研究生学习书 / 整合修订版 / 2026.10

具身智能学习书

从模型原理到机器人实践

模型原理、学习方法与系统实践

以机器人操作与视觉语言动作学习为主线

模型与学习 → 物理与数据 → 预测与决策 → 训练与部署

在线版本、练习代码与勘误

https://github.com/tangtianxu/embodied-ai-book

从模型怎样学习，到机器人怎样行动。

连接建模思想、公式、训练流程与实际使用。

12 章 · 104 节 · 160 幅图

重要机制与高频原理 · 完整推导与独立参考答案

本书由 GPT 生成，仅供学习参考，内容可能存在疏漏。

模型资料范围：截至 2026 年 10 月

正文、算例、实现与练习相互连接。

目录

第1章：模型基础与技术路线 6

1.1 从机器人任务建立学习系统的模型 . . . 6
1.2 神经网络、概率目标与一次真实参数更新 . . . 8
1.3 表示学习、视觉编码与预训练信号 . . . 14
1.4 多模态 token、注意力与信息掩码 . . . 25
1.5 完整 Transformer、位置关系与序列计算 . . . 37
1.6 条件融合、几何输入与多模态可控性 . . . 55
1.7 动作接口、条件分布与分块生成 . . . 68
1.8 潜变量、变分目标与 ACT 的训练部署差别 . . . 73
1.9 扩散模型：从前向过程到训练与反向生成 . . . 83
1.10 Flow Matching：从配对直线到分布运输 . . . 103
1.11 动作编码、条件引导与少步生成 . . . 122
1.12 从视觉语言表示到分层动作策略 . . . 146
1.13 模型演进、记忆与多模态行为条件 . . . 158
1.14 参数适配、联合训练与知识隔离 . . . 173

第2章：模仿学习与策略预训练 192

2.1 示范监督、经验分布与行为克隆风险 . . . 192
2.2 闭环分布偏移、纠错与DAgger . . . 195
2.3 动作块策略在操作任务中的训练和检验 . . . 209
2.4 跨机器人迁移与接口适配 . . . 216
2.5 异质联合训练与语言条件的可控性 . . . 226
2.6 从人类视频、逆向推断和奖励中学习 . . . 229
2.7 模仿学习实践流程与失败诊断 . . . 231

第3章：强化学习与 VLA 后训练 234

3.1 序列决策、状态假设与部分可观测性 . . . 234
3.2 奖励、折扣、动作块与长期信用 . . . 237
3.3 价值、优势、贝尔曼关系与估计误差 . . . 241
3.4 策略梯度怎样改变行动分布 . . . 248
3.5 PPO、相对比较与更新幅度控制 . . . 254
3.6 经验回放、目标网络与连续动作学习 . . . 258
3.7 离线强化学习的数据支持范围与保守改进 . . . 262
3.8 生成式 VLA 的概率、梯度与后训练接口 . . . 265
3.9 奖励模型、偏好、纠错与策略蒸馏 . . . 271
3.10 探索、目标重标记与受约束的经验采集 . . . 274
3.11 完整后训练循环与独立能力验证 . . . 276

第4章：机器人运动学、动力学与物理交互 283

4.1 坐标、旋转、位姿与一致的空间记法 . . . 283
4.2 正逆运动学、雅可比与动作可达性 . . . 287
4.3 力、能量、惯性与机器人动力学方程 . . . 291
4.4 位置、力矩与阻抗控制的策略接口 . . . 295
4.5 时序、接触、摩擦和离散物理变化 . . . 298
4.6 抓取、推拉、插入、工具与柔性物体 . . . 304

4.7 物理测量、系统辨识与可辨识条件 . . . 307
4.8 学习动力学、物理先验与机器人适应 . . . 311
4.9 物理操作项目、移动平台与研究验证 . . . 318

第5章：世界模型、预测表示与想象决策 324

5.1 世界模型的预测对象、动作条件与因果边界 . . . 324
5.2 物理量、图像、潜在表示与不确定性 . . . 329
5.3 RSSM 的先验、后验与潜在动力学 . . . 334
5.4 预测表征、JEPA 与时空生成模型 . . . 340
5.5 轨迹训练、多步展开与误差传播 . . . 344
5.6 奖励、终止、目标与价值的联合学习 . . . 350
5.7 模型预测规划、候选搜索与风险 . . . 352
5.8 Dyna、Dreamer、短想象与离线模型改进 . . . 359
5.9 VLA、世界动作模型与生成式模拟器 . . . 365
5.10 预测、决策和闭环能力的分层评价 . . . 370

第6章：仿真环境、软件实践与 Sim2Real 380

6.1 物理引擎、软件层次与机器人资产 . . . 380
6.2 时钟、执行器和环境重置 . . . 383
6.3 MuJoCo 中的模型、状态、接触与相机 . . . 386
6.4 环境接口与回合边界：一条转移到底从哪里到哪里 . . . 390
6.5 Isaac Sim 与 Isaac Lab 的场景和学习任务 . . . 395
6.6 操作与导航框架、Gazebo 和 ROS 2 集成 . . . 398
6.7 随机化、辨识与迁移：先确定差异，再选择机制 . . . 400
6.8 从单关节到机械臂抓放：搭建可检验的仿真实验 . . . 406

第7章：具身数据生产、质量与科学评测 416

7.1 从任务范围建立数据和标签定义 . . . 416
7.2 遥操作、重定向、采集接口与时间对齐 . . . 418
7.3 示范质量、覆盖和语义保持的数据处理 . . . 421
7.4 混合来源、采样配方与跨形态权重 . . . 423
7.5 语言标签、视频证据与合成监督 . . . 425
7.6 数据集、存储格式与可复现数据工具 . . . 427
7.7 拆分泄漏、统计单位与公平实验 . . . 429
7.8 主动采集、数据贡献与质量闭环 . . . 432

第8章：任务规划、运动规划与技能执行 435

8.1 任务状态、前提效果与搜索 . . . 435
8.2 技能接口、行为树与语言可执行性 . . . 438
8.3 配置空间、碰撞与采样规划：一棵树怎样找到绕行路径 . . . 441
8.4 轨迹优化、时间分配与任务运动联合规划 . . . 455
8.5 分层 VLA、长任务与条件成功概率 . . . 458
8.6 进度监测、重规划与恢复 . . . 460
8.7 从柜内取杯到托盘的完整规划项目 . . . 461

第9章：机器人感知、状态估计与 SLAM 463

9.1 相机、深度、惯性与标定 . . . 463
9.2 贝叶斯估计、滤波与残差优化 . . . 466
9.3 视觉前端、运动估计与数据关联 . . . 470

9.4 点云配准、匹配误差与几何约束 . . . 473
9.5 因子图、回环与视觉激光惯性融合 . . . 476
9.6 地图、深度学习与神经场景表示 . . . 481
9.7 检测、分割、物体位姿与时间跟踪 . . . 484
9.8 语言指代、可供性、主动感知与触觉 . . . 487
9.9 移动操作感知项目与评价诊断 . . . 490

第10章：工程专题：AI Infra 训练基础设施 495

10.1 训练迭代、模型状态与显存需求 . . . 495
10.2 有效批次、混合精度与激活重计算 . . . 499
10.3 分布式进程、集合通信与 DDP . . . 502
10.4 ZeRO、FSDP 与张量并行 . . . 506
10.5 流水线、上下文、专家并行与通信拓扑 . . . 510
10.6 轨迹存储、数据加载与流水线 . . . 514
10.7 性能剖析、检查点与运行恢复 . . . 517
10.8 采样学习系统与多卡 VLA 训练项目 . . . 520

第11章：工程专题：端侧部署与推理加速 524

11.1 部署目标、性能需求与硬件瓶颈 . . . 524
11.2 计算图、算子语义与部署运行时 . . . 529
11.3 量化表示、误差、校准与精度分配 . . . 535
11.4 量化训练、低比特适配、剪枝与蒸馏 . . . 542
11.5 Token、缓存与 VLA 生成工作量 . . . 548
11.6 CUDA 并行、访存与矩阵分块 . . . 552
11.7 归约、融合、注意力与自定义算子 . . . 557
11.8 异步调度、观测时效与稳定运行 . . . 562
11.9 Jetson、CPU、NPU 与模型编译器 . . . 567
11.10 端侧策略优化项目与可信性能研究 . . . 569

第12章：综合项目、研究设计与学习检验 574

12.1 把数据、策略、物理与执行组成完整系统 . . . 574
12.2 从研究问题到公平可验证的实验 . . . 577
12.3 独立学习与实践路线 . . . 579

参考文献与进一步阅读 582

写在前面

本版整合已完成的机制修订，并重点核对常见原理、公式和习题。保留各方法的推导、算例与适用条件；仿真和代码结果按各自实际验证范围理解。学习包同时提供可编辑原稿与检查说明。

这本书从一个问题出发：机器人怎样根据所见、所知和任务要求采取行动，又怎样从示范与行动后果中学得更好。模型原理、模仿学习和强化学习构成主线；动力学、世界模型、仿真、数据、规划与感知解释行动所处的环境；训练基础设施与端侧推理说明系统怎样实际运行。

阅读公式时，先看它描述什么对象，再看哪些量来自数据、哪些由网络估计、最后看它怎样改变训练或决策。例如 V 是当前策略的平均长期回报，价值网络是估计 V 的工具，回报标签是训练该网络的依据。分清这几层，公式就能与实际流程连接。

每节先讲机制，再给算例或实现，最后分别列出练习题和参考答案。可以先独立作答，再核对标签、计算与条件。手写实现中的步骤编号与前面的算法步骤对应；配套代码保存在学习包的代码目录和各机制单元中，正文给出相应入口。

基础模型、模仿与强化学习宜连贯阅读。后续章节可按项目补充：操作任务优先关注坐标、接触、数据与仿真；长任务关注记忆、世界模型和规划；移动操作关注定位与感知；多卡训练和端侧部署可作为工程专题。模型资料包含截至2026年10月的公开路线，图源和进一步阅读入口随正文列出。

第1章：模型基础与技术路线

机器人学习先要把任务转换为网络能够处理的输入、目标与输出。本章从表示和注意力讲到动作生成，再比较不同的**视觉—语言—动作模型**（Vision-Language-Action，VLA）路线。这类模型依据视觉、语言和可用的机器人状态产生动作。重点是每个结构怎样连接语义、几何和动作。

1.1 从机器人任务建立学习系统的模型

1.1.1 先把任务变成可以检查的系统

“会抓杯子”不是一个足够精确的学习问题。杯子在哪些位置，机器人使用什么相机，输出末端增量还是关节角，杯子倾倒算不算失败，指令是否允许同义表达，都决定了训练数据和评价范围。先约定：桌面上有多个物体；固定相机与腕部相机提供图像；本体状态提供机械臂和夹爪的测量；策略输出短时间内的末端位姿增量及夹爪命令；执行器受到速度、工作空间和力的限制；杯子稳定放到托盘才算成功。这是一个教学任务定义，后续章节可以逐项改变条件。

这里最重要的区分是“选择怎么做”和“使电机完成这个选择”。策略可能每 100 毫秒产生一次**动作块**，控制器可能每 2 毫秒跟踪目标。规划器搜索候选路径；**价值网络**给候选行为打预期长期分数；**世界模型**预测动作后的变化。它们都可能是神经网络，也可能包含确定性程序，但训练目标不同。因此不能因为系统都用了 **Transformer**，就认为这些模块可以互换。

1.1.2 从研究问题分清三个变量

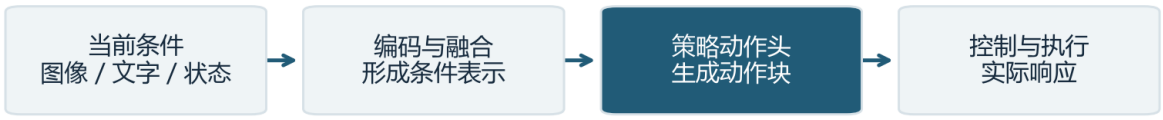
假设机器人没拿稳红杯。训练策略时，我们调整的是权重 θ；规划下一次抓取时，我们调整的是候选动作序列；执行时，改变的是机器人和杯子的物理状态。一个损失下降，可能只是参数更符合训练标签；一条规划代价降低，可能只是预测模型认为路线更好；物理成功必须在执行后另行测量。后面的许多误解，都来自把这三种变化混成一个“优化”。

视觉语言动作模型，Vision-Language-Action，简称 **VLA**，读取视觉、语言及可能的本体状态，产生机器人动作。本体状态是机器人关于自己的测量，例如关节角、角速度、夹爪开度。策略是选择动作的规则；神经网络实现这一规则时称**策略模型**。VLA 可以扮演策略，但“策略”与“VLA”不是同义词：不带语言的视觉动作网络也可以是策略。

红杯任务中，视觉编码器把相机图像变成特征，语言编码器或语言模型处理指令，融合模块连接这些信息，动作头把内部表示变成动作。动作头是负责输出动作的子网络，不是一定独立训练的新模型。控制器再解释动作：目标关节角、末端位移和力矩是不同接口。模型预测“向左 2 厘米”，不意味着电机直接接收这一数字。

还可能存在世界模型，预测“给定动作后会发生什么”；评价模型，判断是否成功或未来回报；规划器，比较候选路线。它们都能使用 Transformer，但职责不同。预测未来图像的模型没有自动变成行动策略；一个会描述红杯的视觉语言模型，也没有自动学会可靠抓杯。

先定位职责，再解释内部模型



世界模型预测后果，评价器判断结果；它们与动作策略的目标不同。

图1.1：同一任务中，不同模块预测不同对象。训练策略修改网络参数，执行控制改变真实状态，规划搜索修改候选动作。

我们用条件 c 表示决策时实际可用的图像、指令、状态和历史，用 A 表示将要输出的动作块。确定性策略直接计算 $A = f_\theta(c)$；随机策略则描述**条件分布** $p_\theta(A \mid c)$，再按规定选择或采样动作。后面学习不同生成路线时，首先要分清模型输出的是一个动作，还是能够产生多种动作的分布。θ 是网络权重，A 是预测对象。真实未来动作可以作为训练标签，部署时却不能作为输入，因为机器人正在决定的正是这些动作。

第一道机制检查始终是：网络输出的数值究竟是什么，在哪个时刻使用，由谁执行？如果这一层不清楚，再准确的损失公式也无法解释系统行为。

训练、生成与物理：三个不同的更新对象

相同“优化”一词，需要追问谁被改变、依据什么证据

损失低、候选代价低与任务成功分别需要证据。不要把其中一个指标当作另外两个的保证。

图1.2：参数更新、候选更新和物理执行分别改变不同对象。模型计算和物理结果之间需要明确接口与评价。

1.1.3 练习题

题1.1-1 策略模型、世界模型、**价值模型**分别回答什么问题？机器人把杯子放入柜子时，三者的输入、输出怎样不同？它们是否必须各有一个独立网络？

1.1.4 参考答案

题1.1-1答案 策略模型回答“现在该怎样行动”。输入可以是相机图像、关节状态、近期观测历史和“把杯子放入柜子”的指令，输出是下一步动作或一段动作，例如末端向杯柄移动、闭合夹爪。

世界模型回答“如果这样行动，接下来会发生什么”。除当前信息外，它还接收候选动作，例如“保持夹紧并抬高”，输出未来状态、未来观测或这些量的分布，也可以同时预测奖励与终止概率。它预测后果，本身不必负责决定执行哪个动作。

价值模型回答“从这里继续，预期能得到多少累计回报”。状态价值 V 的输入不固定下一动作，评价当前策略从该情形继续的平均结果；动作价值 Q 还接收具体候选动作，评价先做该动作再继续的结果。输出通常是一个回报数值，例如预测未来完成任务的折扣得分。

三种职责可以共享一个视觉编码器，再接动作、状态预测和价值三个输出头；也可以分别训练。是否独立取决于监督、更新和部署需求，不能由名称决定。即使共享网络，也要分别说明每个输出用什么标签训练。

1.2 神经网络、概率目标与一次真实参数更新

1.2.1 张量、线性层、嵌入和 MLP 到底在做什么

张量是存放数值的多维数组，维度名称比总元素数更重要。一个训练批次有 B 条样本，每条有 N 个 token、每个 token 有 d 维特征，可以存成 B×N×d。B 是独立样本的批次轴，N 是信息单元轴，d 是每个单元的表示宽度。把 N 与 d 对调，即使程序能运行，含义也可能完全改变。

线性层对每个输入向量计算 y=Wx+b。W 是可训练的映射矩阵，b 是偏置。它能把 7 维关节角投影成 256 维内部表示，但增加维数不会凭空增加传感器信息。输入缺少杯子位置时，线性层没有把缺失位置测出来；它只能利用已有输入与训练统计关系。

多层感知机（Multi-Layer Perceptron，MLP）交替使用线性层和非线性函数，例如 $y = W_2\sigma(W_1x + b_1) + b_2$。若去掉非线性，多层线性变换仍可合成一个线性变换，表达能力不会因此发生本质变化。非线性让网络对不同输入区域采用不同响应：关节接近目标时可以学习小幅修正，远离目标时可以学习另一种变化规律。结构提供表达这些规律的能力，训练数据和损失决定最终学到哪一种规律。

嵌入，embedding，是把信息转换成可计算向量的表示。文字 token 可以用整数查可训练的向量表；连续关节状态通常通过线性层或 MLP 编码。两者都叫嵌入，但操作不同。整数“机器人编号 3”若表示类别，不应机械当作机器人 2 与 4 的中间物理形态。

小例子：输入 x=(杯子水平位置 0.2，夹爪位置 0.1)，取 W=(1,−1)、b=0，输出 0.1，是一个相对位置特征。若训练发现这一差值有用，权重可以接近这种关系；另一维可以记录夹爪是否张开。内部特征没有预先保证对应人类命名的量，需要探测与对照验证解释。

线性层通常可用于独立处理每个 token；它本身不让图像第 1 个 token 读取第 2 个 token。跨 token 的信息交换需要**注意力**、卷积、循环或其他结构。区分“每个信息单元内部的变换”与“信息单元之间的交换”，后面 **Transformer** 的结构就容易理解。

1.2.2 训练目标怎样规定模型要学什么

同一个 Transformer 可以输出文字概率、动作、噪声、生成速度或成功概率。结构提供计算能力，训练目标指定学习职责。网络输出 7 个数不意味着它们就是 7 个关节角；需要由标签、损失和输出接口共同赋予含义。

监督学习给模型输入 c 和标签 y，前向计算得到预测 fθ(c)，损失衡量预测与目标的差异，优化器调整 θ。标签可以来自人类示范、测量、图文配对、遮挡前的原始图像，也可以来自我们人工采样的噪声。不是所有监督都必须由人工逐项标注。

训练动作模型时，一个重要选择是：给定条件后，直接输出一个确定动作，还是学习可能动作的分布？下面先看**单值回归**。它把条件 c 映射为一个预测 $f_\theta(c)$，用平方损失衡量预测与示范动作的差异。这里的“单值”指只给出一个确定结果，结果本身也可以是动作向量或整段动作块。在能够充分拟合数据的理想条件下，这个损失的最优预测是示范动作的条件均值 $\mathbb{E}[A \mid c]$。

假设图像、指令和机器人状态完全相同，示范中有一半从障碍左侧绕行，一半从右侧绕行。用 -2 和 $+2$ 简化表示这两种动作，并假设两者都可行，而中间动作 0 会撞向障碍。输出 0 的平均平方误差是 $\frac{1}{2}(0+2)^2+\frac{1}{2}(0-2)^2=4$；输出 -2 的平均平方误差却是 $\frac{1}{2}(-2+2)^2+\frac{1}{2}(-2-2)^2=8$。因此，平方损失偏好中间结果。计算没有错，问题出在评价标准：它奖励“平均接近全部示范”，没有直接评价“能否成功绕行”。这个例子说明，多种合理动作的平均值可能成为不合理动作。

一种解决办法是学习能表示不同路线的**条件分布** $p_\theta(A \mid c)$，例如让概率分别集中在 -2 和 $+2$ 附近，而不是集中在 0 附近。示范仍然提供具体动作标签，训练不必为每个输入手工标注一整张分布。常用的**负对数似然**目标为

$$L(\theta)=\mathbb{E}_{(c,A)\sim D}[-\log p_\theta(A \mid c)].$$

D 表示训练数据的分布。每读到一个条件与示范动作，模型就计算这个动作在自身分布中的概率密度；密度越低，损失越大。训练反复看到左绕和右绕示范，便要求模型同时解释两类数据。要实现这一点，分布形式必须能容纳两个分开的峰；单个高斯即使增大方差，也可能把大量概率放在不合适的中间区域。

训练后的使用方式同样关键：可以选定一个峰，或者采样一条完整路线；若把两种路线的均值拿去执行，仍会得到 0。因此，这里应得出的结论是“多种不能直接平均的动作，需要被区分或保留下来”。分布建模是一种办法，也可以增加“从左侧绕行”的条件，或先选择路线，再回归具体动作。当条件已足以确定一种合理动作时，直接回归仍是合适的选择。是否需要保留分布，取决于预测对象的用途：价值网络若要预测平均回报，拟合平均值正是所需结果；动作最终却要落到一条可执行路线。动作分布学习的是示范中的行为规律，任务成功还需要执行与评价来检验。

1.2.3 计算图、反向传播与冻结参数

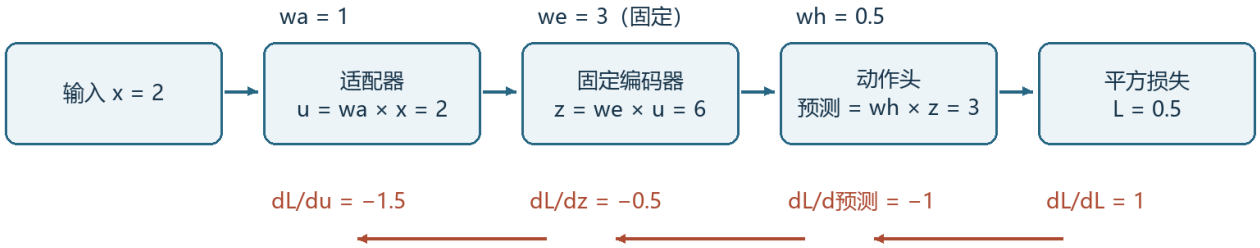
写出一个网络的前向公式，只说明它怎样得到预测。训练还要知道：预测误差是经过哪些计算产生的，调整哪一个参数会怎样改变误差。**计算图**（Computational Graph）就是把这条依赖关系展开：节点表示输入、参数或一次运算，箭头表示某个结果被后面的运算使用。它不是另一个神经网络，也不是机器人环境的地图，而是当前这次计算的“来龙去脉”。

例如，一个可训练适配器先把输入改成编码器更容易使用的形式，视觉编码器提取特征，动作头再给出动作预测。把三个模块暂时简化成标量乘法：

$$u = w_a x, \qquad z = w_e u, \qquad \hat{a} = w_h z, \qquad L = \frac{1}{2}(\hat{a} - a^*)^2.$$

x 是输入，a^* 是示范标签，w_a, w_e, w_h 分别代表适配器、编码器和动作头的参数。真实模块有很多层，但“前一个输出成为后一个输入”的关系相同。令 $x = 2$、$a^* = 4$、$w_a = 1$、$w_e = 3$、$w_h = 0.5$，顺着图计算，得到 $u = 2, z = 6, \hat{a} = 3, L = 0.5$。

计算图：前向记录依赖，反向逐步计算导数



适配器参数梯度：dL/dwa = (dL/du) × x = −1.5 × 2 = −3。
冻结 we：不更新它的权重，但仍用 dz/du = 3 计算适配器的梯度。
no_grad 前向：仍得到 z = 6，却不保存 z 怎样依赖 u 的求导记录。

图1.3：上方沿箭头计算预测，下方沿相反方向计算损失对中间量和参数的导数。编码器权重即使固定，其“输入怎样影响输出”的关系仍可以供适配器训练使用。

反向传播（Backpropagation）把链式法则按计算图逐步执行。先算输出误差 $\partial L/\partial \hat{a} = 3 - 4 = -1$；再经过动作头，得到 $\partial L/\partial z = -1 \times 0.5 = -0.5$；经过编码器，得到 $\partial L/\partial u = -0.5 \times 3 = -1.5$；最后到适配器参数：

$$\frac{\partial L}{\partial w_a} = \frac{\partial L}{\partial u}\frac{\partial u}{\partial w_a} = -1.5 \times 2 = -3, \qquad \frac{\partial L}{\partial w_h} = (\hat{a} - a^*)z = -6.$$

适配器导数为负，说明在当前点稍微增大 w_a 能降低损失；梯度下降因此会增大它。反向传播算的是敏感程度，**优化器**（Optimizer）才据此改变参数。loss.backward() 和 optimizer.step() 分别对应这两个环节：前者计算、累积梯度，后者更新权重。CS231n 反向传播课程

requires_grad 和 no_grad 究竟控制什么

如果编码器已经预训练好，我们希望保持它的权重，同时让前面的适配器学习“怎样把输入送进去”。此时必须区分两种操作。requires_grad 字面意思是“是否需要梯度”，设在参数上，用于决定自动微分是否为该参数计算梯度；torch.no_grad() 是一段代码的运行方式，让其中的运算不被记录到反向传播所需的图中。

把 w_e 的 requires_grad 设为 False，相当于把编码器参数当作固定常数。固定的乘数仍会影响输入输出关系：$z = 3u$ 的导数 $\partial z/\partial u = 3$ 仍然存在。只要 u 来自可训练适配器，框架就需要记录这次乘法，才能得到前面算出的 $\partial L/\partial w_a = -3$。因此，**冻结参数**并不等于禁止梯度经过整个模块。

如果把编码器前向放进 no_grad()，仍会得到同样的数值 $z = 6$，但框架不再记录“6 是怎样由 u 和编码器计算出来的”。动作头可以把这个数当输入继续学习；反向求导走到这里时却没有可用的依赖记录，无法继续回到适配器。这就是“梯度路径被截断”的具体意思。“适配器收不到梯度”指它的参数没有得到由这项损失计算出的导数，因而这项损失不能训练它；不是说梯度很小，也不是说数据没有经过适配器。

操作	编码器参数是否求梯度	损失能否经过编码器训练前面的适配器	适用情况
编码器参数 requires_grad =False，正常前向	不求	可以，前提是上游输入需要梯度	固定编码器，训练输入适配器
编码器前向在 no_grad() 中	不为这次前向求	不可以	只训练编码器后面的头，或独立提取固定特征
仅调用 encoder.eval()	仍可求	仍可以	切换 Dropout、BatchNorm 等模块的运行行为

eval() 的意思是切换评估运行方式。它不负责冻结参数，也不关闭求导。另一个常见操作 z.detach() 返回数值相同、但不再连着此前求导路径的张量，常用于把教师输出或自举标签当作当前步骤的固定目标。公式中常写成 $\mathrm{sg}(z)$，sg 是 Stop Gradient（停止梯度）：前向数值仍为 z，当前反传则把这条输出当作常量。它与 no_grad() 的使用位置不同，但都能有意切断一条训练路径。PyTorch 自动微分机制

用代码观察“固定权重”与“丢掉求导路径”

下面代码复现刚才的数值。None 表示这个参数没有由本次反传获得梯度；数值 0 才表示导数已经算出、恰好为零。这一区别有助于排查一个模块为什么始终没有学到东西。

算法步骤：

1. 创建两个可训练参数和一个固定的编码器参数。
2. 先计算适配器输出，再选择是否记录编码器前向。
3. 计算动作预测、损失并执行反向传播。
4. 比较各参数是否得到梯度。

```
import torch

def inspect_gradients(stop_encoder_path=False):
    # 步骤1：适配器、动作头可训练，编码器权重固定
    wa = torch.tensor(1.0, requires_grad=True)
    we = torch.tensor(3.0, requires_grad=False)
    wh = torch.tensor(0.5, requires_grad=True)
    # 步骤2：冻结 we，仍可以记录 z 对 u 的依赖
    u = wa * 2.0
    if stop_encoder_path:
        with torch.no_grad():
            z = we * u
    else:
        z = we * u
    # 步骤3：动作头拟合标签 4，计算本次损失的梯度
    prediction = wh * z
    loss = 0.5 * (prediction - 4.0).square()
    loss.backward()
    # 步骤4：None 与数值零有不同含义
    return [None if p.grad is None else p.grad.item()
            for p in (wa, we, wh)]

print(inspect_gradients(False))  # [-3.0, None, -6.0]
```

```
print(inspect_gradients(True))   # [None, None, -6.0]
```

两个分支的预测和损失相同，训练效果却会不同，因为优化器能使用的梯度不同。学习计算图的价值就在这里：读到“冻结主干”“停止梯度”“教师不反传”“知识隔离”时，你能具体追踪哪项损失在训练哪个模块。第1.14节的知识隔离和第5章的世界模型目标都会使用这条判断方法。

从梯度到稳定训练

小批次训练以部分样本估计总体目标。**Adam**（Adaptive Moment Estimation，自适应矩估计）记录梯度和平方梯度的移动统计，调整不同参数的更新尺度；AdamW 再把权重衰减单独处理。学习率预热先采用较小步长，梯度裁剪则限制一次更新所依据的梯度规模。这些方法帮助控制优化过程，不能替代正确的标签和求导路径。AdamW 原论文

归一化也会影响目标。动作某一维除以标准差 s 后，该维平方误差相当于乘了 $1/s^2$；数值尺度接近，不代表各种物理误差同样重要。训练时可先让小模型拟合少量样本，检查标签与梯度，再检查独立数据和闭环执行。这样能逐步分清：程序有没有正确训练，目标有没有下降，以及机器人是否真的学会了任务。

1.2.4 损失是你为问题选择的答案标准

同一任务可以采用不同的答案标准。直接回归要求输出接近示范动作；分布学习要求模型解释示范中出现的各种动作；强化学习则利用行动后果，偏向长期回报较高的选择。三种目标可能互补，例如先从示范学习动作分布，再用任务反馈改进它。选择网络之前先确定预测对象和评价标准，才能判断“损失下降”具体意味着模型在哪一方面学得更好。

设一个线性动作头输出 $\hat{a} = wx + b$，x 是归一化后的杯子水平偏移，a 是示范动作。输入 x=2，标签 a=1，当前 w=0.2、b=0.1，则预测为 0.5。令损失为 $L = \frac{1}{2}(\hat{a} - a)^2$，得到 $\partial L/\partial w = (\hat{a} - a)x = -1$，$\partial L/\partial b = -0.5$。学习率为 0.1 时，w 变为 0.3、b 变为 0.15，预测变为 0.75。这次求导的思想是：观察输出误差，沿着产生误差的计算链，把责任分给可调整参数。它没有直接告诉机器人怎样避免碰撞，也没有修改杯子的真实位置。

批次维度 B、**动作块**长度 H、动作维度 d 产生标签张量 $B \times H \times d$。如果动作块尾部已经越过轨迹结束，那里没有真实标签，需要掩码。若把补零当成真实动作训练，模型会学到人为的“轨迹尾部停住”；若对所有元素直接求平均，长块、不同身体和不同有效维度的样本可能获得不一致权重。张量形状是问题定义的一部分，不能只检查程序是否报错。

优化器保存历史梯度统计，并不因此获得任务历史记忆。前者服务参数更新；后者是策略运行时理解速度、接触和身份的信息。学习率、正则化和梯度裁剪控制更新行为；它们不能补回数据里没有的恢复动作。自动微分的计算链可结合 PyTorch 教程练习，但本书的手算不依赖特定框架。

1.2.5 练习题

题1.2-1 同一条件下，示范动作 -2 与 $+2$ 各出现一半。直接用平方损失预测一个动作，最优结果是什么？如果改为预测单个高斯的均值和标准差，用负对数似然训练，理想最优参数又是什么？这是否足以解决中间动作撞障碍的问题？提示：一维高斯忽略常数后的负对数似然为 $(a-\mu)^2/(2\sigma^2) + \log\sigma$，可先对两类标签取平均。

题1.2-2 计算图为“可训练输入适配器→冻结编码器→可训练动作头→损失”。怎样冻结编码器而继续训练输入适配器？

1.2.6 参考答案

题1.2-1答案 这里先讨论单值动作回归。把预测记为 b，平均平方误差为

$$L(b) = \tfrac{1}{2}(b+2)^2 + \tfrac{1}{2}(b-2)^2 = b^2 + 4.$$

因此最优预测 b=0，误差为 4。它位于两个示范之间，但题目规定中间会撞障碍，所以“损失最优”并不等于“动作可行”。

再讨论单个高斯分布的拟合。对两种标签平均后，负对数似然为

$$L(\mu, \sigma) = \frac{\mu^2 + 4}{2\sigma^2} + \log\sigma + \text{常数}.$$

先对 μ 求导，得到 μ=0；代入后对 σ 求导，得到 $-4/\sigma^3 + 1/\sigma = 0$，因此 $\sigma^2 = 4$、$\sigma = 2$。

高斯方差记录了标签的变化范围，却没有正确表示左右两条路线：单高斯的密度峰仍在 0。直接执行均值仍会撞障碍，采样也会产生许多中间动作。更合适的表示可以是两个分量集中在 −2、+2 的混合分布，并先选定一个分量再生成完整路线；也可以把“左绕/右绕”加入条件，让确定性模型只拟合对应路线。需要的是匹配动作的多种可行模式和执行方式，并非只要输出名为“分布”就足够。

题1.2-2答案 计算图记录一次前向计算中“哪些结果由哪些输入经过什么运算得到”，反向传播再沿这些关系应用链式法则。设输入适配器输出 z=gα(x)，冻结编码器输出 h=Eβ(z)，动作头输出 a=fθ(h)。训练适配器需要

$$\frac{\partial L}{\partial \alpha} = \frac{\partial L}{\partial a}\frac{\partial a}{\partial h}\frac{\partial E_\beta(z)}{\partial z}\frac{\partial g_\alpha(x)}{\partial \alpha}.$$

编码器参数 β 不更新，但编码器对输入 z 的导数仍必须参与计算。

把编码器参数设为 requires_grad=False，并让其前向计算在正常梯度模式中执行即可。这个标记控制的是是否为该参数计算并累积梯度。no_grad() 则让包住的运算不被记录为反向传播路径；若包住整个编码器前向，损失只能训练后面的动作头，无法通过 h 找回对 z、α 的依赖。这就是本题“梯度被截断”：不是前向结果消失，而是这条反向求导关系未被保留。

```
# 1. 固定编码器的参数值。
encoder.zero_grad(set_to_none=True)  # 清除此前训练遗留的梯度
encoder.requires_grad_(False)
optimizer = torch.optim.Adam(
    list(adapter.parameters()) + list(action_head.parameters())
)
# 2. 保留编码器对其输入的求导关系。
z = adapter(x)
h = encoder(z)  # 此处不要使用 no_grad() 或 detach()
loss = criterion(action_head(h), target)
# 3. 损失经动作头和编码器传回适配器。
optimizer.zero_grad()
loss.backward()
optimizer.step()
```

清除旧梯度后再检查本轮反向传播：适配器和动作头的参数会累积梯度，编码器参数的 .grad 保持 None。requires_grad=False 控制本轮是否求导，不会自动删除上一次已经保存的 .grad。eval() 只改变 Dropout、BatchNorm 等层的行为，也不等于冻结参数。

进一步阅读：PyTorch：Autograd mechanics（梯度标记与梯度模式）。

1.3 表示学习、视觉编码与预训练信号

1.3.1 表示学习为什么是基础，而不是预处理名词

表示学习把高维原始输入变成对任务有用的特征。一个图像像素阵列包含纹理、光照、位置和背景；抓杯需要区分目标、几何和接触条件。一个适合图文检索的表示可能擅长识别“杯子”，却压缩掉抓取所需的几毫米位移。特征是否“好”，必须问对哪个下游问题好。

不变性指某种输入变化后表示基本保持相同，例如背景颜色改变后仍识别红杯。**等变性**指表示随输入按可预测规律变化，例如杯子向右移动，位置表示也向右变化。分类常希望不变，操作同时需要保留几何变化。把所有图像增强都当作无害的鲁棒性训练，会把动作任务依赖的信息抹掉。

自监督学习利用数据自身构造目标。遮住图像一部分，要求恢复原来的内容；对同一图像的两种视图要求表示对应；预测下一段观察，这些都能提供信号。目标由原始数据构造，不代表完全无假设。把“左右翻转”作为同一图像的正样本，对杯子类别合理；若动作标签没有同时翻转，对左移命令可能错误。

迁移学习使用已有表示初始化新任务。冻结编码器节省更新量，但其保留的信息成为限制；解冻可以适配几何与相机，也可能损伤原有语义。可以用冻结、局部解冻和完全解冻三组对照，固定数据与预算，检查语义辨识、动作精度、泛化和成本，而不是只根据损失最低选择结构。

单张图像可能看不出夹爪速度；连续帧或本体测量可以补充这项信息。如果两种实际状态在输入中完全相同，却需要不同动作，增加编码器容量仍无法把它们区分。此时应先补足观测，再考虑模型容量。

1.3.2 CNN、ViT 与 U-Net：图像怎样变成可用特征

卷积为什么在不同位置使用同一组权重

卷积神经网络（Convolutional Neural Network，CNN）用较小的权重模板处理局部图像，并在不同位置重复使用它。**卷积核**（Convolution Kernel）就是这个可训练模板，**特征图**（Feature Map）记录模板在各位置得到的响应。例如用一行教学核 $(-1, 0, 1)$ 处理灰度 $(0.2, 0.2, 0.9)$，响应为0.7；处理平坦区域 $(0.2, 0.2, 0.2)$，响应为0。这个核对从暗到亮的变化敏感，无论边缘出现在图片左边还是右边，都使用同一规则。

真实卷积同时处理二维邻域和多个颜色、特征通道，可以写为

$$y_{u,v,o} = b_o + \sum_{i,j,c} K_{i,j,c,o}\, x_{u+i,v+j,c}.$$

(u, v) 是输出位置，(i, j) 是核内偏移，c 是输入通道，o 是输出通道；每个输出通道是一种可学习的局部特征。K 不随输出位置改变，这就是**权重共享**。一个 3×3、输入3通道、输出32通道的核组，只需 $3 \times 3 \times 3 \times 32 = 864$ 个权重，而不是为图片每个位置单独建立一套。示例边缘核是手工取的；实际核通过后续识别或动作损失反传学习，不需要先标出每一个边缘的答案。

这种结构预先偏好“局部规律会在不同位置重复出现”，称为**归纳偏置**（Inductive Bias）。逐层组合后，局部边缘可以形成更大的形状；一个深层特征能受多大原图区域影响，称为**感受野**（Receptive Field）。步幅或池化可以降低分辨率、扩大处理范围，但也可能丢掉细小位置差异。物体移位时，卷积响应通常随之移位；要让最终类别判断不受移位影响，还需要后续汇集等设计，不能把局部等变直接称成整个网络完全不变。《动手学深度学习》卷积教程

ViT 如何把图块接到 Transformer

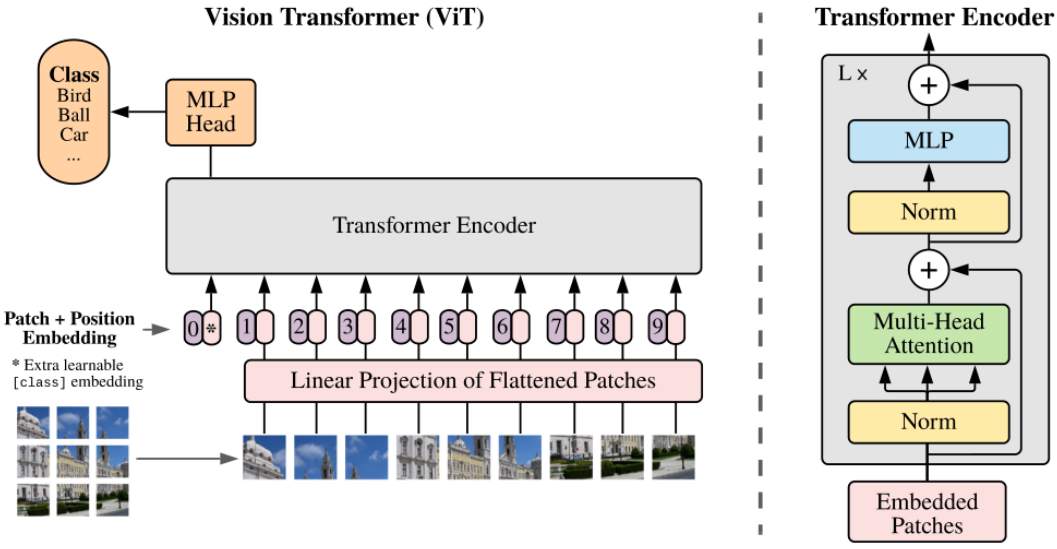


图1.4：Dosovitskiy 等，ViT 论文 Figure 1。图片先切块并线性嵌入，加入位置与汇总 token，再由 Transformer 交换信息。原论文。

视觉 Transformer（Vision Transformer，ViT）先把图片切成图块 patch。一张 224×224 的 RGB（Red, Green, Blue，红绿蓝三通道）图按 16×16 切分，得到196块；每块含 $16 \times 16 \times 3 = 768$ 个像素数，展开后经同一线性层变成 D 维向量。图中的 Linear Projection of Flattened Patches 就是这一步，不是先让模型猜每一块属于什么物体。加上位置嵌入后，这些向量成为图像 token，送入 Transformer 编码器。

图左上方的 MLP Head 是分类输出头，特殊的 [CLS]（Classification Token，分类汇总 token）在注意力中读取其他图块，最终提供全图摘要。图右边展开编码器：Norm 是归一化，Multi-Head Attention 是多头注意力，MLP 是逐位置的多层感知机，加号是残差相加，$L\times$ 表示堆叠 L 层。具体计算将在第1.4—1.5节逐步展开。《动手学深度学习》ViT 教程

用于机器人时，可以保留所有图块特征，让动作查询读取杯柄附近的位置；也可以仅使用全图摘要。后者较省后续计算，但把多个位置汇成一个向量，可能削弱精细几何。CNN 的局部结构与 ViT 的全局信息交换提供不同偏好，训练数据、预训练目标和保留的输出层也会改变结果。选择视觉编码器，应比较目标辨识、局部位置误差和动作表现，而不能仅凭它是否叫 Transformer 判断能力。

U-Net 为什么既缩小特征，又把早期特征接回来

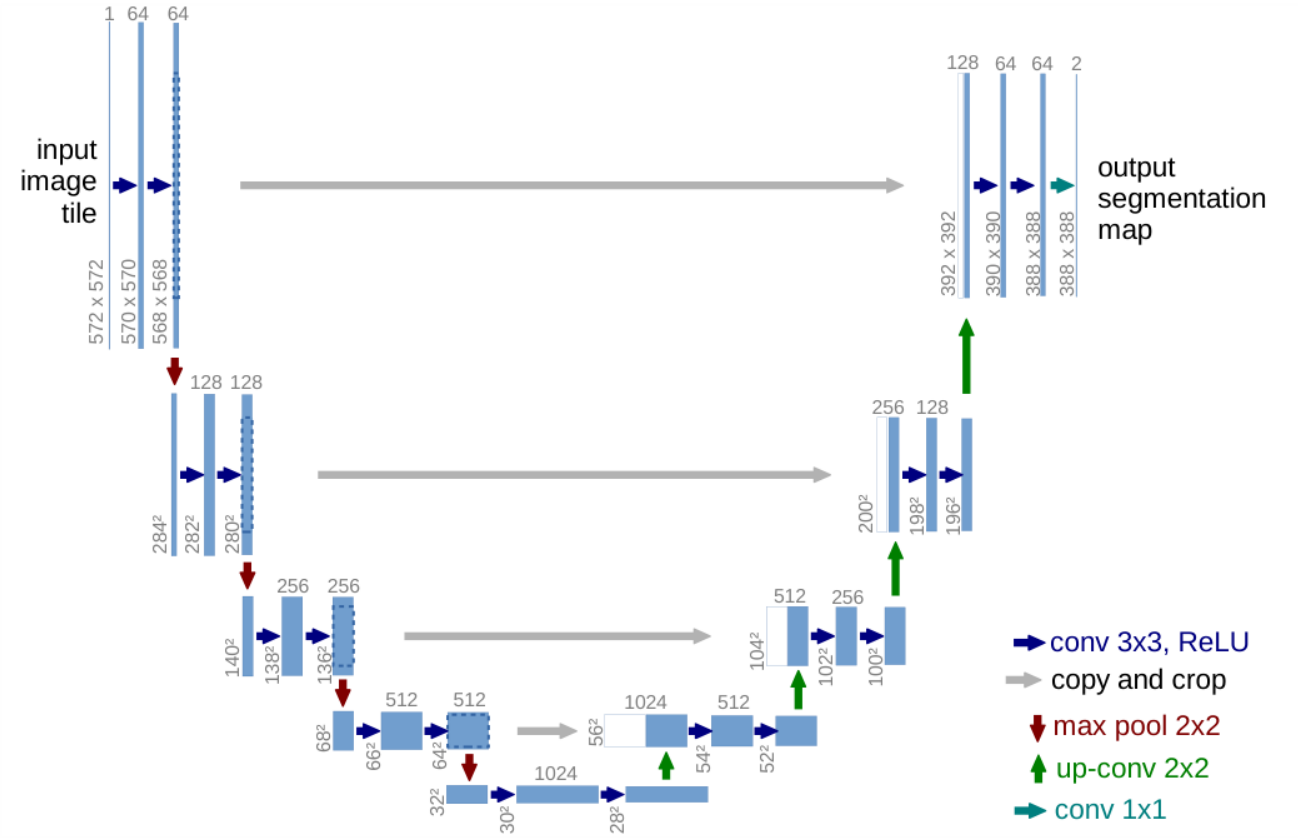


图1.5：Ronneberger 等，U-Net 论文 Figure 1。左侧逐步降低空间分辨率，右侧恢复输出分辨率，横向连接把对应尺度的早期特征送回来。原论文。

U-Net 是一种适合输出空间或序列细节的结构。左侧下采样使特征覆盖更大范围，有利于理解“这片局部属于什么整体”；右侧上采样恢复较密的输出。但缩小后的特征不一定保留准确边缘位置，因此将左侧同尺度的细节直接送到右侧，与粗尺度结果一起加工。这条**跳接**（Skip Connection）在原图采用特征拼接，和残差直接相加的具体操作不同。

原图标签或符号	计算含义
conv 3×3, ReLU	3×3 卷积后使用 ReLU（Rectified Linear Unit，修正线性单元），即 $\max(0, x)$
max pool 2×2	对 2×2 邻域取最大值，降低空间分辨率
up-conv 2×2	上采样侧的可训练卷积运算，增加输出分辨率
copy and crop	将左侧特征复制到右侧，并按原图尺寸对齐后拼接
conv 1×1	在同一位置混合特征通道，映射为输出类别或数值
蓝色、白色方块与数字	蓝色是本支路特征，白色是从另一侧接入的特征；上方数字是通道数，旁边的二维尺寸是空间高、宽

原始图用于像素分割，最后的 segmentation map 是分割结果；扩散模型可使用相似的多尺度结构，输出噪声或干净数据预测，并让噪声级别与任务条件进入各层。动作块则可沿时间轴使用一维版本。相同骨架可以承担不同预测任务，最终输出是什么由训练标签和损失决定。看 VLA 的去噪网络时，既要认出结构，也要找到它正在拟合的量。

ResNet（Residual Network，残差网络）用残差相加帮助深层特征训练，U-Net 用跨尺度连接保留细节，ViT 用注意力交换图块信息。它们都可以服务视觉或生成模块，但改变的是不同计算关系，读架构图时应把这些关系落实到输入、输出和连接上。

1.3.3 CLIP、SigLIP、DINO 与 MAE：没有动作标签，究竟怎样学习

机器人策略通常不会从随机像素表示开始。它会复用已经通过大量图像学到的编码器，再用动作数据建立控制关系。本节要理解的不是四个模型名字，而是四种获取训练目标的方法：图文配对、独立判断图文关系、同一图片的不同裁剪互相提供目标，以及恢复被遮住的像素。它们规定了编码器应保留什么信息，也解释了预训练表示可能遗漏什么。

CLIP：让一张图片找到批次中与它配对的文字

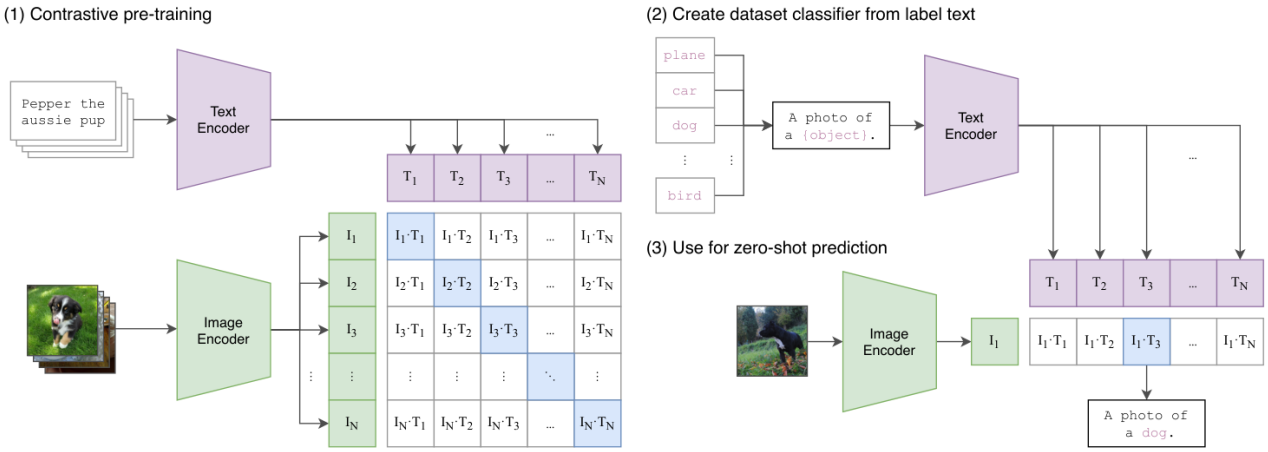


图1.6：Radford 等，CLIP 论文 Figure 1。左侧是训练时的图文匹配，右侧是用类别文字描述进行识别。原论文。

CLIP（Contrastive Language-Image Pre-training，图文对比预训练）的训练数据是一对对图片和描述。图中的图像编码器、文字编码器分别把它们变成向量。假设当前批次是“红杯图片—红杯描述”“蓝碗图片—蓝碗描述”，我们计算所有图像与文字的相似度，得到一个 2×2 矩阵；对角线是数据中给定的配对，另外两项是不配对。

令归一化图像向量为 v_i，文字向量为 t_j，分数 $s_{ij} = v_i^\top t_j / T$。$T$ 是**温度**（Temperature），控制相似度差异转成概率后有多明显。对于图片 i，模型要在整个批次的文字中选中第 i 条：

$$p(j \mid i) = \frac{e^{s_{ij}}}{\sum_k e^{s_{ik}}}, \qquad L_i = -\log p(i \mid i).$$

正确描述分数为 2、错误描述为 0 时，正确概率为 $e^2/(e^2+1) \approx 0.881$；两项同分时只有 0.5。损失要求正确配对相对于其他候选更突出。文字也反过来选择对应图片，两种方向共同训练编码器。图右侧的识别就是复用这一能力：先编码“一个杯子”“一个碗”等候选描述，再与新图片比较，而不是为每个类别重新训练一个识别网络。

SigLIP：每个图文对分别回答“是否配对”

SigLIP 的名称来自 Sigmoid Loss for Language Image Pre-Training，即用于图文预训练的 sigmoid 损失。它保留图文编码器，但换了训练目标。CLIP 问“一行候选中选谁”，SigLIP 对每个图文对问“这两个是否配对”。它没有要求同一行各对的匹配概率加起来为 1。

设一对图文的分数为 s，标签 $y=1$ 表示配对，$y=0$ 表示不配对，预测匹配概率为 $p=\sigma(s)=1/(1+e^{-s})$。二元交叉熵为

$$\ell=-y\log p-(1-y)\log(1-p),\qquad \frac{\partial \ell}{\partial s}=p-y.$$

配对样本当前只有 $p=0.2$，导数为 -0.8，梯度下降会提高分数；不配对样本却给出 $p=0.8$，导数为 $+0.8$，梯度下降会降低分数。标签依然来自原始配对关系，不是模型自己宣布“红杯与蓝碗应该一样”。真实 SigLIP 会对分数设置可学习的尺度和偏置，并处理正负对的数量差异；这里先保留判断机制。SigLIP 论文与算法

这一区别对阅读 VLA 架构有用：使用 SigLIP 视觉编码器，意味着复用了通过这种图文目标学到的视觉特征，不意味着动作模型也使用同一种损失。图文训练主要建立语义对应；把“红杯”变成夹爪的运动，还需要动作监督、空间信息和闭环反馈。

DINO：同一张图片，两种裁剪，怎样互相提供目标

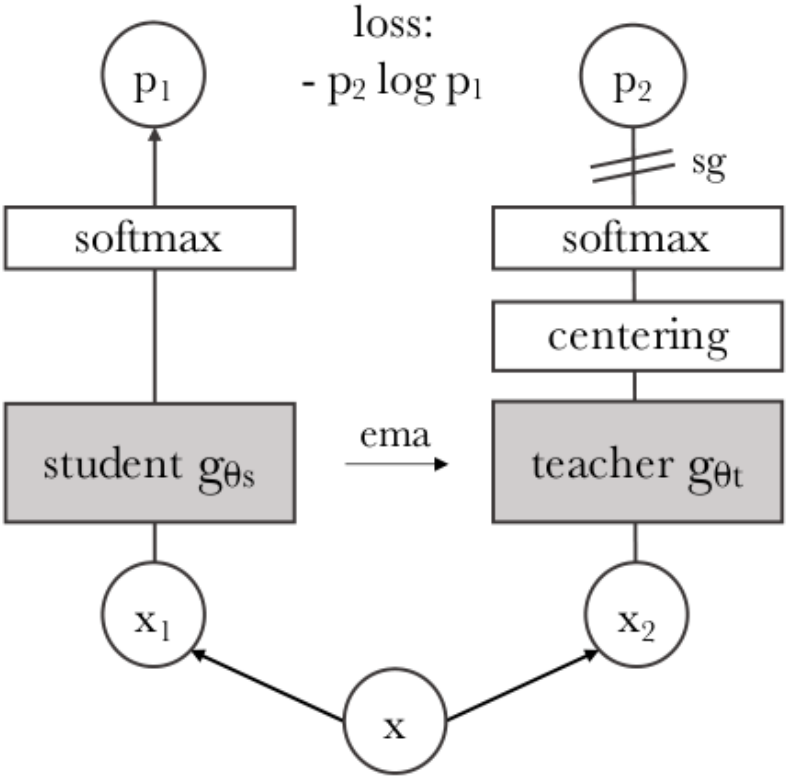


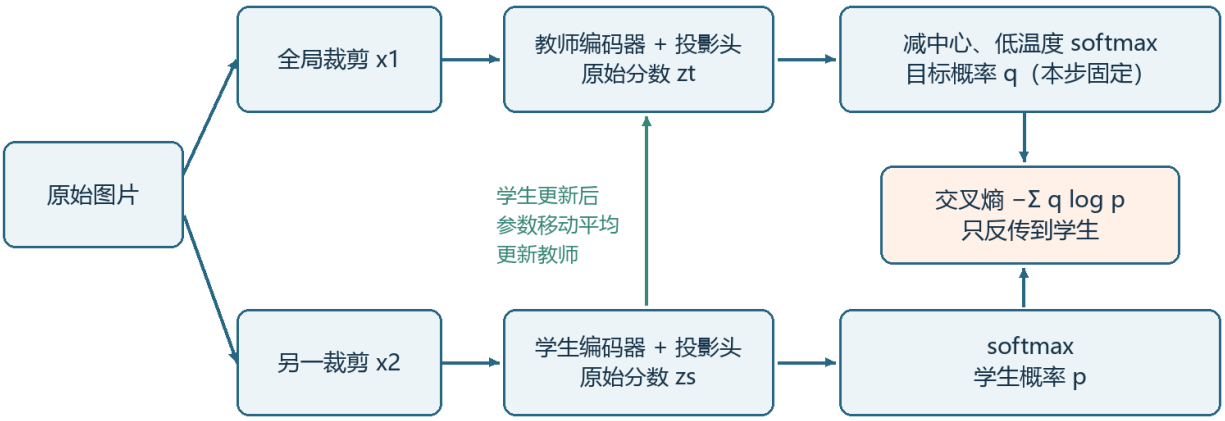
图1.7：Caron 等，DINO 论文 Figure 2。左右分支分别是学生与教师，底部的两个视图来自同一图片。原论文。

图中 student、teacher 分别是学生和教师，centering 是居中，ema 是 Exponential Moving Average（指数移动平均），sg 是 Stop Gradient（停止梯度）。x_1, x_2 是原图 x 的两种变化，p_1, p_2 是两条支路的输出概率；顶部损失将教师概率作为目标。图上的 ema 箭头指参数更新方式，不是把学生本次输出直接当成教师输入。下面用一组具体概率与分数解释这些步骤。

图文方法需要描述。只有大量无文字图片时，**DINO**（Self-Distillation with No Labels，无标签自蒸馏）用图片自身构造训练任务。这里的**图像视图**（Image View）指同一张图片经过裁剪、缩放、颜色调整等操作得到的版本，不是一定来自另一台相机。**教师网络**提供本步的训练目标，**学生网络**调整参数去接近这个目标；“教师”是训练职责，不表示它预先拥有人工答案。

比如原图里有红杯和桌面。一次裁剪保留大部分场景，另一次裁剪保留杯身和杯柄。我们希望编码器在看到局部或不同颜色光照时，仍能提取与同一图像内容对应的特征。原始 DINO 让教师读取覆盖较大区域的全局裁剪，学生读取全局和局部裁剪，并要求学生匹配教师对另一视图的输出。全局视图通常含有更多内容，因而为局部视图提供一个较稳定的参照；裁剪并不保证每次都恰好保留同一个物体，这也是该目标的假设与局限。

DINO：训练目标来自同图另一视图的教师预测



下游动作模型使用视觉特征 f(x)；图中的预测头用于预训练，不直接输出动作。

图1.8：两个裁剪来自同一原图。教师分支在当前步骤提供固定目标，损失只反传到学生；学生更新之后，再用参数移动平均更新教师。训练用预测头与下游使用的视觉特征分开标出。

两条分支先用视觉编码器得到特征 $f(x)$，再经训练用的**投影头**（Projection Head）得到 K 个分数 $z = g(f(x))$。分数经过 softmax 变成长度为 K 的概率向量。K 个位置可以理解为训练中形成的特征分组编号，但没有预先规定“第1项就是杯子、第2项就是桌子”；它们不是人工物体类别，也不是图片中的 K 个像素。引入这个预测头，是为了给特征一致性提供一个便于训练的目标；下游任务通常使用编码器特征，而不是把这些分组编号直接当成机器人动作。

设教师看到视图 x_1，给出 $q = (0.8, 0.1, 0.1)$；学生看到同图的另一视图 x_2，给出 $p = (0.2, 0.5, 0.3)$。这是一个训练中某时刻的构造例子，不是人工指定的红杯标签。损失为

$$L = -\sum_{k=1}^{3} q_k \log p_k \approx 1.477.$$

先把学生温度取为 1，学生各分数的导数就是 $p - q = (-0.6, 0.4, 0.2)$。更新会提高学生第一项分数，降低其余两项，让局部裁剪的预测接近全局裁剪的目标。由于学生分数来自编码器，梯度也会更新视觉特征。一次训练可以同时做“教师看 x_1、学生看 x_2”和相反方向；有多个裁剪时，再对不同视图配对求平均。

教师在这一步不接收损失的梯度，而在学生更新后使用**指数移动平均**（Exponential Moving Average，EMA）：

$$\theta_t \leftarrow m\theta_t + (1 - m)\theta_s.$$

例如 $m = 0.99$，新教师参数保留旧值的 99%，吸收学生新值的 1%。教师和学生通常从相同参数开始，之后走两种更新路径。缓慢变化的教师使目标不会随学生的一次更新立刻剧烈改变；停止目标分支的梯度则让本步明确地由学生去匹配目标。单有移动平均还不能保证学到有用特征，还必须排除下一种取巧方式。

为什么会坍缩，坍缩后会怎样

表示坍缩（Representation Collapse）是不同输入被映射成几乎一样的表示。如果所有图片的特征都等于一个向量，红杯和蓝碗就无法再根据该特征区分。动作头接收到相同特征，也无法从这部分信息判断该抓谁。这不是“特征维度少了一点”，而是训练目标失去了我们需要的信息差异。

只要求同图两种视图相同，存在这个漏洞。假如教师和学生对所有图片都输出 $(1, 0, 0)$，跨视图预测当然一致，交叉熵还可以接近零，却完全不需要辨别图片。另一种退化是每张图片都输出 $(1/3, 1/3, 1/3)$：此时交叉熵不为零，但学生分数梯度 $p - q = 0$，没有区分图片的学习信号。这里说的是预测头的两种退化；检查最终表示时，还应实际查看编码器特征是否失去差异。

因此，训练需要兼顾两件事：**同一图片的不同视图应对应，不同图片仍要保留可区分的信息。**把所有输入都变得一样，只满足了前半句。DINO 的防坍缩设计就是围绕这个缺口安排的。

居中和温度，分别在改变什么

居中（Centering）针对“某一输出维度总是占优势”。设教师对三张不同图片的原始分数为 $(6, 2, 0)$、$(5, 3, 0)$、$(5, 0, 3)$。第一维每次都很大，softmax 容易让所有图片都偏向第一项。按批次逐维平均，得到 $c = (16/3, 5/3, 1)$；减去它后，三张图片分别变成

$$\left(\frac{2}{3}, \frac{1}{3}, -1\right), \qquad \left(-\frac{1}{3}, \frac{4}{3}, -1\right), \qquad \left(-\frac{1}{3}, -\frac{5}{3}, 2\right).$$

现在最突出的维度分别是第1、2、3项。这个例子展示：减去跨图片普遍存在的偏高分数，可以让各图自身的差异重新显现。实际 DINO 用历史批次的移动平均中心，不是每张图减自己的均值，也不是在某一张图的概率向量里强行平均分配。DINO 作者实现

居中可能把分数拉得很接近，于是还需要**锐化**（Sharpening）：在教师 softmax 中使用较低的温度。对分数 $(0.2, 0, -0.2)$，温度 1 得到的概率约为 $(0.402, 0.329, 0.269)$；温度 0.1 则得到 $(0.867, 0.117, 0.016)$。原有的差异没有改变方向，却形成更明确的目标，使学生不能总是用一个模糊的平均答案应付。

两种操作解决不同问题：居中抑制一个维度对全部图片的长期垄断，较低教师温度保留每张图片内部的明确偏好。仅锐化可能让所有图片更坚决地选择同一项；仅居中又可能把目标推向均匀。它们与移动平均教师、多视图目标一起构成原始 DINO 的训练设计。若所有分数已经严格相等，降低温度仍得到均匀分布，因此不应把这些操作理解成“任何常量解从此在数学上都不可能”。原始 DINO 的防坍缩分析

把这些关系写成一步训练，教师目标为 $q = \mathrm{softmax}((z_t - c)/\tau_t)$，学生预测为 $p = \mathrm{softmax}(z_s/\tau_s)$。先用二者的交叉熵更新学生，再更新教师与中心。下面直接用三张图片的教师分数，查看一次损失怎样产生不同的学习方向。

算法步骤：

1. 准备同图跨视图的教师和学生分数，每行对应一张图片。
2. 从教师分数减去逐维中心，再得到固定的目标概率。
3. 对学生预测计算交叉熵，反传到学生分数。
4. 检查每张图片的目标和梯度是否有区别。

```
import torch
from torch.nn import functional as F

# 步骤1：用前面的三张图片分数，学生暂时都预测均匀
teacher_logits = torch.tensor([[6., 2., 0.],
                               [5., 3., 0.],
                               [5., 0., 3.]])
student_logits = torch.zeros(3, 3, requires_grad=True)
center = teacher_logits.mean(dim=0, keepdim=True)
# 步骤2：这个例子用当前批次均值展示居中；实际用移动中心
with torch.no_grad():
    target = F.softmax((teacher_logits - center) / 0.1, dim=-1)
# 步骤3：固定教师目标，只训练学生去匹配
log_prediction = F.log_softmax(student_logits, dim=-1)
loss = -(target * log_prediction).sum(dim=-1).mean()
loss.backward()
# 步骤4：三张图的主要目标维度分别为 0、1、2
print(target.argmax(dim=-1))
print(student_logits.grad)
```

输出中的每行梯度与 $(p - q)/3$ 对应，因为这里对三张图片求均值，学生温度取1。三张图片最需要提高的分数不再是同一个维度。代码只展开预测头的一个批次计算；完整网络中这些分数来自学生编码器，梯度会继续经过投影头和编码器，随后才按前面的公式更新移动平均教师。

学这一类视觉预训练，对机器人有什么用

对于使用现成编码器的 VLA 项目，你首先需要知道它学到什么、应该提取哪一层特征，以及哪些变化被训练目标要求忽略。DINO 不用文字也能从大量图片形成视觉表示，可作为物体对应、局部视觉特征和策略输入的基础；它不是一个直接预测夹爪动作的模型。DINOv2 延续这条路线，并结合更大数据和图像、图块层面的训练目标改进特征，不应把原始 DINO 的简化步骤当成所有后续版本的完整算法。DINOv2 作者仓库

更可迁移的思想是：自监督目标也会有“损失不错、信息却没学到”的取巧解。为世界模型、蒸馏或跨模态对齐设计损失时，应实际检查输出是否有区别，以及这种区别是否与任务相关。机器人尤其还要问：我们希望光照变化不影响物体身份，但杯子向左移动后，特征是否保留了位置变化？全局识别表示、局部图块特征和动作精度分别需要检查，不能用一次图像分类分数替代。

MAE：用被遮住的内容作标签

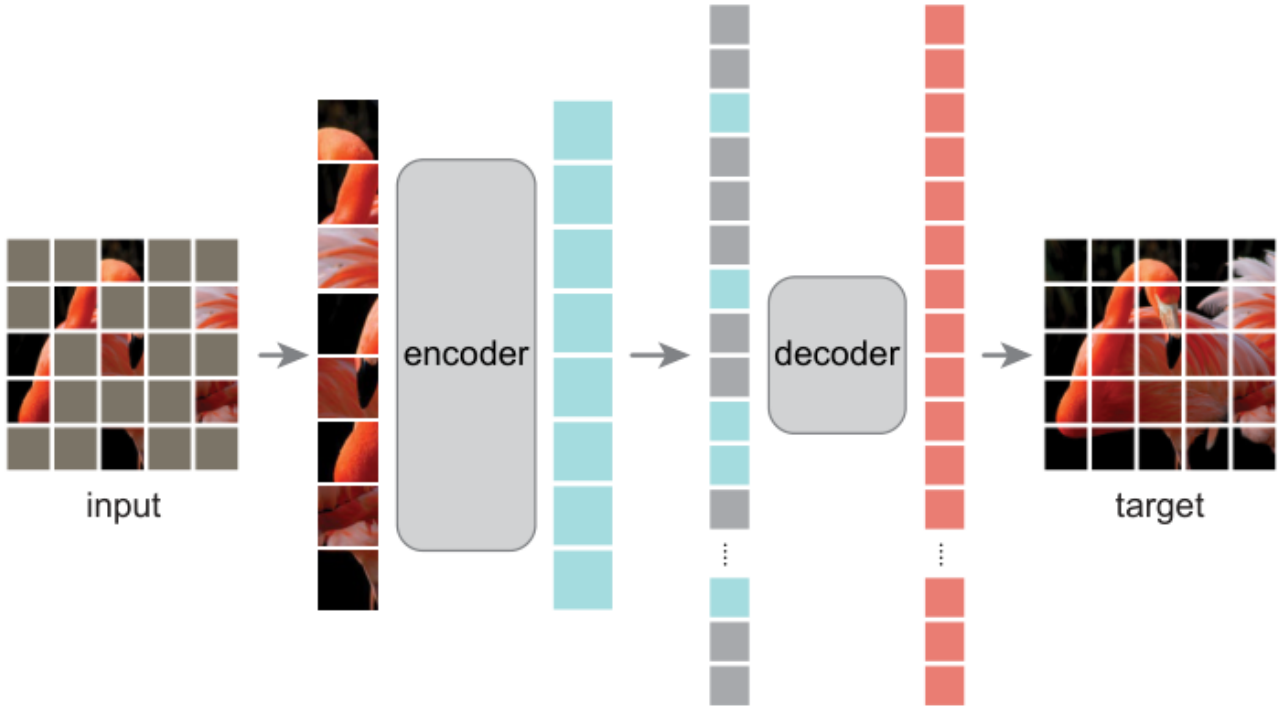


图1.9：He 等，MAE 论文 Figure 1。可见图块先编码，再与代表缺失位置的掩码 token 一起用于重建。原论文。

原图 input 是仅保留部分图块的图片，encoder（编码器）处理这些可见内容；decoder（解码器）接收可见特征与缺失位置的占位向量，输出完整图块预测；target 是遮挡前的原图。占位向量只说明“这里有待恢复的位置”，没有包含被遮住的真实像素。损失主要对照被遮挡区域，使预测必须利用上下文。

MAE（Masked Autoencoder，掩码自编码器）采用另一个直观目标。先把原图切成图块，遮住其中一部分；编码器处理可见图块，解码器根据可见信息和位置预测被遮住部分。标签就是遮挡前的原始像素，不需要人另外标注。编码器由此学习图像中哪些结构可以相互解释，训练后可留下编码器供下游使用。MAE 作者实现

像素重建也有自己的偏好：大量背景纹理可能占据损失，却未必对抓取最重要。比较视觉预训练路线时，重点应是训练目标保留的信息能否服务当前任务。CLIP、SigLIP 借文字建立语义对应，DINO 借跨视图预测形成视觉一致性，MAE 借缺失内容重建学习结构；是否冻结、保留图块特征或用动作数据适配，再由机器人任务决定。

1.3.4 表示应保留什么，取决于后续任务

一张杯子照片有颜色、纹理、背景、位姿、把手、反光和遮挡。把它压缩成向量，意味着选择保留哪些差异。用于“是不是杯子”的表示，可以忽略细小姿态；用于夹爪对准杯口的表示，却需要区分毫米级偏移。语义预训练有助于识别物体，不能据此假定它已经保留了控制所需的精细空间信息。

对比学习希望同一对象的不同视图在表示空间接近，这需要决定哪些变化不应改变答案。若任务是识别杯子，轻微裁剪可能合理；若任务是判断杯子在图像左边还是右边，水平翻转就必须同时修改语言、几何和动作标签。所谓不变性，是主动丢弃某类信息；所谓等变性，是输入变化时输出按已知规则变化。前者适用于身份稳定，后者常适用于几何关系，不能互相替代。

检验表示可以分成三层：冻结编码器后，小模型能否读出物体位姿；端到端训练后策略能否完成动作；改变背景、视角和指令后能力是否保留。线性探针表现好，只说明相应信息可被简单读出，不证明闭环控制稳定。重建图像清晰，也不证明潜在表示正确表达接触。

1.3.5 练习题

题1.3-1 CLIP 对一个批次的 N 对图文构造 N×N 相似度矩阵。为何只把正配对的相似度增大还不够？为什么目标识别更准确后，机器人仍可能抓偏？

题1.3-2 一种跨视图自监督训练中，教师与学生对所有图片都输出 $(1/2, 1/2)$。求交叉熵及对学生分数的梯度（温度取1），说明为何预测一致却没有辨识信息。若二者对所有图片都趋于同一个 one-hot 输出，又会怎样？原始 DINO 的居中与教师低温分别针对什么问题，为什么不能说其中任何一项单独保证不坍缩？

1.3.6 参考答案

题1.3-1答案 CLIP 的任务是让一张图像在本批次的 N 条文字中找出与它配对的那一条，也让文字反过来找到图像。第 i 张图像对应的概率为

$$p_{ij} = \frac{e^{s_{ij}}}{\sum_k e^{s_{ik}}}, \qquad L_i = -\log p_{ii}.$$

如果所有图文都映射到同一个向量，正配对相似度也可以很高，但每个候选都一样高，$p_{ii}=1/N$，模型仍分不出哪条文字对应图像。目标要增大正配对相对于其他候选的优势，而非只增大一个绝对分数。对于一行，梯度为 $\partial L_i/\partial s_{ij}=p_{ij}-1[j=i]$：正配对被拉高，其余候选被压低。

“这是杯子”回答类别或语义匹配；“夹爪应移动到哪里”还需要杯柄位置、三维姿态、相机到机械臂的变换、夹爪几何及控制误差。前者改善不会自动补齐后者。例如图像中的杯柄识别正确，但相机外参有偏差，转换后的抓取点仍可能离杯柄数厘米。可分别测语义目标选择准确率、位置/姿态误差及闭环抓取成功率，以确定瓶颈在哪个环节。

题1.3-2答案 把教师分布记为 q、学生分布记为 p。单个跨视图配对的交叉熵为

$$L = -\sum_i q_i \log p_i = -\tfrac{1}{2}\log\tfrac{1}{2} - \tfrac{1}{2}\log\tfrac{1}{2} = \log 2 \approx 0.6931.$$

温度为1时，对学生logits s 的梯度为 $\partial L/\partial s_i = p_i - q_i$，两个分量都是0。损失不为0，却已是“教师固定给均匀标签”情况下学生的最优值。由于每张图都给出同一个分布，这个输出无法区分杯子、柜门或背景，跨视图吻合并没有带来辨识信息。

若教师和学生对所有图都趋向同一个one-hot，例如(1,0)，交叉熵趋于0，梯度也趋于0。这样甚至能达到看起来更漂亮的损失，但仍无法区分图片。若部署所用特征也变成常量，后续分类或动作头就无法依据图像变化选择动作；仅看到投影头输出坍缩，则至少说明当前学习目标没有提供区分信息，不能直接断言主干所有特征严格相同。

原始DINO居中减去教师各输出维跨批次的滑动均值，抑制某一维对所有图都持续偏高；它却可能把目标推向均匀分布。教师低温放大一张图各维之间的相对差异，使目标更尖锐，抵抗这种均匀趋势；单用它又可能强化所有图占同一维。两种操作配合动量教师是论文验证的训练配方。严格相等的logits无论温度多低仍均匀，停止梯度也能保留无信息目标，因此任何一项都不能单独作为“不坍缩”的保证。

1.4 多模态 token、注意力与信息掩码

1.4.1 文字、图像、状态与动作如何变成 token

Token 是序列模型处理的一个信息单元。文字经过分词后可以产生一个或多个 token；图像小块、关节状态和动作向量也可以投影成 token。它们最后都用向量参与计算，但原始含义不同：有的表示词片段，有的表示局部图像，有的表示机器人状态。

一张 224×224 图像按 16×16 切块得到 196 个图像 token。两台相机、两帧历史会得到 784 个，尚未计入指令和状态。较小的 patch 保留更多局部细节，也增加后续**注意力**成本；缩小图像节省计算，却可能丢失夹爪接触等细小线索。分辨率选择需要同时考虑任务信息与推理预算。

图1.10：文字、图像与状态通过各自编码器变成可连接的向量。训练还需建立它们之间的语义和动作关系。

每个 token 往往还需要来源、位置、相机和时间标识。两个数值相同的向量若一个来自腕部相机、一个来自外部相机，几何含义不同。把它们拼接而不区分视角，可能迫使模型从外观猜来源。填充 token 用于统一批次长度，掩码必须阻止模型把无效内容当作真实信息。

输入编码与输出离散化要分开。关节角投影成 token，并没有把执行动作限制到有限档位；把动作**量化**成 256 个类别，才引入输出分辨率。模型论文中的“tokenized actions”要核对是连续嵌入、固定分箱还是学习到的压缩码，这三种方式的误差来源不同。

1.4.2 注意力：先决定读谁，再决定读出什么

一个动作生成位置需要同时利用目标、场景和机器人状态。假如直接把所有图像区域求平均，红杯、蓝杯和背景会混成一个摘要；如果总是读取固定位置，目标换了位置就可能读错。**注意力**（Attention）让每个输出位置根据当前输入，为可读取的信息分配不同权重。

先从一个查询向量 q 出发。假设有 N 个信息位置，每个位置提供一个键 k_j 和一个值 v_j。我们依次计算

$$s_j = \frac{q^\top k_j}{\sqrt{d_k}}, \qquad p_j = \frac{e^{s_j}}{\sum_{\ell=1}^{N} e^{s_\ell}}, \qquad o = \sum_{j=1}^{N} p_j v_j.$$

这里的**查询**（Query，Q）表示当前输出位置用于寻找信息的特征，**键**（Key，K）表示各信息位置供比较的特征，**值**（Value，V）则是最终读取的内容。s_j 是匹配分数，p_j 是读取权重，o 是读出的向量。三个公式分别对应“比较”“分配”“汇集”，输出不是最大分数的键，也不是权重本身。

如果两个分数为 0 和 $\log 3$，权重为 $1/4$ 和 $3/4$。值是 2 和 6 时，输出为 $\frac{1}{4}\times 2+\frac{3}{4}\times 6=5$。分数相同而值改变，读取比例仍相同，内容会改变；值相同而分数改变，读取比例会改变，内容却可能不变。这说明匹配与内容是两个职责，把它们分开投影比强制使用同一种特征更灵活。

在神经网络中，Q、K、V 通常由输入经过可训练线性层得到。本节把每个 token 写成矩阵的一行，自注意力输入 $X \in \mathbb{R}^{N\times D}$，计算 $Q=XW^Q, K=XW^K, V=XW^V$。W^Q, W^K, W^V 都由最终任务损失训练，不需要人工给每个图块标注“它的正确键向量”。从红杯任务看，网络有机会学习用颜色与语义进行匹配，再把几何等内容读出来；这只是可能形成的分工，实际特征不一定对应这些人类名称。

将多个查询一并计算，得到矩阵公式

$$S = \frac{QK^\top}{\sqrt{d_k}}, \qquad P = \mathrm{softmax}_{\text{每一行}}(S + M), \qquad O = PV.$$

如果有 N_q 个查询、N_k 个键，$QK^\top$ 的形状是 $N_q \times N_k$：每一行回答“这个查询怎样读取所有键”。softmax 沿键轴计算，使同一行允许位置的权重之和为 1；P 再乘 V，才得到每个查询自己的内容摘要。**掩码**（Mask）M 在允许位置为 0，禁止位置为负无穷，使禁止位置不参与分配。

为什么要除以键维度的平方根

缩放不是把“越多的特征变得越不重要”，而是控制点积分数的数值尺度。假设初始化附近 q_r, k_r 独立、均值为 0、方差为 1，且各维乘积相互独立，点积 $s = \sum_{r=1}^{d_k} q_r k_r$ 满足

$$\mathbb{E}[s] = 0, \qquad \mathrm{Var}(s) = d_k, \qquad \mathrm{Var}\left(\frac{s}{\sqrt{d_k}}\right) = 1.$$

若不缩放，维度越大，随机分数的波动通常越大。softmax([10,0]) 约为 [0.99995,0.00005]；大多数位置几乎读不到内容，而 $p(1-p)$ 也很小，不利于通过分配权重学习。除以 $\sqrt{d_k}$ 使不同维度下的初始分数较可比，避免模型一开始就过于确信某个位置。训练后向量不一定仍满足独立单位方差假设，缩放是依据这一分析采用的结构设计。Transformer 原论文

本节的 V 是供加权汇集的内容向量；第3章的 V^π 是平均长期回报。二者只是使用了同一个字母。

1.4.3 从经典图读懂单头与多头计算

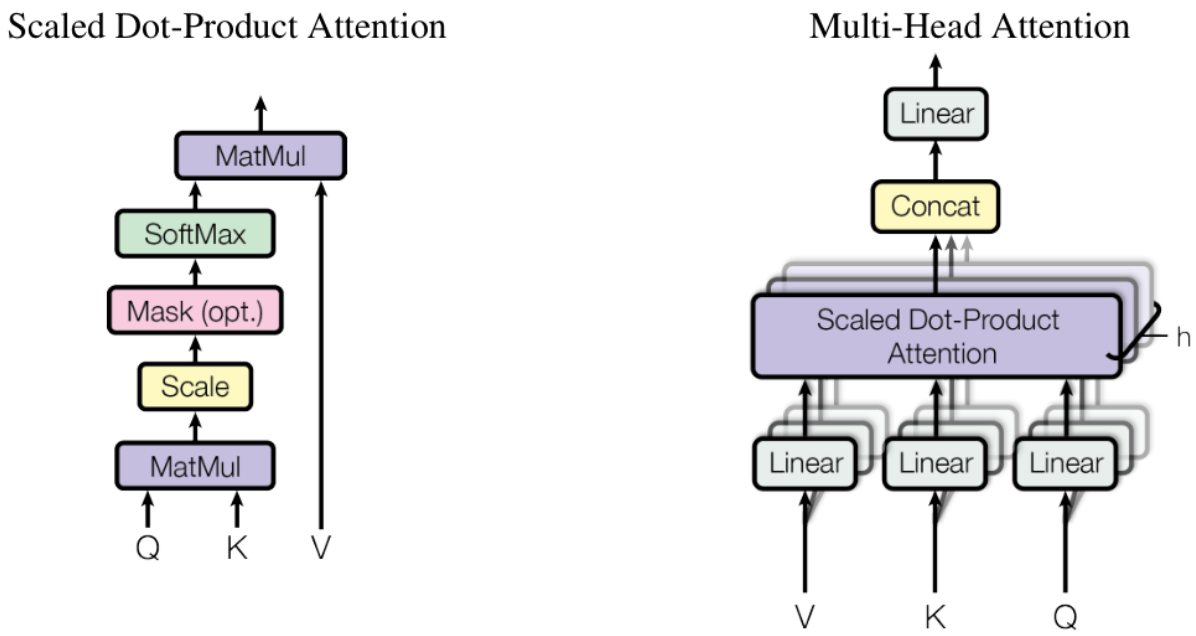


图1.11：Vaswani 等，《Attention Is All You Need》Figure 2。左侧是一次缩放点积注意力，右侧是在不同可学习投影下并行进行多次注意力。原论文。

读左图时从下往上看。Q 与 K 进入第一处 MatMul，算出所有查询与键的分数；Scale 缩放分数；Mask 屏蔽不允许的位置；SoftMax 得到权重；第二处 MatMul 把这些权重乘 V，输出内容。右图把这条计算重复在多个头上，再合并结果。

原图标签	英文或中文含义	在公式中的具体运算
MatMul	Matrix Multiplication，矩阵乘法	下方是 $QK^\top$；上方是 PV
Scale	缩放	把分数除以 $\sqrt{d_k}$
Mask（opt.）	掩码；optional 表示可按任务选用	对禁止位置加负无穷

原图标签	英文或中文含义	在公式中的具体运算
SoftMax	将分数指数化并归一化的函数	每个查询在可读键之间分配权重
Linear	可训练线性映射	生成每头 Q、K、V，以及最终输出映射
Concat	Concatenation，拼接	沿特征轴放在一起，保留各头输出
h	注意力头数	右图并行分支的数量

一个头的限制：所有内容维度共用一套读取比例

单头输出 $o = \sum_j p_j v_j$ 中，同一个位置 j 的所有值维度都乘同一权重 p_j。假设动作既需要红杯的几何，又需要夹爪的姿态，一个头只能在本次计算里给出一套位置分配。**多头注意力**（Multi-Head Attention，MHA）引入多组投影，让同一输出位置形成多套读取比例，再把各自读出的结果交给后续网络组合。

设输入宽度为 D，头数为 H，常取每头宽度 $d = D/H$。第 r 个头使用自己的三个矩阵：

$$Q_r = X_q W_r^Q, \qquad K_r = X_c W_r^K, \qquad V_r = X_c W_r^V,$$

$$P_r = \mathrm{softmax}\left(\frac{Q_r K_r^\top}{\sqrt{d}} + M\right), \qquad O_r = P_r V_r,$$

$$O = \mathrm{Concat}(O_1, \ldots, O_H) W^O.$$

X_q 是发出查询的表示，X_c 是可读取的上下文；每个 W_r^Q, W_r^K, W_r^V 的大小为 $D \times d$，因此一个头可以读取输入的全部 D 个维度，再学出自己的 d 维表示。**分头不是先把原始信息切开、禁止各头看到其他部分**。它先进行可训练投影，再把不同投影的输出分别用于匹配。输出矩阵 $W^O \in \mathbb{R}^{Hd \times D}$ 则把各头内容组合回后续层使用的宽度。

为什么一次大投影再 reshape，等于分别做各头投影

实际代码常只写一个 Q 线性层，看起来没有 H 套矩阵。原因是可以把各头矩阵并排放在一起：

$$W^Q = [W_1^Q \mid W_2^Q \mid \cdots \mid W_H^Q], \qquad X_q W^Q = [X_q W_1^Q \mid \cdots \mid X_q W_H^Q].$$

右边本来就是每头投影的结果。把最后一轴拆成 $H \times d$，只是把这些已经得到的数放到各自的头轴；随后每头独立计算 softmax，才形成不同的读取关系。不能先计算一个大头的 softmax，再把结果切开，因为 softmax 是非线性的，两个过程通常不相等。UvA 多头注意力教程

取 $D = 256, H = 8, d = 32$，八个 Q 投影合计参数数为 $8 \times 256 \times 32 = 256^2$，等于一个 256×256 的 Q 投影。K、V 也一样。这解释了在总宽度固定时，增加头数不一定增加投影参数量：每头同时变窄。不过头越多，每头能够使用的表示宽度越少，显式保存的注意力分数也从一个矩阵变为 H 个矩阵。多个头提供不同匹配的机会，不保证每个头都学到独特语义，也不保证头数越多越好。

两个头的完整数值例子

为了看见多套读取比例，设投影后有两个头，每头一个查询和两个可读位置：

$$q_1 = (1, 0), \quad q_2 = (0, 2), \qquad K_1 = K_2 = \begin{bmatrix} 1 & 0 \\ 0 & 2 \end{bmatrix}, \quad V_1 = V_2 = \begin{bmatrix} 2 & 0 \\ 0 & 4 \end{bmatrix}.$$

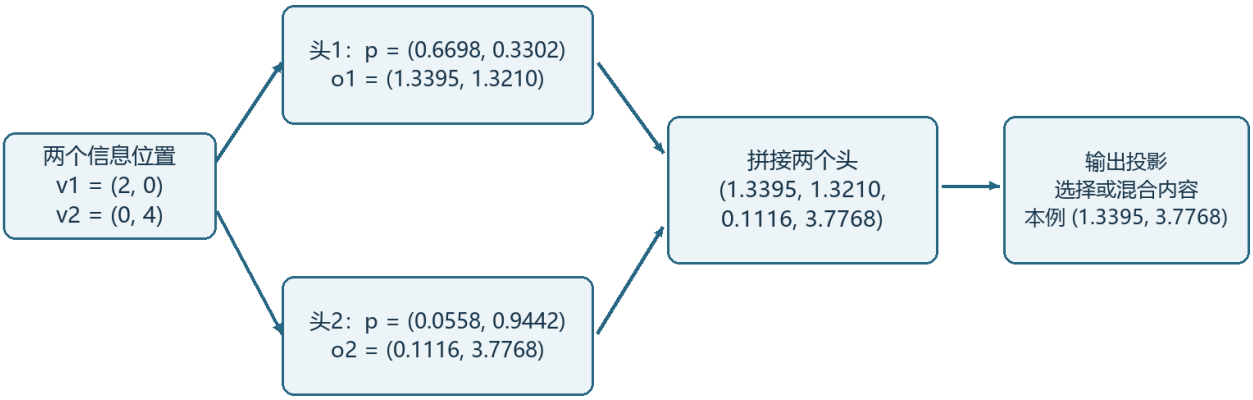
这些数是不同投影的示例输出，不预先规定每个头负责什么。头1得分为 $(1/\sqrt{2}, 0)$，权重约 $(0.6698, 0.3302)$，读出 $o_1 \approx (1.3395, 1.3210)$；头2得分为 $(0, 4/\sqrt{2})$，权重约 $(0.0558, 0.9442)$，读出 $o_2 \approx (0.1116, 3.7768)$。一个头较偏向第一个位置，另一个较偏向第二个位置。

拼接后得到 $(1.3395, 1.3210, 0.1116, 3.7768)$。若输出投影取

$$W^O = \begin{bmatrix} 1 & 0 \\ 0 & 0 \\ 0 & 0 \\ 0 & 1 \end{bmatrix},$$

最终两维输出约为 $(1.3395, 3.7768)$：第一维使用头1读出的第一维内容，第二维使用头2读出的第二维内容。相比只保留一套读取比例，这里能让不同内容采用不同位置侧重。本例用两维输出展示选择；接入残差块时，通常将输出恢复为输入宽度 D，才能与输入相加。真实 W^O 会学习连续组合，不一定像本例只选两项。

多头注意力：同一输出位置，形成两套读取比例



各头可以读取完整输入，再通过不同投影学习不同的匹配；头的功能不是人工指定的。

图1.12：同一查询位置在两个投影空间形成不同权重，得到两个内容摘要。拼接保留两份结果，输出投影再选择或混合它们。

自注意力、交叉注意力与可读取的信息

自注意力（Self-Attention）取 $X_q = X_c$，同一组 token 相互读取；**交叉注意力**（Cross-Attention）让查询和上下文来自不同来源，例如三个动作查询读取七个视觉语言位置。因此查询长度与键长度不必相同，矩阵只要求对应的键、值位置彼此对齐。

因果掩码（Causal Mask）用于预测尚未产生的后续 token，只允许读取已知前缀。扩散、流匹配的动作块则读取当前带噪候选，各候选位置通常可以互相利用；候选中的数值不等于把真实答案提前告诉模型。训练输入到底是已知前缀、带噪候选，还是标签，决定了应采用什么信息边界。

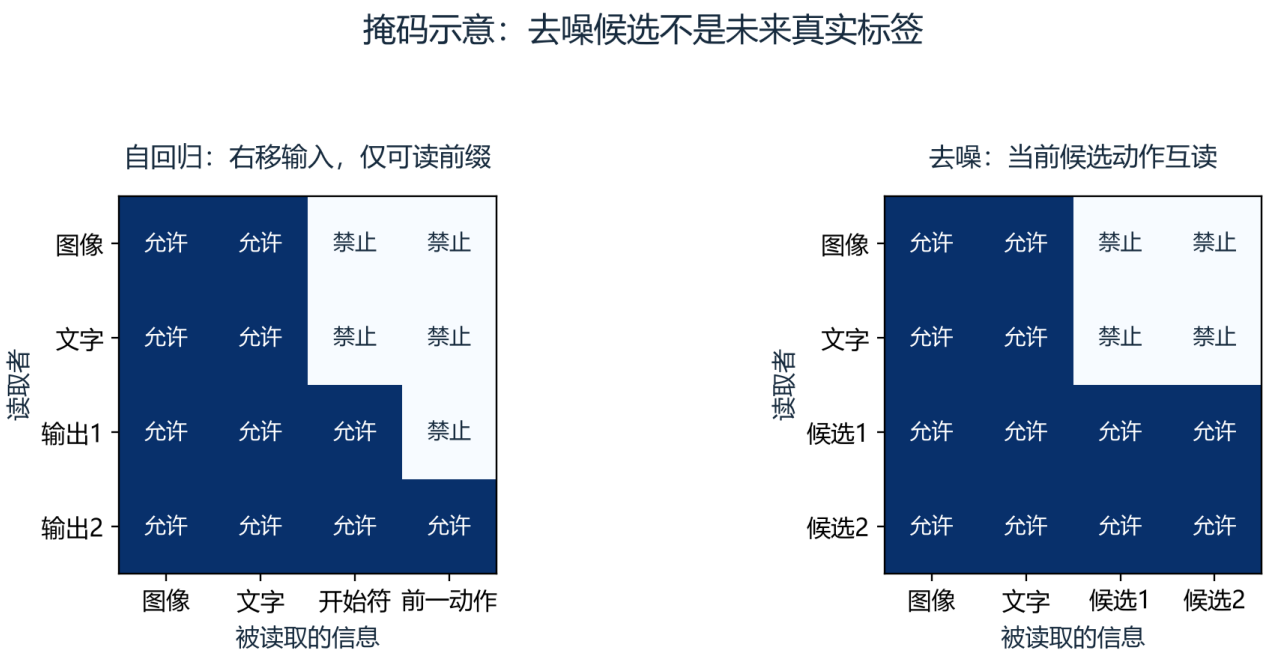


图1.13：行表示发出查询的位置，列表示被读取的位置。左侧只读取已知前缀；右侧动作位置读取图像文字条件和当前候选动作。图中的允许格子对应掩码为0的位置。

自回归训练还要将标签右移：预测第二个动作时可以看到第一个动作，不能把第二个动作本身放在第二个输入位置再声称已经遮住未来。后面完整 Transformer 图中的“Outputs shifted right”，就是这一步输入组织。

1.4.4 从公式到张量形状

把注意力实现成代码，先为每个轴确定含义。本节模块接收已经编码的 token；视觉编码器负责产生图像特征，动作头再将注意力输出变成动作。这样就能把手写模块放回完整策略的数据流。

设输入 X 的形状为 [B,T,D]，分别代表批次、token 数和模型维度。它不是一条机器人轨迹必然具有的形状：图像 patch、语言 token、历史观测和动作 token 都可能进入序列，必须另有位置与模态约定。线性投影生成 Q、K、V；这里的 V 是供聚合的内容向量，与强化学习价值函数表示不同对象。分成 H 个头后，它们的形状为 [B,H,T,d]，其中 d=D/H。先让模型学习投影，再把输出按头划分；不是把原始输入硬切成几份就完成了多头学习。

运算	输入或输出形状	它解决的问题
X 乘投影矩阵	[B,T,D]	为匹配与被聚合内容学习不同表示
分头并调整轴	[B,H,T,d]	每个头分别建立 token 间关系
Q 与 K 的转置相乘	[B,H,Tq,Tk]	每个查询与每个键得到一个分数
对最后一轴 softmax	[B,H,Tq,Tk]	对同一查询可读取的键分配权重

运算	输入或输出形状	它解决的问题
权重乘 V	[B,H,Tq,d]	按权重汇集信息
合并头、输出投影	[B,Tq,D]	让后续网络组合各头结果

Tq 与 Tk 可以不同。动作查询从视觉与语言上下文读取信息时，三十个动作查询可以对应数百个上下文 token。这是交叉注意力；自注意力只是 Q、K、V 来自同一输入的特殊情况。

1.4.5 允许位置内的 softmax

掩码注意力要让禁止位置完全退出归一化。实现时先屏蔽分数，再减可读最大值计算指数；若一行没有可读键，这段实现返回零权重，避免出现零除。有效查询通常应至少保留一个键。这个零值只描述注意力聚合分支，残差仍会保留原输入。

算法步骤：

1. 把允许掩码扩展到分数矩阵的形状。
2. 将禁止位置的分数设为负无穷。
3. 取可读位置的最大分数；全遮蔽行改用零。
4. 减最大值后取指数，并在可读位置归一化。

```
def safe_softmax(scores, allow):
    # 步骤1：把允许掩码扩展到分数矩阵的形状
    allow = torch.broadcast_to(allow, scores.shape)
    # 步骤2：将禁止位置的分数设为负无穷
    masked = scores.masked_fill(~allow, float('-inf'))
    # 步骤3：取可读位置的最大分数；全遮蔽行改用零
    maximum = masked.amax(dim=-1, keepdim=True)
    maximum = torch.where(torch.isfinite(maximum), maximum, 0.0)
    # 步骤4：减最大值后取指数，并在可读位置归一化
    exp = torch.exp(masked - maximum)
    tiny = torch.finfo(exp.dtype).tiny
    return exp / exp.sum(dim=-1, keepdim=True).clamp_min(tiny)
```

在实际库函数中，掩码还要按接口约定转换。PyTorch 的函数式缩放点积注意力用 True 表示允许读取，而 nn.MultiheadAttention 的布尔填充掩码用 True 表示忽略。前面的手写函数采用允许掩码；接入后者时需要取反，使同一信息关系保持一致。缩放点积注意力文档，多头注意力文档。

1.4.6 手写多头注意力：把每个公式放回代码

下面给出一个可以直接实例化的多头模块。输入 x 是查询表示；没有额外上下文时做自注意力，有 context 时从它生成键和值。输出既包括融合后的特征，也包括每头的权重，便于检查刚才的矩阵计算。

算法步骤：

1. 为 Q、K、V 和合头输出建立可训练投影。
2. 投影后把最后一轴分成头数与每头维度，换成批次、头、位置、特征的顺序。

3. 计算每头所有查询与键的缩放点积。

4. 按可读取的位置计算 softmax，并用权重汇集值。

5. 把各头内容拼接，再经过输出投影。

这个模块的分数形状是[B,H,Tq,Tk]，掩码也要按这些轴对齐。若key_valid为[B,Tk]，表示每条样本哪些键有效，应传入key_valid[:,None,None,:]，使所有头和查询共用该样本的键掩码；全批共用的因果掩码可以是[Tq,Tk]。广播从最右一轴对齐，直接传[B,Tk]可能把批次错当成查询轴。需要同时满足因果性时，把两种允许掩码按逻辑“与”合并。

```
import math
import torch
from torch import nn

class TutorialMHA(nn.Module):
    def __init__(self, width, heads):
        super().__init__()
        assert width % heads == 0
        self.heads = heads
        self.d = width // heads
        # 步骤1：每头矩阵并排存放为一个大投影
        self.q = nn.Linear(width, width)
        self.k = nn.Linear(width, width)
        self.v = nn.Linear(width, width)
        self.out = nn.Linear(width, width)

    def split(self, x):
        b, t, _ = x.shape
        # 步骤2：[B,T,H*d] -> [B,T,H,d] -> [B,H,T,d]
        return x.reshape(b, t, self.heads, self.d).transpose(1, 2)

    def forward(self, x, context=None, allow=None):
        source = x if context is None else context
```

```
        q = self.split(self.q(x))
        k = self.split(self.k(source))
        v = self.split(self.v(source))
        # 步骤3：每个头分别得到 [B,H,Tq,Tk] 的分数
        scores = (q @ k.transpose(-2, -1)) / math.sqrt(self.d)
        if allow is None:
            allow = torch.ones_like(scores, dtype=torch.bool)
        # 步骤4：safe_softmax 见上节，禁止位置不参与归一化
        weights = safe_softmax(scores, allow)
        per_head = weights @ v
        b, _, tq, _ = per_head.shape
        # 步骤5：先换回位置轴，再把每头内容并排放在一起
        merged = per_head.transpose(1, 2).reshape(b, tq, -1)
        return self.out(merged), weights

model = TutorialMHA(width=12, heads=3)
x = torch.randn(2, 3, 12)        # 3 个查询位置
context = torch.randn(2, 7, 12)  # 7 个可读位置
output, weights = model(x, context)
print(output.shape)   # torch.Size([2, 3, 12])
print(weights.shape)  # torch.Size([2, 3, 3, 7])
```

reshape 和 transpose 没有学习新的关系。它们把已经投影得到的数按正确轴组织起来，使矩阵乘法发生在各头内部；真正可学习的是投影参数。最后 weights 的第三轴是查询位置、第四轴是键位置，因此查看一个查询的读取比例，应固定批次、头和查询，沿最后一轴观察。

1.4.7 注意力的反向传播：任务误差怎样改变“读谁”

Q、K、V 没有各自的人工标签，但动作误差能够训练它们。为看清路径，暂时只考虑单头，不加输出投影：$S = QK^\top/\sqrt{d}$、$P = \mathrm{softmax}(S)$、$O = PV$。设后续网络已经给出 $G = \partial L/\partial O$，这表示注意力输出每一项怎样改变会影响最终损失。

先经过聚合 $O = PV$。对一个输出元素 $O_{i\ell} = \sum_j P_{ij}V_{j\ell}$，有 $\partial O_{i\ell}/\partial V_{j\ell} = P_{ij}$。所以值向量获得的梯度要按它被读取的程度加权，矩阵形式为

$$\frac{\partial L}{\partial V} = P^\top G, \qquad G_P = \frac{\partial L}{\partial P} = GV^\top.$$

第一项告诉各位置“读出的内容应怎样改”；第二项告诉每个查询“增加某个位置的权重，会给损失带来什么变化”。接着要将权重梯度传回分数。对某一行 $p_i = e^{s_i}/\sum_j e^{s_j}$，softmax 的导数为 $\partial p_j/\partial s_i = p_j(\mathbf{1}_{i=j} - p_i)$。代入链式法则，得到

$$\frac{\partial L}{\partial s_i} = \sum_j \frac{\partial L}{\partial p_j}\frac{\partial p_j}{\partial s_i} = p_i\left(g_i - \sum_j p_j g_j\right), \qquad g_i = \frac{\partial L}{\partial p_i}.$$

减去加权平均不是额外正则化。权重总和固定为1，提高一个位置的权重就意味着其他位置相对减少，因此必须比较它与当前这套分配的平均贡献。如果所有 g_i 相同，换任何分配都不能改善目标，分数梯度为零。

用一行数值看方向：值为 2、6，权重为 $(0.25, 0.75)$，输出为5。令任务损失为 $L = \frac{1}{2}(O-6)^2$，则输出梯度为 -1，权重梯度为 $g = (-2, -6)$，加权平均为 -5，分数梯度为 $(0.75, -0.75)$。梯度下降会降低第一个位置的分数、提高第二个位置的分数，使输出更偏向值6。同时值向量也会由自己的梯度调整，两种变化共同服务最终任务。

最后经过点积，将分数梯度 G_S 传回查询与键：

$$\frac{\partial L}{\partial Q} = \frac{G_S K}{\sqrt{d}}, \qquad \frac{\partial L}{\partial K} = \frac{G_S^\top Q}{\sqrt{d}}.$$

再经过 $Q = XW^Q$ 等线性投影，例如 $\partial L/\partial W^Q = X^\top(\partial L/\partial Q)$，投影矩阵便能被优化器更新。这样，一项最终动作损失就能改变网络怎样匹配、怎样汇集内容。多头只是为各头重复这条路径，再把输出投影的梯度分给各头。

算法步骤：

1. 从输出梯度求值向量和读取权重的梯度。
2. 每行减去权重梯度的加权平均，求分数梯度。
3. 经缩放点积求查询与键的梯度。

```
import numpy as np

# 步骤1：grad_out 是后续网络给出的输出梯度
grad_v = p.T @ grad_out
grad_p = grad_out @ v.T
# 步骤2：每个查询的一行权重单独求导
mean = (grad_p * p).sum(axis=-1, keepdims=True)
grad_s = p * (grad_p - mean)
# 步骤3：对点积的两组输入分别求导
scale = np.sqrt(q.shape[-1])
grad_q = grad_s @ k / scale
grad_k = grad_s.T @ q / scale
```

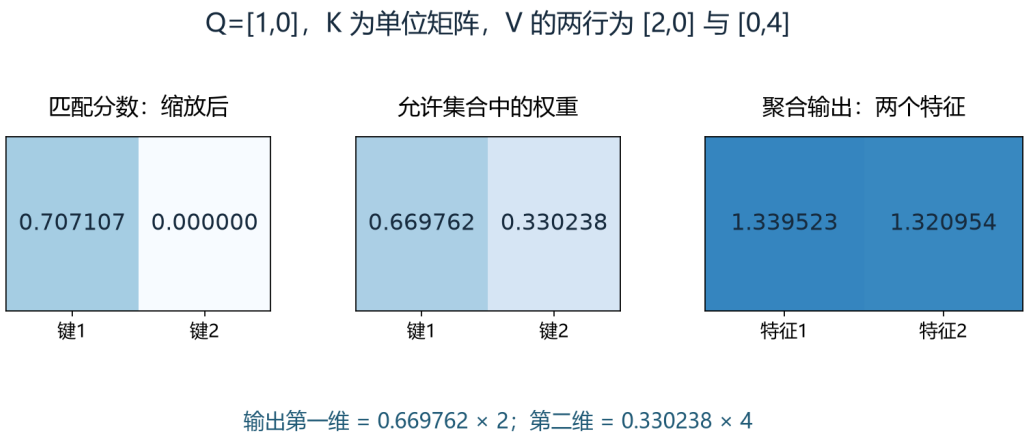


图1.14：查询和键先确定读取比例，再由值向量产生内容。本节练习同时检查前向数值、各轴形状和反向更新，三者应对应同一套计算。

1.4.8 练习题

题1.4-1 单个查询 Q=[1,0]，两个键 K=[[1,0],[0,1]]，值 V=[[2,0],[0,4]]。按键维度 d=2 缩放，求分数、权重和输出；再遮蔽第二个键。

题1.4-2 手写多头注意力时，B=2、Tq=3、Tk=7、D=12、H=3。写出分头后的 Q、K、V，分数矩阵和合头输出的形状，并解释查询长度为什么可以不同于键长度。

题1.4-3 为什么“先 softmax，再把禁止位置清零”通常不同于掩码注意力？怎样检查因果掩码确实阻止未来信息进入输出？

题1.4-4 对 O=PV、P=softmax(S)、S=$QK^T/\sqrt{d}$，已知后续梯度 G=$\partial L/\partial O$。推导 Q、K、V 的梯度，并解释 softmax 梯度中的减平均项。

题1.4-5 输入每个 token 的宽度为 $D=4$，头数 $H=2$，每头宽度 $d=2$，令 $W^Q=[W_1^Q \mid W_2^Q]$。证明一次大投影再重排，与各头分别投影等价。为什么先用全部特征计算一个 softmax，再切分成头，通常不等价？保持总宽度相同时，投影参数数量怎样变化？

1.4.9 参考答案

题1.4-1答案 先计算查询与两个键的点积：[1,0]。键维度为 2，因此缩放后的分数为

$$S=[1/\sqrt{2},0]\approx[0.7071,0].$$

沿两个候选键做 softmax：

$$P=\left[\frac{e^{0.7071}}{e^{0.7071}+1},\frac{1}{e^{0.7071}+1}\right]\approx[0.6698,0.3302].$$

输出是按这些权重加权的值，而非加权的键：

$$O=0.6698[2,0]+0.3302[0,4]\approx[1.3395,1.3209].$$

第一个键与查询更相关，所以它取得较大权重；第二个值的第二维较大，因此其贡献仍明显。

遮蔽第二个键，要在 softmax 前把它的分数变为 −∞。允许集合中只剩第一个键，权重重新归一化为 [1,0]，输出 [2,0]。这里 Scale 指除以 √d，MatMul 指矩阵乘法；第一次矩阵乘法得到查询对键的匹配分数，第二次把值按权重汇总。

题1.4-2答案 每头维度 d=D/H=12/3=4。将投影后的最后一维拆成“3 个头×4 个特征”，再把头维移到序列维之前：

对象	形状	每个轴的含义
Q	[2,3,3,4]	批次、头、查询位置、每头特征
K、V	[2,3,7,4]	批次、头、键/值位置、每头特征
$QK^T/\sqrt{4}$	[2,3,3,7]	每个查询对 7 个键的分数
每头输出	[2,3,3,4]	每个查询的加权值

对象	形状	每个轴的含义
合头及输出投影后	[2,3,12]	批次、查询位置、总特征

之所以可用一次 D→D 的投影再拆头，是因为大投影矩阵可以按输出列分成 W_1、W_2、W_3；计算 XW 后切片，与分别计算 XW_1、XW_2、XW_3 等价。各头有自己学到的投影和注意力权重，并非把同一权重平均复制三遍。

查询长度与键长度分别表示“要产生几个输出”和“每个输出可从多少内容中读取”。交叉注意力中，3 个动作查询可读取 7 个视觉 token；缓存生成中，新查询也可读取更长的历史。要求 Q、K 每头特征维度相同，K、V 的位置数相同，不要求查询数等于键数。

题1.4-3答案 普通 softmax 的分母包括所有位置。随后把禁止位置的权重清零，只删掉它们的贡献，却没有把剩余权重重新归一化。例如两个分数同为 0，softmax 得到 [0.5,0.5]；禁用第二项后变成 [0.5,0]，输出为第一个值的一半。掩码注意力是在允许集合内做 softmax，结果应为 [1,0]。若清零后再对允许项归一化，且不存在整行遮蔽，才可得到数学上等价的结果。

因果性最好用输出不变性检查：关闭 Dropout，固定 token 0…t，只改变 t+1 之后的输入，完整模块在位置 ≤t 的输出应保持一致。也可以检查过去输出对未来输入的梯度为零；但某个样本梯度为零可能来自偶然抵消，因此还应多次改变未来值。检查完整网络时，嵌入、归一化及其他操作也不能跨未来位置混合信息。

同时查看掩码矩阵：查询 t 允许读取的键应恰为 j≤t。使用键值（Key-Value，KV）缓存时比较绝对位置，不能简单按矩阵行号生成下三角。

题1.4-4答案 假设 Q 为 n×d、K 为 m×d、V 为 m×d_v，G 为 n×d_v。先从最后一步 O=PV 反向计算：

$$\frac{\partial L}{\partial V} = P^\top G, \qquad H = \frac{\partial L}{\partial P} = GV^\top.$$

含义是：某个值被哪些查询以多大权重读出，就从这些输出收到相应梯度；H_{ij}则评价“增大查询 i 对值 j 的权重，会怎样改变损失”。

对每一行 p=softmax(s)，有 $\partial p_j/\partial s_k = p_j(1[j=k]-p_k)$。把它与 H 做链式法则，得到

$$D_{ij} = \frac{\partial L}{\partial S_{ij}} = P_{ij}\left(H_{ij} - \sum_k P_{ik}H_{ik}\right).$$

这里减掉的是该行 H 的注意力加权平均。若某个值比当前加权结果更有助于降低损失，其分数就向有利方向变化；只抬高全部分数同样的量不会改变 softmax，所以一行 D 的和为零。

最后对 $S=QK^T/\sqrt{d}$ 求导：

$$\frac{\partial L}{\partial Q} = \frac{DK}{\sqrt{d}}, \qquad \frac{\partial L}{\partial K} = \frac{D^\top Q}{\sqrt{d}}.$$

实现顺序可以对应为 dV=P.T@G; H=G@V.T; D=P*(H-(P*H).sum(-1,keepdim=True)); dQ=D@K/sqrt(d); dK=D.T@Q/sqrt(d)。批次和头维分别独立计算。题中没有掩码；有掩码时，禁止位置的梯度应为零。

题1.4-5答案 设X有T行、每行4维，W_1Q、W_2Q各为4×2。矩阵按列拼接的乘法性质给出

$$XW_Q = X[W_1^Q \mid W_2^Q] = [XW_1^Q \mid XW_2^Q].$$

右侧前两列与后两列分别是两个头的查询。把结果[T,4]改排成[2,T,2]只是重排元素，没有改变数值。因此“一次大投影后拆头”与“两次各头投影”完全等价；K、V同理。学习到的是每头不同的投影矩阵，重排本身不是新的学习步骤。

各头需要分别计算

$$P_h = \mathrm{softmax}(Q_h K_h^\top / \sqrt{2}), \qquad O_h = P_h V_h.$$

若先按全部4维计算，点积为 $Q_1K_1{}^{\mathrm{T}}+Q_2K_2{}^{\mathrm{T}}$，只得到一个共同softmax。比如一个查询在两头的未缩放分数分别为[2,0]和[0,2]，两头权重约为[0.8044,0.1956]及[0.1956,0.8044]；全维相加后分数相同，共同权重为[0.5,0.5]。之后切分值的特征无法恢复丢掉的两套读取权重，因为softmax是非线性且已经归一化。

只计矩阵权重，Q、K、V总计3×4×4=48，输出投影4×4=16，共64个参数；相同总宽度的单头也是64。多头的增加来自多套注意力关系，并非把每套完整4维投影复制两份，所以参数量不随头数直接翻倍。

1.5 完整 Transformer、位置关系与序列计算

前一节只回答“怎样读取信息”。完整模型还要把已知输入编码好，在生成输出时使用它，并对每层读到的内容继续加工。下面这张经典图同时展示了这些职责。可以先把左边理解为处理已知输入的部分，右边理解为逐步生成目标序列的部分，再沿箭头看它们怎样连接。

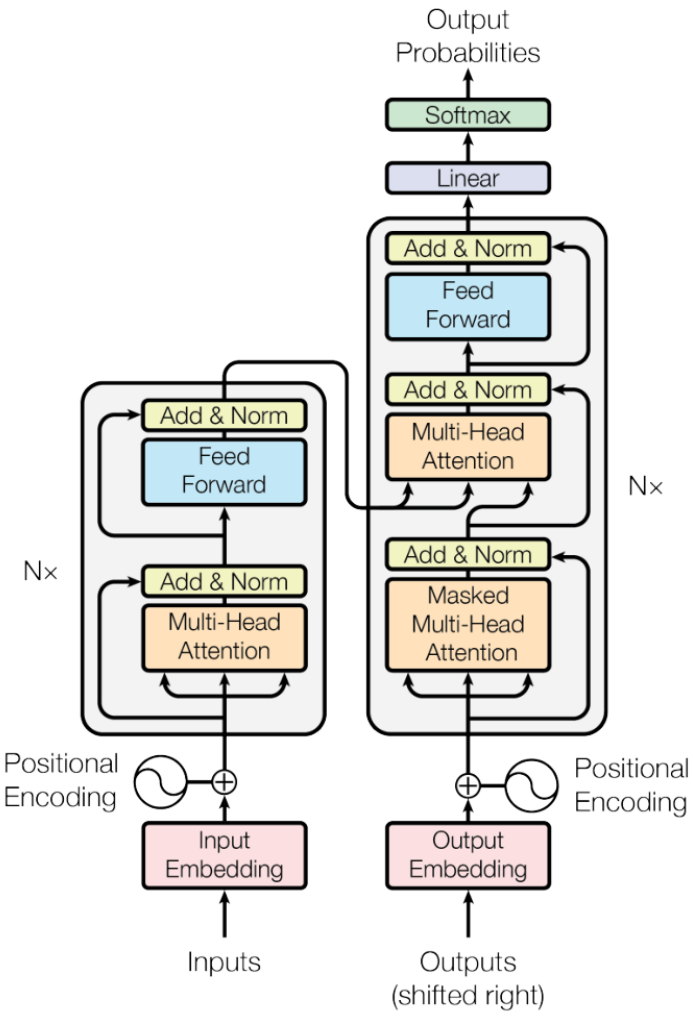


图1.5-1：Vaswani 等，《Attention Is All You Need》Figure 1。左侧编码器处理完整输入，右侧解码器读取输出前缀与编码结果。右侧中间一层注意力同时接收两路信息。原论文。

从左下方的 Inputs 开始：离散输入先经 Input Embedding 变成向量，再加位置编码。编码器的自注意力让输入位置彼此交换信息，前馈网络加工每个位置的内容，重复 N 层后输出一组条件表示。$N\times$ 表示堆叠 N 个结构相同的层，通常每层有自己的参数，不是同一组权重循环执行 N 次。

再看右边。若目标序列是“打开夹爪、接近杯子、闭合夹爪”，训练输入应是“开始符、打开夹爪、接近杯子”，对应标签才是原目标序列。这就是 Outputs shifted right（输出右移）。底部的 Masked Multi-Head Attention 先在输出前缀内部交换信息；它的掩码阻止当前位置读取后面的标签。中间的 Multi-Head Attention 再根据当前解码器表示发出 Q，从左侧编码器输出生成 K、V，读取与本次预测有关的输入信息。

所以，中间注意力的三条入口并非同一段内容重复输入三次。两条来自编码器的连线提供匹配键与待读取的值，一条来自解码器下方的连线提供查询。图中这个连接就是交叉注意力。原论文处理机器翻译；上面的动作词仅用于说明序列关系，连续动作 VLA 的输出不一定采用这套离散词表。

原图标签	具体职责
Input / Output Embedding	将输入或已知输出 token 转换为可计算向量

原图标签	具体职责
Positional Encoding	让表示包含位置与顺序线索
Multi-Head Attention	按前节公式建立多套信息读取关系
Masked Multi-Head Attention	同样做注意力，但限制可读取位置
Feed Forward	前馈网络；对每个位置分别做非线性特征变换
Add & Norm	先将子层输出加回原输入，再进行层归一化
Linear	将解码器最后的表示映射为词表中各候选的分数
Softmax / Output Probabilities	把候选分数转成下一个 token 的概率分布
弯曲的旁路箭头	残差路径；原表示绕过子层，与新结果相加

图中 Add & Norm 对应 $y = \mathrm{LayerNorm}(x + F(x))$，即先算子层 F、相加，再归一化。后面手写模块采用 $y = x + F(\mathrm{LayerNorm}(x))$，即先归一化再算修正。前者称为后归一化（Post-Layer Normalization，Post-LN），后者称为预归一化（Pre-Layer Normalization，Pre-LN）。它们都使用相同的注意力、残差等构件，但具体计算图不同。《动手学深度学习》Transformer 教学

VLA 可以采用编码器解码器，也可以把图像、语言与动作位置放进一个共同主干，再用掩码规定关系；连续动作专家又可能采用单独的去噪网络。掌握经典图，是为了在读新架构时辨认输入、查询来源和输出目标，随后判断哪些连接被保留或改变。

1.5.1 完整 Transformer 块如何连接起来

注意力是 **Transformer** 的一个子层，不是完整 Transformer。常见块还包含逐 token 的前馈网络（Feed-Forward Network，FFN）、**残差连接**和归一化。设输入 X 的形状为[批大小 B，位置数 T，特征宽度 D]，记Attn为注意力、LN为层归一化。一个预归一化块依次计算：

$$Y = X + \mathrm{Attn}(\mathrm{LN}(X)), \qquad Z = Y + \mathrm{FFN}(\mathrm{LN}(Y)). \tag{1.5-1}$$

注意力在不同位置间交换信息，前馈网络对每个位置分别进行非线性变换；两次加法将学到的修正叠加回已有表示。残差中的直接路径帮助梯度穿过深层网络，也使每层可以从较小修正开始学习。

LayerNorm（Layer Normalization，层归一化）在每个 token 的特征维上求均值与方差，再用可学习缩放和偏移调节表示。它对当前特征计算统计量，通常不依赖其他样本；**RMSNorm**（Root Mean Square Normalization，均方根归一化）只按均方根缩放，不减均值。二者调节内部数值尺度，帮助多层计算和优化。LayerNorm，RMSNorm。

1.5.2 从归一化的数值看它在稳定什么

层归一化中的“层”容易让人误以为需要把整层所有样本混在一起。这里针对一个 token 的特征向量 $x = (x_1, \ldots, x_D)$，计算它自己的均值和方差：

$$\mu = \frac{1}{D}\sum_i x_i, \qquad \sigma^2 = \frac{1}{D}\sum_i (x_i - \mu)^2, \qquad y_i = \gamma_i \frac{x_i - \mu}{\sqrt{\sigma^2 + \epsilon}} + \beta_i. \tag{1.5-2}$$

例如特征 $(1, 3, 5)$ 的均值为3、方差为 $8/3$，在暂不考虑 ϵ 且缩放为1、偏移为0时，得到约 $(-1.225, 0, 1.225)$。特征相对大小仍被保留，但共同的偏移与尺度被移除。γ_i, β_i 是训练得到的共享参数，用于再次缩放和偏移归一化特征；它们没有逐个保存各token原来的均值和方差，因此这种调节并非恢复原始数值。ϵ 是防止分母为零的小常数。

RMSNorm 使用 $x_i / \sqrt{D^{-1}\sum_j x_j^2 + \epsilon}$ 再缩放，不会像层归一化那样去掉共同偏移，但仍会缩放它的幅度，计算也较简洁。DINO 的居中则是从教师分数中减去跨样本、跨训练步的逐维平均，目标是抑制长期占优势的输出维度。两者虽然都有“减一个均值”，统计对象与设计目的不同。PyTorch LayerNorm 定义

前馈网络先扩展特征，再经激活和投影返回原宽度。**高斯误差线性单元**（Gaussian Error Linear Unit，GELU）用平滑激活加工输入；**Swish 门控线性单元**（Swish-Gated Linear Unit，SwiGLU）则有两条投影分支，一条生成候选特征，另一条经 Swish 激活后逐维调节它，再投影回原维度。这个门控由任务损失学习，使不同输入采用不同的特征组合。注意力决定从哪些位置取信息，前馈网络决定在当前位置怎样加工信息。门控前馈网络。

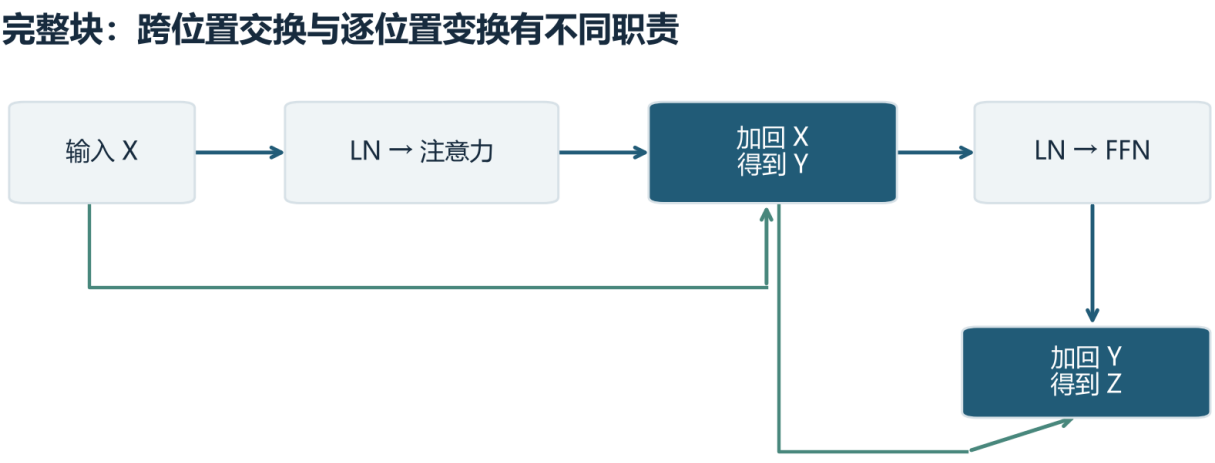


图1.5-2：残差保留主通路，注意力交换信息，前馈层改变各位置的特征。各子模块通常由同一任务损失共同更新。

训练时一次处理整个批次的序列，输出损失后反向传播。推理时使用固定权重计算新输入；模型在图中堆叠许多块，不表示它在每次执行时重新训练。结构图中的循环生成与训练优化循环必须区分。

1.5.3 残差怎样改变梯度路径

1.5.3.1 学一份修正，而不是每层重建全部表示

设前面的网络已经得到表示 x，新增子层继续加工它。若直接令 $y = F_\theta(x)$，已有信息都要经过这个新变换；即使当前不需要改变表示，子层也要学会输出 x。改成 $y = x + F_\theta(x)$ 后，分支只需学习要加上的修正；输出0就能保留当前表示。这使“保留已有计算”更容易表达，但优化器是否学到有用修正仍取决于数据、初始化与目标。D2L残差网络教学、Prince教材，第11.2节

这里的残差是表示上的加性修正。修正本身没有独立的人工标签，而是通过最终任务损失的梯度学习；注意力分支与前馈分支都可以产生正、负修正。相加要求形状兼容；输入输出宽度不同时，若旁路也加了投影，下面的恒等路径就要改成那个投影的导数。

1.5.3.2 直接梯度项从哪来

先把同形状的输入、输出看成同长列向量 x, y；序列张量也可展平后分析。**雅可比矩阵**记录“每个输出分量随每个输入分量怎样变化”，其第 i, j 项是 $\partial y_i / \partial x_j$。记分支雅可比为 J_F，输出处由后续损失传回的列梯度为 $g = \nabla_y L$，则：

$$y = x + F_\theta(x), \qquad \frac{\partial y}{\partial x} = I + J_F(x), \qquad \nabla_x L = g + J_F(x)^\top g. \tag{1.5-3}$$

I 是单位矩阵，来自输入 x 对自己的导数。反向传播时，加法把同一份输出梯度送到两条路径：直接路径给出 g，分支再贡献 $J_F^\top g$。若分支导数很小，输入梯度仍有直接项；若没有相加，梯度就只能经过分支导数。恒等映射原论文，第2节

多层时，梯度仍由各层雅可比连乘，只是每个因子从 J_l 变为 $I + J_l$。展开这些乘积，会出现经过少量分支的路径和一路选择恒等项的路径。它改变了深层优化的条件，并没有让各层彼此独立。

1.5.3.3 归一化放在哪里，会改变哪条路

记层归一化为 N。对预归一化子层 $y = x + F_\theta(N(x))$，直接路径上的 x 不经过本次归一化：

$$\nabla_x L = g + J_N(x)^\top J_F(N(x))^\top g. \tag{1.5-4}$$

对后归一化子层，先令 $h = x + F_\theta(x)$，再输出 $y = N(h)$，则：

$$\nabla_x L = \left[I + J_F(x)^\top\right] J_N(h)^\top g. \tag{1.5-5}$$

此时输出梯度先经过归一化的导数，再分到相加的两条路径。两种写法的分支位置相近，直接路径经历的运算却不同。

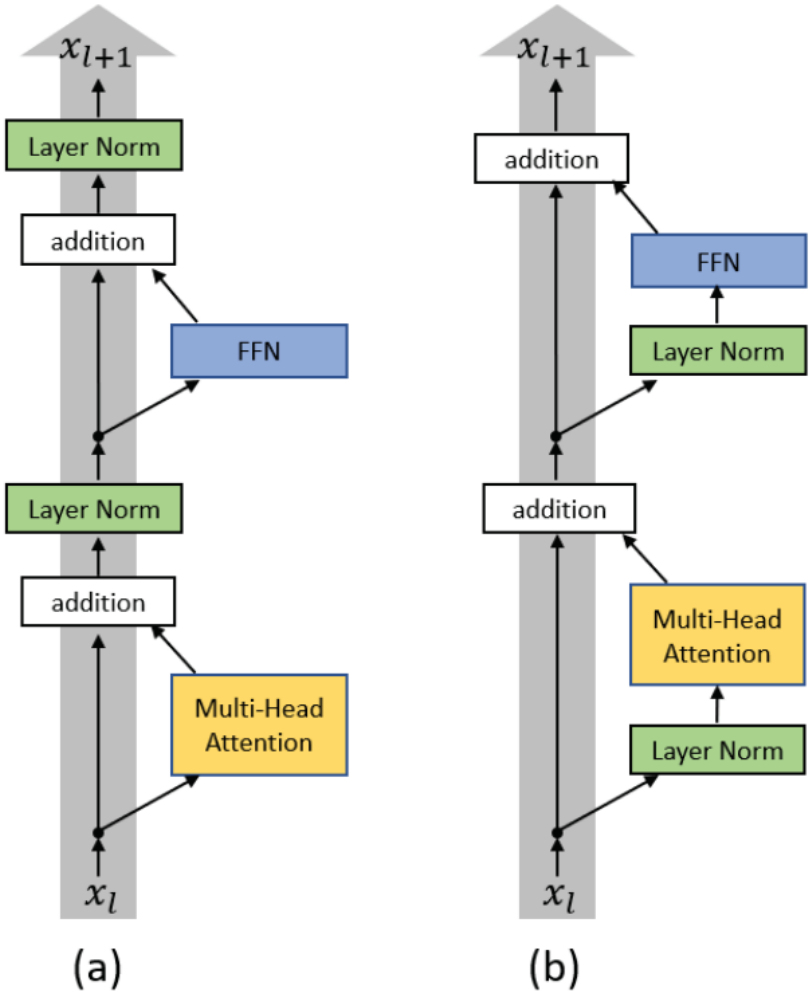


图1.5-3：Xiong等，Figure 1，左为后归一化，右为预归一化。灰色主路径从 x_l 到 x_{l+1}：左侧两次经过Layer Norm，右侧的归一化位于旁边分支。addition是相加，FFN是前馈网络，Multi-Head Attention是多头注意力。图中每层包含注意力与前馈两个残差子层。原论文

常见预归一化模型还在整个堆叠末尾、预测头之前加一次最终归一化。因此，式(1.5-4)说的是各残差子层之间保留直接路径；从任务损失返回时，仍会经过预测头和最终归一化。具体训练稳定性还受分支尺度、深度和学习率等因素影响。

可以用一个三维例子直接核对这些式子。取 $x=[1,3,5]^\top$、$F(z)=Wz$、$W=\mathrm{diag}(0.2,-0.1,0.3)$，归一化缩放为1、偏移为0，$\epsilon=10^{-5}$。先令 $r=x-\mu\mathbf{1}$、$s=\sqrt{D^{-1}r^\top r+\epsilon}$。由 $d\mu=\mathbf{1}^\top dx/D$、$ds=r^\top dx/(Ds)$，在本例的固定缩放、偏移下得到：

$$J_N(x)=\frac{1}{s}\left[I-\frac{\mathbf{1}\mathbf{1}^\top}{D}-\frac{rr^\top}{Ds^2}\right]. \tag{1.5-6}$$

其中 $\mathbf{1}$ 是全1向量。第一个减项去掉共同平移方向的变化，第二个来自分母也随输入变化；若只把标准差当常数求导，就会漏掉它。此处 $D=3$，$N(x)\approx[-1.224743,0,1.224743]^\top$。

选取子层输出处的下游梯度 $g=[1,-2,0.5]^\top$，单独观察它经过两张计算图后的结果：

路径贡献	第1分量	第2分量	第3分量
预归一化的直接项 g	1.000000	−2.000000	0.500000
预归一化的分支项	−0.005103	0.010206	−0.005103
预归一化的总输入梯度	0.994897	−1.989794	0.494897
后归一化的总输入梯度	0.691627	−0.723479	0.295762

这个对照说明，同一份下游信号会因归一化位置不同而沿不同路径传播。practice/gradient_case.py 按下面四步，同时用式(1.5-6)和自动微分逐项核对：

```
G1 指定输入x、分支参数和传入子层输出的梯度g
G2 分别计算预归一化与后归一化输出
G3 用g与输出的内积构造标量，反向求输入梯度
G4 将自动微分结果与显式雅可比链式公式比较
```

G3中的内积对输出求导恰好得到 g，因此能单独测试给定下游梯度如何经过这个子层。

直接项也可能与分支项相消。另设预归一化分支为 $F(z) = -s_0 z$，其中 $s_0 = \sqrt{8/3 + \epsilon}$ 是固定常数。在同一个输入处，取下游梯度 $g_c = [1, -2, 1]^\top$，它同时垂直于全1方向和 $r = [-2, 0, 2]^\top$，所以 $J_N^\top g_c = g_c / s_0$。总输入梯度便为 $g_c - s_0 J_N^\top g_c = 0$。这个局部例子说明，保留恒等项有利于传播，却不保证任意任务梯度都不为0；它也没有使整个雅可比变成零矩阵。

1.5.3.4 输入有梯度，不代表每个分支都在训练

上面求的是损失对输入的梯度。对只出现在这个分支里的参数 θ，保持子层输入与 N 固定，其局部参数梯度为：

$$\nabla_\theta L = \left[\frac{\partial F_\theta(N(x))}{\partial \theta}\right]^\top g. \tag{1.5-7}$$

直接加回的 x 不含分支参数，因此不会给参数梯度额外加一个1。在线性分支中，令 $n = N(x)$，参数梯度就是 $\nabla_W L = g n^\top$。若同一参数在多个位置共享，应累加各处使用它时产生的贡献。

为了看一次真实更新，固定上面的输入，把 W 作为完整的 3×3 参数矩阵，以前述对角矩阵初始化；训练时允许非对角元素变化。把目标表示设为0，使用 $L = \frac{1}{2}\|x + Wn\|^2$。此时下游梯度应由这个损失重新计算，等于当前输出 y。初始输出约 $[0.755051, 3, 5.367423]$，所以 $\partial L / \partial W_{11} = 0.755051(-1.224743) \approx -0.924744$。学习率0.1的梯度下降把 W_{11} 从0.2更新到0.292474；该权重乘的是负的第一个归一化特征，增加它会压低当前偏正的第一输出。

已完成参数更新次数	当前输出	损失
0	[0.755051, 3.000000, 5.367423]	19.189665
1	[0.528537, 2.100003, 3.757202]	9.402966
2	[0.369976, 1.470005, 2.630046]	4.607468

已完成参数更新次数	当前输出	损失
3	[0.258984, 1.029005, 1.841035]	2.257667

这三次更新是在同一个三维样本上拟合零目标，表示不对应机器人动作。程序按T1固定样本和目标、T2反向求参数梯度、T3更新权重循环执行；每次前向都用新权重重新计算输出。实际模型的注意力、前馈网络和可训练归一化参数，由相应任务损失沿同一计算图共同更新。

调试时还要区分“冻结分支参数”和“停止记录整个分支”。前者可以保留损失对上游输入的导数；后者若用 no_grad() 包住分支，输入仍可能通过残差得到 g，但分支参数已没有这条反向路径。只看到输入梯度不为0，无法证明分支正在学习。配套程序给出了相同前向、三种梯度路径的对照，与1.2节的计算图说明对应。

1.5.4 位置编码怎样参与序列计算

暂不加入位置编码，且所有位置都可互相读取时，**自注意力**满足**重排等变性**（permutation equivariance）：换了输入排列，输出只跟着换位置。这里的等变指输出与输入一起重排。固定一个“只看相邻位置”的窗口，即使窗口左右对称，也已按编号规定了读取关系；比较重排前后时，应一起重排这个掩码。**位置编码**把顺序或空间关系加入计算：绝对位置可加在输入上，相对位置可作用于查询与键的匹配。图像二维位置、相机来源和历史时间描述不同关系，宜分别编码。

文本位置、视频物理时间、相机视角编号和**动作块**位置并非同一种位置。把两个相机图像拼接后使用序列下标，并不会自动告诉模型相机的外参。

旋转位置编码（Rotary Position Embedding，RoPE）把位置 m 的查询、位置 n 的键的成对分量分别旋转 $m\omega$、$n\omega$，让点积包含相对角 $(n-m)\omega$。旋转的是网络内部表示分量，并没有对机器人坐标系做物理旋转。RoPE原论文

具体看一对特征。定义旋转矩阵 $R(\phi)=\begin{bmatrix}\cos\phi & -\sin\phi\\ \sin\phi & \cos\phi\end{bmatrix}$，位置 m 的查询为 $R(m\omega)q$，位置 n 的键为 $R(n\omega)k$。利用旋转的转置是逆旋转：

$$[R(m\omega)q]^\top[R(n\omega)k]=q^\top R(m\omega)^\top R(n\omega)k=q^\top R((n-m)\omega)k. \tag{1.5-8}$$

内容向量决定匹配什么，相对位置 $n-m$ 决定以什么角度比较这两个通道。高维时把通道两两分组，各组采用不同频率，再将点积相加。取 $q=[1,0]^\top$、$k=[1,1]^\top$、$\omega=\pi/2$、$m=0,n=1$，旋转后的键为 $[-1,1]^\top$，分数为−1；两者编号同时加2，直接旋转点积仍为−1。

这个不变性要求内容 q,k 固定。完整模型中的内容、掩码或序列边界若随编号改变，最终输出可以改变；它也不自动保证训练长度外准确。

编码器强调把已知输入变成条件表示；解码器强调生成目标序列。经典 encoder-decoder 用**交叉注意力**让输出读取输入；decoder-only 可以把输入条件与输出位置放在同一序列，用掩码规定关系。这里的“解码”不必是文字，连续动作去噪网络也会使用解码模块。

1.5.5 把注意力组成完整块

完整可运行入口是 practice/legacy/transformer_block.py。它把前节的 TutorialMHA 接成一个预归一化块，以下保留该类的实现。两个归一化层分别服务注意力与前馈支路；残差相加始终发生在同一位置、同一特征宽度上。输入和输出形状相同，便可以继续堆叠下一层。

算法步骤：

1. 对每个 token 归一化，再用注意力读取其他允许位置。
2. 将读取后的修正加回原表示。
3. 对当前表示归一化，再逐位置进行前馈变换。
4. 将前馈修正加回，得到本层输出。

```
import torch
from torch import nn
from attention_mha import TutorialMHA

class TutorialBlock(nn.Module):
    def __init__(self, width, heads):
        super().__init__()
        self.attn = TutorialMHA(width, heads)
        self.norm1 = nn.LayerNorm(width)
        self.norm2 = nn.LayerNorm(width)
        self.ffn = nn.Sequential(
            nn.Linear(width, 4 * width),
            nn.GELU(),
            nn.Linear(4 * width, width),
        )

    def forward(self, x, allow=None):
        # 步骤1：归一化后，按掩码交换位置间信息
        delta, _ = self.attn(self.norm1(x), allow=allow)
        # 步骤2：直接路径上的 x 与注意力修正相加
        x = x + delta
        # 步骤3：同一个 FFN 分别加工每个 token
        delta = self.ffn(self.norm2(x))
```

```
        # 步骤4：再加入残差，输出仍是 [B,T,D]
        return x + delta

block = TutorialBlock(width=12, heads=3)
x = torch.randn(2, 5, 12)
allow = torch.ones(5, 5, dtype=torch.bool).tril()
print(block(x, allow).shape)  # torch.Size([2, 5, 12])
```

这个例子由5个序列位置构造因果掩码，输出位置不能读取其后输入。把同一组5个 token 当作已知图像条件时，则可使用全部允许的掩码。模块名称相同，可用信息关系由任务规定；第1.4节的因果测试应对这个完整块也成立。

1.5.6 缓存哪些量，怎样证明复用仍然正确

1.5.6.1 因果关系使旧表示保持有效

键值缓存（Key-Value Cache，KV cache）保存各层已经计算的键和值。固定权重、输入前缀和位置后，因果注意力中的旧位置不能读取后来的token，因此追加新token不会改变那些旧表示。新查询仍要读取旧K、V，但无需重算旧位置的整段前向。Hugging Face缓存教学

对一层注意力，新增 m 个位置、历史长度为 p 时，当前查询为 Q_{new}，可读键值为 $[K_{\text{past}}, K_{\text{new}}]$、$[V_{\text{past}}, V_{\text{new}}]$，每头的分数矩阵形状为 $m \times (p+m)$。当 $m=1$，仍需与全部历史键匹配；缓存省去重复计算，没有省去对历史的读取。每一层接收的表示不同，必须分别保存自己的K、V，即使它们形状相同。

先用入门版本理解层内流程，再用带缓存接口的版本验证复用，分别对应以下入口：

用途与文件	类和返回值	读取关系
完整块入门：practice/legacy/transformer_block.py，调用同目录 attention_mha.py	TutorialMHA 返回输出、权重	接受 allow 允许掩码，没有缓存参数
缓存实验：practice/cache_execution.py，调用 legacy/attention_torch.py	HandMHA 返回输出、权重、新缓存	接受 causal、cache 和 use_cache；本实现缓存仅用于自注意力

同一文件 legacy/attention_torch.py 还提供 PreNormBlock，把 HandMHA 接成注意力、前馈两个残差子层。它与前面的 TutorialBlock 结构相同，使用的注意力接口不同；其 forward 只返回整块输出，没有接收缓存的参数。

缓存对照始终从同一个 HandMHA 实例出发，固定权重。其 key_valid 如果使用，需覆盖历史与新增键。这里的 cache=(K,V) 保存两个张量，每个张量按[批、头、位置、头内特征]排列；新查询只计算新位置的投影，再与历史拼接。

从计算成本看，整段密集注意力的分数矩阵随token数近似按平方增长。多查询注意力或**分组查询注意力**（Grouped-Query Attention，GQA）通过共享部分 K、V 减少缓存等成本，通常仍保留多个 Q 头；FlashAttention 改变计算与访存方式，实现相同注意力形式。第11章会展开这些缓存和访存实现。GQA 原论文、FlashAttention

1.5.6.2 新查询的位置要包含历史偏移

有4个历史位置后再加入3个查询，它们的绝对位置是4、5、6，可读键的位置是0至6。因果规则为：

$$\text{allow}_{ij} = [\, j \le p + i \,], \qquad p = 4. \tag{1.5-9}$$

方括号表示条件成立时允许。正确矩阵为：

$$\begin{bmatrix} 1 & 1 & 1 & 1 & 1 & 0 & 0 \\ 1 & 1 & 1 & 1 & 1 & 1 & 0 \\ 1 & 1 & 1 & 1 & 1 & 1 & 1 \end{bmatrix}.$$

普通 3×7 下三角只按局部行号 $j \le i$ 判断，第一行只读到第0个键，错把查询位置4当成0。应先求实际位置，再比较：

```
# M1：查询位置从历史长度开始
past = cache[0].shape[-2]
tq = x.shape[1]  # x只包含本次新增位置
qi = past + torch.arange(tq, device=x.device)
# M2：键包括全部历史和新增位置
tk = past + tq
ki = torch.arange(tk, device=x.device)
# M3：只允许不晚于当前查询的键
allow = ki[None, :] <= qi[:, None]
```

允许掩码：1 表示可读取。新查询不是从位置零重新开始。

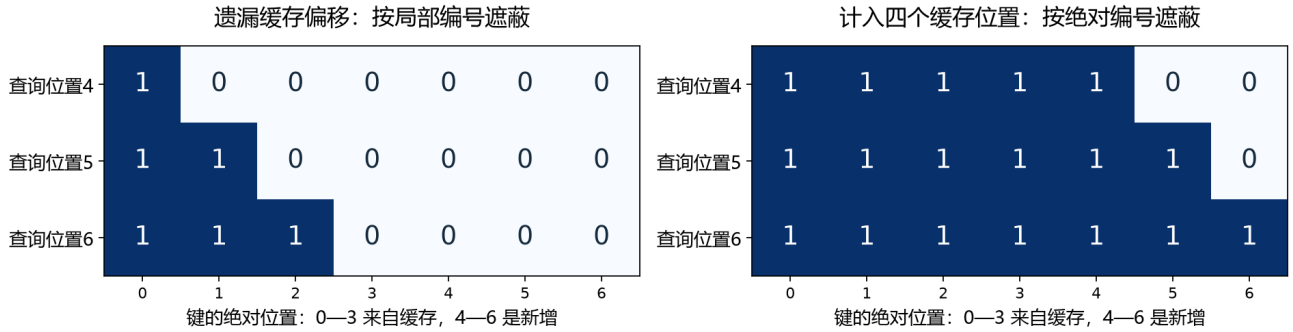


图1.5-4：左图遗漏历史偏移，右图的三行分别可读五、六、七个键。横轴是键位置，纵轴是新增查询的实际位置；浅格禁止，深格允许。

这里的三个新增输入已经给定，可以同时算三行。若真正自回归生成，后一个token依赖前一个刚生成的内容，仍需依次决定它们。下面的实验验证同一段已知输入怎样分块前向，没有在缺少前驱内容时一次生成三个token。

1.5.6.3 从一层注意力接到完整块

给缓存加上残差和前馈时，每层按同样的四步处理新位置：

```
K1 对本层新增表示做norm1
K2 生成本层新K、V，拼接本层历史；新查询按因果关系读取
K3 完成注意力残差，再做norm2、逐位置FFN和第二次残差
K4 保存本层缓存，把新增输出送给下一层
```

下面为 legacy/attention_torch.py 中的 PreNormBlock 增加一个分块执行函数。函数直接使用块内的归一化、注意力与前馈模块，复用同一组权重；原类的 forward 仍负责整段计算。

```
def cached_block(block, x, cache=None):
    attention_input = block.norm1(x)       # K1：只处理新增位置
    delta, _, new_cache = block.attn(      # K2：本层自己的历史K、V
        attention_input, cache=cache, use_cache=True, causal=True
    )
    x = x + delta                          # K3：两个残差与逐位置FFN
    x = x + block.ffn(block.norm2(x))
    return x, new_cache                    # K4：本层新增输出与缓存
```

多层时保存 caches[layer_index]；当前这批新增token在第0层的输出，再送入第1层。第1层的旧K、V来自第0层已经算好的旧输出，不能借用第0层缓存来代替。

实际程序使用固定随机权重的两层块，输入为[2,7,12]，三个注意力头，每头4维。比较时使用推理模式，关闭训练用的随机失活，各组输入、权重、位置与因果关系一致。逐层缓存在4个位置后为[2,3,4,4]，追加后为[2,3,7,4]。整段与分块结果的最大绝对差为：

分块方式	最大差
4 + 3	0
七次各1个	8.89×10^{-16} 以下
2 + 1 + 4	4.72×10^{-16} 以下

这是同权重、同输入、同因果关系下的数值等价检查。缓存中的块没有加入位置编码，RoPE仅按1.5.4的二维例另行核对；若接入RoPE，新增Q、K也要使用包含历史偏移的正确位置，已经旋转过的历史K应保留原有位置含义。

1.5.6.4 哪些变化会让旧缓存失效

是否能够复用，取决于生成这些K、V的计算是否仍相同。程序保留了三组具体对照：

对照	实际结果与含义
把正确偏移掩码换成普通局部下三角	同一注意力模块的新增输出最大差约0.687953；形状正确仍可能读取错误位置
修改第一个旧token的一个特征，仍使用旧缓存	两层块的新增输出最大差约0.125958；重新计算前缀并建立缓存后，与整段结果恢复一致
在两层双向注意力中沿用同样的4 + 3缓存流程	新增输出与完整七位置计算最大差约0.036258；新增内容会改变旧的底层表示，上一轮上层缓存已经过时

这三项偏差来自不同原因：读取集合错了、输入依赖改了、原来的因果不变性不成立。检查输出形状或缓存长度，无法替代同权重输出对照。

VLA使用新图像、修改条件或进入下一轮动作生成时，应检查哪些表示真正发生了变化。固定条件编码、且其计算不依赖不断变化的动作候选时，可以复用相应条件K、V；若条件表示也读取了变化中的候选，或受生成进度调制，就需重新判断。追加一帧历史与把旧帧替换成新观测，依赖关系也不同。第11章将进一步讨论缓存范围、耗时和观测更新。

1.5.7 练习题

题1.5-1：沿经典结构追踪一次预测。 在图1.5-1中，编码器输入已经给定，训练目标序列简化为“打开夹爪、接近杯子、闭合夹爪”。写出解码器训练输入与标签，说明编码器自注意力、解码器自注意力与交叉注意力的Q、K、V分别来自哪里，哪处需要因果掩码。再说明前馈、残差和归一化怎样接在注意力之后；预归一化与原图的顺序有何不同？

题1.5-2：缓存后的允许关系。 已缓存4个位置，再输入3个已知的新token。写出 3×7 允许掩码及其位置判断，说明普通局部下三角错在哪里。若这3个位置还都是未知的自回归输出，仅有这张掩码是否就能一次生成它们？解释计算并行与内容依赖的区别。

题1.5-3：RoPE怎样把相对位置放进分数？ 推导 $[R(m\omega)q]^\top[R(n\omega)k]$，解释共同平移编号时何者保持不变。用 $q=[1,0]^\top$、$k=[1,1]^\top$、$\omega=\pi/2$、$m=0,n=1$ 算一次分数，再把两者编号都加2核对。为什么这不能证明整个VLA对位置改变都不敏感？如果不加位置编码且所有位置互相可读，自注意力怎样响应输入重排？使用局部掩码时还需要什么条件？

题1.5-4：直接梯度项能保证什么？ 对预归一化 $y=x+F_\theta(N(x))$ 与后归一化 $y=N(x+F_\theta(x))$，以列梯度推导 $\nabla_x L$。使用1.5.3.3的三维输入、对角矩阵和下游梯度，复算两种输入梯度，明确直接项位于哪一步。再验证该节给出的局部相消例，说明“存在I项”为什么不足以保证总输入梯度非零，也不等于全部分支参数都收到一个固定梯度。

题1.5-5：前向相同，梯度会相同吗？ 对 $y=x+WN(x)$，仍用题4的输入、初始矩阵及给定下游梯度，比较三种做法：正常训练 W；只令 W 不求参数梯度；将整个 $WN(x)$ 放在 no_grad() 中。每种做法对输入、W 的梯度是什么？仅检查输入有梯度能发现第三种错误吗？随后改用损失 $L=\frac{1}{2}\|y\|^2$、学习率0.1，把 W 作为完整矩阵更新；此时应从该损失重新求下游梯度，计算第一次 W_{11} 更新及输出变化。

题1.5-6：缓存一致性怎样验收？ 为两层预归一化块设计整段与4 + 3分块的核对，说明每层缓存由什么产生、比较前需要固定哪些条件。再分别分析：所有层共用一份K、V；改了旧观测特征却沿用缓存；双向注意力直接照搬该流程。为什么“张量形状相同”不足以支持复用？最后说明动作候选随生成进度改变时，哪些条件表示可以缓存，判断依据是什么。

1.5.8 参考答案

题1.5-1

把三个目标依次记为 a_1, a_2, a_3，开始符记为BOS（Beginning of Sequence，序列开始符）。没有额外加入结束符时，对齐关系是：

解码器位置	训练输入	本位置预测标签
0	BOS	a_1：打开夹爪
1	a_1：打开夹爪	a_2：接近杯子

解码器位置	训练输入	本位置预测标签
2	a_2：接近杯子	a_3：闭合夹爪

对角位置可以读取，因为输入已经右移：预测 a_2 时，该位置读到的是 a_1。训练时三行输入都由示范给定，可以并行计算；生成时必须先决定 a_1，才有预测 a_2 所需的实际前缀。

以下“来自某表示”是指该表示再经过各自的Q、K、V可学习投影，三者通常不共用投影权重：

注意力子层	Q来源	K、V来源	允许读取范围
编码器自注意力	当前编码器层输入	同一组输入	经典离线编码器可读完整已知输入
解码器自注意力	当前解码器层输入	同一组解码器输入	只能读当前位置及之前的右移输入
解码器交叉注意力	本层自注意力处理后的解码器表示	编码器最终输出 H	可读取完整条件 H

这里第二行需要因果三角掩码。其他两行若有填充位置，也要屏蔽填充，但填充掩码与因果关系解决的是不同问题。

原图的编码器层按 $U=\mathrm{LN}_1(X+\mathrm{SelfAttn}(X))$、$Z=\mathrm{LN}_2(U+\mathrm{FFN}(U))$ 计算。解码器则在这两个子层之间多插入一次交叉注意力：先得到自注意力残差与归一化后的 U，用 U 查询 H，再将读到的修正加回 U 并归一化，最后执行FFN及其残差、归一化。每次残差都加回该子层自己的输入，FFN对各位置分别使用同一套参数。

预归一化把每个残差子层改成 $x+F(\mathrm{LN}(x))$：先归一化分支输入，再算修正，相加之后直接输出。于是本节两子层块为 $Y=X+\mathrm{Attn}(\mathrm{LN}_1(X))$、$Z=Y+\mathrm{FFN}(\mathrm{LN}_2(Y))$，直接旁路没有经过这两次LN。

题1.5-2

历史位置是0至3，新查询的实际位置是4、5、6。对局部查询行 $i=0,1,2$ 和键列 $j=0,\dots,6$，允许条件为 $j\le 4+i$：

$$M=\begin{bmatrix}1&1&1&1&1&0&0\\1&1&1&1&1&1&0\\1&1&1&1&1&1&1\end{bmatrix}.$$

普通局部下三角使用 $j\le i$，将实际位置4当成0，第一行只剩一个键可读。程序只替换这个掩码，新增输出就出现约0.687953的最大差。

掩码只规定怎样读取已有内容。如果 a_4,a_5,a_6 是未知的自回归输出，给定历史 h 后，通常要依次计算：

$$p(a_4,a_5,a_6\mid h)=p(a_4\mid h)\,p(a_5\mid h,a_4)\,p(a_6\mid h,a_4,a_5).$$

第一项决定后，第二项才获得它需要的实际内容；第二项又参与第三项。三行掩码可以同时写出来，三个输入却尚未同时具备。训练时右移的示范输入已经给定，所以可并行；本节4 + 3实验也属于输入已知的前向核对。

题1.5-3

旋转满足 $R(\alpha)^\top = R(-\alpha)$ 和 $R(\alpha)R(\beta) = R(\alpha+\beta)$，因此：

$$[R(m\omega)q]^\top[R(n\omega)k] = q^\top R(-m\omega)R(n\omega)k = q^\top R((n-m)\omega)k.$$

固定内容 q, k 与频率时，两者编号同时加 c，相对位置 $n-m$ 不变。这一对通道的点积分数保持不变，两个旋转后的向量各自仍会改变。

题中 $R(0)q = [1,0]^\top$，$R(\pi/2)k = [-1,1]^\top$，点积为−1。编号都加2后，$R(\pi)q = [-1,0]^\top$，$R(3\pi/2)k = [1,-1]^\top$，点积仍为−1。它只比较固定内容的共同编号偏移；实际VLA的位置变化还可能改变图像内容、物理时间、可读集合和边界，不能由这一条点积等式推出整个任务输出不变。

对没有位置编码、所有位置互相可读的自注意力，令 P 为重排矩阵。按位置排列的行向量满足 $Q' = PQ, K' = PK, V' = PV$，分数矩阵相应变为 $S' = PSP^\top$。逐行softmax在行、列同步重排后也跟着重排，所以：

$$\mathrm{softmax}_{\mathrm{row}}(PSP^\top) = P\,\mathrm{softmax}_{\mathrm{row}}(S)\,P^\top, \qquad \mathrm{Attn}(PX) = P\,\mathrm{Attn}(X).$$

输出跟随输入换位置，这就是等变。若有允许矩阵 M，还须同步改为 $PMP^\top$；一般应写成 $\mathrm{Attn}(PX; PMP^\top) = P\,\mathrm{Attn}(X; M)$。固定的局部窗口规定了哪些编号相邻，即使左右对称也提供了位置关系。只交换内容却保持窗口不变，会改变各token读到的对象。

题1.5-4

先确定反向传播的方向。若 $J_{ij} = \partial y_i/\partial x_j$、$g_i = \partial L/\partial y_i$，分量链式法则给出 $\partial L/\partial x_j = \sum_i J_{ij} g_i$，用列向量写就是 $\nabla_x L = J^\top g$。

预归一化中，dx 一路直接相加，一路先经过 N、再经过 F，所以 $dy = [I + J_F(N(x))J_N(x)]dx$。后归一化中，两路先相加，再经过 N，所以 $dy = J_N(h)[I + J_F(x)]dx$，其中 $h = x + F(x)$。转为反向次序：

$$\nabla_x L_{\mathrm{pre}} = g + J_N(x)^\top J_F(N(x))^\top g, \qquad \nabla_x L_{\mathrm{post}} = [I + J_F(x)^\top]J_N(h)^\top g.$$

本题 $J_F = W$。对 $x = [1,3,5]^\top$，有 $\mu_x = 3$、$r = [-2,0,2]^\top$、$s_x \approx 1.632996224$。由式(1.5-6)：

$$J_N(x) \approx \begin{bmatrix} 0.102063029 & -0.204123763 & 0.102060733 \\ -0.204123763 & 0.408247525 & -0.204123763 \\ 0.102060733 & -0.204123763 & 0.102063029 \end{bmatrix}.$$

先算 $W^\top g = [0.2, 0.2, 0.15]^\top$，再经 $J_N(x)^\top$ 得到分支项约 $[-0.005103037, 0.010206188, -0.005103151]^\top$，加上 g：

$$\nabla_x L_{\mathrm{pre}} \approx [0.994896963, -1.989793812, 0.494896849]^\top.$$

后归一化必须在另一个输入 $h=(I+W)x=[1.2,2.7,6.5]^\top$ 处重新求统计量。其均值约3.466666667、方差约4.975555556、分母约2.230597578。代入同一雅可比公式，得到：

$$g_h = J_N(h)^\top g \approx [0.576356019, -0.803865160, 0.227509141]^\top,$$

$$W^\top g_h \approx [0.115271204, 0.080386516, 0.068252742]^\top, \qquad \nabla_x L_{\text{post}} \approx [0.691627223, -0.723478644, 0.295761883]^\top.$$

因此，预归一化旁路直接收到 g；后归一化的加法先收到 g_h，旁路与分支再分别贡献 g_h、$W^\top g_h$。两张计算图使用同一个人为给定的下游信号，并未假设它们的前向输出或任务损失相同。

局部相消例中 $F(z)=-s_0 z$、$s_0=\sqrt{8/3+\epsilon}$。因为 $g_c^\top \mathbf{1}=0$、$g_c^\top r=0$，有 $J_N(x)^\top g_c = g_c/s_0$，所以 $\nabla_x L = g_c - s_0 J_N(x)^\top g_c = 0$。但完整雅可比是：

$$I - s_0 J_N(x) = \frac{\mathbf{1}\mathbf{1}^\top}{3} + \frac{rr^\top}{3s_0^2}.$$

它作用于全1向量仍得到全1向量，因此不是零矩阵。相消发生在指定输入和梯度方向上；求导时 s_0 是固定常数。残差保留的是直接贡献，它与分支贡献相加后仍可能抵消。

对分支参数则是另一条导数：$\nabla_\theta L = J_{F,\theta}^\top g$，没有恒等项“加1”。在线性分支中为 $gn^\top$；本例 $n_2=0$，于是 W 的第二列梯度都为0，即使整条反向计算正常。

题1.5-5

先用标量 $L_g = g^\top y$ 固定题4的下游信号。三种写法的前向输出都约为 $[0.755051485, 3, 5.367422773]^\top$，但记录的求导关系不同：

做法	输入 x 的梯度	W 的梯度记录
正常训练	$g + J_N(x)^\top W^\top g$，约[0.994897,-1.989794,0.494897]	$gn^\top$
只设 $W.\text{requires_grad}=\text{False}$	与正常训练相同	W.grad为None
分支 $WN(x)$ 在no_grad()内	只剩旁路 $g=[1,-2,0.5]^\top$	W.grad为None

正常情况下：

$$gn^\top \approx \begin{bmatrix} -1.224742575 & 0 & 1.224742575 \\ 2.449485150 & 0 & -2.449485150 \\ -0.612371288 & 0 & 0.612371288 \end{bmatrix}.$$

冻结 W 时，只要 x 仍需梯度，分支运算就会保留求 x 导数的路径；no_grad()让本次分支运算不进入反向计算，分支输出被当作常量。None表示没有取得该参数的梯度记录，和正常求导后得到零矩阵不同。每次对照应清除旧梯度或使用新张量，避免把上次遗留的.grad当成本次结果。

残差旁路使第三种写法的输入梯度仍然非零，所以只检查x.grad无法发现误用。还应检查待训练参数有没有梯度，并核对输入梯度是否包含预期的分支贡献。

换成 $L=\frac{1}{2}\|y\|^2$ 后，下游梯度为当前 y，不再是前一对照指定的 g。固定输入及归一化参数，完整矩阵的梯度和更新为：

$$\nabla_W L = yn^\top \approx \begin{bmatrix} -0.924743700 & 0 & 0.924743700 \\ -3.674227725 & 0 & 3.674227725 \\ -6.573711188 & 0 & 6.573711188 \end{bmatrix}, \qquad W' = W - 0.1yn^\top.$$

其中 $W'_{11} = 0.2 - 0.1(-0.924743700) \approx 0.292474370$。整步更新后的矩阵为：

$$W' \approx \begin{bmatrix} 0.292474370 & 0 & -0.092474370 \\ 0.367422773 & -0.1 & -0.367422773 \\ 0.657371119 & 0 & -0.357371119 \end{bmatrix}.$$

不必逐元素重乘也能核对输出，因为 n 固定：

$$y' = x + (W - 0.1yn^\top)n = (1 - 0.1\|n\|^2)y, \qquad \|n\|^2 \approx 2.999988750.$$

因此 $y' \approx [0.528536889, 2.100003375, 3.757201979]^\top$，损失从19.189665降到9.402966。这里同步更新了完整 W 的所有可训练元素；部分元素的梯度恰为0。若约束 W 永远对角，只更新三个对角元素，输出会变成约[0.641794,3,4.562312]，属于另一种参数化。

题1.5-6

设第 ℓ 层输入为 H_ℓ，本层先求 $C_\ell = \mathrm{LN}_{1,\ell}(H_\ell)$，再由本层投影得到 $K_\ell = C_\ell W_{K,\ell}$、$V_\ell = C_\ell W_{V,\ell}$。第0层读取输入token，第1层读取第0层完成两个残差子层后的输出；输入表示与投影参数都不同，缓存自然要分层保存。

验收时固定输入、权重、顺序、允许关系、位置编号与数值精度，使用推理模式并关闭训练随机失活。整段路线依次通过两层；分块路线对前四个位置依次算两层，分别保存缓存，再让后三个位置带着各自层的历史依次计算。可在1.5.6.3的cached_block外加：

```
def full_stack(blocks, x, causal=True):
    for block in blocks:
        x = block(x, causal=causal)  # 整段输入依次经过每个完整块
    return x

# 路线一：同样的七个已知输入，一次通过两层
full = full_stack(blocks, tokens, causal=True)
# 路线二：每层独立保存历史，新增表示逐层传递
caches, pieces = [None, None], []
for start, end in [(0, 4), (4, 7)]:
    x = tokens[:, start:end]
    for layer, block in enumerate(blocks):
        x, caches[layer] = cached_block(block, x, caches[layer])
    pieces.append(x)
cached = torch.cat(pieces, dim=1)
torch.testing.assert_close(cached, full, rtol=1e-12, atol=1e-12)
```

本节两层块的每层缓存从[2,3,4,4]增长到[2,3,7,4]，4 + 3结果与整段一致，其他两种分块的最大差也在 10^{-15} 量级内。这里测试的块不含位置编码；若接入RoPE，还须核对实际位置、频率与历史偏移，二维点积例不能代替这项完整测试。

成立依据是逐层的依赖不变性：第0层旧位置只能读取旧前缀，逐位置LN和FFN也不读取未来，因此旧输出不变；第1层读取的旧输入于是也不变，其旧K、V继续有效。下面几种改动会破坏不同环节：

改动	失效原因与可检验后果
两层共用一份旧K、V	第1层需要自己的输入表示与投影；独立复算中，借用第0层缓存使新增输出偏差约0.382299
修改旧观测的一个特征但保留缓存	被缓存函数的输入已经改变；配套实验偏差约0.125958，重新建立前缀缓存后恢复一致
两层双向注意力直接套用4 + 3	新增位置会改变旧的第0层输出，进而使上层旧缓存过时；实验新增输出偏差约0.036258

双向反例需要区分层次：第一层的旧K、V若直接来自固定输入，它们本身可以保持不变，新增查询仍可复用这些投影；但旧位置的注意力输出会因新增内容而改变，第二层的旧K、V便不再是原来的值。上述两层失败不能简化成“单层双向计算中的旧投影一律不能复用”。

在动作生成中，若条件编码及其归一化、投影保持固定，且不读取变化中的动作候选，也不受生成进度调制，那么相应条件K、V可以缓存。查询Q随候选改变，并不妨碍它读取同一组条件。若条件在共同主干中反过来读取候选，或它的内部变换随去噪进度改变，即使原始图像相同，内部K、V也可能已经改变。判断依据始终是产生缓存值的完整计算依赖。

1.6 条件融合、几何输入与多模态可控性

同一幅画面里同时有红杯和蓝杯，机器人收到不同指令，应当选择不同目标。视觉网络可以提取两个杯子的特征，但“看到了什么”还没有决定“这次要处理什么”。**条件融合**要解决的就是：让当前任务、观测和生成进度参与动作计算，并明确它们在网络的哪一步起作用。

前一节已经说明注意力怎样读取信息、残差怎样传递表示。本节进一步区分三种操作：把信息变成可兼容的输入，让动作表示读取条件，或者让条件改变中间特征的加工方式。最后再看一个容易被混淆的接口：图像位置怎样与机器人空间位置对应。

1.6.1 投影、读取和压缩各自解决什么

图像特征和文字嵌入宽度不同，常用**投影器**（projector）映射到兼容维数。投影器通常是线性层或多层感知机（MLP）。后者由线性变换和非线性激活组成，其参数由图文或下游任务损失训练。宽度变成一样只是接口兼容，不能保证图像和文字已经被有意义地连接。

最直接的融合是拼接token，让自注意力在一个序列中读取多种信息。另一种是交叉注意力，让动作查询读取视觉语言条件。**重采样器**（resampler）使用少量可学习查询从许多视觉token中读出固定数量的摘要，减少后续序列长度；这些查询是可训练向量，不是已经生成的机器人动作。压缩可能丢掉细小目标信息，必须检验保留的摘要是否足够。

图1.6-1：拼接、交叉读取、压缩查询与条件调制改变信息怎样进入动作模块。它们可以组合使用，例如先压缩视觉token，再由动作查询读取；生成进度则通过条件调制进入同一动作模块。

设动作中间表示为 $X \in \mathbb{R}^{B\times T\times D}$，其中 B 是批大小，T 是动作位置数，D 是特征宽度；视觉语言条件token记为 $C \in \mathbb{R}^{B\times N\times D}$。拼接后共有 $T+N$ 个位置，哪些位置能互相读取仍由掩码规定。交叉注意力则保留两组位置，由 X 产生查询、C 产生键和值，形成 $T\times N$ 的读取关系。重采样器把许多条件位置压成较少的摘要，改变的是后续可读的信息量。

条件调制采用另一种接口：条件网络输出一组通道系数，用它们改变已有特征。相同系数可以作用于多个位置；它没有直接给每个位置分配注意力权重，但可以改变后续网络用于选择目标的特征。实际

模型可以同时保留条件token C 和一个全局条件向量 c，分别服务读取与调制，不必把全部图像细节都挤进一个向量。Distill条件变换教程

若把语言向量只加到动作头末端，几何视觉表示在此前可能没有按任务筛选；若让语言过早强烈影响全部视觉路径，又可能损伤精细几何。融合位置是一种归纳假设，也就是用结构预先规定信息在何处、以什么方式相遇；哪种选择合适，还要由任务和数据检验。

1.6.2 FiLM：条件怎样改变特征，又怎样学会这种改变

逐特征线性调制（Feature-wise Linear Modulation，FiLM）让条件网络读取指令等条件 c，输出逐特征的缩放 $\gamma(c)$ 和偏移 $\beta(c)$，再计算：

$$h' = \gamma(c) \odot h + \beta(c). \tag{1.6-1}$$

$\odot$ 表示对应元素相乘。这是逐通道的仿射变换，也就是每个通道先乘一个数、再加一个数。它使同一视觉特征在不同任务下受到不同处理，而不是每次都用一套固定变换。教学标量 $h = 2$，若一条条件得到 $\gamma = 2, \beta = 1$，结果为5；另一条得到 $\gamma = 1, \beta = -1$，结果为1。FiLM原论文，第2节

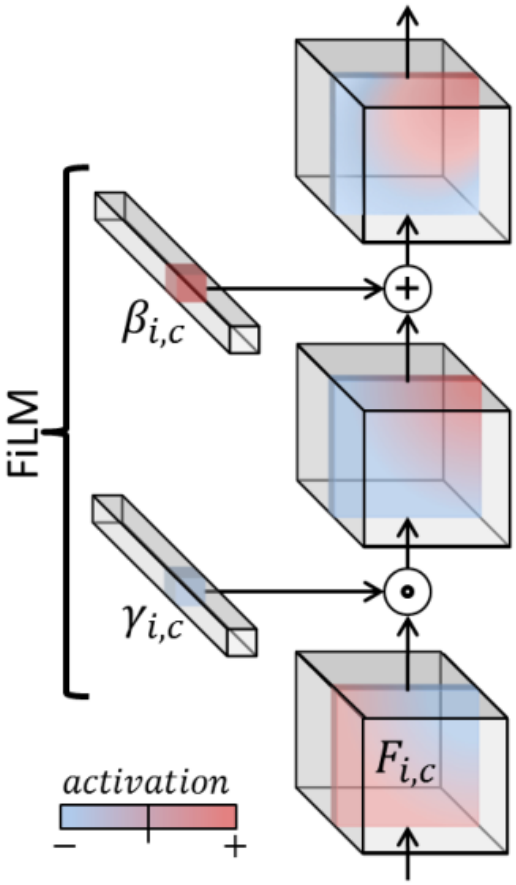


图1.6-2：Perez等，Figure 2。沿中间箭头由下向上看，输入特征先逐元素乘缩放，再加偏移。图中 $F_{i,c}$ 指第 i 个样本、第 c 个通道的特征图；此处下标 c 是原图的通道编号，正文公式中的条件则记为向量 $\boldsymbol{c}$。同一通道上的各空间位置共享一组系数，色标表示特征值的正负。

这里有两类不同的“参数”。条件生成器自身的权重由优化器长期更新；$\gamma(c), \beta(c)$ 则是它对本次条件计算出的输出，换一条指令就可能变化。训练数据没有逐层标出应该使用哪一组缩放和偏移。最终动作损失的梯度同时更新条件网络与视觉路径，使调制朝有助于任务预测的方向学习。

令动作损失传到该层的梯度为 $g = \partial L / \partial h'$，则：

$$\frac{\partial L}{\partial \gamma} = g \odot h, \qquad \frac{\partial L}{\partial \beta} = g, \qquad \frac{\partial L}{\partial h} = \gamma(c) \odot g. \tag{1.6-2}$$

例如视觉特征为 $h = [2, 0.5]$，当前任务传回 $g = [-0.1, 0.1]$，缩放参数的梯度为 $[-0.2, 0.05]$。梯度下降倾向于提高第一个通道的缩放、降低第二个通道的缩放。这些信号再沿条件网络返回，使它逐渐学会在不同指令下怎样调节视觉特征；传回视觉路径的梯度也按当前缩放系数改变。通道未必逐一对应红杯或蓝杯；其含义由表示与动作任务共同形成。

如果同一条件的系数用于多个位置，参数梯度要累加这些位置的贡献。对 $h'_\ell = \gamma(c) \odot h_\ell + \beta(c)$，有：

$$\nabla_\gamma L = \sum_{\ell=1}^{T} g_\ell \odot h_\ell, \qquad \nabla_\beta L = \sum_{\ell=1}^{T} g_\ell. \tag{1.6-3}$$

这也解释了实现中的形状：[批、通道]的条件系数要扩展一个位置轴，再作用于[批、位置、通道]的特征；反向传播会对共享的位置轴求和。某个通道的总梯度为0，也可能是多个位置的贡献相消，不能据此断定这条路径没有参与计算。

条件拼接在线性层中给出 $W_h h + W_c c$，其中 $W_c c$ 相当于条件相关偏移；FiLM还引入逐通道乘法，能够随条件改变特征的影响大小。这个等价关系针对所写的线性层，不意味着带非线性的深层拼接网络只能学偏移。交叉注意力则按查询选择读取哪些位置。三者都能传递任务条件，采用的交互形式不同，融合位置和数据覆盖决定它们是否适合具体操作。

1.6.3 AdaLN-Zero：从条件向量到两次残差相加

自适应层归一化（Adaptive Layer Normalization，AdaLN）把条件调制接在归一化之后。归一化先根据当前特征求统计量，条件网络再决定怎样缩放、偏移这些标准化特征。条件网络不是在替代均值、方差的计算。

以下采用**扩散Transformer**（Diffusion Transformer，DiT）作者实现中的形式。原始DiT用于条件图像生成，输入是加噪后的图像潜表示；生成进度与图像类别通过嵌入形成条件。动作扩散或流匹配网络也可以使用类似的调制，但输入、预测目标和条件组合须由各自模型定义。DiT作者讲解

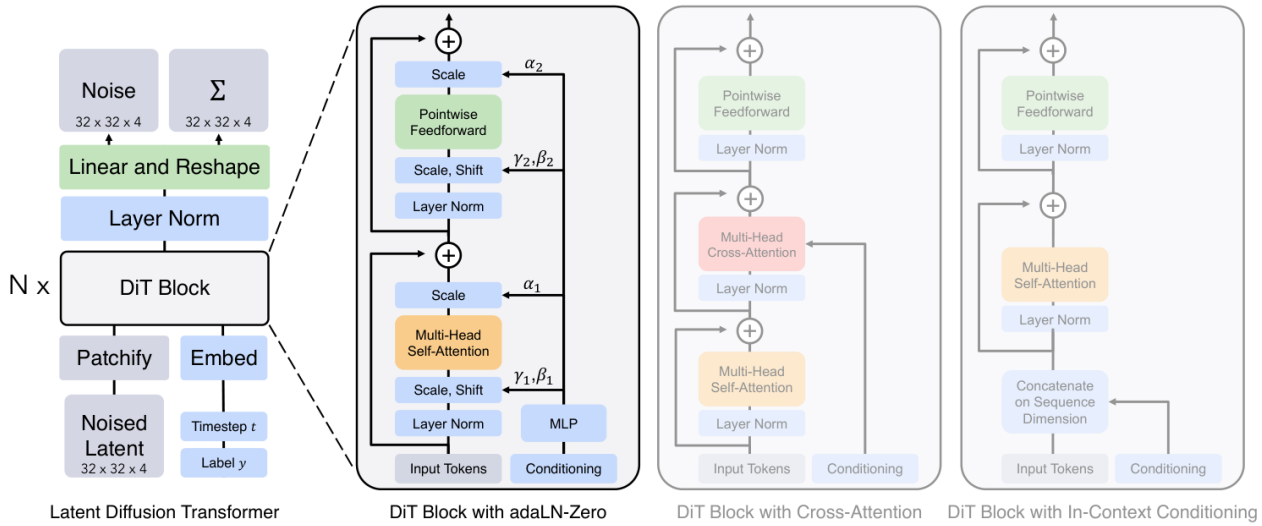


图1.6-3：Peebles与Xie，Figure 3。左边是完整图像生成网络，中间突出AdaLN-Zero块，右边对比交叉注意力和条件token拼接。浅色是原图的强调方式。原图Timestep t 是生成步，本书记为 τ，以区别机器人物理时刻；Label y 是类别条件。原论文，第3—5页

读左边时，Noised Latent是已经加噪的压缩图像；Patchify将它分成小块并投影为token；Embed把生成步和类别变成向量。上方Linear and Reshape将最后的表示映射并重新排列为预测量。Noise表示预测噪声，Σ 表示反向过程的协方差预测，不是LayerNorm的特征方差。图里的 $32 \times 32 \times 4$ 是图像潜表示的形状，不能直接当成机器人动作形状。

现在只沿中间的AdaLN-Zero块计算。条件向量 c 经调制网络产生六组 D 维量：

$$(b_1, s_1, \alpha_1, b_2, s_2, \alpha_2) = M_\phi(c). \tag{1.6-4}$$

b 是偏移，s 是相对于1的缩放增量，α 是控制残差修正的逐通道门控；下标1、2分别服务注意力和前馈子层。对一个批次，调制输出形状为[批、$6D$]，再拆成六个[批、D]向量。它们是每个样本本次计算的输出，不是六组固定网络权重。

用不带固定可学习缩放、偏移的层归一化 N_1, N_2，两条支路依次计算：

$$U_1 = (1 + s_1) \odot N_1(X) + b_1, \qquad Y = X + \alpha_1 \odot \mathrm{SelfAttn}_\theta(U_1), \tag{1.6-5}$$

$$U_2 = (1 + s_2) \odot N_2(Y) + b_2, \qquad Z = Y + \alpha_2 \odot \mathrm{FFN}_\theta(U_2). \tag{1.6-6}$$

各条件向量沿位置轴共享。原图的Scale, Shift对应 $(1 + s) \odot N(\cdot) + b$；单独的Scale对应分支输出前的 α，它后面才是残差加法。Pointwise Feedforward是逐位置前馈网络，MLP产生右侧六组调制量。图中 γ 表示条件缩放，本节用 s 明确表示其相对1的增量。

调制和门控作用在不同位置：s, b 改变送入注意力或FFN的特征，α 决定分支算出的修正怎样加回主路。这里的门控没有被sigmoid限制在0到1，可以为负，也可以大于1。

AdaLN-Zero的“零”在这份实现中有具体含义：把 M_ϕ 最后线性层的权重与偏置初始化为0。因此六组输出最初都为0，$U_1 = N_1(X)$，但 $\alpha_1 = 0$，所以 $Y = X$；同理 $Z = Y = X$。分支内部仍可计算非零候选修正，只是暂未加回主路。$(1 + s)$ 中的1使零缩放增量保留标准化特征，而不是把输入清成0。作者固定版本代码

看一条已经获得非零调制量的子层。沿用前节 $x=[1,3,5]^\top$、$\epsilon=10^{-5}$，有 $N(x)\approx[-1.224743,0,1.224743]^\top$。为便于复算，把注意力等复杂分支暂换成 $F(u)=\mathrm{diag}(0.2,-0.1,0.3)u$，给定 $s=[0.5,0,-0.5]^\top$、$b=[0.1,0.2,-0.1]^\top$、$\alpha=[0.2,-0.1,0]^\top$：

中间量	计算结果
调制后 $u=(1+s)\odot N(x)+b$	[−1.737114, 0.200000, 0.512371]
候选修正 $F(u)$	[−0.347423, −0.020000, 0.153711]
门控修正 $\alpha\odot F(u)$	[−0.069485, 0.002000, 0]
残差输出 $y=x+\alpha\odot F(u)$	[0.930515, 3.002000, 5.000000]

第二通道的负门控改变了修正的符号，第三通道虽然算出了候选值，却因门控为0而不改变主路。这些都是内部特征值，没有赋予米或弧度等动作单位。

配套文件 practice/conditioning_paths.py 的 ConditionedBlock 使用真正的多头自注意力与FFN。核心前向与上面四步对应，完整类与初始化在同一文件中：

```
A1 由条件产生两组偏移、缩放增量、门控
A2 归一化并调制注意力输入，算出候选修正
A3 乘注意力门控，加回原输入
A4 对新表示重复归一化、调制、前馈与门控残差
```

```
def forward(self, x, condition):
    # A1：六个向量均按位置共享
    b1, s1, a1, b2, s2, a2 = self.modulation(condition).chunk(6, dim=-1)
    # A2：先归一化并调制，再计算自注意力修正
    u = (1 + s1[:, None]) * self.norm1(x) + b1[:, None]
    delta, _ = self.attention(u, u, u, need_weights=False)
    # A3：本次分支修正按通道门控，再与主路相加
    x = x + a1[:, None] * delta
    # A4：第二套调制与门控用于前馈子层
    u = (1 + s2[:, None]) * self.norm2(x) + b2[:, None]
    return x + a2[:, None] * self.feedforward(u)
```

1.6.4 门控为0，训练为何还能开始

先只分析一个子层 $y=x+\alpha\odot F_\theta(u)$，并假定后续损失已经传来 $g=\nabla_y L$。按对应元素的乘法求导：

$$\nabla_\alpha L=g\odot F_\theta(u),\qquad \nabla_\theta L=J_{F,\theta}^\top(\alpha\odot g),\qquad \nabla_u L=J_{F,u}^\top(\alpha\odot g). \tag{1.6-7}$$

这里 $J_{F,\theta}$、$J_{F,u}$ 分别记录分支输出对参数、输入的导数。若一组门控在多个位置共享，第一项还要沿位置求和。最初整个门控向量 $\alpha=0$，这条分支传向参数和 u 的梯度因此为0，直接残差路径仍保留传

回 x 的 g。只要候选修正和下游信号的乘积在求和后不为0，门控梯度就可以非零。优化器先改变门控，下一次前向与反向才可能进一步更新分支。

若只把某一个输出通道的门控设为0，则只关闭这个输出的分支贡献。注意力或FFN可能混合通道，其他输出仍可给同一个输入通道或共享参数提供梯度。前面的对角线性算例便于逐项计算，这一特点不能直接推广到混合通道的真实分支。

这说明为何不能把分支和门控都简单置零。若 $F_\theta(u)=0$ 且 $\alpha=0$，式(1.6-7)的两条学习路径同时为0。以零权重线性分支、零门控为例，只靠这个损失的普通梯度下降，二者都没有启动信号；保留非零初始化的候选分支就避免了这个具体问题。

再看条件编码器。将调制网络的最后一步写成 $m=W_m r_\psi(c)+d_m$，其中 m 收集六组输出，$r_\psi(c)$ 是前面的条件表示。单个样本的梯度为：

$$\nabla_{W_m}L=g_m r_\psi(c)^\top,\qquad \nabla_{r_\psi(c)}L=W_m^\top g_m,\quad g_m=\nabla_m L. \tag{1.6-8}$$

$W_m=0$ 时，条件编码器最初收不到这条路径的非零梯度，但 W_m 中对应门控的行仍可能更新。偏移、缩放增量在分支内部，它们此时受零门控阻挡。改变最后线性层后，后续梯度才可能继续传回条件编码器。这里“暂时为0”来自计算值，不是把这些运算从计算图中删掉。

到这里还有一个关键前提：我们假定子层已经收到非零的 g。DiT在最后一个块之后，还先做一次归一化和条件调制，再接输出线性层。记最后块输出为 Z，则 $v=(1+s_f)\odot N_f(Z)+b_f$，其中 $(b_f,s_f)=M_f(c)$。作者实现也将 M_f 的最后线性层置零，所以起初 $v=N_f(Z)$；这一处调制之后没有块内的残差门控。

最终预测为 $\hat r=Pv+d$，输出线性层同样初始化为 $P=0,d=0$。先按一个位置的列向量写，其他位置和样本的参数梯度需要累加。从预测误差传回最后特征的梯度为：

$$\nabla_v L=P^\top\nabla_{\hat r}L=0,\qquad \nabla_P L=(\nabla_{\hat r}L)v^\top. \tag{1.6-9}$$

所以完整模型首次反向时，输出头可以先获得非零梯度，块门控却可能还没有信号。输出头更新后，再用新参数前向和反向，才轮到前面的通路。残差块初始为恒等与完整网络初始预测为零，是两个不同层次的性质。预测零的物理含义还取决于它预测的是噪声、速度场还是动作，不能直接解释成机器人保持静止。

输出头打开之后，最终调制可直接接收 $\nabla_{s_f}L=(\nabla_v L)\odot N_f(Z)$、$\nabla_{b_f}L=\nabla_v L$。它没有乘零残差门控，因此可以和块门控在同一次反向中开始学习；块内的缩放、偏移仍须等门控打开。至于更早的条件编码器，还要等这些调制线性层的零权重先改变，沿式(1.6-8)这条路径回传的梯度才可能非零。下面表格中各行启动先后不同，正是这些连接造成的。

配套实验用一个小型条件块，输入形状[2,2,4]，两个注意力头；条件编码器读取两个二维示例条件，输出头预测每位置两个数。它保留上述调制、门控与输出头关系，没有使用图像、语言编码器或真实动作标签。以相同初始基础权重，分别采用普通输出头、零输出头，各执行三次无动量、无权重衰减的梯度下降，学习率0.05。零输出头一组得到以下梯度范数：

参数组	已更新0次的反向	已更新1次的反向	已更新2次的反向
最终输出头	2.993795	1.863424	1.302028
块调制层的门控行	0	0.085579	0.113215
块调制层的缩放、偏移行	0	0	0.000641
注意力与FFN参数	0	0	0.001613
条件编码器	0	0	0.001375
最终输出前的条件调制	0	0.185338	0.219354

“门控行”指最后调制线性层中产生 α_1, α_2 的权重行及偏置。表中范数合并该组参数的梯度；非零说明该组已有更新信号，不表示每个元素都非零。与之对照，普通输出头在首次反向就使块门控梯度范数约为0.882048。

三次更新后的零输出头组损失从1.020000降至0.434950；中间两次更新后分别约为0.660376和0.512152。各步始终保留完整计算图，梯度为0的参数也有梯度张量，和前节整个分支使用 no_grad() 后没有梯度记录不同。程序按U1重新计算输出和误差、U2反向并记录各组梯度、U3更新参数循环执行。

这组记录验证的是指定初始化下的学习先后关系，不是所有AdaLN网络固定要经过三步。其他损失路径、非零初始化、共享条件方式或恰好相消的梯度都会改变过程。零初始化的设计价值在于规定初始函数及修正如何进入；它是否改善具体任务的训练，还需相同数据和训练预算下的对照实验。

1.6.5 几何输入：特征位置怎样对应到机器人空间

上一节区分了token编号与相机外参。这里再往前走一步：若模型需要读取“杯子相对机器人在哪里”，我们怎样构造可用输入？视觉特征可以带有二维图像位置，但单个像素通常不能唯一确定三维点。

相机内参描述相机坐标到像素的投影。理想针孔模型中，$u = f_x X^C/Z^C + c_x$、$v = f_y Y^C/Z^C + c_y$；f_x, f_y 是以像素为单位的焦距，(c_x, c_y) 是主点位置，通常接近图像中心。这些量组成矩阵 $K = \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix}$。除以 Z^C，正是远近不同的点可能落到同一像素的原因。

先设镜头畸变已校正，使像素符合这里的理想模型，且 K 对应当前图像尺寸；深度 z 指相机坐标系的Z轴坐标，而不是到相机光心的欧氏距离。反过来已知像素和深度时，解出相机坐标点：

$$p^C = zK^{-1}[u, v, 1]^\top. \tag{1.6-10}$$

$K^{-1}[u, v, 1]^\top$ 给出沿该像素观察的方向，其第三分量为1；z 决定在这条线上取哪个点。没有深度或其他几何约束时，二维位置只确定一条射线。这里的深度必须来自当前系统实际提供的测量或估计，不能从训练时标签暗中带入部署输入。OpenCV相机几何教程

相机外参描述相机与另一个坐标系之间的位置、朝向关系。若已知从相机坐标系 C 到机器人基座坐标系 B 的变换，记为 $R_{B\leftarrow C}, t_{B\leftarrow C}$，则：

$$p^B = R_{B\leftarrow C} p^C + t_{B\leftarrow C}, \qquad p^C = R_{B\leftarrow C}^{\top}(p^B - t_{B\leftarrow C}). \tag{1.6-11}$$

箭头明确变换方向：输入用相机坐标表示，输出用基座坐标表示。若标定文件给的是反方向，先求逆，不能只交换变量名称。

取 $K = \begin{bmatrix} 600 & 0 & 320 \\ 0 & 600 & 240 \\ 0 & 0 & 1 \end{bmatrix}$，像素为(380,240)，$z = 1$米，则 $p^C = [0.1, 0, 1]^{\top}$米。再取：

$$R_{B\leftarrow C} = \begin{bmatrix} 0 & -1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix}, \qquad t_{B\leftarrow C} = [1, 0, 0]^{\top}\ \mathrm{m},$$

得到 $p^B = [1, 0.1, 1]^{\top}$米。若把深度改成2米，相机点变为[0.2,0,2]，重新投影仍是同一个像素。这两个点对机器人而言显然不是同一个抓取位置。

点坐标与位移向量还要区别处理。两点相减时平移抵消，$\Delta p^B = R_{B\leftarrow C}\Delta p^C$。本例相机坐标中的位移[0.1,0,0]米变为基座坐标中的[0,0.1,0]米；若错误地再加上平移，会凭空多出1米。这是把坐标接口接到动作增量时需要检查的关系，后续动力学与控制章节再展开。

几何量进入网络可以有多种方式。一种明确的输入接口，是把对应视觉特征 v_i 与基座坐标点 p_i^B 分别投影后相加：

$$e_i = W_v v_i + \mathrm{MLP}_{\mathrm{geom}}(p_i^B) + e_{\mathrm{camera}}. \tag{1.6-12}$$

三项输出宽度都是 D，e_i 才能作为后续网络的一个token。相机来源嵌入 e_{camera} 区分传感器，几何分支则提供坐标，两者职责不同。若一个图像块含有多个深度，还须规定哪个点与它对应，或保留多个点；不能默认一块图像就是一个三维物体。

输入字段	在进入网络前要明确什么	一种可用接口
图像特征与像素位置	当前裁剪、缩放尺寸，特征与像素的对应关系	视觉投影加二维位置编码
三维点或观察射线	来源、有效性、坐标系；点带米等单位，射线未确定距离	几何MLP后与视觉特征融合
关节角、夹爪等本体观测	每个字段的单位、归一化和采样时刻	状态MLP形成一个或多个条件token
相机来源与历史时刻	传感器身份、与机器人时钟的对应	来源嵌入、时间编码

式(1.6-12)是可选择的建模接口，不是每种VLA的固定结构。RGB模型也可能从数据中学习几何线索。这里要掌握的是：输入里实际提供了什么信息，网络又通过哪条路径使用它；向量宽度相同不能补出缺失的深度、外参或时间对应关系。

practice/geometry_inputs.py 按G1由像素和相机Z深度求点、G2变到基座坐标、G3逆变换和重投影、G4区分点与位移四步检查上述关系。它使用给定的理想参数，不是一次相机标定或真实机器人试验。

1.6.6 接入条件以后，怎样判断模型确实在使用它

语言真正有用需要受控修改输入：图像与状态完全一样，只把“红杯”换成“蓝杯”，动作是否改向另一个目标？如果训练里某背景总对应某指令，模型可能靠背景做任务。仅把文字接入网络或画出注意力图，不能证明语言控制了行动。

考虑“抓红杯”和“抓蓝杯”。如果训练场景一直只有一只红杯，网络只看图像就可以降低动作损失。损失没有要求它使用语言。更好的数据设计是在相同布局下给出不同目标，并提供对应动作；评价时再交叉组合未见过的布局和指令。结构上允许读取语言，只提供了能力条件；数据和目标才决定这条路径是否被学会。

同样，对AdaLN做实验时，应分开考察条件注入位置、初始化和数据覆盖。换掉初始化却同时增加训练步数，无法单独判断零门控的作用。对于几何分支，还应检验坐标变换、深度质量和视角变化：即使训练损失下降，错误坐标也可能被网络在固定布局里暂时补偿，换相机后才暴露。练习1要求把语言控制的验证写成完整对照实验，练习5检验几何接口中哪些信息能够被唯一确定。

1.6.7 练习题

题1.6-1：接入语言之后，如何验证控制作用？ 把语言、视觉和状态都拼到一个序列，并不必然让动作受语言控制。设计一个能够判断模型是否使用了指令的对照实验。明确哪些量保持相同、怎样改变条件、记录什么结果，以及怎样区分目标语义、位置捷径与固定动作记忆。

题1.6-2：共享的FiLM系数怎样接收梯度？ 同一个条件作用于两个位置，特征为 $h_1=[2,0.5]$、$h_2=[1,-1]$，下游梯度为 $g_1=[-0.1,0.1]$、$g_2=[0.2,-0.3]$。推导并计算 $\nabla_\gamma L,\nabla_\beta L$，解释第一通道的缩放梯度为何为0。说明条件生成器的权重与本次输出的 γ,β 有何区别，梯度怎样继续传回。再解释线性层上的条件拼接为何等价于条件偏移，这能否推出深层拼接网络不能学习乘法交互？

题1.6-3：从调制输入算到残差输出。 使用1.6.3的三维输入、s,b,α 和线性分支，复算 $u,F(u),y$；说明六组条件量分别对应图1.6-3中的哪两个子层。若六组量全为0，为何整个块输出仍是输入？有人把 $(1+s)N(x)$ 写成 $sN(x)$，检查初始“输出等于输入”能发现这个错误吗？设计一个打开门控的对照，并给出其预期输出。

题1.6-4：零初始化怎样影响第一次和后续更新？ 对 $y=x+\alpha\odot F_\theta(u)$，推导损失对 α,θ,u 的梯度。解释门控为0时哪些路径暂时没有非零梯度，门控为什么仍可能学习，以及分支也为0时会发生什么。再对 $m=W_m r_\psi(c)+d_m$ 分析 $W_m=0$ 对条件编码器的影响。最后把零初始化输出头 $\hat{r}=Pv+d$ 接在网络后，说明为什么完整网络的第一次反向与单独残差块的分析不同，并按正文实测表解释前三次更新的先后关系。

题1.6-5：二维位置、三维点与动作位移能混用吗？ 使用1.6.5的内参、像素和相机到基座变换，计算1米、2米深度对应的相机点并验证它们投影到同一像素。求1米深度点的基座坐标，再用逆变换恢复。相机坐标位移[0.1,0,0]米应怎样变换，为什么不能加平移？如果部署时只有RGB和相机编号，这些输入是否足以按上述公式得到唯一三维点？说明还缺什么，以及几何量通过式(1.6-12)进入模型时应检查的对应关系。

1.6.8 参考答案

题1.6-1

构造“同一场景、不同指令应产生不同正确动作”的配对实验。例如画面同时有红杯和蓝杯，机器人初始姿态、物体位置、图像和历史保持一致，分别输入“拿红杯”和“拿蓝杯”。从相同初始状态独立执行两次，记录选中物体、动作轨迹和成功率。训练与测试中红蓝杯应平衡出现，避免模型只学到“总拿左边杯子”。

在仿真中可保存、恢复同一初始快照。每次换指令都先恢复，清除上个回合的历史；随机策略的成对试验使用相同初始噪声和随机种子。真机则需要重复复位并记录误差，不能把两次先后执行当成完全相同的输入。

主要看“是否按新指令选对目标”，而非两条动作是否不同。两种任务可能共享最初的抬手动作，随机采样也可能造成无关的轨迹差异。可令 $S_{i,c}$ 表示布局 i、指令 c 是否抓到指定杯子，除单条指令成功率外，报告：

$$\text{成对成功率} = \frac{1}{n}\sum_{i}\mathbf{1}[S_{i,\text{红}} = 1 \wedge S_{i,\text{蓝}} = 1].$$

$\mathbf{1}[\cdot]$ 在条件成立时取1。始终抓左杯的策略可能在部分回合成功，却不能在相同布局下完成两个互斥目标。还应分开记录误抓另一杯和完全没有抓到，区分目标选择与操作失败。

对照	保持或改变什么	要排查的替代解释
固定布局，交换红蓝目标指令	只改变当前指令	语言是否造成符合语义的目标变化
交换杯子的左右、远近位置	目标指令不变，物体位置和相应观测一起改变	是否只记住某个位置
新位置、间距和机器人初始姿态	目标身份不变，所需轨迹变化	是否把颜色映射到固定动作
同义改写	目标含义不变，表达改变	是否只把两个固定字符串当作索引

再训练无语言或独立随机打乱语言的对照模型，固定数据量、训练预算与执行设置。若所有红蓝词都被固定互换，任务信息仍完整存在，并没有去掉语言信息；应破坏指令与样本目标的对应关系。仅在测试时塞入从未训练过的全零文本向量，其性能下降还可能来自陌生输入，不能独自证明语义控制。

训练、测试按完整布局和物体实例拆分，避免同一轨迹的相邻帧泄漏。使用多个训练及执行种子，复用成对布局，保留每回合的指令、目标、实际抓取物体、种子和轨迹。统计不确定性时以布局为单位，不能把一条轨迹的许多帧当作许多独立场景。

换指令能正确切换目标、换位置后仍跟随物体、新位置下也能调整轨迹，且优于无语言基线，才支持这组任务中的语言控制。它不等同于任意指令泛化，也不能仅靠行为结果确定内部用了哪个特征通道。

题1.6-2

对通道 j，因为同一 γ_j, β_j 被两个位置共同使用，链式法则累加两处贡献：

$$\frac{\partial L}{\partial \gamma_j} = g_{1j} h_{1j} + g_{2j} h_{2j}, \qquad \frac{\partial L}{\partial \beta_j} = g_{1j} + g_{2j}.$$

贡献	第一通道	第二通道
$g_1 \odot h_1$	−0.2	0.05
$g_2 \odot h_2$	0.2	0.3
$\nabla_\gamma L$	0	0.35
$\nabla_\beta L$	0.1	−0.2

第一通道的缩放梯度是两处相消，它仍参与了前向，偏移梯度也不为0。传回特征的局部梯度为 $\nabla_{h_\ell} L = \gamma \odot g_\ell$，也未必为0。给定的 g_ℓ 已经包含最终损失的求和或平均系数，不应再无依据地除以位置数。

令 $(\gamma, \beta) = f_\phi(c)$，$\phi$ 是优化器更新的共享权重，而 γ, β 是本次条件的一次计算结果。将两个系数的梯度拼成 $q = [\nabla_\gamma L; \nabla_\beta L]$，有：

$$\nabla_\phi L = J_{f,\phi}^\top q, \qquad \nabla_c L = J_{f,c}^\top q.$$

若最后一步是 $m = Wc + d$，则 $\nabla_W L = qc^\top$、$\nabla_d L = q$、$\nabla_c L = W^\top q$。这些信号可继续传入更早的条件编码器；条件若还经过别的路径，其梯度还要相加。一次共享权重更新也可能同时改变多个条件下的系数，不是直接为每个样本单独修改一个 γ。

对单个仿射层，$W[h; c] + a = W_h h + W_c c + a$，所以条件贡献相当于偏移。FiLM显式增加逐通道乘法；但上述分解不能限定带非线性的深层网络。以标量为例，$hc = [(h+c)^2 - (h-c)^2]/4$，网络若能近似平方，就能通过这类组合近似乘法。应比较交互形式、结构偏置与学习效率，而不是断言深层拼接网络无法表达乘法。

题1.6-3

均值为3、方差为 $8/3$，所以 $N(x) = [-2, 0, 2]^\top / \sqrt{8/3 + 10^{-5}} \approx [-1.224742575, 0, 1.224742575]^\top$。先乘 $1 + s = [1.5, 1, 0.5]^\top$，再加偏移，最后计算候选及门控修正：

$$u \approx [-1.737113863, 0.2, 0.512371288]^\top, \qquad F(u) \approx [-0.347422773, -0.02, 0.153711386]^\top,$$

$$\alpha \odot F(u) \approx [-0.069484555, 0.002, 0]^\top, \qquad y \approx [0.930515445, 3.002, 5]^\top.$$

(b_1, s_1) 对应注意力前的Scale, Shift，α_1 对应注意力后的Scale；第二组以同样方式服务FFN。六组量全为0时，分支输入分别仍是归一化结果，但两条修正都乘0，故 $Y = X, Z = Y = X$。

错误的 $sN(x)$ 在 $s=0$ 时会清空内部输入，但门控仍为0时，正确、错误两种分支都无法改变最终输出，恒等测试发现不了它。

可以令偏移和缩放增量仍为0，只把这一条子层的门控打开为[0.1,0.1,0.1]。保留题设无偏置线性分支，第二子层门控保持0，则正确输出为：

$$x+0.1F(N(x))\approx[0.975505148,3,5.036742277]^\top.$$

错误实现的内部输入为0，$F(0)=0$，输出仍是[1,3,5]。也可以直接检查内部输入是否等于 $N(x)$。这里利用了所选线性分支没有偏置；真实带偏置分支的 $F(0)$ 未必为0，不能直接套用本例的错误输出数值。

题1.6-4

先假设子层已经收到 $g=\nabla_y L$。乘法先将传给分支输出的梯度变为 $\alpha\odot g$，再经分支雅可比回传；门控的导数则读取候选修正：

$$\nabla_\alpha L=g\odot F_\theta(u),\quad \nabla_\theta L=J_{F,\theta}^\top(\alpha\odot g),\quad \nabla_u L=J_{F,u}^\top(\alpha\odot g).$$

门控按位置共享时，第一项沿位置求和。整个 $\alpha=0$ 会使分支参数和输入梯度为0，主路仍可以传回 g；但只要求和后的 $g\odot F(u)$ 非零，门控就能先更新，下一次前向与反向才轮到分支。候选为0、各位置相消等情况，都可能使启动信号也为0。

单个输出门控为0的结论更局部。例如 $F(u)=Au$、$A=\begin{bmatrix}1&2\\3&4\end{bmatrix}$、$\alpha=[0,0.5]^\top$、$g=[2,-1]^\top$，则 $\nabla_u L=A^\top[0,-0.5]^\top=[-1.5,-2]^\top$。第一个输出的路径关闭了，第一个输入仍通过第二输出收到梯度；真实注意力和FFN也会混合信息。

如果候选分支是零权重、零偏置的线性层，且门控也全为0，两者的梯度同时为0；在没有其他损失路径的普通梯度下降下，它们无法相互启动。如果只是某一批输入碰巧产生零候选，只能判断这一批，不能推出以后永远没有信号。分支为零而门控非零时，分支参数仍可能学习。

对调制最后一层，令 $r=r_\psi(c)$、$g_m=\nabla_m L$：

$$\nabla_{W_m}L=g_m r^\top,\qquad \nabla_{d_m}L=g_m,\qquad \nabla_\psi L=J_{r,\psi}^\top W_m^\top g_m.$$

$W_m=0$ 阻断了这条路径的非零编码器梯度，但门控对应的权重行、偏置仍可能先学习。要使六组输出也都为0，还需实际初始化中的 $d_m=0$。权重改变后，后续反向才可能继续训练编码器。

完整网络还要经过输出前调制 $v=(1+s_f)\odot N_f(Z)+b_f$，再由 $\hat r=Pv+d$ 预测。令 $q=\nabla_{\hat r}L$，则 $\nabla_v L=P^\top q$、$\nabla_P L=qv^\top$、$\nabla_d L=q$。初始 $P=0$ 时，输出头可能有梯度，前面的块却没有收到独立子层分析假定的非零 g。正文零输出头组的先后关系是：

本次反向之前	哪些参数已经改变	当前能产生的更新信号
已更新0次	尚无	输出头先学习，前面各组为0

本次反向之前	哪些参数已经改变	当前能产生的更新信号
已更新1次	输出头已可回传	块门控与最终调制有信号；块内缩放、偏移和候选分支仍被零门控挡住，编码器仍受零调制权重影响
已更新2次	门控及调制末层已可能非零	块内缩放、偏移、候选分支及条件编码器也出现非零梯度

最终调制后面没有额外残差门控，输出头打开后它可以直接学习，所以启动早于块内缩放和偏移。条件编码器由多条调制路径共享，看到它的总梯度非零，还不能单独确定贡献来自哪一条路径。

表中的零是计算图中的数值结果，相应参数仍有梯度张量；no_grad()则不记录被包住的运算。这不是固定的三阶段训练算法，而是本例逐次参数变化造成的结果。其他损失、初始化或相消情况会改变时序，也不能保证每个参数元素都更新。

题1.6-5

由内参反投影得 $K^{-1}[380,240,1]^\top=[0.1,0,1]^\top$。因此两种Z深度对应：

$$p_1^C=[0.1,0,1]^\top\ \mathrm{m},\qquad p_2^C=[0.2,0,2]^\top\ \mathrm{m}.$$

用 $u=600X^C/Z^C+320$、$v=600Y^C/Z^C+240$ 重投影，二者都得到(380,240)。像素本身没有区分沿射线的距离。

旋转把[0.1,0,1]变为[0,0.1,1]，再加平移，得到 $p_1^B=[1,0.1,1]^\top$米。逆变换先减 t，再乘 $R^\top$：

$$R^\top(p_1^B-t)=R^\top[0,0.1,1]^\top=[0.1,0,1]^\top.$$

若拿到的是反方向外参，其旋转为 $R^\top$，平移为 $-R^\top t$，不能把原平移原样沿用。

位移来自两个点之差，$(Rp_2+t)-(Rp_1+t)=R(p_2-p_1)$，所以[0.1,0,0]米只旋转为[0,0.1,0]米。再加平移会得到错误的[1,0.1,0]，凭空多出1米。若传感器给的是欧氏距离 d，也应沿单位方向取点，即 $p^C=dq/\|q\|$、$q=K^{-1}[u,v,1]^\top$，不能把 d 直接当成Z深度。

只有RGB和相机编号，一般不足以按这些公式确定唯一三维点。还需当前内参、深度或其他约束，以及有效的相机到基座变换。深度可来自传感器、多视角或明确作为估计量使用的单目模型；已知平面也能约束交点，但都是额外信息。相机编号可以用来查找已有标定数据，仅将编号变成可学习嵌入不等于提供标定。腕部相机的外参还可能依赖当前关节状态和手眼标定。

将几何量与视觉特征相加前，应核对同一相机、时刻和图像区域的对应，图像与深度配准，深度有效性与单位、外参方向，以及各分支的维度和归一化。如果一个patch跨越多个深度，须说明采用哪个代表点或保留哪些点；无效深度不能直接当成坐标原点。

裁剪、缩放尤其容易破坏对应。例如原图先裁掉左侧100像素、上侧40像素，再缩小一半，原像素(380,240)变为(140,100)，内参应同步变为 $f_x=f_y=300, c_x=110, c_y=100$。反投影仍为[0.1,0,1]；若沿用旧内参，则得到错误方向[−0.3,−0.233333,1]。投影到相同宽度的网络特征，无法修复这类输入错误。

1.7 动作接口、条件分布与分块生成

1.7.1 动作表示决定预测问题的含义

绝对关节位置指下一目标角；关节增量指相对变化；末端位移还依赖坐标系与**逆运动学**；力矩要求与更快的物理控制直接连接。这些不同表示，不能只通过补齐向量长度互换。输出“0.1”时必须知道单位、坐标、周期和控制模式。

动作归一化让不同维度落在适合训练的尺度。例如米制位移 a 的均值为 0.01、标准差为 0.02，标准化值 z=(a−0.01)/0.02。标签 0.03 米对应 z=1；预测 z=0.5 对应物理量 0.02 米。部署必须使用匹配统计量反归一化；用另一机器人的统计量会改变实际命令。

标准化平方损失实际上按各维尺度改变权重。1 毫米位置误差与 0.1 弧度旋转误差，哪个更严重由任务决定；把它们缩到相同数值范围并不自动给出正确偏好。夹爪的开闭还可能是离散量，与连续位移采用不同损失。

动作块 A 的维度为 H×da：H 个未来控制时刻，每个动作 da 维。每维还可能有有效掩码 m，区分该机器人不存在的关节、缺失标签和真实零动作。把填充的零也当作标签训练，会教模型把缺失维度输出为零，并可能污染共享表示。

训练预测的是未来命令还是未来实测响应，也必须明说。命令、执行器延迟后的关节角和图像中的末端位移，时间与语义不同。后面比较算法时，固定动作接口和时间对齐是前提，否则“某损失更好”可能只是预测对象更容易。

1.7.2 自回归、并行输出与动作分块

自回归生成把一个动作序列分解成多个条件预测：先生成第一个输出，再根据前面的输出生成下一个。离散动作序列 b_1...bL 的概率为

$$p_\theta(b_{1:L} \mid c) = \prod_{i=1}^{L} p_\theta(b_i \mid c, b_{<i}).$$

它能表示输出之间的依赖，例如第一步选择左绕，后面继续左绕，而不是每个动作独立选一边。这里的顺序可能是一个时刻内的动作维度，也可能是跨时刻压缩 token 的顺序，需看分词方案；L 不一定等于机器人动作块长度 H。

训练常用教师强制 teacher forcing：第 i 个位置读取真实标签的前缀 b<i，但不能读取自己的答案 bi 或未来答案。这样所有位置的损失可以并行计算。部署没有真实前缀，使用模型已生成的前缀，因此错误可能累积。训练并行不意味着自回归推理也能一次获得全部答案。

并行动作头给固定的一组输出位置，让它们一次输出整个动作块，再用回归或生成目标训练。它减少逐 token 解码，但分布表达和条件结构可能不同。不能认为“并行”就完全没有动作间联系：动作位置可以通过**注意力**交流，也可以共享主干表示。

分块预测有两个时间长度。控制周期 0.05 秒，H=8，预测覆盖 0.4 秒；若只执行前两步，0.1 秒后再读新观察。这种滚动执行减少完全开环持续时间，但仍会受计算延迟、观测时效和接触突变影响。生成步骤数、动作块长度和执行前缀长度是三个独立设置。

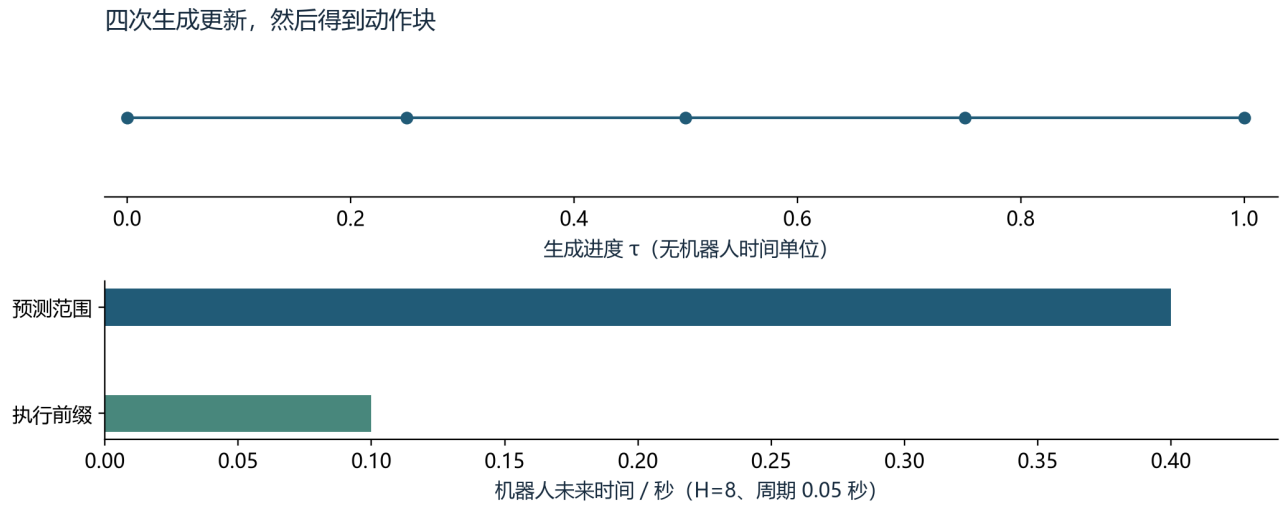


图1.19：生成过程里的迭代轴与真实控制轴不同。预测长动作块不等于机器人必须把整块执行完。

1.7.3 回归、高斯、混合分布与多模态动作

确定性回归输出一个动作。平方损失的理想最优解是条件均值，绝对误差 L1 的理想最优解是条件中位数。L1 对较大误差的惩罚增长较慢，但不自动解决多个等价动作模式：对称的 −2、+2 例子中，两者之间的很多位置都可成为中位数，仍可能落在不可行区域。

高斯策略输出均值 $\mu_\theta(c)$ 和标准差 $\sigma_\theta(c)$，用它们描述动作分布。对一维示范动作 a，忽略常数后的负对数似然为 $\frac{(a-\mu)^2}{2\sigma^2} + \log\sigma$。第一项要求均值接近示范，但允许模型在难以确定的条件下给出较宽的分布；第二项惩罚无节制地扩大范围。两项共同决定“中心在哪里、合理范围有多宽”。部署可以按该高斯采样，或使用其均值，选择哪一种必须与任务要求一致。一个高斯适合描述围绕一个中心的变化，不能完整表示左绕、右绕这样的两个分离选择。

单个高斯只有一座峰，可能把两条正确路线连成宽而危险的分布。混合密度网络可以输出 K 个成分的权重、均值和方差

$$p_\theta(a \mid c) = \sum_{k=1}^{K} w_k(c)\,\mathcal{N}(a; \mu_k(c), \Sigma_k(c)).$$

这里 $w_k(c)$ 是第 k 个成分的权重，各权重非负且总和为 1；μ_k、Σ_k 描述该成分的中心与范围。网络读取条件，一次输出这些参数，训练标签仍是具体示范动作。对一个标签计算全部成分的混合概率，再取负对数：越能解释该标签的成分，承担的更新越大。因此不必事先人工给每条示范指定成分。左绕和右绕数据可以分别由不同成分解释。

部署时可先按权重选成分，再在选中的成分内采样；若需要确定性行为，也可以按事先规定选择某个成分的代表动作。直接对全部成分取加权均值，会再次混合不同路线。混合模型保留了选择，执行规则决定最终使用哪一个选择。

模式坍缩是多个成分最终学到相同或少量模式。成分过多也可能拟合噪声。多步动作若每一步独立选成分，可能左、右路线来回切换；需要动作块、共同**潜变量**或自回归依赖保持协调。

概率分布模型主要回答“训练数据支持哪些动作”，不等于评价器回答“哪一个动作长期更好”。若希望偏向更高回报，需要质量条件、评价选择或后训练目标；不能靠增大分布复杂度自动得到优化后的策略。

一个示范怎样训练多个分布成分

观察到动作 a 后，第 k 个成分对这个标签的解释份额为

$$r_k = \frac{w_k \mathcal{N}(a; \mu_k, \Sigma_k)}{\sum_j w_j \mathcal{N}(a; \mu_j, \Sigma_j)}.$$

这叫**成分责任度**。它回答“在当前混合模型中，哪个成分更能解释这一条示范”，与执行时先按 w_k 选成分是不同步骤。设左、右成分均值分别为 $-2, 2$，标准差均为 0.5，权重各一半；左绕标签 $a = -2$ 几乎全部由左成分解释，右绕标签则反过来。因此两类标签能分别更新不同成分，而无需提前标注“这是第几个成分”。World Models 的混合密度预测教学

若混合权重由 softmax 得分 b_k 得到，负对数似然对该得分的梯度是 $w_k - r_k$。一个成分承担的解释份额比当前权重大，梯度下降就提高它的权重；均值等参数也按责任度获得相应信号。这样，拟合分布连接到了具体的参数更新。

```
import torch
from torch.distributions import Normal

def mixture_nll(action, logits, means, log_stds):
    # action: (B,)，各成分参数：(B,K)；这里示范一维混合高斯。
    # 1. 每个成分评价同一个真实动作，得到其对数密度。
    component_log_prob = Normal(means, log_stds.exp()).log_prob(action[:, None])
    # 2. 成分权重乘密度，在对数形式中变成相加。
    weighted_log_prob = torch.log_softmax(logits, dim=-1) + component_log_prob
    # 3. 汇总成分后取负对数，再按样本平均；所有成分共同参与训练。
    return -torch.logsumexp(weighted_log_prob, dim=-1).mean()
```

代码先求混合概率再取对数，对应 $-\log \sum_k w_k p_k(a)$；分别求每个成分的负对数后再平均，会要求每个成分都解释同一个标签，训练问题就变了。部署保留整块的成分或潜变量选择，有助于避免动作在不同路线之间来回切换。

1.7.4 一份动作定义比一个输出维度更完整

七维动作可能表示三维平移、三维旋转增量和夹爪，也可能表示七个关节目标。即使都是三维平移，还要说明在世界系、基座系还是工具系，单位为米还是厘米，数值是绝对目标还是单步增量。夹爪数字可能表示开度、归一化命令或二元开闭；它们的学习和执行语义不同。

动作块预测 H 步，执行前 k 步后再观察，意味着模型的预测范围是 H 个控制周期，实际开放环路范围是 k 个周期。H 增大有助于协调多个动作，却可能增加训练标签多样性和输出计算；k 增大降低重规划频率，却增加对新接触和遮挡的反应延迟。H 和 k 应作为两个实验变量，不能只称“使用长动作块”。

动作分布中的变化可能来自不同原因：同一条件下本来就有多种合理路线，也可能是相机没有看清位置，或策略尚未学好。一个多峰动作输出不会替我们识别这些原因。若希望保留示范的不同选择，需

要合适的联合分布；若任务只需稳定执行一条路线，也可以明确选择该路线。若关键状态不可见，增加观察或触摸则可能比在当前输入下反复采样更有用。

1.7.5 只在有效动作上计算损失

轨迹尾部和不同身体可能存在无效位置。先选有效标量，再相减求平均，使缺失位置既不影响分子，也不改变样本权重。

算法步骤：

1. 合并时间与动作维度的有效位置。
2. 确认本批次存在有效监督。
3. 只对有效标量计算误差，再取平均。

```
# 步骤1：合并时间与动作维度的有效位置
valid = np.broadcast_to(valid, pred.shape)
# 步骤2：确认本批次存在有效监督
count = int(valid.sum())
if count == 0:
    raise ValueError('empty supervision')
# 步骤3：只对有效标量计算误差，再取平均
residual = pred[valid] - target[valid]
loss = np.mean(residual ** 2)
```

1.7.6 练习题

题1.7-1 一个两步动作块有两种可行示范：左绕 $(-1,-1)$、右绕 $(1,1)$，各占一半。本例规定中途换边的两种序列不可行。若模型在每一步都独立按正确的边缘概率采样，完整动作块有多大概率可行？怎样改建模才能保留两种路线，又避免中途随机换边？

题1.7-2 一个两步、两维动作块只有第一步有效，预测 [1,2]，标签 [0,0]。按有效标量计算均方误差及梯度；如果把填充步也算进分母，有什么后果？

题1.7-3 一维混合高斯用 softmax 得到成分权重 w，真实动作 a 的成分责任度为 r。推导负对数混合似然对权重 logits 的梯度为什么是 $w-r$；当 $w=[0.5,0.5]$、$r=[0.9,0.1]$ 时，更新倾向是什么？为什么不能先分别计算每个成分的负对数似然再等权平均？

1.7.7 参考答案

题1.7-1答案 两个时刻分别以 1/2 采样 −1、+1，且相互独立，会产生四种序列，每种概率都是 1/4：(-1,-1)、(-1,+1)、(+1,-1)、(+1,+1)。只有不换边的两种可行，因此可行概率为 1/2。虽然每一步的边缘概率都与示范一致，联合分布却错了：示范中第二步与第一步完全相关。

一种改法是先采样一次路线变量 z∈{左,右}，再由同一个 z 决定两步动作；左右各 1/2，完整块就各以 1/2 产生 (-1,-1) 或 (+1,+1)。另一种是自回归建模 $p(a_1,a_2|c)=p(a_1|c)p(a_2|a_1,c)$，学到第一步为左时第二步必为左、第一步为右时第二步必为右。还可用扩散或流匹配直接生成整个动作块，使训练目标能够表达时间相关性。

执行时也应保留这种一致性：若每次重规划都重新随机选路线，即使块内连贯，块间仍可能来回切换。可把上一段动作、已选路线或历史加入条件。本题真正缺失的是动作之间的相关结构，而不是某一步的概率值。

题1.7-2答案 只有第一步的两个标量有效，所以分母为 2：

$$L = \frac{(1-0)^2 + (2-0)^2}{2} = 2.5.$$

对预测第一步求导为 [2×1/2,2×2/2]=[1,2]；填充步不参与损失，其梯度为 [0,0]。

若分子仍只保留有效项，却把两步四个标量都放入分母，L=5/4=1.25，有效梯度变成 [0.5,1]。目标数值看似更小，只是学习信号被缩小了。不同样本填充比例不同，还会让较短有效序列获得更小权重。若连分子也不遮蔽，模型还会被要求预测没有实际含义的填充值。

正确做法先对有效标量加权求和，再除以有效标量数量。若研究目的改为让每条轨迹等权，也可以先分别求每条轨迹有效平均，再跨轨迹平均；两种平均方式代表不同的数据权重，应事先确定。

题1.7-3答案 设第 i 个高斯在动作 a 处的密度为 $\phi_i(a)$，权重分数为 α_i。对这些分数求导时，先固定各高斯的均值与方差：

$$w_i = \frac{e^{\alpha_i}}{\sum_j e^{\alpha_j}}, \qquad p(a) = \sum_i w_i \phi_i(a), \qquad L = -\log p(a).$$

权重 w_i 表示看到动作标签之前的成分比例；责任度

$$r_i = \frac{w_i \phi_i(a)}{\sum_j w_j \phi_j(a)}$$

则表示看到这条动作后，各成分对解释它的贡献比例，且 $\sum_i r_i = 1$。

利用 $\partial w_i / \partial \alpha_j = w_i(\mathbf{1}_{i=j} - w_j)$，链式法则得到

$$\frac{\partial L}{\partial \alpha_j} = -\frac{1}{p(a)} \sum_i \phi_i(a) w_i (\mathbf{1}_{i=j} - w_j) = -r_j + w_j \sum_i r_i = w_j - r_j.$$

本题梯度为 $[-0.4, +0.4]$。梯度下降使第一个分数上升、第二个下降，因此第一成分权重提高。原因是它解释这条标签的贡献达到90%，却只分到50%的原始权重。真实训练会汇总所有样本，并分别更新均值与方差；不能仅凭这一条标签就断言整个数据集都应归第一成分。

混合模型假定一条动作可由某个成分解释，故应先加权相加密度，再取负对数。等权平均 $-\frac{1}{2}\log\phi_1(a) - \frac{1}{2}\log\phi_2(a)$ 则同时要求两个成分都拟合每条动作，也忽略了可学习权重，变成另一种目标。例如 $\phi(a) = [0.9, 0.1]$、$w = [0.5, 0.5]$ 时，正确损失为 $-\log 0.5 \approx 0.6931$，等权平均却为约1.2040。后者会强烈惩罚第二成分，即使第一成分已能解释该动作，容易削弱不同成分各自负责不同路线的分工。实现应计算 -logsumexp(log_w + component_log_density)，再对样本平均。

进一步阅读：Stan User’ s Guide：Finite Mixtures，混合似然与后验责任度、Keras：Approximating non-Function Mappings with Mixture Density Networks，混合分布负对数似然实现。

1.8 潜变量、变分目标与 ACT 的训练部署差别

同一张桌面图像下，机械臂可以从物体左侧或右侧绕行。若条件里没有指定路线，示范动作就未必唯一。**潜变量** z 是模型增加的中间随机量，用来容纳条件尚未解释的变化，例如路线或操作习惯。条件变分自编码器 **CVAE**（Conditional Variational Autoencoder）在训练时用推断网络 $q_\phi(z \mid c, A)$ 读取条件 c 与真实动作块 A，再让解码器依据 c, z 重建动作。这里的“路线”说明潜变量可能承担的职责，不表示每一维都会自动学成可命名的语义。

部署时没有真实 A，模型改从**先验** $p(z \mid c)$ 获得 z，或使用实现规定的固定值。先验是在未读取这条示范动作时给出的潜变量分布；训练推断网络读到动作以后给出的 q，则是在解释这条示范，称为**近似后验**。CVAE要解决的困难是：训练时可以借助答案整理变化，运行时却必须在没有答案的情况下生成动作。

为什么需要先验匹配？只用重建损失，推断网络可能把整段答案编码进 z，训练重建很好，但部署拿到的先验 z 根本不在训练使用区域。教学目标为

$$L = L_{\text{rec}} + \beta\, D_{\text{KL}}(q_\phi(z \mid c, A) \| p(z \mid c)).$$

重建项让 z 与条件支持动作预测；**KL 散度**（Kullback-Leibler Divergence）衡量两个概率分布的差异；把它加入损失，是为了约束训练时的推断分布，使其不要任意偏离部署时使用的先验。β 控制重建与约束的取舍；β 太小可能训练部署不匹配，太大可能忽略 z，称**后验坍缩**。KL 小和重建准都不单独保证闭环任务成功。变分自编码器（Variational Autoencoder，VAE）原论文

高斯推断网络输出均值 μ 和标准差 σ，用 $z = \mu + \sigma \odot \epsilon$、$\epsilon \sim \mathcal{N}(0, I)$ 采样，这称为**重参数化**。随机数 ϵ 独立于网络参数，采到它以后，z 对均值的导数为 1，对尺度的导数为 ϵ；重建误差便能沿 z 回传，教推断网络怎样调整这次示范对应的潜变量分布。同时，若 $q = \mathcal{N}(1, 1)$、$p = \mathcal{N}(0, 1)$，KL 为 0.5，约束倾向把偏离先验的均值拉回来。重建项要求 z 有助于解释动作，KL 项限制编码不要任意偏移，两者共同决定保留多少额外信息。

ACT（Action Chunking with Transformers）是一种用 Transformer 预测**动作块**的策略，代表实现采用 CVAE 训练，并结合重叠预测的时间集成。在原始实现中，训练推断编码器利用本体状态与未来动作标签，视觉进入动作解码路径；不要把本书通用 c 符号误读成每个实际分支都读取所有模态。常见部署使用规定的零潜变量，省去读取未来标签。ACT

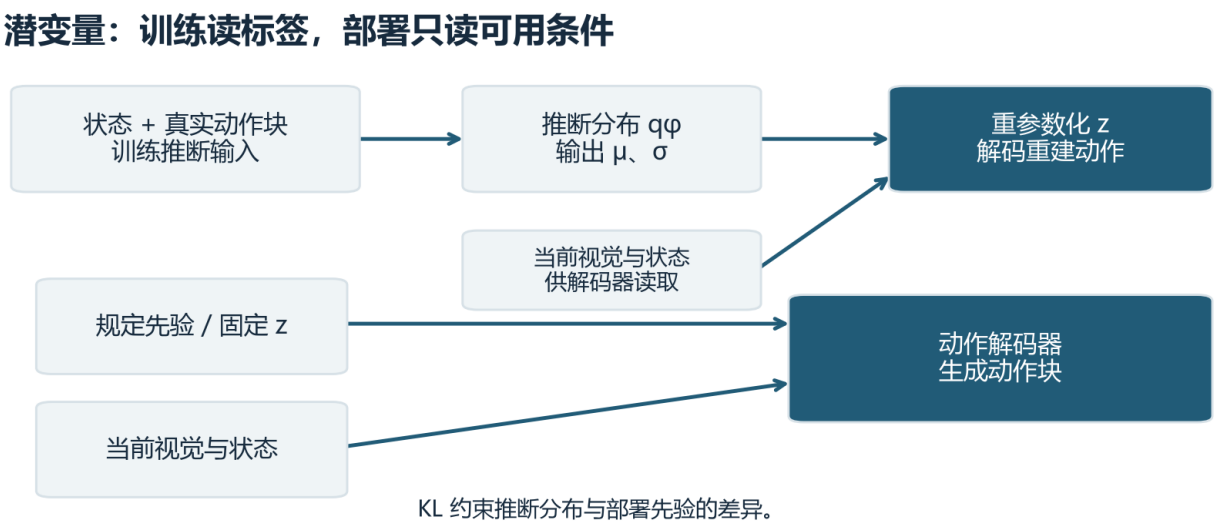


图1.8-1：训练允许推断网络读动作标签，部署只读可用条件。图中以均值 μ、标准差 σ 描述分布；后文网络实际输出 μ, ℓ，再计算 $\sigma = e^{\ell/2}$。两者是同一高斯采样关系的不同参数写法。

潜变量生成能表达多种行为，但固定 z 的具体实现不会自动在运行时采样所有模式。比较 ACT 时要分别检查动作分块、潜变量、时间集成和执行周期，不能把全部收益归因于 CVAE。

1.8.1 变分目标为何需要一个训练时额外的推断网络

潜变量 z 用来解释“相同条件下，示范为何有不同动作风格或路线”。若直接计算所有 z 对动作概率的贡献，积分可能难以求出。训练推断网络读取条件和真实动作，为当前示范找一个可能的 z；解码器再用条件和 z 重建动作。推断网络看到了未来标签，因此它只能用于训练，不能原样作为运行时输入。

条件变分目标可以写为

$$\log p_\theta(A \mid c) \geq \mathbb{E}_{q_\phi(z|c,A)}[\log p_\theta(A \mid c, z)] - D_{\mathrm{KL}}(q_\phi(z \mid c, A) \| p(z \mid c)).$$

右边称为**证据下界**（Evidence Lower Bound，ELBO）。第一项衡量：从当前示范推断出的 z，能否帮助解码器解释动作；第二项约束推断分布与未读标签时规定的先验之间的差别。真实对数似然等于右边再加上 $D_{\mathrm{KL}}(q_\phi(z \mid c, A) \| p_\theta(z \mid c, A))$，而KL非负，所以右边是下界。固定生成模型时，让推断分布接近真实后验会收紧下界；训练生成模型时，则优化这个可采样估计的似然替代目标。由于差距也会随生成模型改变，一次下界上升并不保证真实似然同步上升。

前面的“重建损失加 KL”可由这个目标得到，但需要看重建项怎样定义：若它是相应观测分布的负对数似然，且 KL 权重为 1，就对应上述负下界；改变权重或采用其他重建误差，是在调整训练取舍。ACT 的具体损失与部署潜变量选择应按其实现理解。

1.8.2 用生成图解释先验、后验与变分训练

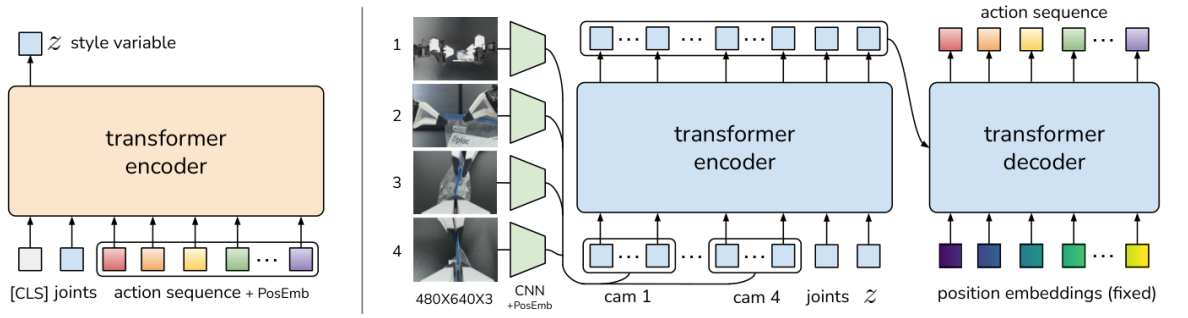


图1.8-2：Zhao 等，ACT 论文 Figure 4。左侧编码器读取训练动作序列，右侧策略网络根据图像、关节状态和潜变量预测动作块。原论文。

先看图左侧：joints 是关节状态，action sequence 是真实示范动作序列，PosEmb 是 Positional Embedding（位置嵌入），[CLS] 是用来汇总序列的特殊 token。左侧的编码器用它们产生潜变量分布，z 可以编码这条示范的动作方式。图右侧的 CNN（Convolutional Neural Network，卷积神经网络）处理不同相机图像；cam 是 camera（相机），图像特征、本体状态与 z 一起进入策略。最右下方的位置嵌入为各未来动作位置提供查询，解码器同时输出整段动作序列。

图里有两处 Transformer encoder，它们承担不同职责。左侧是读取动作答案、推断 z 的训练编码器；右侧是融合图像、本体状态与 z 的观测编码器，部署时仍然保留。CVAE所说的“解码器”，包括右半边整条动作生成路径，并不只指图上最后一个 Transformer decoder。否则，“部署时丢弃编码器”很容易被误解成把观测处理也删掉。

这里需要明确两种分布。**先验**（Prior）是在没有读取这条真实动作答案时，为潜变量规定的分布；**后验**（Posterior）是在动作已经观测到以后，对哪个潜变量可能解释它的判断。CVAE 的 $q_\phi(z \mid c, A)$ 是推断网络给出的近似后验，不等于生成模型的真实后验 $p_\theta(z \mid c, A)$。真实后验通常也因为要计算动作的边缘概率而难求，所以才需要近似。

生成模型按 $p(z \mid c)p_\theta(A \mid c, z)$ 定义动作分布。为了判断一条动作 A 有多合理，应把所有 z 的贡献加起来：$p_\theta(A \mid c) = \int p_\theta(A, z \mid c)\, dz$。这不是要求动作网络每次真的枚举无限多个风格，而是模型所定义的概率。训练困难是这个积分；推断网络则帮助我们优先采到与当前示范有关的 z。

把积分改写成 q 下的期望时，q 必须覆盖联合分布可能取值的区域，否则会漏掉那部分贡献。本节的正方差高斯在整个实数空间都有正密度，后面的两状态例也没有概率为零的项；以下假设相关对数期望有限。插入可采样的 q，再利用对数的凹性，可得到

$$\begin{aligned}\log p_\theta(A \mid c) &= \log \mathbb{E}_{z\sim q}\left[\frac{p_\theta(A, z \mid c)}{q(z \mid c, A)}\right] \\ &\geq \mathbb{E}_q\left[\log p_\theta(A \mid c, z) + \log p(z \mid c) - \log q(z \mid c, A)\right] \\ &= \mathbb{E}_q[\log p_\theta(A \mid c, z)] - D_{\mathrm{KL}}(q \| p(z \mid c)).\end{aligned}$$

这就解释了为什么目标同时有重建和先验约束：第一项让当前示范对应的潜变量能解释动作，第二项防止推断分布完全离开生成时可用的先验。两项来自同一个边缘似然下界，而不是先凭直觉拼出两个

损失。推断网络帮助收紧下界，生成网络优化这个可计算目标；动作在真实任务中是否可行，仍要通过执行检验。Stanford 深度生成模型课程：VAE

原始 ACT 的部署常将 z 取为先验均值0。左侧读取真实动作的网络此时不参与预测；右侧动作查询也不是未来真值。它可以与前面的经典编解码器图共享一些结构，但整块并行输出与自回归逐 token 输出的输入组织不同。看图时先找标签在哪一侧、部署时哪一侧保留，就能识别这一区别。

1.8.3 网络到底输出什么，损失为什么能更新它

先选定一个常见、便于推导的CVAE：先验为 $p(z)=\mathcal{N}(0,I)$，推断网络输出 $\mu_\phi(c,A),\ell_\phi(c,A)\in\mathbb{R}^{d_z}$，其中 $\ell=\log\sigma^2$ 是逐维对数方差。因此：

$$q_\phi(z\mid c,A)=\mathcal{N}\big(\mu,\operatorname{diag}(e^{\ell})\big),\qquad \sigma=e^{\ell/2}. \tag{1.8-1}$$

网络输出的是分布参数，还没有抽出具体的 z。使用对数方差，可以让网络自由输出实数，再通过指数保证方差为正。注意标准差是 $e^{\ell/2}$；若把 e^{ℓ} 直接乘噪声，就把方差当成标准差，采到的是另一个分布。

解码网络 $f_\theta(c,z)$ 输出动作块的预测值 $\hat{A}\in\mathbb{R}^{H\times d_a}$。$H$ 是未来动作个数，d_a 是每个动作的维数。要把预测值写成概率模型，还需规定 $p_\theta(A\mid c,z)$。例如把展平的动作块视为均值 $\hat{A}$、固定方差 $s_A^2 I$ 的高斯，负对数似然中与参数有关的部分就是：

$$L_{\text{rec}}=\frac{1}{2s_A^2}\|\hat{A}-A\|^2. \tag{1.8-2}$$

这一步解释了“重建”在做什么：它让给定 c,z 的预测靠近这条示范。平方损失与高斯假设相对应，而整个动作分布还要对 z 积分；不能因为每个 z 下使用高斯，就认为边缘动作分布只能有一个峰。不同 z 可以经非线性解码器产生不同路线，组合后形成更复杂的分布。

再看先验约束。将两个高斯的对数密度相减并在 q 下取期望，用到 $\mathbb{E}_q[(z_j-\mu_j)^2]=\sigma_j^2$ 和 $\mathbb{E}_q[z_j^2]=\mu_j^2+\sigma_j^2$，得到：

$$\begin{aligned} D_{\text{KL}}(q\|p)&=\mathbb{E}_q[\log q(z)-\log p(z)]\\ &=\frac{1}{2}\sum_{j=1}^{d_z}\left(\mu_j^2+\sigma_j^2-1-\log\sigma_j^2\right)\\ &=\frac{1}{2}\sum_{j}\left(\mu_j^2+e^{\ell_j}-1-\ell_j\right). \end{aligned} \tag{1.8-3}$$

均值项惩罚远离0；方差相关的 $e^{\ell}-1-\ell$ 在 $\ell=0$ 时最小，对应方差1。这个式子约束的是每条示范产生的推断分布与标准正态先验的差别，并不是让所有 z 都变成数值0。方差1的正态分布仍包含大量不同取值。

若推断网络与解码网络不共享参数、先验也固定，重建项同时影响 ϕ,θ，KL项只直接影响 ϕ。重建误差必须经过“解码器→z→μ,ℓ→推断网络”才能到达 ϕ。重参数化使这条路径可以求导：

$$\epsilon\sim\mathcal{N}(0,I),\qquad z=\mu+\exp(\ell/2)\odot\epsilon. \tag{1.8-4}$$

随机的是外部输入 ϵ。对已经抽到的这一份噪声求导时，它保持不变，z 随网络输出连续变化；下一训练样本仍可以重新抽噪声。在适当可微与可积条件下，对这些路径梯度取平均，就估计了期望目标的梯度。Kingma与Welling的VAE教程，重参数化与高斯后验

1.8.4 同一样本：从采样到两套网络的一次更新

将动作块暂时缩成一个无量纲动作坐标，以便完整追踪更新。输入条件为 $c = 1$，示范标签为 $A = 2$。该线性小模型用来追踪梯度，不承担验证多峰生成能力的任务，也不是Panda的米制控制接口。推断网络是两个线性输出：

$$\mu = a_\mu c + b_\mu A, \qquad \ell = a_\ell c + b_\ell A,$$

初始权重为 $(a_\mu, b_\mu) = (0.2, 0.4)$、$(a_\ell, b_\ell) = (0, 0)$，所以 $\mu = 1, \ell = 0, \sigma = 1$。本次抽到 $\epsilon = 0.5$，得到 $z = 1 + 1 \times 0.5 = 1.5$。解码器为：

$$\hat{A} = w_c c + w_z z + b, \qquad (w_c, w_z, b) = (0.5, 0.8, 0),$$

于是 $\hat{A} = 1.7$。取 $s_A^2 = 1$、$\beta = 0.1$：

$$L_{\text{rec}} = \tfrac{1}{2}(1.7 - 2)^2 = 0.045, \qquad D_{\text{KL}} = \tfrac{1}{2}(1^2 + 1 - 1 - 0) = 0.5,$$

$$L = L_{\text{rec}} + 0.1 D_{\text{KL}} = 0.095.$$

预测比标签小0.3，重建项因此要把输出往上推。先沿解码器回传：

$$\frac{\partial L_{\text{rec}}}{\partial \hat{A}} = -0.3, \qquad \frac{\partial L_{\text{rec}}}{\partial z} = -0.3 \times 0.8 = -0.24.$$

再沿重参数化回传。$\partial z/\partial \mu = 1$；$\partial z/\partial \ell = \tfrac{1}{2}\sigma\epsilon = 0.25$，所以重建项对 μ, ℓ 的梯度分别为 $-0.24, -0.06$。KL对这两个量的加权梯度则为：

$$\frac{\partial (0.1 D_{\text{KL}})}{\partial \mu} = 0.1\mu = 0.1, \qquad \frac{\partial (0.1 D_{\text{KL}})}{\partial \ell} = 0.05(e^\ell - 1) = 0.$$

因此总梯度为 $\partial L/\partial \mu = -0.14$、$\partial L/\partial \ell = -0.06$。对均值来说，重建项要求它上移、使本次 z 增大；先验约束要求它靠近0，两股信号在这里方向相反，总结果仍是上移。对尺度来说，这次正噪声使增大标准差能够增大 z，因此本样本给出增大尺度的信号；换一份噪声，单样本信号可以不同。

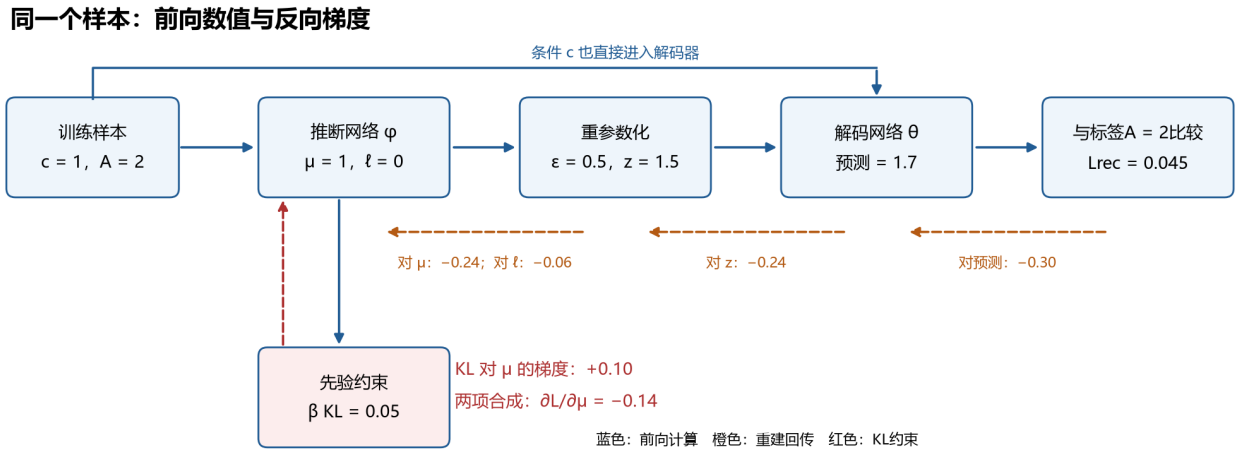


图1.8-3：蓝线为前向计算，橙色虚线为重建梯度，红色支路为KL对推断参数的约束。数值均来自本节同一个样本。两项损失共享 μ, ℓ，但只有重建误差经过解码器。

μ, ℓ 是网络输出，并非本例中直接保存的可训练参数。还要乘上输入，才能得到各权重的梯度：例如 $\partial L/\partial b_\mu = (-0.14)A = -0.28$。用学习率 $\eta = 0.1$ 同时执行一次普通梯度下降，得到：

可训练参数	初值	重建梯度	加权KL梯度	总梯度	更新后
a_μ	0.2	−0.24	0.10	−0.14	0.214
b_μ	0.4	−0.48	0.20	−0.28	0.428
a_ℓ	0	−0.06	0	−0.06	0.006
b_ℓ	0	−0.12	0	−0.12	0.012
w_c	0.5	−0.30	0	−0.30	0.530
w_z	0.8	−0.45	0	−0.45	0.845
b	0	−0.30	0	−0.30	0.030

将同一输入和同一份 ϵ 再算一次，μ 变为1.07，ℓ 变为0.03，$z \approx 1.57756$，预测变为 $\hat{A} \approx 1.89304$。重建损失降到约0.005721，KL反而升到约0.572677，总损失降到约0.062988。优化总目标并不要求每一项都下降；本次更新接受了更大的先验偏离，换来更明显的重建改善。

这次固定噪声的对照用来检查梯度。实际训练对许多示范与噪声取平均，不能由一次下降推出新样本一定改善，更不能推出机器人任务已经成功。

1.8.5 采样值相同，为什么梯度仍可能错

现在只改一处：把采到的 z 在送进解码器前执行 z.detach()。前向数值仍是1.5，预测和损失也完全一样，但自动求导不再沿这条边追溯 z 如何由 μ, ℓ 产生。解码器还能学“怎样利用当前这个 z”，推断网络却收不到“怎样改变 z 才有助于重建”的信号。

在本例初始点，总损失对 a_μ, b_μ 的梯度会从 $(-0.14, -0.28)$ 变成 $(0.1, 0.2)$，只剩KL项；尺度权重的梯度都变成0。更新方向已经反转，尽管打印出来的损失一模一样。因此，检查训练不能只看损失是否有数值，还要确认该损失能到达本来应该更新的参数。

沿这条被切断的路径仍对全部参数更新一次，固定噪声的总损失也能降到约0.068184。解码器与KL支路仍在改变参数，所以“损失在下降”也不足以证明重建目标正确地训练了推断器。这个数值与正确路径的0.062988分开记录在配套结果中。

把随机采样移到外部，并不是把潜变量从计算图中删除。重参数化保留了“分布参数改变，会怎样改变本次样本”的可微关系；detach 则主动删除这段求导依赖。这与第1.2节冻结参数、保留输入梯度的区别相连。

1.8.6 对照伪代码运行完整算例

```
P1  从一条训练样本读取 c、A，推断网络输出 μ、ℓ
P2  采 ε，令 z = μ + exp(ℓ/2)·ε，解码得到 A_hat
P3  计算重建损失与 KL，按 β 组成总目标
P4  反向传播，计算两条损失路径对推断和解码参数的梯度
P5  用优化器同时更新两套网络；换一批示范与噪声后重复
```

五步的更新核心如下；完整网络、分项梯度和数值核对在 practice/cvae_step.py。

```
# P1：编码这条示范，输出均值和对数方差。
mu, logvar = model.posterior(torch.stack([condition, label])).unbind()

# P2：噪声不带参数，mu 和 logvar 到 z 的求导路径仍然保留。
z = mu + torch.exp(0.5 * logvar) * epsilon
prediction = model.decoder(torch.stack([condition, z])).squeeze()

# P3：本算例采用固定方差1的高斯重建目标。
reconstruction = 0.5 * (prediction - label).square()
kl = 0.5 * (mu.square() + logvar.exp() - 1 - logvar)
loss = reconstruction + beta * kl

# P4：一次反向传播同时给两套网络计算梯度。
optimizer.zero_grad()
loss.backward()

# P5：同时更新两套网络。
optimizer.step()
```

在学习包根目录安装 practice/requirements.txt 后，运行 python practice/cvae_step.py。结果保存为 results/cvae_step.json，包括更新前后值、三组梯度、切断路径的反例和有限差分核对。本次运行的总梯度与中心有限差分最大差约 1.46×10^{-11}。这里运行的是标量CVAE的一次更新；ACT的完整视觉策略训练在后续模仿学习项目中展开。

1.8.7 从通用CVAE到ACT，还要作哪些选择

ACT沿用潜变量训练思路，但输入、重建误差和部署规则都有具体选择。作者实现中，推断支路读当前关节位置与示范动作块；动作生成支路读图像、关节位置与 z。动作是双臂的14维绝对关节目标，输出为 $H \times 14$。重建使用带填充遮蔽的L1误差，而不是本节用于手算的半平方误差。ACT作者策略损失

部署时，作者实现用 $z = 0$，不再执行读取动作标签的左侧推断器。潜变量训练、动作分块、时间集成因而是三个不同环节：前者影响训练时如何解释示范，第二项规定一次输出的联合预测范围，第三项决定运行时怎样处理多个重叠块。第2.3节将把这个模型接到逐控制步的选择过程。

固定 $z = 0$ 规定的是解码器输入，并没有求动作分布中密度最高的动作。先验均值经过非线性解码，一般也不等于动作分布的均值：例如 $Z \sim \mathcal{N}(0, 1)$、$g(z) = z^2$，有 $g(\mathbb{E}[Z]) = 0$，而 $\mathbb{E}[g(Z)] = 1$。

再考虑构造的解码函数 $f(z) = 2\tanh(5z)$：$z = -1, 1$ 分别输出约 $-1.9998, 1.9998$，$z = 0$ 却输出0。若正、负两侧代表绕过障碍的两种选择，中间目标可能不可行。这个奇对称函数在标准正态先验下恰好满足 $\mathbb{E}[f(Z)] = f(\mathbb{E}[Z]) = 0$；它说明均值动作也可能无效，不用于证明两个均值不等。固定零潜变量也没有执行任何随机选择。“模型能够表达多种输出”与“部署规则会采样这些输出”仍需分别判断。

还能在这个确定性例子中直接检查动作密度。本例的 f 严格递增，有限 z 处导数为正，每个输出只对应一个潜变量值。令 $Y = f(Z)$，一个很小的潜变量区间 dz 被映射成约 $|f'(z)|dz$ 长的动作区间，概率质量不变，密度便满足：

$$p_Y(f(z)) = \frac{p_Z(z)}{|f'(z)|}, \qquad f'(z) = 10\big(1 - \tanh^2(5z)\big).$$

原点附近的斜率为10，映射把样本拉开；函数接近两侧的 ± 2 时斜率变小，许多潜变量取值被压进较窄的动作区间。计算得 $p_Y(0) \approx 0.039894$、$p_Y(1.9) \approx 0.382614$，所以0并不是密度最高的动作，尽管它是标准正态潜变量密度最高的位置。这个结论来自映射后的密度，不能仅由潜变量密度最高处的位置来决定。若改用平方函数，同一正输出可由两个潜变量值产生，就须把两支的密度贡献相加。

后验坍缩是另一种情况：潜变量不再携带对动作有用的额外信息，推断分布接近先验，解码器主要依赖条件预测。当条件已能解释大部分动作时，利用 z 所带来的重建收益可能抵不过KL代价，训练就可能少用这条路径。固定条件改变 z，若动作几乎不变、各示范的推断分布又接近先验，说明这条路径可能没有被利用。若同一条件确实存在不同路线，应同时检查路线覆盖、重建、逐样本KL及先验输入下的动作；若动作本就几乎唯一，少用潜变量未必有害。只看训练重建也不够：推断器读过答案，重建好仍可能伴随先验生成差。

第6章Panda项目的动作是四维末端增量与夹爪，观测也与原ACT不同。把ACT架构用于该项目时，必须重新定义条件、动作头与训练数据，保持第6章的动作语义；原模型的14维关节目标不能直接作为四维环境动作。

1.8.8 练习题

题1.8-1 ACT 的潜变量编码器为何在训练时能读未来示范动作，而部署时不能读？重建项与 KL 项分别约束什么？

题1.8-2 一个教学潜变量模型中，$p(z=0)=p(z=1)=0.5$。对已观测动作 A，$p_\theta(A\mid z=0)=0.7$，$p_\theta(A\mid z=1)=0.3$；推断分布为 $q(z=0\mid A)=0.8$，另一项为0.2。计算边缘对数概率、ELBO 和真实后验，验证两种目标的差等于 $D_{\mathrm{KL}}(q\|p_\theta(z\mid A))$，并解释为什么训练要优化下界。

题1.8-3 使用1.8.4的全部初值，固定 $\epsilon=0.5$，先手算总损失对 b_μ, b_ℓ, w_z 的梯度及学习率0.1下的更新值。再沿用正文对全部七个参数同时更新的设置，复算重建损失、KL和总目标。为什么KL可以上升而总目标下降？若送入解码器前切断 z 的梯度，这三个参数在初始点各会收到什么梯度？初始损失值或后续损失下降能否单独揭示该错误？

题1.8-4 有人说“标准正态先验的均值是0，因此ACT部署令 $z=0$，就相当于执行最可能的动作；如果有多种路线，CVAE会自动随机选一条”。指出两处推理跳跃。结合 $f(z)=2\tanh(5z)$ 的例子，说明应怎样区分固定潜变量、先验采样与后验坍缩，并设计一组可以定位问题的检查。

1.8.9 参考答案

题1.8-1答案 训练样本来自已经记录的示范，当前条件 c_t 与动作标签 A_t 都已知。推断器读取标签，是在解释“这条示范适合怎样的潜变量”；原ACT的这个支路读关节状态和动作，图像由动作生成支路另行处理。部署时未来动作正是待决定的结果，不能再向推断器提供正确答案，所以移除左侧推断器，保留右侧观测处理与动作生成路径。原ACT实现使用 $z=0$；一般CVAE也可以选择先验采样，但这是另一项明确的运行规则。

重建项让给定 c, z 的预测接近示范，同时教解码器怎样利用 z、教推断器怎样安排这条示范的分布。KL项限制 $q_\phi(z\mid c, A)$ 与未读标签的先验之间的差别，避免只在部署不会使用的潜变量区域完成重建。在两网不共享参数、先验固定的本例中，重建项更新两网，KL只直接更新推断器。它不要求每个 z 都为0，也不单独保证固定零输入下的动作有效。

题1.8-2答案 以下使用自然对数，省略固定条件。先消去潜变量：

$$p_\theta(A)=0.5\times0.7+0.5\times0.3=0.5,\qquad \log p_\theta(A)\approx-0.693147.$$

ELBO中的重建对数期望与先验KL分别为：

$$\mathbb{E}_q[\log p_\theta(A\mid z)]=0.8\log0.7+0.2\log0.3\approx-0.526135,$$

$$D_{\mathrm{KL}}(q\|p)=0.8\log\frac{0.8}{0.5}+0.2\log\frac{0.2}{0.5}\approx0.192745.$$

所以 $\mathrm{ELBO}\approx-0.718879$。这里先对每种潜变量的对数概率加权，不能把它换成“先对概率加权，再取对数”。

真实后验由贝叶斯公式得到：

$$p_\theta(z=0\mid A)=\frac{0.5\times0.7}{0.5}=0.7,\qquad p_\theta(z=1\mid A)=0.3.$$

近似后验仍给 $(0.8, 0.2)$，因此：

$$D_{\mathrm{KL}}(q \| p_\theta(z \mid A)) = 0.8 \log \frac{0.8}{0.7} + 0.2 \log \frac{0.2}{0.3} \approx 0.025732.$$

它恰好等于 $-0.693147 - (-0.718879)$。若 q 改为真实后验 $(0.7, 0.3)$，差距变成0，下界与真实值相等。

两状态例可以直接求和；高维连续模型往往难以精确计算潜变量积分。ELBO把这个困难改成可由 q 采样估计的目标。固定生成模型时，改善推断会收紧下界；联合更新时，差距也会变化，因此下界单步上升不意味着真实似然每步都上升。

题1.8-3答案 初始点有 $\mu = 1, \ell = 0, \sigma = 1, z = 1.5, \hat{A} = 1.7$，残差为 $r = -0.3$。重建梯度先经过解码器到达 z：

$$\frac{\partial L_{\mathrm{rec}}}{\partial z} = r w_z = -0.24, \qquad \frac{\partial z}{\partial \mu} = 1, \qquad \frac{\partial z}{\partial \ell} = \tfrac{1}{2}\sigma\epsilon = 0.25.$$

因此它对 μ, ℓ 的梯度为 $-0.24, -0.06$。加权KL分别贡献 $0.1\mu = 0.1$ 与 $0.05(e^{\ell} - 1) = 0$，总梯度为 $-0.14, -0.06$。b_μ, b_ℓ 是乘标签 A 的权重，还要乘上 $A = 2$；w_z 的梯度则为 rz。

参数	重建梯度	加权KL梯度	总梯度	学习率0.1下的新值
b_μ	−0.48	0.20	−0.28	0.428
b_ℓ	−0.12	0	−0.12	0.012
w_z	−0.45	0	−0.45	0.845

题目要求全部七个参数同时更新，其余新值为 $a'_\mu = 0.214, a'_\ell = 0.006, w'_c = 0.530, b' = 0.030$。所有梯度都在更新前计算，不能边改参数边计算下一项。再前向：

$$\begin{aligned} \mu' &= 0.214 + 0.428 \times 2 = 1.07, \\ \ell' &= 0.006 + 0.012 \times 2 = 0.03, \\ z' &= 1.07 + e^{0.015} \times 0.5 \approx 1.577557, \\ \hat{A}' &= 0.530 + 0.845 z' + 0.030 \approx 1.893035. \end{aligned}$$

重建损失约0.005721，KL约0.572677，总损失约0.062988。重建项减少约0.039279，加权KL只增加约0.007268，总和因而下降。优化加权总目标，不要求各项同时下降。

若在解码器输入前切断 z，前向不变，重建到推断器的路径却消失。初始点对三个参数的总梯度变为 $(0.20, 0, -0.45)$，对应更新值为 $(0.380, 0, 0.845)$；b_μ 的方向反转，解码器的 w_z 仍能接到重建信号。这里 b_ℓ 的KL梯度为0，依赖于初始 $\ell = 0$，一般位置不必为0。

初始损失仍为0.095；沿错误路径对全部参数更新后，总损失也会降到约0.068184。因此初始损失有值、甚至后来下降，都无法单独发现这一问题。应检查分项梯度或用保持噪声不变的有限差分核对所需路径。

题1.8-4答案 第一处跳跃是把潜变量的均值或密度最高点，直接当成动作分布的密度最高点。非线性解码会重新分配概率；例如 $g(z)=z^2$ 下，$g(\mathbb{E}[Z])=0$ 而 $\mathbb{E}[g(Z)]=1$。第二处跳跃是把“训练里有随机潜变量”当成“部署正在随机选择”。固定条件、固定 $z=0$、关闭其他随机来源后，确定性解码器只输出同一个结果；先验采样需要显式执行 $z\sim p(z\mid c)$。

对 $f(z)=2\tanh(5z)$，$f(-1)\approx -1.9998, f(0)=0, f(1)\approx 1.9998$。若中央位置有障碍，固定0可以反复给出无效目标，但改变 z 明明能改变输出，所以该现象本身不是后验坍缩。这个奇函数在对称先验下的输出均值也为0，它说明均值可能不可行，不用于说明两个均值不相等。

也可以利用本例一一对应映射的密度变换核对第一处跳跃：$p_Y(f(z))=p_Z(z)/|f'(z)|$。本例 $p_Y(0)\approx 0.039894$，而 $p_Y(1.9)\approx 0.382614$；映射拉伸或压缩了区间，因此潜变量密度最大的0并未变成动作密度最大处。

可按以下顺序诊断：

1. 选条件相近但路线确有差异的保留示范，确认差异没有被目标指令或观测解释。
2. 冻结模型、固定条件，对比固定0、预定潜变量扫描和重复先验采样，记录路线覆盖、有效动作与碰撞；整块使用同一份 z。
3. 只在离线诊断时读标签，记录逐示范的 μ,ℓ、KL及后验重建，不能把这一路当作部署表现。
4. 后验重建好、先验生成差且推断分布明显偏离先验，优先查训练部署匹配；采样有效而固定0失败，优先查固定潜变量规则；改变相关范围的 z 几乎不影响动作且各推断分布接近先验，才支持潜变量未被利用的判断。动作本就近乎唯一时，少用 z 不必是缺陷。

1.9 扩散模型：从前向过程到训练与反向生成

1.9.1 扩散为什么把动作生成改成去噪

直接从条件预测唯一动作，可能平均不同路线。**扩散**把“生成完整**动作块**”改成“在不同噪声水平下，如何把候选往数据支持的方向修正”。训练从真实动作块出发加入已知噪声；部署从噪声出发，重复使用学到的修正规则。

噪声并不是传感器测量误差。它是人为构造的训练变量，帮助建立一条从简单分布到动作分布的生成过程。模型读取当前候选、噪声水平和条件，预测噪声、干净动作或其他等价参数化。它不是在部署时拿到真实动作，再把我们刚加的噪声去掉。

本节把决策时可用的图像、语言指令、本体状态及观测历史统记为条件 c。它不包含待预测的未来示范动作。x_0 是训练标签，也是部署想生成的对象；x_k 是人为加噪或反向计算中的候选。后面的小实验将 c 固定，以便单独观察同一条件下的动作多样性。

在一次动作块的内部生成中，通常保持当前条件 c 不变；执行动作前缀并取得新观测后，再为下一块更新条件。固定观测不意味着固定动作答案，当前候选仍会影响网络对不同模式的判断。

考虑同一条件下的左绕与右绕。随机起点提供不同生成路径的可能性；在每个中间位置，模型根据条件和候选选择修正方向。不同起点经过整个过程形成多种动作。噪声很大时两种路线难以区分，模型可能给出平均修正；噪声逐渐降低后，候选位置携带更多模式信息。关键是学习随候选和生成阶段变化的函数，不是只在最初给动作加随机抖动。

条件越充分，未解释的多模态可能越少。若左绕、右绕其实对应不同目标指令，正确加入指令后直接回归也可能工作。不能看到动作多样性就无条件认为必须扩散。扩散的收益应与数据覆盖、条件信息和推理预算一起评价。去噪扩散概率模型（Denoising Diffusion Probabilistic Model，DDPM）原论文、Diffusion Policy

这里生成的是 H×da 的动作块，不是未来图像。相同扩散原理可以用于不同对象；图像扩散模型的视觉效果，不直接说明动作扩散在关节、接触和实时闭环上有效。

1.9.2 已知加噪过程，怎样得到反向均值

去噪扩散概率模型（Denoising Diffusion Probabilistic Model，DDPM）先规定一条加噪链。把干净动作块展平为 $x_0 = A \in \mathbb{R}^d$，其中 $d = Hd_a$；H 是动作个数，d_a 是每个动作的维数。生成索引 k 表示候选被污染到哪一级，不是机器人控制时刻。每一级规定噪声方差 $0 < \beta_k < 1$，令 $\alpha_k = 1 - \beta_k$：

$$q(x_k \mid x_{k-1}) = \mathcal{N}(\sqrt{\alpha_k}x_{k-1}, \beta_k I). \tag{1.9-1}$$

前一个候选先缩小，再加入独立高斯噪声。定义累计保留系数 $\bar{\alpha}_k = \prod_{i=1}^{k} \alpha_i$，并约定干净端点 $\bar{\alpha}_0 = 1$，便可直接从干净动作得到任意一级：

$$x_k = \sqrt{\bar{\alpha}_k}x_0 + \sqrt{1 - \bar{\alpha}_k}\epsilon, \qquad \epsilon \sim \mathcal{N}(0, I). \tag{1.9-2}$$

为什么不用顺序运行前面所有级别？若上一级噪声方差是 $1 - \bar{\alpha}_{k-1}$，这一级缩放后贡献 $\alpha_k(1 - \bar{\alpha}_{k-1})$，再加上 β_k，总方差恰好是 $1 - \bar{\alpha}_k$。独立高斯噪声可以合并，所以训练只要读示范、抽 k 和 ϵ，就能制造输入和标签。网络读取 x_k, k, c，输出噪声估计，再与本次所抽标签比较；它不读取干净动作标签。同一候选可能有多种干净来源，噪声未必能逐样本唯一识别，后面将用条件均值算例说明。

1.9.2.1 先解决一个训练时能够精确计算的问题

先考虑 $k \geq 2$。训练时已知 x_0 和当前候选 x_k，想知道前一级 $u = x_{k-1}$ 可能是多少。此时 u 已经被加过噪，即使知道 x_0，它仍不是确定值。边界 $k = 1$ 则不同：前一级就是已知的 x_0，条件分布集中在一个点，不能套用下面含方差倒数的配方；它对应1.9.4中单列的最后恢复项。对于 $k \geq 2$，我们有两条线索：

$$q(u \mid x_0) = \mathcal{N}(\sqrt{\bar{\alpha}_{k-1}}x_0, (1 - \bar{\alpha}_{k-1})I),$$

$$q(x_k \mid u) = \mathcal{N}(\sqrt{\alpha_k}u, \beta_k I).$$

第一条来自干净标签，说明前一级大致在哪里；第二条说明哪个 u 与已经看到的 x_k 相容。后者作为 u 的函数称为**似然**：固定观测，考察不同候选对它的解释程度。贝叶斯公式将两份信息结合：

$$q(u \mid x_k, x_0) \propto q(x_k \mid u)\, q(u \mid x_0).$$

先在一个坐标上推导，向量情形逐坐标相同。忽略不含 u 的项，负对数密度为：

$$\frac{1}{2}\left[\frac{(x_k - \sqrt{\alpha_k}u)^2}{\beta_k} + \frac{(u - \sqrt{\bar{\alpha}_{k-1}}x_0)^2}{1 - \bar{\alpha}_{k-1}}\right] = \frac{1}{2}(Pu^2 - 2hu) + C,$$

$$P = \frac{\alpha_k}{\beta_k} + \frac{1}{1-\bar{\alpha}_{k-1}}, \qquad h = \frac{\sqrt{\alpha_k}x_k}{\beta_k} + \frac{\sqrt{\bar{\alpha}_{k-1}}x_0}{1-\bar{\alpha}_{k-1}}.$$

P 是合并后的精度，即方差的倒数；h 汇集两份线索带来的位置项。将二次式配成 $P(u-h/P)^2$，便读出均值 h/P 和方差 $1/P$。利用 $\alpha_k+\beta_k=1$、$\bar{\alpha}_k=\alpha_k\bar{\alpha}_{k-1}$ 化简：

$$\tilde{\beta}_k = \frac{1}{P} = \frac{\beta_k(1-\bar{\alpha}_{k-1})}{1-\bar{\alpha}_k}, \tag{1.9-3}$$

$$\tilde{\mu}_k(x_k, x_0) = \frac{\sqrt{\bar{\alpha}_{k-1}}\beta_k}{1-\bar{\alpha}_k}x_0 + \frac{\sqrt{\alpha_k}(1-\bar{\alpha}_{k-1})}{1-\bar{\alpha}_k}x_k. \tag{1.9-4}$$

这两个系数来自高斯信息融合，并非随意设置的去噪比例。换个角度看，似然在 u 空间的中心是 $x_k/\sqrt{\alpha_k}$、方差是 β_k/α_k；它与第一条线索的中心按各自精度加权。可靠程度越高，位置判断的影响越大。式(1.9-4)写回 x_0, x_k 后又包含缩放，所以不能把这两个表面系数直接当成和为1的普通平均权重。

DDPM原论文，式(6)—(7)

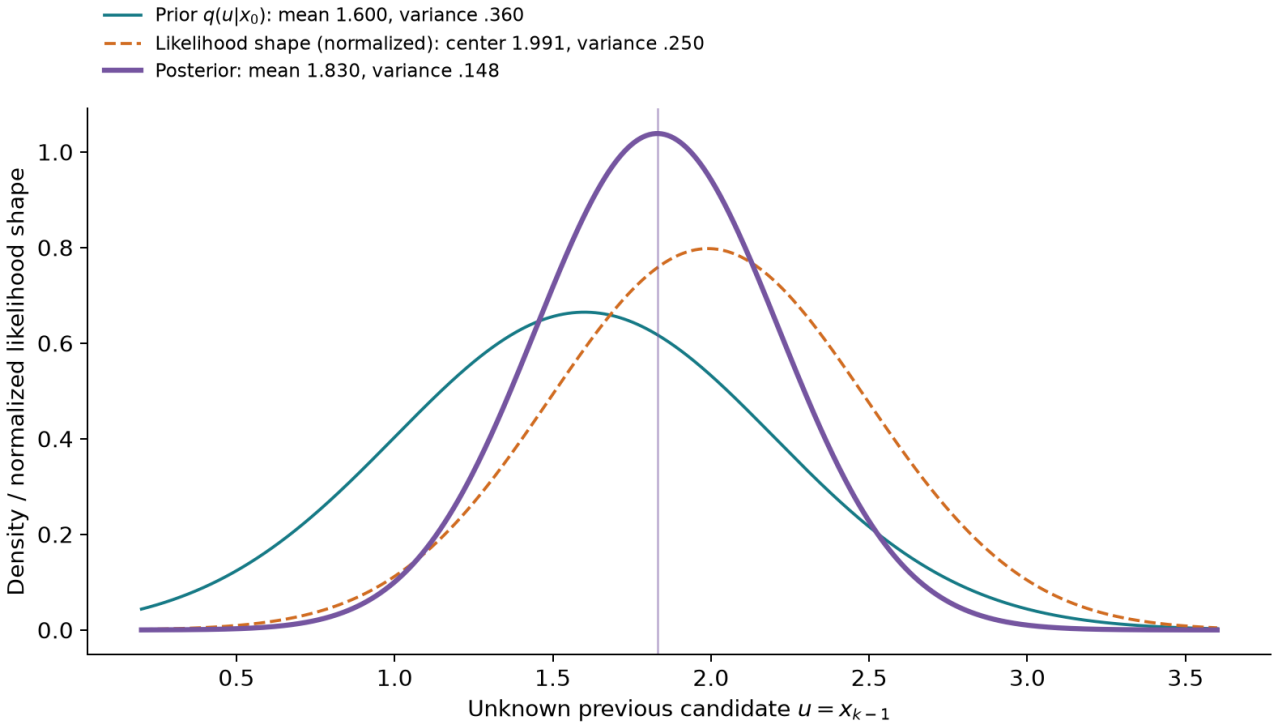


图1.9-1：干净标签给出前一级分布，当前候选提供另一条位置约束，两者相乘后得到更集中的后验。似然曲线按 u 归一化以比较形状；这个归一化只用于画图。

用同一组数贯穿后面的反向参数化与损失。设 $\bar{\alpha}_{k-1}=0.64, \alpha_k=0.8, \beta_k=0.2$，因此 $\bar{\alpha}_k=0.512$。干净动作坐标 $x_0=2$，本次采到 $\epsilon=0.5$，得到 $x_k\approx 1.780368$。由上式：

量	结果	所指
P	6.777778	两条信息合并后的精度

量	结果	所指
$\tilde{\beta}_k$	0.147541	已知 x_0, x_k 后，前一级仍有的方差
$\tilde{\mu}_k$	1.830466	前一级的后验均值

这里计算的是 $q(x_{k-1} \mid x_k, x_0)$，不是已经训练好的网络输出。因为训练知道 x_0，它可以作为可计算的目标来源。

1.9.2.2 部署没有干净动作，网络怎样接替它

部署时 x_0 正是待生成的动作，不能把它放进反向更新。网络先预测噪声 $\epsilon_\theta(x_k, k, c)$，按前向关系构造：

$$\hat{x}_0 = \frac{x_k - \sqrt{1-\bar{\alpha}_k}\epsilon_\theta(x_k, k, c)}{\sqrt{\bar{\alpha}_k}}. \tag{1.9-5}$$

再把 $\hat{x}_0$ 代入式(1.9-4)。整理 x_k 的系数：

$$\frac{\beta_k + \alpha_k(1-\bar{\alpha}_{k-1})}{\sqrt{\alpha_k}(1-\bar{\alpha}_k)} = \frac{1-\bar{\alpha}_k}{\sqrt{\alpha_k}(1-\bar{\alpha}_k)} = \frac{1}{\sqrt{\alpha_k}}.$$

噪声预测的系数则化为 $-\beta_k/(\sqrt{\alpha_k}\sqrt{1-\bar{\alpha}_k})$，因此：

$$\mu_\theta(x_k, k, c) = \frac{1}{\sqrt{\alpha_k}}\left(x_k - \frac{\beta_k}{\sqrt{1-\bar{\alpha}_k}}\epsilon_\theta(x_k, k, c)\right). \tag{1.9-6}$$

这才是常见反向均值公式的来历：用网络估计替换部署不可得的干净标签，再整理已知系数。若本次噪声预测正好为0.5，便得到前面的1.830466；若预测为0.3，模型均值变成约1.894485。网络误差已通过确定的系数转化成下一候选的偏差。

DDPM再按规定方差采样，例如 $x_{k-1} = \mu_\theta + \sigma_k\xi$，其中 $\xi \sim \mathcal{N}(0, I)$ 是这一反向步新抽的随机数。σ_k^2 可以按具体方法规定或学习，必须与所用采样过程一起说明。

部署时没有 x_0，为什么用训练时知道答案的后验均值作目标仍然有意义？把式(1.9-4)写成 $\tilde{\mu}_k = C_0 x_0 + C_x x_k$。给定同一个候选与条件，再对仍然可能的干净动作求平均：

$$\mathbb{E}[x_{k-1} \mid x_k, c] = C_0\mathbb{E}[x_0 \mid x_k, c] + C_x x_k.$$

噪声平方误差的最优输出是 $\mathbb{E}[\epsilon \mid x_k, k, c]$，代入式(1.9-5)恰好得到 $\mathbb{E}[x_0 \mid x_k, c]$。所以理想噪声预测器产生的模型均值，等于只读候选与条件时的真实反向均值；实际网络则近似它。训练并没有把某条部署不可得的干净答案直接交给采样器。

均值正确不代表整个分布已正确。给定干净动作后的后验是精确高斯；只给定 x_k, c 的真实反向分布，还要考虑哪些干净动作可能产生它，可能是多个成分的混合。全方差关系给出：

$$\mathrm{Var}(x_{k-1} \mid x_k, c) = \tilde{\beta}_k I + C_0^2\,\mathrm{Var}(x_0 \mid x_k, c).$$

向量情形的Var表示协方差矩阵。第一项是知道动作答案后仍有的前向噪声不确定性，第二项来自“答案到底属于哪条路线”。所以 $\tilde{\beta}_k$ 不是任意情况下真实反向分布的全部方差。学习高斯反向模型是一项建模选择；足够细的噪声步骤有助于局部近似，却不意味着任意大步、任意数据的真实反向条件都精确为高斯。

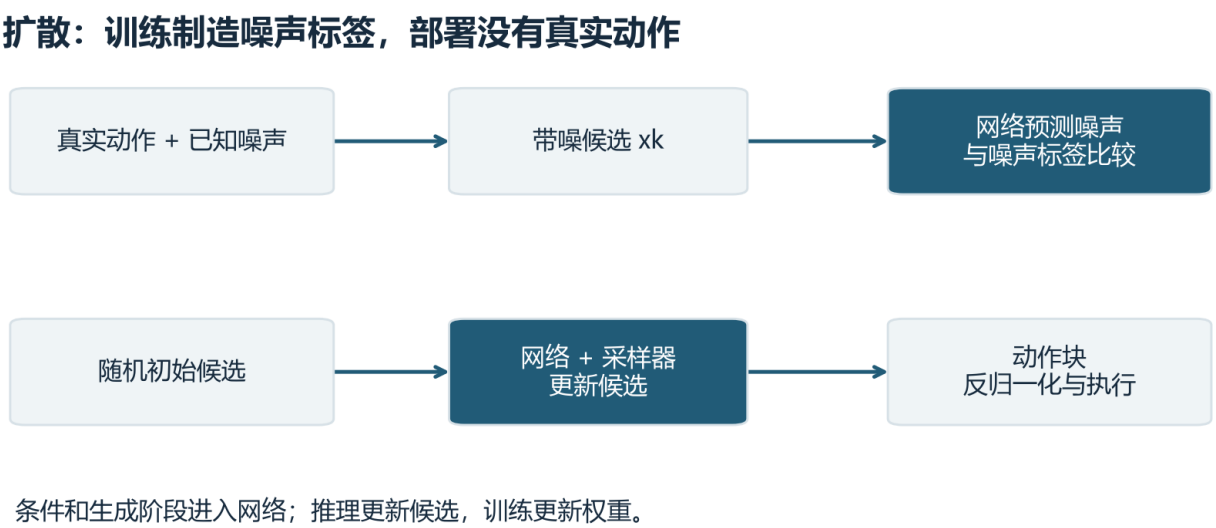


图1.9-2：训练用真实动作制造带噪输入与已知噪声标签，更新网络权重；部署用当前条件和随机候选，更新候选。训练的辅助噪声不来自机器人传感器。

1.9.3 噪声、干净动作、score和v：输出对象怎样转换

为看清几种输出的关系，暂把式(1.9-2)写成 $x_k = ax_0 + b\epsilon$，其中 $a = \sqrt{\bar{\alpha}_k}$、$b = \sqrt{1-\bar{\alpha}_k}$。噪声预测输出 ϵ_θ，干净动作预测输出 $\hat{x}_0$；另一种常见的**v参数化**定义 $v_{\text{noise}} = a\epsilon - bx_0$。由这两条线性关系解得：

$$\hat{x}_0 = \frac{x_k - b\epsilon_\theta}{a}, \qquad \hat{x}_0 = ax_k - bv_\theta, \qquad \hat{\epsilon} = bx_k + av_\theta.$$

v_{noise} 是特定的输出坐标，不等于后面Flow Matching的生成速度，也不等于强化学习价值。上述转换中的 $a > 0$；若某个噪声端点恰好令 $a = 0$，第一条含除法的转换便不能直接使用。

输出可以转换，为什么还要选择参数化？因为误差会随噪声水平重新缩放。若噪声预测误差为 $\delta\epsilon$，换成干净动作误差就是 $-(b/a)\delta\epsilon$。高噪声时 a 很小，微小噪声误差也可能放大为很大的动作误差。不同输出都采用不加权平方损失，通常并不对应同一个加权优化问题；输出坐标既影响数值尺度，也改变网络优先拟合哪些噪声级别。

分数函数（score function，简称 score）描述对数密度对当前位置的变化率：$s_k(y, c) = \nabla_y \log q_k(y \mid c)$。这里的score不是机器人任务评分。它回答的是“在当前噪声水平下，朝哪个方向移动，会提高局部对数密度”。q_k 是把所有可能的干净动作加噪后形成的整体分布：

$$q_k(y \mid c) = \int q(y \mid x_0) q(x_0 \mid c)\, dx_0.$$

对这个积分求导，再除以 $q_k(y \mid c)$，积分权重便变成看到 y 后的后验概率：

$$\begin{aligned}\nabla_y \log q_k(y \mid c) &= \int q(x_0 \mid y, c)\, \nabla_y \log q(y \mid x_0)\, dx_0 \\ &= \mathbb{E}\left[-\frac{y - a x_0}{b^2} \,\middle|\, x_k = y, c \right] \\ &= -\frac{\mathbb{E}[\epsilon \mid x_k = y, k, c]}{b}.\end{aligned} \tag{1.9-11}$$

第二行利用了前向高斯的对数密度；第三行利用 $y - a x_0 = b\epsilon$。关键在于先观察当前候选，再判断哪些干净动作更可能产生它。因此这里按 $q(x_0 \mid y, c)$ 平均，不能不加区别地把全部示范等权平均。

由此，噪声回归也能学到整体带噪分布的score：$s_\theta = -\epsilon_\theta / b$。定义score时对候选 y 求导；训练网络时则对参数 θ 求导。这是两个不同的求导对象。采样器如何使用这个位置场，还取决于具体更新方程和步长，不能把任意一次沿score移动都当成准确抽样。Score-based生成模型

为什么这里仍用平方损失，却能生成分布？直接动作回归的输入只有条件 c，于是可能把两条完整路线平均成一个动作。去噪网络还读取候选 x_k 与噪声级别 k，估计的是这个位置上的噪声条件均值。同一任务下，靠近左模式和右模式的候选可以得到不同修正。真正形成分布的是这个随位置变化的函数与完整生成过程的结合。第1.9.6节先算一个局部例子，第1.9.7节再跟踪同一候选连续变化，避免把局部回归和最终采样混为一谈。

1.9.4 噪声平方损失怎样连接到生成分布

前面已经有一个可计算的目标分布 $q(x_{k-1} \mid x_k, x_0)$。现在要让只读 x_k, k, c 的反向模型与它匹配。为什么逐级匹配能连接到最终动作分布的学习？先写出整条生成链：

$$p_\theta(x_{0:K} \mid c) = p(x_K) \prod_{k=1}^{K} p_\theta(x_{k-1} \mid x_k, c), \qquad q_{\mathrm{f}}(x_{1:K} \mid x_0) = \prod_{k=1}^{K} q(x_k \mid x_{k-1}).$$

$x_{0:K}$ 表示从 x_0 到 x_K 的全部变量。第一条链从简单先验向动作生成；第二条 q_{f} 从示范动作加噪，完全已知。要算 $p_\theta(x_0 \mid c)$，需要把中间变量全部积分掉。沿用第1.8节的变分思路，插入已知的前向链，由对数函数的Jensen不等式得到：

$$-\log p_\theta(x_0 \mid c) \le \mathbb{E}_{q_{\mathrm{f}}}\left[\log \frac{q_{\mathrm{f}}(x_{1:K} \mid x_0)}{p_\theta(x_{0:K} \mid c)} \right].$$

右边是负对数似然的上界，也就是对数似然变分下界的负值。接下来并非另加一组启发式损失，而是把同一前向联合分布倒过来分解：

$$q_{\mathrm{f}}(x_{1:K} \mid x_0) = q(x_K \mid x_0) \prod_{k=2}^{K} q(x_{k-1} \mid x_k, x_0).$$

把这条分解代入对数比，乘积变成求和。末端的比值形成先验KL，各相邻条件的比值形成局部KL；生成链还剩 $p_\theta(x_0 \mid x_1, c)$，负责最后一级恢复。再对示范动作取平均，得到：

$$\begin{aligned}\mathcal{J} = \mathbb{E}_q\Big[& D_{\mathrm{KL}}(q(x_K \mid x_0)\|p(x_K)) \\ & + \sum_{k=2}^{K} D_{\mathrm{KL}}(q(x_{k-1} \mid x_k, x_0)\|p_\theta(x_{k-1} \mid x_k, c)) \\ & - \log p_\theta(x_0 \mid x_1, c)\Big].\end{aligned} \tag{1.9-7}$$

这个期望覆盖示范动作和按前向链构造的带噪变量。第一项检查链末端是否接近规定的简单先验；固定前向规则时，它不更新去噪网络。中间各项匹配相邻反向条件，利用已知标签给出可计算的高斯目标。最后一项负责最靠近干净数据的恢复。扩散并没有要求训练时从随机噪声完整生成一次，才能得到梯度。

1.9.4.1 固定方差以后，局部匹配变成加权回归

令反向模型为 $\mathcal{N}(\mu_\theta, \sigma_k^2 I)$，其中 $\sigma_k^2 > 0$ 固定。高斯KL中与均值网络参数有关的部分为：

$$D_{\mathrm{KL}}(q\|p_\theta) = \frac{1}{2\sigma_k^2}\|\tilde{\mu}_k - \mu_\theta\|^2 + \text{不依赖均值参数的项}. \tag{1.9-8}$$

若还要学习方差，剩余项就不能一并当成常数丢掉。本节先只看固定方差的均值训练。训练时真实 ϵ 已知，将式(1.9-6)分别用真实噪声和预测噪声表示，两项中的 x_k 相消：

$$\tilde{\mu}_k - \mu_\theta = -\frac{\beta_k}{\sqrt{\alpha_k}\sqrt{1-\bar{\alpha}_k}}(\epsilon - \epsilon_\theta).$$

代入得到：

$$L_k = \underbrace{\frac{\beta_k^2}{2\sigma_k^2\alpha_k(1-\bar{\alpha}_k)}}_{w_k}\|\epsilon - \epsilon_\theta(x_k, k, c)\|^2 + \text{常数}. \tag{1.9-9}$$

噪声平方误差由此与局部概率匹配连接起来。w_k 来自噪声预测误差对反向均值的影响，以及模型方差允许多大偏差，不是随意附在损失前面的装饰。DDPM原论文，式(8)—(12)

沿用1.9.2的数值，取 $\sigma_k^2 = \tilde{\beta}_k \approx 0.147541$。真实噪声0.5、预测0.3，噪声平方误差为0.04；权重 $w_k \approx 0.347222$，所以带权误差为0.013889。直接用两个高斯均值算KL，结果同样为0.013889。对预测噪声的局部梯度为 $2w_k(0.3 - 0.5) \approx -0.138889$：它要求提高该输出，从而把偏高的反向均值拉回目标附近。网络参数再通过链式求导接收这个信号。

$k = 1$ 要单独处理：给定 x_0 后，所谓前一级就是 x_0 本身，$\tilde{\beta}_1 = 0$，不能把零方差直接塞进上面的非退化高斯KL。本节的简化训练用噪声回归覆盖这些级别；它是一项明确的目标选择，不是对除零公式作数值补丁。

1.9.4.2 为什么常见代码没有这些权重

常用简化目标直接均匀抽一个噪声级别，训练：

$$L_{\text{simple}} = \mathbb{E}_{x_0,k,\epsilon}\|\epsilon - \epsilon_\theta(\sqrt{\bar{\alpha}_k}x_0 + \sqrt{1-\bar{\alpha}_k}\epsilon, k, c)\|^2. \tag{1.9-10}$$

它保留了每一级的噪声回归任务，改变了各任务在训练中的相对比重。若每一级都有独立、无限容量的函数，正权重不改变该级平方误差的条件均值最优解；实际网络共享参数、容量和训练预算有限，不同权重会改变拟合重点。这里MSE指均方误差（Mean Squared Error）。因此，简化MSE、带权变分目标以及改变 k 的抽样频率，不能笼统说成同一个有限模型优化问题。

一次训练只抽 k，就能更新去噪网络及允许训练的条件模块；无需反向传播穿过完整采样链。只有在研究额外的采样后任务损失时，才需要重新设计那条计算与反馈路径，不能将其与这里的噪声监督混成一件事。

即使预测器达到平方损失最优，也是在估计 $\mathbb{E}[\epsilon \mid x_k, k, c]$。有歧义时，这个值不一定等于生成某一条训练样本时的隐藏噪声；因此标签MSE未必能降到0。它的作用是学出各候选位置上的反向规律，再与采样器一起生成分布，而不是逐样本唯一辨认原始噪声。

动作标准化同样是模型定义的一部分。若平移、转动和夹爪数值的尺度不同，直接加同一标准差的噪声意味着不同的物理扰动幅度。训练在明确的数值空间中进行，生成后再反标准化、解释为控制命令，并检查动作约束。

1.9.5 从原图到动作模型：哪条箭头需要学习

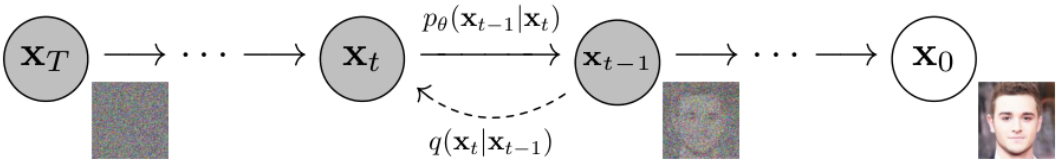


图1.9-3：Ho 等，DDPM 论文 Figure 2。实线表示从噪声向数据生成的反向模型，虚线表示预先规定的加噪过程。原论文。

原图的右端 x_0 是干净样本，左端 x_T 是接近标准高斯的噪声。加噪方向从右向左走；生成方向从左向右走。虚线的 $q(x_t \mid x_{t-1})$ 是人为规定的噪声规则，不需要先找真实世界的配对标签；实线的 $p_\theta(x_{t-1} \mid x_t)$ 才是由网络学习的反向条件分布。这里原图用 t 标生成索引，本书动作章节用 k 标它，避免与机器人控制时间混淆。

图中是一张图片的生成。迁移到动作时，每个 x_k 改为一整个带噪动作块，形状可以是 $H \times d_a$。反向网络还读取图像、语言和本体状态作为条件。它在一次去噪中修改整个候选块，并没有让机器人执行图中的每一步噪声状态。Hugging Face 扩散课程

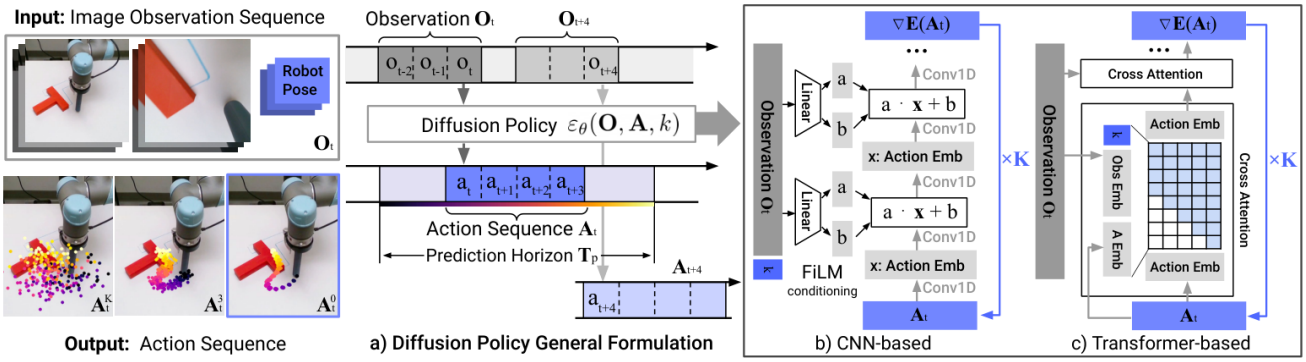


图1.9-4：Chi 等，Diffusion Policy 论文 Figure 3。左侧给出观测历史与动作块的关系，右侧分别展示卷积和 Transformer 去噪网络的条件连接。原论文。

图左侧 O_t 表示本次决策可用的观测历史，A_t 表示待生成的动作块。图中 T_p 标预测长度；本书另用 T_o 记观测历史长度、T_a 记本次执行长度。图里执行动作只是预测块的一段，之后获得新观测，再预测下一块。这正是动作块能够同时保留时间协调性、又进行闭环修正的原因。

图右侧的 A_t^k 是当前噪声级别的候选，ϵ_θ 是网络预测的噪声，k 告诉它现在应处理什么噪声程度。FiLM（Feature-wise Linear Modulation，逐特征线性调制）把观测条件变成特征缩放和偏移；Cross Attention（交叉注意力）则让候选动作查询观测特征。两者改变条件怎样进入去噪计算，不改变“标签噪声来自我们自己采样”的事实。图中的历史长度、动作块长度和去噪步数是三个独立概念。

图右侧CNN指卷积神经网络（Convolutional Neural Network），Conv1D表示沿动作时间序列做一维卷积；Emb是嵌入（Embedding）的缩写，即送进后续网络的特征表示。FiLM分支中的 $a \cdot x + b$ 表示用条件生成的缩放与偏移调整动作特征，这里的 a 不是前向噪声的保留系数。蓝色“×K”表示重复去噪计算。顶部 $\nabla E(A_t)$ 用能量梯度的视角表示去噪修正，具体实现可以直接输出噪声预测，不要求另建一个标量能量网络。原图中的英文Input、Output分别指输入与输出。

生成整个动作块还涉及时间依赖。例如两步示范只有 $(-2,-2)$ 与 $(2,2)$ 两种，若分别独立拟合每一步的双峰，就会把 $(-2,2)$、$(2,-2)$ 也组合出来；两步独立各选一侧时，这两类不一致组合占一半。联合去噪网络同时读取整块候选与条件，才有机会学会这种跨步联系。人为加入的高斯噪声可以逐坐标独立，但反向预测不能据此假定各动作坐标独立。联合建模提供表达这种联系的条件，是否学会仍要检查生成动作块。

1.9.6 双峰动作为什么可以用局部均值来生成

把第1.2节的两条路线带入去噪模型，能看清“平方损失仍估计均值”为什么没有自相矛盾。条件固定，干净动作 $x_0 = -2$ 或 $+2$ 各一半，某个噪声级别取 $x_k = 0.8x_0 + 0.6\epsilon$。因此，左路线的带噪分布为 $\mathcal{N}(-1.6, 0.36)$，右路线为 $\mathcal{N}(1.6, 0.36)$。

当前候选若等于 y，贝叶斯公式给出右路线的后验概率

$$P(x_0 = 2 \mid x_k = y) = \frac{1}{1 + \exp(-3.2y/0.36)}.$$

右边是**logistic函数**，也常写成 $\sigma(z) = 1/(1+e^{-z})$；这里不使用此前表示标准差的 σ_k。这个结果来自比较两个高斯的似然：对数比为 $[(y+1.6)^2 - (y-1.6)^2]/(2 \times 0.36)$。候选是0时，两条路线概率

各半，干净动作后验的均值为0，**众数**即概率最大的取值则同时有−2、+2；均值并不是从两条路线中抽到一条。候选是1时，右路线概率约为0.999862，干净均值约为1.999448，众数为+2。不同候选产生不同后验，去噪网络便不必给出同一个修正。

平方损失的最优噪声预测为 $\mathbb{E}[\epsilon \mid x_k = y]$。利用 $\epsilon = (y - 0.8x_0)/0.6$，得到

$$\mathbb{E}[\epsilon \mid x_k = y] = \frac{y - 0.8\mathbb{E}[x_0 \mid x_k = y]}{0.6}.$$

在 $y = 0$ 处，这个预测为0；在 $y = 1$ 处，约为 -0.99926。这是每个候选位置上的条件均值，不是把全部完整路线平均成一个固定输出。不同随机候选经过随噪声级别变化的修正过程，才能形成不同的最终动作。网络是否学全两个模式、有限步采样是否准确，仍需用生成样本检验；本例只是把局部回归目标与最终分布之间的关系算清。

1.9.7 从一次均值估计到完整生成：沿八次更新走完

1.9.7.1 减少网络调用，需要换成对应的采样公式

DDPM通常按相邻噪声级别更新。机器人希望尽快生成动作块，可以只访问部分训练级别，但跨过若干级时必须重新计算更新系数。**去噪扩散隐式模型**（Denoising Diffusion Implicit Model，DDIM）提供了一类这样的采样规则。本节使用其中不在中途追加随机数的确定性版本；随机性来自最初候选，不意味着所有输入得到同一动作。DDIM原论文，第3—4节

设当前级别为 k，下一个较干净级别为 $s < k$。先预测噪声，并估计干净动作：

$$\hat{\epsilon} = \epsilon_\theta(x_k, k, c), \qquad \hat{x}_0 = \frac{x_k - \sqrt{1 - \bar{\alpha}_k}\hat{\epsilon}}{\sqrt{\bar{\alpha}_k}}.$$

再按下一级的信号与噪声比例，重新组合这两部分：

$$x_s = \sqrt{\bar{\alpha}_s}\hat{x}_0 + \sqrt{1 - \bar{\alpha}_s}\hat{\epsilon}. \tag{1.9-12}$$

为什么可以复用前面训练的网络？本节的简化噪声监督每次只使用某一级的边缘加噪关系，即式(1.9-2)，不需要实际经历相邻加噪链。DDIM构造了保留这些边缘关系、改变级别间连接方式的一族过程，因此仍可使用相同的噪声预测任务。它的确定性更新暂时按当前估计的“干净成分 + 残余噪声”表示，移到下一个噪声水平；若两部分都是真值，重新组合便符合那一级的前向边缘形式。部署使用的是估计值，不能因此推断有限步生成分布精确。到达新位置后，网络需要重新判断，而非一直固定第一次的噪声预测。最后 $s = 0$，$\bar{\alpha}_0 = 1$，此时 $x_s = \hat{x}_0$。

DDIM还有可加入随机性的版本；式(1.9-12)对应其随机强度参数为0的情形。它与“从DDPM公式中直接删掉随机项”不同：后者留下的是DDPM条件均值，式(1.9-12)还按所跨的两个级别改变了系数。即使局部条件均值完全准确，有限大步也不自动保证准确搬运整个分布，后面要用分布结果检验。

```
D1 选取递减噪声级别 K=k0>k1>…>kS=0，初始化候选 x
D2 对当前级别 k：用网络预测噪声 ε̂
D3 用当前 x、k、ε̂ 计算干净动作估计 x̂0
D4 用下一级 s 的系数重组 x←√ᾱs x̂0+√(1−ᾱs) ε̂
D5 重复D2—D4，完成后返回动作块；此时才交给执行模块
```

配套的 practice/diffusion_core.py 按D1—D5注释实现。训练可以使用1000个噪声级别，采样只访问其中的16、64或256个；训练级别数与部署网络调用次数是两个量。本节的单坐标例子没有物理执行，动作块策略则在整块生成后执行一个前缀，再读取新观测。内部去噪候选不会逐个发送到机械臂。

具体比较采用 $k_j = \mathrm{round}(1000(1 - j/S))$，$j = 0, \dots, S$，其中 S 是采样步数，恰好半整数时取最近偶数。这样每个候选都在整数训练级别上调用网络；例如64步中间会访问 $k = 500$。八步表则正好每次跨125级。

1.9.7.2 完整八步：候选为什么先接近中间，后来转向一侧

上一节用两个精确取值说明后验平均。为了同时观察最终两个峰的宽度，这里把数据改成两个窄高斯：

$$q(x_0 \mid c) = \tfrac{1}{2}\mathcal{N}(-2, 0.15^2) + \tfrac{1}{2}\mathcal{N}(2, 0.15^2).$$

条件 c 固定，正负仅表示两种动作模式；这些数没有指定为米或关节弧度。与两个精确点相比，模式内部现在也有连续变化，后验必须按这个新分布重新计算。

先使用已知分布计算出来的解析去噪器，排除网络尚未学好的影响。令 $a = \sqrt{\bar{\alpha}_k}$、$b = \sqrt{1 - \bar{\alpha}_k}$，模式均值 $m_- = -2, m_+ = 2$。给定模式后，x_k 的方差为 $v_k = a^2(0.15)^2 + b^2$；看见候选 y 后，该模式的权重为：

$$w_j(y) = \frac{\exp[-(y - am_j)^2/(2v_k)]}{\sum_{\ell \in \{-,+\}} \exp[-(y - am_\ell)^2/(2v_k)]}.$$

每个高斯的score是 $-(y - am_j)/v_k$，整体score按这些后验权重加权。因此最优噪声条件均值可以直接算为：

$$\epsilon_*(y, k) = b \sum_{j \in \{-,+\}} w_j(y) \frac{y - am_j}{v_k}. \tag{1.9-13}$$

这给出一个可核对的参考函数，不需要训练网络，也没有在采样时偷偷给出某条轨迹的干净答案。它利用的是已知的整个数据分布；实际机器人数据通常没有这个解析条件。

取 $\bar{\alpha}_k = \cos^2(\arccos(0.01)k/1000)$，使末端信号幅度为0.01。候选从 $x_{1000} = 0.5$ 出发，每次跨125级，按式(1.9-12)完成八次更新：

当前级别→下一级	当前候选 x_k	噪声预测 $\hat{\epsilon}$	本次干净估计 $\hat{x}_0$	更新后候选 x_s
1000→875	0.500000	0.499824	0.020114	0.493444
875→750	0.493444	0.417697	0.414909	0.546341
750→625	0.546341	0.201506	0.925964	0.686083
625→500	0.686083	−0.258265	1.604836	0.958745
500→375	0.958745	−0.624570	1.967496	1.294958
375→250	1.294958	−0.640782	1.978241	1.585807
250→125	1.585807	−0.612250	1.966506	1.810505
125→0	1.810505	−0.495976	1.943529	1.943529

表中每行先读前一行的最后一列，再重新预测。第一行的干净估计只有0.020114：高噪声下，候选几乎不能辨认模式，条件均值接近两峰之间的低密度区域。对真实绕障任务，这类中间动作是否不安全，还取决于它怎样对应运动和环境几何；当前标量例子只验证了分布上的平均现象。DDIM保留当前残余噪声，再到较低噪声级重新判断。候选逐渐带上模式信息，后验越来越偏向右侧，干净估计才接近2。

噪声预测不是实际位移方向。表中它由正变负，候选却要结合两个系数才能决定如何变化；“每步减去一个噪声”不足以解释这条轨迹。最终值也不必恰好等于2，因为本例的右峰有宽度，而且八步仍有离散近似误差。

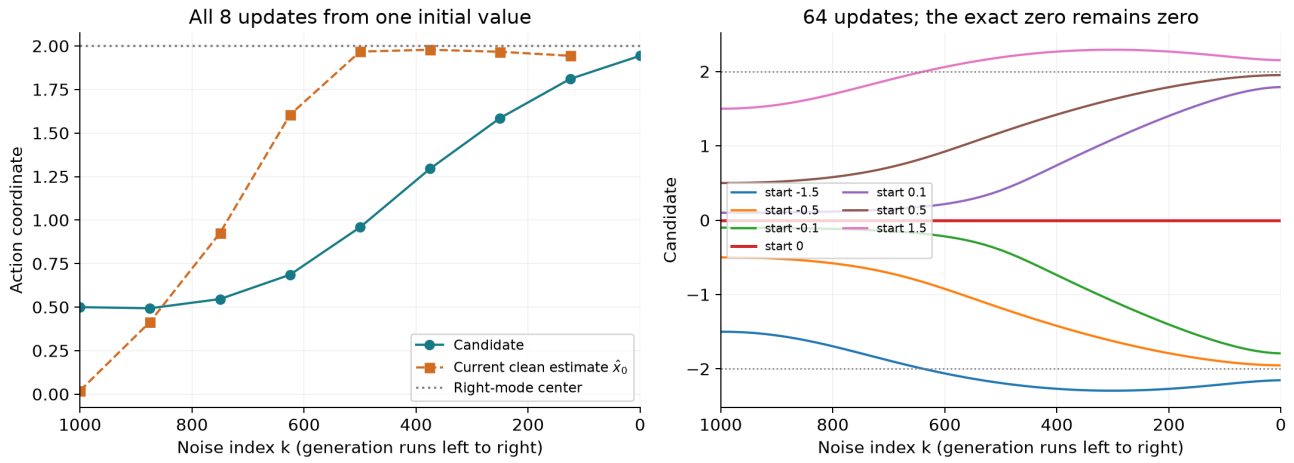


图1.9-5：左图对应表中全部八步，区分候选与当步的干净估计；右图用64步跟踪不同初值。横轴是递减的生成级别，全部计算发生在一个动作生成过程中。

初值为−0.5时，解析例子给出镜像轨迹；不同的正初值也不必到同一点。还有一个重要反例：严格等于0的候选，在这个对称分布、确定性采样和精确计算下始终停在0。连续高斯恰好抽到0的概率为0，但把所有候选人为初始化为0，会让这一实现全部停在中间。多步生成依赖起点分布与更新规则共同作用，不能只看到公式里有去噪网络就认定具备多样性。

1.9.7.3 看一群候选，才知道是否生成了正确分布

将4096个标准高斯的等概率分位点作为共同起点，可以稳定比较不同采样步数。具体取 $u_i = (i + 0.5)/4096$，$i = 0, \ldots, 4095$，再令初值 $z_i = \Phi^{-1}(u_i)$，其中 Φ 是标准高斯的累积分布函数；这里采用确定性分位点，不是4096次随机抽样。末端真实加噪分布与标准高斯非常接近，但在 $\bar{\alpha}_{1000} = 10^{-4}$ 时仍不完全相同。这个初始化近似在各组中保持一致。

把生成结果从小到大排序，与真实双高斯的相同分位点逐个比较。若 F_0 是目标双高斯的累积分布函数，平均绝对间隔为 $E_{\mathrm{q}} = \frac{1}{4096} \sum_i |x_{(i)} - F_0^{-1}(u_i)|$，其中 $x_{(i)}$ 为排序后的第 i 个生成值。它同时反映模式位置、比例和宽度的偏差。单看有无两个峰，无法区分这些问题。

解析去噪器的采样步数	左侧占比	左侧均值 / 标准差	右侧均值 / 标准差	E_{q}
16	0.5000	−1.9782 / 0.1256	1.9782 / 0.1256	0.02565
64	0.5000	−1.9937 / 0.1433	1.9937 / 0.1433	0.00709
256	0.5000	−1.9983 / 0.1482	1.9983 / 0.1482	0.00185

目标的两个成分均值为±2、标准差为0.15。表中“左侧”指生成值小于0，组内标准差按总体标准差计算。在本组实现中，更多步数使结果接近目标；左右各半还受对称初值与解析函数共同保证。有限步数仍会把峰压窄，说明“每一级的条件均值准确”和“有限步采样准确”是不同问题。这里的结果支持这组步长设置，不能推出任意模型增加步数都必然改善。

如果只在最高噪声级调用一次解析去噪器，直接返回 $\hat{x}_0$，结果标准差仅0.0402，全部落在 $|x| \le 1$ 内，$E_{\mathrm{q}} \approx 1.968$。而64步最终形成两个接近±2的峰。这个对照直接回答了建模疑问：局部平方误差最优的干净均值，不是一次抽取完整动作的结果；它要参与逐级生成，才能把不同起点送向不同模式。

1.9.8 把解析规律换成一个真正训练的去噪网络

1.9.8.1 网络输入、标签和参数怎样更新

解析例子让我们知道“正确函数”长什么样，但没有说明网络能否学会。现在从同一双高斯分布不断抽取新的示范标量，训练一个**多层感知机**（Multilayer Perceptron，MLP）。它读取当前候选、噪声级别的正余弦特征和条件，输出一个噪声预测。这里条件始终为0，只检验一个固定条件下的双峰建模；它没有学习区分语言任务或图像场景。

正余弦特征把归一化级别 $r = k/1000$ 写成 $\sin(\pi r), \cos(\pi r), \sin(2\pi r), \cos(2\pi r)$，供网络判断当前噪声程度。候选1维、时间特征4维、条件1维，共6维输入，依次经过两个64维隐层，输出1维噪声；网络共4673个参数。

每个隐层后使用SiLU（Sigmoid Linear Unit，sigmoid线性单元）激活，函数为 $f(u) = u/(1 + e^{-u})$。非线性使预测可以随候选位置弯曲变化；若只叠加线性层，最后仍是一个线性映射，无法表达本例双峰后验随位置切换的修正规律。

```
P1 抽取干净示范 x0 与可用条件 c
P2 抽 k 和标准高斯噪声 ε，计算 xk=√ᾱk x0+√(1−ᾱk) ε
P3 读取 xk、k、c，输出预测噪声 εθ
P4 比较 εθ 与本次已知标签 ε，计算批平均平方损失并反向传播
P5 优化器更新 θ；下一个批次重新抽示范、级别和噪声
```

下面摘出实际训练循环的核心。batch 实现P1、P2，level 是归一化噪声级别，model 内部把它转换为上述特征。完整版本见 practice/train_denoiser.py。

```
clean, condition, index, level, candidate, noise = batch(generator, 256)  # P1、P2
prediction = model(candidate, level, condition)                            # P3
loss = (prediction - noise).square().mean()                                # P4：标签是P2抽出的noise
optimizer.zero_grad()
loss.backward()                                                            # P4：误差沿网络传给参数
optimizer.step()                                                           # P5：更新网络参数
```

以种子7的第一次训练为例，批内第一个示范为1.939429，级别 $k = 336$，抽得噪声1.921532，构造的带噪输入为2.640935。网络尚未训练，输出−0.027333。批大小256，因而本行对自身预测的损失导数为：

$$\frac{\partial L}{\partial \hat{\epsilon}_i} = \frac{2}{256}(\hat{\epsilon}_i - \epsilon_i) \approx -0.015226.$$

负号表示此样本要求提高输出。该信号沿当前样本经过的线性层和激活传到参数，再与同批其他样本的贡献相加。第一次更新后，全部参数变化的二范数约0.067887；这表明权重实际发生了更新，不能据此断言这一个样本或所有输入的输出必然同步提高。参数共享、批内信号和优化器共同决定变化。

整个训练循环没有调用完整DDIM，也没有用解析去噪器当训练标签。解析函数只用于训练之外的评估。每次训练从干净示范制造一个随机级别的输入，部署时才把训练好的网络反复用于多个级别：前者更新权重，后者固定权重、更新候选。

1.9.8.2 标签损失不为零，意味着没有学好吗

三组实验分别用种子7、11、23初始化并产生训练数据，均采用批大小256、学习率 10^{-3} 的Adam优化器，固定训练8000次。另用种子90901生成同一组2048个验证输入，不参加梯度更新，也不用于挑选中间最优版本；下表均评估最后一次更新后的网络。

训练种子	训练前噪声标签MSE	训练后噪声标签MSE	对解析条件均值的MSE
7	0.95497	0.27961	0.00618
11	0.99110	0.28114	0.00746

训练种子	训练前噪声标签MSE	训练后噪声标签MSE	对解析条件均值的MSE
23	0.99734	0.28464	0.01244

在同一验证集上，解析最优条件均值对噪声标签的MSE仍为0.26901。原因不是解析函数出错，而是给定 x_k, k, c 后，原始噪声通常还有不确定性。平方损失对某个函数 f 的总体期望可以分解为：

$$\mathbb{E}[(\epsilon - f(x_k,k,c))^2] = \mathbb{E}[\mathrm{Var}(\epsilon \mid x_k,k,c)] + \mathbb{E}[(\mathbb{E}[\epsilon \mid x_k,k,c] - f(x_k,k,c))^2]. \quad (1.9\text{-}14)$$

第一项是输入不足以唯一识别标签而留下的误差，第二项才是函数偏离条件均值的误差。推导只需在平方中加减条件均值；交叉项的条件期望为0。有限验证集中的交叉项不必恰好为0，所以表中“网络标签MSE−解析标签MSE”不必严格等于最后一列。这个区别也提醒我们：训练损失的绝对大小要放回目标和条件中解释。

1.9.8.3 能生成双峰，仍可能学错比例与宽度

部署评估沿用前面的4096个起点和相同采样器。下表比较三个网络的模式比例及分位点误差。

种子	采样步数	左侧占比	E_{q}
7	16	0.3535	0.58050
7	64	0.4456	0.21885
7	256	0.4683	0.13166
11	16	0.4797	0.15149
11	64	0.4958	0.06639
11	256	0.5007	0.04819
23	16	0.5552	0.31580
23	64	0.5298	0.17892
23	256	0.5217	0.14192

三组都形成双峰，比例却不完全是各半。以256步为例，种子7的左、右标准差约0.1912、0.1640，种子23的右侧均值与标准差约2.1234、0.2146，均偏离目标。候选会沿学到的场持续移动，局部预测误差也可能改变最后落在哪一侧。因此，只看一个噪声标签损失，或者只数是否出现两个峰，都不足以检验分布。

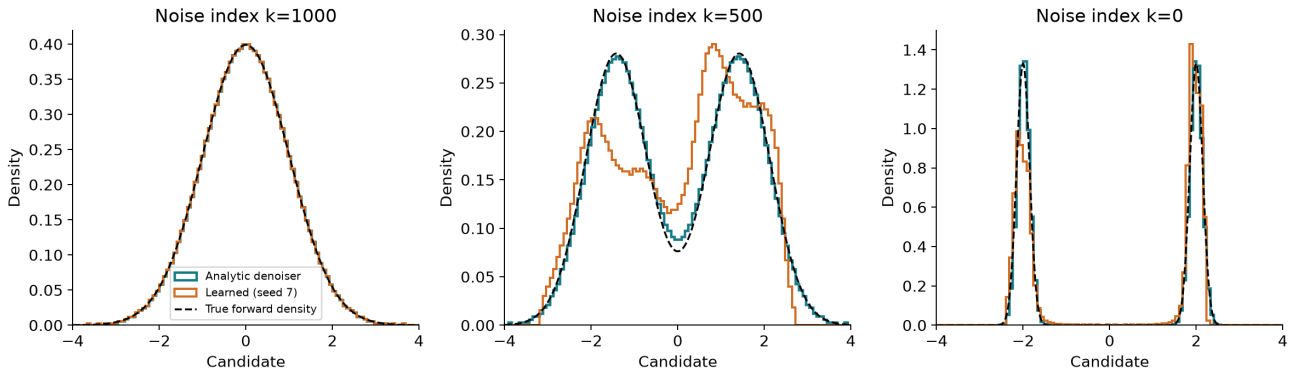


图1.9-6：共同初值在不同噪声级别的分布，比较解析去噪器与种子7网络的64步采样。虚线是该噪声级别的真实加噪密度。最后一幅可以看到模式比例与峰宽误差。

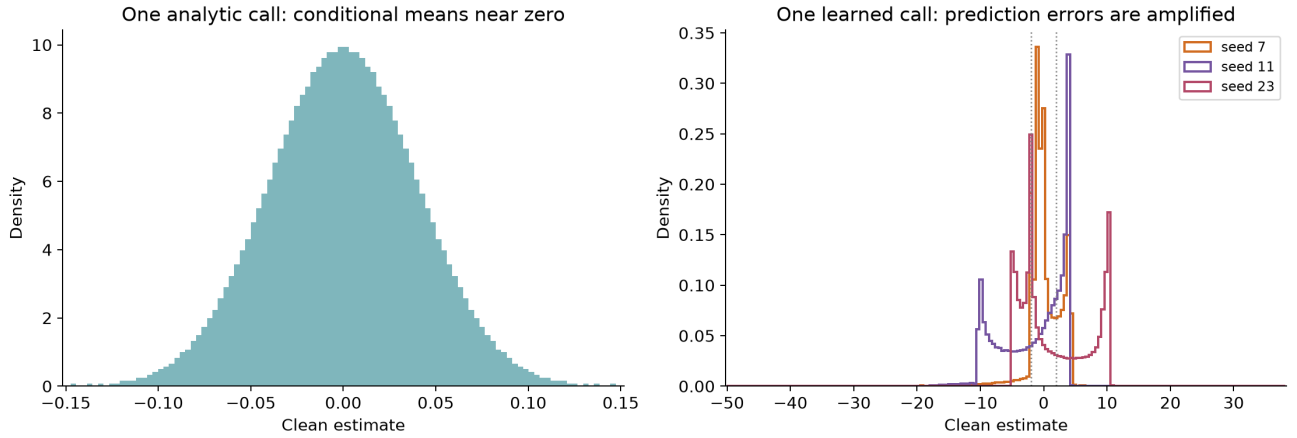


图1.9-7：左图是解析条件均值的一次还原，集中在0附近；右图是三个学习网络的一次还原，保留其全部取值范围。两图横纵尺度不同。学习网络的一次还原还可能被高噪声级的误差放大，不能笼统说成“都会集中在0”。

这个误差放大在本例尤其明显：末端 $a = 0.01$，因此噪声预测的小误差在换算干净动作时约放大100倍。三个网络的一次还原标准差约为3.35、5.25、5.46，并没有像解析条件均值那样全部挤到中间。解析条件均值的一次恢复主要落在两峰中间，学习网络还叠加了预测误差放大的问题。两类失败需要结合解析对照来判断。

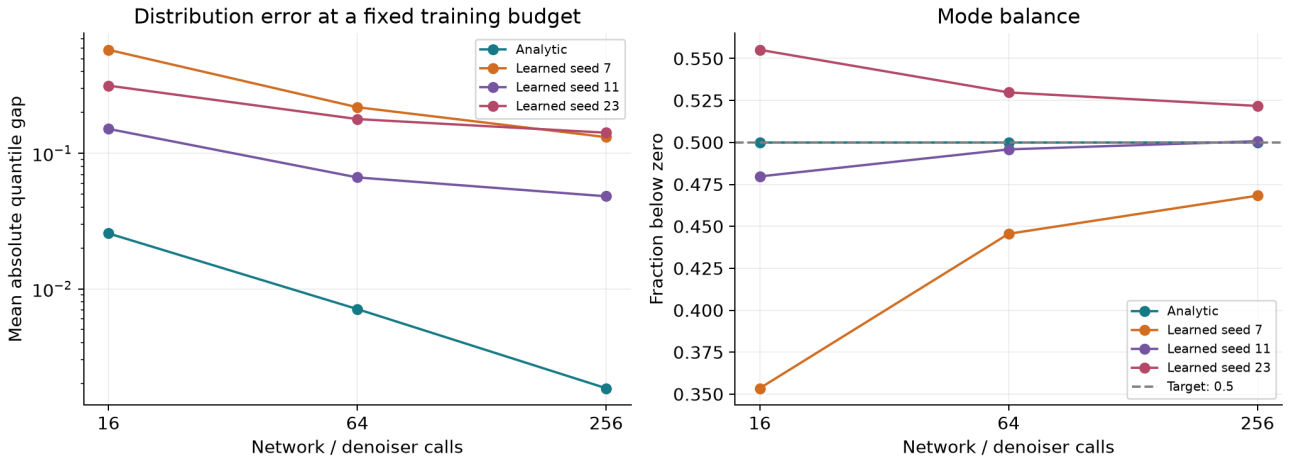


图1.9-8：解析函数和三个学习网络使用完全相同的起点。左图比较分位点平均间隔，右图比较左右模式比例。更多计算有时能减轻步长过大的影响，但不能替代学习准确的去噪函数。

在这组实验中，从16步增加到256步都降低了分布误差；即便256步，学习网络与解析结果仍有明显差距。要区分问题，可以先固定网络增加采样精度，再固定采样器改进训练，并检查各噪声级别的预测。实际动作策略还要检查任务成功率、约束、动作连续性和闭环反馈；标量分布拟合没有测量这些能力。

这套小实验的学习价值，在于把四件事拆开观察：平方损失学的是哪个条件均值；已知的反向系数怎样使用它；有限步数怎样带来近似；有限网络又怎样改变生成分布。把对象换成动作块后，这些关系仍然存在，但条件编码、跨时间耦合和机器人执行需要另外验证。复现命令、预期输出与完整记录见本单元README。

1.9.9 练习题

题1.9-1 高斯后验、噪声回归与梯度。 取 $\bar{\alpha}_{k-1}=0.64, \alpha_k=0.8, \beta_k=0.2, x_0=2, \epsilon=0.5$。从两个高斯因子的乘积推导前一级的后验均值、方差，核对正文数值。若模型噪声预测为0.3，反向方差固定为后验方差，计算两个高斯的KL、等价噪声MSE权重及对预测噪声的梯度，并解释梯度怎样修正反向均值。为什么此处的后验方差不等于部署时只给定 x_k, c 的完整不确定性？

题1.9-2 平方损失到底平均了什么？ 固定条件，干净动作 $x_0=\pm 2$ 各半，加噪为 $y=0.8x_0+0.6\epsilon$。推导 $P(x_0=2 \mid y)$ 和最优噪声条件均值，计算 $y=0$、$y=1$ 时的结果；比较后验均值与后验众数。为什么这个函数不同于直接把动作回归成0？它能否保证一次干净还原就是某条可行路线？

题1.9-3 手写确定性DDIM的一步。 使用式(1.9-12)，输入当前候选 $x_{1000}=0.5$、解析噪声预测0.4998238543，以及本节的余弦累计系数。写一个只依赖当前候选、当前/下一噪声系数和预测噪声的函数，复算1000→875的更新。它为何不等于直接返回 $\hat{x}_0$，也不等于任意跨级时照搬DDPM的相邻均值？若所有候选都初始化为0，在本节对称解析例子中会怎样？

题1.9-4 损失下降以后，网络究竟学了什么？ 写出批平均噪声MSE对某一行预测的导数，并用本节第一次更新的数据复算。说明已知噪声标签来自哪里，为什么训练不需要展开完整采样链。再用条件期望分解解释：解析最优预测的验证标签MSE为什么仍是0.26901？网络标签MSE与它的差，能否在有限验证集上直接当成对解析条件均值的MSE？

题1.9-5 怎样区分两种生成误差？ 解析去噪器的分位点误差随16、64、256步从0.02565降到0.00185，种子7网络对应误差从0.58050降到0.13166。解释这些对照支持什么、尚不能证明什么；给出区分采样误差与网络误差的实验安排。若两个模型的标签MSE接近，为什么还要比较模式比例、峰宽和动作块的跨时间联系？

题1.9-6 从标量分布走向机器人动作块。 示范动作块只含 $(-2,-2)$ 和 $(2,2)$，各半。独立拟合两个时间位置的双峰再分别采样，会产生哪些组合、概率各是多少？联合去噪为什么有机会避免错误组合，却不能仅凭“输入是一整块”就保证成功？最后说明动作块长度、去噪迭代数和真实执行前缀的关系，设计一个能检查跨时间联系的指标。

1.9.10 参考答案

题1.9-1答案。 先算 $\bar{\alpha}_k = 0.512$，$x_k = \sqrt{0.512}\times 2 + \sqrt{0.488}\times 0.5 \approx 1.780368$。令 $u = x_{k-1}$，两份信息为 $q(u \mid x_0) = \mathcal{N}(1.6, 0.36)$ 与 $q(x_k \mid u) = \mathcal{N}(\sqrt{0.8}u, 0.2)$。展开它们负对数中的平方项：

$$\tfrac{1}{2}\left[\frac{(x_k - \sqrt{0.8}u)^2}{0.2} + \frac{(u-1.6)^2}{0.36}\right] = \tfrac{1}{2}(Pu^2 - 2hu) + C,$$

$$P = 0.8/0.2 + 1/0.36 = 6.777778, \qquad h = \sqrt{0.8}x_k/0.2 + 1.6/0.36 = 12.406494.$$

配成 $\frac{P}{2}(u - h/P)^2$，得到后验均值 $h/P = 1.830466$、方差 $1/P = 0.147541$。这一步用的是高斯密度相乘后的信息融合，而非任意两个动作数值的平均。

预测噪声0.3代入式(1.9-6)，模型均值为1.894485。固定两者方差相同，KL只剩均值差：

$$D_{\mathrm{KL}} = \frac{(1.830466 - 1.894485)^2}{2(0.147541)} \approx 0.013889.$$

由未截断数值计算，$w_k = 0.347222$，带权噪声误差为 $0.347222(0.5-0.3)^2 = 0.013889$，与KL相符。预测噪声的梯度为 $2w_k(0.3-0.5) = -0.138889$，要求提高该局部输出。又有 $\partial\mu_\theta/\partial\hat{\epsilon} = -0.320092$，所以提高噪声估计会降低目前偏高的反向均值。网络共享参数，批量更新仍要合并其他样本的梯度。

本题后验知道 x_0；部署不知道。式(1.9-4)中 $C_0 = 0.327869$，因此

$$\mathrm{Var}(x_{k-1} \mid x_k, c) = 0.147541 + C_0^2\,\mathrm{Var}(x_0 \mid x_k, c).$$

第二项表示干净答案的歧义。题目未给 $q(x_0 \mid c)$，不能继续算出完整部署方差，更不能由前面那个高斯推出真实反向分布一定为高斯。

题1.9-2答案。 两种先验各半，后验赔率等于似然比。记 $p_+(y) = P(x_0 = 2 \mid y)$：

$$\log\frac{p_+(y)}{1-p_+(y)} = \frac{(y+1.6)^2 - (y-1.6)^2}{2(0.36)} = \frac{3.2y}{0.36}.$$

所以 $p_+(y) = 1/[1+\exp(-3.2y/0.36)]$，干净均值为 $m(y) = 4p_+(y) - 2$，最优噪声预测为 $f_*(y) = [y - 0.8m(y)]/0.6$。

候选 y	右侧后验概率	干净均值 $m(y)$	噪声均值 $f_*(y)$
0	0.5	0	0
1	0.999862	1.999448	−0.999265

$y=0$ 时，干净后验的两个众数为±2，均值0不属于任何一个数据点；噪声后验在±8/3上各半，均值也是0。$y=1$ 时，干净众数为+2、噪声众数为−1，与各自均值仍有小差别。均值使平均平方误差最小，众数选择概率最大的取值，抽样则按后验随机取值，三者作用不同。

直接动作回归只读固定条件，得到 $\mathbb{E}[x_0 \mid c]=0$；本题还读当前候选，得到随 y 变化的函数。然而一次还原 $\hat{x}_0=[y-0.6f_*(y)]/0.8=m(y)$ 仍是后验均值，$y=0$ 时仍为0。局部预测随位置变化，为完整多步生成提供了条件；它本身不保证一次输出某条路线，也不保证最终物理可行。

题1.9-3答案。 D2已经给出当前噪声预测，函数实现D3、D4：

```
import math

def ddim_step(candidate, alpha_bar_now, alpha_bar_next, predicted_noise):
    # D3: 当前噪声比例下的干净动作估计；alpha_bar_now须大于0。
    clean = (candidate - math.sqrt(1 - alpha_bar_now) * predicted_noise)
    clean /= math.sqrt(alpha_bar_now)
    # D4: 按下一噪声级别重组干净成分与残余噪声。
    following = (math.sqrt(alpha_bar_next) * clean
                 + math.sqrt(1 - alpha_bar_next) * predicted_noise)
    return clean, following
```

当前 $\bar{\alpha}_{1000}=0.0001$，下一级 $\bar{\alpha}_{875}\approx 0.041479333749$。代入题给噪声得到 $\hat{x}_0\approx 0.020113752$、$x_{875}\approx 0.493444352$。尚未到干净端点，残余噪声系数不为0；直接返回0.020114会跳过后续模式判断。到 $s=0$ 才能按这个更新直接返回 $\hat{x}_0$。

DDPM相邻均值对应特定相邻前向核；跨级时不能照搬其系数。可先用 $\alpha_{\text{eff}}=\bar{\alpha}_{1000}/\bar{\alpha}_{875}$ 构造跨级前向核：给定 x_0 与 x_{1000} 时，x_{875} 的后验是高斯，其均值对 x_0 线性。再把本次 $\mathbb{E}[x_0 \mid x_{1000}, c]\approx 0.020114$ 代入，得到只给定候选与条件时的真实反向均值约0.027621；未知干净答案的完整后验一般仍是混合分布。确定性DDIM的结果为0.493444，说明它保留残余噪声的方式不同。“没有中途新抽噪声”并不唯一决定更新规则。

在本节对称解析分布中，$x_k=0$ 时两个模式的后验各半，$\epsilon_*(0,k)=0$、$\hat{x}_0=0$，所以每次更新仍为0。全部零初始化会全部停在中间。这个结论依赖对称函数与确定性更新；不能直接套到非对称学习网络或带额外随机性的采样器。

题1.9-4答案。 标量批损失 $L=B^{-1}\sum_i(\hat{\epsilon}_i-\epsilon_i)^2$，故 $\partial L/\partial\hat{\epsilon}_i=2(\hat{\epsilon}_i-\epsilon_i)/B$。代入 $B=256$、预测−0.027333、标签1.921532，得到约−0.015226。它先作用于本行输出，再经链式法则传到共享权重；不是直接把所有参数都增加这个数。

训练程序读取干净示范，再自己抽级别和标准高斯噪声，因此输入与噪声标签都已知。一次噪声回归已经能提供监督，不必先跑完整生成链。训练更新参数；部署固定参数、重复更新候选。

令 $X=(x_k,k,c)$、$m(X)=\mathbb{E}[\epsilon\mid X]$。在平方中加减 $m(X)$，交叉项因 $\mathbb{E}[\epsilon-m(X)\mid X]=0$ 而在总体期望下消失：

$$\mathbb{E}[(\epsilon-f(X))^2]=\mathbb{E}[\mathrm{Var}(\epsilon\mid X)]+\mathbb{E}[(m(X)-f(X))^2].$$

因此，最优函数仍会留下输入无法唯一决定标签的误差。0.26901是本次有限验证集上的经验值，不是已经精确求出的总体不可约误差。

有限集合中交叉项不必抵消。记 $d_i=f(X_i)-m(X_i)$、$r_i=\epsilon_i-m(X_i)$，有

$$L_{\mathrm{net}}-L_{\mathrm{oracle}}=\frac{1}{N}\sum_i d_i^2-\frac{2}{N}\sum_i d_i r_i.$$

种子7的标签损失差约0.01060，对解析函数的MSE却约0.00618，两者可以同时成立。要测函数拟合误差，应直接比较网络与解析条件均值，而不是把两个有限样本标签损失相减。

题1.9-5答案。 固定解析函数时，加密当前这组采样级别降低误差，说明有限步长确有影响；它没有测试网络学习能力，也没有证明所有模型多用几步都改善。解析实验还使用近似先验及有限分位点，因此其误差不能完全称为纯离散误差。

固定种子7网络时，更多步数也有帮助，但256步仍明显偏离解析结果。可以按两个方向安排对照：固定网络，在同一初值和合法级别网格中加密采样；固定采样器，再改变训练预算、容量或目标权重并保留多个种子。解析对照另比较真实末端加噪分布与标准高斯初始化，检查先验近似的影响。对每个噪声级别，既在真实前向输入上、也在实际生成所访问的候选上比较 ϵ_θ 与 ϵ_*，能发现平均损失掩盖的位置误差。

最终分位点误差不是“采样误差 + 网络误差”的可加公式，不能简单把0.13166减去0.00185就命名为纯网络误差。预测函数会改变后面访问的位置，二者在生成过程中相互影响。

标签MSE接近，也可能在模式分界附近有不同误差，进而改变左右比例；其他位置的误差可能改变峰宽。动作块还存在各位置边缘分布都正确、联合关系却错误的情况。因此要直接检查生成分布及跨时联系，再用真实任务的执行指标评估策略。

题1.9-6答案。 独立采样四种组合 $(-2,-2),(-2,2),(2,-2),(2,2)$，各占1/4；示范只包含第一和第四种，各占1/2。错误跨模式组合共占一半，尽管两个时间位置分别看都分布正确。

联合网络能利用两个位置属于同一模式的信息。例如加噪后 $y_i=ax_{0,i}+b\epsilon_i$，两个完整模式的后验对数赔率是：

$$\log\frac{P(x_0=(2,2)\mid y_1,y_2)}{P(x_0=(-2,-2)\mid y_1,y_2)}=\frac{4a(y_1+y_2)}{b^2}.$$

第一个位置应怎样去噪，因而也取决于第二个位置。前向噪声独立，不意味着干净动作后验独立。网络需要通过跨时间卷积、注意力等连接传递这类信息；如果只逐坐标处理，或者训练与采样有偏，输入拼成整块也不保证学好联合关系。

可以统计模式不一致率 $R = N^{-1}\sum_n \mathbf{1}\{\mathrm{sign}(x_1^{(n)}) \neq \mathrm{sign}(x_2^{(n)})\}$：目标为0，独立采样期望为1/2。还应检查四种组合的频率和到示范模式的距离，避免全部生成左侧或全部接近0时，仅靠一致率就误判成功。真实动作要依据任务定义协调关系，不能一概要求两个时刻的命令同号。

H 决定一次生成包含多少个真实动作，S 决定内部调用网络多少次，K 决定训练噪声网格大小。完成内部生成、反标准化后，执行前 T_a 个动作，通常 $1 \le T_a \le H$，再取新观测重规划。减少 S 可以改变推理预算，缩短 H 改变预测时域，调整 T_a 改变获得新反馈前的执行长度；内部带噪候选不逐个执行。

进一步阅读：Step-by-Step Diffusion的两点分布与多步运输、DDPM、DDIM及Diffusion Policy。

1.10 Flow Matching：从配对直线到分布运输

1.10.1 为什么把动作生成写成一个速度场

上一节通过逐级去噪生成动作。**流匹配**（Flow Matching，FM）从另一种容易制造的监督入手：先规定噪声怎样与示范动作连接，再学习途中应怎样改变候选。它要解决的困难仍然是：我们有动作样本，却通常不知道复杂动作分布的密度，更不知道怎样一次抽到合适的完整动作块。

把归一化动作块展平为 $A \in \mathbb{R}^d$，$d = Hd_a$，其中 H 是块内动作个数，d_a 是单步动作维数。训练还会抽取同形状的标准高斯噪声 Z。条件 c 包含做出这次决策时可用的图像、语言、本体状态或观测历史，不含未来示范动作。A 是标签，Z 是人为产生的起点，两者都在动作表示空间中。

向量场在这里就是一个查询函数：把当前候选 x、生成进度 τ 和条件 c 送进去，它返回候选每单位生成进度应怎样改变。网络记为 $v_\theta(x, \tau, c)$，输出与动作块同形状。部署时从噪声出发，按照这个函数连续推进候选：

$$\frac{dY_\tau}{d\tau} = v_\theta(Y_\tau, \tau, c), \qquad Y_0 = Z, \qquad 0 \le \tau \le 1. \tag{1.10-1}$$

这是**常微分方程**（Ordinary Differential Equation，ODE）：右边根据现在的位置给出变化率。求解它，就是从起点出发反复查询方向并累积变化，最后用 Y_1 作为生成的动作块。给定同一个初值、条件和求解规则，结果可以确定；换不同噪声起点，整组结果仍可以形成一个分布。

τ 是模型内部进度，通常从0走到1，不是机器人经过了1秒。v_θ 的单位是“动作表示单位/生成进度”，不能直接当作末端米/秒或关节弧度/秒执行。一次动作块生成时条件通常固定；整块算好后反标准化，执行一个前缀，再取得新观测重规划。训练修改网络参数，积分修改候选，这两种更新的对象不同。

1.10.2 训练为什么不必先解出这条ODE

如果直接根据最终生成结果训练，梯度可能要穿过整条积分链。FM改为先设计一条能直接抽样、能直接求速度的训练路径。最常见的构造是独立抽取 $Z \sim \mathcal{N}(0, I)$、$A \sim p_{\mathrm{data}}(\cdot \mid c)$ 和 $\tau \sim \mathcal{U}(0, 1)$，再计算：

$$X_\tau = (1 - \tau)Z + \tau A, \qquad U = A - Z. \tag{1.10-2}$$

固定一对端点时，X_τ 是连接它们的直线，导数 U 已知。例如 $Z = -1, A = 3, \tau = 0.25$，训练输入位置为0，速度标签为4。这个4来自我们选定的生成路径，不需要额外测量机器人真实速度。

网络读 X_τ, τ, c，预测速度。已知端点只负责制造输入和标签，不作为额外答案送进网络：

$$L_{\mathrm{CFM}}(\theta) = \mathbb{E}\|v_\theta(X_\tau, \tau, c) - U\|^2. \tag{1.10-3}$$

这个目标称为**条件流匹配**（Conditional Flow Matching，CFM）。名称里的“条件”首先指用训练端点构造容易处理的路径；机器人任务条件 c 是另一层信息。固定某条示范 $A = a$，只让噪声变化，会得到一条以该示范为条件的概率路径。把各种示范混合起来，才得到网络实际要生成的整体动作分布。Flow Matching原论文，第3节

```
P1 读取干净动作A与当前可用条件c
P2 抽Z、τ，计算Xτ=(1−τ)Z+τA与标签U=A−Z
P3 网络只读Xτ、τ、c，预测速度
P4 比较预测与U，计算平方损失并反向传播
P5 更新网络参数；下一批重新抽端点、噪声与进度
```

配套程序 practice/train_bimodal_flow.py 按P1—P5标注。损失只依赖一个随机进度的预测，不必在每个训练批次先运行完整生成ODE。这就是这里“无需模拟训练”（simulation-free training）的含义：省去训练时对生成ODE的求解与求导，并不表示机器人学习不需要数据、物理仿真或执行评估。

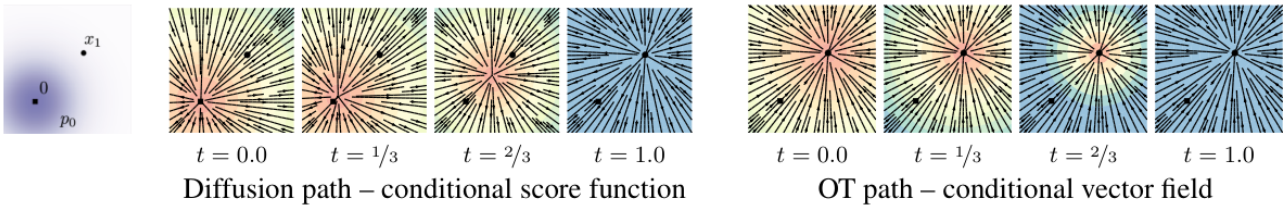


图1.10-1：Lipman等，Flow Matching论文Figure 2。左侧是扩散路径的条件score，右侧是最优传输型条件路径的速度场。原图的 t 对应这里的生成进度 τ，x_1 对应固定的目标样本 $A = a$，p_0 是噪声分布。原论文

图中“conditional”指给定一个目标样本，不是给定机器人文本指令。箭头表示二维示意空间中的函数输出：左侧score是对数密度梯度，右侧是生成速度；蓝色表示较大模长，红色表示较小模长。两侧画的是不同数学量，不能按箭头长度直接比较控制速度或效率。右侧“OT”是Optimal Transport（最优传输）；原图在目标周围保留标准差 $\sigma_{\min} > 0$ 的窄高斯，本节直线构造则取 $\sigma_{\min} = 0$，含 $1 - \tau$ 分母的条件场只在 $\tau < 1$ 使用。这里的OT描述给定端点后的路径设计，尚未说明整个数据分布的端点配对已经最优。1.10.7节会进一步区分。

1.10.3 平均不同路径的速度，为什么没有把分布平均成一点

1.10.3.1 平方损失估计的是当前位置的后验平均

同一个中间候选可能来自不同端点。部署看不到原始配对，网络便无法保证找回那条示范的速度。固定输入 x, τ, c，平方损失的理想解是：

$$v_*(x, \tau, c) = \mathbb{E}[U \mid X_\tau = x, \tau, c]. \tag{1.10-4}$$

它按“哪些配对可能经过这里”取平均，不是把全部动作终点平均成一个固定动作。不同位置具有不同后验，因此可以得到不同速度。

将标签写成 $U = v_* + (U - v_*)$，展开平方，交叉项的条件期望为0，于是：

$$L_{\mathrm{CFM}} = \underbrace{\mathbb{E}\|v_\theta(X_\tau,\tau,c) - v_*(X_\tau,\tau,c)\|^2}_{L_{\mathrm{FM}}} + \underbrace{\mathbb{E}\|U - v_*(X_\tau,\tau,c)\|^2}_{\text{与网络参数无关}}. \tag{1.10-5}$$

这说明用容易获得的单路径标签训练，与直接拟合整体速度场，在期望上具有相同参数梯度。我们不必先算出所有示范的后验概率，再制造一个昂贵的整体速度标签。不同批次的梯度仍有随机性；有限网络也未必能完全表示 v_*。式(1.10-5)没有保证标签MSE能降到0。作者指南，第4.5节

问题还没有结束。条件平均解释了网络学什么，接下来还要解释：为什么恰好是这个平均，能够搬运正确的概率分布？

1.10.3.2 先看一个区间的样本怎样流入和流出

固定某个生成进度，把所有训练配对产生的候选混在一起，得到密度 $p_\tau(x \mid c)$。这叫这一时刻的**边缘分布**：它只问“现在的候选在哪里”，不记录每个候选原来配给了哪个动作。随 τ 排列的一组分布称为**概率路径**；单个样本随进度移动的线才是样本轨迹。

一维空间里，若密度为 p、速度为 v，边界附近一个小区间宽度为 $v\,d\tau$，其中约有 $p\,v\,d\tau$ 的概率质量跨过边界。于是定义**概率通量** $j = pv$：正值表示向右穿过，负值表示向左穿过。它统计一群样本的流量，不是某个点本身的概率。

令固定区间 $[l, r]$ 中的概率质量为 $M_\tau = \int_l^r p_\tau(x)\,dx$。左边界向右的流量增加区间中的质量，右边界向右的流量减少它：

$$\frac{dM_\tau}{d\tau} = j_\tau(l) - j_\tau(r).$$

让区间足够小，就得到**连续性方程**：

$$\partial_\tau p_\tau(x) + \partial_x\big[p_\tau(x) v(x,\tau)\big] = 0. \tag{1.10-6}$$

若右侧流出量比左侧流入量大，$\partial_x(pv) > 0$，当地密度便下降。这就是负号的来源。多维时，把各坐标方向上的净流出相加，写成 $\nabla_x \cdot (p_\tau v)$；这个运算称为**散度**。它在此处描述局部概率质量的净流出，不是另一个待学习网络。

1.10.3.3 条件速度要按当地密度组合

现在固定一个示范 $A = a$。独立高斯噪声给出的条件密度与对应速度为：

$$r_\tau(x \mid a) = \mathcal{N}(x; \tau a, (1-\tau)^2 I), \qquad u_\tau(x \mid a) = \frac{a - x}{1 - \tau}, \qquad \tau < 1.$$

沿着该配对的直线代入 $x = (1-\tau)Z + \tau a$，便有 $u_\tau = a - Z = U$，与训练标签一致。虽然每条配对直线的斜率固定，作为一个空间查询函数，$u_\tau(x \mid a)$ 仍会随位置和进度变化。

整体密度是各种条件密度的混合：

$$p_\tau(x \mid c) = \int r_\tau(x \mid a) p_{\text{data}}(a \mid c)\, da.$$

不同示范在同一个位置贡献的通量可以相加。用贝叶斯公式改写：

$$\begin{aligned} j_\tau(x \mid c) &= \int r_\tau(x \mid a) u_\tau(x \mid a) p_{\text{data}}(a \mid c)\, da \\ &= p_\tau(x \mid c) \int u_\tau(x \mid a) \underbrace{\frac{r_\tau(x \mid a) p_{\text{data}}(a \mid c)}{p_\tau(x \mid c)}}_{p(a \mid X_\tau = x, \tau, c)} da \\ &= p_\tau(x \mid c) v_*(x, \tau, c). \end{aligned} \tag{1.10-7}$$

这一步说明平均方式为什么不能随意选：每条条件路径先贡献“当地密度×自己的速度”，加起来才是总通量；再除以当地总密度，就得到后验加权的平均速度。稀少的路径不能与大量经过此处的路径平起平坐。

每条已知条件路径满足自己的连续性方程。将它们相加，得到：

$$\begin{aligned} \partial_\tau p_\tau &= \int \partial_\tau r_\tau \, p_{\text{data}}(a \mid c)\, da \\ &= -\nabla_x \cdot \int r_\tau u_\tau p_{\text{data}}(a \mid c)\, da \\ &= -\nabla_x \cdot (p_\tau v_*). \end{aligned} \tag{1.10-8}$$

因此，后验平均保留了所选概率路径的流量关系。若从正确的初始分布出发，速度场足够规则，使ODE有唯一解且相应概率演化成立，沿 v_* 积分便能得到同一组时刻分布。这里要求积分与求导可以交换、所需期望有限等条件，不能把任意奇异或不连续场都代入这个结论。原论文第3.1节、作者指南第3.5、4.3—4.4节

保留的是每个进度上的整体分布，不是每条训练配对的身份。不同配对直线可以交叉；满足唯一性条件的同一ODE，在同一时刻相遇后不能又沿不同方向分开。因此，生成轨迹一般会重新组织这些流量，而不会逐条照着训练直线走。

上面的条件表达式只在 $\tau < 1$ 使用。若终点集中在离散数据点，端点需要按极限理解，不能在含 $1 - \tau$ 的式子里直接除以0。后面的连续高斯例能给出有限的整体端点速度；训练标签 $A - Z$ 本身始终无需做这次除法。

1.10.3.4 用概率质量变化核对这个解释

取与扩散节相同的双峰动作分布：两侧均值为±2，成分标准差0.15，概率各半，条件固定。观察区间 $[-0.5, 0.5]$。配套程序分别用真实分布CDF（Cumulative Distribution Function，累积分布函数）的中心差分计算区间质量的时间变化，用密度与解析速度计算边界通量，得到：

τ	区间质量 M_τ	左边界通量 $j(l)$	右边界通量 $j(r)$	$dM_\tau/d\tau$
0.10	0.412159	0.105003	−0.105003	0.210005
0.25	0.408516	−0.167895	0.167895	−0.335790
0.50	0.159843	−0.691619	0.691619	−1.383238
0.75	0.000132	−0.004790	0.004790	−0.009580

$\tau = 0.1$ 时，两边都向区间内流，中央质量增加；到0.5时，两边向外流，中央质量快速减少。看似“平均速度”的规则并没有把样本一直聚到中间。速度随位置与进度变化，局部质量便可以先聚集、再分开，最后形成两个峰。

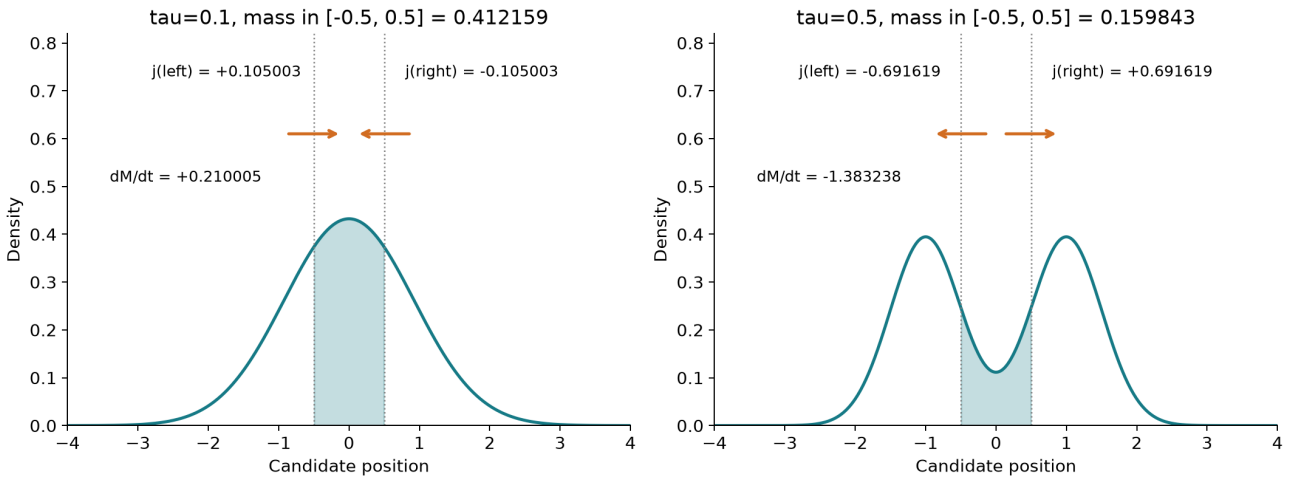


图1.10-2：同一中央区间在两个进度的密度与边界流向。数字为有正负号的概率通量；区间质量的变化等于左通量减右通量，和表中独立CDF计算吻合。

1.10.4 一个能从头算完的高斯例子

先保留一个简单基线：$Z \sim \mathcal{N}(0,1)$、$A \sim \mathcal{N}(m,1)$，两者独立。固定条件后，训练位置 $X_\tau = (1-\tau)Z + \tau A$ 的均值、方差及速度标签的协方差为：

$$\mathbb{E}[X_\tau] = \tau m, \qquad S_\tau = \mathrm{Var}(X_\tau) = (1-\tau)^2 + \tau^2,$$

$$\mathbb{E}[U] = m, \qquad \mathrm{Cov}(U, X_\tau) = 2\tau - 1.$$

这些量由端点的独立性直接得到。例如协方差中，A 与 τA 贡献 τ，$-Z$ 与 $(1-\tau)Z$ 贡献 $-(1-\tau)$，交叉贡献为0。

U 与 X_τ 联合为高斯。将 $U - m$ 分成 $b(X_\tau - \tau m)$ 和剩余量，选 $b = \mathrm{Cov}(U, X_\tau)/S_\tau$，可使剩余量与 X_τ 不相关；联合高斯下它们也独立，剩余量的条件均值为0。因此：

$$v_*(x,\tau) = m + \frac{2\tau - 1}{S_\tau}(x - \tau m). \tag{1.10-9}$$

这个条件均值公式依赖联合高斯性质；一般多峰分布不能只代一个整体协方差就得到正确速度场。1.10.5节会在每个高斯成分内计算，再按后验权重混合。

第一项推动总体均值，第二项根据候选偏离均值的程度改变分布宽度。$\tau < 0.5$ 时系数为负，候选朝当前均值收缩；$\tau > 0.5$ 时系数为正，偏离均值的程度重新扩大。两者共同把起点分布搬到目标，而不是始终把所有候选往终点均值拉。

我们甚至可以写出部署ODE的精确轨迹：

$$Y_\tau = \tau m + \sqrt{S_\tau} Z_{\text{initial}}. \tag{1.10-10}$$

对它求导，$dY_\tau/d\tau = m + (2\tau - 1)Z_{\text{initial}}/\sqrt{S_\tau}$；将 $Z_{\text{initial}} = (Y_\tau - \tau m)/\sqrt{S_\tau}$ 代回，正好得到式(1.10-9)。它从 $Y_0 = Z_{\text{initial}}$ 出发，最后到 $Y_1 = m + Z_{\text{initial}}$，保留了方差1。

取 $m = 2$，中间 $\tau = 0.5$ 时均值为1、标准差为 $\sqrt{0.5}$；终点均值为2、标准差回到1。不同初值的轨迹如下。

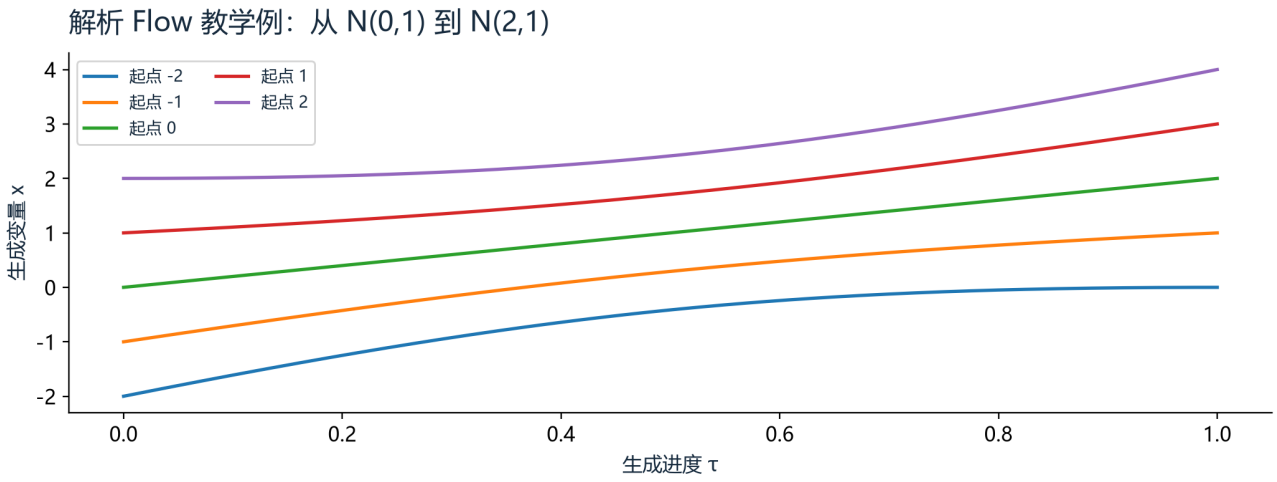


图1.10-3：原书已有的解析高斯轨迹，展示式(1.10-10)的五个初值。曲线的弯曲来自分布宽度先收缩、后扩大；它不是训练配对直线图，也不是机械臂运动轨迹。

再看一条具体训练配对：$Z = 1, A = 1$，它的插值始终为1、速度标签为0。但在 $\tau = 0.5, x = 1$ 处，最优整体速度为2。这个训练样本没有出错：同一位置还可能由其他端点产生，网络要对那些可能来源一起平均。部署从初值1出发，沿整体场得到 $Y_{0.5} = 1 + \sqrt{0.5} \approx 1.7071$，终点为3，不必返回这条训练配对的终点1。所有初值一起看，终点分布仍是 $\mathcal{N}(2, 1)$。

这个例子也能定位积分错误。$\tau = 0$ 的理想速度是 $m - x$；若只做一次步长为1的欧拉更新，$Y_1 = x + (m - x) = m$，所有候选都被压成均值。场完全正确，生成仍会失败，因为一步近似没有跟踪后续的收缩与扩展。增加细分步数后，配套程序对精确终点的均方根误差在16、64、256步时约为0.07728、0.01988、0.00501；这里使用共同的4096个高斯分位点，具体初值约定见下一节。

1.10.5 同一条件下的双峰：跟踪八次更新及整群候选

1.10.5.1 先算出各位置真正应该用的平均速度

现在把动作分布换为 $\frac{1}{2}\mathcal{N}(-2, 0.15^2) + \frac{1}{2}\mathcal{N}(2, 0.15^2)$，条件固定为0，噪声仍为标准高斯。这个分布与扩散节的连续双峰实验相同；生成路径换成线性插值，输出换成速度。数值没有米或弧度语义。

记成分均值为 $m_j \in \{-2, 2\}$、成分标准差为 $s = 0.15$。在给定成分后：

$$\mathbb{E}[X_\tau \mid j] = \tau m_j, \qquad S_\tau = (1-\tau)^2 + s^2\tau^2, \qquad C_\tau = s^2\tau - (1-\tau).$$

其中 $C_\tau = \mathrm{Cov}(A - Z, X_\tau \mid j)$。沿用上一节的高斯条件化，每个成分对当前位置提出的平均速度为：

$$v_j(x, \tau) = m_j + \frac{C_\tau}{S_\tau}(x - \tau m_j).$$

看见候选 x 后，属于该成分的后验权重是：

$$w_j(x, \tau) = \frac{\exp[-(x - \tau m_j)^2/(2S_\tau)]}{\sum_\ell \exp[-(x - \tau m_\ell)^2/(2S_\tau)]}.$$

于是整体解析场为：

$$v_*(x, \tau) = \sum_j w_j(x, \tau) v_j(x, \tau). \tag{1.10-11}$$

这就是后验平均的具体计算。权重随位置、进度改变，不是把左右速度永远各取一半。在 $\tau = 0$ 时，位置等于源噪声、无法提供动作模式信息，场为 $-x$；后续位置逐渐带上模式线索，局部速度便能把不同候选送向两侧。

1.10.5.2 八步欧拉：每一步都在新位置重新查询

欧拉法在一个小区间内把当前预测速度暂当常数，按 $Y_{\tau+h} = Y_\tau + h\, v(Y_\tau, \tau, c)$ 推进。取 $h = 1/8$、初值0.5，用上述解析场完成全部八步：

当前 τ	当前候选	本次平均速度	下一 τ	更新后候选
0	0.500000	−0.500000	0.125	0.437500
0.125	0.437500	−0.174132	0.25	0.415733
0.25	0.415733	0.389628	0.375	0.464437
0.375	0.464437	1.522420	0.50	0.654739
0.50	0.654739	2.613776	0.625	0.981461
0.625	0.981461	2.648704	0.75	1.312549
0.75	1.312549	2.581448	0.875	1.635230
0.875	1.635230	2.367918	1	1.931220

每行读取上一行的最后一列，再计算新的速度。开始时分布先收缩，候选向中间靠；后面逐渐偏向右模式，候选才向右移动。变化的速度是局部后验平均，并不是最初某一对端点的恒定差。机器人不会在这些中间值之间来回运动，内部候选只用于生成最后的动作。

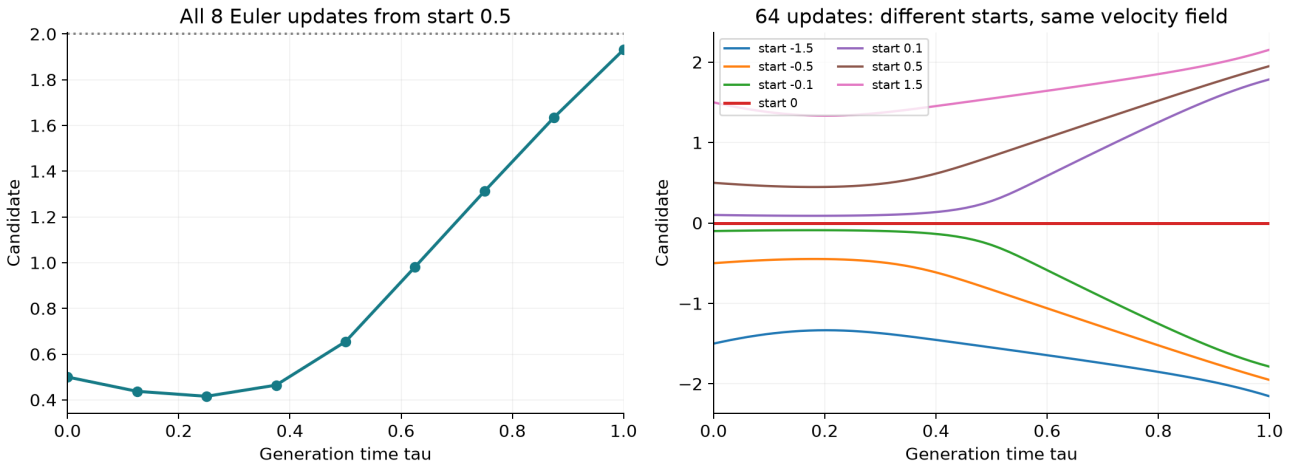


图1.10-4：左图对应全部八次欧拉更新，右图用64步比较七个初值。对称解析场下初值0始终为0；它是连续高斯起点中的零概率单点，不能由此推出整群样本停在中间。若人为把全部起点设成0，则已经改变了生成初始分布。

0处速度为0，为什么那里的密度仍能下降？密度统计的是附近一小段区间容纳了多少质量。由连续性方程，在对称中心有 $\partial_\tau p(0) = -p(0)\partial_x v(0,\tau)$。虽然正中心不动，若附近候选向两侧分开，$\partial_x v > 0$，中心密度就会降低。本例 $\tau = 0.5$ 时，$\partial_x v(0, 0.5) \approx 13.39161$、$p_{0.5}(0) \approx 0.11159$，所以密度变化率约为−1.49440。静止轨迹与邻域质量减少描述的是不同对象。

程序中的生成步骤为：

```
G1 从源分布抽候选，设置进度网格，固定模型参数与当前条件
G2 在当前位置查询速度，先做欧拉预测
G3 若采用Heun，再在预测位置和下一进度查询一次，取两次斜率的平均
G4 重复更新到终点，返回生成动作块；再由执行模块处理
```

只使用G2的核心实现是：

```
def euler_generate(field, source_noise, steps):
    candidate = source_noise.copy()         # G1: 初始化候选
    for i in range(steps):
        tau = i / steps
        velocity = field(candidate, tau)    # G2: 当前条件在field内部保持固定
        candidate = candidate + velocity / steps
    return candidate                        # G4: 返回终点，不执行中间候选
```

完整版本在 practice/flow_core.py。这个循环不读取示范终点，也不更新网络参数。

1.10.5.3 比较求解器，要对齐函数调用次数

Heun法再估一次区间末端的斜率：

$$\tilde{Y} = Y_\tau + h\,v(Y_\tau,\tau), \qquad Y_{\tau+h} = Y_\tau + \frac{h}{2}\Big[v(Y_\tau,\tau) + v(\tilde{Y},\tau+h)\Big]. \tag{1.10-12}$$

第一步预测候选可能到哪里，第二步检查到那里后速度是否改变，再用平均斜率修正。它每步调用两次场，所以不能拿“相同更新步数”直接当作相同推理预算。

下表沿用4096个共同起点 $z_i = \Phi^{-1}((i+0.5)/4096)$，$i = 0,\dots,4095$；$\Phi$ 为标准高斯CDF。它们是确定性分位点，不是独立随机抽样。访问网格为 $\tau_i = i/S$。将终点排序，与目标双高斯的相同分位点比较，平均绝对间隔记为 E_{q}，定义与1.9.7相同。

场调用次数	欧拉更新步数 / E_{q}	Heun更新步数 / E_{q}
16	16 / 0.025444	8 / 0.008324
64	64 / 0.006819	32 / 0.000841
256	256 / 0.001735	128 / 0.000060

在这个光滑解析场上，两次斜率的修正提高了相同调用预算下的精度。64步欧拉的左右均值约±1.99318，组内标准差约0.14664；目标成分标准差为0.15。有限积分仍有偏差，不能因为场由解析式给定就认为采样已经精确。

一次欧拉的理论终点全部为0，程序只有舍入量级的残差，分位点误差为2。这个对照再次表明，准确的局部平均不等于可以用任意步长跳到终点。

1.10.6 从原有条件高斯训练到同条件双峰学习

1.10.6.1 原实验说明了什么

原书的 generative_actions.py 实际训练了一个条件流网络，数据为 $c = -1$ 或 $+1$，$A \mid c \sim \mathcal{N}(c, 0.2^2)$。输入是三维 $[X_\tau, \tau, c]$，输出一维速度；两个48维隐层后使用SiLU非线性。噪声、示范和进度每批重新抽取，只有网络参数接受优化。

本学习包保留未修改的原程序为 practice/legacy_conditional_flow.py，明确启动入口为 practice/run_legacy_conditional.py。按种子13训练1600次，重新运行得到标签MSE约0.34432，始终预测0的MSE约2.05571；两个条件的终点均值约−1.02249、1.01289，标准差约0.18197、0.18369，目标标准差为0.2。

它验证了在这个简单任务中学习条件生成的流程，但每个条件只有一个高斯。把两个条件的样本混在一起看到两个峰，不能据此说明模型已经学会“同一条件下的双峰”。

1.10.6.2 固定条件后，网络怎样学习两种模式

新实验沿用三输入、两个48维隐层的网络，将数据改为1.10.5的同条件双峰。三个种子13、17、29各训练8000次，批大小256，Adam学习率0.002；单独用种子90901生成2048个验证输入，不参加梯度更新，也不用它挑选最优检查点。下面均报告最后一次更新后的模型。

训练核心仍然只有P1—P5：

```
clean, noise, tau, condition, candidate, label = batch(generator, 256)  # P1、P2
prediction = model(candidate, tau, condition)                           # P3
loss = (prediction - label).square().mean()                             # P4, label=clean-noise
optimizer.zero_grad()
loss.backward()
optimizer.step()                                                        # P5
```

种子13第一批的一个样本中，$Z = 0.445580$、$A = -1.783544$、$\tau = 0.562007$，所以输入位置约−0.807203、速度标签约−2.229124。尚未训练的网络预测0.093972。对本行预测的批平均损失梯度为：

$$\frac{\partial L}{\partial \hat{v}_i} = \frac{2}{256}(\hat{v}_i - U_i) \approx 0.018149.$$

该样本要求降低当前偏正的输出；信号经网络的链式求导传到权重，与同批其他样本共同形成参数更新。它监督的是已知端点差，训练循环没有调用ODE采样器，也没有使用解析场当老师。解析场只在训练之外提供核对基准。

种子	训练前标签MSE	训练后标签MSE	对解析速度场的MSE
13	5.18145	2.39720	0.03317
17	5.15505	2.39130	0.03111
29	5.20443	2.40166	0.04085

解析条件均值对同一批标签的MSE仍为2.35647。它是有限验证集的经验值，原因与式(1.10-5)一致：给定中间输入后，原配对速度仍有歧义。标签误差不为零，不等于整体速度没有学会；对解析场的误差与最终生成结果应分别检查。

表中还有一个容易疑惑的细节：种子13的两项标签MSE相减约为0.04073，并不等于场误差0.03317。式(1.10-5)对总体期望成立；有限验证集的平方展开还保留：

$$\frac{1}{N}\sum_i (\hat{v}_i - U_i)^2 - \frac{1}{N}\sum_i (v_{*,i} - U_i)^2 = \frac{1}{N}\sum_i (\hat{v}_i - v_{*,i})^2 + \frac{2}{N}\sum_i (\hat{v}_i - v_{*,i})(v_{*,i} - U_i).$$

最后一项在总体期望下为0，在这批2048个输入上约为0.00756。因此应直接比较预测与解析场，不能把两个经验标签误差之差当成精确的场误差。条件均值的最优性针对总体分布；其他预测器在某批特定标签上的误差偶然更低，也不违反这个结论。真实任务通常没有解析场可供比较，便更需要独立验证数据和生成结果来判断拟合。

随后固定权重，从相同4096个起点生成。欧拉法结果为：

种子	调用次数	左侧占比	E_{q}
13	16	0.5137	0.07572

种子	调用次数	左侧占比	E_{q}
13	64	0.5125	0.05882
13	256	0.5122	0.05531
17	16	0.4915	0.07567
17	64	0.4927	0.05341
17	256	0.4929	0.04826
29	16	0.5010	0.03372
29	64	0.5022	0.03791
29	256	0.5024	0.04123

三组都形成双峰，比例、位置和峰宽仍有偏差。种子29的256步结果，左侧均值约−2.04550、标准差0.17513，偏离目标的−2与0.15。仅有左右数量接近各半，不能代替完整分布检验。

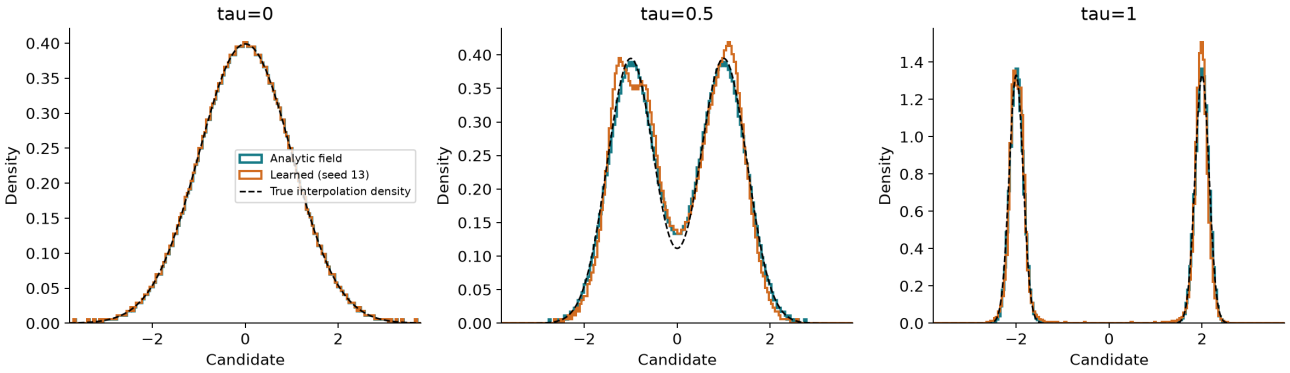


图1.10-5：64步欧拉在 $\tau = 0, 0.5, 1$ 的候选分布，比较解析场与种子13网络；虚线为所选训练概率路径的真实密度。初值相同，不意味着有限网络会保持同样的中间密度或终点模式比例。

1.10.6.3 为什么更多步数反而可能更偏离数据

种子29值得单独检查：欧拉从16增加到256次调用，目标分布误差反而增大。为分清这是不是仍由积分太粗造成，固定同一个最终网络，用更密的Heun积分作数值参考。Heun从256步加到512步，配对终点的平均变化约 7.83×10^{-6}，已远小于下表中的主要偏差。

这里比较两个不同的量。E_{q} 是对真实目标分布的误差；D_{solver} 是各方法从相同初值出发，与Heun512终点逐一配对后的平均绝对差。后者衡量对这个已学速度场的数值求解接近程度，参考结果仍是数值近似。

固定种子29网络的求解方式	调用次数	D_{solver}	对数据的 E_{q}
欧拉16步	16	0.038390	0.033717
欧拉64步	64	0.009172	0.037910

固定种子29网络的求解方式	调用次数	D_{solver}	对数据的 E_{q}
欧拉256步	256	0.002322	0.041229
Heun32步	64	0.000617	0.042573
Heun512步	1024	0	0.042585

积分越来越接近这个网络定义的生成过程，数据分布却没有随之接近。粗积分的偏差可能碰巧抵消一部分场误差；把前者减小，后者便更明显。这不是“欧拉16步学到了更准确的场”，因为各行使用完全相同的权重。

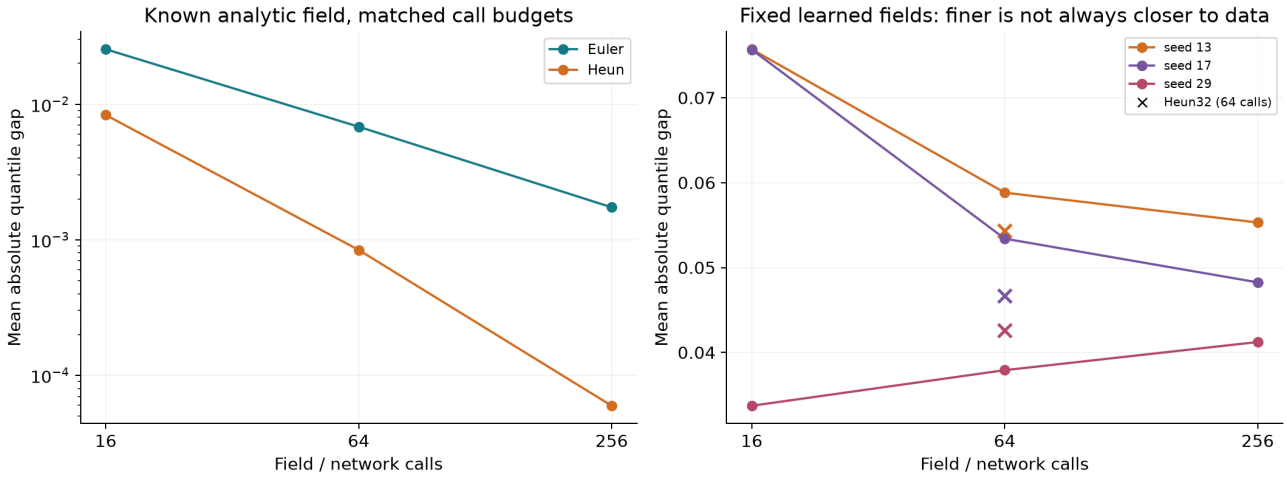


图1.10-6：左图为解析场在相同调用预算下的求解器对比；右图为三个固定学习网络。叉号表示64次调用预算下的Heun32步，种子29出现了更精确积分却更偏离数据的结果。

因此，优化生成要分开做对照：固定网络检查积分是否稳定，固定求解器再改训练数据、容量或目标权重。还应在真实前向候选与生成实际访问的位置上分别检查场预测。最终分布误差不是可以直接相加的“学习误差 + 积分误差”两个标量；它们会通过候选路径相互影响。

1.10.7 改变配对和输出，究竟改变了什么

1.10.7.1 直线训练路径不等于全局最优配对

耦合（coupling）指噪声与动作两端怎样配对的联合分布。独立抽取 Z, A 是一种耦合；保留相同的两端边缘分布，还可以采用其他配对。配对不同，中间候选分布、速度标签和实际要学习的场都可能改变。

例如噪声端点−1、+1和动作端点−2、+2都各半。同侧配对的平方距离平均为1，交叉配对则为9。每对端点都能画成直线，但整体运输代价不同。因此“给定目标样本的条件OT路径”，不等于“噪声分布到动作分布的全局最优耦合”。前者规定一对已知端点怎样连接，后者还要决定哪些端点应配成一对。作者指南第4.7、4.9节

1.10.7.2 Reflow为什么需要重新生成端点对

整流流（Rectified Flow）通过直线插值标签学习因果生成ODE；**Reflow**再利用已学ODE产生新的起点—终点对。具体过程是：先从噪声 Z 运行教师模型，得到 $\hat{A}$；把 $(Z, \hat{A})$ 保持成对，重新训练两端之间的直线速度。新模型尝试把教师原来弯曲的生成过程改得更直接，从而减轻少步积分的困难。RF作者教程、原始论文

回到1.10.4的高斯例子。原来的独立配对给出弯曲的整体轨迹 $Y_\tau = \tau m + \sqrt{S_\tau} Z$，但其精确终点为 $\hat{A} = m + Z$。用教师产生的新配对重新插值：

$$R_\tau = (1-\tau)Z + \tau(m+Z) = Z + \tau m, \qquad \frac{dR_\tau}{d\tau} = m. \tag{1.10-13}$$

新场为常数 m，一次欧拉就得到 $Z + m$，与目标高斯完全对应。这里能一步准确，是因为这个特例的配对与场已解析给定；不是把旧场的步数改成1就有同样效果。旧场在初始时刻给出 $m - x$，一步会把样本全部压成 m。差别来自重新配对改变了目标场：新场平移整群样本，旧场的一步近似却消去了样本间的差异。

中间分布也随新配对改变了。旧精确轨迹的方差是 $S_\tau = (1-\tau)^2 + \tau^2$，新路径 $R_\tau = Z + \tau m$ 的方差始终为1。这里要求新旧共享起点和教师终点分布，并不要求保留教师每个中间进度的密度。1.10.3的结论则是在给定同一套插值路径后，把单路径标签平均为整体场，保留那套路径的各时刻边缘分布；两个结论比较的对象不同。

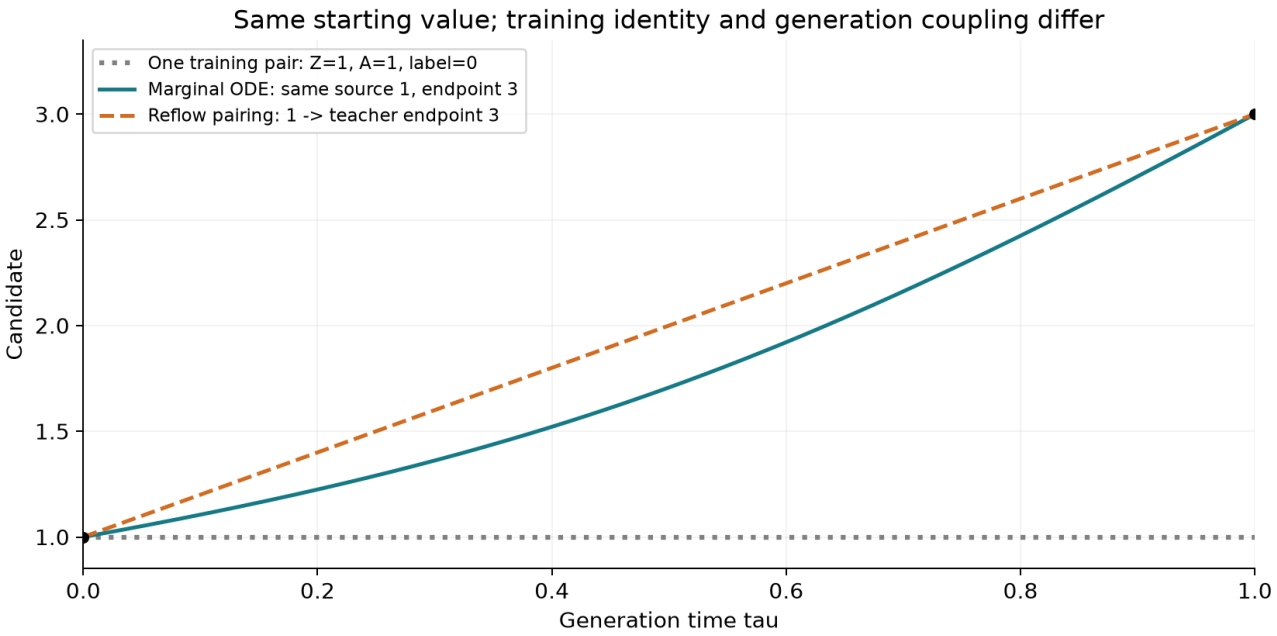


图1.10-7：单高斯 $m = 2$ 的同一个噪声初值1。示意训练配对 $Z = 1, A = 1$ 的直线保持在1；整体ODE从1到3；用教师端点3重新配对后，形成另一条从1到3的直线。三条线承担不同角色，不能把训练配对身份当成部署要求。

实际Reflow需要用数值教师产生数据，再训练有限网络，是否足够直仍要测量。教师如果漏掉模式，新配对的终点分布也会继承这种偏差。Reflow继续训练局部场；直接蒸馏终点映射是另一种可配合的操作，不能仅凭“重新配对”就保证真实高维模型一步生成准确。

1.10.7.3 Flow与扩散的联系要放在相同路径下比较

对本节线性路径，记网络预测为 $\hat{v}$，可同时构造干净端点和噪声端点的估计：

$$\hat{A} = x + (1-\tau)\hat{v}, \qquad \hat{Z} = x - \tau\hat{v}. \tag{1.10-14}$$

它们来自 $x = (1-\tau)Z + \tau A$、$U = A - Z$ 的联立求解。输出速度、动作或噪声并非互不相干的任务，但损失权重会变化。例如 $\hat{Z} - Z = -\tau(\hat{v} - U)$，所以 $\tau > 0$ 时：

$$\|\hat{v} - U\|^2 = \frac{1}{\tau^2}\|\hat{Z} - Z\|^2.$$

相同路径上的不加权速度MSE，对应一个随进度变化的噪声MSE权重，不等于直接沿用扩散节的所有系数。端点转换含除法时还要检查其定义域；直接速度训练不需要除以 τ。

采样也有具体联系。若采用这里同一线性路径，从 τ 前进到 $s > \tau$，按DDIM式的端点重组：

$$x_s = s\hat{A} + (1-s)\hat{Z} = x + (s-\tau)\hat{v}.$$

这正好是欧拉更新。这个等价要求路径、输出转换和访问时刻对应；上一节的方差保留余弦路径还涉及坐标与时间变换，不能直接把两个已经分别训练的模型当成同一个模型。作者教程：扩散与Flow Matching的对应

确定性ODE之外，也可以设计带连续随机扰动的**随机微分方程**（Stochastic Differential Equation，SDE）。在正确的分布信息与相应规则下，ODE和SDE能够具有相同的各时刻边缘分布，单个样本轨迹却不同。额外随机扰动会改变概率流量，因此需要配套修正确定部分；在本文欧拉循环里直接加随机数，并不能保持原来的目标路径。作者指南第10.4—10.5节

对VLA而言，路径、输出参数化、网络结构和求解器是彼此相关的设计选择：局部监督决定学习任务，求解器决定怎样使用输出，最后还需把整个动作块接入物理闭环。上述标量程序解释这些关系，没有训练图像/语言模型或验证动作约束。真实系统应同时测量网络调用、条件编码与数据传输耗时、动作质量和闭环任务结果，不能只按步数推断速度与成功率。

1.10.8 练习题

题1.10-1：标签有歧义，为什么仍能训练正确的场？ 令网络输入为 $W = (X_\tau, \tau, c)$，标签为 $U = A - Z$，$v_*(W) = \mathbb{E}[U \mid W]$。从平方展开推导式(1.10-5)，说明交叉项为什么消失、两个目标在哪个意义下具有相同梯度。若某批次的标签MSE没有下降，能否断言对 v_* 的拟合也没有改善？为什么直接回归 $\mathbb{E}[A \mid c]$ 和这里的条件平均不会产生相同的生成结果？

题1.10-2：平均速度为什么要按当前位置重新加权？ 为便于手算，只在本题把目标端点改成等概率的 $A = -2$ 或 $+2$，源仍为标准高斯，采用线性插值。在 $\tau = 0.5, x = 0.25$ 处，分别计算两条条件密度的后验权重、条件速度和整体速度。若直接将两个条件速度各取一半，会得到什么？解释这种平均为什么不符合当地通量。再用1.10.3.4表中 $\tau = 0.5$ 的边界通量判断中央区间质量如何变化，并说明“中央密度下降”与“某个恰好位于0的样本速度为0”为什么不矛盾。

提示：本题固定目标端点，所以条件方差是 $(1-\tau)^2$，不是1.10.5中含成分方差 0.15^2 的 S_τ。两个端点仅用于计算 $\tau < 1$ 时的局部场，不据此宣称端点处仍是光滑密度。

题1.10-3：训练直线与生成轨迹为什么不同？ 设 $Z \sim \mathcal{N}(0,1)$、$A \sim \mathcal{N}(2,1)$ 独立。推导 X_τ 的均值、方差与 $\mathrm{Cov}(A-Z, X_\tau)$，据此得到整体场，并验证 $Y_\tau = 2\tau + \sqrt{(1-\tau)^2+\tau^2}Z_{\text{initial}}$ 满足生成ODE。比较训练配对 $Z=A=1$ 与部署初值 $Y_0=1$：中间位置、速度与终点各是什么？最后说明为什么一步欧拉会把正确的场用成方差为0的生成器。

题1.10-4：两次场调用怎样分配？ 使用1.10.5的连续双峰解析场，从初值0.5出发，计算两步欧拉（每步 $h=0.125$）与一步Heun（$h=0.25$）到 $\tau=0.25$ 的结果，列出实际查询的进度、位置和速度。两者调用次数相同，这一例的终点大小能否直接判断哪个求解器更准？设计一个能区分积分误差与场学习误差的比较，并解释为什么不能以初值0的静止轨迹代替分布测试。

题1.10-5：一次梯度更新到底监督了什么？ 在1.10.6.2给出的首批样本上，算出 X_τ, U 及损失对该行预测的梯度，解释这个梯度怎样参与网络权重更新。有人将模型输入从 $[X_\tau, \tau, c]$ 改成 $[X_\tau, \tau, A]$，声称“动作标签越充分，训练越容易”，这一修改在哪个环节失去部署意义？已知解析场在固定验证集上的标签MSE为2.35647，而种子13网络为2.39720，能否把二者之差直接认作网络对解析场的MSE？请区分总体恒等式与有限验证集中的交叉项。

题1.10-6：均值正确、方差接近0，该怎样定位原因？ 某条件流模型在固定可用条件下，测试示范有明显的动作多样性，生成均值正确但方差几乎为0。设计一组分步骤对照，分别排查生成实现、条件与数据覆盖、积分和场拟合。每项对照都要说明固定什么、改变什么、哪种结果支持哪类原因。再分析种子29的实际结果：欧拉16、64、256步的 D_{solver} 依次为0.038390、0.009172、0.002322，而对数据的 E_{q} 依次为0.033717、0.037910、0.041229。它支持什么判断，仍不能排除什么？下一步应怎样改实验？

题1.10-7：直线、最优配对与Reflow分别改变了什么？ 两端分别为等概率的噪声点−1、+1和目标点−2、+2。计算同侧、交叉和独立配对的平均平方距离，解释为什么每条训练线都是直线仍不代表整体配对最优。再用题1.10-3的高斯场作精确教师，推导Reflow的新配对、新路径和新场，并与“直接把旧场改成一步”比较。若实际教师漏掉一个动作模式，Reflow能否仅靠重新配对补回该模式？

题1.10-8：输出速度、噪声或动作，哪些可以互换？ 对线性路径，给定 $\tau=0.25, x=0.5, \hat{v}=2$，求 $\hat{A}, \hat{Z}$；再用二者重组到 $s=0.5$，核对与欧拉更新是否相同。推导速度MSE和噪声MSE的权重关系，说明为什么直接使用同一个不加权MSE会改变训练目标。最后指出，哪些条件满足时“DDIM式重组等于欧拉一步”成立，为什么不能据此交换任意扩散模型和Flow模型的采样代码。

1.10.9 参考答案

题1.10-1答案 记 $e = v_\theta(W) - v_*(W)$、$r = v_*(W) - U$，便有：

$$\|v_\theta - U\|^2 = \|e\|^2 + \|r\|^2 + 2e^\top r.$$

给定 W，网络误差 e 已确定，而 $\mathbb{E}[r \mid W] = v_*(W) - \mathbb{E}[U \mid W] = 0$，所以 $\mathbb{E}[e^\top r] = 0$。取期望即得到 $L_{\text{CFM}} = L_{\text{FM}} + \mathbb{E}\|r\|^2$。末项来自给定输入后仍无法辨认的配对差异，不随网络参数改变。训练抽样分布固定、可以交换求导与期望时，两个总体损失具有相同的参数梯度。

相同的是期望梯度，不是每行标签或每个批次的梯度。某输入的标签若等概率为±1，条件均值为0；在恰好抽到标签+1的那一行，把预测从0.5改为0，对条件均值的误差从0.25降到0，对这行标签的误差

却从0.25升到1。因此，一批标签MSE没有下降不足以判断总体场拟合。有限验证集还保留经验交叉项，见题5。

直接从 c 回归动作，理想解是 $\mathbb{E}[A \mid c]$，固定条件每次只返回同一个数或向量。FM保留随机起点，并用 x, τ, c 决定局部速度；不同候选看到的后验不同，随后还要积分。这里取平均的对象和生成方式都变了。准确的后验场在相应正则条件下运输分布；有限网络、有限数据和求解器仍需各自验证。

题1.10-2答案 本题固定 $A = -2$ 或 $+2$，在 $\tau = 0.5$ 时两条条件密度分别是 $\mathcal{N}(-1, 0.25)$、$\mathcal{N}(1, 0.25)$。在 $x = 0.25$ 处：

$$r_- = \frac{\exp(-1.25^2/0.5)}{0.5\sqrt{2\pi}} \approx 0.03505660, \qquad r_+ = \frac{\exp(-0.75^2/0.5)}{0.5\sqrt{2\pi}} \approx 0.25903519.$$

先验各半，后验由当地密度重新加权。因为 $r_+/r_- = e^2$，所以 $w_- \approx 0.11920292$、$w_+ \approx 0.88079708$。条件速度为：

$$u_- = \frac{-2 - 0.25}{0.5} = -4.5, \qquad u_+ = \frac{2 - 0.25}{0.5} = 3.5.$$

整体速度约为 $0.11920292(-4.5) + 0.88079708(3.5) = 2.54637662$。各取一半则得到−0.5，连方向都变了。正确通量是 $\frac{1}{2}r_-u_- + \frac{1}{2}r_+u_+ \approx 0.37443423$；总密度约0.14704590，除回去正好得到2.54637662。错误平均给出的通量约−0.07352295。端点先验各半，不表示看见当前位置后两个来源仍各半。

后半问回到正文的连续双高斯，不能把上面离散端点的密度直接代入表格。表中中央区间的质量变化为 $-0.691619 - 0.691619 = -1.383238$，说明两端都在向外流。0处轨迹静止，附近候选却可以向两侧分开；$\partial_\tau p(0) = -p(0)\partial_x v(0, \tau)$ 仍能为负。密度统计邻域质量，单点静止不要求邻域密度保持不变。

题1.10-3答案 写 $A = 2 + \epsilon$，其中 ϵ, Z 为独立标准高斯。则：

$$X_\tau = 2\tau + (1 - \tau)Z + \tau\epsilon, \qquad U = 2 + \epsilon - Z.$$

因此 $\mathbb{E}[X_\tau] = 2\tau$、$S_\tau = (1 - \tau)^2 + \tau^2$、$\mathbb{E}[U] = 2$，且协方差为 $\tau - (1 - \tau) = 2\tau - 1$。联合高斯条件均值给出：

$$v_*(x, \tau) = 2 + \frac{2\tau - 1}{S_\tau}(x - 2\tau).$$

对 $Y_\tau = 2\tau + \sqrt{S_\tau}z_0$ 求导，利用 $S'_\tau = 4\tau - 2$：

$$\dot{Y}_\tau = 2 + \frac{2\tau - 1}{\sqrt{S_\tau}}z_0 = 2 + \frac{2\tau - 1}{S_\tau}(Y_\tau - 2\tau),$$

正好等于场在该候选处的输出，且 $Y_0 = z_0$。于是随机初值形成的边缘为 $\mathcal{N}(2\tau, S_\tau)$，终点为 $\mathcal{N}(2, 1)$。

训练配对 $Z = A = 1$ 始终位于1，标签始终为0。部署只知道初值1，使用的是整体场：

τ	训练配对位置 / 标签	部署位置 Y_τ	部署位置处的速度
0	1 / 0	1.000000	1.000000
0.25	1 / 0	1.290569	1.367544
0.50	1 / 0	1.707107	2.000000
0.75	1 / 0	2.290569	2.632456
1	1 / 0	3.000000	3.000000

在训练配对经过的 $(1, 0.5)$，整体场也为2，不等于这条配对的标签0。网络对可能来源做平均，没有承诺保留训练配对身份。

若只用一次欧拉，初始速度为 $2 - z_0$，终点 $z_0 + (2 - z_0) = 2$，所有随机差异都被消去。精确轨迹的终点则是 $2 + z_0$，方差仍为1。失败来自一步近似没有跟踪场随进度的变化。

题1.10-4答案 两步欧拉的调用与更新为：

查询进度	查询位置	返回速度	更新后位置
0	0.500000	−0.500000	$0.5 + 0.125(-0.5) = 0.4375$
0.125	0.437500	−0.174132451	0.415733444

Heun第一次也查询 $(0.5, 0)$，速度为−0.5，但步长0.25给出的预测位置是0.375。第二次查询 $(0.375, 0.25)$，速度约0.358233644；修正后：

$$Y_{0.25} = 0.5 + \frac{0.25}{2}(-0.5 + 0.358233644) \approx 0.482279205.$$

第二次查询的预测点和修正后的终点不同。两种方案都用了两次场调用，终点大小本身不能说明谁更准。

这个一维解析例还能独立制造参考。设 F_τ 为候选CDF，无穷远通量为0时，连续性方程给出 $\partial_\tau F_\tau = -p_\tau v_*$。沿精确ODE求导：

$$\frac{d}{d\tau} F_\tau(Y_\tau) = -p_\tau(Y_\tau) v_*(Y_\tau, \tau) + p_\tau(Y_\tau)\dot{Y}_\tau = 0.$$

因此 $F_{0.25}(Y_{0.25}) = \Phi(0.5)$。用正文双高斯CDF反解，得到参考位置约0.466176344；两步欧拉的绝对误差约0.050442900，Heun约0.016102862。本例后者更近，结论仍只针对这个场和预算。

测试时先固定解析场、共同初值和条件，只改求解器与步长，检验积分误差。再固定某个学习网络，为它自身的ODE建立加密后稳定的数值参考，区分“接近网络定义的轨迹”与“接近真实数据分布”。最后固定求解器比较学习场与解析场，并用整群起点检查模式比例、峰宽与分位误差。只取初值0，会把一个特殊静止轨迹当成全部样本；把所有初值改成0，还改变了源分布。

题1.10-5答案 由题给数值：

$$U=-1.783544-0.445580=-2.229124,\qquad X_\tau=0.445580+0.562007(-2.229124)\approx-0.80720329.$$

批平均损失对该行输出的梯度约为：

$$\frac{2}{256}(0.093972+2.229124)=0.01814919.$$

正号表示这一行希望预测下降。它经链式求导参与 $\nabla_\theta L=\frac{2}{256}\sum_i(\hat v_i-U_i)\nabla_\theta v_\theta(X_i,\tau_i,c_i)$；实际参数梯度汇总全部行，再由优化器更新共享权重。因此单行提出的方向不保证在一次共同更新后单独实现。训练监督的是 $A-Z$，候选本身没有变成网络参数。

把未来动作 A 送入模型会泄漏答案。给定 X_τ,τ,A，在 $\tau<1$ 可直接反解 $Z=(X_\tau-\tau A)/(1-\tau)$，再得到 $U=(A-X_\tau)/(1-\tau)$。训练变容易，是因为提供了部署正要生成的未知终点；运行时无法构成相同输入。随意用一个猜测动作代入，也没有学到原任务所需的整体场。

有限验证集保留交叉项。令 $e_i=\hat v_i-v_{*,i}$、$r_i=v_{*,i}-U_i$，有：

$$\widehat L_{\mathrm{CFM}}-\widehat L_*=\frac{1}{N}\sum_i e_i^2+\frac{2}{N}\sum_i e_i r_i.$$

种子13的两个标签误差相减约0.04073，其中场MSE约0.03317、经验交叉项约0.00756。该交叉项的总体期望为0；有限验证集中的取值可能为正、为负或恰好为0，不能直接删去。解析场的经验标签误差也不是该有限集上所有预测器都无法突破的下界；它的最优性针对总体条件风险。

题1.10-6答案 先保存共同初值、固定条件、权重、每步候选及反标准化前后输出，找到方差从哪一步消失，再做下列对照。

环节	固定与改变	可支持的判断
随机起点与输出处理	固定网络和网格，显式送入不同高斯起点，逐步保存候选与最终接口值	初值已相同，应查随机种子重置、零值或广播；候选有方差而接口输出相同，应查批平均、缓存覆盖或反标准化
生成循环	固定起点和代码，先换成常数平移场 $v=2$，再换成题3高斯场	平移场应得到 $Z+2$；若仍丢失方差，优先查实现。高斯场一步压缩、多步恢复，则支持粗积分原因
条件和示范覆盖	使用同一组部署可用信息分组，比较训练与测试的条件内分布、预处理及未来信息	只有混合不同条件才多峰、训练缺某模式或训练读了未来标签，分别支持条件定义、覆盖或信息泄漏问题
积分	固定最终权重和全部起点，细化欧拉并用同调用预算Heun对照	方差随细化恢复，支持数值误差；不同方法收敛到同一窄分布，则需要继续查场和数据
场拟合与结果	固定稳定求解器；合成例分别检查真实前向输入和生成访问位置上的场残差，再逐项改变容量、训练量或覆盖	可定位分支区域拟合不足或访问分布偏移；结合模式比例、峰宽、分位误差与多个种子验证，均值一致不足以定论

真实数据通常没有解析场，只能结合独立标签风险、条件均值估计和生成分布检验；单条配对标签不是正确整体速度的唯一答案。数据对照还要确认相同条件确实指相同的可用观测与指令，而非训练时多提供了路线标签。

种子29的权重固定，D_{solver} 随步数减小而 E_{q} 增大，支持“求解更接近该网络的ODE，却更偏离目标数据”。Heun256与512终点平均差约 7.83×10^{-6}，其512步结果对数据仍有约0.042585的误差；粗欧拉偏差可能抵消了一部分场偏差。该表不能区分容量、优化或数据问题，也不能排除少量尾部候选的数值误差，更不能推出增加步数通常有害。

下一步先固定稳定求解器，按进度和位置检查分支附近、终点附近的场残差；再分别只增加训练量、容量或数据覆盖，跨独立种子核对完整分布。若目标是少步生成，则另做同调用预算下的训练或蒸馏对照。实际机器人还需验证动作约束与闭环结果，这些标量指标没有承担该职责。

题1.10-7答案 四种平方距离为：

$$\begin{array}{c|cc} & A=-2 & A=+2 \\ Z=-1 & 1 & 9 \\ Z=+1 & 9 & 1 \end{array}.$$

同侧配对平均代价1，交叉配对9，独立配对四种各四分之一，平均代价5。直线决定已知两端怎样连接，耦合决定哪两端配成一对；每对之间都直，也可能选择了代价更大的配对。

精确高斯教师从 Z 产生终点 $\hat{A}=Z+2$。保持这组起点—终点身份，新路径为 $R_\tau=(1-\tau)Z+\tau(Z+2)=Z+2\tau$，标签恒为2，所以理想新场也恒为2，一次欧拉得到 $Z+2$，方差为1。把配对打乱后重新独立抽取，就失去了这个新耦合。旧场直接一步则得到 $Z+(2-Z)=2$，方差为0。

新旧共享起点和教师终点分布；旧中间方差为 $(1-\tau)^2+\tau^2$，新中间方差始终为1。Reflow改变了训练耦合及概率路径，不是对旧场仅仅换一个步长。实际教师漏掉模式时，新端点数据也缺少该模式，重新配对没有提供恢复它的额外信息。应先改善教师覆盖，或明确加入有覆盖的真实监督，再独立检验，而不能把学生偶然生成类似动作当成覆盖已经恢复。

题1.10-8答案 代入端点转换：

$$\hat{A}=0.5+0.75(2)=2,\qquad \hat{Z}=0.5-0.25(2)=0.$$

到 $s=0.5$ 的重组为 $0.5(2)+0.5(0)=1$，欧拉为 $0.5+(0.5-0.25)2=1$。一般情况下：

$$s[x+(1-\tau)\hat{v}]+(1-s)[x-\tau\hat{v}]=x+(s-\tau)\hat{v}.$$

同一条路径上的误差满足 $\hat{Z}-Z=-\tau(\hat{v}-U)$、$\hat{A}-A=(1-\tau)(\hat{v}-U)$。所以 $\tau>0$ 时速度MSE等于 $1/\tau^2$ 倍噪声MSE；$\tau<1$ 时也等于 $1/(1-\tau)^2$ 倍动作MSE。本题两个权重分别为16和 $16/9$。

不加权噪声MSE相当于给速度误差乘 τ^2，减轻早期的惩罚；不加权动作MSE则乘 $(1-\tau)^2$，减轻后期惩罚。有限网络共享容量与参数，改变权重一般会改变拟合取舍。在每个输入都能独立达到理想解、内部时刻权重为正且转换可逆时，对应的条件均值仍可互相转换；不能把有限训练目标不同说成理想输出绝对无法对应。

用一个刻意受限的模型可以看到这种取舍：它只能输出同一个常数 w，两个训练输入的进度为0.25、0.75，速度标签分别为1、3。速度MSE取两者平均，最优 $w = 2$；不加权噪声MSE对应的速度权重是 $0.25^2, 0.75^2$，最优 $w = (0.25^2 + 3 \times 0.75^2)/(0.25^2 + 0.75^2) = 2.8$；不加权动作MSE反过来更重视早期，得到 $w = 1.2$。同一参数不能同时满足两个输入时，权重决定优先拟合谁。

欧拉等价还要求同一状态坐标、噪声到数据的方向、线性系数、当前同次预测构造的两端估计，以及对应的访问时刻；采用确定性重组，不额外加随机项或改变端点估计。任意扩散模型可能使用不同日程、缩放、时间方向和输出参数化，分别训练的网络也不会自动相等。从噪声预测反推速度的除法在 $\tau = 0$ 还需单独处理，从动作预测反推则在 $\tau = 1$ 需处理，因此采样代码必须与具体模型的定义配套。

1.11 动作编码、条件引导与少步生成

第1.9、1.10节已经解释了怎样从噪声生成动作块。接下来会遇到几个不同的问题：动作序列太长，能否换一种更紧凑的表示？模型偶尔忽略指令，能否在生成时加强条件影响？多次网络计算来不及完成，能否把原来的多步过程交给一个少步模型？

这些问题发生在不同环节。压缩动作编码，减少的是需要预测的离散符号；条件引导，改变的是生成过程中采用的方向；一致性训练和蒸馏，则让模型学习跨越更长生成区间。本节沿着这些问题分别展开，再把它们放回同一个机器人系统中比较。

1.11.1 网络结构、动作表示与学习目标怎样组合

扩散Transformer（Diffusion Transformer，DiT）用Transformer处理带噪候选，并读取生成时刻和任务条件。原始DiT以图像潜表示的patch为输入；用于机器人时，可以把一个动作块的每个控制位置投影为token，输出与动作块同形状的噪声、干净量或速度。第1.6节已给出它的条件调制、门控和初始化。采用这类网络结构，并不要求一定使用某一种扩散损失。DiT原论文

U-Net采用下采样、上采样和跨层跳接。在时间序列版本中，卷积先组合相邻动作，再在较粗时间尺度上整合较长片段；上采样时结合前面保留的细节。Transformer则用注意力直接建立token之间的关系。这是两种组织信息的方式。动作长度、局部变化、条件接口和计算预算不同，适合的结构也会变化。

VLA中的**动作专家**（action expert）是一组负责精细动作生成的参数，通常通过交叉注意力或分块注意力读取视觉语言主干。这里的“专家”描述模块分工。**混合专家**（Mixture of Experts，MoE）中的专家则由路由器选择参与计算；一个动作专家可以完全没有这种路由结构。

先分清设计轴，再比较模型名

示例组合：视觉语言底座 + DiT 动作专家 + 连续动作块 + Flow 监督。

图1.11-1：输入与表示、网络结构、输出对象、训练范式与目标可以组合。图中CNN是卷积神经网络（Convolutional Neural Network），RL是强化学习（Reinforcement Learning）；Flow在示例中指流匹配动作生成。

例如，“视觉语言主干 + DiT动作专家 + 连续动作块 + Flow Matching监督”描述了一个完整组合的不同部分。FAST改变动作怎样编码，DiT改变计算结构，Flow Matching改变速度场怎样训练和使用。后文比较效率时，还要补上实际生成多少轮、每轮计算多大，以及生成后执行多少步。

1.11.2 条件引导：为什么两条预测的差能加强指令

考虑桌上同时有红杯和蓝杯的场景。当前图像和关节状态记为r，文字“拿红杯”记为ℓ。如果带文字与不带文字的预测几乎相同，文字对这次生成的影响就很弱。**无分类器引导**（Classifier-Free Guidance，CFG）把这两条预测的差放大，从而加强指定条件的影响。训练时随机丢弃文字，让同一个模型见过(r, ℓ)和$(r, \varnothing)$两种输入；图像与关节状态仍保留。CFG原论文

设f_{cond}、f_{base}分别是这两种条件下的方向预测，二者必须针对同一个候选、同一个生成时刻：

$$f_{\text{guided}} = f_{\text{base}} + w(f_{\text{cond}} - f_{\text{base}}). \tag{1.11-1}$$

$w = 0$使用缺少文字的预测；$w = 1$使用普通文字条件预测；$w > 1$沿着“加入文字造成的变化”继续外推。这里的base仍然知道机器人正面对什么场景。“去掉文字”与“去掉所有观测”是两个不同输入，训练和部署必须一致。

这个组合为什么有条件偏好的含义？先在一个固定噪声尺度σ上用**分数函数**（score）解释。它是对当前变量求log密度的梯度。下面先假设模型准确给出了这个梯度，再看两条预测如何组合；实际估计误差会影响引导结果。令

$$s_{\text{cond}} = \nabla_x \log p_\sigma(x \mid r, \ell), \qquad s_{\text{base}} = \nabla_x \log p_\sigma(x \mid r).$$

由贝叶斯公式，

$$\log p_\sigma(x \mid r, \ell) = \log p_\sigma(x \mid r) + \log p_\sigma(\ell \mid x, r) - \log p_\sigma(\ell \mid r).$$

最后一项与x无关，求导后消失，因此

$$s_{\text{cond}} - s_{\text{base}} = \nabla_x \log p_\sigma(\ell \mid x, r). \tag{1.11-2}$$

差值指向使文字条件更能由当前候选解释的局部方向。相应的引导score可以写成

$$s_{\text{guided}} = \nabla_x \log \left[p_\sigma(x \mid r, \ell)^w p_\sigma(x \mid r)^{1-w} \right]. \tag{1.11-3}$$

式(1.11-3)描述固定噪声层上的密度倾斜；括号内还需能够归一化。它并没有证明整个多步采样器的最终分布恰好等于干净数据密度的同样幂次组合。噪声预测可以通过与score的关系得到对应引导；若采用其他速度参数化，则还要考虑它所用的路径和方向转换。

一个可直接核算的例子是：在某个固定噪声层，基础密度为$\mathcal{N}(0,1)$，条件密度为$\mathcal{N}(1,1/4)$。取$w=2$，组合密度的对数中，与x有关的部分为

$$2\left[-\frac{(x-1)^2}{2(1/4)}\right] - \left[-\frac{x^2}{2}\right] = -\frac{7}{2}\left(x - \frac{8}{7}\right)^2 + \text{常数}.$$

因此这个层上的倾斜密度均值为$8/7$、方差为$1/7$：分布更集中，均值还越过了普通条件均值1。加强条件是一种外推，不是把原预测简单限制在更安全的范围内。若条件与可行接触动作之间的关系学得不好，增大w也可能放大错误。

CFG在每个生成时刻需要两条条件预测。把它们组成一个batch可以减少调度开销，却仍要计算两组预测；图像特征是否共享则取决于实现。没有训练条件丢弃分支，部署时临时塞入空文字，也不具备上述训练前提。选择w时，应同时查看目标选择、成功率、多样性、异常动作和端到端延迟。

1.11.3 FAST：一段动作怎样变成少量离散token

从逐维分箱的局限出发

一个动作块记为$A \in \mathbb{R}^{H \times d_a}$，$H$是控制步数，$d_a$是每步动作维度。若把每个标量分到一个离散区间，再逐项预测，$H=16, d_a=7$就有112个标量编号。即使前后两步只差一点，仍要分别给出编号。

分箱本身很清楚：把$[-1,1]$等分为256档，档宽为$2/256$；以档中心解码时，范围内误差不超过半档。问题在于，这种表示没有利用动作在时间上的关联。高采样率下，相邻动作重复很多，序列却依然很长；自回归模型还要逐个完成这些预测。

频域动作序列编码（Frequency-space Action Sequence Tokenization，FAST）先把时间序列变成频率系数，再量化并压缩。它希望把“这一段整体怎样变化”集中到更少的符号里。下面按照作者教程和原论文的处理顺序走一遍。作者教学、FAST原论文

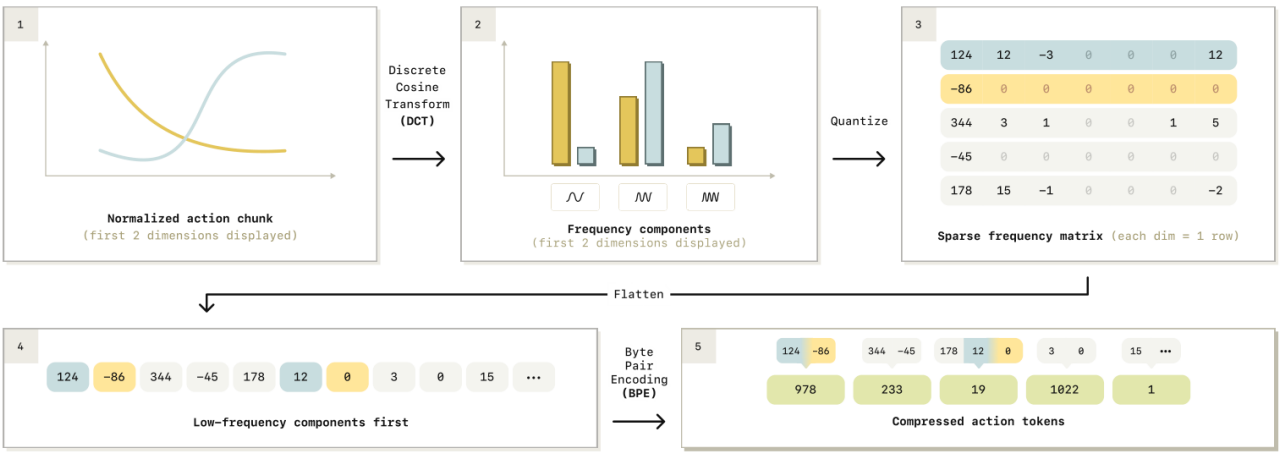


图1.11-2：FAST论文Figure 4。Normalized action chunk是归一化动作块；Frequency components是时间频率成分；Quantize是量化；Flatten是按约定顺序展开；Byte Pair Encoding是BPE压缩；Compressed action tokens是最终离散编号。图中Sparse frequency matrix每行表示一个动作维度，按列从低频到高频取数；本节程序用转置布局$[\text{频率}, \text{动作维}]$，展开后顺序相同。

DCT改变表示，尚未压缩信息

离散余弦变换（Discrete Cosine Transform，DCT）把一段序列写成若干余弦基的加权和。对第j个动作维度，

$$C_{kj} = \alpha_k \sum_{t=0}^{H-1} A_{tj} \cos\left[\frac{\pi}{H}(t+\tfrac{1}{2})k\right], \quad \alpha_0 = \sqrt{\frac{1}{H}}, \quad \alpha_{k>0} = \sqrt{\frac{2}{H}}. \tag{1.11-4}$$

t在这里是动作块内的控制索引，k是频率索引。$k=0$的余弦恒为1，记录整个片段的平均水平；较大的k记录更快的起伏。正交归一化使这些基互相独立、长度为1。把各个基作为矩阵D的行，就有$C = DA$、$D^\top D = I$，所以$A = D^\top C$。保留全部精确系数时，这只是换坐标。

取一个模拟的、已经归一化的四步双通道动作块：

$$A = \begin{bmatrix} 0 & 0.2 \\ 0.2 & 0.4 \\ 0.4 & 0.6 \\ 0.6 & 0.8 \end{bmatrix}, \qquad D \approx \begin{bmatrix} .5 & +.5 & +.5 & +.5 \\ .653281 & +.270598 & -.270598 & -.653281 \\ .5 & -.5 & -.5 & +.5 \\ .270598 & -.653281 & +.653281 & -.270598 \end{bmatrix}.$$

第一通道的零频项为$0.5(0+0.2+0.4+0.6) = 0.6$；一阶项为$0.653281 \times 0 + 0.270598 \times 0.2 - 0.270598 \times 0.4 - 0.653281 \times 0.6 \approx -0.446088$。整块结果是

$$C \approx \begin{bmatrix} .6 & 1 \\ -.446088 & -.446088 \\ 0 & 0 \\ -.031703 & -.031703 \end{bmatrix}.$$

第二通道始终比第一通道高0.2，因此只改变零频项；其他变化形状相同，对应系数也相同。四步动作依然有八个系数，DCT还没有减少数量。它的价值在于，平滑序列的许多高频系数较小，接下来量化后容易产生重复的零和短片段。

量化开始损失精度，排列决定哪些片段相邻

给定尺度$s > 0$，把系数放大后取整：

$$Q_{kj} = \mathrm{round}(sC_{kj}), \qquad \widehat{C}_{kj} = Q_{kj}/s. \tag{1.11-5}$$

$\widehat{C}$是恢复的近似值，不是原系数。s越大，系数网格越细；但整数范围和后续压缩情况也会改变。取$s = 10$，

$$Q = \begin{bmatrix} 6 & 10 \\ -4 & -4 \\ 0 & 0 \\ 0 & 0 \end{bmatrix} \longrightarrow [6, 10, -4, -4, 0, 0, 0, 0].$$

先列出两个通道的零频项，再列出两个通道的一阶项，依次向高频展开。这样，控制整段动作的大尺度变化会较早进入自回归序列；不同通道的相似频率模式也有机会形成相邻片段。它没有把整段动作简单截成前两个频率：本例最后一阶变零，是取整造成的。

BPE怎样学习合并，又怎样还原

字节对编码（Byte Pair Encoding，BPE）先统计训练序列中常见的相邻符号对，把它们合并成新符号，再重复统计。编码新序列时使用已学的合并顺序；解码时把每个新符号展开。BPE词表的学习发生在策略训练之前，不是部署时见到一段动作就重新学习。Hugging Face官方BPE教学

先用整数作为基本符号手算。训练语料是本例序列，再加$[8, 12, -4, -4, 0, 0, 0, 0]$与$[10, 14, -4, -4, 0, 0, 0, 0]$。程序按出现次数选合并对；次数相同时按展开后的整数序列排序。三个实际合并如下：

轮次	新符号代表的片段	本轮相邻对次数	第一条序列变成
1	$Z = (0, 0)$	9	$[6, 10, -4, -4, Z, Z]$
2	$R = (-4, -4)$	3	$[6, 10, R, Z, Z]$
3	$S = (R, Z) = (-4, -4, 0, 0)$	3	$[6, 10, S, Z]$

第一轮的9次来自每条四连零中三个相邻对，但替换时不重叠，所以每条四连零只变成两个Z。最后4个符号依次展开，仍精确得到原来的8个整数。词表可能因语料或平局选择而不同；BPE保存的是离散片段，不能把一个token的编号当作它代表的动作数值。

发布版FAST还要经过字符与字节层：先把整数减去固定最小值，转成字符，再交给字节级BPE。因此上表是合并思想的教学模型。配套程序另运行了固定版本的FAST+处理器和已发布词表。同一个Q实际编码为$[294, 308, 745, 258]$，解码后八个整数逐项相等。这四个编号与上表的符号不是同一词表。此例恰好也为4个token，不能据此推断所有动作都能压缩一半。固定版本处理器与词表

完整解码后，误差到底来自哪里

把token展开为Q，按原来的H, d_a恢复矩阵，再计算$\widehat{A} = D^\top(Q/s)$，得到

$$\widehat{A} \approx \begin{bmatrix} .038687 & .238687 \\ .191761 & .391761 \\ .408239 & .608239 \\ .561313 & .761313 \end{bmatrix}.$$

例如第一通道首项由$.5 \times .6 + .653281 \times (-.4) = .038687$得到；原来的高阶小系数已被舍成零，逆变换无法知道它们原本是多少。手写矩阵、SciPy的DCT和发布版FAST解码得到相同结果。

DCT正交，所以动作平方误差与系数平方误差相同：

$$\|\widehat{A} - A\|_F^2 = \|\widehat{C} - C\|_F^2, \qquad \|\widehat{A} - A\|_F \le \frac{\sqrt{Hd_a}}{2s}. \tag{1.11-6}$$

下标F表示把矩阵所有元素平方求和后开根号。右侧界限来自每个系数取整误差不超过$1/(2s)$，再累加Hd_a项。本例平方误差约0.0062584，单个动作元素最大绝对误差约0.0386874。这个界限假设只有取整、没有额外截断和越界裁剪；它也不能直接换算成抓取成功率。反归一化后，不同动作通道还对应不同物理尺度。

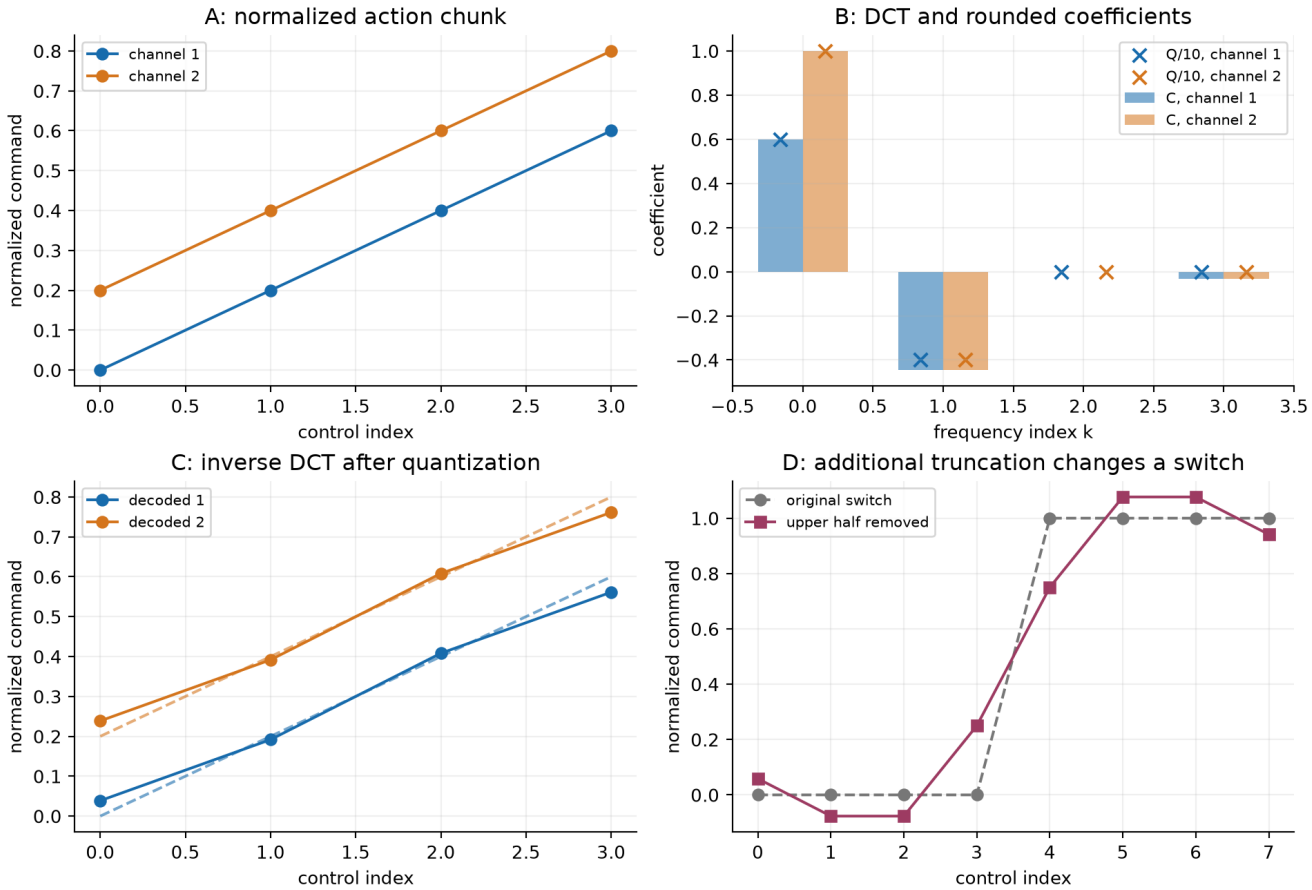


图1.11-3：A为两通道模拟动作；B中柱为原DCT系数C、叉号为$Q/10$；C比较原动作虚线与解码动作实线。D是另外做的高频截断实验：阶跃指令上半频率系数被主动删去后，出现提前变化与过冲，甚至超出原来的$[0, 1]$。D并不是FAST默认删除一半频率的实现。

接触瞬间、夹爪切换等事件可能依赖较快变化。动作“总体看起来接近”仍可能改变接触时序，因而应单独检查这些事件及闭环结果。BPE正常解码不会额外丢掉整数；真正需要检查的是归一化、取整、可能的截断、生成错误和部署解码约定。

分词器、策略训练与机器人执行各做什么

离线先用训练集确定归一化及BPE词表，或采用适配后的已发布词表。FAST本地编号还要通过固定映射接入VLA的词表位置；例如本地294并不天然就是VLA第294个输出类别。将映射后的示范动作编号记为$y_1, \ldots, y_L$，再由VLA序列协议约定动作开始标记BOS（Beginning of Sequence）与结束标记EOS（End of Sequence）。令$y_{L+1} = \mathrm{EOS}$，下一token目标为：

$$L_{\text{token}} = -\sum_{i=1}^{L+1} \log p_\theta(y_i \mid c, \mathrm{BOS}, y_{<i}). \tag{1.11-7}$$

训练的动作输入是$[\mathrm{BOS}, y_1, \ldots, y_L]$，对应标签是$[y_1, \ldots, y_L, \mathrm{EOS}]$：BOS位置预测第一个动作编号，最后一个动作编号的位置预测EOS。这种给出示范前缀的训练称为**教师强制**（teacher forcing）。因果掩码保证对应位置只能读取过去前缀。若训练接口已经在内部错开输入与标签，就不能再手动错开一次；否则预测位置与目标会相差两步。

交叉熵更新选定的VLA参数θ；固定DCT、取整和BPE只制作标签。部署时示范动作不存在，模型根据自己已生成的前缀继续预测，到EOS后还要检查展开的系数数量，再解码和反归一化。示范前缀变成生成前缀，也使离线分类准确率不能代替闭环检查。对应程序中的F1—F4，整体过程是：

```
F1 已归一化动作A → 沿时间做DCT，得到C
F2 Q = round(s C) → 按频率优先展开整数
F3 使用已固定BPE词表编码 → 得到动作token标签
训练：观测c + 示范前缀 → 下一token交叉熵 → 更新VLA
部署：观测c + 自己生成的前缀 → 生成至EOS
F4 token逆展开 → 核验Q与元数据 → Q/s → 逆DCT
反归一化、核对动作接口 → 控制器执行约定前缀
```

fast_steps.py用整数符号演示F1—F4与词表合并；fast_official.py运行真实发布版的字符/字节路径。两者均没有实施伪代码中的VLA训练或物理执行。

一个可变长token序列不能独自告诉控制器动作维度、控制频率、单位及归一化方式。本例的8个整数既能排成4×2，也能排成2×4；错误元数据甚至不会触发元素数量错误，却已把时间和通道混在一起。配套程序实际展示了这两种解码。

执行前还要校验展开后的系数数量，但数量正确只是必要条件。学习包的check_coefficient_count只演示这一项检查；合法动作编号、结束协议、字节完整性及元数据与已登记动作接口的匹配，还需另外核验。例如容错字节解码可能用替换字符填补不完整内容，字符数恰好正确，整数却已损坏。所核对的固定处理器还会对某些异常返回全零数组；返回了数组，不等于得到有效动作。全零是否表示保持、回零或其他命令，取决于动作定义，不能作为通用后备方案。

FAST+发布配置的$s = 10$、词表2048以及最小系数编号属于这份发布资产；输入仍需外部归一化。其代码还会把低于最小可表示整数的输入截到下界，超出支持范围时，式(1.11-6)便不再涵盖全部误差。新机器人需要核对归一化、时长和动作接口，之后才能谈复用词表。

压缩后的自回归序列可能比逐维分箱短，但仍需串行解码。作者在2025年FAST发布说明中同时报告了训练效率改善和当时π_0-FAST推理慢于原π_0流匹配版本。训练吞吐、输出序列长度和真实生成延迟应分别比较。作者对训练与推理的说明

1.11.4 VQ：学习哪些连续片段值得共用一个码

FAST采用固定时间变换再压缩；**向量量化**（Vector Quantization，VQ）则学习一组代表向量。编码器把动作片段变成$z = E_\phi(A)$，码本保存$e_1, \dots, e_K$，选取最近码$k = \arg\min_j \|z - e_j\|^2$，解码器$G_\psi(e_k)$恢复动作。策略随后可以学习预测码编号。一个码代表多少控制步由编码器设计决定，并不自动具有“抓住杯子”这样的技能语义。

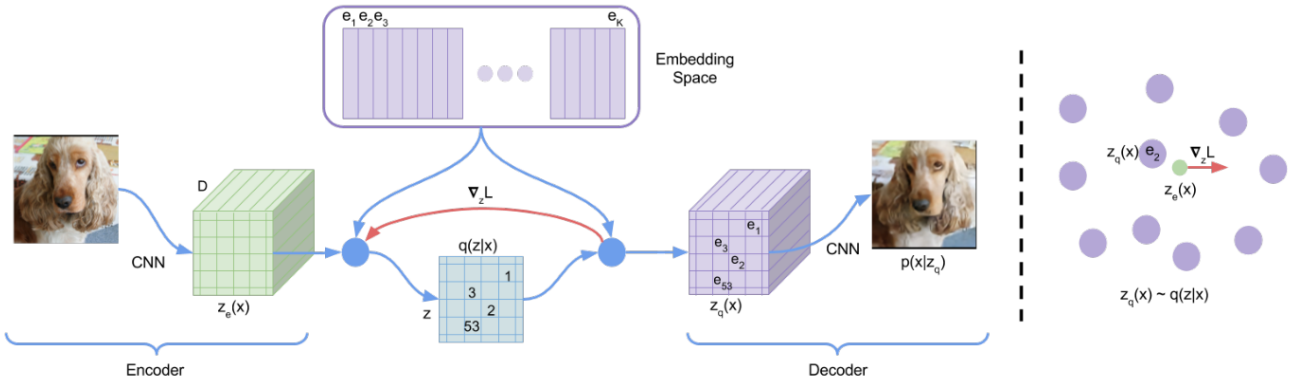


图1.11-4：VQ-VAE原论文Figure 1。Encoder、Decoder分别为编码器、解码器；Embedding Space是码本；原图$z_e(x)$对应本节z，$z_q(x)$是选中的码。CNN表示卷积网络，D表示嵌入维度，$q(z \mid x)$表示离散选码，$p(x \mid z_q)$表示由码重建输入。整数网格保存码编号；右侧紫点为码、绿点为编码输出，附近的e_2被选中。红色梯度箭头表示用于训练编码器的替代梯度。将图片输入替换为动作片段后，选码与重建的关系相同。VQ-VAE原论文

难处是：微调z时，只要仍在同一个最近码区域，选中的e_k完全不变；越过边界，输出又会跳变。直接沿离散选择求导，无法给编码器提供通常所需的重建反馈。**直通估计器**（Straight-Through Estimator，STE）为此规定一条替代反向路径：

$$z_{\text{st}} = z + \text{sg}(e_k - z). \tag{1.11-8}$$

sg前向保留数值，反向在该路径取零导数。因此前向读到e_k，反向对z使用单位导数，对e_k取零。这不是最近码选择的真实导数，而是训练时选择的近似。重建反馈可以借它改变编码器，下一轮前向再重新选码。

梯度式码本学习还要分配另外两项训练职责：

$$L = L_{\text{recon}}(G_\psi(z_{\text{st}}), A) + \|\text{sg}(z) - e_k\|^2 + \beta\|z - \text{sg}(e_k)\|^2. \tag{1.11-9}$$

重建项训练解码器，并经直通路径训练编码器。第二项只把所选码拉向当前编码输出；第三项只让编码输出靠近所选码，称为**承诺损失**（commitment loss）。码本也可用被分配向量的移动平均更新，届时需相应调整码本更新方法，而不能把两套规则无条件叠加。Keras官方VQ教学与实现

取$z=.8, e_k=1$，匹配误差都为.04。码本项对e_k的梯度为.4，下降时让码靠近.8；承诺项对z的梯度为$-.4\beta$，下降时让编码输出靠近1。二者数值相同，直接训练的对象不同。若编码器有参数ϕ，这还只是中间梯度，最终需乘$\partial z/\partial\phi$；码本训练也不会直接等于机器人控制器的更新。

若数据长期只选择少数码，其余码缺少有效更新，码本容量就没有被充分利用。应同时查看码使用分布、动作重建、关键接触事件和闭环成功率。重建漂亮的片段编码器，也不保证上层策略能在正确场景下选对码。

1.11.5 一致性模型：直接学习一条生成轨迹的端点

预测局部方向，还是预测沿它走到底的结果

扩散或流匹配的方向模型回答：“当前候选应该往哪里移动一点？”要走到动作分布，通常需反复调用。**一致性模型**（Consistency Model，CM）改为回答：“当前候选所在的生成轨迹，最终通向哪里？”如果这个映射已经学好，就有机会从高噪声状态一次预测近干净端点。原论文

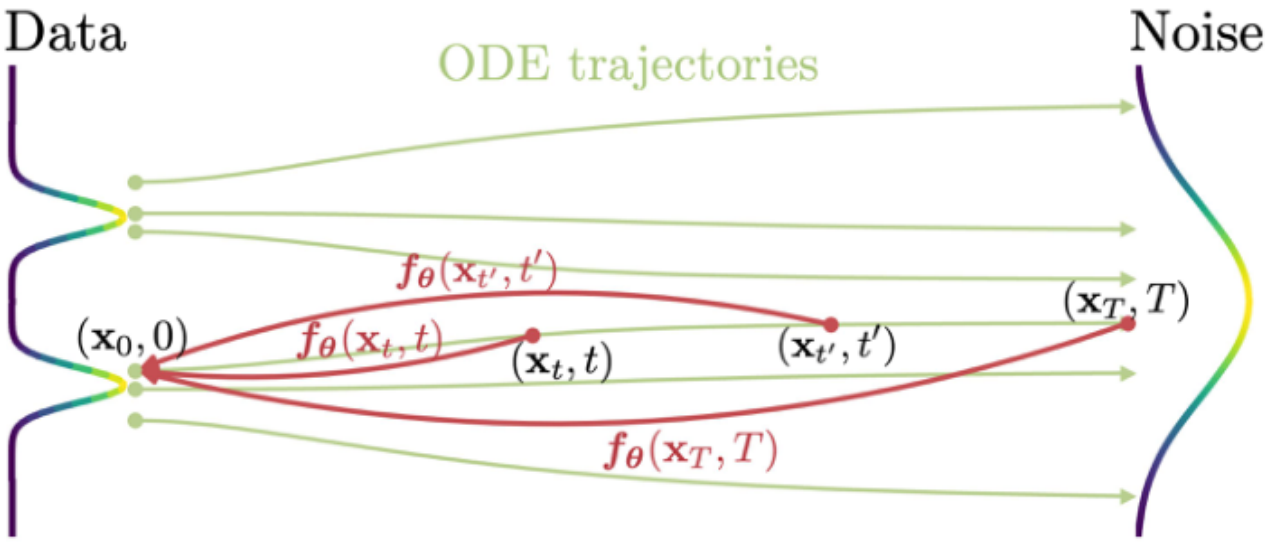


图1.11-5：Consistency Models论文Figure 2。Data与Noise分别是数据与噪声分布；绿色ODE trajectories是概率流常微分方程的不同轨迹；红色f_θ将同一轨迹上的不同点映射到同一个端点。原图的t, t', T表示噪声时刻，本节改用σ，与机器人控制时间分开。不同绿色轨迹通常有不同端点，不能把所有输入都送到同一个动作。

概率流常微分方程（Probability Flow Ordinary Differential Equation，PF ODE）是与加噪过程相配的一种确定性输运：它的各时刻边缘分布与对应加噪分布一致，但用一条确定性轨迹连接这些分布。先选定初始噪声，方向场就确定了后续轨迹；换一个噪声，可能走向另一个动作。PF轨迹也不等于某个示范动作逐次加噪时保存的那条随机历史。

记轨迹在噪声尺度σ上的点为x_σ，在很小正数ε处终止。一致性函数应满足

$$f^*(x_\sigma, \sigma, c) = x_\varepsilon, \qquad f^*(x_{\sigma'}, \sigma', c) = x_\varepsilon, \tag{1.11-10}$$

其中两个输入必须对应相同条件c、相同轨迹。输出与完整动作块同形状，代表端点，不是速度，也不是一步控制反馈。边界条件是$f^*(x_\varepsilon, \varepsilon, c) = x_\varepsilon$；当$\varepsilon > 0$时，它是近干净端点，不必等于当初某一条示范的原动作。

用一个可解的分布看清轨迹与端点

固定任务条件，用一个不设物理上下限的标量模拟动作：干净分布为$A_0 \sim \mathcal{N}(\mu, \delta^2)$，加独立标准正态噪声$\eta$得到$x_\sigma = A_0 + \sigma\eta$。其边缘分布为$\mathcal{N}(\mu, \delta^2 + \sigma^2)$，score为

$$s_\sigma(x) = -\frac{x-\mu}{\delta^2+\sigma^2}.$$

在这个加噪约定下，PF ODE为

$$\frac{dx_\sigma}{d\sigma} = -\sigma s_\sigma(x_\sigma) = \frac{\sigma}{\delta^2+\sigma^2}(x_\sigma - \mu). \tag{1.11-11}$$

直接求导可以验证

$$x_\sigma = \mu + \sqrt{\delta^2+\sigma^2}\, z \tag{1.11-12}$$

满足该方程，z沿轨迹保持不变。若$z \sim \mathcal{N}(0,1)$，它也正好给出所需的各尺度边缘分布。随着σ降低，偏离均值的距离收缩；不同z仍保留不同端点，并没有全部变成均值。

从任意输入求出$z = (x_\sigma - \mu)/\sqrt{\delta^2+\sigma^2}$，代回$\varepsilon$处，得到完整端点映射：

$$f^*(x,\sigma) = \mu + \sqrt{\frac{\delta^2+\varepsilon^2}{\delta^2+\sigma^2}}\,(x-\mu). \tag{1.11-13}$$

取$\mu = 1, \delta = .5, \varepsilon = .05$，从$\sigma_h = 1, x_h = 1.5$出发，$z = .5/\sqrt{1.25} \approx .447214$。在$\sigma_l = .5$处，

$$x_l = 1 + \sqrt{.5} \times .447214 \approx 1.316228.$$

分别从两点计算端点，均为1.224722。同一轨迹上的$\sigma = .75, .25, .05$处，状态依次约为$1.403113, 1.25, 1.224722$，代入式(1.11-13)也得到同一结果。

这个端点不是“给定带噪输入后，原动作的条件均值”。同一高噪声输入$x_h = 1.5$下，高斯后验均值是

$$\mathbb{E}[A_0 \mid x_h] = 1 + \frac{.25}{1.25}(1.5-1) = 1.1.$$

它与1.224722不同。后验均值是在所有可能原动作中做平方误差最优预测；PF端点则由选定的输运轨迹确定，用来把一整份输入分布送到端点分布。二者服务于不同建模任务。不能把一个普通单步均方去噪器直接当作已经学好的端点生成器。

教师给相邻状态，学生与目标网络给端点预测

实际分布通常没有式(1.11-13)，可以借助已经训练的扩散方向场。**一致性蒸馏**（Consistency Distillation，CD）中有三个不同对象：

对象	输入与输出	本次反向中的职责
固定教师方向场d_ϕ	当前状态、噪声尺度、条件→局部方向	通过求解器产生相邻状态，不更新教师参数
在线学生f_θ	较高噪声状态与条件→端点预测	接收损失梯度，由优化器更新
目标网络$f_{\bar\theta}$	较低噪声状态与条件→端点参照	本次停止梯度，之后按规则更新参数

先采一个较高噪声状态x_h，让教师求解器从σ_h推进到σ_l，得到$\widetilde{x}_l$。再比较在线学生在高处的端点预测与目标网络在低处的预测：

$$\widetilde{x}_l = \mathrm{Solver}(x_h, \sigma_h \to \sigma_l; d_\phi), \quad L_{\mathrm{CD}} = \frac{1}{2}\left\| f_\theta(x_h, \sigma_h, c) - \mathrm{sg}\left[f_{\bar\theta}(\widetilde{x}_l, \sigma_l, c)\right]\right\|^2. \tag{1.11-14}$$

这里用平方距离说明训练，实际方法还会选择时间采样、权重和距离函数。教师推进得到的是另一个带噪状态；端点参照来自目标网络，不是教师这一步直接给出了准确干净动作。初期目标也可能不准，需要边界与跨尺度关系逐步建立约束。

令预测$y = f_\theta(x_h, \sigma_h, c)$、目标$q = f_{\bar\theta}(\widetilde{x}_l, \sigma_l, c)$，单样本梯度是$J_\theta^\top(y - q)$。停止梯度意味着这个损失不通过$q$反向改变目标网络，也不反向改变用于造相邻点的教师。使用**指数移动平均**（Exponential Moving Average，EMA）时，先更新在线参数，再执行

$$\bar\theta \leftarrow \rho\bar\theta + (1-\rho)\theta. \tag{1.11-15}$$

EMA让参照随学生较缓慢变化，它是参数赋值规则，不是对目标分支做梯度下降。降低相邻尺度的预测差，有助于学习端点关系；但两边一起输出错误常量，也能使这个差为零，所以还需要下面的边界约束。

边界怎样约束常量解，为什么还不能替代训练

如果所有输入都输出固定b，式(1.11-14)的差当然为零。但在最低噪声端，模型应分别返回不同的x_ε。边界使“所有轨迹都返回b”与要求冲突。常用结构直接保证这一点：

$$f_\theta(x, \sigma) = c_{\mathrm{skip}}(\sigma)x + c_{\mathrm{out}}(\sigma)F_\theta(c_{\mathrm{in}}(\sigma)x, \log\sigma), \tag{1.11-16}$$

$$c_{\mathrm{skip}} = \frac{\delta^2}{(\sigma-\varepsilon)^2+\delta^2}, \quad c_{\mathrm{out}} = \frac{\delta(\sigma-\varepsilon)}{\sqrt{\sigma^2+\delta^2}}, \quad c_{\mathrm{in}} = \frac{1}{\sqrt{\sigma^2+\delta^2}}.$$

$\sigma = \varepsilon$时，$c_{\mathrm{skip}} = 1, c_{\mathrm{out}} = 0$，所以任意网络权重都给出$f_\theta(x, \varepsilon) = x$。这里$\delta$是预设的数据尺度，调节跳接与网络输出的相对大小；这些系数是固定作者实现采用的一组选择，满足边界的结构并不唯一。

固定版本的边界与训练实现

为什么只比较相邻点，有机会学会远处的端点？理想情况下，若同一轨迹上的每条相邻关系都准确，并一直连到边界，就能逐段相接：

$$f(x_{\sigma_3}, \sigma_3) = f(x_{\sigma_2}, \sigma_2) = f(x_{\sigma_1}, \sigma_1) = f(x_\varepsilon, \varepsilon) = x_\varepsilon.$$

边界为不同轨迹提供不同端点，训练再把关系延伸到高噪声处。若完全没有训练到连接边界的尺度，模型仍可能在已采样区域近似输出常量，只在未覆盖的窄区间突然变化。硬边界排除了全局常量函数，却没有保证数据覆盖、拟合和优化都已经正确；EMA同样不能单独给出这种保证。

教师求解误差与无教师训练的区别

回到$x_h = 1.5, \sigma_h = 1$。式(1.11-11)在这里给出方向.4；一个Euler步到$\sigma_l = .5$，得到

$$\widetilde{x}_l = 1.5 + (.5 - 1) \times .4 = 1.3.$$

精确相邻点为1.316228，两者不同。即使后续使用完美端点函数，输入1.3得到的端点也是1.213190，而非原轨迹的1.224722。学生可能很准确地拟合一个带有求解误差的参照。缩小步长、改善求解器与降低学生拟合误差，解决的是不同问题；原作者实现的教师CD采用Heun推进，本节的Euler只用于分开观察误差来源。

也可以不预训练教师方向场，采用**一致性训练**（Consistency Training，CT）。原始路线使用同一个干净样本A_0与同一个噪声η，构造$A_0 + \sigma_h\eta$、$A_0 + \sigma_l\eta$两点，再建立一致性训练关系。

这两个点一般不严格属于边缘分布PF ODE的同一轨迹。例如$A_0 = 1.3, \eta = .4$，两点为1.7与1.5；高斯PF从1.7精确推进到$\sigma = .5$则得到1.442719。

那共享噪声为什么仍能用来训练？对于这类加性高斯路径，理想PF方向等于给定带噪输入后的噪声条件平均：

$$v_\sigma(x) = \mathbb{E}[\eta \mid X_\sigma = x] = \mathbb{E}\left[\frac{x - A_0}{\sigma} \,\middle|\, X_\sigma = x\right].$$

一对示范与噪声提供的是这个平均方向的一次样本估计。把它用作Euler方向，低尺度状态恰好变成

$$x_h + (\sigma_l - \sigma_h)\frac{x_h - A_0}{\sigma_h} = A_0 + \sigma_l\eta.$$

这就把无教师配对与局部方向估计接起来了。不过，样本方向与其条件平均一般不同；目标网络又是非线性的，所以方向估计的无偏性并不直接变成端点标签的逐样本精确性。CT与CD的损失联系还依赖离散尺度趋细和相应光滑性等条件。共享加噪样本提供了训练构造，不能把这两点直接称为精确PF轨迹。

一次真实更新里，哪些数进入了梯度

配套实验用上面的解析高斯方向场作为教师。学生读取缩放后x和$\log\sigma$，经过两个32维tanh隐藏层输出标量，外面包式(1.11-16)。在线与目标网络从相同参数数值开始，保存为两个独立对象；目标采用$\rho = .95$的EMA。为观察学生学习，训练相邻点使用精确高斯推进，Euler误差另算，不混入标签。

首个batch有64个样本，损失为$\frac{1}{2\times 64}\sum_i (y_i - q_i)^2$。其中第一个样本实际经过：

中间量	数值约为
σ_h, σ_l	$1,\ .88125$
高噪声输入x_h	$.265042$

中间量	数值约为
教师精确相邻点$\widetilde{x}_l$	$.333947$
$c_{\text{skip}}, c_{\text{out}}$	$.216920,\ .424853$
网络F_θ的输出	$.089123$
在线端点预测y	$.095357$
目标端点预测q	$.116624$
解析正确端点	$.669677$

在线输出确实由$.216920 \times .265042 + .424853 \times .089123$得到。目标只是当前较低尺度的网络输出，此时离正确端点还很远；训练并没有偷偷把$.669677$作为本样本标签。

这一个样本对y的梯度为$(y-q)/64 \approx -.000332301$。若$F_\theta$最后一层某个权重连接的隐藏特征为$h_0 = -.156400$，其梯度贡献是

$$\frac{\partial L_i}{\partial W_0} = \frac{y-q}{64}\, c_{\text{out}} h_0 \approx 2.20804 \times 10^{-5}.$$

同一个权重由64个样本共享，所以还要把所有贡献相加；本batch实际总梯度约为$.004664193$。程序逐项累加与自动求导相符。其他层继续按链式法则得到梯度；Adam更新在线权重，再用式(1.11-15)更新目标。目标网络在反向后没有梯度张量。

已完成更新次数	本次在线梯度范数	一个首层权重更新前→更新后
0	$.072174$	$.336533 \to .334533$
1	$.059664$	$.334533 \to .332577$
2	$.029300$	$.332577 \to .330841$

这个小模型实际完成2000次更新。固定检查网格上，从$\sigma = 1$的41个状态单次预测端点，相对解析端点的均方误差从$.677339$降为约4.79×10^{-6}。沿同一条测试轨迹，输入$1.5, 1.403113, 1.316228, 1.25$得到的端点预测约为$1.223543, 1.224968, 1.223991, 1.224259$；最低尺度由结构精确返回$1.224722$。

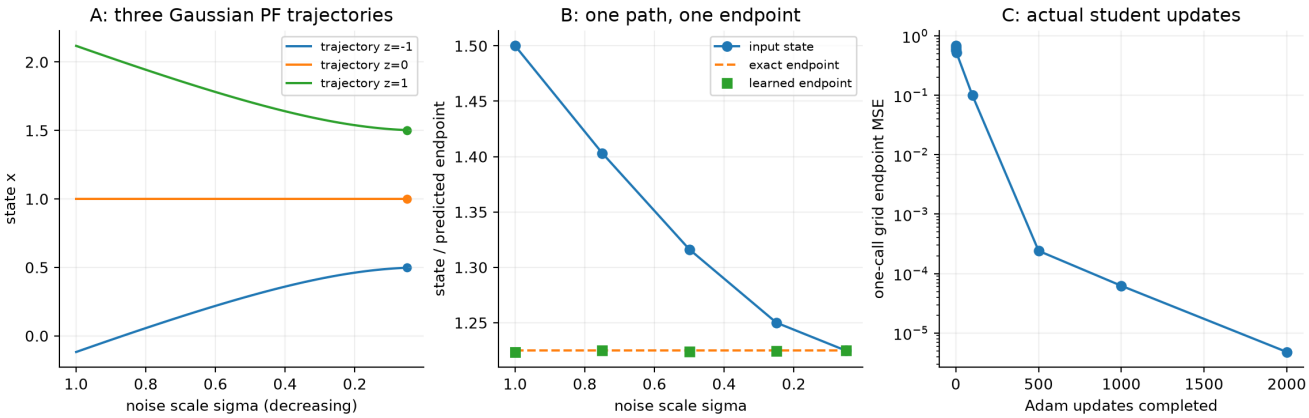


图1.11-6：A中固定z标识不同高斯PF轨迹，横轴为逐渐降低的噪声尺度；B比较同一轨迹的输入状态、解析端点与学生端点；C是实际更新后的单次端点预测误差，MSE为均方误差（Mean Squared Error）。这是一维高斯教学实验，不是图像或机器人策略复现，也未验证多峰动作覆盖。

训练中的高噪声状态按已知的精确带噪边缘分布采样。上述误差检查则固定$\sigma = 1$，使用标准化坐标$z \in [-2, 2]$上的41个等距网格点，MSE是这些点的等权平均，不是高斯加权期望误差，也没有覆盖全部尾部。

若单独讨论整体分布输运，正确初始边缘$\mathcal{N}(1, 1.25)$经解析映射得到$\mathcal{N}(1, .2525)$；比干净分布多出的$.0025$是端点噪声方差ε^2。若把初始分布直接换成$\mathcal{N}(0, 1)$，映射再准确也会得到另一输出分布。实际生成通常选择足够大的末端噪声，使带噪数据可由简单先验近似；这个近似与学生端点拟合是两件事。

从训练过程对应到代码和机器人

训练的关键顺序可以写成：

```
C1 取任务条件c、较高噪声状态x_h与相邻的两个噪声尺度
C2 固定教师将x_h推进到x_l；此处不更新教师
C3 在线学生预测y；目标网络预测q，停止q的反向路径
C4 对y与q计算距离，只反向计算在线参数梯度
C5 优化器更新在线参数
C6 按EMA规则更新目标参数
```

配套的consistency_gaussian.py在相同注释位置执行：

```
# C2：教师输出相邻状态；目标输出端点参照。
with torch.no_grad():
    x_low = advance(x_high, high, low)
    reference = target(x_low, low)
# C3—C5：本次只更新在线学生。
prediction = model(x_high, high)
loss = 0.5 * (prediction - reference).square().mean()
optimizer.zero_grad(set_to_none=True)
loss.backward()
optimizer.step()
# C6：目标参数由在线参数移动平均得到，不是反向传播。
with torch.no_grad():
    for target_parameter, parameter in zip(target.parameters(), model.parameters()):
        target_parameter.mul_(0.95).add_(parameter, alpha=0.05)
```

部署时不再运行这段训练循环，也不需要为每个动作调用教师。读入当前观测c、采一个高噪声动作候选，再调用学生得到整个端点动作块；之后仍需解码、反归一化、检查动作接口，并由控制器执行约定前缀。模型内部一次生成，与机器人只执行一步、执行整块，或能否应对新接触反馈没有等号。

一致性模型也可以多步采样：先预测端点，再按采样算法加回适量噪声并再次预测，以计算换取质量。它不同于沿同一方向场直接积分。少步是否足够，需要在成功率、模式覆盖、接触精度和失败恢复上检验。

1.11.6 其他蒸馏与归一化流各解决什么

蒸馏（distillation）泛指把教师的预测或中间计算转成学生监督，并不只指一致性训练。以确定性ODE教师为例，给定初始噪声z和条件c，让教师多步生成完整动作$A_\phi(z, c)$，再训练学生$G_\theta(z, c)$拟合它。若教师还有其他独立采样噪声，需要一并固定、作为学生输入，或另行定义学生要拟合的分布。昂贵的教师计算转移到造训练样本时，学生部署时可以少算。

噪声输入在这里保留了“同一条件下选择哪条行为”的信息。如果删掉z，只训练$G_\theta(c)$并用平方误差拟合多个教师动作，仍可能回到第1.7节的平均动作问题。学生容量、教师错误、样本覆盖和执行误差也不会因为用了蒸馏就消失。

少步方案至少有四类：保持旧模型、直接减少采样索引；保持方向场、换更好的求解器；重新训练端点或大跳跃目标；用教师蒸馏学生。前两类主要改变数值求解，后两类改变模型所学的任务。它们应分别报告原模型训练成本、新学生训练成本与部署计算。

另一个常见术语是**归一化流**（Normalizing Flow）：用可逆变换把简单变量z变成动作$A = g_\theta(z, c)$，并用体积变化计算密度。为看清密度为什么要修正，取$A = 2z + 1$。z附近宽.01的一小段，被变成宽.02的一小段；概率质量不变，单位长度的密度便减半。

$$\log p_A(A \mid c) = \log p_Z(g_\theta^{-1}(A, c)) - \log \left| \det \frac{\partial g_\theta}{\partial z} \right|. \tag{1.11-17}$$

雅可比矩阵$\partial g_\theta/\partial z$记录各输出对各输入的局部变化，行列式绝对值给出体积伸缩。标准正态z经$A = 2z + 1$后，在$A = 1$处的密度为$(1/\sqrt{2\pi})/2 \approx .199471$。减去$\log 2$使扩大后的空间仍分到正确概率质量。归一化流原理综述

为便于训练和求密度，离散归一化流会专门选择容易求逆和计算行列式的结构。连续时间流在满足解的存在唯一性等条件时，也可通过逆向ODE和散度累计密度变化，但这需要额外计算及数值误差控制。Flow Matching训练出的方向场可以用于这种密度计算，普通动作采样接口却通常只返回样本。若部署采用粗步离散求解或额外后处理，还要核对所算密度是否对应实际部署分布。

这对后训练有实际影响：PPO这类算法需要新旧策略对同一动作的概率比，就必须明确概率来自哪里。离散token序列概率、量化动作的概率质量和连续动作密度不是同一对象；生成动作的接口与计算相应概率的接口都要定义。后面的强化学习章节再把这些对象接到具体损失。

1.11.7 把表示、生成与执行成本放在一起比较

路线	主要训练对象	部署的主要计算	首要检查
确定性回归	动作预测靠近示范	一次输出	条件下多种动作会否被平均
高斯或混合密度	分布参数的负对数似然	选成分、采样或取代表值	分布形式能否表达行为
CVAE	动作重建与潜变量先验匹配	给定潜变量解码	先验与训练后验、潜变量使用
分箱或FAST自回归	动作token交叉熵	串行预测token，再解码	表示精度、有效长度与解码正确性
扩散	噪声、score或干净量	多次方向预测与采样更新	去噪误差、采样误差与动作覆盖
Flow Matching	路径速度	多次场预测与积分	路径、求解步数及终点分布
一致性模型	同轨迹端点关系	一次或少数端点预测	边界、参照误差与少步质量

连续动作块的一次场预测可以并行处理整块，离散自回归则依赖前一个token；但一次大网络计算与一次小专家计算成本也不同。比较需要固定任务、数据、观测模态、动作接口、执行前缀、控制周期和评价拆分。若某方法多读腕部相机，或多生成候选后用强评价器挑选，额外信息与计算也要计入。

还有其他生成路线。能量模型用条件和候选动作产生能量，再通过训练塑造相对偏好，部署需要搜索或采样；能量本身不等于任务回报。**生成对抗网络**（Generative Adversarial Network，GAN）通过真假判别器的反馈训练生成器；它的训练反馈与扩散的噪声监督不同，需要另查稳定性和模式覆盖。它们不是本节的实现主线，但说明“动作生成”并不只存在表中的一种组织方式。

窄任务中，条件已经明确区分了正确动作，简单回归可能够用；同一条件下需要保留多条绕行路线时，才需要能够表达这种选择的分布或额外模式条件。表示压缩、条件引导与少步生成都可能有用，取舍依据应是任务中必须保留的行为和可用计算，而不是某个名字看上去更新。

1.11.8 练习题

题1.11-1：条件引导的概率含义与边界。 设始终保留机器人观测r，只丢弃文字ℓ。推导$s_{\text{cond}} - s_{\text{base}}$与$p_\sigma(\ell \mid x, r)$的关系，并解释$w = 0, 1, 2$。在正文高斯例中求$w = 2$的均值、方差，以及$x = .3$处的引导score。为什么不能据此声称最终采样动作必然服从同样形式的干净分布，或保证更安全？给出训练条件丢弃与部署预测的伪代码。

题1.11-2：压缩与量化误差。 对本节4×2动作块，采用$s = 20$重新计算Q和逆DCT动作。与$s = 10$比较最大元素误差及平方误差，并推导式(1.11-6)。如果有越界裁剪、提前截断token序列或元数据错误，这个误差界还能否使用？DCT、取整、BPE和逆变换分别在哪一步保留或损失信息？

题1.11-3：从分词器走到VLA训练。 已发布词表被冻结，示范动作编码为$[294, 308, 745, 258]$。说明BPE词表学习、VLA训练、VLA推理、物理执行的输入输出及更新对象；写出带结束标记的下一token训练与生成伪代码，说明损失位置怎样与标签对齐。为什么给解码器错误的$H = 2, d_a = 4$仍可能返回数组？设计有效解码检查，区分能检查出的错误与只能靠动作接口约定发现的错误。

题1.11-4：VQ的梯度分工。 取$z = .8, e = 1, \beta = .25$，解码器为恒等映射，标签为1.2。使用式(1.11-8)、(1.11-9)，重建损失取$(z_{\text{st}} - 1.2)^2$。分别计算三个损失的数值、对z和e的梯度及总梯度；再做学习率.1的一次普通梯度下降。说明真实编码器、码本、解码器分别如何更新，以及STE为什么不是最近码选择的真实导数。

题1.11-5：PF端点与单步去噪的差别。 对$\mu = 1, \delta = .5, \varepsilon = .05$的高斯例，推导式(1.11-11)—(1.11-13)，复算$x_h = 1.5$从$\sigma_h = 1$到$\sigma_l = .5$的精确状态、Euler状态及各自端点。求$\mathbb{E}[A_0 \mid x_h]$，解释它为什么不同于PF端点。若输入初始分布从$\mathcal{N}(1, 1.25)$改为$\mathcal{N}(0, 1)$，即使使用解析端点，输出的均值与方差怎样变化？

题1.11-6：一致性训练信号从哪里来。 按表中首个样本，复算y、$(y - q)/64$和输出权重的单样本梯度贡献；解释为什么总梯度还需跨样本相加。画出教师、在线学生、目标网络的依赖关系，说明哪些由反向更新、哪些由EMA更新。证明固定常量解与边界冲突，同时给出“有硬边界、采样区间内却仍近似坍缩”的可能情形。为什么配对损失小不能代替端点误差检查？

题1.11-7：CD、CT和少步执行。 复算$A_0 = 1.3, \eta = .4$时CT的两个共享噪声输入，并与高斯PF的精确相邻点比较。解释CT的配对为什么不等于精确PF轨迹；列出固定教师、目标EMA网络与无教师CT之间的区别。若将一个机器人策略从10轮场预测换成1轮端点预测，设计能区分计算收益、近似误差与闭环效果的评价，不只比较离线重建。

题1.11-8：为什么生成动作不等于拿到策略密度。 对$A = 2z + 1, z \sim \mathcal{N}(0, 1)$推导动作密度，并计算$A = 1$处的值。随后比较可逆流、FAST token序列与通常的多步Flow Matching动作采样器：各自直接提供什么概率对象，要为使用动作概率比的后训练补充什么？若控制器再把动作饱和裁剪到上下限，可逆换元公式还能否直接沿用？

1.11.9 参考答案

题1.11-1

两条score必须针对同一个x,σ,r。由$p_\sigma(x\mid r,\ell)=p_\sigma(\ell\mid x,r)p_\sigma(x\mid r)/p_\sigma(\ell\mid r)$，取对数再对$x$求导，分母与$x$无关，因此

$$s_{\text{cond}}-s_{\text{base}}=\nabla_x\log p_\sigma(\ell\mid x,r).$$

$w=0$保留无文字但有观测的基础分支，$w=1$为普通条件分支，$w=2$为$2s_{\text{cond}}-s_{\text{base}}$。若其他资料写成$s_{\text{cond}}+\gamma(s_{\text{cond}}-s_{\text{base}})$，则本节$w=1+\gamma$。

高斯例中$s_{\text{base}}=-x$、$s_{\text{cond}}=4-4x$，所以$s_w=4w-(1+3w)x$。$w=2$给出$8-7x$，对应均值$8/7$、方差$1/7$，在$x=.3$处为5.9。配方时二次项是$-\frac{7}{2}(x-\frac{8}{7})^2$，与高斯形式$-(x-m)^2/(2v)$比较即可读出方差。

固定层的幂次密度不一定组成同一前向加噪过程的各尺度边缘。一个具体反例是：把本例两个高斯暂视为干净密度，先分别加方差1的噪声，再做$w=2$倾斜，会得到$\mathcal{N}(16/11,10/11)$；先在干净层倾斜，再加同样噪声，则为$\mathcal{N}(8/7,8/7)$。两者不相同。因此逐层score等式本身不足以确定最终干净采样分布，还需要跨层演化、初始分布和求解过程。条件偏好也没有编码全部碰撞、接触和动作约束，不能推出安全保证。

为明确预测对象，下面采用加性高斯路径和噪声预测器。训练中丢弃事件与样本内容独立，始终保留r：

```
训练：
  取示范(r, ell, A0)，采sigma与标准噪声eta
  x = A0 + sigma * eta
  ell_in = NULL_TEXT (独立以p_drop概率) 或ell
  eta_hat = model_theta(x, sigma, r, ell_in)
  用mean((eta_hat-eta)^2)反向更新theta
部署：
  读取r与ell，按约定末端先验采x
  对递减噪声网格的每个(sigma, sigma_next)：
      e0 = model_theta(x, sigma, r, NULL_TEXT)
      ec = model_theta(x, sigma, r, ell)
      eg = e0 + w * (ec-e0)
      x = x + (sigma_next-sigma) * eg
  按采样器约定取最终状态或做终点估计，再反归一化并执行
```

这里理想噪声预测满足$e=-\sigma s$，该路径PF ODE的$dx/d\sigma=-\sigma s=e$，所以下降σ的Euler更新有上述形式。换噪声参数化或求解器，要相应换更新公式。模型、条件丢弃接口和输出语义必须在训练与部署间对应；把两条预测并成一个batch也仍需计算两组预测。

题1.11-2

$s=20$时，原一阶系数乘20约为-8.921770，三阶系数乘20约为$-.634051$。取整后

$$Q_{20}=\begin{bmatrix}12 & 20\\ -9 & -9\\ 0 & 0\\ -1 & -1\end{bmatrix},\qquad \widehat{C}_{20}=\begin{bmatrix}.6 & 1\\ -.45 & -.45\\ 0 & 0\\ -.05 & -.05\end{bmatrix}.$$

与$s=10$不同，三阶项现在保留下来。例如第一通道首项为
$.5(.6)+.653281(-.45)+.270598(-.05)\approx -.007507$；第二项为
$.5(.6)+.270598(-.45)-.653281(-.05)\approx .210895$。完整逆变换是

$$\widehat{A}_{20}\approx\begin{bmatrix}-.007506570 & .192493430\\ .210894952 & .410894952\\ .389105048 & .589105048\\ .607506570 & .807506570\end{bmatrix}.$$

尺度	最大元素绝对误差	全块平方误差	全块误差范数	范数上界
10	.038687407	.006258401	.079110055	.141421356
20	.010894952	.000700194	.026461183	.070710678

平方误差是八项之和，若要求MSE才再除以8。令$E_C=Q/s-C$，利用$DD^{\top}=I$:

$$\|\widehat{A}-A\|_F^2=\|D^{\top}E_C\|_F^2=\operatorname{tr}(E_C^{\top}DD^{\top}E_C)=\|E_C\|_F^2\le\frac{Hd_a}{4s^2}.$$

开根号得到式(1.11-6)。它约束整块误差，不能解释成每个逆变换动作元素都小于半个系数档宽。这里$s=20$的网格包含$s=10$的网格，最近点量化的系数总平方误差不会增加；但逆变换后，各系数误差可能相加或抵消，单个最大元素误差在一般例子中未必单调下降。

DCT及固定排列保留信息，取整把一段实数范围合到一个整数，损失连续精度。完整BPE及逆展开保留这些整数，逆DCT只能恢复量化后的近似。越界裁剪使系数误差可能超过$1/(2s)$；截断token可能导致缺系数，补零也不是最近点取整；错误H,d_a、顺序或尺度则改变了逆变换。因此这些情形不能直接套用该界。若比较原始物理动作，还必须把归一化裁剪和各通道反归一化尺度计入。

题1.11-3

四个阶段不能共用一个“训练模型”的说法：

阶段	输入→输出	改变的对象
BPE拟合	训练动作的量化序列→合并规则和词表	统计规则；可能另确定归一化与整数映射

阶段	输入→输出	改变的对象
VLA训练	可用观测c、示范前缀→下一动作token及EOS	选定的VLA参数，固定分词器不反向更新
VLA推理	当前观测、已生成前缀→后续token	本次前缀与缓存，网络权重不变
物理执行	有效编码及动作接口→控制命令与新反馈	机器人状态改变，执行本身不是梯度更新

$[294, 308, 745, 258]$先是FAST本地编号。令g为接入VLA词表的一一映射，$a_i = g(u_i)$；BOS与EOS由VLA协议另设。不能把它们当作已发布FAST词表必然自带的特殊项。

```
训练：
  u = frozen_FAST.encode(normalize(A_demo))
  a = [g(id) for id in u]
  action_input  = [BOS, a1, a2, a3, a4]
  action_target = [a1,  a2, a3, a4, EOS]
  logits = VLA(c, action_input, causal_mask=True)
  loss = CE(logits, action_target, 屏蔽padding)
  反向更新指定VLA参数
生成：
  prefix = [BOS]; generated = []
  根据c与prefix预测下一个合法动作编号或EOS
  若为动作编号：记入generated并接到prefix，再预测
  若为EOS：结束；若预算耗尽仍未结束：返回无效序列
  用g的逆映射取回FAST编号，再做解码与接口检查
```

BOS位置的logit监督a_1，a_1位置监督a_2，依次到a_4位置监督EOS。若使用内部自动错位的训练接口，传入同长度序列与标签，只对动作目标/EOS启用损失；由该接口执行一次错位，不能再提前手动错位。预测a_3时只能读到示范a_1, a_2及c，因果掩码阻止读取未来动作。

八个恢复整数既可按4×2解释，也可按2×4解释。后一种得到两行相同的$[.424264, .707107, -.282843, -.282843]$；形状、有限数值和元素个数均可通过，却已不是原来的四步双通道动作。

因此，先把元数据与独立登记的机器人动作接口核对，再做以下检查：

```
核验结束协议与合法动作ID；去掉协议标记并逆映射到FAST词表
严格展开BPE与字节，拒绝未知ID、不完整字节和静默替换
恢复整数，核验数量等于H * action_dim
使用登记的尺度、顺序、DCT约定恢复矩阵与动作
核验有限值、反归一化、单位、通道顺序、控制周期及适用约束
任一步不匹配就显式返回错误
```

这些检查能发现协议、字节、数量和接口不匹配，不能仅凭token推断机器人原本想用什么单位和形状，也不能保证格式正确的动作适合当前接触。固定词表中的孤立字节token若被容错替换，甚至可能保留正确字符数，所以数量检查不能替代字节校验。正文的公开helper只演示数量检查；完整接口方案是

本题要求的设计。若另外要求规范BPE分段，还需明确该规则，因为不同token分段可能展开成同一整数串。

题1.11-4

前向$z_{\text{st}} = .8 + (1 - .8) = 1$，规定的反向导数为$\partial z_{\text{st}}/\partial z = 1$、$\partial z_{\text{st}}/\partial e = 0$。逐项计算：

项	损失值	对z的梯度	对e的梯度
重建$(z_{\text{st}} - 1.2)^2$	$.04$	$2(1 - 1.2) = -.4$	0
码本$(\text{sg}(z) - e)^2$	$.04$	0	$2(1 - .8) = .4$
承诺$.25(z - \text{sg}(e))^2$	$.01$	$2(.25)(.8 - 1) = -.1$	0
合计	$.09$	$-.5$	$.4$

以更新前梯度同时更新，$z^+ = .8 - .1(-.5) = .85$、$e^+ = 1 - .1(.4) = .96$。若仍选该码，下一次重建项为$(.96 - 1.2)^2 = .0576$，反而比原来的$.04$大；码本与承诺项合起来下降更多，总损失为$.072725$。码本此步按“靠近编码输出”的职责移动，不是直接朝重建标签移动。

真实网络中$z = E_\phi(A)$不是独立参数，编码器接收$J_\phi^\top \nabla_z L$，更新ϕ后重新计算z，不保证新z恰好为$.85$。解码器参数只接收重建项梯度；本题恒等解码器没有参数。所选码接收码本项，未选码在该样本上没有这项梯度；共享码还要累加其他样本贡献。

固定码本，在一个最近码区域内部微调z，真实选码输出不变，对z的导数为0；跨选择边界又会跳变。STE却把重建反馈对z的反向导数规定为1，同时阻止该重建项直接更新e。它是一条代理训练路径，不是把离散选择变成真实可微函数。停梯度后的前向数值也不能用普通有限差分去验证其全部代理导数。

题1.11-5

加噪后边缘为$\mathcal{N}(\mu, \delta^2 + \sigma^2)$。保持标准化坐标$z$不变，用$x_\sigma = \mu + \sqrt{\delta^2 + \sigma^2}\, z$连接各尺度，恰好具有这些边缘。对它求导：

$$\frac{dx_\sigma}{d\sigma} = \frac{\sigma z}{\sqrt{\delta^2 + \sigma^2}} = \frac{\sigma}{\delta^2 + \sigma^2}(x_\sigma - \mu) = -\sigma s_\sigma(x_\sigma).$$

这验证了本例PF方向。反过来分离变量积分，非零偏移满足$\log|x_\sigma - \mu| = \frac{1}{2}\log(\delta^2 + \sigma^2) + C$；零偏移也由$z = 0$包含。解出$z$并在$\varepsilon$处取值，就得到式(1.11-13)。

计算	结果约为
轨迹坐标$(1.5 - 1)/\sqrt{1.25}$	$.447213595$
$\sigma = .5$的精确状态	1.316227766
高处方向$1/1.25 \times .5$	$.4$
一次Euler状态$1.5 - .5(.4)$	1.3
高点及精确低点的端点	1.224722051

计算	结果约为
Euler低点的端点	1.213190056

Euler偏到另一条轨迹，完美端点函数也只能给出那条轨迹的端点。生成时$\Delta\sigma<0$，方向为正不表示状态增大。

把高斯先验与噪声似然相乘并配方，后验均值为

$$m_{\text{post}}=\mu+\frac{\delta^2}{\delta^2+\sigma_h^2}(x_h-\mu)=1.1,\qquad v_{\text{post}}=\frac{\delta^2\sigma_h^2}{\delta^2+\sigma_h^2}=.2.$$

后验均值平均了可能的原动作；PF端点选择一条确定性输运轨迹。即使$\varepsilon\to0$，本例PF端点也为1.223607，仍与1.1不同。把正确高噪声分布直接交给后验均值函数，输出方差只有.05；它没有恢复干净方差.25。这就是单步均方去噪最优估计不等于完整分布生成器的具体差别。

在$\sigma_h=1$处，解析端点为$1+\alpha(X-1)$，$\alpha=\sqrt{.202}\approx.449444101$。若$X\sim\mathcal{N}(m,v)$，输出均值为$1+\alpha(m-1)$、方差为$\alpha^2v$。所以

$$\mathcal{N}(1,1.25)\to\mathcal{N}(1,.2525),\qquad\mathcal{N}(0,1)\to\mathcal{N}(.550555899,.202).$$

前者仍比严格干净分布多ε^2的方差；后者还存在先验失配。教材所报网格MSE只检验固定41点上的端点误差，不是对这些高斯输出分布做了随机采样评价。

题1.11-6

用表内近似值，$y=.216920(.265042)+.424853(.089123)\approx.09535708$，故$(y-q)/64\approx-.00033230$。末层权重的单样本贡献是

$$\frac{y-q}{64}c_{\text{out}}h_0\approx(-.00033230)(.424853)(-.156400)\approx2.2080\times10^{-5}.$$

表格已取六位小数，末位与完整记录略有差别。这里已经含批平均因子，不能再除一次64。共享权重的总梯度为$\sum_i(y_i-q_i)c_{\text{out},i}h_{i0}/64$；其他样本贡献可能正负抵消，不能用第一项乘64来得到正文的全批数值。

```
x_h ──────────────────→ 在线端点网络theta → y ──→ 距离损失
 └→ 固定方向教师+求解器 → x_1 → 目标网络bar_theta → q（停止梯度）
损失反向只更新theta
优化器完成后：bar_theta ← rho*bar_theta + (1-rho)*theta_new
```

教师输出相邻状态，目标网络输出端点参照；它们不是同一个模块。目标预测很错时，配对损失仍只要求高低处预测相接，不是直接监督真实端点。目标与在线可以从相同数值初始化，但须是两个独立参数对象。EMA是赋值规则；若二者实际引用同一对象，就不会形成该参照机制。

若全局都输出b，边界对$x_1 \neq x_2$分别要求$b = x_1$、$b = x_2$，矛盾。但有限覆盖仍可失效：取$a = .1$，令$r = (\sigma - \varepsilon)/(a - \varepsilon)$，在$[\varepsilon, a)$上设$\chi = 1 - 3r^2 + 2r^3$，在$\sigma \geq a$时设$\chi = 0$。函数

$$f_{\mathrm{col}}(x, \sigma) = b + \chi(\sigma)(x - b)$$

在边界返回x，在$\sigma \geq .1$却恒为b。若训练配对两端都在该区域，损失为0，而不同轨迹端点仍会被合并。这种平滑过渡与边界跳接并不矛盾：在$\sigma > \varepsilon$处，可令网络残差为$(f_{\mathrm{col}} - c_{\mathrm{skip}} x)/c_{\mathrm{out}}$；分子在边界附近为二阶小量，分母为一阶小量，残差可以有有限极限。

因此要让相邻关系连到边界，并检查状态、尺度、任务条件和动作模式覆盖。配对差小也可能因为两边共同偏差、求解器误差，或只是尺度间隔变小。端点误差、生成分布与机器人闭环评价检验不同主张，不能被一个配对损失替代。

题1.11-7

共享样本给出$x_h = 1.3 + 1(.4) = 1.7$、$x_l^{\mathrm{CT}} = 1.3 + .5(.4) = 1.5$。精确PF低点则为$1 + \sqrt{.5/1.25}(.7) = 1.442718872$；两点的轨迹坐标分别为.626099与.707107，所以不是同一条精确PF轨迹。

单样本加噪直线的方向为$\eta = .4$，高点PF方向则为.56。二者通过条件平均连接：$v_\sigma(x) = \mathbb{E}[\eta \mid X_\sigma = x]$。本例在$x_h = 1.7$的后验干净均值为1.14，所以噪声条件均值为.56，并不要求每次抽到的噪声都是.56。

把样本方向$(x_h - A_0)/\sigma_h$放进一次Euler更新，恰好得到$A_0 + \sigma_l \eta$。这解释CT的无教师配对来源；但有限步和非线性目标网络下，它不等于逐样本精确CD标签。原始CT与CD的关系依赖尺度间距趋零、相应光滑性和估计条件。

对象	是否需要预训练方向场	如何更新
CD固定教师	需要，或用本节解析教师	不由学生损失更新
在线学生	自己不必预先会预测端点	接收一致性损失梯度
目标EMA网络	不是另一个预训练方向教师	停止梯度，随后移动平均
无教师CT	不需要方向教师，仍需要数据与目标分支	按共享样本构造配对，训练在线网络

比较10轮场预测和1轮端点预测时，应至少设置原10轮、原方向模型直接1轮、按实际调用成本配平的改进求解器，以及学生1轮/少数轮。用高精度教师端点作数值参照，可以分析求解与学生拟合误差，但教师端点不是物理任务真值。

计算方面报告视觉编码、动作生成、解码通信、端到端中位与尾部延迟、超时比例和实际网络评估次数。Heun一轮可能算多次方向，CFG还有两条分支，调用次数也不能忽略网络大小。离线检查应涵盖逐通道误差、关键接触时序、无效动作与多噪声的行为覆盖，不能只对一条示范算MSE。

闭环先固定动作接口、观测、执行前缀、重规划规则、控制周期及成对初始状态，比较成功率、完成时间、异常接触和扰动恢复，并跨任务/种子统计。再单独允许快模型更早接收新观测，或给它注入与

原模型相同的延迟，区分“生成更准”和“反馈更及时”的收益。教师造标签与学生训练的成本也应计入。

题1.11-8

逆变换为$z=(a-1)/2$，$|dz/da|=1/2$，故

$$p_A(a)=\frac{1}{2\sqrt{2\pi}}\exp[-(a-1)^2/8],\qquad p_A(1)\approx .199471140,\quad \log p_A(1)\approx -1.612085714.$$

它是$\mathcal{N}(1,4)$的密度。连续变量的单点概率仍为0；这里讨论的是单位长度上的密度，不是某个数值的离散概率。

可逆流直接提供相应坐标下的密度：对同一个a,c，新旧模型分别求逆并计算基密度与雅可比，再相减得到对数概率比。若还有固定、可逆的反归一化，要一致地变换坐标；相同且与策略参数无关的雅可比可以在比值中抵消。

FAST策略直接给的是完整token序列（包括EOS）的质量$P_\theta(Y\mid c)=\prod_i p_\theta(y_i\mid c,y_{<i})$。若多个序列解码为同一个$a$，

$$P_\theta(A=a\mid c)=\sum_{Y:D(Y)=a}P_\theta(Y\mid c).$$

合并token与其未合并子序列可能表达同一整数串。若旧策略两种序列质量为$.2,.3$，新策略为$.1,.4$，序列比分别为$.5,4/3$，而解码动作质量都为$.5$，动作比为1。序列质量不能自动替代全部解码动作质量，更不能直接叫连续动作密度。

可以明确选择token序列作为策略动作，把固定解码当作环境接口：保存实际序列及旧log-prob，在同一序列上计算新log-prob。此时做的是序列策略的概率比后训练。实际温度、允许词表或截断采样改变了概率时，也要使用相应重新归一化后的采样分布。

连续FM在方向场足够光滑、解存在唯一等条件下，可以累计密度。令生成进度为τ、$da_\tau/d\tau=u_\theta(a_\tau,\tau,c)$，连续性方程与链式法则给出

$$\frac{d}{d\tau}\log p_\tau(a_\tau)=-\nabla_a\cdot u_\theta(a_\tau,\tau,c),$$

$$\log p_\theta(a_1\mid c)=\log p_0(z)-\int_0^1\operatorname{tr}\left(\frac{\partial u_\theta}{\partial a}\right)d\tau.$$

对给定动作求概率，需要在各自模型下找回初始z并累计散度。高维迹计算、随机迹估计和积分都有成本或误差；普通采样程序没有执行这些计算。连续ODE公式也未必精确对应粗步Euler程序，后者需单独核对离散映射与雅可比，或把数值误差控制到所定义连续策略允许的范围。

若执行$B=\operatorname{clip}(A,l,u)$，映射多对一，没有唯一逆。其一维分布是

$$P_B(db)=\mathbf{1}_{\{l<b<u\}}p_A(b)\,db+P(A\le l)\,\delta_l(db)+P(A\ge u)\,\delta_u(db),$$

其中δ_l, δ_u表示端点的单位点质量。内部是连续密度，两端是非零质量，不能再用一个可逆换元密度覆盖全部。以$l = -1, u = 3$为例，两端质量各约$.158655254$，内部质量约$.682689492$。

若比较实际裁剪动作，内部应比较密度，边界应比较相应尾部质量。也可以保存未裁剪的随机动作A，把固定裁剪视为环境接口，按A定义概率比。后一种仍可后训练，只是概率对象与裁剪后动作的边缘分布不同；关键是让采样、记录和损失中的对象保持一致。

1.12 从视觉语言表示到分层动作策略

前几节解释了动作可以怎样生成。现在把这些机制放回完整的机器人策略：摄像头拍到红杯和蓝杯，指令要求把杯子收进托盘，模型怎样从这些输入得到下一段动作？如果还要先决定收哪个杯子，这个决定由谁学习，怎样传给执行动作的部分？

VLA（Vision-Language-Action，视觉—语言—动作模型）承担的是策略职责：根据机器人当时能够获得的图像、语言及其他条件，产生动作。视觉语言模型提供的知识是起点。它能辨认“杯子”和“托盘”，并不意味着它已经学会夹爪从哪里接近、抓住后如何抬起。连接这两者的关键，是动作标签、输出结构和训练目标。

1.12.1 同样读取图像与指令，输出可以很不一样

RT-1（Robotics Transformer 1）较早展示了用同一个策略学习大量机器人任务的路线。它从一小段图像和语言指令提取特征，压缩成较少的视觉标记，再由Transformer预测离散动作。语言参与视觉特征的调制；模型学习的主体仍是机器人示范中的感知与动作关系。RT-1作者讲解

RT-2向前推进了一步：把机器人动作编码成视觉语言模型能够输出的标记，使图文回答和机器人动作可以进入同一种序列训练形式。这样，辨认未见物体、理解某些新指令时，有机会利用视觉语言预训练。机器人动作如何执行，仍由示范和控制接口约束。RT-2原论文

原始**OpenVLA**提供了这一思路的开放实现。图像分别经过DINOv2和SigLIP，所得特征沿通道拼接，再通过投影器进入Llama 2语言主干；文字指令也进入同一主干。训练用的动作按维度分桶，变成词表中的标记。模型预测下一个标记，生成完整动作后，再把编号还原成控制数值。这里DINOv2与SigLIP提供视觉表示，语言主干负责结合上下文，动作解码器负责数值还原；它们的职责不能相互替代。OpenVLA固定版本，第3节

例如，一个末端控制动作包含位置变化、旋转变化和夹爪命令，共7个分量。原始OpenVLA把每个分量离散为256个桶，逐个预测。分桶是在指定数值区间内划分辨率，并不意味着编号128天然等于某个机器人的0弧度或0米。训练与部署必须使用同一套归一化和解码规则。原论文用训练数据各维的1%与99%分位数定义区间，以减轻极端值对桶宽的影响；区间外的细节无法靠一个桶编号保留。

OpenVLA-OFT 中 OFT 指 Optimized Fine-Tuning，即优化的微调方案。它保留预训练底座，却改变了输出过程。模型输入中加入动作位置的占位槽，槽位通过位置关系彼此区分；它们读取视觉、语言等条件，一次前向得到整段动作的隐藏表示，再由连续动作头输出数值。动作槽里没有填入待预测的真实动作。训练直接比较预测数值和示范数值，采用L1损失，即绝对误差的平均。OFT作者教程，固定论文，第IV节

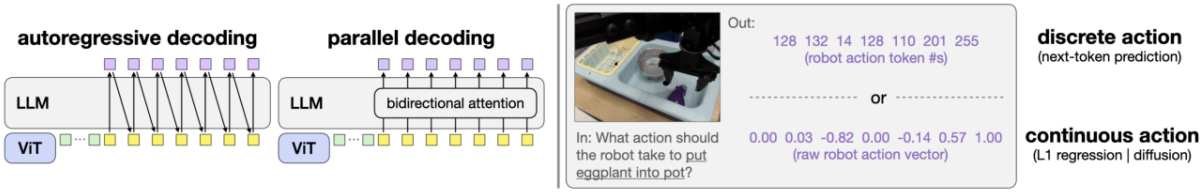


Fig. 2: Key design decisions for VLA fine-tuning. Left: Comparison between autoregressive decoding, which generates actions sequentially, and parallel decoding, which leverages bidirectional attention and generates all actions in a single forward pass. **Right**: Comparison between discrete action tokens with next-token prediction and continuous action values with L1 regression or diffusion modeling objectives. The original OpenVLA training scheme includes autoregressive decoding, discrete actions, and next-token prediction.

图1.12-A：OFT论文Figure 2。左侧的autoregressive decoding是逐标记生成，parallel decoding是同时计算所有动作槽；bidirectional attention表示动作槽可以相互读取。LLM是大语言模型（Large Language Model），ViT是视觉Transformer（Vision Transformer）。右侧比较离散动作编号与连续动作向量；L1 regression是绝对误差回归，diffusion是扩散生成。图中把“生成顺序”和“输出表示”分成两个设计轴，二者可以分别改变。图源

这个改变为什么能加速？若动作块有 H 步、每步 d_a 个分量，逐分量自回归通常需要生成 Hd_a 个标记，后面的计算要等前面的输出。并行回归把这些位置一起交给网络，消除了这条串行生成依赖。不过，一次前向仍然要处理多个槽位，视觉编码和注意力也有成本，因此“前向次数减少”与“总延迟按同一倍数减少”不是一回事。

回归的表达能力需要结合任务判断。单个确定性输出适合在已给条件下存在稳定动作的情况；若相同条件下有多条差异明显的合法路径，它可能丢失这些选择。平方损失的理想最优值是条件均值，L1的理想最优值是条件中位数，都不能一般保证落在一个合法动作模式内。对于等概率的−2与+2，任何位于两者之间的数都是L1最优解，其中的0也可能对应障碍。OFT在其任务上观察到回归有效，是关于具体数据和执行系统的经验结果；这与多峰建模的数学限制可以同时成立。

另一条路线由**Octo**展示：语言经过语言编码器形成条件，观测经过相应编码器进入Transformer，专门的读出标记汇总当前所需信息，再交给较小的扩散动作头。读出标记只读取观测与任务，不反过来改变这些输入标记。这里用到了语言表示，却没有要求整个策略必须以生成文字的语言模型为主干，也没有要求动作逐词输出。Octo作者介绍，固定论文，第III节

路线	训练预测什么	部署怎样得到动作	主要取舍
原始OpenVLA	下一个离散动作标记，使用交叉熵	串行生成标记，再恢复数值	便于利用序列训练；存在离散误差和串行依赖
OpenVLA-OFT	连续动作块，使用L1回归	并行前向产生整块	计算简洁；单值输出难以保留多个合法模式
Octo等扩散策略	带噪动作对应的去噪目标	从噪声多步生成	可表达随机动作分布；增加迭代成本
π_0等流匹配策略	当前带噪候选的生成速度	积分速度场，得到连续块	便于连续动作建模与条件缓存；需控制积分误差和延迟

因此，判断两个VLA是否采用同一路线，应追踪动作标签、损失和部署循环。共同使用Transformer或同一个视觉语言底座，并不足以回答这个问题。

1.12.2 π_0：语义底座怎样与连续动作专家相连

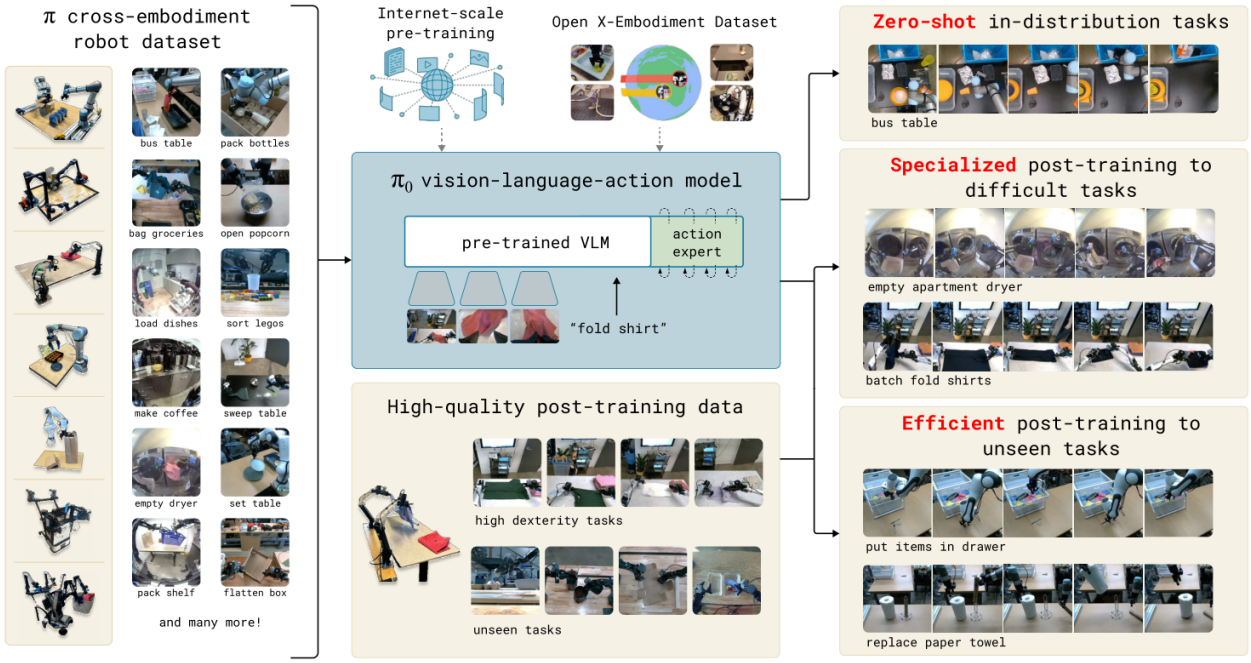


图1.12-B：π_0论文Figure 1。左侧cross-embodiment robot dataset是来自不同机器人身体的数据；上方Internet-scale pre-training表示视觉语言底座已有的互联网预训练，Open X-Embodiment是汇集机器人数据的公开数据来源；中央action expert是连续动作专家。下方high-quality post-training data是后续适配所用的高质量数据。右侧分别表示直接使用基础模型、对困难任务专门适配，以及向未见任务适配；图中zero-shot in-distribution说的是无需该次任务适配的分布内使用，不能据此理解为任意未知任务都能完成。图源

π_0把图像与文字交给预训练视觉语言参数，把机器人状态和带噪动作交给另一组较小参数，即**动作专家**。两组参数在注意力中交换信息。动作专家中的“专家”表示这组参数负责连续动作计算，并不是强化学习中评价动作好坏的Q网络，也不等于后面要介绍的稀疏专家路由。

设机器人时刻为 t，当前观测为 o_t，语言指令为 ℓ，示范动作块为

$$A_t = [a_t, \dots, a_{t+H-1}] \in \mathbb{R}^{H \times d_a}. \tag{1.12-1}$$

H 表示块内动作数，d_a 表示每步控制维数。原始π_0实验常用50个动作目标；每个目标对应一个物理控制时刻。它们与生成算法内部的10次积分属于两条时间轴。

沿用第1.10节从噪声走向数据的约定，训练抽取噪声 ω 和生成进度 τ，构造

$$x_\tau = (1-\tau)\omega + \tau A_t, \qquad u = A_t - \omega. \tag{1.12-2}$$

动作专家接收 x_τ, τ 与条件特征，预测同样为 $H \times d_a$ 的速度矩阵。这个速度描述候选动作数值怎样随生成进度改变；它不直接表示机械臂在空间中的运动速度。流匹配损失让预测接近 u。为什么这个局部回归能够训练动作分布，已在第1.10节从条件期望和概率流展开；这里关注它怎样接入视觉语言模型。

原始π_0把状态单独放成一组，把带噪动作放成另一组。图像与指令不能读取后面的状态及动作组；状态可以读取较早条件，但不能读取正在变化的动作候选；候选动作可以读取前面的条件，也可以在块内相互读取。这样一次积分改变 x_τ 时，图像、指令和状态产生的键值不需要随它重算，便能复用缓存。掩码同时规定了信息关系和计算复用范围。π_0固定版本，第IV节及附录B

部署从新噪声出发，按网络预测逐步修改整块候选，得到归一化动作，再恢复到机器人的控制单位。执行其中一段后，重新读取真实观测并生成新块。完整流程包含“视觉语言理解—条件动作生成—控制目标跟踪—再观测”，而不是生成一段文字后让机器人自行理解如何执行。

原论文使用机器人示范进行预训练和任务后训练。这里的“后训练”表示训练阶段及用途，还没有因此变成强化学习。是否属于强化学习，要看是否使用回报及相应策略改进机制。

1.12.3 $\pi_{0.5}$：为什么再引入高层子任务

“把杯子收进托盘”既包含离散的任务选择，也包含连续的运动。先收红杯还是蓝杯？红杯已拿起时，应继续收紧夹爪还是移向托盘？如果每次直接从宽泛指令预测下一小段动作，同一个网络必须同时解决这些不同尺度的问题。

分层策略把一个可学习的中间决定显式写出来。高层根据场景和总体目标产生子任务，如“拿起红杯”；低层依据当前观测和这个子任务生成动作。这个接口给低层更明确的意图，也让高层可以使用子任务文字监督。高层输出的仍然是模型预测，不是已经发生的物理事实。

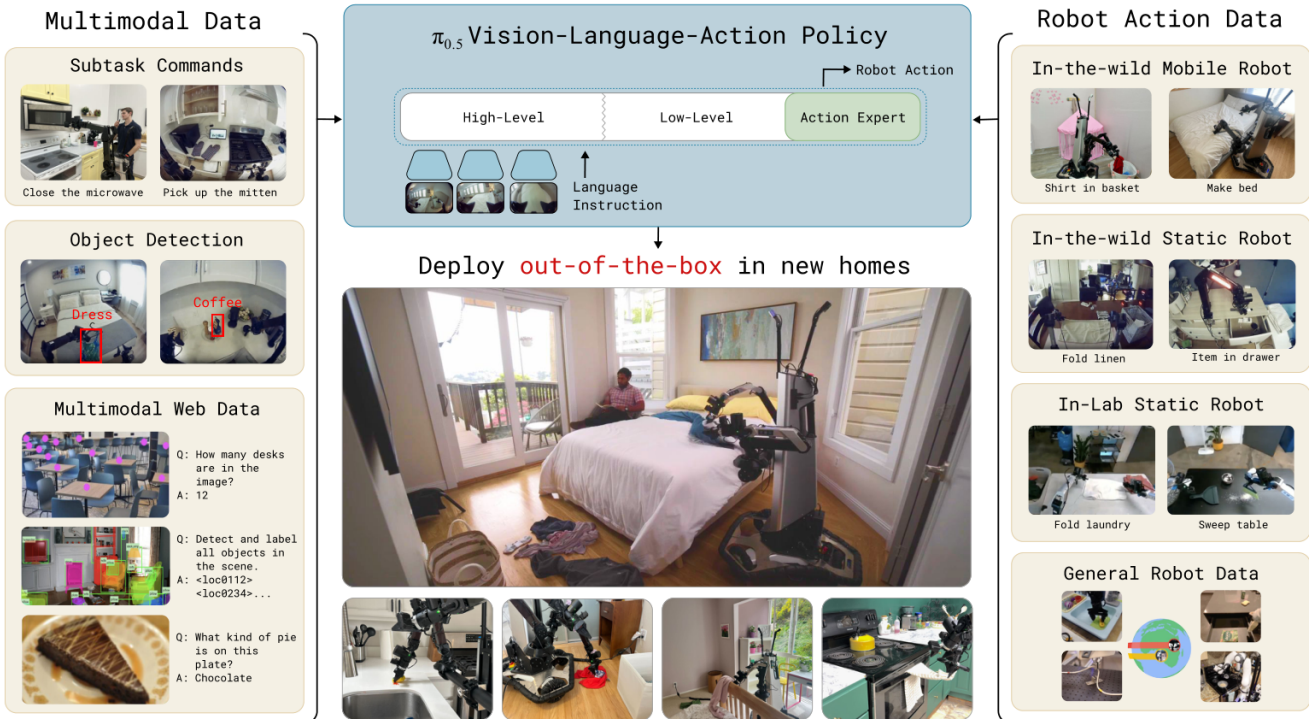


图1.12-C：$\pi_{0.5}$论文Figure 1。左侧multimodal data包含子任务指令、物体检测和图文任务；右侧robot action data包含不同环境和身体的执行数据。中央high-level是子任务预测职责，low-level与action expert负责产生控制动作。下方out-of-the-box表示作者对新家庭环境的直接评估；其范围由论文测试环境和任务确定。图源

原论文图3给出的具体流程是：“清理卧室”先得到“拿起枕头”，随后低层依据这条子任务和当前图像生成连续动作。两种职责由同一模型承担，只是调用时使用不同提示与输出方式。用 g 表示总体任务、ℓ 表示中间子任务，其建模分解是

$$p_\theta(A_t, \ell \mid o_t, g) = p_\theta(\ell \mid o_t, g)\, p_\theta(A_t \mid o_t, \ell). \tag{1.12-3}$$

右边第一项回答“接下来做哪个子任务”，第二项回答“执行这个子任务应怎样运动”。一般概率链式分解的第二项还应保留 g；这里省去它，是该版本对低层条件所作的选择：希望子任务文字已把低层

需要的任务意图说清。若“拿杯子”没有交代是哪只杯子或之后如何处理，接口可能丢失必要信息。此时要改进子任务描述、增加条件或更换模型分解，而不能靠分层这个名称弥补。

同一个 θ 表示模型参数共享，不表示训练时每条数据都提供相同标签，也不表示两个职责必须用同样频率运行。高层决定通常持续覆盖多个低层动作块；低层则随新观测反复调整。原论文只规定了高层运行得更慢，不能把机器人50Hz控制频率直接写成高层每秒生成50条子任务。$\pi_{0.5}$固定版本，第IV节

如果不固定某个子任务，而希望得到总体动作分布，需要把可能的子任务都考虑进去：

$$p_\theta(A_t \mid o_t, g) = \sum_{\ell} p_\theta(\ell \mid o_t, g) p_\theta(A_t \mid o_t, \ell). \tag{1.12-4}$$

式中求和针对中间的离散选择。先从第一项采样子任务，再从对应低层采样动作，可以实现这个混合分布；先选择最高概率子任务，则只使用其中一个分支。部署采用哪种选择规则，还要结合任务和风险评价。

一个小例子可以看出两者的差别。设高层选择 ℓ_0, ℓ_1 的概率为0.55、0.45。低层在 ℓ_0 下等概率给出动作方案 a_0, a_1，在 ℓ_1 下只给出 a_2。于是三种动作的边际概率分别为0.275、0.275、0.45。先取高层最大值会走向 ℓ_0，而联合概率最大的配对是 (ℓ_1, a_2)。这不是说机器人应该总选概率最大的动作，而是说明“逐层取最大值”与“对完整联合模型取最大值”是不同的计算。模型概率反映训练分布中的行为频率，也不是任务回报的替代品。

1.12.4 沿一条示范追踪标签、损失和更新

继续用收杯子的任务。下面是一条教学记录，用文字描述图像字段，并把连续动作缩成两步、两维；数值不来自作者私有轨迹。

字段	本条记录的内容	训练与使用中的职责
总体目标 g	把桌上的杯子收进托盘	告诉高层整个任务的意图
当前观测 o_t	红、蓝杯仍在桌上，托盘在右侧；当前关节与夹爪状态	来自相机与传感器，只含当时已经观测到的信息
子任务标签 ℓ^*	拿起红杯	来自这条示范的语义标注，训练高层；也是低层训练条件
真实动作 A_t	$[[0.2, 0.4], [0.4, 0.6]]$	示范者实际给出的归一化控制目标
离散动作标签 D	由 A_t 经FAST编码得到的标记序列	训练离散动作预测，具体编码见第1.11节
噪声 ω	$[[-0.2, 0.2], [0.6, -0.4]]$	训练时采样，用于构造流匹配输入及标签
生成进度 τ	0.25	指定当前动作候选的噪声程度

示范选择先拿红杯，并不说明先拿蓝杯一定错误。高层的文字交叉熵首先让模型拟合这条数据中选用的子任务；它本身没有比较两种顺序的任务回报。

先看高层调用。模型读取 o_t, g，生成子任务文字；教师强制训练把正确前文交给模型，要求它预测下一个子任务文字标记。对完整子任务的负对数概率记为

$$L_H = -\log p_\theta(\ell^* \mid o_t, g). \tag{1.12-5}$$

这个量由子任务各文字位置的交叉熵组成。梯度会推动网络在相似场景与目标下提高该标注的概率。它没有告诉连续动作头下一步应该输出多少弧度。

低层调用读取当前观测与 ℓ^*，学习对应的离散动作和连续动作。对表中数值，式（1.12-2）给出

$$x_{0.25} = \begin{bmatrix} -0.10 & 0.25 \\ 0.55 & -0.15 \end{bmatrix}, \qquad u = \begin{bmatrix} 0.4 & 0.2 \\ -0.2 & 1.0 \end{bmatrix}. \tag{1.12-6}$$

每一行对应一个未来物理时刻，每一列对应一个控制分量。连续专家接收左边的候选矩阵，预测右边这样的变化方向；真实动作 A_t 用来构造训练标签，不直接作为专家的干净输入。若使用

$$L_F = \frac{1}{2Hd_a} \sum_{h,j} \left[v_\theta(x_\tau, \tau, o_t, \ell^*)_{h,j} - u_{h,j} \right]^2, \tag{1.12-7}$$

则零预测的损失为 $(0.4^2 + 0.2^2 + (-0.2)^2 + 1^2)/8 = 0.155$，对四个预测值的导数依次为 $[-0.1, -0.05, 0.05, -0.25]$。若暂时把四个预测值当作独立可训练表项，梯度下降会分别把它们推向速度标签；真实网络还要沿“预测值怎样依赖权重”继续回传，更新后的输出取决于共享参数和其他联合损失。第1.12.7节先用独立表项让更新可手算。分母表示按有效动作元素平均；真实批次含不同身体和填充时，应先明确有效位置及归约规则。

离散分支则把同一 A_t 变成FAST标签，用下一标记预测损失 L_D 训练。预测第 k 个标记时，训练可以读取移位后的正确前文 $D_{<k}$；部署改用此前已经生成的前文。原始π₀.₅在后训练中联合使用文字/离散标记预测与连续流匹配。可以把其监督来源理解为 L_H, L_D, L_F，但实际训练还混合图文、定位等数据，各项仅作用于有相应标签的记录。这个分解用于看清监督职责，不要求每个训练样本同时具备三项。

三者的前向可见性也不同。低层可以读取作为条件的子任务文字；它不能偷看正在预测的真实FAST动作答案。子任务由部署时的高层先产生，因而可以成为低层输入；真实未来动作在部署时未知。第1.14节给出的允许矩阵和逐层键值检查，正是为了保持这一差别。

原始π₀.₅论文采用先离散预训练、再加入连续专家的两阶段监督方案。后来的知识隔离配方又调整了连续损失回传底座的路径。二者要分别描述：本节介绍原论文中的阶段和职责，第1.14节解释停止梯度怎样实现；不能把后来的配方自动加进所有旧版本。π₀.₅原始训练方案，知识隔离

1.12.5 两个阶段分别从什么数据中学习

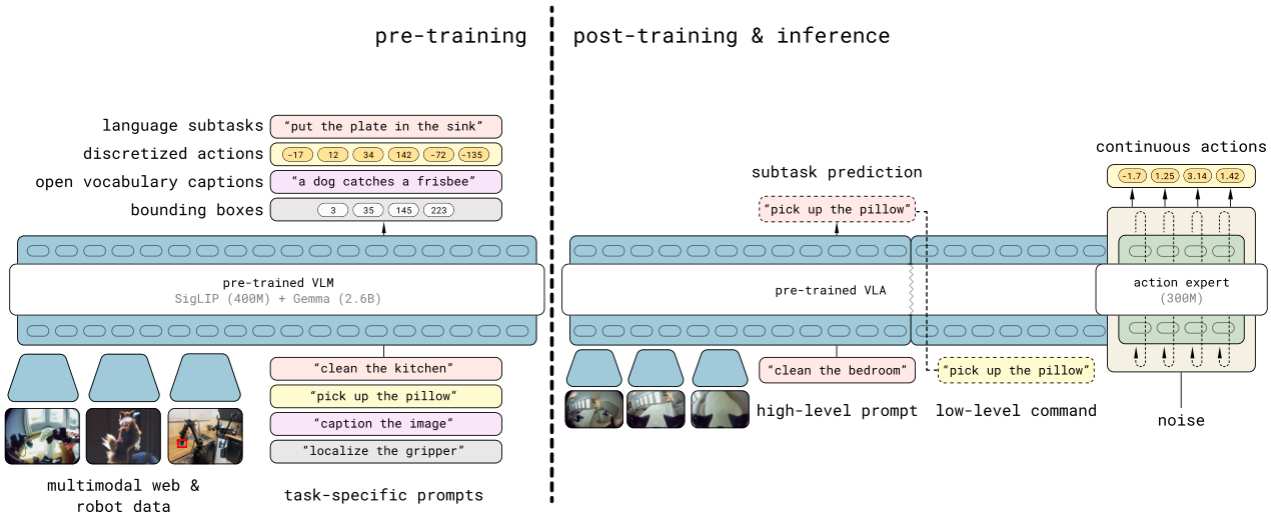


图1.12-D：$\pi_{0.5}$论文Figure 3。pre-training是预训练，post-training是后训练。左边的language subtasks、discretized actions、open vocabulary captions、bounding boxes分别是子任务文字、离散动作、开放词汇图像描述和边界框标签；它们对应不同数据任务的输出。右边先根据high-level prompt生成subtask prediction，再把它作为low-level command交给连续动作专家；noise表示生成动作所用的噪声。虚线连接的是同一模型在不同调用中的文本接口。图源

与π_0相比，这个版本还调整了输入：身体状态离散成文字，进入底座前缀；动作仍以连续候选进入专家。π_0把生成进度的编码与候选动作拼接后送入网络，$\pi_{0.5}$则用进度产生的条件调节各层自适应RMS归一化，让噪声阶段直接影响层内计算。条件调制的原理见第1.6节，具体输入变化见原论文附录E。

预训练先把多种机器人执行数据、子任务语义及互联网图文任务组织成离散序列监督。机器人数据告诉模型动作如何随场景变化，图文数据扩充物体、属性和场景知识，子任务标注训练中间决策。图像描述没有机器人关节标签，不能通过“补一个零动作”变成控制示范。

后训练一方面把模型集中到家庭移动操作，另一方面新增随机初始化的连续专家，并继续使用离散监督。原论文保留多环境移动和非移动机器人数据、相应高层标注及图文任务，加入人类实时语言指导形成的数据。人类可以让已有低层策略依次执行更合适的子任务，由此得到高层示范；这与人类逐个关节遥操作提供连续动作标签，提供的是不同层次的信息。

读原图时还会遇到MM、ME、CE、HL、WD、VI：它们依次指移动操作数据、多环境非移动机器人数据、实验室跨身体数据、高层子任务监督、网页图文数据和语言指导数据。这里数据类别CE是Cross-Embodiment；损失中的CE常指Cross-Entropy（交叉熵），两者含义不同。正文讨论损失时使用“交叉熵”，讨论原论文数据混合时写明数据类别。

联合使用异质数据的目的，是让不同标签训练它们真正能够支持的能力。只有图文知识而没有控制覆盖，不能得到精确接触；只有同一个桌面的动作示范，也难以覆盖陌生家庭里的语义和布局。数据之间是否产生有效迁移，需要在场景、物体和身体变化的测试中分别判断。

1.12.6 推理时哪些量仍然存在

部署时，真实未来动作、它的FAST编码以及人工子任务标签都不再可用。流程先根据真实观测和总体目标得到 $\hat{\ell}$，再从新噪声生成条件动作块。低层每次重新观测时可以继续使用同一 $\hat{\ell}$，不需要每个控制周期都重复生成子任务。何时换子任务，应根据高层的更新规则和新观测决定，而不是仅因为已经发送过一条“拿起红杯”就把抓取记为完成。

```
H1 读取当前图像、身体状态及总体目标
H2 若需要更新语义决定：高层生成子任务，保存为当前低层条件
H3 从新噪声创建完整动作候选；固定本次调用的观测与子任务
H4 多次预测流速度并积分，得到一个动作块
H5 反归一化并转换控制接口，执行前缀，重新读取真实观测
H6 根据实际进展再次进入H1；真实未来标签始终不参与部署
```

源模型输出的控制分量不一定具有统一单位。原始$\pi_{0.5}$移动机器人中，手臂和夹爪使用位置目标，底盘使用速度目标，升降机构又有自己的位置接口。归一化向量只是网络接口，执行器必须知道各列分别是什么。在第6章Panda项目中，动作是末端增量与夹爪开度；这个四维接口不能直接接收另一个模型的关节位置向量，需要相应的数据与适配映射。

即使目标关节位置都合法，连接它们的运动也可能碰撞，目标也可能跟踪失败。高层计划是否合适、低层动作是否可行、控制器是否跟上，是三个不同的检验环节。第8章讨论规划与约束，第11章讨论推理等待和动作衔接；在这里先保留它们在系统中的位置。

1.12.7 用小程序观察监督更新，也观察它做不到什么

配套的分层计算程序先计算式（1.12-4）的三个动作概率，再用最小的可训练量追踪两类标签。高层只有两个可训练logit，即softmax前的分类分数；低层用按子任务索引的 2×2 速度表。它们刻意省去了图像编码和共享主干，使每个更新能直接手算。

```
程序H1 计算高层概率×低层条件概率，再对子任务求和
程序H2 用示范动作、噪声和生成进度形成带噪输入与速度标签
程序H3 用真实子任务标签监督分类和对应速度表；连续执行三次SGD
程序H4 部署探针改用新噪声，不读取示范动作
```

程序注释的H1—H4对应上面这组计算步骤。高层从两个相同分数开始，用子任务0作为标签；速度表从零开始，学习式（1.12-6）的目标。学习率为0.2、损失为分类交叉熵加式（1.12-7）时，实际更新得到：

已更新次数	子任务0的概率	分类损失	连续损失	速度表第一行
0	0.500000	0.693147	0.155000	[0,0]
1	0.549834	0.598139	0.139888	[0.020,0.010]
2	0.593889	0.521063	0.126248	[0.039,0.0195]
3	0.632396	0.458239	0.113939	[0.05705,0.028525]

这个表告诉我们分类标签和速度标签分别把什么数值往哪里推，却没有说明策略已经能够完成抓取。尤其是低层速度表根本没有读取 x_τ 或 τ，它只能记住这一次噪声形成的目标。即使继续训练到表中预测完全等于 $A_t-\omega_{\text{train}}$，从另一份噪声 ω_{new} 积分得到的也会是

$$\omega_{\text{new}}+(A_t-\omega_{\text{train}})=A_t+(\omega_{\text{new}}-\omega_{\text{train}}), \tag{1.12-8}$$

一般不等于 A_t。这个速度场是常量，欧拉法无论分成几步都累计同样的变化，甚至没有积分离散误差可供缩小。这里可以把训练误差降为零，却仍然没有学会生成。这解释了真实流匹配为什么要让网络读取当前候选和生成进度，并在训练中覆盖不同噪声与进度；只把一个固定速度标签背下来不够。完整向量场的可训练实现与多峰验证在第1.10节。

1.12.8 练习题

题1.12-1：分层概率与逐层选择。 高层对 ℓ_0,ℓ_1 的概率为0.55、0.45；低层条件分布为 $p(a_0,a_1,a_2\mid\ell_0)=(0.5,0.5,0)$，$p(a_0,a_1,a_2\mid\ell_1)=(0,0,1)$。写出完整联合表和动作边际，比较逐层取最大值、联合取最大值及逐层采样。为什么这些计算都还不能单独说明哪种动作物理上更好？若从一般链式分解得到式（1.12-3），省去总体目标 g 增加了什么建模条件？

题1.12-2：同一条记录的三种监督。 使用式（1.12-6）的记录，设高层最初两个logit为0，连续速度预测为零。分别计算高层交叉熵的logit梯度与式（1.12-7）对预测矩阵的梯度，再用学习率0.2更新一次。说明哪些量是输入、哪些是标签，为什么训练可把真实子任务交给低层，却不能把真实FAST动作交给连续专家。纯图文问答样本应怎样进入同一训练配方？原始$\pi_{0.5}$后训练是否已经属于强化学习？

题1.12-3：零训练误差为什么仍不会生成。 速度表仅记住本节固定训练噪声对应的 $A_t-\omega_{\text{train}}$。假设表已经精确拟合，用新噪声 $[[0.1,-0.3],[-0.2,0.2]]$ 积分，总进度为1。算出终点并与示范动作比较。增加积分步数能消除什么误差，不能消除什么错误？真正的流模型怎样改变输入和采样训练过程，才有机会解决该问题？

题1.12-4：输出结构与动作执行。 原始OpenVLA、OFT和π_0在动作表示、预测目标、信息依赖与部署前向上分别怎样不同？为什么OFT动作槽双向读取不意味着读到了真实未来动作？再解释L1对等概率−2/+2的最优预测集合，以及为何这不否定它在某些机器人任务上的实用性。最后指出把一个关节位置模型直接接到第6章四维Panda接口会在哪一步出错。

1.12.9 参考答案

题1.12-1

固定观测与总体目标后，联合概率等于“高层选中该子任务的概率”乘“该子任务下选中动作的概率”。本题动作是三个离散方案，表中的数是概率质量。

子任务	a_0	a_1	a_2	行和
ℓ_0	0.275	0.275	0	0.55
ℓ_1	0	0	0.45	0.45
动作边际	0.275	0.275	0.45	1

逐层取最大值先选0.55对应的 ℓ_0，然后在 a_0, a_1 间打破平局，所选配对的联合概率为0.275。联合取最大值则比较六个配对，选择 (ℓ_1, a_2)，其概率为0.45。高层较大的0.55分散到了两个动作，高层稍小的0.45却集中在一个动作上，所以逐层贪心没有找到联合最大值。

逐层采样先以0.55、0.45抽子任务，再按对应分支抽动作。例如得到 a_0 的总概率为 $0.55 \times 0.5 + 0.45 \times 0 = 0.275$，恰好实现表中的边际。如果先固定 ℓ_0，再等概率打破动作平局，只会得到 $(0.5, 0.5, 0)$，已经改变了分布。本题边际最大值也为 a_2，但一般情况下多个子任务可以对同一动作贡献概率，边际求和最大值未必对应联合最大的单个配对。

这几种规则都没有单独评价物理后果。模型概率主要描述学到的行为分布，可能反映示范者偏好；任务回报、碰撞风险和时间代价没有由本题给出。概率较大的动作不必更好，按正确分布采样也不保证每次安全。

一般链式法则实际给出

$$p(A, \ell \mid o, g) = p(\ell \mid o, g) p(A \mid o, \ell, g).$$

进一步省去第二项的 g，增加了 $A \perp g \mid (o, \ell)$ 的建模条件：知道观测和子任务后，总体目标不再提供当前动作需要的额外信息。若两个总体目标分别要求放入水槽和柜子，却都只传“移动手中杯子”，低层可能仍需知道目的地。可以把子任务写完整、传递必要参数，或让低层继续读取总体目标。省略条件是接口选择，不是链式法则自动成立的性质。

题1.12-2

以子任务0为标签，分类概率为 $p_k = e^{z_k}/(e^{z_0} + e^{z_1})$，损失为

$$L_H = -z_0 + \log(e^{z_0} + e^{z_1}), \qquad \nabla_z L_H = (p_0 - 1, p_1).$$

初始 $z = (0, 0)$ 时，概率各0.5，损失为 $\log 2 = 0.69314718$，梯度为 $(-0.5, 0.5)$。一步SGD给出

$$z^+ = (0, 0) - 0.2(-0.5, 0.5) = (0.1, -0.1).$$

所以新的标签概率为 $1/(1 + e^{-0.2}) = 0.5498339973$，交叉熵为0.5981388694。这表示更接近示范选择，尚未评价该顺序是否回报更高。

连续目标为 $u = [[0.4, 0.2], [-0.2, 1.0]]$。本题有4个有效元素，式（1.12-7）对每个预测值的导数为 $(v_{h,j} - u_{h,j})/4$。初始预测为零，因而

$$\nabla_v L_F = \begin{bmatrix} -0.10 & -0.05 \\ 0.05 & -0.25 \end{bmatrix}.$$

将预测作为独立可训练表项，一次更新得到

$$v^+ = \begin{bmatrix} 0.020 & 0.010 \\ -0.010 & 0.050 \end{bmatrix} = 0.05u.$$

误差变成原来的0.95倍，损失从0.155降为 $0.155 \times 0.95^2 = 0.1398875$。未选中子任务的速度表没有收到这条样本的梯度。若预测由共享网络产生，实际更新的是权重，梯度还需乘上预测对权重的导数：

$$\nabla_\theta L_F = \frac{1}{4}\sum_{h,j}(v_{\theta,h,j} - u_{h,j})\nabla_\theta v_{\theta,h,j}.$$

因此同样的输出误差，在不同参数化或联合损失下可以造成不同的后续输出变化。独立表项的更新数值不能直接推广到真实VLA的每个输出。

监督	输入与标签	直接学习的内容
高层文字	读当前观测、总体目标及正确文字前文；标签为下一子任务文字	场景与目标下的子任务选择
离散动作	读当前观测、子任务及移位动作前文；标签为FAST编码	下一个动作标记
连续动作	读当前观测、子任务、x_τ, τ；标签为 $A - \omega$	当前候选的生成方向

题面没有给FAST分支的预测概率，所以不能算出 L_D 的具体数值。高层交叉熵也不能代替离散动作交叉熵。

训练低层使用真实子任务，是在拟合“给定意图怎样行动”；部署先产生 $\hat{\ell}$，再提供同类条件。高层预测可能出错，所以真实子任务辅助下的低层成绩不等于完整系统成绩。FAST标签却编码了未知真实未来动作；若连续专家读取它，训练就能依赖部署时没有的答案。离散分支自身使用移位前文属于教师强制，部署可用已有预测替代；这不允许连续专家读取整份干净答案。流匹配的 x_τ 是算法在生成中能够逐步提供的候选变量，与额外输入干净未来答案的职责不同。

纯图文样本以图片和问题为输入，以回答文字为标签，仅计算相应有效位置的文字交叉熵；缺少动作就省略需要动作的监督，不伪造零动作。共享底座被图文损失更新后，可以间接改变专家读到的条件。原始$\pi_{0.5}$后训练仍采用标记预测和流匹配监督，轨迹筛选或语言指导本身不使它成为强化学习。

题1.12-3

表格模型输出固定的 $v = A - \omega_{\text{train}}$。从新噪声积分到进度1，终点是

$$x_1 = \begin{bmatrix} 0.1 & -0.3 \\ -0.2 & 0.2 \end{bmatrix} + \begin{bmatrix} 0.4 & 0.2 \\ -0.2 & 1.0 \end{bmatrix} = \begin{bmatrix} 0.5 & -0.1 \\ -0.4 & 1.2 \end{bmatrix}.$$

它与示范动作的差为 $[[0.3, -0.5], [-0.8, 0.6]] = \omega_{\text{new}} - \omega_{\text{train}}$，平方误差和为1.34，按元素平均为0.335。

分成 N 个等长欧拉步，每步只增加 v/N，最终仍是 $\omega_{\text{new}} + Nv/N = \omega_{\text{new}} + v$。对这个常量场，精确算术下的积分离散误差已经为零；错误来自模型输出的场本身。一般速度场中，增加步数在适当稳定性和正则条件下可以减少求解器相对于该已学场的近似误差，却不能纠正学错的方向、缺失条件或训练覆盖，也不保证真实任务指标逐步单调改善。

真正的流模型应接收 x_τ, τ, c，其中 c 包含部署可用的观测和任务条件。训练反复抽取数据动作、噪声和进度，为不同路径点提供对应标签。例如换成本题的新噪声，在进度0.25有

$$x_{0.25}^{\text{new}} = \begin{bmatrix} 0.125 & -0.125 \\ -0.05 & 0.30 \end{bmatrix}, \qquad u^{\text{new}} = \begin{bmatrix} 0.1 & 0.7 \\ 0.6 & 0.4 \end{bmatrix}.$$

同一目标从不同起点出发，需要不同局部方向，固定表格无法区分。若数据只有一个固定动作 A，理想解析场 $v^*(x,\tau) = (A-x)/(1-\tau)$ 在 $\tau < 1$ 时可说明这种区别：代入直线路径就得到对应的 $A-\omega$，因而能从不同噪声趋向同一目标。这里固定 A 表示模型已从极简单数据分布中学到的目标，不是部署时再交给它一份未知标签。

一般多动作数据中，理想局部回归是 $\mathbb{E}[A-\omega \mid x_\tau = x, \tau, c]$，不必对每条配对都零误差；生成目标是输运整体条件分布，也不是每次复制某条指定示范。应从新噪声、新观测实际生成，分别检验分布、约束和闭环结果。增加输入与训练覆盖提供了正确的建模方向，仍需验证有限模型是否学好。

题1.12-4

项目	原始OpenVLA	OFT并行L1方案	π_0
输出	每维256桶的离散动作标记	连续动作块数值	生成动作块所需的速度场
标签与损失	示范分桶，动作位置交叉熵	示范动作，L1误差	动作与噪声构造局部速度，平方误差
动作间依赖	后一个标记读取此前动作标记	空动作槽双向交互	同一进度下的候选动作双向交互
部署	逐标记生成、解码	一次并行前向	新噪声起步，多次前向与积分
共同执行环节	还原动作单位与接口	还原动作单位与接口	还原动作单位与接口

原始OpenVLA用图像和语言产生动作；OFT支持多视角与状态等输入，改变了动作输出和训练目标。π_0还必须读取当前候选与生成进度。把逐分量自回归扩为 H 步、d_a 维，通常产生 Hd_a 个串行标记依赖；训练时的因果损失可以并行计算，并未消除部署时对前一新输出的等待。并行回归虽减少生成前向次数，仍要处理视觉和所有槽位；流模型则还要多次更新候选。

OFT动作槽最初装的是占位嵌入及位置信息，读到其他槽时得到的是根据当前条件算出的隐藏表示。输出对应未来时刻，不代表输入含有未来真值。若把真实动作填入槽位并保持双向读取，才改变了信息边界；应同时检查数据与掩码。双向交互能协调多个输出，却不自动把确定性回归变成随机多峰生成。

对 $Y = -2, +2$ 各一半，预测 m 的绝对误差风险为

$$R(m) = \tfrac{1}{2}|m+2| + \tfrac{1}{2}|m-2| = \begin{cases} -m, & m < -2, \\ 2, & -2 \le m \le 2, \\ m, & m > 2. \end{cases}$$

在两标签之间向右移动，减少的一侧误差与增加的另一侧误差相等，所以全部 $[-2, 2]$ 都是最优解，0也在其中。如果两端是合法绕行、0会碰撞，目标仍可接受0。L1的条件中位数不保证是分布众数。若给

定观测与指令后动作已比较一致，或示范明显偏向某种合法策略，单值预测仍可能足够好；例如−2的概率变成0.6后，它就是唯一中位数。因此数学反例与OFT在具体任务上有效并不矛盾。

第6章Panda接口接受 $[-1,1]^4$ 中的三维末端平移增量和夹爪命令；环境再把末端目标变成七个关节目标，交给底层执行。七维关节位置模型的输出直接接入，首先会违反形状约定；强行截取四维，则把弧度值误当平移增量、把一个关节角误当夹爪命令。裁剪范围不能修复单位、坐标和含义。可以按四维接口重新收集并训练标签，或设计相匹配的关节控制接口；即使同一机器人经正运动学转换，四维接口也无法一般保留任意姿态和冗余关节选择。生成目标与真实跟踪结果仍需分别验证。

1.13 模型演进、记忆与多模态行为条件

第1.12节把动作表示、高层子任务与低层执行接在了一起。接下来遇到的新问题是：机器人转身后怎样记住杯子放在哪个柜子，怎样避免一次次重复失败，怎样从质量不同的数据中学到更好的行为？这些问题分别涉及历史信息、训练信号和行为条件，不能只通过扩大动作生成器解决。

1.13.1 用“改变了哪一环”理解π系列

图1.13-A：π系列的模型与方法关系。上排组织主要模型，下排按动作编码、梯度、经验学习、记忆和执行问题组织方法。箭头概括研究发展关系，既可能涉及模型设计延续，也可能涉及训练数据来源；例如$\pi_{0.7}$利用了早期强化学习策略收集的数据，不等于把$\pi^*_{0.6}$的某次任务更新直接当作它的初始化。

π_0建立视觉语言条件与连续动作专家的连接，$\pi_{0.5}$把更广的数据和高层子任务接入监督训练。$\pi_{0.6}$更换底座、扩大动作专家，并采用知识隔离组织联合学习；RECAP把经验和纠错加入策略改进，形成$\pi^*_{0.6}$路线。MEM补充短期视觉和长期语义记忆。$\pi_{0.7}$继续扩充上下文，使总体任务、子任务、视觉子目标和质量信息可以共同指定行为。

版本或方法	主要改变	标签和更新的关键区别	部署时新增什么
π_0原始方案	视觉语言底座连接连续动作专家	示范动作提供流匹配监督；状态作为连续输入进入专家	从噪声生成动作块
$\pi_{0.5}$原始方案	异质监督与高低层分工	先离散预训练，再加入连续专家联合监督；状态离散成文字	先产生子任务，再生成条件动作
$\pi_{0.6}$	更大的底座、动作专家及更丰富条件	底座接受FAST及其他离散监督，连续专家梯度经知识隔离分开	继续使用分层推理，可加入行为元数据
$\pi^*_{0.6}$ / RECAP	利用不同质量的经验改进策略	另训价值模型，产生优势相关条件，再训练动作策略；包含离线预训练及后续任务经验学习	请求较好的优势条件
$\pi_{0.6}$-MEM	短期视频与长期文字记忆	训练视频条件、摘要更新和子任务预测；连续专家仍按对应配方训练	保留近期观测，读写任务摘要
$\pi_{0.7}$	用多模态条件利用更广数据	混合示范、失败及自主执行等数据；训练时随机省略部分条件	接受细化指令、质量/速度条件及生成的子目标图像

这个表中的一行通常同时改变多个因素。直接比较两个完整版本的成功率，可以评价整套系统，却不能单独证明“多了记忆”或“换了底座”带来全部增益。具体机制需要单独对照。

同样要区分版本名与方法名。**FAST（Frequency-space Action Sequence Tokenization，频域动作序列编码）**改变动作如何表示，第1.11节已经给出变换、量化、压缩及还原过程；知识隔离改变连续损失能更新哪些参数，第1.14节逐层检查了键和值；它们可以进入多种模型，不是各自独立的一代机器人模型。$\pi_{0.6}$模型卡，RECAP固定版本

1.13.2 从$\pi_{0.6}$到$\pi^*_{0.6}$，新增的是质量相关学习

$\pi_{0.6}$保留高层子任务和低层动作分工，视觉语言底座初始化自Gemma 3 4B，动作专家约860M参数。M表示百万，B表示十亿。与原始$\pi_{0.5}$相比，模型卡还改变了输入分辨率、文本注意力、数据和提示设计，因此不能只用参数规模描述其变化。其身体状态仍离散为文字；后面的MEM和$\pi_{0.7}$则改用连续状态投影，以免历史状态产生过多文字标记。模型设计与训练数据

RECAP 的英文名称为 Reinforcement Learning with Experience and Corrections via Advantage-conditioned Policies，意思是利用经验和纠正数据，通过优势条件化策略进行强化学习。它处理的是另一类问题：已有VLA会做任务，却也会失败、绕远或需要人类纠正，怎样利用这些不同质量的经验继续改进？

它先训练一个独立价值模型。输入是观测和任务，标签来自轨迹的后续回报；该模型给出价值的离散分布，再由分布求得数值估计。随后结合实际反馈和价值估计，为数据动作生成优势相关的改进标签：

以按任务设定的阈值区分相对较好的行为。动作策略在训练时同时看到动作及其条件，部署时请求较好的那一类。标记中的positive表示通过该改进阈值，不必等同于所有任务统一按“优势大于0”分组。连续动作仍可用流匹配学习，无需先把整个生成器改造成能直接计算动作密度的PPO策略。

$\pi^*_{0.6}$不是“给基础模型加一颗星后部署”的单一步骤。原方法先在已有数据上训练价值和优势条件化策略，再针对任务加入示范、自主执行与人类纠错，反复更新数据、价值模型和策略。在线收集的经验、价值标签和动作标签各有来源。第3.8节将展开这条改进循环，本节先把它放在版本路线中的正确位置。RECAP，第IV—V节

质量条件也有能力边界。若某种情形下的数据只有失败动作，写上“正优势”没有提供正确动作证据；还需要恢复示范、纠错或新的交互。价值模型的误差也会传入标签。这类机制的评价既要看最终成功率，也要看额外数据、人类纠错和交互成本。

1.13.3 只有当前画面，为什么会反复做错

设机器人刚把红杯放进左柜，再转身面对台面。现在左右柜门都关闭，画面中看不到杯子。接到“把红杯拿出来”时，仅凭当前图像和身体状态，可能无法判断该开哪一扇门。这不是视觉编码器不够清楚，而是当前输入缺少过去发生的事件。

还有一种困难只持续很短时间。夹爪刚尝试抓一根筷子，第一次从正上方接近后滑开了。下一帧看起来又像尝试之前；没有近期历史的策略可能再次采用同一个接近方式。近期图像能保留滑落方向、夹爪相对位置与尝试过程，较长的文字摘要则更适合保留“红杯已放左柜”这样的事件。两类信息需要不同的保存精度。

MEM（Multi-Scale Embodied Memory，多尺度具身记忆）据此把记忆分成两个部分：短期视频保留近期感知细节，长期语言记忆保留任务相关事件。这里“记忆”是下次推理时继续作为输入的信息，不是每次任务中都更新神经网络权重。MEM作者讲解

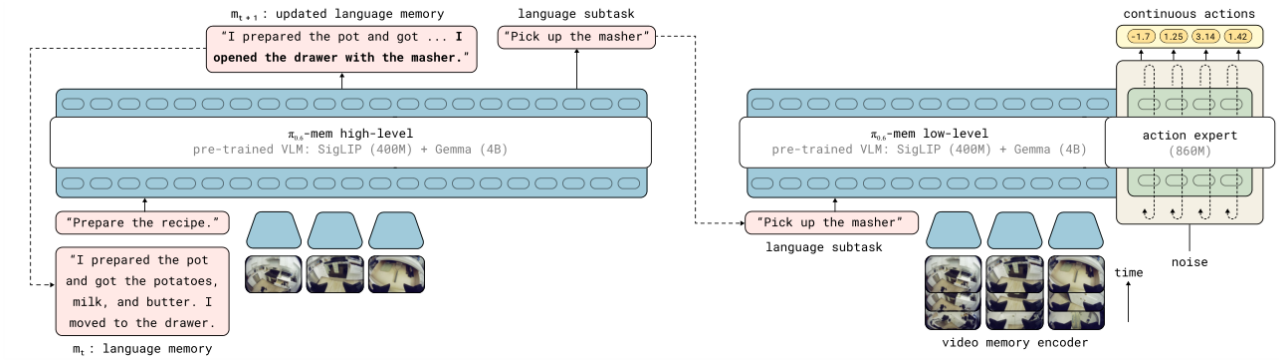


图1.13-B：MEM论文Figure 2。左侧language memory是上一份任务摘要 m，updated language memory是本次更新后的摘要，language subtask是接下来要执行的子任务；虚线把新摘要送回下一次高层调用。右侧video memory encoder把近期图像序列编码给低层，action expert从noise生成continuous actions。time箭头表示过去到当前的观测顺序，专家中的迭代箭头表示动作生成，两者不是同一条时间轴。图源，第III节

用 m 表示旧摘要、m' 表示更新后的摘要、ℓ' 表示新子任务，可以把高层职责写为

$$(\ell', m') \sim p_H(\ell', m' \mid o_t, m, g). \tag{1.13-1}$$

高层根据已经看到的情形改写过去事件，并决定接下来做什么。m' 的撇号表示摘要被更新，不表示把将来的预期动作写成已经完成。发送“拿起红杯”后，只有新的观测支持杯子确实已被拿起，摘要才应记录成功。

低层则使用近期观测、总体目标及高层给出的子任务：

$$A_t \sim p_L(A_t \mid o_{t-(K-1)\Delta:t}, \ell', g). \tag{1.13-2}$$

这里 K 是保留的观测帧数，Δ 是相邻历史帧间隔，下标表示按该间隔取样的序列。低层无需直接读取无限长的全部历史；长期相关事件经摘要影响高层选择，再通过子任务影响执行。原方法也讨论用近期观测帮助高层更新。式（1.13-1）突出摘要递推，式（1.13-2）突出短期感知，它们不是要求所有实现使用同一组相机或同一时间窗口。

1.13.4 语言记忆怎样学会写、压缩和继续读取

“把历史变成一句摘要”还不是完整训练方法。模型需要看到旧摘要、当前观测与下一份摘要之间的配对，才能学习哪些信息该保留，什么时候应更新。

MEM用已有机器人片段的子任务文字、成功或失败标记，交给预训练语言模型制作摘要监督。摘要要保留对后续任务仍有帮助的过去信息，同时合并冗余。高层VLA随后通过文字预测学习摘要和子任务的输出。部署时由这个学到的高层递推自己的摘要，并不要求每个低层控制周期都调用外部语言模型。

例如，原记录依次为“红碗放入上柜”“蓝碗放入上柜”“黄碗放入上柜”。如果后续只需要知道柜内是否已经收好碗，可以压缩成“三只碗已放入上柜”。如果下一步要单独取回红碗，就应保留足以定位它的信息。压缩的目标是保留决策需要的信息，不是字数越少越好。

训练样本还必须按时间对齐。高层输入里的旧摘要只能包含此前事件，新的摘要是要学习预测的文字标签。把“稍后成功放杯”提前写进当前输入，会让训练看起来容易，却使部署无法获得同样条件。摘要制作可以使用离线标注来教模型判断已发生的事，但不能把未来事实当成当时已有的记忆。

当前已经观察到的变化	旧摘要	本次可写的新摘要	接下来的子任务
红杯已放入左柜，手中为空	红杯在夹爪中	红杯已放左柜；蓝杯尚未收好	拿蓝杯
第一次抓蓝杯后滑落，仍在桌面	红杯已放左柜；蓝杯尚未收好	保留未完成状态，必要时记下失败线索	重新抓蓝杯
蓝杯已进入托盘，手中为空	蓝杯在夹爪中	红杯在左柜；蓝杯已放托盘	关闭尚开着的柜门

表中的摘要与子任务为教学示例，关键是每次改写都对应已观察到的变化。若只是把所有发过的指令拼起来，三次“抓蓝杯”会占据三段历史，却仍没有说明哪次成功。MEM论文观察到，训练示范往往比较流畅，部署则会出现重复尝试；保留大量重复指令会加大两种历史的差异。压缩可以让摘要继续表达“蓝杯未收好”，同时近期视频保留刚才怎样失败的细节。MEM，第III-B及IV-A节

这不意味着所有失败都应删除。“冰箱门向左拉不动”可能对下一次选择方向很有用。需要区分不再提供信息的重复记录与仍能改变决策的失败事实。语言摘要会写错或遗漏，错误还可能随递推保存，因此应让新观测能够修正它，而不是把旧摘要当作永远正确的环境状态。

一个明确的失效反例是：红杯可能在左柜，也可能在右柜，当前画面与身体状态完全相同；压缩器却对两段历史都只留下“已经收好一只杯子”。若两种历史等概率，且要求第一次就打开正确柜门、不能先探索，那么只读这份摘要的策略最多有50%的首次选择正确率。因为它在两种情形下接收到完全相同的输入，只能给出同一个选择分布。保留杯子所在柜子，才消除了这个信息缺口。50%的界限针对这次受限决定，不是允许反复搜索后整回合成功率的上限。

1.13.5 视频记忆怎样压缩，而不把历史先扔掉

最直接的办法是把 K 帧的每个图像块都交给后续主干。若每帧有 n 个块，共 Kn 个标记，联合注意力的分数矩阵大小随 $(Kn)^2$ 增长。长期保留高分辨率观测，很快会占用大量计算；只留下当前帧又会失去需要的历史。

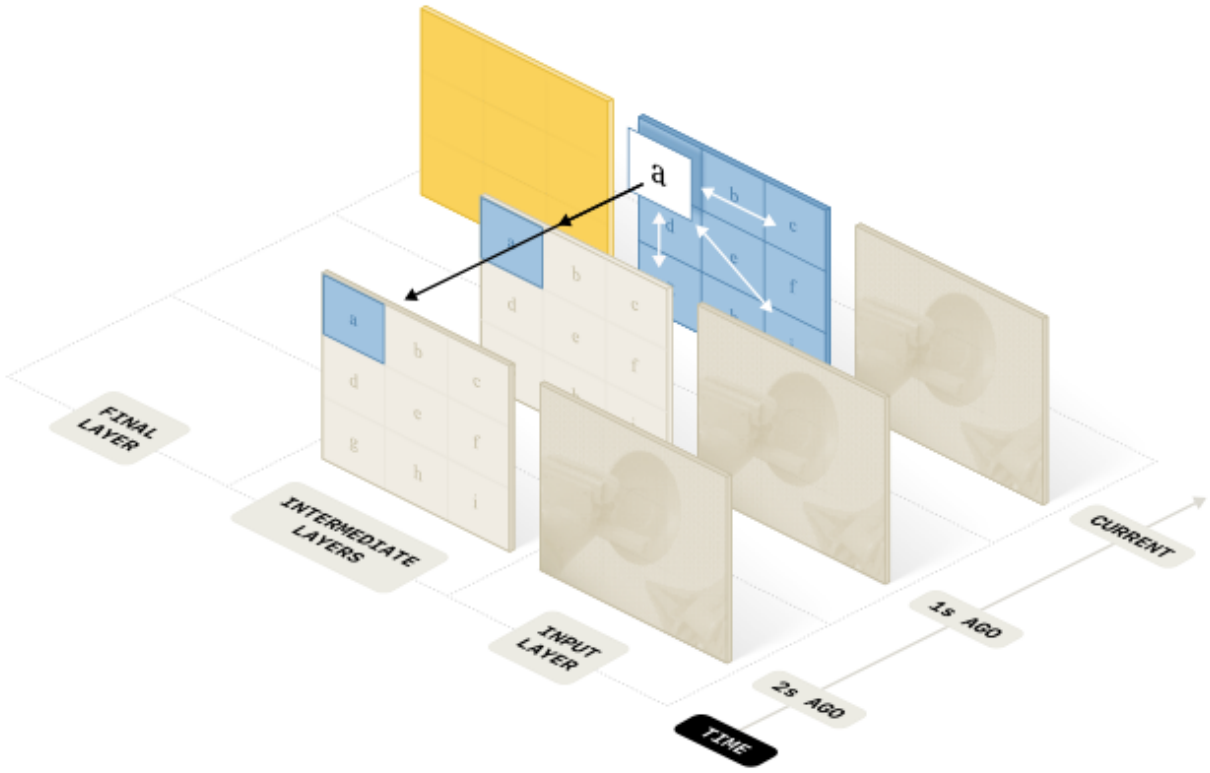


图1.13-C：MEM论文Figure 4。input layer是输入层，intermediate layers是中间层，final layer是送往后续主干的输出层；current、1s ago、2s ago分别表示当前、一秒前、两秒前。a、b等字母标记空间图像块。白箭头在同一帧的空间块间交换信息，黑箭头把同一空间位置的过去信息送向较新时刻；输出只保留当前时刻的一组标记。图源，第III-C节

MEM把空间读取和时间读取拆开。多数层继续处理单帧中的空间关系；在选定层中，增加同一图像块位置跨时间的读取，并用因果掩码禁止较早帧读取之后的帧。历史信息先进入当前帧的表示，最后再删去较早帧的输出标记。于是后续语言主干收到的视觉标记数可以保持为单帧规模，但这些标记已经包含过去信息。

这种分开读取的主要注意力分数规模为

$$O(Kn^2 + nK^2), \tag{1.13-3}$$

第一项是在 K 帧内各做空间注意力，第二项是在 n 个空间位置各做时间注意力。比如 $K = 6, n = 16$，完整时空分数矩阵有 $96^2 = 9216$ 个位置；分开的空间与时间矩阵合计 $6 \times 16^2 + 16 \times 6^2 = 2112$ 个位置。这比较的是注意力分数规模，尚未计算投影、前馈、图像编码等成本，也没有把一个掩码中的无效位置都当成实际省掉的计算。

下面把每帧缩成两个标量图像块，进一步追踪“保留当前表示为什么仍能保留过去”。三帧输入分别是

$$F_0 = [0, 2], \qquad F_1 = [4, 6], \qquad F_2 = [8, 10]. \tag{1.13-4}$$

教学例中，空间注意力分数都为0，经softmax后两块等权。空间读取得到 $[1, 1]$、$[5, 5]$、$[9, 9]$。再对同一空间位置做等权的因果时间读取：第0帧只读自己，得到1；第1帧读前两帧，得到 $(1 + 5)/2 = 3$；第2帧读三帧，得到 $(1 + 5 + 9)/3 = 5$。最终保留 $[5, 5]$，它已经包含过去两帧。

如果在时间读取之前就删除历史，结果会是 $[9, 9]$。若把第一帧两个值都加6，最终当前表示变成 $[7, 7]$，说明旧帧仍可影响保留下来的输出。若只在序列末尾追加一帧 $[100, 100]$，前三帧的因果输出不应变化。这三个干预分别检查了处理顺序、过去信息是否进入输出，以及是否偷看未来。

这个算例只有等权读取，省略了投影、残差、前馈和位置编码，并非复现MEM的视觉网络。它展示的是必要的信息关系：先吸收历史，再缩减输出。真实网络还要学习哪些图像块与时间有用；输出数量固定并不保证细节都被保留，也不意味着增加历史后整个视觉计算量不再增长。

MEM复用预训练视觉编码器参数并改变注意力结构，加入固定的时间位置编码。对应$\pi_{0.6}$-MEM还把每个历史身体状态投影为一个连续标记，避免每个时刻都展开成很多文字。训练混合机器人、图文及视频语言数据，高层学习摘要与子任务，底座和连续专家仍按知识隔离分工。MEM，第III-C—D节

1.13.6 记忆更新、动作执行和参数更新是三件事

当机器人第二次抓筷子时改了接近角度，可能只是网络读到了第一次失败的历史，当前推理因此产生不同输出。这个过程称为**上下文适应（in-context adaptation）**：行为因输入上下文改变，权重可以保持不变。若收集失败和纠错后执行优化器更新，才是另一种训练适应。

```
R1 新回合建立空的观测历史与任务摘要
R2 接收真实观测，按时间顺序写入近期窗口
R3 视觉编码先读取空间及过去，再保留压缩后的当前表示
R4 需要更新高层时：读取旧摘要和新观测，生成新摘要与下一子任务
R5 低层读取近期表示与子任务，生成动作块并执行一段
R6 回到R2，以实际反馈修正下一次判断；不因已发送指令就标记完成
```

开始独立新回合时，应清空属于上一回合的图像缓冲、摘要和正在使用的任务条件；模型权重不因清空这些输入而重置。配套视频读取程序把视频部分缩成下面五步；代码中的M1—M5注释逐项对应。它没有训练摘要器，而是让信息流与失效条件可直接复算。

```
M1 在每帧内部做空间等权读取
M2 沿同一空间位置做因果时间读取
M3 读取完成后仅保留当前帧标记
M4 改变过去或追加未来，检查保留信息与因果关系
M5 新回合清空缓冲，并与误留旧回合数据比较
```

程序中新回合第一帧为 $[20, 22]$，正确清空后得到 $[21, 21]$。如果误把上一回合末两帧留在三帧窗口，会得到 $[(5+9+21)/3, (5+9+21)/3] = [35/3, 35/3]$。模型没有更新一项参数，输入却已被另一回合污染。评测记忆系统时，重置是否正确和网络是否有效必须分别检查。

记忆也不总能改善控制。训练数据可能让“上一动作”比当前视觉更容易预测下一动作，模型便只会延续旧动作；环境被扰动后，这个捷径可能失效。可以在相同当前场景下改变合理的历史，检查输出是否利用了真正相关事件；也可以保留失败、遮挡和恢复场景，检验历史是否帮助修正，而非固化错误。增加记忆扩大了可用信息，是否学会使用它仍是实验问题。

1.13.7 $\pi_{0.7}$：把希望怎样做也写进条件

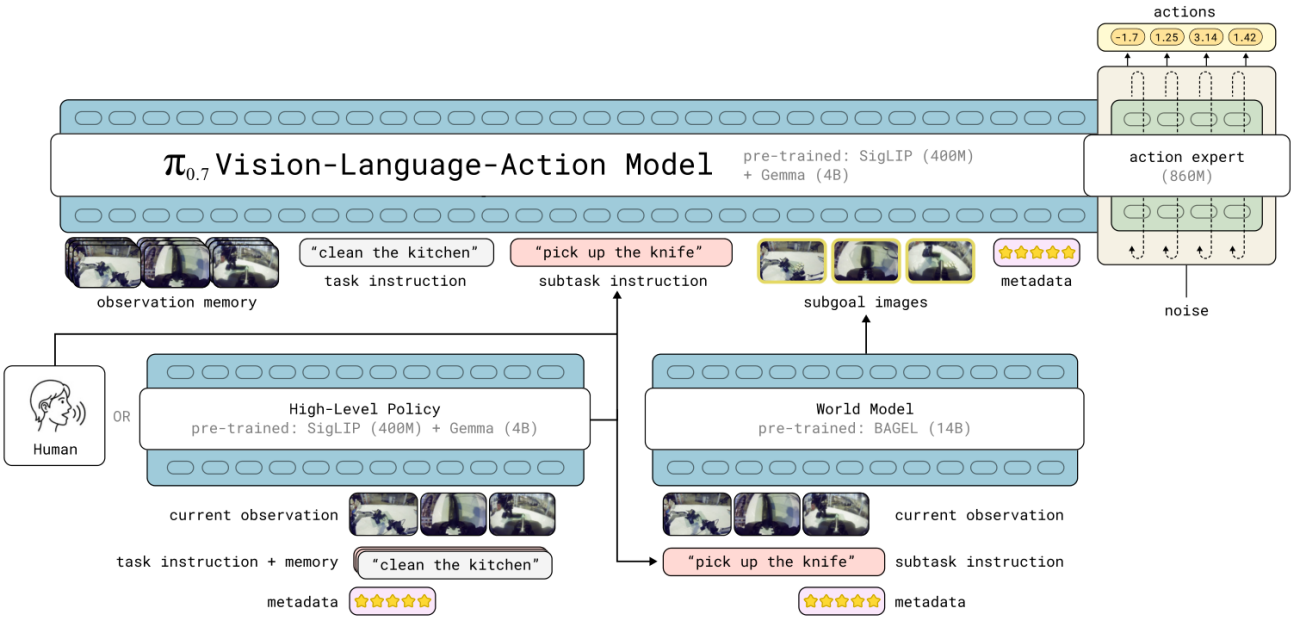


图1.13-D：$\pi_{0.7}$论文Figure 2。上方是产生actions的VLA与action expert；observation memory是近期观测表示，task instruction是总体任务，subtask instruction是当前子任务，subgoal images是希望达到的视觉状态，metadata描述质量、速度等行为属性。下方human或high-level policy提供子任务，world model根据当前观测、子任务及元数据生成子目标图像。SigLIP/Gemma标明视觉语言初始化，BAGEL标明图像生成底座；14B图像生成模型是额外模块，不能把上方VLA的约5B规模当成整个运行系统的全部成本。固定论文，第IV—VII节

仅用任务名称组织数据时，同一任务中的不同方式和质量很容易混在一起。例如两条示范都写“把杯子放托盘”：一条直接完成，另一条先抓空两次，再绕远放好。如果条件里看不出这些差异，训练就

要用同一个条件解释多种行为。$\pi_{0.7}$通过更丰富的上下文描述这条数据“在怎样做”，再在使用时提出希望采取的行为条件。$\pi_{0.7}$作者讲解

元数据（metadata）是描述一条样本属性的附加字段。在这里可以包括完成步数、质量评分、片段是否出错和控制模式。训练中的评分来自该条记录的标注，动作仍来自那条轨迹。部署写“高质量、无错误”是在请求相应条件下的行为，不是预先知道机器人将成功。它能否按这种请求行动，取决于条件覆盖、标签质量和模型学习；提示字段不是保证。

更细的子任务文字说明语义意图，**视觉子目标**进一步说明希望达到的空间状态。例如“把红杯放托盘”没有明确杯子在托盘上的位置，子目标图像可以表示杯子应在左侧空位、夹爪已松开。它是条件中的目标图，不是相机此刻看到的观测，也不是机器人已经执行成功的证明。

原方法用片段后续真实图像训练子目标生成模型，并让策略在训练中读取真实未来图像与模型生成图像的混合。部署时，图像由以BAGEL为起点训练的生成模块根据当前观测和子任务产生。让动作策略在训练中也看到生成图，是为了减小“训练目标图很准确、部署目标图带误差”的差异。子目标与动作联合生成的世界动作模型则是另一种架构，下一小节会比较。

这里使用未来图像作为训练条件，并不等于把未来观测合法化为部署输入。训练可以提供目标状态，部署必须明确由谁给出对应目标：用户、规划器或生成模型。若测试时仍把真实未来帧直接交给动作策略，测到的是理想目标辅助下的能力，不能与只读当前观测的策略当作同条件比较。

训练还随机省略部分条件，使策略能在没有子目标图像、没有某项元数据等情况下工作。这既支持不同调用方式，也使有条件与省去某项条件的预测可以用于无分类器引导。省略条件的含义是该字段不可用，而不是填入一个容易被误认为真实观测的“零状态”。具体引导公式和其适用条件见第1.11节。

部署有多种更新节奏：低层根据新观测生成动作；高层在需要时更新子任务；图像模块在子任务改变或间隔达到阈值时更新子目标。原论文的子目标刷新阈值为4秒，动作块有50个目标，生成用5步迭代，并只执行部分动作后重算。几个数字分别控制目标刷新、预测长度、生成成本和闭环频率，不能混成一个“模型频率”。

异步还带来条件一致性问题。高层已从“抓杯”切换到“放杯”，较慢的图像模块却刚返回旧子任务的抓取图。如果把它与新指令直接拼在一起，两个条件可能冲突。实际系统可以给请求携带子任务版本和时间戳，在结果返回时检查所属任务，并据实际延迟决定继续使用还是重算。这是由多线程条件接口提出的工程要求；具体取舍应在自己的执行系统中验证。

$\pi_{0.7}$使用MEM式视频历史编码和连续状态标记，继续用知识隔离训练底座与动作专家，也使用训练时模拟延迟的RTC。它还利用早期策略包括强化学习专家收集的数据。因此“从强化学习专家学到行为”与“当前这次训练直接执行PPO”是不同的机制；数据来源、损失和部署过程应分别说明。$\pi_{0.7}$，第VI—VII节

1.13.8 其他路线分别解决什么问题

GR00T同样把视觉语言条件与连续动作生成分工。阅读版本演进时，要对照具体实现：本节所引用的N1.7快照使用Cosmos-Reason2-2B/Qwen3-VL视觉语言底座，以及流匹配DiT动作头，并改进相对末端动作表示、人体视频数据利用及部署接口。DiT是Diffusion Transformer，即承担迭代生成的Transformer；名称中有Diffusion，并不说明它一定预测DDPM噪声，预测目标要看实际训练配方。N1.7固定仓库快照

相对末端表示把动作表达成相对当前位置的变化，便于不同身体在同类操作中共享部分几何关系；它仍需要坐标系、旋转表示、关节限位和底层跟踪的对应。人的手部运动也不是机器人关节命令，跨身体数据必须经过重建与动作表示转换。模型的数据接口和身体标签，正是这些差异在训练系统中的落点。

SmolVLA关注较小资源预算。它使用紧凑的SmolVLM-2，减少图像标记，并截取视觉语言模型的较早层输出，为较小的流匹配专家提供条件。原论文的层跳过发生在视觉语言解码路径，不能笼统理解成“删掉一半视觉编码器”。它还交替使用交叉注意力和动作自注意力：前者由动作查询读取底座键值，后者在候选动作内部交换信息。SmolVLA作者教程，固定论文，第3节

SmolVLA的动作自注意力使用因果掩码，π系列相应连续动作块则可双向读取。两者都能在每次流迭代中同时计算整个块，因为本轮输入的所有带噪候选已经存在。因果掩码限制一个位置可读哪些候选，不自动要求等待“这一轮刚生成的前一个真实动作”。自回归生成之所以串行，是因为下一个输入需要前一个新输出；这里需要区分掩码与生成变量之间的依赖。

SmolVLA还把预测和执行放进异步队列。机器人消费旧动作期间请求新块，返回后处理时间对齐与重叠。收益取决于推理是否能及时完成、旧条件过时程度及衔接规则；它不是仅凭“小模型”就自动获得的闭环能力。**RTC（Real-Time Chunking，实时动作块衔接）**进一步把推理期间已执行或已承诺的前缀纳入生成，避免新块像从旧时刻重新开始。第11.8节将沿时间轴展开不同衔接方案。

X-VLA针对跨身体与数据来源差异，引入按数据来源索引的可训练软提示。软提示不是一段必须读懂的自然语言，而是一组参与模型计算的向量；动作损失通过梯度调整它们，使共享网络能够区分当前身体或数据域的条件。适配新域时可以学习新的提示及相应适配参数。这给共享与专用部分提供了接口，却没有自动识别未知机器人的关节意义，仍需正确的数据和控制映射。X-VLA固定论文，第3—4节

MoE（Mixture of Experts，混合专家）采用另一种容量分配方式：在网络某层放多组前馈参数，由路由器根据当前标记选择部分组参与计算。LingBot-VLA 2.0把这种稀疏层放在动作专家内部，结合共享专家与被选择的专家，并引入未来视频表示和几何相关监督。一个“动作专家模块”内部还可以包含多个“稀疏专家”，两处专家对应不同层次。路由通常不预先保证某组只负责左臂或抓取，所学分工和负载均衡都需要验证。LingBot-VLA 2.0，第4节

WAM（World Action Model，世界动作模型）则把未来视觉变化与动作共同建模。DreamZero从视频生成底座出发，在同一个网络中联合处理视频和动作的带噪变量，学习二者对应的生成方向；它不是先独立生成一张目标图，再把图交给另一个低层策略。视频监督提供密集的时空变化信息，但也增加训练、生成和对齐成本。DreamZero固定论文，第3节

DreamZero执行一段动作后，用真实新观测更新相应视觉上下文，而不是始终把先前生成的画面当作真实历史继续预测。这样能够修正部分预测历史误差，仍不能保证之前的动作没有碰撞或之后的预测必然准确。评价WAM要区分视频外观、与动作对应的物理后果，以及接入闭环后是否真正改善决策。

联合生成视频和动作，也不自动提供“输入任意外部动作，预测其后果”的程序接口。若模型支持的是联合采样，可以从同一初始状态采样若干视频—动作对，在可复位环境中分别执行所配动作，再核对预测与真实后果；若要固定人为指定动作进行反事实预测，则还需实现相应条件推断。概率模型能够作某种数学分解，与代码已经提供这种操作，是两件事。

RLT（Reinforcement Learning Tokens，强化学习标记）把大VLA的表示接到小型在线学习器。它先训练读出模块，把底座的多个标记压缩为一个可供强化学习使用的表示，再固定VLA和读出模块，在线训练较小的行动网络与评价器。小行动网络还读取VLA采样的参考动作块，并受到接近该参考的约束；在线学习因此围绕已有行为改进。它并非只取一个现成隐藏向量后就默认适合强化学习，也不是直接在线更新整个VLA。RLT，第III—IV节

1.13.9 怎样进行有解释力的路线比较

图1.13-E：四类动作建模路线。图中列出的图像、语言、状态与历史是可能的条件集合，具体模型未必全部使用。离散生成、单值回归、扩散与流匹配各行对应不同的标签和部署计算；它们的对照不应只用参数量或模型名字。

若要比较语言能力，可以在同一场景放两个候选目标，仅切换指令并配对重置，分别测目标选择和选对后的操作成功。若要比较记忆，可以让当前观测相同、过去事件不同，使任务确实需要历史；同时检查历史是否同回合、是否包含未来。若要比较动作分布，可以在同一条件下保留多种合法路径，观察覆盖与碰撞，而不只看平均动作误差。

对视觉子目标，至少区分无目标图、真实未来图辅助、生成目标图三种条件。真实图能帮助而生成图无效，问题可能在目标生成或训练时的条件差异；两者都无效，则应检查策略是否读取目标、任务是否真的需要它。目标图好看与低层使用正确，也是两项不同检查。

对速度或质量条件，应保持当前观测和随机种子相同，只改变条件，再在独立闭环中测成功、耗时和失效方式。若高速度条件让动作更激进但碰撞更多，不能只按耗时判断改善。若某项元数据只与相机背景或机器人型号相关，还应通过跨场景、跨身体测试排查捷径。

最终比较需要固定或分别报告示范、额外视觉语言与人体视频数据、模型初始化、调参预算、硬件、生成步数和执行前缀。相同生成步数不等于相同耗时，相同动作误差也不等于相同成功率。用按墙钟时间、计算和机器人交互量分别统计的学习或运行曲线，可以看清收益来自哪里。

1.13.10 练习题

题1.13-1：视频压缩后的信息在哪里。 对式（1.13-4）完成空间等权读取和因果时间等权读取，列出各帧中间结果。分别计算提前删除历史、修改第一帧以及末尾追加未来帧的结果，解释这些干预检验了什么。再计算6帧、每帧16块时两种注意力分数规模，说明为什么它不等于总延迟之比。单独新回合第一帧为 $[20, 22]$ 时，重置与误留旧回合末两帧的输出各是什么？

题1.13-2：摘要压缩何时丢掉了决定所需的信息。 两段历史分别把红杯放在左柜、右柜，当前图像、身体状态和目标完全相同。摘要都写“收好一只杯子”。两种历史各占一半，评价只允许首次选择一扇柜门。证明任意只读取这些输入的随机策略，其首次选对概率不超过0.5。保留什么信息能解除限制？若允许继续探索，这个界限还约束整回合成功率吗？说明语言摘要的训练标签怎样制作，为什么不能把已发出的指令直接写成已完成事件，以及重复失败应如何在短期与长期记忆中处理。

题1.13-3：$\pi_{0.7}$的目标图与质量条件。 训练用真实未来图像和实际质量标签，部署用生成图像并请求质量5。分别指出两种条件的训练来源、部署来源、改变了什么预测问题，以及为什么都不是成功保证。设计一个能区分目标生成能力和低层目标跟随能力的对照。若新子任务已生效，而旧子任务的图像异步返回，怎样避免把矛盾条件送入低层？为什么直接把真实未来图交给测试策略会改变比较条件？

题1.13-4：把模型版本、方法与执行依赖分开。 画出π_0、$\pi_{0.5}$、$\pi_{0.6}$、$\pi^*_{0.6}$、MEM与$\pi_{0.7}$之间的机制及数据关系，注明FAST、知识隔离、RECAP与RTC各改变什么。解释SmolVLA使用因果动作注意力却仍可逐轮并行计算，与OpenVLA自回归的区别；再比较$\pi_{0.7}$的子目标条件与DreamZero的视频—动作联合建模。最后设计两类实验，分别检验世界动作模型的行动信息与闭环收益，不能只评价画面清晰度。

1.13.11 参考答案

题1.13-1

空间分数为零时，两个键的softmax权重各为1/2；因果时间读取则只在当前和过去的允许集合内重新归一化。因此两组权重矩阵为

$$W_S = \begin{bmatrix} 1/2 & 1/2 \\ 1/2 & 1/2 \end{bmatrix}, \qquad W_T = \begin{bmatrix} 1 & 0 & 0 \\ 1/2 & 1/2 & 0 \\ 1/3 & 1/3 & 1/3 \end{bmatrix}.$$

时刻	原输入	空间读取后	时间读取后
0	[0,2]	[1,1]	[1,1]
1	[4,6]	[5,5]	[3,3]

时刻	原输入	空间读取后	时间读取后
2	[8,10]	[9,9]	[5,5]

最后只保留第三行 $[5,5]$。旧帧输出虽然被删除，旧帧信息已经参与当前表示的计算。先删历史再编码，则只有当前空间均值，结果为 $[9,9]$，处理顺序改变了输入能够影响哪里。

第一帧两块都加6，变成 $[6,8]$，其空间均值变为7。三个时间输出依次为 $[7,7]$、$[6,6]$、$[7,7]$。当前值增加2，正是旧帧均值增加6、以1/3权重进入当前输出的结果。

末尾追加 $[100,100]$ 后，旧三个时刻仍为 $[1,1],[3,3],[5,5]$；新时刻自己可以读取四帧，其输出为 $[(1+5+9+100)/4,\ldots]=[28.75,28.75]$。因果检查应比较同一旧时刻是否改变，不能要求“追加后最后一帧”与“追加前最后一帧”相同，它们代表不同物理时刻。去掉因果限制后，旧时刻若也读新增帧，就会受到未来污染。

这三项检查分别回答：历史是否先进入表示、过去是否影响保留输出、未来是否被禁止。它们没有证明保留了所有细节；例如第一帧从 $[0,2]$ 改为 $[1,1]$，均值不变，最终输出也不变，说明本例压缩确实丢失了部分差别。

6帧、16块时，完整分数矩阵有 $(6\times16)^2=9216$ 个位置；分开读取有 $6\times16^2+16\times6^2=2112$ 个位置，规模比为 $48/11\approx4.36$。只数时间掩码允许项时，时间部分为 $16\times6\times7/2=336$；稠密计算仍可能先生成576个时间分数再遮蔽，不能直接把允许项减少当作实际计算减少。

分数矩阵也只占整网成本的一部分。投影、前馈、图像编码、主干、动作生成和数据搬运都会耗时；分开读取还有自己的调用开销。因此不能据4.36倍推断系统快了4.36倍。原MEM也只在选定层加入时间操作，完整延迟需按实际网络与硬件测量。

新回合只读 $[20,22]$ 时输出为 $[21,21]$。误留旧回合末两帧时，空间均值为5、9、21，输出为 $[35/3,35/3]\approx[11.67,11.67]$。这说明输入混入另一回合的信息，未涉及参数更新；独立新回合应重置相应观测、状态、摘要和旧任务请求，模型权重保持不变。

题1.13-2

令隐藏情况 $H=L,R$ 表示红杯所在柜子，各占1/2；令 x 包含策略全部可读的当前观测、身体状态、目标及摘要。题目给定两种情况下 x 相同，所以策略只能在两种情况下使用同一选择分布。设首次选左、右的概率为 p_L,p_R，则

$$P(\text{首次选对})=\tfrac{1}{2}p_L+\tfrac{1}{2}p_R\le\tfrac{1}{2}.$$

必须二选一时 $p_L+p_R=1$，首次正确率恰为50%；允许放弃时才可能更低。随机数无法恢复输入丢掉的信息。若还存有包含位置的视频或隐藏记忆，就不满足“全部可读输入相同”的前提；若两种情况不是等概率，例如左柜占80%，只按先验选择左柜也可以达到80%。

保留“红杯在左柜”或“红杯在右柜”即可解除本题的信息限制，理论上一个可靠的位置比特就足够。更复杂任务可能还需柜层、杯子身份或门的状态，不能据此认定所有摘要只需一比特。若允许开门观察后继续查另一柜，并有足够时间、检查可靠且开错不立即失败，整回合找到概率可以为1；50%只约束受限的第一次决定。缺信息可能使策略必须探索，不等于永远无法完成。

MEM的摘要监督来自带子任务文字和成功/失败标记的机器人回合。预训练语言模型按这些过去记录制作摘要转移，保留仍对任务有用的事件并压缩重复；高层VLA再学习从旧摘要、已有观测和任务生成新摘要及子任务。若标注输出序列为 y^*，其文字监督可写为

$$L_H = -\sum_j \log p_\theta(y_j^* \mid o_{\le t}, m_t, g, y_{<j}^*).$$

式中观测取实现规定的可用窗口，m_t 是旧摘要，$y_{<j}^*$ 是训练时的正确输出前文；部署改为已生成的前文。该式说明标签怎样提高相应文字的概率，不预设作者实现的具体分词顺序。低层还有自己的动作标签，摘要文字不是关节命令。

时间对齐必须先确定摘要对应哪个观测边界。若当时杯子仍在桌上，不能在输入摘要中写入三秒后才发生的成功；执行已结束、已有观测支持拿起时，新的摘要标签才可以记“杯子已拿起”。离线标注可以校正已发生的事件，不能把将来事实提前变成当前输入。部署判断依靠可得观测，仍可能出错，也应允许新观测纠正旧记忆；论文没有因此提供一个永远准确的成功检测器。

“拿起红杯”是意图，“红杯已在夹爪中”是事件。抓空后若仍记成功，高层就可能让空夹爪继续放杯。重复失败时，长期摘要可以保持“蓝杯未收好”，不必把三条相同指令都保留为已完成事项；短期视频保留接近角度、偏移和滑落等细节。若失败揭示了长期有用的事实，如某门被卡住，摘要也应保留。压缩去除的是冗余，不能一律删除失败。

示范中的一次完成与部署中的多次尝试会产生不同历史。摘要合并可减轻重复指令积累，这是作者任务中的实验解释；它不保证压缩越强越好。丢掉杯子位置的反例已经说明，压缩必须以未来决定需要的信息为准。

题1.13-3

视觉子目标指定希望达到的状态，质量条件区分行为怎样完成。二者的来源如下：

条件	训练来源	部署来源
子目标图	轨迹后续真实图像，以及生成器产生的目标图	生成器根据当前观测、子任务和元数据产生，也可由合法的用户/规划接口提供
质量与错误字段	该回合实际质量评分、片段错误标注；速度来自回合长度	调用者请求质量5、无错误及希望的速度

图像生成器以当前观测和子任务等为条件，学习片段结束状态；动作策略则学习在目标图条件下产生动作，原方法还使用片段内未来图与生成图的混合。生成器的预测标签与策略读取的条件不是同一职责。

把原条件统称为 C，用 b 表示质量、速度等元数据，原来的 $p_\theta(A \mid C)$ 变为 $p_\theta(A \mid C, G, b)$。目标图 G 区分杯子应落在托盘哪个位置，b 区分同一任务中顺畅或反复抓空等不同行为。训练依然使用这条记录实际发生的动作，不会因添加低质量标签就产生一条从未发生的优质动作。

已知当前图和未来目标图时，动作预测带有**逆动力学（inverse dynamics）**的性质，即根据起点与希望到达的状态推断连接它们的动作。图像不能一般唯一确定动作，遮挡、时长、接触和多条合法

路径仍有影响。质量条件也不同于评价任意候选后果的价值网络：部署请求质量5是在要求采取类似高质量数据的行为，尚未测量未来结果。

目标可能生成错误、视角不一致或不可达，低层也可能不会跟随；某场景缺少高质量数据，质量字段也不能补出正确动作。训练中混入生成图，是让策略面对生成器误差，减轻只见完美真实图造成的条件差异，并非保证所有误差都能恢复。

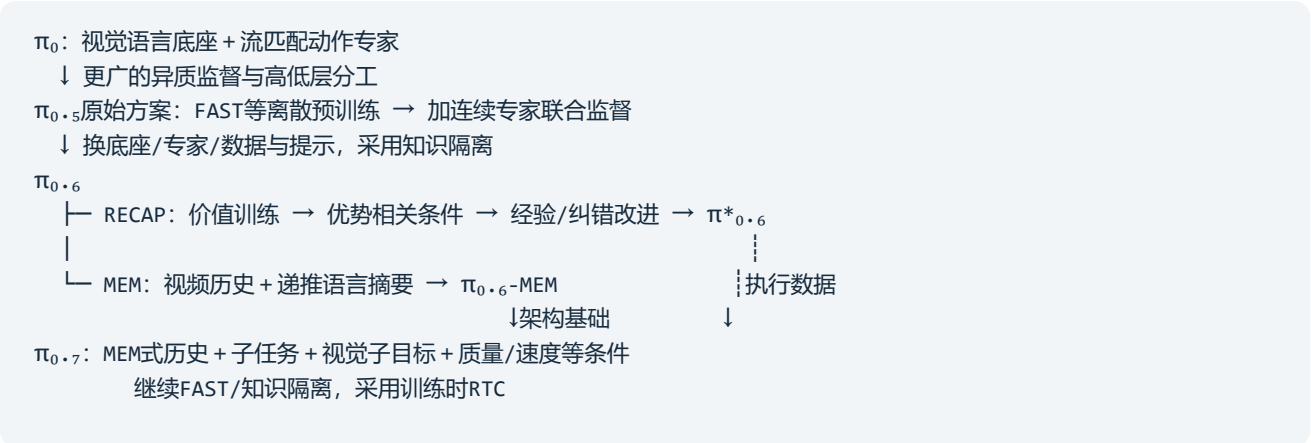
可固定低层权重、任务、元数据、当前观测与生成预算，在配对初始状态比较三组：无目标图、真实可达目标图辅助、实际生成图。真实图可以在可精确复位的环境中从独立参考执行获得，必须明确它提供了额外信息。还可保持“放托盘”文字不变，把目标图中的落点改为两个可达区域，检验真实落点是否跟着改变，排除模型忽略图像只靠语言完成的解释。

对生成器检查意图、几何和多视角一致性，对低层检查到达状态、成功、碰撞和耗时。真实图有用、生成图无用，提示生成或条件差异可能是瓶颈；两者都无用，则检查是否读取图、图是否可达、任务是否需要图。还需这些辅助检查，才能进一步定位原因，不能仅凭一项成功率变化唯一归因。

异步接口可附带回合ID、递增子任务版本、元数据版本和观测时间。抓杯版本11发出的图像请求，在高层切换到放杯版本12后返回，不能再与版本12指令混用；只看文字是否相同也不够，因为稍后可能再次出现同名子任务。返回时检查所属版本与时效，再提交一致的条件快照。新图未就绪时，可以使用训练支持的无目标图条件，或采用执行系统已验证的等待方式；具体选择需按任务测量。这个版本规则是接口建议，不冒充论文已公开的实现。作者4秒刷新阈值也不是任意旧图都有效4秒的保证。

若测试把实际未来帧提供给策略，就增加了未来状态或路线信息，不能再与仅读当前观测的基线当作同条件比较；从同一评测行为的结果回填，还会引入结果与条件相互依赖。应明确标为目标辅助诊断，或让各方法获得相同的合法目标条件；部署结果仍使用部署真正能提供的目标来源。

题1.13-4



竖向箭头表示设计或配方延续，虚线表示数据来源。$\pi_{0.7}$使用早期强化学习策略收集的执行记录，不等于直接复制某次任务专家的参数。各版本还同时改变数据、规模和接口，整版本差异不能单独归因于某个方法。

方法	实际改变什么
FAST	把连续动作序列压成离散监督；它本身不是连续动作采样器

方法	实际改变什么
知识隔离	规定连续损失到哪些参数停止回传；不阻止专家前向读取底座，也不冻结所有底座学习
RECAP	用独立价值与回报构造优势相关标签，把不同质量经验接入改进；positive按任务阈值划分，不必等于数值优势大于0
MEM	保存和压缩可读的过去；回合内更新摘要可以不改变权重
RTC	推理期间机器人仍在执行，新动作需考虑已执行或已承诺前缀；训练时模拟延迟与推理时衔接是相应方案

SmolVLA某轮已有完整候选 $X^{(r)} = [x_1^{(r)}, \dots, x_H^{(r)}]$。因果掩码规定

$$v_i^{(r)} = f_i(X_{\le i}^{(r)}, C, \tau_r), \qquad X^{(r+1)} = X^{(r)} + \Delta\tau\, v^{(r)}.$$

所有位置读取的旧候选在本轮开始时已知，因此可以通过下三角掩码一次算出全块速度；位置 i 不等待刚更新的 $x_{i-1}^{(r+1)}$。例如候选 $[2, 6, 10]$ 做因果等权读取，结果为 $[2, 4, 6]$，取步长0.1可同时得到 $[2.2, 6.4, 10.6]$。若边算边覆盖，再让后位置读取新值，就换了一种算法。整块本轮可并行，下一轮仍需等本轮结束。

OpenVLA按 $\prod_i p(z_i \mid C, z_{<i})$ 生成动作标记，推理开始时只有 C；先生成 z_1，才能得到预测 z_2 所需前缀，所以有串行依赖。训练用已知正确前文可以并行计算损失，缓存也能减少重复，却不取消部署时对新输出的等待。掩码规定读谁，生成流程还取决于输入是否已存在。

$\pi_{0.7}$先由图像模块产生目标 G，动作策略再以 G 为条件生成 A；两个模块可以异步运行。DreamZero则在一个共享生成网络中联合处理未来视频潜变量 Y 与动作 A，学习 $p(Y, A \mid \text{历史、任务、身体状态})$。联合分布可在数学上因子分解，但它的实现不是两个独立模型的串联。DreamZero还在视频块之间使用自回归上下文，块内通过流匹配迭代生成；自回归和流匹配可以处于不同层次。

第一类实验检查行动相关信息。固定初始场景和生成预算，联合采样多组 $(\hat{Y}_i, \hat{A}_i)$，在可复位环境中分别执行所配动作，比较预测与实际的接触、杯子位姿、抓取成败、放置及碰撞事件，并对齐物理时间。可打乱视频与动作配对，检查原配对是否更能解释实际后果；若候选动作几乎相同，错配测试也缺少分辨力。测量应来自独立状态记录或经过核对的感知，不能只看模型自评分。若要输入外部任意动作做反事实预测，要先实现并验证该条件推断接口；联合采样本身不提供这一保证。

第二类实验检查闭环收益。对配对初始状态，比较完整WAM、资源匹配的动作策略及关键的视频监督/历史反馈消融；保持相机、动作接口、成功标准可比，分别报告成功、约束违反、耗时、恢复和接管。记录模型耗时与动作执行衔接，避免一个系统异步、另一个系统空等却把差异全归给世界建模。示范、额外视频、初始化、调参、计算和交互成本也需控制或报告。

执行后写入真实观测与继续沿用预测视觉历史，可在受控环境中比较反馈路径的作用。更准确的视频可能因延迟而没有改善闭环；更好的闭环也可能来自动作专家或重规划方式。两类实验分别检验行动信息和系统使用该信息的效果，结合起来才能支持具体机制，而非只凭画面清晰度下结论。

1.14 参数适配、联合训练与知识隔离

将视觉语言模型接到机器人上，至少要解决两件事：让原有表示适应机器人任务，并让模型输出能及时交给控制器的动作。它们相互联系，却不必使用同一种训练目标。一个视觉语言底座可以通过离散动作预测学习“哪些图像信息与动作有关”，再由较小的连续动作专家学习“怎样根据这些信息生成连续动作块”。

这里的**底座**是承接预训练权重、处理视觉和语言等输入的网络，**动作专家**是专门处理连续动作生成的网络。专家并不是一个外部标注者，也不必先生成文字再把文字翻译成动作。本节关心的是这两个网络怎样连接、哪些参数参与更新，以及训练时能看到的信息在部署时是否仍然存在。

1.14.1 适配模型时，究竟在改变什么

预训练通过较广的数据学习可迁移表示，**监督微调**（Supervised Fine-Tuning，SFT）再用目标任务的标签改变模型。用示范中的观测和动作训练策略，既可以称为**行为克隆**（Behavior Cloning，BC），也可以是一次任务监督微调。“微调”描述训练所处的阶段，“行为克隆”进一步说明监督来自专家动作。仅凭阶段名称，无法判断模型是否使用了强化学习。

如果保留预训练权重而只训练新的动作头，底座表示不会被动作损失改变。这样可以减少需要更新的参数，但动作头只能利用底座已经提供的信息。如果原表示没有充分保留精细的夹爪位置、接触状态或机器人构型，仅增加输出层未必够用。反过来，放开全部参数又需要更多训练资源，也可能改变原有的语言和视觉能力。适配方法实际在选择：允许模型在哪里、以多大的自由度改变。

几种常见选择可以沿着这条线理解。**局部解冻**直接更新部分原权重；**适配器**（adapter）在网络内部增加较小的可训练模块；**软提示**增加可训练的输入向量；低秩适配则为指定权重增加受约束的修正。它们节省的资源、能改变的计算关系和部署开销各不相同。选择时除了比较目标任务，还要在相同数据和训练预算下检查旧任务表现、语言跟随和实际运行成本。

1.14.2 LoRA：给原映射增加一条可学习的修正

LoRA（Low-Rank Adaptation，低秩适配）的出发点是：不直接训练一个与原权重同样大的增量，而是用两个较小矩阵表达增量。为避免与后面的动作块 A 混淆，这一小节将它们写成 A_L, B_L。设原线性层为 $y = W_0 x$，其中 $x \in \mathbb{R}^{d_{\text{in}}}$、$W_0 \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$，则适配后的计算为

$$y = (W_0 + sB_L A_L)x, \qquad A_L \in \mathbb{R}^{r \times d_{\text{in}}}, \quad B_L \in \mathbb{R}^{d_{\text{out}} \times r}. \tag{1.14-1}$$

W_0 固定，A_L, B_L 可训练，s 是固定缩放系数。$A_L x$ 先将输入组合成 r 个方向，再由 B_L 把这些方向组合成输出修正。因此 $\operatorname{rank}(B_L A_L) \le r$：限制的是本次适配的增量，原来的 W_0 可以是满秩矩阵。较小的 r 减少训练参数，也减少允许的变化；如果任务需要复杂的表示调整，秩太低或插入位置不合适就可能限制效果。LoRA 原论文

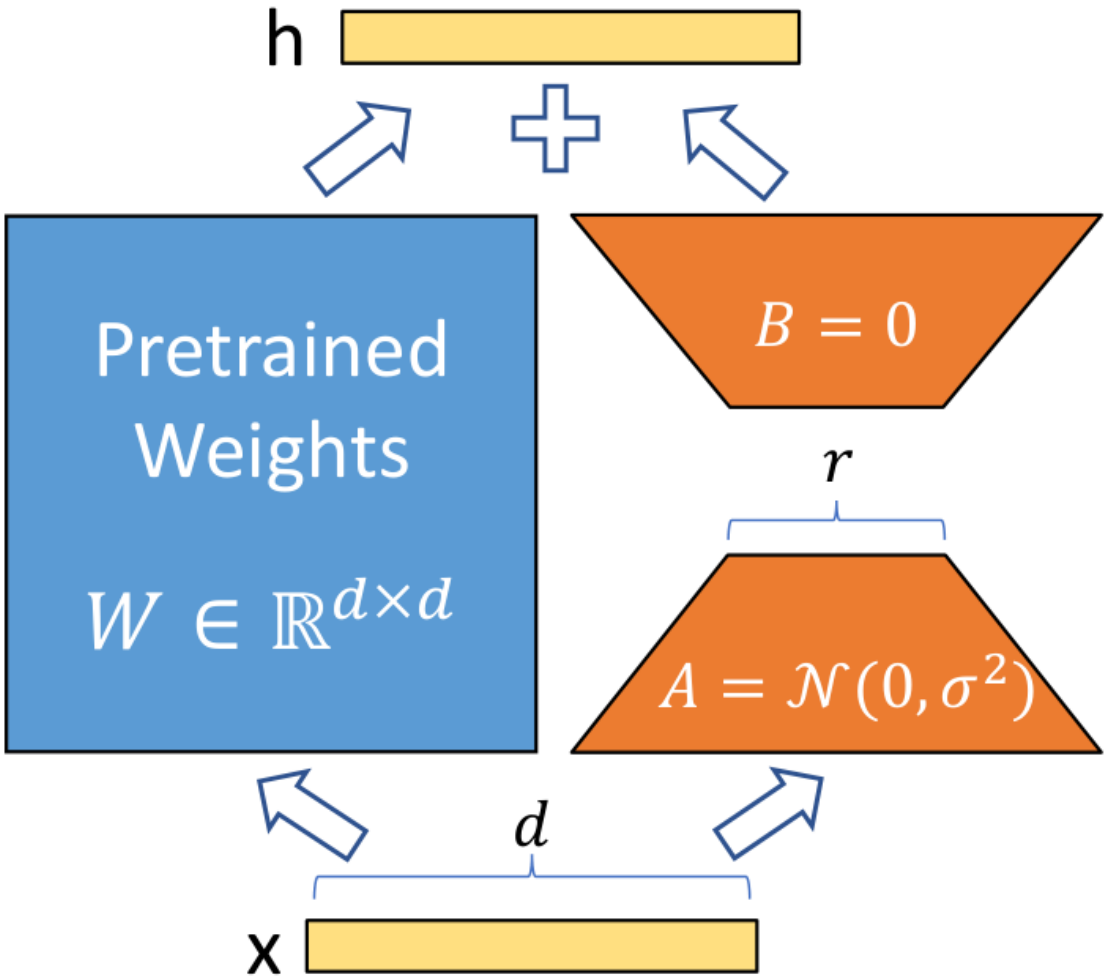


图1.14-A Hu等，LoRA论文Figure 1。由原始PDF中的图重新渲染，图内内容保持原样。

图从下向上读。x 是层输入，h 是层输出，Pretrained Weights表示预训练权重。左侧直接算 W_0x；右侧先经过 A_L，再经过 B_L，最后在加号处相加。图里的 d 是示意用的输入、输出宽度，r 是中间宽度。$A = \mathcal{N}(0, \sigma^2)$ 表示随机初始化，$B = 0$ 表示零初始化。不同实现可以使用不同的非零随机初始化方式；决定下面启动过程的是“一侧非零、另一侧为零”的关系。

为什么不把两个小矩阵都初始化成零？令输出处的损失梯度为列向量 $g = \partial L/\partial y$。先记 $z = A_Lx$，则修正为 sB_Lz。按链式法则，

$$\frac{\partial L}{\partial B_L} = s\, gz^\top = s\, g(A_Lx)^\top, \qquad \frac{\partial L}{\partial A_L} = s\, B_L^\top gx^\top. \tag{1.14-2}$$

第一式说，B_L 根据输出误差和当前低维表示 z 改变；第二式说，A_L 要先通过 $B_L^\top$ 接收输出误差。如果两者都为零，两条梯度都为零，分支无法从这个起点开始学习。只令 $B_L = 0$，初始增量仍为零，模型保留原映射；只要 A_Lx 和输出误差提供有效信号，B_L 就能先更新，随后 A_L 也开始改变。

“A_L 非零”不保证每个样本都使 B_L 更新。例如 $A_L = [1, -1]$、$x = [1, 1]^\top$，仍有 $A_Lx = 0$；即使预测有误差，该样本的两个因子梯度也都是零。批量训练时，还可能出现不同样本的梯度相互抵消。初始化提供了通常可用的学习通道，并没有保证每一步都出现非零更新。

输入也有自己的梯度：

$$\frac{\partial L}{\partial x} = (W_0 + sB_LA_L)^\top g. \tag{1.14-3}$$

假如 x 来自前面一个可训练视觉模块，这个梯度就会继续更新视觉模块。固定 W_0 只是不更新它，不会删除乘法 W_0x 对输入的导数。把整个层放进 torch.no_grad() 会使这次输出不再连接到 A_L, B_L 或输入的求导路径；这与只固定 W_0 有不同的训练效果。第1.2节的计算图区分，在这里直接决定哪些模块能够学习。

1.14.3 从零增量到三次实际更新

取 $s=1$、$W_0 = I_2$、$A_L = [1, -1]$、$B_L = [0, 0]^\top$，输入 $x = [1, 2]^\top$，目标 $y^* = [1, 0]^\top$。损失取 $L = \frac{1}{2}\|y - y^*\|^2$，用学习率 0.1 的**随机梯度下降**（Stochastic Gradient Descent，SGD）更新两个因子。

初始 $A_Lx = -1$，但 $B_L = 0$，所以输出仍是 $[1, 2]^\top$，$g = [0, 2]^\top$。式（1.14-2）给出 $\nabla_{A_L}L = [0, 0]$、$\nabla_{B_L}L = [0, -2]^\top$。更新后 $B_L = [0, 0.2]^\top$，而 A_L 未变。第二次前向的修正是 $[0, -0.2]^\top$，输出变成 $[1, 1.8]^\top$；此时

$$\nabla_{A_L}L = [0.36, 0.72], \qquad \nabla_{B_L}L = [0, -1.8]^\top.$$

第二次更新得到 $A_L = [0.964, -1.072]$、$B_L = [0, 0.38]^\top$，下次前向的低维表示也变了：$A_Lx = -1.18$。以下各行表示一次更新之前的数值，最后一行表示已经完成三次更新：

已完成更新次数	A_L	$B_L^\top$	输出的第二维	损失
0	$[1, -1]$	$[0, 0]$	2.000000	2.000000
1	$[1, -1]$	$[0, 0.2]$	1.800000	1.620000
2	$[0.964, -1.072]$	$[0, 0.38]$	1.551600	1.203731
3	$[0.9050392, -1.1899216]$	$[0, 0.5630888]$	1.169554	约0.683929

这里的每一步同时使用更新前的两个因子计算梯度，再一起更新；不是先改 B_L，再用新 B_L 计算同一步的 A_L 梯度。原权重始终是单位矩阵。这个例子只拟合一个线性样本，作用是展示分支如何启动，不代表已经学会了一个机器人任务。

对应实现可以完整保留原线性层。伪代码与注释使用相同的顺序：

```
L1  固定原线性层W0，初始化非零A、零B。
L2  计算低维表示Ax，再计算修正B(Ax)。
L3  输出W0x+B(Ax)，由损失反传并更新A、B。
L4  部署前需要时形成Wmerged=W0+BA，替代两条线性分支。
```

```
import torch
from torch import nn
import torch.nn.functional as F

class LowRankLinear(nn.Module):
    def __init__(self, base_layer, rank):
        super().__init__()
        # L1：保留原映射，固定其参数；A非零、B为零。
        self.base = base_layer
        self.base.requires_grad_(False)
        self.A = nn.Parameter(
            base_layer.weight.new_empty(rank, base_layer.in_features)
            .normal_(0.0, 0.01)
        )
        self.B = nn.Parameter(
            base_layer.weight.new_zeros(base_layer.out_features, rank)
        )

    def forward(self, x):
        # L2：先降到低维，再生成输出修正。
        correction = F.linear(F.linear(x, self.A), self.B)
        # L3：两条分支都保留对输入的求导关系。
        return self.base(x) + correction
```

代码中的 A,B 对应 A_L, B_L，缩放取1。完整运行入口是 practice/lora_paths.py，它复用 practice/low_rank.py 中的层，记录三次前向、因子梯度、输入梯度和更新结果。训练完的线性修正可合并：

$$W_{\text{merged}} = W_0 + sB_LA_L. \tag{1.14-4}$$

合并后只需一次原尺寸的线性映射。它省去单独执行适配分支，不能把原矩阵自动变成小矩阵。一个 1024×1024 权重有1,048,576个元素，秩8的两个因子共有16,384个可训练元素，约占1.56%；训练时减少的主要是可训练参数的梯度和优化器状态，部署合并后的稠密矩阵仍有1,048,576个元素。底座前向与必要的激活也仍然存在。PEFT的LoRA指南

量化LoRA还涉及用低精度保存底座、用适配参数学习修正，不能与第1.11节的动作编码混用：前者处理模型权重的存储与计算，后者处理动作的表示。训练显存、通信和量化部署分别见第10、11章。

1.14.4 知识隔离：让两种监督承担不同职责

LoRA回答的是“允许怎样修改权重”。接下来是另一个问题：如果同一个底座同时服务于文字预测、离散动作预测和连续动作生成，是否让所有损失都回传到底座？

设底座参数为 ϕ，连续动作专家参数为 ψ。专家读取底座特征 $h_\phi(c)$，从带噪动作 x_τ 预测流速度 $v_\psi(x_\tau, \tau, h_\phi(c))$。如果直接联合训练，连续损失到达底座时，必须先经过专家对特征的导数：

$$\nabla_\phi L_F = \left(\frac{\partial h_\phi}{\partial \phi}\right)^\top \left(\frac{\partial v_\psi}{\partial h_\phi}\right)^\top \frac{\partial L_F}{\partial v_\psi}. \tag{1.14-5}$$

新加入的专家开始时通常尚未训练。中间这项导数由它当前的参数决定，未必把动作误差转换成有利于原视觉语言表示的修改方向。再加上机器人数据可能让场景与动作高度相关，模型即使弱化语言也能降低训练误差。这样就出现了一个值得检验的风险：连续动作损失下降，原有语言条件却越来越难影响行为。它是训练动力学和数据共同造成的可能结果，不是连续动作目标必然有害的定理。

知识隔离（Knowledge Insulation）为此分配两种学习职责：底座用下一token预测学习语言与离散动作相关表示；连续专家读取这些表示，用流匹配学习连续动作，但其损失不经读取接口回传到底座。视觉语言数据还可以与机器人数据混合，使底座继续接触原有类型的监督。作者讲解

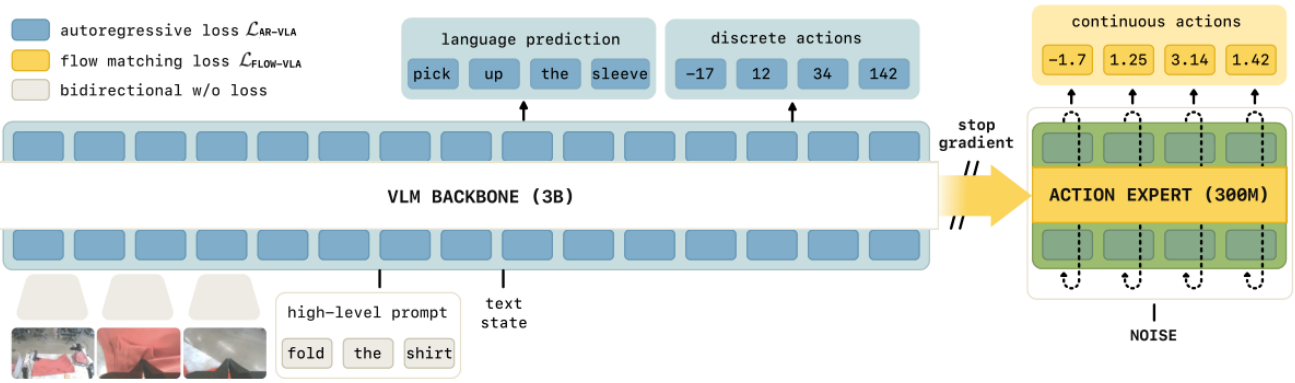


图1.14-B Driess等，知识隔离论文Figure 1。图展示同一训练系统中的离散监督与连续动作生成。原论文

图中VLM BACKBONE是视觉语言模型（Vision-Language Model，VLM）底座，ACTION EXPERT是动作专家；3B与300M分别是图示模型约30亿、3亿参数的规模。high-level prompt是给定的任务要求，text state是用文字编码的机器人状态。上方language prediction和discrete actions分别表示文字及离散动作预测；continuous actions是连续输出。左侧autoregressive loss表示自回归下一token损失，flow matching loss表示流匹配损失，bidirectional w/o loss表示可双向读取、该位置不直接承担预测损失。NOISE是连续生成的初始噪声。

底座到专家的箭头表示前向读取，stop gradient表示在这个接口停止反向导数；它没有把特征数值删掉。专家内的虚线回路表示反复细化连续候选，不能理解成多次执行物理动作。部署时可使用较小专家生成连续动作，不必为了这项辅助监督再逐个解码训练中的离散动作标签。

知识隔离也不是“把底座全部冻住”。底座仍由离散动作和文字监督改变。只停止连续梯度却不给底座任何任务监督，会留下固定表示是否足够的问题。反过来，保留所有连续梯度也并非必然失效；论文的消融中，加入视觉语言数据后，不使用停止梯度的联合训练也能取得较好的语言跟随。方法的价值在于一种可验证的训练分工，而不是保证旧知识永远不变。原论文§6

从技术演进看，π0提供连续动作专家，π0-FAST使用离散动作编码；π0.5最初采用先离散训练、再加入连续专家的两阶段过程。知识隔离论文进一步将离散表示学习与连续专家训练组织成可同时进行的一阶段配方。本节解释这份配方的机制，各版本的数据和完整训练流程见第1.13节。原论文§2

1.14.5 先放进一个共同训练样本，再区分三种掩码

假设机器人当前要把红杯放到托盘。决策条件 c 可以包括此刻已经获得的图像、任务指令和本体状态。示范额外提供未来 H 步的动作块 $A \in \mathbb{R}^{H \times d_a}$。同一个 A 可以生成离散动作标签 $d_1, \ldots, d_m$，也可以用来构造连续流匹配训练对。标签是训练记录，并不会因为两条分支一起训练，就自动成为连续策略部署时可用的条件。

字段	从哪里来	训练时的职责	连续动作部署时是否可直接获得
c	当时传感记录与给定任务	共同条件	是，受实际传感时延限制
A	示范中的未来动作块	连续标签；动作编码的来源	否
$d_1, \ldots, d_m$	动作编码器处理A	离散分支的监督目标	真实标签不可得
ω	训练时采样的噪声	构造带噪候选	可另行采样初始噪声
τ	采样的内部生成进度	告诉专家当前噪声阶段	可由生成器设置
x_τ	$(1-\tau)\omega + \tau A$	连续分支的输入	推理时由当前生成候选提供
$u = A - \omega$	示范与本次噪声	连续分支的速度标签	否

这里沿用第1.10节的噪声到数据方向：$\tau = 0$ 对应噪声，$\tau = 1$ 对应动作。u 是模型内部生成路径的速度，单位和机器人末端物理速度不同。推理时不使用未知的真实 A 重建 x_τ，而是从噪声开始，按网络预测的速度逐步更新候选。

离散分支使用**教师强制**（teacher forcing）：将正确序列右移后输入，一次计算各位置的下一token预测。例如目标是 $[d_1, d_2, \mathrm{EOS}]$，输入为 $[\mathrm{BOS}, d_1, d_2]$。BOS（Beginning of Sequence）是开始标记，EOS（End of Sequence）是结束标记。输入位置 D_0 预测 d_1，D_1 预测 d_2，D_2 预测EOS。因此，允许一个位置读取自己的输入，并不等于允许它读取自己应预测的下一标签。

这时需要区分三种操作。

操作	它回答的问题	在本例中的作用
损失掩码	哪些位置、哪些样本有监督标签？	离散后缀承担下一token损失；没有动作标注的视觉语言样本不承担连续动作损失
注意力掩码	某个查询可以读取哪些输入？	连续动作候选不能读取教师强制中的真实动作token；离散分支也不读取带噪连续动作
停止梯度	某项损失沿已存在的计算，能更新哪些上游参数？	专家仍读取底座键和值，但连续损失在读取处不回到底座

损失掩码不会阻止中间特征偷看答案；停止梯度也不会。假如专家看到了真实动作token，它仍能利用这些数值降低预测误差，即使不能修改生成它们的参数。要消除这种输入依赖，必须改变前向可见性。

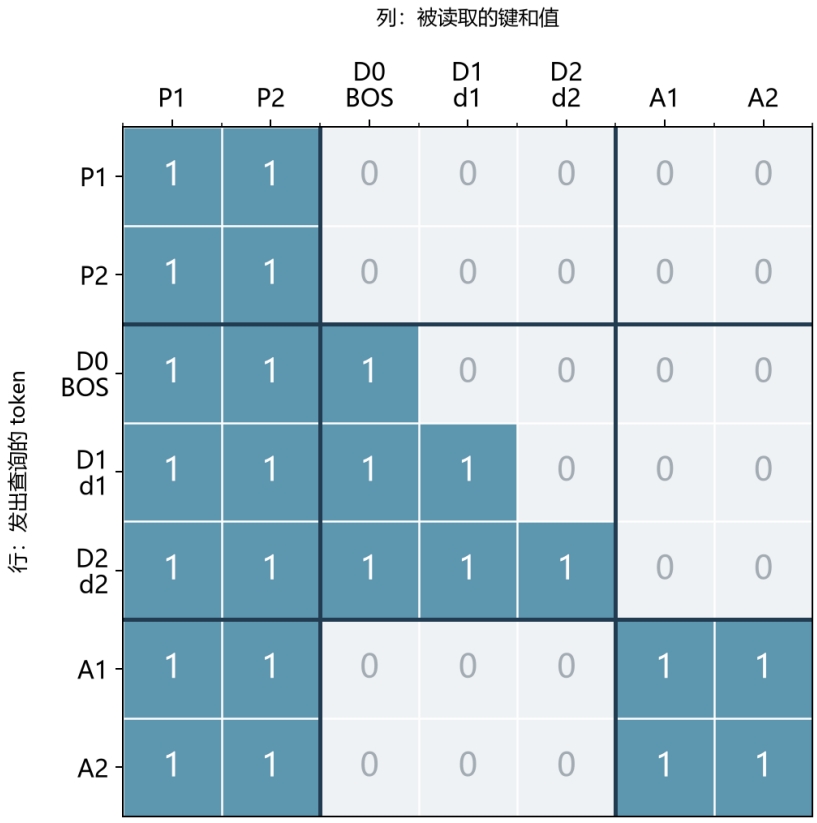


图1.14-C 两条件token、三离散输入位置、两连续候选位置的允许矩阵。行发出查询，列提供键和值；1允许读取，0在softmax之前屏蔽。矩阵与本节运行代码一致。

图中的 P_1, P_2 表示部署可用条件，D_0, D_1, D_2 是右移的离散输入，A_1, A_2 是两个带噪连续候选。P 之间可以相互读取，但不能读取未来答案；D 读取 P 与截至当前位置的 D；A 读取 P 和所有当前候选 A，不读取 D。尤其要保留 $P \not\leftarrow D$：否则即使禁止 A 直接读 D，真实标签仍可能先进入 P，再被下一层的专家间接读走。

连续候选之间双向读取是合理的：整个带噪动作块在这一次网络调用时已经存在。它们不是尚未观测到的真实未来。离散分支不能读取这些候选，是因为训练构造的 x_τ 含有真实动作信息，而自回归解码离散动作时没有这份训练候选。两种生成过程所拥有的条件不同，训练连接必须与各自使用方式相容。

1.14.6 为什么每层的键和值都要处理

把一个注意力层中的底座表示记为 X_b，专家表示记为 X_a。线性投影得到底座的键和值 K_b, V_b，以及专家的查询、键和值 Q_a, K_a, V_a。若本头宽度为 d_h，底座有 N_b 个token、专家有 N_a 个token，则 K_b, V_b 为 $N_b \times d_h$，专家的三个投影为 $N_a \times d_h$。专家读取的注意力结果可写成

$$Z_a = \mathrm{softmax}\left(\frac{Q_a[\mathrm{sg}(K_b); K_a]^\top}{\sqrt{d_h}} + M_a\right)[\mathrm{sg}(V_b); V_a]. \tag{1.14-6}$$

分号表示沿token维拼接；d_h 是键、查询的宽度；M_a 在允许位置为0，在禁止位置为 $-\infty$。这样禁止位置的softmax权重为0。底座的离散动作位置仍被图1.14-C屏蔽，只读取其中可用的条件部分。**停止梯度算子** $\mathrm{sg}(z)$ 前向返回同一个 z，反向将经过该处的导数置为零；PyTorch中的 z.detach() 可以实现这一读取方式。这是有意指定的反向规则：本项损失将读入条件看作给定数值，不通过它调整来源网络。其他底座分支仍可使用未分离的张量求导。知识隔离论文§5.2

为什么同时处理 K_b 和 V_b？因为它们各有一条回传路线。值决定“被读出的内容”，键参与决定“各份内容占多大权重”。将值分离后，损失仍可通过注意力分数更新键；将键分离后，损失仍可通过读出的内容更新值。分离键也不固定整个注意力权重，因为专家查询仍可改变分数。用一个标量查询、两个键和值，就能把两条路线算清。

取 $q = 1$，$k = [0, \log 3]$，$v = [0, 2]$。分数为 $[0, \log 3]$，注意力权重 $\alpha = [1/4, 3/4]$，输出 $z = \sum_j \alpha_j v_j = 1.5$。假设目标0，$L = \frac{1}{2} z^2 = 1.125$，那么

$$\frac{\partial L}{\partial v_j} = z\alpha_j, \qquad \frac{\partial L}{\partial k_j} = z\alpha_j (v_j - z)\, q. \tag{1.14-7}$$

第二式来自softmax的导数。若把某个键的分数增加一点，该位置权重增加，其他位置权重相应减少，所以输出变化取决于它的值 v_j 与当前加权平均 z 的差。代入本例，

$$\nabla_v L = [0.375, 1.125], \qquad \nabla_k L = [-0.5625, 0.5625].$$

因此只分离键时，两个值仍收到梯度；只分离值时，两个键仍收到梯度。两者都分离后，来自此读取接口的底座梯度才停止。查询仍然可以学习：本例 $\partial L / \partial q = 0.5625 \log 3 \approx 0.6179694$。专家可以改变自己的提问方式，底座提供的内容和检索地址则在这项损失下作为给定条件。

专家读取方式	当前输出	沿键回到底座	沿值回到底座	专家查询可否学习
不停止	1.5	可以	可以	可以
仅停止键	1.5	停止	可以	可以
仅停止值	1.5	可以	停止	可以
键和值都停止	1.5	停止	停止	可以

practice/attention_routes.py 计算这四种情况。前向输出与损失相同，自动求导路径不同。这也是为什么不能仅凭loss相等判断训练等价。

如果专家在每个Transformer层都读取底座，就要在每次跨模块读取处处理这些路径。只对最后一层做分离，前面层读入的底座信息已经进入专家隐藏表示，损失还可以经这些较早的读取回传。实现时需要检查实际连接；若还有共享投影或其他读取支路，也必须一并分析。

可以逐层理解这个结论：如果专家的初始表示不依赖 ϕ，第一层又只读取分离的底座特征，那么第一层专家输出对 ϕ 的导数为0。第二层只连接这样的专家表示与新的分离特征，导数仍为0，后续各层同理。反之，只要第一层保留了未隔离的读取，最后一层即使停止自己的键和值梯度，也仍可通过上一层专家表示回到那条早期路径。

1.14.7 联合损失怎样分配梯度，三次更新发生了什么

底座使用的下一token损失是**交叉熵**（Cross-Entropy，CE）：给定正确目标，惩罚模型为它分配过低的概率。先明确平均方式。用 b 表示样本、j 表示下一token预测位置，m_{bj} 表示该位置是否有监督。总有效token数为 $N_C = \sum_{b,j} m_{bj}$，则

$$L_C = -\frac{1}{N_C} \sum_{b,j} m_{bj} \log p_\phi(d_{bj} \mid \text{该位置之前的允许输入}). \tag{1.14-8}$$

其中 d_{bj} 泛指该位置的文字或动作目标。连续损失只在具有动作标注的样本上计算。如果所有有效动作块共有 N_F 个标量元素，取

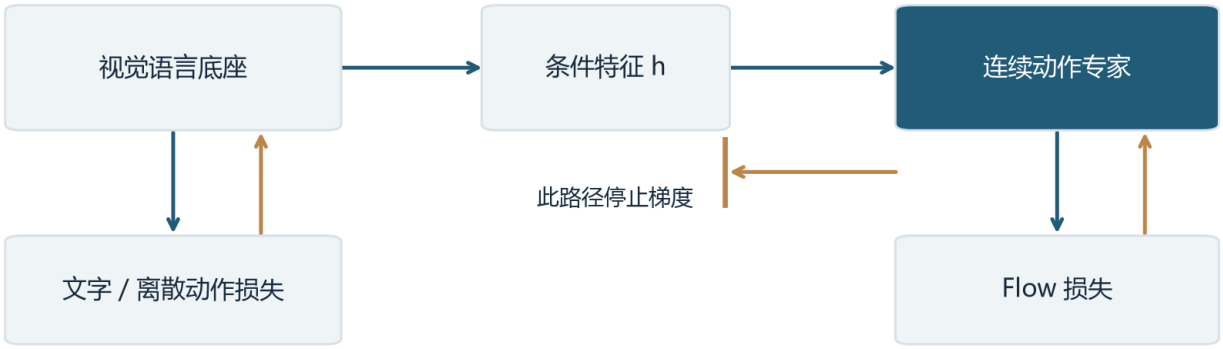
$$L_F = \frac{1}{2N_F} \sum_{b:\ \text{有动作}} \left\| v_\psi(x_\tau^{(b)}, \tau, \mathrm{sg}(h_\phi(c_b))) - (A_b - \omega_b) \right\|_F^2, \qquad L = L_C + \lambda L_F. \tag{1.14-9}$$

$\|\cdot\|_F^2$ 是动作矩阵各元素平方之和。这里将逐层底座键和值简写为条件 h_ϕ，实际实现仍按式（1.14-6）逐层读取。若整个批次没有动作标签，就不计算连续项；把缺少的动作填成0再纳入损失，会变成一项错误的“执行零动作”监督。

在底座与专家参数分离、没有其他跨模块路径的条件下，更新关系为

$$\nabla_\phi L = \nabla_\phi L_C, \qquad \nabla_\psi L = \lambda \nabla_\psi L_F. \tag{1.14-10}$$

知识隔离示意：底座仍可由其他损失更新



蓝色：前向信息。橙色：梯度去向。连续专家不读真实离散动作答案。

图1.14-D 两种监督的分工。蓝色箭头传递前向数值，橙色箭头表示梯度方向；连续损失在读取接口停止，离散监督仍可更新底座。图里的条件特征h概括了实际每层的键和值接口。

连续监督不会直接更新底座，底座也不会因此静止。一次文字或离散动作更新后，$h_{\phi_{\text{new}}}(c)$ 通常与原来不同；下一次专家读取的是新特征。知识隔离改变的是每一步的导数路径，仍然允许两个网络在训练过程中相互适应。它也不禁止在底座内部使用LoRA：这时 ϕ 中只有选定的适配参数参与式（1.14-10）的底座更新。

下面用两层单头注意力检验这些关系。每个表示宽度为2，省略前馈层与归一化，只保留投影、允许掩码和残差，便于单独观察读取路径。两个条件向量是 $P_1 = [1, 0.1]$、$P_2 = [-0.2, 0.8]$，不是由机器人相机提取的真实特征。词表只有6个标记，其中两个教学动作标记的ID为1和2。真实训练中它们应由动作编码器从示范得到，本例直接给定这两个ID，不运行真实FAST编码。

机器人样本的连续数据为

$$A = \begin{bmatrix} 0.5 & -0.2 \\ 0.1 & 0.4 \end{bmatrix}, \quad \omega = \begin{bmatrix} -0.3 & 0.2 \\ 0.7 & -0.6 \end{bmatrix}, \quad \tau = 0.4.$$

所以

$$x_\tau = \begin{bmatrix} 0.02 & 0.04 \\ 0.46 & -0.2 \end{bmatrix}, \qquad u = A - \omega = \begin{bmatrix} 0.8 & -0.4 \\ -0.6 & 1.0 \end{bmatrix}.$$

离散输入为 $[0, 1, 2]$，目标为 $[1, 2, 5]$，0和5分别代表BOS与EOS。再加入一个只有视觉语言监督的样本：它的输入为 $[0, 3, 4]$、目标为 $[3, 4, 5]$，没有连续动作标签。两个样本共有6个有效token位置，连续项只有4个动作元素。

固定初始化时，机器人样本三个目标的预测概率约为 $[0.054122, 0.146821, 0.209251]$，视觉语言样本约为 $[0.238264, 0.094326, 0.163050]$。将六个负对数相加再除以6，得到 $L_C = 2.00139133$。专家初始预测为

$$\hat{u} = \begin{bmatrix} 0.15886364 & 0.11278034 \\ 0.07469761 & 0.04829347 \end{bmatrix}.$$

四个元素与 u 比较，按式（1.14-9）得到 $L_F = 0.25437021$。取 $\lambda = 0.5$，总损失约为2.12857644。网络此时在预测内部流速度，还没有生成并执行一个完整机器人动作。

对同一次前向分别求两项损失的梯度，结果如下。表中是底座参数梯度合并后的欧氏范数，连续项尚未乘 λ：

跨模块读取方式	L_C 对底座的梯度范数	L_F 对底座的梯度范数
两层均不停止	0.59027515	0.00767614
两层只停止键	0.59027515	0.00767866
两层只停止值	0.59027515	0.00018557
两层键和值均停止	0.59027515	0
仅最后一层键和值停止	0.59027515	0.00612959

各设置的 L_C, L_F 和专家前向预测在这次初始化下完全相同。专家收到的连续梯度范数均约为0.04160467，文字梯度为0。部分停止后剩余梯度不一定按大小单调减小，因为不同路径的梯度原本就可能相加或抵消；不能把“少了一条路线”解释成梯度范数必然更小。

接着用学习率0.03的SGD实际更新三次，所有比较从同一初始化开始。下表记录隔离方案更新前的损失与这次更新造成的参数变化：

即将执行的更新	L_C	L_F	底座参数变化范数	专家参数变化范数
第1次	2.00139133	0.25437021	0.01770825	0.00062407
第2次	1.99099172	0.25434231	0.01752912	0.00062293
第3次	1.98080027	0.25431445	0.01735584	0.00062180

完成三次后，$L_C = 1.97080818$、$L_F = 0.25428663$。底座参数确实变化了，原因是它接收离散监督，而不是连续梯度绕过了隔离。代码还保持专家参数固定，只让底座做一次 L_C 更新；下一次专家预测相对原预测的变化范数约为 4.36×10^{-5}，直接显示前向条件仍在改变。

未隔离方案三次后的连续损失约为0.25428402，与隔离方案非常接近。这不是隔离效果的性能比较：两个合成样本没有语言语义，也没有旧任务，无法测量知识保持。本实验检验的是前向依赖、梯度去向和实际参数更新。

实现可按以下顺序对应到 practice/joint_paths.py：

```
J1  从样本取得可用条件；构造右移的离散输入，以及有标签样本的带噪动作。
J2  每层先设置P/D/A可见关系，底座计算自己的下一token表示。
J3  专家读取每层底座K、V的分离副本；专家自己的计算继续记录梯度。
J4  按有效token和动作元素计算两项损失，反传后更新各自参数。
```

其中决定知识隔离的读取代码如下。kb,vb 是底座投影，qa,ka,va 是专家投影；allowed 使用图1.14-C中专家对应的行。

```
# J2：先用允许矩阵决定前向能读取哪些位置。
# J3：底座的原kb、vb仍供离散分支使用，这里只分离专家读取的副本。
keys = torch.cat([kb.detach(), ka], dim=0)
values = torch.cat([vb.detach(), va], dim=0)
scores = qa @ keys.T / (qa.shape[-1] ** 0.5)
weights = torch.softmax(scores.masked_fill(~allowed, -torch.inf), dim=-1)
expert_attention = weights @ values

# J4：底座接收CE；专家接收flow。两者仍在同一步训练中更新。
loss = token_loss + flow_weight * flow_loss
optimizer.zero_grad(set_to_none=True)
loss.backward()
optimizer.step()
```

最后做一个反例：保持真实连续动作、噪声和全部参数不变，仅将教师强制输入中的动作标记交换。正确掩码下，专家预测严格不变；若开放专家读取离散动作位置，即使键和值都分离，预测变化范数仍约为0.0011238。这是对输入依赖的干预检查，不是构造一条新的有效示范。它说明代码里出现 detach()，并不能证明训练输入没有答案泄漏。

两层还会产生间接泄漏。用标量特征看这个过程：初始条件 $p^{(0)}=2$，第一层误把真实动作标签的某个特征 d 混入条件，得到 $p^{(1)}=1.5p^{(0)}+0.4d$。专家两层内部表示分别为

$$z^{(1)}=0.8\times 0.5+0.1\,\mathrm{sg}(p^{(0)})=0.6,\qquad z^{(2)}=1.2z^{(1)}+0.5\,\mathrm{sg}(p^{(1)}).$$

专家没有直接读 d，跨模块读取也都停止了梯度；但 d 从3变为5时，$p^{(1)}$ 从4.2变为5，专家输出从2.82变为3.22。标签通过“第一层改条件、第二层读条件”进入预测。这里的 d 是标签编码后的特征值，$z^{(1)},z^{(2)}$ 是网络内部表示，没有执行两步机器人动作。

检查也有局限：一次替换后输出改变，能揭示这次输入依赖；一次不变却不能证明不存在依赖。例如无位置依赖的注意力，对键值对的共同排列保持不变。应结合逐层数据流检查，使用多种标签内容替换和多组样本，而不是把一次交换测试当成充分证明。

1.14.8 怎样使用这套训练分工，怎样判断它值得采用

先看任务是否需要改变底座。如果数据量小、现有特征已经足够，固定底座训练专家可能是合理基线；若需要机器人特有的表示，可以尝试局部解冻或LoRA，再决定这些可训练参数接收哪些损失。知识隔离与低秩适配可以组合，二者分别决定更新路径和更新形式。

联合损失还必须说明平均方式。若机器人样本有3个有效token、负对数损失和6，视觉语言样本有9个有效token、损失和9，样本内均值分别是2和1。全部token平均得到 $15/12=1.25=\frac{1}{4}\times 2+\frac{3}{4}\times 1$；先对每个样本平均再平均，则为 $\frac{1}{2}(2+1)=1.5$。后一种方式提高了短样本的相对权重。这不仅改变loss尺度，也改变监督之间的权重，通常无法用一个全局学习率补偿。

在式（1.14-10）的分工中，λ 缩放专家收到的连续梯度，而机器人与视觉语言样本的抽样比例还会改变底座的CE监督组成，二者一般不能互相替代。连续项按有效动作元素数 N_F 平均，增加每批机器人样本也不必成比例地放大其均值梯度；它可能主要影响方差、连续更新出现的频率和底座后来提供的条件。课程学习逐步改变难度，多任务配方改变监督来源，二者也都需要检查稀少任务是否长期得不到有效更新。

损失系数与更新幅度的关系还取决于优化器。若专家只承担这一项损失，使用无动量、无衰减、无裁剪的普通SGD，一步为 $\Delta\psi=-\eta_\psi\lambda\nabla_\psi L_F$；在数据、底座条件和随机数固定时，放大 λ 等同于放大专家学习率。**Adam**则分别累积梯度的一阶矩和平方梯度的二阶矩，再用 $\hat{m}/(\sqrt{\hat{v}}+\epsilon)$ 形成更新方向。如果矩从零开始，从训练开始将全部专家梯度乘同一个正数，一阶矩同比放大、二阶矩按平方放大；在 $\epsilon=0$、分母非零的理想情况下，两种缩放相消。因此使用Adam时，λ 加倍并不意味着参数步长加倍。已有历史矩、训练中途改变系数、裁剪或额外正则项，还会改变这个关系。Adam原论文

评估知识保持，需要同时测试“做对动作”和“按指定条件做动作”。例如同一桌面上同时放红杯与蓝杯，固定图像、状态、历史与初始噪声，仅切换目标颜色指令，检查选择的杯子是否正确切换；真实闭环从配对初始状态分别重置。将目标选择正确率、选对后的放置成功率和整体成功率分开报告，再交换物体位置、加入未见物体与新背景，排查视觉捷径。旧知识还需要训练前后的独立视觉语言评估；只有初始策略已经具备的机器人技能，才有相应的旧技能保持问题。

一个有解释力的消融应分别改变连续梯度、离散动作监督、视觉语言监督，并保留固定底座参照。比较其中一个因素时，固定其他因素、初始化、专家和动作表示；保持机器人样本曝光与主要更新步数

可比，记录额外视觉语言token和计算，再补等训练时间或计算量的性能曲线。每步成本、达到同一性能的耗时与部署延迟分别回答不同问题。

使用LoRA时还要固定插入位置、秩、缩放约定、初始化和其他可训练模块；若一边增加秩或放开视觉编码器，容量变化也进入了比较，难以再把差异归因于梯度隔离。完整的对照矩阵与指标分工见题1.14-6的答案。

如果隔离后任务表现不足，可能是底座的辅助监督没有保留所需控制信息，也可能是专家容量、数据覆盖或动作接口有问题。可依次检查：离散分支是否确实学习了任务；连续目标、噪声方向与动作归一化是否匹配；掩码是否阻断了合法状态信息；专家是否能在足够信息下拟合训练动作。只有这些环节清楚，才值得讨论是否需要放开部分连续梯度或改变网络连接。

1.14.9 历史与异步执行：训练之外还要保留哪些信息

前面把条件写成 c，不代表一张图像总能提供全部决策依据。相同的红杯图像可能对应“夹爪正在接近”和“夹爪正在离开”；刚发生的接触、物体是否已被夹起，也可能需要前后帧才能判断。**历史**提供这些变化信息，**记忆**则用某种表示保存其中与后续行动有关的部分。

固定窗口直接保留最近若干观测，历史token把过去编码进序列，循环状态则递推压缩过去：$m_t = F_\theta(m_{t-1}, o_t)$。这里的 m_t 是网络内部表示，不是传感器直接测得的机器人物理状态。循环神经网络（Recurrent Neural Network，RNN）与状态空间模型（State-Space Model，SSM）都可用于这类序列处理，具体结构影响保存能力和计算成本；例如选择性SSM让状态更新随输入内容改变。Mamba 原论文 压缩也可能丢失一次短暂接触；比较时要检查保留了什么、回合何时重置、是否混入其他回合，而不能仅比较能接收多长的序列。第1.13节讨论具体模型的历史方案，第5章再区分隐藏状态与真实状态。

执行动作块时还会遇到另一个时间问题：推理期间机器人继续运动。同步执行等待新结果，可能形成停顿；**异步执行**让控制器继续执行已有动作，同时生成下一块。新块返回时，最初观测对应的时刻已经过去，直接从新块第一个动作开始，可能重放过时的命令。

例如在物理时刻 $t = 0$ 读取图像，推理耗时0.08秒、控制周期0.02秒。新块返回时，旧块已经执行四步。此时必须明确新块预测的是哪个时间段：是否利用已排队的动作作为条件，是否只接入尚未执行的部分，重叠区域怎样平滑。这个问题需要时间对齐与生成条件配合解决，仅把推理放到另一个线程并不充分。

较长动作块减少调用次数，却可能延长对变化不敏感的区间；只执行短前缀能更频繁接收新观测，也增加生成开销。接触任务还可能依靠更快的本体或力反馈，而不必每次都重算全部视觉语言底座。端到端策略与底层控制器可以共同工作。实际闭环要同时记录观测年龄、生成延迟、控制周期、块长度与执行前缀；第4章解释动作怎样进入控制器，第11章讨论部署和异步接续。

1.14.10 练习题

题1.14-1 LoRA的启动、输入梯度与合并。 使用第1.14.3节的初始化、样本和损失，手算前两次SGD更新后的 A_L, B_L，以及第三次前向的输出。解释第二次前向时，W_0 虽然固定，为什么仍有 $\partial L/\partial x = [0.36, 1.44]^\top$。比较固定 W_0 与用 no_grad() 包住整个层。若两个因子都初始化成0，或只令 $B_L = 0$ 但当前输入满足 $A_L x = 0$，本轮能否更新因子？重复同一固定样本与改用其他样本有何不同？最后说明合并为什么保持线性输出，却不缩小原矩阵尺寸。

题1.14-2 从样本到两种标签。 给定本节 2×2 的动作与噪声矩阵、$\tau = 0.4$，构造连续输入和速度目标；写出离散目标 $[d_1, d_2, \mathrm{EOS}]$ 对应的教师强制输入、各位置能读到的离散内容，以及两条分支的训练标签。一个只有图文标注的样本应该参与哪些损失？说明为什么连续候选之间可以双向读取，而真实动作token不能因此被专家读取；推理中又从哪里取得连续候选。

题1.14-3 手推键和值的两条梯度。 取第1.14.6节的标量注意力，推导 $\partial L/\partial k_j$、$\partial L/\partial v_j$ 和 $\partial L/\partial q$，给出数值。分别停止键、值、两者，哪些导数保留，哪些被停止？说明为什么只处理多层专家的最后一层仍可能回传到底座；如果专家还有一条未经隔离的共享投影，结论如何变化。

题1.14-4 相同前向不代表相同训练。 一位同学看到“隔离前后初始预测、loss完全相同”，认为 detach() 没有作用；又看到“隔离后底座仍变化、专家输出仍变化”，认为隔离实现失败。分别判断并解释。设计两个不依赖最终成功率的检查，区分“梯度隔离失败”与“真实标签泄漏”。还需解释为什么允许 P 读 D 后，只屏蔽专家直接读取 D 仍不充分。

题1.14-5 联合目标的权重究竟在哪里。 同一批次有一个3-token机器人样本，负对数损失之和6；一个9-token视觉语言样本，负对数损失之和9。比较“全部token一起平均”和“每个样本先平均再平均”的目标、两个样本的权重及单个位置的权重。在知识隔离下，调大 λ 与增加机器人样本采样频率，能否互相替代？对没有动量和权重衰减的普通SGD，固定条件下放大 λ 能否用专家学习率吸收？使用Adam时还应检查什么，为什么不能直接沿用同一结论？

题1.14-6 怎样验证知识保持的机制。 设计一组以“将指定颜色杯子放到托盘”为任务的对照，分别检验连续梯度、离散动作监督、视觉语言混合数据的作用。说明训练条件怎样控制，语言是否真的影响动作怎样测量，旧知识与新任务如何同时评估。若未隔离方案也有良好语言跟随，或隔离方案机器人成功率更低，怎样分析而不是直接宣布机制正确或错误？若使用LoRA，必须额外控制哪些变量？

1.14.11 参考答案

题1.14-1

令 $g = y - y^*$，则 $\nabla_{B_L} L = g(A_L x)^\top$、$\nabla_{A_L} L = B_L^\top g x^\top$。初始 $A_L x = -1$，$y = [1, 2]^\top$、$g = [0, 2]^\top$，所以第一轮梯度是 $\nabla_{A_L} L = [0, 0]$、$\nabla_{B_L} L = [0, -2]^\top$。学习率0.1给出

$$A_L^{(1)} = [1, -1], \qquad B_L^{(1)} = [0, 0.2]^\top.$$

第二次前向 $y = [1, 1.8]^\top$，$g = [0, 1.8]^\top$，所以 $\nabla_{A_L} L = [0.36, 0.72]$、$\nabla_{B_L} L = [0, -1.8]^\top$，更新得到

$$A_L^{(2)} = [0.964, -1.072], \qquad B_L^{(2)} = [0, 0.38]^\top.$$

第三次前向 $A_L^{(2)} x = -1.18$，输出为 $[1, 2]^\top + [0, -0.4484]^\top = [1, 1.5516]^\top$，损失1.20373128。这里两项梯度都根据同一轮的旧参数计算，不能先更新一个因子，再用它计算同一步另一个因子的梯度。

第二次前向时，有效权重为

$$W_0 + B_L^{(1)} A_L^{(1)} = \begin{bmatrix} 1 & 0 \\ 0.2 & 0.8 \end{bmatrix}, \qquad \nabla_x L = \begin{bmatrix} 1 & 0.2 \\ 0 & 0.8 \end{bmatrix} \begin{bmatrix} 0 \\ 1.8 \end{bmatrix} = \begin{bmatrix} 0.36 \\ 1.44 \end{bmatrix}.$$

其中原映射贡献 $W_0^\top g = [0, 1.8]^\top$，修正分支贡献 $[0.36, -0.36]^\top$。固定 W_0 不删除它对输入的导数；如果输入来自可训练视觉模块，这个梯度仍可继续向前面的模块传播。整个层使用 no_grad()，则本次输出到两个因子和输入的路径都不再记录。若损失只有这条路径，直接反传会因没有求导图而报错；若另有可训练分支，总损失仍能反传，但这条路径不贡献梯度。只给原分支使用 no_grad() 也不同：输入仅剩修正分支的 $[0.36, -0.36]^\top$。

两个因子同时为零时，普通梯度更新无法启动这条分支，因为两项因子梯度对任何输入都为零。非零 A_L 配合当前 $A_L x = 0$，则只是该样本这一轮不能启动 B_L；若始终重复同一固定输入，也会停在这里，但换用不在零空间中的样本，或由上游学习改变输入，可能提供新信号。即使单样本梯度非零，批次相加也可能抵消。这些情况都不等于原网络对输入没有梯度。

第二次更新后的合并权重为

$$W_{\text{merged}} = \begin{bmatrix} 1 & 0 \\ 0.36632 & 0.59264 \end{bmatrix},$$

乘 $[1, 2]^\top$ 仍得到 $[1, 1.5516]^\top$。这是线性映射的代数合并；低秩约束只作用于增量，合并矩阵仍为原尺寸，本例甚至仍是满秩。这个极小例子的两个因子共有4个参数，与原 2×2 矩阵一样多；它用于看清梯度，真正的参数节省要求 $r(d_{\text{in}} + d_{\text{out}}) < d_{\text{in}} d_{\text{out}}$。

题1.14-2

连续输入和速度标签分别为

$$x_{0.4} = 0.6\omega + 0.4A = \begin{bmatrix} 0.02 & 0.04 \\ 0.46 & -0.2 \end{bmatrix}, \qquad u = A - \omega = \begin{bmatrix} 0.8 & -0.4 \\ -0.6 & 1.0 \end{bmatrix}.$$

专家读取条件、当前候选和内部进度，预测的是 u；本题不是直接用干净动作 A 作为速度回归标签。离散输入与目标则为

$$\text{输入} = [\text{BOS}, d_1, d_2], \qquad \text{目标} = [d_1, d_2, \text{EOS}].$$

位置	可读取的离散输入	预测目标
D_0	BOS	d_1
D_1	BOS、d_1	d_2
D_2	BOS、d_1、d_2	EOS

三个位置还都可以读部署可用的条件 P，不能读未来离散输入或连续候选。D_1 读取的自身输入是上一个标签 d_1，不是其目标 d_2。本题ID为1、2的动作标记直接给定，没有从这两个矩阵实际运行FAST得到；真实数据流程中，动作编码器才负责从示范生成相应序列。

图文样本参与文字交叉熵，不参与连续动作损失。本节两条记录合计6个有效token，只有机器人记录的4个动作标量进入连续分母。缺失动作不能当作零动作标签。

连续块的所有带噪候选在一次网络调用前已经存在，因此可以相互读取。这些候选是当前生成变量，不是真实未来观测；教师强制中的真实离散动作则是另一条部署时没有的答案通道，不能交给专家。还必须禁止标签先混入 P，再被下一层读走。

推理从新噪声 $X_0 = \omega$ 开始，例如欧拉更新

$$X_{n+1} = X_n + (\tau_{n+1} - \tau_n) v_\psi(X_n, \tau_n, h_\phi(c)).$$

下一次候选来自这次积分结果，不能使用未知真实动作重新构造。生成结束后再按动作接口执行所选前缀；内部积分步、块内位置和物理控制步各有自己的含义。

题1.14-3

令 $s_j = qk_j$、$\alpha_j = e^{s_j}/\sum_i e^{s_i}$、$z = \sum_i \alpha_i v_i$。softmax导数给出

$$\frac{\partial \alpha_i}{\partial s_j} = \alpha_i(\mathbf{1}_{i=j} - \alpha_j), \qquad \frac{\partial z}{\partial s_j} = \sum_i v_i \alpha_i(\mathbf{1}_{i=j} - \alpha_j) = \alpha_j(v_j - z).$$

再用 $\partial L/\partial z = z$，得到

$$\frac{\partial L}{\partial v_j} = z\alpha_j, \quad \frac{\partial L}{\partial k_j} = z\alpha_j(v_j - z)q, \quad \frac{\partial L}{\partial q} = z\sum_j \alpha_j(v_j - z)k_j.$$

代入 $\alpha = [1/4, 3/4]$、$z = 1.5$：

$$\nabla_v L = [0.375, 1.125], \qquad \nabla_k L = [-0.5625, 0.5625], \qquad \frac{\partial L}{\partial q} = \frac{9}{16}\log 3 \approx 0.6179694124.$$

键梯度两项之和为0，与“所有分数同时加同一常数不改变softmax”一致。只停止键，则此接口不再给原键传梯度，值和查询仍保留上式梯度；只停止值，则键和查询保留；两者都停止，则只保留查询梯度。停止键不会冻结权重，因为查询仍可改变分数。PyTorch可能用 None 表示某个原张量没有来自此图的路径，不一定存一个全零梯度张量。

多层中若第一层仍读取未分离的底座，路径 $\phi \to K_b^{(1)}, V_b^{(1)} \to X_a^{(1)} \to X_a^{(2)} \to L_F$ 仍然存在。一个最小代数例是 $z_1 = \phi$、$z_2 = z_1 + \mathrm{sg}(\phi)$、$L = \frac{1}{2}z_2^2$；$\phi = 1$ 时，最后一项虽然已停止，第一条路径仍使 $\partial L/\partial \phi = 2$。逐层处理读取接口，才会移除这条早期路线。

共享投影也要按位置分析。如果底座与专家共享 R，专家直接用 $q = R \times 1$，即使底座键和值都分离，仍有 $\partial L_F/\partial R = (9/16)\log 3 \neq 0$。只分离投影的输入，阻止不了投影参数本身更新。可以给专家独立参数，或分离需要作为给定条件的跨模块输出；若共享参数所有通往连续损失的输出都已经分离，它则未必构成旁路。

题1.14-4

两项判断都不成立。detach() 保留数值、指定停止反向的规则，所以初始预测和loss相同是预期结果；下一次参数更新却可能不同。完整隔离时，L_F 不回底座，L_C 仍可更新底座，下一次专家读取的特征因而可能改变。专家参数固定也不保证输出固定。反过来，一次更新后输出没有变化，也可能只是变化方向暂时未被专家使用，不能据此判定底座已冻结。

第一项检查是分别求 L_C, L_F 对实际参数的梯度，清除累积影响并固定噪声、生成时间和随机状态。完整隔离应阻断连续项到所有底座参数的路径，同时在非退化输入上保留专家梯度；还要检查每层键和值、共享参数与其他旁路。只看总loss或参数是否变化不足，因为其他损失、动量和权重衰减都可能改变参数。一个数值零也可能来自恰好零误差或路径抵消，应结合图结构、多个输入与未隔离对照检查。

第二项检查是固定条件、A, ω, τ, x_τ、标签目标、参数和随机数，只替换教师强制输入中的动作标记，比较专家输出和逐层条件。输出变化说明被禁止的输入仍被使用；这属于前向依赖，不能以“它的自动梯度是零”来排除。

间接泄漏的路径是 $D^0 \to P^1 \to X_a^2$：第一层将真实标签混入条件，第二层专家才读到。第1.14.7节的标量例中，标签特征从3变为5，输出从2.82变成3.22，跨模块停止梯度始终存在。因此每层都要维护 P 不读真实动作标签的边界。

一次干预不变也不是无泄漏的充分证据。取两个真实标记的键值对 $(0, 0)$、$(\log 3, 2)$，查询为1，专家读取它们时输出为

$$\frac{1 \times 0 + 3 \times 2}{1 + 3} = 1.5.$$

交换两个键值对后，加权和仍为1.5；但将第一个标记替换成键值对 $(0, 4)$，输出就变为 $(4 + 6)/4 = 2.5$。原先的交换只是重排求和项，未暴露已有的答案通道。应以逐层结构检查配合多种内容替换和多组样本；有限干预的阴性结果只是局部检查通过。

题1.14-5

全部token平均为 $(6 + 9)/(3 + 9) = 1.25$；样本内平均再平均为 $\frac{1}{2}(6/3 + 9/9) = 1.5$。将两样本的均值2和1作为比较对象：

方式	机器人样本均值权重	视觉语言样本均值权重	机器人单个位置系数	视觉语言单个位置系数
全部token平均	$1/4$	$3/4$	$1/12$	$1/12$
样本内平均再平均	$1/2$	$1/2$	$1/6$	$1/18$

权重是目标的线性系数，不是该样本当前loss的数值贡献。例如第一行两样本实际贡献是0.5和0.75。改变平均方式会改变相对监督权重，通常也改变梯度方向。

完整隔离下，λ 只缩放专家的连续梯度；提高机器人抽样频率还会改变底座的离散监督组成、条件分布和有效更新次数，所以一般不能替代。连续项按 N_F 平均时，若每批本来就有机器人样本，多放几个样本不必放大期望梯度，可能主要减少方差；若原来有些批次完全没有动作，则还会影响连续更新出现的频率。

无动量、无衰减、无裁剪的普通SGD，专家只承担 λL_F 时，有 $\Delta\psi = -\eta_\psi \lambda g$。固定初始化、数据、噪声、底座条件与调度，$(c\lambda, \eta_\psi)$ 和 $(\lambda, c\eta_\psi)$ 等价；若要保持原更新不变，放大 λ 后应把专家学习率除以 c。同时改变底座的学习率会改变条件序列，就不再满足这个比较的前提。

Adam保存两种移动统计。用 G_k 表示第 k 次实际送给优化器的专家梯度，包括损失系数；β_1, β_2 控制保留历史的比例，初始 $m_0 = v_0 = 0$：

$$m_k = \beta_1 m_{k-1} + (1-\beta_1) G_k, \qquad v_k = \beta_2 v_{k-1} + (1-\beta_2) G_k^2,$$

$$\hat{m}_k = \frac{m_k}{1-\beta_1^k}, \qquad \hat{v}_k = \frac{v_k}{1-\beta_2^k}.$$

平方和除法按元素进行，帽号校正从零初始化造成的早期偏差。对给定的梯度序列，若从第一步起全部乘固定正数 c，上述递推使 $m' = cm$、$v' = c^2 v$，偏差校正后也如此。使用常见分母形式时，

$$\Delta\psi' = -\eta \frac{c\hat{m}}{c\sqrt{\hat{v}} + \epsilon} = -\eta \frac{\hat{m}}{\sqrt{\hat{v}} + \epsilon/c}.$$

例如第一步 $G = [0.4, -0.2]$、$\beta_1 = 0.9$、$\beta_2 = 0.99$，有 $m = [0.04, -0.02]$、$v = [0.0016, 0.0004]$；将梯度乘4，两个统计分别乘4和16。偏差校正后，$\hat{m}/\sqrt{\hat{v}}$ 在两种设置下都是 $[1, -1]$，没有放大4倍。

因此 $\epsilon = 0$、分母非零时，同一学习率下更新不变；ϵ 很小时才近似不变，不能套用SGD的 $\eta\lambda$ 等价。比较实际训练还需要相同的数据与条件，给定梯度序列的尺度关系不能自动证明两次完整训练轨迹相同。中途改变损失系数不会自动重缩放已有矩；其他损失、共享参数、裁剪和额外正则也可能使实际梯度不再满足统一缩放。耦合L2把未同时缩放的正则梯度加进总梯度，通常会破坏上述关系。AdamW的权重衰减与损失梯度分开执行：在同一学习率和衰减系数下，单纯缩放损失不会改变衰减步，所以理想条件下的尺度不变性仍可能成立；若为补偿损失缩放而改变学习率，衰减量也会随之改变。实际比较应检查这些处理，不能据理想尺度关系推断任意 λ 都不影响效果。

题1.14-6

先构造需要语言作出选择的任务：红杯、蓝杯同时出现，颜色与位置交叉平衡，两种指令都出现，示范的动作同时提供离散和连续监督。将连续梯度是否回底座、离散动作监督和视觉语言监督组成三个二元因素：

方案	连续梯度回底座	离散动作监督	视觉语言监督
I00	否	否	否
I10	否	是	否
I01	否	否	是
I11	否	是	是
J00	是	否	否
J10	是	是	否
J01	是	否	是
J11	是	是	是

所有行都训练连续专家。相同两项因素下只切换第三项，才能归因相应作用；例如I11与J11比较隔离，I10与I11比较视觉语言监督。只比较I11与J00，会同时改变三个因素。I00没有其他底座目标，在无动量或衰减漂移时相当于固定表示，可以再显式冻结底座作实现参照。若资源有限，应优先保留只改变一个关键因素的成对对照，并说明未能区分的交互。

对照应共享底座与专家初始化、机器人示范与切分、编码、动作接口、噪声、生成方法和执行协议。离散动作与视觉语言项分别按有效位置归一化，避免关闭其中一项后另一项因分母改变而被放大。保持机器人曝光次数和主要更新步数可比；额外视觉语言数据有额外计算，另报告等时间或计算量的学习曲线。使用相同验证选择与调参预算，多种子和配对场景评估，同时记录推理延迟与失败类型。

语言测试固定初始图像、状态、历史和随机种子，只切换“红杯/蓝杯”，检查选取对象是否正确变化；真实闭环分别重置场景。分别报告目标选择、选对后的运动放置、整体成功；用位置交换、同义改写、新背景和未见组合排查捷径，动作仅仅“有变化”不是语言跟随。旧知识使用训练前后独立视觉问答、物体属性和关系理解等测试；旧机器人技能必须是初始策略已有的技能，新接专家不能声称保留从未具备的机器人能力。

J11也能良好跟随语言，说明当前数据和预算下隔离不是取得该结果的必要条件，应继续检查视觉语言数据的作用、测试天花板和不确定性。I11控制成功率较低，则先排查合法输入是否被遮蔽、动作归一化与流方向、执行时序及训练预算，再区分目标选择和选对后的控制瓶颈。可检查底座是否提供精细状态、专家能否在足够条件下拟合，以及离散辅助监督是否过粗。必要时研究局部开放连续梯度或改善辅助目标，仍须重新对照验证。

使用LoRA时，固定插入层与模块、秩、缩放、初始化、其他可训练参数和数值精度，使用可比优化器、预算与推理合并方式；检查专家是否意外共享适配参数。若同时更换秩、放开编码器或改量化，容量与数值变化也进入比较，应作为额外因素报告。

第2章：模仿学习与策略预训练

已有动作模型之后，最直接的学习证据是示范。这里从行为克隆开始，解释监督标签、闭环偏移和纠错，再讨论跨任务与跨身体适配。

2.1 示范监督、经验分布与行为克隆风险

2.1.1 从看似容易的监督问题开始

我们有示范条件 c 和专家动作 a，训练网络在这些条件下给出相似动作，这就是**行为克隆**的基本任务。一个专家动作标签回答“专家在这个条件下做了什么”，并不保证它是唯一最优动作，也不告诉模型未示范状态该怎么办。读取训练目标时，应先明确数据从哪种采集策略、哪些场景和哪些筛选规则产生。

2.1.2 策略模型怎样处在机器人系统中

输入“把蓝杯放到托盘上”，机器人需要根据当前视野、自身姿态和历史，给出下一动作。学习策略可以写成 $\pi_\theta(a \mid c)$：θ 是模型参数，c 是部署时可获得的条件，a 是动作。它表示条件动作分布；有些实现直接输出确定值，有些通过分布采样动作。

条件 c 可以包括图像、指令、关节状态、历史和机器人标识。它不是神秘的“上下文向量”：实际代码中会先分别读这些数据，再编码和融合。网络输入真实观测，动作输出经过反归一化、坐标转换和控制器执行，产生新的观测。

模型职责见第1.1节，动作块时间关系见第1.7节。

模仿学习用示范中的动作提供监督；强化学习用行动后的反馈改进策略；**世界模型**预测行动后会发生什么。模仿策略可以没有**价值网络**，**VLA** 可以没有显式世界模型，普通视觉动作策略也不必有语言输入。

同一个输出“末端向右移动两厘米”，对底层是位置增量而非力矩。模型不直接控制电机的所有物理细节。了解这种接口，才能解释“预测对了但执行错了”可能来自哪里。

学习终点不是背模型名称，而是能说出它的输入、输出、标签、损失和运行流程。后文每类策略都沿这条链解释。

2.1.3 示范数据如何变成训练问题

一次示范是按时间记录的轨迹，包含观测、状态、动作和结束信息。训练样本可以是当前条件与下一动作，也可以是历史窗口与未来**动作块**。时间对齐、单位和动作定义决定标签到底代表什么。

例如用时刻 t 的图像预测从 t 开始的 H 步示范动作。未来动作是监督标签，当前策略输入则只包含决策时可获得的观测。这个区分使训练任务与部署条件对应。

行为克隆（Behavior Cloning，**BC**）把这类样本当作条件预测问题。数据由示范者产生，训练通常不需要每一步都运行机器人。网络的预测与示范动作比较，产生梯度更新 θ，再用新网络处理新输入。

示范由人类、遥操作设备和底层控制共同产生，可能包含迟疑、不同路径和恢复。记录的发送命令也可能与实际位移不同。选择动作标签时，应结合设备记录和控制响应，明确模型是在模仿命令还是运动结果。

动作块尾部不足 H 时，要标记哪些步有效。不同回合之间发生重置，不能把前一回合最后观测与后一回合动作拼接。训练集与测试集按任务、对象、场景或采集回合划分，而不是随机把相邻帧分开。

网络看到的数据分布定义了它被训练解决的范围。只采集杯子永远在桌子中央的示范，不能仅凭低训练损失宣称具备桌面任意位置的抓取能力。

2.1.4 行为克隆为什么最小化动作预测损失

设示范数据分布为 D_E，其中 E 表示示范来源，一个常见目标是

$$L(\theta) = \mathbb{E}_{(c,a)\sim D_E}[-\log \pi_\theta(a \mid c)].$$

期望对示范条件与动作取平均，更新的对象是策略参数。**负对数似然**鼓励网络给示范动作更高概率或密度。它没有直接比较整段任务回报，也不自动判断示范是否成功。

如果使用固定方差高斯输出，其均值是网络预测，最大似然等价于在相应尺度下减少平方误差。方差固定意味着建模时把某种误差尺度视为给定；不同动作分量的单位和尺度需要归一化或加权。

回归目标的选择也会影响模仿结果。假设同一条件下，动作标签 1 与 3 各出现一半，它们代表障碍左侧和右侧的两条可行路线，而动作 2 对应不可行的中间路线。单值模型的平均平方误差是 $[(\mu-1)^2+(\mu-3)^2]/2$，最优预测为 $\mu=2$。这是第1.2节所讨论的模式平均：模型正确优化了示范误差，却输出了不应执行的动作。

增加训练轮数只会让模型更接近这个目标的最优解，不会让目标自行区分路线。可以加入能决定绕行方向的信息，或学习多峰动作分布并在执行时选择一条路线。改进应针对歧义来源：若左右路线对应不同任务，就补任务条件；若在同一任务下都合理，就保留不同选择。

预测误差与任务成功之间还隔着动力学、误差累积和执行约束。BC 是合理基线，条件清楚、数据覆盖足够时可以有效；它的限制也应通过闭环测试判断。离线示范学习研究。

2.1.5 训练误差与应用风险为什么不同

设专家的数据分布为 $d_E(c)$，行为克隆最小化 $\mathbb{E}_{c\sim d_E}\ell(\pi_\theta(c), a_E)$。执行时，**条件分布**却是 $d_{\pi_\theta}(c)$，因为此前动作改变了下一帧看到什么。两式中的损失可以完全相同，取平均的对象却不同。训练损失低，只能先支持专家分布上的拟合；任务成功还要求执行分布上的错误不会把系统推向无法恢复的状态。

以杯沿对准为例，专家示范一直离杯口中心很近。模型在这一区域输出准确，略有偏差后却可能从未见过“杯口已经偏到图像角落”的观测。恢复数据不是把原示范复制更多次，而是加入偏离状态和对应纠正。若专家标签也有延迟或错误，应先清理接口；否则更忠实的模仿可能复现这些错误。

2.1.6 行为克隆的训练循环

```
从训练轨迹读取批次，并保留 episode 边界
仅用训练集统计量变换观测与动作标签
inputs = 当前图像、当前状态、语言、允许的历史
prediction = policy(inputs)
mask = 有效未来时间 AND 有效动作维度
loss = 在有效监督元素上计算路线对应的损失
清空本轮旧梯度，反向传播，按配置裁剪梯度，更新参数
在固定留出任务上执行闭环，保存成功与失败轨迹
```

2.1.7 把一次模仿更新与一次机器人执行接起来

设训练样本的图像里杯柄位于右侧，专家动作是在机器人基座系向右移动 $0.02\,\mathrm{m}$。本例以 $0.04\,\mathrm{m}$ 为尺度，计算 $u_{\mathrm{norm}} = u/(0.04\,\mathrm{m})$，标签因此为0.5。模型预测0.3，对应 $0.012\,\mathrm{m}$；平方误差为 $(0.3-0.5)^2 = 0.04$，对预测的导数为 -0.4，反向传播据此更新网络参数。训练阶段机器人尚未移动；部署时再将新观测送进网络，按相同尺度还原输出，并交给控制器，才发生真实运动。

因此，调试时可固定一条样本，依次检查原始动作、归一化标签、网络输出、还原后的命令和实际位移。若网络输出与标签一致，但还原后的方向错了，扩大模型无法修正这个接口错误；若这五项一致，却在抓偏后的观测上不会恢复，问题才转到状态覆盖。这个顺序来自监督学习与闭环执行的分工。
LeRobot 动作表示教程、CS285 行为克隆课程

2.1.8 练习题

题2.1-1 为什么同一示范集上动作误差更小的行为克隆模型，闭环成功率可能更低？给出一个具体机制和一个对应实验。

2.1.9 参考答案

题2.1-1答案 动作误差通常在示范经过的状态上计算，闭环成功率却取决于模型自己行动后访问到的状态。模型可能在正常抓取轨迹上误差很小，但稍微抓偏后就进入示范里没出现的状态，接着没有纠偏能力。另一个模型在示范状态上平均误差略大，却学过偏离后的恢复，反而完成更多任务。

具体例子是抓杯柄：一个模型把少量纠错样本视为噪声，主要拟合最常见的接近路径；其离线平均误差下降，但真实执行出现横向偏差后继续闭夹爪，抓到空气。

对应实验是固定测试初始状态，分别进行正常执行和受控扰动执行：接近杯柄时施加相同的小位姿偏移，再比较纠偏动作、恢复率和最终成功率。同时在示范状态与策略访问状态上分别测动作误差。若正常误差小却扰动恢复差，说明问题在状态覆盖和闭环纠错。实验中还要控制相机、控制器和动作限幅，避免把执行接口差异误认为行为克隆机制的差异。

2.2 闭环分布偏移、纠错与DAgger

2.2.1 模仿的是动作，动作却会改变下一道题

示范者把夹爪对准杯柄，训练图像大多对应顺利接近的状态。策略部署后，一次小偏差可能把杯子碰歪。下一张图像里，杯柄换了位置，夹爪还遮住了部分杯身。模型接下来要回答的，已经不是示范里反复练习的那道题。

这里的**策略**是根据可用信息选择动作的规则；神经网络实现这个规则时，就是策略模型，记为π_θ。VLA通常在机器人系统中承担这个职责。它预测的动作会被执行，执行又改变下一次输入。因此，机器人模仿学习不仅要问“给定这一张图，动作预测得准不准”，还要问“模型连续行动以后，会把自己带到哪些情形”。

用$d_E^t(s)$表示专家执行到第t步时的状态分布，用$d_{\pi_\theta}^t(s)$表示学习策略执行到同一步的状态分布。这里的“分布”可以直观理解为：反复重置任务并运行，哪些状态经常出现，哪些状态几乎不出现。对固定的策略，取各时刻的平均，得到$d_{\pi_\theta}(s) = T^{-1}\sum_{t=1}^{T} d_{\pi_\theta}^t(s)$。它既受初始条件和环境影响，也受此前动作影响；改了策略，通常就会改掉它。

普通**行为克隆**（Behavior Cloning，BC）主要在专家数据上拟合动作。其目标与真正想要的闭环目标可写成：

$$\underbrace{\mathbb{E}_{s\sim d_E}[\ell(\pi_\theta(s),\pi_E(s))]}_{\text{原示范里的动作误差}},\qquad \underbrace{\mathbb{E}_{s\sim d_{\pi_\theta}}[\ell(\pi_\theta(s),\pi_E(s))]}_{\text{自己运行时的动作误差}}. \tag{2.2-1}$$

$\pi_E(s)$是专家面对**当前状态**给出的动作标签，ℓ衡量预测与标签的差异。两个目标的标签规则可以完全相同，差别在于在哪些状态上练习。第一个目标小，不足以说明第二个也小：模型可能只熟悉“杯子没碰歪”的图，却不懂偏离以后如何恢复。真实策略若读取图像、语言和历史，应把上式的s替换为它实际可得的条件c；本节先用完全可观测状态说明分布问题。

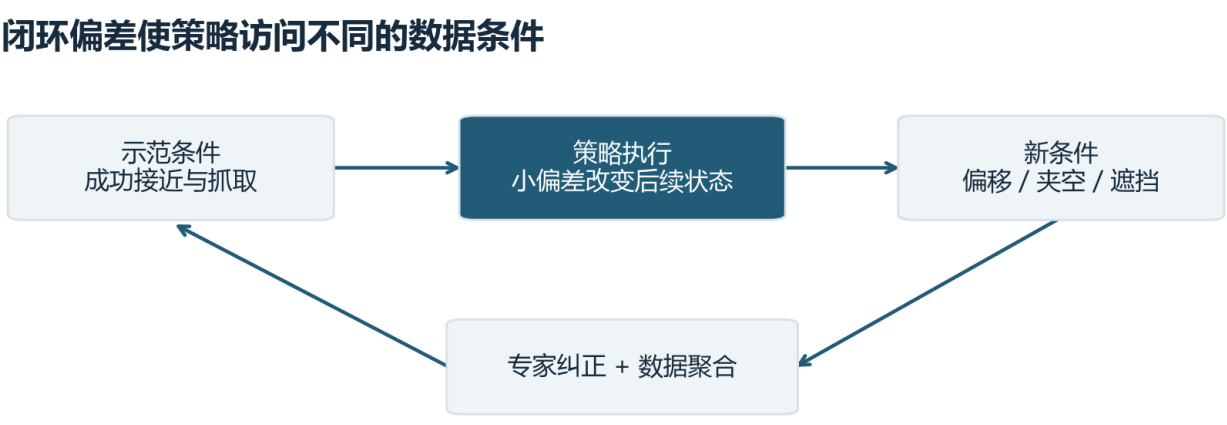


图2.2-A：原有闭环示意。恢复示范增加的是“已经偏离后应该怎么办”的监督。它仍需要覆盖有意义的状态，并提供正确、对齐的标签。

DAgger（Dataset Aggregation，数据聚合）沿着这一思路改变数据来源：让当前策略运行，在它实际访问的状态上询问专家，把这些纠正与旧示范合并，再训练下一版策略。它没有要求把策略换成某一种网络，也不要求奖励标签；它让原来的监督学习接触到自己将要遇到的输入。CS285第二讲以分布偏移和纠错解释这一动机；imitation教程则把专家、采样器和监督训练器放在同一个运行流程中。

2.2.2 执行动作、专家标签和下一观测分别是什么

设第i轮开始时模型参数为θ_{i-1}。采样期间先固定这版模型。机器人在s_t上执行a_t^{exec}，转移到s_{t+1}；专家对s_t给出的正确动作记为a_t^E。一条有用的记录至少能区分：

$$(s_t,\ a_t^{\text{exec}},\ a_t^E,\ s_{t+1},\ \text{执行者},\ \text{策略版本}). \tag{2.2-2}$$

用于监督拟合的是(s_t, a_t^E)。记录学习者实际做过什么，则有助于还原为什么到了下一状态。这两个动作可能不同，而它们的不同正是纠错信号的来源。如果误把学习者的错误动作当成标签，模型只会被要求重复刚才的行为。

运行日志不一定已经有专家标签。本节一轮记录960次访问，只选160次查询；其余记录的专家标签仍然缺失，不能填成0，因为0也可能是真正的停车动作。可以给日志加上标签有效标记，也可以像配套程序一样，分别保存访问记录和已查询的监督表，再以访问编号关联。环境分析可以使用全部日志，动作损失只从有效的输入—专家标签对计算。

专家不一定亲自执行。离线重标注可以让专家看当时的记录，回答“如果处在这里，你会怎么动”；实时接管则让专家实际改变后续状态。无论哪一种，都要把标签对应到专家作出判断时的输入。把接管后才出现的画面配上接管前的动作，会制造错误的监督关系。

为了让早期轨迹更可控，有些实现会在每一步随机选择由谁控制：

$$\mu_i(a \mid s) = \beta_i \pi_E(a \mid s) + (1-\beta_i)\pi_{\theta_{i-1}}(a \mid s). \tag{2.2-3}$$

这里β_i是选择专家执行的概率。例如$\beta_i = 0.25$表示每次按规定的随机规则，有25%的概率执行专家动作，其余执行学生动作。它不是把两个动作向量按0.25和0.75相加。对离散控制而言，两种选择尤其不同；对连续控制而言，平均动作也可能走到两种原动作都没走的地方。这里采用的是每步选择执行者的定义，整回合选择一次会产生另一种轨迹分布。

采样分布此时是d_{μ_i}，不是纯学生的$d_{\pi_{\theta_{i-1}}}$。在采到的状态上，标签仍应由专家给出。imitation的采集器接口很能说明这种区分：它接收并保存专家动作，同时可以选择执行学习者动作。若把“传给采集器的动作”一概理解成“环境一定执行的动作”，就会读错数据。

人工只在危险或困难时接管，通常能节省标注，但它也改变了哪些状态会被标注。接管数据适合帮助恢复，不自动等价于“每一个学生访问状态都能获得专家标签”的基础版本。本节数值实验取$\beta_i = 0$：初始示范训练完成后，由学生独立采样，再对选中的访问状态查询专家。

2.2.3 数据为什么累计，参数又在什么时候更新

把本轮新标签记为$\Delta\mathcal{D}_i$。更新过程是：

$$\mathcal{D}_i = \mathcal{D}_{i-1} \,\|\, \Delta\mathcal{D}_i, \qquad L_i(\theta) = \frac{1}{|\mathcal{D}_i|} \sum_{(s,a^E)\in\mathcal{D}_i} \ell(\pi_\theta(s), a^E). \tag{2.2-4}$$

||表示拼接样本记录，重复状态也保留其样本权重。训练器通过若干次梯度更新，得到θ_i；随后才由新模型执行下一轮。访问状态的变化不发生在一个静态训练批次里，而是通过“参数改变→动作改变→环境状态改变”这条路径传递。

为什么不只训练最新一轮？因为最新轨迹是某一版模型造成的，可能集中在一个狭窄的错误区域。只留下这些标签，旧的正常动作可能被遗忘。累计数据让正常行为和多轮恢复一起约束模型。当然，均匀采样全部记录也会产生取舍：原示范在总数据中的比例逐渐降低，某个停滞状态反复出现时会占较大权重。可以研究重新加权、按回合采样或挑选纠错片段，但应说明改掉了哪一种权重，不能同时把它们当成完全相同的算法。

聚合后的训练集是历史多版采样策略的混合。即使本轮全由学生执行，也不能说“更新后的模型与训练集具有完全相同的状态分布”：新模型尚未运行，它可能带来新的状态。DAgger正是通过下一轮交互继续修正这个差异。原始算法也保留了重复采样、聚合和重新训练的循环。

```
D1  用初始专家示范训练 π₀，保留数据 D₀。
对 i = 1, 2, …:
    D2  固定 πᵢ₋₁，让它或预先规定的混合策略采样轨迹。
    D3  从访问状态中选择查询对象，取得对应时刻的专家动作。
    D4  把新监督样本拼接到已有数据，保留来源、回合和时间。
    D5  在累计数据上做监督训练，得到 πᵢ。
    D6  在独立重置条件下评估，再用 πᵢ采下一轮。
```

D2中模型负责执行；D3中专家负责标签；D5中优化器改变参数。D6的测试记录可以用于选择版本，不能为了让结果更好而回填成未经声明的新训练数据。下一节将用同一程序连续跟踪这六步。

2.2.4 第一轮纠正了方向，却引出了新的错误

考虑一个一维环境，状态是x，目标是0.8，转移为$x_{t+1} = x_t + a_t$。这些数没有机器人长度或时间单位，用来观察学习闭环。专家给出：

$$a^E(x) = \mathrm{clip}\big(0.35(0.8 - x), -0.12, 0.12\big). \tag{2.2-5}$$

离目标远时，专家以限幅动作接近；靠近后，动作逐渐减小；越过目标则反向。策略是一个输入一维、隐藏层32维、输出一维的网络，动作范围为$[-0.15, 0.15]$。初始160个专家标注状态都在$[-0.9, -0.5]$，对应的专家动作全部为$+0.12$。模型因此主要练习过“向右走”，没有从这些标签中得到目标附近该如何停车的信息。

固定种子3，先训练初始策略π_0。在$x = 0.4$，它输出约-0.000871，专家却要求$+0.12$。在自己的闭环里，这版策略容易停在约0.396附近。原示范区间上的拟合，并没有决定它在右侧应该输出什么。

第一轮从24个规定区间内的起点运行π_0，得到960个访问记录，再随机选160个查询专家。所选状态最大约0.609，尚未覆盖越过目标后的情形。其中一条真实记录为：

字段	数值与含义
当前状态x_t	0.460042066
执行的学生动作	−0.010425779
随后状态x_{t+1}	0.449616287
专家对x_t的标签	+0.118985277

这一条要教的是：面对0.460这个状态，应向目标方向移动。专家标签没有被拿去替换已经发生的转移；原记录仍反映学生先往左走了。它也不是原始示范里“第10步动作”的复制，因为标注针对的是这里实际看到的状态。

把160条新标签和160条旧标签合并，做350次小批监督更新，得到π_1。它在$x = 0.4$的输出变为$+0.118007$，方向修正了。然而，在$x = 0.8$它仍输出$+0.115924$，在$x = 1.2$仍输出$+0.113608$。所收标签没有覆盖目标两侧的停车与返回，这次拟合得到的动作曲线接近“一直向右”。

因此，同样从0.4出发，新状态依次约为：

$$0.400 \to 0.518 \to 0.635 \to 0.752 \to 0.868 \to 0.984 \to 1.099 \to \cdots.$$

40步后，它距离目标约3.805，比初始模型更远。这个变化不能被概括为“DAgger无效”：它表明本轮标签修正了什么，也暴露了尚未学到什么。算法下一轮需要面对的，恰好是这版模型新造成的越界状态。

2.2.5 第二轮学会返回，第三轮补上停稳

第二轮运行π_1。访问状态延伸到约4.729，新的160条标签就包括“越过目标后向左返回”。例如$x = 1.157554174$时，学生还执行$+0.113856210$，专家标签却是-0.12。这与上一轮的纠正方向相反，却并不矛盾：标签对应的是两个不同状态。

现在总数据为480条，初始示范、第一轮和第二轮各160条。训练π_2后，在$x = 1.2$的输出已变为-0.067343，学到了返回方向。不过，它在$x = 0.8$仍输出$+0.023103$。从0.4出发，轨迹约为：

$$0.400 \to 0.507 \to 0.599 \to 0.672 \to 0.729 \to 0.771 \to 0.802 \to 0.825 \to \cdots \to 0.885.$$

它不再跑远，却停在约0.885，而非0.8。**闭环固定点**是执行后状态保持不变的位置；在这个环境中，它满足$\pi_\theta(x) = 0$。π_2在0.885附近的动作接近零，形成了错误的固定点。训练误差不大，仍可能把“停在哪里”学偏。

第三轮运行π_2，大量状态集中在这个停滞区域。所选一条$x = 0.885331783$的记录，学生动作只有$+0.000000554$，专家标签则为-0.029866124。这次要学的不是大幅返回，而是“这里还没到目标，不能就此停下”。合并为640条数据再训练，π_3在$x = 0.8$的输出降至约0.003222。从同一0.4出发，它经过0.781、0.792等状态后，满足进入目标容差并连续停留三步的成功判据。

版本	累计专家标签	$\pi(0.4)$	$\pi(0.8)$	$\pi(1.2)$	100个固定测试起点成功比例
π_0	160	−0.000871	−0.056634	−0.091763	0
π_1	320	0.118007	0.115924	0.113608	0
π_2	480	0.107209	0.023103	−0.067343	0
π_3	640	0.112644	0.003222	−0.099572	1

成功比例使用最多40步、距离小于0.025且连续三步满足条件的定义。它不是“精确到达0.8”的证明，也不是未知初始条件上的保证。表里最值得观察的是同一位置的动作怎样改变，以及这种改变怎样重新塑造下一轮状态。

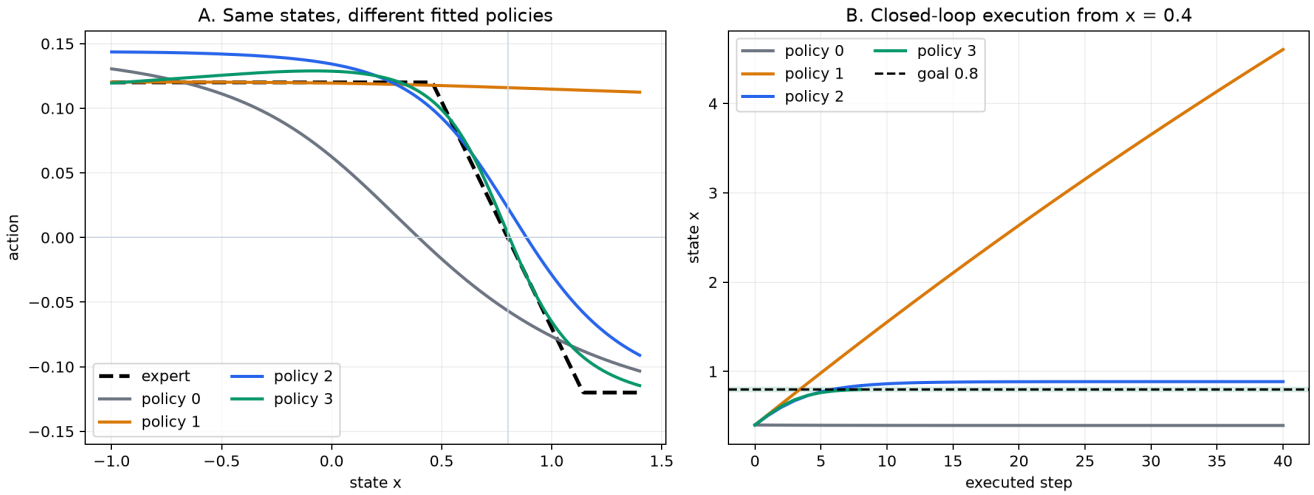


图2.2-B：A图横轴是状态，纵轴是动作；黑色虚线为专家，policy 0—3为四版策略。B图固定从0.4出发，横轴是实际执行步，纵轴是状态；goal是目标，浅绿色带是容差。图中的“策略曲线更接近”与“实际运行到达”是两个可以分别核对的对象。

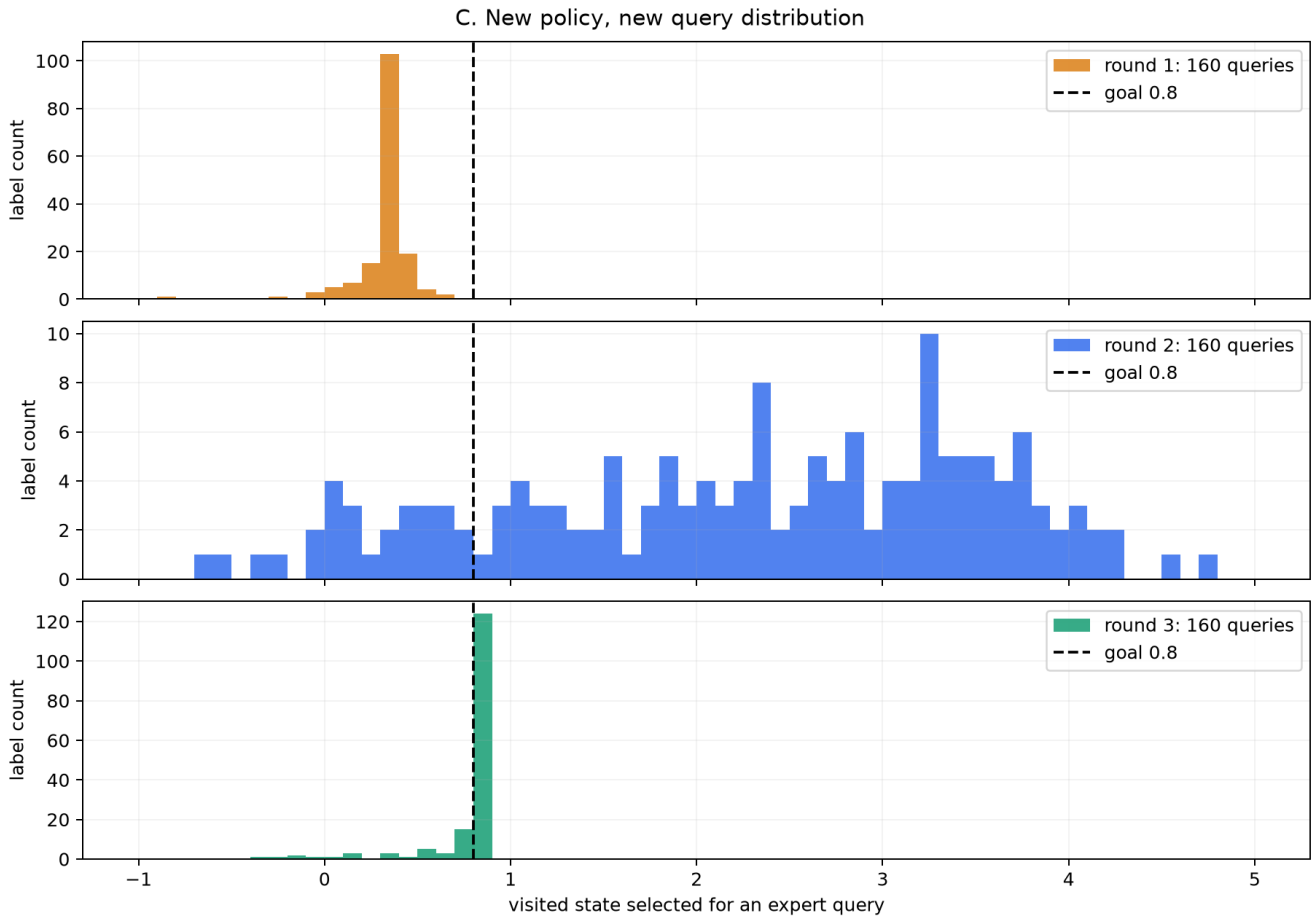


图2.2-C：三幅图各有160次查询，横轴是查询状态，纵轴是该区间内的标签数；各图纵轴刻度不同。第一轮集中在初始停滞区，第二轮出现越过目标后的广泛状态，第三轮集中在新的停滞区。相同标签数量不等于相同信息：每轮学习到的行为决定下一轮主要需要什么纠正。

2.2.6 一个纠正标签怎样改变网络参数

把策略展开为：

$$h = \tanh(xW_1 + b_1), \qquad r = \tanh(hW_2 + b_2), \qquad \hat{a} = 0.15r. \tag{2.2-6}$$

x按1×1输入，W_1, b_1均为1×32，h为1×32，W_2为32×1，b_2是标量。对单条专家标签y，取$L = (\hat{a} - y)^2$。从动作误差到输出偏置的导数为：

$$\frac{\partial L}{\partial b_2} = 2(\hat{a} - y)\, 0.15(1 - r^2). \tag{2.2-7}$$

第一轮那条0.460的记录中，$\hat{a} = -0.010425779$，$y = 0.118985277$，$r = -0.069505193$。代入后，导数约为-0.038635762。若只在模型副本上做一次学习率0.1的**随机梯度下降**（Stochastic Gradient Descent，SGD），则b_2增加约0.003864，预测动作由-0.010426变成-0.009849，单样本平方误差由0.016747降为0.016598。动作还没纠正完，但已经沿着减少该误差的方向变化。

这个小切片说明标签怎样产生更新方向。完整训练还把同一误差信号乘上隐藏特征，传到W_2，再经过第一层的导数传到W_1, b_1。实际轮次使用累计数据的小批次和**Adam**（Adaptive Moment Estimation，

自适应矩估计）优化器，每轮350次更新；前面的单样本SGD没有混入轮次训练。共享参数会同时影响其他状态，因而某条误差下降不意味着所有状态都改善，也不保证本轮闭环成功比例上升。

若把错误的已执行动作-0.010425779作为监督标签，当前模型在这条记录上的预测恰好等于它，动作误差和这条记录的梯度都是零。即使把这样的失败轨迹反复加入数据集，也没有告诉网络应该向哪边纠正。DAgger的关键是“学习者提供访问状态，专家提供应做动作”，并非笼统的“多训练失败数据”。

2.2.7 历史信息不足时，更多标签仍可能相互冲突

杯子当前都在夹爪右侧，但一个正在向左滑，另一个正在向右滑。只输入当前位置x_t，两种情况看起来相同；加入前一位置后，可以用$(x_t - x_{t-1})/\Delta t$区分运动方向。这里不要求视觉策略先精确计算速度，而是说明历史提供了什么信息。示范者若根据运动方向决定追上杯子还是等待，缺少历史时，这些标签在模型看来就会冲突。

把最近几帧一起输入，是直接保留历史；循环模型则逐步压缩历史。设隐藏状态为h_t，写成$h_t = F_\theta(h_{t-1}, o_t, a_{t-1}^{\text{exec}})$，动作头再读取$h_t$。若具体模型读取上一动作，这里必须使用日志中实际执行的动作，不能用未执行的专家纠正改写历史；不读取动作的模型则省去这一输入。这个h_t是一组训练得到的内部表示，不是人工列出的速度或接触类别。动作损失通过递推关系，促使模型保留有助于预测动作的信息；没有帮助训练目标的信息也可能被丢掉。这里的序列隐藏状态与上一小节一次前向中的隐藏特征承担不同职责。

每个控制周期应保留上一隐藏状态，回合重置时清空。若训练每个窗口都从零开始，部署却连续运行数百步，隐藏状态的使用条件可能不同。**预热历史**（burn-in）是在需要计分的训练窗口之前，先用一段真实历史计算起始隐藏状态，再只对后段动作计算损失；是否把梯度继续传到预热段，需要随具体训练设计说明。Transformer历史输入同样要保持时间顺序、采样间隔和可读掩码。第1.13节的MEM进一步讨论了短期视频与长期摘要怎样分工。

DAgger主要补访问覆盖，不会凭空让不可见信息变得可见。专家如果读取完整仿真状态，而学生只见一张被遮挡的图，必须检查这些标签是否能由学生输入决定。可比较“增加历史或视角”与“只增加同类标签”的效果，判断瓶颈是覆盖还是信息不足。CS285第二讲也把改善模型输入与改善数据分布作为不同的改进方向。

2.2.8 为什么会有平方级误差，又为什么不能保证线性改善

先看一个离散动作的教学模型。专家始终处于正常路径；学生只要还没偏离，每步有概率ϵ选错。一旦选错，就进入无法恢复的区域，此后每步都付出1的代价；在正常区域正确行动时代价为零。这里的“错误”定义为未选择专家动作，不是前面连续实验里的动作均方误差。

第t步已经出错的概率为$1-(1-\epsilon)^t$，因此T步的期望总代价是：

$$\sum_{t=1}^{T}\left[1-(1-\epsilon)^t\right] \;\le\; \sum_{t=1}^{T}\min(1, t\epsilon). \tag{2.2-8}$$

右侧使用“前t步任意一次出错”的并集上界。对$\epsilon = 0.1, T = 4$，各步代价概率是0.1、0.19、0.271、0.3439，合计0.9049；只把四步分别看作原专家输入上的独立预测，会误以为合计只是$4\epsilon = 0.4$。当$t\epsilon$

还小时，上界求和出现$O(T^2\epsilon)$。它描述错误改变后续输入后可能付出的最坏代价；$T^2\epsilon$大于T时，还要记得总代价本来就不超过T。

如果专家能很快从偏离状态恢复，一次错误的后续损失就可能小得多。设在学生实际访问的状态上，选择不同于专家动作的概率平均为ϵ_{own}；再设“一次改用学生动作、随后交回专家”的额外剩余总代价至多为u。通过比较逐步把专家动作换成学生动作的混合执行，可以把总差距写成各步额外后果的和，于是得到形如：

$$J(\pi) - J(\pi_E) \;\le\; uT\epsilon_{\text{own}}. \tag{2.2-9}$$

这里J是期望总代价，越小越好。u衡量一次错误及其恢复造成的总后果，不只是当前一步的代价。如果一次错误会让剩余整段任务都失败，u自身仍可随T增长，线性形式就没有带来所期待的改进。更重要的是，式中的误差来自学生自己的访问分布；只在专家测试集上测得的小误差，不能直接填进去。连续动作的平方误差与任务代价之间还需要额外关系，不能把前面表中的训练MSE当成这里的错误概率。

DAgger的理论还需要监督学习在多轮数据上具有合适的学习性质。可以把第i轮定义成一个损失函数$\ell_i(\pi) = \mathbb{E}_{s\sim d_{\mu_i}}\ell(\pi(s), \pi_E(s))$：输入分布由当轮采样策略产生，但我们可以在这批状态上比较任意候选策略的预测。用Π表示给定模型结构与可用输入允许表示的策略集合。**无遗憾**（no regret）要求，随着轮数N增大，每轮平均累计损失不显著差于“事后从Π中选同一个固定策略处理全部轮次”的最优值：

$$\frac{1}{N}\sum_{i=1}^{N}\ell_i(\pi_{\theta_{i-1}}) - \min_{\pi\in\Pi}\frac{1}{N}\sum_{i=1}^{N}\ell_i(\pi) \;\le\; \gamma_N, \qquad \gamma_N \longrightarrow 0. \tag{2.2-10}$$

γ_N是平均遗憾的上界；“趋近零”要求的是长期不比这个固定比较者更差，实际差值也可能为负。比较者在所有轮次保持不变；如果专家用了学生没有的信息，或模型表达能力不足，Π中的最好策略仍可能有非零损失。配合专家混合逐渐减小、足够数据和相应损失条件，分析才能把多轮训练联系到某个候选策略在自身分布上的表现。对一般深度网络做有限次Adam更新，不会自动获得上述学习保证。原始结果可以通过独立验证选择候选；也可以在回合开始时均匀抽取一版候选，并在全部T步保持它。每步重新切换候选会产生另一种状态分布，没有自动继承这个结论。分析同样不保证最后一版深度网络最好，实际仍要保存版本并做闭环评价。Ross等的理论分析给出了这些条件与有限样本项。

2.2.9 复现入口与怎样判断收益来自哪里

trace_dagger_rounds.py中的D1—D6注释对应本节伪代码。它沿用原有网络与训练程序，保留原采样和优化顺序，新增访问记录、专家标签、累计样本来源及四个模型检查点。G1—G3是隔离的单标签梯度探针，不参与模型轮次。运行：

```
python practice/trace_dagger_rounds.py
```

结果写入dagger_round_trace.json，检查点位于results/checkpoints。可以先找第一轮r1_label51，沿visit_id回到原执行记录，再与第二轮r2_label77、第三轮r3_label0比较。程序核对前三轮与已有种子3记录一致，而不是为了得到单调曲线重新挑采样。

原有完整对照则运行六轮、五个种子。窄覆盖BC每轮从原示范区间补160条标签；DAgger从当前访问状态补160条；宽覆盖BC从预先规定的较宽区间补160条。各法使用相同初始化训练后的拷贝、相同总标签数和新增更新步数，测试起点和成功判据一致。它们各自抽取的小批次不相同；原实现共享随机数生成器并按固定顺序调用，属于明确的有限教学比较。

这里可以直接抽取一个数值状态x，再调用解析专家。宽覆盖BC因此不必先执行到$[-1, 1.3]$中的每个采样位置；初始与窄覆盖标签也采用这样的查询。真机通常不能免费设置任意物体状态、接触关系和机械臂姿态。本例匹配的是有效标签条数与训练步数，没有匹配状态获取、失败和复位成本。这个对照用于判断覆盖本身能否解释收益，不能据此宣称两种真机采集方案成本相同。

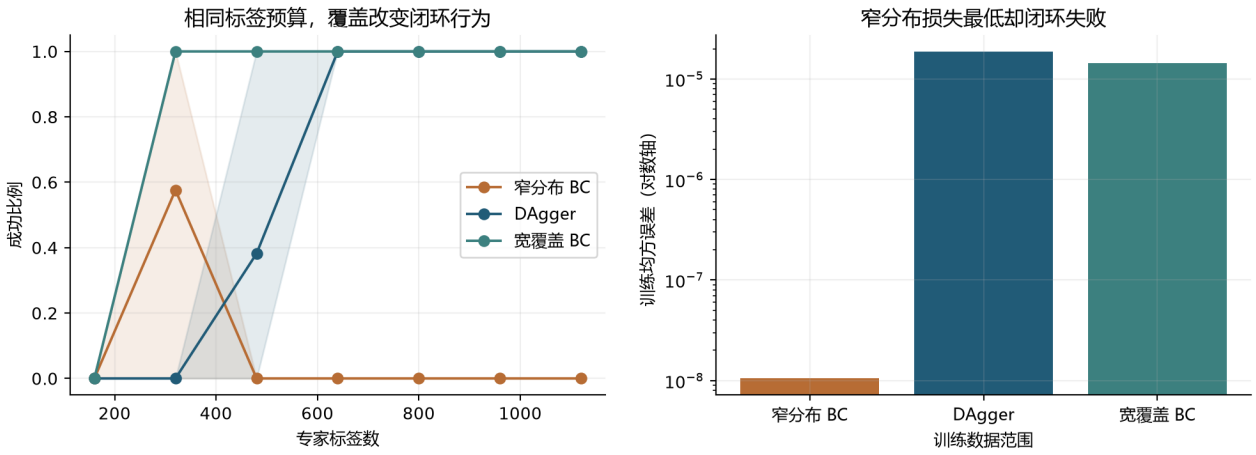


图2.2-D：保留的五种子结果。左图横轴为累计专家标签，纵轴为成功比例，阴影为种子范围；右图为各法最终训练集上的均方误差。窄覆盖BC可以把自己的训练误差压得更低，却在闭环中失败；宽覆盖BC同样成功，说明本任务不能把“增加覆盖”的全部收益独归于DAgger。

每种方法最终都有1120条标签，五个种子中DAgger与宽覆盖BC的成功比例都为1，窄覆盖BC都为0。这一结果支持本例中恢复覆盖有用；它没有证明DAgger比充分覆盖的BC更省标签，也没有测量机器人专家的时间成本。窄覆盖BC、DAgger和宽覆盖BC的训练误差来自不同数据分布，右图的柱高不能直接当成同一难度上的准确率排名。

在具身系统里，查询专家、接管、复位和执行失败都有成本。比较纠错方案时，除了标签条数和训练计算，还应记录专家耗时、访问/接管次数、有效恢复片段、独立闭环成功与失败类型。若专家只肯标一部分状态，要测哪些失败状态仍没有监督；若学生增加历史后才改善，则还要把信息变化与数据覆盖变化分开。一个适用方案应回答：错在什么状态，我们能否得到该状态下可信的标签，学生输入是否足以解释标签，以及更新后如何验证真正改善了执行。

2.2.10 练习题

题2.2-1：执行记录与监督标签。 第一轮记录为$x_t = 0.460042066$，学生动作-0.010425779，专家动作$+0.118985277$，随后记录到$x_{t+1} = 0.449616287$。分别说明环境转移、动作损失和下一轮数据分析应使用哪些字段。若采用式(2.2-3)且$\beta = 0.25$，它与执行加权平均动作有什么差别？若专家事后才标注，哪些时间关系必须保持？解释把学生动作作标签时为什么可能出现“训练很容易，行为不改”。

题2.2-2：连续轮次与公平对照。 结合本节三轮记录，说明为什么第一轮监督误差下降仍可能造成更差的闭环行为，第二轮为什么又产生新的停滞区。两轮聚合后，初始、第一轮、第二轮的样本各占多少权重？如果只留下最新160条，优化目标怎样改变？设计一个比较“访问分布选择”和“单纯增加标签”的对照，明确至少应匹配的训练、数据、评价及专家成本条件，并说明本书宽覆盖BC结果能支持和不能支持什么。

题2.2-3：把纠正传到参数。 使用式(2.2-6)，单样本损失为$(\hat{a}-y)^2$。推导W_1, b_1, W_2, b_2的梯度及维度；用第一轮给出的$\hat{a}, y, r$复算输出偏置梯度和一次隔离SGD后的方向。解释为什么“对输出求导”不等于把该数直接加到动作上，也解释为什么该样本损失下降仍不能保证全体状态或闭环成功率改善。可将推导与原程序的四组grads对应。

题2.2-4：误差界究竟比较什么。 对式(2.2-8)的吸收失败模型，推导第t步的代价概率，复算$\epsilon = 0.1, T = 4$的总代价和并集上界。说明为什么不能直接写成$T\epsilon$。再解释式(2.2-9)的u与ϵ_{own}分别约束什么，何时所谓“线性界”仍可能随T平方增长。DAgger的无遗憾比较对象是否每轮都可更换？为什么原始分析不保证最后一个神经网络版本最好？

题2.2-5：专家更强是否一定有用。 两条历史当前图像相同，但杯子分别向左、向右运动；专家看完整状态，学生只看当前图像，专家给出相反动作。解释增加DAgger标签为什么可能无法解除冲突。提出一个区分信息不足、恢复覆盖不足和接管时间错位的诊断实验；说明怎样采样、用什么输入和标签、看哪些预测及闭环结果。若改用循环历史，训练窗口起点、预热、回合重置各需要处理什么？

2.2.11 参考答案

题2.2-1

环境执行的是学生动作，所以实际转移为$0.460042066 - 0.010425779 = 0.449616287$。监督样本却应是$(0.460042066,\ 0.118985277)$：输入为动作前状态，标签为专家对这一状态的正确决定。不能用“专家本来执行以后会到哪里”替换已经发生的下一状态；下一状态也不是这里动作回归的标签。

下一轮分析需要保留回合号、步号、策略版本、执行者、观测与动作时间，再把查询结果关联到同一个访问编号。它们分别用于还原轨迹、选择需要纠正的状态，以及检查标签是否配对。960条访问日志中只有160条查询，并不意味着其余800条的专家动作是0。未标注记录可以继续用于轨迹分析，动作损失只使用有标签的记录。若以$m_j \in \{0,1\}$表示有效性，可写成$L = \sum_j m_j \ell_j / \sum_j m_j$；实现时先筛选有效样本，批次没有标签就跳过这项更新。

$\beta = 0.25$时，按每步混合规则，执行专家动作与学生动作后，下一状态分布分别为：

$$x_{t+1} = \begin{cases} 0.579027343, & \text{概率 } 0.25, \\ 0.449616287, & \text{概率 } 0.75. \end{cases}$$

插值执行则每次都取$\bar{a} = 0.25(0.118985277) + 0.75(-0.010425779) = 0.021926985$，到达0.481969051。在线性转移下，它恰好等于随机混合的下一状态均值，却没有相同分布：前者始终到同一点，后者的方差约为0.003140104。碰撞、阈值和后续非线性决策都取决于实际到了哪里，不能只凭均值相同认定两套轨迹等价。无论本步谁执行，已查询状态的监督仍是专家动作，β不控制标签中掺多少学生动作。

事后标注可以晚于实际执行，但标签必须回答原来t时刻的问题，并关联到当时可用的观测和历史。接管后把物体扶正的画面，不能配上接管前的纠正动作。回放暂停在指定时刻标注、记录人工反应延迟、在可重建状态的仿真里重新查询，都能帮助校准配对。文件写入时间相同，不代表动作与观测语义上对齐。

对同一版确定性模型，若直接令标签为刚执行的$\pi_{\theta_{i-1}}(x_t)$，则当前预测与标签相同，平方损失和梯度都为零。失败虽然被记录了，目标却没有告诉网络应怎样纠正。后续训练可能让它不再零误差，但这种标签仍约束模型靠近旧行为。可信的纠正、偏好或其他明确反馈，才会为失败记录提供相应改进方向。

题2.2-2

π_0主要看过左侧的正动作标签，在右侧怎样延伸并未被数据决定。第一轮的访问与查询集中在0.396附近等位置，专家给出向右动作，π_1因而修正了原来向左或停住的行为；但查询尚未覆盖停车和返回，它在目标及其右侧仍输出正动作，便把自己带到更远的越界状态。

第二轮恰好运行这版会越界的模型，查询池里就出现1.157等状态和负向纠正。π_2学会返回，却把动作零点拟合在约0.885，形成错误固定点。第三轮的访问又集中到这里，专家小幅向左的标签补上了“还不能停”的信息。正文的三轮记录依次补了前进、返回和停稳。它们不是同一批静态标签上的重复训练。

现有标签上的损失下降，不保证数据以外的动作函数正确；共享参数还会同时影响其他位置。闭环评价进一步包含这些输出怎样改变后续输入。因此，局部纠正有效与第一轮任务表现变差可以同时发生，有限非凸训练也没有逐轮单调保证。

两轮聚合后共有480条记录，目标为：

$$L_2 = \frac{1}{3}L_{\mathcal{D}_0} + \frac{1}{3}L_{\Delta\mathcal{D}_1} + \frac{1}{3}L_{\Delta\mathcal{D}_2}.$$

三个来源各160条，均匀抽记录时各占$1/3$。重复访问会叠加权重；这不是对所有不同状态均匀采样。若只留最新160条，目标改为$L_{\Delta\mathcal{D}_2}$，来源权重变成$(0,0,1)$，旧接近和旧恢复行为不再直接约束拟合，可能遗忘。累计也不是没有取舍：旧样本和高频停滞区可能占较大权重，需要结合失效分析选择采样方式。

比较数据选择，应保留同初始数据和初始化的窄覆盖BC、访问状态DAgger及预先规定的宽覆盖BC，而不只与“完全不增加标签”的BC相比。需要匹配或报告：

对象	应控制的条件
训练	网络输入、容量、初始化、损失、优化器、更新步数、批量、调参与版本选择预算
监督	每轮新增有效标签、专家质量、聚合和查询规则
状态获取	采样轨迹、交互步数、可否任意设状态、失败与复位次数
专家成本	查询、观看、等待、接管和恢复工时，以及无效标注
评价	独立且共同的起点、扰动、时域、成功判据和失败分类

本例宽覆盖BC可以直接在$[-1, 1.3]$抽取x，向解析专家查询；它不必实际执行到该状态。初始和窄覆盖标注也有这种便利。标签条数相同没有消除真实机器人获取状态、布置物体和复位的成本。旧程序重复计算确定性标签，也不能按函数调用次数直接换算为人工工时。

五种子结果支持本例的窄覆盖不足、两种扩大覆盖方式都可成功；它不能证明DAgger普遍优于充分覆盖BC、更加节省专家时间，或在VLA上具有同样效果。各法训练MSE来自不同分布，不能直接按柱高排名；应在共同独立状态集和闭环任务上比较。本实验的有限种子、固定随机调用顺序也不能替代研究中的充分对照。

题2.2-3

先把前向计算中的两个线性输入写出：

$$z_1 = xW_1 + b_1, \quad h = \tanh z_1, \qquad z_2 = hW_2 + b_2, \quad r = \tanh z_2, \quad \hat{a} = cr, \quad c = 0.15.$$

$x, z_2, r, \hat{a}, y$都是1×1，W_1, b_1, z_1, h为1×32，W_2为32×1，b_2为1×1。动作误差先产生$\partial L/\partial \hat{a} = 2(\hat{a} - y)$；经过缩放和输出$\tanh$，得到：

$$\delta_2 = \frac{\partial L}{\partial z_2} = 2(\hat{a} - y)c(1 - r^2), \qquad \frac{\partial L}{\partial W_2} = h^{\mathsf{T}}\delta_2, \quad \frac{\partial L}{\partial b_2} = \delta_2.$$

δ_2为1×1，后两项分别为32×1和1×1。再经过第二层权重和第一层$\tanh$：

$$\delta_1 = (\delta_2 W_2^{\mathsf{T}}) \odot (1 - h \odot h), \qquad \frac{\partial L}{\partial W_1} = x^{\mathsf{T}}\delta_1, \quad \frac{\partial L}{\partial b_1} = \delta_1.$$

$\odot$是逐元素乘法；这三项均为1×32。同一动作误差沿不同参数影响动作的路径传播，W_2决定哪些隐藏特征承担修正，$1 - h^2$则体现隐藏单元接近饱和时的梯度减弱。

对B条样本的平均平方损失，令X为$B \times 1$，H为$B \times 32$，$\hat{A}, R, Y$为$B \times 1$，则：

$$\Delta_2 = \frac{2}{B}(\hat{A} - Y) \odot c(1 - R \odot R), \qquad \Delta_1 = (\Delta_2 W_2^{\mathsf{T}}) \odot (1 - H \odot H).$$

原代码中的gz对应Δ_2，gh对应Δ_1；grads依次为 $[X^{\mathsf{T}}\Delta_1,\ \sum_j \Delta_{1,j}, H^{\mathsf{T}}\Delta_2,\ \sum_j \Delta_{2,j}]$，对应$[W_1, b_1, W_2, b_2]$。除以$B$已在$\Delta_2$中完成，偏置求和不能再除一次。若动作变成多维，还要按实际损失是“每样本求和”还是“所有元素取平均”调整归约。

用题面舍入值，$r = \hat{a}/0.15 \approx -0.0695051933$，所以：

$$\frac{\partial L}{\partial b_2} = 2(-0.010425779 - 0.118985277)(0.15)(1 - r^2) \approx -0.0386357624.$$

学习率0.1的SGD令b_2增加0.0038635762。只改这个偏置，z_2增加同样的量，故可以直接复算新预测：

$$\hat{a}' = 0.15 \tanh(\operatorname{atanh}(r) + 0.0038635762) \approx -0.0098488902.$$

单条损失约从0.0167472214降至0.0165982426。梯度方向已让动作靠近专家，但这一次还没有让动作变成正向。

对输出的导数回答“改变预测会怎样影响损失”，训练真正改的是参数。相同偏置变化在tanh线性区和饱和区会产生不同动作变化；只有把动作本身定义为优化变量的算法，才直接优化动作。这里也只在副本上改b_2：真实轮次使用批量梯度、四组共享参数及Adam的历史梯度统计，其他样本还可能产生相反更新。一条损失下降无法保证未覆盖状态的输出正确，更无法代替闭环执行。

题2.2-4

设F_t表示第t步动作后已进入吸收失败区。每一步在此前仍正常的条件下，以概率ϵ首次出错，所以：

$$\Pr(\neg F_t) = \prod_{k=1}^{t} \Pr(\text{第 } k \text{步 正 确} \mid \text{此 前 均 正 确}) = (1-\epsilon)^t.$$

这依赖题设的条件错误率，不能只凭任意相关过程的无条件单步错误率使用乘积。第t步代价是F_t的指示函数，因此期望总代价为$\sum_t [1-(1-\epsilon)^t]$。也可以按首次出错时刻$k$分解：概率为$(1-\epsilon)^{k-1}\epsilon$，后续付出$T-k+1$步代价。

$\epsilon = 0.1, T = 4$时，逐步为$0.1, 0.19, 0.271, 0.3439$，合计0.9049。并集上界$\sum_t \min(1, t\epsilon) = 1.0$是上界，不是精确值。未截断的和为$\epsilon T(T+1)/2$；当$T\epsilon$小时，精确过程也出现平方级首项，总代价则始终不超过T。写成$T\epsilon = 0.4$漏掉了早期错误导致后面持续失败的成本。

为了理解恢复界，定义$Q_k^E(s,a)$为：从s先执行a，此后交给专家，在剩余k步中产生的期望总代价。这里是分析用的后果函数，不是另一个已训练网络；它使用“代价越小越好”的约定。若对于学生会访问的状态和相关时刻，有：

$$Q_{T-t+1}^E(s,a) - Q_{T-t+1}^E(s,\pi_E(s)) \le u,$$

那么u限制一次改用学生动作后，连同专家恢复，多造成多少**后续总代价**。它不是动作差的数值，也不只是一帧损失。

比较“前0步用学生、剩余用专家”“前1步用学生、剩余用专家”，直到“全部T步用学生”。相邻两种执行只多替换一个动作；把相邻总代价差相加，中间项消去，得到：

$$J(\pi) - J(\pi_E) = \sum_{t=1}^{T} \mathbb{E}_{s\sim d_\pi^t}\left[Q_{T-t+1}^E(s,\pi(s)) - Q_{T-t+1}^E(s,\pi_E(s))\right].$$

前缀已经用学生，所以状态来自d_π^t；后缀用专家，所以后果由Q^E描述。相同动作时差值为0，不同动作时至多为u，便得到$uT\epsilon_{\text{own}}$。ϵ_{own}是在学生各时刻访问分布上的平均不一致概率，而非专家数据上的误差。

若专家几步内可恢复，u可能与T无关；若一次错误损坏后面全部任务，$u = O(T)$，这个界仍可为$O(T^2\epsilon_{\text{own}})$。连续动作几乎处处不完全相等，离散不一致概率也未必有用。若想用连续MSE控制任务代价，需另建动作距离与后果之间的界，例如$|Q^E(s,a) - Q^E(s,a^E)| \le L\|a - a^E\|$；碰撞、接触和不可恢复失败未必满足这样的平滑关系，常数也可能随时域变大。

无遗憾的比较者是Π中的**同一个固定策略**。例如策略A在两轮的损失为$(0,1)$，B为$(1,0)$，最好固定比较者的平均损失是0.5，每轮分别取最优才能得0。后者是另一个更强的要求。模型缺少必要信息或表达能力不足时，固定比较者的最佳损失也可能不为零。

平均遗憾趋零不保证最后迭代好。例如A每轮损失0、B每轮损失1；学习者只在平方数轮次选B，其余选A。到第N轮，平均遗憾约为$1/\sqrt{N}$，但平方数末轮仍可能损失1。取$N = 10000$，平均遗憾为0.01，末轮损失仍是1。这是关于平均保证的反例，不是重现本书训练。

原DAgger分析还控制专家混合引起的分布差异和有限采样误差，结论可支持候选中存在较好的策略。可以用独立验证选择，也可以回合开始时均匀抽一版，并在整个回合保持；不能据此推断每个控制步随意换网络版本也有同样保证。按本节索引，π_N是在最后一轮采样后才训练出来的，更不能直接把π_{N-1}的采样结果当成它已经测过的表现。平方损失和有限次Adam本身也不保证深度网络满足所需的无遗憾性质。

题2.2-5

专家看见速度，学生当前图像却把两种速度合并成同一输入，因而相反标签可能都正确。若等概率标签为$-v, +v$，确定性回归输出同一个a，条件平方风险为：

$$\frac{1}{2}(a-v)^2 + \frac{1}{2}(a+v)^2 = a^2 + v^2.$$

最优输出是0，仍留下v^2的不可消除误差；$v = 0.12$时为0.0144。增加同样的标签，可以更准确地估计这个冲突分布，却没有提供区分当前方向的新信息。生成策略可以保留两种模式，避免平均动作，但在平衡且必须立即二选一时，仍无法仅凭相同输入把正确率提高到$1/2$以上。额外观察、等待或主动探测改变了信息条件。

可在仿真中构造当前图像相近、速度相反的成对状态，保存完整历史、真实状态、执行动作和时间。专家在指定时刻按真实状态标注，普通单帧学生仍只读单帧。按回合和初始条件划分数据，建立共同测试集，包含正常、离轨恢复及观测混叠案例。主要对照为：

输入	数据覆盖	主要检验
单帧	窄覆盖	原基线
单帧	恢复覆盖	仅补访问状态的作用
短历史	窄覆盖	增加过去信息的作用
短历史	恢复覆盖	信息与恢复覆盖是否同时需要

尽量匹配标签、专家、容量和训练预算，无法匹配的计算差别单独报告。另用同一日志比较原始配对与校正时间后的训练；评价标签始终正确对齐，不能为了对照而保留已知错误。看共同测试集动作误差、相反运动的动作方向和输出分布，再看相同独立重置条件下的成功、恢复时间、接管和失败类型。

短历史主要改善混叠案例，支持信息不足；恢复覆盖主要改善离轨输入，支持覆盖不足；校正时间后动作不再系统性追随旧位置，支持时间错位。这些瓶颈可以同时存在，需要逐因素比较。完整状态学生可作诊断参照，不能把它的得分当成单帧部署成绩。

循环模型若输入动作，应读取实际执行的a_{t-1}^{exec}，包括专家接管。未执行的纠正标签不能成为过去事实。训练窗口从回合中间开始时，可从回合起点重算，或用前一段真实历史预热，再对后段计损失。预热长度有限，得到的隐藏状态仍可能只是近似；缓存若来自旧参数，还可能与当前模型不一致。

预热放在no_grad中仍会算出携带过去信息的隐藏状态，只是不通过那段计算反传后续梯度；截断计算图也可以保留状态数值。这些操作与把状态置零不同。部署在同一回合连续更新隐藏状态，真正重置时清空对应环境；并行环境分别处理，不能因一个环境结束就清空全部历史。比较完整重算、不同预热长度、窗口置零和不做回合重置，可以观察异常是否集中在窗口或回合起点；历史完整并对齐后仍失败，再检查历史长度、遮挡和表示能力。

2.3 动作块策略在操作任务中的训练和检验

2.3.1 从示范窗口到可执行动作块

一次根据当前条件预测未来 H 个动作，称为**动作块**。它将多个未来位置的动作作为一段序列共同预测，有机会表达动作之间的协调。预测长度与模型调用频率是两件事：可以执行若干步后再算新块，也可以每个控制步都重新预测。只有实际减少调用次数时，才会摊薄模型推理开销；每步预测并做时间集成仍需每步推理。

第1.8节讲的是动作块怎样通过潜变量目标训练；本节接着说明它怎样进入控制循环。以原ACT为例，一份训练样本包含当前图像、本体关节位置，以及从该时刻开始的未来示范动作块。推断器借助标签产生 z，动作生成支路根据当前观测与 z 预测整块，再与标签比较。模型更新后，部署保留动作生成支路、固定 $z = 0$，输出尚未执行的关节目标序列。

阶段	读取什么	产生什么	更新或执行什么
训练取样	同一回合的当前观测与后续示范窗口	条件 c_t、标签 A_t、填充标记	此时不执行机器人动作
潜变量训练	本体状态、A_t；动作支路另读图像	μ, ℓ, z、预测动作块与损失	梯度更新推断器和动作生成网络
部署预测	当前观测、规定的 $z = 0$	$\hat{A}^{(t)} = [\hat{a}_t^{(t)}, \ldots, \hat{a}_{t+H-1}^{(t)}]$	不更新模型参数
控制选择	覆盖当前物理步的一个或多个预测	当前命令 a_t	下发低层控制器，推进物理并获得下一观测

上标 (t) 表示这份预测在哪个时刻生成，下标表示它打算控制哪个物理步。例如 $\hat{a}_{12}^{(10)}$ 是在10时刻预测、准备在12时刻使用的命令；它不是已经发生的动作，也不是环境后来实际达到的关节位置。

训练窗口靠近回合末尾时，后面可能不足 H 个真实动作。补齐数组仅为了形成同形状批次；填充项应在推断器的读入和重建监督中遮蔽，不应当作“机器人需要执行零命令”的示范。完整训练推导见 1.8 节。

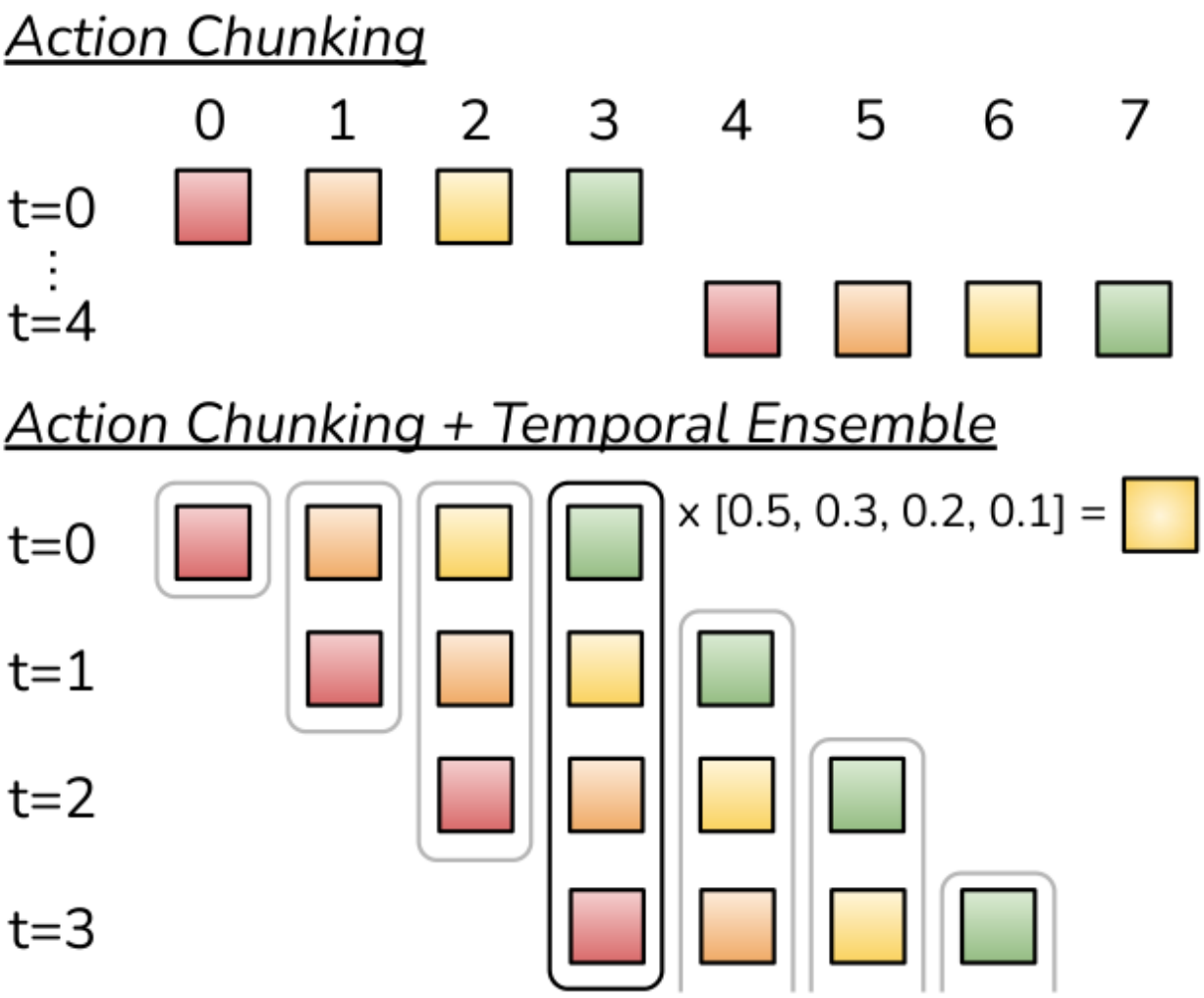


图2.3-1：Zhao等，ACT原论文Figure 5。上半图每隔一块才重新预测；下半图每个物理步都预测，斜排的块因此重叠。每一列对应同一物理时刻，集成沿列进行；图中的乘号与权重列表表示加权，实际命令还需按权重总和归一化。原图来源

设物理控制周期为 Δt，实际开环执行的前缀长为 k。在同步、忽略额外推理延迟的设置中，先执行前 k 步，再读取新条件并重新预测，**重规划周期**就是 $T_{\text{replan}} = k\Delta t$。例如 $\Delta t = 0.05$秒、$H = 8, k = 2$，预测覆盖0.4秒的命令时段，每次执行0.1秒就更新。若每个物理步生成新块并时间集成，新预测周期则为 Δt。异步运行还需单独记录生成延迟，不能直接套用这个等式。

模型职责见第1.1节，动作块时间关系见第1.7节。

增大 k 会减少这个同步设置中的推理次数，也延长新观测进入策略前的时间，目标移动、接触变化或早期偏差因此更晚得到修正。增大 H 改变的是预测范围，不会自动改变调用频率，还会增加有效标签覆盖与较远未来预测的难度。这些量需要结合任务动态、推理延迟和控制接口分别选择。

同一控制时刻可能被多个历史动作块预测过，**时间集成**便把这些预测按权重组合。例如最近两个块对当前时刻给出 $a_t^{(1)}$ 与 $a_t^{(2)}$，执行 $\bar{a}_t = w_1 a_t^{(1)} + w_2 a_t^{(2)}$，其中权重和为 1。这样可以减小模型切换产生的命令跳变，但旧块依据的是旧观测。若两块只是同一路线上的小幅误差，组合可能更平稳；若一块左绕、一块右绕，平均仍可能撞障碍。因此要同时检查预测年龄、路线一致性和平滑性。

动作块训练不等于强化学习中的一个环境宏动作。接入**强化学习**（Reinforcement Learning，RL）时，还需重新明确回报、折扣和**信用分配**（Credit Assignment）。信用分配要回答的是：较晚获得的结果，应当怎样归因于此前的行动或计算？这些问题详见第3章。

2.3.2 不同动作生成路线应怎样比较

路线	表达与使用方式	主要检查
回归	直接给一个连续预测	模式平均、尺度和任务误差
混合分布	输出多个成分并选择或采样	模式坍缩与跨时间一致性
离散自回归	逐项输出动作 token	分箱、前缀误差与延迟
ACT 类	用条件和潜变量预测动作块	训练推断与部署先验是否一致
扩散 / Flow	从噪声逐步生成动作块	生成预算、有效轨迹与闭环修正

多模态表示有助于避免平均两种正确路线，但并非复杂方法必然胜出。若条件已明确唯一合适动作、数据有限且延迟紧，简单回归可能是有竞争力的方案。

同样使用平方损失，为何**扩散**和 Flow 可以生成分布？单值 **BC** 回归的是仅给当前条件的最终动作；去噪与 Flow 回归的是给定带噪或中间变量、生成进度与条件的噪声或方向。不同随机起点和逐步生成共同形成最终分布，因此不等于先把两条完整动作取均值。有限模型仍可能丢模式或生成无效轨迹，需要验证。

比较时固定训练数据、输入信息、有效动作定义、控制周期和执行前缀。生成式策略抽十次再用强评价器筛选，不能与只输出一次的模型直接归因比较。可同时报告固定预算性能与各自最佳成本收益。

离线动作误差、动作分布覆盖和真实任务成功是不同指标。生成多条视觉上合理轨迹，也可能全都不可达。模型选择应通过闭环和失效分析接到任务，而非按参数量、模型名或生成步数排序。

2.3.3 时间集成先对齐同一物理时刻，再合并预测

动作块的第0项永远表示“从该块起点开始的第一步”，不是某个固定的全局控制时刻。设时刻10生成 $[a_{10}, a_{11}, a_{12}]$，时刻11生成 $[b_{11}, b_{12}, b_{13}]$。运行到时刻12，可以组合的是 a_{12} 与 b_{12}，即旧块第2项和新块第1项。直接把两块同一数组位置平均，会混合不同时间的命令。

若这两项都在同一坐标系、同一单位下表示同一时刻的目标，并且属于一致的运动路线，可按预定权重合并。例如 $\bar{a}_{12} = 0.25a_{12} + 0.75b_{12}$ 偏向较新观测。这是教学权重；具体算法可以采用不同的年龄权重。若动作是逐步位移增量，还要确认各预测的起始状态是否一致；若一条左绕、一条右绕，则先检查路线是否冲突，平均并不天然可行。

实现中应让每个块同时保存起始控制编号、预测长度、动作语义和观测时间。执行时用当前编号减块起点找到相应项，忽略已过期或无覆盖的块，再做组合。ACT 的时间集成和异步动作块的衔接都依赖这个对齐过程；后者还要处理新块生成期间已经执行的前缀。把预测年龄、块长度和执行前缀分别改变，能判断改善来自平滑、未来协调还是更及时的闭环。ACT 作者项目与时间集成说明

2.3.4 三条动作块记录，怎样连续选择命令

先在同步、无额外推理延迟的教学设置中运行：预测起点就是当前观测的控制编号。每块长3步，只展示一个关节的绝对角度目标，单位为弧度；它不是第6章Panda的末端位移增量。三次预测记录为：

块起点	局部索引0	局部索引1	局部索引2	对应物理步
10	0.10	0.12	0.15	10、11、12
11	0.13	0.17	0.21	11、12、13
13	0.00	0.05	0.09	13、14、15

到物理步12，起点10的块应取第2项0.15，起点11的块取第1项0.17；起点13的块尚未生成。只有前两项可以比较，它们都在预测同一时刻的关节目标。

原ACT把这些候选按生成时间从旧到新排列，用：

$$w_i = \frac{\exp(-mi)}{\sum_{j=0}^{n-1}\exp(-mj)}, \qquad a_t = \sum_{i=0}^{n-1} w_i \hat{a}_t^{(s_i)}, \quad s_0 < s_1 < \cdots < s_{n-1}. \tag{2.3-1}$$

i 是候选从旧到新的排列序号，n 是当前候选个数。因此 $m > 0$ 时旧预测的权重更大；它并不是按“越新越重要”定义的年龄衰减。作者代码取 $m = 0.01$，使相邻候选的差别较小。下面为方便手算取 $m = \log 2$，两项权重便为 $2/3, 1/3$，这是本例的设置。ACT作者时间集成实现

物理步12的命令为 $0.15 \times 2/3 + 0.17 \times 1/3 \approx 0.156667$。物理步13，起点10的块已经过期，起点11的第2项0.21与新块的第0项0组成候选，命令为 $0.21 \times 2/3 + 0 \times 1/3 = 0.14$。物理步14只剩起点13的第1项，命令为0.05。

这里的零目标仍是有效预测。判断一行是否存在，应根据块起点与长度，不能根据动作是否非零。否则物理步13会错误地丢掉新块，只执行0.21。到了物理步16，三条记录都不覆盖该时刻，情况才是真正的“没有预测”；这也不能悄悄改成目标0。

图2.3-2：每行是一份动作块，列是物理步。实心格表示有对应预测，空白表示没有覆盖；0.00所在格仍然有效。黄色列突出物理步12的两项候选。

最小选择过程如下，对应 practice/temporal_ensemble.py 的T1—T4注释。程序同时保存观测编号，便于检查预测年龄；异步推理还需要处理生成期间已执行的前缀，见第11.8节。

```
T1  对每块计算局部索引 j = 当前控制编号 − 块起点
T2  只保留 0 ≤ j < 块长度的项；无候选时明确报告缺失
T3  按块起点排序，计算并归一化约定权重
T4  检查命令含义与单位一致，对同一物理步的候选加权
```

```
# T1—T2：按时间覆盖判断有效性；零角度目标也应保留。
candidates = []
for chunk in chunks:
    index = current_step - chunk.start_step
    if 0 <= index < len(chunk.targets):
        candidates.append((chunk.start_step, chunk.targets[index]))
if not candidates:
    raise LookupError("当前物理步没有对应预测")

# T3：本例遵循从旧到新的排列，正decay使旧预测权重更大。
candidates.sort(key=lambda item: item[0])
raw = [math.exp(-decay * i) for i in range(len(candidates))]
weights = [value / sum(raw) for value in raw]

# T4：这里每项都是同一个关节、同一物理时刻的绝对角度目标。
command = sum(w * item[1] for w, item in zip(weights, candidates))
```

在学习包根目录运行 python practice/temporal_ensemble.py，无需安装PyTorch。结果应依次为约0.156667、0.14、0.05，并记录第16步无覆盖、零目标保留、拒绝不同动作语义和未来观测等检查。它验证索引与组合规则，没有驱动真实或仿真机械臂。

2.3.5 平滑命令之前，先看候选是否能合并

偏向旧预测可以维持原轨迹的连续性，也会延缓新信息对命令的影响；偏向新预测通常更快跟随观测变化，也可能放大新预测的抖动。这是闭环响应的取舍，不能仅用轨迹看起来更平滑来评价。

同一时刻、同一单位仍不足以保证平均可行。先把末端简化为一个点，只检查目标点是否占据障碍：两个位置目标为 $(-0.2, 0.1)$ m、$(0.2, 0.1)$ m，中心障碍占据 $[-0.05, 0.05] \times [0, 0.2]$ m。两个候选点都在障碍外，等权平均 $(0, 0.1)$ 却在障碍内。这个例子说明目标点满足约束，不保证它们的平均仍满足约束；到达各点的完整路径、机器人形状与动力学尚未在此验证。

因而应分别检查时间对齐、动作语义、候选自身的可行性和组合结果的可行性。若候选代表冲突路线，可考虑保持一条一致路线或重新规划，再通过碰撞检查与闭环结果验证。若同一路线只是小幅预测误差，时间集成更符合“减少切换抖动”的出发点，但组合命令仍须满足实际约束。

与第6章环境结合时，还需区分网络预测的命令与实际状态。关节目标经执行器和物理推进后才产生下一观测；将目标平均得更平滑，并不保证接触力更好或抓取成功率更高。评测应同时记录命令连续性、跟踪误差、扰动响应和任务成功。

2.3.6 练习题

题2.3-1 动作块长度 H 与执行前缀长度 k 分别改变什么？同步、忽略额外推理延迟时，重规划周期与 $k, \Delta t$ 有什么关系？每步重新预测并做时间集成时，为什么较大的 H 不代表更少的模型调用？为什么只增大 H 不一定使操作更稳定？

题2.3-2 使用2.3.4的三条记录和 $m = \log 2$，分别列出物理步12、13、14的有效块、局部索引、权重及命令。若程序用“动作不为0”判断有效性，哪一步会错，命令变成多少？第16步无覆盖与第13步零目标有何区别？

题2.3-3 原ACT从旧到新取 e^{-mi} 权重，是否优先使用新观测？如果只追求相邻命令变化小，可能遗漏什么失败？用2.3.5的两个位置目标计算平均值，在目标点障碍检查的简化条件下，说明时间对齐、候选可行和平均可行为什么是三个不同检查，再提出一项能检验时间集成取舍的受控实验。

2.3.7 参考答案

题2.3-1答案 H 决定联合预测多长，k 决定本次先实际执行多少步。在同步、执行 k 步后立即重新预测且忽略额外推理延迟时，重规划周期为：

$$T_{\text{replan}} = k\Delta t,$$

而非 $H\Delta t$。例如 $H = 8, k = 2, \Delta t = 0.05$秒，预测覆盖0.4秒，却每0.1秒就读取新观测更新。每步生成新块并时间集成时，策略仍每 Δt 调用一次；增大 H 不会自动减少调用。

长块有机会协调接近、夹紧、抬起等未来动作，也增加了联合拟合与标签覆盖的要求。较远预测更容易受目标移动、接触切换和跟踪偏差影响。若同时增大 k，纠偏还会更迟；若保持小 k 或每步重算，则是否获益取决于较远监督是否有用及增加的推理开销。“稳定”需要分开看命令平滑、扰动响应与任务成功，不能由 H 单独判断。

题2.3-2答案 局部索引为 $j = t - s$，只有 $0 \le j < 3$ 才有覆盖。两个候选从旧到新排列，未归一化权重为 $(1, 1/2)$，归一化后为 $(2/3, 1/3)$；只剩一个候选时其权重重新归一化为1。

物理步	有效块起点	局部索引	目标/rad	权重
12	10	2	0.15	$2/3$
12	11	1	0.17	$1/3$
13	11	2	0.21	$2/3$
13	13	0	0.00	$1/3$
14	13	1	0.05	1

$$
\begin{aligned}
a_{12} &= \tfrac{2}{3}(0.15) + \tfrac{1}{3}(0.17) \approx 0.156667 \text{ rad}, \\
a_{13} &= \tfrac{2}{3}(0.21) + \tfrac{1}{3}(0) = 0.14 \text{ rad}, \\
a_{14} &= 0.05 \text{ rad}.
\end{aligned}
$$

第12步新块13尚未生成；第13步旧块10已经过期；第14步块11也过期。若按数值非零筛选，第13步会误删新块的有效0目标，只剩0.21，错误命令为0.21 rad。绝对角度目标0也不等于“不给命令”或“保持当前位置”。

第16步的三个局部索引是6、5、3，均无覆盖，根本没有可加权的候选。教学选择器报告缺失；实际系统之后重新预测、等待还是采用别的回退行为，需要另外定义，不能偷偷补零。

题2.3-3答案 候选从旧到新编号，$m > 0$ 时 $w_{i+1}/w_i = e^{-m} < 1$，所以偏向旧预测。序号差不一定等于实际年龄差；如果中间未生成某块，不能把两者混同。旧命令即使很平滑，也可能已经过时，延迟对目标或接触变化的响应。

两个目标 $(-0.2, 0.1)$ m、$(0.2, 0.1)$ m 等权平均为 $(0, 0.1)$ m。候选横坐标都在障碍的 $[-0.05, 0.05]$ 外，但平均点的两个坐标都落在障碍矩形内。因此，同一时刻的目标可以对齐正确，各目标点也可以通过简化障碍检查，组合结果却失败。这里没有证明到达两个目标的完整路径或真实机器人运动可行。

可固定同一训练后策略、输入模态、H、控制周期、每步推理预算和低层控制器，只改变权重：旧预测偏重、等权、新预测偏重，并可加入只取最新候选的参照。在配对初始条件与相同外部扰动下测试静态和突然移动目标，重复多次，记录命令变化量、响应延迟、实际跟踪误差、碰撞与任务成功，并报告差异的不确定性。不同控制导致后续观测不同，是闭环结果的一部分；不能为了公平而强行规定整条观测轨迹相同。只有平滑改善而碰撞或响应恶化，不能据此称整体更好。

2.4 跨机器人迁移与接口适配

2.4.1 同一句“向杯子移动”，为什么不能直接复制动作

两台机械臂都要把红杯放到托盘，视觉上需要识别的对象、目标关系和操作阶段有许多共同之处。但一台有七个关节，另一台有六个；基座方向不同，夹爪宽度不同，底层控制器接收的命令也可能不同。共同任务并没有让两个输出向量自动拥有相同含义。

跨本体迁移（cross-embodiment transfer）讨论的，就是怎样把一个机器人或设备上学到的能力用于另一个身体。“本体”在这里指机械结构、传感器、执行器和它们的使用条件。需要区分两种迁移对象：一是可复用的表示，例如杯子在哪、下一步应该接近还是释放；二是身体相关的动作生成与执行，例如关节怎样运动才能让夹爪达到目标。

把短动作向量补零到同一长度，只解决了数组形状；把两路相机都改名为front，只解决了字段名。它们没有解决坐标、标定、视角、动力学或可达性。本节用一条连续过程说明：哪些差别可以按规则转换，哪些需要学习新参数，最后又该用什么实验判断迁移有效。

LeRobot的动作表示教程把关节 / 末端空间与绝对 / 相对表示分开介绍，适合作为实现入口。UMI作者的接口设计则说明，观察方式、动作表示和执行时序应一起考虑；单独改输出维度并不能完成迁移。

2.4.2 两种共享方式：共享动作含义，或共享内部表示

一种方式是让不同机器人使用共同的末端动作语义。例如策略都输出“相对本次预测起点的末端目标轨迹和夹爪宽度”，再由各设备的确定性转换器处理坐标、单位和命令格式。转换器本身不需要从示范中猜“米怎样换成毫米”；这些已知关系应明确写入接口。UMI采用相对末端轨迹，并通过相近的手腕相机 / 夹爪观察条件减小采集与部署差异，属于这一思路。共同语义仍需各身体有能力实现，不能越过关节限位和工作空间。

另一种方式是共享处理观测和任务的网络主干，为新身体增加或替换输入编码器、动作头。它允许输出保留不同身体的原生含义，例如某个头产生七维关节目标，另一个产生双臂十四维目标。共享的是内部表示和已有参数，不要求两个头的数字逐项相同。

共享方式	主要复用什么	还要明确什么
共同末端语义 + 设备转换器	几何动作和操作决策	参考系、动作基准、单位、时序、夹爪与本体执行约束
共享主干 + 身体专用输入 / 输出模块	视觉、语言及动作相关表示	新模块输入输出、标签、初始化、可训练参数和目标身体数据

两者可以组合，但不能混为一个已经完成的步骤。共同末端表示可能减少输出差异，却不能保证所有身体执行同一运动；身体专用头能表达不同命令，却仍需要正确标签。对于本来就已知的坐标和单位关系，先做确定性对齐，通常比让少量示范重新学习这些规则更容易检查。

2.4.3 相对轨迹不是“每一步再走这么多”

设本次预测开始时，末端位置是p_0，模型给出的未来目标为p_1, p_2, p_3。三种表示分别为：

$$\begin{aligned} \text{绝对目标：} &\quad p_1, p_2, p_3, \\ \text{共同起点相对量：} &\quad p_1 - p_0,\ p_2 - p_0,\ p_3 - p_0, \\ \text{逐步增量：} &\quad p_1 - p_0,\ p_2 - p_1,\ p_3 - p_2. \end{aligned} \tag{2.4-1}$$

共同起点相对量把每个目标都连回同一个p_0。恢复第h个目标只需$p_h = p_0 + r_h$；逐步增量则要累加$p_h = p_0 + \sum_{j=1}^{h} d_j$。前者的某一预测误差不必通过累加传播到后面所有目标，后者则具有这种编码上的依赖。这不是“相对动作消除了全部闭环误差”：起点估计、模型预测、控制跟踪仍然可能出错。

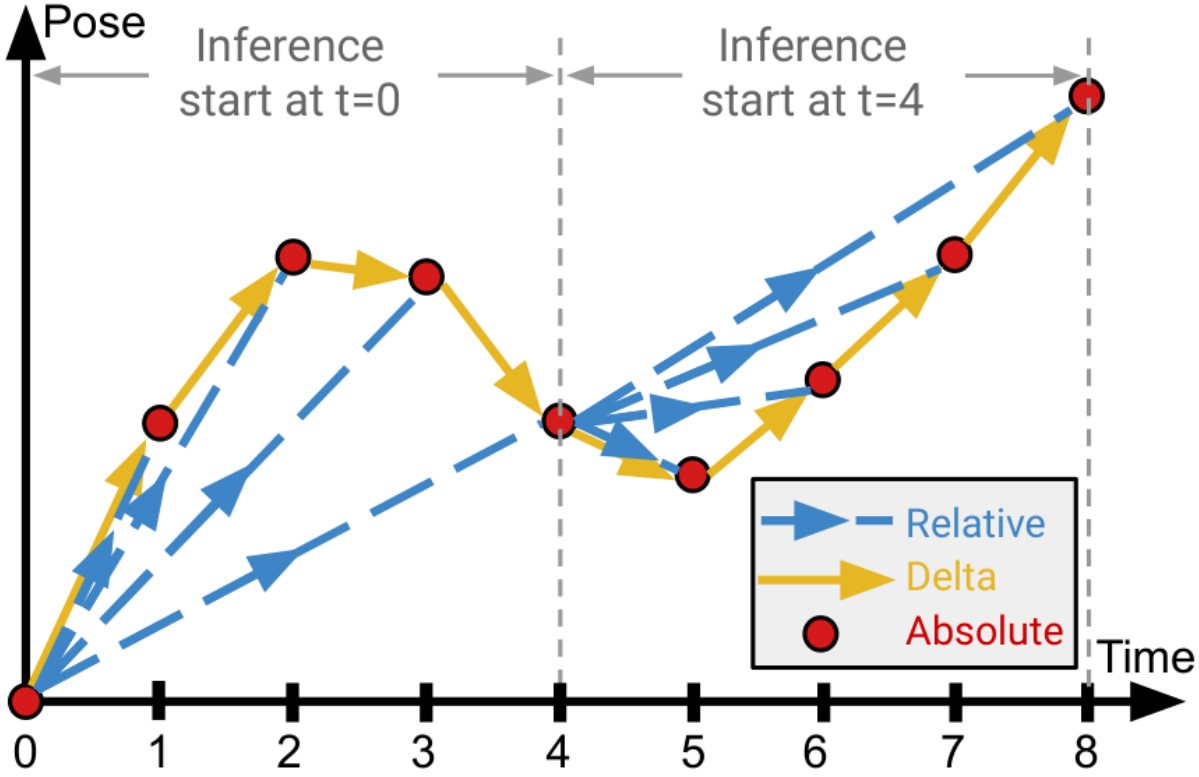


图2.4-A：UMI原论文Figure6。Time是时间，Pose是位姿的示意纵轴；红点Absolute表示绝对目标，黄箭头Delta连接相邻目标，蓝色Relative把同一预测块的目标连接到共同起点。Inference start at t=0 / t=4表示两次开始预测，第二次改用$t = 4$时的当前位姿作为新起点。图用于解释参照关系，不是某关节真实运行曲线。

因此，一块动作还没用完时，解码器不能每收到一帧新状态，就用这个新状态替换原块的参考起点。假设一维$p_0 = 0.4$，共同起点相对量的第二项为0.03；它表示绝对目标0.43。若第一项执行到0.41后改用0.41作起点，同一项就被解释成0.44，目标变了。

固定版本LeRobot的相对动作处理器采用配对的正向 / 逆向转换，并能在动作队列未耗尽时保持缓存起点。夹爪等不适合做位置差的通道可排除在相对转换之外。固定实现说明了这一接口；应用到具体系统时，还要核对动作队列和观测顺序。

完整位姿包含位置与朝向。UMI使用相对起点的刚体变换，不是把三个欧拉角逐项相减。设T_0, T_h为末端到同一参考系的齐次变换，即把旋转与平移放在同一个4×4矩阵中；相对变换为$T_0^{-1} T_h$。若起点与

目标的旋转分别为R_0, R_h，它包含的相对位置和相对朝向分别是$R_0^{\mathsf{T}}(p_h - p_0)$与$R_0^{\mathsf{T}} R_h$，还原时再左乘$T_0$。关于这些矩阵的完整推导见第4.1节；下面先固定朝向，只算位置与夹爪，以看清动作接口。

2.4.4 一条共同动作，怎样变成A、B两种命令

规定一个任务坐标系F。预测时末端位置为$p_0^F = (0.4, 0.1, 0.2)$米，两个机器人的工具朝向暂时保持不变。共同动作每行是“相对p_0^F的位置偏移、绝对夹爪宽度”，经反标准化后得到：

$$U = \begin{bmatrix} 0.01 & 0 & 0 & 0.04 \\ 0.03 & 0.02 & 0 & 0.02 \\ 0.04 & 0.02 & 0.02 & 0 \end{bmatrix}. \tag{2.4-2}$$

前三列单位为米、参考任务坐标轴，最后一列也是米，但表示绝对开口宽度，不与当前宽度做差。于是共同位置目标为$(0.41, 0.10, 0.20)$、$(0.43, 0.12, 0.20)$和$(0.44, 0.12, 0.22)$。约定它们分别期望在预测起点后的0.1、0.2、0.3秒到达；这些是目标时间，不是已测跟踪结果。本例中工具轴与任务轴平行；一般UMI相对工具位姿的方向还需要按起点朝向变换，不能忽略这一步。

两台教学设备都提供笛卡尔位置接口，但规定不同：

接口	机器人A	机器人B
位置坐标	基座A与任务系重合	基座B相对任务系绕z轴转90°，原点在$(0.5, 0, 0)$米
接收的位置命令	绝对目标，米	相邻目标间增量，毫米
夹爪标量	0闭合、1最大张开；最大0.08米	0最大张开、1闭合；最大0.10米

这里的A、B是用于解释转换的设备合同，不是某两款真实控制器的复现。它们的底层仍须把笛卡尔命令变成各自可执行的关节运动；当前数值程序没有求逆运动学或运行控制器。

设R和t把基座B坐标映射到任务系，即$p^F = Rp^B + t$。那么反向转换为：

$$R = \begin{bmatrix} 0 & -1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix}, \quad t = \begin{bmatrix} 0.5 \\ 0 \\ 0 \end{bmatrix}, \qquad p^B = R^{\mathsf{T}}(p^F - t). \tag{2.4-3}$$

为什么绝对点要减t，增量却不用？两个点相减时，平移项正好消去，所以$\Delta p^B = R^{\mathsf{T}} \Delta p^F$。相对旋转矩阵的转置才是这里的逆变换；把$R$直接当成$R^{\mathsf{T}}$，会把任务系向右0.01米误解成向左0.01米。

B的初始基座位置为$(0.1, 0.1, 0.2)$米，三个目标依次为$(0.10, 0.09, 0.20)$、$(0.12, 0.07, 0.20)$、$(0.12, 0.06, 0.22)$。对这些点取相邻差，再从米换成毫米；夹爪则分别用$w/0.08$和$1 - w/0.10$编码。最终两台设备接收：

目标序号	A：绝对位置 / 米；张开比例	B：顺序增量 / 毫米；闭合比例
1	$(0.41, 0.10, 0.20;\ 0.50)$	$(0, -10, 0;\ 0.60)$
2	$(0.43, 0.12, 0.20;\ 0.25)$	$(20, -20, 0;\ 0.80)$

目标序号	A：绝对位置 / 米；张开比例	B：顺序增量 / 毫米；闭合比例
3	$(0.44, 0.12, 0.22;\ 0)$	$(0, -10, 20;\ 1)$

两组数字很不一样，却能还原成相同的任务系位置与物理宽度。反过来，把同样四个数直接交给两个接口，往往才会产生不同运动。检查语义时，应解码回共同物理量比较，而非要求两个向量逐项相等。

若错误地把B的共同起点相对量当成逐步增量累加，三个任务系目标会变成$(0.41, 0.10, 0.20)$、$(0.44, 0.12, 0.20)$、$(0.48, 0.14, 0.22)$。第一项看起来正确，后面却逐渐重复累加。因此只测第一个动作不足以发现接口错误，至少要检查完整动作块的还原结果。

如果加入朝向，还要区分绕工具轴还是任务轴转动：$R_{\text{new}} = R_0 \Delta R_{\text{tool}}$与$R_{\text{new}} = \Delta R_F R_0$通常不同。本节程序用初始绕$z$轴90°、再绕$x$轴30°的例子展示区别，详细运动学放在第4章。位置换算正确也不等于路径可达、无碰撞或能稳定接触。

2.4.5 归一化统计应属于哪一种动作

模型常在标准化后的标签上训练：$z = (u - \mu)/\sigma$，推断后再还原为$u = z\sigma + \mu$。这里的μ, σ必须对应**送入模型的那种表示**。绝对位置、相对起点的位置差、逐步增量是不同随机变量，不能只因都占前三列就共用未经核对的统计。

例如当前位置0.4米、目标0.41米，相对标签是0.01米。按本例指定的均值0、标准差0.02米，得到$z = 0.5$；把它代入绝对位置常用的均值0.5、标准差0.2，则得到-2.45；先分别标准化绝对目标和状态再相减，得到0.05。三者使用了不同的数值坐标，不能交给同一固定解码器任意互换。

另一套仿射变换并非数学上不可用。若从训练开始就按$(0.5, 0.2)$编码相对动作，部署也用同一套逆变换，则$(-2.45) \times 0.2 + 0.5 = 0.01$米，仍能还原。错误发生在混用时：把$-2.45$交给原来的$u = 0.02z$，会得到$-0.049$米偏移，绝对目标变成0.351米。尺度同时改变损失：标准化平方误差等于物理平方误差除以σ^2；两种尺度对同一物理误差的权重相差100倍。多维或多身体训练时，这还会改变各项相对权重。可逆与学习效果相同，是两个不同问题。

原因不只在均值：$\mathrm{Var}(a - s) = \mathrm{Var}(a) + \mathrm{Var}(s) - 2\,\mathrm{Cov}(a, s)$。相对动作的变化量取决于目标与当前状态共同怎样变化，不能从单独的绝对动作标准差直接推断。应先完成语义转换，再从训练划分计算对应统计；推断端按相反顺序还原。这里的标准化不要求数值都落在$[-1, 1]$，本节例子的某些标准化偏移大于1仍然有效。

式(2.4-2)对应的教学标准差为$(0.02, 0.02, 0.02, 0.04)$米、均值为0，因此标准化输出是$[0.5, 0, 0, 1]$、$[1.5, 1, 0, 0.5]$、$[2, 1, 1, 0]$。前三维反标准化后才加起点，夹爪宽度直接按设备规则编码。真实训练应使用实际训练数据的统计；这里给定这些值，只为让完整链条可复算。

2.4.6 补齐维度以后，哪些零是真的

假设把七维与六维关节目标都存进七列数组。六维机器人最后一列只是占位，不代表一个需要学会停在零的第七关节；其余有效维度的真实零值却应该继续受监督。因此需要另存动作有效掩码m_j，例如六维样本为$(1, 1, 1, 1, 1, 1, 0)$。

若希望每条样本按自身有效维度取平均，可定义：

$$\ell_r = \frac{\sum_j m_{r,j}(\hat{y}_{r,j} - y_{r,j})^2}{\sum_j m_{r,j}}. \tag{2.4-4}$$

分母决定了不同身体样本的相对权重。只把无效误差乘零，却仍统一除以7，会把六维样本整体缩小为原来的$6/7$。这也可以是有意选择，但必须理解它改变了优化权重，不能把两种归约当成同一目标。若一条样本没有任何有效动作标签，应跳过该项监督，不能除以零。

配套例中，六个有效维度的平方误差和是0.12，正确有效均值是0.02；误除以7则为约0.017143。如果占位第七维预测为9而标签填0，忘记掩码会得到约11.588571，训练主要在优化一个不存在的关节。这个例子与前面四维末端 / 夹爪合同是两个不同表示的教学切片，不把七维关节直接塞进四维接口。

观测也有类似问题。缺失相机和真正拍到黑色图像不是一回事，改名也不等于视角配准。实际LeRobot π_0实现为缺失图像构造占位和无效掩码；Octo按观测元数据生成有效标记，缺少相关元数据时还可能回退成全有效。因此，不能只看填了什么像素值，还要检查它是否真的从读取中排除。LeRobot观测接口教程讲解了改名与空相机的区别；注意力掩码原理见第1.4节。

2.4.7 新身体到底训练哪些参数：看一个完整适配过程

Octo提供了一个具体例子：把预训练模型用于新的单相机、本体状态输入和双臂动作数据。它没有仅把旧动作补齐到14维，而是调整模型输入和输出：

1. 按新数据的实际字段读取相机、本体状态与任务标签，建立新的训练批次。
2. 移除不再使用的腕相机编码入口，加入本体状态编码入口。
3. 把原动作头换成预测50个十四维目标的L1动作头；这里的十四维由新数据的动作语义决定。
4. 创建新结构，再复制参数路径和形状都能匹配的预训练权重。新输入位置编码和新头的一部分参数没有对应旧值，需要初始化。
5. 用目标身体示范计算动作损失，按选择的训练范围更新新模块及主干；推断使用对应的新数据统计和动作解码器。

这是固定作者适配示例的结构过程；本书没有运行其JAX训练或下载该示例检查点。其原始预训练动作头使用扩散，不意味着所有下游头必须继续用扩散。对新标签和任务选用不同头，是可适配设计的一部分。

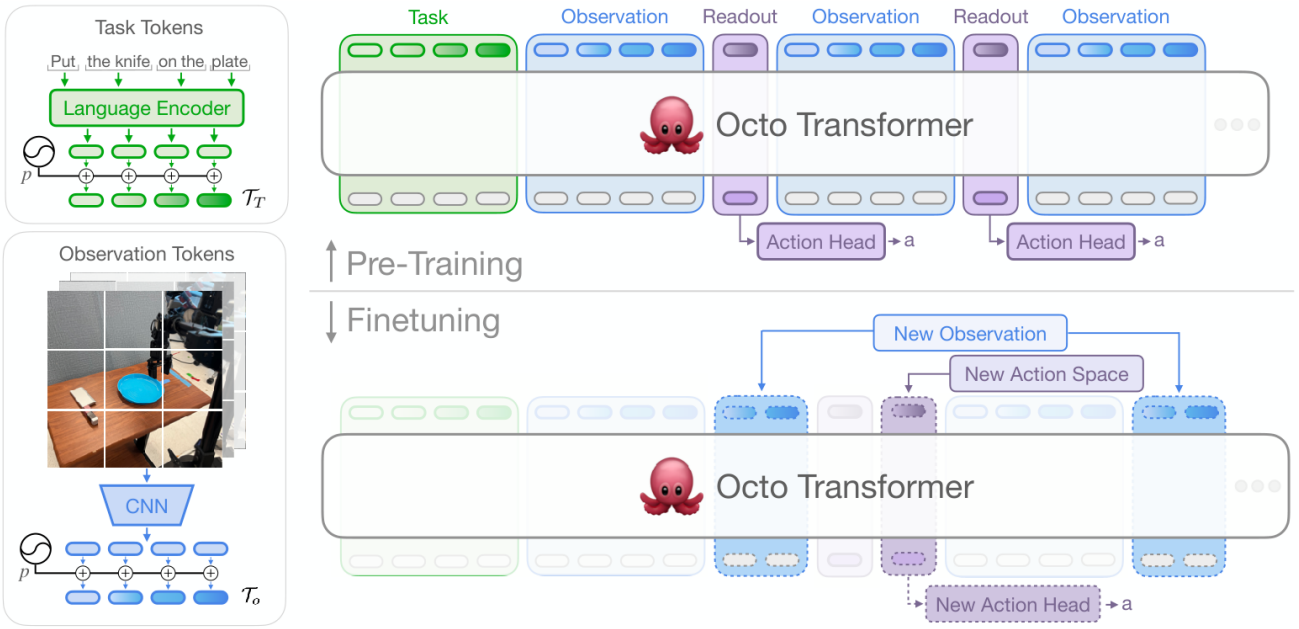


图2.4-B：Octo原论文Figure2。绿色Task是任务标记，蓝色Observation是观测标记，紫色Readout是读取当前信息供动作头使用的标记；Action Head把它转成动作。左侧Language Encoder是语言编码器，CNN（Convolutional Neural Network，卷积神经网络）把图像编码为标记，p表示加入的位置编码。上半部Pre-Training是预训练，下半部Finetuning是适配；New Observation和New Action Space对应新增输入与输出。虚线标出需要新增的部分，不代表网络已经无需新数据就会执行新身体。

可以把“共享主干 + 专用模块”的一般形式写为：

$$z^r = f_\phi(E_{\psi_r}(c^r)), \qquad \hat{y}^r = H_{\omega_r}(z^r). \tag{2.4-5}$$

r是身体编号，c^r是该身体可用的输入，E_{ψ_r}将它编码为公共内部格式，f_ϕ是共享主干，H_{ω_r}产生该身体的输出。这个式子帮助说明参数职责，不声称每个公开VLA都严格按三个独立模块实现。身体编号或身体状态也可作为共享模型条件，而不必总建立完全独立的头。

新身体的监督首先教会新头“怎样从已有表示产生新标签”。只训练新头时，主干仍要前向产生表示，只是它的参数不更新；同时训练新输入模块时，梯度还可能需要穿过冻结主干传回输入侧。冻结与no_grad的区别见第1.2节，低秩适配的更新机制见第1.14节，不在这里重复推导。

实际冻结还应检查参数分区。该固定示例向冻结模式追加BlockTransformer_0，底层却对完整点分隔叶路径做fnmatch；裸模块名并不能据此认定已匹配全部主干参数。复用时应列出实际匹配范围，再核对一次更新前后的数组。新输入模块若也需要训练，还要防止宽泛的主干冻结模式把它一并冻结；固定主干参数与停止输入侧梯度是两回事。

2.4.8 共享参数如何同时收到两个身体的训练信号

用一个标量例看参数更新。令共享表示为$z = \phi x$，A、B的头分别为$\hat{y}_A = w_A z$、$\hat{y}_B = w_B z$。两个样本的x都取1，标签分别为2和−3，损失为$L = \frac{1}{2}[(\hat{y}_A - 2)^2 + (\hat{y}_B + 3)^2]$。这些是无物理单位的表示 / 头参数例，不是前面设备命令。

初始$(\phi, w_A, w_B) = (1, 1, -1)$，预测为1、−1，误差为−1、2。由链式法则：

$$\frac{\partial L}{\partial \phi} = (-1)(1) + (2)(-1) = -3, \quad \frac{\partial L}{\partial w_A} = -1, \quad \frac{\partial L}{\partial w_B} = 2. \tag{2.4-6}$$

共享参数ϕ收到两者通过各自头传回的梯度，A头只从A样本产生直接梯度，B头同理。学习率0.1并同时更新，得到$(1.3, 1.1, -1.2)$，新预测为1.43和−1.56，损失从2.5降到1.19925。冻结ϕ、只更新两个头，则参数为$(1, 1.1, -1.2)$，损失为2.025。它说明自由度怎样改变拟合过程，不证明实际少量数据上全参数训练一定更好；后者还要考虑过拟合与遗忘。

共享也可能冲突。另设两个身体被迫用同一标量θ预测，标签分别为$+1, -1$，采样权重为0.9和0.1。初始$\theta = 0$，对加权半平方损失做学习率0.1的一次更新，θ变为0.08。A的平方误差由1降为0.8464，B却从1升为1.1664，总加权损失仍从0.5降为0.4392。这就是总体指标变好、少数身体变差的一种机制。

这个例子故意省掉身体条件，表示能力不足。可以通过输入身体信息、建立专用头或先统一已知动作语义，消除这类本来可区分的冲突。**负迁移**（negative transfer）指迁移或共同训练反而损害目标表现；要用匹配条件下的合适基线比较，单步梯度冲突只是可能的成因。真实负迁移还可能来自视觉域、任务和数据质量差别。改变身体采样权重p_r，会改变目标$\sum_r p_r \mathbb{E}_{\mathcal{D}_r} \ell_r$。重采样是目标选择，而不是无条件的“数据越均衡越好”；应同时看每种身体和任务的结果。

2.4.9 从可复算转换走到可执行迁移

配套程序的P1—P5对应下面的处理顺序，M1—M2处理缺失维度，G1—G3计算上一节参数例，R1展示朝向乘法差别。

```
P1  读取身体、动作表示、统计与本次预测起点；反标准化共同输出。
P2  把任务系目标变到对应基座系，明确变换方向。
P3  按绝对 / 共同起点相对 / 逐步增量编码，换单位并编码夹爪。
P4  检查字段、维度、数值范围；交给本体约束与控制检查。
P5  反向解码到共同物理量，核对完整动作块与宽度。
```

P4前半部分只能发现数据接口错误；可达性、碰撞、速度和稳定接触还需要运动学、控制与真实执行。程序只做表示转换和参数算术，没有实现这些物理检查。即使解码位置一致，不同控制周期也需要明确的插值或调度，不能把三个目标直接当成任意频率下的三个控制步。第6.8节PandaTray有自己每步累积末端位置、夹爪与控制周期的合同，不能把这里的绝对目标或毫米增量直接当成它的四维动作。

真实迁移先验证同一目标在各身体上被正确解释，再测实际跟踪，最后测试抓取、放置和恢复。测试条件至少分开：已见身体的新任务、同类任务的新身体、身体与任务都新。报告目标身体使用了多少示范、是否更新了模块、归一化统计来自哪里，以及是否用了额外状态或标定。常规独立测试不应再用测试数据计算训练统计或反复调配置；若协议允许未标注的目标域数据参与适配，也要单独说明，不能与完全没有目标数据的直接部署混称。

微调可以从只训练新模块的基线开始，再按错误类型扩大更新范围。若视觉表示本就不适合新相机，只训练动作头可能不够；全参数更新更灵活，也可能遗忘源任务。匹配目标示范、初始化、训练与调参预算，比较共享预训练适配、合适的单身体基线及从头训练；再测源身体任务，才能区分迁移收益与遗忘。旧任务回放可以改变这个取舍，但其数据量、质量和计算同样要报告。

评价源身体时，还要保留原观测处理、源动作头、统计和解码路径。可保留源专用分支，或按预先约定把更新后的共享参数放回源结构；目标适配若已删去源输入并替换源头，不能只给新模型一个“源任务”名称就算测过遗忘。

2.4.10 练习题

题2.4-1：一条动作的完整语义。 从式(2.4-2)和A、B合同出发，推导三个目标在B基座系的位置、顺序增量及夹爪标量，并反向还原到任务系。解释点变换为何包含平移、增量为何没有平移。若把共同起点相对量当成逐步增量，哪些步骤第一次出现错误，最后偏到哪里？若动作块尚未用完就更新解码起点，又会发生什么？再用$R_0 = R_z(90^\circ)$与$\Delta R = R_x(30^\circ)$说明左右乘为什么表示不同动作。

题2.4-2：同样的列数是否就是同样的目标。 复算0.4到0.41米的三种标准化结果，解释相对标签的均值 / 方差为什么不能照搬绝对动作统计。六维目标补到七维后，有效六维平方误差和为0.12，占位第七维误差为9，分别计算三种归约；某个有效动作维度的标签数值恰为0时，应不应该掩码？若一条样本完全没有有效标签呢？缺失相机填0为何也不够？说明训练与推断端应按什么顺序做表示转换、标准化和逆转换。

题2.4-3：共享与专用参数如何更新。 推导式(2.4-6)，分别算全参数与只训练头的一次更新。解释B样本怎样影响共享主干，而不直接给A头产生梯度；A头不更新是否意味着A行为不变？再复算0.9 / 0.1采样权重下的冲突例，说明总损失下降为什么不能证明所有身体受益，并给出能消除该例表示冲突的改进方向与适用前提。

题2.4-4：怎样判断新身体适配真的学到了东西。 结合Octo示例说明新增输入、换头、权重合并和冻结各解决什么问题，为什么“参数形状可加载”不足以证明语义可用。设计一个同时检查动作接口、目标身体收益与源身体遗忘的实验：明确数据划分、训练统计、共同评价、更新范围与成本；区分预训练身体的新任务、全新身体适配和无需适配的直接部署，不把它们混成一个分数。

2.4.11 参考答案

题2.4-1

先还原共同任务系目标：$p_h^F = p_0^F + r_h$，依次为$(0.41, 0.10, 0.20)$、$(0.43, 0.12, 0.20)$、$(0.44, 0.12, 0.22)$米。B到任务系的关系是$p^F = Rp^B + t$，所以逆变换为$p^B = R^\mathsf{T}(p^F - t)$。按分量展开就是$(x^B, y^B, z^B) = (y^F, 0.5 - x^F, z^F)$。代入起点和三个目标，得到：

点	B基座系位置 / 米
起点	$(0.10, 0.10, 0.20)$
目标1	$(0.10, 0.09, 0.20)$
目标2	$(0.12, 0.07, 0.20)$
目标3	$(0.12, 0.06, 0.22)$

B要的是相邻目标差，因此取第一项减起点、后项减前项，再乘1000，得到$(0, -10, 0)$、$(20, -20, 0)$、$(0, -10, 20)$毫米。夹爪三项是绝对宽度0.04、0.02、0米，按$1 - w/0.10$变成闭合比例0.6、

0.8、1；它没有随位置一起做差。反向从B起点累加毫米命令除以1000，再用$Rp^B + t$变回任务系，恰好恢复上述三目标；宽度用$0.10(1 - s_B)$恢复。

点坐标随原点改变而改变，所以要处理t；两个点相减时，平移抵消：

$$\Delta p^B = R^{\mathsf{T}}(p^F_{\text{next}} - t) - R^{\mathsf{T}}(p^F_{\text{prev}} - t) = R^{\mathsf{T}}\Delta p^F.$$

若把共同起点相对量直接累加，得到$\tilde{p}_h = p_0 + \sum_{j=1}^{h} r_j$。第一目标仍正确；第二、第三目标变为$(0.44, 0.12, 0.20)$和$(0.48, 0.14, 0.22)$，终点多走$(0.04, 0.02, 0)$米。问题不是坐标旋转，而是同一段偏移被重复累加。若第一项执行后只把解码起点改成p_1，旧的r_2就被解释为$p_1 + r_2 = (0.44, 0.12, 0.20)$。可以保持旧块起点，也可以重新预测；若只想改写到新起点$p_0 + \delta$而保持原目标，则必须同时令$r'_h = r_h - \delta$。

朝向不是普通向量相加。令$c = \cos 30^\circ = \sqrt{3}/2$、$s = \sin 30^\circ = 1/2$，两种乘法分别给出：

$$R_0 R_x(30^\circ) = \begin{bmatrix} 0 & -c & s \\ 1 & 0 & 0 \\ 0 & s & c \end{bmatrix}, \qquad R_x(30^\circ) R_0 = \begin{bmatrix} 0 & -1 & 0 \\ c & 0 & -s \\ s & 0 & c \end{bmatrix}.$$

右乘绕当前工具x轴旋转，第一列即工具x轴仍为$(0, 1, 0)$；左乘绕固定任务系x轴旋转，工具x轴变成$(0, c, s)$。转轴不同，结果自然不同。完整工具相对位姿$T_0^{-1} T_h$中的平移为$R_0^{\mathsf{T}}(p_h - p_0)$，已经转到起点工具轴；本节主算例固定工具朝向并使用任务轴位置差，不能直接省略一般情况下的这次旋转。

题2.4-2

实际相对量为$u = 0.41 - 0.4 = 0.01$米。按指定相对统计得到$u/0.02 = 0.5$；代入另一套仿射参数得到$(0.01 - 0.5)/0.2 = -2.45$；分别标准化绝对目标和状态再相减，则为$-0.45 - (-0.5) = 0.05$。最后一种仍是$u/0.2$，与题目指定的尺度相差十倍。

这些数不同，并不意味着第二套仿射不可用。始终配对时，$(-2.45) \times 0.2 + 0.5 = 0.01$米；混用时，例如把$-2.45$交给原相对解码器$u = 0.02z$，会得到$-0.049$米，绝对目标错成0.351米。应检查完整的编码—解码关系。尺度还改变训练权重：$(\hat{z} - z)^2 = (\hat{u} - u)^2/\sigma^2$，0.02与0.2对应的物理平方误差权重相差100倍。可逆并不等于有限模型、有限步优化和输出限幅下的学习效果相同。

相对量$U = A - S$的均值为$\mu_A - \mu_S$，方差为：

$$\operatorname{Var}(U) = \operatorname{Var}(A) + \operatorname{Var}(S) - 2\operatorname{Cov}(A, S).$$

目标常随当前状态共同变化，协方差不能任意删去。例如等概率记录$S = (0.3, 0.4, 0.5, 0.6)$、$U = (0.01, 0.02, 0.01, 0.04)$，则$A = S + U$。三项分别为0.01465、0.0125、0.0135平方米，组合后得到相对方差0.00015平方米，恰好等于直接对U计算。忽略协方差会得到0.02715，描述的是另一种变化范围。

六个有效维度平方误差和0.12：按有效维平均为0.02；掩码后仍除7为约0.017143；把第七维误差9也计入，则为$(0.12 + 81)/7 \approx 11.588571$。第二种改变了样本权重，第三种在训练不存在的关节。有效标签恰为0仍应监督；完全无标签时跳过这项，并按有效样本归约。先按每条样本取有效均值，再平均样本，与全批所有有效元素一起平均也有不同权重。

缺失图像填零并不会自动退出注意力。例如真实值2、占位值0、两者分数均为0，均匀注意力输出1；若正确排除占位，输出应为2。必须另传有效标记，并确认它真正用于读取掩码；动作损失掩码则负责排除不存在的监督，二者作用不同。

训练顺序是统一单位和参考系→构造实际动作表示→仅用训练划分计算统计→标准化标签。部署按相反顺序：反标准化→用缓存起点恢复目标→转换基座坐标和绝对 / 增量格式→换单位、编码夹爪。测试集不再用来改训练统计。每一步都要沿用同一种动作含义。

题2.4-3

令$e_r = w_r\phi x_r - y_r$。链式法则给出：

$$\frac{\partial L}{\partial \phi} = e_A w_A x_A + e_B w_B x_B, \qquad \frac{\partial L}{\partial w_A} = e_A \phi x_A, \qquad \frac{\partial L}{\partial w_B} = e_B \phi x_B.$$

在初始点，两误差为−1、2，梯度为$(-3,-1,2)$。同一点计算梯度后同时更新，参数变成$(1.3, 1.1, -1.2)$，预测为1.43、−1.56；损失为$\frac{1}{2}(0.57^2 + 1.44^2) = 1.19925$。固定$\phi$只更新头时，预测为1.1、−1.2，损失$\frac{1}{2}(0.9^2 + 1.8^2) = 2.025$。标签符号相反也未必梯度冲突：这里它们通过不同符号的头，对共享参数贡献恰好都为负。

仅用B样本时，梯度为$(-2, 0, 2)$；一次更新后参数为$(1.2, 1, -1.2)$。A头仍为1，但A表示已变成1.2，因而A输出由1变为1.2。冻结头不等于冻结行为；本例A误差反而下降，也不能把一切输出变化叫遗忘。实际优化器还可能有动量或权重衰减；要严格冻结，应明确排除该参数的更新。

若新输入编码器位于冻结主干前，仍需要主干对输入的导数。举例$u = \psi c$，主干固定为$z = 2u$，输出为z，$c = 1, \psi = 1$、目标3，半平方损失梯度是$(2-3)\times 2 = -2$。学习率0.1使ψ变为1.2，输出2.4，损失从0.5降到0.18。固定主干系数2仍然提供导数2；用no_grad包住整段主干会丢掉这条训练路径。

冲突例目标为$L_p(\theta) = \frac{1}{2}[p(\theta-1)^2 + (1-p)(\theta+1)^2]$，导数为$\theta + 1 - 2p$。在$p = 0.9, \theta = 0$处为−0.8，更新得0.08。两身体平方误差为0.8464、1.1664，总损失降到0.4392。继续优化也不能让两者同时完美：最优$\theta^\star = 2p - 1 = 0.8$，最小损失0.18，而B平方误差为3.24。

平衡采样只把最优折中改成0，不能让同一标量同时等于两个标签。加入部署可得的身体条件且模型能使用它，或建立专用头，才可消除本例的不可区分冲突；已知坐标正负差异可以先统一语义。真实身体需要不同控制时不能强行抹平标签。负迁移仍应和匹配数据、能力与预算的基线比较，不能由一次梯度或总体均值单独判断。

题2.4-4

Octo适配先建立目标数据，再修改输入和输出，最后合并权重并训练。固定示例读取top相机、本体state和语言，构造50步动作块；删除腕相机入口，新增本体编码；改用50×14输出的L1动作头。新结构先初始化，路径与形状均匹配的参数再从预训练模型复制；新位置编码和没有对应旧值的新头参数保留初始化。复制回答初始化，冻结回答哪些参数更新，旧能力保留则需要重新评价。

加载成功只检查数组兼容。例如两列从(x, y)改为(y, x)，形状不变，含义已变；相机、单位、控制周期和统计变化同样不会由形状检查自动发现。固定示例中，冻结函数按完整点分隔参数路径匹配模式；裸的BlockTransformer_0不等于通配模式octo_transformer.*。因此应检查实际参数分区和一次更新前后数组，不能仅凭开关名字认定已冻结。本书只核对固定源码和构造路径，没有运行作者JAX参数树。

实验先按物理合同检查完整动作块的正反转换，再测跟踪、限位与时间，最后测真实任务。目标数据按整回合划分训练、验证、测试，统计只来自训练；泛化测试按宣称留出身体、对象或任务组合。已见身体的新任务、新身体有示范适配、无需参数适配的直接部署分开报告。新随机动作头尚未学习，不构成可用的直接部署基线；没有目标标签也不等于没有使用目标域图像或统计。

在同一目标示范和评价条件下，比较从头训练、预训练只训新头、新输入 / 输出适配、扩大主干更新范围，并加入合适的单身体基线。比较表示迁移效果时，任务应落在各身体共同可达的范围内，夹爪宽度等目标也应可执行；各身体额外能够覆盖的任务另行报告，区分能力范围差别与学习效果。报告训练与调参预算、目标示范成本、额外源回放和实际计算。看每身体任务成功、完成时间、约束违反与失败类型；标准化动作损失只作训练诊断，不能直接跨不同单位和维度比较。

源能力评价要保留原输入、源头、统计和解码。可以一直保留源分支，也可以按预定规则把适配后的共享参数放回保存的源结构，在源任务同一测试条件上比较适配前后。删除源输入和源头后，仅给新模型一个“源任务”名称并不能完成遗忘测试。接口往返正确、目标收益、源能力变化与部署成本应各有证据，不合成一个平均分掩盖差异。

2.5 异质联合训练与语言条件的可控性

2.5.1 语言监督怎样排除视觉捷径

若训练集中红杯永远在左边，“拿红杯”的所有动作都向左，忽略文字、只按位置行动也能取得低损失。损失并不知道模型内部是否使用了语言。要让“文字改变动作”成为训练必须学会的关系，数据需要包含同一视觉条件下不同指令对应不同标签，也需要包含同一指令下目标换位置的样本。

场景	指令	需要的目标选择
红左、蓝右	拿红杯	左侧
红左、蓝右	拿蓝杯	右侧
蓝左、红右	拿红杯	右侧
蓝左、红右	拿蓝杯	左侧

这个小表不是完整的操作数据，而是展示哪些变化使视觉与文字的作用能被分开。模型先编码图像与指令，再通过拼接、交叉注意力或条件调制让动作预测读取两者。动作标签使“这条指令应选择哪个对象”进入训练目标；夹爪怎样靠近、如何接触，则还需要连续机器人轨迹提供。

验证时沿同样关系改变条件：保留场景、换指令；保留指令、换目标位置；加入同类干扰物。看选择是否随正确条件改变，再看抓取是否成功。预训练图文知识可以帮助识别颜色和物体，但动作可控性仍要靠这种条件—行为对应检验。CS285 目标条件模仿学习、LeRobot 语言标签与样本组织

2.5.2 视觉语言预训练提供了什么，没提供什么

VLM（Vision-Language Model，视觉语言模型）根据图像与文字进行匹配、问答或生成。预训练数据可能来自图文配对、描述、问答等任务。标签是文本或配对关系，不是机器人力矩。

对比学习让对应图文表示接近，同时与某些非对应样本区分。负样本选择决定网络需要区分什么：若文字总是简单类别，模型未必学会把手朝向；若背景强烈相关，也可能学背景捷径。

生成式视觉语言训练让模型在图像条件下预测文字 token。预测句子“蓝杯在左侧”能支持语义信息，但不能单独确定机器人系下的厘米距离、夹爪宽度或接触结果。

迁移到动作学习，预训练表示可以降低从少量示范学语义的负担；动作监督再学习这些条件如何映射到身体行为。要验证迁移贡献，可控制模型容量与动作数据，比较不同初始化、冻结和适配方式，而不是只比较有无大模型名称。

精细操作可能需要更高空间分辨率、本体状态或触觉。互联网语义知识有价值，其信息边界也应明确：知道物体用途与会执行该用途，是不同证据。

2.5.3 VLA 预训练的混合目标与数据配方

VLA 可以从预训练 VLM 出发，用机器人轨迹学习动作；也可以在多任务训练中共同使用语言视觉任务和动作任务。每种任务有对应输入、答案和有效输出掩码。

一个教学混合目标是 $L = \lambda_A L_A + \lambda_V L_V$：动作损失约束行动，视觉语言损失保留或学习语义能力。λ 表示训练方案对两类目标的权重。不同样本数量、损失尺度与采样频率都会影响实际贡献，不能只看 λ 数值。

输入机器人图像、指令和状态时，动作 token 或连续动作头产生动作监督；输入图文问答时，输出文字。共享主干可以迁移信息，也可能产生梯度冲突或让常见任务支配少见动作。

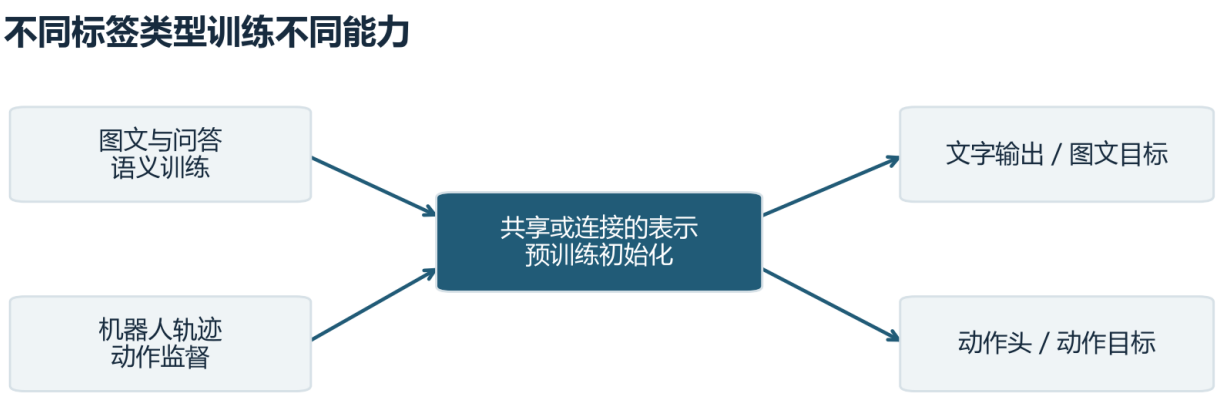


图2.3：标签类型决定训练约束。预训练提供初始化，动作数据接通执行语义，微调适配目标条件。

机器人数据多样性包括任务、对象、环境、视角、机器人和失败恢复。数量增长与覆盖扩大不是同义词。重复同一场景一万次，不能替代未见场景的有效变化。

Open X-Embodiment 提供多机器人数据与策略研究入口，支持研究跨数据与跨身体迁移，但需要核对各数据源的动作和传感器语义。Open X-Embodiment。

2.5.4 混合训练改变的是模型被要求兼顾的风险

互联网图文样本、机器人示范和人类视频的标签类型不同。设训练风险为 $L = \lambda_1 L_{text} + \lambda_2 L_{action} + \lambda_3 L_{future}$，权重控制各目标在总优化中的贡献，但实际梯度还取决于抽样频率、序列长度、有效标签数和损失尺度。三个权重相等，并不意味着三类任务得到相同学习资源。

若语言目标主导，动作头可能学不充分；若动作数据窄且完整微调很强，语义能力可能退化。解决路径包括调节采样、限制更新模块、分阶段训练、联合评测和冲突诊断。判断语言是否被用到，应该设置同场景异指令、同指令异目标以及语言缺失的对照；仅看模型能输出流畅解释，不足以评价动作可控性。

2.5.5 同样写等权，模型实际在学谁

混合训练可以明确写成 $L = \sum_j p_j \lambda_j \mathbb{E}_{D_j}[\ell_j]$。$p_j$ 是训练时抽中任务 j 的概率，λ_j 是该任务损失的显式权重，ℓ_j 还需要说明是按 token、动作标量还是整条序列平均。数据集大，只表示它有更多候选样本；如果先等概率选任务，数据量并不自动改变 p_j。

例如每次以一半概率选语言或动作，语言样本有100个有效 token，动作样本有10个有效标量。若两类各自求有效元素均值，平均单元素损失都为1，那么两类样本的损失都为1；若直接求和，数值就变为100和10。这一差异来自归一化，而不是“语言天生应该更重要”。即使均值都为1，进入共享参数的梯度方向与大小也可能不同，需要按目标检查。理解这三层权重，才能解释增加某类数据究竟改变了训练覆盖，还是改变了优化侧重。

2.5.6 练习题

题2.5-1 多任务训练应怎样区分采样概率、任务内损失归一化和显式任务权重？若各任务等概率采样，且各任务按有效动作标量求均值，样本总量或动作维度多是否仍会自动获得更大的权重？怎样通过对照实验判断任务间的不平衡来自哪里？

2.5.7 参考答案

题2.5-1答案 先把 L_i 定义为任务 i 内有效动作标量的平均损失。若每次以概率 p_i 抽取任务，再给该样本乘显式权重 λ_i，实际期望目标为

$$J = \sum_i p_i \lambda_i \, \mathbb{E}[L_i].$$

p_i 决定任务出现多频繁；L_i 的归一化决定一条样本的各元素如何合并；λ_i 决定已经抽到的样本影响多强。三者分别作用，不能把其中一项隐含进“平均损失”四个字里。

按本题设定，任务等概率、各任务按有效标量求均值且权重相同，样本总量或动作维度较大不会自动得到更大的显式系数。若每次都分别采齐各任务，再把各任务均值相加，则目标为 $\sum_i \lambda_i \mathbb{E}[L_i]$；与等概率任务抽样只相差任务数量这个整体尺度，但必须在比较时保持损失总尺度或更新幅度一致。

从拼接数据集中均匀抽样会产生 $p_i = N_i / \sum_j N_j$，此时样本多的任务出现更频繁；把动作误差求和而不取有效平均，又会使维度多的任务通常贡献更大的数值。即使平均后系数相同，单位、噪声和共享网络中的梯度大小或方向仍可能不同。

对照时先固定 L_i 与 λ_i，只改变任务采样比例；再固定采样与归一化，只改变权重。记录任务出现次数、各任务闭环表现以及共享层的梯度。这样才能分辨数据覆盖的改善、任务关注比例的改变与损失尺度的影响，不能仅凭总体损失下降判断配方更好。

进一步阅读：PyTorch：MSELoss，mean 与 sum 的归一化定义、PyTorch：WeightedRandomSampler，样本权重与抽样概率。

2.6 从人类视频、逆向推断和奖励中学习

人类视频丰富地记录物体、场景与行为变化，但通常缺少机器人动作和电机命令。利用它可以学习表示、推断可能动作，或推断行为偏好。这三项任务使用不同监督，下面分别解释它们怎样接到机器人学习。

2.6.1 视频提供哪些证据

视频能直接提供外观、时序、物体变化和部分接触线索。它通常不能唯一决定机械臂关节角、力矩或夹爪受力：同一物体移动可以由不同身体、不同抓法和不同控制产生。相机运动还可能被误当成物体运动。利用视频预训练表示时，目标可以是时间一致、未来特征预测或动作阶段识别；这不等于已经学到了可执行策略。

将人类手轨迹映射为机器人末端目标，需要人体或手部估计、坐标对齐、接触推断、可达性检查与机器人重定向。手掌能够完成的姿态未必在机器人工作空间内；人类视觉成功并不保证夹爪几何允许相同接触。适合的使用方式是把视频作为语义和时序**先验**，再用机器人数据**校准**动作接口与物理约束。

2.6.2 逆动力学把结果转成候选动作

前向模型预测 $p(s_{t+1} \mid s_t, a_t)$，逆动力学模型预测 $p(a_t \mid s_t, s_{t+1})$。后者可以用机器人自身轨迹监督：输入相邻状态或图像特征，标签为实际执行动作。训练完成后，对无动作视频产生伪动作，或者在目标状态已知时提出动作候选。

这依赖状态表示可对齐以及转移可解释。假设小车在一个控制周期后向右移动 2 厘米，可能来自不同速度、延迟和此前动量；仅看位置不能唯一推断当前命令。需要加入历史或速度，并在必要时预测多峰分布。若视频来自人类，逆模型在机器人数据上的正确性也不保证迁移后的伪标签正确。可以先在留出机器人视频上遮掉动作标签验证，再测试跨域伪标签的误差和下游贡献。

2.6.3 从轨迹概率推导奖励学习的更新方向

逆强化学习（Inverse Reinforcement Learning，IRL）从示范寻找能解释行为的奖励，再用奖励求策略。它面对的困难是：同一行为常能由多种奖励解释。在等长、同结束规则的候选中，给每步奖励加相同常数不会改变路线排序；其他某些奖励变化也可能保留同一策略。因此通常要规定奖励的形式和尺度，检验它在新选择上是否有用。

先考虑已知确定动力学中的有限候选路线：选定路线后，结果也被确定。最大熵建模分配 $p_\psi(\tau) = p_0(\tau) \exp R_\psi(\tau) / Z_\psi$。$p_0$ 是基础轨迹权重，R_ψ 表达偏好，Z_ψ 负责把候选概率归一化。若只提高专家奖励而省去 Z_ψ，抬高所有奖励也能降低这种不完整目标，模型却没有学出路线间的偏好。随机动力学还要保留不能由策略任意选择的转移结果，不能把任意倾斜环境转移的轨迹概率都解释为可执行策略。

用一个独立构造的两轨迹例子看清更新。基础概率相等，轨迹A的特征 $f=1$，轨迹B为0，奖励设为 $R_w=wf$。于是 $p_w(A)=e^w/(1+e^w)$。八条专家示范中六条是A，平均负对数似然为

$$L(w)=-0.75w+\log(1+e^w),\qquad \frac{dL}{dw}=p_w(A)-0.75.$$

从 $w=0$ 开始，模型认为A占一半，导数为 -0.25。梯度下降增大 w，让A更常出现；若模型已经把A概率推到0.9，导数转正，更新会降低 w。这里奖励权重由“模型预测的行为比例与专家比例之差”调整，而不是给每条专家轨迹随意加一个大分数。最优时 $p_w(A)=0.75$，得到 $w=\log 3$。

线性特征奖励的多维版本比较模型与专家的平均特征。真正机器人问题不能枚举全部轨迹，需借助规划、采样或策略学习估计模型一侧的平均。奖励网络学好后仍需训练或规划动作；是否能迁移到新环境，还取决于示范中的偏好是否被正确识别。最大熵 IRL 原始推导

2.6.4 判别器提供学习信号，策略通过交互改变访问分布

生成对抗模仿学习（Generative Adversarial Imitation Learning，GAIL）把专家和学习者访问过的状态—动作对交给一个分类网络。约定 $D_\psi(s,a)$ 输出“来自专家”的概率，训练标签由数据来源直接获得：专家样本标1，学习者样本标0。

$$L_D=-\mathbb{E}_E\log D_\psi(s,a)\qquad -\mathbb{E}_\pi\log(1-D_\psi(s,a)).$$

输入不是整段文字任务的对错判断，而是具体状态与动作的表示。分类器当前认为一条学习者行为很像专家时，例如 $D=0.8$，奖励 $-\log(1-D)$ 约为1.61；认为不太像时，例如 $D=0.2$，奖励约为0.22。这是与上述类别约定匹配的一种奖励写法。策略算法利用这些奖励与实际状态转移改进动作，随后产生新轨迹，判别器再学习区分新的两类样本。

整条数据流是：采专家样本与当前策略轨迹，更新判别器；固定本轮判别器，算策略轨迹奖励，更新策略；再执行新策略。判别器损失更新 ψ，策略损失更新策略参数，两者不是同一份动作回归标签。判别器需要不断重训，因为学习者行为也在改变。imitation 作者实现教程

“访问分布”可理解为反复执行后，哪些状态和动作经常出现。它还受环境转移影响：策略不能只在一张静态图片上复制动作，必须通过前面的行动把自己带到类似专家的状态。标准状态—动作 GAIL 需要专家动作；只有人类视频时，要另外设计观测或转移层面的监督。背景纹理或身体外形若暴露来源，判别器可能学到这些捷径，因此最后应以任务成功和可比较的任务状态检验模仿效果。GAIL 原论文

2.6.5 练习题

题2.6-1 只观察人类“把杯子放入柜子”的视频，为什么不能直接把帧间运动当成机器人动作标签？逆动力学模型能补什么，又依赖什么？

2.6.6 参考答案

题2.6-1答案 帧间像素运动由物体、人体和相机共同产生，通常没有直接给出三维接触状态，也没有说明机器人控制命令。人的手指轨迹无法原样映射到不同关节结构的机械臂；同一可见运动还可能需要不同力量，例如空杯与满杯抬起相似，但负载不同。因此视频能表达目标、顺序和可见变化，不能直接作为机器人力矩或关节命令标签。

逆动力学模型学习“给定前后状态，哪种动作可能导致这一变化”，例如由 s_t、s_{t+1}预测 a_t。它需要带真实动作标签的机器人转移数据来训练，且观测必须含有足够的运动和接触信息。有了模型，可以从适合其机器人和观测条件的无动作轨迹估计伪标签；若直接输入人类视频，还需跨身体、坐标与视觉域的映射。

这个推断也可能不是唯一的：多种动作可产生近似相同后果，遮挡与控制延迟会进一步增加歧义。应在有标签的机器人测试转移上测预测误差，并真正执行伪标签检验可行性，再决定它能提供动作监督，还是只能提供更高层目标监督。

2.7 模仿学习实践流程与失败诊断

2.7.1 放杯策略的训练与评价流程

先限定任务：桌面静态、固定夹爪、指定相机、杯子范围与托盘位置变化。定义成功为正确杯子稳定放入托盘，而非夹爪移动到某坐标。采集成功与恢复示范，记录原始时间、动作语义和控制配置。

数据按回合和场景划分，训练统计只用训练集。样本为当前或历史条件与未来 H 步动作；尾部有有效掩码。先在少量数据上检查图像、语言和动作方向，确认标签可被正确回放。

建立回归或短**动作块 BC** 基线，再根据多模态与协调需求尝试 **ACT**、**扩散**或 Flow。不要同时改变全部数据、架构和控制周期，否则很难知道哪项起作用。

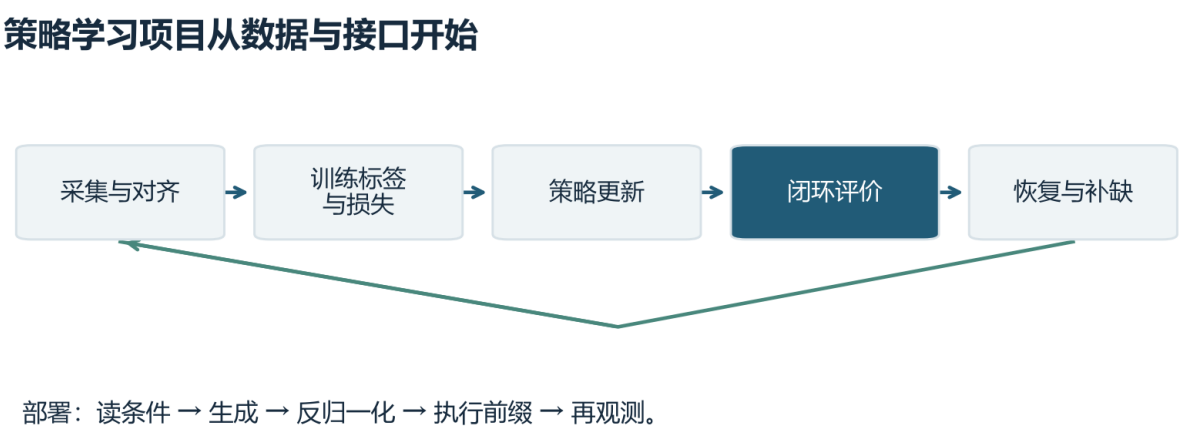


图2.4：先检查数据与接口，再训练和闭环评价；失败分析决定补数据、改表示还是修执行链。

一次训练：读取条件和有效标签；编码观测；生成动作预测或构造噪声/路径监督；计算对应损失；更新可训练参数；周期性保存与验证。部署：读取当前条件；生成动作块；反归一化；检查约束；执行前缀；用新观测继续。

2.7.2 评价必须区分模仿、泛化与任务成功

离线误差检验预测与示范是否一致，闭环成功检验行动后任务是否完成。多模态示范中，预测不同于某条标签，也可能是有效动作；反过来，误差小也可能累积成失败。

泛化可沿对象、位置、背景、视角、语言、任务组合和机器人变化分别测试。训练里见过相同对象不同角度，不应自动称为新对象泛化。测试条件需要按主张设计。

LIBERO 等基准提供受控任务与迁移评价入口，但仿真与真实机器人存在条件差异。一个基准上的高成功率不能代表所有任务。LIBERO。

比较生成策略时，报告采样次数、推理时间、执行前缀和控制频率。比较预训练时，报告预训练数据与额外信息；比较微调时，报告目标示范数、更新参数和总计算。公平不总意味着所有模型参数相同，但差异必须足以让读者解释收益来源。

研究的机制证据可以包括语言干扰实验、动作模式选择、偏离轨迹后的恢复率和跨身体负迁移。再用整体任务结果说明实际作用，避免以模型热力图或单个视频代替验证。

2.7.3 从失败症状回到建模选择

症状	可能缺口	可验证的改进
总往两个绕行方向中间走	单峰回归平均、多模态条件缺失	比较多模态策略或增加区分信息
初始正确、偏一点后无法恢复	执行分布缺乏示范覆盖	收恢复标签，测试偏移强度曲线
换指令不改动作	语言捷径或标签关联不足	同场景反事实指令测试
离线好、运行幅度离谱	归一化、坐标或控制语义不一致	手算一个输出并核对设备执行
生成轨迹跳变	逐步模式不一致或块集成冲突	比较块级一致性与重规划
新机器人明显退化	身体条件与接口未对齐	分身体报告，检查动作有效掩码

读新论文时沿数据流提问，而不是先寻找熟悉缩写。尤其检查训练时是否读取答案、部署如何替代这部分信息，以及真实行动得到的新观测如何再次进入策略。

2.7.4 从能跑通到能定位失败

先用很少量固定数据过拟合一个短动作任务，验证输入、标签、掩码和反归一化，再扩大数据。若连这一步也失败，应检查接口、优化和模型容量；若训练拟合很好而新场景失败，再检查覆盖与泛化；若开放环路预测良好而闭环失败，检查延迟、恢复数据和接触条件。分层排查能减少盲目更换架构。

最低项目记录包含数据版本、机器人接口、归一化统计、采样配方、可训练模块、有效批次、训练步数和评价协议。一个模型文件不等于可复现策略，缺少相机和动作定义时，权重相同也可能产生不同物理行为。

2.7.5 练习题

题2.7-1 行为克隆在训练环境成功、新相机环境失败。按输入、标签、访问状态三个层次提出排查与对照，而非立即更换模型。

2.7.6 参考答案

题2.7-1答案 先查输入：新相机是否改变视野、裁剪、颜色、内外参或图像时间戳？把同一机器人状态从旧新两机同时拍摄，输入模型但不执行，比较对象特征和动作输出。若恢复旧预处理或使用双相机对齐后输出恢复，先修输入接口和相机覆盖。

再查标签：动作在什么坐标系、单位与时间间隔中定义？若动作在相机坐标中表示，相机移动后必须转换；若用基座坐标，也要核对相机图像与命令的同步。用专家执行的对应画面测离线误差，并检查同一物理目标转换后的动作是否一致，避免误把标定错误当视觉泛化不足。

最后查访问状态：让模型在新相机下闭环执行，记录首次偏离以及之后的状态。比较“专家带着模型经过正常状态”和“模型自行执行”的误差；若前者尚好、后者迅速恶化，可加入新环境中的纠错状态并检验恢复率。按输入、标签、闭环状态逐层控制变量，才能区分视觉域变化、监督错位和误差累积，而非用更大模型同时掩盖三个问题。

第3章：强化学习与 VLA 后训练

示范告诉模型某种行为怎样做，行动后果则帮助判断做得是否更好。本章围绕长期回报，将价值估计、策略更新与 VLA 后训练逐项连接。

3.1 序列决策、状态假设与部分可观测性

3.1.1 先明确强化学习究竟多学习了什么

模仿学习主要比较模型动作与示范动作；强化学习把后续结果作为改进依据。它通常不需要专家为每个状态写正确动作，却需要奖励、交互或离线轨迹，并承担长期结果估计的误差。**策略模型**、**价值网络**、奖励函数和环境各有职责：策略选择动作，价值预测未来回报，奖励定义即时评价，环境产生真实后果。

3.1.2 强化学习公式首先是一套建模选择

为什么优化对象不是单个正确动作

监督学习通常有现成动作标签。强化学习通常只有某种评价，因此先要明确我们希望整段行为达到什么结果。一次从开始到结束的交互叫作一个回合，episode；其中按时间记录的观测、动作和奖励组成一条轨迹，trajectory。

把“做完任务得到更多分”写成一个目标，可以得到

$$J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta} \left[\sum_{t=0}^{T-1} \gamma^t r_t \right].$$

这句话的完整意思是：让参数为 θ 的策略反复完成任务；每次把沿途奖励按时间折扣后加起来；选择能让这种总分平均更高的参数。T 是本回合的步数，r_t 是第 t 个动作之后得到的奖励，γ 是折扣系数。$\tau \sim \pi_\theta$ 是简写：轨迹由策略和环境共同产生，还受到初始条件、传感器和物理随机性影响。

目标取轨迹总回报的期望。策略参数改变后，当前动作和后续访问状态都会改变，因此数据分布也随策略变化。强化学习既要判断行动后果，又要处理这种由自身行为造成的分布变化。

同一个目标可以有不同算法

目标函数说明想达到什么，算法说明怎样靠有限数据接近它。我们不能直接枚举真实机器人的全部可能未来，因此必须采样、估计和近似。

例如，我们可以等每条轨迹结束以后，用实测总分更新动作概率；也可以训练一个网络预估未来总分，提前给动作反馈。前者不依赖**价值模型**但往往噪声大，后者利用预测但引入估计误差。很多算法差别来自这种信息与误差的取舍，而不是任务目标完全不同。

“损失”也不一定就是任务目标本身。**近端策略优化**（Proximal Policy Optimization，PPO）的训练损失是用于更新策略的代理目标。例如，PPO-Clip 通过裁剪部分更新激励，抑制对旧样本的过度利用；价值网络的平方误差用来拟合未来回报；模仿损失让动作留在数据支持的范围。只有把这些损失各自服务的对象分清，才不会误以为所有公式都在独立定义任务。

四种对象，不要混在一起

对象	示例	它的作用
任务目标	方块最终进入左盒，且动作满足约束	决定什么行为算好
数学量	当前策略从某状态开始的期望回报	明确“好”怎样被度量
估计器	价值网络、采样回报、优势估计	用有限信息估计数学量
优化过程	梯度更新、裁剪、目标网络更新	利用估计值改变网络

真实的 V^{π} 是按定义得到的期望；网络 V_{ϕ} 是对它的近似。二者都被口头称作“V”，但理论对象和计算对象不相同。训练时通常只能访问后者。

A^{π} 定义动作相对当前策略平均水平的优势；**广义优势估计**（Generalized Advantage Estimation，GAE）则用于从有限轨迹中估计这个量。它利用奖励和价值预测，把当前及后续各步的时序差分误差按衰减权重累加，得到用于策略更新的优势估计。定义告诉我们想评价什么，估计器告诉我们有限数据下怎样得到用于更新的数值。

平均回报与任务约束

平均回报反映总体收益。若任务另有碰撞概率或受力上限，应把这些条件作为约束或风险目标，并独立验证。奖励中的碰撞惩罚会影响取舍，但有限平均惩罚仍可能允许某些高回报的危险行为。

任务分布也属于建模。只在固定相机、固定方块位置上采样，优化的是这种分布下的平均表现；新背景和新物体不属于同一个保证范围。训练成功、测试成功、真机成功分别需要证据。更大的模型或更高训练奖励不会自动补齐这些证据。

3.1.3 状态、观测和历史，为什么要分开

状态是预测后果所需的信息，观测是传感器真正给的信息

方块位置、夹爪速度、接触力、物体是否滑动，都可能影响下一个动作的结果。理想化地，把足以预测后续变化的信息叫作状态，state。相机图像和关节读数则是观测，observation。相机没看见的接触力，不会因为图像很清晰就消失。

如果给定当前状态和动作，未来变化不再需要额外依赖更早历史，就满足马尔可夫性质。它的用途是让复杂的历史依赖问题能够写成“当前状态—动作—下一状态”的递推问题。马尔可夫决策过程，Markov Decision Process，缩写 **MDP**，就是包含状态、动作、转移、奖励等对象的一种形式化模型。

马尔可夫性质要求状态已经概括了与未来有关的历史信息。例如在一个简化推杯任务中，物体位置、速度和夹爪状态可能足以预测下一步；只保留位置却遗漏动量，就可能不够。同一位置上的静止杯子和正在滑动的杯子，执行同一个动作也会产生不同后果。状态设计是否充分，决定后面的一步价值递推能否合理使用。

为什么一张相同的图片可能需要不同动作

两张图都显示夹爪接近方块。但第一种情况夹爪正稳定下降，第二种情况刚撞到桌面后反弹。静态图像看起来相近，合适动作却不同。只用图片作为输入时，策略和价值网络会把不同情形压到同一个输入上，出现难以解释的平均预测。

部分可观测马尔可夫决策过程（Partially Observable Markov Decision Process，POMDP）描述“真实状态未完全可见”的问题。严格地说，决策可以依赖对隐藏状态的信念，即根据历史推断出的状态概率分布。实际 **VLA** 往往用多帧图像、动作历史、循环网络或**注意力**模型中的历史表示近似这一信息。

后文的 $V(x)$ 使用算法实际可获得的决策输入 x，可以是完整状态，也可以是历史编码。只用当前观测时，价值会对未观测的信息取条件平均；加入速度或历史则可能区分原来混在一起的未来。

语言指令是任务条件，不是装饰

同样看到方块和两个盒子，“放左盒”与“放右盒”的**动作价值**不同。若价值网络只看图像、不看指令，它会把两个目标下的回报混在一起。策略有语言条件，而价值网络没有相应条件，训练信号可能自相矛盾。

因此一个常用决策输入可以是 $x_t = (o_t, \ell, h_t)$：当前观测、任务指令、历史表示。这只是输入设计示例，不要求每个模型都显式拼接三个变量。真正原则是：影响奖励与后续动作的条件，应被相应模块获得。

训练时可以多看信息，但要知道谁在使用它

仿真中，价值网络可能得到精确物体坐标，策略只得到相机图像。这叫不对称的 Actor-Critic 设计：评价者信息更多，执行者仍使用部署可得输入。它有时能让价值训练更稳定。

部署时若要用价值网络筛动作，其输入必须可获得。特权教师监督学生时，也应在学生实际访问的输入上提供可用标签，避免要求学生推断不可观察信息。

3.1.4 信念状态怎样把观测历史变成决策信息

信念状态（Belief State）不是模型“相信自己”的分数，而是对当前隐藏物理状态的概率分布。设图像无法判断杯子是否夹牢，简化成“夹牢、松动”两种状态，当前各有一半概率。触觉传来一个稳定接触信号；假设夹牢时出现此信号的概率为0.9，松动时为0.2，那么更新后的夹牢概率是 $0.9 \times 0.5 / (0.9 \times 0.5 + 0.2 \times 0.5) \approx 0.818$。这个观测没有把未知状态变成确定真值，却改变了后续搬运和试探动作应依据的信息。

动作也会改变信念：上一时刻夹牢不意味着快速摆动后仍夹牢。先用状态转移预测现在的可能状态，再用新观测修正它。有限状态下可写成

$$b_{t+1}(s') \propto p(o_{t+1} \mid s') \sum_{s} P(s' \mid s, a_t) b_t(s).$$

求和部分是动作后的预测概率，观测似然再给这些预测重新分配权重，最后归一化。转移和观测模型正确时，这个分布概括了决策所需的历史；高维机器人通常难以精确维护它，因此使用滤波、世界模型后验或学习到的历史编码来近似。给网络更多帧提供了推断机会，仍要检查它是否保留了关键状态。
Cassandra 的 POMDP 教程与信念说明

3.1.5 练习题

题3.1-1 两张图像看起来相同，杯子一次静止、一次正在滑动。仅用当前图像的策略和 V 网络分别会遇到什么问题？怎样改善决策输入？

3.1.6 参考答案

题3.1-1答案 当前图像若无法区分静止杯与滑动杯，就不是足以决策的完整状态。当前图像策略只能对两个情形输出相同的动作分布，无法在滑动时提前拦截、静止时精确接近；它可能学到折中动作，却在两种情形都不理想。

V 网络也只能输出一个值。相同图像对应的真实成功前景不同，因此它拟合的是数据中这两种隐藏情形混合后的平均回报。若策略改变了静止/滑动情形的出现比例，这个平均值又会变化，价值误差可能传入优势估计。

可以加入相邻帧、时间间隔和历史动作，让模型从位置变化估计速度；也可使用递归状态、视频编码器或状态估计器，表示“依据目前历史，我认为杯子以何种速度运动”。历史不是越长越好，关键是包含区分隐藏状态的信息，并在训练部署保持同样可用。用“当前画面近似相同、运动历史不同”的配对测试，检查动作与价值是否分别作出合理变化。

3.2 奖励、折扣、动作块与长期信用

3.2.1 奖励与回报，怎样表达长期目的

奖励评价一个事件，回报评价后续一段行为

本书约定：执行 a_t，环境进入下一个情形后给出 r_t。奖励 reward 是这次反馈。回报 return 是从某时刻开始，把随后奖励按规则累计起来。

假设三步分别是抓取、搬运、放入，只有完成放入得到 10 分。奖励序列是 0、0、10。没有折扣时，从抓取开始的回报也是 10，而不是 0。这使得前面为成功铺路的动作有机会得到正向学习信号。

若采用折扣 $\gamma = 0.9$，从时刻 t 开始的回报写成

$$G_t = r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \cdots .$$

三步对应 $G_0 = 8.1$、$G_1 = 9$、$G_2 = 10$。早期状态离成功更远，所以同一笔终局奖励在早期被折得更小。这些是已经观察到的一条轨迹的累计结果，还不是所有可能未来的平均价值。

即时奖励为零，也可以有长期贡献

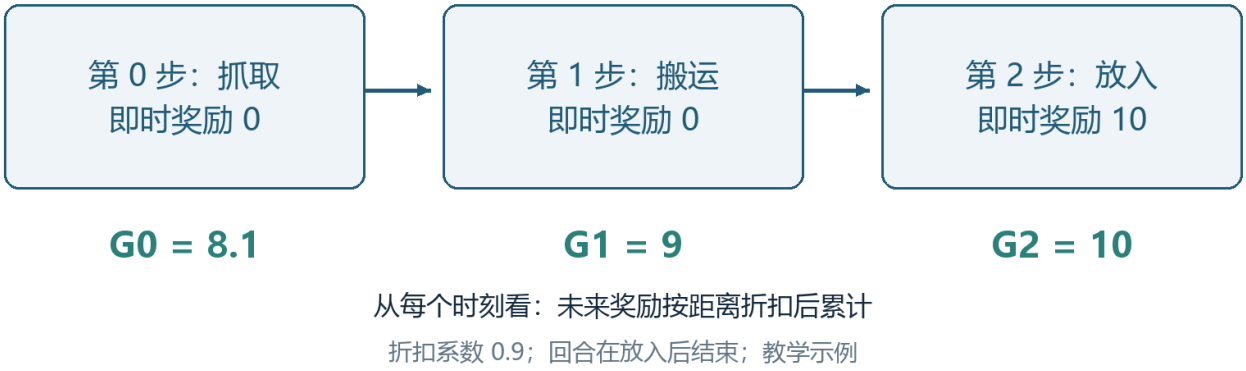


图3.1：同一条轨迹中的即时奖励和未来回报。

折扣并非只是方便收敛的技巧

折扣可以表达更偏好早点完成，也使无限时域中的回报在一定条件下有限。但选取不同折扣，就在改变优化目标。

成功都得 10 分时，$\gamma < 1$ 偏好早成功。若机器人为了快而采用更冒险的行为，不能把这种倾向全归因于实现错误。折扣可能让速度和成功概率发生权衡。真实系统还要考虑每个控制步对应多少秒；同样的 γ 在 5 Hz 和 50 Hz 控制频率下，隐含的时间偏好不同。

有限回合可以取 $\gamma = 1$。若奖励只在结束时给 0 或 1，且没有其他成本，期望回报可解释为成功概率。一旦加入时间折扣、动作惩罚、过程奖励，就通常不能再把价值数值直接叫作成功概率。

过程奖励增加信号，也可能改写任务

只有最后得分叫稀疏奖励。距离盒子更近得分、稳定抓住得分，是过程中的稠密奖励。它们帮助探索，但也规定了模型追求的中间目标。

如果每一步“方块在盒子附近”都得 1 分，机器人可能学会一直停在那里。如果只奖励夹爪靠近方块，它可能靠近而不抓。问题不是“奖励不够复杂”，而是中间指标与最终目的未必一致。

势函数塑形是一种有理论条件的辅助奖励设计

$$r'_t = r_t + \gamma\Phi(s_{t+1}) - \Phi(s_t).$$

$\Phi(s)$ 为状态指定一个潜力值，例如用到目标的负距离表达接近程度。把新增奖励沿轨迹折扣求和，中间项会相消，得到 $G'_0 = G_0 - \Phi(s_0) + \gamma^T\Phi(s_T)$。若有限回合的终止潜力统一为零，或无限时域中末项趋于零，同一起点下新增部分与行动选择无关，原来的策略排序便被保留。与此同时，每步仍能得到与进展相关的训练信号。这就是该设计兼顾反馈密度与原任务目标的原因。若终点潜力随结果任意变化，末项便可能改变排序，不能省略这个条件。势函数塑形。

奖励尺度也影响学习过程

把所有奖励乘以 100，某些理想问题的最优行为不变，但价值目标、优势尺度、损失平衡和数值稳定性会改变。若同时还有固定权重的熵奖励或参考策略约束，相对重要性也会改变。

因此奖励设计至少要分别检查三件事：它是否表达正确任务；它是否提供可学信号；它是否与优化器和其他损失的尺度协调。算法无法替代第一件事。

3.2.2 动作块为什么会改变时间与信用分配

预测八步，执行四步，再看一次环境

动作块，action chunk，是一次生成多个未来控制动作。它减少大模型调用次数，也让相邻动作由同一次计划共同生成，可能更连贯。

例如模型输出八步动作，只执行前四步，然后读取新图像重新规划。其余四步是未执行候选，不应把后面的实际奖励算作它们的执行结果。

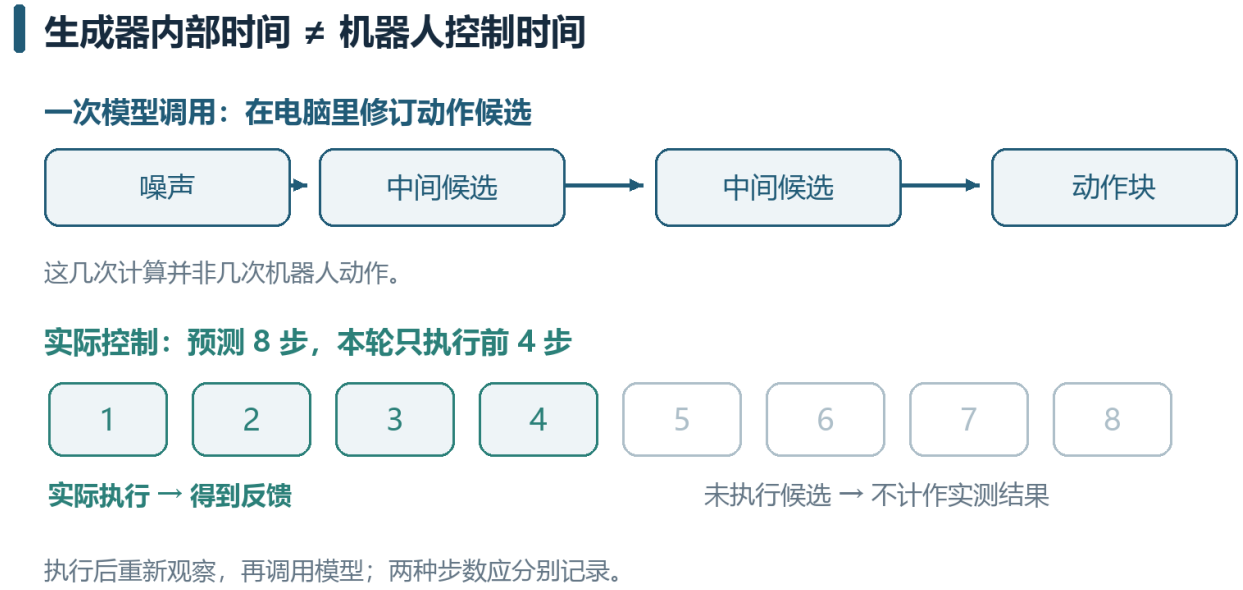


图3.2：生成进度与机器人控制时间是不同的轴。

这里有三种步数：机器人实际控制步，模型重新决策的次数，生成器内部的去噪或积分步。论文中只写“step”容易混淆，阅读时应辨认。

把四个控制步视为一次决策，需要同步改奖励

若原任务按每个控制步折扣 γ，一次宏观决策实际执行 k 步，其奖励应对应

$$R_{\text{block}} = r_0 + \gamma r_1 + \cdots + \gamma^{k-1} r_{k-1}, \qquad \Gamma = \gamma^k.$$

下一次决策的价值再乘 Γ。这保持了与原控制时间一致的回报定义。不能因为模型调用频率变低，就随意把每个动作块仍只折扣一次原 γ。

设四步奖励为 0、0、0、10，$\gamma = 0.9$。块奖励是 7.29。若四步后任务未结束，下一决策价值为 8，当前目标再加 $0.9^4 \times 8 = 5.2488$。若已真正结束，则不加这项。

实际执行步数可能少于计划：动作到第二步就成功或碰撞终止，应该按实际转移和任务边界处理，而不是仍当作完整执行四步。

块越长，不只是效率越高

长块减少计算调用，但也减少传感器反馈。搬运途中物体滑动时，下一次重规划来得更晚。块级回报给整段动作同一个方向，也使“哪一小步有问题”更难区分。

短块更容易纠错，但调用大模型频繁，训练样本更多且决策链更长。最适合的长度依赖控制频率、生成延迟、环境变化速度和反馈质量。需要报告实际执行长度和时延，而不是只报预测长度。

算概率时也要明确随机变量

模型生成完整八步，环境执行前四步。完整块的联合密度和前四步的边缘密度不是相同量。一般不能直接把未执行后四步对应的项随意删除，除非模型的概率分解和算法定义允许。

一种合理设计可以把完整生成过程作为内部随机决策，奖励由实际执行前缀产生；另一种设计直接定义执行前缀的策略。两者都需要清楚的数学对象，不能一边按完整块采样、一边默认自己计算了前缀的精确密度。

3.2.3 折扣连接评价偏好与决策时钟

若每步间隔为 Δt，同一个折扣因子在不同频率下对应不同物理时间偏好。可以用 $\gamma = e^{-\rho \Delta t}$ 表示按连续时间速率 ρ 折扣的教学形式。将控制频率从 10 Hz 改为 50 Hz 而保留每步折扣，不仅改变数据量，也改变较远未来在目标中的权重。

执行 k 步动作块后得到一次宏观转移，应积累块内奖励，并用 γ^k 对块后的价值折扣。如果接触导致提前终止，需要按真实执行步数处理。只把块末奖励当全部奖励，可能遗漏夹爪撞击或过程损失。动作块长度、奖励采样和算法时钟必须共同定义。

3.2.4 练习题

题3.2-1 奖励 [0,0,10]，γ=0.9。计算每步回报。若前两步合成一个动作块，块奖励和块折扣应怎样定义，才能保留相同目标?

题3.2-2 每步成功进度奖和势函数塑形 r′=r+γΦ(s′)−Φ(s) 有何区别？为何要关心终止状态的 Φ?

3.2.5 参考答案

题3.2-1答案 按 $G_t=r_t+\gamma r_{t+1}+\dots$，三步回报分别为

$$G_2 = 10, \qquad G_1 = 0 + 0.9(10) = 9, \qquad G_0 = 0 + 0.9(0) + 0.9^2(10) = 8.1.$$

若从原时刻 0 起把前两步合成一个块，块内奖励应为 $R_0=r_0+\gamma r_1=0$，跨块折扣应为 $\Gamma_0=\gamma^2=0.81$。下一块开始于原时刻 2，回报为 10，所以 $G_0=R_0+\Gamma_0 G_2=8.1$，原目标保持不变。

一般持续 k 个控制步的块，需要 $R=\Sigma_{j=0}^{k-1}\gamma^j r_{t+j}$、$\Gamma=\gamma^k$。若只是把两步奖励直接相加，并仍按 γ 跨块折扣，就把等待两步的后果当作只等待一步，改变了优化目标。控制步数可变时，每个转移要保存其真实持续长度。

题3.2-2答案 一般“每步进度奖”只是另加一个奖励函数，可能改变原来的最优行为。例如每次杯子进入靠近目标区域都给分，机器人可能反复进出区域获取分数，而不把杯子真正放稳。

势函数塑形使用相邻状态势值之差 $F=\gamma\Phi(s_{t+1})-\Phi(s_t)$。累积到终止时 T，有

$$\sum_{t=0}^{T-1}\gamma^t F_t = -\Phi(s_0)+\gamma^T\Phi(s_T).$$

中间项抵消，因此在边界项不影响动作选择的条件下，塑形只改变学习过程中的局部反馈，不改变原回报对应的策略排序。它可以更早告诉机器人是否接近目标，同时保留原任务。

若终止状态的 Φ 不为零，最后的 $\gamma^T\Phi(sT)$ 可能随到达状态或耗时变化，从而额外偏爱某些结束方式。有限回合常把真正终止状态势值设为零，或采用一致的吸收状态/终止修正。外部时间截断又不能直接视为任务终止，需要按剩余任务的价值边界处理。不是任何“距离越来越近”的奖励都自动具有这种不改变目标的性质。

3.3 价值、优势、贝尔曼关系与估计误差

3.3.1 V、Q、A 分别评价什么，怎样算出来

先有比较问题，才需要三个量

机器人处在“方块已经夹住”的情形。它可以稳定搬运，也可以快速甩过去，还可以过早松手。我们需要知道：这个情形总体有多好？选某个动作后有多好？这个动作比当前策略平常的选择好多少？

这三个问题分别对应**状态价值** V、**动作价值** Q、优势 A。它们不是三个必须同时训练的网络，而是三种数学量。不同算法用不同估计方法，可能只训练 V，也可能主要训练 Q，还可能不用单独的**价值网络**。

V：从这里按当前策略继续，平均能得到多少

$$V^\pi(x)=\mathbb{E}[G_t \mid x_t=x, \text{之后按 } \pi \text{ 行动}].$$

条件“按 π 继续”很重要。同一个抓取状态，熟练策略和容易松手的策略有不同预期回报。V 不是状态天然带有的绝对分数，而是状态与后续策略共同决定的量。

通常用价值网络 $V_\phi(x)$ 近似它。输入是图像、任务和必要历史等决策信息；输出是一个标量，单位与回报一致。网络可能共享 **VLA** 编码器，也可能独立。它不一定输出动作，也不一定控制机器人。

Q：先固定这个动作，再按当前策略继续

$$Q^\pi(x,a)=\mathbb{E}[G_t \mid x_t=x, a_t=a, \text{之后按 } \pi \text{ 行动}].$$

Q 与 V 的差别在于，Q 明确指定了第一步动作。后续仍按相应策略继续。因此 Q 的比较同时包含即时反馈和动作改变未来情形的效果。

训练 Q 网络时，输入通常是决策信息加一个候选动作，输出该动作的预期长期回报。也可以在某些条件下用真实轨迹回报、模型滚动或其他估计来近似 Q。没有试过的动作，网络只能凭泛化或模型假设预测；这类预测并不因写成 Q 就变成可靠事实。

A：这个动作比当前策略的平均选择好多少

$$A^{\pi}(x,a) = Q^{\pi}(x,a) - V^{\pi}(x).$$

优势的参照是“当前策略在同一情形下的平均表现”。它解决的是：不要只因为一个状态很容易成功，就把在那里采样的所有动作都当作好选择；也不要只因为状态很困难，就把有帮助的动作都当作坏选择。

假设三个候选动作的选择概率是 0.5、0.3、0.2，Q 值分别是 8、6、1。则

$$V^{\pi}(x) = 0.5 \times 8 + 0.3 \times 6 + 0.2 \times 1 = 6.$$

对应优势是 2、0、−5。第三个动作回报仍为正，但它比当前策略通常能达到的水平差，因此优势为负。第二个动作不是完全无用，只是没有超过这个比较基准。

Q 评价指定动作；A 与当前策略平均水平比较

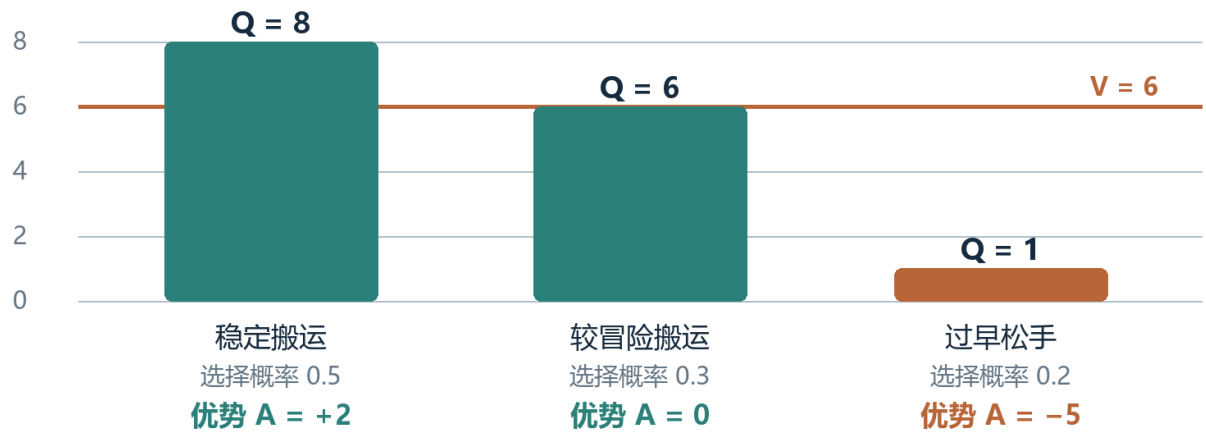


图3.3：Q 的绝对水平与相对当前策略的优势。

在精确估计下，按当前策略加权的优势平均为零：$0.5 \times 2 + 0.3 \times 0 + 0.2 \times (-5) = 0$。这是比较基准的结果，不要求实际一小批采样的优势恰好均值为零。

价值网络的训练标签从哪里来

最直观的方法是等任务结束，用实际回报当标签。回到三步奖励 0、0、10 的例子，折扣为 0.9。三个输入的标签分别是 8.1、9、10。若当前价值预测为 2、4、6，第一条的平方误差是 $(2-8.1)^2 = 37.21$。

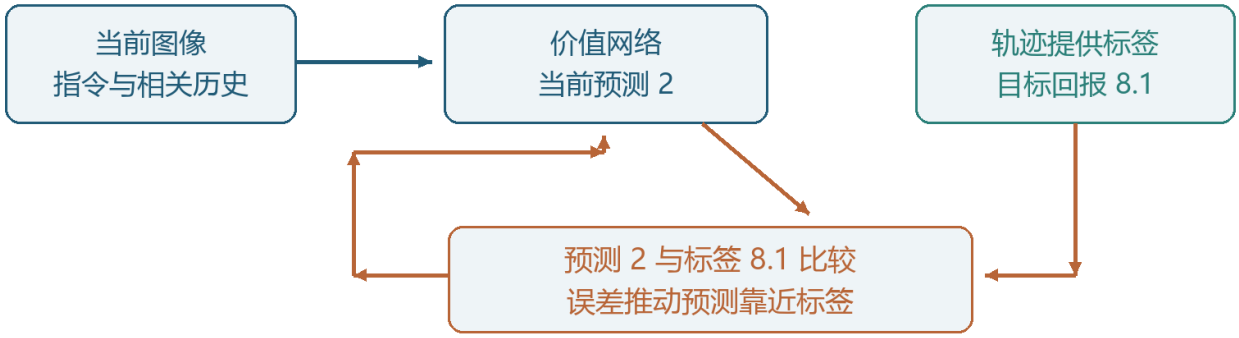
价值损失可以写成

$$L_V(\phi) = \frac{1}{N}\sum_i \big(V_\phi(x_i) - y_i\big)^2.$$

y_i 是从轨迹或递推方法得到的训练目标。平方误差让过低预测上升、过高预测下降。它的目的不是让模型动作更接近示范，而是让预测的长期回报更接近标签。

一次成功或失败是回报的一个样本。相同决策输入可能产生多种未来，价值网络对多条标签拟合，学习其条件平均。若后续行为来自不同策略，标签也对应不同后续选择；训练目标需要说明是在评价哪种行为。

价值网络：输入当前情况，预测后续回报



网络拟合的是未来累计反馈；不是把 8.1 当作动作输出

图3.4：价值预测与回报标签怎样形成训练信号。

为什么不直接训练一个优势网络

可以直接训练优势估计器，但常见 PPO 已经有一套更方便的标签来源：价值网络预测当前与下一时刻的回报，轨迹提供实际奖励，把两者组合便能得到时序差分（Temporal Difference，TD）或 GAE 优势。V 网络只需根据决策输入输出一个回报值，不必对每个连续候选动作单独评价。优势在这里是给策略更新用的权重，通常随采集批次重新计算，因此不必再维护第三个网络。

Q 型算法则训练 Q，把动作评价直接放进网络。究竟训练哪个量，取决于动作维度、数据复用方式和策略更新方法。不要把 V、Q、A 看成任何强化学习必须按顺序输出的三个模块。

3.3.2 为什么用“下一步价值”估计未来

等到任务结束很直接，却不总是好用

用完整实测回报作标签叫蒙特卡洛估计，Monte Carlo，缩写 MC。它不需要用网络预测尚未发生的后果，但得等待未来，而且成功与失败的随机性会传到早期动作上。长任务里，相同动作后的总分可能因为后续许多偶然因素变化很大。

另一条路线是：当前一步奖励已经知道，剩余未来用下一个状态的价值预测代替。这样把长问题拆成反复可学的短问题。这种用已有估计构造新估计目标的做法，称为**自举**（Bootstrapping）。它的代价是，你拿不完全准确的预测去监督另一个预测。

贝尔曼关系为什么成立

回报可以拆成“这一笔奖励，加上之后那段回报的折扣”。对未来取期望，便得到

$$V^{\pi}(x_t) \quad = \mathbb{E}_{\pi}[r_t + \gamma V^{\pi}(x_{t+1}) \mid x_t].$$

贝尔曼期望关系来自长期回报的逐步分解：当前奖励加上折扣后的下一价值。它提供自举训练的依据。若输入缺少必要状态或历史，这个递推只能作为近似，预测误差也会包含未观测信息的影响。

训练时只有一条采样转移，因此常把 $r_t + \gamma V_{\phi_{\text{old}}}(x_{t+1})$ 当作一个近似目标。将目标当作标签时通常不让梯度穿过它；否则网络可以通过改变“答案”来降低误差，失去预测目标应有的语义。具体实现可能采用当前网络的停止梯度版本，也可能用独立、缓慢更新的**目标网络**。

TD 误差在比较哪两种判断

时序差分（Temporal Difference，TD）根据相邻时刻的预测与实际奖励形成更新。常用的 TD 误差为

$$\delta_t = r_t + \gamma V_{\text{old}}(x_{t+1}) - V_{\text{old}}(x_t).$$

左边的名字虽然是“误差”，但右边是在比较两种对未来的判断：动作前预测的总回报，和动作后利用新反馈修正的总回报。如果后者更高，这个转移比原先预期好。

在精确价值、正确条件和按策略采样等条件下，给定当前输入和动作，一步 TD 误差的期望对应动作优势。实际网络不精确，因此单步 TD 不是神谕。它既含动作后果，也含环境随机性和**价值模型**误差。

一个完整 GAE 算例

广义优势估计，Generalized Advantage Estimation，缩写 **GAE**，把之后多个 TD 误差按衰减权重相加。它试图平衡“依赖短期价值预测的偏差”和“依赖长程实测结果的方差”

$$\hat{A}_t = \delta_t + \gamma\lambda\delta_{t+1} \qquad + (\gamma\lambda)^2\delta_{t+2} + \cdots.$$

λ 决定信号追溯多远。$\lambda = 0$ 只使用当前一步修正；λ 接近 1 时，更接近由长段实际结果判断。γ 主要属于任务回报定义，λ 属于估计器设计，二者作用不相同。

用三步奖励 0、0、10，$\gamma = 0.9$，$\lambda = 0.8$。本例旧价值预测为 2、4、6，真正结束后的价值为 0。这些是故意设置的不准确预测，不是上一章三个候选动作算例里的精确 V。

时刻与阶段	旧价值	本步奖励	TD 误差
0：抓取	2	0	0 + 0.9 × 4 − 2 = 1.6
1：搬运	4	0	0 + 0.9 × 6 − 4 = 1.4
2：放入并结束	6	10	10 + 0 − 6 = 4

从后往前算，衰减因子是 0.9 × 0.8 = 0.72。最后一步优势为 4；搬运阶段为 1.4 + 0.72 × 4 = 4.28；抓取阶段为 1.6 + 0.72 × 4.28 = 4.6816。

读懂这个计算，不是只会代入数字。它说明：最后成功比价值网络预测的更好，这个意外的好结果被逐步传回前面的动作；传得越远，权重越小。最后 TD 为 4 不代表“放入动作在所有可能轨迹中优势精确等于 4”，而是这次尝试给出的估计信号。

用于拟合价值的常见目标是 $\hat{y}_t = V_{\text{old}}(x_t) + \hat{A}_t$，本例为 6.6816、8.28、10。它们没有全部等于完整回报 8.1、9、10，因为 $\lambda < 1$ 时保留了短程自举预测的影响。取 $\lambda = 1$ 且回合完整、边界正确时，递推会得到 $G_t - V_{\text{old}}(x_t)$，目标回到实测回报。

优势标准化与回合边界为什么不能随意处理

把一批优势减均值、除标准差，常用来改善策略更新的尺度。它是一种优化处理，不是在重新定义真实动作价值。价值目标应使用原始尺度的优势；若把标准化后的优势再加到 V 上，标签单位就错了。

真正任务结束，后面没有本任务回报，因此末尾价值为零。若只因采集缓冲区满而暂停，任务仍会继续，末尾通常要保留价值预测。外部时间截断也需按任务定义判断。与此同时，GAE 递推不能跨到重置后另一个回合上。能否用末尾价值、能否继续递推，是两个不同问题。

3.3.3 GAE 的衰减权重从哪些估计组合出来

一步优势估计是 $\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t)$。若多看一步，就得到 $r_t + \gamma r_{t+1} + \gamma^2 V(s_{t+2}) - V(s_t)$；代入两个 TD 残差，恰好等于 $\delta_t + \gamma\delta_{t+1}$。一般的 n 步估计为

$$\hat{A}_t^{(n)} = \sum_{k=0}^{n-1} \gamma^k \delta_{t+k}.$$

这已经展示了取舍：n 小时较早用价值预测替代未来，n 大时读到更多真实奖励。GAE 再将不同长度的估计按几何权重混合。对无限展开、$0 \le \lambda < 1$ 的形式，

$$\hat{A}_t^{\text{GAE}} = (1-\lambda) \sum_{n=1}^{\infty} \lambda^{n-1} \hat{A}_t^{(n)} \qquad = \sum_{k=0}^{\infty} (\gamma\lambda)^k \delta_{t+k}.$$

第二个等号的理由是：第 k 个残差出现在所有 $n \ge k+1$ 的估计里，其混合权重之和为 λ^k，再乘原有的 γ^k。因此 λ 调的是“多信任多长的实测展开”，不是又给任务加一次偏好折扣。$\lambda = 0$ 只剩一步；λ 趋近1时，完整终止轨迹上的残差相消，趋向实际回报减初始价值。

有限数据要在边界截住展开。真实结束时末尾价值为0；缓冲区结束或外部截断时，可用最后真实观测的价值自举，但不能把重置后的另一条轨迹接进来。下一小节的反向递推，正是把上式实现为一次从后向前的累积。GAE 原始推导

3.3.4 GAE 的反向时间递推

先为每条转移计算 TD 残差，再从后向前累积。terminated 表示任务真正结束，此时不使用下一价值；任务结束或轨迹被截断时，都不把当前残差递推接到重置后的另一回合。这里连接的是同一轨迹上的估计，不是 Transformer 中的残差连接。

算法步骤：

1. 用奖励和下一价值构造 TD 残差，终止时不自举。
2. 从末尾初始化递推。
3. 只在同一连续轨迹内传播后续残差。
4. 按 γλ 衰减累加，保存当前优势。

```
# 步骤1：用奖励和下一价值构造 TD 残差，终止时不自举
delta = rewards + gamma * (~terminated) * next_values - values
# 步骤2：从末尾初始化递推
carry = 0
for t in range(len(rewards) - 1, -1, -1):
    # 步骤3：只在同一连续轨迹内传播后续残差
    continuation = ~(terminated[t] | truncated[t])
    # 步骤4：按 γλ 衰减累加，保存当前优势
    carry = delta[t] + gamma * lam * continuation * carry
    advantages[t] = carry
```

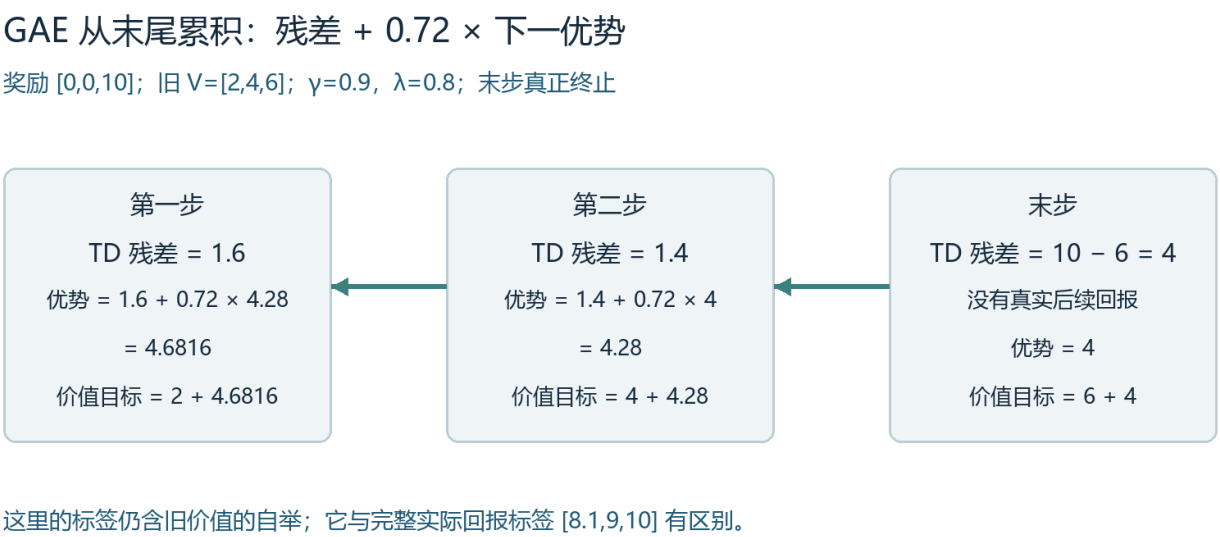


图3.5：前述算例的逆序计算与价值标签。末步真正终止，不再做价值自举；前面通过 γλ=0.72 传递下一优势。

3.3.5 练习题

题3.3-1 三个动作的 Q 分别为 8、6、1，当前策略概率为 0.5、0.3、0.2。求 V 和各动作 A。价值网络怎样得到训练标签?

题3.3-2 奖励 [0,0,10]，旧 V=[2,4,6]，γ=0.9、λ=0.8，末步真正终止。求 TD 残差、GAE 和价值目标。

题3.3-3 最后一步 r=1、V当前=2、V最后观测=5、γ=0.9。真正终止和外部时间截断的 TD 残差分别是什么？GAE 能否接到重置后的轨迹?

3.3.6 参考答案

题3.3-1答案 V 是按照当前策略选择第一动作，再继续执行该策略的平均长期回报：

$$V = 0.5(8) + 0.3(6) + 0.2(1) = 6.$$

各动作优势 A=Q−V，因此分别为 2、0、−5。第一个动作比策略平常的平均选择高 2，第二个相当，第三个低 5；第三个 Q 仍为正，但相对当前选择水平较差。这也是为什么策略更新不能只看回报是否大于零。

实际没有给定精确 Q 时，可以运行策略采集轨迹，用后续折扣奖励 G_t给价值网络作标签。网络输入状态或观测历史/任务条件，输出预测 Vψ，最小化 $(V_\psi - G_t)^2$。它学的是未来回报，不是动作标签。

也可用自举标签 $r_t+\gamma V_{旧}(s_{t+1})$，或多步回报加末端旧价值。标签通常停止梯度，防止网络同时追着自己正在改变的目标跑。完整回报更依赖真实后续样本、方差较高；自举减少等待与方差，却会带入旧价值误差。Q 网络如果单独训练，还必须把当前动作作为输入。

题3.3-2答案 末步真正终止，因此终止后的价值取 0。TD（Temporal Difference，时序差分）残差为

$$\delta_0 = 0 + 0.9(4) - 2 = 1.6, \quad \delta_1 = 0 + 0.9(6) - 4 = 1.4, \quad \delta_2 = 10 - 6 = 4.$$

广义优势估计（GAE）按下式反向递推 $\hat{A}_t=\delta_t+\gamma\lambda\hat{A}_{t+1}$，这里 γλ=0.72：

$$\hat{A}_2 = 4, \quad \hat{A}_1 = 1.4 + 0.72(4) = 4.28, \quad \hat{A}_0 = 1.6 + 0.72(4.28) = 4.6816.$$

用采集时的旧 V 加优势作价值目标，得到

$$\hat{V}^{\text{target}} = [2 + 4.6816, 4 + 4.28, 6 + 4] = [6.6816, 8.28, 10].$$

这些目标不是完整轨迹回报 [8.1,9,10]，因为 λ=0.8 对远期 TD 残差做了额外衰减，仍保留部分旧价值自举。λ=1、轨迹完整且终止处理正确时，递推才会抵消为 $G_t - V_{旧}(s_t)$。优势用于决定策略对动作的相对增减，价值目标用于训练 V 网络，两者用途不同。

题3.3-3答案 真正终止意味着任务后面没有继续回报，应取消下一状态价值：

$$\delta_{\text{terminal}} = 1 - 2 = -1.$$

外部时间截断只表示采集暂时停止，任务本身原本可以继续，应对最后实际观测自举：

$$\delta_{\text{trunc}} = 1 + 0.9(5) - 2 = 3.5.$$

这里的 5 必须来自截断前最后观测，不能来自环境重置后的初始观测。

GAE 也不能接到重置后的轨迹。截断时可以让本段最后一步的 δ 带上终点自举值，但优势递推在这段采集边界停止；重置之后属于新的回合，不是该动作的未来。可用两个标记分别控制：真正终止标记决定 δ 是否保留下一价值，回合/采集边界标记决定是否继续累计后面的优势。

如果时间限制本来就是任务定义的一部分，例如必须在规定期限内完成，那么到期可是真正终止；此时剩余时间也应进入状态。本题的“外部时间截断”特指不是任务结束条件的采集限制。

3.4 策略梯度怎样改变行动分布

3.4.1 只有结果，没有正确动作标签，怎样训练

假设在“夹爪接近方块”的输入下，策略采样了向下移动这个动作，最后任务成功。我们没有专家告诉我们最优下降幅度，但可以增加本次动作在相似输入下出现的倾向。失败或明显低于预期时，则可降低某些动作的倾向。

这种思路叫**策略梯度**：直接对策略参数求改善长期回报的方向。这里“直接”指更新策略分布，不表示省去采样、奖励和价值估计。一个常见形式是

$$L_{\text{actor}}(\theta) = -\frac{1}{N}\sum_{t} \log \pi_\theta(a_t \mid x_t)\, \hat{A}_t.$$

这是需要最小化的损失。a_t 是已经执行并记录下来的动作，$\hat{A}_t$ 是由这批数据估计的优势，更新此项时通常视为固定标签。正优势乘上负对数概率，推动网络提高采样动作的概率；负优势则给出相反方向。不是直接把优势当成新动作，也不是让策略输出优势数值。

这个平均式是常用的采样损失。若要严格对应第3.1节“从回合起点折扣”的目标，而样本又按轨迹时刻等权抽取，还要加入相应的γ^t权重；也可以让采样的状态分布已包含这种折扣。第3.4.7节将推导这一区别。比较算法时，应同时看任务目标和采样方式。

3.4.2 为什么对数概率能承载长期结果

一条轨迹的概率包含每一步动作的概率，以及环境转移的概率。如果环境本身的参数不依赖策略参数，求轨迹对数概率的梯度时，可训练部分就变成各步动作对数概率梯度之和。

于是，期望回报的梯度可以通过“轨迹结果乘以动作对数概率梯度”来估计，不需要对真实接触动力学求导。进一步减去基线后，优于预期的结果推动相应动作更常出现，低于预期的结果给出相反方向。这里的结果是统计性信号，不是一条轨迹就能证明某动作的因果作用。第3.4.7小节给出从轨迹分布到策略梯度的具体推导。

这是统计性学习方向，不是一次成功就能证明某个早期动作导致成功。大量采样、合理基线和受控任务条件，帮助平均掉随机结果。机器人探索昂贵时，正因为这种方向噪声可能很大，价值估计与数据复用才有实际意义。

3.4.3 为什么减去 V，通常仍能保留正确方向

如果把所有动作都乘以总回报，容易状态中的每个动作都可能得到很大的正信号，困难状态中的好动作却得分很小。我们希望比较“在当前情形下是否超出平常水平”。

在同一输入下，一个不依赖当前采样动作的基线，其对策略梯度期望的贡献为零。直觉上，它对所有可能动作减去同一个参照，不改变动作之间的相对优劣，而能减少“状态本来容易或困难”带来的波动。因此用 V 作基线，将回报替换为优势。

状态基线不随本次动作变化时，它乘以 log 策略梯度的条件期望为零，因此可以减去来降低波动。动作相关基线需额外处理，才能保持所需期望方向。实际算法还会受有限样本与优势近似影响。

3.4.4 Actor 和 Critic 分工在什么地方

Actor 是行动者，即策略网络，负责动作分布。Critic 是评价者，负责价值估计；它可以是 V 网络，也可以是 Q 网络。Actor-Critic 就是把行动选择与长期后果预测协作起来。

行动者与评价者：两个不同的预测任务

图示 PPO 分工：优势训练策略；回报目标训练价值

图3.6：策略网络与**价值网络**的输出和训练职责。

PPO 中，Critic 通常预测**状态价值**，结合轨迹形成优势；Actor 用优势调整动作概率。**软演员—评论家算法**（Soft Actor-Critic，SAC）把熵正则化纳入强化学习目标。在常见的连续动作实现中，Critic 评价输入动作的 Q 值，Actor 同时依据 Q 值与熵项更新。共享编码器不取消这两个角色的差别。

实际训练包括两种损失：动作更新损失、价值拟合损失。若共享骨干，还要选择两个目标的权重，防止价值预测的优化破坏动作表示。是否独立建价值网络，是算力、稳定性与信息使用之间的设计选择。

3.4.5 更新一个动作概率，会连带改变很多行为

网络参数共享，一个输入上的更新会影响其他相似输入；输出概率还必须满足分布结构。算法并不能独立拨动每个动作的概率旋钮。

所以“提高正优势动作概率”只是局部目标方向。训练步长太大，其他动作和任务也可能受到影响。策略也会到达新状态，使下一批数据改变。这就是后面需要控制单轮更新幅度、重新采集数据和检查旧能力的原因。

3.4.6 为什么基线能降低波动而不改变期望方向

给定状态，若基线 b 不依赖被抽中的动作，则 $\mathbb{E}_a[b\nabla \log \pi(a \mid s)] = b\nabla \sum_a \pi(a \mid s) = 0$。所以减去基线可以改变每个样本的更新量，平均方向保持一致。直观上，好坏是相对于当前可选行为的水平来衡量，避免所有正奖励动作都被一味提高。

如果基线依赖当前动作，这个零期望结论一般不成立，除非额外设计校正。优势由价值网络算出后，策略损失中通常把它当固定权重；否则梯度可能通过优势本身进入评价器，改变原来的估计含义。共享编码器时仍要分别解释演员损失与评价器损失的梯度路径。

3.4.7 从轨迹目标到策略梯度，究竟省去了什么

先用完全状态记号，固定环境与初始分布。轨迹密度分解为

$$p_\theta(\tau) = p(s_0) \prod_t \pi_\theta(a_t \mid s_t)\, P(s_{t+1} \mid s_t, a_t).$$

令整条轨迹的折扣总分为 $R(\tau)$，则

$$\nabla_\theta J = \int p_\theta(\tau) R(\tau) \nabla_\theta \log p_\theta(\tau)\, d\tau.$$

这一步将“对参数求导”移到采样分布的对数密度上。环境转移 P 不依赖策略参数时，其对数导数为零，剩下各步策略的对数导数。因此无需对真实物理过程求导。

过去已经得到的奖励不需要归功于当前动作，在相应条件下可以从期望梯度中消去。对于第3.1节从初始时刻定义的折扣目标，可写成

$$\nabla_\theta J = \mathbb{E}\left[\sum_t \gamma^t \nabla_\theta \log \pi_\theta(a_t \mid s_t) G_t\right].$$

这里 G_t 的折扣从当前时刻重新起算，所以另有 γ^t。有些文献把时间权重放入状态访问分布，有些控制实现采用相应近似或不同采样目标，公式外观会不同。不能无视目标与采样定义，只按符号判断谁遗漏了一项。

减去只依赖当前输入的基线 $b(s_t)$，其期望梯度贡献为

$$\sum_a \pi_\theta(a \mid s) b(s) \nabla_\theta \log \pi_\theta(a \mid s) = b(s) \nabla_\theta \sum_a \pi_\theta(a \mid s) = 0.$$

连续动作下相应地改为积分。这个等式说明基线为什么可降低波动而不改变理想期望方向；它不说明任意估计误差、动作相关基线或离线数据都自动符合条件。

3.4.8 贝尔曼期望与贝尔曼最优，为什么不能互换

对于指定策略

$$Q^\pi(s,a) = \mathbb{E}\left[r + \gamma \mathbb{E}_{a' \sim \pi(\cdot \mid s')} Q^\pi(s', a') \mid s, a\right].$$

内层平均说的是“以后按这套策略选动作”。如果换成 $\max_{a'} Q(s', a')$，就是在追求每个未来状态选择最优动作的另一种目标，而不是评价原策略。

异策略价值学习之所以可以使用旧转移，是因为一步条件后果可以保留，后续行为由目标定义。离线条件下，直接最大化 Q 容易选到未被数据约束的动作；**隐式 Q 学习**（Implicit Q-Learning，IQL）和**保守 Q 学习**（Conservative Q-Learning，CQL）针对的正是这类估计风险：前者利用数据内价值估计和加权行为学习改进策略，后者通过保守约束抑制未受数据支持的动作价值高估。

SAC 加入熵后的递推又不同，因为它评价的是最大熵目标。论文都写 Q，并不代表训练标签具有同一种语义。

3.4.9 GAE 为什么是短期与长期估计的混合

在无限或边界正确处理的递推中，一个 n 步**优势估计**可写成

$$\hat{A}_t^{(n)} = \sum_{j=0}^{n-1} \gamma^j r_{t+j} + \gamma^n V_{\text{old}}(x_{t+n}) - V_{\text{old}}(x_t).$$

它等于前 n 个 **TD** 误差的折扣和。短 n 更多依赖末尾预测，长 n 更多依赖真实后果。**GAE** 在适用条件下可看成这些不同长度估计的几何加权混合

$$\hat{A}_t^{\text{GAE}} = (1-\lambda) \sum_{n=1}^{\infty} \lambda^{n-1} \hat{A}_t^{(n)}.$$

有限轨迹需要按末尾条件处理尾项，不能直接把缺失项当真实零。这个关系解释了 λ 为什么改变估计器而非任务奖励。

实现时可分别使用两个指示量：b_t 表示下一输入是否有本任务未来可**自举**，c_t 表示递推是否可接到记录中的下一项。于是

$$\delta_t = r_t + \gamma b_t V_{\text{old}}(x_{t+1}) - V_{\text{old}}(x_t), \qquad \hat{A}_t = \delta_t + \gamma\lambda c_t \hat{A}_{t+1}.$$

真正终止时二者通常都为零。采集暂停但任务仍继续时，末尾可取 $b_t = 1$，但若下一项不是同一连续轨迹，取 $c_t = 0$。这是信息边界，而非为了方便而增加的两个随意开关。

3.4.10 PPO 的局部代理，为什么与真实回报相关又不相同

先固定一个输入 x，旧策略对动作采样。如果新旧分布满足必要支持条件

$$\mathbb{E}_{a\sim\pi_{\text{old}}}\left[\frac{\pi_\theta(a \mid x)}{\pi_{\text{old}}(a \mid x)} f(x,a)\right] = \mathbb{E}_{a\sim\pi_\theta}[f(x,a)].$$

在这个固定输入上，概率比能修正动作采样分布。PPO 用 $f = \hat{A}_{\text{old}}$ 构造旧状态上的局部收益估计。

但新策略还会改变将来访问哪些状态。普通代理保留旧状态分布与旧优势，所以只在变化较小时更可信。裁剪、有限轮更新与 **KL** 监测共同帮助限制这种偏离，却不把它变成任意更新下的精确回报。

旧动作概率极低时，比值还可能很大；高维动作的联合概率会对细小变化敏感。对动作 token、**动作块**或生成路径选择不同概率对象，都将改变代理的数值和约束含义。

3.4.11 为什么优势加权会出现指数

固定输入，设行为分布为 $\mu(a \mid x)$，希望更偏向高优势动作，同时不离原分布太远。一种理想化局部问题是

$$\max_{\pi} \{\mathbb{E}_{a\sim\pi}[A(x,a)] - \beta D_{\text{KL}}(\pi(\cdot \mid x) \| \mu(\cdot \mid x))\}.$$

加上概率归一化约束，可得到

$$\pi^*(a \mid x) = \frac{\mu(a \mid x)\exp(A(x,a)/\beta)}{Z(x)}.$$

$Z(x)$ 是使分布和为 1 的归一化常数。这说明指数权重来自“偏好较好动作”与“保持接近原行为”之间的一种优化关系，不是随意选择的激活函数。

在数据上用加权对数似然训练一个参数策略，可以视为向这种理想局部分布做投影。但真实行为分布、准确优势、充分数据和模型表达未必具备；归一化及采样也会影响不同状态的贡献。简化实现不能被直接声称为完整问题的精确解。

这个推导同时说明数据支持的限制：若 μ 对某动作没有支持，理想公式不会无依据地给它质量。离线改进的保守性具有建模依据。

3.4.12 生成损失、生成密度与生成路径的关系

考虑固定输入下的 Flow：$dz_u/du = v_\theta(z_u, u)$。在足够正则、可逆等条件下，密度沿流的变化与向量场散度有关

$$\frac{d}{du}\log p_u(z_u) = -\operatorname{div} v_\theta(z_u, u).$$

因此计算最终密度，需要考虑整个变换如何扩张或压缩体积。匹配某个方向的平方误差，不直接等于这个体积变化项。这是 Flow 训练损失与最终动作密度不能随意等同的一层原因。

对于随机生成链，内部路径联合密度可按条件核相乘。最终动作密度则要把其他内部变量积分掉

$$p_\theta(b \mid x) = \int p_\theta(b, z_{\text{internal}} \mid x)\, dz_{\text{internal}}.$$

一条可记录的路径可以提供策略梯度信号；一条路径的密度数值不是这个积分的值。这个区别同样解释了为什么路径优化合理，却不能直接宣称获得最终动作的精确概率比。

3.4.13 熵、KL 与尺度，分别在控制什么

熵只看当前策略自身分布；新旧 KL 比较本轮行为变化；对基础参考策略的 KL 比较对预训练行为的偏离。三者不是可以随意替换的“稳定项”。

对离散分布，熵较大一般表示选择更分散。连续分布的微分熵还依赖坐标尺度，同一动作从米换成毫米，数值会变化。动作归一化与熵系数需要对应定义。奖励缩放也会改变最大熵目标中的相对权重。

这些差异解释了为什么把一种模型的超参数直接抄到另一种动作单位或概率对象上，效果可能明显不同。公式上的常数看似工程细节，实际上承载了一部分建模与尺度选择。

3.4.14 价值标签与反事实，哪里需要额外假设

一条数据可以告诉你“在这个情形做了 a，后来发生了什么”；它不能直接告诉你“若改做另一个动作，会怎样”。Q 网络、**世界模型**和策略梯度分别用函数近似、转移预测和统计采样等方式处理这种未知。

如果两条轨迹的初始难度不同，结果差不能直接当作当前动作效果差。如果后续策略不同，累计回报也评价了不同后续行为。建模必须明确条件和目标策略，实验必须控制关键替代解释。

理解公式背后的思想，最终就是知道哪些关系来自定义，哪些来自可观察数据，哪些来自近似假设。把这三层分清，才知道什么时候可以相信一个机制，什么时候需要额外验证。

3.4.15 已经测得的奖励不能求导，为什么仍能更新策略

先考虑只有一步、两个动作的教学问题。动作1回报为8，动作2回报为2；策略选动作1的概率为 $p=\sigma(w)$。期望回报 $J=8p+2(1-p)$，直接对 w 求导得到 $dJ/dw=6p(1-p)$。在 $p=0.5$ 时，导数为1.5，提高 w 会让好动作更常出现。

真实任务通常不知道每个动作的回报，只能采样。于是将同一导数改写为

$$\nabla_\theta J=\sum_a R(a)\nabla_\theta\pi_\theta(a)=\mathbb{E}_{a\sim\pi_\theta}[R(a)\nabla_\theta\log\pi_\theta(a)].$$

在上述例子中，选动作1时 $\partial\log p/\partial w=1-p$，选动作2时 $\partial\log(1-p)/\partial w=-p$。取平均，$p\times 8(1-p)+(1-p)\times 2(-p)=6p(1-p)$，与直接求导相同。奖励当作采到的结果使用，需要求导的是动作概率。这条恒等关系是用有限交互估计更新方向的依据，不需要对机器人成功检测器本身求导。

若减去与本次动作无关的基线5，$p=0.5$ 时两个动作的优势为3和−3：选好动作产生 3×0.5 的正信号，选差动作产生 $(-3)\times(-0.5)$ 的正信号，二者都倾向提高好动作的选择概率。基线消掉状态整体好坏带来的共同变化，却不改变这一期望导数。多步问题再把即时回报换成后续回报或优势估计，便连接到本章策略梯度目标。Spinning Up 策略梯度推导

3.4.16 练习题

题3.4-1 为什么策略梯度用 A·∇logπ(a|s)，而不是直接对实际奖励求导？减去只依赖状态的基线为何不改变理想期望梯度？

3.4.17 参考答案

题3.4-1答案 真实奖励是机器人执行动作后由环境给出的反馈。相机、接触、采样等过程通常不提供对策略参数的可微计算图，所以不能直接把一次测到的奖励对网络求导。策略梯度通过改变“哪些动作被采到的概率”优化期望回报。

对期望作求导，利用 ∇θπθ=πθ∇θlogπθ，可得到在策略采样下用回报乘 ∇logπ 的估计。优势把回报换成“这个动作比同状态下平常选择好多少”，因此正优势增加该动作的概率，负优势降低其概率。

只依赖状态的基线 b(s) 不改变理想梯度，因为对固定 s 有

$$\mathbb{E}_{a\sim\pi}[b(s)\nabla\log\pi(a|s)]=b(s)\sum_a\nabla\pi(a|s)=b(s)\nabla 1=0.$$

连续动作把求和改为积分。在估计器中应把基线/优势视为固定权重，不把其对参数的导数混入这个推导。减基线有助于降低方差，但用近似 Q、自举和有限批次时，实际估计仍可能有误差；“期望不变”不等于每个样本更新完全相同。

3.5 PPO、相对比较与更新幅度控制

3.5.1 PPO 为什么要求每轮不要改得太快

用旧策略的数据，评价新策略的局部变化

采集机器人数据很贵，所以一批轨迹通常会训练多次。但轨迹是旧策略采样的，第一次更新之后，新策略已经不同了。若仍把所有样本看作从新策略直接采样，会忽略数据分布差异。

PPO 是 Proximal Policy Optimization，近端策略优化。“近端”表达在旧策略附近改进的思路。它用一个概率比衡量：相同输入和已记录动作，新策略比旧策略更倾向执行它多少。

$$\rho_t(\theta) = \frac{\pi_\theta(a_t \mid x_t)}{\pi_{\text{old}}(a_t \mid x_t)}.$$

分母属于采集这批数据时的固定策略。初始新旧参数相同时，正确重算应有 $\rho_t \approx 1$。之后 $\rho_t > 1$ 表示增加这个动作的倾向，$\rho_t < 1$ 表示减少。

概率比用于在旧策略采集的动作上比较新策略倾向。PPO 保留旧数据状态与固定优势，近似评价旧策略附近的变化。更新较大时，新策略访问的状态也会变化，这部分由后续重新采集的数据反映。

为什么需要裁剪

如果某个样本优势很高，一直增大它的概率会持续改善未裁剪的目标。有限数据中的高优势可能只是偶然，而大更新会让策略跑到**价值模型**没见过的区域。

PPO 用裁剪的代理目标减少这种过度追逐

$$L_{\text{clip}} = \frac{1}{N} \sum_t \min\left(\rho_t \hat{A}_t,\ \text{clip}(\rho_t, 1-\epsilon, 1+\epsilon)\hat{A}_t\right).$$

这里目标是最大化；实现成损失时取负号。ϵ 是允许局部变化的参数。clip 把比值压在区间内，min 选择更保守的收益评估。

正优势时，继续增大概率为什么不再额外得分

设旧动作概率 0.2，优势为 3，$\epsilon = 0.2$。新概率为 0.22 时，比值 1.1，贡献为 3.3；新概率为 0.24 时，比值 1.2，贡献为 3.6；新概率继续变成 0.36 时，比值 1.8，但裁剪项贡献仍为 3.6，目标取较小值，不再给额外收益。

PPO：正优势样本的收益激励在上界后变平

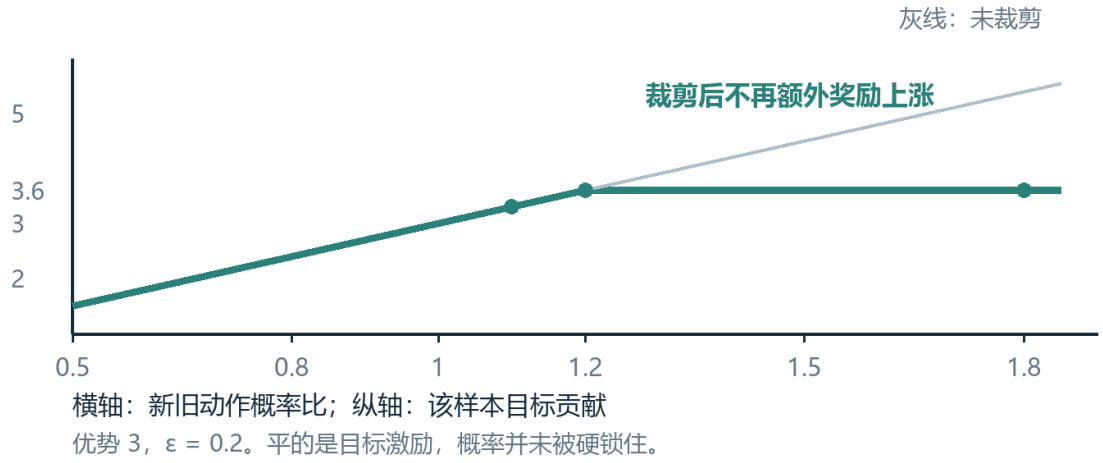


图3.7：PPO 限制的是样本的改进激励。

注意，公式没有把模型实际概率硬锁在 0.24。其他样本、共享参数或优化过程仍可能使它更大。它抑制的是这个样本继续推动上涨的激励。

负优势时，min 不是多余的

设优势为 −3。如果比值已经降到 0.5，未裁剪贡献为 −1.5，裁剪到 0.8 后贡献为 −2.4；min 选择 −2.4。因为要最大化目标，算法不再额外奖励把坏动作概率降得更低。

负优势为 −3、概率比升到 1.8 时，未裁剪项为 −5.4，裁剪项为 −3.6，min 仍选择 −5.4，继续惩罚坏动作概率上升。因此 PPO 按优势符号限制有利方向的进一步收益，而非对所有概率比施加同一种硬限制。

为什么还要看 KL、熵和价值损失

KL 是 Kullback–Leibler divergence，KL 散度，衡量两个分布的差异。训练中可能监测新旧策略 KL，并在变化太大时停止该轮更新。PPO 裁剪本身不保证全局 KL 一定小。

熵衡量动作分布的不确定性。加入适度熵激励可以避免过早集中在少数动作，但机器人动作更随机不自动更好；噪声尺度、约束与安全执行仍要协调。

价值损失训练 Critic，和策略裁剪是不同目标。训练日志里策略损失改善、价值损失改善、真实成功率改善也不是同一现象。还需观察奖励独立验证、动作平滑性和训练分布外表现。

若另外使用“与最初预训练模型保持接近”的参考策略约束，它的参照是固定基础模型；PPO 的旧策略则每轮采集后刷新。前者主要限制偏离原能力，后者主要控制本轮数据上的局部更新。

3.5.2 不训练价值网络，也能比较多次尝试吗

为什么想到用同题多次尝试作参照

训练大型 **VLA** 已经昂贵，再加一个视觉**价值网络**还需要额外参数、输入处理和价值标签。如果同一任务能低成本重复尝试，可以直接比较若干结果，让较好的尝试得到正信号，较差的得到负信号。

组相对策略优化，Group Relative Policy Optimization，缩写 **GRPO**，是采用这类组内比较思想的一种方法。其原始算法来自语言模型训练；迁移到机器人需要重新处理任务重置、轨迹长度、动作密度和反馈分配，不能照搬一段文本生成的所有假设。原始入口见 DeepSeekMath。

一组四条轨迹的例子

从相同或可比较初始条件运行四次，总回报为 10、10、0、0。平均为 5，按本例总体标准差计算为 5。标准化组内得分为

$$\hat{A}_i^{\text{group}} = \frac{R_i - \bar{R}}{\text{std}(R) + \eta}.$$

R_i 是第 i 次尝试的回报，η 防止除零。忽略很小的 η，得分近似为 1、1、−1、−1。它们可以参与类似 PPO 的策略更新，表达“这次尝试比本组平均好或差”。

这里仍可能沿用“优势”这个名字，但它不是严格的同一状态动作优势 $Q^\pi - V^\pi$。它是轨迹质量的相对信号。一整条成功轨迹中的动作都收到正向权重时，算法并没有因此识别哪个动作真正关键。

为什么比较条件决定信号质量

若一条轨迹的方块靠近夹爪，另一条要绕障碍物，分数差可能主要来自难度而非动作策略。组内归一化能调整数值尺度，不能自动消除这个混杂因素。

因此应尽可能匹配任务、初始条件和评估规则。若使用不同初始状态，还需说明为何这种比较有意义。有真实机器人重置成本时，“不用价值网络”节省的计算可能被多次尝试的交互成本抵消。

四次全失败时，若奖励均为零，组内差异也为零，不能知道哪次更接近成功。可考虑适当的过程反馈、成功课程、增加多样探索或引入示范，不能仅靠归一化创造任务信息。

留一法基线在避免什么相关性

留一法，Leave-One-Out，缩写 LOO，用其他样本的平均回报作为本样本的基线。例如第一条成功轨迹，其余三条平均回报是 $10/3$，本条相对得分是 $10 - 10/3$。

这样本样本的回报不直接进入自己的平均基线。在独立、同分布及相应估计条件下，这有助于基线满足需要的独立性。若还用全组标准差缩放，统计相关性会更复杂。使用 LOO 不能自动证明整个机器人优化器无偏。

RIPT-VLA 使用留一法处理稀疏反馈，TGRPO 考察轨迹与步级信号。组内比较不自动识别每个动作贡献。RIPT-VLA，TGRPO。

3.5.3 概率比修正了动作抽样，为什么还要限制更新

固定某个状态 s，旧策略采样动作，新策略却想评价这些动作的优势贡献。离散动作下，

$$\sum_a \pi_{\text{old}}(a \mid s) \frac{\pi_\theta(a \mid s)}{\pi_{\text{old}}(a \mid s)} A^{\text{old}}(s,a) = \sum_a \pi_\theta(a \mid s) A^{\text{old}}(s,a).$$

旧动作概率与分母抵消，说明比值怎样把旧动作抽样改成新动作加权。这个推导要求新策略需要的动作在旧策略下有支持。但机器人数据不只有动作分布，还包括此前行动形成的状态分布。PPO 用旧轨迹中的状态和优势，不能靠这个单步比值同时把全部状态都换成新策略将要访问的状态。

所以未裁剪目标是局部代理：在旧状态上，提高旧优势较好的动作倾向。小幅更新时它较有参考价值，更新太大时，新状态、新价值误差和采样噪声可能使代理与真实回报脱节。裁剪抑制某些样本继续推动有利方向上的大变化；重新采集则补上新策略实际访问的状态。两者分别处理一轮优化和下一轮数据，并共同组成 PPO 流程。Spinning Up PPO 教程

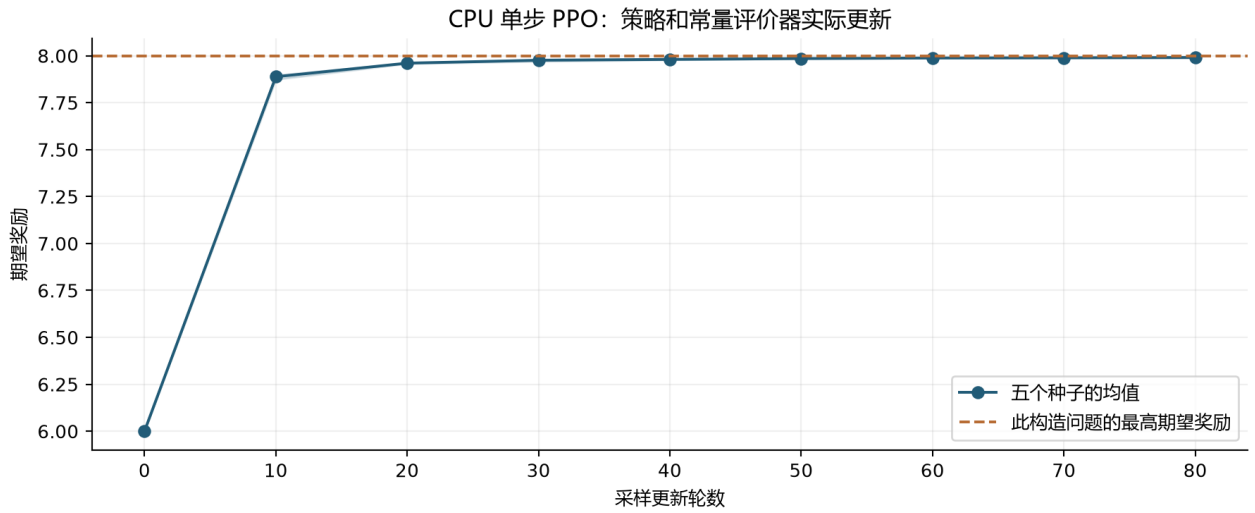


图3.8：实际更新的单步教学策略与常量评价器。曲线表示精确概率下的期望奖励，阴影为五个种子范围，不是机器人任务结果。

3.5.4 PPO 的采集与更新

```
用当前策略采集轨迹，保存每个已执行动作的旧对数概率
用轨迹奖励与旧价值估计计算优势和回报目标，切断标签梯度
重复若干优化轮：
    用更新后的策略评价这批原动作，得到新对数概率
    ratio = exp(新对数概率 - 已保存旧对数概率)
    actor_loss = 负的平均裁剪代理
    critic_loss = 新价值预测与固定回报目标的误差
    更新策略及价值参数；记录散度、裁剪比例与独立回报
完成后重新采集下一批轨迹
```

3.5.5 练习题

题3.5-1 PPO 取 ε=0.2。分别计算 (A,ρ)=(2,1.3)、(2,0.7)、(−3,0.7)、(−3,1.3) 的裁剪代理贡献，并指出哪两项对 ρ 的局部梯度为零。

题3.5-2 同一条件下四条轨迹奖励 [0,0,1,1]，以总体标准差作组内标准化得到什么？全部失败时为什么没有区分信号？

3.5.6 参考答案

题3.5-1答案 裁剪代理为 min[ρA,clip(ρ,0.8,1.2)A]，这是要最大化的贡献：

A、ρ	未裁剪项	裁剪项	取较小值	对 ρ 的局部梯度
2、1.3	2.6	2.4	2.4	0
2、0.7	1.4	1.6	1.4	2
−3、0.7	−2.1	−2.4	−2.4	0
−3、1.3	−3.9	−3.6	−3.9	−3

第一行已经过多提高好动作的概率，继续增大 ρ 不再增加代理收益。第三行已经过多降低差动作的概率，继续降低 ρ 也不再增加收益。这两项为零梯度。第二行对好动作的概率降低过头，第四行对差动作的概率提高过头，目标仍给出纠正信号。

这是单条样本贡献对 ρ 的梯度；其他样本、熵损失或共享参数仍可能改变这个动作的实际概率。PPO 因而限制的是部分继续改进的激励；它没有对全部动作的新旧概率比施加硬约束。这里的 [0.8,1.2] 是比值区间，不是动作概率区间。

进一步阅读：Berkeley CS 285：Off-Policy Policy Gradient，裁剪代理目标。

题3.5-2答案 四条奖励的均值为 μ=0.5。按总体标准差计算，方差为

$$\sigma^2 = \tfrac{1}{4}[(0-0.5)^2 + (0-0.5)^2 + (1-0.5)^2 + (1-0.5)^2] = 0.25,$$

所以 σ=0.5，标准化结果为 [-1,-1,+1,+1]。它表达同组内相对好坏，而非任务奖励的绝对单位。

如果全部失败且奖励都为 0，均值和标准差都为 0。实现通常用 σ+ε 作分母或跳过该组，此时标准化后都为 0，不能判断哪个失败动作更接近成功。没有区分不是公式“坏了”，而是给定标签没有包含差别。

可以收集更多探索结果，让组内出现可辨别的成功或进度；也可在有可靠依据时加入中间质量评价或专家纠正。不能靠把分母 ε 调小凭空制造正确排序。全失败组仍可用于价值或监督目标，但不应声称其零组优势能提供偏好方向。

3.6 经验回放、目标网络与连续动作学习

3.6.1 两组经常被混淆的分类

在线学习允许继续与环境交互收集新数据；离线学习主要依赖固定数据。另一组分类是同策略 on-policy 与异策略 off-policy：前者依赖当前或刚采集策略的数据，后者能利用其他行为策略的数据。

在线与离线描述“能否继续获得数据”，同策略与异策略描述“数据由谁产生、怎样被利用”。在线训练也可以是异策略，比如从经验池取出几小时前的旧数据；离线训练通常需要处理行为策略和目标策略不一致。

PPO 使用近期旧策略数据做有限次局部更新，通常归为同策略方法。**SAC** 等方法把转移保存在回放缓冲区，replay buffer，即经验池，允许跨很多轮反复学习。数据复用可能更省交互，但要处理价值误差和分布差异。

3.6.2 Q 学习为什么有复用一条旧转移的可能

一条旧数据记录了输入 x、实际动作 a、奖励 r 和下一输入 x'。如果环境规则与奖励定义没变，这次动作引起的转移仍提供信息，即便采集它的策略已经旧了。

但以后怎样继续可以换成新策略。于是 Q 的目标可以用“实际本步结果 + 当前策略在下一情形的预期价值”。旧数据提供一步物理事实，网络估计新策略后续。这是复用的建模依据，不是旧轨迹总回报自动等于新策略回报。

3.6.3 SAC 怎样同时考虑好结果和适度探索

SAC 是 Soft Actor-Critic，软行动者—评价者。“软”主要指其最大熵目标：在高回报之外，鼓励策略保留一定分布不确定性，而不是过早只选择一个动作。

常见 SAC 包含随机策略、两个 Q 网络及慢速更新的目标 Q。Q 输入决策信息与候选动作，输出软回报估计。取两个估计中的较小值，可以减轻策略利用偶然高估的倾向；数据不足时仍需关注实际执行结果。

一个常见目标为

$$y = r + \gamma \left[\min_j Q_{\text{target},j}(x', a') - \alpha \log \pi_\theta(a' \mid x') \right], \qquad a' \sim \pi_\theta(\cdot \mid x').$$

请沿数据流读它：旧数据给出 r 和 x'；当前策略在 x' 采样将来动作 a'；目标 Q 评价接下去的回报；熵项表达该算法对分布不确定性的偏好；整个括号再被折扣。真实任务结束时不加未来项。

SAC 的 Q 是与这种“奖励加熵”的目标一致的软价值，不能不加说明地当作仅累计任务奖励的普通 Q。α 控制熵与奖励的权衡，奖励尺度变了，这个权衡也会变。

3.6.4 两个网络分别怎样训练

Q 网络用旧转移的实际动作作输入，以 y 作停止梯度标签，最小化平方误差。策略则在经验池中的输入上生成新动作，并优化

$$L_\pi = \mathbb{E}_{x \sim \mathcal{D},\ a \sim \pi_\theta(\cdot|x)} \left[\alpha \log \pi_\theta(a \mid x) - \min_j Q_j(x, a) \right].$$

这表示想生成评价更高的动作，同时维持目标所需的熵。可**重参数化**的连续策略允许梯度穿过所生成动作，再经过 Q 对动作的导数返回策略。

策略更新时，通常不更新 Q 的参数，但需要保留 Q 输出相对于动作的梯度。冻结 Q 参数与把 Q 输出整体断开梯度是两回事；后者可能切断策略所需的改进信号。

目标网络提供更新较慢的标签。若当前 Q 每变一次，下一状态估值也立刻跟着变，网络就会同时追逐自己改变的预测与目标，估计误差可能互相放大。常用缓慢更新 $\bar{\phi} \leftarrow (1-\eta)\bar{\phi} + \eta\phi$，其中 ϕ 是正在

训练的 Q 参数，$\bar{\phi}$ 是目标参数，η 较小。训练网络通过梯度追随标签，目标网络通过移动平均追随训练网络，两种更新速度被分开。它主要改善标签稳定性，不能补充数据中缺失的动作后果。

3.6.5 这种路线适合什么，代价是什么

SAC 适合能够采样连续动作并计算密度的策略。动作维度高或生成机制复杂时，Q 的训练可能更困难。Actor 会追求 Q 预测的高分，其中也可能包含评价错误，所以应把 Q 分数与真实回报一起观察。

TD3（Twin Delayed Deep Deterministic Policy Gradient，双重延迟深度确定性策略梯度）使用确定性策略 $a = \mu_\theta(x)$，通过 Q 对动作的梯度调整它。三个设计分别针对评价误差：下一价值取两个 Q 中较小者，减轻高估；策略比 Q 更新得慢，给评价器更多时间修正；构造目标时在下一动作附近加入小扰动并限幅，使标签不只依赖一个尖锐的虚高点。采集经验时另加动作噪声用于探索，它与目标动作平滑中的噪声作用不同。SAC 则把随机策略与熵偏好纳入目标，两者的动作输出和回报定义因此不同。TD3。

3.6.6 压缩高斯动作后，怎样保留采样梯度与正确密度

SAC 常让网络根据状态输出 μ_θ 与 σ_θ，采样 $\epsilon \sim \mathcal{N}(0, I)$，计算 $u = \mu_\theta + \sigma_\theta \epsilon$，再用 $a = \tanh u$ 限制动作范围。固定一次噪声后，动作仍是参数的可微函数；评价器告诉它 $\partial Q / \partial a$，梯度便能经 a 和 u 回到策略。这就是这里使用**重参数化**的目的。

同时，压缩改变了密度。对一维变量，概率质量满足 $p_a(a)\, da = p_u(u)\, du$；因为 $da/du = 1 - a^2$，得到 $\log p_a(a) = \log p_u(u) - \log(1 - a^2)$。靠近动作边界，同样的概率质量被压进更窄区间，密度会改变。SAC 的熵项必须使用动作密度，不能仍使用压缩前高斯的密度。

算法步骤：

1. 网络提供均值与对数标准差，构造高斯。
2. 用重参数化采样，再压缩到动作范围。
3. 对高斯对数密度扣除变换的对数导数，并合并动作各维。

```
import math
import torch
from torch.nn import functional as F

def sample_squashed(mu, log_std):
    # 步骤1：本函数接收网络输出，不再独立训练另一组分布参数
    normal = torch.distributions.Normal(mu, log_std.exp())
    # 步骤2：rsample 保留 u 对均值、标准差的求导路径
    u = normal.rsample()
    action = torch.tanh(u)
    # 步骤3：这是 log(1-tanh(u)^2) 的稳定等价形式
    log_derivative = 2 * (math.log(2) - u - F.softplus(-2 * u))
    log_prob = (normal.log_prob(u) - log_derivative).sum(dim=-1)
    return action, log_prob
```

代码避免先算 $1-a^2$ 再在边界附近取对数，但关键数学关系仍是概率质量守恒。若把 $[-1,1]$ 继续缩放为真实动作区间，还需计入该线性变换的密度尺度。相同的变量变换问题也会出现在连续流模型中；可采样与可计算密度是两个需要分别确认的接口。Spinning Up SAC 教程

3.6.7 SAC 的策略与评价器更新

```
从 replay 读取 (s, a, r, final_s_next, terminated)
不记录梯度地生成下一动作和熵修正的目标 Q
target = r + gamma * 非终止 * 下一步柔性价值
用 target 拟合两个 Q 网络，更新 Q 参数
用重参数化策略生成当前动作；暂时冻结 Q 参数
actor_loss = mean(alpha * log_pi(action|s) - min(Q1,Q2))
保留 Q 对 action 的梯度，更新策略，再恢复 Q 参数设置
按规定更新温度；用慢速混合更新目标 Q 参数
```

3.6.8 目标网络与冻结评价器，是两种不同的稳定办法

Q 网络的标签若为 $y=r+\gamma Q_{\bar{\phi}}(s',a')$，$\bar{\phi}$ 是缓慢更新的目标参数。一次训练先计算这个数，再把它当固定标签，让当前 $Q_\phi(s,a)$ 接近它。否则预测和答案同时随同一项损失变化，网络可能通过改答案来减小误差。目标网络像 DINO 的移动平均教师一样缓慢吸收参数，但它在这里提供未来价值标签，而不是视觉表示目标。

DDPG（Deep Deterministic Policy Gradient，深度确定性策略梯度）用价值对动作的导数训练确定性策略。训练演员时，当前评价器固定，演员提出 $a=\pi_\theta(s)$，用 $-Q_\phi(s,a)$ 更新动作选择。此时需要 $\partial Q/\partial a$，再经过 $\partial a/\partial\theta$；不需要更新评价器参数。标量例子 $Q(a)=-(a-2)^2$、$a=\theta=0$ 中，演员损失导数为 $2(a-2)=-4$，梯度下降会把动作向2移动。若把 Q 前向放进 no_grad()，这个方向也被丢掉，演员便不能通过价值学习。两次“固定目标”分别固定不同对象，计算图必须相应安排。
Spinning Up DDPG 教学

3.6.9 练习题

题3.6-1 SAC 更新策略时冻结 Q 网络参数，却仍要对 Q(s,aθ) 求导。为何 no_grad() 包住整个 Q 前向会破坏策略更新?

题3.6-2 a=tanh(u)，原变量密度为 p(u)。写出动作 log 密度；双 Q、目标网络和熵分别解决什么问题?

3.6.10 参考答案

题3.6-1答案 软演员—评论家算法（SAC）的策略损失包含 −Qφ(s,aθ)。这里策略通过重参数化产生动作 aθ=fθ(s,ε)，想知道怎样改变动作能提高 Q，因此需要

$$\nabla_\theta Q_\phi(s,a_\theta)=\frac{\partial Q_\phi}{\partial a}\frac{\partial a_\theta}{\partial\theta}.$$

冻结 Q 的参数 φ 只是本次不更新评价器；∂Q/∂a 仍要保留。可把 Q 参数 requires_grad=False，让前向在正常梯度模式中执行，策略损失反向传到动作及策略参数。

若用 no_grad() 包住整个 Q 前向，这个 Q 值在策略计算图中被当成常数，策略收不到“哪个动作能提高价值”的导数。若熵项仍保留梯度，策略可能只朝增大熵的方向学习，因此并非一定报错，而是学习目标被悄悄改变。

这与题1.2-2同一原则：固定模块权重不意味着固定模块输出对输入的关系。可核查策略梯度是否包含价值项，并用固定噪声微调动作检查 Q 的数值变化；下一次训练评价器前再恢复所需参数的梯度设置。

题3.6-2答案 对每维 $a_i=\tanh(u_i)$，导数为 $1-a_i^2$。变量变换后

$$\log \pi(a|s) = \log p(u|s) - \sum_i \log(1 - a_i^2).$$

求 log 密度时用生成该动作的原始 u；减去变换的对数雅可比，是因为同样的 u 空间概率质量被压到不同宽度的动作区间。接近边界时 tanh 压缩强，不能直接把原高斯 log 密度当作动作密度。若之后还做有尺度的线性变换，还应计入对应的尺度项。

双 Q 用两个评价器的较小估计构造目标，减少策略和自举对偶然高估动作的追逐；它会引入一定低估倾向，并不消除所有误差。目标网络缓慢跟随当前网络，让“预测值”与“训练标签”不同时剧烈移动。熵项鼓励策略保留多种动作、避免过早确定，有利于探索；系数过大却可能妨碍精密操作。

三者分别针对价值高估、移动目标和探索/确定性取舍，不应把它们都概括为同一种“稳定训练技巧”。

3.7 离线强化学习的数据支持范围与保守改进

3.7.1 离线强化学习的核心困难

有一个抓取数据集，记录了人类和旧机器人策略的成功失败轨迹。我们希望学出更好策略，但暂时不能真机试错。

如果直接让 Actor 最大化 Q，它可能找到数据里从未出现过的动作。例如 Q 对“快速甩动后松手”预测很高，但这只是函数近似在空白区域的错误。在线训练可以尝试后修正，离线训练却没有这个纠错通道。这叫分布外动作的价值外推问题。

因此离线方法通常要同时做两件事：从数据里辨别更好行为，并限制没有证据的乐观改进。不同方法在价值训练与策略限制上采用不同结构。

3.7.2 指数质量权重怎样从“改进但保留数据支持”推出来

设在状态 s 上，数据行为分布为 $b(a \mid s)$。我们希望选较高 Q 的动作，又不想随意离开这份有证据的分布。先不限制神经网络形式，把待选择的动作分布写成 p，优化

$$\max_p \sum_a p(a)Q(s,a) \qquad -\beta \sum_a p(a) \log \frac{p(a)}{b(a \mid s)}, \qquad \sum_a p(a) = 1.$$

第一项偏向高价值，第二项用 KL 代价限制偏离，$\beta > 0$ 控制两者取舍。加上归一化乘子并对每个 $p(a)$ 求导，得到 $Q - \beta[\log(p/b) + 1] + \nu = 0$；整理和归一化后，

$$p^*(a \mid s) = \frac{b(a \mid s)e^{Q(s,a)/\beta}}{Z(s)}.$$

指数不是临时选的“好样本放大函数”，而是这个受约束改进问题的解。两个数据动作原来占0.8和0.2，Q 为0和2，$\beta = 1$ 时，新分布约为0.351与0.649：较少出现但较好的动作被提高，仍没有给数据零支持的动作创造证据。

下一步用参数化策略模仿 p^*。将其到策略的 KL 改写为交叉熵，便得到对数据动作的加权负对数似然。减去只依赖状态的基准 $V(s)$，指数中的公共因子可吸收到 $Z(s)$，于是常写成 $w = \exp[(Q - V)/\beta]$。

$$L_\pi = -\mathbb{E}_{(s,a)\sim\mathcal{D}}\left[w(s,a)\log \pi_\theta(a \mid s)\right].$$

这些权重总为正：低优势动作被较轻地模仿，不是像带符号策略梯度那样被直接反向推开。实践中会限制极端权重，且常省略每状态归一化；省略后也改变了不同状态在共享网络里的学习权重。对扩散或 Flow，可让质量权重作用在原监督损失上，但不能由此把它叫作精确概率比优化。CS285 离线强化学习推导讲义，第2节

3.7.3 IQL 为什么需要一个“偏向较好动作”的 V

IQL 是 Implicit Q-Learning，隐式 Q 学习。它的一项重要设计是：价值学习尽量利用数据中已有动作，避免在训练 Q 时直接对未见动作做显式最大化。

它训练 Q、V 和策略。这个 V 常通过非对称平方误差拟合数据动作的 Q，使预测偏向较高 Q，而不是所有数据动作的普通均值。这种回归叫 expectile，期望分位回归；它与按排序位置定义的 quantile 分位数不是同一个概念。

设同一输入下两个数据动作 Q 值为 0 和 10，出现权重相等。采用参数 $\tau = 0.7$，高于 V 的残差给予 0.7 权重，低于 V 的给予 0.3。对介于 0 和 10 的 V，目标为

$$L(V) = \tfrac{1}{2}\left[0.3V^2 + 0.7(10 - V)^2\right].$$

其最优值是 7。它比均值 5 更偏向高质量动作。建模意图是：在数据支持的动作里，形成一个偏向更好后续行为的价值参照。这不是随机当前策略的平均**状态价值** V^π；同一个字母在不同算法里必须按训练目标解释。

3.7.4 IQL 的三个目标各自读取哪些数据

IQL 每条样本仍是离线转移 (s, a, r, s')。第一步把已观察动作的 Q 当成停止梯度标签，训练状态网络 V。令 $u = Q_{\bar{\phi}}(s,a) - V_\psi(s)$，使用 $|\tau - \mathbf{1}_{u<0}|u^2$：Q 高于 V 时给予 τ 权重，低于时给予 $1 - \tau$。$\tau > 0.5$ 使 V 更偏向较好数据动作；上一小节的0、10例子展示了这种偏向。

第二步用 $y = r + \gamma(1 - d)V_\psi(s')$ 训练 $Q_\phi(s,a)$，标签停止梯度，d 表示真实终止。这里给 Q 输入的仍是数据实际动作，下一目标读取 V，没有让当前策略在未见动作上寻找一个最大的 Q。V 和 Q 的更新交替进行，目标 Q 的参数按算法缓慢跟随。

第三步计算数据动作相对基准的差 $Q_{\bar{\phi}}(s,a)-V_{\psi}(s)$，作为固定质量信号，以指数权重训练策略。部署才让策略根据新观测生成动作。V 本身不是一个“成功分类器”，Q 也不是新策略已经真实执行过的结果；它们在这套目标中近似数据支持内较好后续行为的价值。

连续状态下，同一个精确状态通常只有一个已记录动作，不能机械地在每个状态的样本表里做最大化。IQL 依靠共享函数近似与非对称回归在相关状态间学习这种较好水平；覆盖不足和拟合误差仍会影响结果。理解它的价值在于看到：离线改进可以把“选更好动作”放进价值目标与监督权重，而不必在训练时显式遍历所有候选动作。CS285 IQL 讲义，第3节、IQL 原论文

3.7.5 其他路线在约束什么

AWAC 是 Advantage-Weighted Actor-Critic，优势加权行动者—评价者。它用优势加权策略更新，并结合异策略价值学习，强调离线数据起步再在线改进。其价值估计和在线接续方式不应直接等同于 IQL。

CQL（Conservative Q-Learning，保守 Q 学习）在贝尔曼拟合之外，抑制没有数据支持的相对高 Q。以离散动作的一种目标为例，附加项为 $\alpha[\log\sum_a \exp Q(x,a)-\mathbb{E}_{a\sim\mathcal{D}(\cdot|x)}Q(x,a)]$。第一部分由各候选动作的 Q 共同决定，特别关注高值；第二部分对数据中真正出现的动作作补偿。两者结合，使未观察动作的虚高更难被策略利用。连续动作中需要采样近似相应项，具体算法还包括自己的权重和约束设置。“保守”描述价值学习的方向，并不意味着机器人一定选择最慢动作，也不是逐次安全保证。CQL。

选择时要先问数据条件：有多少成功、失败和恢复？动作是否覆盖所需行为？**价值模型**能看见任务条件吗？能否补充交互？如果数据很窄，优先改善数据和监督基线，可能比增加一个复杂价值目标更有效。

3.7.6 CQL 怎样压低数据外的虚高，而不是所有 Q 一起压低

用离散动作看附加项 $L_C=\log\sum_a e^{Q_a}-\sum_a b_aQ_a$。$b_a$ 是数据中各动作的比例，对第 a 个输出的导数为

$$\frac{\partial L_C}{\partial Q_a}=\mathrm{softmax}(Q)_a-b_a.$$

例如数据只做动作1，$b=(1,0)$，当前 Q 预测为 $(1,5)$。其 softmax 约为 $(0.018,0.982)$，导数约为 $(-0.982,0.982)$。只看这一附加项的下降方向，动作1的 Q 上升，未观察动作2的 Q 下降。前一项关注高预测，后一项补偿数据动作，因此限制的是未支持的相对乐观。

如果所有 Q 同时加一个常数，前项和数据平均项增加量相同，附加项不变。真实奖励与贝尔曼拟合仍负责约束价值的回报尺度；CQL 的附加项提供相对抑制，并非单独定义全部价值。其强度过大会妨碍必要的改进，过小又挡不住外推误差，需要连同数据覆盖和独立任务表现选择。CS285 CQL 讲义，第4节

3.7.7 练习题

题3.7-1 IQL 的 V 用 expectile 目标拟合数据动作的 Q。取 τ=0.8、两个 Q 标签为 1 和 3、当前 V=2，写出两项权重和梯度方向；策略之后用什么标签训练？

3.7.8 参考答案

题3.7-1答案 记残差 $u = Q - V$，expectile 的平方损失为 $w(Q-V)^2$：$u \geq 0$ 时 $w = \tau$，$u < 0$ 时 $w = 1 - \tau$。当前 $V = 2$，$Q = 1$ 的残差为 -1、权重为0.2；$Q = 3$ 的残差为 $+1$、权重为0.8。

把Q当作固定标签，每项对V的导数为 $2w(V - Q)$，分别为 $+0.4$ 和 -1.6。梯度下降让第一项把V拉低，第二项把V拉高。若两标签等权，平均梯度为 -0.6，所以本次总更新提高V。处于两个标签之间的最优值满足

$$0.2(V-1) + 0.8(V-3) = 0, \qquad V = 2.6.$$

它偏向较高的数据动作价值，而非普通均值2，也不是直接取最大值3。

策略仍以数据中实际执行的动作 a 为监督标签。本书用温度 $\beta > 0$ 写质量权重

$$w_\pi(s,a) = \exp\left(\frac{Q(s,a) - V(s)}{\beta}\right), \qquad L_\pi = -\mathbb{E}_{\mathcal{D}}[w_\pi \log \pi_\theta(a \mid s)].$$

权重作为固定信号参与策略更新，必要时裁剪。β 越小，较好动作和较差动作之间的学习强度差异越大；原论文的逆温度对应这里的 $1/\beta$。例如本题当前V=2、$\beta = 1$ 时，两标签对应权重约0.368与2.718。标签本身仍是两条数据动作，没有让策略生成未见动作再用Q当答案。这个V是数据动作Q的上侧expectile，应与当前策略的平均价值 V^π 区分。

进一步阅读：IQL 原论文，第4.1—4.3节（expectile 与逆温度）、Berkeley CS 285：Offline RL 推导讲义。

3.8 生成式 VLA 的概率、梯度与后训练接口

3.8.1 VLA 的动作接口，决定怎样接入强化学习

先识别从输入到动作的实际路径

一个典型 **VLA** 会把图像转换为视觉表示，把语言指令转换为语言表示，再通过联合模型或动作模块生成控制信号。表示是供后续计算使用的内部特征，不一定能逐维解释为物体坐标。

动作头是把内部特征转成动作输出的网络部分。动作可能是关节目标、末端位姿增量、夹爪开合，也可能是多个未来控制步组成的**动作块**。判断某篇算法能否用于你的 VLA，首先应查这个接口，而不是只看它也叫“大模型”。

动作 token、连续回归、生成式动作的差别

动作 token 化的一类方法把连续动作分箱或压缩为离散符号，再预测这些符号的概率，例如把位移划成若干档。这里的 token 是离散动作编码。前面注意力章节还把连续动作向量的嵌入称为 token，那只表示参与序列计算的信息单元。接入强化学习时，应先确认输出是否真的有离散类别分布，不能仅凭“使用动作 token”判断能否计算概率。

这种策略通常能提供已采样 token 的对数概率，便于连接**策略梯度**。若动作由多个 token 顺序生成，联合对数概率是各条件对数概率之和；条件包含已生成前缀。它仍需要处理**量化**误差、解码成本和动作执行映射。

连续回归直接输出数值向量。确定性回归没有自动给出随机策略密度；若要使用普通 **PPO**，可以额外定义随机输出分布，但这同时改变了探索与动作模型，不是给平方误差随意取负号就得到概率。

扩散与 Flow 则把简单噪声逐步变成动作或动作块。这有助于表示多种合法做法，例如从障碍物左侧或右侧绕行，但复杂的生成过程让密度与更新接口更难处理。第1.9、3.2、3.8节会展开。

OpenVLA 可帮助理解动作 token 接口；OFT，即 Optimized Fine-Tuning，优化微调，讨论连续动作、并行生成和动作块等适配；π_0 是理解 Flow VLA 的入口。它们是具体模型与训练设计，不代表所有 VLA 都采用同一种动作表示。OpenVLA，OFT，π_0。

更新哪里，意味着能改变什么

只更新动作头，可以调整已有表示下的控制；若模型根本没识别正确目标，只改动作头可能不够。更新视觉与语言骨干可能改善任务理解，但成本和旧能力变化更大。

LoRA 是 Low-Rank Adaptation，低秩适配。它用较小的可训练矩阵增量调整部分权重，减少需要训练和存储的参数。它不是强化学习算法：相同适配器既可用监督损失，也可用策略优化损失训练。

冻结大部分参数并不意味着一次采样不需要运行大模型。交互速度仍受到完整前向计算、动作生成步数和图像处理影响。参数效率、采样效率、部署延迟应分开评价。

动作坐标和归一化也是策略定义的一部分

末端增量是以世界坐标、相机坐标还是机器人自身坐标表示？数值是毫米还是米？夹爪是连续控制还是二值开合？这些差异会改变一个输出向量的实际含义。

归一化把动作按固定统计量缩放到方便训练的范围；反归一化恢复真实控制单位。训练与部署使用不同统计量时，同一个网络输出会变成不同物理动作。

动作裁剪、执行器限幅和控制器转换也会使“模型采样动作”与“实际执行动作”不同。奖励来自后者，概率来自前者，算法需要明确二者之间的映射。采集时额外加入噪声，却仍记录未加噪动作的概率，也可能破坏同策略更新的对应关系。

3.8.2 生成式 VLA 接入 RL，困难究竟在哪里

难点不是没有随机性，而是怎样评价这份随机性的变化

一个高斯策略直接给均值与方差，很容易计算已采样动作的对数密度。扩散或 Flow 策略可能先采样噪声，再经过多次转换才得到动作。最终动作分布存在，但最终密度不一定方便计算。

PPO 要问“相同动作新旧策略的概率比是多少”。去噪误差小、Flow 匹配误差小，不能自动回答这个问题。误差是训练代理量，密度是生成分布的量，二者必须通过对应理论连接。

路线一：把随机生成过程展开，优化内部选择

若生成过程每一步有明确的随机转移密度，可以记录中间变量，计算各步条件密度。策略梯度就可以评价“产生这次最终动作的这条生成路径”。

DPPO，即 Diffusion Policy Policy Optimization，扩散策略的策略优化，将去噪过程与环境决策结合，用可计算的生成转移连接策略更新。它的关键不是把去噪误差叫作概率，而是改变优化对象的表示。DPPO。

一般原理是：内部路径改变最终动作，最终动作改变任务结果，所以奖励可以训练内部随机选择。但记录一条路径的联合概率，并不等于计算出了积分掉所有其他路径后的最终动作边缘概率。路径级 PPO 裁剪也不等价于对最终动作分布施加同样的硬限制。

因此必须保存并重算相同条件、相同随机变量和相同采样机制。更新后只重新随机生成一条新路径，不能冒充原路径的新概率。

路线二：给 Flow 定义适合优化的随机机制

Flow 常通过常微分方程，Ordinary Differential Equation，缩写 **ODE**，从初始噪声确定地积分到动作。随机性可以只在初始噪声中；这不意味着每个中间转移都有可直接套用高斯密度的随机核。

有些方法在 Flow 的生成过程中加入明确的随机机制，使每一步的变化都有可计算的条件密度，从而连接策略更新。**随机微分方程**（Stochastic Differential Equation，SDE）是在连续演化中加入随机扰动的一种描述。πRL 讨论 Flow-Noise 与 Flow-SDE 两类设计；理解时要具体追踪噪声加在哪里、改变哪个随机决策，以及怎样记录新旧概率。πRL。

建模上需要问：新增噪声怎样改变动作分布？探索会不会破坏原动作结构？训练与部署的生成过程是否一致？“能计算某个噪声密度”只有在该噪声确实定义了行为策略时，才能服务相应更新。

路线三：保留生成训练接口，用质量信号改造目标

一条直观路线是用优势给扩散或 Flow 监督损失加权。高质量动作被更强地拟合，低质量动作贡献更小。这避免直接计算复杂最终密度，但仍依赖价值或回报估计，并主要在数据支持范围内学习。

另一类工作尝试从生成损失构造策略优化代理。Flow Matching Policy Gradients 使用 FPO 缩写；另有一篇 VLA Flow 强化微调论文也使用 FPO。缩写相同不意味着推导和算法相同。Flow Matching Policy Gradients，Reinforcement Fine-Tuning of Flow-Matching Policies for VLA。

采用代理目标时，需要说明它怎样连接最终生成分布与策略改进。**流匹配**误差评价局部速度，概率比评价同一随机对象在新旧分布中的相对密度；二者之间需要相应推导。

路线四：冻结大生成器，只学怎样给它选噪声

设预训练生成器为 $b = F(x, z)$。冻结 F，另训练小策略选择**潜变量**或噪声 z。小策略的输出影响生成动作，因此可以根据最终任务反馈学习。

这个思想像同一台已经会生成多种路线的机器，改变选择哪种路线的入口。DSRL 是 Diffusion Steering via Reinforcement Learning，强化学习扩散引导；其论文标题为 Steering Your Diffusion Policy with Latent Space Reinforcement Learning。DSRL。

需要直接理解的机制是“改变生成器输入分布，从而改变动作分布”。它可以减少需要训练的大模型参数，但仍受冻结生成器可达行为的限制。若生成器根本没有产生稳定抓取的能力，仅调整入口未必能得到该技能。

该怎样选择，而不是怎样记住更多缩写

你的条件	可以优先考察的路线	关键代价
生成随机核明确，能保存路径	路径策略优化	路径存储、训练时延、概率一致性
Flow 输出，愿意定义随机机制	明确的随机 Flow 优化	分布变化与探索设计
有较好固定数据和质量估计	优势加权生成训练	数据覆盖与权重偏差
希望借助生成损失代理优化	有对应推导的代理方法	代理目标与真实目标的差距
大模型昂贵，已有能力较强	冻结生成器、训练潜变量策略	可达行为与小策略表达

这些选择没有通用优先级。先写出采样过程和实际动作，再确认训练信号、概率对象及更新范围，通常能排除很多表面上相似、实际不适用的方法。

3.8.3 Flow VLA 怎样从历史数据走向经验改进

先选择一条明确路线

本节展示“质量估计 + 生成式动作训练”的教学流程。它不需要把所有 Flow VLA 都改造成 PPO，也不是某篇论文的逐项复现。

我们有预训练 Flow VLA，以及包含成功、失败和人类恢复的机器人轨迹。希望保留已有动作表达，同时更偏向长期效果好的行为。策略训练仍使用第1.10节的路径速度标签；新增的质量信号决定哪些动作得到更强监督，或作为额外条件区分希望生成的行为。

给历史行为估计质量，先说明在评价谁

从轨迹计算回报，可以训练输入任务观测的**价值模型**。若轨迹来自许多不同策略，人类与机器人混合，直接拟合得到的是数据后续行为对应的回报规律，不等于当前 VLA 的精确 V^{π}。

若采用离线 Q 方法，还要按其算法定义构造**动作价值**与基准；如果采用实测回报减价值，得到的是轨迹相对质量估计。选择哪一种取决于数据条件，不能给所有信号都贴上同一个“真实优势”标签。

为使例子容易理解，假设三个类似输入下的数据动作质量信号为 2、0、−2，取 $\beta = 2$，指数权重约为 2.718、1、0.368。它们表达相对学习强度，实际需限制极端权重和估计噪声。

权重怎样进入 Flow 训练

对每条已有动作块 b，采样噪声与生成进度，构造 z_u 与目标方向。加权目标为

$$L_{\text{quality}} = \mathbb{E}_{\mathcal{D},z_0,u}\left[w(x,b)\|v_\theta(x,z_u,u) - (b - z_0)\|^2 \right].$$

网络输入输出并未改变，动作标签仍来自数据。改变的是哪个样本更强地影响共享参数。同一条轨迹得分很高，可能因任务容易，因此权重必须控制任务条件与状态难度，不能只按所有轨迹总分排序。

权重过尖会让少量高分样本主导梯度，高分估计错误也会被放大。ARFM，即 Adaptive Reinforced Flow Matching，自适应强化流匹配这一方法标识，研究 Flow 离线后训练中的优势信号与训练方差问题。理解其设计前，先理解上述权重怎样改变梯度构成；具体自适应规则应查原文。ARFM。

质量条件化为何能保留不同质量数据

除了给样本加权，还可以增加质量标签条件 I。训练时，高质量样本带 $I = 1$，较低质量样本带 $I = 0$，模型学习两类条件下的动作分布。部署时请求 $I = 1$，相当于让它生成学到的高质量行为模式。

这不是部署时先知道未来动作会成功；$I = 1$ 是对生成行为类别的请求，类似“按熟练模式来”。其有效性取决于高质量数据覆盖和模型是否学到可泛化差异。某状态只有失败数据，条件标签不能凭空补出正确动作。

如何定义质量标签、是否连续条件、阈值如何设置、如何利用纠正数据，都会影响效果。它不是简单的“把成功标签加进去就能得到最优策略”。

回报分布输出为什么还是价值模型

价值网络也可以输出回报区间的概率，而不直接回归一个标量。教学例子有三个代表值 0、5、10，预测概率为 0.2、0.3、0.5，则平均预测为 6.5。真实回报落在 10 对应区间时，可以用分类**交叉熵**训练。

分类对象是回报区间，不是动作类别；输出概率也不是三个策略的投票。这个表示可以描述回报的不确定性或多峰，但需要注意分箱范围与**校准**。

RECAP 用价值估计、**质量条件化**及多来源经验连接 VLA 改进，价值输出采用回报分布表示。这里的三个区间是本书示例，不是论文实现参数。RECAP / $\pi^*_{0.6}$。

在线接续不是只把新数据扔进旧数据池

开始交互后，新增轨迹反映新策略实际访问的状态。需要重新评价数据质量、处理人工干预、控制旧新数据比例，检查策略偏离后价值估计是否仍适用。

如果走异策略 Q 路线，经验可以复用，但新动作价值的可靠性要持续校正；如果走同策略概率优化，必须按对应采集与概率定义组织数据。不同路线不能仅因都叫“在线接续”就混用标签与损失。

ConRFT 使用一致性策略等结构连接离线与在线学习。这里的一致性策略，consistency policy，学习让同一生成路径中不同噪声层级映射到一致的干净动作结果，追求较少生成调用；不是要求机器人连续两步输出相同动作。网络可接收任务条件、含噪动作与噪声层级，输出干净动作估计。数据动作提供监督参照，价值评价可对生成动作提供改善方向；这两类信号分别保持已有行为与追求较好结果。具体一致性训练、Q 引导和损失权重需按论文核对。ConRFT。

HIL-SERL，即 Human-in-the-Loop Sample-Efficient Reinforcement Learning，人在回路中的样本高效强化学习，提供真实机器人示范、奖励分类与接管闭环的参考。它与 ConRFT 并非同一个 VLA 结构。HIL-SERL。

怎样验证质量信号确实有用

至少比较等数据继续监督训练、只保留成功数据的模仿、你的质量估计方法。若主张长期评价优于终局成功标签，要控制数据量，比较同样轨迹上的标签与权重。

打乱质量标签后效果不变，可能表示真正作用来自新增数据、额外训练或模型容量。只有模块加入后分数上涨，还不足以说明长期**信用分配**成立。

3.8.4 路径概率与最终动作概率，差在是否合并了其他路径

用一个离散教学生成器说明这个区别。内部有路径P1、P2、P3，P1与P2都生成最终动作A，P3生成动作B。旧路径概率是 $(0.4, 0.4, 0.2)$，新概率是 $(0.7, 0.1, 0.2)$。最终动作分布在更新前后都是 $(0.8, 0.2)$；但若记录的是P1，其新旧比值为1.75，记录P2则为0.25。

一般地，路径联合密度由初始随机量和各步随机核相乘得到；最终动作密度还要对未保留的内部变量求和或积分。本例中 $\pi(A \mid s) = p(P1 \mid s) + p(P2 \mid s)$，已经清楚显示：改变哪条路径更常用，不一定改变最终动作比例。因此路径级裁剪、动作级裁剪和直接使用生成误差，是不同的优化对象。

如果算法把内部路径明确当作随机决策，最终奖励可以沿这条路径训练相应选择。这要求记录真实采样的中间变量，在新参数下评价同一条记录路径；重新采一条新路径不能代替这个重算。确定性 ODE 的一步更新没有自动获得高斯随机核，不能给它随意套一个转移密度。DPPO 作者说明与实现入口

扩散噪声损失和 Flow 速度损失则评价局部监督预测。它们不包含上述对全部生成路径的边缘化过程，取负号不等于最终动作 log 密度。若选择质量加权监督，应明确是在改变哪些数据动作的拟合强度；若选择路径策略优化，应明确随机变量与奖励映射。两条路线都能研究，所需证据和实现接口不同。

3.8.5 练习题

题3.8-1 扩散或流匹配训练的**均方误差**，为什么不能直接取负号当最终动作 logπ 接入 PPO？给出两种有明确目标的替代路线。

3.8.6 参考答案

题3.8-1答案 PPO 的比值需要同一个已执行动作在新旧策略下的归一化概率密度。扩散噪声均方误差或流速度均方误差，比较的是随机中间级别上的预测与人工构造标签；它随采样级别、噪声及权重变化，并非最终动作在策略下的精确 logπ。取负号不能补上概率归一化，也不能保证新旧指数之比是正确的动作概率比。

一条路线是把随机扩散生成过程的整条去噪链作为扩展的策略决策过程，为每个随机转移定义明确的条件密度，用链上实际采到的转移计算概率比，并把最终动作执行后的回报用于更新。此时优化对象是明确规定的随机链，不能随意把确定性流积分当成具有同样密度的转移。

另一条路线保留原生成监督目标，以明确的质量权重做回报/优势加权拟合：从真实经验取得动作标签与经核验的权重，让好动作得到更强噪声或速度监督；也可做优势条件生成，部署时选择高优势条件。它们是加权模仿或条件策略改进，不应名为 PPO。无论选择哪条，都要明确实际生成程序、数据分布和损失改变的对象。

3.9 奖励模型、偏好、纠错与策略蒸馏

3.9.1 奖励模型与偏好，怎样把目的变成反馈

奖励函数、奖励模型与价值网络

在仿真里，可以读取方块坐标判断是否进盒。真机中可能只能看图像，需要一个模型判断完成程度。把感知输入映射为任务反馈的模型，通常叫**奖励模型**，reward model。

奖励模型可以输入图像或视频加语言指令，输出成功评分、进度等级或事件分数。训练标签来自人工标注、可验证规则、任务结果或偏好比较。它评价哪些内容，取决于标签设计。

价值网络也输出一个分数，但预测对象是“从这里按相应后续行为继续的累计反馈”。一张图显示方块已经被拿起，奖励模型可能判断“抓取阶段完成”；价值网络还要考虑当前策略能否搬运并放入。因此二者不是简单换个名字。

一个成功分类奖励模型怎样训练

收集任务结束的视频片段，人工核查成功与失败；模型输入片段和指令，输出成功概率，使用二分类损失训练。部署训练中，可以将分数阈值化成终局奖励，或者采用另有依据的反馈转换。

关键并不只有分类准确率。若模型学到“夹爪在盒子上方就算成功”，它可能漏掉方块仍被夹着的失败。需要专门构造近成功但未完成的负例：放错盒、未松手、方块掉在盒外、拿错物体、完成后再掉出。它们检验任务语义而不只是图像风格。

RoboReward 是机器人奖励模型的研究入口；SOLE-R1 考察视频语言推理反馈。它们可帮助理解反馈学习，但应结合部署行为检验反馈分数与真实完成条件的对应关系。RoboReward，SOLE-R1。

为什么策略会利用奖励模型的错误

普通分类评估只考察固定数据。**RL** 则让策略持续寻找高分行为，数据会被主动推向奖励模型的高分区域。如果模型在那里有漏洞，策略可能放大漏洞。这叫奖励投机，reward hacking。

解决思路包括独立核查真实结果、补充错误高分负例、将完成验证与过程评分分开、检查视觉遮挡与时序一致性。多加一个模型不自动消除问题；如果两个模型共享同样偏差，仍可能互相确认错误。

语言模型生成奖励代码，是另一种机制。Text2Reward 和 Eureka 让语言模型协助设计可执行奖励函数；这与“每次看视频输出分数”的奖励模型不同。前者最终执行的是代码定义的度量，后者执行的是学习到的判断。Text2Reward，Eureka。

偏好比较在学习什么

有时人不愿精确打分，但能判断同一任务下两段行为哪段更好：都成功，哪段更平稳？都失败，哪段更接近正确恢复？偏好数据表达的是相对质量。

一种做法是先训练评分模型。对同一任务下更受偏好的片段 τ^+ 和较差片段 τ^-，令偏好概率为 $\sigma(R_\psi(\tau^+) - R_\psi(\tau^-))$，再用偏好标签训练二分类损失。模型学习的核心是分数差，而非人工给出的绝对奖励。训练好后，可把它的评价接入策略优化；若想把片段评分分解成逐步奖励，还要明确这层建模。另一种做法直接训练策略的相对输出概率，直接偏好优化（Direct Preference Optimization，DPO）就属于后一条路线。从人类偏好学习奖励。

DPO（Direct Preference Optimization，直接偏好优化）把偏好学习写成策略自身的分类目标。设同一条件为 c，优选输出为 y^+，劣选输出为 y^-，参考策略为 π_{ref}，一条样本的目标为

$$\ell_{\text{DPO}} = -\log \sigma\left(\beta\left[\log \frac{\pi_\theta(y^+ \mid c)}{\pi_{\text{ref}}(y^+ \mid c)} - \log \frac{\pi_\theta(y^- \mid c)}{\pi_{\text{ref}}(y^- \mid c)}\right]\right).$$

括号比较的是：相对于参考策略，新策略对优选输出的倾向增长，是否超过对劣选输出的倾向增长。σ 是 sigmoid，将分数差映射为偏好概率；$\beta > 0$ 来自经典推导中的参考 KL 约束系数。这样同时利用“哪一个更好”与原模型的概率信息。经典推导把带参考约束的奖励优化与偏好概率连接起来，训练无需先单独拟合一个奖励网络。它仍需要可用的输出概率及合理偏好数据；机器人中的输出究竟是动作块还是轨迹，要按下一段进一步定义。DPO。

在机器人里，需要先明确“输出”是什么：同一观测下的一个动作、**动作块**，还是从同一起点开始的一段轨迹。两条来自不同初始状态、不同后续策略的轨迹，结果差未必能归结为被比较的动作。文本偏好的公式不能自动解决机器人环境变化与长程**信用分配**。

哪种反馈适合哪种问题

如果最终结果能可靠验证，终局规则奖励通常最容易解释；若成功非常稀少，可考虑对探索有帮助的过程反馈，并检验是否改写目标；若关心风格与舒适度，偏好可能更合适；若没有可靠评分器，就应先解决反馈验证，而不是先优化复杂策略。

反馈的评价应包括错误类型和影响。把成功误判为失败会损失信号；把失败误判为成功可能直接训练错误行为。两者的系统代价不一定对称。

3.9.2 人类纠错与教师蒸馏，补充的是什么信息

人类可以提供动作，也可以提供评价

机器人抓偏，人类接管并重新对齐。这个过程产生至少两种可能信号：正确恢复动作的监督，以及干预发生位置的信息。二者并不等于一个明确奖励。

把纠正动作加入监督训练，教模型“在自己会犯错的状态里怎样恢复”；根据干预次数设计惩罚，则是在改变任务反馈。必须说明哪一种被算法实际使用。

DAgger 为什么让学生先去走自己的轨迹

DAgger 是 Dataset Aggregation，数据聚合。一种基本流程是：用当前学生策略交互，取得它实际会遇到的状态；让专家在这些状态提供动作标签；聚合到数据集并重新训练学生。

它针对模仿学习的**分布偏移**：专家轨迹上的状态可能很干净，学生一犯错就进入未被覆盖的状态。补充学生访问状态上的专家标签，比只重复学习专家成功轨迹更直接。

它仍是以动作标签为核心的模仿方法，而非仅靠奖励的 RL。优势是纠错方向明确；代价是专家标注和执行能力，且理论条件与真实系统的有限采样仍有差别。DAgger 原论文。

教师蒸馏怎样利用更强模型

蒸馏 distillation 是让学生模仿教师输出。教师可以更大、更慢、拥有特权信息，或者结合规划与搜索。学生输入实际部署信息，输出较便宜的动作策略。

如果教师是概率策略，可以训练学生分布接近教师分布，例如最小化教师到学生的 **KL**；如果教师给动作标签，可以用动作监督损失。训练目标是保留教师行为的有用信息，不是让学生理解教师每个内部步骤。

在学生自己采集的输入上查询教师，叫同策略蒸馏的一种思路。**VLA**-OPD 中 OPD 是 On-Policy Distillation，同策略蒸馏。它把教师监督放到学生实际访问的轨迹上，帮助连接离线微调与在线改善。VLA-OPD。

为什么不能混淆人类动作与策略采样

普通 **PPO** 的概率比针对旧策略实际采样的动作。如果机器人动作被人类替换，再把接管动作当作原策略采样，并配上原策略概率，就改变了行为分布。

常用处理是分开保存自主轨迹与干预数据：前者进入对应 RL 流程，后者做监督纠错或由明确处理行为差异的算法使用。终止、奖励和重置也要记录真实干预边界。

人工接管可以避免浪费昂贵失败回合，也能提供恢复数据，但不能用它掩盖自主能力评估。测试应说明是否允许接管，人工完成是否计为成功。

多种信号可以组合，但每项应有明确职责

一种系统可以用终局反馈优化完成率，用人类纠错保持恢复能力，用教师蒸馏降低部署成本。组合本身有研究空间，关键是明确哪个机制解决哪个已观察问题，并通过对应对照验证。

如果三种信号都来自同一位教师，最终提升可能主要是更多教师监督，而不一定来自奖励信用分配。需要同数据监督基线和信号替换实验。合理的叙事来自可检验的必要性，而不是堆积模块名字。

3.9.3 DPO 为什么能把奖励优化改写为策略的偏好分类

第3.7节已经推过“高奖励加参考分布约束”的最优分布。把行为分布替换为固定参考策略，同样得到 $\pi^*(y \mid c) = \pi_{\text{ref}}(y \mid c) e^{r(c,y)/\beta} / Z(c)$。反过来整理奖励，

$$r(c, y) = \beta \log \frac{\pi^*(y \mid c)}{\pi_{\text{ref}}(y \mid c)} + \beta \log Z(c).$$

偏好模型只需要同一条件下两个输出的奖励差，$\log Z(c)$ 在相减时消失。于是可以直接用可训练策略替代 π^*，把两个参考对数概率比之差当作分类分数，得到前面的 DPO 损失。这条连接使训练无需先拟合一个单独的奖励网络，但仍需要偏好标签和可计算的输出概率。RLHF 教材：直接偏好优化的推导

一个重要区别是，DPO 的偏好概率不等于策略采样优选输出的概率。设参考策略对优选、劣选输出给 $(0.8, 0.2)$，新策略给 $(0.6, 0.4)$，取 $\beta = 1$。分类分数为 $\log(0.6/0.8) - \log(0.4/0.2) = \log 0.375$，偏好概率约为0.273。尽管新策略仍更常输出前者，它相对参考策略却降低了优选、提高了劣选的倾向，所以这条优选标签要求继续修正。

对分类分数 z，优选标签损失 $-\log \sigma(z)$ 的导数是 $\sigma(z) - 1$。更新提高优选相对参考的对数倾向，降低劣选的相对倾向；共享参数和其他样本仍决定实际总体变化。这里不是承诺每个输出的绝对概率都单调上升或下降。

经典推导以同一条件下可比较的输出为基础。机器人若比较两段环境交互轨迹，需要交代初始状态、真实转移、行动概率和评分条件；Flow 模型若没有现成动作密度，也不能仅把误差代入此式。理解这条推导，能帮助判断一个“机器人 DPO”究竟保留了哪些假设、又用了什么近似。TRL 的 DPO 实现说明

3.9.4 练习题

题3.9-1 成功判定模型把“夹爪在盒子上方”当成功，策略会如何利用它？怎样补数据与评价，才能区分评分器修复和策略真实改善？

3.9.5 参考答案

题3.9-1答案 策略会寻找容易触发高分的行为，例如把夹爪停在盒子上方、做出放入动作的外观，却不把物体放稳。奖励模型学到的是视觉相关线索，策略优化则会放大这条线索，造成分数上升但任务没完成。

补数据时加入难负例：“夹爪在上方但没抓到物体”“放入后又掉出”“物体在盒沿”等，并保留完成后夹爪离开、物体稳定等正例。必要时提供时间片段、目标物体状态或独立传感信息，令判定依据更贴合真正成功条件。保留一批不参与训练的困难样本检验评分器泛化。

评价上做两步区分：先固定同一批轨迹，比较新旧评分器与人工/独立传感真值的一致性，证明评分器是否修复；再重新训练或运行策略，用独立成功判据比较真实完成率，证明策略是否改善。若只换评分器，给同一策略重新计分，不能把分数变化当成能力变化。还要检查策略是否发现新的误判方式，而非只堵住一个固定漏洞。

3.10 探索、目标重标记与受约束的经验采集

3.10.1 探索不是随便加大动作噪声

奖励稀疏时，机器人可能很少第一次成功，无法判断哪类行为值得重复。探索的目标是获得能区分决策效果的新经验。平移方向加随机噪声可能扩大状态覆盖，也可能破坏已学会的接触序列；在动作维度很多时，独立随机扰动很难产生协调的抓取。

可以从成功示范开始，用较小扰动学习恢复；按目标或技能组织探索；通过课程学习逐步增加距离、遮挡和接触难度；使用不确定性指导补充数据。熵鼓励策略保持多样性，却不等于针对未知动力学的有效信息增益。评价探索应同时报告成功发现次数、交互数量、风险事件和最终策略能力。

3.10.2 HER 把已经发生的转移换一个目标解释

假设希望把杯子推到位置 g=0.8，实际只推到 0.5。原任务奖励为失败；若任务族允许目标位置变化，就可把这段真实轨迹的**目标重标记**为 $g' = 0.5$，重新计算每一步奖励，并把策略和评价器条件改为新目标。这叫事后经验回放（Hindsight Experience Replay，HER）。它不声称原任务成功，而让算法学会“怎样达到一些实际到过的位置”。HER 原论文研究稀疏目标奖励下的这种利用方式。

重标记要求：给定同一物理状态和同一动作，改变目标标签后，记录的下一状态仍服从同一套转移规则。目标可以改变策略本来想选的动作，却不能暗中改变环境动力学。例如“推到左边”和“推到右边”会引导不同选择，但已发生的一次右推，仍可作为左目标下的失败转移；HER不要求重标记动作对新目标有利。奖励、成功标记、目标相关观测和终止条件都应按新目标重算。对于“倒空杯子”与“保

留液体”，必须记录判断新任务所需的液体状态或过程信息，不能只换指令而保留旧反馈。如果目标还改变物体质量、门锁状态等实际转移规则，就不能直接复用旧转移。未来状态取作目标在随机环境中还可能引入选择偏差，因此不能把所有目标重标记都称为严格无偏。

3.10.3 约束需要独立于任务得分定义

把碰撞、超力或越界记为成本，便可以在优化任务回报 $J(\theta)$ 时另要求期望累计成本 $C(\theta) \le d$。拉格朗日方法使用 $J(\theta) - \lambda(C(\theta) - d)$，其中 $\lambda \ge 0$：策略提高回报并减少被加权的成本，乘子则随预算违反程度调整。例如观测到成本超预算时，可按 $\lambda \leftarrow \max(0, \lambda + \eta(\hat{C} - d))$ 增大惩罚；成本较低时减小它。与固定碰撞惩罚相比，惩罚强度会响应约束违反。但估计的是平均累计成本，仍要单独检查单次峰值受力等要求。

运行时过滤器可以修改候选动作，例如将越界增量投影回允许集合。此时真正执行的动作与原策略采样不同。记录必须同时保存提议动作、执行动作、过滤原因和策略版本；算法使用的概率也需与随机变量定义一致。若过滤器掩盖了大量错误，表面成功率不能证明策略自身可靠。

3.10.4 一套可验证的采样顺序

先在低风险环境验证动作范围、时钟和紧急停止；再收集短任务和恢复轨迹；逐步开放更多初始条件；每轮评估自主完成、人工接管与成本事件。将探索方式与普通噪声基线在相同交互预算下比较，再做课程顺序和示范起点的消融。安全问题应落到可记录事件和明确约束，避免用“训练稳定”代替物理风险评价。

3.10.5 练习题

题3.10-1 事后经验回放（Hindsight Experience Replay，HER）把未达到原目标的轨迹重标为“达到后来实际到达的状态”。哪些量必须一起重算？什么任务不能只替换一句目标文字？

3.10.6 参考答案

题3.10-1答案 目标重标记后，凡依赖目标的量都应保持一致：状态/观测中的目标字段、策略和价值输入、每步奖励、成功标记，以及由目标决定的终止条件和掩码。动作与已发生的物理状态转移保持原值。若成功即终止，改目标后可能更早成功，重标记序列也需按该语义截取，不能把成功后的动作继续当作同一回合。

例如原目标位置 g 没达到，却到达了位置 g′，可用同一实际轨迹重新计算是否接近 g′、相应距离奖励及成功条件。它告诉模型“这些动作确实能到达 g′”，没有声称完成了原目标 g。

只替换一句文字对依赖完整过程的任务往往不成立，例如“先开门再把杯子放入”“抓取后不得碰墙”“不得弄湿物体”。是否完成不由最终位置一个变量决定，需要保持阶段、约束和历史判据。HER 还要求能计算新目标下的奖励，且物理转移本身不因隐藏的目标机制改变；若目标会改变环境规则，不能直接复用同一转移。

3.11 完整后训练循环与独立能力验证

3.11.1 把一轮 PPO 后训练从头走到尾

先写清系统里的四个对象

我们已有一个能完成部分抓取任务的 **VLA**。为方便教学，设它有可计算的随机动作概率，并每个决策执行一个控制步。实际 token 或生成式动作要按第1.9、3.2、3.8节定义概率对象。

这一循环包含策略、**价值网络**、环境或奖励器，以及轨迹缓冲区。策略选择动作，价值预测长期回报，反馈提供训练证据，缓冲区保存这些记录供后续更新。

本节只展示一条成功轨迹，便于手算；真实训练要用多条成功、失败和恢复轨迹。单条轨迹不能识别所有动作的真实优势，也不足以可靠优化大型模型。

采集时记录什么，为什么记录

把本轮采集策略固定为旧策略。每一步保存：准确输入、实际采样动作、旧动作对数概率、奖励、旧价值预测、下一输入和结束类型。图像预处理、语言条件、历史和动作生成设置也必须可追溯。

旧概率不只是日志信息。更新时分母需要它来判断行为倾向改变了多少。旧价值用来计算这一批固定优势和训练标签；若在每个小批次中随意用更新后的价值重算全部标签，算法语义与稳定性会改变。

示意记录如下。表中的概率来自简化离散动作，连续策略对应密度。

阶段	已采样动作的旧概率	奖励	旧价值	结果
抓取	0.2	0	2	任务继续
搬运	0.3	0	4	任务继续
放入	0.4	10	6	真实结束

从反馈生成两套不同的训练标签

采用第3.3节的 $\gamma = 0.9$、$\lambda = 0.8$，**TD** 误差为 1.6、1.4、4；**GAE** 优势为 4.6816、4.28、4；价值目标为 6.6816、8.28、10。

策略用的是“比预期好多少”，价值网络用的是“未来总分应该预测到多少”。这两组标签不同，不能交换。优势可以为策略优化进行标准化；价值目标保留回报原单位。

最后一步成功的意外好结果传回前面，使抓取和搬运也获得正信号。但它并未证明三个动作同等关键。价值估计、失败对照和更多样本将逐步改善**信用分配**。

固定样本，重算新概率，更新策略

设某次更新后，相同记录动作的新概率为 0.22、0.36、0.50，比值分别是 1.1、1.2、1.25。取 $\epsilon = 0.2$，三个正优势样本的裁剪目标贡献为：

阶段	概率比	优势	PPO 样本贡献
抓取	1.1	4.6816	5.14976

阶段	概率比	优势	PPO 样本贡献
搬运	1.2	4.28	5.136
放入	1.25	4	4.8

最后一个样本虽然比值为 1.25，但只按 1.2 给收益。优化器通过这些贡献对共享网络求梯度；不是把表里的新概率直接写入模型。这里的新概率是假设值，用来展示损失如何判断变化。

实际训练中，样本被分成小批次，mini-batch，多次经过网络。一次遍历该批数据叫一个 epoch，训练轮次。多训练几轮可以复用采集成本，但继续训练过久会让新旧策略偏离，优势也越来越不能描述当前策略。需要按对应算法监测更新幅度。

训练价值网络，目标为什么不随它一起移动

初始预测 2、4、6，与上述目标的平方误差分别约为 21.9174、18.3184、16。价值梯度推动它更准确地估计类似输入下的后续回报。

假设后来预测变为 3、5、7，三个误差会减小，但这只是方向演示。优化器更新共享参数，真实输出不保证每个样本都单调改善。多条数据可能有相互冲突的标签。

价值目标通常在这一批训练中固定或按明确规则生成。否则网络一边改变预测，一边把同样变化写入目标，损失看似容易下降，却没有稳定的监督含义。

下一轮必须重新面对自己的新行为

本轮更新结束，保存检查点 checkpoint，即模型参数及必要训练状态快照。新策略去采集下一批数据；这时它可能抓得更稳，也可能到达此前没见过的搬运状态。下一轮旧策略就指这个新采集版本，而不是最初预训练模型。

一轮后训练：数据变成标签，标签推动两个网络

旧策略在一轮内固定；更新后重新采集，才能学习新的行为后果。

图3.9：**PPO** 一轮的数据、标签、更新和重新采集。

完整闭环可以读成：

固定本轮采集策略与生成设置
→ 用它交互，保存输入、动作、旧概率、奖励、价值与边界
→ 在回报原尺度上计算优势和价值目标
→ 用固定记录动作更新策略，用回报目标训练价值网络
→ 检查概率一致性、更新幅度和独立任务结果
→ 保存模型，使用更新后的策略重新采集

检查不是额外的形式流程，而是确认训练信号的语义。相同参数重算概率比不接近 1，意味着新旧概率很可能没有针对相同随机对象；奖励与视频不对应，意味着动作在学习错误结果。

3.11.2 从训练到真实机器人，还缺哪些环节

算法里的动作必须真的能被执行

确认输入图像尺寸、相机顺序、语言模板、状态单位与训练匹配；确认输出动作坐标、尺度、夹爪定义与控制器匹配。模型参数正确，而动作接口错，仍会表现为任务失败。

控制频率与端到端延迟决定机器人多长时间能纠正一次。一次前向只耗时很短，但如果图像传输、生成多步和动作下发占据大部分时间，整体系统仍可能反应慢。

仿真到现实的差异，应映射到具体机制

Sim-to-real 指仿真到真实系统的迁移。差异包括视觉外观、相机标定、摩擦、接触、控制延迟与物体柔顺性。域随机化，domain randomization，在训练中随机改变相关条件，让策略不只适应某组仿真参数。

随机化不是越多越好。超出真实分布太远，可能增加学习难度；没有覆盖关键接触误差，背景变化再多也无济于事。应根据实际失败定位需要覆盖的变化。

如果策略在仿真通过错误物理漏洞获得奖励，视觉随机化不能修正这个问题。需要独立验证物理过程。

训练时探索与测试时执行如何对应

训练为了收集信息可能采样随机动作；测试可能用均值、低温度采样或固定噪声。温度 temperature 调节某些概率分布的集中程度，是采样设置的一部分。

测试确定性动作不必等于随机训练分布的平均性能。生成模型的平均动作也可能落在不合理的模式之间。应报告采样方式，并验证部署设置下的实际效果。

对生成式 **RL**，改变生成步数、噪声方差或随机核可能改变策略本身。降低推理成本是合理优化，但需要重新测量，不能默认行为分布保持不变。

旧能力怎样保留

后训练可能提高一个任务，却损害其他物体、指令或动作风格。原因可能是新数据窄、奖励过强、共享参数迁移或动作分布过度集中。

可以混合旧数据、限制更新范围、使用参考策略约束或采用多任务训练，但每种方法都有代价。评估应有旧任务集合，并区分真正遗忘与输入或执行设置变化。

成本应覆盖完整闭环

报告环境控制步、回合、真实交互时间、人类标注与接管时间，以及训练和推理计算。并行仿真能缩短墙钟时间，wall-clock time，即实际经过时间，但总交互量可能很大。

一种方法只训练小适配器，另一种全量训练；前者可能省训练显存，却不一定更省交互。论文若主张“高效”，应说明高效指什么资源，并在该资源下比较。

3.11.3 怎样知道模型真的学到了，而不是评分变高

先把主张改成可测问题

“后训练更强”过于宽泛。可以改成：固定新增数据量，完成率是否提高？在相同交互预算下，是否更快学会恢复？对未见位置是否更稳定？旧任务下降多少？

研究指标应对应任务定义。成功率看最终完成；完成时间看效率；干预率看自主性；碰撞、掉落和错误目标看失败类型。平均奖励可以作为训练指标，但不是这些结果的替代品。

测试分布如何检验泛化

训练与测试最好区分初始位置、物体、场景和语言条件，并说明哪种变化才是本研究目标。多个随机种子 random seeds，是不同随机运行配置；它们帮助观察训练偶然性。一次训练很多回合的置信区间，不替代多次训练之间的差异。

随机初始化、探索、数据抽样和环境条件都影响结果。条件允许时，给出运行间变化和任务级结果，而不只一个总体均值。小样本真机试验应明确样本数量与不确定性，不把小幅差异叙述为稳定规律。

基准选什么，取决于要验证什么

Benchmark 是统一任务与评价协议的基准。它有助于比较，但不自动覆盖你的研究机制。

基准	可以帮助检验的方向	需要额外说明
LIBERO	多任务操作与知识迁移	使用的任务套件、示范与训练拆分
RoboTwin 2.0	双臂操作与随机化场景	物体、随机化设置、数据和执行协议
CALVIN	语言条件下的长程任务链	链长、起点、语言拆分与成功定义
SimplerEnv	在仿真协议中评价真实机器人策略	对应机器人配置及模拟现实差异

原始入口：LIBERO、RoboTwin 2.0、CALVIN、SimplerEnv。这些是不同研究问题的入口，不是一个可以按分数直接横向排行的统一榜单。

公平基线是在排除什么解释

Baseline 是对照方法。你提出 RL 后训练，应至少问提升是否来自 RL 信号，而非更多数据、更多训练或更大模型。

同新增数据继续 **SFT**，排除“只要再训练就好”；成功筛选 **BC**，排除“只学成功就足够”；相同奖励的已有算法，比较更新机制；相同模型、更新范围与交互预算，控制容量和成本。

没有能力复现所有方法时，优先选择能排除核心替代解释的强对照，并准确说明比较边界。不是列的算法越多，证据就越强。

消融实验不只是删除模块

Ablation 是消融：改变一个关键设计，观察结果。删除价值网络导致系统根本无法训练，不一定能证明你的价值估计机制比其他估计好。更有信息的对照可能是等算力**价值模型**、简单回报基线或相同数据的随机权重。

假设主张“GAE 帮助长程信用分配”，可比较不同任务长度和稀疏程度下的估计方差、学习速度与最终效果。假设主张“**奖励模型**降低真机成本”，应比较相同真实验证标准下的交互成本及误判行为。

模块有用、机制解释成立、系统比别人好，是三种层次的结论。实验应支持论文实际声称的层次。

3.11.4 把概念变成自己的理解

第一阶段：不看公式，讲清整条链

拿抓取放置例子回答：策略输入什么、输出什么？奖励在什么事件后产生？为什么前面的零奖励动作也可能好？V 依赖谁的后续行为？Q 如何固定第一步？A 相对谁比较？

如果卡住，回到对应概念的现实问题，不必重复背定义。能说出“价值网络预测当前策略从这里继续的累计回报”，比只说“V 是**状态价值**函数”更完整。

第二阶段：手算一个闭环

算三步奖励的**折扣回报**；用 2、4、6 的旧价值算 TD 与 GAE；区分优势和价值标签；给正、负优势各算两个 PPO 比值；解释为什么裁剪不锁住概率。

小规模手算可以把奖励、价值标签与概率比的对应关系显露出来。更有价值的检查是解释每个数来自哪条转移、改变了哪个训练目标，再将同一关系放进程序。

第三阶段：画出一个 VLA 的实际接口

选择你研究中会用的模型，标出图像、语言、状态、动作空间、生成随机性、预测与执行块长度。问它是否能计算精确动作概率，能否保存生成路径，价值网络怎样获得任务条件。

此时再选后训练算法。不要先决定必须用 PPO，再把不对应的生成误差塞进概率接口；也不要只因一个方法适配 Flow，就忽略它所需数据和反馈条件。

三个有明确学习目的的小实验

第一个实验检验优势的比较基准。用一个小离散环境改变初始任务难度，比较原回报和减去状态基线的信号。观察是否降低更新波动，而不是只看一次最终得分。

第二个实验检验**自举**的取舍。保持数据、模型和奖励不变，比较完整回报、一步 TD 与不同 λ 的 GAE。看长任务和价值预测误差如何改变效果。没有通用最优 λ，这个实验帮助理解条件。

第三个实验检验生成式质量训练。固定同一数据集，比较均匀监督、成功筛选、质量加权与打乱质量权重。若研究目标是 VLA，应再在对应动作接口与任务条件下验证，不能用小环境结果替代大模型结论。

这些实验是机制学习路线，不是自动成立的论文方案。要成为研究，需要明确已有工作的缺口、控制变量、评价任务和资源条件。

把研究主张写成可以失败的句子

一个具体机制假设是：“在训练与测试都存在恢复状态，且数据包含多种恢复动作时，状态条件的质量估计比仅按终局成功筛选更有效地选择后续行为。”

它指出适用条件、比较对象和可能失败的原因。如果所有提升都来自新增恢复数据，机制假设就没有得到支持，但这仍能帮助改进研究方向。

不够明确的写法是“引入价值模块增强智能决策”。它没有说明价值预测对象、使用位置和可验证结果。研究的严谨性来自把这些空缺补上。

最后的一次整体自检

请给一个没有读过本书的人讲十分钟：VLA 已经会模仿，为什么还要后训练？什么情况下只补充演示可能更合适？为什么需要长期回报而不只看即时奖励？价值、优势和策略损失各自做什么？生成式策略增加了哪个接口难题？怎样判断任务进步真实而且公平？

如果能够把这些问题连成自然的一段解释，并知道自己哪些地方仍不确定，就已经可以开始以机制为中心阅读多数相关论文。需要查表和查原文很正常；目标是理解逻辑，不是记住所有标识。

3.11.5 后训练的一轮究竟留下什么

每轮固定采样策略与版本，获取轨迹、奖励、终止信息和旧概率；训练或更新价值评价；计算优势；更新策略；保存状态；用独立条件评价，再决定下一轮采样。优势统计来自训练批，测试成功率来自独立执行，二者用途不同。只有模型权重而没有优化器、归一化、采样进度和环境配置，可能无法接着进行同一训练。

如果策略奖励上升而真实成功下降，优先检查奖励漏洞、终止语义、接管计分和输入变化；如果价值误差下降但策略不变，检查优势尺度、概率计算和截断；如果训练效果可观而新任务退化，检查更新范围与保持能力。后训练应报告改善了哪些能力、付出多少交互和计算、损失哪些能力，不能只展示最好的成功视频。

3.11.6 练习题

题3.11-1 写出一次同策略后训练的数据与参数流，明确哪些量采集后固定、策略和价值各用什么目标、何时必须重新交互。

3.11.7 参考答案

题3.11-1答案 一次 PPO 型同策略更新可按以下流程理解：

1. 保存采集策略 π旧和价值网络 V旧。用 π旧与环境交互，记录观测/历史、实际动作、奖励、终止与截断、最后观测、动作 logπ旧以及 V旧输出。动作接口必须与计算密度的策略一致。

2. 用这些真实奖励和边界标记算 TD 残差、GAE 及价值目标 V旧+Â。这批数据里的动作、奖励、旧 log 概率、旧价值、优势及标签在更新阶段固定，不再随正在更新的网络重新定义。用于策略的优势可以标准化，价值标签使用标准化前的优势。

3. 当前策略重新算同一观测和已执行动作的 logπθ，形成 exp(logπθ−logπ旧)，优化裁剪代理及规定的熵/约束项。价值网络则拟合固定价值目标；两者若共享参数，还要说明总损失和梯度怎样合并。

4. 在这批数据上做有限轮更新，检查与采集策略的偏离。更新结束后用新策略重新交互，再形成下一批固定数据。

同一批数据可重复使用几轮，不代表可以无限训练。策略改变后，状态分布和优势评价都渐渐不再匹配新策略；应重新采集，超出允许偏离时还可提前停止当前轮。独立测试使用单独环境与成功判据，避免把训练奖励上升直接当作能力结论。

第4章：机器人运动学、动力学与物理交互

模型输出最终要经过控制器作用于真实物体。本章关注具身学习所需的坐标、运动、受力与接触知识，用它们解释标签、执行与预测之间的差异。

4.1 坐标、旋转、位姿与一致的空间记法

4.1.1 一次错误变换可以让正确策略做错动作

腕部相机看到杯子在图像右边，不代表它在机器人基座坐标系的右边。相机可能随腕部转动，工具坐标系还可能与夹爪开合方向不同。空间记法负责把“在哪里”和“往哪里移动”转成一致的数值。本节统一这些概念；感知和规划部分会引用它，不再重新建立另一套方向约定。

4.1.2 坐标、姿态与单位，是动作语义的一部分

“向右”相对哪个坐标系

相机右侧、机器人基座右侧、夹爪自身右侧，可能是不同方向。坐标系 frame 是给位置和方向提供参照的原点与轴。世界坐标常作为场景参考，基座坐标随安装位置确定，末端坐标跟随夹爪，相机坐标跟随相机。

点的位置既依赖物体，也依赖坐标描述。同一个方块不会因为换坐标而移动，但坐标数值会改变。策略、数据、控制器与物理评价必须对“这些数值是相对谁”达成一致。

一次旋转加平移，为什么写成变换

设 $\mathbf{p}_A = R_{AB}\mathbf{p}_B + \mathbf{t}_{AB}$，表示把 B 系中的点转为 A 系坐标。$R_{AB}$ 的列描述 B 的轴在 A 中的方向，$\mathbf{t}_{AB}$ 是 B 原点在 A 中的位置。

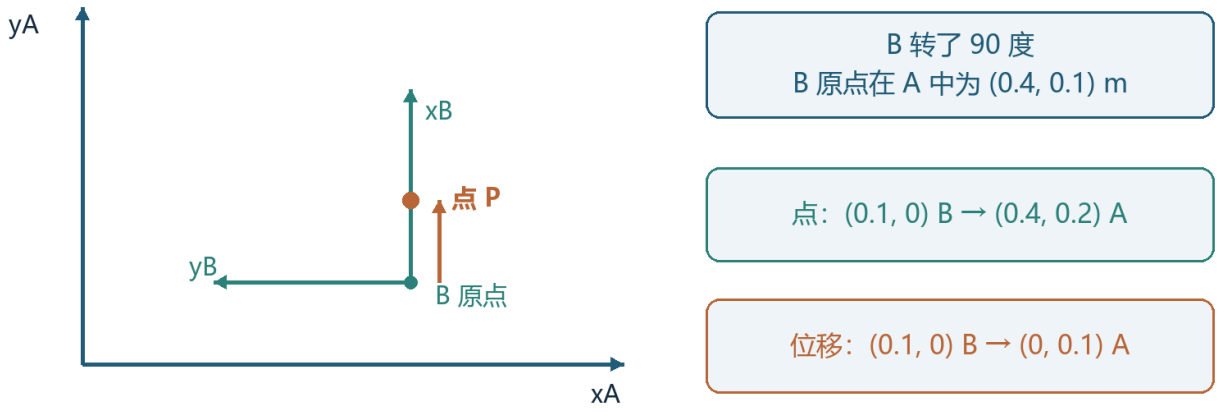
它同时编码姿态与位置，可以写成齐次变换

$$T_{AB} = \begin{bmatrix} R_{AB} & \mathbf{t}_{AB} \\ 0 & 1 \end{bmatrix}.$$

这个四行四列矩阵不是四维物理空间，而是把旋转和平移放进同一次矩阵运算的表示。连续变换可以组合为 $T_{AC} = T_{AB}T_{BC}$，顺序表达“先从 C 到 B，再从 B 到 A”。详细定义见 Modern Robotics 的变换讲解。

二维例子：B 相对 A 转了 90 度，B 原点在 A 中为 (0.4, 0.1) 米。B 中的点 (0.1, 0) 旋转后为 (0, 0.1)，转到 A 后为 (0.4, 0.2)。B 中的位移 (0.1, 0) 则只旋转为 (0, 0.1)，不用再加原点平移。

坐标改变的是描述，点与位移的转换不同



点：旋转再平移；位移：只旋转。数值为教学例子。

图4.1：点和位移换坐标时，平移的作用不同。

姿态不是三个坐标逐项相减那么简单

位置是平移，姿态 orientation 是方向。三维旋转一般不能交换顺序：先绕一个轴转，再绕另一个轴，与反过来不同。

欧拉角用若干连续转角表示姿态，必须说明轴顺序和参考系；某些姿态附近会出现表示奇异。单位四元数 quaternion 用四个数表示三维旋转，有归一化约束；同一个旋转可由一对相反符号的四元数表示，因此直接逐项平方误差可能把相同姿态判为不同。

旋转向量的方向表示轴，长度表示转角。**三维特殊正交群**（Special Orthogonal Group，SO(3)）由三维旋转矩阵构成，**三维特殊欧氏群**（Special Euclidean Group，SE(3)）由三维旋转与平移组成的刚体变换构成；它们帮助你辨认论文对象，并不要求本书完整展开李群理论。

对于小姿态增量，可以用局部三维旋转量；较大旋转仍需按实际表示组合，不能把任意姿态当作普通六维向量随意相加。

单位与标定决定标签是否有意义

一米与一毫米相差千倍，弧度与角度也不同。归一化动作的 0.5 不是天然的 0.5 米，它需要反归一化规则。

标定 calibration 是建立传感器、机器人和工具坐标对应关系的过程。工具中心点，Tool Center Point，缩写 **TCP**，是控制或测量末端位置时选定的参考点。夹爪换了、工具长度变了，旧 TCP 会让接触位置系统性偏移。

学习模型可能在固定偏差下拟合训练数据，却在换相机或换工具后失败。不能期待“大模型会理解空间”自动修正未给出的坐标和标定变化。

4.1.3 坐标系是信息能否用于行动的前提

同一个点为什么有多个坐标

杯子在相机坐标系中可能是“前方两米、右方二十厘米”，在机器人底座坐标系中则是另一组数。物体没有移动，只是描述它的原点和坐标轴变了。机器人通常涉及世界 W、底座 B、相机 C、末端 E 和物体 O 等坐标系。

本书采用列向量，并约定 T_{AB} 把 B 系中的坐标变成 A 系中的坐标。它由旋转 R_{AB} 与平移 t_{AB} 构成

$$p_A = R_{AB}p_B + t_{AB}, \qquad T_{AB} = \begin{bmatrix} R_{AB} & t_{AB} \\ 0 & 1 \end{bmatrix}.$$

这不是“一个预测网络”的公式，而是同一个空间点在不同坐标系之间的描述规则。旋转先改变方向，平移再改变原点。齐次坐标是在点向量后加一个 1，让两步操作合成一次矩阵乘法。

先沿哪一条链转换

相机观测点到世界系的转换是

$$\tilde{p}_W = T_{WB}T_{BC}\tilde{p}_C.$$

这条链从右向左计算：先把相机坐标转换到底座，再转换到世界。相邻变换的输入与输出坐标系相接，因而可用下标检查连接。实际代码也应先确认变换方向，再做矩阵相乘。

图4.2：转换链与每个变换的来源。外参由标定提供，机器人运动位姿由运行时估计提供。

做一个二维算例。点在 B 系是 $(1, 0)$，B 系相对 A 系逆时针旋转 90 度，B 系原点在 A 系中是 $(1, 2)$。旋转后点为 $(0, 1)$，再加平移得到 $(1, 3)$。如果直接把平移加到原坐标上，会得到 $(2, 2)$，含义完全不同。

反向变换也不是仅把平移取负。应先逆转旋转，因此逆变换的平移为 $-R_{AB}^{\mathsf{T}}t_{AB}$。抓取系统常见的错误，就是把物体到相机的变换当成相机到物体的变换。

哪些误差会被转换放大

相机外参旋转误差会让远处目标产生更大的位置偏差。小角度下，两米距离处一度误差约对应 3.5 厘米横向偏移。一个视觉网络即使在相机系中估计很准，外参错误仍会让机械臂抓偏。因此标定是任务链的一部分，而非安装完后永远不变的配置。

4.1.4 用局部转角更新姿态，为什么能保持有效旋转

一个刚体位姿有三个平移自由度与三个旋转自由度。旋转矩阵虽然有九个元素，却要求列向量单位长度、彼此正交且行列式为1。直接给九个元素各加一点，可能把旋转改成拉伸。**SO(3)** 表示满足这些条件的三维旋转矩阵集合；加上平移形成 **SE(3)**。它们的实际用途之一，就是规定如何更新位姿而不破坏几何含义。

可先求一个三维小旋转向量 $\delta\theta$，再将它变成旋转矩阵。若仅绕 z 轴转 α，叉乘矩阵与对应旋转是

$$[\delta\theta]_\times = \begin{bmatrix} 0 & -\alpha & 0 \\ \alpha & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}, \qquad \exp([\delta\theta]_\times) = \begin{bmatrix} \cos\alpha & -\sin\alpha & 0 \\ \sin\alpha & \cos\alpha & 0 \\ 0 & 0 & 1 \end{bmatrix}.$$

这里的矩阵指数把局部转角转换成完整旋转；小角度时 $\cos\alpha \approx 1, \sin\alpha \approx \alpha$，与 $I + [\delta\theta]_\times$ 的一阶近似相符。任意轴也可按同样原则处理。反对称矩阵 S 满足 $S^T = -S$，所以 $\exp(S)^T \exp(S) = \exp(-S)\exp(S) = I$；这样的更新保持正交，而从零连续增加转角保持行列式为1。

右乘 $R \leftarrow R\exp([\delta\theta]_\times)$ 表示在当前旋转所连接的局部坐标里追加变化；左乘则用另一参考坐标表达增量。它们通常不同。机械臂工具自身“向右转”与世界系“绕竖直轴转”不能只靠三个相同数值互换，求误差和雅可比时也要沿用同一约定。Modern Robotics：变换与运动表示

四元数是另一种旋转表示，需保持单位范数，q 与 $-q$ 表示同一个旋转；因此逐元素平方误差可能误罚相同姿态。欧拉角适合显示，但受旋转顺序与表示奇异影响。选择表示是为了让训练、优化、插值与执行具有一致的几何含义，而不是给同一个六维数组换一个名称。

4.1.5 点、方向、速度和协方差不能同样变换

点的位置受旋转和平移影响；方向向量只受旋转影响。若把速度当作齐次点乘同一变换，会凭空加上位置平移。六维速度还取决于选取的参考点，平移参考点会把角速度贡献到线速度。力矩也随作用参考点改变，因此空间速度与力应采用配套变换，保持功率内积一致。

估计的点有不确定性时，坐标变换还应传播协方差。简单线性旋转 $p_A = Rp_B + t$，若 R、t 暂当已知，则 $\Sigma_A = R\Sigma_B R^\top$。若标定本身不确定，还需加入外参误差的贡献。一个准确到厘米的目标估计，不能仅通过坐标转换就变成毫米精度。反复使用同一误差源产生相关性，不应将其当成许多独立观测平均掉。

4.1.6 练习题

题4.1-1 点 p相机=[1,0,0]，相机到机器人基座的旋转为绕 z 轴 90°，平移为 [0,2,0]。求 p基座；再写出逆位姿。为什么旋转与平移不能换顺序?

4.1.7 参考答案

题4.1-1答案 相机到基座变换作用为 p基座=Rp相机+t，其中

$$R=\begin{bmatrix}0 & -1 & 0\\ 1 & 0 & 0\\ 0 & 0 & 1\end{bmatrix},\qquad t=\begin{bmatrix}0\\ 2\\ 0\end{bmatrix}.$$

所以 Rp相机=[0,1,0]，再加平移，得到 p基座=[0,3,0]。

齐次位姿 T=[R,t;0,1] 的逆为

$$T^{-1}=\begin{bmatrix}R^{\top} & -R^{\top}t\\ 0 & 1\end{bmatrix},\qquad R^{\top}=\begin{bmatrix}0 & 1 & 0\\ -1 & 0 & 0\\ 0 & 0 & 1\end{bmatrix},\quad -R^{\top}t=[-2,0,0]^{\top}.$$

代回逆变换 R^{T}[0,3,0]+[-2,0,0]，确实得到原来的点 [1,0,0]。

若先给点加同一数值的平移再旋转，会得到 R(p+t)=Rp+Rt，而原式是 Rp+t；Rt 一般不等于 t。旋转会改变平移方向。更根本地说，题中的 t 是在基座坐标里表达的相机原点位置，不能不经坐标变换就拿它当相机坐标中的平移。顺序与坐标表达必须同时明确。

4.2 正逆运动学、雅可比与动作可达性

4.2.1 关节动作与末端运动怎样连接

配置空间和任务空间

关节配置 $\mathbf{q}$ 包含各关节位置，转动关节常以弧度表示，移动关节常以米表示。自由度，Degree of Freedom，缩写 DOF，表示系统可独立运动的维度；七自由度机械臂通常有七个可动关节。

末端位姿属于任务空间。正运动学 forward kinematics 把关节配置转换为末端位姿；**逆运动学** Inverse Kinematics，缩写 **IK**，给定末端目标求关节配置。

相同末端位姿可能对应多个关节配置。冗余机械臂可以用这种自由度避免障碍、调整姿态或远离关节极限。末端目标相同，不表示身体动作或可用力方向相同。

雅可比不是新的物理定律，而是局部变化关系

雅可比矩阵 Jacobian 表示关节小变化如何引起末端小变化。只看位置时

$$\dot{\mathbf{p}}=J_p(\mathbf{q})\dot{\mathbf{q}}.$$

左边是末端速度，右边是各关节速度对它的共同贡献。若末端有三维位置、机器人有七个关节，J_p 是三行七列。它随配置变化，因为杆件方向与力臂改变。

本书需要六维末端运动时采用几何速度 $[\mathbf{v};\boldsymbol{\omega}]$，先线速度后角速度，并使用与它对应的雅可比。其他文献可能采用不同排列或空间运动定义，要核对坐标与参考点。Modern Robotics 的雅可比章节提供另一套常见记号的阅读入口。

两关节例子解释耦合和奇异

平面机械臂两段长度为 0.3、0.2 米，末端位置为

$$x = 0.3\cos q_1 + 0.2\cos(q_1+q_2), \qquad y = 0.3\sin q_1 + 0.2\sin(q_1+q_2).$$

对关节求局部变化关系，得到

$$J_p = \begin{bmatrix} -0.3\sin q_1 - 0.2\sin(q_1+q_2) & -0.2\sin(q_1+q_2) \\ 0.3\cos q_1 + 0.2\cos(q_1+q_2) & 0.2\cos(q_1+q_2) \end{bmatrix}.$$

当 $q_1 = 0, q_2 = \pi/2$，矩阵为第一行 (-0.2, -0.2)，第二行 (0.3, 0)。只让第二关节以 -0.05 rad/s 运动，末端局部速度就是 (0.01, 0) m/s。

当两关节都为零，手臂完全伸直，第一行变成零，第二行为 (0.5, 0.2)。这时关节微小转动不能产生一阶的横向伸长。这个配置存在局部运动方向缺失，叫**奇异性** singularity；不是机器人完全不能动，而是某些要求变得难以实现。

为什么学习策略也受这些几何关系影响

末端动作接口可以隐藏 IK 细节，但不能取消可达性、关节极限与奇异问题。相同目标位移，在不同身体配置下可能需要不同关节速度，有时被限幅或投影。

若策略或**世界模型**只看末端位置，不看关节配置，就可能把不同执行能力混在一起。加入机器人状态的作用不只是多几个数字，它提供了决定动作后果的信息。

完整逆运动学求解器通常还包含约束与正则化。训练数据应记录实际使用的求解器设置，不能把“目标可写出来”当作“动作可执行”。

4.2.2 逆运动学为什么是一组可行解

逆运动学（Inverse Kinematics，IK）根据目标末端位姿求关节配置。正运动学是 $x = f(q)$，IK 不一定存在唯一反函数：目标可能不可达，也可能有多个姿态实现。

二维两连杆各长 1，末端目标 (1,1)。关节角 (0,π/2) 与 (π/2,−π/2) 都到达目标，但肘部位于不同位置。环境碰撞可能只允许其中一种。

可将 IK 写为在限位与碰撞条件下，最小化目标误差和姿态偏好：
$\min_q \|f(q) - x^*\|_W^2 + \lambda\|q - q_{ref}\|^2$。$x^*$ 是目标，q_{ref} 是参考配置，W 平衡位置与朝向误差，λ 表达姿态偏好。旋转误差应使用适合的局部表示，不随意减旋转矩阵元素。

软代价低不保证满足硬约束，局部优化也依赖初值。可尝试多个合理初值，检查返回解的残差、限位与碰撞，再交给路径规划。直接拿到一个 IK 数值就发送整段运动，会漏掉途中障碍。

4.2.3 雅可比是局部关系，不是全局动作解答

雅可比给出当前位置附近的关系 $\Delta x \approx J(q)\Delta q$。想用关节增量修正末端误差 e，可解最小二乘问题 $\min_{\Delta q} \|J\Delta q - e\|^2$。冗余机械臂有多种解，伪逆选取其中最小范数的一种；接近奇异位形时，小末端误差却可能需要很大的关节变化。

阻尼最小二乘在目标中加入 $\lambda^2\|\Delta q\|^2$，得到 $\Delta q = J^\top(JJ^\top + \lambda^2 I)^{-1}e$。它允许牺牲一些末端精度，换取较小的关节修正，因此缓和奇异附近的放大。计算后仍要限制步长、更新姿态、重新计算 J，并检查误差与碰撞。局部求解器在近处修正目标，不能用一个固定 J 替代整条大范围运动规划。

多个关节可以实现同样末端目标，剩余自由度可以服务避障、姿态或限位。将这些目标放进零空间或约束优化时，要明确其优先级和可行性。若主任务本身不可达，不应把求解器输出的某个近似向量说成“已经解决 IK”。Modern Robotics提供系统的几何基础。

4.2.4 一次完整的二维逆运动学迭代

下面给出可运行的两连杆位置求解：先从当前关节算末端位置，再求局部雅可比，用阻尼最小二乘得到小修正。每次更新都重新计算位置与雅可比，因为它们随构型改变。这里的输入是末端目标，优化对象是关节角；没有训练神经网络。

算法步骤：

1. 定义前向运动学与其局部导数。
2. 计算当前末端到目标的误差。
3. 求阻尼修正，限制单次关节变化并重新计算。
4. 返回配置，检查实际末端残差。

```
import numpy as np

def fk2(q):
    # 步骤1：两根连杆长0.3和0.2米
    a, b = q[0], q.sum()
    return np.array([0.3*np.cos(a)+0.2*np.cos(b),
                     0.3*np.sin(a)+0.2*np.sin(b)])

def jacobian2(q):
    a, b = q[0], q.sum()
    return np.array([[-0.3*np.sin(a)-0.2*np.sin(b), -0.2*np.sin(b)],
                     [ 0.3*np.cos(a)+0.2*np.cos(b),  0.2*np.cos(b)]])

def solve_position(target, initial, damping=0.01):
    q = np.array(initial, dtype=float)
    for _ in range(60):
        # 步骤2：这里仅求平面位置，不包含姿态误差
        error = np.asarray(target) - fk2(q)
        if np.linalg.norm(error) < 1e-5:
            break
        # 步骤3：解线性方程而非显式求逆，每次最多转0.1弧度
        j = jacobian2(q)
        dq = j.T @ np.linalg.solve(j @ j.T + damping**2*np.eye(2), error)
```

```
        dq *= min(1.0, 0.1 / max(np.linalg.norm(dq), 1e-12))
        q += dq
    # 步骤4：求解返回后仍应核对目标残差
    return q, np.linalg.norm(np.asarray(target) - fk2(q))

q, error = solve_position([0.35, 0.2], [0.2, 0.9])
print(q, error)
```

阻尼让关节修正较平缓，不能让不可达目标变得可达。真实机器人还需加入关节限位、碰撞与姿态要求，并检查返回状态和完整运动路径；本例帮助读者看清局部误差怎样逐步变成关节变化。MIT：微分逆运动学与约束

4.2.5 雅可比为什么只能帮助局部可达的运动

雅可比矩阵（Jacobian Matrix）把关节的小变化与末端的小变化连接起来。第 j 列具体表示：只让第 j 个关节增加一点，其余暂不变，末端朝什么方向、以多大比例变化。$\Delta x \approx J(q)\Delta q$ 是当前位置附近的一阶近似，末端位置改变很多以后，需要重新计算 J。

用一个局部矩阵 $J = \begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix}$ 看奇异性。任何关节增量都只能改变末端第一方向，第二方向始终为0。若目标增量是 $(0, 1)$，不存在能实现它的 Δq；阻尼最小二乘只能给出使误差和关节变化折中的解，不能补出机器人失去的运动方向。若目标为 $(1, 0)$，阻尼 λ 的解为 $(1/(1+\lambda^2), 0)$，与未阻尼的 $(1, 0)$ 相比更保守。

这说明“数值计算有一个稳定解”与“目标在物理上能实现”是两回事。VLA 输出末端增量后，逆运动学、关节限位和碰撞检查仍需要处理可达性；接近奇异状态时可以改变构型、移动基座或缩小局部步长，而不是只把阻尼调大。MIT 操作课程：机器人接口与运动

4.2.6 练习题

题4.2-1 平面二连杆长度均为1，关节角为 q=[0,0]。求末端位置雅可比，说明为什么一阶局部关系无法产生非零x方向速度。很小但非零的关节变化是否可能产生向−x的位移？用 q=(η,−2η) 检查。阻尼最小二乘对本位姿的纯x误差会返回什么，为什么？

4.2.7 参考答案

题4.2-1答案 约定两连杆长度均为1、关节角为 q_1, q_2，末端位置为

$$x = \cos q_1 + \cos(q_1 + q_2), \qquad y = \sin q_1 + \sin(q_1 + q_2).$$

对关节角求导，在 $q = (0, 0)$ 得到位置雅可比

$$J = \begin{bmatrix} 0 & 0 \\ 2 & 1 \end{bmatrix}.$$

一阶关系 $\Delta p \approx J\Delta q$ 因而只能表达y方向位移：本位姿不能产生非零的瞬时x速度。

这不表示任何小关节变化都不改变x。令 $\Delta q = (\eta, -2\eta)$，两杆方向分别为 $\eta, -\eta$，精确末端为

$$x = 2\cos\eta \approx 2 - \eta^2, \qquad y = 0.$$

它产生二阶的向内收缩；对应的一阶结果 $J\Delta q = 0$，恰好说明变化不在雅可比的一阶项中。向+x进一步伸长则超过总长度2，无法实现。

阻尼最小二乘最小化 $\|J\Delta q - \Delta p\|^2 + \lambda^2\|\Delta q\|^2$，解为

$$\Delta q = (J^\top J + \lambda^2 I)^{-1} J^\top \Delta p.$$

对纯x误差，$J^\top \Delta p = 0$，所以该局部步骤仍返回0。阻尼控制的是近奇异处关节修正的放大，不能增加缺失方向。要利用二阶向内运动，应改变构型、采用有限弯曲或调整路径后重新线性化；仅提高阻尼无法让这一步发现二阶解。

进一步阅读：Modern Robotics：Singularities（雅可比的一阶关系）。

4.3 力、能量、惯性与机器人动力学方程

4.3.1 力、力矩、惯性与能量，怎样影响操作

移动物体需要的是合力，而非某一项力

质量 m 的物体质心满足 $\sum \mathbf{F} = m\mathbf{a}$。重力、夹持摩擦、桌面支撑和碰撞作用都在合力里。桌面上的静止方块没有加速度，是支撑力与重力平衡，而非它没有受力。

质量 0.3 kg 的方块受到向上 4 N 的合成夹持作用，重力取 2.94 N。在不考虑其他力的教学模型中，向上净力为 1.06 N，加速度约 3.53 m/s²。把 4 N 直接除以质量，会漏掉重力。

力矩为什么取决于作用位置

力矩 moment 或 torque 描述使物体转动的作用。相对于参考点，$\boldsymbol{\tau} = \mathbf{r} \times \mathbf{F}$，其中 $\mathbf{r}$ 是从参考点到作用点的向量。

垂直于 0.1 m 力臂施加 5 N，力矩大小是 0.5 N·m；沿着力臂推，力矩可为零。抓住杯子边缘和抓住杯子中间，可能需要不同的稳定接触，即便握力一样。

六维力旋量 wrench 汇总力与力矩。本书采用 $[\mathbf{F}; \boldsymbol{\tau}]$ 顺序，必须说明相对哪个点、在哪个坐标系表达。更换参考点会改变力矩部分；不能只旋转六个数字而忘记力臂变化。

惯性为什么不仅是重量

平移惯性与质量有关，转动惯性还与质量相对转轴的分布有关。相同质量的长工具与紧凑方块，转动响应可能不同。

惯性张量 I 用矩阵描述三维转动的方向差异。相对质心、在合适的刚体坐标表达中，角动量是 $\mathbf{L} = I\boldsymbol{\omega}$。快速三维转动还出现方向耦合，因此一般不只有“力矩等于一个标量乘角加速度”。

学习系统可能需要质量、质心或惯性相关信息，不一定需要精确显式数值；也可利用动作响应估计它们。仅给物体类别或几何外形，不能保证知道内部质量分布。

能量和功率为什么是好诊断量

功率描述作用与运动的即时能量交换：$P = \mathbf{F}^\top \mathbf{v} + \boldsymbol{\tau}^\top \boldsymbol{\omega}$。力很大但速度为零，机械功率可以为零，不过电机仍可能消耗电能或发热。

动能是运动相关能量，重力势能是高度相关能量。碰撞、摩擦和阻尼会耗散部分机械能。仿真中一个不受驱动的物体越弹越高，可能说明数值或接触模型正在非物理地增加能量。

这类检查帮助判断预测与仿真是否可信。不能仅因运动轨迹视觉平滑，就认为它满足正确能量和作用关系。

4.3.2 怎样读懂机器人动力学方程

方程在回答“哪些作用一起产生运动”

对固定基座、使用普通关节坐标的机械臂，一种常见写法是

$$M(\mathbf{q})\ddot{\mathbf{q}} + C(\mathbf{q}, \dot{\mathbf{q}})\dot{\mathbf{q}} + \mathbf{g}(\mathbf{q}) + \boldsymbol{\tau}_{\text{fr}} = \boldsymbol{\tau}_{\text{act}} + \sum_c J_c(\mathbf{q})^\top \mathbf{f}_c.$$

M 是**惯性矩阵**，将关节加速度映射到所需广义作用。C 乘速度概括运动中的速度耦合项。$\mathbf{g}(\mathbf{q})$ 是重力相关项，$\boldsymbol{\tau}_{\text{fr}}$ 表示按本式符号约定移到左边的摩擦等阻力。右边是执行器作用与环境接触的广义作用。

接触力 $\mathbf{f}_c$ 在这里定义为环境作用于机器人的力或相应力旋量。J 与它必须使用相容的坐标、参考点和排列。对转动关节，方程分量单位为 N·m；移动关节对应 N。

给定关节位置、速度、执行器作用和接触作用后，这个方程可以求出关节加速度，再由积分器推进状态。反过来，若规定期望加速度，也可以计算需要的执行器作用。这就是正动力学与逆动力学的用途。策略负责选择任务动作，动力学解释所选动作怎样通过控制与受力变成运动。接触力不能任意指定，还需要结合接触几何、摩擦和约束求解。

为什么 M 是矩阵，而且随姿态改变

改变一个关节会移动多个杆件，因此别的关节也感受到相应惯性。惯性矩阵的非对角元素体现这种耦合。手臂伸开与收拢时，质量相对各关节的分布不同，因此 M 随配置变化。惯性矩阵的原始教材讲解可帮助进一步阅读。

取一个忽略重力、速度与接触的两关节瞬时例子

$$M = \begin{bmatrix} 2 & 0.5 \\ 0.5 & 1 \end{bmatrix}, \qquad \boldsymbol{\tau} = \begin{bmatrix} 1 \\ 0 \end{bmatrix}.$$

解 $M\ddot{\mathbf{q}} = \boldsymbol{\tau}$，得到加速度约为 (0.5714, -0.2857)。第二关节没有主动施加力矩，仍可能加速，因为整个机构是耦合系统。这个例子不代表某台真实机器人参数。

速度项与重力项各在什么时候重要

重力项说明保持静止也可能需要力矩。拿着重物时，策略即使输出不变目标，执行器也在持续做补偿。

速度耦合项在快速多关节运动中更明显，它们来自机构运动的几何与惯性关系。不同文献可把它们合并为 $\mathbf{h}(\mathbf{q},\dot{\mathbf{q}})$。不应把 C 的每个元素都想象成一个真实独立阻尼器：这些项与耗散摩擦不是同一种物理机制。

慢速**准静态**任务可以近似忽略某些惯性和速度项，但必须检查速度、加速度和接触切换。说“操作很慢”不是任何碰撞都可忽略的理由。

为什么末端外力通过 J 的转置进入关节

关节作用与末端作用要产生相同机械功率。由于末端速度由 J 映射关节速度，力的对应关系是 $\boldsymbol{\tau}_{\text{ext}} = J^{\top}\mathbf{F}_{\text{ext}}$，而不是 J 直接乘力。静力关系的教材说明给出功率推导。

用第4.2节弯折配置的二维 J，末端受到向右 5 N 外力时，广义外力矩为 (-1, -1) N·m。若要静止抵消，还需相反方向的执行器力矩，以及重力等其他补偿。外力与机器人主动施加给物体的力是作用反作用关系，符号不能混用。

正动力学与逆动力学是不同查询

正动力学 forward dynamics：给定状态和作用，计算加速度或下一状态。逆动力学 inverse dynamics：给定状态和期望加速度，计算相应所需作用，仍依赖接触和模型条件。正动力学入口。

数据学习里的“逆模型”也可能指给定前后图像预测动作。那是从转移恢复动作的学习任务，不等于解析逆动力学，而且可能有多个合法动作。

4.3.3 动力学方程怎样从能量模型推出来

取一个独立教学模型：无质量杆长 l，杆端集中质量 m，绕关节在竖直平面转动，$q=0$ 指向下方。末端速度大小为 $l\dot{q}$，因此动能 $T = ml^2\dot{q}^2/2$；相对最低点的势能为 $U = mgl(1-\cos q)$。这两个量同时表达了几何、质量与重力。

定义拉格朗日量 $L = T - U$。在这一关节坐标下，动力学用

$$\frac{d}{dt}\frac{\partial L}{\partial \dot{q}} - \frac{\partial L}{\partial q} = \tau.$$

第一项为 $ml^2\ddot{q}$，第二项的负号给出 $mgl\sin q$，于是

$$ml^2\ddot{q} + mgl\sin q = \tau.$$

ml^2 是关于转轴的惯性，$mgl\sin q$ 是重力在该关节上的作用需求。下方静止时重力力矩为0，横向举起时需要持续力矩支撑。这个单关节模型的惯性常数不随姿态改变，也没有多关节速度耦合；换成多个杆件，能量含有姿态相关系数与交叉速度项，求导后便出现 M 与速度相关项。MIT：从能量到机器人动力学

因此机器人动力学不是把几个名字拼到一个等式里，而是把所有杆件的运动与能量整理到同一组坐标上。VLA 即使只输出位置目标，实际控制器仍必须产生这些作用；负载或姿态改变后，同一几何目标就可能产生不同跟踪误差。Modern Robotics：惯性矩阵的物理解释

4.3.4 为什么速度用 J，而力用 J 的转置

设几何速度满足 $\mathbf{v} = J\dot{\mathbf{q}}$。同一机械作用的功率要求

$$\boldsymbol{\tau}^\top \dot{\mathbf{q}} = \mathbf{F}^\top \mathbf{v} = \mathbf{F}^\top J\dot{\mathbf{q}}.$$

对任意允许的关节速度，这给出 $\boldsymbol{\tau} = J^\top \mathbf{F}$。它表达作用与运动的对偶关系，不是凭经验把矩阵转置一下。

如果六维速度与力旋量使用不同排列，或者它们相对不同参考点，内积就可能不再代表同一功率。坐标变换和力矩参考点正是这条推导的条件。

4.3.5 惯性矩阵怎样来自所有杆件的动能

第 i 个刚体的质心速度和角速度可分别由局部雅可比映射关节速度。其动能包含平移与转动

$$E_i = \tfrac{1}{2} m_i \mathbf{v}_i^\top \mathbf{v}_i + \tfrac{1}{2} \boldsymbol{\omega}_i^\top I_i \boldsymbol{\omega}_i.$$

代入速度关系并对杆件求和，得到 $E = \tfrac{1}{2}\dot{\mathbf{q}}^\top M(\mathbf{q})\dot{\mathbf{q}}$。M 汇总了整个机构在当前姿态下的能量结构，所以包含配置相关性与关节耦合。

在对应无退化的机械系统条件下，非零关节速度产生正动能；这解释了常见惯性矩阵的对称正定性质。它不是“每个对角元素都是一块物体质量”。

4.3.6 准静态：何时可以忽略惯性变化

若物体在长度尺度 L 上用时间 T 完成运动，典型加速度尺度约为 L/T^2，惯性作用尺度约为 mL/T^2。把它与摩擦、支撑或任务作用尺度比较，可以判断惯性是否可能很小。

时间减半，典型加速度尺度可增大四倍。慢速演示变成快速策略，原准静态模型可能不再足够。接触切换还有更短时间尺度，不能只看整回合平均速度。

这是估算而非普遍误差定理。实际验证要观察作用、速度、模式与任务后果，必要时选择更完整动态。

4.3.7 学习模型的平均与充分状态假设

如果同一输入后的未来有多个可能状态，使用平方误差回归在理想充分数据条件下倾向于预测条件平均。平均不一定是一个真实可发生的接触结果，例如把“继续夹住”与“滑落”混成中间状态。

一种原因是真实随机性，另一种原因是输入缺了速度、负载、历史或接触信息。增加**条件分布**输出与补充状态信息解决不同问题；模型结构更复杂，不自动补齐缺失信息。

延迟系统还可能依赖过去未生效命令。把命令队列与必要历史加入状态，可使转移描述更充分；若省略它们，当前动作与下一状态之间的关系可能看似随机。

理解这些假设后，才能判断应该增加数据、输入、概率结构还是物理模型，而不是把所有预测失败都叫作网络不够强。

4.3.8 练习题

题4.3-1 动力学 $M(q)\ddot{q} + C(q,\dot{q})\dot{q} + g(q) = \tau + J^\top f$ 中各项对应什么物理作用？若策略输出位置而非力矩，为什么仍要理解这条方程？

4.3.9 参考答案

题4.3-1答案 $M(q)\ddot{q}$是产生关节加速度所需的惯性作用：质量分布随姿态变化，各关节也会相互耦合。$C(q,\dot{q})\dot{q}$描述运动产生的科里奥利与离心作用，$g(q)$是重力对应的广义力，τ是执行器提供的广义驱动力。对转动关节，这些分量是力矩；对移动关节，则是力。$J^\top f$把环境作用在机器人上的末端力或力旋量映射到关节广义力，J与f必须使用相容的坐标、参考点和分量顺序。方程描述这些作用如何共同决定关节加速度。

若策略输出位置目标，底层控制器仍需根据位置误差、速度、负载和接触情况生成 τ，机器人再按这条动力学运动。相同位置命令在空载、重载、自由空间与接触中可能产生不同实际轨迹，且受到电机力矩、速度与控制带宽限制。

例如策略命令夹住杯子后快速抬起，若负载较大或加速度过高，跟踪会滞后、夹持可能滑落。理解动力学帮助解释“预测位置正确但执行失败”，决定是否加入速度/力反馈、调整轨迹加速度或更换控制接口。高层不用直接算力矩，也不能把机器人当作目标位置一发出就立即到达的系统。

4.4 位置、力矩与阻抗控制的策略接口

4.4.1 位置目标不是把机器人瞬间放到那里

位置控制接口接收目标位置，控制器不断根据误差调整驱动，尽量到达目标。关节速度接口接收期望速度，力矩接口接收关节作用。它们定义不同动作空间，也让策略承担不同程度的底层决策。

位置目标通常容易用于模仿，但接触时目标和实际位置可能不同。力矩接口更直接影响动力学，策略也要处理更多跟踪、惯性和反馈问题。这里没有统一最好的接口，选择取决于任务、机器人支持和数据来源。

robosuite 等具身仿真系统明确区分关节位置、速度、力矩与末端动作接口，并通过控制器将策略命令变为底层作用。具体默认坐标与增量约定应按版本核对。robosuite 控制器文档。

4.4.2 PD 的作用为什么像纠偏加刹车

比例微分，Proportional-Derivative，缩写 **PD**，是常见跟踪机制。一个单转动关节的简化形式是

$$\tau = k_p(q_{\text{ref}} - q) - k_d\dot{q} + \tau_{\text{other}}.$$

这里采用单次保持位置目标、期望速度为零的简化。位置误差项使关节朝目标运动；速度项抵抗过快运动，提供阻尼。τ_{other} 可以包含重力等补偿。增益 k_p 的单位在此为 N·m/rad，k_d 为 N·m·s/rad。真实实现可能有惯性缩放、滤波和更多环节。

例如误差 0.02 rad、速度 0.1 rad/s，取 $k_p = 20$、$k_d = 2$，不计其他项时，输出为 $0.4 - 0.2 = 0.2$ N·m。它是在算本时刻所需作用，不是保证下一时刻恰好到达目标。

策略每 50 ms 更新一次目标，PD 可在这期间多次根据新状态重新算力矩。因此“策略动作保持不变”不表示执行器力矩不变。

4.4.3 阻抗与顺应为什么与接触操作有关

阻抗 impedance 描述力与运动之间的动态关系。任务空间阻抗的一种教学图像是虚拟弹簧与阻尼器：夹爪被目标吸引，但受到接触时允许一定偏离。

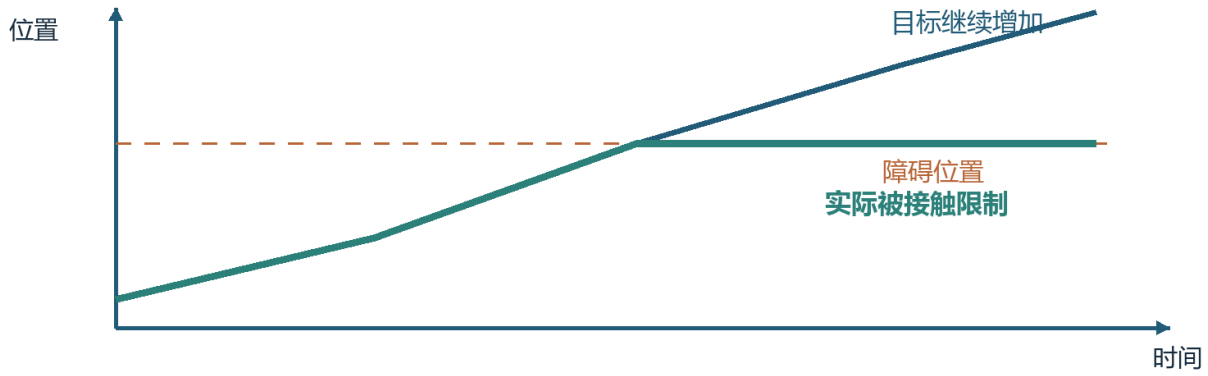
一维期望作用可写成

$$F_{\mathrm{cmd}} = K(x_{\mathrm{ref}} - x) - B\dot{x}.$$

K 是刚度，stiffness，表示位置偏离一单位时的作用变化；B 是阻尼，damping，表示运动速度带来的抵抗。K 不是物体材料硬度本身，而是这里控制器希望呈现的关系。

忽略其他作用与动态，在刚墙阻挡下，目标超出墙面 5 mm，K 为 200 N/m 时，弹簧项是 1 N；K 为 2000 N/m 时是 10 N。姿态路径相同，接触作用却可能相差十倍。

接触时，目标位置与实际位置可以分开



示意曲线；不是实测轨迹。相同偏差下，刚度影响作用大小。

图4.3：目标继续前进，真实位置可能被接触限制。

顺应 compliance 表示受到作用时允许一定变形或位移的特性。软一些可能帮助吸收接触误差，过软也可能拿不稳或跟踪差。不能把低刚度当作所有任务的正确答案。

4.4.4 导纳与力控制，了解它们的输入输出即可

导纳 admittance 通常根据测得外力生成期望运动响应，例如用虚拟质量、阻尼、刚度关系将力转成目标位移。阻抗常强调“期望运动偏差产生什么作用”，导纳常强调“作用引起什么运动”。它们在具体硬件中仍要通过内层控制实现。

力控制关心接触作用跟踪。没有接触时，持续要求某个接触力可能没有对应稳定状态；接触方向、观测噪声与约束必须一起考虑。

学习系统若输出目标位置加刚度，实际动作空间已经扩展。其优势可能来自更合适的接触行为，而不只是更精确的几何目标。实验需要控制是否额外给予这些接口能力。

4.4.5 执行器也有自己的状态与限制

电机、齿轮、驱动和机械结构具有响应时间、摩擦、传动间隙和作用限幅。请求大力矩，实际作用可能饱和；持续输出能力与短时峰值也可能不同。

带内部激活或滤波的执行器，当前输出依赖过去命令。学习状态只放当前目标，未必足够预测其响应。MuJoCo 的计算文档明确包含执行器与激活状态等建模对象；使用时应检查模型实际采用的定义。MuJoCo 动力学计算。

4.4.6 导纳怎样把测到的外力变成新的位置目标

若机器人主要接受位置目标，而任务又需要受力后适当让步，可以在位置接口外面加一个导纳模块。它读取测得的外力，用虚拟动态计算允许的运动，再把所得位置交给原位置控制器。虚拟质量 M_v、阻尼 B_v、刚度 K_v 规定的是想呈现的响应，未必等于机器人的真实质量或材料参数。

$$\ddot{x} = \frac{F_{\text{ext}} - B_v\dot{x} - K_v(x - x_d)}{M_v}.$$

例如 $M_v = 0.5\,\text{kg}$、$B_v = 5\,\text{Ns/m}$、$K_v = 50\,\text{N/m}$，测得外力3 N，当前偏移0.01 m、速度0.1 m/s，则虚拟加速度为4 m/s²。按控制间隔推进虚拟速度和位置，新的位置目标体现了这次外力；它不要求位置控制器直接接收3 N作为动作。实际模块还要处理测力偏置、坐标变换、速度限幅和内层响应。ROS 2 导纳控制器说明

阻抗模块则从运动误差计算所需作用，导纳模块从外力计算允许运动。二者的模型都可以具有弹簧、阻尼和惯性，但输入输出方向不同。学习策略可以选择目标、阶段或这些响应参数；这样做扩展了策略的接口能力，比较时应让对照具有相容的控制条件。插入任务还要区分轴向推进与横向让步，单纯把所有刚度一起降低可能既不对齐，也推不进去。

4.4.7 练习题

题4.4-1 插入任务中，用很高位置增益追踪目标，与设置较柔顺的阻抗有什么不同？为何降低刚度后仍可能失败？

4.4.8 参考答案

题4.4-1答案 高位置增益使机器人强烈抵抗位置误差。若孔位略有偏差，机械臂仍把插头推向错误目标，接触力近似随刚度与误差增大，可能卡住或损伤物体。柔顺阻抗则规定接触时希望表现出的弹簧和阻尼关系，使外力能引起一定偏移；合适的横向柔顺可让插头沿孔口几何调整，而不把微小误差放大成很大的力。

降低刚度仍可能失败：姿态偏差过大不会自动对正；横向与轴向都很软可能没有足够插入力；缺少阻尼会振荡；摩擦和插入深度估计错误会导致卡滞；控制延迟与力传感噪声也会破坏预期响应。

所以应按方向选择刚度与阻尼，区分接近、寻找接触、对齐和插入阶段，并监测力、位移与进度。例如保持轴向缓慢推进、横向允许微调，遇到异常力后退并重新对齐。验证要看接触力、轨迹误差、插入成功率及物体损伤，而非只看“刚度较低”。柔顺提供的是接触响应机制，不能代替正确几何、反馈和任务策略。

4.5 时序、接触、摩擦和离散物理变化

4.5.1 时间、延迟与动作块为什么改变动力学

系统里通常不只有一个频率

策略频率是生成新目标的频率，控制频率是更新底层作用的频率，仿真步长是**数值积分**的时间间隔，相机还有自己的采样频率。它们不必相同。

教学例子中，策略为 20 Hz，每次隔 50 ms；控制器为 200 Hz，每隔 5 ms 更新；物理仿真每 1 ms 积分一次。一次策略命令期间包含十次控制更新和五十次物理步。具体机器人频率以其真实接口为准。

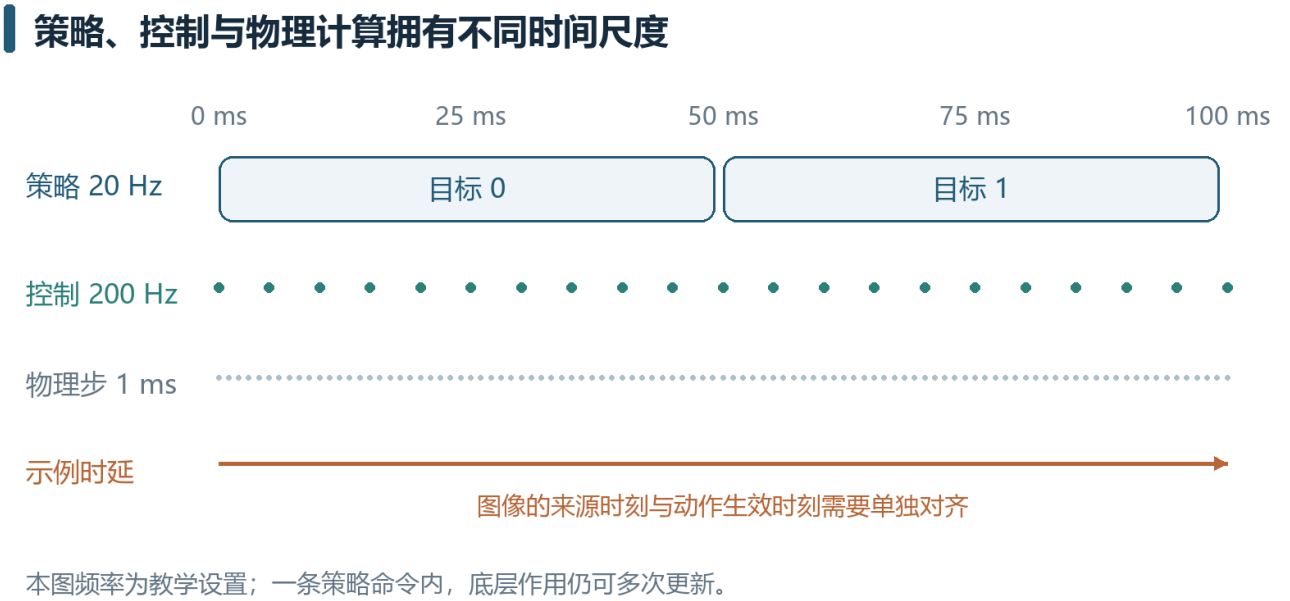


图4.4：策略、控制器、物理步与传感器有不同时间尺度。

只记录“step=1”而不说明是哪一种步，会使动作持续时间、回报折扣和模型预测标签变得模糊。

相同数值动作，持续时间不同，后果也不同

速度命令 0.1 m/s 持续 50 ms，理想位移为 5 mm；持续 200 ms，理想位移为 20 mm。位移目标 20 mm 则是另一种命令，不应仅按速度乘时间解释其实际跟踪。

力矩命令持续更久还会改变速度与位置，不能只改采样频率而认为策略动作含义不变。不同数据集的动作尺度、持续时间和控制器约定必须共同对齐。

延迟使模型对“现在”的认识变旧

延迟 latency 包括图像采集、传输、推理、队列和执行响应。某张图像代表 100 ms 前的场景，而它经过处理后才产生动作。

延迟使决策依据落后于实际状态。物体运动时，旧位置误差随速度和观测年龄增长；接触切换时，即使位移不大，受力模式也可能已改变。因此补偿需要判断任务变化的时间尺度，不能只追求离线位置精度。

状态可加入历史观测、已下发动作、时间戳、队列等信息，用以推断当前状态；也可使用延迟补偿。但估计仍依赖运动与延迟假设，接触突然变化时可能不准确。

动作块是在减少反馈次数

动作块 action chunk 一次输出多步，随后执行部分或全部。它减少模型调用，也可能让相邻动作连贯；代价是中途不一定重新看到滑动、碰撞或目标变化。

20 Hz 的四步控制对应 200 ms。若这四步期间物体滑落，重新规划可能太晚。预测块长度与实际执行长度要分别记录，未执行候选没有实测后果。

“给每个未来动作一个正确位置”仍可能不足：物体承受的加速度、夹持余量与接触阶段变化也影响是否可执行。动作序列的物理时间结构不是可随意压缩的文本长度。

为什么从位置差分得到速度与加速度会很吵

速度可近似为位置差除以时间间隔，加速度再对速度做差分。时间间隔越小，相同位置测量误差对差分影响越明显。机器人静止时，带噪位置也可能产生不小的估计加速度。

辨识和学习应尽量使用可靠速度读数、合适滤波或积分型目标，并保留采样时间。滤波会改变信号与延迟，不能只说“滤过以后更平滑”而忽略标签时序。

4.5.2 接触为什么让运动规则发生切换

桌面不是一种普通的向上推力标签

物体在桌面上时，桌面提供需要的支撑；物体离开桌面时，支撑消失。接触既取决于几何，也与允许的力有关。

理想无黏附刚性接触不能穿透，也不能凭空把离开的物体拉回来。设 d 是表面间隙，f_n 是压缩方向法向力

$$d \geq 0, \qquad f_n \geq 0, \qquad d f_n = 0.$$

这个互补关系说：分离时法向力为零；有压缩作用时，两面应接触。刚好接触也可能作用为零。它是一种理想模型，不要求你把真实表面微观变形也设为零。

约束如何决定支撑，而不是任意指定支撑

静止的 0.3 kg 方块在水平桌面上，垂直作用只有重力与支撑，则支撑为 2.94 N。手把它向上提，接触可能减弱并消失；手往下压，桌面支撑会增加。

接触力是满足运动、几何与作用条件的结果，不是每次固定等于 mg。多个接触点还存在作用分配问题，材料顺应与求解规则会影响具体结果。

从**世界模型**角度，相同位置附近可能有未接触、刚触碰、稳定支撑、即将脱离等不同情形。只预测连续位置而不区分模式，可能把不同未来平均在一起。

接触模式是“当前允许怎样相对运动”

常见模式包括分离、黏着、滑动、滚动和脱离。这里黏着 sticking 指接触面没有相对切向滑动，不一定表示用了黏胶。

刚体动力学在某个模式内可以相对平滑，模式改变时规律会突然变化。这类同时有连续状态和离散模式的描述叫混合系统 hybrid system。模型中显式分类模式或用**潜变量**表示，都是可能的设计。

理想接触：间隙、法向作用与切向作用

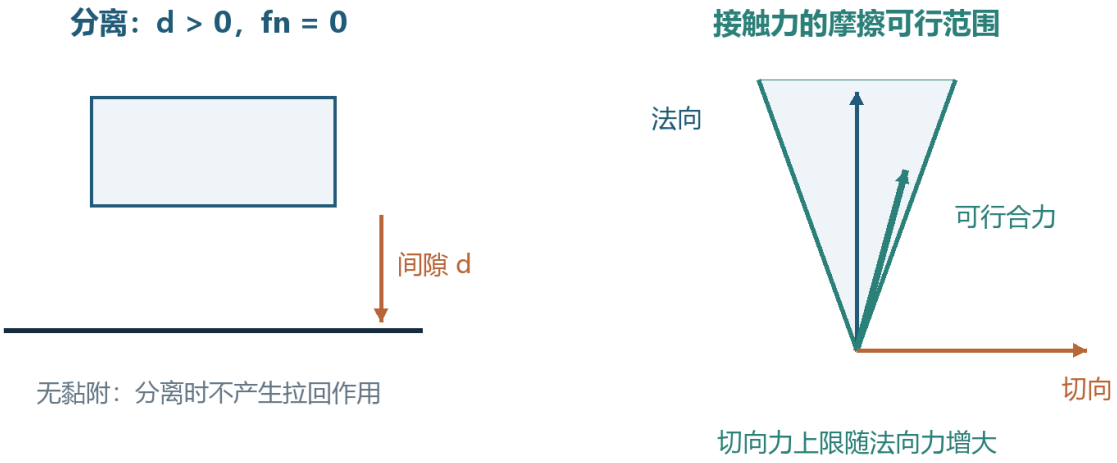


图4.5：接触几何、法向作用与摩擦约束。

软接触为什么允许一点穿入

真实物体和表面会变形。仿真也常用有限刚度近似接触，例如令压缩量 δ 产生弹簧与阻尼作用

$$f_n = \max(0, k\delta + b\dot{\delta}).$$

这是无黏附一维教学模型，具体适用范围需定义。压缩产生推力，阻尼随压缩变化率耗散；截到零避免该简化模型在脱离时产生拉力。其他软接触实现可能采用不同关系。

因此模拟中小穿入不一定是碰撞检测完全失效，也可能是软接触的建模结果。但穿入过大、作用不合理或被策略利用时，应检查参数、几何与积分设置。

接触状态为什么需要数据支持

视觉中两面距离很小，未必能辨认有没有真实接触。关节误差、作用传感、触觉和历史可以补充信息。

如果训练数据只包含稳定抓住的阶段，没有首次触碰和滑动恢复，模型很难学到模式切换。单步平均误差也可能把少见但决定成败的事件淹没。

接触与操作的模型化入口可参见 MIT 操作课程；本节的力与间隙示例是独立教学推演。

4.5.3 摩擦怎样决定推得动、抓得住和是否滑落

静摩擦不是永远等于 μN

法向力垂直接触面，切向力沿接触面。库仑摩擦模型 Coulomb friction 用法向压缩限制切向作用

$$\|\mathbf{f}_t\| \le \mu_s f_n.$$

静摩擦在这个范围内抵抗相对滑动，实际大小由运动与其他作用共同决定。它不是无论怎么推都固定取上限。滑动后，理想动摩擦通常按与滑动相反的方向给出相应作用，常用不同系数 μ_k。

摩擦锥 friction cone 是三维可行力集合：法向作用越大，允许切向作用越大。它给可行范围，不单独决定最终力。Modern Robotics 摩擦讲解。

推方块时，什么时候没有动

0.3 kg 方块静止在水平桌面，法向力为 2.94 N，设 $\mu_s = 0.4$。最大静摩擦约 1.176 N。水平推力 0.5 N 时，静摩擦可平衡它，方块仍静止。

推力超过这个阈值，在该简化条件下可能开始滑动。运动中还需考虑动摩擦、推的高度导致的翻转、接触形状和加速度。不能将“位移目标没被跟踪”都当作位置模型错误。

两指夹持为什么需要法向力

两侧手指分别压住竖直物体，夹持法向作用水平，相应摩擦可向上抵抗重量。设每指法向力大小 N，理想对称无其他接触时

$$2\mu_s N \geq m g_{\text{grav}}.$$

g_{grav} 是重力加速度。质量 0.3 kg、摩擦系数 0.4，静止提起所需每指法向力至少 3.675 N。这里 N 是每指作用，不是两指总作用，也不是夹爪厂商接口天然采用的数值。

两指夹持：水平挤压，通过摩擦向上承载

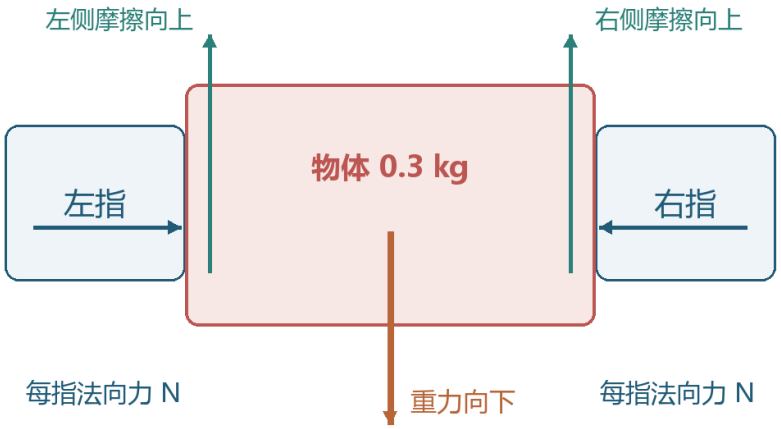


图4.6：两指法向夹持，通过摩擦支撑物体。

若还要求向上加速度 1 m/s²，条件变成 $2\mu_s N \geq m(g_{\text{grav}} + 1)$，下界为 4.05 N。搬运更快需要更大作用余量。实际还需检查几何、偏心力矩、材料变形与接口标定。

抓到几何正确位置，也可能抓不稳

接触点、表面方向与摩擦可行范围共同决定可抵抗哪些外部力与力矩。力闭合 force closure 是在相应理想接触假设下能够抵抗任意方向的小外部力旋量的一类性质。

它不保证执行器力足够、物体不会被压坏或实际摩擦已知。一个物体也可能不满足完整力闭合，却足以完成有限方向负载的当前任务。应按任务需要比较，而非把性质名称当作成功标签。

质量、偏心和抓取位置变化会改变搬运需求。语言模型能识别“杯子”，并不能因此知道这个杯子是否装满水。

滑动信息为什么适合进入学习

触觉图案变化、微小相对运动、夹爪状态与作用响应可以提示即将滑动。策略可以调整夹持、速度或重新抓取，世界模型可以预测不同模式的后果。

摩擦系数还会随材料、表面和状态变化。用一个固定 μ 作为所有真实物体的标签可能有偏；可估计范围、识别任务相关响应，或训练对合理变化稳健的行为。

4.5.4 碰撞、冲量和仿真步长怎样影响结果

碰撞发生得快，为什么改用冲量

力描述单位时间内动量改变；冲量 impulse 是作用在一段时间上的积分

$$\mathbf{j} = \int \mathbf{F}\, dt, \qquad m(\mathbf{v}^{+} - \mathbf{v}^{-}) = \mathbf{j}.$$

$\mathbf{v}^{-}$ 和 $\mathbf{v}^{+}$ 是碰撞前后速度。峰值力依赖碰撞持续时间，冲量则直接描述整体速度变化。快速碰撞中相机可能只看到前后两帧，无法观察完整作用曲线。

一个球碰墙的算例

取指向离开墙面的法向为正。质量 0.3 kg，碰前速度 -0.2 m/s。若简化恢复系数 restitution coefficient 为 0.5，则碰后法向速度为 0.1 m/s，冲量为 $0.3(0.1 - (-0.2)) = 0.09$ N·s。

同一冲量在 10 ms 内平均作用约 9 N，在 1 ms 内约 90 N。只测低频平均作用可能遗漏峰值；轨迹结果相似不意味着接触作用过程相同。

这个恢复系数模型描述特定法向碰撞关系，不能替代所有摩擦、转动和软材料影响。

数值积分在做什么

连续动力学给加速度，模拟器必须按有限时间步更新速度与位置。一种半隐式 Euler 教学写法是

$$\mathbf{v}_{k+1} = \mathbf{v}_k + \Delta t\, \mathbf{a}_k, \qquad \mathbf{q}_{k+1} = \mathbf{q}_k + \Delta t\, \mathbf{v}_{k+1}.$$

这里适用于对应的普通坐标示例；三维自由刚体的姿态要用相容的旋转更新，不能照搬位置加法。

积分方法和步长影响误差与稳定性。接触越刚，响应时间尺度越短，过粗步长越容易遗漏或错误放大变化。更小步长通常提高计算成本，也不修正错误物理参数。

碰撞几何与可视几何不一定相同

渲染网格决定看起来怎样，碰撞几何决定在哪里发生作用。为了效率，模拟器可能用盒、球、凸分解等近似。细槽、薄边或工具尖端的近似会改变任务可行性。

接触求解器 solver 根据约束计算作用或冲量，还可能采用软化、有限迭代和数值容差。迭代次数不是摩擦系数，求解误差也不是物体真实随机性。需要分清物理参数与数值参数。

可微仿真也需要模型可信

可微 differentiable 指可获得某种输出对输入或参数的梯度，可能用于辨识、动作优化或训练。接触切换本身可能不光滑，梯度常依赖特定处理或近似。

梯度存在不表示方向在真机中有效。优化器还可能利用软接触穿入、错误弹性或求解器偏差。应先看独立物理后果与任务结果，再解释优化收益。

4.5.5 接触是条件改变，不只是一个连续的小扰动

夹爪离开物体时，物体不受夹爪约束；夹住后，物体运动与夹爪耦合；滑动后，这种关系再次改变。单个平滑函数可以近似部分现象，却可能在切换附近丢失关键规律。模型输入应包含足够接触线索，评价要在接触前、建立接触、滑移和释放阶段分别检查。

摩擦参数与接触几何共同影响抓取。把仿真摩擦调大使任务轻易成功，并不证明策略稳定；把参数任意随机到不现实的范围，也可能把学习目标变成无法完成的任务。应从测量和失败记录构造合理范围，再分别测量范围内鲁棒性和范围外退化。

4.5.6 练习题

题4.5-1 接触从滑动变为粘着时，为什么单个平滑动力学网络容易在转变处预测错误？只降低一步预测误差是否足够？

4.5.7 参考答案

题4.5-1答案 滑动时摩擦力与相对速度方向相关；粘着时相对速度为零，摩擦力在允许范围内抵消外部切向作用。两种情形满足的关系不同，临界附近小小的力或速度变化可能决定物体继续滑、停住还是重新滑。

单个平滑网络若没有接触历史、受力或模式信息，同样的可见状态可能对应不同后果；用平方误差训练时还可能在“滑动”和“停止”的标签之间做平均，预测一种现实中难以发生的缓慢漂移。即使输入完整，跨转变区域逼近平滑函数也可能把转变边界抹宽。

只降低全数据的一步平均误差不够，因为非接触或稳定阶段通常很多，少量边界错误会被平均掩盖；规划把这些小错重复推进后，可能预测假接触、错失抓取或过度推进。应增加转变附近的数据，比较粘滑判定、接触时间、长段滚动与候选动作排序。可加入接触/历史信息、分模式模型或物理摩擦约束，并验证它们是否改善实际接触操作，而非只改善总体误差。

4.6 抓取、推拉、插入、工具与柔性物体

4.6.1 推、拉、抓和插入，各需要哪些物理理解

推物体，是改变接触系统而非复制手的轨迹

夹爪在桌面上向右运动，物体是否同向移动取决于有没有接触、推力是否超过摩擦、推点是否导致转动。物体运动还受桌面支撑与接触分布影响。

如果推在偏离物体中心的位置，既产生平移也产生转动。学习模型只用“末端位移”预测“物体位移”，却不输入接触几何或能够推断它的观测，可能把不同后果平均掉。

准静态 quasi-static 是假定惯性相对其他作用足够小，近似使用作用平衡描述过程。它可能适合慢推，但启动、急停、碰撞和快速滑动不一定满足。采用这个假设应验证误差，而不是只因为任务名字叫“推”。

拉抽屉，轨迹必须符合约束方向

理想抽屉主要沿滑轨运动。手向侧面拉，未必增加开口，反而可能增加滑轨接触作用或卡住。

约束 constraint 限制允许的运动。几何上可通过关节或接触表示，物理上还产生抵抗被禁止方向的作用。**世界模型**应区分“沿允许方向走”和“使约束变紧”的候选动作。

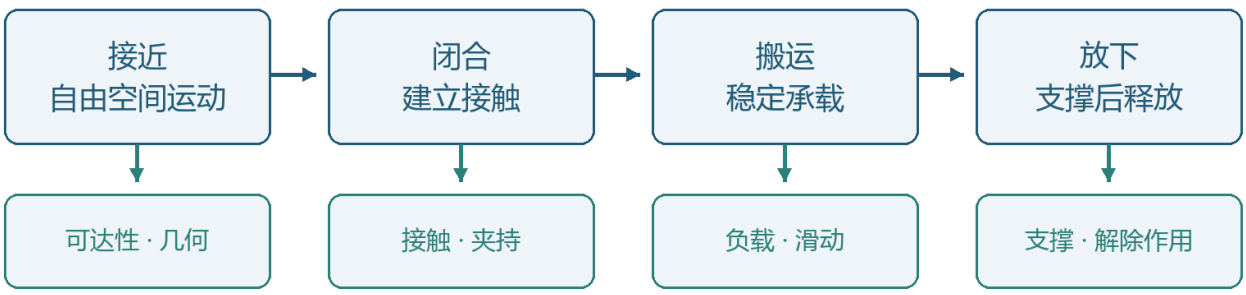
阻力还可能来自摩擦、锁扣、弹簧或物体挤压。图像看不出原因时，有限接触尝试与作用历史可能提供信息；同一局部图像不一定决定唯一正确动作。

抓取应拆成接近、闭合、承载与释放

接近需要可达性与碰撞几何；闭合需要正确接触与可行夹持；承载需要抵抗重力、加速与转矩；释放需要在合适支撑条件下解除夹持。

一个分类器只判“夹爪围住了物体”，可能没有检验承载稳定。奖励或数据标签应明确阶段目的，避免把接触发生当作整个任务成功。

任务阶段改变，接触模式与所需信息也改变



目标到位、接触正确、物体稳定是不同事件；转换时需要新的反馈。

图4.7：操作阶段转换，伴随不同接触与观测需求。

动作块越长，跨越模式时越可能沿用旧假设。是否需要更短执行块、触觉反馈或阶段条件，应由任务失败和成本证据决定。

插入不是只找对几何中心

插孔操作同时依赖位置、姿态、接触方向、刚度与摩擦。略有偏差时，刚硬追踪可能使边缘作用迅速增大；适当顺应与反馈可能允许沿接触导向修正。

某个方向控制位置、另一个方向限制作用，是一种任务分解思路。这里需要理解受约束方向与允许运动方向，并非要求学习者推导一套完整混合力位控制器。

小范围试探可以辨认孔壁或卡住模式，但训练需覆盖相关状态。只收集无偏差成功演示，模型不一定学会接触恢复。低速度可能减小冲击，不自动消除几何锁死。

动力学与任务语义相互制约

“小心放下易碎物体”需要语义识别目标，也需要控制接触速度与作用。“拧开盖子”需要知道任务顺序，也需要合适力矩、摩擦与约束。

物理信息不是语言模型之外的独立装饰。如果评价只检查最终外观，它可能忽略损伤或不稳定承载；如果只检查力很小，也可能永远不完成任务。任务定义要把目的与允许的物理行为连接起来。

操作中的顺应与接触利用，可进一步读 MIT 的机械臂接触操作讲解。

4.6.2 柔性物体、工具与液体，何时不能只用刚体

刚体模型的含义与代价

刚体 rigid body 假设内部距离不变，用整体位姿和速度简化状态，适用于变形可忽略的任务。

布料、海绵、软包装和线缆会形变 deformation。相同整体位置下，内部形状不同，后续抓取和受力也可能不同。只预测一个物体中心位置，会遗漏决定任务的状态。

需要多少变形信息应由任务决定。搬运一个封闭软袋，整体近似可能足够；整理布料折痕或穿过线缆时，形状、接触和张力更关键。

刚度、弹性、阻尼与滞回不是同一个词

弹性 elasticity 指变形后有恢复倾向。刚度是局部作用与变形的关系，阻尼描述与运动有关的耗散。滞回 hysteresis 表示当前响应还依赖过去加载过程，单看当下形状可能不足。

一维弹簧 $F = k\delta$ 是局部教学近似。k 为 100 N/m 时，压缩 10 mm 产生 1 N；软材料可能非线性、黏弹性明显，继续压缩不能保证仍用同一个 k。

控制器虚拟刚度和物体材料刚度会共同决定接触响应。看到力随位移改变，不能不加分离就把斜率叫作物体真实刚度。

工具改变的不只有末端外形

装上长工具，**TCP**、质量、质心、转动惯性和接触位置都改变。抓住工具柄时，工具尖端运动由末端姿态与工具几何共同决定。

例如工具尖端离夹持点 0.2 m，小转角 0.01 rad 可带来约 2 mm 横向位移。工具更长，姿态误差对接触位置的影响通常更明显。

学习工具使用，需要知道动作目标指夹爪还是工具尖端。跨工具数据若只对齐视觉类别，不对齐控制参照与负载，行为语义可能失配。

液体会改变负载与内部状态

装液体的杯子有总质量、液面与晃动 sloshing。杯子外部位姿相同，液体内部运动不同，急停后的后果也不同。密闭负载和开口杯子的任务约束不一样。

是否需要显式流体模型取决于目标。慢速搬运可尝试更简单状态或历史表示，快速倾倒与防溢出可能需要更丰富预测。加入流体求解器也不能替代数据与反馈验证。

学习表示应保留任务需要的物理区别

潜在表示 latent representation 不必逐维对应质量或张力，但应能区分导致不同行为后果的状态。若关心布料是否已折好，仅编码纹理而遗漏形状与接触，就可能失败。

可以用形状、接触、张力或任务结果监督表示，也可通过动作条件预测学习。还要验证动作选择和任务后果；表示可视化不足以证明有效。

4.6.3 柔性物体需要改写“状态足够”的假设

毛巾的一个角点位置不能唯一确定褶皱、张力和隐藏折叠；电线两端坐标也不能唯一确定中间接触。刚体位姿仍可描述夹爪，却不足以描述被操作对象。可以采用关键点、网格、图结构、图像历史或潜在表示，选择取决于可观测信息和任务所需精度。

工具使用进一步改变动作与效果之间的关系：夹爪末端对准，不等于工具工作端对准。应标定或估计工具几何，记录接触和力的传播，并检验新工具的有效范围。**表示学习**的价值在于捕获任务相关差异，不能因为用了图网络就假定所有物理约束已经学到。

4.6.4 练习题

题4.6-1 视觉上夹爪已围住物体，却在提起时滑落。单纯增大夹紧命令是否合理？说明至少三个可能原因与需要的观测。

4.6.5 参考答案

题4.6-1答案 盲目增大夹紧命令不合理。至少应区分以下原因与证据：

1. 摩擦或夹力不足：平行夹爪的简化抗滑能力约随 μN 增加，物体重量或抬升加速度超出它就会滑。需要估计负载、材料摩擦，并观察实际夹力与滑移，而非只读目标开合值。

2. 抓取几何不合适：接触只在边缘、物体重心偏离或杯壁形状使夹持容易转动。需要接触位置、物体姿态和提起后的转动轨迹。增加力也可能把物体挤出去，应调整抓取位置与方向。

3. 夹爪没有真正达到命令：位置控制、力限幅、行程限制或硬件饱和使“闭合更多”不等于“受力更多”。需查看编码器、实际夹力/电流与控制模式。

4. 抬升动作太快或有碰撞：突然加速增大所需摩擦力，擦到柜沿也会施加额外扰动。需同步检查末端加速度和环境接触。

在确认确为夹力不足且物体可承受时才适度增加力；否则应改接触几何、降低加速度或修执行接口。软物体、薄壁杯还要考虑形变与损伤，因此动作选择依赖实际观察到的失败机制。

4.7 物理测量、系统辨识与可辨识条件

4.7.1 机器人能测到什么，又有哪些物理量是隐藏的

本体感知补充图像看不见的信息

本体感知 proprioception 是机器人自身状态读数，例如关节位置、速度、夹爪开度、驱动或作用相关量。它帮助辨认身体配置、跟踪状态与执行限制。

图像相似但关节速度不同，下一动作结果可不同；末端位置相同但身体配置不同，可用运动方向可不同。把本体状态加入策略或**世界模型**，应说明它提供的决策信息，而不只说“多模态融合”。

传感器读数仍有噪声、滤波、时延和标定误差。数据里的 effort 字段可能是指令、测量、估计或某种代理量，必须查看接口定义，不能默认就是准确接触力。

力力矩传感器测的是指定参考处的合成作用

腕部六轴传感器提供力与力矩，但读数可能包含工具自重、加速产生的作用、接触、偏置和噪声。传感器坐标与世界坐标也不同。

接触估计通常需要空载标定、负载重力和必要动态影响的处理。不能把非零读数全部标成“碰到物体”，也不能在工具更换后继续使用旧补偿。

若由关节力矩残差估计外部作用，还依赖本体动力学模型。模型不准时，负载或摩擦误差会被解释为外力。估计器输出与直接传感真值是不同证据。

触觉能提供局部接触结构

触觉 tactile sensing 可提供接触位置、压力、剪切或表面形变的相关信号，具体输出取决于传感器。它有助于区分接触、滑动趋势与抓取稳定。

触觉图像并不天然等于牛顿单位的压力地图，需要标定或学习。其监督可以来自已知加载、滑动标签或任务结果。输入、输出与标签应按具体方法解释。GelSight 的原始研究是理解从局部形变推断作用的入口：GelSight。

视觉提供全局几何与目标，触觉提供局部交互信息。融合是否值得，需要与同数据、相近容量及实际延迟的对照比较。

IMU 的加速度读数需要正确解释

惯性测量单元，Inertial Measurement Unit，缩写 **IMU**，通常包含角速度与加速度相关读数。理想加速度计感受的是比力 specific force，即与引力作用区分后的非引力作用响应，并在自身坐标表达。

静止放置的加速度计常报告约一个重力加速度大小的读数，并非物体真的在持续向上加速。根据姿态和传感器约定恢复惯性系加速度，需要正确旋转与处理重力。

角速度积分与加速度积分会累积偏置误差。仅靠它们不能无条件恢复精确全局位置或所有姿态方向。移动和腿足学习要核对实际提供的姿态估计来源。相关接口约定可读 ROS 官方 IMU 草案，并核对实际传感器驱动。

为什么历史能帮助识别隐藏物理状态

质量、摩擦、接触是否滑动、驱动响应等不一定能从一帧图像识别。施加动作后观察响应，可以提供额外信息。

例如同样的小推力，一个物体动了，一个没动。差别可能来自摩擦、质量、接触状态或实际执行力；历史有信息，但不自动唯一确定原因。

状态估计 state estimation 的目的，是综合观测、模型和历史得到对当前状态的判断。学习式历史编码器可以近似这种功能，应验证其在关键隐藏条件下是否真的改善预测或行为。

4.7.2 动力学数据应该记录什么

一条转移不是随意拼接前后两帧

动力学数据需要表达“在什么状态，施加了什么作用，经过多久，发生什么”。若只保存两张图，没有准确动作、时间和执行条件，就难以解释因果与转移关系。

可以把一条学习转移写成 $(x_t, u_t, \Delta t, x_{t+1})$。这里 x 是所选状态表示，u 是该模型定义的输入。若 u 是高层目标，模型包含执行闭环；若是**校准**后的实际作用，模型所指对象可能不同。

一个可用记录的字段

字段	物理含义与用途
观测及采集时间	当时场景，需与动作生效时间对齐
关节位置、速度、末端位姿	身体配置与运动，核对坐标和单位
高层命令	策略真正选择的目标或动作块
执行器或控制器输出	可能更接近实际驱动，需要注明来源
实测运动与接触相关量	结果与辅助监督，区分传感和估计
控制设置与机器人配置	控制器、刚度、工具、频率与归一化
环境和物体条件	几何、负载、材料或可获得的条件标签
结束、人工干预与缺帧	防止跨回合、跨接管或时间间断拼接

不要求每个数据集拥有所有真值。需要明确缺哪些、怎样估计，以及哪些推断因此不能被验证。

记录目标和记录执行结果，训练的是不同对象

行为克隆，Behavior Cloning，缩写 **BC**，用数据动作训练策略。目标位姿标签教模型发出相应目标；实际末端位移标签描述执行结果。二者不应在不了解控制器的情况下直接互换。

动力学预测则把命令和状态映射到执行结果。若把结果又当作输入动作，模型可能接收到部署时不可能预先知道的信息。应在采集与训练时明确字段职责。

多机器人数据需要对齐动作参照、单位、时间和接口。Open X-Embodiment 提供跨机器人数据与学习的研究入口，但共享动作维度并不保证相同物理语义。Open X-Embodiment。

数据覆盖决定哪些动力学可辨认

只收集静止或匀速轨迹，很难估计惯性。只在稳定接触中采集，很难学会首次碰撞与滑动。只录成功，不一定覆盖恢复状态。

可以设计有信息且与设备能力相容的激励和任务变化，使质量、速度、作用与**接触模式**有可区分响应。数据量增加而条件完全重复，未必增加可辨识信息。

接触切换样本少却影响成败，训练可采用分阶段抽样或加权，但要评价它是否改善关键事件而非仅减少总体**均方误差**。

测试集要保留真正独立条件

随机拆相邻视频帧可能让同一轨迹、物体和参数出现在训练与测试中，低估泛化难度。按回合、物体、物理条件或机器人拆分，更能检验相应主张。

辨识新的质量与摩擦、在旧参数下预测新动作、换控制器迁移，是不同测试。不要将它们合成一个模糊“泛化”分数。

4.7.3 系统辨识怎样从动作响应估计物理规律

辨识不是先假定参数真值已知

系统辨识 system identification 用输入输出数据估计模型参数或结构。质量、阻尼、摩擦、驱动响应与时延，都可能是辨识对象。

先选模型，再确定可观察量和输入，最后比较模型预测与真实响应。若模型结构错误，精确优化参数也可能得到有偏结果。辨识不能自动解决不可观察或数据不充分的问题。MIT 系统辨识课程提供更深入的理论入口。

用两条数据识别质量与阻尼

考虑水平低摩擦导轨上的一维物体，忽略干摩擦与其他水平外力。F 是已校准的实际水平作用。

模型将输入作用拆成加速与速度阻力两项

$$F = m\ddot{x} + b\dot{x}.$$

m 描述加速所需惯性，b 描述与速度相关的阻力。两次记录如下：

记录	加速度 m/s²	速度 m/s	实际作用 N
1	2	0.2	0.7
2	1	0.4	0.5

得到 $2m + 0.2b = 0.7$ 与 $m + 0.4b = 0.5$，解得 m 为 0.3 kg，b 为 0.5 N·s/m。它们与前面的质量示例兼容，但这是特定导轨教学模型，不是夹爪桌面摩擦模型。

多个带噪记录可以写成回归问题，最小化模型作用与实测作用的误差。要使用更多数据、合理权重与不确定性分析，而不是把两条记录的精确解当成真机精度。

为什么“激励”比样本数量更关键

若所有数据都没有加速度，质量项消失，无法从这些记录估计 m。若加速度和速度始终成固定比例，两个参数的影响也可能无法分开。

激励 excitation 指使不同模型因素产生可区分响应的输入变化。它不是无约束随机动作，而是根据辨识对象与设备能力设计信息来源。

例如需要区分惯性与阻尼，就应有不同加速度和速度关系；需要区分静摩擦与动摩擦，就要覆盖启动和运动。更多相同匀速数据不能补齐缺失维度。

为什么不能用位置目标冒充实际作用

上一例 F 是实际作用，不是位置命令。若输入是目标位移，实际作用还由控制器、误差、饱和和状态决定。直接把位移当牛顿代入方程，单位与因果链都错了。

可选择记录校准后的作用，也可把整个闭环响应当作辨识对象，例如识别“目标序列如何影响实测位置”。后者依赖当前控制器设置，不能宣称得到纯物体质量而忽略执行系统。

类似地，从推力与运动猜摩擦时，要核对法向作用、接触位置、质量和作用测量。多个因素可以互相补偿，让参数估计看似拟合好却语义不正确。

主动感知怎样帮助学习

主动感知 active perception 或 probing 指通过行为获得原来缺少的信息。例如轻微移动观察负载响应、改变夹持观察滑动趋势。

行为既要推进任务，又可能帮助识别状态，这是一种信息与任务效果的取舍。可以显式设计信息目标，也可用历史条件策略学会适应。但如果数据没有相应差异，模型不能仅凭架构恢复未知物理量。

4.7.4 可辨识性决定数据能否回答参数问题

假设 $x_{t+1} = ax_t + bu_t$，采集时总令 $u_t = 0.5x_t$，便只得到组合系数 a+0.5b。回归矩阵两列相关，多组 a、b 都解释同一数据。需要独立变化的输入，才能分别辨认两个参数；增加相同激励的数据主要减小组合估计的噪声。

配套实践采用真实教学参数 a=0.93、b=0.18。独立变化 x 和 u 时，最小二乘估得约 0.93129、0.17909；共线输入时，另一个参数对约 0.816、0.408 也能使训练残差接近零。两组参数在新控制输入下给出不同预测。因此“训练残差小”与“参数被辨识”必须分开。辨识知识可结合 机器人与感知教材的学习章节继续学习。

实际辨识还要检查测量噪声、输入误差、延迟、摩擦和执行器动态。选择激励时兼顾安全约束、频率覆盖与设计矩阵的条件数；在未用于拟合的动作波形、负载和速度下验证预测，再决定是否把参数送回仿真。

4.7.5 从多条响应到参数回归，什么时候能得到可信的质量

将上一小节的 $F = m\ddot{x} + b\dot{x}$ 写成矩阵，每条记录提供一行 $[\ddot{x}_i, \dot{x}_i]$，组成设计矩阵 X，参数向量为 $w = (m, b)$。回归最小化 $\|Xw - F\|^2$，其驻点满足 $X^T Xw = X^T F$。这不是又训练了一个任意神经网络，而是用数据确定已选物理结构里的系数。

两条示例记录给出 $X = \begin{bmatrix} 2 & 0.2 \\ 1 & 0.4 \end{bmatrix}$、$F = (0.7, 0.5)$，求解得到 $(m, b) = (0.3, 0.5)$。实践中采用最小二乘求解器，通常不显式计算矩阵逆。

若所有加速度始终是速度的两倍，X 的两列相关，数据只能识别 $2m + b$。把样本数量增加十倍也不能把这两个参数分开。若两列近似相关，测量噪声还会显著放大参数波动。改变加速、匀速与减速阶段，使它们在数据中具有不同响应，才增加了可辨识的信息。

导数标签的噪声、未校准力、执行延迟或遗漏摩擦也可能被参数吸收。因而既要检查拟合误差，也要换未参与拟合的动作波形与负载，比较预测，并在需要时与独立测量核对。把学出的参数送回仿真，成立条件是这套模型与实际接口仍对应。MIT 系统辨识教材

4.7.6 练习题

题4.7-1 辨识 x下一=ax+bu，只采集 u=0.5x 的数据。为什么不能分别可靠估计 a 和 b？怎样设计更有信息的采集？

4.7.7 参考答案

题4.7-1答案 把采集规律 u=0.5x 代入：

$$x_{t+1} = (a + 0.5b)x_t.$$

数据只能确定组合 c=a+0.5b。对任何数 η，令 a′=a+η、b′=b−2η，会得到相同 c，因此所有现有样本都无法区分这些参数。增加同样闭环关系下的样本，能更精确估计 c，却不会增加区分 a、b 的信息。

从线性回归看，设计矩阵的两列 [x_t] 与 [u_t] 成比例，秩不足；(X^TX) 没有可用于唯一求解两个参数的逆。正则化可以选出一个数值解，但这个解由额外偏好决定，不表示参数被数据识别出来。

采集时可在保证安全的范围内加入独立扰动 u=0.5x+e，或用不同反馈系数与多种输入频率，使 x、u 不再总成比例。检查设计矩阵的最小奇异值、在新输入下的预测及参数不确定性。辨识目标是未来改变控制时仍能预测后果，所以测试也应包含训练中未使用的输入关系，而不只重复同一控制器。

4.8 学习动力学、物理先验与机器人适应

4.8.1 学习动力学模型时，输入、标签和结构怎样选

先定义模型预测哪一个系统

预测机器人本体、被操作物体，还是包含控制器的整个交互？这个选择决定输入动作、状态和监督标签。

一种可计算状态模型可以输入 $x_t = (\mathbf{q}_t, \dot{\mathbf{q}}_t, \text{物体状态}, \text{相关历史})$、命令 u_t 与时间间隔，输出下一状态或变化量。某些真值状态在部署时拿不到，就要使用观测编码与状态估计，或者明确只用于训练评价。

不是所有任务都需要同时输入全部物理状态，但省略相关变量会让模型面对不可区分的后果。例如忽略速度，相同位置向左滑与向右滑被混在一起。

预测变化量，怎样形成普通监督训练

网络 Δ_θ 可以预测状态变化

$$\hat{x}_{t+1} = x_t + \Delta_\theta(x_t, u_t, \Delta t).$$

标签来自真实相邻状态差，平方误差让预测接近记录后果。网络参数更新，不是把下一状态标签写进部署输入。对于姿态等非欧式变量，变化量应按相容表示计算，不能对任意四元数直接相加。

为什么常用变化量？短间隔下状态本身变化小，模型可以集中学习转移部分，减少复制当前状态的负担。它不是普遍保证；长间隔、接触切换和不当归一化仍可能使学习困难。

状态增量、加速度与图像预测，各承担什么误差

预测加速度可以与物理方程对应，但差分标签可能噪声大，积分还会累积误差。直接预测下一状态，避免显式加速度标签，却依赖训练间隔和动作持续定义。

预测图像提供视觉未来，但视角、遮挡和背景占据大量输出。视觉平均误差低不代表接触后果正确。潜在模型可以减少输出维度，但要检验表示是否保留速度、物体与接触信息。

选择输出应围绕下游决策：需要判断滑不滑，就应有能支持这一判断的预测与验证，而非只增加图像清晰度。

物理残差模型怎样结合先验与数据

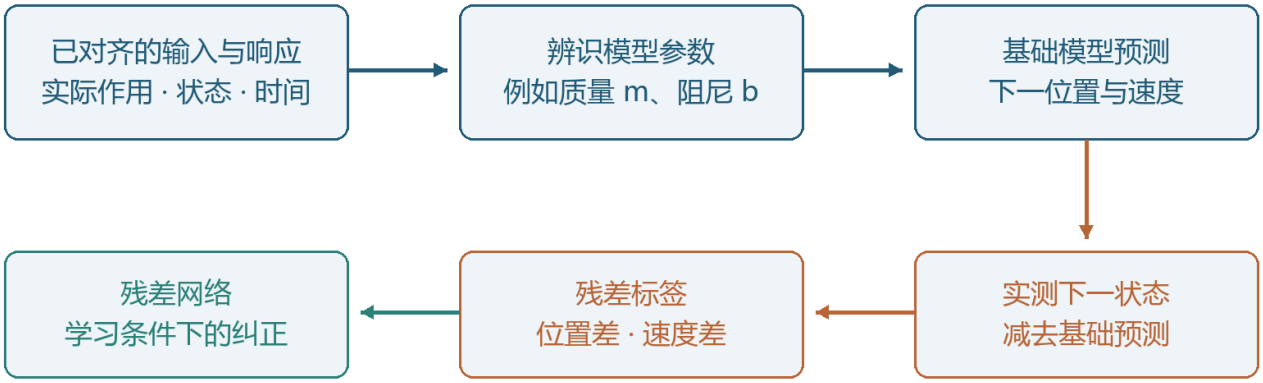
残差 residual 模型在基础预测上学习纠正

$$\hat{x}_{t+1} = f_{\text{phys}}(x_t, u_t) + r_\theta(x_t, u_t).$$

f_{phys} 可以是明确的简化物理模型，r_θ 输出预测残差。训练标签是实测下一状态与基础预测之差，或通过总体误差联合构造。

在第4.7节教学模型中，可用辨识质量与阻尼预测速度，再用网络修正未建模作用。但网络可能补偿错误质量、延迟或单位问题，而不是真实额外物理机制。残差大并不能直接解释成某个物理力。

从实测轨迹到物理参数，再到残差标签



残差可以混合多类误差；预测变好不自动说明识别了某个真实物理量。

图4.8：物理辨识与学习残差使用不同模型对象。

参数化质量、阻尼或摩擦时，可约束其合理范围；仅有正值也不保证完整物理一致性。若论文声称更可解释，应检验参数与独立测量的关系，而不只看预测精度。

不确定性输出为什么有用

概率动力学模型可以输出下一状态的均值和方差，表达在给定信息下仍可能出现的不同结果。**模型集成**则训练多个预测器，用它们的分歧提示规律尚不确定的区域。例如同一推杯动作在已知粗糙桌面上有小幅随机变化，单模型方差可以描述这种变化；遇到未见表面时，各模型可能给出不同平均位移，分歧便提供补数据的线索。实际网络也可能共同预测错，因此两类量都要用留出动作后果检验。规划器可以据此对同一动作模拟多个未来，再选择平均或风险敏感的评分，而非只比较一条均值轨迹。

PETS 是 Probabilistic Ensembles with Trajectory Sampling，概率集成与轨迹采样。它使用概率动力学与集成处理预测不确定性，是学习动态和决策结合的代表入口。PETS。

任务中可以在预测不可靠时缩短展开、请求真实反馈或限制动作搜索。但必须**校准**“不确定性高是否真的容易错”，不能把方差大简单等同于危险、方差小等同于可信。

多步训练与观测校正是两个过程

训练时将预测状态再次送入模型，可以测自主多步展开误差；每步输入真实状态则主要测局部转移。二者解决不同问题。

部署时新观测可校正状态，但预测未来期间没有真实未来标签。模型规划若每步借用真实未来观测，评价结果会过于乐观。

4.8.2 仿真迁移与动力学适应为什么需要条件化

从仿真到现实，差异发生在哪条链上

Sim-to-real 指从仿真训练或设计迁移到真实系统。差异可来自物理、接口、观测与计算：摩擦不同、负载未知、控制器实际响应不同、图像延迟或视觉分布变化。

如果主要错误是末端动作坐标，随机化背景不能修复它；如果主要错误是接触太硬，增加图像纹理也不能修复。先定位相关后果，比无差别扩大随机化更有用。

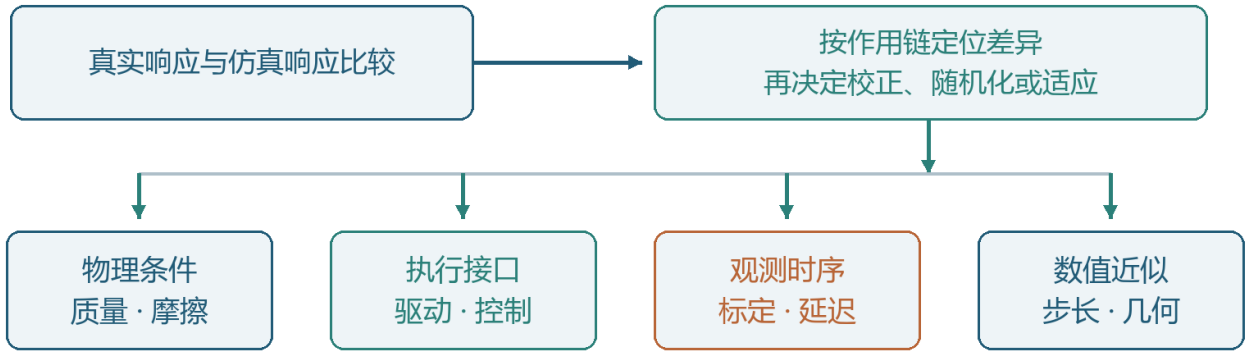
域随机化为什么要改变有意义的条件

域随机化 domain randomization 在训练中改变物体、机器人或观测条件，使策略不只适应一组设定。动力学随机化是其中与质量、摩擦、驱动等有关的部分。

参数范围应覆盖合理变化，并保持重要物理关系。任意改变质量却保持不相容的惯性，可能制造怪异对象；每帧独立更换一个本应整回合固定的物体质量，也改变了学习问题。

物理参数可以回合固定、缓慢变化或事件改变，但要与任务假设相符。传感噪声与参数未知不是同一种随机性。

迁移之前，先定位哪类关系发生偏差



应改变与失败机制相关的条件，保持参数变化和数据语义有物理意义。

图4.9：把迁移误差定位到物理、执行、观测与计算。

稳健策略与适应策略回答不同问题

稳健 robust 策略希望在一组可能条件下都表现合理，可以不显式判断当前参数。适应 adaptive 策略根据历史响应调整行为或内部表示，更针对当前条件。

例如未知重量时，稳健策略可能统一采用较慢搬运；适应策略可能从承载响应识别重负载，再降低加速度。谁更好取决于辨认成本、变化速度与任务需求，不是所有不确定性都值得在线辨识。

特权教师与历史适应器的分工

仿真中可给教师真实物理参数，训练参数条件策略。部署时参数不可得，就训练历史编码器输出内部条件，再让策略使用它。

历史编码器输入过去观测、动作和响应，标签可以是参数相关**潜变量**、教师表示或任务学习信号。输出不一定逐项等于真实质量与摩擦，应按训练目标解释。

RMA，即 Rapid Motor Adaptation，快速运动适应，在腿足机器人中研究这类快速适应思想。RMA。抓取负载也有类似的信息结构：训练时可读取重量，部署时只能从历史响应推断。

适应器为什么可能辨认不出来

历史没有有区分力的动作响应，就缺少辨识信息。控制器过强、传感器噪声大或动作很单一，都可能使不同条件看起来相同。

参数突然变化还会让过去信息失效。应检验适应延迟、变化后的恢复、对未见参数的表现和信息消融，而不只展示编码器输出聚类。

对照可包括无历史策略、同容量历史策略、参数真值策略，以及简单保守执行。特权真值方法是信息更充分的参照，不是可部署公平竞争者。

4.8.3 动力学怎样影响 VLA 与强化学习

不显式预测动力学，也仍然受它制约

模型自由 model-free **RL** 通常不以显式环境预测模型为核心更新工具。它仍通过真实或仿真交互，学到当前执行系统下哪些动作有较好长期后果。

若动作、控制器或时间条件改变，经验与价值估计可能失配。模型自由不是“没有动力学”，而是改善策略时不必显式构造可查询的转移模型。

VLA 行为克隆同样依赖演示的执行条件。模型能生成相似动作标签，不等于换负载、换接口后产生相似物理轨迹。

动作空间在分配谁处理什么困难

末端目标接口让策略关注任务几何，底层系统承担部分跟踪；关节目标接口更暴露身体配置；力矩接口让策略直接处理更多驱动与动态关系。

接口是否简化学习，需要看任务。插入需要的顺应若完全被固定刚度隐藏，策略可能只通过缓慢移动间接调作用；额外给刚度或力目标，可能提高表达能力，也改变对照的公平性。

比较方法时应说明动作维度、坐标、尺度、执行周期和控制器，而不只说明用相同视觉骨干。

物理历史为什么可成为重要表示

视觉语言编码器善于语义与外观，不保证保存速度、滑动或执行器状态。可以加入关节、触觉、作用和动作历史，或通过预测与反馈训练任务相关表示。

历史编码长度应覆盖有用响应，又不能期待无限历史自然解决所有隐藏变量。验证可选择外观相似但物理不同的物体，比较无历史、真实历史与打乱历史的效果，并控制模型容量。

如果表示改进只让动作分类准确，却没有改善负载变化后的任务表现，动力学适应主张仍需更多证据。

奖励应表达目的，也要捕捉物理失败

奖励 reward 是训练反馈，回报 return 是未来反馈累计。最终放入是任务目的，掉落、压坏、错误接触或拖拽可能是物理失败。

动作平滑惩罚可以减少快速改变，但过强会让必要接触或恢复也被压制。力惩罚可以限制作用，但动作尚未推进任务时很小力也不算成功。不能把方便记录的物理量自动等同于最终质量。

成功验证最好独立检查物体与目标的真实关系。学习式奖励器看到“夹爪在盒子上方”就给高分，可能让策略利用感知漏洞，而非改善动力学。

动作块与信用分配需要实际时间

若一次宏观决策实际执行 k 个底层任务步，原每步折扣为 γ，相容的块奖励与后续折扣为

$$R_{\text{block}} = \sum_{j=0}^{k-1} \gamma^j r_j, \qquad \Gamma = \gamma^k.$$

这是把相同物理时间重新分组，避免仅因模型调用频率改变任务偏好。若提前终止，按实际执行与结束规则处理。

块级信号还可能不清楚哪一步造成滑动。缩短执行、增加阶段反馈或利用价值估计是不同解决思路，应根据计算与交互成本选择。

数据纠错应定位到物理后果

失败轨迹可以标出触碰、滑动、跟踪慢、动作不可达等事件。人类恢复提供相应状态下可用动作，RL 反馈提供长期质量比较，两类信号可以结合。

人工接管替换了策略动作时，不能仍当作原策略采样。应保存实际动作来源与干预边界，按监督或明确处理行为差异的方法使用。

4.8.4 世界模型怎样学到对决策有用的物理

世界模型的任务是区分动作后果

世界模型 world model 根据当前信息与候选动作预测后续状态、观测或任务相关结果。若它只生成常见成功画面，忽略动作方向，就不能可靠选择动作。

有用的物理预测需要在相似起点下区分：推力不足是否不动，接触是否使物体转动，夹持是否滑落，快搬是否损失稳定性。这些区别有时只占很少像素，却决定决策。

状态表示应保留哪些物理信息

显式状态容易检查位置、速度与作用；**潜在状态**可以由观测历史学习，但语义更难直接确认。关键不在是否长得像物理坐标，而在是否保存预测和控制需要的区别。

如果同一潜在表示混合了向左滑与向右滑状态，后续预测可能给出平均不动。可通过速度、接触监督、动作条件预测或更合适历史结构改进，并分别检验表示、动态与控制。

不能只凭表示重建出当前图像，就认为它包含了隐藏摩擦或**接触模式**。

模型规划中，优化的是候选动作

模型预测控制，Model Predictive Control，缩写 **MPC**，用模型展开候选动作序列，按任务代价评分，选较好序列，只执行开头部分，再用新观测重规划。

一种教学目标为

$$J(U) = \sum_{j=0}^{H-1} \ell(\hat{x}_j, u_j) + \ell_f(\hat{x}_H).$$

U 是候选动作序列，H 是预测长度。每步代价可以含动作或接触要求，终点代价表达目标到达。它们的单位与权重要相容。有限搜索通常只找到候选中的较好方案，而非保证全局最优。

若对动作求梯度，可冻结模型参数，但保留通过模型输出返回候选动作的梯度。更新 U 不等于训练模型参数，也不等于已经完成策略后训练。

接触误差为什么容易被优化器放大

模型稍微低估滑落，规划器可能选择极快搬运，因为在模型里既快又成功。模型允许穿入几毫米，优化器可能把它当作绕过障碍的办法。

低平均预测误差不足以排除这些问题。可以缩短展开、限制可信范围、使用多个模型、真实数据校正或独立反馈，但每种方案都有条件与成本。

真实新观测能纠正状态，却不能恢复已经掉落或损伤的物体。持续重规划减轻部分误差，不是万能保障。

物理约束应该约束哪个对象

可以约束预测速度、作用、接触不穿透、能量或动作可达性，也可提供接触事件与几何辅助标签。但如果隐藏控制器和未知外力没有被建模，直接要求所有观测严格满足某个简化方程，可能反而产生错误监督。

应先检查条件：式子描述机器人还是物体？作用是指令还是实测？传感是否同步？数据是否有真实负载与接触标签？

物理**先验**有价值的证据，应是有针对性的后果预测、动作排序或任务改善，而非损失多了一个物理名称。

4.8.5 适应方法到底在识别什么

历史编码器可能从几步运动推断负载、摩擦或电机响应，并把一个潜在向量提供给策略。这个向量不一定就是物理质量或摩擦系数，它可能同时编码相机、接触和任务阶段。若论文把它解释为某个物理因素，应通过控制其他变量、改变该因素和干预向量检查。

运行时适应需要经历足够有信息的动作。机器人停着不动时，许多动力学参数无法从历史识别。主动试探可以增加信息，但有成本和风险；教师训练使用特权物理参数，学生部署只能用历史时，应评价估计延迟和突然参数变化。

4.8.6 练习题

题4.8-1 用物理模型加残差网络预测下一状态，残差应输入什么、学习什么标签？什么时候它会把错误标定学进去？

4.8.7 参考答案

题4.8-1答案 先明确残差预测的是离散下一状态，而不是额外力矩。若物理模型给出 $\hat{s}_{t+1}^{\text{phys}} = f_{\text{phys}}(s_t, u_t, \Delta t)$，残差网络读取当前状态、实际控制命令、时间间隔及必要的历史或负载条件，输出 δ_θ。对于位置、速度等可直接相加的量，预测为 $\hat{s}_{t+1} = \hat{s}_{t+1}^{\text{phys}} + \delta_\theta$，标签是

$$\delta^* = s_{t+1} - \hat{s}_{t+1}^{\text{phys}}.$$

标签来自对齐的真实转移，损失更新残差网络参数；未来真值 s_{t+1} 只用于构造标签，不进入部署输入。

姿态若用旋转矩阵R表示，不能任意相加矩阵差。可以先算相对旋转 $R_\Delta = R_{t+1}(\hat{R}_{t+1}^{\text{phys}})^\top$，让网络预测用旋转向量表示的这项修正，转成旋转矩阵后，再以 $\hat{R}_{t+1} = \hat{R}_\Delta \hat{R}_{t+1}^{\text{phys}}$ 组合。这样得到的仍是旋转，而任意矩阵加法可能破坏正交约束。左右乘法应与选择的参考系一致。

残差可以补偿未建模摩擦或负载，但不会自动区分它们与接口错误。若外参、工具中心、单位或时间戳错误，真实下一状态与物理预测的差也包含这些偏差；网络可能在原环境学到补偿，换相机或修复标定后反而失效。应先校准可测接口，再用不同速度、负载与接触条件检验残差；比较正确标定与受控标定扰动，判断它学习的是物理响应还是测量偏差。

进一步阅读：MIT Underactuated Robotics：System Identification、Modern Robotics：Homogeneous Transformation Matrices。

4.9 物理操作项目、移动平台与研究验证

4.9.1 应用流程：从数据到稳定搬运

定义目标与系统，而不是先选大模型

任务是把红方块从桌面抓起并放进指定盒子。核心变化包括物体负载与接触表面，关注自主完成、掉落、接触质量和时间。

假设机器人支持末端增量目标与夹爪闭合命令，策略为 20 Hz，控制器更高频运行。图像、关节与夹爪状态可用；若有可靠力或触觉读数，也可加入。夹持力是否可直接设置，以硬件实际接口为准。

物理实验先做几何与执行对齐，再看负载与摩擦。对于单纯语义选错物体，不应期待动力学模型解决全部问题。

先检查没有接触时的执行关系

给几个代表目标，比较实际轨迹、时延与可达性，核对坐标、尺度、**TCP** 与控制设置。记录高层命令和真实运动，避免把两者混成一个动作标签。

然后比较有负载与无负载响应。若负载使跟踪误差变化，就检查执行能力、重力补偿或策略信息，而非立即把误差全归因于感知。

用简单数据估计一个可解释转移

作为可手算子任务，使用第4.7节导轨数据，得到 m 为 0.3 kg，b 为 0.5 N·s/m。取当前位置 0.2 m、速度 0.1 m/s、实际输入 0.7 N，预测间隔 0.05 s。

不含干摩擦的基础模型给加速度约 2.1667 m/s²。半隐式一步给下一速度约 0.20833 m/s，下一位置约 0.2104167 m。

再构造一条含额外 0.05 N 反向阻力的教学记录。按相同一步更新，其加速度为 2 m/s²，下一速度为 0.2 m/s，下一位置为 0.21 m。基础模型的残差标签因此为速度约 -0.00833 m/s、位置约 -0.0004167 m。

对象	输入	输出或标签
基础物理模型	当前位置、速度、实际作用、时间与参数	下一位置和速度预测
真实或构造记录	动作后的实际响应	下一状态监督
残差网络	当前状态、输入与必要条件	记录减基础预测的变化

上述算例用已知阻力构造残差标签。真实数据的残差还可能包含积分误差、未建模作用与测量偏差，需要结合独立实验辨认来源。

给夹持与搬运设定有物理意义的测试条件

0.3 kg 方块与两指对称摩擦系数 0.4 时，静止承载每指理论下界为 3.675 N；向上加速度 1 m/s² 时为 4.05 N。4 N 在前者够、后者不够，说明相同抓取姿态在不同速度计划下可能失败。

若实际 μ 只有 0.3，后一条件下界升到 5.4 N。提高作用是否可行，还取决于材料、执行器、接触与接口。若只有开合命令，应标定它与实际夹持的关系，不能把理论 N 直接作为现成控制值。

测试应覆盖负载、表面与加速条件，而不只是重复同一个物体、同一慢速轨迹。相关条件应独立记录，才能判断模型真的利用了物理变化。

训练策略与训练动态，分别使用什么信号

策略监督用示范和纠正动作；动力学监督用动作后的真实状态与事件；价值或 **RL** 更新用任务回报及相应估计。三者可共享表示，但标签语义不同。

可以先用同数据**行为克隆**作为对照，再考虑历史、触觉或动态辅助。若加入**世界模型**，说明它是训练辅助、测试时筛选还是模型内交互。若加入 RL，说明奖励和概率或价值更新接口。

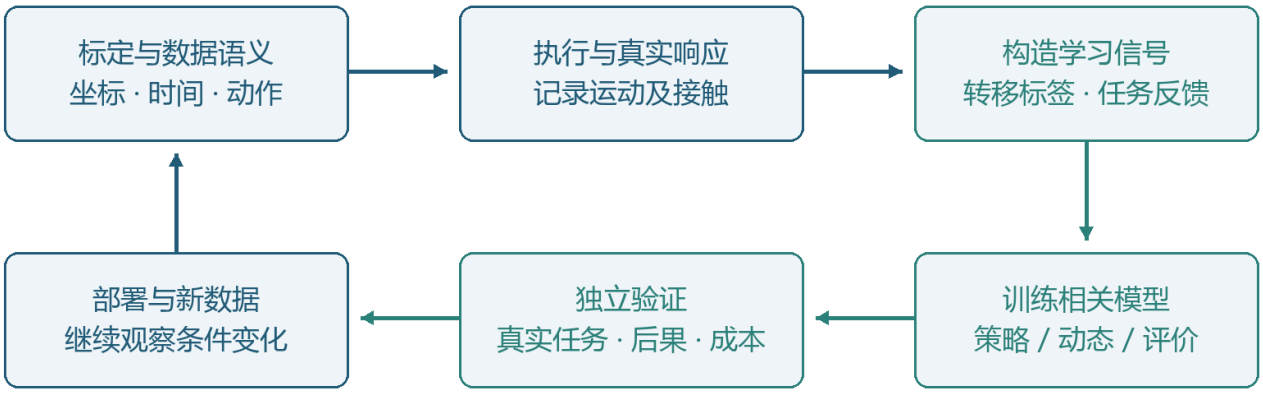
不需要一开始同时增加所有模块。根据已观察的失效选择，例如滑动信息不足优先检查感知和历史，目标正确但作用不足优先检查接口与执行，预测不区分负载则检查状态与数据。

部署闭环和独立验证

读取并对齐观测，生成目标，按实际接口执行，获得新状态与接触反馈，再重新决策。发生接管、重置或缺帧时，按真实边界记录。

比较同数据、同控制器、同时间预算的基础策略与新方法。完成率、掉落、完成时间和必要作用指标来自独立评价，而非全部由训练中的世界模型或奖励器确认。

具身项目闭环：物理语义、学习与独立验证



每个模块有不同对象和标签；训练分数与真实任务结果分别检查。

图4.10：项目中的物理、学习、执行与验证闭环。

4.9.2 移动与腿足机器人，还需要哪些共通知识

普通轮式底盘不能任意平移

对于理想无侧滑差速或车式运动的简化模型

$$\dot{x} = v\cos\psi, \qquad \dot{y} = v\sin\psi, \qquad \dot{\psi} = \omega.$$

v是车体前向速度，ψ是朝向，ω是航向变化率。这是一种统一运动描述，实际允许的v, ω还取决于机构。差速底盘受到左右轮速限制；对以后轴中点为参考的理想车式模型，轴距为L、前轮等效转向角为δ时，$\omega = v\tan\delta/L$，有限转向角使它不能任意原地旋转。直接给“向左平移”目标，也不表示底盘能像全向机构一样执行。非完整轮式机器人。

这种对瞬时速度的约束叫非完整约束 nonholonomic constraint。它不表示最终任何侧向目标都到不了，而是允许路径与瞬时运动不同。打滑时，理想模型本身也会偏离。非完整轮式模型的教材入口。

浮动基座使全身动作与支撑相互关联

固定机械臂基座不动，腿足机器人身体可以平移旋转。浮动基座 floating base 的运动不能直接当作几个有电机的普通关节；身体加速依赖关节作用与地面接触。

三维姿态表示可能比角速度多一个坐标维度，例如四元数。因此位置坐标维度与广义速度维度不一定相同，不能机械把每个 q 分量都按普通二阶关节公式处理。

学习策略需要知道身体姿态、速度、关节、足接触与目标，具体可获得信息取决于机器人估计系统。

质心、支撑与角动量

质心 Center of Mass，缩写 CoM，是质量分布的整体位置。身体质心运动满足合外力关系；足地接触力、重力和其他外力决定加速度。

身体绕质心的角动量变化由合外力矩决定。只让脚的位置看起来正确，未必形成足够支撑或正确身体姿态。

水平地面、无额外作用、**准静态**等条件下，质心投影位于接触支撑区域内是一类常用平衡检查；动态行走中，速度、角动量、摩擦与未来接触也重要。不能将静态支撑多边形作为所有动态稳定的充分条件。

接触切换与欠驱动为什么增加学习难度

摆动足与支撑足允许的运动不同，着地又产生冲量与摩擦约束。机器人整体的一些运动不能被执行器直接独立指定，这叫欠驱动 underactuation 的相关问题。

策略需要学会在这些条件下组织动作。模拟器若给过强摩擦、错误着地恢复或没有执行延迟，可能让不现实步态成功。腿足 RL 的结果应同时核对执行模型与真实任务。

并行腿足训练可参见 Learning to Walk in Minutes；该工作提供具体系统入口，不意味着并行环境数量本身解决物理迁移。

对具身模型的直接启示

移动操作还要同时考虑底盘、机械臂、相机和负载。底盘加速可能改变抓取状态，相机随身体变化也会改变视觉坐标。

跨形态策略必须描述动作与身体能力，不仅对齐动作向量长度。可用统一目标接口、形态条件表示或分层执行，但都应验证转移与任务条件。

4.9.3 怎样评价动力学相关机制，怎样继续研究

先定义你声称改善了哪一层

第一层是物理响应或预测：位置、速度、力与关键事件是否准确。第二层是决策：预测是否把候选动作排对。第三层是任务：实际成功、掉落、接触质量与成本是否改善。

三个层次不等价。平均位置误差低，首次接触预测仍可能错；动作排序改善，控制器不可执行时仍可能失败。研究叙事应把问题、机制与相应证据对齐。

评价指标应对关键后果敏感

可以分别报告不同展开长度、不同物理条件、不同**接触模式**下的状态误差；接触发生、滑落与失败预测可用分类和**校准**指标。作用误差要说明单位、测量来源与参考坐标。

视觉指标可以描述画面，不替代动作条件正确性。对模型做移除或打乱动作的消融，结合关键物体后果与候选排序，检查它是否真的使用动作。

哪些对照最容易被遗漏

新方法用了触觉，基线没有；新方法控制更慢，基线更快；新方法控制器更顺应，基线固定刚度；新方法有真实参数，基线只能看图像。这些差异都可能解释提升。

应根据主张比较同接口、同信息、同数据和相近预算的方案。额外真值可以作为上界性信息对照，但必须标出它不能部署的条件。

如果主张物理残差有效，与同数据纯学习模型、基础物理模型及合适容量的替代模型比较；若主张历史适应有效，控制容量与历史信息，查看参数变化后的适应过程。

可验证研究问题比大标签更有用

“让 **VLA** 更懂物理”过于宽泛。可改成：“在外观相近而负载不同的搬运任务中，当前策略未能利用承载响应调整加速度，导致滑落。”

候选路径可能是加入相关历史表示、接触观测或动态辅助；需要检验这些信息是否可辨认负载相关状态、是否改变动作选择、是否降低真实滑落。它们是有条件假设，不能预设一定成功。

组合已有方法也可以形成贡献，关键是明确必要的机制联系和整体差异，检索目的与流程接近的系统。更换骨干或增加数据，可能有实证价值，但不自动成为新动力学原理。

建议的学习与诊断顺序

先画执行链与时间轴，检查动作语义、坐标、标定和持续时间；再检查自由空间跟踪、接触、摩擦和负载；随后考虑感知充分性、数据覆盖和模型结构；最后分析规划、策略更新与迁移。

这不是认定低层总是错，而是避免在物理接口未对齐时，把失败解释成高层智能不足。已确认语义错误时，直接研究语义和任务表示更合适。

读论文时做一张卡：预测对象、输入动作、隐藏状态、训练标签、物理假设、使用方式、对照和实际成本。能把重要公式放到这张卡上，就更容易理解具体贡献。

学到什么程度，就足以开展多数相关工作

你应能解释目标动作为什么不等于执行结果，坐标与时间为什么属于动作语义，惯性与接触如何影响转移，数据能识别什么，模型训练标签从哪里来，以及怎样检验物理知识是否改善行为。

需要更深的控制或机构推导时再读对应原始章节。本书的目标是给学习系统提供正确物理接口与判断能力，不要求先完成所有控制课程再开始具身研究。

4.9.4 具身研究需要动力学的哪些深度

操作方向首先掌握坐标、动作接口、接触、摩擦、时钟与**系统辨识**，能够解释一个策略为什么碰撞、滑移或震荡。移动平台需要轮地接触、底座状态与机械臂耦合；腿式平台还需要平衡、支撑和全身限制。可以按任务补深控制理论，但不能把纯控制课程全部搬来替代具身学习主线。

研究验证应区分“物理参数变化后仍成功”和“模型估计出了正确参数”。前者证明行为鲁棒性，后者需要独立参数或响应证据。两者都可能有研究价值，叙事应与实际实验一致。

4.9.5 练习题

题4.9-1 移动底盘和机械臂一起取杯，为什么单独证明机械臂末端可达还不够？完整可行性至少包含什么？

4.9.6 参考答案

题4.9-1答案 机械臂末端可达只回答“在某个固定基座位姿下，有没有关节配置能到达杯子”。移动操作还要求底盘能从当前位置走到这个基座位姿，整个过程不碰桌腿、障碍和自身；机械臂、底盘与杯子的动作应在时间上相容。

完整可行性至少包括：底盘运动约束与路径通行性；机械臂关节限位、速度和力矩能力；整机与物体的碰撞；抬升后的负载与底盘稳定性；夹持能否抵抗行走加速度；传感视野、定位与坐标变换的可靠性；以及控制延迟下实际能执行的轨迹。底盘是非完整约束系统时，还不能把任意平面方向的瞬时速度直接当作可执行命令。

例如手臂从远处向前完全伸展虽然碰到杯子，但可能使重心移出支撑区域；先把底盘靠近、手臂保持较短伸展才安全可行。验证可以先规划底盘与手臂联合轨迹，再在带负载和定位误差条件下测取杯、移动及放置全流程；仅报告末端到达率无法支持移动操作能力的结论。

第5章：世界模型、预测表示与想象决策

理解物理接口后，可以进一步学习行动后果的模型。本章从预测对象与潜在状态讲到模型训练、规划和想象学习，并分别评价预测与决策能力。

5.1 世界模型的预测对象、动作条件与因果边界

5.1.1 预测必须回答一个具体的动作问题

预测“杯子下一帧在哪里”与预测“如果夹爪向右移动会发生什么”不同。前者可能只利用过去轨迹惯性，后者要求模型理解动作干预。**世界模型**的对象可以是物理状态、图像、潜在特征、奖励和终止，但必须说明它预测的是哪一个，以及预测怎样被决策使用。

5.1.2 世界模型到底是什么，为什么机器人需要它

从一个具体任务开始

机器人收到指令：“把红色方块推到桌上的圆圈里。”它有摄像头、机械臂、夹爪和控制器。摄像头提供图像，机器人自身传感器提供关节位置等信息。动作可以是“末端执行器沿水平方向移动 2 厘米”，这里的**末端执行器**指夹爪、吸盘等直接与环境交互的工具。

一种做法是直接训练模型：输入图像和指令，输出动作。这是策略。另一种做法是先考虑几种动作，预测各自会使方块怎样移动，再选择最有利的一种。后一种做法需要世界模型。

世界模型（world model） 是对环境变化规律的可计算近似。在机器人学习中，它通常由一个或多个神经网络实现。它接收当前状态的表示、历史信息以及可能的动作，预测未来状态、观测或相关结果的分布。它不是只存储“世界知识”的文字数据库，也不必生成完整视频。

一个最简单的世界模型就是函数 $\hat{x}_{t+1} = f_\theta(x_t, a_t)$。这里 x_t 是方块当前的位置，a_t 是推的动作，$\hat{x}_{t+1}$ 是预测位置，θ 是可学习的网络参数。如果训练数据表明推 2 厘米通常只使方块移动 1.6 厘米，模型可以学到这种关系。帽子符号表示“模型预测”，它并不保证等于真实结果。

几个模型各自负责什么

策略（policy） 是从决策信息到动作的规则，记作 $\pi_\phi(a_t \mid b_t, g)$。用神经网络实现时，就是**策略模型**。**VLA** 常承担这个角色：根据视觉、语言和机器人状态输出动作。g 表示目标，b_t 表示机器人对当前环境的内部估计。策略可以输出一个确定动作，也可以输出动作分布。

世界模型回答“这样做会怎样”，策略回答“现在做什么”。**奖励模型** 回答“这个结果符合目标到什么程度”，通常输出标量。**价值模型** 回答“从这里继续按某个策略行动，未来总收益大概有多大”。它们可以共享视觉编码器，但训练标签和输出含义不同。

模块	典型输入	典型输出
策略模型	当前观测或内部状态、任务目标	动作或动作分布
世界模型	当前状态表示、候选动作	下一状态或未来观测分布

模块	典型输入	典型输出
奖励模型	状态、动作、结果、目标	即时奖励预测
价值模型	当前内部状态、目标	按指定策略继续行动的预期回报
规划器	世界模型、目标、当前状态	搜索得到的动作序列

规划器不一定是神经网络。例如，它可以随机提出一千条动作序列，调用世界模型预测，选取最好的一条。执行时还需要底层控制器把动作指令转换成电机控制。把这些模块混成一个“智能模型”，会使训练流程变得难以理解。

为什么增加一个预测模块

首先，真实交互可能昂贵。机器人试错会占用设备、需要复位，还可能碰撞物体。模型可以在计算机中比较动作，或产生训练策略所需的虚拟经验。其次，目标会变化。学到方块的运动规律后，可以针对不同目标位置重新规划，未必每次都重新训练策略。再次，预测提供一种诊断工具：如果动作执行后与预测明显不符，可以发现接触、感知或动力学估计出了问题。

这些收益都依赖预测在有关动作和状态上足够可信。模型若错误地预测“手穿过桌面仍能拿到方块”，规划器可能主动选择这个错误。更多虚拟试验只能让策略更熟练地利用该错误，并不会自动纠正它。

因此，世界模型不是免费的真实世界替代品。其实际价值取决于“预测是否保留决策需要的信息、是否覆盖将要执行的动作、使用它的成本是否值得”。

本书采用的范围

本书讨论四种主要用途：通过预测学习表示；通过模型规划动作；在模型中训练策略；生成用于训练和评测的交互数据。模型可以预测低维物理量、**潜在状态**、物体结构或像素视频。

世界模型的用法包括视频预测与行动条件动力学。两者可以共享结构，但规划需要明确的候选动作接口：给定同一起点、改变动作，模型应能比较不同后果。仅根据文本生成目标视频则更适合作为计划提议。

历史上，Ha 与 Schmidhuber 的 World Models 展示了学习视觉压缩和潜在动态、再学习控制器的组合。本书从这个模块分工继续展开，不把它当作世界模型的唯一结构。原始论文

一条贯穿全书的路线

```
真实观测 + 已执行动作 → 训练世界模型
当前观测 → 估计内部状态
候选动作 → 模型预测未来 → 根据目标评分
选择动作 → 真实执行 → 获取新观测
新观测既修正当前判断，也补充下一轮训练数据
```

理解世界模型，首先要能指认每个箭头上的数据。下面各章会把这些箭头展开成网络、标签、损失函数和决策过程。

5.1.3 真实状态、观测、历史和信念

摄像头看到的不等于完整状态

真实状态 s_t 是足以决定环境后续变化的变量集合。对于推方块，它可能包括位置、速度、摩擦、手与物体的接触关系、控制器内部状态。若当前状态与动作给定后，下一状态的分布不再依赖更早历史，就满足马尔可夫性质

$$p(s_{t+1} \mid s_{0:t}, a_{0:t}) = p(s_{t+1} \mid s_t, a_t).$$

这个等式是对建模变量的要求，不是“所有真实机器人观测天然满足”的定理。

观测 o_t 是机器人实际获得的传感器信息，如图像、关节角、夹爪宽度、力觉。两个物理状态可能产生相近图像，却对同一动作产生不同后果。例如方块看起来静止，但一种情况正在缓慢滑动，另一种情况已经被卡住。

仅用单帧预测，会把这些不同状态混合成一个分布。混合有时来自真实随机性，有时只是模型缺少速度或接触信息。加上历史和触觉，可能把后一类不确定性降低。

为什么模型需要记忆

从连续两帧可以估计运动方向，从连续推的过程可以判断摩擦是否较大。模型因此可以接收历史

$$\mathcal{H}_t = (o_0, a_0, o_1, \ldots, a_{t-1}, o_t).$$

历史理论上越来越长，直接处理既昂贵又不方便。常用循环网络或 **Transformer** 把它压缩为内部状态 b_t。这个内部状态是网络计算出来的，不是真实状态传感器给出的答案。

如果压缩丢失了“夹爪已经接触方块”这一信息，未来预测仍会出错。网络有记忆结构，也不代表记忆已经包含控制所需的一切。

信念状态是对未知状态的分布

在部分可观测决策中，**信念状态（belief state）** 严格指 $b_t(s) = p(s_t = s \mid \mathcal{H}_t)$。它描述机器人对隐藏真实状态的判断。例如，它认为方块处于“低摩擦”的概率是 0.7，“高摩擦”的概率是 0.3。

理论上的更新分为预测和校正

$$b_{t+1}^{-}(s') = \int p(s' \mid s, a_t) b_t(s)\, ds,$$

$$b_{t+1}(s') \propto p(o_{t+1} \mid s') b_{t+1}^{-}(s').$$

先用动作和转移规律推测新状态，再用真实新观测修正。$\propto$ 表示还需要归一化，使所有状态的概率之和为 1。

神经网络方法通常不显式存储这个完整分布，而用一个向量或潜在变量分布近似它。本书也用 b_t 表示这种内部状态，但会明确它只是近似的信念表示。

一个隐藏摩擦的数值例子

假设执行动作后，观察到方块移动得很少。在“低摩擦”情况下，这种观测发生概率为 0.1；在“高摩擦”情况下，为 0.8。动作前两者概率分别为 0.7、0.3。

观测后高摩擦的概率为

$$\frac{0.8 \times 0.3}{0.1 \times 0.7 + 0.8 \times 0.3} \approx 0.774.$$

同一次真实交互不仅改变了方块位置，还改变了机器人对摩擦的判断。下一次规划应基于修正后的判断。这里用两个离散摩擦类别是为了手算，不是实际机器人必须采用的分类方法。

对设计和实验意味着什么

当任务涉及遮挡、速度、接触、延迟时，应比较单帧模型与历史模型，并检查历史长度。若触觉可用，应验证它是否改善接触预测，而不只改善图像重建。

训练时应在轨迹开始处重置记忆，并在轨迹中连续更新。随机抽取一帧却把模型当作已经知道完整历史，会造成训练与部署不一致。后文的 **循环状态空间模型**（Recurrent State-Space Model，RSSM）结合循环记忆与随机潜在状态，把“没有新观测时的预测”和“看见新观测后的校正”写成两个不同网络。

5.1.4 动作条件、前向模型、逆向模型与干预

预测自然发生的未来与预测行动后果

只看视频训练的模型可以学习 $p(o_{t+1} \mid o_{\le t})$：根据历史猜测下一帧。它可能猜到手会继续向右移动。但机器人控制需要 $p(o_{t+1} \mid \mathcal{H}_t, a_t)$：如果现在给机器人一个特定动作，会出现什么结果。

动作条件（action conditioning） 就是把动作作为预测网络的显式输入。具体可把动作向量与视觉特征拼接，用动作 token 参与**注意力**，或用动作特征调制网络层。无论怎样接入，都应检验改变动作是否会使预测按正确方向改变。

训练数据只有“向右推成功”的轨迹时，动作输入可能被模型忽略，因为当前图像和行为惯例已经足以猜测未来。加入动作接口是必要步骤，模型学会动作影响仍需要数据与验证。

前向与逆向，输入输出相反

前向动力学模型 接收当前状态和动作，输出后续状态。例如 $f_\theta(b_t, a_t) \to \hat{b}_{t+1}$。它用于模拟动作后果。

逆向动力学模型 接收当前状态与期望后续状态，输出使这个转移可能发生的动作。例如 $q_\psi(a_t \mid b_t, b_{t+1})$。训练标签是轨迹中实际执行的动作，常用动作回归或动作似然损失。

逆向关系不一定唯一。机械臂可能从上方或侧面到达同一目标；相邻图像也未必能识别关节冗余。确定性**均方误差**可能输出两种动作的平均，而这个平均可能碰撞障碍物。因此逆向模型有时也需要多峰动作分布。

在“先生成目标视频，再从视频恢复动作”的系统里，逆向模型是视频与真实控制之间的桥。桥本身的误差必须评估，不能因为生成视频成功就认为动作执行成功。

动作必须有物理含义

同一个 7 维向量，可以表示关节角目标、末端位姿增量、速度、力矩或夹爪命令。世界模型必须知道使用的是哪一种；还要知道坐标系、单位、执行频率和底层控制器。

一个 0.02 的平移分量，在米制下是 2 厘米，在归一化区间内则需要反归一化才知道实际距离。若每条速度命令保持一个控制周期，10 Hz 的一条命令覆盖0.1秒，100 Hz 的一条覆盖0.01秒，对应位移便不同。若持续时间和实际跟踪相同，总位移不会仅因调用频率不同而改变。数据处理不能只“把维度补齐”，却丢掉这些语义。

动作块 是连续多个控制时刻的动作序列。若预测一个**动作块**执行后的观测，模型的一个转移步覆盖了多个物理控制步。若机器人只执行前几步后就重新规划，模型接口也应支持这个实际执行时长。

为什么有动作数据仍不等于因果识别完成

机器人操作者通常会根据隐藏情况选动作。例如看到物体快滑落就突然夹紧。如果模型没有观察到这个隐藏情况，“夹紧动作”可能与“滑落”高度相关。仅凭相关关系，它可能错误地预测夹紧导致滑落。

在控制问题中，希望模型近似行动干预后的分布

$$p(o_{t+1} \mid \mathcal{H}_t, \mathrm{do}(a_t)).$$

$\mathrm{do}(a_t)$ 表示主动设定动作。要把数据中的条件预测当作这种干预预测，需要足够的状态信息、动作覆盖和适当的数据采集；有隐藏混杂时并不能自动成立。动作本身随机化或在相似状态下采集多种动作，可以帮助检验其作用，但也应控制可执行范围。

同一初始状态下比较“向左”“向右”“不动”的真实与预测转移，是比只看演示视频更直接的动作敏感性测试。

目标条件放在哪里

物理转移主要由当前物理状态和动作决定。“把方块推左边”与“把方块推右边”不应在同一实际动作下凭空改变摩擦。因此在完整状态给定时，动力学通常不必依赖目标；策略和奖励则必须知道目标。

在视频生成中，文本可以表达期望动作或场景变化，也可以只是描述任务。若模型把“拿起杯子”的目标直接变成成功未来，却没有接受对应机器人动作，它更接近目标条件的视频规划器。不能把这种结果当成物理模拟器对任意动作的响应。

5.1.5 观测相关性不自动支持动作反事实

采集策略在杯子容易抓取时才闭合夹爪，数据中“闭合夹爪”与成功相关。模型可能将容易状态当成原因，而没有学到不同夹爪动作在同状态的影响。检验动作条件作用，应尽量在相同或配对初始条件下执行不同动作，并比较预测与实际变化；乱序动作输入的对照也能发现模型是否忽略动作，但不能替代真实干预验证。

世界模型依赖数据支持范围。对从未见过的高速撞击给出流畅视频，不代表其反事实可信。规划器可能主动找到模型漏洞，因此用于控制的验证要比仅预测测试更严格。

5.1.6 练习题

题5.1-1 一个模型仅用历史图像预测下一帧，另一个还输入动作。前者为何可能在视频预测上表现好，却不适合比较候选动作？

5.1.7 参考答案

题5.1-1答案 只读历史的模型回答“在训练视频所呈现的行为习惯下，接下来通常出现什么”。输入动作的模型才有机会回答“当前情况相同，分别执行这两个动作，后果有什么不同”。

例如，训练视频中机械臂接近柜门后几乎总会拉门。仅看历史的预测器可以准确生成开门画面；规划器却还要比较拉门、向前推、保持不动。若三种候选动作都没有进入预测器，它就会给出相同预测，无法评价它们。

用 h_t 表示历史，动作无关预测相当于把数据中的动作选择混在一起：

$$p(o_{t+1} \mid h_t) = \int p(o_{t+1} \mid h_t, a)\, p_{\text{data}}(a \mid h_t)\, da.$$

这里的平均由采集者常用什么动作决定。规划需要保留动作这个条件，并用记录的“观测—动作—后续观测”训练。动作条件仍不自动保证可靠的因果预测：未记录的接触状态、几乎从未尝试过的动作，都会使模型对候选动作的评价失真。

5.2 物理量、图像、潜在表示与不确定性

5.2.1 预测物理量、像素还是潜在表示

表示选择决定模型主要学习什么

表示（representation） 是把原始观测转换为模型可以处理的变量。选择表示，相当于选择保留哪些信息、忽略哪些细节。

低维状态模型可以预测物体位姿、关节角和速度。像素模型直接预测图像。**潜在表示（latent representation）** 是编码器学习得到的特征向量；模型预测向量，再按需要用解码器恢复图像。物体模型则按物体组织位姿、形状、关系等变量。

表示方式	主要好处	关键代价与前提
物理状态	量纲清楚，规划容易检查	需要可靠状态估计；复杂场景难以完整定义
像素或视频	接近相机输入，便于生成视觉经验	计算量大；大量误差可能来自无关背景
潜在向量	紧凑，可快速大量展开	含义不一定直观；关键接触信息可能被丢掉
物体与三维结构	支持几何关系与组合	分割、跟踪和遮挡估计可能失败

这不是从低级到高级的排名。若桌面位置可准确测量，简单状态模型可能比大型视频模型更可靠。若需要训练直接看图像的 **VLA**，只有位置向量又未必满足其输入需求。

编码器、动力学和解码器

一个常见分工是

$$z_t = E_\eta(o_t), \qquad \hat{z}_{t+1} = F_\theta(z_t, a_t), \qquad \hat{o}_{t+1} = D_\omega(\hat{z}_{t+1}).$$

E_η 是编码器，输入图像、输出特征；F_θ 是动力学网络，输入当前特征和动作、输出未来特征；D_ω 是解码器，输入特征、输出图像或图像分布参数。

训练时，可比较未来解码图像与真实下一帧，也可比较预测特征与真实下一帧编码得到的特征。部署时，如果规划只依赖潜在奖励或目标特征，就可以不调用解码器，从而节省计算。是否解码是用途决定的，不是潜在模型必须一直生成图像。

为什么重建漂亮不保证控制有效

桌面背景占图像的大部分，方块与夹爪只占小部分。按全部像素**均方误差**训练，网络可能优先恢复桌面纹理，却模糊夹爪接触点。重建指标很好，控制仍然失败。

反过来，一个模型可能不能恢复每根木纹，却能准确表示方块位置、速度和是否被夹住。对推方块规划，它可能更有用。

理想的控制表示应使具有不同可行动后果的状态能够区分。若两个表示几乎相同，但执行相同动作一个会成功、一个会碰撞，那么这个表示对当前任务不够充分。

“表示充分”也有条件：只为推方块充分的表示，不一定为倒水充分。把颜色完全丢弃可能不影响运动预测，却破坏“选择红色物体”的任务。因此任务无关表示与任务相关表示之间存在取舍。

为什么不只训练奖励和价值

如果只关心任务收益，可以学习预测奖励与未来价值的内部模型，不一定重建图像。MuZero 和 **TD**-MPC2 说明了不同形式的决策导向潜在建模；但它们的结构、训练目标和搜索机制并不相同。MuZero 用学习模型支持树搜索，TD-MPC2 面向连续控制采用潜在**模型预测控制**。MuZero、TD-MPC2

这种取舍可减少无关视觉负担，但也可能让表示更依赖已有奖励任务。如果后来需要新任务、视觉评估或生成训练图像，原模型可能缺少必要信息。

怎样检验表示是否有用

冻结编码器后，用少量数据训练物体位置、接触状态或成功标签的预测头，能检查某些信息是否容易读出。但一个线性头读不出，不严格等于信息完全不存在；复杂头读得出，也不证明规划一定有效。

更直接的证据是：相同真实交互预算下，基于该表示的预测和控制是否改善；改变目标、相机或物体后是否仍然有效。表示指标、动力学指标和真实任务指标应该分别报告，并通过消融分析其关系。

5.2.2 确定性、随机性与概率预测

为什么一个均值可能是不可能的未来

假设方块碰到尖角后，可能向左或向右绕开，两种结果各占一半。如果用均方误差训练一个确定性网络，在信息不足时，它倾向预测两种结果的平均。平均轨迹可能恰好穿过尖角，而这在真实环境中不可能发生。

这个例子说明，预测均值与预测可能发生的完整后果，承担不同职责。尤其在非线性任务评分下，先平均未来再打分，通常不等于对各未来的分数取平均。假设下一位置为 -1 或 $+1$，各占一半，目标是 0，平方距离代价的真实期望为 1；若只给均值 0 打分，代价却成为 0，误以为已经到达目标。因此涉及多种后果、碰撞或成败判断时，应选择足以支持该评分的分布或候选表示。若后果近乎确定、只需短时局部预测，均值模型也可能适用。

随机世界模型 用分布表示未来的不确定性。它可以输出高斯分布参数，混合分布参数，离散类别概率，也可以通过潜在随机变量或**扩散**采样产生复杂未来。

高斯网络怎样训练

对一维下一位置 y，网络输出均值 $\mu_\theta(x,a)$ 和正标准差 $\sigma_\theta(x,a)$。通常预测未约束的数 u，再用 $\sigma = \mathrm{softplus}(u) + \epsilon$ 保证标准差为正。ϵ 是很小的正数，防止除零。

以真实下一位置为标签，最小化**负对数似然**，忽略常数项后为

$$\mathcal{L}_{\mathrm{NLL}} = \frac{(y-\mu_\theta)^2}{2\sigma_\theta^2} + \log \sigma_\theta.$$

似然 是模型给实际观测赋予的概率密度。负对数似然越小，实际观测越符合模型分布。第一项惩罚“误差相对于预测尺度太大”；第二项惩罚“无限扩大标准差”。两者共同避免用一个极宽分布掩盖全部预测错误。

假设一条样本的预测误差为 0.2。$\sigma = 0.1$ 时，平方误差项为 2，总损失约为 -0.303；$\sigma = 0.5$ 时，误差项降到 0.08，但加上 $\log\sigma$ 后总损失约为 -0.613。还可以取 $\sigma = 0.2$，此时总损失为 $0.5 + \log 0.2 \approx -1.109$，比前两者都低。这个例子说明，模型需要为误差选择相称的尺度，既不能盲目宣称很确定，也不能无限扩大范围。这里只比较同一误差下的尺度；真实训练会同时调整均值与尺度，在全部样本上权衡。省略常数的连续密度负对数似然允许为负。

随机性从哪里来

偶然不确定性（aleatoric uncertainty） 指在给定当前建模信息后仍存在的结果变化，如测量噪声或没有被观测到的微小接触差异。增加同类数据不一定能消除它；增加触觉或更完整状态，有时可以使原先被当作随机性的变化更可预测。

认知不确定性（epistemic uncertainty） 指模型因数据不足或参数未确定而不知道规律。例如从未见过湿滑桌面。补充相关数据可以减少它。

两者的边界依赖你给模型提供了哪些信息。高斯网络输出较大标准差，不足以证明它识别出了分布外输入；它可能只是在训练分布里学习了噪声尺度。

集成模型与粒子

集成（ensemble） 是训练多个模型，使用不同初始化或重采样数据，使它们形成不同的预测。模型间意见差异可作为认知不确定性的近似信号。每个模型内部的预测分布则表达该模型认为仍存在的随机性。

规划时可模拟多个**粒子（particle）**。粒子不是物理小颗粒，而是一条采样得到的可能状态轨迹。它可以携带一个模型身份以及每步采样的状态。不同的模型身份采样方式对应不同的不确定性近似，应说明是否在一条轨迹内保持同一模型。

PETS 把概率动力学集成与轨迹采样结合，用预测的不确定性支持控制。该结构的意义是避免把单一均值当作所有未来，并不是集成可以识别所有未知环境。PETS 原始论文

使用概率模型时应该检查什么

既要检查均值误差，也要检查概率区间。声称“90% 的预测区间”如果只覆盖 55% 的真实结果，模型明显过于自信。反过来，区间覆盖全部结果但宽到无法区分动作，也没有实际决策价值。

单高斯不能表达明显分开的两个未来；许多维度独立的高斯也可能忽略物体间关系。此时可考虑混合分布、潜在变量模型或生成模型。选择更复杂分布的理由应来自数据中的多种模式和控制需求，而不是“随机模型听起来更先进”。

5.2.3 物体、三维几何、触觉与物理约束

为什么按物体组织世界

单个向量可能把多个物体的信息混合。**物体中心表示** 将场景拆成若干实体，每个实体携带位置、形状、速度或类别等信息，再描述它们的关系。

这样做的动机是：物体之间的接触和相对位置往往决定后果，而且物体数量变化时，希望规律能复用。比如“被推的方块推动相邻方块”依赖接触关系，而不是场景里第几个像素。

困难是物体的定义与跟踪。柔软布料可以视为一个物体，也可以视为许多节点；遮挡可能使同一物体被识别为两个；非接触的力或固定连接也可能不是简单图像邻接关系。

图网络怎样预测相互作用

图神经网络 用节点表示物体或粒子，用边表示关系。网络先从节点状态和相对关系计算边消息，再把相关消息汇总到每个节点，输出更新量

$$m_{ij} = M_\theta(x_i, x_j, e_{ij}), \qquad \Delta x_i = U_\theta\left(x_i, \sum_j m_{ij}\right).$$

e_{ij} 可包含距离、接触类型或连接关系。输入是真实当前节点状态，标签是下一步位移、速度或加速度等。若预测加速度，还需要按照时间步长积分得到位置。

共享消息函数使同一相互作用规律可以应用到不同节点组合。学习式图物理模拟已有具体研究实例，但“使用图网络”本身不保证能量守恒、无穿透或接触准确。Learning to Simulate Complex Physics with Graph Networks

三维重建与动力学模型的区别

多视角、深度和点云可以帮助恢复空间结构，减少单视角的尺度与遮挡歧义。机器人规划可以在三维中计算物体相对位置、抓取空间和碰撞距离。

但静态三维表示主要解释“从某个视角看起来怎样”。例如 3D Gaussian Splatting 的原始工作面向场景表示和渲染；仅获得渲染表示，还没有学到施加动作后的物体运动。3D Gaussian Splatting 原始论文

要形成行动**世界模型**，必须增加对象运动、形变、接触和控制条件的动态规则。画面在多视角下一致，有助于几何可信，却仍不能推出摩擦和力学正确。

触觉为什么可能比更清晰视频重要

拿起方块时，视觉可能看见夹爪闭合，但不知道方块是否稳定夹住。触觉、力矩、夹爪宽度等信号可以提供接触、滑移和负载信息。

一个多模态模型可以把视觉特征与触觉特征共同用于**后验**推断，再预测未来视觉和接触状态。训练时，每类输出使用对应传感器的未来读数作为标签，并处理传感器时延和不同采样频率。

如果力传感器读数时间错位，模型可能把接触结果提前当成动作前条件，形成信息泄漏。不能仅把传感器向量拼接，就认为完成了多模态融合。

物理先验与残差学习

混合模型 可以保留已知物理规律，再学习未知部分

$$\hat{s}_{t+1} = F_{\text{phys}}(s_t, a_t) + \Delta_\theta(s_t, a_t).$$

F_{phys} 是已有物理模型，Δ_θ 是神经网络预测的残差。标签是实际下一状态与物理预测之间的差。它适合已知基本机械结构、但摩擦或接触参数难以精确建模的场景。

残差项也可以把错误物理模型“抵消”掉，所以需要限制、分布外测试和量纲检查。对姿态、约束流形或离散**接触模式**，不能总按普通向量直接相加，应使用合适的状态更新方式。

软约束与硬约束不是一回事

在损失中加入穿透惩罚、能量误差或几何一致性，是软约束：优化器会权衡它与其他目标，仍可能违反。将动作投影到可执行集合、用专门物理求解器保证某些约束，则更接近硬约束，但保证只覆盖明确规定的变量和假设。

一篇论文称模型“具有物理一致性”，应该进一步查明一致性的定义、如何训练、哪些场景测试、是否真实接触有效。不要仅凭术语就推断它具备通用物理正确性。

5.2.4 不确定性应通过可检验事件校准

输出一个方差并不保证方差正确。模型可以把大量噪声都解释成大方差，或者在未知动作上极度自信。应检查预测区间在留出转移上的覆盖率，按动作幅度、接触阶段和数据距离分层；同时看区间宽度，避免用无限宽的范围获得虚假的覆盖。

集成模型分歧有助于发现部分知识不足，但成员共享架构和数据时可能一起犯错。观测噪声、真实随机性和参数知识不足有不同来源，应按决策需求区分。接触任务常更需要“是否会滑脱”的风险预测，而不是单个平均像素误差。

5.2.5 练习题

题5.2-1 当前观测下，杯子可能被遮挡在两个位置。确定性均值预测与随机预测各有什么问题？如何区分数据本身不确定与模型缺少知识？

5.2.6 参考答案

题5.2-1答案 假设杯子在左右两个位置 $x = -1$ 和 $x = 1$ 的概率各为一半，中间实际没有杯子。用平方损失训练单值位置预测，最优输出是 $E[x] = 0$。它是合理的统计均值，却不是杯子真实可能出现的位置；按这个均值抓取，机器人会伸向空处。

随机模型可保留“左右各一半”的分布，但从中抽到左边不代表杯子已经被确认在左边。规划应比较一个动作在多个可能位置下的后果，或先换视角消除遮挡。预测整段未来时，还应保持一次采样中物体状态的连贯，不能逐帧独立重抽位置，让杯子左右跳动。

区分不确定性要看增加什么信息能减少它。在当前视角下两边都可能，即使已经学过大量同类数据，图像仍无法唯一判断，这是给定当前信息后的不确定性；获得另一个视角可能消除它。若图像其实足以判断，但模型没学过透明杯，则是模型缺少知识，补充这类训练数据可能改善。

多个模型可以提供诊断线索。若它们都预测“左右各一半”，可能一致认为输入证据不足；若有的强烈预测左边，有的强烈预测右边，分歧提示知识不足。常用模型内部方差与模型预测均值之间的方差近似区分这两部分，但前提是模型已学到合理的条件分布。模型共同犯错、概率未校准时，这种解释也会失效，仍需在独立数据上检查预测概率与真实频率。

进一步阅读：Kendall 与 Gal：视觉模型中的数据不确定性和模型不确定性。

5.3 RSSM 的先验、后验与潜在动力学

5.3.1 本节奖励下标的约定

RSSM 常在“到达当前状态”时预测奖励，因此本节的 r_t 表示从 t−1 到 t 的转移反馈，等于其他章节通常写的 r_{t-1}。训练字段按实际转移连接即可：哪个动作造成哪个下一观测，就与这次到达的奖励配对。

5.3.2 为什么同时需要确定性记忆与随机状态

变分自编码器（Variational Autoencoder，VAE）用潜变量解释输入，通过推断网络和变分目标训练。静态 VAE 每次只解释当前图像，未必记得刚才怎样推、是否接触、物体是否有速度。**循环状态空间模型（RSSM）** 用确定性记忆 h_t 加随机潜在变量 z_t 表示环境。

h_t 由上一内部状态和动作递推得到：给定输入与参数，其计算结果是确定的；这不意味着模型已经确定了真实世界的一切。随机变量 z_t 则允许同一记忆对应不同的当前状态解释，观测到来后再修正它的分布。二者合成 $b_t = (h_t, z_t)$：记忆保留历史，随机状态保留当前仍需推断的差异。网络通过预测与重建目标学习这种分工，不必把某一维预先指定为物理摩擦或速度。

PlaNet 与 Dreamer 系列采用相关的潜在动态结构，但各版本的分布、损失和行为学习细节有区别。以下用统一索引介绍核心机制。PlaNet、Dreamer

5.3.3 三个关键网络的输入和输出

先根据上一状态与已执行动作更新记忆

$$h_t = f_\theta(h_{t-1}, z_{t-1}, a_{t-1}).$$

然后，**先验网络** 接收 h_t，输出 $p_\theta(z_t \mid h_t)$。它表示在还没有看到当前观测时，模型预测当前随机状态应该怎样分布。

如果真实当前观测 o_t 已经到来，先计算图像特征 $e_t = E_\eta(o_t)$，再由**后验网络** 接收 h_t, e_t，输出 $q_\eta(z_t \mid h_t, e_t)$。它用观测修正预测。

网络	输入	输出及训练作用
记忆转移网络	上一记忆、上一潜在样本、上一动作	当前确定性记忆
先验网络	当前记忆	当前潜在状态分布；供无新观测的预测使用
后验网络	当前记忆、当前图像特征	观测校正后的潜在状态分布
解码器及辅助头	当前记忆与潜在样本	图像、奖励、继续概率等预测

这些网络可以联合训练，但不能因此把**先验**和**后验**当成同一个功能。

5.3.4 有真实观测时怎样训练

从真实轨迹抽取一段观测与动作。每一步按照记录的上一动作更新 h_t，读取真实 o_t，计算后验并抽样 z_t。用 (h_t, z_t) 重建当前图像，并预测相应监督信号。

这里我们约定 r_t 是从 $t-1$ 到 t 的转移所得奖励，c_t 表示这次转移后是否还可继续。奖励和继续标签有记录时，训练损失可写为

$$\mathcal{L}_t = -\log p_\omega(o_t \mid b_t) - \log p_\psi(r_t \mid b_t) - \log p_\xi(c_t \mid b_t) + \beta D_{\mathrm{KL}}(q_\eta(z_t \mid h_t, e_t) \| p_\theta(z_t \mid h_t)).$$

图像项提供观测解释，奖励项提供任务反馈，继续项提供终止判断；**KL** 项教先验在没有当前观测时预测后验所认为的状态。若任务不提供奖励或终止标签，可以只训练其他项，之后另设目标评分。

每个头的分布假设不同：连续位置可以是高斯，二元继续标签可以是伯努利，图像可以使用不同像素或生成式似然。不能把所有**负对数似然**默认看成同一种平方误差。

5.3.5 没有真实未来时怎样想象

部署或策略想象时，当前真实观测仍可用于获得后验状态。但一旦开始模拟未来，就没有未来真实图像。每个模拟步只能用当前**潜在状态**和候选动作更新记忆，再从先验取新的 z。

```
真实当前图像 → 编码器 + 后验 → 当前内部状态
当前内部状态 + 候选动作 → 下一记忆 → 下一先验分布
从先验采样 → 预测内部状态 → 奖励等预测
再次输入候选动作 → 继续模拟
真实新图像到来 → 重新使用后验修正
```

若评测时在每个预测步偷偷输入真实未来图像，实际上只在测观测校正后的单步模型，不能证明模型具备多步自主预测能力。

5.3.6 为什么要调整 KL 的梯度方向

后验可能比先验更了解当前状态，因为它看到了图像。若直接让两者同时追逐彼此，模型可能通过削弱后验信息来轻易降低 KL，而没有让先验真正学好动力学。

一种做法将损失分为两部分

$$\mathcal{L}_{\text{dyn}} = D_{\text{KL}}(\text{sg}(q) \| p), \qquad \mathcal{L}_{\text{rep}} = D_{\text{KL}}(q \| \text{sg}(p)).$$

sg 表示停止梯度，数值继续参与 KL 计算，但不由这一项直接更新。第一项固定后验作为目标，让先验学习预测；第二项固定先验，让后验表示受到“应当可被历史预测”的约束。用一个二元潜在变量说明：后验看过图像后给取值 1 的概率为 0.9，先验仅凭历史给 0.5。动力学项推动先验向 0.9 靠近；表示项则对后验偏离 0.5 施加代价。重建和奖励目标又要求后验保留有用观测，几项损失共同形成取舍。若分支共享参数，应继续沿计算图检查梯度，而不是认为停止一处分布梯度就冻结了整个网络。

Free bits 常以 $\max(\kappa, D_{\text{KL}})$ 形式让小于阈值的 KL 不继续受到减小压力。它的动机是给表示留一定信息容量，防止为了节省很小的 KL 把有用信息也压掉。它不是强制每个变量携带指定数量的物理信息。DreamerV3 使用了这类稳定化设计。DreamerV3 原始论文

5.3.7 如何判断 RSSM 真的有效

分别测试有观测校正的重建、只用先验的多步预测、以及基于预测的控制。检查移除动作后结果怎样、历史长度是否足够、遮挡时是否还能维持对象状态。

潜在变量可视化好看、KL 数值平稳或重建清晰，都不足以单独证明动力学可靠。最重要的是：使用相同起点和不同可执行动作时，模型是否能预测真实后果的差异。

5.3.8 同一个 KL 数值，为什么能产生两种更新

上一小节的“停止梯度”可以直接用一个二元状态算出来。先看未经 free-bits 阈值裁剪的 KL，以便分清两条更新路径。设 $z = 1$ 表示已经接触杯子：读过图像的后验给 $q = 0.9$，只读历史的先验给 $p = 0.5$。二元分布的 KL 为

$$D(q\|p) = q \log \frac{q}{p} + (1-q) \log \frac{1-q}{1-p}.$$

先验和后验通常由网络输出的 **logit** 转成概率：$p = \sigma(v)$、$q = \sigma(u)$。这里的 logit 是未经 sigmoid 的实数得分。计算动力学损失时把 q 当固定答案，对先验得分求导可得

$$\frac{\partial D(\mathrm{sg}(q)\|p)}{\partial v} = p - q = -0.4.$$

梯度下降会增加 v，从而提高先验预测接触的概率。它在学习：“这种历史和动作之后，接触比我原先估计的更常见。”后验不接受这一项的直接更新。

表示损失反过来固定先验，对后验得分求导：

$$\frac{\partial D(q\|\,\mathrm{sg}(p))}{\partial u} = q(1-q) \log \frac{q(1-p)}{p(1-q)} = 0.09 \log 9 \approx 0.198.$$

这一项会降低后验的 u，让后验减少与先验的差异。但后验还承担重建图像、预测奖励的任务；这些目标可能要求它保留“已经接触”的证据。因此，最终更新由各项梯度及其权重共同决定。表示正则的作用是让状态容易预测，重建等任务的作用是让状态仍有内容。DreamerV3 的模型训练

```
import torch
from torch.distributions import Bernoulli, kl_divergence

# 1. 两个独立得分分别代表后验与先验网络的输出。
u = torch.tensor(9.0).log().requires_grad_()  # sigmoid(u) = 0.9
v = torch.tensor(0.0, requires_grad=True)    # sigmoid(v) = 0.5

# 2. 后验提供固定目标，梯度只直接进入先验得分。
loss_dyn = kl_divergence(Bernoulli(logits=u.detach()),
                         Bernoulli(logits=v))
grad_prior, = torch.autograd.grad(loss_dyn, v)

# 3. 先验提供固定参照，梯度只直接进入后验得分。
loss_rep = kl_divergence(Bernoulli(logits=u),
                         Bernoulli(logits=v.detach()))
grad_posterior, = torch.autograd.grad(loss_rep, u)
print(grad_prior.item(), grad_posterior.item())  # 约 -0.4, 0.198
```

两项 KL 的数值相同，求导路径不同。代码把两个得分独立设置，是为了看清各分支的直接作用；在完整 RSSM 中，记忆状态可能被多个头共享，还要沿共享状态继续传播梯度。

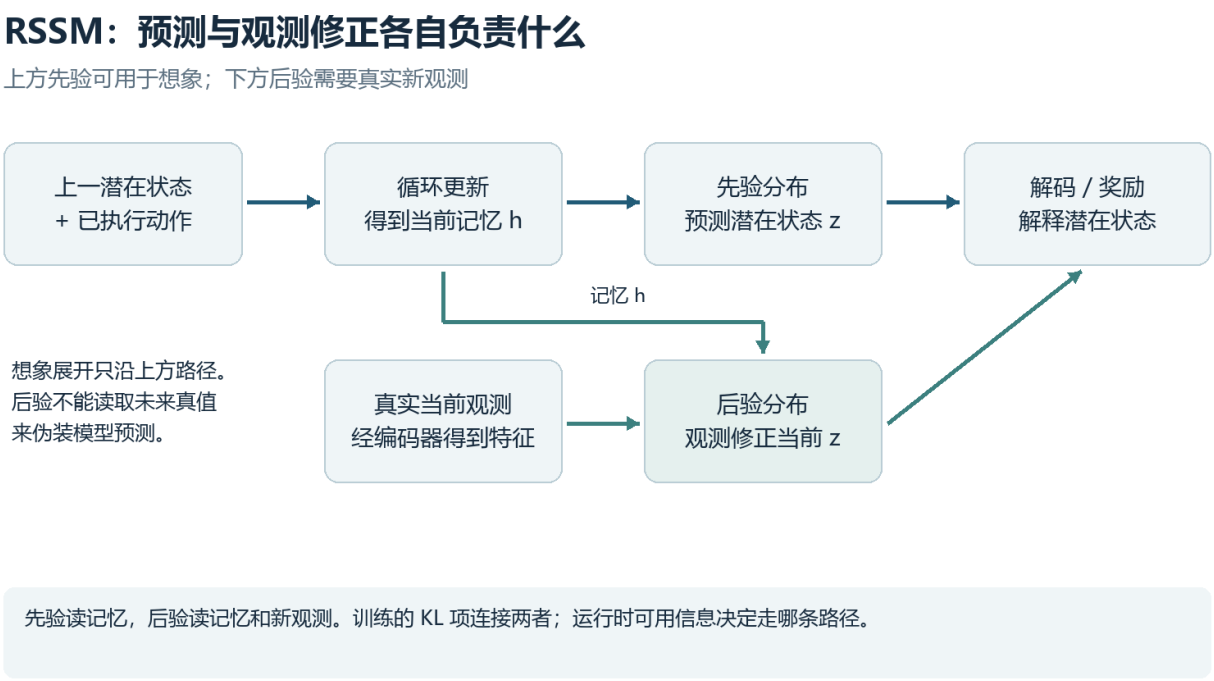


图5.1：先验预测尚未看到的状态，后验用新观测修正；训练损失分别教先验预测、教表示保留有用且可预测的信息。

5.3.9 RSSM 训练与想象

训练一条真实转移：

- 用已有观测形成当前潜在状态
- 用当前潜在状态和已执行动作预测下一潜在先验
- 读取真实下一观测，形成下一潜在后验
- 用重建、执行后奖励、继续信号等真实标签训练预测头
- 以规定的 KL 约束连接先验与后验，更新对应参数

想象未来：

- 从当前真实观测得到当前潜在状态
- 用策略或搜索候选提供动作
- 只用先验推进未来潜在状态，并预测后果
- 累积想象奖励，更新搜索变量或按算法更新策略

5.3.10 用一次遮挡后的预测理解先验与后验

想象夹爪正在接近杯子，下一帧暂时看不见。模型利用前面的内部状态与已执行动作，预测“杯子可能仍在原处、夹爪应该已接近”——这一步是先验分布。新图像到达后发现杯子被推走，读取图像的后验模块便重新判断当前潜在状态。**潜在状态**是模型内部用于预测的向量或离散编码，不是传感器直接测量到的一组真实关节和物体坐标。

如果先验预测“还未接触”的概率为0.8，而观测后形成的后验将“已接触”概率提高到0.9，KL 匹配会给先验提供学习信号：这些历史与动作之后，接触比原先预计更可能。重建、奖励或继续信号又要求后验保留有助于解释真实后果的信息。只把每一帧变成任意编号，虽然可以重建，动力学却很难预测编号怎样变化；只追求先验后验完全相同，又可能忽略新图像。这两类目标的取舍共同规定了什么状态值得记忆。

实际行动时，每次新观测都可修正后验；向前想象尚未发生的帧时，只能调用先验。它与第1章CVAE 的“读到答案再推断”和“没有答案时生成”具有相似信息边界，但这里还要按时间递推。
Dreamer 作者说明

5.3.11 练习题

题5.3-1 RSSM 的后验可读当前真实观测，先验只读此前潜在状态与动作。为什么训练中需要两者，想象未来却主要用先验?

题5.3-2 两个独立网络输出 logits u、v，q=sigmoid(u)=0.9、p=sigmoid(v)=0.5。二元 KL 为D(q||p)=q log(q/p)+(1-q)log((1-q)/(1-p))。损失D(sg(q)||p)和D(q||sg(p))的数值是否相同？分别对先验得分v、后验得分u求梯度，解释梯度下降怎样改变概率，为什么仍需重建/奖励损失?

5.3.12 参考答案

题5.3-1答案 RSSM（Recurrent State-Space Model，循环状态空间模型）需要同时学习“尚未看见时怎样预测”和“看见之后怎样修正”。令

$$h_t = f(h_{t-1}, z_{t-1}, a_{t-1}), \quad p(z_t \mid h_t), \quad q(z_t \mid h_t, o_t).$$

h_t 汇总此前信息；先验 p 不读当前图像，预测可能的潜在状态；后验 q 额外读真实 o_t，把预测与新证据结合。比如先验仍不确定杯子在左还是右，新图像露出杯柄后，后验就能集中到右侧。

训练中，真实观测帮助后验提取有信息的状态；由该状态重建图像、预测奖励和继续标志，使它保留任务需要的信息。再用 KL 损失让先验学会预测后验所确认的状态。否则，后验可以每步依赖图像重建得很好，而先验仍不会独立推进。

想象未来时还没有未来真实图像，不能使用读取该图像的后验；模型先由动作推进 h，再从先验产生 z。实际执行后得到新图像时，仍可重新用后验校正。区别在于真实观测是否已到达，并不是“训练用后验、部署完全不用后验”。

题5.3-2答案 把 q 看成读取当前观测的后验概率，p 看成尚未读取该观测的先验概率。u, v 是两个独立网络输出的可训练得分，因此没有共享参数或其他交叉路径。本题使用自然对数，只计算给出的未裁剪KL项。

停止梯度 sg 不改变前向数值，只把所在分支的数值当成本次反向传播中的常量。因此两项当前损失相同：

$$D = 0.9\ln(0.9/0.5) + 0.1\ln(0.1/0.5) \approx 0.368064.$$

差别在于靠改变哪一个网络降低损失。

对 $L_{\text{prior}} = D(\mathrm{sg}(q)\|p)$，后验0.9是固定目标。先对 p 求导，再经过Sigmoid：

$$\frac{\partial D}{\partial p} = -\frac{q}{p} + \frac{1-q}{1-p} = \frac{p-q}{p(1-p)}, \qquad \frac{\partial p}{\partial v} = p(1-p).$$

所以

$$\frac{\partial L_{\text{prior}}}{\partial v} = p - q = -0.4, \qquad \frac{\partial L_{\text{prior}}}{\partial u} = 0.$$

用小的正学习率 η 做梯度下降，v 更新为 $v + 0.4\eta$，Sigmoid输出 p 随之增大，向0.9靠近；后验不受这一项更新。它要求先验学习后验从真实观测中得到的判断。

对 $L_{\text{post}} = D(q \| \operatorname{sg}(p))$，固定的是先验0.5。此时

$$\frac{\partial D}{\partial q} = \ln \frac{q(1-p)}{p(1-q)} = u - v,$$

$$\frac{\partial L_{\text{post}}}{\partial u} = (u - v)q(1-q).$$

由 $u = \ln 9,\ v = 0$ 得

$$\frac{\partial L_{\text{post}}}{\partial u} = 0.09 \ln 9 \approx 0.197750, \qquad \frac{\partial L_{\text{post}}}{\partial v} = 0.$$

梯度下降降低 u，使 q 从0.9向0.5靠近。它把后验表示约束到先验较容易预测的范围。梯度为0的那一路，是指这一个损失项不通过它更新对应网络；其他损失仍可更新该网络。

KL只要求两者一致，不要求它们包含有用信息。如果全部图像都输出 $q = p = 0.5$，KL已经为零，表示却不再区分杯子位置或是否被抓住。重建目标要求状态能解释真实观测，奖励目标要求状态能区分实际任务后果，给表示提供额外的内容约束。两类目标共同决定“保留多少新证据”和“这些证据多容易由历史预测”的取舍。

进一步阅读：Stanford CS231n：计算图、链式法则、Sigmoid导数与共享变量梯度累加、PyTorch官方教程：Automatic Differentiation with torch.autograd，停止梯度与计算图、DreamerV3作者论文：动态KL与表示KL的停止梯度位置及预测损失。

5.4 预测表征、JEPA 与时空生成模型

5.4.1 换一个预测目标

像素生成要解释阴影、背景纹理和相机噪声，其中一些细节与决策无关。**联合嵌入预测架构**（Joint-Embedding Predictive Architecture，JEPA）选择在编码器的表示空间中预测目标观测，而不恢复全部像素。例如预测杯子的未来特征，再将它与真实未来图像编码得到的特征比较。目标变得紧凑，但编码器必须保留任务需要的差异，否则预测器也无法恢复被丢掉的信息。

“嵌入”就是编码器输出的特征。一个简化的时间预测形式是

$$\hat{z}_{t+1} = P_\theta(E_\eta(o_{\le t}), a_t), \qquad z^{\text{tar}}_{t+1} = E_{\bar{\eta}}(o_{t+1}).$$

预测器输入可见历史特征和动作，输出未来特征；目标编码器输入真实未来图像，提供特征标签。训练时知道未来，因此可以产生这个标签；部署预测未来时，只使用左侧路径。

上式是适合行动预测的教学形式。**JEPA** 的视频预训练还可以预测被遮住的时空区域，不一定是严格按机器人时间方向预测下一帧。掩码补全能力不能直接当作行动条件控制能力。

5.4.2 两个编码器为什么不能一起任意改变

如果预测与目标都能任意变化，最容易的解法是所有图像都编码为同一个常数。此时预测误差为零，却没有学到场景区别。这是**表示坍缩**。

常见做法让目标特征停止梯度，并用在线编码器参数的指数滑动平均更新目标编码器

$$\bar{\eta} \leftarrow \tau \bar{\eta} + (1-\tau)\eta.$$

τ 越接近 1，目标编码器更新越慢。这是**指数移动平均**（Exponential Moving Average，EMA）：把旧参数与当前在线参数按固定比例组合。若学生每次更新都改变目标特征，预测器会不断追逐移动的答案；让目标缓慢变化，可以在一段训练中保持较稳定的参照，同时逐渐吸收新学到的表示。停止梯度负责控制本次误差传到哪里，EMA 负责规定目标参数怎样跨训练步变化。

停止梯度让目标编码器不接受当前预测误差，**EMA** 让目标缓慢跟随，二者提供较稳定的学习对象。常量表示仍可能满足某些简化目标；不同方法通过掩码、结构、归一化或显式表示约束维持有用变化。分析时应沿具体目标寻找防止坍缩的机制。

5.4.3 损失、标签和梯度要分别看

一般特征预测损失可写为

$$\mathcal{L}_{\text{pred}} = \|\hat{z} - \text{sg}(z^{\text{tar}})\|_1.$$

L_1 是各分量绝对误差之和；也有方法使用平方误差等形式。预测器和在线编码器通过预测误差更新，目标编码器通过 EMA 更新。若编码器已经冻结，则只更新预测器。

特征标签由模型产生，不等于没有监督信号。它也不保证每个特征分量对应一个可解释物理量。因此可以用下游状态读出、动作敏感性和真实规划检验，判断这种预测对象是否具有控制价值。

5.4.4 V-JEPA 与 V-JEPA 2-AC 的分工

V-JEPA 展示了从视频学习特征预测表示的路线。V-JEPA 2 先进行不含机器人动作条件的大规模视觉预训练；其行动条件版本 V-JEPA 2-AC 再冻结编码器，利用机器人轨迹训练预测器，支持图像目标下的规划。V-JEPA、V-JEPA 2

特别需要读清“无标注”：该行动条件后训练不需要任务类型、奖励和成功标签，但使用末端状态，并从相邻状态差构造动作。它不是仅凭无任何控制信息的视频就获得机器人电机接口。状态差与原始控制命令也并不天然等价，其对应关系依赖动作参数化和底层控制。

5.4.5 图像目标怎样产生动作

给定目标图像 o_g，先得到目标特征 z_g。对每条候选动作序列，在潜在空间展开，比较终点与目标

$$J(A) = d(\hat{z}_{t+H}(A), z_g) + \rho\,\text{Cost}(A).$$

这里是最小化代价；d 是特征距离，ρ 控制动作代价权重。规划不一定需要把预测特征解码成视频。

这样可以在不训练任务奖励头的情况下使用图像目标，但仍然有任务目标：目标图像和距离函数共同定义了什么结果“更好”。“不需要奖励标签”不等于“完全没有目标评分”。

目标图像可能包含不同光照、相机位置或夹爪姿态。若特征距离对这些变化敏感，规划可能追求画面相似而不是物体任务成功。应测试目标特征是否保留位置、接触和物体身份，并是否忽略真正无关的外观差异。

5.4.6 优势和适用边界

特征预测可以节省像素解码成本，也可以减少对随机纹理的关注。它适合规划主要依赖可在表示中识别的几何和状态变化的情况。

若下游策略必须接收图像，潜在预测还需要解码或让策略适配潜在输入。若细微视觉变化决定成败，例如夹爪是否触到薄片，压缩也可能丢失它。把“预测特征”作为研究设计时，应验证优势确实来自信息选择，而非更强编码器、更多数据或更大的模型。

5.4.7 预测特征需要验证任务相关信息仍在

像素预测承担许多与动作无关的纹理细节；特征预测可以把容量集中在语义与结构，却可能丢掉杯沿和接触精度。教师编码器、目标停止梯度、参数滑动平均和防坍缩设计，为学习稳定提供不同约束。教师输出不是天然真实标签，而是当前表示系统定义的学习目标。

如果只检验特征距离小，无法知道目标是否易于通过常量表示达到。应检查表示多样性、冻结表示后的状态读出、动作相关预测以及下游决策。把预测特征距离当规划代价时，还要证明它对任务进度和失败具有合理敏感性，而不只是视觉相似。

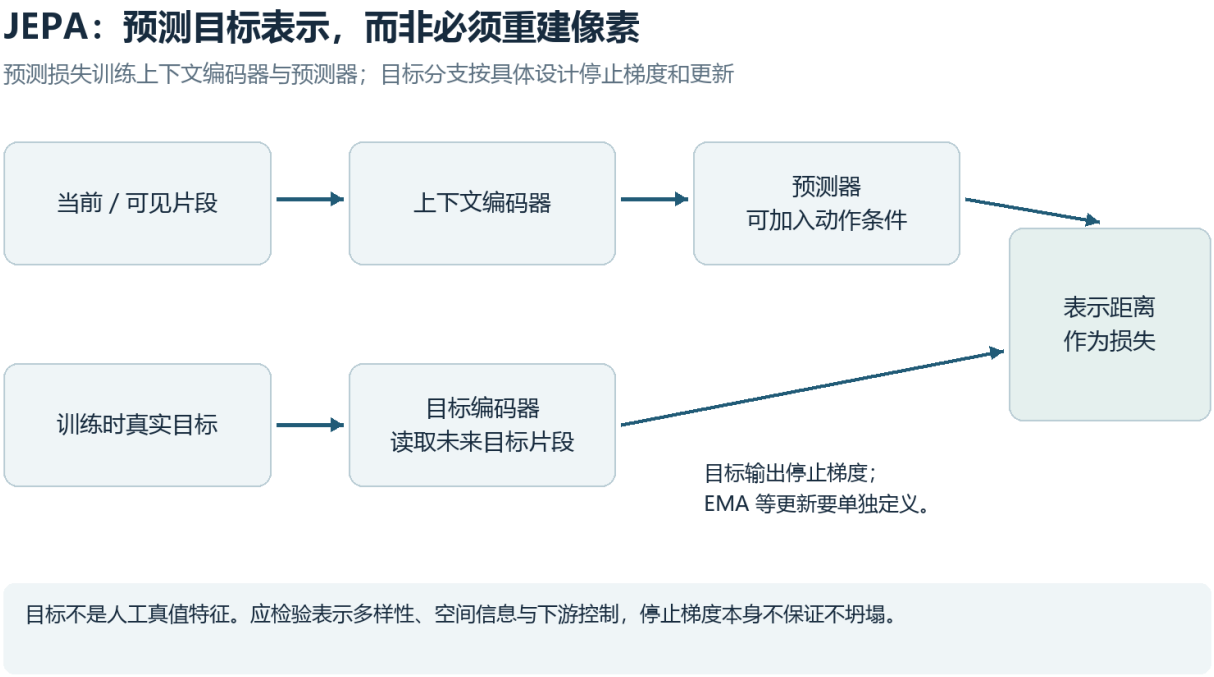


图5.2：目标分支产生训练表示，预测分支学习匹配；停止梯度与目标更新机制须按方法定义，不保证自动避免坍缩。

5.4.8 从 I-JEPA 原图看“预测特征”是什么

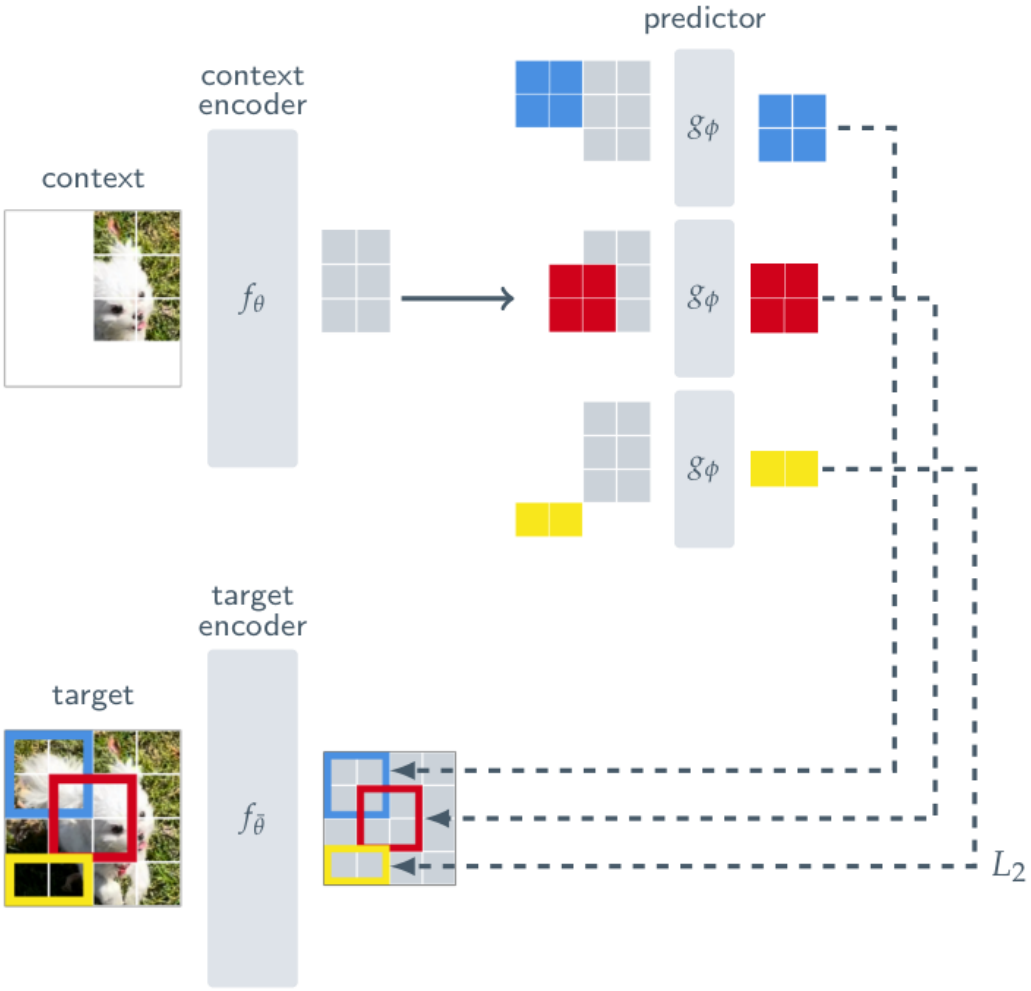


图5.3：Assran 等，I-JEPA 论文 Figure 3。上支路从可见区域预测被遮挡区域的特征，下支路为训练提供相应目标特征。原论文。

图左上的 context 是可见图像区域，context encoder（上下文编码器）把它变成特征；predictor（预测器）接收这些特征和目标区域的位置信息，输出蓝、红、黄区域各自的预测表示。下方 target encoder（目标编码器）能够读取原图，为这些区域提供目标表示。虚线将对应预测与目标配对计算损失，并不是把下方真实区域特征提前送给上方预测器。

这张图解释的是单张图片中的遮挡区域补全。预测器回答“根据可见内容，这个位置应有什么特征”，不是直接回答“机器人执行这个动作后会怎样”。把它用于视频或动作条件世界模型，还需要加入时间与动作，并用相应真实轨迹训练。目标编码器的缓慢更新稳定目标；是否保留几何、是否发生表示坍缩，还要结合掩码设计与特征检验理解，不能由两条编码器支路自动推出。I-JEPA 作者教学说明

5.4.9 练习题

题5.4-1 JEPA 在潜在空间预测目标表示，像素重建预测图像。两种目标分别保留哪些信息？为什么潜在误差更小仍不一定有利于操作？

5.4.10 参考答案

题5.4-1答案 像素重建要求预测每个位置的颜色，因此会惩罚物体位置错误，也会惩罚光照、纹理和背景细节错误。JEPA（Joint-Embedding Predictive Architecture，联合嵌入预测架构）先把真实目标区域或未来画面编码为向量，再要求预测器逼近这个向量。目标中保留什么，取决于编码器及其训练目标；它可以保留物体、运动和空间关系，弱化难以预测的纹理变化，但不能仅凭“潜在空间”四个字保证任务信息充分。

对操作来说，两个杯子的整体外观相近，杯柄却相差几厘米，抓取动作可能完全不同。若目标表示对这几厘米不敏感，预测器忽略杯柄也能得到小误差。更简单的反例是把全部表示乘以 0.01：平方误差缩小为原来的万分之一，机器人能力没有随之改善。不同表示的原始误差数值通常也不能直接比较。

因此应检查表示能否读出目标位置、接触状态、可抓取姿态等量，并比较候选动作排序和闭环成功率。表示预测的学习价值是把建模容量用在可预测、可用于决策的信息上；真正需要保留的信息，仍须由任务监督或决策评价来检验。

5.5 轨迹训练、多步展开与误差传播

5.5.1 从轨迹到一次完整训练

一条轨迹应保存什么

最基本记录为 (o_t, a_t, o_{t+1})。实际项目通常还需要时间戳、机器人自身状态、相机信息、轨迹标识、终止原因及可用任务标签。若要训练奖励或继续概率，还需要 r_{t+1} 和 c_{t+1}。

“机器人自身状态”也称本体感知，如关节角、速度、末端位姿。它常是输入，不应与环境中方块的真实状态混为一谈。

相机时间戳和动作时间戳要明确对应：动作开始前的图像作为当前观测，动作作用一段时间后的图像作为下一观测。存在控制延迟时，可以加入动作历史或估计延迟；简单假设同步可能造成模型学到错误规律。

划分数据应先于随机抽窗口

若相邻帧随机分到训练和测试，测试几乎是训练场景的重复，误差容易过于乐观。先按完整轨迹、场景、物体、采集者或时间段划分，再在各集合内部抽取窗口。

不同划分回答不同问题。留出轨迹测试同分布泛化；留出物体测试对象泛化；留出相机测试视觉变化；留出动作幅度测试控制外推。一个“测试集”标签不足以说明究竟检验了哪种能力。

归一化均值和标准差只用训练集估计。对位置、角度和夹爪命令分别处理，避免把不同语义变量混成同一种尺度。测试期间复用同一映射。

窗口、记忆初始化与掩码

训练批次可包含 B 段长度为 L 的转移窗口，对应 $L+1$ 个观测。轨迹较短时补齐，并用有效位置掩码 m_t 排除补齐位置

$$\mathcal{L} = \frac{\sum_{i,t} m_{i,t}\mathcal{L}_{i,t}}{\sum_{i,t} m_{i,t}}.$$

分母是有效转移数，而不是补齐后的总长度。图像项若使用不同索引，应该相应建立观测掩码，不能不加区分套用动作掩码。

轨迹起始观测没有对应的上一转移奖励，因此仍可用于编码或重建，但该位置的奖励与继续损失应排除。一个中间窗口若保留了前一转移标签，则按实际记录对齐，避免把第一条未来奖励错误地配给起始观测。

循环模型从中间窗口开始时，隐藏状态可能缺少此前历史。可先输入一段 **burn-in** 历史，只用于建立状态，再在后续部分计算主要损失；也可明确采用重置状态训练，并评估这种近似是否适合部署。轨迹结束后的下一条轨迹必须重置，不能把两个不同场景接成连续转移。

一次潜在模型更新

1. 抽取真实轨迹窗口和有效位置掩码
2. 检查观测、动作、下一观测的时间对应
3. 编码真实图像，初始化或预热记忆
4. 按已记录动作更新动态，计算先验和后验
5. 用后验状态生成重建及辅助预测
6. 计算观测、潜在动态、奖励等有效位置损失
7. 清空旧梯度，反向传播，更新指定网络
8. 在独立验证轨迹上做单步与自主展开评估

每一步数据都来自真实记录，没有策略更新也可以训练**世界模型**。若后来要在想象中训练策略，应该再增加策略与价值训练流程，不能把它们隐含在“训练模型”四个字里。

预训练与后训练分别学什么

大规模视频预训练可以学习视觉表示、对象运动规律或生成分布；机器人行动条件后训练则让预测器理解具体机器人状态和动作接口。后训练可能只更新预测器，也可能更新部分编码器或整个模型。

是否冻结编码器是取舍。冻结可以稳定特征标签、保留预训练能力并降低成本；但预训练特征可能忽略接触细节。端到端更新可适应任务，也可能遗忘或造成标签空间漂移。应有冻结、部分更新与完整更新的对照。

预训练见过的网络视频数量，不能替代机器人动作覆盖。没有机器人动作语义的数据主要提供视觉和时序经验，后训练仍需建立动作与后果的对应。

训练目标与部署用途对齐

规划要比较动作，训练就应检查动作敏感性；策略想象需要多步分布，训练就应检查自主展开；生成 **VLA** 数据需要视觉与动作对齐，训练就应检查生成图像中控制相关对象。

训练日志至少应分别显示各损失、有效样本量和展开误差。总损失下降可能只是背景重建改善，也可能只是某一损失的尺度大。不要仅以加权总数决定模型已经可用于真实机器人。

5.5.2 多步展开为什么会错，怎样减少错误

单步监督与自主展开

单步测试在每一步输入真实当前状态。自主展开只在起点使用真实状态，之后连续使用自己的预测。后者才接近规划或策略想象。

设模型为 $\hat{s}_{k+1} = f_\theta(\hat{s}_k, a_k)$。如果第一步把方块位置预测偏差 1 厘米，第二步就会从偏差状态继续模拟，可能把“接触”判断成“未接触”，造成结构性错误。接触切换处的误差不一定是缓慢平滑增长。

一个有条件的误差关系

若真实动态 F 在相关区域满足 Lipschitz 条件，常数为 L，并且模型在每个展开访问的状态上具有统一单步误差上界 ϵ，则

$$e_{k+1} \le L e_k + \epsilon, \qquad e_H \le \epsilon \sum_{j=0}^{H-1} L^j,$$

这里起点误差为零，e_k 是第 k 步状态误差。L 表示状态差异经过真实动态可能放大多少；ϵ 是模型误差上界。

若 $\epsilon = 0.01, L = 1$，5 步上界为 0.05；若 $L = 1.2$，为 0.074416。这个推演解释为何长时域更难，却不能直接拿训练集平均误差代替统一上界。真实接触、分布外状态和不完整状态可能破坏这些假设。

多步损失训练什么

从真实起点出发，按真实记录的动作自主展开，并比较第 k 步预测与真实第 k 步

$$\mathcal{L}_{\text{roll}} = \sum_{k=1}^{H} w_k d\big(\hat{z}_{t+k}, \text{sg}(z^{\text{tar}}_{t+k})\big).$$

这个目标直接让模型适应“上一输入可能是自己预测的”情况。反向传播可以穿过多个模型步，教前面的表示与动态为后面的预测负责。

代价是计算和梯度路径更长；随机未来也可能使确定性特征距离偏向均值。需要选择合适分布目标、权重和展开长度。短展开训练可以改善局部稳定，但不能证明任意长时域可靠。

Overshooting 与普通多步误差的关系

潜在 overshooting 通常把多步**先验**预测分布与未来观测得到的**后验**分布对齐，例如比较 k 步预测分布和第 $t+k$ 步后验的 **KL**。它希望潜在动态不仅在一步中一致，也在跨多步时与观测推断一致。

它与“每一步都输入真实下一图像重建”不同，因为中间跨越的预测部分没有依赖这些未来图像；但目标后验仍在训练时由真实未来产生。PlaNet 讨论了这类多步潜在训练。PlaNet 原始论文

后验如果本身没有保留重要物理信息，与它对齐也无法恢复被丢掉的信息。多步一致性是表示与动态的一种约束，不等于物理正确性证明。

噪声扰动、短时域和观测校正

在训练输入中加入适度状态噪声，可让模型学会从小偏差恢复。噪声应与实际误差范围和状态约束相符；任意扰动姿态或接触标签可能形成不存在的状态。

控制时缩短预测时域，可以减少错误累积，但也可能看不到绕障或延迟收益。执行少量动作后重新观察，可以频繁校正状态，这是 **MPC** 的关键。短预测配上终点价值，是另一种取舍：把远期后果交给**价值模型**估计。

起点感知、多步训练、数据覆盖与重新观察分别作用于起点误差、展开失配、未知区域和执行后的状态校正。可以通过固定其他部分的对照，判断预测失败主要来自哪一环，再选择相应改进。

长视频“连贯”与长期可控

模型可以生成一段视觉连贯的拿杯子视频，却在每一步都把动作影响稍微改错。视觉连贯性主要检验时间上的外观关系；长期可控还要求动作变化对应真实后果。

应测试固定动作序列的真实与预测轨迹，并检查位置、接触、终止及对象身份，不能只播放漂亮视频作为长期动力学证据。

5.5.3 自主展开的误差，怎样传回较早的预测

单步预测准确，并不意味着把自己的预测连续当输入时仍然准确。用最小模型看这一区别：真实系统将状态乘以 2，轨迹为 $x_0 = 1, x_1 = 2, x_2 = 4$；模型为 $f_\theta(x) = \theta x$。

如果第二步训练总读取真实 x_1，预测为 2θ，终点损失及梯度为

$$L_{\text{teacher}} = \tfrac{1}{2}(2\theta - 4)^2, \qquad \frac{dL_{\text{teacher}}}{d\theta} = 2(2\theta - 4).$$

如果模型从 x_0 自主展开两步，第一步预测 $\hat{x}_1 = \theta$，第二步读取自己的预测，得到 $\hat{x}_2 = \theta^2$：

$$L_{\text{rollout}} = \tfrac{1}{2}(\theta^2 - 4)^2, \qquad \frac{dL_{\text{rollout}}}{d\theta} = (\theta^2 - 4)\,2\theta.$$

设当前 $\theta = 1$，两者梯度分别为 -4 和 -6。自主展开的终点误差沿两条参数路径返回：第二步的转移使用了 θ，第一步产生的输入也由 θ 决定。它在训练模型处理自己生成的状态，而不是每步都等真实标签纠正输入。这就是通过时间展开求导在动力学学习中的具体含义。

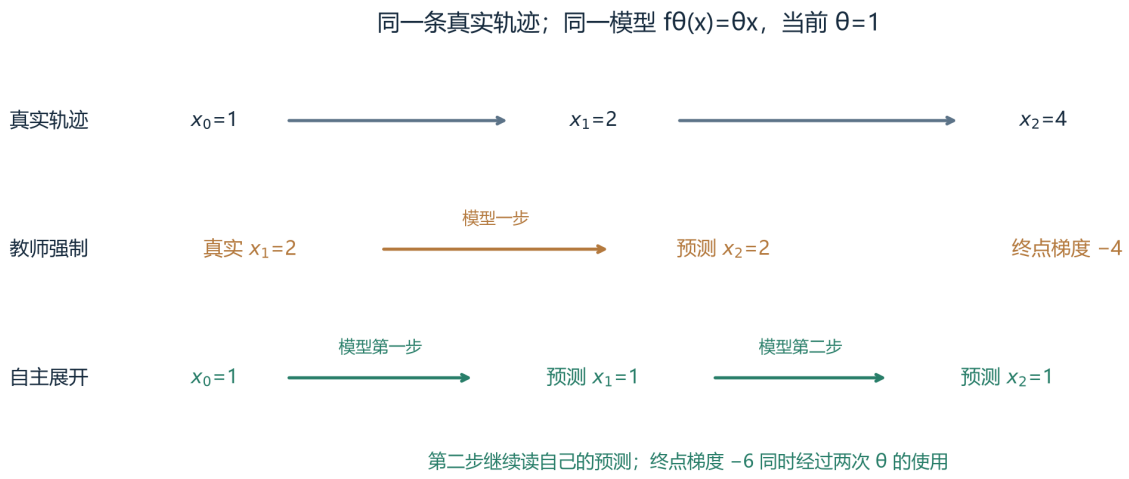


图5.4：第二步是否换回真实状态，会改变预测与梯度。图中 $\theta = 1$，两种终点损失都以真实 $x_2 = 4$ 为标签。

多步损失通常还会加入中间各步的误差，避免只把终点拟合好。展开越长，计算和存储越多，梯度也可能被连续转移放大或缩小；模型预测进入缺少数据的区域时，长期监督仍然可能不足。因此训练与评价都应比较多个展开长度，并把“每步读真实观测”和“只读起点后自主预测”分别报告。CS285 的模型展开与表示学习

5.5.4 练习题

题5.5-1 训练世界模型时每步都输入真实上一状态，部署规划时却输入自己预测的状态。这会产生什么偏移？多步训练应怎样构造？

题5.5-2 真实轨迹x0=1,x1=2,x2=4，模型fθ(x)=θx。当前θ=1.5。只看第二步终点半平方损失，分别对教师强制输入真实x1和从x0自主展开两步求预测、损失及θ梯度。为什么两种训练梯度不同？它们各匹配怎样的实际模型输入分布？

5.5.5 参考答案

题5.5-1答案 单步训练时，模型总从真实状态附近开始预测。规划时，第一步稍有偏差，第二步就从这个有偏差的预测继续，输入可能落在训练中很少见的区域，后续误差因此扩大。变化的是模型实际读到的输入分布，而不只是预测误差的数值。

多步训练可从真实轨迹的状态 s_t 开始，然后只使用自己的预测推进：

$$\hat{s}_t = s_t, \qquad \hat{s}_{t+j+1} = f_\theta(\hat{s}_{t+j}, a_{t+j}),$$

$$\mathcal{L} = \sum_{j=1}^{H} w_j \, d(\hat{s}_{t+j}, s_{t+j}).$$

H 是训练展开步数；w_j 决定第几步误差占多大权重；d 比较预测与标签，例如可取状态各分量的加权平方误差。位置、速度等单位不同，比较前应规定尺度，不能任意把原数值相加。

例如从 s_t 预测得到 $\hat{s}_{t+1}$ 后，第二步计算的是 $f_\theta(\hat{s}_{t+1}, a_{t+1})$，而不是重新换回真实 s_{t+1}。未来真实状态只作为各步标签；动作取自同一真实轨迹，才能把预测与实际后果配对。在潜在模型中，可由读取真实未来观测的编码器或后验提取目标表示；这些真实未来表示用于监督，想象分支仍从自身预测推进。

多步误差经展开的计算路径更新参数，使第一步预测不仅接近标签，也更适合后续继续预测。展开过长可能增加随机性和优化难度，可从短展开开始。它改善的是已覆盖动作序列附近的稳定性，不能补足规划器提出的新动作，仍需检查候选动作分布上的多步表现。

进一步阅读：Berkeley CS285：Model-Based Policy Learning 课程讲义。

题5.5-2答案 本题的标签 x_0, x_1, x_2 都是常量，只计算第二步损失 $\frac{1}{2}(\hat{x}_2 - x_2)^2$；两次模型调用共用同一个 θ。自主展开时保留第一步到第二步的完整求导路径。

教师强制把真实 $x_1 = 2$ 送入第二步：

$$\hat{x}_2^{\mathrm{TF}} = \theta x_1 = 1.5 \times 2 = 3, \qquad L_{\mathrm{TF}} = \tfrac{1}{2}(3-4)^2 = 0.5.$$

由于第二步输入是固定标签，

$$\frac{\partial L_{\mathrm{TF}}}{\partial \theta} = (\theta x_1 - x_2)x_1 = (3-4)\times 2 = -2.$$

即使先算出第一步预测1.5，第二步仍会用真实的2替换它，所以第一步预测不影响本题的终点损失。

自主展开将第一步预测直接送入第二步：

$$\hat{x}_1 = \theta x_0 = 1.5, \qquad \hat{x}_2 = \theta \hat{x}_1 = \theta^2 x_0 = 2.25,$$

$$L_{\mathrm{roll}} = \tfrac{1}{2}(2.25-4)^2 = 1.53125.$$

此时改变 θ 有两种作用：既改变第二次乘法，也改变进入第二次乘法的 $\hat{x}_1$。两条梯度路径相加：

$$\frac{\partial \hat{x}_2}{\partial \theta} = \hat{x}_1 + \theta \frac{\partial \hat{x}_1}{\partial \theta} = 1.5 + 1.5 \times 1 = 3.$$

因此

$$\frac{\partial L_{\mathrm{roll}}}{\partial \theta} = (\hat{x}_2 - x_2)\frac{\partial \hat{x}_2}{\partial \theta} = (2.25-4)\times 3 = -5.25.$$

两个梯度都使小步梯度下降增大 θ，但自主展开同时计入了较大的终点误差，以及参数对前后两次调用的影响。梯度更大不代表训练一定更好，而是它在优化另一个输入与目标组合。

教师强制训练“从真实状态2继续预测”，适合学习真实状态附近的一步转移；自主展开训练“从当前模型生成的1.5继续预测”，更接近没有未来真值可用的想象规划。模型更新后生成状态也会变化，所以自主展开的输入分布依赖模型参数。部署中每步都能读到可靠真实状态，和一次开环想象多步，对模型输入的要求并不相同。

本题还说明只监督终点的限制：$\theta = -2$ 也会让两步终点等于4，但第一步预测为-2，与真实2相反。实际多步训练通常同时约束中间状态，并使用多条轨迹，才能检验模型是否真的学对了过程。

进一步阅读：Stanford CS231n：计算图、链式法则、Sigmoid导数与共享变量梯度累加、Dive into Deep Learning：Sequence-to-Sequence Learning，Teacher Forcing与预测输入、Berkeley CS285：Model-Based Policy Learning，多步模型展开与输入分布偏移。

5.6 奖励、终止、目标与价值的联合学习

5.6.1 预测后果不等于评价后果

模型正确预测“方块会向右移动”，还不知道向右移动是否符合当前任务。目标在左侧时，这个后果可能更差；目标在右侧时，可能更好。**世界模型**描述后果，目标评分决定后果的价值。

奖励 r_{t+1} 是动作 a_t 造成转移后得到的即时任务反馈。它可以来自环境规则、人工标签、可验证物理量或学习模型。**奖励模型** $\hat{r}_\psi(b_t, a_t, b_{t+1}, g)$ 输出即时反馈的估计。

训练奖励模型时，输入是真实轨迹的对应状态、动作、目标，标签是实际奖励。连续奖励可用回归或概率似然；成功分类可用二元**交叉熵**。在任务不同或奖励规则不同的情况下，应明确目标条件与标签含义。

5.6.2 几种奖励怎样得到

推方块可以用负的目标距离作为密集奖励，用进入圆圈并保持稳定作为成功奖励。密集奖励容易提供学习信号，但可能鼓励“接近目标却不完成”；稀疏奖励更贴近任务定义，却在早期少有正样本。

图像目标距离可以直接作为评分，不一定训练奖励头。**VLM** 可以从图像和任务估计进展，但这种奖励仍需校验，尤其是遮挡、看似成功却未稳定等情况。

模型既生成未来图像，又由另一个模型评价图像，两者可能共同接受不真实的成功画面。独立的真实成功判据可以帮助发现这种问题。

5.6.3 继续概率为什么也是一个输出

继续概率 $\hat{c}_{t+1} \in [0, 1]$ 表示转移后回合还能继续的概率。网络输入未来内部状态，输出一个伯努利参数；标签为真实是否终止。用二元交叉熵训练。

如果机器人已经完成任务，模型还继续模拟无穷后续收益，会扭曲动作评价。有效折扣常写成

$$d_{t+1} = \gamma \hat{c}_{t+1}.$$

γ 表示时间折扣，$\hat{c}$ 表示环境结束的可能性，两者含义不同。真实终止后不再**自举**后续价值，但最后一次转移的奖励仍应保留。

时间限制截断不总等于物理终止。若任务本来是无限时长、记录只因采集时间到而停止，通常还应估计后续价值；若“必须在 30 秒内完成”就是任务规则，剩余时间需要成为状态或目标条件，超时也可能是真实失败终止。

5.6.4 价值网络具体评价什么

状态价值 是从当前内部状态开始，按指定策略 π 继续行动的平均预期回报

$$V^{\pi}(b_t, g) = \mathbb{E}_{\pi}\left[\sum_{k=0}^{\infty} \gamma^{k} r_{t+k+1} \mid b_t, g\right].$$

输入是内部状态和目标，输出通常为一个标量或回报分布。它不是预测下一步物体位置，也不是“不考虑后续策略的绝对好坏”。同一状态在不同策略下可具有不同价值。

动作价值 $Q^{\pi}(b_t, a_t, g)$ 评价先执行某个指定动作、之后按该策略继续的预期长期回报。**优势** $A^{\pi} = Q^{\pi} - V^{\pi}$ 表示该动作相对于策略在当前状态平均水平的好坏。世界模型可以帮助估计这些量，但不会自动替代**价值网络**。

5.6.5 价值网络怎样训练

最简单的标签是轨迹后半段实际累计回报。也可使用一步自举标签

$$y_t = r_{t+1} + d_{t+1} V_{\bar{\nu}}(b_{t+1}, g),$$

$$\mathcal{L}_V = \big(V_{\nu}(b_t, g) - \mathrm{sg}(y_t)\big)^2.$$

ν 是价值网络参数，$\bar{\nu}$ 可以是延迟更新的目标参数。标签既包含实际或模型预测的即时奖励，也包含对未来的价值估计。停止梯度使网络学习这个标签，而不通过改变标签来降低当前误差。

真实轨迹标签与想象轨迹标签的可信度不同。后一种同时受到动力学、奖励、继续概率和价值自举误差影响。因此价值损失下降并不保证真实收益改善。

5.6.6 终点价值为什么帮助短时规划

只预测未来 5 步，可能看不到长期目标。可以对前 5 步用世界模型显式模拟，在终点用价值网络估计余下回报。这样减少展开长度，但需要**价值模型**在规划到达的状态上可靠。

若规划器把终点带到训练分布外，价值模型可能过高估计。用价值替代远期展开是误差来源的重新分配，不能当作消除了长期不确定性。

5.6.7 奖励与继续概率怎样参与想象回报

预测继续概率是为了估计未来分支是否还应贡献回报，不是把所有超时都当成真实死亡。奖励头需要来自任务规则或标注的监督，价值头需要由回报或自举目标训练。图像重建好，不会自动产生正确奖励和终止。

各头共享**潜在状态**时，奖励梯度可能推动模型保留任务相关信息，视觉目标可能推动保留纹理。冲突应通过任务与表示指标诊断。若希望模型跨任务使用，应说明奖励是模型内固定头、条件化头，还是运行时外部代价。

5.6.8 练习题

题5.6-1 世界模型预测状态很准，但终止头漏掉碰撞终止。规划或想象策略训练会怎样利用这个错误？继续概率应怎样进入回报？

5.6.9 参考答案

题5.6-1答案 规划器或策略优化器寻找模型评分高的动作。若模型认为碰撞后任务仍会继续，就可能选“先撞上障碍，再达到目标”的序列，把碰撞之后实际上不会发生的收益也算进去。状态预测整体很准，并不能抵消结束规则的错误。

沿用正文索引：执行 a_t 后得到奖励 r_{t+1}；$\hat{c}_{t+1}$ 是这一转移后仍可继续的预测概率，γ 是时间折扣。想象回报按下面的递推计算：

$$\hat{G}_t = \hat{r}_{t+1} + \gamma \hat{c}_{t+1} \hat{G}_{t+1}.$$

前 H 步展开，并在终点用价值补上余下收益时，

$$\hat{G}_t^{(H)} = \sum_{j=0}^{H-1} \gamma^j \left(\prod_{i=1}^{j} \hat{c}_{t+i} \right) \hat{r}_{t+j+1} + \gamma^H \left(\prod_{i=1}^{H} \hat{c}_{t+i} \right) V(\hat{b}_{t+H}).$$

$j = 0$ 时乘积为空，取一，因此当前奖励完整保留。后面的奖励必须乘上此前每次转移的继续概率：只有任务还在进行，它才可能贡献收益。使用真实轨迹时，可把预测概率换成真实的0/1继续标志；对随机世界模型，应沿各条采样轨迹分别累计，再对回报求平均，保留奖励、状态和终止之间的关系。

例如碰撞得到 -2，模型又预测下一步得到 10，取 $\gamma = 0.9$。正确判为终止时只得到 -2；漏报终止、令继续概率为一，则估成 $-2 + 0.9 \times 10 = 7$，危险动作反而受偏好。继续头的监督标签为 $c_{t+1} = 1 - \text{terminated}_t$：真正终止标0，正常继续标1，并单独检查碰撞漏报。持续任务因采集时间到而截断时，不能直接标成任务终止。

进一步阅读：DreamerV3：RSSM 与继续概率的作者论文说明、Gymnasium 官方教程：Handling Time Limits。

5.7 模型预测规划、候选搜索与风险

5.7.1 不确定性怎样进入规划

先区分分布与可信度

世界模型认为“方块左滑或右滑”是两个可能结果，不等于模型已经知道这些概率是准确的。预测分布是模型内部的判断；可信度需要独立数据检验。

在训练数据覆盖的区域，模型可以较好表达随机未来；到了未见过的物体，多个模型也可能一致地预测错。集成分歧是有用指标，但不是通用的错误探测器。

总方差怎样拆分

若先随机选择一个集成模型 M，再从其预测分布采样未来 Y，全概率下有

$$\mathrm{Var}(Y) = \mathbb{E}_M[\mathrm{Var}(Y \mid M)] + \mathrm{Var}_M(\mathbb{E}[Y \mid M]).$$

第一项是每个模型内部方差的平均；第二项是模型预测均值之间的方差。在合适集成假设下，它们分别作为偶然与认知不确定性的近似。这个代数等式成立，不代表模型集成就是精确的参数**后验**。

例如两个模型均值分别为 1 和 3，各自方差 0.25，等概率选择。均值间方差为 1，总方差为 1.25。单个模型内部看来稳定，模型之间却存在显著分歧。

校准与区分能力

校准（calibration） 指模型概率与真实发生频率是否一致。对“抓取成功概率 0.8”的一批预测，若真实成功率接近 0.8，才说明这一概率段较**校准**。

区分能力 指模型能否把较好动作与较差动作排开。一个模型可以总预测成功概率 0.5，在某个成功率恰好一半的数据集里大致校准，却完全不能筛选动作。相反，排序不错的模型也可能过度自信。

验证时同时看概率校准、区间宽度、动作排序和分布外变化。不同场景应分组报告，整体平均可能掩盖接触阶段的严重错误。

最大化平均回报与规避坏结果

假设动作 A 有 95% 概率获得收益 10，5% 概率导致严重失败并得 -100；动作 B 总是得 4。A 的平均收益为 4.5，按期望最大化会选 A。若严重失败不可接受，就需要改变决策准则，而不是期待模型自动“懂风险”。

一种风险敏感代价为

$$J_{\mathrm{risk}}(A) = \mathbb{E}[C(A)] + \beta\sqrt{\mathrm{Var}(C(A))}.$$

C 是代价，越小越好；β 决定对波动的惩罚强度。这使两个平均代价相近的动作中，结果更稳定的一个受到偏好。还可以直接约束失败概率，或采用**条件风险价值**（Conditional Value at Risk，CVaR），关注高代价尾部。例如对最坏 10% 的结果取平均代价，便重点评价较严重的失败。离散样本与分位点并列时需要按所占概率处理边界，但设计意图都是让“少见而严重的后果”在决策中获得明确权重。

方差惩罚把有利和不利波动都计入；CVaR 更关注坏尾部。二者选择依赖实际任务，不能声称一个固定系数保证安全。

不确定性惩罚的使用条件

对模型分歧大、数据距离远的区域加入惩罚，可以减少规划器追逐不可靠预测。但惩罚太强，会使系统拒绝所有新动作，无法探索或改进。

若误差指标与真实误差不相关，惩罚可能只是偏向某类外观或动作尺度。应该在独立数据上检查分歧与误差的关系，再测试不同惩罚强度下的成功率、碰撞率和探索范围。

主动采集最不确定区域的数据也是一条路径。应先筛选可执行动作，再采集能帮助区分模型的交互；“不确定就去试”并不是完整采集方案。

把不确定性放在哪个模块

模型可以输出分布，规划器可以对分布使用风险准则，执行器还可以依据实际观测停止或修正。这三层分别负责“表达可能性”“选择风险偏好”“处理实际异常”。

清楚的系统设计应说明哪一层做了什么，并通过真实测试验证其效果。不能用一张不确定性热图替代控制层的证据。

5.7.2 MPC、CEM 与采样规划怎样选择动作

MPC 是一种决策流程

模型预测控制（MPC） 的基本过程是：从当前估计状态出发，用模型预测有限时域的候选动作后果；优化动作序列；只执行第一个或前几个动作；看到新的真实观测后重新规划。

一次规划得到 $A = (a_t, \dots, a_{t+H-1})$。$H$ 是预测时域，不一定等于总任务长度。目标可以最大化奖励，也可以最小化代价。为了便于手算，本节使用代价

$$A^\star \approx \arg\min_A \mathbb{E}\left[\sum_{k=0}^{H-1} \ell(\hat{b}_{t+k}, a_{t+k}, g) + \ell_f(\hat{b}_{t+H}, g)\right].$$

ℓ 是每步代价，ℓ_f 是终点代价。$\approx$ 表示通常只能做有限计算的近似搜索。若轨迹终止，应按终止规则停止累积或使用正确继续权重。

一个候选排序例子

使用 $\hat{x}' = x + 0.8a$，当前 $x = 0.2$，目标为 0.8，预测 3 步，动作范围 $0 \le a \le 0.3$。定义

$$J(A) = (\hat{x}_3 - 0.8)^2 + 0.1\sum_{k=0}^{2} a_k^2.$$

第一项鼓励到达目标；第二项鼓励动作不要太大。动作的单位和状态尺度必须与系数相容。

候选动作序列	三步预测位置	总代价
A：0.2，0.2，0.2	0.36，0.52，0.68	0.0264
B：0.3，0.3，0	0.44，0.68，0.68	0.0324
C：0.25，0.25，0.25	0.40，0.60，0.80	0.01875

在这三条候选中，C 最好。它不是所有连续序列中的全局最优解，因为动作惩罚可能使最优终点略低于 0.8；有限候选只能保证候选集内最好。

真正执行时只执行第一个 0.25。如果看到实际位置是 0.38，应从 0.38 重新规划，不应继续假定位置是预测的 0.40。

随机 shooting 的具体操作

随机 shooting 指采样多条完整动作序列，逐条展开、评分、选出最好。输入是当前状态、模型和任务评分；输出是一个动作序列。它不训练神经网络参数。

若动作有 7 维，时域为 10，搜索对象有 70 个连续变量。盲目随机采样可能很难找到好序列。可从上一规划序列、**VLA** 策略建议或平滑动作分布附近采样，提高候选质量。

随机 shooting 简单且易并行，但候选数量受计算时限约束。评价它应报告候选数、每条粒子数、展开步数和总延迟，而不是只报告网络一次前向耗时。

CEM 如何逐轮改进候选分布

交叉熵方法（Cross-Entropy Method，CEM）逐轮调整动作序列的采样分布。先从当前分布采一批完整序列，用世界模型展开并评分，保留代价最低的精英样本，再用这些样本拟合下一轮分布。若采用高斯，主要更新精英序列的均值与方差。下一轮围绕更有希望的区域搜索，经过有限轮次后执行选中序列的前缀。这里训练的是一次规划中的搜索分布，世界模型参数保持不变。

```
初始化动作序列分布，常用均值和方差
从分布采样 N 条序列，裁剪或约束到可执行范围
调用世界模型展开并评分
保留得分最好的 K 条精英序列
用精英的均值和方差更新采样分布
重复若干轮，选出动作序列并执行开头部分
```

例如初始动作均值为零，精英序列主要向右推，下一轮分布均值就向右移动。这里更新的均值和方差是此次规划的搜索变量，不是世界模型参数。

名称中的“**交叉熵**”来自分布拟合：把精英样本的经验分布作为目标，使新的采样分布更接近它。在常见高斯设置中，对精英样本做最大似然拟合就得到样本均值和方差。与只记住当前最好序列相比，这保留了一片有希望的搜索区域，下一轮仍能在附近探索。

方差过快缩小可能错过另一条绕障路线。常设置最小方差、平滑更新或保留策略候选。**CEM** 不依赖模型对动作可微，但仍依赖模型评分准确。

MPPI 的加权思路

MPPI（Model Predictive Path Integral，模型预测路径积分）类采样控制从当前动作序列附近加入扰动，对低代价样本给更大权重。一个用于直观理解的权重形式是

$$w_i = \frac{\exp(-(C_i - C_{\min})/\lambda)}{\sum_j \exp(-(C_j - C_{\min})/\lambda)}.$$

较小的温度 λ 使权重集中在少数低代价样本；较大温度保留更多样本贡献。然后用加权扰动更新动作序列。减去 $C_{\min}$ 主要为数值稳定。

完整 MPPI 推导还涉及采样分布、控制代价和重要性修正，上式只是核心加权直觉，不是所有条件下的完整算法。信息论 **MPC** 原始工作给出了更完整的理论和控制实例。Information Theoretic Model Predictive Control

梯度规划改变哪个变量

若模型与评分对动作可微，也可计算 $\nabla_A J$，直接更新候选动作

$$A \leftarrow \mathrm{Proj}_{\mathcal{A}}\left(A - \eta \nabla_A J(A)\right).$$

投影使动作落在允许集合。这里冻结世界模型参数，但仍保留从评分经过模型到动作的梯度路径。冻结参数不等于禁止一切梯度传播。

接触不连续、离散夹爪命令或复杂生成采样会使梯度不稳定，也可能让优化器利用模型局部误差。采样与梯度各有适用条件，可以结合使用。

MPC 为什么仍可能失败

重新观察可纠正状态误差，但若第一次错误动作已经让物体掉落，后续规划来不及补救。模型、目标函数、动作约束、状态估计和控制时限都可能成为瓶颈。

MPC 的优势是对新观测及时适应、任务评分可替换；代价是每次决策都做搜索。对于要求高频反应的系统，可以让快速策略执行，较慢模型负责建议或筛选。

5.7.3 规划搜索会放大预测偏差

从多个动作序列选预测最好的一条，相当于反复询问模型哪个候选最有利。若某些预测带乐观误差，候选越多越容易选到这些错误。风险惩罚、限制动作支持范围、缩短预测范围和每次只执行前缀，都用于控制这种问题，但不能保证模型正确。

配套线性实验中，拟合模型的终点误差约为 0.0114，错误模型约为 0.0617；累计代价分别约为 8.411 和 8.357。两项指标评价不同对象：终点误差只看最后一刻，累计代价则计算整段执行中的取舍。因此前者终点更准，却在这一累计指标上略高，不能据一个指标直接宣布它全面更好。这个无图像、无接触的实验用于观察模型预测、滚动重规划与评价目标的关系。

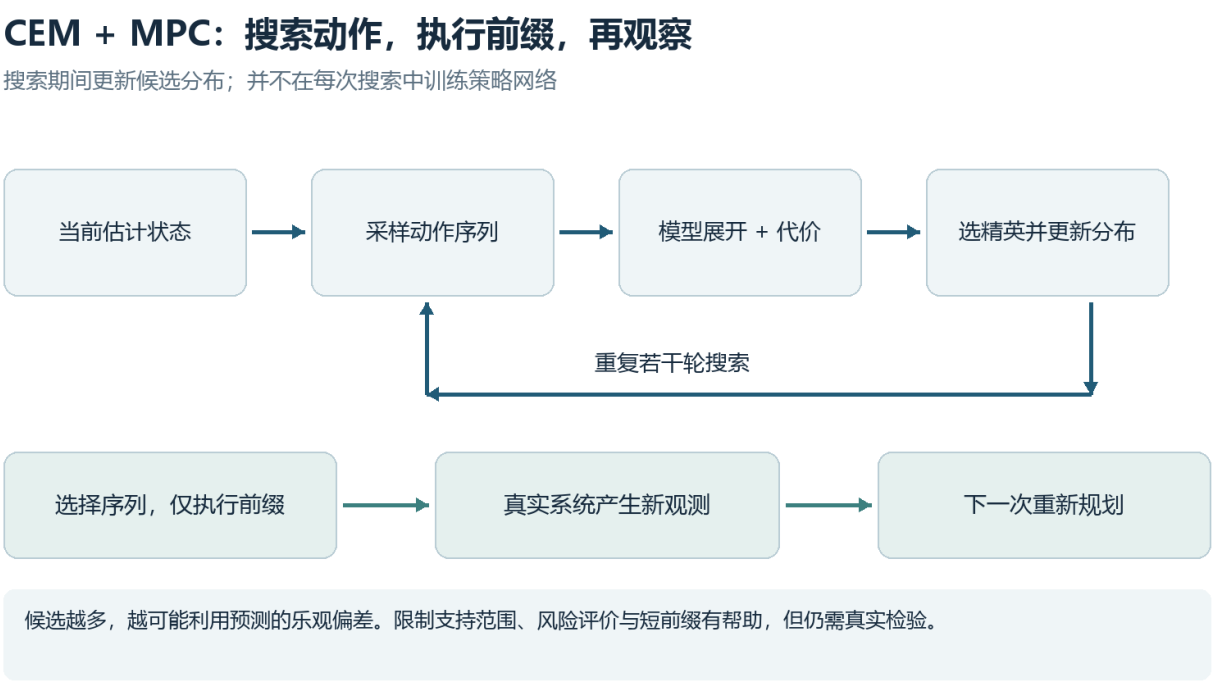


图5.5：搜索更新候选分布，真实系统执行前缀后重新观测。搜索优化的对象与策略训练不同。

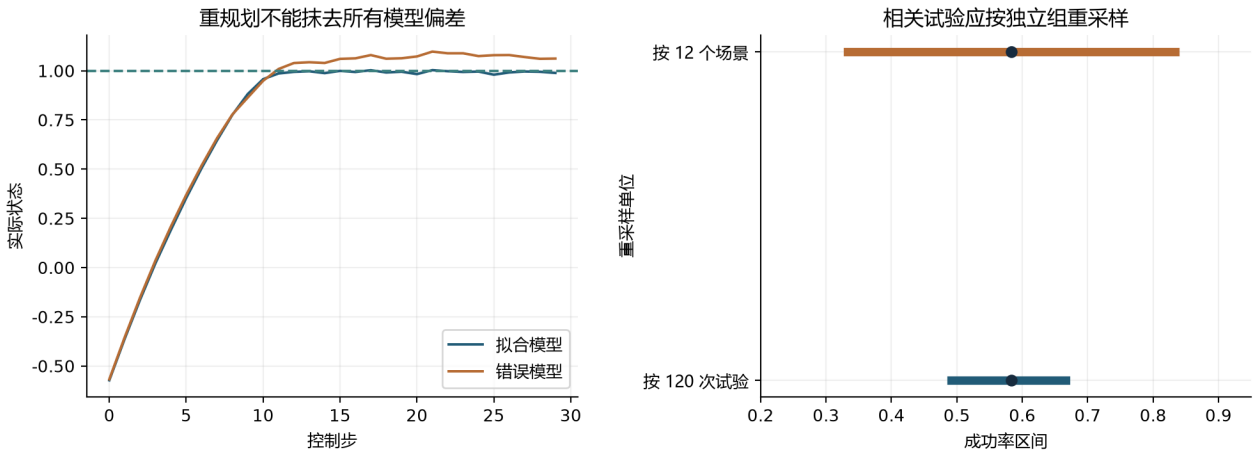


图5.6：左侧为真实线性系统中的**模型预测控制**，右侧为同一组相关试验的不同重采样单位。两者都提示评价口径需要与主张一致。

5.7.4 把 CEM 搜索写成可以运行的程序

下面用一维移动系统完整实现一次搜索。状态转移为 $x_{k+1} = x_k + 0.2a_k$，动作范围为 $[-1, 1]$，从 $x = 0$ 接近目标 $g = 1$。每个候选包含未来六步动作；代价由终点距离和动作幅度组成：

$$J(A) = (x_6 - g)^2 + 0.01 \sum_{k=0}^{5} a_k^2.$$

先采候选、再模拟、再评分、再拟合精英分布；这四步分别对应代码中的编号。搜索只改变候选及其采样参数，状态转移函数保持固定。

```
import numpy as np

def cem_plan(x, goal, seed=0):
    rng = np.random.default_rng(seed)
    horizon, count, elite_count = 6, 256, 32
    mean = np.zeros(horizon)
    std = np.ones(horizon)
    best_cost, best_actions = np.inf, None

    for _ in range(5):
        # 1. 每一行是一整条动作序列，不是一个独立动作。
        actions = rng.normal(mean, std, size=(count, horizon))
        actions = np.clip(actions, -1.0, 1.0)

        # 2. 固定模型并行展开全部候选；这里只需终点状态。
        final_x = x + 0.2 * actions.sum(axis=1)

        # 3. 评价到达目标和动作幅度，并保留实际采过的最好序列。
        cost = (final_x - goal) ** 2 + 0.01 * (actions ** 2).sum(axis=1)
        winner = np.argmin(cost)
        if cost[winner] < best_cost:
            best_cost = float(cost[winner])
            best_actions = actions[winner].copy()
```

```

        # 4. 用较好候选重新拟合搜索分布，留少量方差继续探索。
        elite = actions[np.argsort(cost)[:elite_count]]
        mean = elite.mean(axis=0)
        std = np.maximum(elite.std(axis=0), 0.05)

    return best_actions, best_cost

actions, predicted_cost = cem_plan(x=0.0, goal=1.0)
first_action = actions[0]  # 当前只执行这一项；新观测到来后重新调用搜索。
```

在这个线性例子里，终点等于起点加动作和，所以可以一次算出。换成神经世界模型，就需要沿时间循环调用转移网络；多种可能未来则需要对每个候选展开多个样本再汇总评分。搜索步骤不变，代价计算变得更昂贵。

返回实际采过的最好序列，能保留已经评价过的动作方案。精英均值适合提出下一轮候选，却未必是一条好路径：绕障时，把左绕和右绕求平均可能穿过障碍。CEM 对单峰搜索分布的依赖，与第1章单值回归压平多峰动作的问题相互呼应。CS285 的 CEM 与在线规划课程

5.7.5 CEM 的一次更新怎样产生新候选

交叉熵方法 CEM（Cross-Entropy Method）在这里更新的是候选动作序列的采样分布。用一个标量候选简化说明：本轮提出 $[-2,-1,1,2]$，世界模型给出的代价是 $[9,4,1,2]$，越低越好；保留两个较好候选1、2，其均值为1.5，按总体形式计算的方差为0.25。下一轮就从更靠近1.5、范围更小的分布提出候选，而不是重新训练世界模型去输出更低代价。

真实样本是一整个 $H \times d_a$ 动作序列，更新通常对各位置的采样参数进行；多峰候选也可能在求平均时被压成不合适的中间序列，因此需要检查搜索分布、候选约束和最终选择。迭代若只留下很窄范围，也可能过早放弃其他路线。MPC（Model Predictive Control，模型预测控制）再执行当前选中序列的前缀，用新观测重新启动搜索。前缀执行后的真实反馈，而不是“模型代价已经下降”，构成闭环纠错。Berkeley CS285 模型与规划课程

5.7.6 练习题

题5.7-1 CEM 规划时哪些参数在搜索中改变，哪些保持固定？为什么执行完整最优序列往往不如执行短前缀后重规划？

5.7.7 参考答案

题5.7-1答案 CEM（Cross-Entropy Method，交叉熵方法）在本题中是动作搜索算法。每轮从分布中抽取 N 个长度为 H 的动作序列，经世界模型展开、计算回报，留下评分最高的 K 个，再把搜索分布集中到这些序列附近。使用逐维高斯时，

$$\mu^{\text{new}} = \frac{1}{K}\sum_{i\in E} A^{(i)}, \qquad (\sigma^2)^{\text{new}} = \frac{1}{K}\sum_{i\in E}(A^{(i)} - \mu^{\text{new}})^2.$$

$A^{(i)}$ 是整段动作向量，E 是入选序列集合。改变的是当前候选动作，以及搜索分布的均值和方差；在一次常规规划中，世界模型、奖励模型和策略网络的参数保持固定。改进这一轮搜索不等于训练了网络。

整段执行会连续依赖开环预测：抓取稍偏、杯子滑动或外界扰动，都不能及时修正。执行第一步或短前缀后，读取真实观测重新搜索，可以把预测起点拉回实际状态。前缀太短会增加计算和通信负担；环境稳定、模型准确、动作连续性要求高时可延长。重规划还应处理序列衔接，避免每轮突然切换到另一条路线。

5.8 Dyna、Dreamer、短想象与离线模型改进

5.8.1 Dyna、Dreamer 与想象中的策略学习

为什么不仅在运行时规划

MPC 每次决策都搜索。另一条路线是先在模型里学习一个策略，让它把“好状态下应该做什么”记入参数。部署时策略可以直接输出动作，不一定继续调用**世界模型**。

基于模型的强化学习 指利用学习到或已知的环境模型改善决策。模型用于训练过程，就已经属于这一范畴；不能只按部署时是否显式规划判断。

Dyna 把真实与模拟经验放入同一个学习循环：真实交互得到转移，先用于更新价值或策略，也用于训练环境模型；再选一些已有状态，由模型预测动作后的奖励与下一状态，用这些模拟转移追加学习。模型让一次真实经验支持更多计算中的更新，真实新经验则持续纠正模型。

Dreamer 在紧凑的潜在动力学中产生多步想象轨迹，再训练策略与价值网络。两者都用模型增加学习机会，但模拟对象、梯度和行为学习算法不同。下面按 Dreamer 类模块分工展开，各版本细节以相应论文为准。Dyna，Dreamer。

三组参数，三种目标

世界模型参数 θ 学习“动作造成什么后果”，用真实轨迹监督。策略参数 ϕ 学习“选择什么动作”，目标是提高回报。价值参数 ν 学习“当前策略从这里继续能得多少回报”，使用实际或想象回报标签。

更新对象	输入及监督来源	主要目标
世界模型	真实观测、动作、下一观测及辅助标签	预测真实转移
价值网络	内部状态、目标；回报或自举标签	估计当前策略长期收益
策略网络	内部状态、目标；优势或可微回报	提高动作带来的预计收益

这三种训练不能用一个“世界模型 loss”概括。若采用联合目标，仍应说明每一项更新哪些参数。

想象轨迹怎样产生

从回放缓冲区抽取真实历史，用**后验**网络得到当前内部状态 b_0。之后按策略取动作 $a_k \sim \pi_\phi(\cdot \mid b_k, g)$，由世界模型**先验**生成 b_{k+1}，预测 $\hat{r}_{k+1}$ 和 $\hat{c}_{k+1}$，重复 H 步。

真实观测只用于起点与历史状态推断，想象部分不读取真实未来。回放缓冲区是保存已采集轨迹的容器，不是世界模型本身。

训练策略时通常先冻结世界模型参数。否则策略目标可能通过改变模型，让模型“声称”动作收益更高，而没有让真实行动变好。

Lambda return 为什么把不同长度估计混合

纯累计想象奖励依赖较长展开，误差容易累积；一步**自举**又高度依赖价值估计。λ **回报** 混合短期自举与较长回报。有限时域递推为

$$G_H^\lambda = V_{\bar{\nu}}(b_H, g),$$

$$G_k^\lambda = \hat{r}_{k+1} + d_{k+1}\left[(1-\lambda)V_{\bar{\nu}}(b_{k+1}, g) + \lambda G_{k+1}^\lambda\right].$$

$d_{k+1} = \gamma \hat{c}_{k+1}$。$\lambda$ 接近 0，偏向一步价值自举；接近 1，偏向较长想象回报。它调节偏差与方差等取舍，不是“信任世界模型的概率”。

价值网络读取想象状态，以停止梯度后的 G_k^λ 为目标，学习“从这里继续能得到多少回报”。标量输出可用平方误差，回报分布输出则使用对应的分布损失。停止梯度是为了让预测追随已构造的目标，而不是让这一项同时修改目标。策略随后利用回报相对价值的差异，学习哪些动作值得更常选择。

一段两步想象可以手算

假设 $\gamma = 0.9, \lambda = 0.5$，两步都继续。预测奖励为 1、2，下一**状态价值**为 $V(b_1) = 3, V(b_2) = 4$。

先算尾部 $G_2 = 4$，再得到 $G_1 = 2 + 0.9 \times 4 = 5.6$，然后

$$G_0 = 1 + 0.9\big(0.5 \times 3 + 0.5 \times 5.6\big) = 4.87.$$

若当前价值网络输出 $V(b_0) = 4$，则这条轨迹提供的**优势估计**为 0.87。价值网络被训练向 4.87 靠近；策略可增加产生该较好轨迹的动作概率。4.87 是当前模型和价值估计下的训练标签，不是真实世界已经获得的收益。

策略怎样获得梯度

一种方法使用**策略梯度**

$$\mathcal{L}_\pi = -\sum_k \log \pi_\phi(a_k \mid b_k, g)\, \mathrm{sg}(G_k^\lambda - V_\nu(b_k, g)).$$

优势为正，更新倾向提高该动作概率；优势为负，则降低。还可加熵奖励鼓励一定探索。上式省略了完整算法中的时间或状态采样权重等细节，用于展示优势怎样改变动作概率。具体实现需要与动作采样分布一致的概率计算；用于不同动作生成方式时，应相应调整估计。

另一种方法在连续可微设置中，让策略通过可微模型直接得到回报对动作的梯度。它利用模型的局部变化方向，但容易受到模型梯度错误影响。Dreamer 的不同版本采用不同的梯度与稳定化设计，不能把“所有 Dreamer 都通过图像解码器反向传播”当作事实。DreamerV2、DreamerV3

前一种通常将想象状态与优势视作适当停止梯度的样本；后一种保留策略经动态到回报的路径。实现时应明确选择，不能为了冻结模型就误用禁止全部梯度的操作。

完整训练循环

```
采集真实轨迹 → 放入回放缓冲区
用真实轨迹更新世界模型
从真实历史得到想象起点
冻结世界模型参数，按当前策略构造想象轨迹
计算奖励、继续概率、价值和 lambda 回报
更新价值网络与策略网络
用新策略真实执行，采集纠正模型的经验
重复，并独立评测真实任务成功率
```

模型误差可能被策略放大。策略越善于搜索，越可能找到模型缺口。短时想象、不确定性约束、真实数据刷新和独立真实评估，都是处理该问题的具体路径。

为什么想象样本不等于新事实

想象轨迹来自当前模型。增加采样可以减小某些统计波动，但模型从未学到的接触规律仍不会因此出现。真实数据提供新事实，想象数据提供对当前假设的展开，两者的作用应分别理解。

衡量样本效率时应分别报告真实交互量、预训练数据量、模型训练计算量和想象计算量。只报告“策略用了少量真实步骤”，会漏掉世界模型此前学习所用的经验。

5.8.2 离线数据覆盖、短展开与保守学习

离线场景为什么尤其容易利用模型错误

离线学习 指训练期间不能随意获取新的真实交互。数据由某个行为策略采集，模型只在它覆盖的状态和动作上接受监督。

策略更新可能提出数据中从未出现的动作。模型在这些地方仍会给出预测，优化器可能恰好找到被高估的收益。因为不能在线验证，这种错误可持续累积。

问题不是“离线数据一定不能用”，而是优化范围可能超出证据支持范围。需要约束、覆盖测试和独立验证。

从真实状态分支的短展开

基于模型的策略优化（Model-Based Policy Optimization，MBPO）从真实数据中的状态出发，生成较短模拟轨迹，再把这些数据用于策略学习。短分支减少连续模型误差积累，真实起点减少完全虚构起点的问题。MBPO 原始论文

但是第一步动作也可能分布外，因此短展开并不能解决全部外推。MBPO 的在线交互还允许数据随策略变化更新；把它直接移植成完全离线方案，必须重新处理无法真实纠错的问题。

MOPO 的惩罚思路

基于模型的离线策略优化（Model-Based Offline Policy Optimization，MOPO）用不确定性相关惩罚降低模型在不可靠区域的奖励，核心形式可概括为

$$\tilde{r}(s,a) = \hat{r}(s,a) - \beta u(s,a).$$

u 应与模型误差或未知程度相关。策略在被惩罚的模型中优化，以减少对高风险外推的依赖。理论结论依赖特定误差界与分布假设，实践中的集成不确定性只是近似。MOPO 原始论文

若模型对所有未知状态都过度自信，u 太小，惩罚无效。若视觉变化导致 u 很大而动态并没有变，惩罚可能过分保守。需要检查指标与真实误差之间的关系。

MOReL 的未知区域思路

MOReL 构造悲观模型环境，把被判断为未知的状态动作引向不利的吸收状态。这使策略尽量留在模型较有支持的区域。MOReL 原始论文

“吸收状态”指进入后按设定一直停留、获得规定反馈的状态，不是真实机器人出现了一个新地点。它是训练环境中的保守设计。

这种机制依赖未知区域识别。如果判定太松，会漏掉外推；太严，会把可完成任务的路径排除。

行为约束与策略初始化

可以从已有模仿策略或 **VLA** 初始化，让模型式改进从有经验支持的行为附近开始。规划器也可把该策略作为候选提议分布。

KL、动作偏移或密度约束可以限制策略偏离，但也可能限制改进幅度。若行为数据本身存在系统性错误，紧贴它不能解决任务；应逐步扩大可验证范围，或收集有针对性的补充数据。

动作“在同一数值区间”不等于分布覆盖充分。某个姿态下向右推可以见过，而在另一接触姿态下同一向右动作可能完全没见过。覆盖应是状态与动作的联合覆盖。

数据采集怎样服务纠错

对模型分歧、预测失败和策略经常进入的区域进行定向采集，可以比继续收集重复成功演示更有效。记录失败与恢复过程，有助于模型认识任务边界。

在完全离线限制下，应明确哪些区域仍无证据支持，并限制结论范围。论文不能只凭模型内部收益提升声称真实泛化提高，至少需要独立环境或真实执行评测。

5.8.3 想象学习改变策略，在线规划改变当前行动

模型预测控制在运行时搜索动作序列，主要更新本次候选动作；Dreamer 类方法在潜在模型中产生轨迹，用想象回报更新策略参数，部署时通常由学好的演员选择动作。二者都使用模型，却有不同计算位置、梯度路径和失效方式。

可微模型允许通过预测后果把梯度传到动作和策略，但错误的导数也会误导优化。离散**潜变量**、采样和停止梯度等选择需要明确。想象范围较长时模型误差积累，较短时需要可靠终端价值。评价要同时报告模型训练数据、想象计算和真实交互预算。

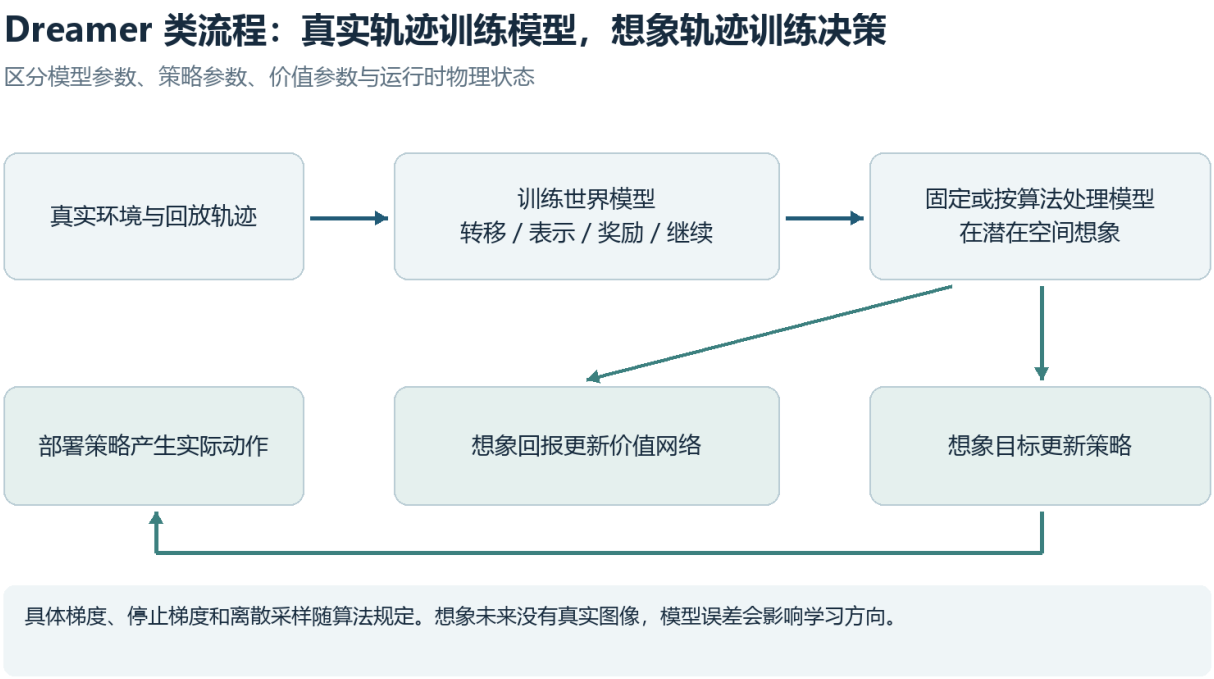


图5.7：真实轨迹训练模型，想象轨迹训练决策，部署演员改变环境。梯度路径由具体算法规定。

5.8.4 Dreamer 原图里的两类训练轨迹

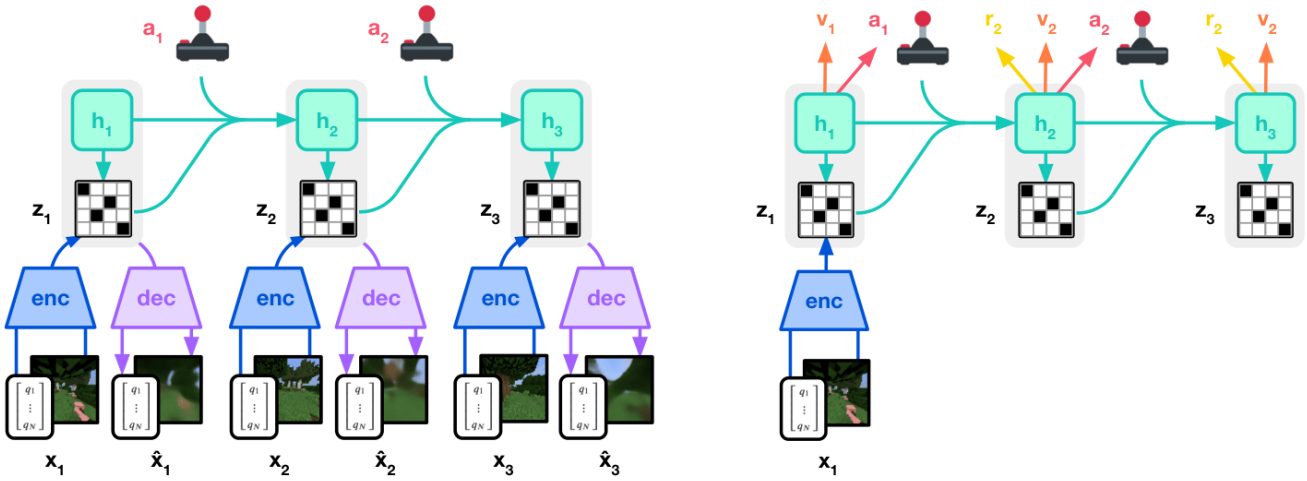


图5.8：Hafner 等，DreamerV3 论文 Figure 3。左侧由真实数据训练世界模型，右侧在模型内部的轨迹上训练演员与评价器。原论文。

图左的 x_t 是真实观测，enc、dec 分别是 encoder（编码器）和 decoder（解码器）；z_t 是由观测推断的随机潜在状态，h_t 是累积历史的循环状态。动作 a_t 影响后续状态预测，重建与真实反馈训练这些模块。这一半主要在学习“世界会怎样变化”。

图右只有起点用真实观测形成状态，后续状态由模型和策略递推。演员 actor 提出动作 a_t，critic（评价器）给出价值 v_t，预测奖励参与构造回报目标。它们学习“在当前模型描述的世界里，应该怎样行动”。右图后面没有真实未来图像，也无需每一步重建一幅图才计算奖励。

看这类模型时，要追踪两条不同的信息链：真实转移更新世界模型，模型生成的转移更新行为。优化行为时若同时任意改动预测模型，系统可能把“预测奖励变好”误当成“动作后果变好”。因此需要按所用算法固定模型参数、安排动作梯度或概率梯度，并最终回到真实任务验证。图的箭头说明计算关系，各代 Dreamer 的具体求导和稳定化方案仍由训练目标决定。

5.8.5 练习题

题5.8-1 Dyna、Dreamer 和模型预测控制怎样使用世界模型？“用模型训练策略”与“每次运行都搜索动作”有何区别？

5.8.6 参考答案

题5.8-1答案 三者的差别是模型产出的预测被送到哪里。

Dyna先用真实转移训练模型、更新价值，再让模型产生额外转移。例如模型生成 $(s, a, \hat{r}, \hat{s}', \hat{c})$，其中 $\hat{c}$ 表示转移后是否或多大概率还能继续；Q学习的标签为

$$y = \hat{r} + \gamma \hat{c} \max_{a'} Q(\hat{s}', a').$$

若合成转移已经终止，$\hat{c} = 0$，只保留最后奖励。这里模型给的是训练样本，更新的是Q或相关策略；执行时可直接按已经学到的价值或策略选动作。

Dreamer先由真实观测形成潜在状态，在潜在空间调用当前行动策略，再由世界模型推进。想象轨迹中的奖励、继续概率和价值提供训练信号，更新行动策略和价值网络。常规执行时，根据当前潜在状态直接调用策略，新真实观测用于修正内部状态，不必每次都搜索一批未来动作序列。

模型预测控制在每次决策时搜索当前最有利的动作序列，用模型评分，然后执行短前缀。搜索主要改变这一次的动作方案，不必同时更新一个策略网络。

“用模型训练策略”把多个状态上的决策经验存入网络参数，运行时前向计算即可出动作；“运行时搜索”则在当前状态重新比较候选。两者可结合，例如用策略提出候选，或让策略学习搜索器选出的动作。

进一步阅读：Berkeley CS285：Model-Based Policy Learning 课程讲义、Dreamer 作者项目说明。

5.9 VLA、世界动作模型与生成式模拟器

5.9.1 世界模型怎样与 VLA 配合

VLA 在系统里承担什么角色

Vision-Language-Action（VLA） 将视觉、语言任务和机器人信息映射到动作。具体输出可能是离散动作 token、连续向量、**动作块**或动作分布，训练通常先利用示范等数据学习。

如果模型只输出动作，它承担策略角色；即使使用了大型视觉语言骨干，也不能据此说它已经显式学会可查询的行动后果。**世界模型**可以成为它的辅助模块，或者与它共享表示。OpenVLA 是 **VLA** 策略的一个具体研究实例。OpenVLA 原始论文

下面四种结合方式的训练流程不同，研究时应分别说明。

第一种：用预测任务改善策略表示

同一视觉骨干同时训练动作预测与未来图像或特征预测。策略损失用示范动作作标签，预测损失用未来观测作标签

$$\mathcal{L} = \mathcal{L}_{\text{action}} + \alpha \mathcal{L}_{\text{future}}.$$

设计动机是让表示不只识别当前物体，还关注与动作相关的变化。部署时可以仅保留动作头，不运行未来生成。

GR-1 将语言、观测历史和机器人状态作为输入，同时预测动作与未来图像，并利用视频预训练。GR-1 原始论文

但辅助预测改善成功率可能来自更多数据、更大网络或训练正则化，不一定证明策略进行了内部规划。要验证机制，可比较相同骨干与预算下移除预测头、打乱未来标签、改用无时间的辅助任务等情况。

第二种：运行时用模型筛选 VLA 候选

VLA 根据当前图像和目标提出多条动作块，世界模型预测各自后果，评分器选出较好动作。这把 VLA 用作高质量候选提议器，减少盲目动作搜索。

数据接口必须一致：VLA 输出的动作参数化、频率、执行时长，应与世界模型训练时相符。若 VLA 输出 20 个高频动作，世界模型却只认识每秒一条状态差，必须建立明确映射。

随机未来需多次评估；候选多样性也重要。如果全部候选都走错误路径，筛选器无法创造新路径。可加入局部扰动、其他策略或更广的搜索。

应与“多采样但不用世界模型”“用**价值模型**筛选”“用真实物理模型筛选”等比较，区分候选数量与预测机制的贡献。

第三种：生成视频计划，再恢复动作

先根据当前画面与语言目标生成未来视频，再用逆向动力学把相邻帧转换成动作。UniPi 研究了这种把策略规划表述为文本引导视频生成的路线。UniPi 原始论文

其动机是让不同机器人的目标都能在视觉空间表达，并利用大规模视频知识。但视频计划可能包含不可实现的物体运动，逆向模型可能遇到多解或看不到隐藏控制量。

要单独评估视频计划可实现性、逆向动作恢复误差和闭环执行。必要时重新观察、重新生成计划，或者先用前向模型检查恢复动作是否真的能产生这些帧。

“生成符合目标的未来”与“在给定动作下预测未来”是不同**条件分布**。前者更像计划提议，后者更像动作检验；组合时不能省掉这一步含义转换。

第四种：在世界模型中后训练 VLA

用当前 VLA 采样动作，让世界模型生成下一观测，评分器产生奖励，终止头判断是否继续，由此构造策略训练轨迹。若 VLA 看图像，模型需要产生它能使用的图像；若策略接受**潜在状态**，则需相应适配和训练。

世界模型先用真实转移训练，奖励或成功评估器另用标签或规则建立。之后冻结或按明确日程更新这些模块，使用强化学习改善 VLA，再在真实环境或独立模拟器验证。

这里策略改变后，采样动作分布也改变。模型原来在示范附近可靠，可能在后训练策略探索区域不可靠。仅在原始演示测试模型误差是不够的。

“世界-动作联合模型”这个名称应怎样读

有些系统联合输出动作与未来观测，或在统一 token 空间中建模二者。其潜在优势是共享表示、跨任务知识和统一生成；代价是目标之间可能冲突，而且条件结构更容易含糊。

检查时需要明确：预测未来是否显式依赖候选动作；动作头能否看到真实未来标签；部署时生成顺序是什么；联合目标是否改善了行动后果预测。

如果动作预测训练时读取真实下一帧，它是在学习逆向映射；部署时却让它只看当前帧，输入条件已经改变。统一模型名称不能代替这些具体说明。

5.9.2 生成式模拟器与世界模型后训练

物理模拟器和学习式模拟器

传统物理模拟器用状态、动力学方程与数值求解器计算转移，往往还能提供精确状态、接触信息和复位接口。学习式模拟器从数据拟合转移，可直接生成视觉经验，减少某些场景建模工作。

二者可以结合。物理模拟器负责几何和接触，生成模型负责外观；学习模型修正未知物理参数；或生成模型给出候选场景，再由物理系统验证动作。

学习式模拟器不因为能反复采样就自动具备真实复位能力。真实世界中的物体破损、液体流失与装配状态变化，在视频生成中可能被任意重置，但这种虚拟可逆性不等于真实可逆性。

UniSim、Genie 与 Cosmos 分别提示什么

UniSim 探索从多种真实数据学习交互式生成模拟器，在控制条件下产生视觉变化。Genie 从视频学习交互环境与潜在动作接口。Cosmos 提供面向物理 AI 的世界基础模型平台。这些工作说明了生成式环境建模的不同方向，训练数据、控制接口和应用范围不能混同。UniSim、Genie、Cosmos

世界基础模型 强调在较广泛数据上预训练，再供多种下游预测、生成或控制任务适配使用。“基础”主要说明复用定位，不是物理正确性的等级。适配到特定机器人仍可能需要行动条件数据、控制映射、任务评分与独立验证；预训练规模大并不能省去这些接口。

尤其是**潜在动作**：它可以表示模型从视频中推断出的变化控制码，并不天然等于某个机器人电机命令。需要额外的标定、对应学习或行动条件后训练，才能用于具体机器人。

摄像机移动、人物移动和机器人末端控制也不是相同条件。论文展示“可交互”，应继续查它究竟允许哪种干预。

生成环境为什么还需要奖励与终止

视觉转移生成器主要提供下一观测。要用于强化学习，还要有任务目标、奖励、终止、有效动作范围、初始状态分布和采样规则。

若奖励依赖真实状态，而生成器没有状态输出，就需学习或重建评分接口。若无法判断杯子实际是否稳定拿起，只判断画面是否像拿起，训练策略可能学会制造假成功。

一个可用于后训练的完整环境应该在实际调用上明确：

```
reset(任务和初始条件) → 初始观测及内部状态
step(动作) → 下一观测、奖励、终止、必要诊断
```

这两个接口看起来像普通模拟器，但内部不确定性、误差范围和状态记忆处理必须说明。

两个具体后训练实例

VLA-RFT 使用从交互数据学习的世界模拟器，预测动作条件未来观测，并用目标参考产生训练反馈。RehearseVLA 使用世界模拟器与 **VLM** 引导的奖励、终止判断支持后训练。VLA-RFT、RehearseVLA

这些是具体方法实例，不是“任何视频模型都能后训练 VLA”的证明。评估其可信度，应看动作对齐、未见策略行为下的预测、奖励误判和独立真实任务结果。

评测中的物理一致性可以具体检查几何、接触、能量、动作响应或可达性。每类检查支持相应结论，组合评价才能反映生成环境的实际控制用途。

为什么同一个生成器不能充当最终裁判

策略在模型 A 中训练，又在模型 A 中评测，主要说明它学会了模型 A 的任务。即使生成视频与奖励看起来一致，也可能共同偏离真实环境。

更有说服力的评测可以使用真实机器人、独立物理模拟器或未参与训练的真实动作后果数据。若只能模型内评测，结论应限于模型内性能，并明确缺少真实迁移证据。

视觉生成本身的训练数据也可能包含评测场景。需要核查场景、目标、轨迹和时间上的重叠，不能只确认策略训练集没有测试任务名称。

把计算成本算完整

生成一次未来可能需要编码历史、维护缓存、多次去噪、解码图像、运行**奖励模型**，再运行 VLA。一个强化学习更新还可能需要许多这样的转移。

报告模型参数规模不足以判断实用性。应分别测每步延迟、峰值内存、每秒环境转移、采样批次和总训练时长。省真实交互与省计算是两种不同收益。

5.9.3 数据集、动作语义与分布变化

数据类型决定可以直接学到什么

人类视频提供丰富物体与场景变化，却通常没有机器人动作命令。机器人演示提供动作与结果，但往往集中于成功行为。主动探索数据可以覆盖不同动作，失败和恢复数据则有助于了解边界。

状态动作数据适合直接学动力学；只有视频的数据适合学视觉表示、视频生成或潜在变化模式。不能因为二者都包含“运动”，就认为具有相同控制监督。

几个常用机器人数据资源

资源	适合关注的方面	使用前要检查
DROID	多样真实场景的机器人操作轨迹	传感器、状态与动作字段及同步
Open X-Embodiment	多机器人、多数据源	动作语义、频率、坐标系和任务差异
BridgeData V2	多任务真实操作	场景划分、控制字段与数据覆盖
RoboNet	多机器人视觉交互	机器人差异与行动预测接口
robomimic 数据与工具	离线示范、基线和数据处理	观测设置、演示质量及评测配置

这些资源面向不同数据收集与研究目标，并非专门为所有世界模型设计。具体字段以官方版本为准。DROID、Open X-Embodiment、BridgeData V2、RoboNet、robomimic

跨机器人不能只统一维度

两个机器人都输出 7 维动作，也可能一个是末端位姿增量，一个是关节命令。末端位移相同，也可能因夹爪形状、控制增益和摩擦不同产生不同后果。

可以增加机器人身份、形态参数和控制模式条件，或使用统一末端动作接口，再让各机器人控制器实现。统一接口只解决部分语义问题，仍应验证低层执行误差。

若关节或工具结构不同，动作映射可能不是一一对应。跨机器人世界模型的泛化目标应说明：是外观泛化、末端动作后果泛化，还是完整控制接口迁移。

数据增强也可能改变物理含义

图像裁剪、颜色变化和背景扰动可提高视觉适应性。但裁剪可能移除接触区域；翻转图像却不变换动作坐标，会让同一个“向右推”在图像中变成相反效果。

增强应考虑几何和控制语义。对多视角图像，独立任意变换也可能破坏标定关系。状态噪声需要符合物理量单位和实际测量误差。

如果增强后的视觉只用于表示预训练，与用于精确行动预测，允许的变换范围可能不同。

覆盖应围绕决策需求检查

看样本总数之外，还应看物体、初始状态、动作方向、动作幅度、**接触模式**和失败比例。十万条几乎相同的成功推方块轨迹，也可能没有一条“推得太大导致掉落”的边界数据。

可画出状态动作分布、分阶段错误与模型分歧，再确定补采集区域。若希望策略学会恢复，模型训练数据也需要包含可恢复状态及相关动作，而不只是任务起点和成功终点。

Sim-to-real 与 real-to-real

Sim-to-real 指从模拟训练迁移到真实环境；**real-to-real** 也可能面临不同设备、光照、控制器和采集流程的差异。

外观域随机化主要处理视觉变化；动力学随机化处理物理参数变化；动作标定处理控制映射。模型失败时应定位是哪一类差异，而不是把一切概括成“域差距”。

迁移评估应固定或记录实际控制接口，并在可比条件下统计多次任务结果。单次成功视频不足以估计稳定性。

5.9.4 生成式模拟器要通过闭环检验

一个根据动作生成未来视频的模型，还需要支持状态连续、动作条件、重置、多步交互和一致奖励，才能成为训练或评价环境。若模型先生成完整视频，再让策略在其中选动作，动作并未真正影响未来，这更像离线观察而不是交互模拟。

有价值的验证包括配对动作的不同后果、接触几何、长期状态一致、策略利用漏洞，以及与真实环境的控制结果差异。世界动作联合模型还需分别检查行动质量与预测质量，不能由一个联合损失下降推断二者都改善。

5.9.5 练习题

题5.9-1 生成式模拟器、世界动作模型与 VLA 的动作条件有什么不同用途？生成未来图像怎样接入 VLA 才具有明确含义？

5.9.6 参考答案

题5.9-1答案 动作这个条件在不同模型里扮演不同角色。普通 VLA（Vision-Language-Action，视觉语言动作模型）通常接收当前观测和任务，输出动作，即 $p(A \mid o, g)$。动作是待生成的结果。动作条件世界模型接收候选动作，再预测后果，即 $p(o_{\text{future}} \mid o, A)$；动作是评价候选方案时给定的输入。交互式生成模拟器也可接收控制输入并生成下一观测，但能生成逼真画面不等于符合真实接触规律。“世界动作模型”没有统一接口，可能联合生成动作与未来，也可能用期望未来反推动作，讨论前需写出其输入输出。

把未来图像接入 VLA，可采用明确的数据流：VLA 先产生几个候选动作块；世界模型分别预测这些动作的未来；评价器比较任务完成、碰撞和风险；执行胜出的短前缀，再读取真实图像。另一种用法是将未来图像当视觉子目标，让受过相应训练的策略从真实当前观测向该目标行动。

前者把预测当动作评价证据，后者把预测当目标条件。两者都不能把虚构未来当作已经发生的真实观测；还需检验动作条件预测是否可靠，或子目标是否可达到。仅把一段好看的视频塞进现有 VLA，不足以确定它学会了哪种决策机制。

5.10 预测、决策和闭环能力的分层评价

5.10.1 怎样评价一个世界模型，而不只评价画面

评价应至少覆盖三层

第一层是预测：真实动作后果与预测是否一致。第二层是决策：预测是否能帮助区分候选动作。第三层是任务：把模型接入控制后，真实成功率和成本怎样变化。

这三层可能不一致。像素误差下降，动作排序可能不变；排序改善，若执行器误差很大，任务仍可能失败。应通过分层指标定位瓶颈。

图像和视频指标衡量什么

PSNR（Peak Signal-to-Noise Ratio，峰值信噪比）基于像素**均方误差**，像素范围为 $[0, 1]$ 时

$$\mathrm{PSNR} = 10 \log_{10} \frac{1}{\mathrm{MSE}}.$$

均方误差（Mean Squared Error，MSE）为 0.01 时 PSNR 为 20 dB。它对整体像素误差敏感，也会惩罚合理但与标签不同的随机未来。

SSIM（Structural Similarity Index，结构相似性指标）关注局部亮度、对比度和结构；**LPIPS**（Learned Perceptual Image Patch Similarity，学习式图像块感知相似性）比较神经网络特征，衡量感知上的差异。它们仍不是接触或动作正确性的直接指标。SSIM 原始论文、LPIPS 原始论文

弗雷歇视频距离（Fréchet Video Distance，FVD）比较真实与生成视频集合在视频特征空间中的分布差异，常用特征均值和**协方差**近似。它评估分布层面的视觉与时间特性，不直接验证每条动作条件预测正确。模型生成像真实数据的常见视频，也可能忽略用户指定动作。FVD 原始论文

随机模型应怎样评分

若实际未来有多种可能，不能只拿一个生成样本与一条标签的像素差，判断整个预测分布。可以评估适用的似然、区间覆盖、成功概率**校准**、样本多样性和任务相关变量。

Best-of-K 取 K 个生成样本中与真实标签最接近的一个。它说明模型是否有能力产生某个接近标签的样本，却不是随机取一个样本时的平均表现。K 越大通常越有利，比较必须固定 K 并同时报告平均或实际选择后的结果。

规划器若没有真实未来标签，就无法在部署时按标签选择 best-of-K。把这个指标当成控制可靠性，会遗漏这一限制。

动作条件正确性

在相近起点下执行不同动作，比较真实与预测的方块位移、接触、夹爪状态和失败概率。可以增加不动、反向和边界动作作为检查。

对模型做“移除动作”或“打乱动作”的消融。如果视觉指标几乎不变，不一定说明动作无关；也可能是数据行为太单一或指标对关键物体不敏感。应查看任务变量和候选排序。

同时测试单步、有真实观测校正的预测，以及完全自主多步展开。把三种结果混成一个误差平均，会掩盖长时问题。

决策后悔量可以揭示排序错误

在可复位的测试环境，对固定候选集 $\mathcal{A}$ 测量真实代价，模型选出 $\hat{A}$。定义

$$\mathrm{Regret} = J_{\mathrm{real}}(\hat{A}) - \min_{A \in \mathcal{A}} J_{\mathrm{real}}(A).$$

它衡量模型选择比候选中真实最好动作差多少。模型整体预测误差略大，却仍把动作排对，可能有低后悔量；平均误差低但把关键失败动作排到首位，后悔量会很大。

真实环境存在随机性和复位偏差时，需多次重复、控制起点差异，并给出不确定范围。这里的最优只限于候选集，不是所有可能动作的全局最优。

Benchmark 选择应配合问题

DeepMind Control Suite 适合较清晰的控制与模型学习实验；LIBERO 可研究多任务操作与迁移；CALVIN 面向语言条件的长时序操作。它们的观测、任务和评测协议不同，不能把得分直接横向排名。DMC、LIBERO、CALVIN

如果研究接触表示，测试集应包含接触差异；如果研究多步稳定，必须测不同展开长度；如果研究模型辅助后训练，必须有独立的任务执行验证。

训练与评测使用同一个学习式生成环境时，结果主要反映模型内能力。若论文目标是真实具身控制，应增加独立物理或真实评估。

公平比较需要记录哪些成本

固定或明确真实交互、预训练数据、模型与策略容量、目标评分信息、候选数、计算预算和控制频率。只给某方法额外未来标签或更强编码器，会混淆机制贡献。

真实任务至少报告成功率、失败类别、完成时间和必要资源；高风险失败应单独统计，而不被平均回报掩盖。多次运行和置信区间帮助判断改进是否稳定，不应只选最好的随机种子。

5.10.2 应用流程：从数据到闭环推方块

先定义一个能验证的项目

任务是：机械臂在桌面上推动一个方块，使其进入目标区域，并保持一小段时间。先在有真实状态输出的物理模拟器中建立原型，再用真实摄像头与机器人逐步验证。

主要研究问题是“学习的动作条件模型能否在有限预测时域内帮助选动作”。暂时不追求语言泛化、复杂物体与通用接触。这样可以先诊断动力学、评分和闭环流程，再扩展到 **VLA**。

输入是两帧或更长图像历史、机器人末端状态与已执行动作。动作是固定频率的平面末端位移和必要的夹爪命令，坐标系及控制器固定。目标是桌面坐标中的位置 g。

成功判据应由独立环境状态计算，例如方块中心进入目标半径、速度足够小并保持规定时长。预测图像“看起来到了”不是原型的最终判据。

数据采集与划分

采集不同起点、方向、幅度和接触位置下的短轨迹，保留成功、失败和恢复。数量需要根据覆盖与验证误差确定；可以先用几百条轨迹调通，再定向补充薄弱区域，而不是先承诺某个数量一定够。

每条转移保存图像、末端状态、动作、下一图像、方块位置与速度、奖励、终止、时间戳。模拟器的方块状态作为训练和评测标签，部署时策略与**世界模型**不能直接读取它。

先按轨迹与起点区域划分训练、验证和测试，再从训练集抽取窗口。另留出一组摩擦或物体质量变化，用于动力学变化测试。相机改变和物理参数改变分别测试，以免无法定位原因。

一个明确的网络方案

采用第5.3节的 **RSSM**：图像与机器人状态编码为特征，记忆维度可以先设 128，随机潜在维度可以先设 32。它们是初始实验配置，需要根据结果调整，不是推荐的普适最优值。

模块	输入	输出与标签
编码器和后验	真实图像、末端状态、历史记忆	当前潜在分布；通过联合损失学习
先验和转移	上一步潜在状态、动作	下一潜在分布；由后验对齐及展开监督训练
图像解码器	内部状态	重建图像；标签为对应真实图像
位置及速度头	内部状态	方块位置、速度；标签来自模拟器
奖励头	转移结果、目标	即时奖励；标签由环境规则计算
继续头	转移结果	是否继续；标签来自真实终止标记

位置与速度头让原型容易诊断，并可直接用于规划评分。这使用了额外训练监督，论文比较时必须让相关基线获得相同信息，或单独做无状态标签版本。不能称它为完全不使用物理状态标签的自监督方法。

模型训练阶段

抽取 16 条、每条 16 次转移的窗口作为一个示意批次，对应 17 个观测。先检查时间与索引，再按真实动作递推，通过真实观测形成**后验**，计算重建、位置、速度、奖励、继续和 **KL** 损失。

连续物理量按训练集尺度归一化；各项损失分别监测，权重根据梯度、任务与验证结果调节。新增短自主展开损失，检查两步、五步等不同长度。训练还没有涉及策略强化学习。

挑选模型不能只看重建损失。验证应包括方块位置多步误差、接触前后预测、不同动作响应、概率区间及候选排序。发现模型在大幅动作下不可靠，可以先限制规划范围，并补采集相应数据。

模型接入 MPC

实时取得图像与末端状态，用后验获得当前内部状态。生成动作序列，在**先验**中展开，用预测位置、速度和动作代价评分，执行最优序列开头，再重新观测。

可以从预测时域 5、候选数 128、每候选 4 个粒子、3 轮 **CEM** 开始测试。一次决策共模拟 $5 \times 128 \times 4 \times 3 = 7680$ 个模型转移。它们可批量计算，但是否满足控制时限仍要实测。

若只有单个随机模型，4 个粒子主要采样它内部的未来分布，并未充分估计认知不确定性。要研究模型未知程度，需增加适当集成或其他不确定性估计。

设动作执行频率为 5 Hz，每次决策预算约 0.2 秒，还要扣除感知与通信。若搜索过慢，可以减少候选、用策略提议或采用更小模型，并检验控制损失。配置本身没有实时性保证。

用一维模型核对训练与规划

为了核对数值链条，暂时把上面的视觉 RSSM 替换为一维可观测模型。假设始终处于接触、没有障碍，$x' = x + wa$。用该教学系统给出的两条转移拟合

$$w^\star = \frac{\sum_i a_i(x'_i - x_i)}{\sum_i a_i^2} = \frac{0.1 \times 0.08 + 0.2 \times 0.16}{0.1^2 + 0.2^2} = 0.8.$$

这是最小二乘解，前提是分母不为零。它没有隐藏状态，也不是完整机器人模型；它的用途是把“先训练参数、再冻结参数搜索动作”的顺序完全算清楚。

从 $x = 0.2$ 到目标 0.8，第5.7节三条候选中 C 代价最低。冻结 $w = 0.8$，选择 C 的第一个动作 0.25。世界模型参数这时没有变化。

新观测怎样纠错

假设本次执行后的观测为0.38，而预测是0.40。这里有 0.02 的位置误差，可能来自摩擦变化、控制误差或模型缺少状态。

MPC 直接从实际 0.38 重新规划。例如未来三步各 0.175，在当前模型中可达到 0.80，动作代价为

$$0.1 \times 3 \times 0.175^2 = 0.0091875.$$

这又是一条候选，并不保证全局最优。关键是使用真实观测重设起点。

同时把 $(0.2, 0.25, 0.38)$ 放入真实缓冲区，用于后续模型更新。这条样本单独隐含的增益是 0.72；是否应该立刻把全部模型改为 0.72，需要考虑测量噪声、此前数据与条件变化。状态修正可立即发生，参数更新则应依据训练方案。

如果改成想象策略训练

先用已经验证的模型生成短轨迹，再按第5.8节计算价值标签和**策略梯度**。策略输入内部状态与目标，输出动作分布；价值输入相同决策信息，输出预计回报。

模型依然用真实转移更新，策略与价值用明确的想象目标更新。定期真实执行，检查模型误差是否随着策略行为变化上升。若上升，应补采集、缩短想象或限制策略偏移。

部署时可以只运行快速策略，也可保留 MPC 筛选。两种部署的计算与效果应该分别比较。

如果进一步接入 VLA

将目标位置扩展为语言任务，同时让 VLA 从真实或生成图像提出**动作块**。先验证动作接口，再选择“候选筛选”或“模型内后训练”路线。

保留独立的成功标签与真实评测。逐步增加对象与任务，避免同时换编码器、生成器、奖励器和策略训练方法后，无法判断改进来自哪里。

项目报告应给出模型错误、动作选择错误和执行失败各占多少。这个拆分比只报告总成功率，更能指导下一轮改进。

5.10.3 从理解机制到设计可发表研究

先把现象改写成可检验问题

“世界模型不懂物理”过于宽泛。可以改成：“在视觉遮挡后的重新接触阶段，现有潜在模型无法区分两种接触状态，导致候选动作排序错误。”这个问题明确了条件、缺失信息和控制后果。

改进假设可以是：“增加接触相关观测或训练目标，有助于保存该区别，并降低决策后悔量。”这不是预设方法有效，而是一个可以通过表示、预测和控制三层证据检验的假设。

如果最终只有重建更好而决策没变，就需要修正“预测改善控制”的叙事，而不是用视觉结果代替控制证据。

代表性路线应该按机制比较

路线与实例	主要机制	审视时的关键问题
PETS	概率动力学集成、采样控制	不确定性是否对应真实错误
PlaNet	潜在动态与在线规划	表示是否保留控制信息
Dreamer 系列	潜在想象中的策略学习	模型偏差是否被策略放大
IRIS、DIAMOND	不同视觉生成动态	视觉细节收益与计算成本
MuZero、TD-MPC2	决策相关潜在建模	任务依赖、价值误差与泛化
V-JEPA 2-AC	特征预测与图像目标规划	目标距离、行动接口与迁移
UniPi、GR-1	视频计划或联合未来预测	可实现性、动作恢复与辅助收益
VLA-RFT、RehearseVLA	学习式环境中的后训练	独立奖励验证与真实迁移

这些类别不是互斥分类，也不是性能排名。具体方法同时可能使用生成、潜在表示和强化学习。原始链接见参考文献；读论文时应回到实际**条件分布**与更新目标。

方法必要性应有对应证据

若声称长期预测问题来自错误累积，应先展示误差随展开长度变化；若声称接触表示不足，应检查状态可区分性与接触阶段错误；若声称控制接口失配，应测试动作尺度、频率和坐标变化。

一种方法可能同时改变表示容量、数据增强和训练目标。要论证机制，最好有逐项消融，并让主要比较在相近数据和计算预算下进行。

消融也需解释失败原因。移除模块后模型容量减少、训练难度增大，会使结果不能只归因于预设机制。可加入等容量替代模块和不同辅助任务。

几个容易不公平的 baseline

新方法使用预训练视觉骨干，基线从头训练；新方法使用物体真值状态监督，基线没有；新方法有更多候选和更长推理时间，基线受更严时限；新方法调了很多参数，基线仅使用一个默认设置。上述差异可能解释表面优势。

建议至少比较：相同骨干的无世界模型策略；相同输入的简单动态；相同世界模型但不同使用方式；已知真实动态或真实状态的上界性对照。上界对照不是可部署竞争方法，应该明确其额外信息。

如果研究目标是实际成本，可以同时报告固定时间预算与固定模型调用预算，而不只给一个最佳配置。

组合已有方法也可以形成贡献

把潜在预测、接触监督和风险规划组合，不因为各模块已有就自动没有创新。可能的贡献在于：针对明确问题建立了必要的系统关系，解决过去组合未处理的接口或训练矛盾，并提供有效证据。

但“没有人用了完全相同模块名称”不足以证明贡献。需要检索目的、数据流、训练目标、表示与使用方式相近的完整系统，说明实际差异和为什么这些差异重要。

例如，已有工作同样用动作条件潜在模型做风险 MPC，你只是更换视觉骨干，贡献可能主要是实证与工程，而不是新规划原理。可以如实定位，或寻找更明确的机制问题。

一套可验证的实验叙事

先确认瓶颈：某类状态下预测与排序失效。再验证表示：新增信息能区分有关状态。然后验证动态：在相同动作下预测更准确。最后验证控制：固定搜索预算下，真实后悔量和失败率下降。

增加域变化与消融，检查改善是否依赖某个数据集或额外监督。记录计算代价，确认收益值得。若证据只支持其中一部分，结论就应限于那一部分。

论文的核心逻辑是问题、机制和证据互相对应，而不是给系统增加尽可能多的模块。

常见审稿问题如何预先解决

同编码器对照可以检查改善是否只是来自更强视觉特征；动作打乱与同起点干预可以检查模型是否使用动作；独立成功判据可以发现评分偏差；跨环境执行可以检查对特定模拟器的依赖。这些实验分别排除不同解释。

“长时改善在哪里？”报告自主展开与长任务。“真实数据成本是否隐藏？”列出预训练和采集预算。“只选了最好种子吗？”报告重复运行与不确定性。

这些检查不是防御性措辞，而是使结论能够被其他研究者检验的证据设计。

5.10.4 学习路线与实现时的诊断顺序

第一阶段，完全算清一个小模型

结合第5.7节的线性模型实践，完成拟合、候选展开、代价比较和真实观测纠错。应能明确指出：什么时候参数变了，什么时候只是动作候选变了，什么时候状态估计变了。

再把模型改成“接触之前方块不动，接触之后按比例移动”，观察只输入方块位置和动作是否足够。加入手位置后比较。这个小实验能直观说明状态信息与模型表达能力的区别。

第二阶段，预测低维真实状态

选择简单物理环境，输入当前状态与动作，训练确定性模型；再加入概率输出或集成。比较单步与多步误差、预测区间和动作排序。

保持策略与环境不变，先调通模型接口。若低维模型在动作对齐上都失败，大型视频模型只会让原因更难定位。

第三阶段，加入视觉与潜在状态

用同一环境把输入改成图像，但保留真实状态作为评测标签。先训练编码与重建，再训练时间与动作预测；随后构建 RSSM，比较真实后验校正和纯先验展开。

查看每个损失与真实状态读出，检查潜在变量是否坍缩。不能只调总损失直到曲线平滑。借助冻结编码器与端到端版本，理解表示和动态共同学习的取舍。

第四阶段，选择一种使用方式

先用 MPC，因为动作搜索过程可直接观察与手算；确认预测能改善候选排序，再考虑在想象中训练策略。若策略训练失败，分别检查奖励、继续标签、价值目标和采样概率计算。

图像生成和 VLA 接入可以作为后续扩展。不要第一步就同时学习视觉、长视频、语言、多机器人与复杂奖励，否则很难知道哪个机制出了问题。

阅读一篇论文时填写一张机制卡

问题	你应写出的具体答案
输入是什么	哪些观测、历史、动作、任务和时间信息
输出是什么	状态、特征、图像、分布或动作
标签从哪里来	真实未来、动作记录、目标编码器或奖励规则
哪些参数更新	编码器、动态、解码器、策略、价值各自的目标
训练与部署怎样区别	是否看见真实未来；怎样采样和修正
模型怎样用于决策	搜索、筛选、策略训练或仅辅助表示
成立依赖什么	观测充分、覆盖、动作接口、评分准确与计算预算
证据是什么	预测、排序、独立任务结果与公平成本

整理机制卡时，对照正文、原始文献与实现，逐项填入输入、输出、标签和参数更新。这样可以按照实际计算比较论文。

出错时按数据流排查

先查时间戳、动作单位、坐标和轨迹重置；再查输入信息是否足够；再查分布假设和多步展开；然后查评分与终止；最后查搜索或策略优化。

模型预测正常但控制失败，可能是评分错或搜索不足。模型内控制好、真实失败，可能是覆盖、接触或生成奖励偏差。真实图像作为输入时好、生成图像作为输入时差，可能是 VLA 对生成外观不适应。

按这条顺序可以减少无目的地更换模型。每次改变应对应一个已经观察到的错误，并用独立结果检查是否真的解决。

学习与实践的连接

你不必背下每一种模型结构，但应能解释一个系统为什么需要预测模块，指出它的输入、输出和监督；分清先验想象与后验校正；区分生成质量和行动正确性；完整描述模型、策略、奖励和价值的更新流程。

当一个新方法声称“世界模型能提升具身智能”，你应能进一步判断：提升的是表示、规划、策略学习还是数据生成；它依赖哪些证据；下一步最需要做什么实验。

5.10.5 一项世界模型研究需要三层证据

第一层预测证据说明模型在什么输入分布、什么展开长度下预测正确；第二层决策证据说明使用模型后在同预算下选择更好动作；第三层闭环证据说明真实或独立仿真执行改善，并排查额外传感、搜索次数和奖励信息的贡献。不是每项论文都要声称覆盖三层，但主张应与证据匹配。

如果目的仅是学习更好的表示，可以重点验证冻结读出与策略适配；如果目的为控制，应补动作干预、模型外推、规划漏洞和终点约束。这样能够围绕明确问题形成研究，而非仅用更大模型生成更漂亮的未来。

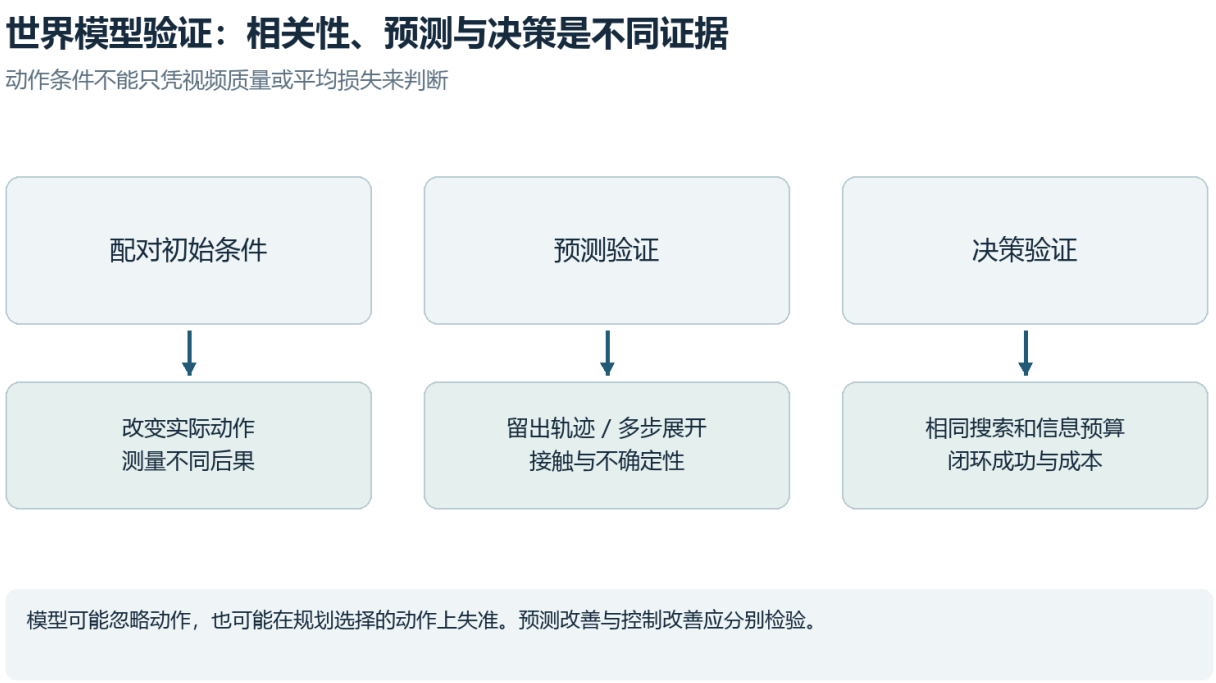


图5.9：动作干预、预测和闭环决策提供不同层次证据。流畅视频与低预测损失不能替代任务验证。

5.10.6 练习题

题5.10-1 世界模型评价包括像素质量、动作后果预测、动作排序、闭环成功。设计逐级检查，定位“视频好看但规划差”的原因。

5.10.7 参考答案

题5.10-1答案 应沿同一批状态逐级检查，并记录哪一级先出现问题。

1. **画面与连续性。**固定真实历史和实际动作，对比预测视频与真实未来，查看清晰度、物体身份和时间连续性。背景清楚而杯子位置错误，说明整体图像指标没有突出关键对象。

2. **动作后果。**在同一可复现初始状态，执行向左、向右、保持等候选动作，比较杯子位移、接触、奖励和结束。若模型对各动作给出几乎相同预测，它主要学到了行为习惯，或动作编码没有有效进入模型。

3. **候选排序。**对同一组动作，同时取得模型分数和真实执行分数。检查模型选中动作的真实损失，以及它是否把危险动作排到前列。后果有偏但排序正确时仍可能可用；像素误差低而排序错时，规划依然困难。

4. **闭环任务。**固定候选数、搜索轮数和延迟预算，比较模型规划、相同规划器接入较可靠模型，以及直接策略。若动作排序已经正确但闭环失败，再查观测时延、动作接口、前缀长度和重新定位。

这样可以分别定位视觉解码、动作敏感性、评价目标和执行闭环的问题。真实多分支后果可先在可重置仿真中取得，真机重复实验则需记录重置误差；不能把不同起点的结果当成相同起点的动作比较。

第6章：仿真环境、软件实践与 Sim2Real

仿真把物理、传感与任务规则组织成可交互环境。本章介绍常见软件层次和最小使用流程，再讨论随机化、辨识与真实迁移。

6.1 物理引擎、软件层次与机器人资产

6.1.1 仿真软件各层承担不同职责

物理引擎推进刚体、关节和接触；场景系统管理机器人、物体和传感器资产；任务环境把场景包装成 reset 与 step；学习框架收集轨迹和更新策略；机器人通信系统连接真实接口。安装一个物理引擎，并不会自动获得奖励、成功判定、示范采集和训练算法。先分清层次，再选择软件，能避免把不同层的工具当作互斥替代品。

6.1.2 物理引擎、模拟器与学习环境不是同一层

物理引擎根据几何、质量、关节、执行器和力，推进物理状态。渲染器根据场景、相机和光照生成图像。模拟器组织资产、物理、传感器和软件接口；任务环境再定义观测、动作、重置、奖励和成功条件。

训练框架调用任务环境，运行策略并更新参数。一个软件可以覆盖多层，但这些职责仍不同。物理引擎能稳定走一步，不等于任务定义或策略训练正确。

图6.1：物理、视觉、任务与训练分别回答不同问题。资产和接口将它们连接起来。

MuJoCo 提供机器人动力学与 Python 接口；Isaac Sim 组织机器人仿真与传感等生态；Isaac Lab 提供学习任务与训练组织；ManiSkill 提供操作任务框架；Gazebo 常用于机器人软件、传感与系统集成。选择需看任务、硬件和工具链，而非统一精度排名。MuJoCo、Isaac Sim、Isaac Lab、ManiSkill、Gazebo。

6.1.3 先把软件版本与运行条件记录下来

软件接口、支持硬件与依赖会变化。固定软件版本、Python、驱动、物理后端、资产与配置，比一句“使用最新版”更可复现。latest 或 main 文档是学习入口，复现实验应改成与本地匹配的版本或提交。

MuJoCo 最小例子可在普通的**中央处理器**（Central Processing Unit，CPU）上推进物理，不需要窗口；图像渲染需要相应图形环境。headless（无界面）表示没有交互窗口，不表示不计算视觉，也不保证不需要 **图形处理器**（Graphics Processing Unit，GPU）。

物理推进与图像渲染有不同依赖。学习单关节响应时，可以先运行无窗口的物理程序；加入视觉策略后，再建立相机、渲染和图像读取路径。这样能先确认“命令怎样变成运动”，再检查“运动怎样变成模型输入”。

问题诊断先区分安装与加载失败、资产路径错误、物理不稳定和任务逻辑错误。它们不应全部归为“仿真精度差”。

6.1.4 机器人资产包含哪些必须核对的信息

机器人模型包含 link、joint、质量与惯性、执行器、视觉几何和碰撞几何。**URDF**（Unified Robot Description Format，统一机器人描述格式）描述机器人结构；**MJCF** 是 MuJoCo 的模型描述格式，采用**可扩展标记语言**（Extensible Markup Language，XML）组织模型元素；**通用场景描述**（Universal Scene Description，USD）用于组织场景资产；Gazebo 常使用 SDF（Simulation Description Format，仿真描述格式）场景和模型描述。

格式转换可以保留结构，但执行器、材料、碰撞与物理参数未必一一对应。模型看起来正确，不证明关节轴、惯性或控制方式正确。

图6.2：视觉网格负责外观，碰撞代理负责接触，惯性和执行器负责响应。几类信息分别检验。

检查长度、质量、角度与轴约定。MJCF 可以设置角度解释，这一示例使用弧度，避免把数字 1 当成一度。自由关节状态维数还可能与速度维数不同，不能假定 qpos 和 qvel 总同长。

合理顺序是先固定底座和重力，检查单关节；再检验整体重力平衡；再加碰撞与工具；最后加复杂场景。初期全部一起加载，难以定位错误来源。

6.1.5 物理仿真包含哪些假设

模拟器不是只读一个三维模型然后自动变真

外形决定可视与部分碰撞几何，动力学还需要质量、质心、惯性、关节、摩擦、驱动与接触配置。**数值积分**、求解器和控制频率又决定怎样计算变化。

URDF 是 Unified Robot Description Format，统一机器人描述格式，常描述连杆、关节与相关属性。MJCF 是 MuJoCo 的模型描述格式。文件能加载，不表示参数与真实机器人已经匹配。

仿真数据是建立在这些设定下的后果，不能自动作为所有真实环境的物理真值。MuJoCo 建模文档说明了模型、坐标、接触与相关配置。

必须区分三组参数

参数类别	例子	它改变什么
物理与对象	质量、惯性、摩擦、真实顺应	希望描述的物理响应
执行与感知	控制器、驱动响应、延迟、噪声	学习策略实际接触的接口
数值与近似	积分步长、求解迭代、碰撞近似	后果计算的精度与代价

有些接触参数既是现象近似又是数值设计，并非都能直接量成真实材料常数。调节它们可以改善模拟拟合，也可能只是在补偿其他误差。

为什么要先验证简单物理实验

空载跟踪检验执行系统；静态承载检验重力、负载与夹持；推物体检验摩擦和接触几何；碰撞检验恢复与时间尺度。用已知或可测条件比较响应，有助于定位偏差。

这不要求所有参数都独立精确测量。它要求先确认模拟中策略依赖的关系是否正确，比如增大负载是否使任务更难，改变摩擦是否以合理方式影响滑动。

任务成功率高，仍可能是策略利用穿模、错误夹持或永不滑动的模型。简单机制验证与完整任务验证相互补充。

并行训练与时间分辨率不能混为一谈

更多并行环境增加采样吞吐量，throughput，即单位时间产出；不会自动提高每个环境接触计算的精度。更粗物理步可以快，但可能改变任务难度。

对照方法使用不同控制频率、求解器、接触设置或碰撞几何时，最终差异可能来自环境，而不只是算法。应固定关键设置或分别报告影响。

软件版本与默认值属于复现实验

软件可能改变接口、参数或求解实现。记录模拟器版本、模型、控制器与运行配置，避免只写“使用 MuJoCo”。

不同引擎对接触、执行器和传感器采用不同近似。读配置时，先找它约束或计算的对象，再比较响应。例如同名的“刚度”可能属于执行器、材料接触或数值软约束，作用位置不同，就不能用相同数字代替相同物理效果。

6.1.6 学软件从一条数据链开始

把一次完整调用画成“资产文件→模型加载→状态重置→控制命令→物理子步→传感器读数→观测组装→奖励终止”。每一步保留最小可观察输出。模型加载成功后，先检查关节数量、轴、单位、惯性和初始姿态；再施加一个已知动作，核对响应；最后才加入图像、复杂任务和学习。

6.1.7 练习题

题6.1-1 物理引擎、场景模拟器、学习任务库分别提供什么？同一机器人资产导入成功为何还不代表环境适合策略训练？

6.1.8 参考答案

题6.1-1答案 物理引擎负责“施加力和控制输入之后，下一时刻的物体状态是什么”，包括动力学积分、碰撞和接触。场景模拟器组织机器人、物体、相机、灯光和渲染，把物理状态转换成可供感知的观测。学习任务库进一步定义策略读哪些观测、动作怎样变成控制输入、奖励怎样计算、何时结束和怎样重置。实际软件可能同时包含几层，但职责仍应分清。

资产导入成功只说明文件能被解析，机器人能出现在场景中。模型还可能有错误的碰撞几何、质量和惯性；关节轴、顺序或执行器也可能不正确。例如画面上的手指正常，碰撞体却比手指厚，抓取接触就与视觉不一致。

训练前应至少走通一条真实转移：已知初始状态和命令，确认运动方向、控制单位和约束；检查相机和机器人状态对应同一时刻；测试成功、失败和重置；确认奖励确实偏向所需行为。否则强化学习仍可优化，只是优化的是一个定义错误的任务。渲染成功与任务适合学习，是两个不同检查层次。

6.2 时钟、执行器和环境重置

6.2.1 物理步、控制步与相机帧怎样配合

物理步长 Δt_p 决定引擎一次推进多长模拟时间。一个控制动作可以保持 n 个物理步，控制周期为 $\Delta t_c = n\Delta t_p$。相机还可以有独立更新频率。

示例中的 Δt_p=0.002 秒，每控制步走 10 个物理步，控制周期 0.02 秒，即 50 Hz。若相机每 40 个物理步更新一次，则周期 0.08 秒，即 12.5 Hz。两路数据需要时间对齐。

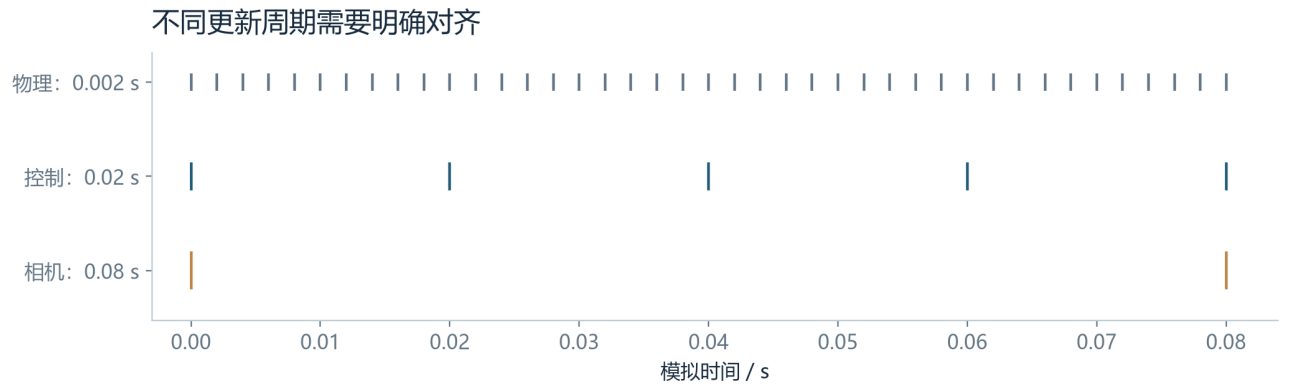


图6.3：保持动作、推进物理与更新相机是不同事件。加快墙钟执行，不应改变模拟时间语义。

模拟时间与现实墙钟时间不同。跑三秒模拟可能只花一小段墙钟，也可能更慢。策略输出速度时应按模拟控制周期解释，而非程序实际每次花的毫秒数。

步长过大可能影响稳定性和接触结果，步长小则增加计算。改变 n 也改变策略观察与修正频率，不能只认为是性能参数。比较训练必须明确物理、控制、传感与策略更新的各个周期。

6.2.2 同一个 ctrl 数字，为什么会产生不同运动

策略输出进入 data.ctrl 后，还要经过**执行器模型**才能变成关节力。理解这层转换，才能把示范动作、仿真训练和真机控制接起来。以传动比例为 1 的单转动关节为例，两种常见定义是：

执行器	输入 u 的含义	产生力矩的简化关系
直接力矩执行器	期望施加的关节力矩	$\tau = u$
位置执行器	期望到达的关节角	$\tau = k_p(u - q)$

设当前角度 $q = 0.2$ rad，输入 $u = 0.5$。第一种定义施加 0.5 N·m；第二种定义在 $k_p = 20$ N·m/rad 时施加 6 N·m，推动关节向 0.5 rad 运动。角度接近目标后，位置误差和这部分力矩都会减小。输入数字相同，代表的量和反馈规律不同。

阻尼或速度反馈还会抵抗运动。若另有 $-k_d\dot{q}$，总响应近似为 $\tau = k_p(u - q) - k_d\dot{q}$。力矩限幅又会把过大的输出裁到允许范围；因此目标角度并不是下一时刻的实际角度。MuJoCo 用增益、偏置和传动组合表达这些关系。MuJoCo 执行器计算说明

完整链路是“归一化动作→反归一化后的命令→执行器产生力→传动映射到关节→动力学推进”。若执行器作用于肌腱，一个输入还可能同时影响多个关节。迁移策略时，要匹配这条链路，而不只是匹配动作数组的长度。

6.2.3 重置与随机种子为什么影响训练和评价

重置需要回到合法物理状态，清掉回合计数、控制命令、任务标记和相关历史。只改关节位置，其他速度、接触缓存或控制器状态可能仍保留旧影响。

MuJoCo 的 mj_resetData 重置运行数据，手动设置需要的状态后，mj_forward 更新相应派生量。模型结构参数与运行状态分别在 model 和 data 中管理。MuJoCo Python 接口。

训练随机化范围与测试范围要分开。选择验证结果最好的随机化配置后，应在独立条件测试，避免把测试信息反复反馈进仿真调参。

6.2.4 一次 step 可以包含很多不同事件

物理步长为 0.002 秒，策略周期为 0.02 秒时，每个策略动作保持 10 个物理子步。相机可能每 0.04 秒更新，所以两个策略周期共享同一帧；控制器还可能在每个子步重新计算力矩。这样一个“step”返回的观测，不一定是所有传感器同时在末尾采样。

重置应恢复关节位置、速度、物体状态、控制目标、历史缓冲、随机参数和传感器状态。只把位置重置而保留上一局速度，会使初始分布错误；只重置物理而保留策略记忆，会让新回合带着旧任务信息。测试 reset 后的前几步，比只检查初始化画面更有效。

6.2.5 一个动作保持多步，奖励目标怎样随之改变

动作重复（Action Repeat）是策略给一次命令，低层在多个物理步保持它。它减少策略调用次数，但把原来的若干决策间隔合成一段。若原任务每物理步折扣为 γ_p，想保留同一回报，应把内部奖励及外部折扣一起改：

$$R^{(k)} = r_0 + \gamma_p r_1 + \cdots + \gamma_p^{k-1} r_{k-1}, \qquad G = R^{(k)} + \gamma_p^{k} G_{\text{next}}.$$

例如两步奖励为1、2，$\gamma_p = 0.9$，这段奖励为2.8、下一段折扣为0.81。直接把两步奖励相加为3，再用原折扣0.9连接下一段，会改变奖励的时间权重。某些环境有意按控制步重新定义目标，可以采用另一约定，但应清楚自己在优化哪种时间尺度。若中途真正结束，只累计实际经历的步数，不继续给未执行的命令奖励。Gymnasium 时间限制教程

这里的奖励是假定已经定义好的每步累计量；若写的是每秒成本率，还需要按物理步长积分。看仿真接口时，应同时确认状态什么时候更新、动作保持多久、奖励在哪里累计，不能只根据一个 step() 名称判断它代表的时间。

6.2.6 练习题

题6.2-1 控制器每次保持命令 k 个物理步。设物理步奖励为 r_j，物理步折扣为 γ_p，要求策略环境保留相同的折扣回报目标，环境该怎样累计块奖励并换算块间折扣？若中途结束应怎样处理？重置遗漏关节速度会产生什么偏差？

6.2.7 参考答案

题6.2-1答案 令物理步奖励为 r_j，物理步折扣为 γ_p，一次动作保持 k 步。要保留原来的物理步回报，环境返回

$$R_n = \sum_{i=0}^{k-1} \gamma_p^{i} r_{nk+i}, \qquad \gamma_e = \gamma_p^{k}.$$

因为

$$\sum_{n\geq 0}\gamma_e^n R_n=\sum_{n\geq 0}\sum_{i=0}^{k-1}\gamma_p^{nk+i}r_{nk+i},$$

右侧正是按每个物理步折扣的回报。块内折扣与块间折扣缺一不可。例如 $k=2,\gamma_p=0.9$，两步奖励为1、2，返回 $R=1+0.9\times 2=2.8$，块间折扣为0.81。当 $\gamma_p<1$ 时，直接把块内奖励相加会改变原目标；每秒奖励率还应乘物理步长，形成相应时间积分。

若当前块在实际执行 $m<k$ 步后停止，只累计这 m 步，剩余价值的折扣应为 γ_p^m。真正任务终止时，后续价值为零；若只是外部采样截断、任务仍可继续，则标签为

$$y=R_n^{(m)}+\gamma_p^m V(s_{\text{final}}).$$

这里必须使用重置前的最终状态，不跨过重置连接到新回合。

重置位置而不重置速度，会让新回合带着上一回合的动量开始，例如手臂回到起点后立即前冲。如果观测显示残留速度，初始分布已经改变；若观测被清零而实际速度未清零，观测还会与物理状态矛盾。执行器内部状态和命令缓存也应按任务定义重置。

进一步阅读：MuJoCo 官方文档：Computation，状态和执行器模型、Isaac Lab 官方教程：Creating a Direct Workflow RL Environment。

6.3 MuJoCo 中的模型、状态、接触与相机

6.3.1 场景文件定义了什么，物理引擎推进什么

MuJoCo 的场景采用其模型描述格式 **MJCF**，以可扩展标记语言（XML）描述刚体、关节、几何体、执行器和传感器。先看最简单的单关节：一根杆绕转轴运动，位置执行器接收目标角度。第6.8节的项目保留了这个例子，入口为 practice/single_joint。

加载后，MjModel 保存关节范围、质量、惯量和执行器参数等模型描述；MjData 保存某次运行中的位置、速度、时间及由它们计算出的量。同一模型可以配多份独立运行状态，例如多份并行环境，或者一份实际状态加一份运动学求解副本。随机化时可以修改模型参数，但本例让它们在每个回合内保持固定。

下面读取单关节 XML，创建状态，把位置目标设为0.5 rad，再推进物理：

```
from pathlib import Path
import mujoco

# 1. 编译物理模型；由Python读取内容，可兼容含中文的文件路径。
xml = Path("practice/single_joint/one_joint.xml").read_text(encoding="utf-8")
model = mujoco.MjModel.from_xml_string(xml)
data = mujoco.MjData(model)

# 2. 写入执行器目标，不直接改关节实际角度。
data.ctrl[0] = 0.5

# 3. 每一步根据力和约束更新位置、速度与时间。
for _ in range(1500):
    mujoco.mj_step(model, data)

print(data.time, data.qpos[0], data.qvel[0])
```

XML把物理步长设为0.002 s，故1500步对应约3 s。此单关节设置重力为零，目的在于先观察执行器目标如何带动角度变化。运行后角度趋近0.5、角速度接近零，只支持这套单关节模型的跟踪结论；机械臂在重力和接触下的响应另在6.8节检验。MuJoCo官方Python教程也按模型、状态、渲染和物理推进逐步组织示例。

6.3.2 位置、速度与控制输入为什么不是同一个数组

qpos 是**广义位置**：用能确定系统构型的坐标描述当前姿态。单个转轴用一个角度；自由运动的物体则用三项平移与四项单位四元数。qvel 是**广义速度**，自由物体用三项线速度和三项角速度。四元数有单位长度约束，四个分量只表示三个独立旋转自由度，所以位置坐标数 nq 可以大于速度维数 nv。

第6.8节的 Panda 场景可逐项核对：

系统部分	项数	项数	为什么
七个机械臂旋转关节	7	7	每轴一个角度、一个角速度
两个手指滑动关节	2	2	每指一个位移、一个速度
自由圆柱	7	6	三维平移加四元数，对应三维线速度与角速度
合计	16	15	与程序加载结果一致

ctrl 的长度 nu 则由执行器数量决定。两指各有滑动位移 q_{f1}, q_{f2}，模型以 $l = (q_{f1} + q_{f2})/2$ 作为共同开合坐标，让一个执行器对此坐标施力，再用同步等式约束协调两指运动。因此，一个开合通道可以驱动两指，位置与速度数组仍各保留两个手指坐标。加上七轴机械臂执行器，得到 nu=8。

关节数、位置坐标数、速度维数和控制数分别描述不同对象，不必相等。读取特定关节时，可通过关节名字及其 jnt_qposadr、jnt_dofadr 查到对应切片；当场景加入自由物体以后，不能继续假设所有数组都按相同维度排列。

ctrl 的物理含义由模型决定：位置伺服器接收位置目标，力矩执行器则可能接收力矩刻度，带传动或激活动力学的执行器还会有另外的映射。先读它怎样产生力，再决定动作如何写入。第6.8节将这种差别落实为“末端位移动作→关节角目标→力矩→实际运动”。

6.3.3 为什么设置完状态，还需要刷新派生量

物体的世界坐标位置不是独立存储的一套真实状态，而是根据关节位置计算出来的。直接设置 qpos 后，旧的末端位置、相机位姿和接触结果可能还没有刷新。mj_forward(model,data) 用当前状态重新计算相关量，不推进时间；mj_step 则进一步积分到下一个物理时刻。重置时常用的顺序为：清空上一局状态，设置本局初始位置、速度和控制输入，调用 mj_forward，最后构造初始观测。

只重置位置而保留上一局速度，会让机器人在新回合开头带着旧动量运动。若使用了动作历史、图像缓存或延迟队列，也应在同一回合边界清空。物理状态重置正确，不代表策略的记忆已经重置。

记录历史时另有一个容易忽略的关系：许多 MuJoCo 数组直接引用内部内存，之后一步会原地改写它们。下面第一行只是保存了一个持续变化的视图，第二行才保存该时刻的值：

```
history.append(data.qpos)          # 所有条目可能都指向同一块内存。
history.append(data.qpos.copy())   # 复制当前数值，后续步进不会改写它。
```

训练数据中的历史必须是过去实际发生的状态。否则，即使文件里有很多行，它们也可能都变成最后一个时刻的值。Python绑定对内存与复制的说明

6.3.4 接触需要看力和运动，不能只看图像

视觉网格决定画面，碰撞几何决定哪些表面参与接触计算。细致外观可以搭配较简洁的碰撞形状，以降低计算量；但若碰撞模型把杯柄孔洞填满，抓取空间也会改变。模型能显示出来，只说明渲染链路有了输入，还需核对关节轴、尺寸、质量、惯量以及接触关系。机器人模型搭建教程

物体接触桌面时，引擎根据当前运动与约束求解接触力；夹爪闭合时，手指与物体之间的法向力和摩擦共同影响能否举起。data.contact 中可以查看接触对象、距离和摩擦参数，mj_contactForce 则读取求解后的接触力。几何体靠近到生成接触记录，不等于这个接触已经承担了足够的重量。

实验中若把圆柱摩擦从1改成0.02，还要检查接触对最终用了哪个值。MuJoCo在同优先级几何体之间，摩擦参数通常逐项取较大值；若手指仍为1，只改物体的0.02，可能根本没有降低两者接触的摩擦。本项目使物体具有较高的接触参数优先级，实际接触记录也核对到了0.02。这是设置与物理作用之间需要检查的环节。接触参数组合规则

6.3.5 相机的三种输出分别告诉我们什么

同一相机可以生成 RGB 图、深度图和分割标识图。RGB近似相机看到的颜色；深度图给出可见表面沿相机前向的深度，单位为米；分割图给出被渲染对象的标识，便于制作标签与排查遮挡。分割ID并不是普通真实RGB相机直接测得的量。

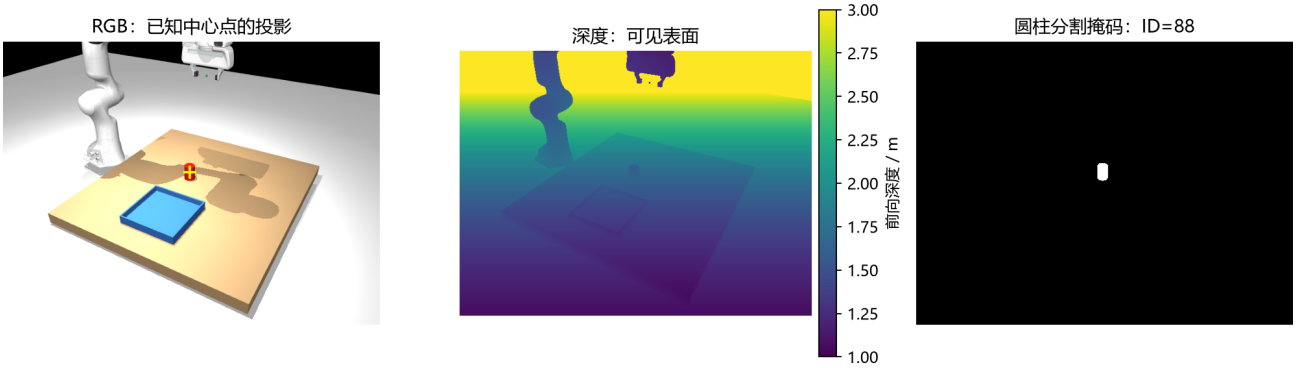


图6.3-1：同一仿真时刻、同一相机。中图按标尺把米制深度映成颜色，右图仅显示圆柱ID对应的掩码。学习包保留原始深度与分割数组，分割配色不代表真实材质。

运行 practice/dump_sensors.py 会在 results/sensors 保存三个数组，并把一个已知世界点投到图像中检验。MuJoCo相机的局部 x 向右、y 向上、负 z 朝前，图像像素的纵坐标却向下。配套 project_point 先将世界点变换到相机坐标，再按这个轴向计算像素，避免相机朝向或纵轴符号写反。

圆柱中心在图像上约为 $(u, v) = (316.32, 220.28)$，该像素的几何体ID为88，与圆柱ID一致。中心的前向深度约1.6355 m，而此像素的渲染深度约1.6063 m。这不矛盾：中心在物体内部，相机先看到的是更近的表面。标定核对应比较相应几何对象及可见表面，不能把中心深度直接当作表面深度标签。

这些输出可用来训练视觉感知、生成仿真监督和检查坐标。在部署设计中还应说明，策略读取的是颜色、实际深度传感器输出，还是仿真标识。6.8节先用状态观测验证控制与物理，图像按单独时间戳采集；这样可以逐项接入感知误差。

6.3.6 练习题

题6.3-1 七个旋转关节、两个手指滑动关节及一个自由刚体，为什么对应 nq=16,nv=15,nu=8？四元数为何有四项而旋转只有三个自由度？同样要让第一个关节到0.5 rad，改 qpos 与写位置执行器的 ctrl 有何区别？

题6.3-2 物体中心前向深度为1.6355 m，其投影像素的表面深度为1.6063 m，是否说明标定错误？怎样用已知世界点和几何体ID检验映射？分割ID图可以如何用于训练，其中哪些信息普通真实RGB相机无法直接提供？

6.3.7 参考答案

题6.3-1答案 七轴机械臂与两个手指各占一个位置和速度坐标，合计9项。自由物体的位置用三维平移和四元数表示，增加7项；速度用三维线速度和三维角速度表示，增加6项，所以 nq=9+7=16、nv=9+6=15。

四元数 (w, x, y, z) 满足 $w^2 + x^2 + y^2 + z^2 = 1$，四个分量只剩三个局部独立变量；正负相反的两个单位四元数还表示同一个旋转。角速度表示旋转的瞬时变化，只有三项，不能把四个四元数分量逐项当成四个独立转轴。

模型保留了两根手指的坐标，并通过关节相等约束使其同步；一个执行器作用在由两指关节组成的耦合坐标上。因此有两个手指关节，却只有一个夹爪控制通道，加七轴执行器得到 nu=8。相等约束产生约束力，不会让存储数组自动删掉一个关节。

写入 ctrl=0.5 是请求位置执行器跟踪0.5 rad，有限力矩随后带动运动。直接把相应 qpos 改成0.5，则立即把构型设置到了那里，没有经历执行器、惯性和碰撞过程；调用 mj_forward 只刷新该构型的派生量。前者用于本例的物理控制，后者适合初始化、构造测试状态或运动学副本。

题6.3-2答案 不一定有标定错误：中心在实体内部，渲染器返回的却是该视线首先遇到的可见表面，所以表面深度更小。两个数都是沿相机前向轴的深度，也不能与相机到该点的欧氏距离混用。

核对投影时，设相机世界位置为 c_W，相机到世界的旋转为 R_{WC}，已知世界点为 p_W。先求：

$$p_C = R_{WC}^{\top}(p_W - c_W) = (x_C, y_C, z_C), \qquad d = -z_C > 0.$$

相机右方为正 x_C、上方为正 y_C，图像向下为正 v，故针孔投影为：

$$u = c_x + f\frac{x_C}{d}, \qquad v = c_y - f\frac{y_C}{d}.$$

本例 $f = 480/[2\tan(45^\circ/2)]$，主点为 $(319.5, 239.5)$。中心投影约 $(316.32, 220.28)$，读取数组时取 [v,u]，并同时核对分割的对象类型与ID：这里是几何体、ID 88。若该点被其他物体遮挡，像素也可能属于遮挡物；要进一步检查深度，应选已知可见表面点，而不是要求中心深度等于表面深度。

分割ID可作为感知监督、调试标签或明确声明的特权输入。普通RGB相机不直接提供精确的实例身份与边界；若策略依赖仿真ID及其配色，部署就需要感知模块生成相应表示，或重新定义输入。分割本身也不自动给出质量、摩擦或完整遮挡形状。

6.4 环境接口与回合边界：一条转移到底从哪里到哪里

6.4.1 用统一接口连接决策与物理

Gymnasium规定环境与算法如何交换数据。reset 返回新回合初始观测和附加信息，step(action) 返回这次动作之后的观测、奖励、terminated、truncated 和附加信息。物理引擎负责推进状态，Gymnasium接口负责把这些状态组织成一个任务。

单关节例子的动作为目标角，观测为角度、角速度和目标角；机械臂例子的动作为三维末端增量加夹爪命令，观测为37维状态。这两种任务可以使用同一套训练程序的调用形式，但动作含义仍由各自环境定义。action_space 与 observation_space 告诉程序数组的形状、类型和范围，不能代替动作的物理解释。

环境检查器能发现返回类型、种子行为和接口错误。本项目另外核对了回合内物理时间、成功/超时、参数重置和历史观测独立保存，因为接口符合格式还不能说明任务判断正确。Gymnasium环境接口、自定义环境教程

6.4.2 奖励、任务终止与采样截断是三个判断

奖励是学习信号；成功条件是我们究竟要完成什么的判据。若只奖励末端接近托盘，机器人可能夹着物体停在托盘上方。距离变小可以作为过程反馈，最终成功还应检验物体是否释放、得到支撑并符合任务要求。

任务终止 terminated 表示到达所定义任务的结束状态。本项目完成抓放，或者物体掉出桌面并满足失败定义，都会终止。**采样截断** truncated 表示采集被外部条件切断，例如程序最多记录800个控制步；动作任务理论上可能还没结束，只是这次不再采下去。

这个差别直接影响价值标签。采用一步 **TD**（Temporal Difference，时序差分）时，若时间上限只是外部采样限制，标签写成：

$$y_k = r_k + \gamma(1 - \text{terminated}_k)V(o_{k+1}). \tag{6.4-1}$$

真正终止以后没有任务后续回报；外部截断只是后半段没有被采到，可以用最后状态的价值补上。这里的 o_{k+1} 必须来自这次动作后的同一回合。若任务本身定义为“剩余十秒内完成”，到时就属于该有限时域任务结束；还应把剩余时间纳入状态/观测，使价值能区分同一位置下不同的剩余机会。时间限制与自举

6.4.3 多份环境为什么让回合边界更容易弄错

批量采样时，同一策略调用产生多份动作，但各环境的回合长度不同。A可能已经成功，B却还在搬运。系统要重置A，不能连B一起清空；同时，上一回合最后一次动作的后继观测，还要保留给训练程序。

配套 vector_boundary.py 把复杂物理暂时省去，仅保留回合边界，实际运行 Gymnasium 1.4.0 的两份环境。观测为 [环境ID,回合ID,回合内步数]。A在第2步真正终止，B在第3步因外部采样上限截断。初始观测分别为 [0,0,0]、[1,0,0]。这三个数字使“同一回合的下一步”和“下一回合的起点”可以直接辨认。

图6.4-1：每个格子显示一次批量调用返回的回合编号与步数。A比B先结束，重置只影响A。SAME_STEP返回的新回合观测，与该动作真正对应的最终观测，用不同标记区分。

6.4.4 SAME_STEP：策略的下一输入与转移的后继观测不同

在**同次重置** SAME_STEP 模式中，某份环境一结束，就在本次 step 内重置。返回数组便可以立刻用于下一次策略调用；然而，这次动作真正达到的最终观测，保存在 infos["final_obs"] 中。下表摘自程序的三次批量调用：

批量调用	A返回值	A标记	A真正的转移后继	B返回值	B标记	B真正的转移后继
1	[0,0,1]	未结束	[0,0,1]	[1,0,1]	未结束	[1,0,1]
2	[0,1,0]	终止	[0,0,2]	[1,0,2]	未结束	[1,0,2]
3	[0,1,1]	未结束	[0,1,1]	[1,1,0]	截断	[1,0,3]

第2次调用的A同时涉及两段数据：奖励与终止标记属于回合0最后一步；返回值 [0,1,0] 则是回合1的起点。如果把它直接写成上一动作的后继，就声称“上一动作把环境送进了新回合”。动力学预测会收到错误标签。A若保留了正确的终止掩码，一步TD的自举项已经为零，数值目标可能暂时不变；B这类仍要自举的截断样本则会直接受到影响。

本模式的数据整理可以按下面顺序实现。代码同时保留 returned_obs 和 transition_next，因为它们分别服务于下一次行动和本次训练样本：

```
returned_obs, reward, term, trunc, infos = envs.step(actions)
ended = term | trunc

# 1. 未结束的环境，返回值就是这次动作的后继。
transition_next = returned_obs.copy()

# 2. 结束的环境，从final_obs取重置前的最终状态。
for i in np.flatnonzero(ended):
    assert infos["_final_obs"][i]
    transition_next[i] = infos["final_obs"][i]

# 3. 写训练转移时用transition_next；下一次行动用returned_obs。
store(observations, actions, reward, transition_next, term, trunc)
observations = returned_obs
```

_final_obs 是逐环境的字段有效标志。以上字段与时机对应所选版本和模式；配套程序会直接输出实际结果，而不是要求读者根据一个泛泛的“自动重置”名称推断。Farama对自动重置模式的讲解

6.4.5 NEXT_STEP：有些返回只是重置，没有执行动作

在**下次重置** NEXT_STEP 模式中，第2次调用的A先返回真实末状态 [0,0,2]。第3次调用再对A重置，返回 [0,1,0]、奖励0，这一槽位没有执行提交给A的动作；与此同时，B仍然正常走了第3步。

因此第3次批量调用不能把两份结果都作为普通转移写入缓存。使用“上次哪份环境结束了”的掩码，可识别本次哪些槽位只做了重置：

```
# previous_ended在本次调用之前保存上一批的回合结束标记。
returned_obs, reward, term, trunc, infos = envs.step(actions)
valid_transition = ~previous_ended
store_only(valid_transition, observations, actions, reward, returned_obs, term, trunc)

# 所有返回值都作为下一次策略输入；只跳过重置槽位的数据写入。
observations = returned_obs
previous_ended = term | trunc
```

第3次的有效掩码为 [False,True]。这里的有效性与TD自举是两个判断：

当前槽位发生什么	是否写入学习样本	是否从真实后继价值自举
执行动作，尚未结束	是	是
本次动作真正终止	是，末动作仍是有效转移	否
本次动作因外部上限截断	是	是，用重置前未观测
NEXT_STEP中本次只reset	否，没有执行提交的动作	不为该槽位构造TD目标

有效掩码问“本次是否发生了动作转移”，所以使用上次结束标记；自举掩码问“这条已有转移后面是否还有任务回报”，所以使用本次终止标记。若仍给只重置的A存一条“末状态→新起点、奖励0”的动作转移，就虚构了一次环境运动。RNN（Recurrent Neural Network，循环神经网络）的隐藏状态、图像历史和动作队列也应在各自的新回合开始时重置，不能用一份全局结束标记处理整批。

第三种 DISABLED 模式不自动重置。程序先保存结束转移，再用 reset_mask=term|trunc 显式重置相应环境。它使边界更直观，但需要采样程序自己维护掩码。三种模式都能正确使用，关键在于按实际时机组织数据。SyncVectorEnv 在这里逐份执行环境，主要便于观察语义；这段实验没有测量并行吞吐。SyncVectorEnv接口

批量推理可共享网络计算，物理是否并行则取决于后端。扩大环境数量后，物理、渲染、处理器和内存争用都可能成为瓶颈；无图像的状态策略与视觉策略负载也不同，吞吐需要按实际输入和完整流程测量。

6.4.6 一个错接回合会如何改变价值标签

仍看B的第3步，奖励为1、$\gamma = 0.9$。为便于核对，假设价值估计是 $V([i, e, t]) = 10e + t$，则同回合末状态 [1,0,3] 的价值为3，新回合起点 [1,1,0] 的价值为10。这个价值函数仅为边界算例指定，不是训练结果。

由于B只是外部截断，正确标签为 $1 + 0.9 \times 3 = 3.7$。错误使用返回的新回合起点，标签就变成 $1 + 0.9 \times 10 = 10$；若把所有 done=terminated|truncated 都当作不自举的终止，标签又变成1。一个错误从另一个回合引入回报，另一个错误丢掉尚未采到的未来，两个方向都改变了学习问题。

A在第2步真正终止，同样奖励1，正确标签就是1。即使存储的末状态正确，也不能因为它的价值网络输出非零而继续自举。准确的后继观测与准确的结束语义，缺一项都不能得到正确标签。

6.4.7 练习题

题6.4-1 采用SAME_STEP模式，A终止时返回 [0,1,0]、final_obs=[0,0,2]；B外部截断时返回 [1,1,0]、final_obs=[1,0,3]。当次奖励均为1，$\gamma = 0.9$，$V([i, e, t]) = 10e + t$。分别计算正确一步TD目标，指出直接用返回观测、把两类结束都当终止时各会错在哪里。若到时结束就是任务定义，还需要改变什么？

题6.4-2 A在第2次调用结束，NEXT_STEP模式第3次对A只做reset，返回 [0,1,0]、奖励0，B仍执行真实动作。为什么不能把两份数据都存为正常转移？怎样构造有效掩码，并维护下一次策略输入？

6.4.8 参考答案

题6.4-1答案 SAME_STEP的返回值用于下一次策略行动，本次转移的后继要取 final_obs。A真正终止，正确目标为 $1 + 0.9 \times 0 \times 2 = 1$；B只是外部截断，正确目标为 $1 + 0.9 \times 3 = 3.7$。

若把返回的新回合起点当作后继，其价值为10。B的标签便从3.7变成 $1 + 0.9 \times 10 = 10$，把另一回合的价值接入旧回合。A若仍正确保留终止掩码，标签数值依然是1，因为自举项乘零；但后继状态仍然存错，会影响动力学预测等使用完整转移的学习。A也得到10，意味着还额外漏掉了终止掩码。

若把 terminated|truncated 都当作不自举的结束，A仍为1，B却从3.7变成1，丢掉未采到的继续价值。因此，自举使用“是否真正终止”，回合拼接和跨步递推则在“终止或截断”处切断，不能把下一回合接上去。

若截止时间本来就是任务定义，截止转移应按该有限时域任务结束处理，不再自举；还需将剩余时间纳入状态/观测，或显式使用 $V(s,h)$。否则，相同物理状态在只剩一步和还剩一百步时价值不同，却被错误视为同一决策条件。

题6.4-2答案 第3次调用的A没有执行所提交动作，只重置到新起点；奖励0也不代表发生了一次“末状态到新起点”的任务运动。B却确实执行了动作，所以需要逐环境筛选，不能整批丢弃或整批保留。

初始化 previous_ended=np.zeros(2,dtype=bool)。每次调用之前，用 valid=~previous_ended 标记将执行真实动作的环境；调用之后仅为有效槽位写转移，再令 observations=returned_obs.copy()，将所有返回值作为下一次行动输入，最后更新 previous_ended=terminated|truncated。第3次的有效掩码是 [False,True]。

更新后的A输入为 [0,1,0]，B输入为 [1,0,3]，下一次由B执行重置。策略的记忆与图像历史也按各自回合重置。不要通过奖励是否为零判断有效性，真实动作同样可以得到零奖励。

6.5 Isaac Sim 与 Isaac Lab 的场景和学习任务

6.5.1 Isaac Sim 如何从资产走到机器人场景

Isaac Sim 的场景通常组织在 **USD** stage 中。先建立世界、机器人、地面、光照和相机，再设置物理与执行接口。stage 是场景组织，不等于学习环境。

官方 **URDF** 导入路线将机器人描述转换为 USD。使用对应版本的导入扩展，选择源文件、输出路径和碰撞等设置，导入后首先查看碰撞代理、关节轴、限位和惯性，而不是立即训练。Isaac Sim URDF 教程。

一个可检验流程是：确认机器人比例与固定基座；单独驱动一个关节；检查目标与反馈；配置相机并验证投影；加入物体与接触；保存资产和配置。这样每一步有明确结果。

程序启动、图形用户界面（Graphical User Interface，GUI）和 ROS 节点导入有不同前提。官方页面中 **机器人操作系统 2**（Robot Operating System 2，ROS 2）的节点导入部分注明 Linux 条件，不能据此推断所有导入方式都相同。扩展名和 **应用程序编程接口**（Application Programming Interface，API） 也会随版本改变，代码应以所用版本文档为准。

6.5.2 从 Isaac Lab 教程读懂一次学习步

Isaac Lab 的直接式任务把环境逻辑写在一个任务类中。以官方 Cartpole 教程为入口，可以先运行其任务，再沿下面五个函数追踪数据：

函数	何时调用	在任务中做什么
_pre_physics_step(actions)	每个策略步一次	缩放并保存策略动作

函数	何时调用	在任务中做什么
_apply_action()	每个物理子步之前	把保存的动作提交给执行器
_get_observations()	需要策略输入时	组装状态或图像观测
_get_rewards() / _get_dones()	物理推进后	计算反馈与结束原因
_reset_idx(env_ids)	指定环境结束后	只恢复这些环境的状态

decimation 表示一次策略步包含多少个物理子步。官方例子的动作先经过尺度变换，再作为关节力目标提交。它与位置目标任务的区别在执行接口，函数名称本身并未规定动作单位。Isaac Lab 直接式任务教程

```
收到一批环境的策略动作
    反归一化或缩放，并缓存动作
    重复 decimation 次：
        将缓存动作提交给各环境的执行器
        推进一步物理并更新相关状态
    根据推进后的状态计算奖励和结束标志
    若需截断前的最终观测或价值，在重置前由任务/采集适配层保存
    对结束的环境调用 _reset_idx
    计算返回观测：未结束实例继续旧回合，已重置实例开始新回合
    返回观测、奖励、terminated、truncated和附加信息
```

先用一个环境检查这些函数，再扩为批量环境。比如只让第 3 个环境掉落，检查第 3 个环境的结束标志、最终观测和重置状态，其他环境应继续自己的轨迹。这样的检查比观察一条平均奖励曲线更容易发现回合串接。

管理式任务将上述职责拆成动作、观测、奖励、结束和事件管理器，适合复用组件。直接式与管理式的组织方法不同，任务定义仍需一致：同一种动作、相同时间尺度、相同成功条件，才适合比较训练结果。

这里需留意自动重置。以Isaac Lab v2.2.0直接式环境为例，step先算奖励和结束标志，再重置结束实例，最后生成返回观测。因此结束标志属于刚结束的回合，相应观测却可能已是下一回合起点。真正终止时不再自举；外部截断若仍需估计剩余价值，应在重置前保存对应观测或价值，或使用明确提供这项信息的适配器，不能拿重置后的观测代替。普通step返回值不自动保证含有全部重置前信息。

6.5.3 Isaac 工作流从场景验证到学习任务

第一步是固定兼容的软件组合、驱动和资产。创建场景后，导入机器人，核对关节、坐标、碰撞和执行器，施加简单位置目标，检查状态读回。第二步加入物体和传感器，验证相机坐标、分辨率、更新周期和深度语义。第三步定义任务观测、动作映射、奖励和结束条件；第四步扩展为向量化环境；第五步接入学习并独立评价。不要把并行环境数量增大当成跳过前几步的理由。

基于管理器的任务将观测、动作、奖励、结束、命令和事件等职责放进独立配置；直接任务把更多逻辑写进环境自身。前者便于组合与复用，后者适合特殊计算流程。无论使用哪种方式，都需要同一份任务定义。配置层并不改变奖励的物理含义。

例如抓杯观察组包括关节位置、速度、夹爪状态、目标位置及图像；动作组定义末端增量如何交给控制器；事件组定义重置、质量和摩擦变化；奖励组定义靠近、抓住、放稳和超力；结束组区分成功、物体掉落与步数限制。特权物体位置可以供评价器或奖励使用，但若策略部署只能读相机，就不能不加说明地放进策略观测组。

6.5.4 并行带来的有效吞吐条件

许多环境共享场景模板，但各自需要独立状态、目标、随机种子和结束处理。图像传感、接触求解和数据拷贝可能成为瓶颈。增大环境数只有在有效轨迹产出、任务定义和数值行为一致时才是优化；若渲染降频改变观测，或环境间对象发生不应有的碰撞，已经改变了任务。

创建任务时，先用单环境和固定条件检查观测、动作、反馈与重置，再启动并行采样与学习。官方教程目录。

6.5.5 后端选择与版本兼容

更换物理后端会改变场景加载、执行器、接触求解和可用传感器接口。传统 Isaac Sim 场景工作流与其他后端的启动路径应分别理解：有的示例先启动完整模拟应用，有的采用后端专用初始化。相同任务配置能否复用，取决于该版本提供的适配，不能把旧脚本与新后端参数直接拼接。官方 Quickstart 与后端说明提供各自入口，实践中先选定一条完整、兼容的路径。Isaac Lab Quickstart。

固定版本后，先运行随该版本提供的小任务，检查已注册任务和后端能力；再修改资产、观测和动作，并对照同一版本的文档验证。若迁移后行为变化，分别检查启动与加载、动作映射、时钟和接触响应，避免把接口变化全部解释成策略泛化失败。

6.5.6 练习题

题6.5-1 并行仿真环境里的某些实例结束后，仅重置这些实例。为何需要同步处理动作历史、计数器与观测缓冲区？

6.5.7 参考答案

题6.5-1答案 重置的是一整个环境实例的回合状态，不能只把机器人位置放回去。例如环境 3 刚失败，若动作历史仍保留最后的猛拉命令，新回合的观测会告诉策略“刚才正在拉门”，动作平滑代价还可能把第一个正常动作判成剧烈变化。若计数器不清零，新回合可能立刻超时；若图像堆叠不更新，观测里会混入上一回合的杯子位置。

应先保存重置前的最终观测、奖励和结束标志，再仅对结束实例重设物理状态，以及动作历史、任务进度、计数器、延迟队列和观测历史。新的第一份观测要从重置后的实际状态生成。循环策略自己的隐状态、未执行完的动作块也属于回合状态，需要同步处理。

Isaac Lab 的管理器式环境将这些职责分给场景、动作、观测、奖励和终止等管理器，并向它们传入同一组 env_ids。自定义缓冲区也要使用这组索引；对全批缓冲区清零会扰乱尚未结束的其他实例。原则是每一条转移都属于确定的回合，新回合的信息不能残留旧回合，也不能修改其他环境正在执行的回合。

6.6 操作与导航框架、Gazebo 和 ROS 2 集成

6.6.1 ManiSkill、robosuite 与 Habitat 怎样选择

ManiSkill 提供操作任务与环境工具。可以先创建已有任务，读取 observation/action space，重置、推进、查看结果，再开发自己的任务。一个按官方任务接口组织的示意为：

算法步骤：

1. 创建包含任务观测和动作接口的环境。
2. 重置为初始状态。
3. 执行一次动作，读取转移结果。
4. 释放环境资源。

```
import gymnasium as gym
import mani_skill.envs
# 步骤1：创建包含任务观测和动作接口的环境
env = gym.make("PickCube-v1", obs_mode="state")
# 步骤2：重置为初始状态
obs, info = env.reset(seed=0)
# 步骤3：执行一次动作，读取转移结果
result = env.step(env.action_space.sample())
# 步骤4：释放环境资源
env.close()
```

动作维度随控制模式变化：末端增量、关节目标与夹爪命令会组成不同向量。创建任务后，应按其动作空间连接策略输出。ManiSkill Quickstart。

自定义任务需要设置资产、初始化、观测、评价和奖励，并按框架要求注册。先用构造状态检查成功条件，再让学习算法访问环境：夹爪闭合却没有抓住物体，应返回抓取失败；物体已放下但仍在滑动，应继续检查稳定性。这样的规则测试能发现训练奖励可能利用的漏洞。ManiSkill 自定义任务。

robosuite 基于 MuJoCo 组织机器人操作任务与控制工具；Habitat 面向具身任务与场景学习生态，适合相应导航与移动操作问题。它们的数据、资产和控制假设不同，需按研究问题选择。robosuite、Habitat-Lab。

软件丰富不意味着必须全部安装。先用目标 benchmark 对应平台建立可信基线，确有跨平台验证或新功能需求时再增加。

6.6.2 Gazebo 与 ROS 2 侧重系统怎样连起来

Gazebo 常用于机器人模型、传感器和软件联调；**ROS 2** 组织节点、消息、坐标与控制接口。两者之间的桥接负责传递数据，并不保证坐标、单位与时间一致。Gazebo 入门。

学习路线可以是：启动已有世界，确认物理推进；查看机器人模型；连接关节控制；读取相机与里程计；桥接到 ROS 2；运行同一感知或规划节点；核对时间和坐标；再替换为自己的机器人。

模型和世界常使用 SDF（Simulation Description Format，仿真描述格式）。从 **URDF** 转换后，需要检查传感器、插件、碰撞与关节控制，不能只看外观。

采用模拟时间时，节点应按约定使用该时间源，不能混用墙钟给数据打时间戳。相同消息类型也需要核对字段语义，真实驱动与仿真桥的延迟可能不同。

这种路线适合验证系统接口和整机流程，不意味着已解决策略训练所需的数据分布与物理迁移。你可以用同一软件节点提高一致性，但仍要测真实响应。

6.6.3 按任务生态选择框架

操作基准框架常提供机器人、物体、任务重置、奖励和示范接口，导航框架重视地图、场景和路径任务。选择时检查任务是否适配自己的机器人接口，资产能否修改，物理与渲染是否满足需求，评价协议是否可导出。相同“抓取”名称可能对应不同初始分布、动作语义和成功规则，不能未经核对横向比较。

6.6.4 ROS 2 的消息系统怎样连接仿真与机器人

ROS 2 通过节点组织软件。相机节点发布图像，状态节点发布关节测量，策略节点订阅这些输入并发布命令，执行节点负责控制。话题适合连续数据，服务适合短请求响应，动作接口适合具有过程反馈与取消的长任务。节点不是神经网络里的一个神经元，而是独立运行的软件组件。

消息应带有采样时间和坐标系标识。变换树记录 base、camera、tool、map 等坐标系之间的关系，查询时需要对应测量时刻；把最新变换应用到几百毫秒前的图像，在移动平台上会产生系统偏差。官方 TF 与里程计教程可用于进一步实践。

服务质量（Quality of Service，QoS）配置包括可靠性、队列深度和持久性等。可靠传输对关键状态有价值，但摄像头堆积大量旧帧可能降低决策时效；最佳努力允许丢部分消息，却要接受其条件。发布和订阅配置不兼容可能导致无数据，不能只检查节点是否存在。相关配置可查 QoS 教程。

6.6.5 仿真加速后，消息时间为什么也必须一起改变

假设物理引擎只花 0.2 秒墙钟时间，就推进了 1 秒模拟时间。传感器在模拟时刻 $t = 1$ 拍了一帧，机器人在该时刻的位置变换也记录为 $t = 1$。感知节点应按这个模拟时刻配对，而不是把图像标为墙钟 0.2 秒，再去查询 0.2 秒的机器人姿态。

ROS 2 的**模拟时间**通过 /clock 传播；启用 use_sim_time 的节点用这一时间源解释消息和定时事件。于是加速、减速和暂停模拟时，各节点可以共同跟随模拟进度。硬件超时则常需单调运行的时钟：模拟暂停了，真实电机通信仍可能需要超时处理。ROS 2 的时钟设计教程

日志回放还可能从 $t = 10$ 跳回 $t = 0$。按时间维护的缓存需要处理这种跳变：旧图像、旧变换、历史速度估计和未完成动作块若被保留，会把上一次回放与这一次连接起来。回放重置因此也是时间状态的重置。

验证完整链路时，先录下图像、关节状态、坐标变换和命令；关闭执行器回放，检查同一测量时刻的数据能否正确配对；再在仿真中加入延迟和丢帧，观察策略。软件回放验证输入如何被解释，实际执行检验命令如何造成运动，两类实验分别定位问题。

6.6.6 练习题

题6.6-1 ROS 2 传感、仿真物理和策略各有时钟。为什么“消息成功送达”不等于“观测与动作对齐”？怎样记录可追溯的转移?

6.6.7 参考答案

题6.6-1答案 消息送达只证明数据到了接收端，并不证明它属于策略正在决策的时刻。一张相机图像可能在仿真时刻 1.00 s 采集，到 1.08 s 才送达；策略在 1.08 s 算动作，执行器在 1.11 s 才应用命令。消息若只保留接收时间，就会把旧图像误当成当前状态。仿真还可快于、慢于或暂停于墙钟时间，不同时钟的数值不能直接比较。

可追溯的一条转移应记录：观测的采集时刻和接收时刻、决策使用了哪些观测、推理开始和结束时刻、原始动作、缩放后的命令、实际开始执行和保持结束的时刻，以及后续观测与奖励对应的区间。还要记录时钟来源、环境与回合编号、命令序号和重置事件。ROS 2 仿真节点通常使用仿真时钟统一语义，硬件耗时可另用单调时钟统计。

随后按“决策时已经可用的历史”构造输入，并按实际执行区间配对动作和后果。存在延迟时，转移不必硬凑成理想同步的三元组；可以记录动作队列或把延迟纳入状态。时间跳回和重置时应切断旧序列、清理缓存。这样才能判断失败来自策略选择，还是选择基于过时观测。

6.7 随机化、辨识与迁移：先确定差异，再选择机制

6.7.1 先让同一个动作在两边具有相同含义

Sim2Sim（Simulation-to-Simulation，仿真之间的迁移验证）将控制器或策略移到另一套仿真模型、版本或引擎，观察行为能否保持。它最先帮助我们检查接口：关节顺序、角度单位、末端坐标系、夹爪开度、动作保持时间和观测归一化是否一致。同一个四维数组，如果一边表示末端增量，另一边表示绝对位置，就没有在比较同一动作。

Sim2Real（Simulation-to-Real，仿真到真实系统的迁移）还涉及真实传感、驱动与接触。颜色、曝光、深度空洞和遮挡改变模型输入；增益、死区、饱和与延迟改变命令执行；质量、摩擦、柔性改变物理响应；对象和初始位置又改变任务分布。解决方法应针对已经观察到的差异。

比如真实夹爪接口使用总开度0.08 m，仿真执行器使用0—255刻度，首先需要正确转换。只有把两边都转换为“张开到同一总开度”，才适合比较张开速度与接触力。单位或坐标错误应修在接口层；把这些错误随机化，会让学习器被迫兼顾错误的控制含义。

长任务成功率下降只能说明整体出了问题。先用同一初始条件和短输入段，对照角度、末端轨迹、夹爪开度与接触力，往往更容易定位。不同引擎中的接触参数并不必然具有相同数值含义，应匹配可观察响应，再讨论任务性能。

6.7.2 用抓放项目完成一次单因素敏感性实验

第6.8节提供了一套已经能抓放的阶段控制器。现在固定它的阶段、目标和时长，在相同种子下只改一个因素。practice/compare_domains.py 对种子0、1、2逐项运行以下设置：

设置	改变的模型量	成功回合 / 总回合	物体最高中心高度范围 / m
名义条件	质量0.10 kg、滑动摩擦1、原伺服参数、零延迟	3/3	0.2329—0.2342
较重物体	圆柱质量与惯量一起乘4	0/3	0.0382—0.0751
低摩擦	圆柱接触的滑动摩擦降为0.02	0/3	0.0352—0.0438
较软伺服	七轴位置与阻尼增益一起乘0.25	3/3	0.2208—0.2253
执行延迟	执行器命令延迟十个控制步，即0.20 s	3/3	0.2305—0.2356

这些数来自同一 MuJoCo 场景的实际运行，原始轨迹在 results/domain_comparison。参数刻意拉开，以便看清作用；它们不是对某台真实 Panda 测得的不确定范围。这里把同引擎内的参数变化称为敏感性实验，第二引擎验证还需要另外执行。

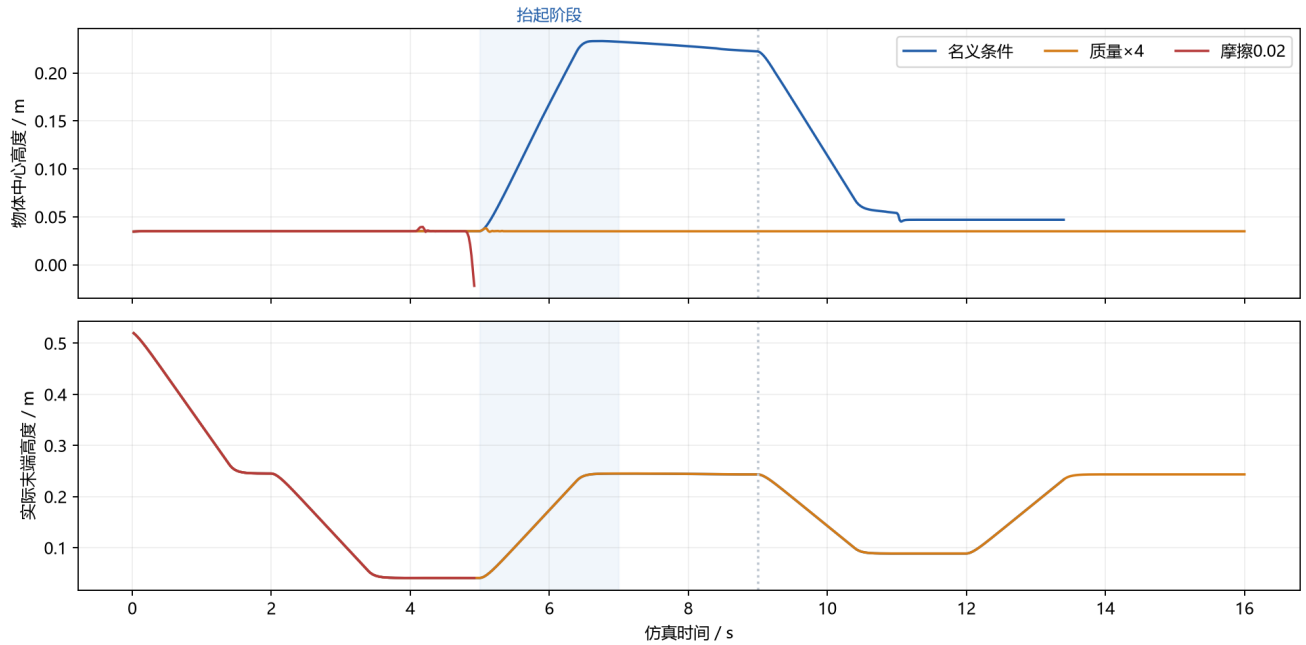


图6.7-1：同一控制器、同一种子下，名义、重物与低摩擦条件的实际高度曲线。低摩擦红线在掉落失败处截止；末端运动与物体运动分别记录，控制阶段走完并不表示物体跟着完成了动作。

较重物体与低摩擦条件在夹紧或抬起时就未能形成可持续抓持。低摩擦种子0在闭爪期间被挤离原处，滑出台面后坠落，回合提前以失败结束；较重物体则没有随夹爪稳定举高。高度、平面位置、双指接触与夹爪开度共同给出了发生阶段的证据。这里的低摩擦同时影响圆柱与手指、桌面的接触，不能把整个失败唯一归为指垫摩擦。

伺服变软与延迟在这三个回合中仍成功，也不表示它们没有影响：软伺服的物体最高点降低，延迟还改变了请求命令与实际命令的对应时刻。控制器的阶段比较慢、保持时间较长，可能为这些扰动留下余量，应从轨迹进一步检验，而不能仅凭成功计数归因。

一次改变一项使比较容易解释。若同时改质量、摩擦与增益后出现倾倒，最多先把失败定位到该组合条件，不能直接断言就是其中某一项造成的。这里每项只有三个回合，用于建立诊断过程，尚不足以估计广泛场景下的可靠成功率。

6.7.3 随机化改变的是训练时会遇到的环境

域随机化（Domain Randomization，DR）让模型在多种环境条件中学习。设 ξ 表示一回合的质量、摩擦、增益和延迟等参数，训练时从 $p_\psi(\xi)$ 抽取它；θ 表示策略网络参数。强化学习中的平均目标可写为：

$$\max_{\theta} \ \mathbb{E}_{\xi\sim p_\psi}\big[J(\pi_\theta;\xi)\big], \qquad J(\pi_\theta;\xi)=\mathbb{E}_{\tau\sim\pi_\theta,\xi}\left[\sum_k \gamma^k r_k\right]. \tag{6.7-1}$$

外层期望问“各种参数各占多少训练机会”，内层期望问“在这一组参数下，当前策略通常得到多少回报”。每轮先抽参数、重置环境、采集交互，再用回报构造策略更新。普通随机化固定抽样规则 p_ψ，更新的是行为规则 θ；改变随机化分布本身则是另一层方法。

这个目标促使有限容量的策略兼顾多种响应。例如同一夹持动作在高摩擦环境中足够，在低摩擦中会滑落；跨环境训练可能促使策略改变抓取位置、减小加速度或根据历史调整动作。究竟学到哪一种，取决于动作接口、可见信息、奖励和训练数据，不能从“加了随机化”直接推出。

平均表现还不是最坏情形保证。若某种困难条件抽到的概率很低，牺牲该条件也可能使平均回报更高。需要控制极端失败时，应明确风险目标或单独验收相关条件。视觉随机化主要改变图像形成过程，动力学随机化主要改变动作后的响应；两类方法分别见视觉域随机化原始研究和机械臂动力学随机化研究。

6.7.4 回合内固定参数与逐步噪声解决不同问题

一个物体的质量通常在一次抓放中固定，未知的是我们还没准确测到它。所以本项目在回合开始抽一次质量，并按相同倍数改变惯量：同样形状和质量分布模式下，密度一起缩放，质量与转动惯量就同比变化。不能只改质量却保留不相符的惯量。

逐步重抽质量会让手里的同一个物体不断变轻变重，改变了学习问题。相反，图像读数噪声可能每次变化，缓慢漂移和突发负载变化也可以有各自的时间模型。应先说清要模拟的是回合间差异、回合内扰动，还是测量误差，再选择采样时机。

程序在每次reset前先恢复名义质量、惯量和增益，再应用本回合的倍率；否则第一个回合乘1.2，第二个又乘1.2，环境就会悄悄漂移。延迟通过先进先出的命令队列实现，而不是推迟读取日志。队列在重置时用初始保持命令填充，避免上一回合命令串进下一回合。

配套示范采集的核心顺序是：

```
为本回合生成独立种子
抽取一组质量、摩擦、增益和延迟，回合内保持
恢复名义模型后应用参数，重置物理状态和历史
运行控制器，保存每一次真实转移与实际命令
保存成功或失败及原因，整条回合归入同一数据子集
```

这里的采集程序不更新任何网络。它把训练环境如何抽样落实成了可检查的数据过程，供后续学习算法使用。

6.7.5 为什么采到随机化示范还不能证明训练有效

collect_dataset.py --episodes-per-split 3 --capture 采集三条名义示范、三条温和随机化示范和三条独立随机化验证轨迹。温和范围为：质量倍率0.8—1.2、圆柱滑动摩擦0.7—1.3、七轴伺服倍率0.8—1.2、延迟0—3个控制步。范围只是教学选取，尚未用真实测量校准。

已有记录中，前两组各3/3成功，独立验证组为1/3。失败轨迹保留在清单中。验证种子1001在抬起和搬运阶段确实举着物体，到了降低阶段发生倾倒，最终没满足直立与低速度条件；把它归为“没抓起来”就会误诊。另一失败也应从自己的轨迹定位。

这组实验说明固定脚本在不同条件下有成功也有失败，并生成了可以检查的观测动作数据。它不能比较“随机化训练”和“不随机化训练”，因为到此没有训练策略。九条记录还不足以支持视觉语言动作模型的泛化结论，数据也没有语言标注。

要验证随机化是否改善学习，可沿用环境再做两个训练组：一组只用名义条件，一组使用指定随机分布；保持网络、输入、动作、优化步骤和总数据/交互预算一致，运行多个训练种子。测试统一使用未参与训练的完整回合，并分别覆盖名义条件、分布内未见参数、合理压力条件。报告所有尝试的成功率、失败阶段和不确定性，失败示范如何参与训练也应预先规定。若随机化组额外收集了更多数据，或测试取自它自己的训练轨迹，就无法把收益归因于随机化机制。

6.7.6 系统辨识怎样帮助选择范围

系统辨识用可观察的输入响应估计未知物理或执行参数。它不要求一次恢复整台机器人全部参数，而是围绕任务最敏感的量设计实验。抓放时，可以先称量物体；在无接触段给关节或末端短目标，比较跟踪滞后和稳态偏差；给夹爪已知开度变化，记录开度与力；通过带时间戳的命令和反馈估计延迟；在已知接触条件下做滑动或抓持实验，约束有效摩擦范围。

同样的轨迹误差可能由不同参数造成。例如增益偏低与负载增大都可能使跟踪落后，只观察一个慢速动作未必能区分。可以改变加速度、方向或负载，比较多个实验的响应；若仍无法区分，就保留一组可信参数，而不是给出没有辨识依据的精确数值。

辨识帮助缩小合理范围，随机化训练让策略适应范围内剩余变化。两者并不冲突。范围过宽会稀释真实关键条件的训练比例，还可能加入难以完成的场景；策略可能因此更保守、样本需求更大。扩大范围应有新的测量或失败证据，而不是默认越宽越好。

6.7.7 运行时适应改变的是当前环境判断

如果策略在动作后观察到滑移或跟踪滞后，它可以利用历史推断“当前环境和先前有什么不同”。**运行时适应**的一种做法是保持行为网络权重固定，更新提供给它的环境上下文；真实数据微调则会改动网络权重，两者改变的对象不同。

以 RMA（Rapid Motor Adaptation，快速运动适应）的思路为例，仿真训练先把环境参数编码为上下文，让策略据此行动；再训练一个适应模块，从观测动作历史预测这个上下文。部署时用历史预测

替代真实参数编码。这样，“仿真知道质量”被转化为“机器人从实际响应判断当前条件”，而不是要求真机直接读取未知参数。

将环境编码器写成 $z = g_\eta(\xi)$、策略写成 $a = \pi_\theta(o, z)$。第一阶段依据回报共同训练 η, θ，使上下文编码保留对行动有用的信息。第二阶段固定二者，训练历史模块 $\hat{z} = f_\omega(h)$，最小化：

$$\mathcal{L}_{\text{adapt}}(\omega) = \mathbb{E}\left[\left\| f_\omega(h) - \text{stopgrad}(g_\eta(\xi)) \right\|^2\right]. \tag{6.7-2}$$

h 是近期观测动作历史；标签来自仿真中已知参数的编码。平方损失只更新 ω，不把标签或基础策略一起拉着改变。原方法还让策略使用当前预测的 $\hat{z}$ 收集第二阶段数据，再用真实编码监督；这样适应模块会见到自己预测不准时引起的偏离，而不只见到教师条件下的平稳运动。部署时权重固定，新历史改变的是 $\hat{z}$。这与用真机损失继续训练策略是两种更新路径，参见原文图2及方法第III节。

这类上下文不必逐项还原质量和摩擦，只需提供有助于行动的信息。但若两种物理条件在已有历史中表现几乎一样，网络也不能凭空区分；需要更多响应、更合适的观测或能区分它们的动作。RMA原工作面向腿足机器人，操作任务应重新检查接触信息、历史长度与验证条件。

6.7.8 怎样把同一项目接到第二引擎或真机

本学习包的实际证据停在同一引擎内的控制与参数实验。向第二引擎或真机推进时，可以分层增加新的证据：

验证层次	固定与施加什么	记录什么	能回答的问题
接口与坐标	同一初始姿态、同一有界命令	关节顺序、单位、实际命令、参考点位姿	两边动作含义是否一致
无接触响应	同一短目标变化与保持时长	位置、速度、力矩/电流、时间戳	增益、饱和、延迟是否显著不同
物体与接触	同一几何、负载、夹爪目标	开度、滑移、物体姿态、接触/力反馈	哪类接触差异影响抓持
感知	固定相机、已知世界点和标定物	投影位置、深度、时间戳、估计误差	输入差异是否来自标定、噪声或时间
完整闭环	固定任务定义，覆盖独立初始条件	所有尝试与失败阶段	整体迁移在目标条件下表现如何

只有目标条件发生在真机上，最后一层才支持真实迁移结论。两种仿真都成功，可以减少对某个引擎特性的依赖，仍可能共同省略柔性、相机延迟等因素；两种仿真不一致，也可能是接口或模型不匹配，不能单独断言真机必然失败。

6.7.9 练习题

题6.7-1 抓放任务的质量、摩擦与延迟随机范围应从什么测量或辨识中得到？回合初抽一次与回合内持续重抽有什么建模区别？为什么更宽未必更好，质量与惯量又应怎样关联处理？

题6.7-2 固定控制器在名义条件成功3/3，在低摩擦和四倍质量下各0/3；又采集了三条名义示范、三条随机化示范及三条独立验证轨迹，前两组全成功，验证组1/3，没有训练网络。能推出什么，不能推出什么？怎样设计公平的固定参数训练与随机化训练对比，明确更新对象、测试集合、失败分母与数据划分？

6.7.10 参考答案

题6.7-1答案 质量可由称重、物体或载荷统计得到；质心与惯量结合几何、密度或动力学辨识估计。摩擦应针对物体—指垫、物体—桌面等接触对，用受控滑移、牵引或抓持实验约束范围。延迟则由命令与反馈时间戳、阶跃响应估计，并分清执行队列延迟、传感延迟和伺服响应慢。范围应包含重复变化和辨识尚不能确定的部分。

回合初采一个固定参数，模拟“这次拿到一件固定但未知的物体”；逐步重抽则模拟时间变化的动力学。同一物体质量不断变化，还可能引入没有物理依据的动量和能量突变，两者不是同一种不确定性。逐步测量噪声、缓慢漂移和突发负载变化可以分别建模，不必全用回合级采样。

固定形状与质量分布模式、仅密度同比改变时，应同时使用 $m' = \alpha m$、$I' = \alpha I$。尺寸或形状改变时，惯量与碰撞几何都要更新；仅内部质量分布改变时，则更新质心与惯量，不必改变外形。不能任意独立抽取不相符的分量。

范围增宽改变训练目标的概率分布。有限数据与模型容量可能花在真实很少出现、甚至无法完成的场景上，关键条件的样本比例反而降低，也可能得到过于保守的策略。先用测量定义可信范围，再根据独立失败逐项调整，比无依据地扩宽更可检验。

题6.7-2答案 现有实验说明这个固定控制器在已测试名义条件中成功，在指定重物和低摩擦条件中失败；独立温和随机化验证还暴露了新的失败。它提供敏感性与数据覆盖证据，不能说明随机化训练提升了网络能力，因为网络尚未更新，也没有两组学习策略可比。3/3与1/3还不足以精确估计总体成功率；同范围内的新组合失败也不能改称为范围外失败。

后续可固定环境、动作、成功规则和学习算法，训练两个相同结构的策略：一组只遇名义参数，一组按规定分布抽参数。两组控制网络容量、初始化规则、交互或示范预算、优化步骤及调参预算，分别运行多个训练种子。强化学习时真正更新的是策略与价值估计器；模仿学习时更新的是学生策略，标签来自教师动作。底层控制接口保持一致。

测试使用两组共用的未见完整回合列表，分别报告名义、分布内新组合与范围外压力条件；所有启动的有效评测回合进入分母，超时、未抓起、倾倒、放偏、未释放均计失败。环境错误按预定规则单列，不能静默删除。若主张真实迁移，还需相应真实硬件任务集合。

数据按完整回合划分，若研究新物体泛化还应按物体身份划分。归一化只用训练数据，验证用于选择方法，最终测试留到选择结束。失败轨迹保留，但不能无条件将其中全部动作视为优质专家标签；若只保留成功示范，两组应采用相同规则，并报告采集尝试数与保留率。

6.8 从单关节到机械臂抓放：搭建可检验的仿真实验

6.8.1 先搭好实验，再替换决策方法

学习策略要回答“此刻该做什么”，仿真环境则要回答“这个动作施加之后发生了什么”。把二者分开，才能判断一次失败来自决策、动作解释、控制跟踪，还是接触与任务规则。如果机械臂连固定动作都执行不稳定，立刻训练一个大模型，往往只会让故障更难定位。

本节先完成一个具体任务：七关节 Panda 机械臂夹起桌面上的红色圆柱，将它直立放进蓝色托盘，松爪并退开。圆柱是红杯任务的简化物体：保留抓持、搬运、释放、支撑和倾倒，暂时省去杯柄、空腔、液体和柜门。这样，第2章可以在同一动作接口上研究模仿学习，第3章可以更换奖励与更新方法，第5章可以学习状态转移；这些方法都需要接受同一个物理结果检验。

决策起点使用**阶段控制器**：按接近、下降、夹紧、抬起、搬运、降低、松爪和退开等阶段给出末端目标。它读取仿真中的物体位置，并使用预设阶段时长，是便于排错的基线。读完本节，应该能够运行这个基线、解释一条完整轨迹、采集动作数据，并改变一个物理因素观察失败。策略网络的训练接在这套环境之后展开。

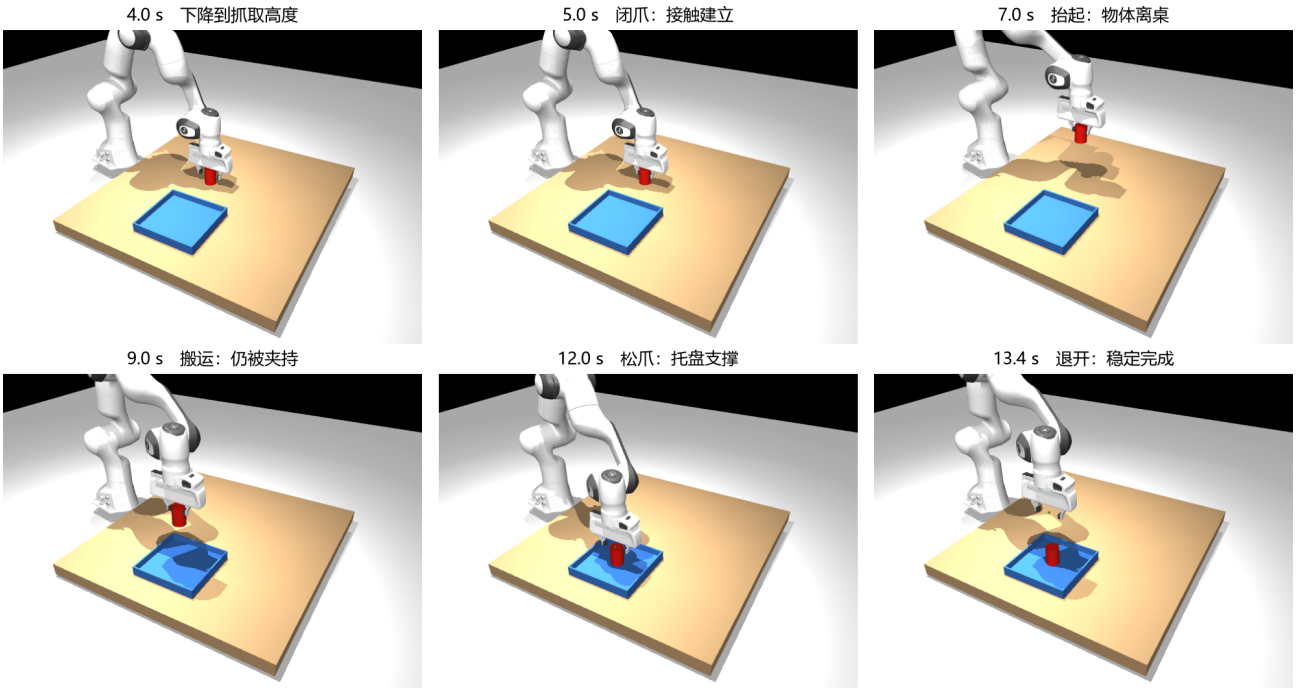


图6.8-1：同一回合的六个时刻，来自配套程序实际渲染。图中的物体靠接触力被夹起；运行过程中没有把物体绑定到夹爪，也没有直接改写它的位置来完成搬运。

6.8.2 场景里的对象、参数和坐标

机器人模型来自 MuJoCo Menagerie 的 Panda 模型。学习包保存了固定版本的模型、网格和原许可；新增的桌面、圆柱、托盘、相机和抓取参考点在 practice/scene.py 中构造。无需先安装一个大型机器人训练平台。

全场景用米、千克、秒和弧度。世界坐标的 z 轴向上，机器人基座位于原点附近，桌面上表面为 $z = 0$。名义圆柱半径0.025 m、高0.07 m、质量0.10 kg；初始中心的平面位置在 $(0.45, 0.05)$ m 附近

小范围变化。托盘中心平面位置为 $(0.5, -0.25)$ m，盘底支撑面为 $z = 0.012$ m。因此，圆柱直立落稳后，中心高度应接近 $0.012 + 0.07/2 = 0.047$ m。后面的成功条件由这段几何关系得到。

Panda 有七个旋转关节，夹爪有两个滑动关节。两根手指通过耦合关系与一个夹爪执行器协调运动；加上七个机械臂执行器，控制数组共有八项。圆柱带自由关节，可以平移和转动，质量、重力与接触决定它的运动。场景加载后得到 nq=16、nv=15、nu=8；三个数的含义与推算见6.3节。

夹爪上的 grasp 是一个**参考点**，用于定义末端位置和计算雅可比矩阵。它本身不施力，也不会吸住物体。托盘与桌面则是真正参与碰撞的几何体。区分“用于计算的位置标记”和“能够产生物理作用的对象”，有助于理解场景文件中每一项的职责。

6.8.3 第一次运行：应该生成什么

配套包以 Python 3.12 为运行环境，核对过的核心依赖是 MuJoCo 3.15.0、Gymnasium 1.4.0、NumPy 2.5.4 和 Pillow 12.3.0。先解压学习包，在包含本文与 practice 的目录中打开终端。Windows 可执行：

```
py -3.12 -m venv .venv
.venv\Scripts\python.exe -m pip install -r practice/requirements.txt
.venv\Scripts\python.exe practice/single_joint/simulate_joint.py
.venv\Scripts\python.exe practice/single_joint/validate_env.py
.venv\Scripts\python.exe practice/run_project.py --seed 0 --capture
```

前两项实验先检查单关节：给定0.5 rad的位置目标，推进约3 s，最终角度接近0.5；环境检查还会验证种子重放、目标跟踪和超时。机械臂命令随后运行抓放任务，--capture 同时保存相机图像。物理计算可在 CPU（Central Processing Unit，中央处理器）上进行；图像采集另外需要可用的 OpenGL 图形环境。本学习包在 Windows 上完成了这两部分运行。其他系统的安装命令见包内 README。

机械臂实验的输出集中在 results/nominal：

文件	用来检查什么
summary.json	本回合参数、结束原因、成功判据及各阶段末状态
trace.csv	每个控制步的物体高度、末端位置、跟踪误差、接触和成功计数
episode.npz	可按数组读取的观测、动作、后继观测、执行器命令、奖励、结束标记和图像
schema.json	各字段的顺序、单位与时间含义
各阶段的 PNG 图像	对照数值查看抓持、倾倒、释放和退开

种子0的一次记录在第670个控制步结束，仿真时间约13.4 s，success=true，物体最高中心高度约0.2333 m。这里报告的是程序实际推进的仿真时间。电脑花多少墙钟时间完成它，会随设备与是否渲染变化。

6.8.4 一个四维动作怎样变成机械臂运动

上层动作写成 $a_k = (a_x, a_y, a_z, a_g)$，四项都限制在 $[-1, 1]$。前三项表示本次要把末端**命令位置**移动多少，比例为每轴最多0.005 m；最后一项控制夹爪，-1 为闭合目标，1 为张开目标。k 表示控制步，命令每0.02 s更新一次。

环境保留上一次命令位置 p_k^{cmd}，然后累积本次位移：

$$p_{k+1}^{\mathrm{cmd}} = \mathrm{clip}_{\mathcal{W}}\left(p_k^{\mathrm{cmd}} + 0.005\, a_{k,xyz}\right). \tag{6.8-1}$$

$\mathcal{W}$ 是允许下发目标的工作区，clip 把超出边界的坐标截回边界。例如 $p_k^{\mathrm{cmd}} = (0.45, 0.05, 0.25)$ m，动作前三项为 $(0.4, -0.2, 0)$，新命令位置为 $(0.452, 0.049, 0.25)$ m。这一步只改变控制器的目标。实际末端位置 p_k^{ee} 要经过逆运动学、执行器和物理积分才会改变。

之所以另外保存命令位置，是因为机械臂可能跟不上命令。若每次都以实际位置为起点再加一个很小的增量，跟踪滞后会不断改变上层命令的解释；这里采用累积目标，所以上层可以稳定地描述一条期望轨迹。两种接口都能设计，但采集数据和部署时必须使用同一种。

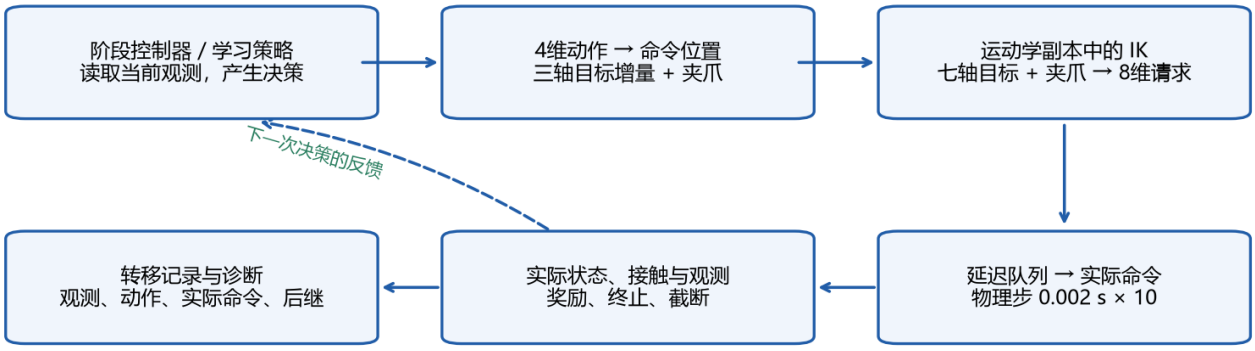


图6.8-2：四维动作先更新命令位置，再转换成八维执行器输入。只有物理引擎推进之后，才得到真正的下一观测。左侧的阶段控制器以后可以换成学习策略。

机械臂关节执行器需要角度目标，不能直接接受三维位置。因此，环境先求一组关节角 q^{ref}，使抓取参考点靠近期望位置，并保持夹爪朝下。这是**逆运动学**（Inverse Kinematics，IK）在本项目中的用途。为免改变真实仿真状态，求解在单独的 kinematics 数据副本中进行。

在一次迭代中，正运动学给出当前参考点位置 $p(q)$，位置误差为 $e_p = p^{\mathrm{cmd}} - p(q)$。姿态也要比较：将当前旋转矩阵的三列记为 r_1, r_2, r_3，目标朝向的三列记为 $r_1^\star, r_2^\star, r_3^\star$，这些列向量分别是末端三根坐标轴在世界系中的方向。本例用：

$$e_R = \frac{1}{2}\sum_{i=1}^{3} r_i \times r_i^\star, \qquad e = \begin{bmatrix} e_p \\ e_R \end{bmatrix}.$$

每个叉积给出把当前轴转向目标轴的局部方向，三项合成姿态修正；在朝向接近时，它近似需要补上的小旋转。将末端雅可比矩阵记为 $J \in \mathbb{R}^{6\times 7}$，上三行描述关节小改变量怎样改变位置，下三行描述怎样改变朝向，便有局部近似 $J\Delta q \approx e$。我们希望减少末端误差，又不希望在某些方向上要求关节突然大幅转动，因而求解：

$$\min_{\Delta q} \|J\Delta q - e\|^2 + \lambda^2 \|\Delta q\|^2.$$

对 Δq 求导并令其为零，得到 $(J^\top J + \lambda^2 I_7)\Delta q = J^\top e$。利用 $(J^\top J + \lambda^2 I_7)J^\top = J^\top (JJ^\top + \lambda^2 I_6)$，可以先解六维线性方程，再左乘 $J^\top$ 得到七维关节改变量。相同的解写为：

$$\Delta q = J^\top \left(JJ^\top + \lambda^2 I\right)^{-1} e. \tag{6.8-2}$$

这就是本例使用的**阻尼最小二乘**。第二项惩罚过大的关节改变量，$\lambda > 0$ 也使求解避免直接逆转奇异矩阵。代码实际解线性方程，没有显式构造逆矩阵；随后限制单次更新幅度与关节范围，再重新计算误差。这里以米和弧度的数值给位置、姿态误差设定相对权重，在朝下姿态附近作局部迭代；$\lambda = 0.03$ 是本项目的数值设置，并非各种机器人通用的物理常数。逆运动学求出了目标附近的一组关节角，还没有完成全臂避障规划。

6.8.5 关节角目标为什么不等于关节实际角度

逆解给出 q^{ref} 后，代码把它写进 data.ctrl 的前七项。此模型的关节执行器根据角度误差和速度产生力矩，可用以下形式理解：

$$\tau_i = K_{p,i}(q_i^{\mathrm{ref}} - q_i) - K_{d,i}\dot{q}_i, \qquad \tau_i \text{受执行器力矩范围限制}. \tag{6.8-3}$$

角度误差越大，比例项越用力地拉向目标；关节运动越快，阻尼项越抑制运动。重力、惯性和接触又共同决定这个力矩最终造成什么运动。因此，即使逆解误差很小，实际末端也可能有稳态偏差、暂态滞后或碰撞阻挡。式中的控制规律由所用 Panda 模型的执行器定义决定，不能把其他模型的任意 ctrl 都理解成角度。执行器的原始定义

配套核对中，目标为 $(0.45, 0.05, 0.25)$ m，运动学副本的残差约 2.0×10^{-6} m；将求得的角度目标交给位置执行器，推进2 s物理后，实际末端距目标约 5.69×10^{-3} m。前者检查“这组关节角在几何模型中是否到位”，后者检查“有重力和执行器的机械臂是否实际到位”。它们不是同一种误差。

这次偏差还可以进一步定位：末态关节速度已经很小，机器人没有发生接触，力矩也未饱和。第二关节的目标减实际角度约为−0.004889 rad，乘以位置增益4500，产生约−21.998 N·m，正好与该姿态所需的重力平衡力矩接近。有限增益的位置伺服靠这点位置静差维持力矩，因而留有末端偏差。检查器把角度差、力矩、偏置力和接触对一并记录，读者可以核对这个因果解释。

夹爪也有相同区别。本模型把高层 a_g 映射为 $u_g = 127.5(a_g + 1)$，所以闭合、张开分别下发0、255。这段刻度对应模型内部的手指目标开度；无阻挡张开时，两指总开度接近0.08 m。夹住直径0.05 m的圆柱时，闭合命令仍为0，实际开度却停在物体两侧。接触阻止了手指继续闭合，这恰好是抓持所需的物理作用。

若为了让画面“准确到位”，在每个控制步直接把真实 data.qpos 覆盖成逆解，便绕过了上面的执行与接触过程。此时即使物体被穿过、关节力矩不足，也可能在画面中完成运动；这类结果无法用来评价动作控制和抓持。

6.8.6 把环境的一步读懂

物理引擎每次推进0.002 s，一个控制步保持同一执行器命令推进十次。下面的伪代码对应 panda_env.py 中标注的 P1—P5：

```
P1 读取四维动作，限制范围，累积并限制新的末端命令位置
P2 在运动学副本中求关节目标；拼上夹爪命令，组成八维输入
   若设置了执行延迟，把输入放入队列，取出本步真正施加的输入
P3 保持输入，推进十个物理小步，更新末端、物体与接触结果
P4 根据物体状态判断成功、失败或采样超时，计算本次奖励
P5 复制下一观测，连同奖励、结束标记和诊断信息一起返回
```

最关键的代码关系可以缩成：

```
# P1：更新的是期望位置；机器人和物体还没有被移动。
command = np.clip(command + 0.005 * action[:3], workspace_low, workspace_high)

# P2：逆解只修改运动学副本，返回七个关节角目标。
joint_target = solve_ik(command)
requested_ctrl = np.r_[joint_target, 127.5 * (action[3] + 1)]
pending.append(requested_ctrl)
data.ctrl[:] = pending.popleft()   # 零延迟时取出的就是刚生成的命令。

# P3：真实状态由力、接触和积分改变。
mujoco.mj_step(model, data, nstep=10)
mujoco.mj_forward(model, data)     # 刷新积分后状态对应的位姿和接触量。

# P4、P5：先判定这次动作的结果，再返回独立保存的下一观测。
success, terminated, truncated = evaluate_task()
next_observation = get_observation().copy()
```

这里的 solve_ik、evaluate_task 与 get_observation 是对配套函数职责的简写。完整实现还会返回逆解残差、原始动作、裁剪后动作、请求命令和实际命令。特别是加入延迟以后，请求命令与实际命令可能来自不同控制步；只记一个名为 action 的数组会丢掉这个差别。

mj_forward 不再推进时间。它在已有位置、速度和输入上刷新派生量。物理小步结束后再次调用它，是为了让记录的末端位置、接触和下一观测对应积分后的状态，而不是残留的旧计算结果。

6.8.7 一次抓放是怎样连续发生的

阶段控制器在每个运动阶段的前70%时间内把目标线性移向下一个位置，余下时间保持；关闭和打开夹爪的阶段各给1 s。种子0的完整过程如下，坐标都是实际状态，而非期望目标。

时刻	已执行的阶段	末端高度 / m	物体中心高度 / m	双指接触	盘底支撑	此时的含义
2 s	接近	0.2452	0.0349	无	无	夹爪到物体上方，物体仍在桌上
4 s	下降	0.0408	0.0349	无	无	指尖来到抓取高度，张爪未夹持
5 s	夹紧	0.0408	0.0349	有	无	接触建立，但尚不能据此宣布举起
7 s	抬起	0.2450	0.2325	有	无	物体离开桌面并随夹爪上升
9 s	搬运	0.2433	0.2224	有	无	已在托盘上方，仍被夹着
11 s	降低	0.0886	0.0539	有	无	接近盘底，尚未由托盘支撑
12 s	松爪	0.0887	0.0469	无	有	物体落稳，夹爪还没退开
13.4 s	退开	0.2335	0.0469	无	有	退开后稳定条件达到保持时长，结束

注意第7 s到第9 s：末端高度接近不变，物体相对指尖却下移了一点。这个模型的接触允许小量滑移，不能把“夹紧”理解为物体与夹爪永久刚性连接。控制器必须通过后续物体运动判断抓持是否维持；只看阶段计数不会发现滑落。

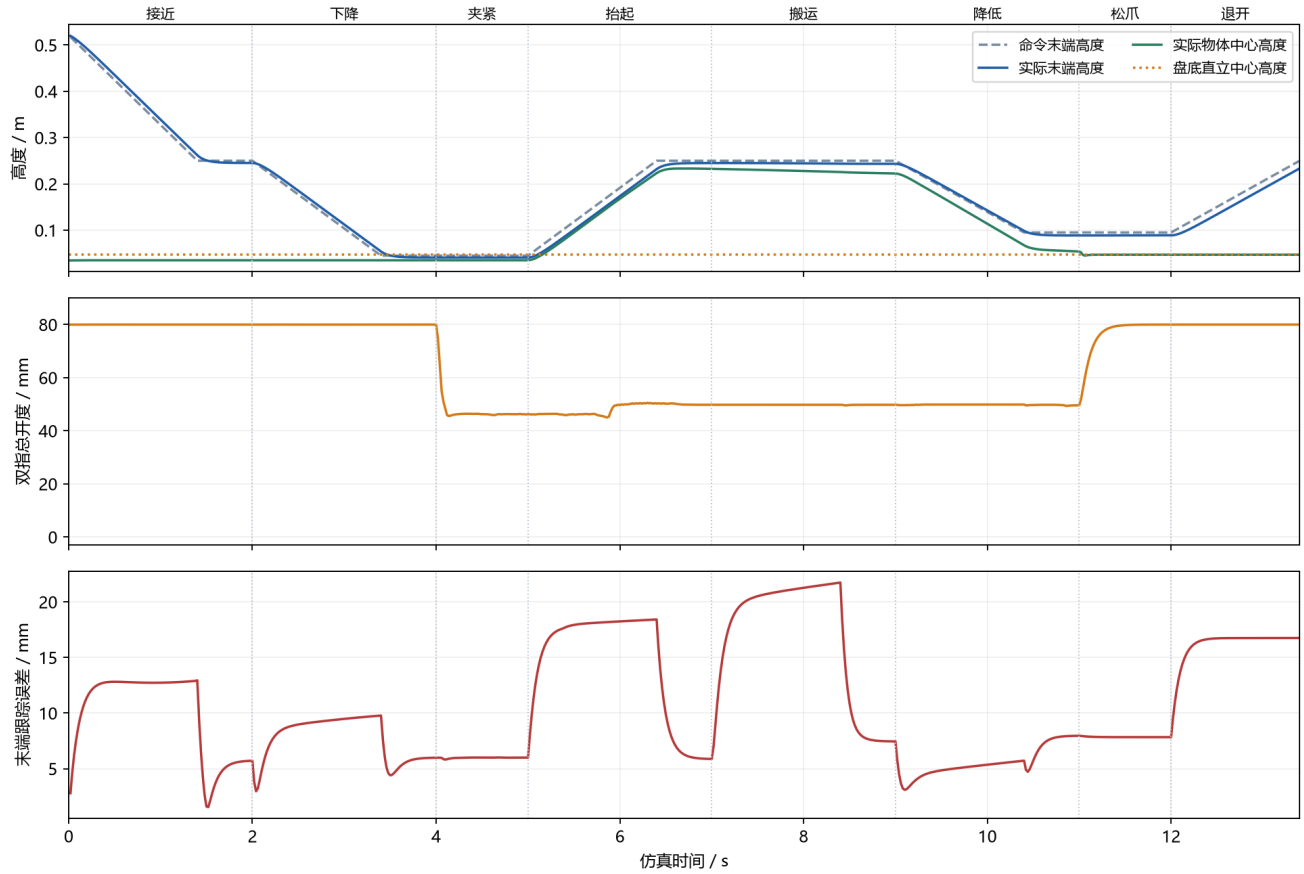


图6.8-3：上图同时画实际末端与物体高度，中图为夹爪开度，下图为末端跟踪误差。阶段边界解释了曲线为何变化；逆解准确、末端跟踪准确、抓持成功分别需要不同证据。

6.8.8 如何判断放置完成

本任务检验“曾在双指接触下举高，最终直立放稳并松爪退开”。成功需要一个历史事件和一组当前条件。历史事件是：物体中心高于0.15 m时，双指同时存在正法向接触力，环境记下lifted_while_grasped=True。当前条件是：中心位于托盘内部目标区域，高度接近0.047 m，圆柱朝上且线速度、角速度较小，盘底确实承担支撑力，两指不再压住物体，夹爪张开且末端已退到0.18 m以上。历史事件已发生、当前条件又连续满足25个控制步，才返回成功。

其中，“直立”通过物体自身竖直轴与世界竖直轴的点积判断；大于0.98，表示两轴夹角较小。“盘底支撑”从接触求解结果累加法向力，本例至少超过物体重力的一半，再结合低速度与正确高度检查。种子0最终盘底法向力约0.9812 N，与0.10 kg物体的重力相符。仅仅存在一个几何接触点，还不足以说明它承担了支撑。

这些阈值是本教学任务的验收定义：允许圆柱横放的任务就应删改直立条件，需要夹爪继续扶持的任务也会使用别的释放条件。夹爪完全张开、退到指定高度，是本例定义的操作结束姿态。当前判据没有要求搬运全程直立，也没有单独标注释放瞬间的姿态；需要这些时序要求的任务，应把相应事件加入记录。

第9 s是一个现成反例：物体平面位置已经靠近托盘中心，速度也很低，但它仍悬在托盘上方，由手指支撑。只检查平面距离或低速度都会误判。第12 s则说明，物体放稳与整个操作结束也可以是不同事件；本任务还要求夹爪退开。

运行 practice/verify_project.py 会核对正常抓放、逆解与真实跟踪的差别、种子重放、重置后参数不累乘，以及一条停止降低和松爪的对照轨迹。该对照在第9 s同样位于托盘上方，却不满足释放与支撑；继续运行后物体还可能滑落。对照检验的是错误判据会接受什么状态，不能把固定夹爪命令视为物体始终被固定住。

6.8.9 把轨迹变成可以学习的数据

episode.npz 保存的基本关系是 (o_k, a_k, r_k, o_{k+1})。本例 $o_k \in \mathbb{R}^{37}$ 包括七个关节位置和速度、夹爪总开度、实际末端位置、累积命令位置、物体位姿与速度以及托盘位置。准确切片见 schema.json。旋转关节用弧度，平移用米；物体四元数顺序为 w, x, y, z，自由关节的平移速度在世界系表达，角速度在物体系表达。

这是**仿真状态观测**：物体位姿来自引擎真值，可以先排除视觉估计误差。真实部署若只有 RGB（Red, Green, Blue，红绿蓝三通道）相机，不能直接读取这些真值；随后需要视觉模型或状态估计来提供相应信息，或者直接训练从图像到动作的策略。这37项也没有包含所有物理内部量，例如手指速度和执行延迟队列，不能未经分析就把它当作完整马尔可夫状态。

为什么同时保存八维命令？高层动作表达了“想移动多少”，请求执行器命令表达了“逆解后下发什么”，实际执行器命令表达了“受延迟后本步用了什么”。学习上层策略一般模仿四维动作；学习带执行器影响的转移，则要明确把哪一层输入交给模型。字段并列保存，可以防止后续把控制误差错误归因于策略。

种子0数据有670条转移，state 与 next_state 形状均为 (670,37)，action 为 (670,4)。相邻记录满足 state[k+1] == next_state[k]；最后一项后继观测仍属于结束回合，没有偷偷换成下一局起点。time[k] 记录这次动作结束时刻，当前观测则来自初始化时刻或前一次动作的结束。因此代码用 observation_time = np.r_[0.0, time[:-1]] 恢复当前观测的时间戳，再与图像配对。结束标记单独保存，后面构造价值标签和历史窗口时才能正确切断回合。

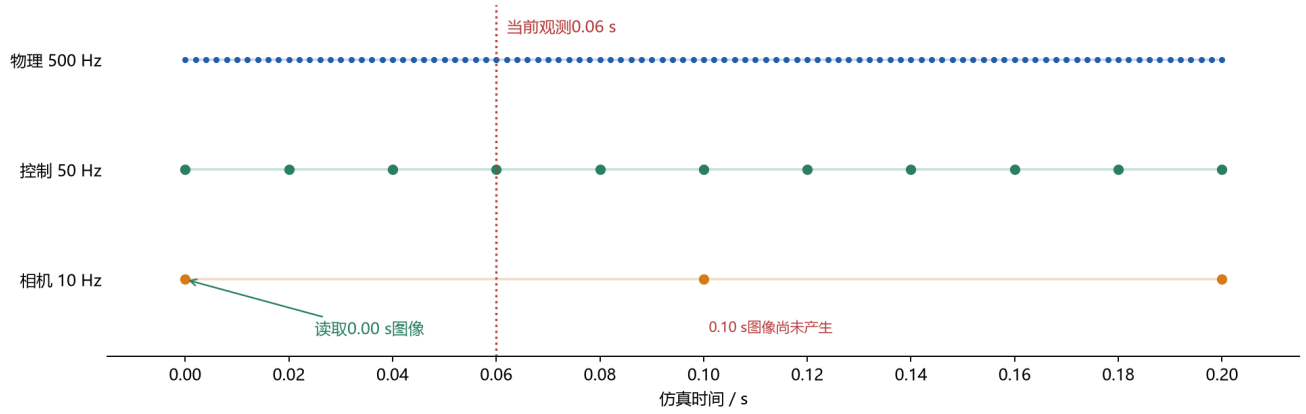


图6.8-4：物理500 Hz、控制50 Hz、图像采样10 Hz。一次动作之前，策略只能读取已经获得的图像。本项目没有额外模拟相机曝光和传输延迟。

图像每五个控制步采集一次：相机渲染640×480图，再将128×96采样图放入数据数组；各阶段结束时另外保存一张640×480图，便于看清接触细节。种子0有134帧采样图。观测时刻0.06 s必须配0.00 s的图像，不能配0.10 s的图像；后一帧包含动作执行后的信息。配对时用“不晚于观测时刻的最后一帧”，同时保存图像时间戳。本例图像最大年龄约0.08 s。practice/inspect_dataset.py 会核对这些关系。

轨迹保存阶段名称、控制步和时间戳，教师使用的阶段内计数没有放进37维观测。固定阶段时长可以结合记录重建计时，学生如果没有同样的历史，就未必能确定教师当时处在哪一阶段。脚本在位置近似不变时也会从等待转为夹紧；若可见观测相似而标签不同，单帧回归策略可能学到互相冲突的动作。后续模仿学习应检查这种条件信息不足，比较历史观测、明确的技能接口或基于反馈的教师。

6.8.10 从同一环境开始做对照

环境能运行后，首先改变一个因素，观察哪条链路变化。执行：

```
.venv\Scripts\python.exe practice/compare_domains.py
.venv\Scripts\python.exe practice/collect_dataset.py --episodes-per-split 3 --capture
.venv\Scripts\python.exe practice/inspect_dataset.py
```

第一条命令保持控制器不变，分别改变质量、摩擦、伺服增益或执行延迟；第二条命令按回合采样参数并收集带图像的示范，整条轨迹分到同一个数据子集；第三条检查时序与字段。不加 --capture 时只采集状态与动作。已有结果和每个参数的含义在6.7节展开。这些记录没有语言标签，也还没有网络参数更新；目录中的 train_* 只表示预定用于训练的数据子集。

进入第2章的训练实验时，环境、动作和成功规则沿用这里的定义，只替换产生 a_k 的方法。先用状态观测建立模仿学习基线，再引入图像和语言，便可以区分低层执行失败、数据覆盖不足与视觉理解失败。接入第5章的预测模型时，也先预测物体和末端的实际变化，而不是把命令位置当作已经实现的位置。

不同引擎和真机迁移还需要各自的实测响应。本学习包已执行的是 MuJoCo 内部参数对照，尚未包含第二引擎或真机结果。6.7节给出如何沿现有输入输出逐级验证的实验设计。

6.8.11 练习题

题6.8-1 同一目标点，运动学副本上的位置残差约为 2.0×10^{-6} m，物理执行2 s后的末端偏差约为 5.69×10^{-3} m。两种误差各测什么？如何区分逆解问题、执行器问题和接触阻挡？为什么每步直接把真实 qpos 改成逆解，会破坏这个实验？

题6.8-2 第9 s物体中心约为 $(0.4967, -0.2433, 0.2224)$ m，托盘中心平面位置为 $(0.5, -0.25)$ m，双指仍接触，盘底支撑力为0，物体速度很低。仅看平面距离、低速度或夹爪目标位置各会怎样误判？给出能检验本任务“曾在双指接触下举高，最终直立放稳并松爪退开”的条件，指出哪些属于任务约定，以及是否足以证明搬运全程或释放瞬间直立。

题6.8-3 轨迹记录37维观测、4维动作、8维实际执行器命令与后继观测，相机10 Hz、控制50 Hz。0.06 s的观测应配哪一帧图像？物体真值和脚本内部阶段分别给部署与学习带来什么问题？为什么不宜把同一轨迹相邻帧随机分到训练与测试？这些数据怎样接到后续模仿学习实验？

6.8.12 参考答案

题6.8-1答案 设正运动学为 $f(q)$，目标为 $p^\star$，逆解为 $q^\star$。两种误差分别是：

$$e_{\mathrm{IK}} = \|f(q^\star) - p^\star\|, \qquad e_{\mathrm{track}} = \|f(q(2\mathrm{s})) - p^\star\|.$$

前者检查一组候选构型的几何位置，后者检查有限力矩执行后的真实构型。若逆解残差、姿态残差或限位已有问题，先查目标和求解；若请求命令与实际命令不同，查裁剪、延迟与执行映射；若命令一致，继续比较目标/实际关节位置、速度、执行器力矩、重力偏置和具体接触对。

本例推进2 s后关节速度已接近零，没有机器人接触，执行器力矩未饱和。第二关节约−0.004889 rad的位置差乘4500，得到约−21.998 N·m，实际力矩与重力偏置也接近该值，支持“有限位置增益的重力静差”这一具体诊断。其他场景可能是延迟、饱和或接触阻挡，需用自己的日志区分。

每步直接写真实 qpos=q_star 会强制构型到位，绕过驱动、惯性、重力和碰撞过程，只剩几何位置检查。运动学副本可以这样迭代，实际执行状态不能用它冒充物理跟踪结果。

题6.8-2答案 第9 s物体平面位置距盘中心只有约7.45 mm，直立圆柱底面却约为 $0.2224 - 0.035 = 0.1874$ m，比支撑面高约0.1754 m。它仍由双指悬持，盘底法向力为0。平面距离遗漏高度和支撑；低速度不能区分被夹住与已放稳；夹爪目标位置只是命令，不能代替物体实际状态。

本例先记录历史事件：物体中心高于0.15 m且双指都有正法向接触力。最终再检查：中心在目标区域，高度接近0.047 m，圆柱轴向接近世界竖直、线速度和角速度低，盘底确实承重，两指已无有效接触，实际张爪且已退开。已记录该抓持举高事件，并且最终条件连续满足25个控制步，才结束成功。

直立要求、各阈值、完全张爪、向上退开和保持时长都是这项教学任务的约定；允许侧放或要求扶持的任务应另行定义。该历史与终点组合也不证明搬运全程直立、释放瞬间始终直立或无限时间稳定。悬停对照在第9 s仍被夹持，后来可能滑落，必须按具体时刻解释。

题6.8-3答案 0.06 s只能读0.00 s图像，0.10 s图像尚未产生。一般选择“不晚于当前观测可用时刻的最新帧”；真实系统还要考虑读出和传输延迟。本例 time[k] 记录动作结束，当前 state[k] 要对应前一时刻，不能因此偷用动作后图像。

37维观测含仿真物体位姿与速度，普通RGB部署没有同样的精确输入；它还遗漏手指速度、延迟队列和任务历史，不能当作完整马尔可夫状态。教师又依赖内部阶段与计时；同样近似静止的可见状态，可能有等待或切换动作两种标签。后续应明确学生能否维护相关历史，或改用可观察反馈驱动的教师。

同一轨迹相邻帧共享场景与参数，多个动作还会复用同一图像。逐帧随机划分可能让同一图像或近乎相同的上下文同时进入训练与测试，低测试误差不能代表新回合泛化。应按完整回合划分，并按研究目标进一步隔离物体、场景或参数组合。

用于模仿学习时，先定义学生的可部署输入、四维输出、因果对齐、有效教师标签和训练损失，实际更新策略参数，再在未见完整回合闭环评测。八维执行器命令与四维高层动作属于不同层级，存在延迟时还要分清请求与实际命令。当前九条数据已补采RGB并核对时序，仍无语言标签，也没有网络训练；能称为脚本示范与失败数据，不能据此声称VLA已经学会任务。

第7章：具身数据生产、质量与科学评测

数据决定模型实际学到的任务范围。这里沿采集、对齐、标注、采样和拆分展开，解释每项处理怎样改变监督与泛化评价。

7.1 从任务范围建立数据和标签定义

7.1.1 数据首先记录事件和语义

一条轨迹不只是若干数组。它记录谁在什么场景，以哪个策略版本，读到了什么观测，提出什么动作，实际执行什么动作，之后发生了什么。数据定义将这些字段的单位、坐标、时间和有效性写清，才允许后续训练正确连接输入与答案。

7.1.2 数据是观测、行动与学习目标之间的关系

机器人数据包含图像、关节、动作、力、任务描述和结果等。它们不是并排存在就能训练。一个样本必须说明“在什么条件下，要预测或评价什么”。

模仿学习需要条件—动作对应；**世界模型**需要当前条件、动作与之后的变化；奖励学习需要任务结果或比较；感知学习需要对象、区域或几何标签。同一段视频可以支持多种目标，但提供的信息边界不同。

图7.1：原始记录经过对齐和标签构造，成为不同学习问题。标签规则决定哪一关系受到监督。

杯子成功进入托盘，只说明整段执行达到某种成功定义，不能直接给每个像素杯子位置真值。根据视频估计的位姿属于伪标签，其误差和依赖需要记录。

数据质量首先看它是否正确支持所声称的关系。图像很清晰，但动作晚了两帧，可能比稍有噪声而同步正确的数据更有害。训练损失低也可能只是学会错误配对。

7.1.3 采集计划要从最终测试条件倒推

先定义机器人、控制器、动作单位、相机、任务和成功条件，再定义希望泛化的变化。如果最终要处理不同杯子与托盘位置，采集只变光照不能覆盖这些需求。

把变化因素写成表：对象形状、材质、位置、背景、视角、指令、接触状态、操作者和设备配置。不是所有组合都必须穷举，但哪些缺失、哪些需要组合泛化，应明确。

一次回合（episode）是从初始化到成功、失败或重置的一段连续任务。回合中可能有多次抓取尝试。一次尝试与一个回合不是同一个统计单位，后续采样与评价要区分。

采集预算包括机器人时间、人工指导、重置、审核和故障处理。某种采集方式每小时轨迹多，若大部分需要重做标签，最终有效数据成本仍可能更高。

建议先做小规模试采，验证动作回放、同步、标签与重置记录，再扩量。先收很多再补定义，容易产生无法恢复的语义缺口。

7.1.4 一条可追溯的轨迹应该记录什么

轨迹记录至少需要身份、时间、原始或可恢复观测、机器人状态、动作定义、控制配置、任务、结束原因与有效标记。可追溯不要求所有信息都进模型，而是支持检查和再构造标签。

记录	作用	常见误解
回合、场景、对象标识	划分、去重与统计	文件名不同就独立
命令与状态时间	核对因果与延迟	一行中的数据天然同时
动作单位和参考系	明确执行语义	七维向量必然同义
成功、终止、截断原因	监督与评价	所有结束都算成功或真正终止
标定、控制器、软件版本	解释响应变化	同型号设备行为永远一致
来源与处理版本	追踪标签与过滤	清洗后不需要保留来源

标签可以后来加工，但无法恢复的原始信息要优先保留。坐标变换、裁剪、动作标准化与语言生成都应记录规则，使别人能知道数值如何得到。

配置错误可能贯穿一批数据。审核应按采集批、设备与日期分层抽样，避免随机抽几帧全都来自正常部分。

每条轨迹应关联任务、身体、相机标定、控制周期、动作解释、归一化版本和采集来源；每一步保存观测时间、动作生效时间、提议与执行动作、反馈和结束原因。模型输入与仅供标签或评价的字段应分别标记。例如训练成功分类器时，把成功标签同时送入输入，会泄漏答案；第3.8节的质量条件化策略则有不同用途：质量标签在训练中区分行为类别，部署时由系统主动请求较好类别。字段能否作为输入，取决于模型预测什么，以及部署时如何获得该字段。

7.1.5 练习题

题7.1-1 数据字段只写 action:[0.1,0,0]，缺少坐标、单位和控制模式。训练损失可以正常下降，为什么部署仍无法解释这个动作？应补哪些字段？

7.1.6 参考答案

题7.1-1答案 训练损失只比较数组的数值，无法检查这些数的物理含义。模型可以准确预测 [0.1,0,0]，但执行端仍不知道它是世界坐标中的目标位置、末端坐标中的位移、线速度，还是三个关节的命令。相同的数值在不同定义下会引起完全不同的运动。

至少应保留：机器人和末端版本；每维名称与顺序；关节空间或末端空间；参考坐标系、轴方向和单位；绝对目标、相对当前状态还是相邻目标差分；位置、速度、力矩或阻抗控制模式；动作开始时刻、保持时长和控制频率；夹爪开合编码；归一化、缩放、限幅以及执行器配置。若是动作块，还要说明每一项相对同一预测起点，还是相对前一项。

例如“base 坐标系下末端位置增量 [0.1,0,0] m，基于决策时刻的末端位姿，经位置控制器在 0.1 s 内执行”才给出了可重放的含义。部署时必须用同一规则将模型输出反归一化并转成执行命令。可以用保存的起始状态和动作重放验证，确认数据采集、训练转换和执行端实现的是同一种动作。

7.2 遥操作、重定向、采集接口与时间对齐

7.2.1 遥操作、示教与 UMI 各自采到了什么

遥操作让人通过设备控制机器人，记录当时机器人观测与命令。拖动示教直接引导机器人或末端；虚拟现实设备可以记录人手目标；不同接口改变动作平滑、速度和可用反馈。

人手姿态到机器人动作需要动作重定向（retargeting）：把人的运动映射为机器人可达的动作。映射受到坐标、关节范围、工具与控制器影响。人完成动作并不说明机器人能以同样关节轨迹完成。

UMI 使用手持夹爪等接口在自然场景采集，并通过姿态估计、动作表示和延迟匹配等设计接到机器人学习。它提供一种降低现场机器人依赖的路线，但需要配套接口和质量验证，不能把任意手机视频视为同等动作示范。UMI。

采集应同时保留原始设备信号、转换后命令与机器人反馈，便于追踪重定向误差。只存最终动作向量，后续可能无法判断偏差来自人、映射还是执行。

操作者间速度与风格差异可以增加覆盖，也会增加条件未解释的变化。需要时记录操作者或采集条件，检验模型学到任务规律还是操作者捷径。

7.2.2 时间对齐怎样改变标签

相机曝光、驱动接收、控制命令发送与关节响应有不同时间。一个统一系统时钟有帮助，但不能自动消除传感器内部延迟。时间戳必须记录其对应事件。

若关节以 0.4 弧度/秒变化，20 毫秒错配约产生 0.008 弧度偏差。接触瞬间状态变化快，误差影响可能更大。判断同步质量应结合运动和事件，而非只看时间差均值。

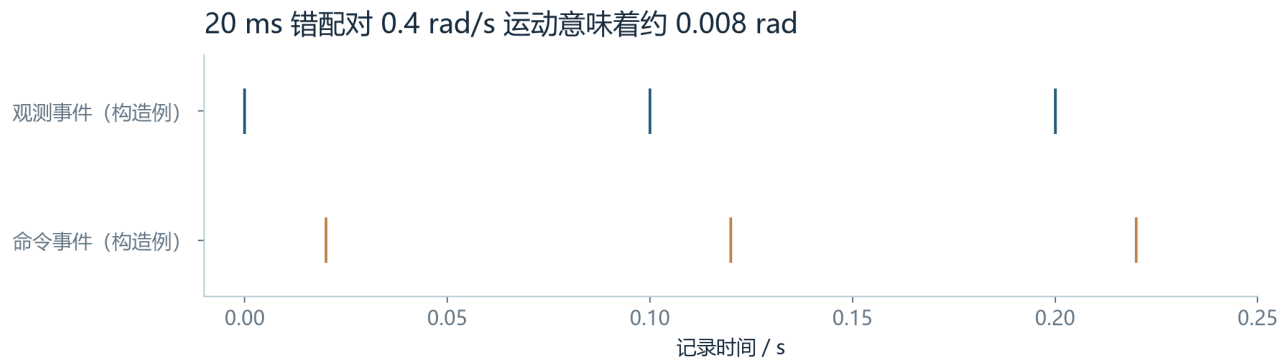


图7.2：同一个控制过程有多个物理事件。标签对齐必须说明希望预测发送命令还是后续真实响应。

把记录统一到某个频率时，连续关节量可按适合的插值方式处理；离散开合事件或成功标签不能机械平均。图像通常选择对应帧，不应通过补重复帧伪装成新增真实观测。

动作重采样还取决于表示。速度可按时间关系积分或重新采样；位置增量改变周期时不能保持数值不变而宣称语义相同。设备丢帧需要有效标记，避免模型把缺失记录当静止。

标定与同步参数应有版本。相机支架移动后，旧外参不一定仍有效；数据应能关联到采集时配置，而不是全部使用今天的配置重新解释。

7.2.3 对齐应依据动作何时生效

相机采样、编码、传输、策略计算与执行器响应发生在不同时间。若专家在看见图像后 80 毫秒下令，机械臂又经过 20 毫秒才响应，把命令简单配给文件中最近一帧可能制造错误标签。应定义用于决策的观测时刻和执行起点，必要时保存最近历史或估计延迟。

验证对齐可以先做明显事件：夹爪开合、指示灯闪烁、关节阶跃或触碰信号，比较跨设备时间戳与实际响应。仅检查时间单调递增不能证明设备时钟同步。对齐不确定时，应记录误差范围并做延迟敏感性实验，而不是用更大的模型吸收错误。

7.2.4 为决策选择当时真正可用的观测

设三帧图像的采集时刻为 $[0.00, 0.04, 0.08]$ 秒，到达程序的时刻为 $[0.02, 0.09, 0.10]$ 秒。决策发生在 0.07 秒：第二帧已经拍到，却还没传到程序，实际只能用第一帧。按采集时间找最近帧会错误选中第二帧。

可以把选择规则写成两个条件：消息已到达，且内容采集于决策之前。候选中再取采集时刻最新的一条，并用最大年龄限制太旧的输入。以下程序同时处理消息乱序与没有可用观测的情况。

```
import numpy as np

def causal_observation_index(sample_times, available_times,
                             decision_time, max_age):
    sample_times = np.asarray(sample_times)
    available_times = np.asarray(available_times)
    if sample_times.shape != available_times.shape:
        raise ValueError("每条观测需要对应的采集时间与可用时间")
    # 1. 找出决策时已经到达、且内容不是未来的消息。
    allowed = ((available_times <= decision_time)
               & (sample_times <= decision_time))
    candidates = np.flatnonzero(allowed)
    if len(candidates) == 0:
        return None
    # 2. 到达次序可能不同，按内容的采集时刻选最新观测。
    index = int(candidates[np.argmax(sample_times[candidates])])
    # 3. 太旧的观测按缺失处理，让上层决定等待或恢复。
    if decision_time - sample_times[index] > max_age:
        return None
    return index

index = causal_observation_index([0.00, 0.04, 0.08],
                                 [0.02, 0.09, 0.10], 0.07, 0.10)
```

```
print(index)  # 0
```

离线训练也需要重建这条信息边界。使用双向插值或未来帧，可以提高回顾性状态估计精度；将这种结果直接当在线策略输入，却会让训练条件比真实部署更有利。保存采集时刻与可用时刻，才能同时检查同步、延迟与因果性。ROS 2 时间设计

7.2.5 对齐观测时，采集时间与可用时间都要看

相机在0.04秒采到一帧，并不表示控制程序在0.04秒就得到它。图像传输和处理可能到0.08秒才完成。若决策发生在0.07秒，这帧在物理上已经存在，在当时的控制程序里却尚不可用。训练样本要模拟部署的信息条件，需要同时满足“采集不晚于决策”与“已经到达或处理完成”。

可以给每条消息记录两个时间：内容对应的采集时刻 t_s 与程序可用时刻 t_a。决策时刻 t_d 只能使用 $t_a \le t_d$ 的消息，再按 t_s 判断它有多旧。如果离线为了平滑，用决策之后的一帧插值，那是在做回顾性估计；不能原样当作在线策略的输入。否则离线模型会利用部署拿不到的未来信息。ROS 2 时钟与时间设计

这也连接端侧加速：模型推理时间下降后，如果输入仍来自很长的队列，实际动作可能继续依据旧场景。数据对齐、消息队列与计算时间共同决定机器人看到的是哪一个时刻的世界。

7.2.6 练习题

题7.2-1 传感帧采集时间为 [0.00,0.04,0.09] s，决策时间为 0.07 s。先假设消息采集后立即可用，在线因果对齐应选哪一帧？再假设 0.04 s 帧直到 0.08 s 才可用，而 0.00 s 帧已到达，应怎样处理？能否使用 0.04 s 与 0.09 s 两帧插值？

7.2.7 参考答案

题7.2-1答案 先区分图像的采集时刻与它到达决策程序的可用时刻。若三条消息在采集后立即可用，0.07 s 决策能读到 0.00 s 和 0.04 s，选最新的 0.04 s；0.09 s 尚未发生，不能作为当前输入。只按时间距离取最近帧，会错误选中 0.09 s。

若 0.04 s 图像直到 0.08 s 才可用，0.07 s 的真实决策还读不到它。只要 0.00 s 图像已到达，当前只能使用它，或依据已有历史进行带不确定性的状态估计。允许等待到 0.08 s 再决策也可以，但这已经改变决策和执行时刻，必须在记录中体现，不能在离线样本中把消息“提前到达”。

可把选择规则写为

$$i^\star = \arg\max_i t_i^{\text{capture}}, \quad t_i^{\text{capture}} \le t^{\text{decision}}, \quad t_i^{\text{available}} \le t^{\text{decision}}.$$

还应限制最大观测年龄并标记丢帧。对 0.04 s 和 0.09 s 的两帧插值虽然可能更接近 0.07 s 状态，却使用了未来证据；它适合离线平滑分析，不适合作为模拟在线决策的信息输入。

7.3 示范质量、覆盖和语义保持的数据处理

7.3.1 好示范不等于只保留最快最顺的成功

成功示范提供任务完成路线，失败提供机制边界，恢复示范提供偏离后如何纠正。数据是否应进入某个训练目标，要看标签含义。失败动作不能不加区分地作为“该模仿的正确动作”，但可以支持价值、失败识别或纠正学习。

慢动作可能含有观察与精细调整；停顿也可能只是操作者走神。仅按速度删数据会把两种情况混在一起。平滑性指标类似：突然动作可能是错误，也可能对应真实接触变化。

质量审核分三层：记录是否损坏；标签是否对应真实事件；行为是否有助于目标学习。层次之间不能互相替代。自动规则可以筛明显异常，边界情况需人工或模型反馈验证。

在困难位置，失败概率高。如果只保留成功，最终数据可能主要覆盖容易位置，模型误以为“世界总是容易的”。应记录尝试分母和筛选条件，必要时为困难条件生产更可靠的纠正。

建立质量规则时，报告被保留和被排除的数据分布，以及规则对最终任务的影响。质量分数的价值来自可验证目标，而不是“看起来专业”的多项加权。

7.3.2 覆盖为什么比帧数更难衡量

相邻一百帧常描述同一个对象和同一段接近。更多帧有助于时间分辨率，但并不等于一百个独立场景。覆盖要看任务相关变化和策略将访问的状态。

十种杯子、十种位置、五种光照有五百种组合；只知道每个因素都出现过，不能证明重要组合已覆盖。可以先分层统计，再对关键交互因素设计测试，例如透明杯在强光下是否总失败。

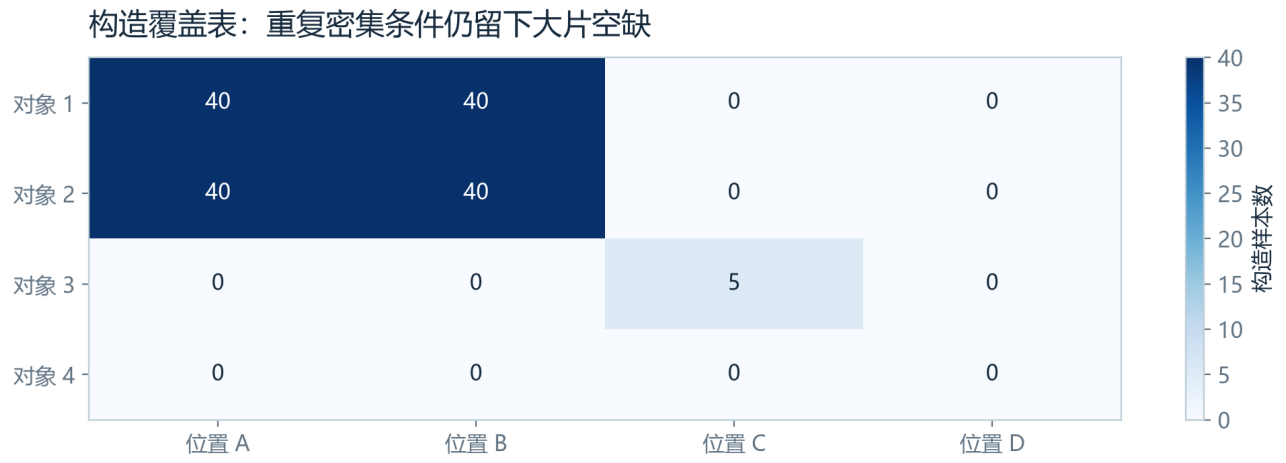


图7.3：大量样本集中在少数条件，仍会留下任务相关空缺。补数据应对准这些空缺。

特征空间距离可以帮助寻找外观新颖样本，但外观差异不等于动作差异。换背景可能距离大，杯子质量变化却视觉几乎不变。覆盖指标应结合语义、几何、动作和物理条件。

数据审计的结果应是一组可验证假设，例如“侧边接近和夹空恢复不足”，而不是一个无法解释的全局多样性分数。

7.3.3 清洗、过滤与增强都可能改变任务

清洗检查丢帧、异常时间、损坏文件、无效传感器、单位错误和明显标签错位。异常规则要保留理由与来源，支持回溯，而非只留下“干净”数据。

几何增强必须同时处理动作和标签。图像水平翻转后，机器人系动作不一定只需把 x 取负；还受相机外参和动作参考系影响。空间裁剪可能删掉目标，颜色增强可能抹去指令要求的蓝红区别。

如果增强应保持标签不变，需要说明任务中的**不变性**假设；如果应变换标签，需要使用一致转换。增强不是一种天然无害的免费多样性。

按模型置信度过滤会偏向模型已会的容易样本，形成反馈偏差。可保留一部分低分样本供人工审查，区分错误标签与真正难例。仅用原模型分数决定新训练数据，可能强化原盲区。

过滤方案应比较相同有效数据量的随机选择、简单规则与新方法，并报告被删条件。如果提升主要来自去掉明显坏文件，而非新颖评分机制，论文应据实说明。

7.3.4 清理坏数据也可能改变你要学习的任务

删除所有碰撞和失败轨迹能提高数据表面的成功比例，却可能失去识别危险与恢复的证据。应根据用途处理：动作模仿中的错误标签可剔除或修正；奖励和**世界模型**学习中的真实失败转移仍可能很重要；明显传感损坏需标记其范围。质量、覆盖与任务难度是不同维度。

覆盖不应只数物体类别。需要包含初始位置、目标指令、视角、接触阶段、负载、失败类型和恢复动作。两万条重复的容易抓取，可能比少量精心设计的纠错轨迹更难支持恢复研究。最有效的新增数据通常来自已知能力缺口，评价也应按这些条件分层。

7.3.5 练习题

题7.3-1 删除所有失败轨迹后，**行为克隆**的留出动作误差下降了。为什么这不一定改善闭环？哪些失败片段值得保留或重新标注？

7.3.6 参考答案

题7.3-1答案 删除失败轨迹后，留出集也可能只剩容易、成功的状态，动作误差自然更低。闭环执行却会产生定位偏差、滑落和没抓牢等状态；策略若从未见过这些状态，就不知道如何恢复。离线误差评测的输入来自示范，闭环输入由策略自己的行为产生，两者不是同一分布。

失败轨迹也不是整条都无价值。一次取杯可能因最后放置出错而失败，但开门、接近和抓取片段仍正确；滑落后的重新定位和再次抓取，恰好提供成功示范很少覆盖的恢复动作。这些可按阶段保留并补标签。若某段动作本身导致错误，直接用普通行为克隆把它当正确标签，会教模型重复错误；可让专家对该状态给出纠正动作，或保留它用于学习失败评价、动力学和带明确目标的强化学习。

例如成功示范都从“杯子稳固在手里”开始搬运，实际策略经常只夹到杯缘。值得补的是从夹偏状态退回、张开、重新对准的纠正序列。应在固定闭环任务上比较恢复率、失败类型和总成功率，再判断清洗是否有效，而不能只依据清洗后留出误差。

7.4 混合来源、采样配方与跨形态权重

7.4.1 数据配方怎样改变模型被训练重视的东西

多个数据源混合时，采样概率定义训练看到它们的频率。按帧数采样会偏向大数据源；各源等概率会相对提高小源权重。没有脱离目标的通用最佳比例。

一个常见教学形式为 $p_i = n_i^\alpha / \sum_j n_j^\alpha$，这里只考虑非空来源。$n_i$ 是选定口径下的数据量；α=1 按量混合，α=0 各源等权，介于两者之间缓和规模差异。它不是质量学习器，只是明确的采样规则。

两个源有 1000 与 100 条记录，按量为约 90.9% 与 9.1%；取 α=0.5 时约为 76.0% 与 24.0%；等权则各 50%。小源被看得更多，但若质量差或条件不同，效果未必改善。

若目标是对分布 p 上的样本损失取平均，实际却按 q 采样，在支持条件满足时可使用**重要性加权**：$\mathbb{E}_q[(p(x)/q(x))\ell(x)] = \mathbb{E}_p[\ell(x)]$。它补偿“某类样本被抽得过多或过少”，使计算仍指向原来的目标风险。若改变采样比例就是为了重新重视某类任务，便不应再用权重把这种改变全部抵消。若 q 根本不包含目标中的某类样本，权重也无法补出这些数据；极大权重还会使少数样本主导梯度。

回合长度、有效动作数与 loss 归一化也影响权重。不能只说“两个数据集各一半”，却让每批有效监督量不同而忽略实际梯度贡献。

7.4.2 跨机器人与跨来源需要语义适配

共享数据要求相机、状态和动作各自的意义清楚。一个源记录末端绝对位置，另一个记录局部速度，不能靠补成相同维度直接混合。

可在统一语义层转换，再保留机器人条件和有效掩码。转换会引入误差，例如由末端轨迹反算关节目标需要**逆运动学**；由相邻位置差分得到速度，会放大测量噪声。

缺失状态或相机需要显式标记。用零填充但不告知缺失，模型可能把零看成真实物理状态。重复帧也不应被当成不同观测。

应分别评价每个来源与机器人。共享训练可能对某些条件有益、另一些负迁移；总体平均不能替代分层结果。若新增源只提高它自身对应场景，也不一定证明广泛跨身体迁移。

7.4.3 数据配方是一项可以消融的设计

按轨迹、按时间步、按任务等概率采样得到不同训练分布。长轨迹若逐步等概率抽样，会贡献更多训练样本；小数据任务若按任务均匀抽样，会被更频繁重复。应记录每类数据的有效曝光次数，而不仅记录文件数量。

比较混合配方时固定总训练步数、有效标签预算和目标任务评价，再改变采样权重。检查共享数据是否带来正迁移，以及是否损害难任务。若从异质来源获得大量免费语义数据，应同时报告来源和额外预训练，而不是称其与仅机器人数据的基线完全同预算。

7.4.4 采样配方的变化，可在概率上算出来

A 来源有10万条轨迹，B 有1000条；若从全部轨迹均匀抽取，A、B 的来源概率约为0.9901、0.0099。若先以各一半概率选来源，再在来源内部均匀选轨迹，一条 B 轨迹的抽中概率为 $0.5/1000$，一条 A 轨迹为 $0.5/100000$，前者是后者的100倍。它增加了少量困难来源在训练中的相对关注。

这不必自动做“纠偏”。如果研究目标就是让困难任务占更多训练权重，重新采样正是在改变目标分布；如果只是为了提高采样效率、最终仍要估计原分布的风险，则需要相应的重要性权重。轨迹抽中以后怎样选择窗口、每条轨迹的长度，以及损失怎样归一化，还会进一步决定哪个实际动作更常被训练。数据配方的作用应顺着这条概率链解释。LeRobot 数据集教程

7.4.5 练习题

题7.4-1 数据源 A 有 10 万条简单轨迹，B 有 1 千条困难轨迹。设每次先抽一条轨迹，再从中按同样规则抽训练窗口。按全部轨迹均匀采样与先按来源各一半采样，来源概率和单条轨迹概率各是多少？两种配方改变了什么训练目标，应怎样在固定预算下评价？

7.4.6 参考答案

题7.4-1答案 先按题意固定采样单位：抽一条轨迹，再用同样规则从该轨迹取窗口。总轨迹数为101000。全部轨迹均匀采样时，

$$P(A)=\frac{100}{101}\approx 0.9901,\quad P(B)=\frac{1}{101}\approx 0.0099,$$

每条轨迹无论来自A还是B，概率都是 $1/101000 \approx 9.901 \times 10^{-6}$。

若先以各一半概率选来源，再在来源内均匀选轨迹，则两来源概率均为0.5，但

$$q(\text{A中的一条}) = \frac{0.5}{100000} = 5 \times 10^{-6}, \qquad q(\text{B中的一条}) = \frac{0.5}{1000} = 5 \times 10^{-4}.$$

B中单条轨迹的概率是A中单条的100倍，相比原配方提高了50.5倍。它增加困难数据的训练份额，也可能反复记住少量B轨迹或放大B的标签错误。

令 L_A, L_B 是按上述窗口规则得到的来源内平均损失。两个目标分别为

$$L_{\text{all}} = \frac{100}{101} L_A + \frac{1}{101} L_B, \qquad L_{\text{balanced}} = \frac{1}{2} L_A + \frac{1}{2} L_B.$$

来源均衡采样是在改变训练目标，而不是自动保留原来的平均。如果想用均衡采样估计原目标，可给A、B样本分别乘 $w_A = 200/101 \approx 1.9802$、$w_B = 2/101 \approx 0.0198$，因为权重等于目标采样概率除以实际采样概率。若本来就希望强调困难来源，则直接优化均衡目标，不必用权重抵消这种设计。

评价时固定模型、更新次数和样本预算，使用独立A、B留出场景分别看闭环成功，再按部署任务比例计算总分，同时检查B过拟合与A退化。按帧均匀会使长轨迹占更多份额，是另一种配方，不能混同。

进一步阅读：LeRobot 官方教程：LeRobotDataset v3.0、Art Owen，Monte Carlo Theory, Methods and Examples，第9章9.1基本重要性采样。

7.5 语言标签、视频证据与合成监督

7.5.1 语言、事件与几何标签怎样得到

语言可以来自任务指令、人工描述或自动生成。回合级“放杯到托盘”不一定解释每个中间片段：片段可能在接近、抓取、调整或恢复。标签粒度应匹配训练目标。

对象区域和位姿可由标注、标定目标、几何重建或感知模型生成。自动预测属于伪标签。其错误可能与待训练模型使用的输入相关，不能当作独立真值评估同类能力。

接触、抓起、滑移与成功事件可结合力、夹爪、图像和状态判断。单个信号容易混淆：夹爪闭合不证明抓到了杯子，杯子高度改变也可能是人手移动。应对关键事件做多证据核对或抽样人工复审。

弱监督使用不完整或有噪声的标签。训练可以对可靠部分加掩码、权重或建模噪声，但必须知道不确定性来自哪种规则。多个标签器投票只在一定误差关系下更可靠，若都依赖相同坏深度，投票不能消除系统偏差。

验证标签器时，应按场景、动作阶段和失败类型分别测精确率、召回率与时间偏差。**精确率**问“判为该事件的记录中，有多少是真的”；**召回率**问“真实发生的该事件中，有多少被找到了”。若接触失败很少，模型把多数片段都判正常就能获得高准确率，却可能漏掉训练最需要的失败信号。后面的数值例子用于说明这种区别。

构造例子：100 个片段中有 20 个接触失败，标签器检出 12 个并误报 3 个。精确率为 12/15=80%，召回为 12/20=60%；其余 77 个正常片段判断正确，总准确率为 89%。准确率看起来较高，却漏掉 40% 的关键失败。

7.5.2 人类视频能贡献什么，为什么不是机器人动作真值

人类视频提供对象、任务关系、运动顺序和操作视觉线索。它通常没有机器人的关节命令、控制频率、接触力和实际可达条件。

可以用视频预训练表示、识别子任务、估计对象运动，或构造与机器人数据对齐的中间目标。但将估计的人手轨迹作为机器人动作，需要尺度、相机姿态、工具、身体约束和控制接口映射。

对象位移可作为结果目标，却不能唯一决定产生它的动作。同样杯子移动十厘米，可以是推、抓、碰撞或桌面移动。逆向推断需要**先验**，视频预测准确也不证明动作因果识别完成。

合理做法是明确视频监督哪一种能力，再用机器人数据接通身体行为。实验可比较视觉预训练、伪动作标签与相同量真实示范，测各自在目标条件下的收益与成本。

7.5.3 合成数据的价值取决于标签与应用条件

仿真可生成图像、对象位姿、接触和动作标签，便于控制变化因素；也可以生成策略轨迹、失败案例或教师示范。程序得到的真值是在仿真定义内准确，不等于真实世界无误差。

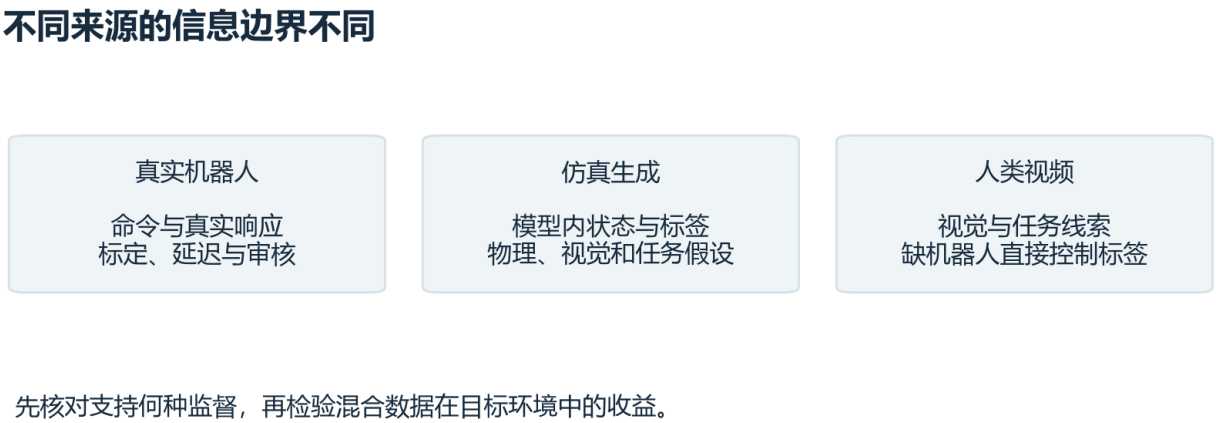


图7.4：不同来源支持不同监督与覆盖。混合前先核对语义，混合后在目标应用条件检验收益。

感知合成数据可能需要外观变化；动力学学习还需要响应和接触分布；策略模仿需要有效动作。渲染更逼真并不自动提高摩擦、延迟和控制器一致性。

教师可读取仿真完整状态，学生部署只读图像。这是允许的**蒸馏**信息差，但学生数据必须足以推断需要的行为；若同一图像对应隐藏状态下不同动作，教师标签仍会冲突。

仿真数据应保留生成器、资产、随机化、控制与标签规则的版本。真实测试不能使用为该测试逐项调过的仿真参数后再宣称零样本泛化。软件操作与迁移实验在仿真专题进一步展开。

7.5.4 伪标签要带着来源和置信边界进入训练

语言描述可能来自人工、模板或模型；动作伪标签可能来自逆动力学；未来视频可能来自生成器。它们提供的证据层次不同。自动标签的流畅和数量不能证明正确，模型也可能偏向常见目标而漏掉罕见恢复。

采用一小部分独立人工审核或传感验证，按阶段与失败类型测量错误，而不只测量总准确率。训练时可以按可靠性加权、隔离不同来源或逐阶段引入；评价应使用独立真实任务，避免用同一个标注模型既产标签又打分。合成数据有助于覆盖变化，但物理规律、语言和动作必须一致。

7.5.5 练习题

题7.5-1 给视频自动生成“成功抓起杯子”的文字标签。如何避免语言标注把不存在的事件写进训练？这种文字能否直接当动作监督？

7.5.6 参考答案

题7.5-1答案 “成功抓起”包含几个需要证据的事件：机械臂接触杯子、杯子离开支撑面，以及杯子被稳定带动。只看到夹爪闭合，或画面中的杯子短暂被遮挡，都不足以断定抓取成功。自动标注应检查一段时序证据，并保留对象身份和事件起止；无法看清时标为不确定或转人工，而不是补成肯定句。

可将视觉追踪、深度或机器人状态作为证据，按统一成功规则核验自动标签，抽查不同任务、遮挡和失败片段，统计误把失败写成成功的比例。标注置信度和来源也应保存，便于过滤和重标；用另一个相同偏差的语言模型复述，不等于获得独立证据。

文字能提供任务描述、阶段说明或成功评价标签，却不能直接替代关节/末端动作监督。“抓起杯子”没有说明从哪一姿态出发、往哪个方向移动、夹爪怎样闭合。行为克隆仍需与观测时间对齐的实际动作；若通过逆动力学从视频推测动作，必须另行说明模型和可辨识条件。过去事件描述和将来任务指令也应区分，避免把未来成功信息放进部署时不存在的当前输入。

7.6 数据集、存储格式与可复现数据工具

DROID 强调多场景真实机器人采集；Open X-Embodiment 整理跨机器人来源；UMI 提供便携采集与策略接口；robomimic 研究离线人类示范的算法与数据条件。它们对应不同目的，不应只按轨迹数量排序。DROID、Open X-Embodiment、UMI、robomimic 研究。

LeRobotDataset 提供机器人数据组织与工具接口。其 v3 格式将数据、视频与元数据关系显式组织，支持分片与索引；不能把旧版本按每回合单文件的路径假设无条件应用到 v3。LeRobotDataset v3。

读取新数据集先查 schema，即字段定义，再查样本、动作语义、时间、终止、来源和拆分。用几十条记录做可视化与检查，比直接启动大训练更能发现不兼容。

格式统一方便读取，不保证标签含义统一。数据集作者的成功定义、相机裁剪与采样策略也要记录。把官方格式读通，只完成了数据使用的第一步。

7.6.1 存储边界怎样变成训练窗口边界

LeRobotDataset v3 将多条回合放入较大的数据和视频文件，通过元数据索引每条回合的位置。这样减少小文件开销，但读取连续几行不一定仍在同一回合。文件边界负责存储，回合边界负责学习语义。LeRobotDataset v3 教程

假设某回合在控制步 0、1、2 记录了三个动作，步 2 后已经结束。训练时从步 1 取长度为 4 的动作块，合法标签只有 a_1, a_2；后面两项需要填充，并提供有效掩码 $[1, 1, 0, 0]$。即使数据文件接下来紧接另一回合的 a_0，它也不是这条动作块的未来。

图像历史有相似问题。从新回合的第一步读取两个历史观测时，前一条数据记录可能属于上一回合。可以重复本回合首帧或使用缺失标记，具体处理由模型输入约定决定；都不应把上一回合末帧当作当前任务历史。

时间偏移还需与采样频率配合。50 Hz 数据中的 0.04 秒是两步，20 Hz 数据中却不是一个整数步。数据加载器可能采用最近采样点并设置容差，动作重采样则另需保持其物理意义。先画出一条完整回合的图像、动作与标签，再检查窗口首尾，可以同时发现存储索引、时间偏移和有效掩码的错误。

7.6.2 练习题

题7.6-1 为什么只保存模型权重和原始视频不足以复现 **VLA** 训练？列出从原数据到训练样本中必须保留的变换。

7.6.3 参考答案

题7.6-1答案 权重只保存训练后的函数，原始视频只保存部分输入；中间怎样把一段机器人经历变成一个监督样本仍然未知。例如同一帧可以配“现在发出的绝对关节目标”，也可以配“0.1 s 后开始执行的末端增量”，两种训练并不相同。

从原数据到训练样本，至少需保留：原始状态和动作记录及其语义；时钟校正和消息对齐；视频解码、抽帧与多相机选择；坐标变换、机器人重定向和动作格式转换；归一化统计及其计算数据；图像裁剪、缩放、颜色处理与随机增强；历史长度、动作块长度、采样步长和回合边界填充；语言和成功标签来源；过滤、去重及质量规则；训练/验证拆分、数据版本和样本清单；来源混合概率、采样器与随机种子。

这些变换的次序也要保存。例如先转相对动作再归一化，与先归一化绝对目标再相减，不是同一计算。统计量只能按既定训练划分估计，不能利用测试集。还需保存训练配置、代码与依赖版本、优化器和调度状态，才能继续或尽可能复现训练；数据预处理记录解决的是“实际训练了哪些输入与标签”这一层。

7.7 拆分泄漏、统计单位与公平实验

7.7.1 拆分、去重与信息泄漏

信息泄漏是训练或模型选择得到本应仅供最终评价的信息。相邻帧跨集合、同一轨迹多个裁剪跨集合、同一物体扫描跨集合，都可能让所谓泛化测试过于容易。

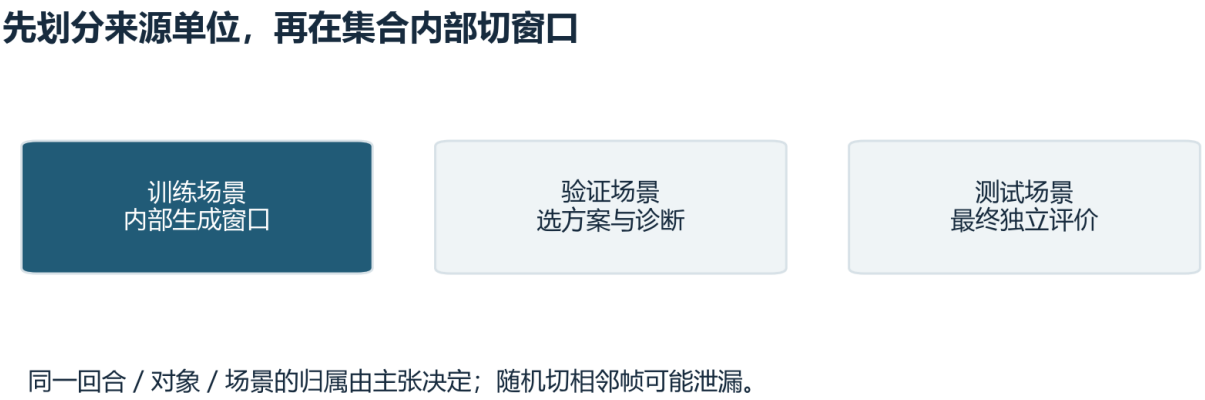


图7.5：先按回合、对象或场景等主张相关单位划分，再在集合内部构造窗口。不同拆分检验不同能力。

验证集用于选择超参数和采集方向，测试集用于最终评价。训练统计、数据筛选模型及标签器是否访问测试来源，也需要记录。一个已在测试场景训练的标签器可能间接带入额外信息。

去重不仅比较文件哈希，还可能比较回合来源、连续时间和近似视觉内容。阈值应结合任务，不能把不同动作结果但图像相近的数据全部删除。

测新场景时可以保留相同对象；测新对象时必须按对象身份划分。二者都有意义，但结论不能交换。报告拆分单位与规则，比一句“随机 80/20 划分”更有解释力。

7.7.2 成功率、统计单位与基准选择

成功定义应有可观测检验：正确对象、正确位置、稳定保持和允许重试等。某些任务只要求到达，另一些要求交接或精细插入，不能共享模糊的成功标签。

两任务 A 成功 90/100，B 成功 1/10。按全部试验合并的微平均为 91/110≈82.7%；按任务等权的宏平均为 (0.9+0.1)/2=50%。它们并非谁算错，而是给任务的权重不同。

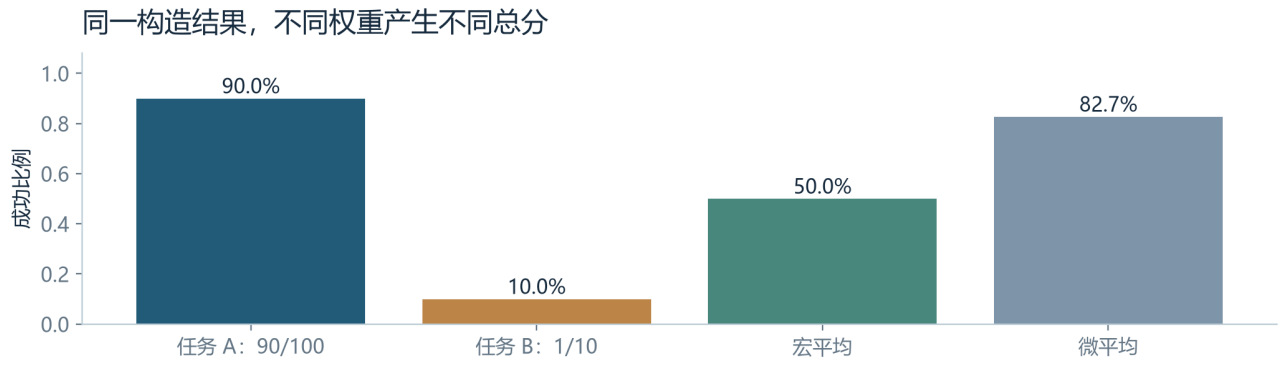


图7.6：样本多的任务会主导微平均。宏平均回答各任务等权时的效果，口径应与目标一致。

独立二项试验下，p=0.8、n=100 的标准误近似为 $\sqrt{p(1-p)/n} = 0.04$。这不是完整置信区间，也不意味着重复同一环境一百次都是独立证据。场景、对象和随机种子相关时，可按相应组重复或重采样估计不确定性。

LIBERO 提供受控迁移任务，真实机器人测试检验真实接口和条件。基准选用应服务问题定义，不因为某套 benchmark 易于跑通就宣称覆盖全部具身能力。LIBERO。

7.7.3 怎样估计某一批数据的贡献

最直接的实验是固定模型与训练预算，比较加入该数据前后的目标测试效果。需要控制有效训练量、采样配方、随机性和其他数据，否则差异可能只是更多梯度更新。

可以比较相同数量的随机补充、按覆盖选择、按失败选择和新评分选择。数据价值依已有数据、模型和任务变化，不是每条记录永远固定的分数。

若想隔离覆盖与数量，可以同时做固定回合数、固定总有效动作数和固定采集成本的对照。每种口径回答不同问题。删除一批数据导致效果变化，也可能是重新训练噪声，需要重复与分层分析。

大型数据价值方法可近似分配边际贡献，但代价、模型依赖和相互作用都很强。研究生可先做清楚的分组增量实验，比直接套复杂评分更容易得到可靠结论。

7.7.4 成功率的分母是什么

成功 70 次、试验 120 次，成功率点估计为 58.3%。若各次试验独立、具有相同成功概率，采用 95% 置信水平的 Wilson 方法，区间约为 49.4% 至 66.8%。点估计给出这批结果的比例，区间反映有限样本下的估计不确定性。这套计算针对规定的采样分布；换成更困难的物体或场景时，需要新的评价数据。

如果 120 次试验来自 12 个场景，每个场景重复 10 次，场景内结果可能高度相关。逐次试验重采样会把这些重复当成许多独立信息；**按场景重采样**则每次抽取整个场景的 10 次结果，保留组内关系，再计算成功率。配套构造例中，两种方法所得区间分别约为 49.2% 至 66.7%，以及 33.3% 至 83.3%。后者更宽，说明跨场景判断实际只有较少的独立场景支持。重采样单位应与泛化主张对应：跨物体按物体分组，跨场景按场景分组，训练偶然性则另外重复训练种子。

配对比较用相同物体、布局和重置条件运行两策略，先计算每组差值，再对组重采样，常比把两套结果当互不相关样本更清楚。记录完整失败和接管分母，避免只对成功片段计时。置信区间方法和重采样工具可参照 NIST 手册与 SciPy Bootstrap 文档。

7.7.5 归一化为什么也要留在训练集合内

按场景或回合拆分以后，还要先用训练集合计算动作均值、标准差和裁剪范围。它们虽然不直接是模型参数，却决定模型看到的数值坐标。例如训练动作是 $[0, 2]$，测试动作是 $[10]$；用训练集得到均值 1，用全数据却得到均值 4。后一种预处理已利用了待评价场景的动作分布。

这与“看了测试答案再更新策略”程度不同，但同样改变了实验的信息条件。在严格的留出评测中，模型选择、标准化和特征选择都应只拟合训练数据，再应用于验证和测试。scikit-learn 的划分与预处理教程

```
import numpy as np

train_actions = np.array([[0.0], [2.0]])
test_actions = np.array([[10.0]])
# 1. 只在训练集合拟合数值变换。
mean = train_actions.mean(axis=0)
std = np.maximum(train_actions.std(axis=0), 1e-6)
# 2. 同一个变换应用到不同集合。
train_normalized = (train_actions - mean) / std
test_normalized = (test_actions - mean) / std
# 3. 部署时沿用这组统计量，并按同一规则恢复动作。
restored = test_normalized * std + mean
```

机器人数据还要按动作定义分别计算统计量。两台机器人的第一个动作维度若一个表示关节角、另一个表示末端速度，它们不应因为索引相同就共享均值。统一语义、拆分数据、拟合预处理、构造窗口，是一条相互依赖的顺序。

7.7.6 练习题

题7.7-1 每个场景录了多条近似轨迹，随机按帧拆训练和测试会有什么泄漏？比较两个策略时为什么宜按相同场景配对统计？

7.7.7 参考答案

题7.7-1答案 相邻帧和同一场景的近似轨迹共享背景、物体布局和运动片段。随机按帧拆分，会让测试集包含训练中几乎已经见过的状态，甚至同一动作块的重叠窗口。模型记住固定布局也能取得低误差，这不能说明能适应新场景。

拆分单位应对应要验证的泛化：新轨迹就按整条轨迹分；新布局就把同一布局全部放在同一侧；新物体、操作者或房间则按这些组划分。划分之后再构造窗口，归一化统计也只用训练组。

比较两种策略时，在相同场景和可匹配的初始条件下分别测得 Y_{Ai}, Y_{Bi}，用差值 $D_i = Y_{Ai} - Y_{Bi}$ 比较。场景本身的难度同时影响两者，配对差值可以减小这种干扰：

$$\mathrm{Var}(D) = \mathrm{Var}(Y_A) + \mathrm{Var}(Y_B) - 2\,\mathrm{Cov}(Y_A, Y_B).$$

当相同场景使二者结果正相关时，差值通常比无配对比较更稳定。置信区间应以独立场景为重采样单位，并把两策略的同一场景一起抽走；不能把数千帧当数千次独立实验。真实实验还应随机或交替执行顺序，避免电量和物体磨损偏向一方。

7.8 主动采集、数据贡献与质量闭环

7.8.1 数据闭环怎样避免只收模型已经会的东西

数据闭环是采集、训练、评估、分析失败，再决定下一批采集。反馈应指向条件和机制，例如“遮挡后认错杯子”“夹空后不会恢复”，而非统一要求增加数据量。

主动学习选择希望获取标签的数据；主动交互可能通过机器人行动获得新的观测和结果。模型分歧、不确定性、稀有条件或任务损失都可作选择依据，但每种评分都有偏差。

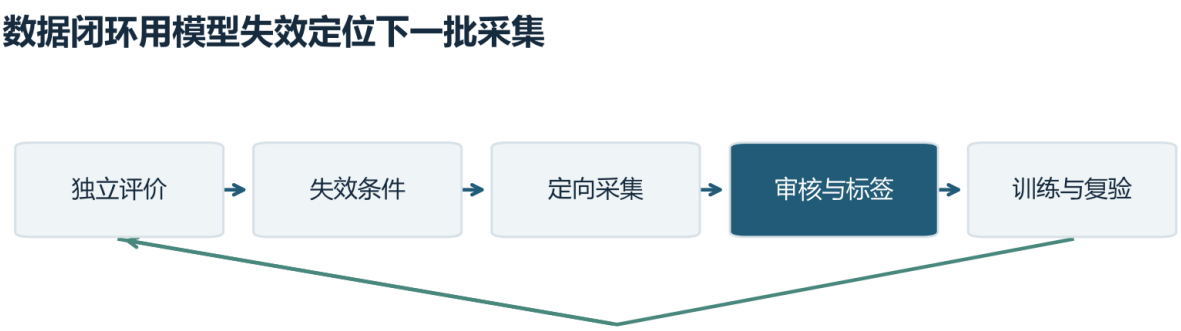


图7.7：独立评价产生可解释的失效条件，采集补缺，训练后重新验证。评价集不进入采集训练答案。

一个样本价值可以概念性地看作“预计减少目标错误，除以采集与审核成本”。实际减少量未知，需要通过试采、对照和后续模型效果估计。它不是一个能直接从单帧准确读取的真值。

对外观新颖但与任务无关的样本，纯特征距离可能高估价值；对模型错误自信的样本，纯不确定性又可能遗漏。可结合分层覆盖、失败记录和人工抽查。

固定测试集用于判断改进，不能反复把其具体失败回合直接变成训练样本后仍用同一测试分数证明泛化。可以利用验证集反馈设计采集，另留真正独立测试。

7.8.2 放杯数据闭环的采集与验证方案

教学预算为八个场景，每场景 25 回合，共 200 回合；四场景训练、两场景验证、两场景测试。此处对象可以共享，所以这套拆分主要检验新场景，不能直接称为新对象泛化。

训练场景每组包含 20 次常规示范和 5 次恢复示范。具体比例不是最佳配方，只用于说明需要提前规划恢复条件。每次记录成功、失败原因、命令、反馈、时间与配置。

先试采并验证动作回放和时间链；再执行采集；按批抽审；根据 schema 检查边界和范围；在各集合内部切窗口；训练基线；验证失败类型；选择下一批补缺条件；最后用独立测试评价。

完整数据项目把记录、审核与训练收益接起来

教学预算：8 场景 × 25 回合；4/2/2 场景对应 100/50/50 回合。

图7.8：计划、记录、审核和模型效果共同构成数据项目。数据存储成功只是其中一步。

若发现侧边抓取失败，先判断是不是该条件缺失、标签错位或控制不可达。缺失则补相应示范；错位先修规则；不可达则需要改变任务或动作接口。不能把全部故障都转成“再采一百条”。

交付物包括数据版本、采集规范、字段说明、质量报告、拆分列表和模型对照结果。配套小程序只检查构造记录的边界与泄漏，不冒充真实采集报告。

7.8.3 研究问题与诊断路线

一个可研究的问题应连接数据机制与能力缺口。例如“在相同采集成本下，按恢复状态覆盖选择示范，是否比按模型不确定性选择更有效减少接近偏差后的失败？”需要定义偏移、预算与恢复成功。

组合采集、标签和筛选技术可以有贡献，但要核对完整流程与已有工作的差异。只是改名称或多加一个分数项，不足以证明新机制必要。

诊断时先检查记录与标签，再看覆盖与采样，随后看模型是否能利用信息，最后看真实执行条件。训练误差低而闭环差，可能是策略访问分布问题；所有模型都在同一位置错，可能是标定或动作定义问题。

数据改进应留下可复现规则和成本。成功案例之外，报告被过滤的困难条件、自动标签错误和未解决的覆盖空缺，才能形成可信的后续路线。

7.8.4 主动采集应选择能够检验假设的数据

若失败集中在杯子遮挡，可增加新视角或遮挡布局；若集中在接触后滑移，可改变负载、摩擦并记录力；若换指令无效，应构造相同场景不同目标。每次采集对应一个机制假设，训练前写下预期改善的子指标，再用未用于选择数据的条件验证。

一个闭环包括失败分类、提出假设、选数据、审计标签、固定训练预算、独立评价和更新覆盖表。数据贡献实验不能只比较“旧小集”和“新大集”；应加入相同数量随机新增数据或重复训练的对照，判断精心采集是否确实比单纯增加数量有效。

7.8.5 练习题

题7.8-1 下一轮采集应该选择模型最不确定的状态，还是最容易失败的状态？怎样避免主动采集只积累无用难例？

7.8.6 参考答案

题7.8-1答案 两种信号都可用，但各自可能选错目标。最不确定的状态可能只是严重遮挡，增加相同视角的数据也无法确定动作；最容易失败的状态可能已经物理不可恢复，专家也给不出有效示范。相反，模型很有把握地把新杯柄识别错，可能会频繁失败，却没有很高的不确定性。

采集价值应结合：状态在目标任务中出现的频率、失败后果、数据是否缺少、专家能否提供有效纠正，以及新样本是否与已有样本重复。例如夹偏后仍能重新抓取，且该状态经常出现，适合采恢复示范；完全超出工作空间的目标可用于学习拒绝或重规划，不宜无休止地采“成功抓取”。

可先把闭环失败按原因分组，再在每组里选有代表性的状态，保留一定比例常规成功和随机覆盖，避免全部预算集中到少量奇异难例。将新增数据与等成本随机采集做对照，使用固定、独立的评测集看对应失败类和总任务成功是否改善。若只增加了训练集上难例的拟合，而部署分布没有改善，应改采集方向。主动采集的目标是获得可减少实际错误的信息，困难或不确定本身并不是最终价值。

第8章：任务规划、运动规划与技能执行

长任务不仅需要一个局部动作，还需要选择步骤、满足前提、规划运动并处理失败。本章把符号任务规划、几何搜索与分层策略连接起来。

8.1 任务状态、前提效果与搜索

8.1.1 规划从“想要什么”走向“哪些步骤可执行”

“把柜子里的红杯放到托盘”需要定位柜门、打开、观察、抓取、取出、放置和验证。每一步有前提，完成后改变后续可行性。策略可以学习整段行为，也可以与任务规划、技能和运动规划组合。选择这种组合的动机应是任务结构、可靠性和恢复需求，而不是默认模块越多越好。

8.1.2 任务、运动与控制分别决定什么

任务层决定先打开柜门再抓杯子，运动层决定机械臂怎样绕过柜壁到达把手，控制层让关节按参考运动并处理实际反馈。学习策略可以替代或辅助其中某些环节，但并不因此消除其他层需要解决的约束。

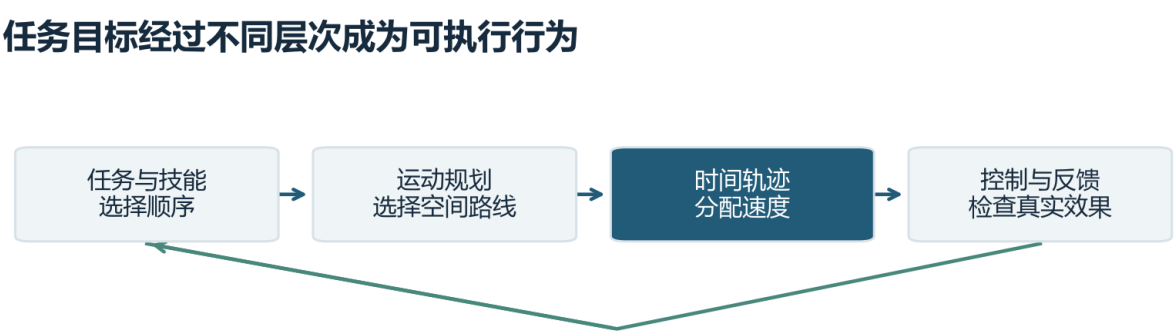


图8.1：不同层输出不同对象。上层目标需要经过可行性检验，下层反馈可以要求上层重规划。

计划（plan）是一组预期行动安排；路径（path）描述空间经过哪里；轨迹（trajectory）还包含时间关系。一个看起来通畅的路径，若速度或加速度不合理，仍不能执行。

规划不一定训练网络。A* 在图上搜索路径；**快速探索随机树**（Rapidly-exploring Random Tree，RRT）通过随机采样扩展配置空间中的搜索树；**轨迹优化**直接修改当前运动变量。这些计算通常在运行时完成。策略训练则改变跨任务使用的参数。二者可以组合：模型给候选，规划检验与细化。

本书的目标是让你能解释“模型知道要抓哪里”到“机器人真正完成一段任务”之间，还需要哪些信息、约束和反馈。

8.1.3 规划状态为什么同时有符号和数值

任务层可记录“柜门关闭”“杯子在柜内”“夹爪空闲”等事实；运动层需要对象位置、关节姿态与碰撞几何；执行层还关心时间、接触和估计可信度。它们描述同一场景的不同精度。

符号化把复杂观测压缩成便于推理的事实。例如 holding(cup) 表示夹爪正持杯子，但它必须有观测依据，不能只是因为刚执行了抓取就自动设为真。

观测不充分时需要保留未知或置信信息。未看到杯子不等于杯子不存在，没检测到接触不等于没有接触。把未知强制改成假，会让计划建立在错误确定性上。

规划使用的地图、位姿和对象状态都有时刻。动态人或被移动杯子需要更新。一个状态缓存接口应说明何时有效、哪个模块更新，以及冲突时如何处理。

符号模型使搜索更容易，但丢掉的几何和物理信息需要在后续可行性检验中补回来。

8.1.4 动作前提与效果是建模承诺

一个任务动作可以包含名称、参数、前提、效果和代价。“抓杯子”可能要求夹爪空闲、杯子可接近；成功效果是夹爪持杯、杯子离开支撑面。执行中失败，则不能按成功效果更新状态。

前提（precondition）决定何时可尝试动作；效果（effect）描述动作成功后改变什么。效果是规划模型的预测，真实反馈才决定它有没有发生。

例如“开门”需要把手可达，但门的铰链约束决定有效运动。只把状态由 closed 改为 open，而没有可执行的技能和几何检验，只完成了符号推理。

动作可以增加或删除事实。夹起杯子后，“杯子在桌上”应失效；机器人持杯子移动时，碰撞几何也变化。忽略删除和附着状态，容易让规划同时相信互相矛盾的事实。

建模应从任务的关键因果变化出发。过粗会漏约束，过细会使状态与搜索复杂。可以通过失败记录逐步增加必要前提，但不能把每个训练场景编号都写成规则来伪装泛化。

8.1.5 任务搜索为何需要代价与启发式

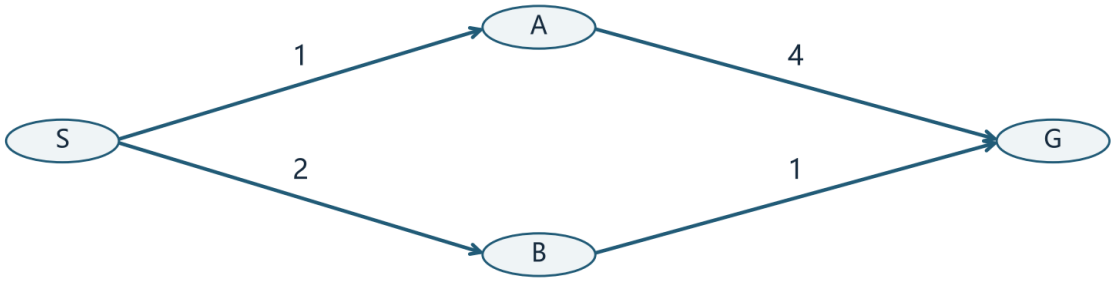
把状态看成节点，合法动作看成边，规划就是找从当前状态到目标的路径。边可以带时间、资源或风险代价。最短动作数与最快完成不一定一致。

A* 使用 $f(n) = g(n) + h(n)$：g 是到节点 n 已付代价，h 是到目标剩余代价估计。h 用来优先扩展看起来有希望的节点，而不是替代真实代价检验。

若 $h(n)$ 不高估从 n 到目标的真实最低代价，称为**可采纳启发式**。A* 因而有依据地优先搜索较有希望的方向，同时保留最优解的机会。更强的一致性条件为 $h(n) \le c(n, n') + h(n')$：沿一条边前进后，估计总成本不会突然下降。采用一致启发式时，标准图搜索可避免某些重复处理；若只有可采纳性而不一致，发现更低代价时可能需要重开已关闭节点。最优性仍依赖正确边代价、搜索规则和预算，无法直接转成真实机器人的任务保证。

教学图有 S→A 代价 1、A→G 代价 4，S→B 代价 2、B→G 代价 1。经过 A 总成本 5，经过 B 为 3。若 h(A)=4、h(B)=1，则 f 分别为 5 和 3；只按第一步代价贪心会更偏向 A。

教学搜索图：更便宜的第一步未必更便宜到达目标



经 A 总成本 5，经 B 总成本 3；A* 比较已付 g 与剩余估计 h。

图8.2：候选顺序由已付与预计剩余成本共同决定。启发式是否合理，要联系具体代价单位与约束。

真实任务搜索还要调用连续几何检查，某条符号边可能不可执行。后文 **任务与运动规划**（Task and Motion Planning，TAMP）联合考虑任务步骤与连续运动可行性，处理这类耦合。

8.1.6 A* 的启发式为什么要低估剩余代价

启发式的作用是给未完成路线一个有用的下界。仍用前面的两条路线：$S \to A \to G$ 总代价 5，$S \to B \to G$ 总代价 3。如果把 B 的剩余代价错估成 10，先扩展 S 后便有 $f(A) = 1 + 4 = 5$、$f(B) = 2 + 10 = 12$。搜索会先经 A 找到代价 5 的目标；若按“目标从队列弹出就结束”的规则，它会提前停下，错过代价 3 的路线。

采用可采纳启发式时，最优路线上的某个前沿节点满足 $g + h \le C^\star$，其中 $C^\star$ 是真正的最优代价。一个代价更大的目标不应先于这个节点被弹出。正确处理节点的更低代价更新后，这个下界关系便构成 A* 最优性的核心理由。Modern Robotics 图搜索教程、CMU 的 A* 推导课程

令 $h = 0$ 也满足条件，但失去了方向提示，退化为按累计代价展开的 Dijkstra 搜索。让 h 更接近真实剩余代价，可以减少无关扩展；直接把它设为精确剩余代价，又可能需要先解决同样困难的规划问题。设计启发式是在便宜、有效和下界性质之间取舍。

上述保证针对已建好的图。机器人使用错误碰撞模型、过时地图或无法执行的边时，图上的最优路径仍可能在现实中失败。图搜索与物理可行性检查因此需要相互配合。

8.1.7 练习题

题8.1-1 “打开柜门→拿杯→放托盘”中，为什么任务规划需要前提与效果，而不只需要动作名称？给出一个执行后必须重新确认的条件。

8.1.8 参考答案

题8.1-1答案 动作名称只说明打算做什么，前提和效果说明它在什么状态下可执行，以及执行后哪些状态会变化。可把本题简化成以下规则：

- 打开柜门：前提是柜门可达、把手可操作；预期效果是柜内取物空间开放。

- 拿杯：前提是柜门开度足够、目标已定位、夹爪空闲且有可行抓取；预期效果是杯子在夹爪中，原位置没有杯子。
- 放托盘：前提是杯子确实被持有、托盘可达且有空位；预期效果是杯子稳定落在托盘内，夹爪空闲。

这些规则让搜索器知道“拿杯”的需要可由“打开柜门”的效果满足，也能拒绝在夹爪没有杯子时直接放置。实际物理执行可能失败，所以预期效果不能被直接写成已确认事实。

例如开门技能返回之后，应重新观察门开度与取杯通道是否足够，而不只确认电机停止了。门弹回或只打开一半时，“柜内可达”仍不成立；高层应继续开门或修改抓取路线。前提和效果用于组织计划，执行后的观测用于判断计划假设是否真实成立。

8.2 技能接口、行为树与语言可执行性

8.2.1 技能应有可检查的接口

技能（skill）是有具体目标和执行边界的一段行为，例如接近把手、开门、抓杯、放杯。它可以由程序、轨迹规划、学习策略或组合实现。

技能接口至少说明输入参数、启动条件、执行输出、成功判据、失败原因和停止方式。参数可以是目标对象与位姿；成功可能由对象移动、夹爪接触和支撑关系共同判断。

训练技能策略时，输入观测与技能目标，标签或奖励围绕该技能结果；执行时调用已训练参数并根据反馈决定是否结束。技能名称本身不提供能力，文本“抓取”必须对应实际可调用的实现。

模块化技能便于诊断和复用，但边界可能不适合需要持续协调的任务。双臂搬运和接触插入若切得过碎，可能丢掉重要联动。技能长度与参数化应服务行为规律。

高层选择正确技能，而技能本身成功率不足，整段任务仍失败。应分别检验选择和执行，不用一个总分掩盖责任边界。

8.2.2 状态机与行为树如何管理执行

有限状态机（Finite State Machine，FSM）用有限模式和转换条件组织流程，例如寻找→接近→抓取→验证→放置；失败进入恢复。转换条件来自反馈，不只是计时器到点。

行为树（Behavior Tree，**BT**）用层次节点组织条件与行动。节点通常返回成功、失败或运行中。顺序节点按规则推进，选择节点尝试替代分支；响应式形式可以重新检查条件。BehaviorTree.CPP。

图8.3：执行有“运行中”状态；成功与失败必须通过条件确认。恢复分支也需要边界与退出规则。

如果抓取返回运行中，上层不能立即执行放置；如果失败，应先判断杯子位置和夹爪状态，再决定重试。中断技能还需要清理控制命令与共享状态，避免旧动作继续执行。

树结构便于组合，状态机便于明确模式，二者都不能自动提供物理可行性或任务推理。并行节点也不意味着任意技能能同时使用同一机械臂。

8.2.3 语言规划怎样与可执行能力连接

语言模型可以提出技能序列，但文字合理不等于机器人具备对应能力。规划应限制到已定义技能、合法参数与已知状态，并用几何或经验成功评估筛选。

SayCan 将语言相关性与技能可执行性结合，是这种分工的代表入口。具体评分与系统实现依原文，本节用最小算例解释目的。SayCan。

假设杯子还在关闭的柜子里。候选 A 是“抓杯子”，语言相关分数为 0.8，但当前可执行分数仅为 0.2；候选 B 是“打开柜门”，相应分数为 0.4 与 0.7。乘积分别为 0.16、0.28，于是优先执行 B。例子说明，高层不仅要选择贴近最终指令的动作，还要选择当前条件下能推进任务的动作。这些分数用于候选排序；未经校准或额外概率建模，不能把乘积直接称为整段任务成功概率。

可执行分数可以来自几何检查、经验统计或网络。若训练成功预测网络，输入为当前可用观测、候选技能及参数，输出尝试成功的概率；标签来自实际或仿真尝试是否达成技能目标，二元**交叉熵**更新预测网络。它评价候选而不直接控制机器人；数据只包含容易条件时，分数在困难条件下可能错误自信。

成功率还依赖具体技能策略与执行条件。技能版本改变后，旧标签和分数未必仍代表当前能力，应记录实现版本并重新校验。

参数解析后还需要验证对象是否存在、坐标是否有效、技能是否满足条件。失败反馈可要求修改顺序、换技能或重新观察，不应只要求语言模型再生成一段更流畅文字。

评价语言规划要控制底层技能库和感知能力。新系统使用更强技能，成功率上升不能全部归于推理提示。

8.2.4 一次 tick 到底检查什么

行为树通过周期性 **tick** 驱动节点：tick 是一次询问“这项条件或行动现在处于什么状态”，不是要求长动作在这次函数调用内全部完成。抓取节点第一次 tick 可以发送命令并返回 RUNNING；后续 tick 检查反馈，成功才返回 SUCCESS，失败则返回 FAILURE。BehaviorTree.CPP 的异步与响应式序列教程

设一个响应式顺序分支为“通道畅通→移动到杯旁”。其过程可以读成：

时刻	通道条件	移动节点	分支结果
第一次 tick	成立	启动运动，尚未到达	RUNNING
第二次 tick	仍成立	查询反馈，仍在移动	RUNNING
第三次 tick	被人挡住	调用停止，取消正在运行的运动	FAILURE

响应式序列会重新检查前面的条件。普通序列在子动作运行中通常继续处理该动作，具体推进规则取决于节点类型；选错类型，可能让“开始时畅通”被当成“整个移动过程始终畅通”。这就是同样树形连接下，节点执行语义仍然重要的原因。

```
移动节点首次启动：发送带目标的运动请求，保存请求标识，返回 RUNNING
移动节点继续运行：检查该请求的反馈；未结束则返回 RUNNING
    到达并满足目标条件 → SUCCESS
    执行失败或目标失效 → FAILURE
移动节点被中断：取消请求，停止继续提交旧命令，清理本次执行状态
```

停止不能只把节点的返回值改成失败。执行器若仍按旧目标运动，树已经切换分支，机器人却还在执行上一项行动。节点状态与物理请求的生命周期必须一致，行为树才能可靠组织长任务与恢复。

8.2.5 练习题

题8.2-1 语言模型输出“抓住杯子”，技能系统应怎样接收它？如果技能返回完成而杯子仍在桌上，行为树或高层状态机应怎样处理？

8.2.6 参考答案

题8.2-1答案 应先将“抓住杯子”落成已注册技能及其具体参数，例如 Grab(target_id=cup_3, grasp_pose=当前杯柄对应姿态)。技能接收目标身份、参考坐标系和位姿、动作限制、超时等明确输入，检查夹爪是否空闲、目标是否可达，再由底层控制器执行。运行中返回进行中、完成或失败，并附实际进度和原因。语言模型负责提出任务，不能用一句文字替代执行接口。

如果底层报告完成而杯子还在桌上，说明它的“完成”可能只是命令执行完、夹爪闭合，不能等同于抓取成功。应把抓取后的验证作为独立条件，例如杯子离开桌面并随夹爪运动，再结合夹爪位置或接触信号。

验证失败时，不进入搬运阶段；更新任务状态为未持杯，记录抓取失败，重新定位后重试、换抓取姿态，必要时请求帮助。行为树可把“抓取→验证持杯”放在同一序列，并给失败配置恢复分支；状态

机则仅在持杯条件成立时转到搬运。要限制重试次数并处理条件不确定，避免靠无休止重试掩盖错误。技能调用、执行完成和任务效果确认是三个环节。

8.3 配置空间、碰撞与采样规划：一棵树怎样找到绕行路径

VLA提出“抓住红杯”之后，系统还需要找到一段可以到达抓取姿态的运动。目标姿态回答的是“最后到哪里”，运动规划回答的是“从当前姿态怎样过去”。如果只让夹爪沿直线靠近杯子，肘部可能扫到柜壁；即使每个动作端点都没有碰撞，连接过程也可能穿过障碍。

本节先把整台机器人的姿态表示成搜索对象，再完整追踪一次带障碍的RRT搜索。最后比较RRT*怎样改进路径，以及PRM为什么适合重复查询。这些算法改变的是候选路径和搜索结构；本节的基本算法不需要训练网络。

8.3.1 搜索对象为什么是一套关节配置

配置（configuration）是确定机器人几何姿态所需的一组变量。固定基座的六关节机械臂可以用六个关节角组成的向量q表示；若底座也参与运动，还要加入底座位置与朝向。**配置空间**（Configuration Space，C-space）则是这些配置构成的集合。

配置与末端位置不是一一对应的。看图8.3-1中的平面二连杆，两根杆长都是1，末端都在$(1, 1)$。一种姿态的肘部在$(1, 0)$，第一根杆沿x轴经过障碍；另一种姿态的肘部在$(0, 1)$，手臂从另一侧伸到同一末端位置。只检查末端坐标，会把这两种情况混在一起。

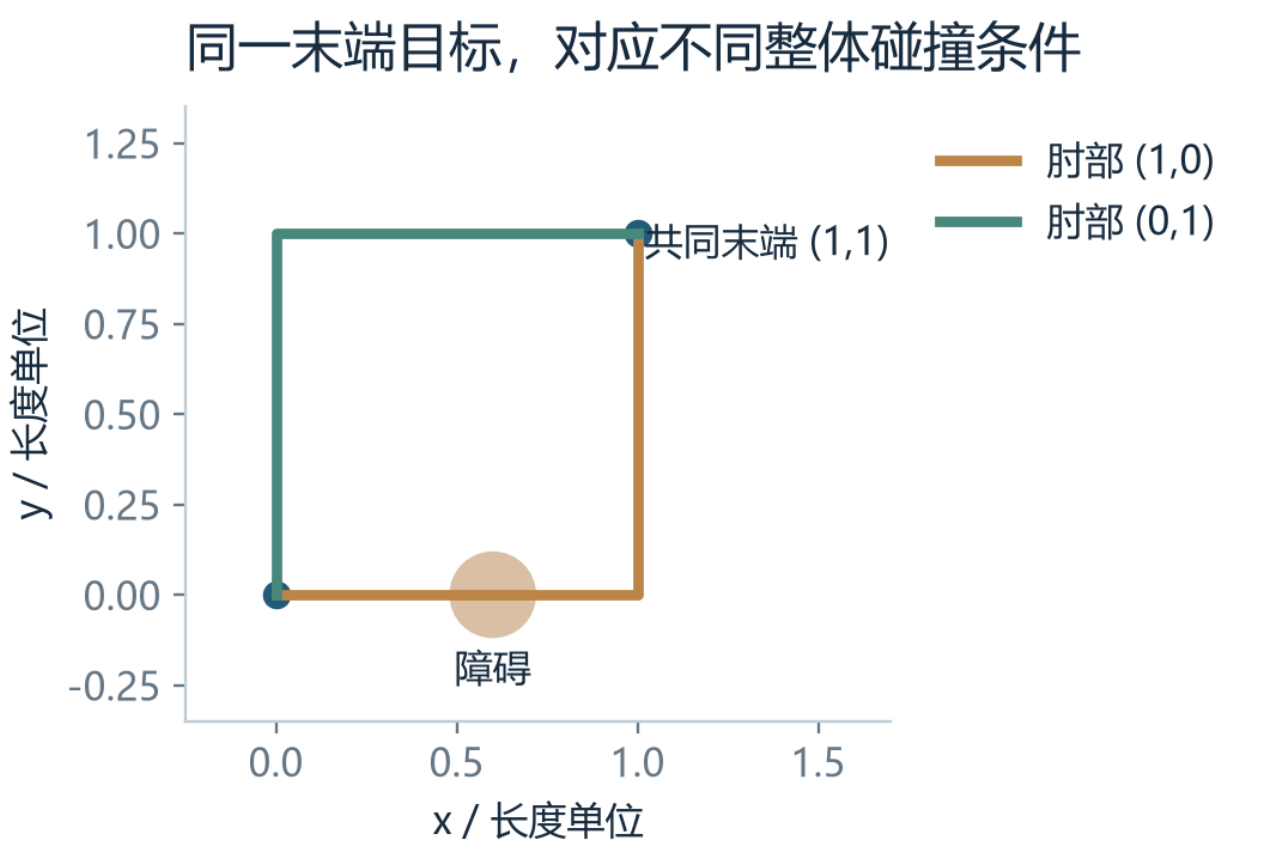


图8.3-1：末端位置相同，连杆经过的区域可以不同。棕色手臂的第一根杆穿过障碍；绿色手臂在图示配置下没有这种碰撞。图中的平面是机器人实际所在的工作空间。

对每个q，先用正向运动学得到各连杆的位置，再检查关节限位、自碰撞，以及连杆、工具和环境之间不允许发生的接触。通过这些检查的配置组成**自由空间**$\mathcal{C}_{\text{free}}$。配置空间中的一个障碍点，表示“机器人处于这套姿态时发生了不允许的碰撞”，并不是相机中的某个障碍像素。

哪些接触允许，需要随任务阶段改变。抓杯时夹爪需要接触杯子，手臂仍不能撞柜壁；抓住杯子之后，杯子随手臂运动，也应加入被检验的几何模型。否则夹爪虽然通过了门框，杯沿仍可能撞上去。MoveIt规划场景说明了环境对象与附着物体的管理方式。

碰撞模型通常使用便于计算的几何代理。模型过粗，可能把实际能通过的缝隙封住；过细，则增加检查代价。若对禁止接触的几何对预留间隔，可以令$d_{\min}(q)$为这些几何对在工作空间中的最小距离，并要求$d_{\min}(q) \geq \delta$。允许的夹爪—杯子接触按任务规则单独处理，不放进同一正距离约束；杯子—柜壁等仍需检查。物体位姿估计发生变化后，应更新模型或重新观察。

8.3.2 从规则网格转向逐步探索

在二维地图上划网格，再用A*搜索，是很自然的做法。但若六个关节各取100个位置，组合就有$100^6 = 10^{12}$个。空间中还有大量不能到达或会碰撞的配置，没有必要先把它们全部列出来。

RRT（Rapidly-exploring Random Tree，快速探索随机树）采用另一种组织方式：从起点长出一棵树，逐次尝试新的局部连接，只为实际探索到的地方保存节点和边。一个节点是一套配置；一条边是

一段由局部规划器给出的运动。本节先采用欧氏空间中的直线连接，后面再说明机器人约束会怎样改变它。

树的每个新节点都接到一个已有节点。因此，只要起点合法、每条新增边合法，新节点就有一条沿树返回起点的合法路径。算法不必在每次加入节点后重新搜索整棵图，只需要保存新节点的**父节点**：它是通过哪个旧节点到达这里的。

采样的目的，是不断提出值得尝试的方向。它没有直接给出解，也不会让障碍检查变得多余。窄通道可能很少被采到，局部连接也可能反复失败。采样规划减少了预先穷举空间的负担，代价是有限预算下不一定找到路径。

8.3.3 一轮扩展：样本、最近节点与新候选

设树中已有节点$q_0, \dots, q_{n-1}$，本轮采到q_{rand}。先找距离它最近的已有节点：

$$i^* = \arg\min_{0 \le i < n} \rho(q_i, q_{\text{rand}}), \qquad q_{\text{near}} = q_{i^*}.$$

ρ是配置距离。在本节的二维点例子中，它就是欧氏距离；机械臂则需要考虑关节尺度、周期和限位。对能够连续转动的关节，179°与-179°按最短转动只差2°；有限位的关节则要服从实际允许的运动范围，不能一概跨越数值边界。

本例从整个有界配置域采样，不先要求样本无碰撞。样本只提供方向，最终是否加入由候选边检查决定；一个较远的障碍内样本，也可能先引出一小段尚未碰到障碍的扩展。

接着向样本走一段。下面的向量差与直线公式用于本节的二维欧氏例子；周期关节需要相容的距离和插值，不能按最短角距选节点，却用未经处理的原始角度差扩展。令$\eta > 0$为最大扩展长度，$d = \|q_{\text{rand}} - q_{\text{near}}\|_2$。当$d > 0$时，候选为

$$q_{\text{new}} = q_{\text{near}} + \frac{\min(\eta, d)}{d}(q_{\text{rand}} - q_{\text{near}}).$$

这里有两个决定。除以d，是把位移变成只表示方向的单位向量；乘$\min(\eta, d)$，是决定本轮沿该方向走多远。样本很远时只走η，样本很近时恰好到样本，不越过它。若$d = 0$，树中已经有这个位置，本轮跳过即可。

例如从$(0, 0)$向$(1, 1)$扩展$\eta = 0.2$：

$$q_{\text{new}} = (0, 0) + 0.2\frac{(1, 1)}{\sqrt{2}} \approx (0.141421, 0.141421).$$

q_{rand}只是提出方向，q_{new}才是准备加入的候选。候选还要通过整条连接边的检查，不能算完坐标就直接加入树。Modern Robotics的RRT讲解按采样、最近节点、局部连接、检查和回溯组织了这一过程。

为什么这种选择倾向于探索树外的区域？暂时考虑均匀采样。每个树节点都对应一片“到它最近”的区域，即它的**Voronoi区域**。采样落入哪片区域，就选中哪个节点。区域越大，被选中的概率越高。

例如单位正方形中只有$(0,0)$与$(0.2,0)$两个节点，二者距离相等的分界线是$x=0.1$。左侧面积为0.1，右侧为0.9，后一个节点因此更常被选中。与“随机挑一个已有节点，再随便长一下”相比，最近节点规则会给覆盖较少的方向更多机会。选中之后能否成功扩展，仍由障碍和局部连接决定。

还需要一个到达目标的办法。在连续空间里，仅等待随机数精确等于一个目标点不合适。可以偶尔直接采目标，称为**目标偏置**；也可以在新节点进入一定范围后尝试连接目标。本节同时使用这两种常见做法：随机模式有15%的概率采目标；新节点距离目标不超过η时，检查最后一条连接边。15%只是本程序的设置，并不是算法的固定常数。

8.3.4 两个合法端点之间，怎样判断整条边

现在给上述第一轮加入障碍：

$$0.06 \le x \le 0.09, \qquad 0.06 \le y \le 0.09.$$

起点$(0,0)$和候选$(0.141421, 0.141421)$都在障碍外，但连接它们的线段经过$(0.075, 0.075)$，所以这条边不合法。采用本节的规则，这次扩展被拒绝，树仍只有起点。

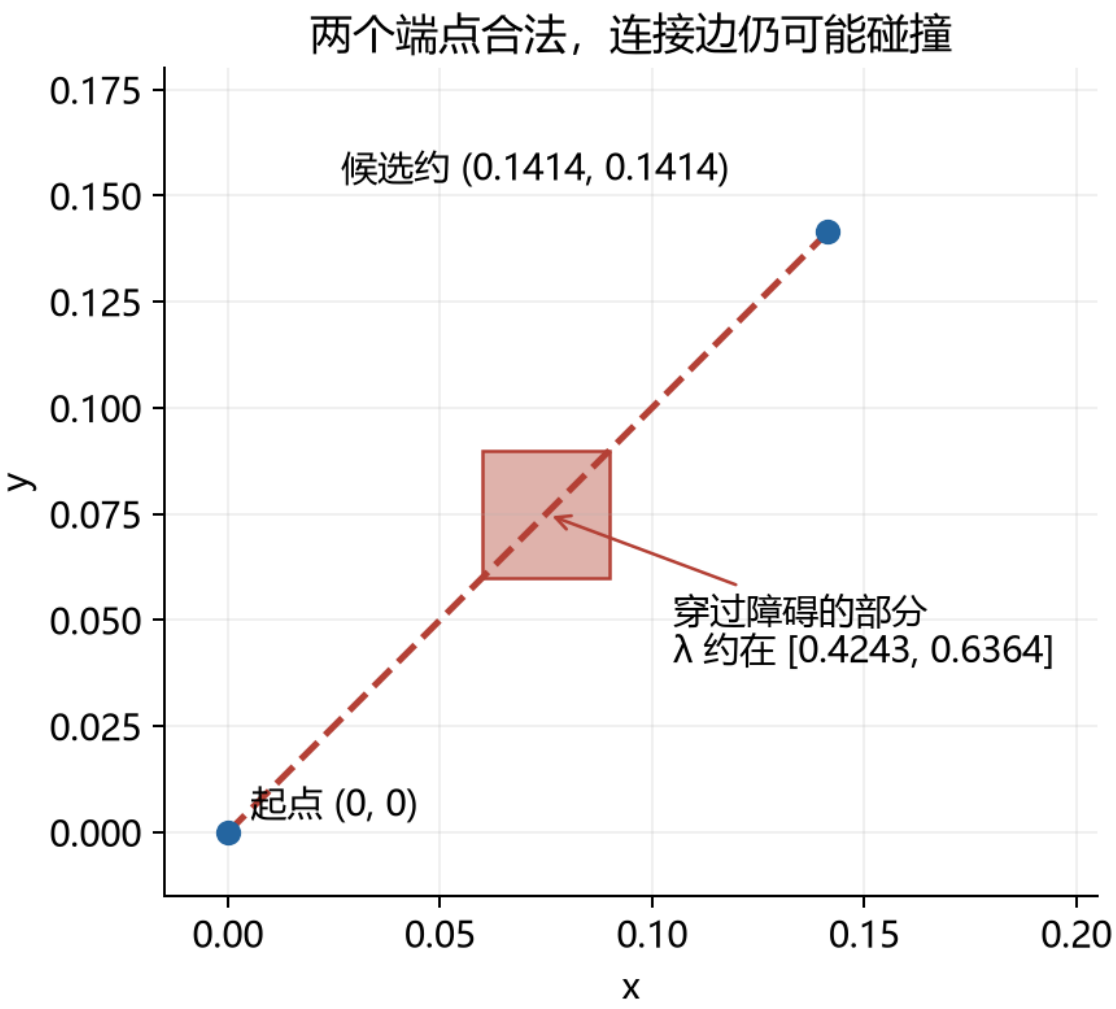


图8.3-2：红虚线表示本轮提出的边，灰红矩形表示障碍。端点检查会放过它；整段检查会发现中间的相交部分。

为了能复算碰撞判断，把线段写为

$$q(\lambda) = a + \lambda(b - a), \qquad 0 \le \lambda \le 1.$$

a是父节点，b是候选，λ表示在线段上走过的比例。对本例的x坐标，进入障碍要求

$$0.06 \le 0.141421\lambda \le 0.09 \quad \Longrightarrow \quad \lambda \in [0.424264, 0.636396].$$

y坐标给出相同区间。两个坐标同时在障碍范围内的比例区间非空，因此线段与障碍相交。

一般闭矩形也可以这样检查。对坐标$j \in \{x, y\}$，记边界为$[l_j, u_j]$、该方向位移为$d_j = b_j - a_j$。若$d_j \neq 0$，允许区间为

$$I_j = \left[\min\left(\frac{l_j - a_j}{d_j}, \frac{u_j - a_j}{d_j}\right), \max\left(\frac{l_j - a_j}{d_j}, \frac{u_j - a_j}{d_j}\right)\right].$$

方向反过来时，先碰到哪条边界也会反过来，因此用两个比例的较小值作为进入位置，较大值作为离开位置。若$d_j = 0$，而a_j在边界范围之外，这条线段就不可能进入矩形；若该坐标一直在范围内，它不再限制λ。

最后求$[0, 1] \cap I_x \cap I_y$。这里求交是因为两个坐标必须同时进入各自范围；交集非空即碰撞，恰好相切也算碰撞。配套程序的collision_interval实现了这个判断。浮点实现另将矩形边界向外扩展10^{-12}，避免反复加0.2的舍入误差把边界接触判成离开；这个数值容差与机器人实际需要的安全余量是两回事。

这同时解释了两个常被混用的参数。扩展长度η决定树一轮最多长多远；碰撞检查的分辨率决定沿途检查多密。把η设小，并不能自动保证不会漏过更薄的障碍。本例用线段与矩形的解析关系检查整段；复杂机器人常采用离散姿态检查或专门的连续碰撞方法。若只检查若干离散点，间隔过大就可能漏掉它们之间的碰撞。OMPL运动合法性文档明确区分了状态检查、运动检查与离散分辨率。

8.3.5 同一张地图上的15轮搜索

下面把上述反例接入完整搜索。机器人简化为二维点，配置$q = (x, y)$，因此本例中的二维平面也直接是配置空间。范围为$[0, 1]^2$，起点$q_0 = (0, 0)$，目标$q_G = (0.9, 0.2)$，最大扩展长度$\eta = 0.2$。除刚才的小障碍，再加入一块挡板：

$$0.38 \le x \le 0.62, \qquad 0 \le y \le 0.60.$$

挡板阻断了从左向右的低处通道，需要先向上绕行。障碍边界也视为碰撞。这里固定15个样本以便逐轮核对；这是展示算法决策的构造序列，随机模式会产生不同的探索顺序。

轮次	样本q_{rand}	最近节点编号	候选q_{new}	判断与树的变化
1	(1, 1)	0	(0.1414, 0.1414)	穿过小障碍，拒绝；仍为1节点
2	(1, 0)	0	(0.2, 0)	加入节点1，父节点0；共2节点
3	(1, 0)	1	(0.4, 0)	穿过挡板，拒绝；仍为2节点

轮次	样本q_{rand}	最近节点编号	候选q_{new}	判断与树的变化
4	(0.2, 1)	1	(0.2, 0.2)	加入节点2，父节点1
5	(0.2, 1)	2	(0.2, 0.4)	加入节点3，父节点2
6	(1, 0.4)	3	(0.4, 0.4)	穿过挡板，拒绝；仍为4节点
7	(0.2, 1)	3	(0.2, 0.6)	加入节点4，父节点3
8	(0.2, 1)	4	(0.2, 0.8)	加入节点5，父节点4
9	(0.1, 0.2)	2	(0.1, 0.2)	样本仅距0.1，直接到样本；加入节点6，父节点2
10	(1, 0.8)	5	(0.4, 0.8)	加入节点7，父节点5
11	(1, 0.8)	7	(0.6, 0.8)	加入节点8，父节点7
12	(1, 0.8)	8	(0.8, 0.8)	加入节点9，父节点8
13	(0.9, 0.2)	9	(0.8329, 0.6027)	加入节点10，父节点9；距目标约0.4083
14	(0.9, 0.2)	10	(0.8658, 0.4054)	加入节点11，父节点10；距目标约0.2083
15	(0.9, 0.2)	11	(0.8986, 0.2082)	加入节点12，再接入目标节点13；共14节点

看第3轮：候选横坐标到了0.4，连接边在$x = 0.38$之后进入挡板，因此被拒绝。再看第6轮：树已经长高了，但从$(0.2, 0.4)$向右仍会撞挡板。算法没有把失败方向当作可走路线，也没有因为目标在右边就强行向右移动。

第9轮显示树可以产生支路。此时最新加入的是节点5，但样本$(0.1, 0.2)$更靠近较早的节点2。因此新节点6接到节点2，而不是接到上一个加入的节点5。下一轮继续向右上方采样时，又从节点5扩展，支路6留在树中。

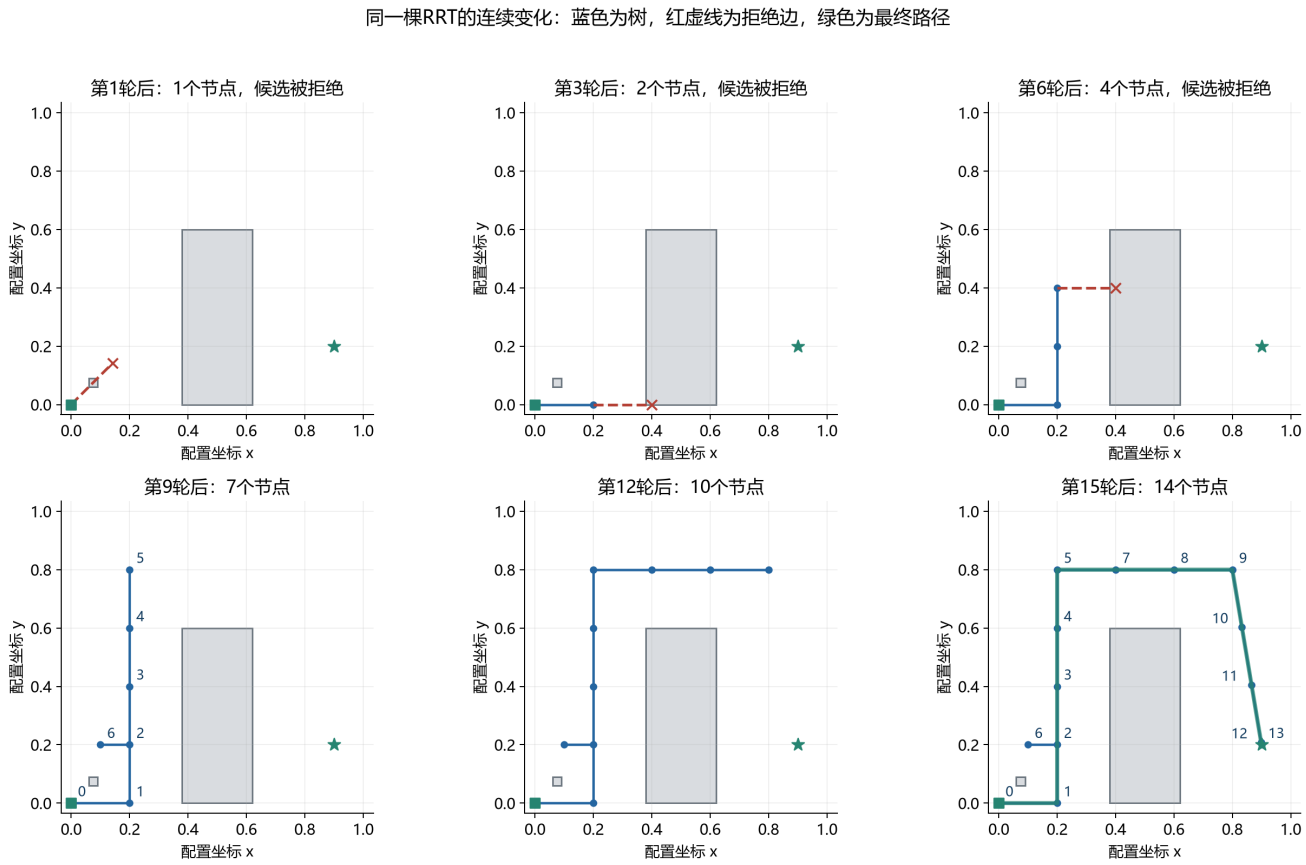


图8.3-3：同一次搜索的六个阶段。蓝线是已经加入的合法边，红虚线是当前被拒绝的候选边，末图绿色线是回溯得到的路径。小障碍在左下角，挡板位于中下部。

最后三轮为什么不能一口气到目标？第12轮末的节点9在$(0.8, 0.8)$，到目标的距离为

$$d = \sqrt{(0.9 - 0.8)^2 + (0.2 - 0.8)^2} = \sqrt{0.37} \approx 0.608276.$$

第13轮沿方向$(0.1, -0.6)$走0.2，得到$(0.832880, 0.602721)$；第14轮再走0.2。此时剩余距离约0.208276，仍大于连接阈值0.2，所以尚不连接目标。第15轮继续走0.2，剩余约0.008276；这最后一小段也通过碰撞检查，才把目标加入树。

若预算只有14轮，这个程序会报告预算耗尽。地图其实存在解，只是当前规则尚未完成最后的连接。这是“本次搜索失败”与“问题无解”的具体区别。

概率完备（probabilistic completeness）讨论的是另一件事：对满足相应假设的可行问题，随着随机采样预算趋于无穷，找到可行解的概率趋于1。几何RRT的分析通常需要一条与障碍保持正净空的可行路线、能够覆盖相关区域的采样，以及相容的局部连接与检查规则。直观上，有余量的路线可以被分成有限个相邻小区域；采样有正概率落入下一片区域，并使树继续推进。只有零宽度的缝隙，或者始终不会被采到的区域，都会破坏这个推理的前提。LaValle教材的完备性定义与几何RRT分析的第III节给出了相应定义和条件。

这是随机算法在给定条件下的性质。前面的固定序列用于检查运行流程；它既不是概率完备性的证明，也不提供“本次100毫秒一定找到路”的保证。

8.3.6 回溯父节点，得到可行路径

找到目标后，从节点13读取父节点12，再读取12的父节点11，如此返回起点：

$$13 \to 12 \to 11 \to 10 \to 9 \to 8 \to 7 \to 5 \to 4 \to 3 \to 2 \to 1 \to 0.$$

反转次序便得到从起点到目标的路径。节点6没有出现在其中：它证明搜索曾探索过一条支路，并不意味着机器人必须沿这条支路先走一趟。采样和建树发生在计算机中；机器人随后才执行选出的路线。

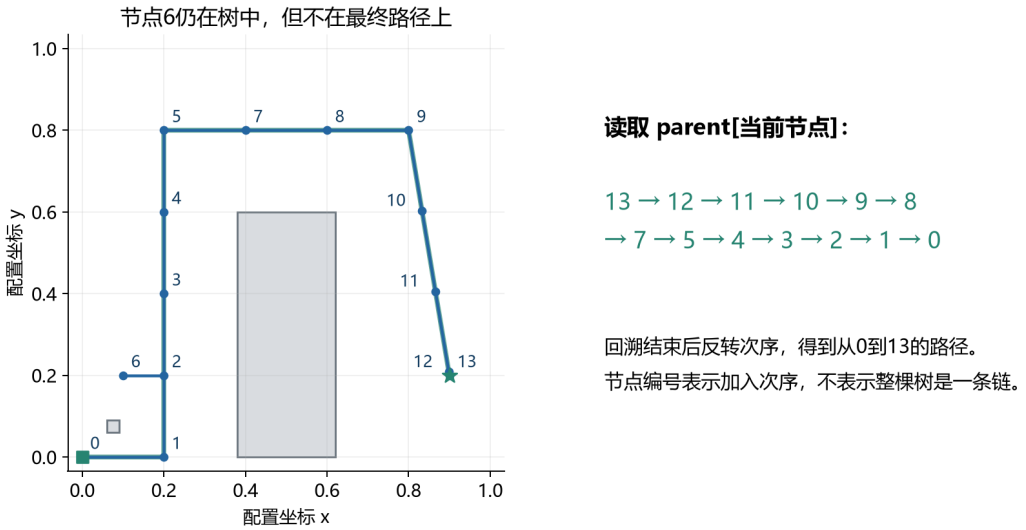


图8.3-4：左图保留整棵树，并用绿色标出最终路径；右图按读取父指针的顺序回溯。节点6的父节点是2，但最终目标的祖先链没有经过6。

路径包含13个节点，欧氏长度约为2.208276。这个数来自各段长度相加。它是一条可行路径的长度，不表示已经求得地图中的最短路径。RRT基本扩展时只选择最近节点，并未为每个候选比较所有可能路线的累计成本。

算法可写成下面的伪代码，完整Python程序中的注释使用相同的步骤编号：

```
节点表 = [合法起点]，父节点表 = [-1]
重复直到预算耗尽：
    1. 从配置域采样，偶尔直接采目标
    2. 在已有节点中找最近节点 q_near
    3. 朝样本走 min(最大扩展长度，当前距离)，得到候选
       若当前距离为零，跳过本轮
    4. 若 q_near 到候选的整段运动碰撞，拒绝；树不变
    5. 否则保存候选，并记录父节点为 q_near
    6. 若候选已到目标，或在连接范围内且最后一段合法：
            保存目标连接
            7. 从目标沿父节点回溯，再反转并返回起点到目标的路径
预算耗尽则返回本次未找到路径
```

本节采用“整条候选边失败就拒绝”的规则。另一些树扩展方法会保留碰撞前可走的部分；这会改变树的变化过程。LaValle的树探索教材展示了这种扩展，而OMPL的几何RRT实现可以看到整段检查后再入树的流程。学习和实现时应先固定这一规则，随后按同一规则核对算例。

8.3.7 RRT*：新增一个点，为什么能让旧路线变短

RRT*在探索之外增加了成本比较。令$g(v)$表示从起点沿当前树走到节点v的累计成本，$\ell(u,v)$表示连接边成本。在本例中，边成本就是欧氏长度。

第一件事是为新节点选父节点。在附近且能够合法连接的节点集合$\mathcal{N}$中，比较

$$g(u)+\ell(u,q_{\text{new}}),\qquad u\in\mathcal{N}.$$

边本身短，不代表整条路线短。设有一棵无障碍平面中的绕行树：

$$S=(0,0)\to A=(0,3)\to B=(3,3)\to C=(3,0)\to D=(4,0).$$

累计长度依次为$0,3,6,9,10$。现在候选$N=(2,0)$位于树附近；本次邻域半径取2.1，附近节点为S,C,D。所有列出的直线连接都可行。这是用于观察局部更新的给定树，不是在重复前面的单位正方形地图。

候选父节点	到该父节点的累计成本	父节点到N的长度	经它到N的总成本
S	0	2	2
C	9	1	10
D	10	2	12

最近节点是C，但应选择S作为N的父节点，因为经S只需2。于是$g(N)=2$。

第二件事是**重连**（rewiring）：检查附近旧节点，看看经新节点能否更便宜地到达。对C，新路线成本为

$$g(N)+\ell(N,C)=2+1=3<g(C)=9.$$

因此把C的父节点由B改为N。D原来是C的子节点，它的位置和连接边都未改变，但到它之前的路线变短了，所以累计成本也应更新为$g(D)=3+1=4$。若只改C的父指针，却仍保留D的旧成本10，后续比较就会使用错误账本。

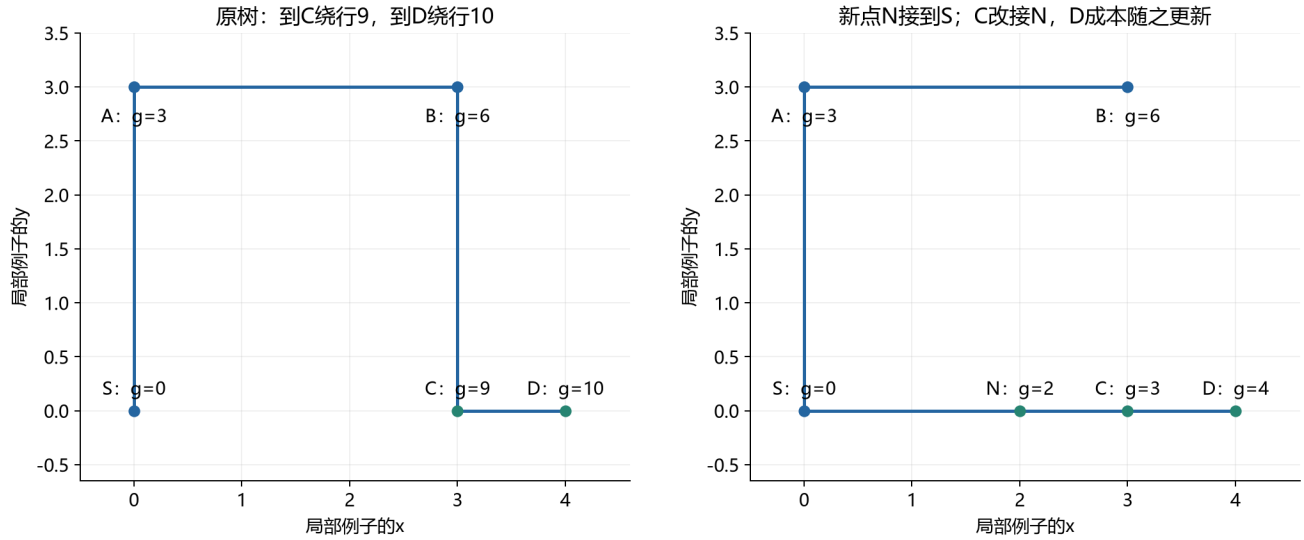


图8.3-5：右图中新节点N直接接到起点，C改接N。D仍是C的子节点，成本却从10降到4。本例按节点编号先检查C，再检查D；相同成本不再改父节点。

这两类比较分别改善“到新节点的路”和“利用新节点到旧节点的路”。每一次更便宜的连接仍需通过运动合法性检查；穿过障碍的捷径不参与更新。重连也要保持树无环，并更新受影响的子孙成本。OMPL RRTstar实现中的父节点选择、重连和updateChildCosts对应这三个环节。

漏掉子孙更新，有时会被当前例子掩盖：即使D还错误地存着10，后面检查邻居D时，程序也可能直接把它改接N，令成本变成4。再考虑原树中D已经有子节点$E=(4.5,0)$的情形：重连前$g(E)=10.5$，而E距N为2.5，不在2.1的邻域内。上游路线缩短后，它的成本应降到4.5，单靠遍历N附近的节点却访问不到它。局部个别节点的成本碰巧修复，不能代替沿受影响子树传播更新。

渐近最优（asymptotic optimality）要求：在满足相应采样、连接与问题条件时，随着预算不断增加，当前找到的最好可行路径成本以概率1趋于问题的最优成本。这比“最终找到某条可行路径”更强，也不保证当前限时结果已经最优。这里取2.1的固定邻域，只用于复算一次成本变化，不由此建立渐近最优保证。实际比较RRT与RRT*时，应同时报告成功率、计算预算、路径成本与检查口径，而不是只比较某一次返回路径的长度。

8.3.8 PRM：把建图代价用于多次查询

RRT通常围绕当前起点增长树；**PRM**（Probabilistic Roadmap，概率路线图）先收集自由配置并尝试局部连接，形成可复用的图。查询时，把新的起点与目标接入，再用图搜索找路线。环境相对稳定、需要多次规划时，预先建图的代价可以在多次查询中分摊。Modern Robotics的PRM讲解区分了建图与查询这两个阶段。

回到同一张带小障碍和挡板的地图，先给出四个自由配置：

$$P_0=(0.2,0),\quad P_1=(0.2,0.8),\quad P_2=(0.8,0.8),\quad P_3=(0.8,0.2).$$

为便于看清建图过程，这里直接给定这四个点；通常它们由采样产生。对距离不超过0.85的点对尝试直线连接：

候选边	长度约值	检查结果
P_0P_1	0.8000	合法，加入图
P_0P_3	0.6325	穿过挡板，拒绝
P_1P_2	0.6000	合法，加入图
P_1P_3	0.8485	穿过挡板，拒绝
P_2P_3	0.6000	合法，加入图

P_0P_2的长度为1，超过邻域范围，本次不尝试。最终共享图是$P_0 \leftrightarrow P_1 \leftrightarrow P_2 \leftrightarrow P_3$。

查询A仍从$(0,0)$到$(0.9,0.2)$。本例为每个查询端点选择最近且可合法连接的路线图节点，于是起点接P_0，目标接P_3。图搜索得到

$$S_A \to P_0 \to P_1 \to P_2 \to P_3 \to G_A,$$

长度为$0.2+0.8+0.6+0.6+0.1=2.3$。查询B改为从$(0.1,0.8)$到$(0.9,0.8)$，分别接入P_1与P_2，得到长度0.8的路线。共享路线图不变，两个查询各自添加临时端点与连接即可。

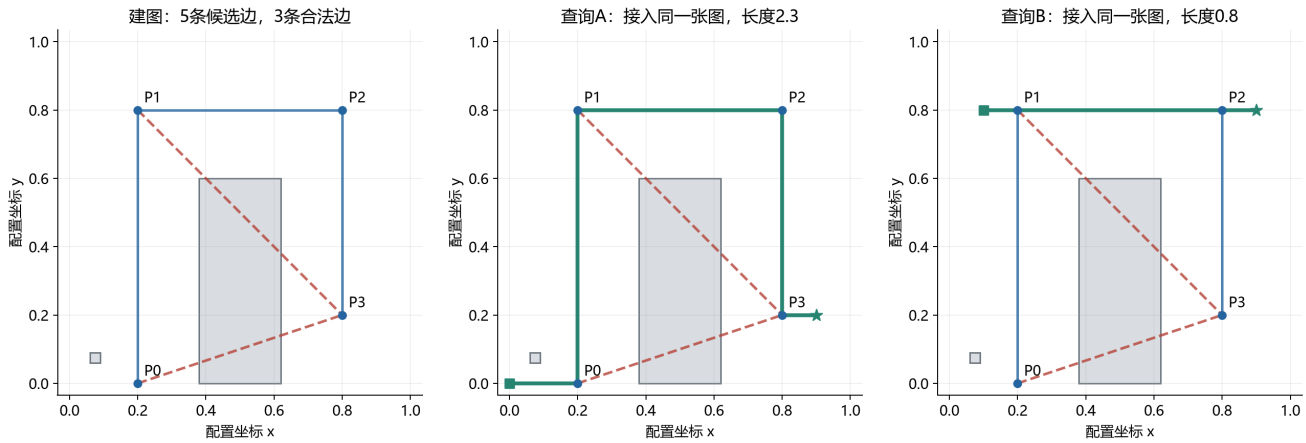


图8.3-6：左图建图时尝试5条边，保留3条；中、右图将不同起终点接入同一张图。红虚线是被拒绝的边，绿色表示查询路线。

实际PRM可以尝试把端点接到多个近邻，而本例只接最近的合法点。采样不够、邻域连接太少或端点接入失败，都可能使有限图断开；图上无路不能直接证明连续空间无路。障碍发生变化后，受影响的旧边也要重新验证。例如把挡板顶部从0.60升到0.90，原来的P_1P_2横边就穿过挡板，缓存图不再能支持原查询。

因此，RRT与PRM的选择主要涉及任务组织：当前问题需要尽快从某个起点探索，还是愿意先为相对稳定的空间建图、供以后重复查询。PRM并不要求每个查询都重新建图，RRT的父节点树也不等同于预先建好的通用路线图。

8.3.9 从几何路径接到机器人执行

以上得到的路径是$q_0, q_1, \ldots$及它们之间的连接关系，尚未为每段分配时间。机械臂执行时，还需要第8.4节的时间参数化或轨迹生成，使速度、加速度与控制接口相容；执行监测则负责在观测变化后决定继续、停止还是重规划。

局部连接也应服从机器人自身的运动能力。平面点可以沿任意方向走直线；有转弯半径限制的移动底座不能直接侧移。带动力学约束时，可以从当前状态试若干控制输入并积分，选出接近样本的可行运动，或使用适合该系统的局部规划器。此时节点可能还要包含速度，边也成为带时间的状态变化。OMPL规划器分类区分了几何规划与控制规划。

在柜内取杯任务中，VLA可以提出抓取姿态或技能参数。若给出的是末端目标位姿，应先用逆运动学求出满足目标的一组或多组关节配置，检查限位与目标碰撞，再作为规划的目标候选；同一末端位姿的不同关节解，可能有不同的可达路径。对其中一个关节解没找到路，也不能据此判定整个末端目标不可达。

运动规划器搜索连接，轨迹生成分配时间，控制器负责跟踪。抓起后更新附着物，观察到柜门或物体移动后更新场景。规划器返回路径，只说明在采用的模型和检查规则下找到连接；真实任务结果还取决于估计、控制与接触。各模块通过实际反馈接成系统。

8.3.10 运行算例与查看中间状态

在本节文件所在目录运行以下命令。算法与验证只使用Python标准库；配图已经随正文提供。

```
python practice/rrt_trace.py --out results
python practice/verify_rrt.py
```

第一条命令生成逐轮记录results/rrt_trace.json与CSV，以及PRM查询、RRT*局部重连的JSON。固定例的主要输出为：

```
status: solved
attempts: 15
path_indices: [0, 1, 2, 3, 4, 5, 7, 8, 9, 10, 11, 12, 13]
path_length: 2.208276253029822
```

完整实现见rrt_trace.py。nodes[i]存配置，parents[i]存父节点编号，events存每轮样本、最近节点、候选、碰撞区间与树大小。将这些量保留下来，就能定位错误发生在采样、扩展、碰撞还是回溯，而不只得到一个“成功/失败”。

下面节选步骤3—5，与伪代码编号对应；这些变量由完整函数run_rrt初始化，运行时使用上面的完整文件入口。

```
# 步骤3：只沿方向走 min(step, distance)，不越过近处样本。
scale = min(1.0, step / distance)  # distance 为零的情况已在上方跳过。
candidate = tuple(a + scale * (b - a) for a, b in zip(origin, sample))
hits = edge_collisions(scene, origin, candidate)

# 步骤4：检查整段；碰撞时不改变节点表与父节点表。
if hits:
    # 完整程序在这里记下拒绝原因，然后进入下一轮。
    continue

# 步骤5：合法候选入树，明确记录它从哪个旧节点接入。
nodes.append(candidate)
parents.append(nearest)
```

原有学习包的mechanisms/robotics_algorithms.py:rrt也是二维点RRT，可以返回最终路径。本节程序增加了逐轮记录与固定采样入口，便于逐项对应正文。随机模式可调用run_rrt(..., samples=None, seed=7)；设置相同种子有助于复现本程序的随机序列，不要求它与其他程序的随机序列相同。

invalid_start和invalid_goal表示输入端点不符合当前场景；budget_exhausted表示在预算内未找到路径；固定序列提前用完则为samples_exhausted。这些结果应分开处理。第二条命令检查几何相交、树与路径、边界情形和局部重连；结果写入results/verification.json。绘图方法见本节README。

8.3.11 练习题

题8.3-1 分别从第8轮结束的同一棵树出发，考虑下一轮采到$(0.3, 0.6)$或$(1, 0.6)$两种情形，最大扩展长度仍为0.2。各自的最近节点是哪一个、候选在哪里、整条连接能否加入？请使用本节的闭障碍边界规则，不只比较候选端点。

题8.3-2 有人把第1轮的碰撞检查改成“只检查线段比例$\lambda = 0, 1/3, 2/3, 1$处的配置”，据此接受了这条边。计算或定位实际碰撞区间，解释错误从哪里来。随后说明：为什么减小RRT扩展长度与加密边检查是不同操作？若要为机械臂宣称整段安全，还缺少什么依据？

题8.3-3 在第8.3.7节的RRT*局部例中，比较“选最近节点C为N的父节点”与“比较累计成本后选父节点”的结果，写出正确重连后的C, D父节点与成本。再考虑漏掉子孙传播的程序：①改接C后立即结束本轮，D缓存仍为10；②仍继续检查邻居D。两种情形下，经D再连接一条长0.5的合法边，成本会被算成多少？为什么②中D的成本即使得到修复，仍不能说明子孙传播可以省略？

题8.3-4 PRM完成两次查询后，挡板顶部变为0.90。请指出哪条已有合法边失效，给出使缓存图不再返回失效路线的最小更新，并说明它是否必然恢复查询连通性。再解释：更新后的四节点图没有跨挡板路线，能否断言这张连续地图无解？

题8.3-5 六关节机械臂的两个配置具有相同末端位置，其中一个肘部靠近柜壁。请说明：为什么不能把两个配置合并为同一个规划节点？两端配置都合法后，还需要验证什么？拿到一条几何路径后，还要经过哪些步骤才能执行？请按信息和模块的先后关系作答。

8.3.12 参考答案

题8.3-1答案 第8轮后节点0—5的位置依次为 $(0,0),(0.2,0),(0.2,0.2),(0.2,0.4),(0.2,0.6),(0.2,0.8)$。对$(0.3,0.6)$，最近的是节点4，距离0.1；相邻的节点3与5距离都是$\sqrt{0.1^2+0.2^2}\approx 0.2236$。候选因此直接到$(0.3,0.6)$。整段横坐标位于$[0.2,0.3]$，尚未到挡板左边界0.38，故可加入，父节点为4。

改采$(1,0.6)$时，最近的仍是节点4，但候选只能到$(0.4,0.6)$。边上$x\in[0.38,0.4]$的部分位于挡板上边界$y=0.6$，按闭障碍规则属于碰撞，应拒绝，树不变。向上绕过挡板需要高于0.6；恰好贴在上边界不满足本例规则。

题8.3-2答案 线段为$q(\lambda)=(0.141421\lambda,0.141421\lambda)$。同时满足两个坐标位于$[0.06,0.09]$，得到$\lambda\in[0.424264,0.636396]$。四个检查比例都不在该区间：$1/3$尚未进入，$2/3$已经离开，因此离散点全部合法，却漏掉中间一段碰撞。

扩展长度限制新边的长度；检查间隔决定一条已经提出的边上观察哪些位置。两者都会影响计算，但一个小扩展仍可能跨过更薄的障碍。对此二维矩形例，解析相交检查能直接检查整段。机械臂还需要可靠的整臂几何、连接路径以及相应运动验证：可采用连续碰撞检测，或利用净空与连杆运动位移界论证采样点之间也不会碰撞。仅说“采得足够密”，没有给出间隔与几何条件，尚不足以支持整段安全结论。

题8.3-3答案 若只选最近节点C，到N的累计长度为$9+1=10$。比较附近节点后，经过S只需$0+2=2$，因此应令$\mathrm{parent}(N)=S$。随后经N到C为3，低于旧成本9，所以$\mathrm{parent}(C)=N$、$g(C)=3$。D保持$\mathrm{parent}(D)=C$，但成本随父路径更新为4。检查直接从N到D也为4，与当前相等，本例不再改父节点。

情形①中，D的错误缓存10会把“再走0.5”算成10.5；正确结果为4.5，高估6。情形②中，后续检查发现经N到D的成本4低于缓存10，会把D直接改接N并令成本为4，于是再走0.5也被算成4.5。此时父节点已经与正确传播版本不同：正确版本在检查D前成本就已降为4，因此不因相等成本改接。

这种修复只发生在仍被逐一检查的邻居上。正文的$E=(4.5,0)$是D的子节点，却在N邻域外；若不沿子树传播，它仍缓存10.5，而实际根到E的长度已是4.5。用这个反例可以检查实现是否维护了所有受影响的成本，而不只碰巧修复一个邻居。

题8.3-4答案 原边P_1P_2沿$y=0.8$跨越$x\in[0.2,0.8]$；挡板升高后，它在$x\in[0.38,0.62]$的部分进入障碍。继续使用旧图，会把一条穿板路径报告为可行。最小更新是重新检查受影响的边，删除或标记这条边不可用，再搜索；两个方向的邻接记录都应更新。若采用延迟检查，也必须在接受路径之前验证实际采用的边。

删除失效边能恢复缓存的合法性，却不必然恢复查询连通性。剩余四节点图断开，只说明这张有限图没有表示出可用绕行路线。连续空间仍可从$y>0.9$绕过挡板，例如

$$(0,0)\to(0.2,0)\to(0.2,0.95)\to(0.8,0.95)\to(0.8,0.2)\to(0.9,0.2).$$

四个路线图点的最高位置只有0.8。需要补采上方配置并建立连接，才能让图表达这条路线。

题8.3-5答案 相同末端位置没有确定各连杆的位置。两个关节配置可能需要不同的避障路线、具有不同的关节限位余量；若合并成一个节点，会丢失后续连接是否合法所需的信息。应保留完整关节配置，用正向运动学放置整台机器人后检查碰撞。

两端合法之后，还要按采用的局部规划器验证中间运动，包括连杆、自身、环境及抓住的物体。得到几何路径后，按速度、加速度和控制接口生成时间轨迹；控制器跟踪，执行监测使用实际观测确认状态。抓取或放置改变碰撞场景，目标或障碍移动时更新场景并按需要重新规划。规划、控制和任务成功分别有各自的检查对象。

8.4 轨迹优化、时间分配与任务运动联合规划

8.4.1 轨迹优化的代价为什么有多项

给定一个初始轨迹，可以优化各时刻配置，让路径更平滑、接近目标并避开障碍。教学目标为

$$J(q_{0:H}) = w_g \|f(q_H) - x^*\|^2 + w_s \sum_{t=0}^{H-1} \|q_{t+1} - q_t\|^2 + w_c \sum_{t=0}^{H} \psi(d(q_t)).$$

第一项要求终点接近目标；第二项抑制相邻配置之间的大变化；第三项惩罚靠障碍太近。若各时间间隔固定，第二项可理解为限制离散速度的规模；若要让速度变化也平缓，可以再约束二阶差分 $q_{t+1} - 2q_t + q_{t-1}$。因此不同“平滑项”约束的运动性质不同，不能只看它们都叫平滑损失。权重组合不同目标与单位，优化的对象是本次候选轨迹。

仅用碰撞软代价可能在目标收益足够大时接受碰撞。必须严格满足的条件，应设成约束或在输出后独立验证。局部求解还可能困在初始路径所在的绕行方向。

只检查离散点会漏掉点间碰撞：一维路径从 −1 到 +1，障碍在 [−0.1,0.1]，两端都自由，但中间穿过障碍。需要足够细运动检查、连续碰撞方法或合理界限。

优化可细化采样搜索的初解，也可在重复任务里利用好初值。比较时应包括失败率、初值来源、检查精度和总时间，不能只展示成功案例的漂亮曲线。

8.4.2 路径怎样变成可跟踪的时间轨迹

路径告诉机器人经过哪些配置，时间参数化决定每段多久走完。要满足速度、加速度、关节和控制条件。改变速度也会改变接触与延迟影响，不是任意播放倍率。

直线位移 D=0.5 米、最大速度 0.2 米/秒，仅速度限制给下界 2.5 秒。若从静止到静止、最大加速度 0.4 米/秒平方，梯形速度方案需要加速、匀速、减速，总时间为 $D/v_{max} + v_{max}/a_{max} = 3$ 秒，条件是距离足够达到最大速度。

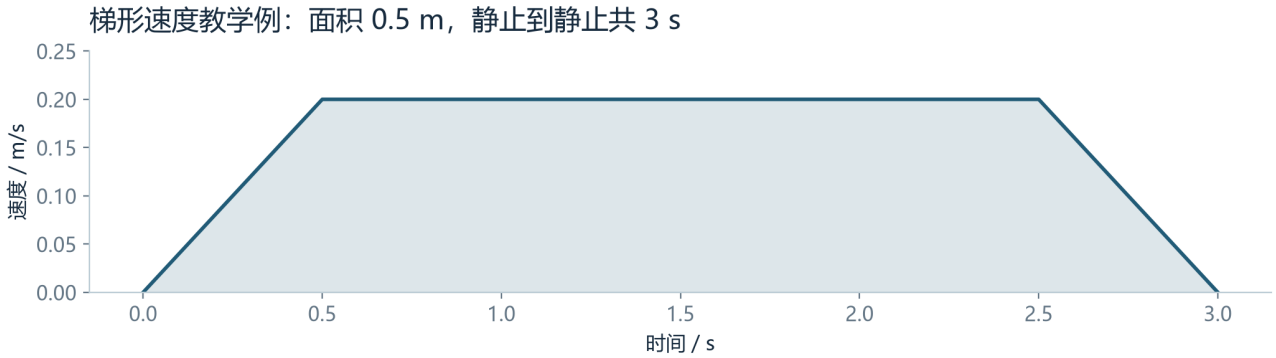


图8.6：同一路径可以有不同时间分配。真实可执行性包含速度、加速度和控制反馈。

关节路径的末端速度还受**雅可比**影响，不能只以末端总距离判断所有关节是否超限。控制器跟踪误差也需要检验，规划给参考并不保证真实运动完全相同。

机器人执行中若偏离参考，可以局部修正或触发重规划。需要防止旧轨迹和新指令同时控制同一资源。

8.4.3 TAMP 为什么要交替检查离散顺序与连续几何

TAMP（Task and Motion Planning，任务与运动规划）同时选择任务动作、对象参数和连续运动。例如先开哪扇门、采用哪个抓取位姿、杯子放哪里，都影响后续可行性。TAMP 综述。

一个符号计划“抓杯→放托盘”可能因为柜门关闭而失败；加上“开门”后，也可能由于所选抓姿导致杯沿无法穿门。任务顺序与连续参数之间相互约束。

一种组织方式先提出离散骨架，再采样抓取、放置与 **IK**，检查运动；失败后更换参数或骨架。另一类方法更紧密地共同搜索。它们都要避免只在最终才发现所有连续条件不成立。

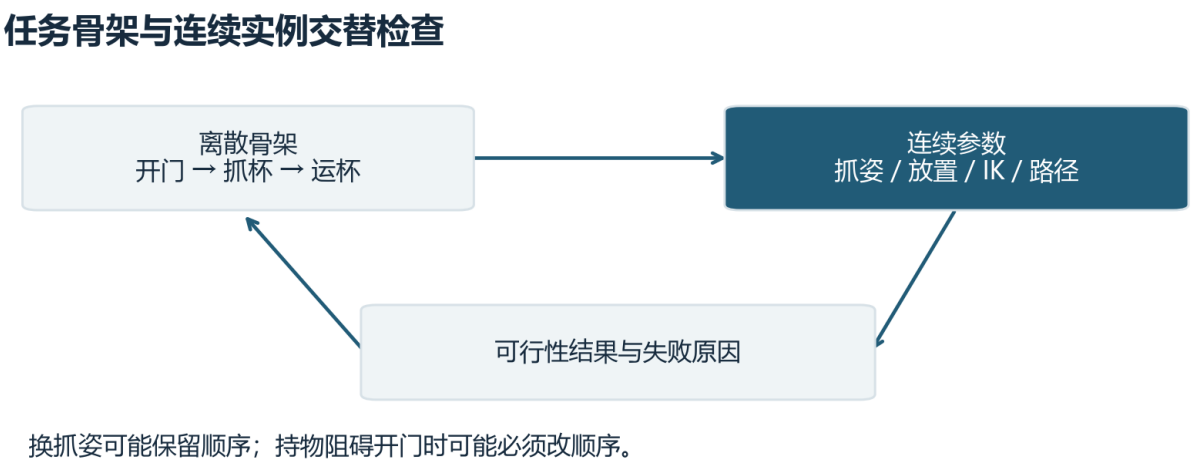


图8.7：同一任务顺序可以对应不同几何实例。连续失败信息决定修改参数还是改变顺序。

若夹着杯子开门不可行，可能需要先放下杯子；这改变任务层顺序。若只是当前抓姿卡住，换抓姿可能保留同一顺序。失败原因的粒度决定搜索效率。

接触、动态对象或柔性操作还会引入更复杂约束。并非所有任务都适合完整 TAMP；短局部操作可以交给学习技能，长任务和强几何依赖更适合显式协调。

8.4.4 用时间缩放把路径变成平稳起停的轨迹

设关节路径为 $q(s) = q_0 + s(q_1 - q_0)$，$s \in [0,1]$。它只规定从哪里到哪里。再设计 $s(t)$，才能决定每一时刻走到路径的哪一段。令总时间为 T、归一化时间 $u = t/T$，希望起终点位置正确，速度和加速度都为零：

$$s(0) = 0,\ s(1) = 1, \quad s'(0) = s'(1) = 0, \quad s''(0) = s''(1) = 0.$$

这里的导数先对 u 求。六个边界条件对应六个系数，可以选五次多项式 $s(u) = \sum_{i=0}^{5} c_i u^i$。前三个起点条件给出 $c_0 = c_1 = c_2 = 0$；终点条件构成

$$c_3 + c_4 + c_5 = 1, \quad 3c_3 + 4c_4 + 5c_5 = 0, \quad 6c_3 + 12c_4 + 20c_5 = 0.$$

解得 $c_3 = 10, c_4 = -15, c_5 = 6$，因此 $s(u) = 10u^3 - 15u^4 + 6u^5$。这些系数来自所要求的起停性质。若只要求起终点速度为零，用三次多项式就够了；增加加速度条件，才需要再增加自由系数。
Modern Robotics 时间缩放教程

回到物理时间：

$$\dot{q} = (q_1 - q_0)\frac{s'(u)}{T}, \qquad \ddot{q} = (q_1 - q_0)\frac{s''(u)}{T^2}.$$

将播放时间缩短为一半，速度变成两倍，加速度变成四倍。这说明一条几何可行路径不能任意加速执行。应据关节限速、加速度和必要力矩约束选择 T，再交给控制器跟踪。五次时间缩放提供平稳起停的参考，并不会自动处理碰撞、接触或跟踪误差。

8.4.5 练习题

题8.4-1 设几何路径光滑且无碰撞，为什么仍需要时间参数化？若沿相同路径把全程时长缩为一半，即 q_fast(t)=q_old(2t)，对应位置处的速度和加速度各如何变化？在刚体动力学中惯性项、科里奥利/离心项和重力项怎样变化，为什么总力矩不能简单判为四倍？

8.4.6 参考答案

题8.4-1答案 几何路径 $q = r(s)$ 只说明经过哪些姿态；$s(t)$ 指定各时刻的路径进度，才得到可执行轨迹。链式法则给出

$$\dot{q} = r'(s)\dot{s}, \qquad \ddot{q} = r''(s)\dot{s}^2 + r'(s)\ddot{s}.$$

弯曲路径上的第一项说明：即使进度速度恒定，关节仍可能需要加速度。时间参数化须使速度、加速度、力矩及接触稳定性满足限制。

全程时长减半时，$q_{\text{fast}}(t) = q_{\text{old}}(2t)$。直接求导得到

$$\dot{q}_{\text{fast}}(t) = 2\dot{q}_{\text{old}}(2t), \qquad \ddot{q}_{\text{fast}}(t) = 4\ddot{q}_{\text{old}}(2t).$$

比较的是同一路径位置：新轨迹在时刻 t 与旧轨迹在 $2t$。因此速度变为2倍，加速度为4倍。

对忽略摩擦和额外接触外力的刚体动力学

$$\tau = M(q)\ddot{q} + C(q,\dot{q})\dot{q} + g(q),$$

同一姿态下 $M(q)$ 和 $g(q)$ 不变。惯性项随加速度变为4倍，标准科里奥利/离心项对速度为二次齐次，也变为4倍，重力项保持原值。所以在这些条件下，

$$\tau_{\text{fast}} = 4[\tau_{\text{old}} - g(q)] + g(q),$$

通常不等于 $4\tau_{\text{old}}$。加入摩擦后，黏性摩擦通常约变为2倍，库仑摩擦不按此倍数变化；接触任务还可能发生滑动，不能只靠上述自由运动公式判断。几何无碰撞只保证经过的姿态安全，不保证任意速度都能被执行器准确跟踪。

进一步阅读：MIT Robotic Manipulation：路径时间参数化。

8.5 分层 VLA、长任务与条件成功概率

8.5.1 VLA 与规划怎样互相补充

VLA 可直接生成动作，也可作为参数化技能。高层规划决定当前子目标，策略根据图像和目标处理局部复杂行为；显式几何检查为候选抓取与运动提供约束。

还可以让策略给初始轨迹，规划器细化；让规划产生示范，训练策略摊薄运行时搜索成本；或由评价模型选择策略候选。训练教师与部署学生的信息差需要明确。

显式规划便于检查顺序与几何，学习策略可处理难以写规则的视觉和接触。混合也增加接口、时间和故障诊断复杂度。选择应围绕某个失效原因，而非把更多模块自动视为更强。

评价可固定底层策略比较任务层，或固定任务层比较技能，再报告完整系统。新模型、额外地图与更强控制器同时变化时，很难归因。

8.5.2 长任务成功率为什么容易下降

十个步骤各自成功率 0.95，在独立且无恢复的教学假设下，整段约为 $0.95^{10} \approx 0.599$。看起来每步都不错，串联结果仍可能明显降低。

一般关系是 $P(E_1,\ldots,E_m) = \prod_i P(E_i \mid E_1,\ldots,E_{i-1})$，其中 E_i 表示第 i 阶段成功。它使用的是“前面已经成功时，这一步还能成功多少”的条件概率，并不需要各阶段独立。单独测试抓取时的杯子位置，与执行开门、接近之后的杯子位置分布可能不同，所以不能直接把独立测出的单技能成功率相乘。阶段日志能帮助定位误差在何处传递，恢复则改变后续的成功机会与时间成本。

长任务还需要记忆已完成效果、当前持物与失败历史。错误状态更新可能让任务重复或跳过关键步骤。终止条件应对应完整目标，而非最后一个技能返回成功。

可以通过更可靠关键技能、有效恢复、合理重观察和顺序调整改善，但每种方案要同时报告时间与成本。层次化不是免费消除误差，而是让不同时间尺度的决策更可管理。

8.5.3 分层策略怎样安排决策频率

高层根据指令、场景状态和已完成步骤选择子目标，例如“打开柜门”；低层在这个子目标保持期间，根据新图像、本体或接触反馈持续生成动作。技能报告成功、失败或需要重新观察后，高层再决定下一步。这样把较慢的任务选择与较快的动作修正分开，减少高层每个控制周期都重新安排任务的负担。关键接口是子目标的含义、低层能执行的范围，以及何时向高层报告结束。

分层方法将目标选择与动作执行放在不同时间尺度。高层可以要求“抓稳红杯”，低层不断根据图像和接触调整。若高层频繁改变目标或低层结束反馈不可靠，分层反而造成不稳定。比较时需控制额外技能示范、状态标签和搜索预算。

8.5.4 长任务的连乘，应该乘哪些成功率

设三个技能的成功事件为 E_1, E_2, E_3。整任务成功概率总能分解为

$$P(E_1, E_2, E_3) = P(E_1)P(E_2 \mid E_1)P(E_3 \mid E_1, E_2).$$

这里没有先假设技能独立。第二个因子是在第一个技能真正完成后，对其留下的状态计算的成功率；它不一定等于从标准起点单独测试技能2得到的0.9。例如“取杯成功”仍可能把杯子放到倾斜位置，接着“放上托盘”就更难。例如三个对应条件概率均为0.9时，乘积为0.729；其他组合也可能得到相同乘积。

分层控制的价值是让上层决定当前子目标，下层专注执行，但接口需要传递真实结果。上层验证杯子是否被稳稳抓住，再决定搬运还是重新抓取；它并非发完三个命令就天然保证三个效果发生。条件成功率、结果验证与恢复是同一条长任务机制的不同环节。

8.5.5 练习题

题8.5-1 三阶段任务中，各技能在各自标准起点单独测试的成功率都是 0.9，整体是否一定为 0.9^3？用阶段成功事件写出完整任务的条件概率分解。什么条件下结果为 0.729，是否必须假设独立？为什么分层模型仍需要结果验证和恢复？

8.5.6 参考答案

题8.5-1答案 不一定。孤立技能测试的0.9是在各自标准起点上测得；串联执行时，第二阶段起点由第一阶段结果决定。例如开门虽被判成功，真实开度仍有差异，会改变取杯难度，不能直接搬用单独取杯测试的成功率。

令 S_1, S_2, S_3 是各阶段成功事件，完整任务的正确分解为

$$P(S_1 \cap S_2 \cap S_3) = P(S_1)P(S_2 \mid S_1)P(S_3 \mid S_1, S_2).$$

若沿实际串联流程测得这三项都是0.9，就得到 $0.9^3 = 0.729$；这个计算不另外要求独立。独立时可把条件概率换成边际概率，但仅凭孤立技能测试率，既不知道这些串联条件概率，也不能断言独立。

例如有10%的场景三阶段都困难而失败，其他90%的场景三阶段都成功。三阶段各自的边际成功率都是0.9，完整任务成功率却是0.9，而不是0.729。共同场景因素使失败关联。

分层模型使技能可复用，却不自动消除阶段间的状态偏差。高层应验证真实效果是否满足下一阶段前提，并在抓取失败后重新定位、换姿态，或在滑落后回到适当阶段。恢复使一次失败不必立即成为整段任务失败；多次重试也未必独立，收益应实测。应同时报告阶段条件成功率、恢复率和最终成功率。

进一步阅读：MIT Robotic Manipulation：任务层程序、状态机与行为树、Introduction to Probability, Statistics, and Random Processes：条件概率与链式法则。

8.6 进度监测、重规划与恢复

执行监测比较预期效果与观测。例如开门后检查门角度，抓杯后检查对象附着，放下后检查支撑与夹爪释放。判断器可能有误差，需要结合时间和证据。

若完成判断器是网络，可输入图像、力历史和任务目标，以人工核验或可靠仿真规则产生成功标签。分类损失更新判断器，部署通过门限产生技能状态。门限决定误报与漏报的取舍，不能仅把“发了抓取命令”当成功标签，否则会复制执行盲区。

如果抓取失败，先更新杯子位置与夹爪状态，再选择重新观察、换抓姿或重试。旧目标位姿可能已经失效，机械重复同一动作未必有用。

恢复也需要预算、停止条件和升级路径。无限重试能让某些成功率看似高，但代价与失败定义变了。报告允许次数、时间和人工干预。

规划更新应处理正在执行的命令：取消、等待停止、核对当前状态，再切到新参考。实时信息年龄和资源所有权是系统约束，不能只在算法示意图中忽略。

主动观察可以用于恢复不可辨识状态，但要计算额外时间与运动代价。不能在评估时偷偷用未来真实状态来决定最佳观察。

8.6.1 恢复是一个新的决策问题

杯子滑落后，原计划前提已经失效；重复上一段动作可能再次撞击。监测器应识别物体是否仍在夹爪、目标是否移动、进度是否停滞和传感是否异常，然后选择重抓、退回、换视角或请求接管。恢复轨迹与普通成功轨迹的数据分布不同，需独立训练和评价。

阈值过敏会频繁打断，阈值过迟则损失恢复机会。应报告检测延迟、误报、漏报和恢复成功率，结合具体后果选择阈值，不把“使用异常检测”当成完整机制。

8.6.2 练习题

题8.6-1 门已打开一半但卡住，按固定时长切换到抓杯会怎样失败？进度监测应使用哪些信号，何时重规划？

8.6.3 参考答案

题8.6-1答案 固定时长假定“执行这么久就已达到足够开度”。门卡住时这个假定失效，系统仍切到抓杯，手臂可能撞门，或无法到达此前规划的抓取位置。经过时间只能说明执行多久，不能证明阶段效果已实现。

监测应优先读与目标有关的进度：门角度或可通行开度、门角度变化率、末端实际位姿与目标误差；再用力/力矩、接触和电机电流判断受阻，用视觉确认把手接触和门的状态。不要只设一个角度阈值，还要检查取杯运动所需的空间是否足够。

例如一段时间内开度几乎不增加，而接触力持续升高，可判定“没有进展且正在受阻”。此时先停止施加原动作，重新观察，再区分夹爪滑脱、门被障碍挡住或开门方向错误。根据原因选择重新抓把手、改变受力方向、移走可移动障碍或修改取杯路线。若门已达到最低可取杯开度，可以对新门姿态重规划抓取。

触发规则应使用短时间窗口与合理阈值，避免传感噪声导致反复切换；危险接触可立即中止，普通停滞则需持续证据。恢复结束后再次验证前提，不能把“做过恢复动作”当成问题已解决。

8.7 从柜内取杯到托盘的完整规划项目

8.7.1 规划应用流程：从柜内取杯到托盘

输入为指令、门与杯子状态、机器人位姿和几何。目标为门打开、正确杯子稳定放入托盘、夹爪完成释放。目标事实需要独立观察检验。

流程为：确认目标→找到门把手→验证开门技能→开门并确认→选择抓姿与 **IK**→规划接近→抓取并确认附着→更新碰撞场景→规划运杯→放下与检验。失败信息回到当前状态，决定局部恢复或高层重新选择。

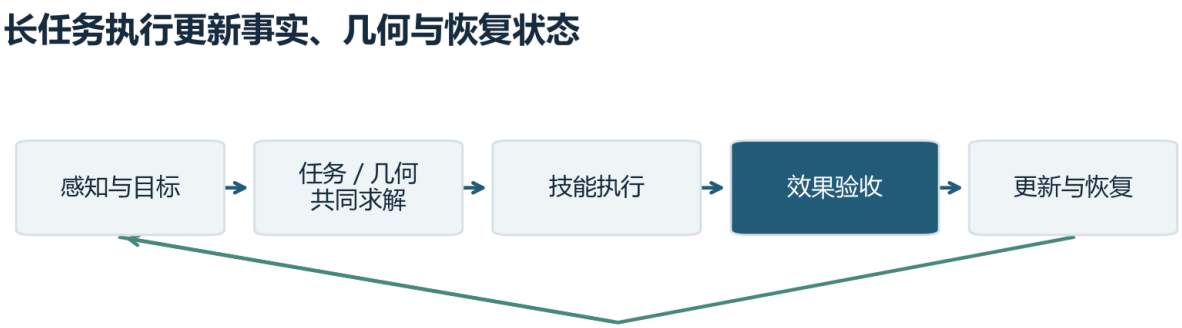


图8.8：每次技能成功都更新任务事实和连续状态。持物改变碰撞模型，失败改变后续可行计划。

日志应记录规划输入时刻、候选与拒绝原因、技能状态、观测结果、重试和耗时。只保存最后视频，很难区分任务搜索失败、运动不可行、策略失误与控制偏差。

先在固定场景完成接口，再变化门角、杯位置与干扰；最后检验完整长任务。配套 A* 小例子可以观察搜索、路径代价与障碍布局的关系。

8.7.2 研究评价与后续学习

研究主张要定位层次：更好任务顺序、更高几何可行率、更低搜索成本、更强恢复，还是更高完整任务成功。各自需要对应指标与公平基线。

规划器比较应固定几何模型、碰撞精度、预算和硬件；语言规划比较应固定技能库；混合系统比较应说明新增输入与评价器。可报告成功率、时间、路径质量、重试和人为干预，而非只给成功轨迹耗时。

组合已有任务规划、学习策略和监测方法可以形成贡献，但需要证明它们之间新增的协调机制解决了真实问题。若只增加更强底层技能，总分提升不能证明高层机制有效。

后续可以深入 MoveIt、OMPL、Drake 与行为树工具。先复现一个可解释任务链，再逐层替换算法；工具能求解或组织问题，但问题定义仍由你负责。MoveIt 教程、MIT 运动规划资料。

完整规划项目应固定物体和布局，先逐技能验证，再串联长任务；有意加入目标移动、抓取失败和传感延迟，检查监测与恢复。记录计划、实际阶段切换、失败原因和重规划次数。多一个恢复模块的结果，应与相同感知、技能和预算的对照比较。

8.7.3 练习题

题8.7-1 在完整取杯项目中，如何区分目标定位错、运动规划错、抓取执行错和成功验证错？为什么只报告总成功率难以改进系统？

8.7.4 参考答案

题8.7-1答案 应在模块交接处保存可核对的中间结果，而不只保存最终成功标签。

- **目标定位。**保存杯子身份、三维位姿、坐标变换和时间戳，用独立标注或较可靠的测量检查。杯子识别正确但相机到机器人变换错误，仍属于定位接口问题。
- **运动规划。**在定位正确的前提下，检查是否找到可达抓取姿态和路径，整条路径是否无碰撞，时间参数化是否满足限制。规划路径本身碰撞与机器人没有跟上安全路径，应分开记录。
- **抓取执行。**定位和计划都有效时，比较指令与实际轨迹、夹爪开度和接触、提起后的杯子运动。手指在正确位置但杯子滑落，可能是力或摩擦问题；执行偏离则应查控制和延迟。
- **成功验证。**用独立结果判断核查“杯子稳定位于托盘内”等条件，区分实际失败却报成功、实际成功却报失败，以及技能命令完成被误当任务成功。

可用替换实验逐级定位：给已标注的杯子位姿是否改善；给人工验证的安全路径是否改善；由人工完成抓取后，搬运与验证是否正常。替换需记录额外信息，不能当作同条件最终性能。总成功率可能同为 60%，一个系统主要定位错，另一个主要滑落，改进方向完全不同。还应记录最早失效环节与后续连锁失败，避免重复计数。

第9章：机器人感知、状态估计与 SLAM

规划与操作需要可靠的空间状态。这里从传感和标定出发，讲解估计、定位、地图、物体感知与多模态反馈，以及它们怎样支持实际行动。

9.1 相机、深度、惯性与标定

9.1.1 感知输出的是估计，不是物理真相

策略可以直接读图像，也可以读取物体位姿、地图和触觉等结构化估计。后者有明确几何语义，却携带标定、关联和观测误差。本章解释这些误差如何产生、怎样融合，以及它们为何影响闭环操作。共同空间记法见第4.1节。

9.1.2 图像为什么丢失深度，深度又怎样回来

投影公式描述了哪一种压缩

针孔相机把三维点投到二维图像。忽略畸变，点在相机系中为 (X, Y, Z)

$$u = f_x X/Z + c_x, \qquad v = f_y Y/Z + c_y.$$

u、v 是像素位置；f_x, f_y 是以像素为单位的焦距；c_x, c_y 是主点。X、Y、Z 使用同一长度单位。除以 Z 表示远处同样大小的东西在图像中更小。这是传感器测量模型，可以用来预测“假设这个三维点和位姿正确，应看到哪个像素”。OpenCV 相机模型。

取 $f_x = 400, c_x = 320, X = 0.2, Z = 2$，得到 u=360。另一个点 X=0.4、Z=4 也投到 u=360。单个像素只限定一条射线，不能单独确定距离。**同时定位与建图**（Simultaneous Localization and Mapping，SLAM）在单目设置下存在尺度问题，其原因正在于此：整体场景和相机平移同时放大，图像可能仍然一样。

双目利用第二个视角增加约束

经过校正的双目相机有已知基线 b，同一目标在左右图像的横向差是视差 d，深度满足

$$Z = f_x b/d.$$

两个视角的射线相交，提供了距离线索；基线的米制长度提供尺度。取 b=0.1 米、焦距 400 像素、视差 20 像素，深度为 2 米。

但这个公式还揭示了远处难测的原因。视差变为 5 像素时，深度是 8 米；同样一像素视差误差引起的深度变化大得多。局部敏感度的绝对值为 $f_x b/d^2$，在 d=20 时为 0.1 米/像素，在 d=5 时为 1.6 米/像素。这是线性近似，不能在大误差时当作精确界限。

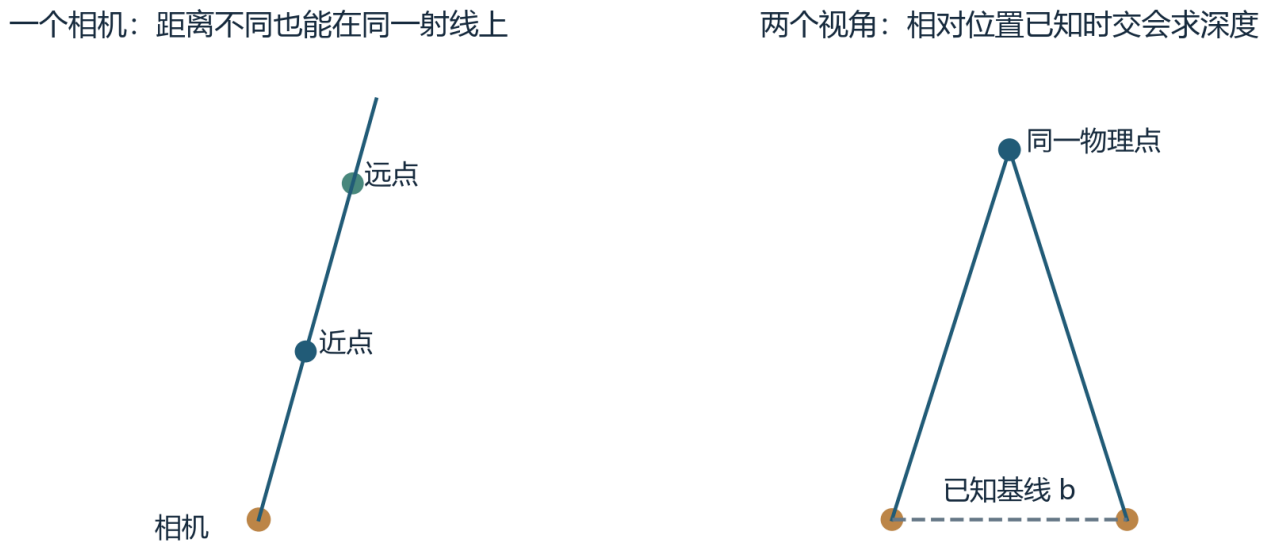


图9.1：一个视角给出射线，两个有已知相对位姿的视角可以三角化。远处射线交角小，深度更敏感。

RGB-D 相机同时提供彩色图像（RGB）与深度（Depth）观测，但其测量也可能受材质、距离、光照和遮挡影响。透明杯、反光杯容易出现无效深度。反投影公式 $X = (u - c_x)Z/f_x$ 可以把像素与深度转换成三维点；使用前必须确认设备输出的是沿光轴的 Z，还是沿射线的距离，以及深度图是否与彩色图对齐。两者混用会得到系统性几何错误。

9.1.3 相机、激光雷达与 IMU 各自看到了什么

相机有丰富纹理与语义，但光照变化、运动模糊和无纹理墙面会削弱匹配。激光雷达提供距离点云，弱光下也能工作，但点云稀疏、扫描随时间展开，透明或特殊反射材质也会造成困难。两者各有测量边界，不是简单的精度排行榜。

IMU（Inertial Measurement Unit，惯性测量单元）通常包括陀螺仪和加速度计。陀螺仪测角速度，加速度计测比力。比力包含运动加速度与重力的关系，静止放在桌上的加速度计通常也有约一个 g 的读数。若世界系重力向量为 g_W，理想比力可以写成 $f_B = R_{BW}(a_W - g_W)$；具体正负号依坐标约定核对。

IMU 的价值在于高频提供短时运动信息。相机两帧之间机器人仍在运动，IMU 能帮助预测位姿、处理快速转动。但积分会放大偏置：恒定加速度偏置 0.01 米/秒平方，在初始速度正确且其他条件理想时，10 秒可产生约 $\frac{1}{2}bt^2 = 0.5$ 米位置误差。这解释了为什么单独长期积分 IMU 难以保持准确位置。

偏置不是每次随机变化后立即抵消的白噪声，而是有时间相关性的误差。融合系统常把陀螺与加速度偏置也放进状态，一边利用外部观测约束运动，一边估计这些偏置。若机器人始终静止或只做非常有限的运动，有些量难以区分，增加传感器并不保证**可观测性**（Observability）。可观测性关心的是：依据系统运动规律和传感器观测，能否区分不同的内部状态。

激光雷达一圈扫描中的点并非同一时刻采集。运动中把整圈当作瞬时刚体点云，会出现墙面弯曲或双层边缘。去畸变（deskew）利用各点时间戳和运动估计，把点转换到同一参考时刻。它校正的是扫描期间的运动，不是相机镜头畸变。

轮式机器人还可能使用轮速里程计；足式机器人可利用接触约束，但打滑、脚离地或错误接触检测会破坏假设。传感器选择要从工作环境与任务精度出发：走廊导航需要长期运动稳定，桌面抓取需要近距离几何，动态人群还需要处理运动物体。

9.1.4 标定与同步为什么常比换模型更重要

内参、外参与时间偏移分别是什么

内参描述图像如何成像，包括焦距、主点和畸变。外参描述传感器相对机器人或另一个传感器的刚体位姿。时间偏移描述两路时间戳是否指向同一物理时刻。三者可以同时出错，并且表现可能相似。

相机标定通常拍摄多个方向和距离的已知几何标定板，利用板上点的三维坐标与图像像素，求使**重投影**残差小的内参、畸变与各帧板位姿。这里优化的是标定参数，不是训练杯子识别网络。只拍正对相机的板，可能不足以有效约束所有参数；只看平均像素误差，也未必发现边缘区域畸变没拟合好。OpenCV 标定教程。

手眼标定到底在求哪只“眼”

考虑**眼在手上**：相机固定在末端，标定板在世界中保持不动。第 i 次拍摄时，机械臂提供末端到世界的变换 $G_i = T_{WE_i}$，图像提供标定板到相机的变换 $H_i = T_{C_iO}$。未知量是相机到末端的固定外参 $X = T_{EC}$。无论采用哪一次拍摄，沿“标定板→相机→末端→世界”转换，都应得到同一个标定板世界位姿。因此多次记录能够约束 X，常整理成

$$AX = XB.$$

具体地，两次拍摄满足 $G_iXH_i = G_jXH_j$。移项后得到 $(G_j^{-1}G_i)X = X(H_jH_i^{-1})$，于是 $A = G_j^{-1}G_i$ 来自机械臂的相对运动，$B = H_jH_i^{-1}$ 来自图像估计的相对运动，待求的 X 是固定外参。公式的设计依据是两条坐标路径描述同一固定目标，结果必须一致。求解器输入多组 A, B，输出能共同解释这些记录的 X；再用未参与拟合的姿态检验标定。眼在手外等配置需要重新写变换链，不能沿用本例的 A、B 定义。OpenCV 手眼标定。

如果只沿一条直线运动，或者旋转轴几乎不变，可能难以约束完整 X。标定需要激励不同方向的运动。机械臂绝对运动学误差、末端柔性和相机支架松动，也会使“固定刚体外参”假设出现偏差。

时间误差如何变成空间误差

机器人以 1 米/秒移动，两路观测相差 30 毫秒，直接融合可能引入约 3 厘米的位移不一致。转弯时还会有旋转误差，距离越远影响越大。看似“点云与图像对不上”，原因可能是同步而非外参。

处理顺序应是确认单位、坐标轴、时间戳定义、数据对齐方式，然后检查模型误差。时间戳究竟是曝光开始、曝光中点、驱动收到数据还是算法处理完成，不能视为同一个时刻。Kalibr 提供多相机、相机—IMU 等标定工具，可帮助理解联合估计的实际流程；使用前仍需要符合工具的采集要求。Kalibr 官方资料。

9.1.5 标定决定不同观测能否互相解释

内参把空间射线连接到像素，外参连接不同坐标系，时间偏差连接不同设备采样。相机内参正确而相机到工具的外参错误，机械臂仍会对错位置。将标定板移动到多个位置与角度，是为了使未知参数得到充分约束，不是越多相同姿态的照片越好。

评价标定不能只看训练照片的重投影误差。需要在独立姿态、深度和机器人动作下检验空间对齐，尤其是目标任务实际使用的距离。深度值是沿光轴的 Z 还是沿射线的距离，也必须与重建公式一致。

9.1.6 练习题

题9.1-1 深度像素如何从图像坐标变成机器人基座中的三维点？为什么相机外参有小偏差时，语义识别仍对而抓取却会偏？

9.1.7 参考答案

题9.1-1答案 先恢复相机坐标中的三维点，再转换到基座坐标。假设像素已去畸变，深度和彩色图已对齐，内参为 K。记 $d = K^{-1}[u, v, 1]^\top$；在通常的针孔内参形式下，d 的第三个分量为 1。

若深度值是沿光轴的距离 Z，则

$$p_C = Zd = \left[\frac{(u - c_x)Z}{f_x}, \frac{(v - c_y)Z}{f_y}, Z\right]^\top.$$

若传感器给的是相机原点到目标的射线距离 ρ，先把射线方向变成单位向量，再乘距离：$p_C = \rho d/\|d\|$，所以 $Z = \rho/\|d\|$。只有图像中心附近，二者才接近。完成单位换算并剔除无效深度后，用相机到基座的外参计算

$$p_B = R_{BC}p_C + t_{BC}.$$

腕部相机的外参由两段变换组成：$T_{BC}(t) = T_{BE}(q(t))T_{EC}$。第一段由曝光时刻的关节角求出，第二段是固定手眼标定。不能拿处理完成时刻的关节角替代曝光时刻。

语义识别主要根据图像判断杯子是什么，外参通常不参与；抓取必须把杯子转换到机器人坐标，因此识别正确仍可夹偏。设表达在基座系中的小旋转误差向量为 $\delta\theta$，其位置影响近似为 $\delta\theta \times (R_{BC}p_C)$，大小不超过 $\|p_C\|\|\delta\theta\|$，还要叠加平移误差。目标越远通常越敏感，但沿旋转轴的点受这项旋转误差较小。

进一步阅读：OpenCV：Camera Calibration and 3D Reconstruction、MIT Robotic Manipulation：Geometric Pose Estimation。

9.2 贝叶斯估计、滤波与残差优化

9.2.1 状态估计如何表达“我还不确定”

状态、观测与测量模型

状态 x 是我们想知道但不能直接完整读取的量，例如位置、速度、朝向与 **IMU** 偏置。观测 z 是传感器数据或从中提取的测量，例如像素、深度、角速度。测量模型 h(x) 描述假设状态为 x 时，应该得到怎样的观测。

系统维护一个估计及其不确定性。**协方差**大的方向表示该方向约束较弱；它不是“网络置信度”的另一种写法。神经网络输出 0.9 的分类分数，也不自然等于位置误差小的概率。

在一个标量例子中，预测位置是 1.0 米，预测误差方差 P=0.04；传感器测到 1.2 米，测量误差方差 R=0.01。在线性、无偏、独立高斯误差假设下

$$K = \frac{P}{P+R}, \qquad \hat{x} = \hat{x}^- + K(z - \hat{x}^-), \qquad P^+ = (1-K)P.$$

K 是**卡尔曼**增益，决定向新观测靠近多少；$z - \hat{x}^-$ 是创新，即观测与预测的差。这里 K=0.8，更新位置为 1.16 米，方差变为 0.008。测量更可靠，所以结果更靠近 1.2。若 R 很大，系统应更多相信预测。

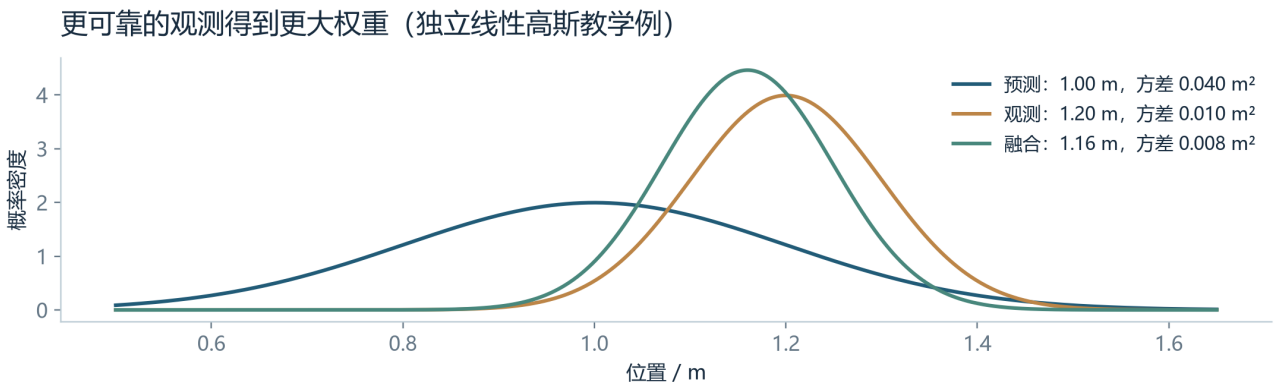


图9.2：精度不同的两个信息源在明确假设下融合。结果取决于不确定性，而非简单平均。

为什么不能把同一个证据算两次

如果用同一幅图像构造两个估计，再当作独立传感器融合，会高估信息量。一个位姿来自图像匹配，另一个地图匹配也大量依赖同一批像素，它们可能相关。融合后协方差变小并不等于实际误差变小。

扩展卡尔曼滤波使用局部线性化处理非线性模型，但近似可能在初值差、强非线性或数据关联错误时失效。大残差可以触发异常检测，不过不能机械地删掉所有大残差：它可能来自真实突然运动，也可能来自噪声模型低估。应检查残差分布、运动条件与故障场景。

对行动而言，不确定性有实际用途：位置把握不足时减速，目标深度不可靠时换视角，地图中未观测区域避免直接当成可通行。只有当概率含义经过合理建模或验证时，才能据此设安全阈值。

9.2.2 残差与优化到底在修改什么

从“猜状态”到“检验这个猜测”

设当前猜测的相机位姿和地图点为 x，把它们通过投影模型得到预测像素 h(x)，与实际像素 z 比较，残差是 $r(x) = h(x) - z$。改变 x 后，预测会改变，残差也会改变。运行时优化就是寻找更能解释观测的一组状态。

常见目标是

$$x^* = \arg\min_x \sum_i \rho\left(r_i(x)^\top \Sigma_i^{-1} r_i(x)\right).$$

若测量误差服从零均值高斯分布，某条观测的负对数似然包含 $\frac{1}{2} r^\top \Sigma^{-1} r$。这就是逆协方差加权的来源：误差尺度小的方向，一点偏差就提供强烈的反证；误差尺度大的方向，同样偏差没那么意外。各观

测的独立似然相乘，取负对数后便成为残差之和。鲁棒函数 ρ 再限制离群残差的影响，避免一条错误匹配支配全部位姿。它能减弱异常值，却不能把大量错误关联变成正确几何。GTSAM 教程。

米与像素不能直接混在一个未经加权的平方和中。即使同为像素，近处清晰角点与远处模糊匹配也未必同样可靠。鲁棒损失能减轻离群匹配影响，但不能把大量错误对应自动变成真观测。

雅可比说明哪个变量能改变哪个误差

雅可比 J 描述当前估计附近，状态小变化如何改变残差：$r(x+\delta) \approx r(x) + J\delta$。把这个近似代入加权平方目标，再对 δ 求最小值，便得到**高斯—牛顿**方程 $J^\top W J\delta = -J^\top W r$。右侧汇总当前观测希望怎样修正状态，左侧描述变量受到的约束强度与耦合。求出增量后更新状态，再重新计算残差和 J，逐步修正原来的非线性问题。位姿增量按第4.1节组合，不能直接破坏旋转约束。初值差或弱约束时，局部近似可能不可靠，因此还需要阻尼、合理初值或补充观测。

若某个方向的变化几乎不影响残差，系统就无法从当前观测确定它。矩阵病态不只是数值烦恼，可能反映真实的**可观测性**不足。例如单个无限平面通常不能确定沿平面滑动了多少。

再看地图整体的参考坐标。只使用相对几何观测、没有绝对位置与方向约束时，把所有相机和地图点一起平移、旋转，并不会改变内部观测；纯单目系统还可能保留整体尺度自由度。这类不改变观测的整体变换称为**规范自由度**（gauge freedom）。具体剩下哪些自由度，取决于传感器和运动条件：标准视觉惯性系统在充分激励、米制IMU及已知重力约定下，通常仍不能确定全局三维位置和绕重力方向的航向；额外绝对观测可以进一步约束它们。固定第一帧或设置锚定先验，是选择参考坐标，不等于获得了新的绝对定位测量。视觉惯性系统的不可观方向。

神经网络训练也最小化损失，但变量通常是跨许多样本共享的网络参数。当前位姿优化的变量是这一段轨迹和地图。二者可以联用，例如网络给初值、几何优化做修正，却不能用“都做梯度下降”来抹去它们在数据、目标和泛化能力上的差别。

9.2.3 从概率乘积推到卡尔曼更新，再写出预测步骤

假设预测位置 x 服从 $\mathcal{N}(\mu^-, P)$，测量满足 $z = x + \epsilon$、$\epsilon \sim \mathcal{N}(0, R)$，且测量噪声与预测误差独立。先验乘似然后，后验负对数中与 x 有关的部分是

$$L(x) = \frac{(x-\mu^-)^2}{2P} + \frac{(z-x)^2}{2R}.$$

第一项要求估计别随意偏离预测，第二项要求它解释新测量；方差规定每项偏差受到多大惩罚。令导数为零，得到

$$\frac{x-\mu^-}{P} + \frac{x-z}{R} = 0 \quad\Longrightarrow\quad \mu^+ = \frac{R\mu^- + Pz}{P+R} = \mu^- + \frac{P}{P+R}(z-\mu^-).$$

所以卡尔曼增益 $K = P/(P+R)$ 来自两份证据的误差模型。二次项系数又给出 $1/P^+ = 1/P + 1/R$，因此 $P^+ = PR/(P+R)$。在线性高斯条件下，求出这个均值和方差，就得到完整后验；多峰位置歧义则不能用一个高斯充分表达。《Kalman and Bayesian Filters in Python》第4章

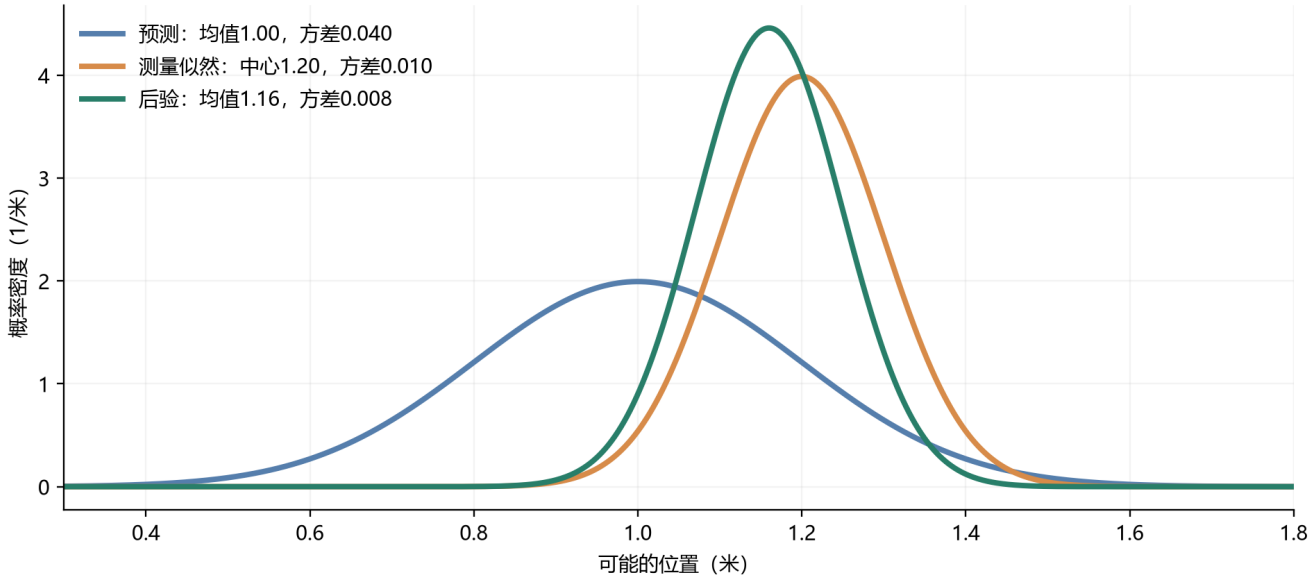


图9.3：沿用前面的均值1.0、测量1.2、方差0.04与0.01，后验均值为1.16、方差为0.008。融合比例来自误差模型，后验比两份输入都更集中。

机器人会运动，还需要在测量之间做**预测**。用位置和速度组成 $x = [p, v]^\top$，短时间内采用恒速模型：

$$F = \begin{bmatrix} 1 & \Delta t \\ 0 & 1 \end{bmatrix}, \quad x^- = Fx^+, \quad P^- = FP^+F^\top + Q.$$

F 将速度转换为下一时刻的位移；Q 表达恒速近似没有覆盖的运动变化。P 描述当前状态估计误差，Q 描述预测新增的不确定性，测量误差 R 则来自传感器。它们在滤波中有不同位置。

```
import numpy as np

def position_velocity_update(x, P, measured_position, dt, Q, R):
    F = np.array([[1.0, dt], [0.0, 1.0]])
    H = np.array([[1.0, 0.0]])  # 传感器只读取位置。
    # 1. 根据运动模型预测状态，并传播原有误差、加入过程误差。
    x_prior = F @ x
    P_prior = F @ P @ F.T + Q
    # 2. 比较实际位置与预测位置；投影协方差得到创新的不确定性。
    residual = measured_position - (H @ x_prior)[0]
    S = (H @ P_prior @ H.T)[0, 0] + R
    K = (P_prior @ H.T)[:, 0] / S
    # 3. 用新证据共同修正位置、速度与协方差。
    x_next = x_prior + K * residual
    J = np.eye(2) - np.outer(K, H[0])
    P_next = J @ P_prior @ J.T + R * np.outer(K, K)
    return x_next, P_next
```

这里的矩阵形式增益为 $K = P^-H^\top(HP^-H^\top + R)^{-1}$。只有位置测量，为什么也能修正速度？因为预测把“速度偏高”和“随后位置偏高”联系起来，P^- 中产生了位置与速度的相关项。新测量反驳较高的位置，也就提供了修正速度的证据。只有持续的一系列位置观测，速度才逐渐受到约束。同书的多变量滤波教学、FilterPy 输入与预测更新示例

滤波更新当前状态与不确定性，不是训练神经网络。噪声和动态假设错误时，矩阵计算仍能顺利运行，估计却可能过度自信；应在真实运动、遮挡和异常测量下检查残差与误差范围。

9.2.4 练习题

题9.2-1 假设状态的先验分布为 N(0,4)，测量噪声独立且服从 N(0,1)，测量模型为 z=x+噪声，观测到 z=3。求卡尔曼增益、后验均值与方差，并解释先验和测量的权重为何如此分配。

9.2.5 参考答案

题9.2-1答案 按独立高斯先验和测量噪声计算。先验方差为 $P^- = 4$，噪声方差为 $R = 1$，测量直接观察状态，所以

$$K = \frac{P^-}{P^- + R} = \frac{4}{5} = 0.8.$$

卡尔曼更新先计算测量与预测的差，再只吸收其中一部分：

$$\hat{x}^+ = \hat{x}^- + K(z - \hat{x}^-) = 0 + 0.8(3 - 0) = 2.4,$$

$$P^+ = (1 - K)P^- = 0.2 \times 4 = 0.8.$$

后验均值也可写成 $0.2 \times 0 + 0.8 \times 3$。测量的方差比先验小，因而更可信，权重更大。这不是人为规定“相信传感器八成”，而是由两个信息源的不确定性得到的最优权重。

从贝叶斯角度看，后验负对数密度去掉常数后是 $\frac{1}{2}[x^2/4 + (3-x)^2/1]$。它同时惩罚偏离先验和偏离测量；对 x 求导令零，得到 $x/4 + x - 3 = 0$，解为 2.4。两个精度，即方差的倒数，相加为 $1/4 + 1 = 1.25$，其倒数就是后验方差 0.8。若只给均值和方差而不假设高斯，这组公式仍给出线性最小均方误差估计，但未必给出完整后验的真实均值和方差。

9.3 视觉前端、运动估计与数据关联

9.3.1 前端怎样从图像中取得运动证据

对应关系比“找到了点”更关键

SLAM 前端把原始图像、点云转换成可以用于估计的约束。视觉前端常找角点或其他局部特征，计算描述子，再在不同帧中寻找可能对应同一物理点的匹配。描述子用于比较局部外观；它不是三维坐标，也不保证匹配正确。

机器人移动后，杯子把手上的同一点在图像中换了位置。如果能建立对应，就可以用几何模型约束相机运动。但是重复桌腿可能造成错配，移动中的人的衣服也可能成为大量清晰特征。能稳定匹配与适合描述静态环境运动，是两个条件。

RANSAC（Random Sample Consensus，随机采样一致性）反复从匹配中抽少量样本，拟合候选模型，检查其余匹配是否支持该模型。目的不是把所有匹配平均，而是找到能够解释一组一致证据的模型。若每个匹配为内点的概率为 w，最小样本数为 s，独立采样下，为以概率 p 至少抽中一次全内点样本，需要

$$N \geq \frac{\log(1-p)}{\log(1-w^s)}.$$

每次抽到全内点最小样本的概率是 w^s，连续 N 次都没抽到的概率为 $(1-w^s)^N$。要求这一失败概率不超过 $1-p$，整理后便得到上式。$w=0.5, s=3, p=0.99$ 时约需 35 次。它用来估算搜索预算，说明内点比例下降时为什么计算会迅速增加。抽到全内点也不保证样本几何不退化，最终仍要检查候选模型和支持它的观测；实际最小样本数由所拟合的几何模型决定。

特征法与直接法在相信不同的证据

特征法先抽取匹配，再优化几何误差。直接法利用像素强度或局部光度一致性，比较“按候选运动把图像对齐后，像素是否一致”。后者能利用更多图像信息，但依赖合理曝光、亮度变化处理和运动初值。镜面反射随视角变化，可能违反亮度一致性。

语义分割可以标出人、车辆等可能运动区域，帮助筛选运动约束。然而“杯子”并不永远静止，“人”也不永远运动。更合理的动态处理会结合几何一致性与时间信息，而非固定删掉某类物体。

前端输出通常包括匹配、观测质量、相对运动初值和关键帧候选。它的错误会进入后端，所以评估不能只看单帧匹配数量，还应看正确率、空间分布、退化场景与跟踪丢失率。

9.3.2 视觉里程计、三角化与 PnP 怎样配合

二维—二维、三维—二维不是同一个问题

两帧中对应同一物理点的射线，可以约束相机相对运动。已知内参后，把像素换成归一化方向 d_1, d_2；若相机1到相机2的变换为 R, t，对应关系满足 $d_2^\top [t]_\times R d_1 = 0$。原因是两条视线与相机间平移共同处于一个平面。矩阵 $E = [t]_\times R$ 称为**本质矩阵**，多组对应点可用来估计它，再恢复相对旋转与平移方向。把 t 乘任意非零尺度仍满足零残差，因此单目两帧一般不能给出绝对平移长度。纯旋转、低视差等情况下，还可能缺少稳定的深度与运动约束。

三角化是在相机相对位姿已知或已估计时，从多条观测射线求三维点。这个点又能进入后续定位。若射线几乎平行，交点对像素误差很敏感；前后两帧几乎只旋转，也不足以稳定产生新深度。关键帧选择不仅是减少计算，也是在选择更有用的几何证据。

PnP（Perspective-n-Point，透视 n 点位姿估计）使用一组已知三维点及其当前图像二维对应，求相机与这些点所在坐标系之间的位姿。输入不是“随便 n 个像素”，而是带三维对应的像素。输出通常还需要通过**重投影**误差和鲁棒筛选验证。OpenCV PnP 文档。

OpenCV 的旋转与平移把对象或世界坐标转换到相机坐标，即本书记号中的 T_{CW} 或 T_{CO}。若要相机在世界中的位姿，需要取逆。机械臂拿到物体在相机系的位姿后，则继续通过相机到机器人系的外参转换。理解输出方向比记住函数名更重要。

一个连续运行的循环

初始帧建立少量地图点；新帧提取匹配，利用已有三维点做 PnP，得到当前相机位姿；有足够视差时加入关键帧并三角化新点；局部后端同时优化位姿和点；跟踪失败时尝试重定位。具体系统可能采取不同初始化和地图管理方式，但这种证据互相补充的关系很常见。

尺度来源应单独检查。双目、有效 **RGB-D** 或充分激励下的视觉惯性系统能够提供米制线索；单目图像本身通常留下尺度自由度。学到的单目深度可以提供数据**先验**，但其尺度在新相机、新领域中未必可靠，不能等同于几何上的绝对可观测。

如果只做桌面局部抓取，固定相机和有效深度可能已经足够，无需构造完整视觉 SLAM。模块复杂度应由任务需要决定。

9.3.3 光流为什么需要一个小区域，而不能只看一个像素

光流描述相邻图像中同一内容在像素平面上怎样移动。它可以来自相机运动，也可以来自物体运动，所以还要结合场景与几何，才能用来估计机器人运动。

Lucas–Kanade 方法从亮度近似不变出发：一个像素附近的内容移动了 $\Delta x, \Delta y$，经过 Δt 后，应在新位置看到相近亮度。写成

$$I(x, y, t) \approx I(x + \Delta x, y + \Delta y, t + \Delta t).$$

对右侧作一阶展开并消掉原亮度，得到 $I_x \Delta x + I_y \Delta y + I_t \Delta t \approx 0$。除以 Δt，令像素速度 $u = \Delta x / \Delta t, v = \Delta y / \Delta t$：

$$I_x u + I_y v = -I_t.$$

图像能给出三个导数，但一个等式约束两个未知速度。看一条竖直边缘时，横向运动会改变像素亮度，沿边缘上下运动却可能看不出来。这叫**孔径问题**：局部证据缺少一个运动方向的约束。OpenCV 光流教程

若一个小窗口内共享近似相同的运动，每个像素便各给一个等式。把 $[I_{x,i}, I_{y,i}]$ 作为矩阵 A 的一行，把 $-I_{t,i}$ 放入 b，求 $\min_w \|Aw - b\|^2$，得到 $A^\top A w = A^\top b$。窗口中的梯度若包含多个方向，两维速度才都能被确定；全是平行梯度时，增加像素数量也补不出缺失方向。这解释了角点适合跟踪，而平坦墙面和单条边缘较困难。

一阶展开适合小位移。图像金字塔先在低分辨率估计大运动，再逐层细化，使每层剩余位移较小。曝光突变、遮挡或反光会破坏亮度假设；运动物体则可能提供可靠光流，却不能当成静态地图的相机运动证据。光流追踪、几何筛选与状态估计因此各自负责不同环节。

9.3.4 RANSAC 的采样与重拟合

```
随机取两个不同且不重合的二维点
由它们建立单位法向量 n 与偏移 b 的直线 n·p+b=0
以 abs(n·p+b) 小于阈值判定内点
保留支持点较多的候选，重复有限轮
对最佳内点去中心化，用 SVD 找垂直于主方向的法向量
返回法向量、偏移、内点标志及必要的失败状态
```

9.3.5 练习题

题9.3-1 在把抽样点的内点事件近似视为独立、各轮独立的轮数预算模型下，RANSAC 拟合需要最小样本数 s=2，内点比例 w=0.5，希望至少一轮抽到全内点的概率不低于 p=0.99，需要多少轮？为什么用 y=kx+b 拟合垂直线会有问题？

9.3.6 参考答案

题9.3-1答案 采用常用的独立抽样预算近似：每个抽样点是内点的概率为 w，两点同时为内点的概率为 $w^2 = 0.25$；假设各轮独立，则 N 轮全部失败的概率为 0.75^N。要求至少成功一次的概率达到 0.99：

$$1 - 0.75^N \geq 0.99 \quad \Rightarrow \quad N \geq \frac{\log 0.01}{\log 0.75} \approx 16.0078.$$

因此至少 17 轮。16 轮成功概率约 0.989977，17 轮约 0.992483。这只是在该抽样模型下保证抽到一次全内点，仍要排除重复点等退化样本，并用合理距离阈值判断模型支持的内点。

若数据集很小且一轮内无放回，设总点数为 M、内点数为 m，两点均为内点的概率应为 $m(m-1)/[M(M-1)]$，再替代上式的 0.25。数据集远大于最小样本时，w^s 近似通常更合适。

$y = kx + b$ 要求每个 x 对应单个 y，不能用有限斜率表示垂直线；而只拟合竖直偏差也不等于拟合二维点到线的距离。用 $ax + by + c = 0$ 并约束 $a^2 + b^2 = 1$，点到线的距离为 $|ax + by + c|$。例如 $(a, b, c) = (1, 0, -2)$ 就表示垂直线 $x = 2$。

进一步阅读：MIT Robotic Manipulation：Geometric Pose Estimation。

9.4 点云配准、匹配误差与几何约束

ICP（Iterative Closest Point，迭代最近点）在两组点云间交替做两件事：根据当前位姿建立最近点对应；固定这些对应，求一个更好的刚体变换。变换更新后再重新对应。它输出的是两点云的相对对齐，不是物体语义。PCL ICP 教程。

最简单的点到点残差是 $Rp_i + t - q_i$。来源点 p_i 经旋转平移后应靠近目标点 q_i。若两次扫描采样位置不同，最近点并不一定对应同一个物理点，点到点残差可能受采样影响。

点到平面残差利用目标点附近的法向 n_i

$$r_i = n_i^{\mathsf{T}}(Rp_i + t - q_i).$$

这个标量只计算沿表面法向的偏离。例如地面为 z=0、法向为 $(0,0,1)$，变换后的点高度为 0.05 米，残差就是 0.05 米。沿地面横向差 0.2 米不会直接进入这一条残差。这适合局部平面，但也揭示了约束缺失：只有一个无限平面时，沿平面平移和绕法向旋转难以确定。

如果初始位姿太差，最近点会对应到错误墙面，ICP 可以稳定收敛到错误结果。惯性预测、轮速或全局特征配准常用于给初值；距离阈值、法向一致性和鲁棒损失用于减少错配。低残差只能说明当前对应下对齐好，不能证明对应真实。

点云体素下采样可减少计算并平衡局部密度；滤除孤立点可减轻噪声。但分辨率过粗会丢掉杯把手等操作几何。导航点云与抓取点云可以使用不同尺度，不能要求一个分辨率兼顾全部任务。

长走廊、单面墙、重复货架等环境要重点检查退化。此时提高点数未必补上缺失的方向约束，应增加不同几何视角、融合其他信息，或在策略层降低速度和重新观察。

9.4.1 对应点已经给定时，怎样求出旋转和平移

ICP 交替寻找对应与更新变换，其中“固定对应后拟合刚体”可以直接求解。设 p_i 与 q_i 已经配对，目标为

$$\min_{R,t} \sum_i \|Rp_i + t - q_i\|^2, \quad R^\top R = I,\ \det R = 1.$$

先对 t 求导，最优平移满足 $t = \bar{q} - R\bar{p}$。代回目标，相当于先减掉两组点各自的中心，再只求旋转。平移改变整体位置，不改变点之间的相对结构；这就是把两类未知量分开的依据。

令 $p_i^c = p_i - \bar{p}, q_i^c = q_i - \bar{q}$。展开平方后，点的长度项不随旋转变化，最小化剩余误差等价于最大化 $\sum_i (q_i^c)^\top R p_i^c = \mathrm{tr}(RH)$，其中 $H = \sum_i p_i^c (q_i^c)^\top$。对 H 作 **SVD**（Singular Value Decomposition，奇异值分解），$H = U\Sigma V^\top$，可得

$$R = VDU^\top, \qquad D = \mathrm{diag}(1, 1, \det(VU^\top)).$$

SVD 把匹配关系整理到几组正交方向上。没有行列式约束时，沿这些方向对齐可得到 $VU^\top$；它有时包含镜像反射。D 在必要时翻转最后一个方向，使结果仍是刚体旋转。中心化与旋转约束的推导可参照 MIT 点云位姿教学和 《Introduction to Robotics and Perception》的点云配准。两处采用的相关矩阵方向可能与本例互为转置，旋转公式也随之调整。

```
import numpy as np

def fit_rigid_transform(source, target):
    # 输入为同样数量的三维点，每一行的点已与另一组同一行配对。
    source = np.asarray(source, dtype=float)
    target = np.asarray(target, dtype=float)
    # 1. 去掉整体平移，将问题改为围绕中心的旋转匹配。
    center_p, center_q = source.mean(0), target.mean(0)
    P, Q = source - center_p, target - center_q
    # 2. 汇总点对关系，求最能对齐它们的正交旋转。
    U, singular_values, Vt = np.linalg.svd(P.T @ Q)
    D = np.eye(3)
    D[-1, -1] = np.linalg.det(Vt.T @ U.T)
    R = Vt.T @ D @ U.T
    # 3. 恢复两个中心之间的平移关系。
    t = center_q - R @ center_p
    return R, t
```

这个函数解决给定对应下的点到点最小二乘问题。ICP 还要反复更新对应，点到平面目标则使用不同残差与求解方式。共线点等退化布局不能唯一约束完整旋转；错误对应也可能给出数值稳定的错误变换。闭环任务需要同时检验对应、几何布局和变换后误差。

9.4.2 练习题

题9.4-1 ICP 反复找最近点并拟合刚体变换，为什么初值差时会收敛到错误配准？点到平面目标与点到点目标怎样不同？

9.4.3 参考答案

题9.4-1答案 ICP（Iterative Closest Point，迭代最近点）交替解决两个较容易的子问题：先用当前位姿为源点找最近目标点，再在这些对应关系固定的条件下拟合刚体变换。困难在于，初值偏得太远时，“最近点”往往不是同一个物理点。例如两个外形相似的桌腿可能被错误配对；随后的变换又会让这组错误配对更近，于是算法停在一个误差较小、物理对应错误的结果上。反复迭代只能改善当前局部对应，不能保证找到全局正确位置。

记变换后的源点为 $Rp_i + t$，对应目标点为 q_i。点到点目标为

$$E_{pp} = \sum_i \|Rp_i + t - q_i\|^2.$$

它惩罚三个方向上的位置差。点到平面目标使用目标表面的单位法向量 n_i：

$$E_{pl} = \sum_i [n_i^\top (Rp_i + t - q_i)]^2.$$

它主要惩罚偏离局部表面的距离，对沿表面滑动较宽容，常能更快修正贴合误差。代价是需要可靠法向量；若点云几乎全是一个平面，沿平面移动和绕法向旋转可能缺乏约束。实践中先用几何特征或全局匹配获得粗位姿，再用 ICP 精修，并检查重叠率、残差和多个候选初值。

9.5 因子图、回环与视觉激光惯性融合

9.5.1 因子图与 BA 把哪些证据放到一起

图中的节点和边有具体含义

因子图包含未知变量节点与约束因子。节点可能是各时刻位姿、三维地图点、速度和偏置；因子表示一个或一组观测对相关变量的约束。相邻位姿之间有里程计因子，位姿与地图点之间有投影因子，初始位姿上可以有锚定因子。

因子图把联合估计写成若干局部证据的组合。位姿与地图点作为待求变量，一条像素观测只约束看见它的相机及对应三维点，一段惯性测量只约束相应时间范围。采用所定义的条件独立关系后，后验可写成各因子的乘积；最大化它，等价于最小化各因子负对数之和。这样既说明“哪条证据约束谁”，也产生了可利用的稀疏结构。后端求解的是这段轨迹和地图的状态值。GTSAM 入门。

BA（Bundle Adjustment，束调整）同时调整相机位姿与三维点，让这些点在各图像上的**重投影**更符合实际观测。只改相机可能把地图点误差归到运动上，只改点又可能吸收位姿误差；共同优化体现了二者耦合。

位姿图：变量节点与观测因子不是神经网络

x0—x3：未知位姿；f：相邻运动观测；回环：跨时间的相对约束。

锚定一个参考位姿用于选择坐标；不等于新增了绝对地理定位。

图9.4：观测连接具体变量。**回环**增加跨越较长时间的约束，让积累的漂移可以被重新分配。

每个点通常只在少量帧中可见，每个观测只关联少数变量，所以**雅可比**和正规矩阵有稀疏结构。利用稀疏性比把所有变量当作密集矩阵更节省计算。消元可以先处理地图点再求位姿；它改变求解方式，没有丢弃点对位姿的约束。

为什么窗口需要边缘化

滑动窗口若直接删除旧位姿和相关观测，会丢失它们对当前状态的约束。**边缘化**是在局部高斯近似下，消去旧变量，把其影响保留为剩余变量的先验。把信息矩阵按旧变量 m 与保留变量 r 分块，保留部分得到 $H'_{rr} = H_{rr} - H_{rm}H_{mm}^{-1}H_{mr}$，右侧的线性项也相应调整。这一消元关系称为**舒尔补**（Schur Complement）。原先需要经过旧位姿联系的证据，被压缩成对保留变量的联合约束，窗口便可以继续向前移动。计算依赖线性化与可消元条件，下一段讨论由此产生的近似限制。

边缘化常建立在某个线性化点附近。后续状态变化很大时，旧先验可能不能像原始观测那样重新准确线性化。固定窗口因此存在效率与一致性的取舍。全局位姿图通常保留较粗的关键帧关系，局部窗口处理更细的几何和惯性状态，两者可以分工。

选择后端时，应看任务轨迹长度、实时预算、回环需求和**可观测性**，而不是看到“图优化”就认为一定比滤波准确。错误因子、错误噪声权重和错误坐标方向会让复杂后端更坚定地给出错误结果。

9.5.2 回环、重定位与漂移修正

回到旧地方为什么提供新信息

里程计每一步都有误差，长期串联后地图可能弯曲。当机器人再次看到曾经经过的地方，可以增加“现在与以前其实在相近位置”的约束。这就是回环。外观检索通常先找候选，随后用几何验证确认对应关系，最后加入后端。

重复走廊、相似门口和同款桌子很容易外观相似。检索得分高只是候选，不应直接变成强位姿约束。错误回环可能把整张地图拉到一起，危害比短时漂移更大，因此验证和异常回环处理很重要。

做一个一维闭环算例。三个相对位移观测为 1.1、1.1、−2.0 米，但真正闭合要求它们的和为 0。观测总和为 0.2。若三条边方差相等，并最小化各边修正量的平方和，最优做法是每条减去 0.2/3。修正后为 1.0333、1.0333、−2.0667，位姿从 0 经 1.0333、2.0667 回到 0。

若某条边更不可靠，它应承担更多修正。这个例子说明回环不是把最后一个点硬拉回起点，而是按约束与可信度调整整段轨迹。三维系统还涉及旋转及非线性关系。

重定位与回环的目的不同

重定位是在跟踪丢失或重新启动时，利用已有地图恢复当前位姿。回环通常针对持续运行中的全局一致性。二者可以共享地点识别和几何匹配，但触发情境不同。

地图中的世界系通常只是内部参考。回环减少内部漂移，并不会自动提供地理坐标或建筑物的绝对方向。若任务要求与建筑地图对齐，需要额外的地图注册、已知标记或其他外部定位信息。

全局回环可能使机器人在地图中的坐标突然修正。控制通常需要连续的局部里程计坐标，而全局规划使用可修正的地图坐标。ROS（Robot Operating System，机器人操作系统）是一套机器人软件框架，常用 odom 与 map 两级坐标关系表达这种分工：局部平滑与全局一致性各自服务不同时间尺度。

9.5.3 视觉惯性与激光惯性融合的设计逻辑

相机或雷达给几何约束，**IMU** 给高频运动约束。两者互补的前提是外参、时间和运动模型正确。融合不是简单把三个位置输出取平均。

松耦合通常融合各子模块已计算的位姿或运动结果，接口较容易管理，但原始信息和相关性可能已经丢失。紧耦合让图像特征、点云残差、惯性测量共同约束状态，即使某个子模块不足以独立输出完整位姿，剩余有效测量仍可能有用；代价是实现、标定和一致性分析更复杂。

IMU 每秒产生许多读数，若每次优化都从头积分所有读数，代价很高。预积分把两个关键帧之间的高频惯性信息压缩成相对旋转、速度变化、位置变化及其不确定性。优化改变端点状态时，可以重用压缩结果；偏置小幅改变时，常通过局部修正更新，变化过大则需要重新积分。它不是把 IMU 变成无误差的里程计。

滤波着重递推当前状态分布；平滑或窗口优化可以用更晚观测修改窗口内较早状态。低延迟与充分利用历史证据有不同取舍。**激光惯性里程计**（LiDAR-Inertial Odometry，LIO）融合激光与惯性信息估计运动。FAST-LIO、LIO-SAM 是这类系统的代表；VINS-Mono 主要融合视觉与惯性信息，ORB-SLAM3 则覆盖视觉及视觉惯性设置。它们是理解不同组织方式的系统入口，而不是在所有机器人上通用的最优排名。FAST-LIO、LIO-SAM、VINS-Mono、ORB-SLAM3。

视觉惯性系统需要估计重力方向、偏置、速度等状态。充分运动有助于区分它们；纯旋转、匀速直线或少量激励可能让某些量难以确定。一个系统在采集初始化时要求移动设备，往往有建模原因，而非交互设计偏好。

检验融合效果时，应同时报告位姿误差、失败率、延迟和计算资源。可以对同步误差、遮挡时长、低纹理程度或激光几何退化逐级施加扰动，验证互补性发生在哪些条件下。若只展示一条容易轨迹，很难证明融合机制真正解决了薄弱场景。

9.5.4 舒尔补怎样保留已经删去的变量所提供的信息

用两个一维位置看窗口边缘化。y 是准备移出的旧位置，x 是保留的位置；三条同等权重的观测给出代价

$$L(x,y)=(x-y-1)^2+(y-2)^2+(x-4)^2.$$

第一条是相对位移，第二条约束旧位置，第三条约束新位置。正规方程为

$$\begin{bmatrix}2 & -1\\ -1 & 2\end{bmatrix}\begin{bmatrix}x\\ y\end{bmatrix}=\begin{bmatrix}5\\ 1\end{bmatrix}.$$

第二行给 $y=(1+x)/2$；代回第一行，得到 $1.5x=5.5$，于是 $x=11/3$，再得到 $y=7/3$。移出 y 后，只要保存 $1.5x=5.5$ 这条合成约束，关于 x 的解仍相同。若把涉及 y 的两条观测全部删除，剩下的目标只有 $(x-4)^2$，解会变成 4，旧观测提供的信息已经丢失。

一般分块方程 $H_{rr}r+H_{rm}m=b_r$、$H_{mr}r+H_{mm}m=b_m$ 中，先解出 m 再代回，得到

$$H'_{rr}=H_{rr}-H_{rm}H_{mm}^{-1}H_{mr},\qquad b'_r=b_r-H_{rm}H_{mm}^{-1}b_m.$$

这就是舒尔补。在线性高斯问题中，它也对应消去旧变量后，剩余变量的边缘信息；非线性位姿系统通常先在当前状态附近线性化，再保存这样的先验。原始因子被压缩后，重新线性化的自由度会减少，因此窗口长度、状态变化与一致性需要一起考虑。GTSAM 的因子图与联合估计教程

9.5.5 把因子图看成一组共同约束，而不是轨迹插值

因子图（Factor Graph）中的变量节点可以是各时刻位姿，因子则是一条测量对相关变量提出的约束。里程计说“第二次位姿相对第一次移动了多少”，回环说“现在与较早时刻看到同一地点，应满足怎样的相对位置”，先验则固定整体参照。它们共同决定一条尽量解释测量的轨迹。

用三个一维位置举例：固定 $x_0 = 0$，两次里程计分别测到 $x_1 - x_0 = 1$、$x_2 - x_1 = 1$，回环或外部测量却说 $x_2 = 1.8$。三条约束同样方差时，最小化

$$L = (x_1 - 1)^2 + (x_2 - x_1 - 1)^2 + (x_2 - 1.8)^2$$

得到 $x_1 \approx 0.9333, x_2 \approx 1.8667$。误差被各约束共同分担，不是只把最后一个点强行改成1.8。若最后一条方差更小，它对解的影响更大；若误将两个相似场景关联成回环，优化也会努力满足错误约束。鲁棒损失减少大残差的影响，不能替代正确的数据关联。GTSAM 因子图教程

9.5.6 练习题

题9.5-1 因子图中里程计边与回环边分别提供什么信息？错误回环为什么可能比没有回环更糟？鲁棒损失是否足以解决所有错误关联？

题9.5-2 两个一维位置 x（保留）、y（旧变量）的代价为 L=(x-y-1)^2+(y-2)^2+(x-4)^2。写出完整正规方程，消去 y，给出舒尔补的信息系数与右侧项，求 x 和 y。若直接删除涉及 y 的两条观测，x 变成多少？说明“移出旧变量却保留其信息”如何实现，以及非线性 SLAM 中这种合成先验为什么受线性化点限制。

9.5.7 参考答案

题9.5-1答案 里程计边约束相邻时刻之间的相对位姿，例如“从第 3 个位姿前进了多少、转了多少”。把这些相对运动串起来可以得到轨迹，但小误差会逐步累积。回环边连接较远的两个时刻，表示机器人重新看到了同一地点，并给出两次观测之间的相对位姿。优化器利用它重新分配整条轨迹的误差，纠正累计漂移。

错误回环把两个不同地点当成同一地点，相当于向优化器输入一条错误但可能很强的约束。例如两个长得相同的走廊被匹配起来，优化后地图可能被拉弯，使多个原本正确的位姿一起偏移。没有回环时通常只是缓慢漂移；错误回环可能造成全局突变。

鲁棒损失会降低大残差的影响，但残差取决于当前估计。错误约束若数量多、权重过大，或优化后的地图恰好满足这些错误约束，其残差也可能很小；反而正确约束变成“大残差”。因此还要在加入回环前验证几何匹配、重叠区域与时间一致性，并检查它是否与已有运动链冲突。可疑匹配先暂存，等更多连续观测确认后再加入；仍有歧义时保留多个位置解释，避免过早强迫整张图满足一条边。只换鲁棒损失不能完成这些关联判断。

题9.5-2答案 对 x, y 求导并令零，约去共同因子 2：

$$\begin{aligned}(x - y - 1) + (x - 4) &= 0,\\ -(x - y - 1) + (y - 2) &= 0.\end{aligned}$$

所以完整正规方程为

$$\underbrace{\begin{bmatrix} 2 & -1 \\ -1 & 2 \end{bmatrix}}_{H} \begin{bmatrix} x \\ y \end{bmatrix} = \underbrace{\begin{bmatrix} 5 \\ 1 \end{bmatrix}}_{b}.$$

这里 H 是平方残差的正规矩阵；若直接写 L 的二阶导数会得到 $2H$，右侧也要同时乘 2，解不变。由第二行得到 $y = (x+1)/2$。代回第一行，等价于做舒尔补：

$$S = H_{xx} - H_{xy} H_{yy}^{-1} H_{yx} = 2 - (-1)\frac{1}{2}(-1) = \frac{3}{2},$$

$$\bar{b} = b_x - H_{xy} H_{yy}^{-1} b_y = 5 - (-1)\frac{1}{2} \times 1 = \frac{11}{2}.$$

因此 $(3/2)x = 11/2$，得到 $x = 11/3 \approx 3.667$，再回代得到 $y = 7/3 \approx 2.333$。

信息保留的含义可以直接从代价看出来。对给定 x，旧变量最合适的值是 $(x+1)/2$，代入涉及它的两项得到

$$\min_{y} \left[(x - y - 1)^2 + (y - 2)^2 \right] = \frac{1}{2}(x-3)^2.$$

所以可删去 y 及原来两条观测，同时在 x 上加入合成先验 $\frac{1}{2}(x-3)^2$。两条旧观测提供的是 $y \approx 2$ 和 $x - y \approx 1$，合起来约束 x 接近 3；若把单位权重残差解释为独立、方差为 1 的高斯测量，这条间接约束的方差为 $1 + 1 = 2$，信息系数便是 $1/2$。新增先验单独的右侧项为 $3/2$；与保留的 $(x-4)^2$ 相加后，才得到完整舒尔系统的 $S = 3/2, \bar{b} = 11/2$。若直接删除两条观测而不加入先验，只剩 $(x-4)^2$，最优值就变为 $x = 4$。

本题是线性平方问题，消元保持完整问题的 x 解；在线性高斯模型中，对旧变量积分也得到同样的 x 相关二次项。非线性 SLAM 则先在当前估计点附近写 $r \approx r_0 + J_x \delta x + J_y \delta y$，再消去 δy。合成先验保存的是当时的残差、雅可比及参考点，是局部近似。以后位姿离该点很远，残差随位姿变化的关系可能已明显不同；若原始因子和旧变量已丢弃，便无法从这个二次先验恢复它们在新位置的雅可比。因此它不是在所有位置都成立的完整非线性约束；移动参考点时必须一致变换已有先验，而重新表达已有二次项不等于重新计算原观测的线性近似。

进一步阅读：Ceres Solver：Solving Non-linear Least Squares，DENSE_SCHUR & SPARSE_SCHUR、GTSAM：Fixed-Lag Smoother Example、GTSAM：Marginals、GTSAM：LinearContainerFactor。

9.6 地图、深度学习与神经场景表示

9.6.1 地图为什么有很多种，行动究竟需要哪一种

稀疏特征地图主要服务定位；它可以告诉机器人在哪里，但少量特征点之间的空白不等于可通行。导航需要表达障碍、未知区域与机器人尺寸，抓取需要局部表面、法向和接触几何。地图形式由后续查询决定。

占据地图给空间单元记录被障碍占据的概率。激光穿过某些格子再击中终点时，可以给前段空闲证据和终点占据证据。没有射线经过的格子保持未知。规划时怎样对待未知，是任务风险与探索策略的选择。

占据概率常用对数几率表示 $\ell = \log(p/(1-p))$，这样独立观测的证据可以相加。采用同一**先验**和一致的逆传感器模型时

$$\ell_t = \ell_{t-1} + \ell(z_t) - \ell_0.$$

减去先验项是避免每次都重复计入先验。先验 p=0.5，单次观测更新给 p=0.7；在条件独立的教学假设下，两次同样证据得到几率 $(7/3)^2$，即概率约 0.845。真实相邻帧常高度相关，重复添加可能让系统过度自信。OctoMap。

高程地图记录地面高度及变化，适合足式行走的落脚与地形判断，但单层高度难以完整表达桥下空间。**TSDF**（Truncated Signed Distance Function，截断有符号距离函数）在表面附近融合有符号距离，适合重建表面；ESDF（Euclidean Signed Distance Field，欧氏有符号距离场）提供到障碍表面的距离，便于查询安全间隔或构造规划代价。TSDF 的局部截断距离不能直接当作任意点的完整障碍距离。Voxblox。

图9.5：地图表示决定支持哪些问题。可视化漂亮、便于定位、适合避障和适合抓取，是不同要求。

语义地图额外记录“桌子”“杯子”等标签，实例地图区分具体对象。标签可以落在体素、点、网格或对象节点上，不必重新发明一张地图。动态目标需要时间和更新规则：昨天桌上的杯子位置不能永远当作当前事实。

9.6.2 学习深度、重建与神经场景表示

学习深度的标签与尺度从哪里来

单目深度网络输入图像，输出每个像素的深度或逆深度。监督训练可以使用有效 **RGB-D**、激光投影或其他重建结果作为标签；无标签方法可以用多视角**重投影**与光度一致性产生训练约束。标签本身也有空洞、噪声和选择偏差，需要有效掩码。

自监督重投影往往同时依赖深度和相机运动，动态物体、遮挡和光照变化会破坏假设。深度与运动尺度还可能共同缩放。因此“没有人工标签”不等于“没有建模假设”，也不等于输出天然是准确米制距离。

NeRF 和 3D Gaussian Splatting 在拟合什么

神经辐射场（Neural Radiance Fields，NeRF）用网络根据三维位置预测密度，并根据位置与观察方向预测颜色。为了与照片训练，沿每个像素的相机射线查询这些量，再做体渲染：靠前且密度较大的部分贡献颜色，也削弱后方贡献；透明区域则允许射线继续读取更远处。渲染像素与真实像素的差异，反向更新场景网络。多视角照片共同约束同一空间表示，因此可从新视角渲染。这里拟合的是场景的外观与几何线索，尚未建立机器人施加动作后的动力学。NeRF。

三维高斯泼溅（3D Gaussian Splatting，3DGS）把场景表示为许多带位置、椭球形状、颜色和不透明度的高斯基元。将它们投影到图像，按遮挡顺序合成颜色，再通过图像误差调整这些参数。与逐点查询网络的 NeRF 相比，它采用显式基元和另一种渲染计算组织。这里的高斯主要规定一个基元在空间中怎样展开，并不是机器人对物体未来位置的不确定性分布。渲染拟合很好，仍需另行检查表面尺寸、空洞与碰撞查询是否可信。3DGS。

这两类表示擅长新视角合成，也可以扩展到定位、几何和语义。用于机器人时必须额外验证尺度、表面精度、动态更新、可见区域以外的不确定性及距离查询。一个重建的透明杯在渲染图中很好看，并不说明其表面可以安全用于夹爪碰撞检查。

经典多视角重建、学习深度和神经场可以组合：几何提供位姿与尺度，学习先验补充弱纹理区域，任务监督强调需要的结构。但补全的部分应明确属于推断，而非已观测事实。评估应分别考察测量区域与补全区域，防止把“合理想象”误报为准确重建。

9.6.3 体渲染的权重怎样表示前后遮挡

沿同一条像素射线，查询得到第 i 段的体积密度 σ_i、长度 Δ_i 和颜色 c_i。这一段吸收的比例为 $\alpha_i = 1 - \exp(-\sigma_i \Delta_i)$；在到达它之前仍透过的比例为 $T_i = \prod_{j<i}(1-\alpha_j)$。于是像素颜色近似为

$$C = \sum_i T_i \alpha_i c_i + T_{\text{end}} c_{\text{background}}.$$

前面的透明部分让光线继续，前面的高密度部分则减少后方贡献。比如前段红色、不透明度 0.8，后段蓝色、不透明度 0.5，背景黑色。红色权重为 0.8，蓝色权重为 $(1-0.8)\times 0.5 = 0.1$，余下 0.1 透到黑色背景，结果是 $C = (0.8, 0, 0.1)$。两段颜色没有直接取平均，因为它们在空间中有前后遮挡。NeRF 的渲染目标与公式

训练通过像素误差调整颜色与密度。错误像素可能由颜色错、表面位置错、遮挡错或相机位姿错造成；多视角观测共同限制这些解释。新视角渲染得好，说明外观模型能解释相应观察，机器人所需的尺度、表面和碰撞距离仍需要专门检验。神经场用于感知，世界模型用于预测动作后变化，它们可以共享表示，但学习问题不同。

9.6.4 NeRF 原图：从空间查询到一张可渲染图片

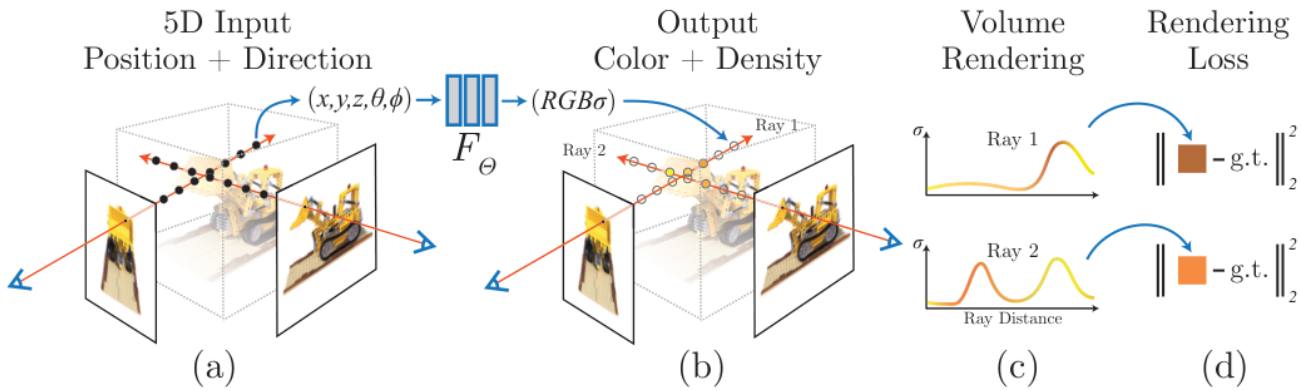


图9.6：Mildenhall 等，NeRF 论文 Figure 2。沿相机射线查询空间位置与方向，再把网络输出组合成像素，并与真实图像比较。原论文。

图(a)沿一个像素对应的相机射线取多个位置；每个查询包含三维位置和观察方向，所以称5D Input（五维输入：三维位置加描述方向的两个自由度）。图(b)的网络为位置输出体积密度，为位置与方向输出 RGB（Red, Green, Blue，红绿蓝）颜色。图(c)进行体积渲染：靠前而密度较高的区域会挡住后面的贡献，沿射线将各处颜色按透过与吸收关系累积成像素。图(d)再把预测像素与已知相机图像的对应像素比较，误差反传到网络参数。

因此，监督是多视角图片，学习对象是可供空间查询的密度和颜色函数，使用时可从新相机位姿渲染图像。密度适合解释光线的可见性，却不天然等于已标定的刚体表面或机器人碰撞距离。若要用于规划，还需建立相应几何查询并检查物理尺度。这个区别能帮助判断“场景看起来重建好了”是否足以服务抓取。NeRF 作者项目说明

9.6.5 练习题

题9.6-1 占据地图、语义地图与神经辐射场分别适合回答什么问题？为什么渲染质量高的场景表示未必直接适合机器人碰撞检测？

9.6.6 参考答案

题9.6-1答案 占据地图回答“某个空间区域有障碍、没有障碍，还是没有观测过”。它适合导航和碰撞检查，未知区域不能自动视为可通行。语义地图在位置或几何上添加类别、物体身份等信息，回答“杯子在哪里”“哪片区域是桌面”。两者可放在同一张地图中，并非互相替代。

NeRF（Neural Radiance Field，神经辐射场）用网络把空间位置和观察方向映射为体积密度与颜色，再沿相机射线合成图像。它主要回答“换到这个观察位置，会看到怎样的画面”，适合新视角合成，也可以辅助几何重建。

高质量渲染只表明已知或相近视角下的像素拟合好。遮挡内部可能没有约束；薄物体、透明物体和表面边界可能含糊；拟合图像的体积密度也不能直接当成可靠占据概率。机器人还要查询整个夹爪或机身的扫过体积，考虑定位误差与未知区域。实际可把重建结果转换为占据、网格或距离场，再用独立深度或接触数据检验几何，并加入安全余量。应分别评价图像重建误差与碰撞漏检率。

9.7 检测、分割、物体位姿与时间跟踪

9.7.1 检测、分割与三维语义学习

框、像素区域与实例分别提供什么

目标检测常输出类别、边界框与得分。框告诉你大致在哪里，框内可能混有背景和其他物体。语义分割给每个像素类别；实例分割进一步区分同类的不同对象。抓杯时，用检测框直接取全部深度，可能把桌面深度也算进去；实例掩码能改善目标点云，但边界错误仍会带入背景。

训练一个分割网络，典型输入是图像，输出是各像素的类别分数，标签是对应像素的类别或实例区域。分类**交叉熵**让正确类别得分提高；区域重叠损失强调前景形状；有效掩码排除无标签区域。更新的是网络参数，运行时图像推理通常不再修改参数。

对二分类像素，损失可写为 $-[y\log p+(1-y)\log(1-p)]$。这里 y 是标签，p 是网络预测前景概率。设计目的不是学习“对数”，而是让错误且自信的预测受到较强惩罚。例如真正杯子像素 y=1，模型给 p=0.1，会比给 p=0.8 受到更大损失。类别不平衡时，大量背景可能掩盖小杯把手的错误，需要采样或合理加权。

二维语义如何落到三维

有深度和相机位姿后，可将带标签的像素反投影到点云或地图。多帧同一位置的标签可以融合，但投影误差、遮挡和动态物体会污染结果。相机看到前面的杯子，不代表后方被遮住的桌面也属于杯子；融合需要深度一致性和可见性检查。

三维网络也可直接输入点云，输出逐点类别、实例或几何特征。训练标签可以来自人工标注、多传感器对齐或仿真。合成标签容易完整，但真实传感器的缺失、噪声和材质分布可能不同。一个三维特征向量不是天然的物理变量，要靠训练目标和验证说明它包含哪些有用信息。

提示式分割模型 SAM 能根据点、框等提示生成区域；语言条件检测模型 Grounding DINO 能结合文字寻找候选对象。二者可组成感知流程，但“能分出一个区域”与“正确理解指令中的对象”仍需分别验证。工具输出不应直接被视为人工真值。SAM 原论文、Grounding DINO 原论文。

9.7.2 物体六自由度位姿为什么难于找一个中心点

位姿依赖物体坐标的定义

物体位姿 T_{CO} 表示物体系 O 到相机系 C 的变换。平移是物体原点在相机系中的位置，旋转是物体系的朝向。若 **计算机辅助设计**（Computer-Aided Design，CAD）模型原点在底部，而另一模型原点在重心，两者的平移数值不同；没有坐标定义就无法比较。

物体中心点不能告诉机器人杯口朝向、把手位置或可接近方向。CAD（Computer-Aided Design，计算机辅助设计）模型可提供对象几何。对带把手的杯子，完整姿态能把 CAD 模型中的把手位置变换到

机器人系，支持指定部位抓取。对于无模型的新物体，也可以直接预测候选抓取或局部接触几何，不一定先求完整物体位姿。

一类典型训练与使用流程

网络可输入 **RGB** 或 **RGB-D**，以及对象**先验**，预测二维关键点、对象表面对应或直接预测姿态。若输出二维关键点，训练标签可由已知 CAD 点与真值位姿投影获得；损失约束关键点位置；推理时把二维预测与三维 CAD 点送入 **PnP**。这时网络预测的是对应，PnP 求的是运行时位姿，它们没有合成一个模糊的“位姿模型”。

另一类方法利用渲染候选姿态与真实图像比较，迭代细化位姿。FoundationPose 提供新对象的位姿估计与跟踪，使用 CAD 或少量参考图像等对象信息，结合预训练与姿态细化。它的“新对象”能力不意味着完全没有对象先验，也不意味着所有透明、形变物体都适用。FoundationPose 论文、官方代码。

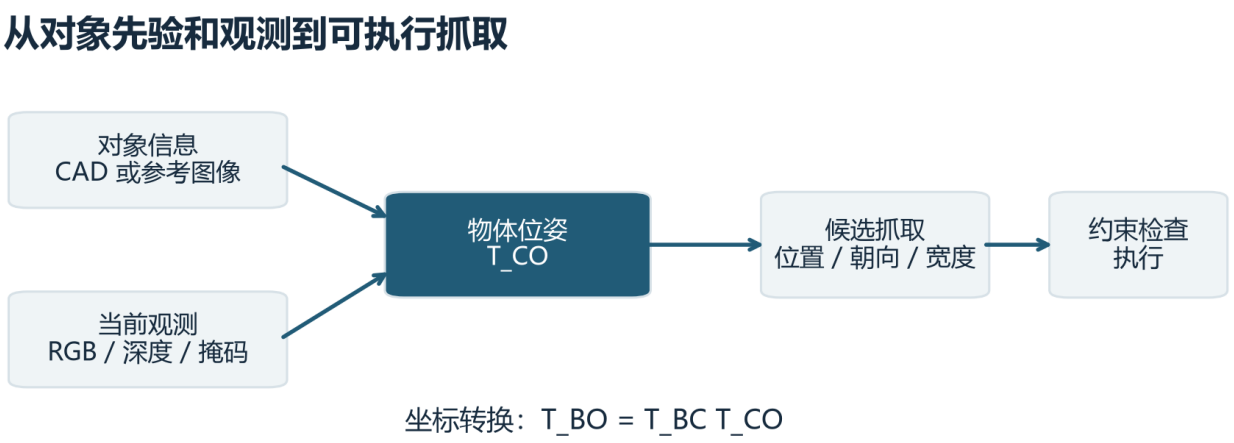


图9.7：位姿估计把对象几何放到当前场景中。抓取还需要接近方向、碰撞、夹爪尺寸和物理接触条件。

对称性意味着有些答案无法由图像区分

无纹理圆柱绕轴旋转，图像可能完全不变。强迫网络给出唯一朝向，会把多个等价解误当成训练冲突。评估也要考虑对称性，而非直接比较三个欧拉角。

ADD 类指标比较对象表面点经过估计位姿与真值位姿变换后的距离。对称对象需要相应处理，否则视觉等价姿态也可能被罚。BOP 的可见表面和对称感知指标提供了更细的评价方式。对任务还要追问：图像等价是否行动等价？如果一个隐藏接口要求固定朝向，单张图像无法消除歧义，需要换视角或接触反馈。BOP 评价定义。

9.7.3 跟踪是跨时间维护同一个对象

单帧检测每次给出一组杯子，不能保证编号稳定。跟踪需要将当前检测与历史对象关联，维护位置、速度、身份和不确定性。机器人若把人的杯子与桌上杯子交换身份，语义类别完全正确也可能执行错任务。

一种常见设计用运动模型预测下一帧位置，用观测更新，再构造关联代价。代价可以包含位置差、外观差与类别不一致。分配算法决定哪些观测对应哪些旧对象；距离门限排除不合理候选。门限应结合预测不确定性：短时遮挡后的可能范围应比稳定可见时大。

例如一个杯子在 x=1.0 米，速度为 0.2 米/秒，0.1 秒后预测为 1.02 米。下一帧有检测 1.03 和 1.8 米，运动一致性偏向前者。但若杯子被人突然拿走，恒速模型失效，不能继续坚信旧预测。运动模型是短时近似，真实操作会产生突变。

遮挡后没有新观测来校正估计，应按运动模型和过程噪声传播不确定性；在常用恒速模型中，位置预测范围通常随遮挡时间扩大。持续失去观测后，可根据预测范围、目标身份和任务需要降级或删除轨迹。重新出现时，外观和场景关系帮助重识别，但同款杯子仍可能不可区分。任务可以通过额外观察、颜色标记或交互确认减少歧义。

跟踪输出应带上时间戳。对 0.2 米/秒移动的目标，200 毫秒旧结果已落后约 4 厘米。低平均位置误差不能替代低信息年龄，控制需要知道估计对应哪个时刻。

训练跟踪网络时，标签通常包括跨帧实例身份与位置，损失可约束关联或运动预测；运行时**卡尔曼**更新和数据关联则更新当前轨迹。评估除了每帧检测准确度，还要看身份切换、丢失恢复和延迟。用静态图像检测指标不能证明跟踪可靠。

9.7.4 物体位姿还包含对称性问题

圆柱杯子绕轴旋转后图像可能接近不变，多个姿态在外观上等价；带把手杯子则可能需要精确朝向。评价旋转误差时应考虑任务与对称性，避免惩罚视觉上不可辨识且任务无关的变化，或忽略影响抓取的把手朝向。

跟踪保存身份和时间连续性，但遮挡后重现可能发生身份交换。语言指令指向“刚才那只红杯”时，这种交换会直接影响动作。应在遮挡、相似物体和目标切换条件下测试，而非只报告单帧检测分数。

9.7.5 练习题

题9.7-1 检测框、实例分割和六自由度物体位姿对抓取各提供什么？追踪为何不能仅逐帧选最高置信度检测？

9.7.6 参考答案

题9.7-1答案 检测框提供图像中的粗位置和类别，适合先找到候选杯子；框内可能仍含桌面或其他物体，不能把框中心直接当成可靠抓取点。实例分割提供“哪些像素属于这一个物体”，结合深度可提取该物体点云，减少邻近物体干扰。六自由度位姿提供物体的三维位置与朝向；若已知物体几何及相对于物体的抓取位姿，就能把抓取模板转换到相机或机器人坐标。它们增加的是不同层次的信息，任何一个都不能单独保证抓取可达、无碰撞且稳定。

追踪要维持同一物体的身份，而置信度主要表示当前检测有多可信。例如两只杯子交错运动，左杯在第 1 帧得分较高、右杯在第 2 帧得分较高，逐帧取最高分会无故换目标，机械臂随后追向另一只杯子。应先根据上一帧位置与运动预测候选范围，再结合位置、外观、三维距离等完成数据关联；遮挡时保留短期轨迹和不确定性，重新出现时再核验身份。评价也需包含身份切换、遮挡恢复和位姿时序误差，而不只看逐帧检测准确率。

9.8 语言指代、可供性、主动感知与触觉

9.8.1 语言指代、场景关系与可供性

“桌上靠左的蓝杯子”怎样成为一个具体对象

语言指代需要将文本与场景实例关联。可将候选杯子的颜色、位置和视觉特征与文本比较，再结合“在桌上”“靠左”等关系筛选。左侧是观察者左侧、机器人左侧还是人的左侧，必须由任务语境定义，不能只把句子编码为一个向量后忽略参照系。

场景图用对象节点和关系边记录“杯子在桌上”“人靠近门”“把手属于杯子”等信息。边可来自几何规则、网络预测或语言**先验**，可靠性不同。语言模型觉得杯子通常在桌上，不等于当前杯子确实在桌上；地图中的关系需要观测和时间维护。

可供性是在任务条件下判断行动机会

可供性（affordance）描述对象或区域可能支持什么行动。例如杯把手可用于某种握持，杯口可用于倒入液体，桌面可用于放置物体。它依赖夹爪形状、机器人可达性、任务目的和物体状态，不只是一个与机器人无关的类别标签。

可供性网络可输入图像、点云、任务文本与机器人状态，输出区域分数或候选动作。训练标签可以来自人工区域标注、仿真接触成功或真实尝试结果。不同标签决定它学到的含义：人工“可抓区域”未必包含摩擦和碰撞，真实成功标签又可能混入控制器能力和探索偏差。

抓取网络也可以直接预测夹爪位姿、开口宽度和得分。Contact-GraspNet 是利用深度场景生成抓取的代表入口。网络得分需要通过碰撞检查、运动学可达性和任务约束筛选；它不是抓取成功的物理保证。Contact-GraspNet 原论文。

对“把杯子交给人”，抓杯身可能方便，但会遮住把手；交接方向与人手可接近性也重要。任务成功不能只由“夹爪闭合后杯子没掉”定义。可供性评估应明确对应抓起、使用、交接还是其他行动，并测试未见对象和机器人条件。

9.8.2 导航与 VLA 怎样使用感知结果

从地图到一条可以执行的路径

全局规划在较大尺度选择路线，局部规划根据附近障碍和机器人运动限制产生短时行动。占据栅格常转成代价地图：障碍附近扩大高代价区域，反映机器人身体尺寸与余量。一个格子空闲并不保证整台机器人都能通过；机械臂伸出后，碰撞体积也会变化。

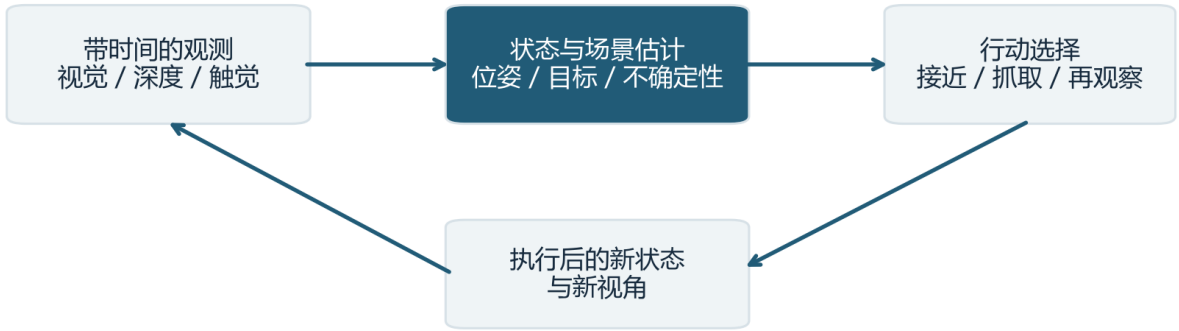
局部规划器需要当前位姿、局部地图、目标和运动限制。物体导航还需要把“蓝杯子”变成可到达的观察或操作位置。目标对象坐标不等于底座应该停的位置，机器人通常应停在满足视野、可达性与安全距离的位置。

显式几何与学习策略各自提供什么

VLA 可以接收当前图像和机器人状态，也可以接收位姿、目标点、局部地图或历史记忆。显式几何使某些约束可解释、可检查；学习模型能结合复杂语义与经验，但输入接口越多，时间与坐标一致性越重要。

有些端到端策略不需要独立地图即可完成短任务；长距离、多房间或跨视角任务可能更依赖记忆和空间一致性。不能仅从模型输入有图像，就推断它解决了长期定位；也不能仅因为有 **SLAM**，就推断语义目标和动作选择已经解决。

行动改变观测；新的观测再修正行动



闭环使用更新信息；检查信息年龄，而非仅看单帧推理速度。

图9.8：每次行动改变机器人视角，新的观测更新目标与空间理解。输出时间戳决定行动使用的是哪一刻的信息。

一个模块平均每帧 50 毫秒，看似满足频率要求，但排队后输出可能已经落后 300 毫秒。系统应监测端到端信息年龄、丢帧与恢复情况。定位丢失、目标歧义或深度无效时，策略需要触发重新观察、减速或停止该动作，而不是默认填入上次结果永久继续。

做模块组合实验时，保持动作策略和控制器相同，替换感知模块，能更清楚检验感知贡献。做系统实验时再报告任务成功、时间、碰撞或人工干预，避免用模块误差代替整机效果。

9.8.3 主动感知、触觉与力信息补上什么

观察不是只能被动发生

如果两个同款杯子重叠，当前图像可能无法判断把手归属。机器人可以侧移相机、转动物体或靠近观察，让未知量变得可辨识。**主动感知**把行动用于获取信息，而不仅用于完成物体搬运。

候选观察动作需要在执行前评价，因此不能用尚未看见的真实结果来计算收益。设 X 是待辨认的量，例如把手属于哪个杯子，当前历史为 $\mathcal{H}$。一种**预期信息增益**为 $I(a) = H(X \mid \mathcal{H}) - \mathbb{E}_{o \sim p(o \mid a, \mathcal{H})}[H(X \mid \mathcal{H}, a, o)]$，其中 $H(\cdot)$ 表示熵。前项是当前不确定性，后项是按观测模型预测各种可能新观察后，平均还剩多少不确定性。选择动作时再扣除运动、时间和风险代价。若没有可信概率模型，也可以用视差、可见面积等代理指标，但要验证它们是否真的减少了任务关键歧义。

主动观察是否值得，取决于额外时间与失败成本。移动五秒才能多看一眼，对简单抓取可能不划算；对插接、液体容器或身份歧义，补充观察可能显著减少损失。不能只比较观察后感知精度而忽略付出的行动成本。

接触带来了视觉没有的证据

力传感器可以感知接触和受力变化，触觉传感器可以感知局部接触分布、滑移或表面信息。它们能帮助判断是否真正夹住杯子，以及视觉估计的接触位置是否准确。力信号还会受到机器人自身加速度、重力和传感器偏置影响，需要补偿和坐标转换。

多模态融合可在状态层进行，也可由网络融合图像与触觉特征。训练网络时要说明输入对应时刻、标签来自何种接触结果，以及是否包含失败数据。只训练成功示范可能使模型难以识别“夹空了”这种关键状态。

触觉并非解决所有视觉歧义的捷径。主动接触需要先满足可达性和安全条件，且只能观察接触附近。更合理的系统是让视觉提供全局候选，几何约束接近过程，触觉和力在接触后修正局部判断。

9.8.4 信息增益要对尚未看到的结果求平均

设杯子被挡住，左、右两个位置各有一半概率。当前熵为 $H(X) = \log 2 \approx 0.693$ nat。候选观察动作 a 能给“左”或“右”的测量，测量有 90% 概率判断正确，并且左右对称。

观察之前不知道会收到哪个结果。收到“左”后，用贝叶斯公式得到后验 $P(X = \text{左} \mid o = \text{左}, a) = 0.9$；收到“右”后，后验相反。两种结果各有一半概率，后验熵都为

$$H(X \mid o, a) = -0.9 \log 0.9 - 0.1 \log 0.1 \approx 0.325.$$

所以执行前能计算的是平均信息增益 $0.693 - 0.325 = 0.368$ nat。若另一观察动作只是随便报左或右、正确率 50%，后验仍各一半，信息增益为零。第一种动作提供了区分位置的证据，第二种只是产生了一条新消息。POMDP 教程中的信念更新

预期信息增益把每个可能观察的后验改善，按它发生的概率加权。评分还要扣除观察动作的时间、移动和交互代价。一个角度更清楚却需要长时间绕行，未必值得；轻触能确认接触，却也会改变物体状态。主动感知的建模价值在于，让“先多获得一条信息”成为有明确目标和代价的行动选择。

9.8.5 练习题

题9.8-1 指令“拿靠近盘子的杯子”需要哪些推理与空间信息？什么时候主动移动相机或使用触觉比继续加大视觉模型更有效？

9.8.6 参考答案

题9.8-1答案 先识别“杯子”这一候选集合和“盘子”这一参照物，再根据“靠近”筛选目标。仅有类别识别还不够：需要把每个物体的位置或表面放到共同坐标系，明确比较的是桌面上的中心距离、最近边界距离，还是其他与任务有关的距离。透视投影会让相距很远的物体在图像中靠近，所以复杂场景通常需要深度、多视角或已知桌面几何。选定杯子后，还要估计可抓取部位、夹爪进入方向和周围空隙。

若盘子后面的杯柄完全被遮挡，再大的视觉模型也没有当前视角的直接观测证据。安全地横移相机可露出杯柄，并用新观测区分原先都可能成立的解释。若接近物体后视觉被手遮住，或问题变成“是否已经接触、是否滑动、插入是否卡住”，触觉与力反馈往往更直接，因为它们测到的是接触状态。

选择主动感知，应比较新观测预计减少的不确定性与移动时间、碰撞风险。可验证的方案是：在同一模型上比较固定视角和换视角，再比较纯视觉与加入触觉，看目标选择、接触判断或任务成功率是否改善。这样才能判断瓶颈是缺少信息，还是现有信息没有被模型利用。

9.9 移动操作感知项目与评价诊断

9.9.1 一个抓杯任务需要哪些“知道”

自身位姿、地图和目标位姿分别回答什么

位姿（pose）是位置与朝向的组合。机器人站在桌子左侧与右侧，位置不同；站在同一个地方面向桌子与背向桌子，朝向不同。只说“距离桌子一米”不足以决定下一步往哪里走。

SLAM（Simultaneous Localization and Mapping，同时定位与建图）在没有完整已知地图的情况下，根据观测同时估计机器人运动和环境结构。难点是相互依赖：要把当前看到的桌子放到地图上，需要知道相机位置；要根据桌子判断相机位置，又需要知道桌子在哪里。系统通过跨时间的共同观测，让两类未知量一起受到约束。

定位不总是 SLAM。如果地图已经给定，只估计机器人在地图中的位姿，是定位；如果相机轨迹已知，只整理环境，是建图。视觉里程计主要估计连续帧之间的运动，通常没有完整的全局地图维护和**回环**修正。它们不是互斥的产品名称，而是不同问题范围。

感知还要回答杯子的类别、实例、几何和运动。“这是杯子”是语义判断；“它是指令中的蓝杯子”是目标关联；“杯口朝上，把手朝右”涉及物体姿态；“杯子正在被人移动”涉及跟踪。SLAM 的相机位姿不能自动替代这些结果。

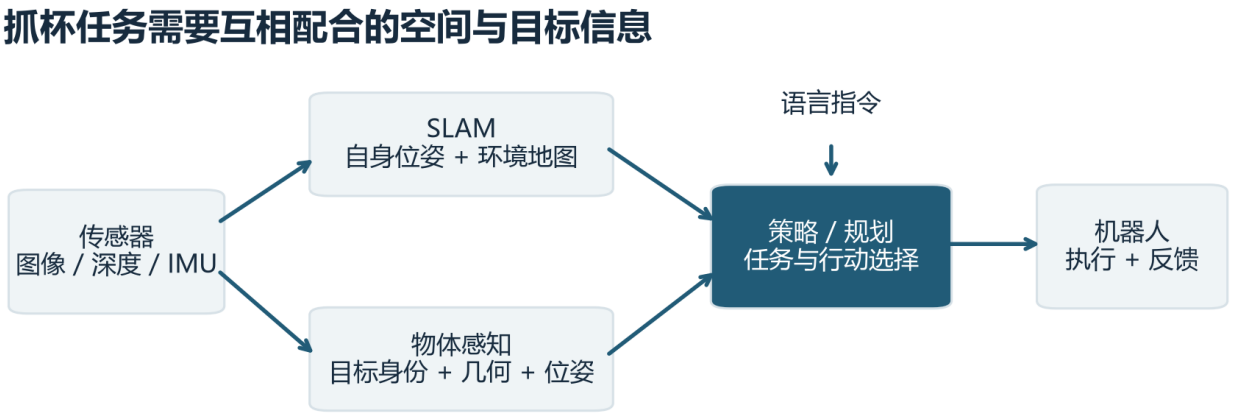


图9.9：不同模块提供不同信息。地图记录空间，物体感知解释目标，策略根据任务和观测选择行动。

它们怎样与 VLA、世界模型协作

VLA（Vision-Language-Action，视觉—语言—动作模型）把视觉、语言及其他状态映射为动作，在这个系统里承担策略的角色。显式 SLAM 可以为它提供地图、位姿或路径信息；也有策略直接利用图像历史行动，并不调用独立 SLAM。不能因此断言所有 VLA 内部都学到了准确地图。

世界模型预测行动后的变化；定位感知主要估计现在和过去。一个物体位姿网络输出当前杯子的姿态，一个世界模型预测推杯后的位置，一个策略选择是否推。即使共享编码器，三者的任务仍有区别。

本书学习的目标是理解这些接口。当机器人抓偏时，你应能分辨是目标选错、坐标转换错、估计过时、几何不准，还是动作策略本身有问题，而不是统一归因于“模型不够大”。

9.9.2 评价指标要与建模自由度和任务对齐

轨迹误差不能绕开坐标对齐

ATE（Absolute Trajectory Error，绝对轨迹误差）比较估计轨迹和参考轨迹的位姿差，常先做合理坐标对齐。**RPE**（Relative Pose Error，相对位姿误差）比较一段时间或距离内的相对运动误差，更侧重局部漂移。两者反映不同方面，不能只挑有利的一个。TUM RGB-D 基准论文。

对纯单目、无米制约束的方法，允许相似变换对齐可以消除任意尺度；对宣称米制准确的方法，任意缩放会掩盖尺度错误。应报告采用 **SE(3)** 还是含尺度的 Sim(3) 对齐、参与对齐的轨迹范围及失败处理。对齐去掉的是坐标规范差异，不应成为抹去系统性错误的手段。

只在成功跟踪段计算均方根误差，可能奖励经常丢失但短段很准的方法。需要同时报告运行完成比例、丢失次数、恢复时间，以及失败轨迹是否计入。KITTI 里程计基准关注不同长度轨迹段上的运动误差，适合其传感器与道路条件，不能直接替代室内抓取验证。KITTI 里程计评价。

感知指标各有盲区

分割 IoU（Intersection over Union，交并比）是预测区域与真值区域交集除以并集。预测 80 个像素、真值 100、交集 60，并集为 120，IoU=0.5。它不直接说明三维位置是否准；边界附近的小误差可能对夹爪接近更关键。

深度要报告单位、有效范围、无效像素处理和尺度对齐。物体位姿要报告对象**先验**、对称处理、遮挡条件、运行时间。地图质量还要看可查询性和动态更新，不能只给新视角图像质量。

任务指标应覆盖找到目标、导航到位、抓起、交接各阶段。阶段成功率不能随意相乘：失败可能相关，而且后续阶段样本是前面筛选过的。报告整段任务成功并记录失败位置，才能定位瓶颈。

9.9.3 移动操作感知流程：走到桌边，抓住蓝杯子

先定义接口与检验

项目目标是让移动机械臂在已知室内范围找到蓝杯子，接近桌子并完成抓取。先限定杯子是否透明、是否有 **CAD**、桌面是否静止、是否允许人工初始定位。这些是方法条件，不是实现完以后再补的注释。

接口表可以这样设计：

模块	输入	输出及检查重点
标定与同步	标定序列、机器人姿态、原始时间戳	内参、外参、时间规则；单位与方向
定位建图	图像、深度或激光、IMU	带时间的底座位姿、地图、状态质量
目标感知	当前图像、指令、深度	实例掩码、对象坐标或位姿、歧义标记

模块	输入	输出及检查重点
导航	底座位姿、障碍地图、操作目标	可执行局部运动与重新规划条件
抓取	目标几何、机器人状态、夹爪条件	候选抓取、可达与碰撞检查、动作
反馈	新图像、触觉、力、执行状态	继续、修正、重试或重新观察

一个可以手算的局部链路

先用教学简化：相机和底座方向相同，相机在底座系的位置为 $(0.2, 0, 0.5)$ 米；底座在世界系为 $(1, 2, 0)$ 米，方向也相同。目标点在相机系为 $(0, 0, 1)$ 米。转换到底座得到 $(0.2, 0, 1.5)$，再到世界得到 $(1.2, 2, 1.5)$。这里的轴约定刻意简化，真实光学相机坐标轴通常与底座不同，必须代入实际旋转。

若错误地漏掉相机高度，目标会低估 0.5 米；若仅漏掉时间对齐，静态时可能看不出，机器人移动时才抓偏。先做这种可预测的输入输出检查，再接复杂网络，有助于排除系统接口错误。

按可诊断的顺序实现

先在固定底座、固定目标条件验证目标几何和坐标转换；再让底座运动，检查同一个静态目标在世界系中是否保持稳定；再加入导航；最后加入动态目标与闭环抓取。每一步记录原始传感器、估计、时间、策略动作和结果。

如果假设物体静止，可以在接近后重估目标位姿，以减少早期远距离深度误差。若物体移动，不能用旧位置规划完整开环动作；应根据运动状态和信息年龄决定是否重规划。成功定义也应包含正确对象，而不是任意一个杯子被抓起。

这个项目不要求每个模块都自己训练。可以使用成熟定位系统和预训练感知网络，把研究重点放在目标关联、鲁棒闭环或信息选择上；但需要记录这些依赖的版本、先验与失败边界，避免把别人的能力误报为自己的机制贡献。

9.4.4 研究问题、基准与公平比较

从明确失效原因提出机制

“加入大模型提升 SLAM”过于宽泛。更可检验的问题是：动态物体占主要视野时，语义与运动一致性联合筛选，能否降低错误运动约束且保持足够静态信息？它指向具体机制、扰动条件和评价指标。

组合已有方法可以形成研究贡献，但要证明组合解决了一个真实接口或建模问题。例如语言指代、三维实例跟踪与闭环抓取组合后，是否减少跨视角身份切换？如果只把现成模块串起来，性能变化可能来自更强检测器或更多计算。应设置模块替换、相同算力与相同先验的基线，并分析故障链。

先验和数据边界必须公平

一个方法使用 CAD，另一个不使用；一个使用 **RGB-D**，另一个纯 **RGB**；一个允许测试对象参考图像，另一个仅类别文本，不能用一个总分直接说明建模更强。可分别比较同条件效果与不同条件下的成本收益。

训练集与测试集应按对象、场景、采集序列或机器人配置合理划分。相邻视频帧随机分开，会让测试场景几乎已被看过；同一物体不同角度分开，也未必验证新对象泛化。仿真标签、预训练数据和自动伪标签的来源都应交代。

TUM 可用于 RGB-D 轨迹评价，KITTI 可用于道路里程计，BOP 可用于对象位姿。选择它们是为了检验各自问题，不是因为分数容易获取。具身结论还需要真实任务或与任务相关的测试：遮挡、光照、目标移动、地图变化及部署延迟。

论文叙事需要证据链

动机指出当前系统在哪个条件失效；方法说明新增信息或约束怎样缓解失效；实验检查对应条件下的中间量、模块效果和任务收益；局限说明哪种假设仍然不成立。除了消融总分，还可以分析正确匹配比例、不确定性**校准**、目标身份切换与重新观察次数。

成功率应有样本数和置信区间，随机性明显时重复实验。失败案例不是装饰：它能判断问题在机制边界、数据不足还是实现错误，也能支持下一轮改进。

9.9.5 排查失败与安排后续学习

症状怎样指向不同层次

症状	优先检查的原因	一个可验证的检查
静态目标随机器人移动而漂移	外参、时间、底座位姿	固定目标，比较不同速度下世界坐标
只在图像边缘抓偏	畸变、内参、深度对齐	分区域统计重投影与目标位置误差
长走廊位置不稳定	几何退化、尺度或弱纹理	比较各方向误差与观测布局
回环后地图折叠	错误回环或过强权重	查看候选匹配与几何验证残差
类别正确却抓到另一只杯子	指代或实例身份错误	记录跨帧身份与指令关联
离线准、在线慢且抓偏	排队、过时数据、资源竞争	记录采集到执行的信息年龄
分割好但抓取失败	三维、可达性、碰撞或接触	固定分割，分别替换几何和动作模块

排查顺序先看原始数据，再看单位、坐标、时间，随后看对应与噪声，最后看模型能力。不是因为学习模型不重要，而是系统错误会让训练目标与实际应用含义不一致；此时更多训练无法消除错误接口。

工具应按问题学习

OpenCV 用于相机、图像几何与 **PnP**；PCL 用于点云处理；GTSAM 用于状态与**因子图**；**ROS 2** 组织机器人通信、时间和坐标变换。Eigen 是线性代数工具，不是定位算法。学习工具时先做一条可解释的小链路：输入对应、求位姿、投影检查，再接数据集或机器人。

建议完成三个练习项目：相机标定与深度反投影；公开序列上的轨迹估计和误差分析；固定相机下目标实例到机器人坐标的转换。随后再做完整移动抓取。暂时不需要从零实现所有 SLAM 系统或读完每一类三维网络。

读完本书，应能给任意新方法提出五个具体问题：它估计什么变量？使用什么观测和先验？训练或优化改变什么？哪些场景不可辨识或假设失效？结果如何接到任务、如何验证？这五个问题比记住更多算法缩写更有用。

9.9.6 感知误差应在任务链中评价

位置平均误差几毫米，可能在宽口杯抓取中足够，在插入任务中却不够。分别报告定位、地图、物体位姿、接触和任务成功，再测试感知误差、延迟与丢帧对动作的敏感性。这样才能判断需要更强感知网络、额外传感，还是调整策略和控制接口。

9.9.7 练习题

题9.9-1 全局定位误差很小，但机械臂仍无法插入小孔。为什么 SLAM 指标不足以说明操作感知准确？需要哪些局部指标？

9.9.8 参考答案

题9.9-1答案 SLAM 轨迹分数主要评价全局定位，插孔需要工具相对孔口的局部位姿。手眼外参、工具中心、孔口深度、控制滞后或关节跟踪有偏差，都可能在全局误差较小时造成失败；评测中的坐标对齐还可能掩盖实际使用的固定偏移。

局部应测工具—孔口的横向、轴向位置误差及轴线夹角，报告高分位误差；分别核对孔口三维重建、遮挡时估计、手眼与工具中心标定，以及图像到控制使用的观测年龄。然后记录插入成功率、峰值接触力、卡滞和碰撞，连接几何误差与实际后果。

用已知精确孔位替换感知再执行：若仍失败，检查控制、机械柔顺性与执行映射；若成功，优先修正局部感知和标定。替换实验能定位误差来自哪里，而单个 SLAM 总分不能区分这几条原因。

第10章：工程专题：AI Infra 训练基础设施

模型训练依赖显存、数据和通信。本章作为工程专题，解释资源需求、分布式训练与采样系统，帮助定位训练规模和效率问题。

10.1 训练迭代、模型状态与显存需求

10.1.1 训练基础设施保存并推进学习状态

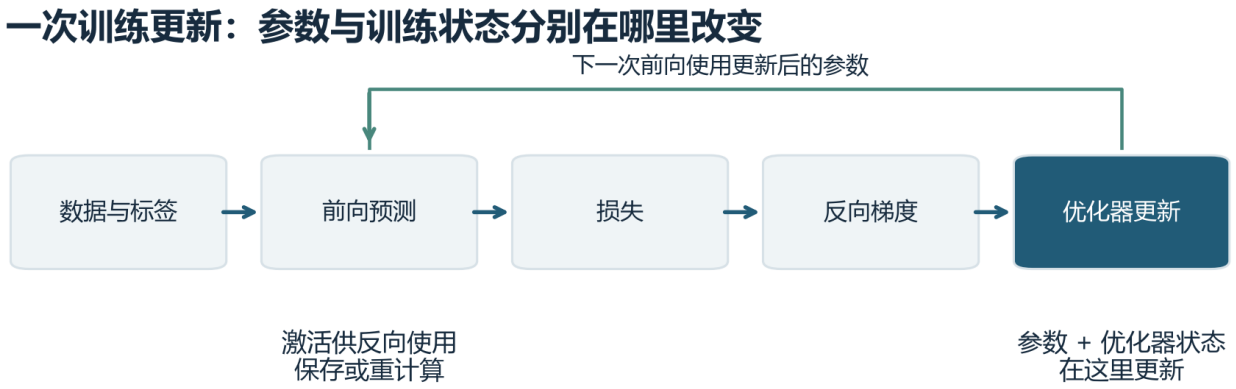
训练系统把数据组成批次，计算输出与损失，反传梯度，更新参数，并保存可恢复的状态。接下来的工程问题围绕这条流程展开：哪些量占显存，哪些运算可分到多卡，数据怎样持续供给，故障后怎样接着训练。

10.1.2 AI Infra 要保证什么，为什么具身训练特别复杂

一个 **VLA** 模型训练需要读视频、语言和机器人状态，构造输入与未来动作标签，把它们送到 **GPU**，算预测和损失，求梯度，更新参数，定期评估并保存状态。训练 Infra 组织这条链路，让它既符合学习问题，又在资源与故障条件下运行。

预训练通常从较大规模数据学习一般表示或生成能力；监督微调（Supervised Fine-Tuning，**SFT**）用目标任务示范调整模型；强化学习后训练根据行动产生的反馈更新策略。名字相同的阶段可能采用不同目标，需要看实际数据和损失。Infra 不决定某个目标是否科学，但必须确保执行的正是定义的目标。

具身数据不是一堆可独立打乱的图片。一次轨迹中，动作、相机和状态有时间关系；多摄像头、不同机器人、连续动作和接触事件需要一致接口。后训练还可能由仿真或真机生成新数据，采样速度、环境重置和策略版本都影响学习过程。



另有周期性评估与磁盘保存；RL 采样还会由策略行动产生新数据。

图10.1：一次训练包括数据构造、计算、更新、评估与保存。强化学习额外存在行动到新数据的回路。

Infra 的目标有三层。正确性要求数据、梯度和参数更新符合训练定义；效率要求资源与时间得到合理利用；可靠性要求中断、异常或规模变化后能够恢复。某个优化把 GPU 利用率拉高，却混入未来图像或漏掉部分梯度，就没有实现正确训练。

人工智能基础设施（Artificial Intelligence Infrastructure，AI Infra）的工作可以涉及数据格式、训练框架、分布式通信、编译、算子、调度和运维。**CUDA**（Compute Unified Device Architecture，统一计算设备架构）是 NVIDIA 的并行计算平台与编程模型。这里会解释必要机制，但不把集群管理员、CUDA 专家和模型研究员的所有任务都堆成一个入门门槛。研究生更需要先能定位瓶颈、理解更改是否改变学习目标，再选择深入方向。

10.1.3 一次训练迭代中，什么被计算，什么被保存

从一个动作预测问题看张量

张量（tensor）是带形状和数据类型的数值数组。假设一批有 b 条样本，每条预测 H 个动作，每个动作有 d 个分量，预测和标签形状可以都是 (b, H, d)。图像可能是 (b, V, C, h, w)，V 为视角数，C 为颜色通道。形状相同不保证意义相同：关节位置、关节增量和末端速度需要不同标签定义。

网络输入图像、语言和当前状态，输出未来**动作块**。一个简化连续动作损失是

$$L = \frac{\sum_{i,t} m_{it} \|\hat{a}_{it} - a_{it}\|^2}{\sum_{i,t} m_{it}}.$$

标签 a_{it} 来自示范动作；预测 $\hat{a}_{it}$ 来自模型；掩码 m_{it} 表示该时间步标签有效。这里每个动作向量的平方误差先求和，再按有效时间步平均；若希望按有效标量平均，还需相应除以 d。设计归一化方式必须前后一致。

轨迹尾部不足 H 时，补零用于存储对齐，这些位置应从动作损失排除。有效监督数为零时，应一致地跳过该批或按指定规则处理，避免无效分母进入参数更新。

前向、反向和更新是三件事

前向计算从参数得到预测，同时产生中间结果，称为激活值。反向传播通过计算图求损失对可训练参数的梯度；自动微分根据算子关系应用链式法则。梯度缓冲是保存这些数值的内存，并不是“又一个神经网络”。

优化器根据梯度和自己的状态更新参数。一次 backward() 通常不会自动完成参数更新；一次 optimizer.step() 也不会自动清空上轮梯度。多个微批次的**梯度累积**恰好利用了这种分离。PyTorch 自动微分说明。

训练除了权重，还要保存或重算反向所需的激活，并维护梯度与优化器状态，所以显存通常明显高于只存权重。冻结参数可减少其梯度和优化器状态，但激活能否省掉取决于梯度路径：若冻结模块之前还有可训练适配器，梯度仍需穿过冻结模块，相关中间量可能仍要保存；若只训练冻结编码器之后的动作头，编码器通常可在不构建反向图的情况下计算特征。先画出要更新哪些参数，再判断能省哪部分内存。

10.1.4 优化器状态为什么比模型权重还占空间

SGD 更新什么

最基本的梯度下降是 $\theta \leftarrow \theta - \eta g$，g 是当前梯度，η 是学习率。参数决定模型输出，梯度说明当前数据下怎样改变参数可能降低损失。小批梯度带有随机性，某次下降方向不保证整个数据集或机器人任务立刻改善。

SGD（Stochastic Gradient Descent，随机梯度下降）用抽取的数据估计梯度并更新参数；带动量的 SGD 会保存一份历史更新方向，让变化更平滑。**Adam**（Adaptive Moment Estimation，自适应矩估计）还为每个参数保存梯度一阶与二阶统计

$$m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t, \quad v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2.$$

m_t 平滑记录梯度方向，v_t 平滑记录梯度平方的尺度。它们从零开始，训练初期会偏小，因此采用 $\hat{m}_t = m_t/(1-\beta_1^t)$、$\hat{v}_t = v_t/(1-\beta_2^t)$ 做初始化偏差校正，再按 $\hat{m}_t/(\sqrt{\hat{v}_t}+\epsilon)$ 更新。分母使长期梯度尺度较大的参数不至于只因数值大就承担同样比例的大步长；分子保留较稳定的方向。Adam 的“自适应”来自这些历史统计，不意味着它会自行判断机器人任务是否成功。系统要为每个可训练参数保存两份统计及相应步数，恢复训练时也要恢复它们。Adam。

第一步 θ=1、g=2、学习率 0.1，忽略 ε，偏差校正后 $\hat{m}=2, \hat{v}=4$，更新量约为 0.1，θ 变为 0.9。这个例子说明 Adam 的更新不是简单等于学习率乘梯度 0.2。它不是对真实收敛或最佳学习率的保证。

系统为什么要关心这些数学细节

每个参数的 m 和 v 常用 **32 位浮点格式**（32-bit Floating Point，FP32）保存，每份四字节；**16 位浮点格式**（16-bit Floating Point，FP16）则占两字节，数值取舍在第10.2节解释。十亿参数，仅这两份状态就要约 8 GB，采用十进制 GB。状态分片可以节省显存，而只把权重变为半精度不解决这 8 GB。

恢复训练时，只加载权重意味着 m、v 与步数可能重新开始，接下来的更新与原训练不同。学习率调度器、梯度缩放器等也有状态。一个可供部署的权重文件和一个用于继续训练的完整检查点，需求不同。

AdamW 将权重衰减与自适应梯度更新分开。梯度裁剪与学习率调度也会改变更新。优化训练系统时，先保持这些学习规则一致，才能把吞吐提升与算法变化分别评价。

10.1.5 训练显存需求要逐项算

一个带条件的 16 字节例子

设每个参数保存 FP16 权重 2 字节、FP16 梯度 2 字节、FP32 主权重 4 字节、FP32 Adam m/v 共 8 字节，总计 16 字节。十亿参数因此需要约 16 GB 的持久训练状态。这个配置便于手算，但不是 PyTorch 所有**混合精度**训练的固定公式：实际权重、梯度、主权重是否存在及其类型取决于框架实现。

16 GB 是特定配置的状态账本；激活与临时峰值另计

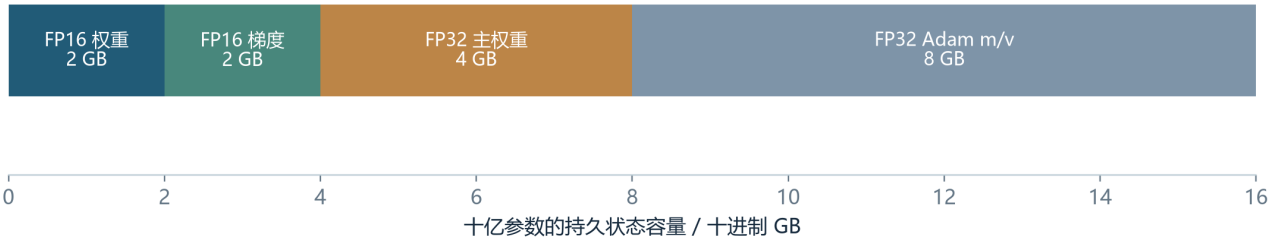


图10.2：十亿参数在特定精度与 Adam 配置下的状态容量。激活和临时内存必须另外计算。

还需增加激活、临时工作区、通信缓冲、数据张量以及分配器保留空间。激活通常随微批大小、序列长度、图像 token 数和层数增长；**注意力**中间量还受实现影响。权重规模不变，输入分辨率或视角数增加仍可能显存溢出。

OOM（Out Of Memory，内存不足）表示某次申请无法满足，未必是静态权重放不下。某个分片层临时聚集完整权重、某个长序列产生大激活或保存检查点时复制状态，都可能造成峰值。平均占用不能代替峰值分析。

活跃内存与保留内存不是同一个量

框架分配器可以保留已经申请过的块供后续重用，监控中保留内存不一定全是活跃张量。释放缓存不能解决仍然被计算图或 Python 引用持有的张量。日志误把带计算图的损失张量长期收进列表，会持续保留图，引起类似“训练越来越吃显存”的现象。

优化应先看主导项。状态主导时考虑分片或参数高效训练；激活主导时考虑微批、序列与重计算；临时峰值主导时考虑算子或聚集粒度。把所有情况都回答成“减 batch size”，容易失去最有效的方案。

容量报告应说明十进制 GB 与二进制 GiB 的区别、精度配置、模型参数数、输入形状和具体测量口径。比“这个模型需 24G”更有价值的是一张带条件的显存需求。

10.1.6 显存需求随计算阶段变化

参数、梯度、优化器状态、激活、通信缓冲和临时算子工作空间有不同生命周期。前向末尾激活可能多，反向中间可能同时出现梯度与通信缓冲，优化器步骤又有其他临时分配。只把参数字节数乘一个固定倍数，无法准确预测峰值。

完整记录包括每阶段已分配与保留显存、序列和图像长度、有效批次、数据类型、缓存和编译状态。在相同任务质量下比较训练成本，同时报告有效动作标签吞吐，避免大量补齐元素让吞吐看上去很高。

10.1.7 练习题

题10.1-1 训练显存包含参数、梯度、优化器状态和激活。冻结大部分参数后，为什么仍可能显存不足？**LoRA** 主要减少哪部分？

10.1.8 参考答案

题10.1-1答案 冻结参数主要省去这些参数的梯度和优化器状态，参数本身仍要放在显存里。激活是前向计算产生的中间张量；反向传播常需要它们来计算输入或其他可训练参数的梯度。若可训练模块位于冻结层之前，或者 LoRA 插在多个冻结层内部，梯度仍需穿过相关冻结层，很多激活就必须保留。长图像序列、长历史和较大批次因此仍可能占满显存。

例如 $y = Wx$，即使固定 W 不更新，为了训练产生 x 的上游模块，仍需计算 $\partial L/\partial x = W^\top \partial L/\partial y$。冻结 W 与不计算这条路径的梯度是两回事。只有一个完全冻结的前缀、输入也不需要梯度时，才通常可以不记录这个前缀的反向路径；固定输入还可考虑缓存其特征。

LoRA（Low-Rank Adaptation，低秩适配）把权重更新写成小矩阵乘积 $\Delta W = BA$，训练 A, B，冻结原始 W。它主要减少可训练参数对应的梯度、优化器状态和增量权重；不自动缩小原始权重，也不自动消除激活。应分别测量模型状态和激活占用，再选择量化底座、缩短序列、降低批次或激活重计算。

10.2 有效批次、混合精度与激活重计算

10.2.1 微批、全局批与梯度累积

为什么分几次算可以得到同一类平均梯度

微批是一次前向反向实际处理的样本数。**梯度累积**是在参数不更新时，连续处理多个微批，将梯度加到同一缓冲，再更新一次。**数据并行**还会让多张卡处理不同数据。等长、等权样本下

$$B_{\text{global}} = D\, b_{\text{micro}}\, A.$$

D 是数据并行副本数，A 是累积次数。四个副本，每卡微批 2，累积 4 次，全局每次更新使用 32 条样本。**张量并行**参与同一条样本的计算，不应再次计入 D。

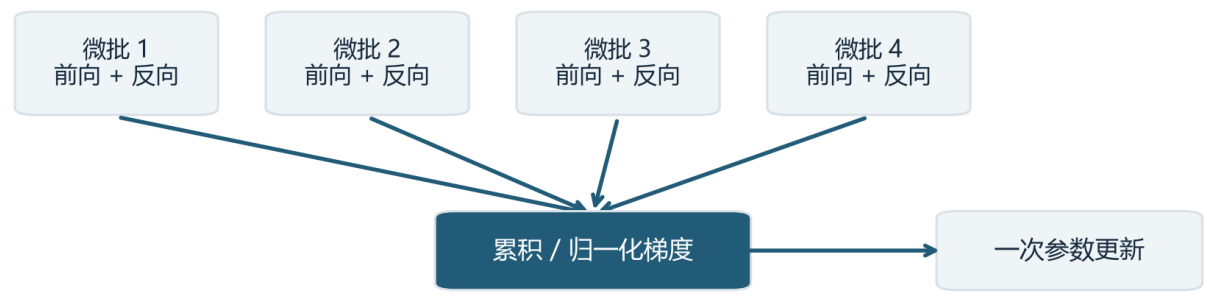


图10.3：参数固定时多个微批贡献梯度，一次更新再改变参数。全局批由数据并行数量决定。

如果每个微批损失已经对相同样本数取均值，通常应把贡献按 A 归一化，否则累积后的梯度放大 A 倍。这只是等权例子。不同微批有不同有效动作数或 token 数时，应该按目标定义加权，不能默认平均各微批的均值。

一个常见但隐蔽的加权错误

两个工作进程分别有 2 和 6 个有效时间步，它们的平均损失为 1 和 3。直接平均得到 2；按所有有效时间步平均则为 $(2 \times 1 + 6 \times 3)/8 = 2.5$。梯度同样需要按有效数量加权，才能对应全局目标。**分布式数据并行**（Distributed Data Parallel，DDP）让每个工作进程保留模型副本并同步梯度；默认的**梯度同步**不会自动按各进程的有效样本数量重新加权。

设这一更新的全局有效标签数为 C，本进程损失总和为 S_r，数据并行进程数为 D。目标是 $\sum_r S_r/C$，但 DDP 通常会把各进程梯度再除以 D。因此本进程应反传 DS_r/C，抵消这次平均：$\frac{1}{D}\sum_r \nabla(DS_r/C) = \nabla(\sum_r S_r/C)$。这里的 D 用来匹配通信规则，C 用来匹配学习目标，作用不同。梯度累积时，C 应覆盖整次更新中的全部有效标签，不能每个微批单独归一化后再等权相加。

为减少通信，可以在前几个累积微批使用 no_sync()，最后一次同步；其作用域要覆盖相应前向与反向。累积也不是在任何网络上都严格等于大批计算：批归一化、随机性、不同数值求和顺序会造成差异。改变全局批还可能影响优化稳定与泛化，应区分“同目标的实现拆分”与“训练方案改变”。

10.2.2 混合精度为什么需要选择算子和数值范围

FP16、**BF16**（bfloat16）和 **FP32** 是不同的浮点格式。FP16 与 BF16 都常占两字节，但指数范围和有效精度不同。BF16 指数范围接近 FP32，FP16 则更容易在某些中间值或梯度中上溢、下溢；格式小不意味着整个训练都能统一改成该格式。

AMP（Automatic Mixed Precision，自动**混合精度**）依据算子与类型规则，让部分计算使用低精度，同时保留需要较大范围或精度的部分。自动转换不是让所有参数、优化器状态和操作全部用半精度。PyTorch AMP 示例。

FP16 中，小梯度可能下溢。梯度缩放把损失乘 S，反向得到约 S 倍梯度；在优化器更新前再除以 S，意图保持原梯度大小并减少中途下溢。若检测到非有限梯度，动态缩放器可以跳过此次更新并调整尺度。它主要处理表示范围问题，不会修复错误标签或不合理目标。

梯度裁剪必须在恢复真实尺度后进行。否则把放大后的梯度按原门限裁剪，会改变真实更新。梯度累积时，缩放与解除缩放的时机还需与整个更新一致，不能每个微批随意改变尺度。

BF16 常不需要 FP16 那样的梯度缩放，但仍有有限精度、累加误差和敏感算子；应检查硬件支持与实际框架实现。混合精度结果需要验证 loss、梯度异常和任务质量，不能只看训练没有报错。

一个实用顺序是建立小规模 FP32 参考，确认数据和损失正常，再启用混合精度比较短程损失与更新。如果误差突然放大，先定位具体算子、范围和掩码问题，再考虑更宽类型或稳定计算，避免把全部差异归到“随机波动”。

10.2.3 激活重计算用计算换什么内存

反向传播需要前向中的部分中间值。保存每层必要激活容易消耗大量显存。激活检查点（activation checkpointing）只保存某些边界状态，反向经过这一段时重新执行前向，恢复所需中间值。它与保存到磁盘用于故障恢复的检查点不是同一个机制。

例如一个分段网络本来保存每层输出，改为每若干层保存边界，在反向时重算中间层。这减少需要长期驻留的激活，增加前向计算，并可能改变峰值发生的位置。若激活主导，收益可能大；若优化器状态主导，它不直接减少那部分状态。

反向重计算要复现原前向所需的中间量。Dropout 使用的随机状态若改变，梯度就可能对应另一份计算；被重算函数若修改外部状态，也可能重复产生副作用。框架对常见随机状态提供处理，自定义算子与外部状态仍需按其行为组织。PyTorch 激活检查点文档。

重计算不是必然拖慢整体训练。如果节省显存后可以使用更有效的微批、减少其他开销，达到同一训练目标的时间可能改善；也可能由于计算已是瓶颈而变慢。应在相同全局批和目标下测量，而不是只比较单个前向速度。

算例：原激活峰值为 16 GB，某种分段方案把它降至 8 GB，但增加 20% 的前反向时间。这个比例只是教学假设，真实节省与开销依网络、保存策略和算子而定。实践中应记录每个候选方案的峰值显存、有效样本吞吐和任务质量，选择符合资源约束的点。

10.2.4 LoRA、QLoRA 与冻结训练

训练少量参数如何改变大模型

LoRA（Low-Rank Adaptation，低秩适配）冻结基础矩阵 W，训练一个低秩增量，例如 $\Delta W = BA$，其中 A 与 B 的中间维度 r 小于原矩阵维度。前向使用 $Wx + BAx$。目的在于减少可训练参数、梯度和优化器状态，而非凭空消除整个模型的前向。LoRA 原论文。

若 W 为 4096×4096，r=8，则增量参数量为 $4096 \times 8 + 8 \times 4096 = 65,536$，而原矩阵有 16,777,216 个参数。两者相差很大，但是否足以适配具身任务，取决于插入位置、秩、数据和原模型表示。低秩是假设有效更新可用较少方向表达，不保证任何新任务都满足。

冻结 W 后不保存其常规可训练梯度和优化器状态；但要训练前面的适配器，梯度可能仍需经过一些冻结计算。冻结编码器并预缓存特征能进一步减少计算，前提是数据增强、输入条件和需要更新的部分允许这种缓存。缓存会限制某些训练变化，必须交代。

QLoRA 节省的又是哪一项

QLoRA（Quantized LoRA，**量化**低秩适配）把冻结基础模型以低比特形式存储，仍训练适配器，并在计算中按需要使用相应表示与类型。它进一步降低基础权重存储。量化相关元数据、解量化、激活和适配器状态仍占资源，不能直接用“参数数×四比特”代表整个训练显存。QLoRA 原论文。

它与为部署学习量化鲁棒性的量化感知训练不是同一个目的，也不自动得到最快端侧模型。训练后需要核对合并、保存、加载和部署数值是否一致。

具身微调时，可以先用 LoRA 建立低成本基线，再比较全参数训练或分层解冻。公平比较应明确训练参数量、数据、步数、总计算和达到的任务质量。不能只因可训练参数少就宣称效果更好，也不能只按参数量判断训练成本。

10.2.5 梯度缩放和裁剪，为什么有先后顺序

设一个参数的真实梯度为 $g = 0.25$，裁剪门限为 1；它本来不需要裁剪。若训练采用缩放系数 $S = 1024$，反向缓冲中暂时存的是 $Sg = 256$。

正确顺序是先除以 1024，恢复 0.25，再按门限裁剪。若先把 256 裁到 1，再解除缩放，就只剩 $1/1024 \approx 0.000977$。一个只是防止数值下溢的机制，被错误顺序变成了极强的梯度缩小。

因此，FP16 混合精度的一次更新可按以下过程组织：

```
清空本次更新的梯度缓冲
    对每个微批：前向得到损失，按有效监督权重归一化
                用同一个缩放系数放大损失，再反向累积
全部微批结束后：解除梯度缩放
                对真实尺度梯度做裁剪
                若梯度有效，执行优化器更新
                调整下一次更新使用的缩放系数
```

同一次累积中缩放系数需要保持一致，才能最后统一解除缩放。遇到溢出而跳过更新，也要由所有相关进程按相容规则处理。梯度累积控制样本如何合成一次更新，梯度缩放控制数值如何表示，裁剪则直接改变过大梯度；三者不能只当作互不相关的开关。PyTorch AMP 的裁剪与累积教程

10.2.6 练习题

题10.2-1 用四个小批次累积梯度，怎样才等价于对应大批次的平均损失？混合精度的梯度缩放为什么要在梯度裁剪前还原？

10.2.7 参考答案

题10.2-1答案 设第 k 个小批次有 n_k 个有效元素，损失和为 S_k。对应大批次的平均损失为 $L = \sum_k S_k / \sum_k n_k$。因此每个小批次反向传播 S_k/N，其中 $N = \sum_k n_k$；梯度相加后正好是 ∇L。若四批有效数量相等，可以把各批平均损失除以 4；若数量不同，不能简单平均四个均值。

累计开始前清空梯度，四次反向期间不清空、不更新参数，累计完再裁剪并执行一次优化器更新。严格等价还要求损失能按样本相加、参数在四次前向中相同；BatchNorm 等依赖当前批次统计的层会改变前向结果，随机层的具体随机数也可能导致数值差异。

梯度缩放先把损失乘以 s，反向得到 sg，防止低精度下很小的梯度变成零。裁剪阈值是为真实梯度 g 设计的，所以累计结束后应先还原 $g = (sg)/s$，再裁剪。若直接用阈值 1 裁剪 sg，随后再除以 s，真实梯度的有效阈值会变成 $1/s$。例如 $s = 1024$ 时，无意中把阈值缩小了 1024 倍。整个累计窗口应保持同一个缩放因子。

10.3 分布式进程、集合通信与 DDP

10.3.1 多进程、rank 与通信组

多 **GPU** 训练常采用一张卡对应一个训练进程的组织方式。rank 是进程在通信组中的编号；world size 是组内进程总数；local rank 常用于表示本机上的编号。它们是执行组织信息，不是网络层编号。

在两台机器各四张 GPU 的常见八进程任务中，全局 rank 可以为 0—7，某台机器的 local rank 为 0—3。每个进程应绑定正确设备，并使用一致的启动、网络和训练配置。两个进程意外绑定同一卡，会产生资源冲突，而不是正常分布式加速。

process group（进程组）指定谁参加某类通信。八个进程可以分成多个**张量并行**组和**数据并行**组。组内成员共同执行相应集合通信；不同组的语义不同。某个 rank 提前跳过一次组通信，其他成员可能一直等待。

GPU 通信常使用 **NCCL**（NVIDIA Collective Communications Library，英伟达集合通信库）等后端。训练框架负责组织调用，通信库根据拓扑和消息规模选择实现。能连上网络不代表带宽、延迟与路径已满足需求。PyTorch 分布式文档。

分布式错误往往需要从所有 rank 的日志找最早原因。一个进程因坏样本或 **OOM** 退出，其他进程可能只报告通信超时；超时是结果而非第一原因。日志应有 rank、步数、数据索引和错误位置，便于把共同训练状态与局部故障对上。

选择并行方案前，先确认单卡小规模训练正确。否则新增进程既增加错误入口，也增加诊断难度。训练 Infra 的关键不是会启动很多卡，而是知道这些卡在为哪一个数学更新共同工作。

10.3.2 集合通信搬的是什么，结果各放在哪里

集合通信（collective communication）是一个进程组共同参与的数据交换与计算。它的名称描述结果布局，不是不同版本的同一条复制命令。

两张卡各有一个梯度向量，g_0=[2,4]，g_1=[6,0]。All-reduce 求和后，两卡都得到 [8,4]；若再除以 2，则得到平均梯度 [4,2]。通信操作是否自动平均，要看实际操作与框架约定，NCCL 的求和本身不能默认当作平均。

All-gather 让各卡拿到所有分片的拼接，例如 rank 0 有 [2]、rank 1 有 [6]，两者都得到 [2,6]。它不把 2 和 6 求平均。Reduce-scatter 则先对对应位置归约，再把结果分片保存。上面两个向量求和并按一元素切片，rank 0 得 [8]，rank 1 得 [4]；若需要平均，还要相应除以 2。

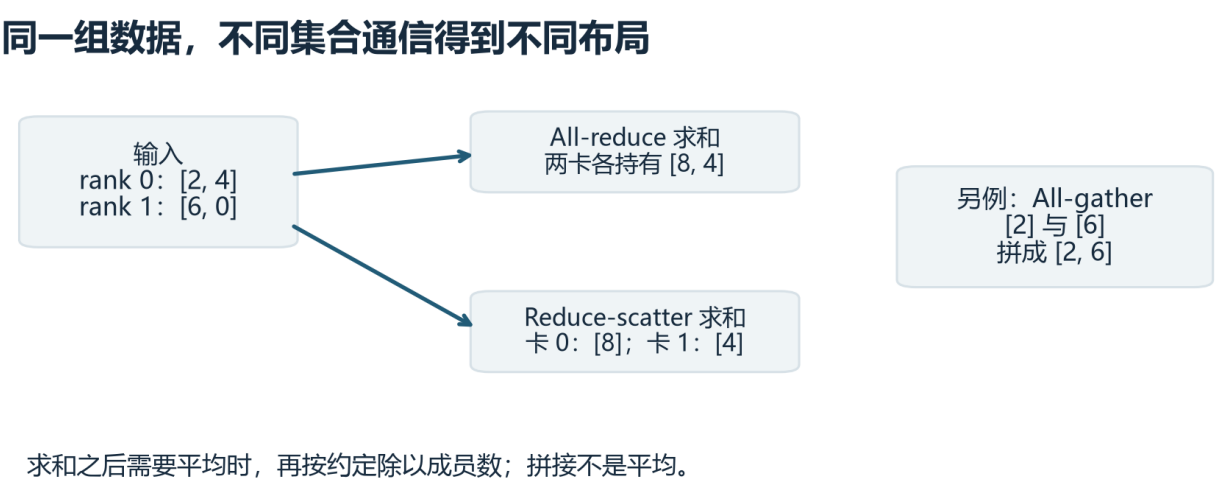


图10.4：相同输入可以产生复制结果、拼接结果或分片结果。分片训练通过改变结果布局节省每卡持有的状态。

广播把某个源进程数据发给其他成员；All-to-all 让各进程分别向其他成员发送不同片段，常用于专家路由等场景。这些操作必须在组内以匹配顺序、形状和类型执行。NCCL 集合通信说明。

为什么有了 All-reduce 还需要 Reduce-scatter？如果后面只更新自己的参数分片，各卡不需要保留整份归约结果。反过来，某一层前向需要完整参数，就可能先 All-gather。理解“谁需要什么数据、何时需要”，才能读懂 **零冗余优化器**（Zero Redundancy Optimizer，ZeRO）和**完全分片数据并行**（Fully Sharded Data Parallel，FSDP）。

小消息频繁发出会受启动延迟支配，大消息可能受带宽支配；把消息合并能减少调用次数，但可能让通信开始得更晚。算法选择和分桶都是在处理这个取舍，而非简单追求消息越大越好。

10.3.3 DDP 为什么快，却不替你把大模型分开

DDP（DistributedDataParallel，分布式数据并行）在各进程保留模型副本，分别处理不同数据，反向时同步梯度。初始参数一致、每步同步结果一致、优化器配置一致时，各进程可以独立做同样更新，保持模型副本一致。PyTorch DDP 设计说明。

同一更新中，rank 0 看到左侧房间的轨迹，rank 1 看到右侧房间的轨迹，平均梯度对应两部分数据的共同贡献。正常同步 DDP 并不是各卡独立训练几小时后再平均权重。

梯度通常按桶组织，某一桶的反向梯度准备好就开始通信，同时计算更前层梯度，形成计算与通信重叠。桶过小调用多，过大又可能等很久才启动。实际最佳组织依模型、顺序和网络而变。

DDP 每卡通常保留完整参数、梯度和优化器状态，主要分担数据计算。因此总体吞吐可以增加，但单卡模型状态的容量问题仍在。下一节的分片方法将从存储布局上解决这一问题。

输入也不会自动合理分配。需要采样器让各 rank 处理相应数据，避免重复整份训练集，并让每个 epoch 的洗牌与种子符合定义。相同随机种子在所有进程上也可能造成不期望的数据重复，需区分模型初始化一致与数据流差异。

比较单卡和多卡时，固定全局批可以检验执行效率；固定每卡批会扩大全局批，影响更新频率和学习动态。两种实验都可以做，但结论不同。学习率是否随批大小改变，也必须明示。

10.3.4 多卡平均需要知道每卡平均了多少有效元素

设两卡分别有 100 与 20 个有效动作标签。将各卡平均损失的梯度再等权平均，相当于给后者每个标签五倍权重。若目标是全局所有有效标签平均，应按有效数量加权；默认梯度归约会平均时，还需与损失缩放方式配合。分布式代码能运行，并不证明它优化的是预期风险。

DDP 同步模型副本的梯度，样本分配仍由数据管线决定。分布式采样器需要处理重复、随机种子、续训位置和验证集覆盖。可用同一小模型比较单卡与多卡的完整更新，判断采样与归约是否对应同一目标。PyTorch DDP 教程。

10.3.5 DDP 的有效监督平均

```
local_sum = 本卡有效元素损失总和
local_count = 本卡有效元素数
global_count = 对 count 做求和通信
若 global_count 为零，所有卡一致地跳过本次更新
backward_loss = world_size * local_sum / global_count
反向传播；DDP 平均后得到所定义全局均值的梯度
```

10.3.6 把全局有效动作平均接到 DDP 反向传播

前面的 DS_r/N 可以直接写进损失计算。设预测与标签形状为 (B, H, d_a)，掩码为 (B, H)；每个有效动作向量内部先求平方误差之和，再对全局有效时间步平均。这与按所有动作标量平均的目标不同，使用时应沿用同一口径。

```
import torch
import torch.distributed as dist

def backward_global_action_mean(prediction, target, valid):
    # 1. 本卡保留损失总和，让每个有效时间步先贡献一次。
    valid = valid.to(dtype=torch.bool)
    per_step = (prediction[valid] - target[valid]).square().sum(dim=-1)
    local_sum = per_step.sum()
    count = valid.detach().sum().to(dtype=torch.float32)
    # 2. 汇总全局有效数量；数量只是归一化依据，不参与求导。
    dist.all_reduce(count, op=dist.ReduceOp.SUM)
    if count.item() == 0:
        local_sum.backward()  # 同步零贡献，完成本次 DDP 的反向流程。
        return False          # 所有卡统一跳过优化器更新。
    # 3. 抵消 DDP 随后的梯度平均，得到按有效时间步平均的梯度。
    loss = dist.get_world_size() * local_sum / count
    loss.backward()
    return True
```

这段函数使用默认全局进程组，适用于该组正好就是模型DDP数据并行组、且采用默认平均梯度的情况。若DDP使用子组，数量求和与卡数都必须改用同一子组：all_reduce传入该group，get_world_size(group)取得该组成员数。张量并行和流水线并行成员不能因为也在全局任务中，就再次计入有效样本数。这里仅处理一次未做微批累积的更新。函数之前清空梯度，返回 True 后调用优化器更新。若一张卡没有有效标签，它仍参与通信并从模型输出构造零损失；单独退出会让其他卡无法完成同一更新。先选有效位置再相减，可以排除无效标签中的占位值；真正参与训练的输入、输出和标签仍应是有效数值。

加入梯度累积时，先汇总整个更新窗口的有效数 N，各微批的损失和都使用这一分母，再只在需要时同步梯度。逐微批平均后等权相加，会重新改变不同长度轨迹的权重。公式、有效数量、通信和优化器边界共同规定了正在实现的训练目标。PyTorch DDP 教程

10.3.7 练习题

题10.3-1 两卡有效损失总和与数量分别是 (6,2)、(12,6)。按卡平均的均值与按有效元素的全局均值各是多少？DDP 默认平均梯度时应怎样缩放本卡损失？

10.3.8 参考答案

题10.3-1答案 第一卡的均值为 $6/2 = 3$，第二卡为 $12/6 = 2$。按卡等权平均得到 $(3 + 2)/2 = 2.5$；按全部有效元素平均得到 $(6 + 12)/(2 + 6) = 2.25$。前者把两卡视为两个等权单位，因此第一卡的两个元素比第二卡的六个元素分别占更大权重。

设卡数为 D，本卡损失和为 S_r，全局有效数为 N。DDP 默认把各卡梯度相加后除以 D。为了得到目标梯度 $\nabla(\sum_r S_r/N)$，本卡应反向传播

$$L_r = \frac{D}{N} S_r.$$

因为同步后 $D^{-1}\sum_r \nabla L_r = N^{-1}\sum_r \nabla S_r$。本题 $D=2, N=8$，本卡反向损失分别是 1.5 和 3；这两个值的平均为 2.25。如果程序已有本卡均值 $\bar{L}_r = S_r/n_r$，则分别乘 Dn_r/N，即第一卡乘 0.5，第二卡乘 1.5。

这里需要在反向前同步有效数量 N，数量不参与求导。空卡仍应按一致的同步流程提供可反向的零贡献；全局数量为零时跳过该次更新。计算的标量均值可用于检查归一化，真正必须等价的是各参数收到的梯度。

10.4 ZeRO、FSDP 与张量并行

10.4.1 ZeRO 三个阶段分别消除哪种冗余

ZeRO（Zero Redundancy Optimizer，零冗余优化器）的思路是：**数据并行**副本共享同一个全局模型与更新，不必在每卡重复保存全部训练状态。它通过分片与通信，在需要时提供相应数据。ZeRO 原论文、DeepSpeed ZeRO 教程。

阶段 1 分片优化器状态；阶段 2 进一步分片梯度；阶段 3 进一步分片参数。它们不是模型质量从低到高的等级，而是不同状态布局。分片越深入，通常越需要在计算期间组织参数和梯度通信。

沿用第10.1节的特定十亿参数配置：权重 2 GB、梯度 2 GB、主权重和 **Adam** 状态共 12 GB，总计 16 GB。四卡时，理想持久状态需求为：

布局	每卡持久状态计算	教学容量
普通复制	2+2+12	16 GB
ZeRO-1	2+2+12/4	7 GB
ZeRO-2	2+(2+12)/4	5.5 GB
ZeRO-3	(2+2+12)/4	4 GB

这里把主权重纳入优化器相关状态。表中不包含激活、参数聚集峰值、通信缓冲、分配器和对齐开销。它说明节省来自哪一项，而不是保证 4 GB 卡能训练该模型。

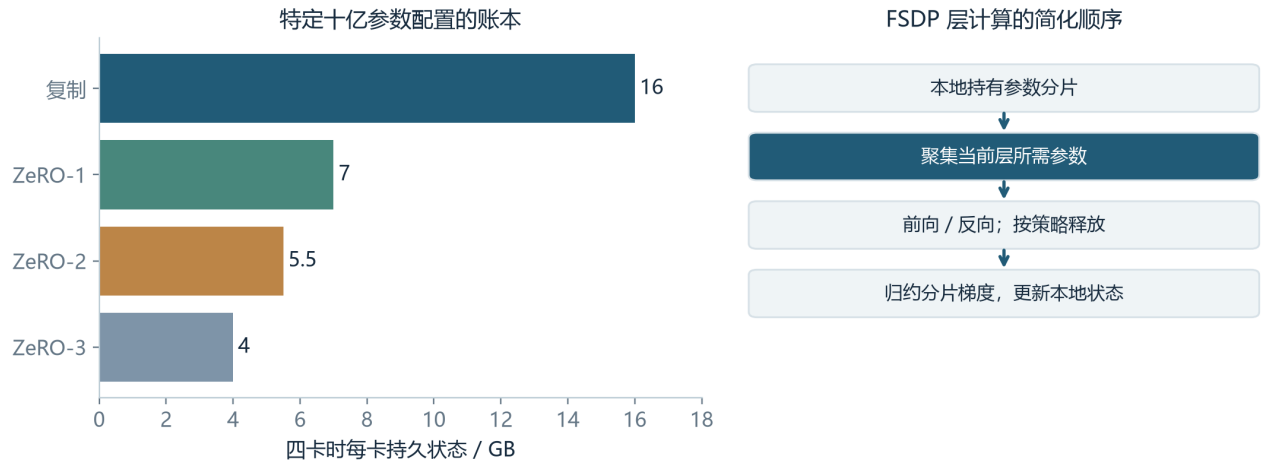


图10.5：分片减少持久副本，计算时仍可能临时取得完整层参数。持久容量与峰值容量不能混淆。

CPU 或存储卸载把部分状态移出 **GPU**，进一步缓解容量，但通过带宽和传输延迟支付代价。若频繁搬运大状态，省显存可能伴随明显降速。选择阶段应先看当前显存主导项：优化器状态主导时阶段 1 或 2 就可能足够，未必一开始就需要最深分片。

10.4.2 FSDP 如何在“用的时候”聚集参数

FSDP（Fully Sharded Data Parallel，完全分片数据并行）把训练状态分到数据并行成员。理解它可以从一层计算的时间线入手：本地持有参数片段；层执行前聚集所需参数；前向结束后按策略释放或重新分片；反向可能再次聚集；梯度归约并分片；各 rank 更新自己负责的部分。

不同实现和配置会改变具体保留、预取和释放时机。PyTorch 的 FSDP2 使用 fully_shard 等组合式接口，旧式 FSDP 接口的包装与状态保存规则不能不加区分地套用。学习时先理解布局，编写代码时再对照所用版本。PyTorch FSDP2 教程。

为什么包装粒度影响峰值

若把很大的模块作为一个聚集单位，计算前需要同时持有它的大量完整参数，峰值较高；单位很小又会增加通信调用、调度和预取管理。分片不是把任何峰值机械除以卡数，至少还要考虑正在计算的层与可能提前聚集的层。

例如某层完整权重需要 3 GB，本地持久状态只有 4 GB，前向时仍可能在此基础上增加完整层和激活临时占用。FSDP 无法保证任意一层自身大到超过单卡容量时还能正常工作，此时可能需要**张量并行**或其他拆分。

FSDP 与**激活重计算**针对不同内存项，可以组合，但重计算也可能触发额外参数聚集，增加通信。只计算重算算力而忽略重新聚集，会低估开销。通信预取让数据提前准备，也会提高同时驻留的内存。

冻结与可训练参数混合、共享权重、优化器创建时机、保存与恢复，都有实现约束。无需在入门书中背所有 **API**，但需要知道它们关系到“优化器到底更新哪一份参数”。最小验证可以检查分片前后同一小批次的损失与参数更新，容忍明确的数值误差，再扩大规模。

10.4.3 张量并行把一个矩阵乘法拆给多张卡

数据并行让各卡看不同数据，**TP**（Tensor Parallelism，张量并行）让多卡共同算同一个模型层。它适合单层计算或容量需要拆分的情况，但要求频繁交换中间结果。Megatron 原论文。

用行向量约定演示：$x = [1, 2]$，$W = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$，结果 $xW = [7, 10]$。按列拆 W，卡 0 算第一列得到 7，卡 1 算第二列得到 10；需要完整输出时拼接即可。按行拆 W，卡 0 算第一行贡献 [1,2]，卡 1 算第二行贡献 [6,8]，相加得到 [7,10]。

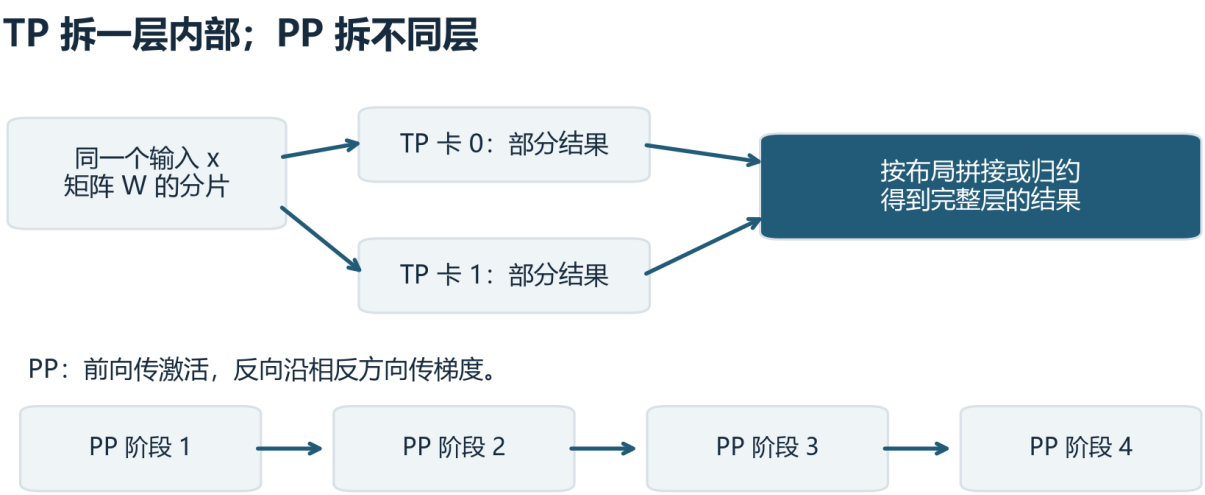


图10.6：张量并行拆一层内部运算，**流水线并行**拆不同层。两者交换的对象与频率不同。

MLP 常把第一层按输出维拆开，得到各卡自己的隐藏片段 $z_i = xW_{1,i}$；逐元素非线性可以在本地计算。第二层按输入维拆开，各卡计算贡献 $y_i = \sigma(z_i)W_{2,i}$，最后归约相加得到完整输出。这样两层之间不必先把隐藏片段全部拼接。能这样安排，是因为非线性按各维独立作用，后层输出可分解为贡献之和；若某运算需要全体特征统计，就要补相应通信。注意力也可按头等结构拆分，但必须沿真实依赖组织张量。

拆分正确性来自矩阵运算可分解，而非把网络随便切成几份。偏置、归一化、残差、共享权重和梯度都必须遵守布局。把分片张量当作完整张量做统计，可能悄悄改变模型。

TP 频繁通信，通常更适合高速互连的相近设备。如果跨慢网络，算力节省可能被通信抵消。对于本来就可放入单卡的小模型，用 TP 未必比数据并行好。选择理由要来自层容量、输入规模与实际通信测量。

10.4.4 分片怎样保持同一个模型更新

分片可以改变谁保存结果，而不改变数学更新。设模型只有两个参数 $\theta = [1, 2]$，两卡对各自数据得到梯度 $g_0 = [2, 4]$、$g_1 = [6, 0]$。全局平均梯度为 $[4, 2]$，学习率为 0.1 的 SGD 应得到新参数 $[0.6, 1.8]$。

普通 DDP 让两卡都取得 $[4, 2]$，各自更新整份参数。采用梯度与优化器分片时，可以让卡0只取得第一个平均梯度 4，更新其负责的参数到 0.6；卡1只取得第二个平均梯度 2，更新到 1.8。下一次需要完整参数时，再收集两份更新结果。两卡分担的是同一个更新的存储与执行，不是在训练两个互不相同的模型。DeepSpeed ZeRO 教程

Adam 的历史统计也可按参数分配给相应卡：负责哪个参数，就保存哪个参数的 m, v 并完成更新。参数全分片时，前向和反向还需要按层临时聚集权重；层算完后释放完整副本，梯度再归约到各卡负责的分片。PyTorch FSDP 教程

所以持久内存减少并不代表计算某层时只需本地小片。画出“分片驻留→参数聚集→层计算→梯度归约分发→本地更新”的时间线，才能同时判断内存峰值、通信开销和优化器究竟更新哪一份状态。

10.4.5 从 ZeRO 原图辨认重复存储与临时计算

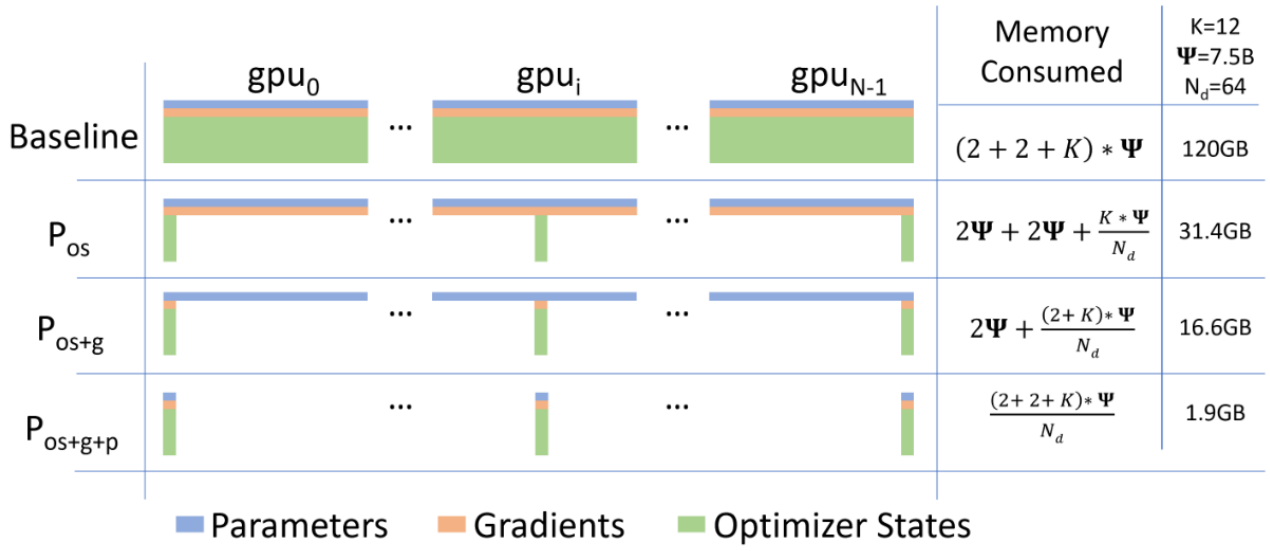


图10.7：Rajbhandari 等，ZeRO 论文 Figure 1。各行依次分片优化器状态、梯度和参数，颜色区分存储对象。原论文。

Baseline（基线）这一行中，每张 GPU 都保存完整参数、梯度和优化器状态。图中的 Parameters 是参数，Gradients 是本次导数，Optimizer States 是 Adam 等优化器跨步骤保存的历史统计。P_{os} 先分片优化器状态，P_{os+g} 再分片梯度，P_{os+g+p} 进一步分片参数；os、g、p 分别对应这三项。图的 Ψ 是参数数量，N_d 是数据并行进程数，K 是该例优化器状态的字节倍率，右列按原论文示例设置计算存储量。

最后一行的短条不是说每张卡永久只执行网络的一小部分。采用参数分片的数据并行时，当前层需要计算，就先 all-gather（全收集）本层参数；反向得到梯度后，再 reduce-scatter（归约分发）到各卡负责的分片。它主要消除长期重复存储，代价是通信和临时聚集。张量并行则把同一层计算也拆开，二者不能只根据“每卡存的参数少”混为一谈。PyTorch FSDP 教程

10.4.6 练习题

题10.4-1 数据并行、ZeRO/FSDP、张量并行怎样分配参数与计算？为何把模型切到更多卡上可能更慢？

10.4.7 参考答案

题10.4-1答案 数据并行让每卡保存完整模型，处理不同样本，最后平均梯度。若全局批次固定为 32，四卡各算 8 个样本、八卡各算 4 个样本，每卡样本工作量下降；若始终每卡算 8 个样本，加卡增加的是全局批次和吞吐，不能认为每卡计算减半。

ZeRO-1 分片优化器状态，ZeRO-2 再分片梯度，ZeRO-3 再分片参数；全分片 FSDP 也把这些模型状态分开保存。执行某层前临时汇集所需完整权重，反向后归约并分发梯度，各卡更新自己负责的参数片段。它消除持久重复存储，各卡仍对自己的样本执行完整模型；单凭状态分片不会减少这份前向计算。

张量并行才把同一层的计算拆给多卡。例如 $Y = XW$，按列拆 $W = [W_1, W_2]$，两卡分别计算 $Y_1 = XW_1, Y_2 = XW_2$，拼接得到 $Y = [Y_1, Y_2]$。按输入维拆则需把各卡的部分贡献相加。它可以降低单层的本卡容量和计算量，同时要求交换中间结果。

更多卡可能带来更多参数汇集、梯度同步或中间结果通信；固定总工作量时，本卡矩阵也可能变小，计算单元难以充分使用。通信等待超过省下的计算时间就会变慢。比较时同时记录全局和每卡批次、峰值显存、每步时间及计算—通信重叠，才能判断加卡到底获得了什么。

进一步阅读：PyTorch：Getting Started with Fully Sharded Data Parallel (FSDP2)、NVIDIA Megatron Core：Parallelism Strategies Guide。

10.5 流水线、上下文、专家并行与通信拓扑

10.5.1 流水线并行为什么有气泡

PP（Pipeline Parallelism，**流水线并行**）把连续网络层分到不同阶段。第一个阶段计算视觉与前部层，将激活传给第二阶段，最终阶段计算损失；反向时梯度沿相反方向传回。模型深度被拆开，单个样本仍须经过各阶段。

如果只送一个微批，其他阶段经常等前面或后面完成，称为气泡。连续送多个微批，可以让不同阶段同时处理不同微批。简化等时前向流水线中，p 个阶段、m 个微批，填充与排空后理想忙碌比例为 $m/(m+p-1)$。p=4、m=8 时约为 72.7%。这只是说明气泡来源的理想模型，不能直接当作包含反向、通信和不平衡的真实利用率。

训练调度还需要保存供反向使用的激活。1F1B（one forward, one backward，一次前向一次反向）调度在适当预热后交替前向和反向，改变激活驻留与气泡结构。不同调度、交错阶段和微批数都有取舍，不是微批越多越好。

阶段负载不均会让最慢阶段成为瓶颈。视觉编码器、语言主干和动作头的计算量不同，按层数平均分配不一定平衡。需要按真实输入形状和算子时间评估，图像分辨率变化可能使原来平衡的切分失衡。

同步流水线训练应保证一个更新窗口内所用权重版本符合算法定义。如果一边处理旧激活一边随意更新阶段参数，反向可能对应不同版本前向，改变训练算法。异步流水线可以有专门处理方式，但必须显式说明，而非当作普通加速技巧。

10.5.2 上下文、序列与专家并行怎样组合

CP（Context Parallelism，上下文并行）沿较长序列划分计算和激活。某张卡只处理一段 token，但**注意力**可能仍需要其他段的键和值，因此必须交换相关信息。它减少每卡处理的序列范围，不意味着把长序列切开后各段互不相干。Megatron CP 文档。

SP（Sequence Parallelism，序列并行）在特定框架中常指与 **TP** 配合，沿序列分片某些归一化、Dropout 等操作的激活；与覆盖更广序列依赖的 CP 不应混为同义词。名称可能随框架改变，应读实际张量布局。

混合专家（Mixture of Experts，MoE）由路由器为每个 token 选择少量专家子网络；**专家并行**（Expert Parallelism，EP）则把这些子网络分到不同设备。token 先按路由结果送到相应专家，计算后再返回并按权重组合，通常涉及 All-to-all 通信。这样总参数容量可以较大，而每个 token 只执行其中一部分。若大量 token 都选同一专家，该设备会拥堵，其他专家则缺少训练。负载平衡损失、容量与溢出规则试图缓解这种问题，也可能改变哪些 token 被计算。专家总数、实际激活计算与路由通信应分别计入成本。

设八张卡采用 **数据并行**（Data Parallelism，DP）的并行度为 2，同时 TP=2、PP=2，每个**数据并行**副本包含四张共同处理同一数据的卡。每卡输入微批 2、累积 4 时，全局批是 $2 \times 2 \times 4 = 16$，不是 $8 \times 2 \times 4 = 64$。

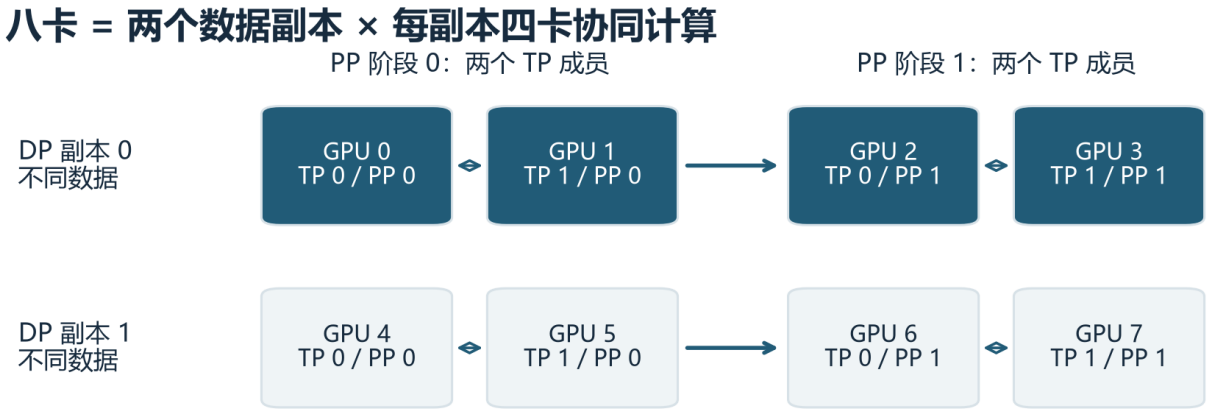


图10.8：每个副本内部共同计算同一组样本，两个数据副本才引入不同样本。通信组由张量布局决定。

并行维度组合不是随意把所有缩写的数量相乘。EP 与 DP、TP 等的映射依专家层和框架组织而定；有的维度共享或采用不同分组。学习时先画设备、模型分片和样本分配，再写组大小。对于中小型 **VLA**，**DDP**、**FSDP** 和数据流水线往往更先需要，CP、EP 则按序列长度和架构选择。Megatron 并行指南。

10.5.3 通信、拓扑与计算重叠

一张卡快不等于一组卡快

通信时间可粗略拆成启动延迟与传输时间。对 p 个成员的理想环形 All-reduce，消息大小 S、有效带宽 β、每轮启动开销 α，一个教学近似是

$$T \approx 2(p-1)\alpha + \frac{2(p-1)}{p}\frac{S}{\beta}.$$

环形 All-reduce 可以按两段理解：先用 Reduce-scatter 汇总并分散结果，再用 All-gather 让各卡获得完整结果。理想模型中，每段有 $p-1$ 轮，每轮各卡发送约 S/p 字节，所以总启动开销是 $2(p-1)\alpha$，传输量是 $2(p-1)S/p$。这就给出了上式的两项。小消息更受每轮启动延迟影响，大消息更受传输带宽影响；实际算法还会考虑拓扑、链路竞争和计算重叠。

四卡、S=100 MB、有效带宽 10 GB/秒、α=10 微秒，理想传输项约 15 毫秒，启动项约 0.06 毫秒，总计约 15.06 毫秒。若模型产生大量很小的通信，单次启动占比会显著增大。MB、GB 此处都使用十进制。

拓扑包括卡间高速互连、PCIe、跨节点网络和 **CPU** NUMA 内存关系。NUMA（Non-Uniform Memory Access，非一致内存访问）意味着某个 CPU 访问某些内存或设备更远，可能影响解码与数据传输。组映射应让通信最频繁的 TP 尽量使用合适高速路径，但需结合硬件实际测量。

重叠能隐藏一部分时间，不能消除数据依赖

当一桶梯度已完成，通信可与其他层反向重叠；参数预取可与当前层计算重叠；数据传输也可与前批计算重叠。前提是操作确实可并发、没有必要依赖，且不争抢到使两者都变慢。

下一层需要远端数据时，必须等数据到达；异步提交可以重叠独立工作，却无法跳过这条依赖。重叠还需要缓冲，可能增加峰值显存。剖析应在时间线上找等待来源，并同时观察存储代价。

扩卡实验可以测强扩展效率：固定总工作量，多卡加速多少；也可以测弱扩展：每卡工作量固定，总吞吐如何增长。二者回答不同问题，具身训练还要固定有效数据口径与输入分布。

10.5.4 并行拆分要顺着计算依赖组织

以两层 MLP 为例：$Y=\mathrm{ReLU}(XW_1)W_2$。若把 W_1 按列分成两片，隐藏特征也分成两片 Z_0, Z_1。ReLU 逐元素作用，不需要知道另一片；再把 W_2 按行对应分片，就有

$$Y=\mathrm{ReLU}(Z_0)W_{2,0}+\mathrm{ReLU}(Z_1)W_{2,1}.$$

两卡可以保留各自的隐藏片段，第二层只交换需要相加的输出，不必先把完整隐藏特征收集起来。这是张量并行选择“先按输出拆、后按输入拆”的依据。PyTorch 张量并行教程

```
import torch

X = torch.tensor([[1.0, 2.0]])
W1 = torch.tensor([[1.0, 2.0], [-1.0, 1.0]])
W2 = torch.tensor([[2.0, 0.0], [1.0, 3.0]])
# 1. 两片分别计算自己的隐藏特征；实际分布式中放在不同卡。
first_columns = torch.chunk(W1, 2, dim=1)
second_rows = torch.chunk(W2, 2, dim=0)
contributions = []
for W1_part, W2_part in zip(first_columns, second_rows):
    hidden_part = torch.relu(X @ W1_part)
    # 2. 每个隐藏片段产生一份完整输出的局部贡献。
    contributions.append(hidden_part @ W2_part)
# 3. 实际分布式中用归约求和取得最终结果。
split_output = sum(contributions)
reference = torch.relu(X @ W1) @ W2
print(split_output, reference)  # 两者均为 [[4, 12]]。
```

若两层之间是需要全部隐藏维统计的 LayerNorm，就不能照此独立处理，还要交换所需统计。反向传播同样必须按依赖汇总梯度。并行方案的正确性来自运算分解，设备拓扑再决定这样分解是否划算；把网络任意切开，既可能改变结果，也可能增加等待。

10.5.5 练习题

题10.5-1 流水线并行的气泡怎样产生？专家并行的 all-to-all 为什么不同于 DDP 的**梯度同步**？

10.5.6 参考答案

题10.5-1答案 流水线把连续模型层放在不同卡上，一个小批次依次经过各阶段。开始时后面的卡还没收到输入，结束时前面的卡已无新批次；中间也可能因某阶段更慢而等待。这些卡有资源却无法继续计算的空档称为气泡。把一个大批次拆成多个微批，可以让前一批在后段计算时，后一批进入前段。

以只计算前向、阶段耗时相等的教学例子为例，p 个阶段处理 m 个微批需要 $m+p-1$ 个时间单位，理想利用率为 $m/(m+p-1)$。四阶段只处理一批时为 $1/4$，处理八个微批时为 $8/11$。实际训练还含反向、通信和调度，不能把这条前向公式直接当成所有训练流水线的利用率。

专家并行的 all-to-all 在传递“每个 token 应交给哪个专家处理”的中间表示。不同卡发送不同内容，专家算完后再把结果送回原 token 所在位置并组合。DDP 梯度同步则把各卡对同一组参数算出的梯度做求和或平均，使模型副本一致。前者是模型计算中的数据路由，前向就需要；后者是数据并行训练中的参数更新同步。专家负载不均还会使其他卡等待最忙专家。

10.6 轨迹存储、数据加载与流水线

10.6.1 具身数据定义先于高速读取

数据的一列不是只要形状正确

每条轨迹应有明确的机器人、任务、摄像头、时间、状态、动作、结束条件与有效性。动作是发送的命令、控制器实际执行值，还是由下一状态反推的目标？末端位姿用哪个参考系，角度用弧度还是度？采样率、控制延迟、夹爪开合编码都决定标签含义。

数据定义是这些语义和格式的明确约定。它不一定需要一个复杂平台，但必须能被数据生产、训练和评估共同遵守。两个机器人都提供七维向量，可能一个是关节目标，一个是末端增量；直接混合会让模型面对相同数字的不同意义。

训练未来动作是合理的监督方式：当前图像和状态作为输入，之后 H 步示范动作作为标签。把未来相机图像混进当前输入则可能泄漏模型部署时得不到的信息。用未来信息构造标签与用未来信息帮助模型作答，要明确区分。

时间对齐还要考虑动作命令和响应延迟。某个命令发送后多久体现在状态和图像中？若标注把它们错误同时配对，模型可能学到偏移关系。先用可控动作检查时间链，再决定输入窗口与**动作块**的相对位置。

划分与归一化也属于训练定义

按帧随机切训练和测试会让同一场景、对象和连续动作同时出现在两边。要验证新环境，按环境划分；验证新对象，按对象划分；验证新机器人，按配置划分。一个测试集不能同时自动证明所有泛化。

均值、标准差、动作范围等归一化统计应由训练数据或预先声明来源获得，避免测试信息流入训练。不同机器人的动作分布是否共享统计，也应符合其语义。数据版本应记录过滤、重采样、标签修正和坐标转换，不能只保存一个路径。

先检查数据的单位、范围、时间顺序、轨迹结束与动作回放，再优化加载速度。数据正确性与覆盖决定学习目标，高速管线则决定同一目标能多快完成训练。

10.6.2 轨迹切片、采样和存储布局

为什么不能跨 episode 拼动作

episode（回合）从任务开始到结束、失败或重置形成一段有连续语义的轨迹。窗口可以取过去几帧和未来动作，但不能跨重置边界，把失败前状态与下一回合的动作接在一起。

一个回合有 10 个动作，从索引 7 取 H=4 的窗口，只存在动作 7、8、9，有效掩码为 [1,1,1,0]。第四个位置用于对齐存储，损失排除它。历史不足时也要采用固定处理，使训练输入与部署一致。

采样可以按回合、窗口、任务或机器人加权。均匀抽窗口会让长轨迹贡献更多；均匀抽回合再抽窗口改变任务权重。稀有成功、接触失败和不同机器人的比例都会影响模型学到的分布。系统采样效率优化不能偷偷改这些比例。

大量小文件为什么容易拖慢训练

存储除了总带宽，还有元数据和打开文件开销。视频解码又需要压缩数据、**CPU** 时间与合适随机访问。每次随机打开许多小文件，可能使 **GPU** 等待；按合理大小组织分片与索引，可改善顺序读取和缓存。

shard（数据分片）把许多样本或轨迹组织为一批存储单元，与参数分片不是同一个对象。数据分片大小需要兼顾随机性、缓存、恢复游标与负载平衡。过大的分片不利于细粒度重启和随机访问，过小又增加元数据负担。

多视角视频与动作数组可以采用不同存储形式，但必须有共同索引。压缩格式节省空间，却可能提高解码成本；预解码节省 CPU，却增加存储与传输。应测完整链路，不能只用压缩文件大小比较优劣。

缓存必须保留版本与语义：相机裁剪、图像缩放、语言 tokenizer、动作归一化或标签规则变了，旧缓存可能失效。缓存命中率高不能证明缓存内容正确。

10.6.3 数据加载流水线怎样让 GPU 少等

训练数据常经过读取、解码、增强、组成批、主机到设备传输几个阶段。只提高 GPU 算力，而 CPU 解码一批需要更久，训练仍会等待数据。流水线用并行工作进程与预取，使下一批准备与当前批计算同时进行。

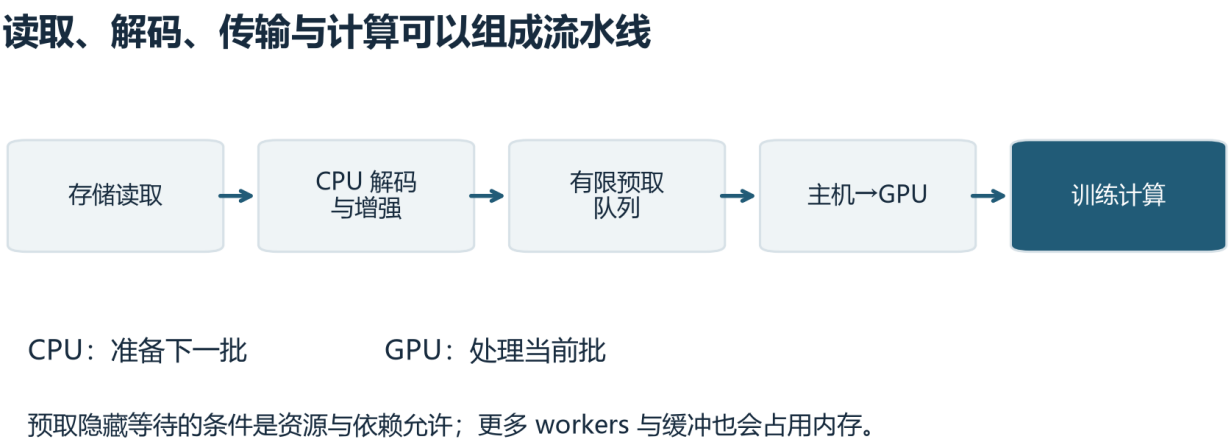


图10.9：预取在阶段间建立有限缓冲。某个阶段慢会形成队列或让下游等待，增加缓冲只能暂时吸收差异。

DataLoader 组织取样、工作进程、批构造和可选预取。pinned memory（页锁定内存）可为高效主机到 GPU 传输提供条件；non_blocking 等设置需要合适内存、流和依赖，不能仅加一个参数就保证传输与计算重叠。PyTorch 数据加载文档。

一个估算：每个 rank 有 8 个工作进程，各预取 4 个批，每批 2 条样本，可能预备约 64 条解码样本，实际还受实现、消费速度和额外缓冲影响。每条有两张图，就可能涉及 128 张图的容量。多 rank 后主机内存、CPU 与存储压力叠加，不能只看单卡设置。

工作进程不是越多越好。超过 CPU、存储和内存带宽能力后，进程调度与资源竞争可能恶化吞吐。持久工作进程减少反复启动开销，但状态和随机种子需要管理。采样器的 epoch 设置、每个 worker 的随机状态及恢复游标关系到数据是否重复或漏读。

适量预取可以吸收数据准备的波动，减少GPU等待下一批的空档；超过需求后，继续增大队列不一定提高吞吐，还会增加内存占用。队列容量与实际积压也不同：更多样本已经排队时，它们往往要等更久才能被使用。离线SFT主要关注容量与资源竞争，在线强化学习还要检查数据年龄和策略版本滞后。数据流水线应同时测量有效吞吐、等待时间、内存与信息时效。

诊断可以用固定已在内存的批次测纯计算，再测正常读取，比较时间线。若两者差距大，检查解码、文件访问、采样与传输；若差距小，再优化计算和通信。不要从 GPU 利用率低直接推断一定是 DataLoader。

10.6.4 数据加载器是训练分布的执行者

数据章节定义采样配方，加载器负责在多进程、缓存和重试条件下实现它。失败样本重试、短轨迹过滤和缓存命中可能悄悄改变曝光次数；断点续训后重新从头抽样，也会改变数据历史。记录有效样本标识与采样状态，才能核对配方是否落实。

压缩视频减少存储，却增加解码和随机访问成本；预解码加快训练，却增加容量。应使用真实图像和轨迹长度进行性能剖析，并测试数据缺失、解码失败与存储抖动，而非只用随机张量评估吞吐。

10.6.5 练习题

题10.6-1 GPU 利用率低但模型前向很快，如何判断瓶颈在数据加载、跨卡通信还是环境采样？怎样避免仅靠增加 workers 误调？

10.6.6 参考答案

题10.6-1答案 先把一个训练迭代拆成可测量的区间：等待批次、CPU 解码与预处理、传到 GPU、前向反向、跨卡通信、参数更新；在线学习再加上等待环境轨迹。看 GPU 时间线上空档出现在谁之后，比看一个平均利用率更有诊断价值。

若预先把固定批次放在 GPU 上反复训练后空档消失，瓶颈多半在存储读取、解码或传输；继续分别测它们，才能选择更快的数据格式、缓存或预取。若前向很快，但反向末端长时间等待集合通信，检查各卡是否同时进入同步、有效序列长度是否均衡、链路是否拥堵。若轨迹队列经常为空且学习器等待，记录环境每秒产生的有效步数和学习器消费速率，判断是否缺少采样能力。

增加 workers 只会并行部分 CPU 数据工作。磁盘已满负荷、视频解码受限、网络或环境采样慢时，多开进程可能加重争用、重复缓存和内存占用。应固定模型与批次，逐步改变 worker 数量，记录等待时间、吞吐和内存，选择收益停止增长附近的配置；不要把“模型前向快”误认为整套训练系统已经快。

10.7 性能剖析、检查点与运行恢复

10.7.1 性能分析、算子与编译优化

利用率只是线索

GPU 利用率说明设备在一段时间中是否忙碌，却未说明完成了多少有效模型计算。**模型浮点运算利用率**（Model FLOPs Utilization，MFU）将一个更新中的模型运算量，与相同设备、精度条件下的峰值能力比较。这里 FLOPs 表示浮点运算数量，其教学形式为

$$\mathrm{MFU} = \frac{F_{\mathrm{model}}}{P\,F_{\mathrm{peak}}\,T_{\mathrm{step}}}.$$

F_{model} 是一个全局更新的模型前向与反向运算量估计，P 是参与设备数量，F_{peak} 为所用精度及条件下每卡峰值每秒运算量。运算计数、是否包含重计算、稀疏峰值与精度都必须说明；不能混用口径得到高分。

硬件实际执行的额外重算也会消耗算力。MFU 常强调有用模型计算，HFU（Hardware FLOPs Utilization，硬件运算利用率）类口径可能包含更多实际计算，但社区用法并非完全一致，应在实验中定义。对多模态变长输入，估算尤其要交代。

看时间线后再动手

profiler（性能分析器）记录 **CPU**/GPU 算子、内存和时间依赖。需要区分等待数据、等待通信、很多小算子、单个大矩阵乘和设备同步。异步 GPU 操作的主机计时可能仅测到提交时间；测端到端应按正确同步或设备事件方式进行，避免把排队计算漏掉。

算子融合减少中间读写和多次启动，例如将若干逐元素操作合并。高效**注意力**实现通过分块与内存访问组织，减少中间量；它不一定改变数学注意力定义。自定义 **CUDA** 或 Triton 算子需要前向和反向正确性、精度与形状边界验证，训练侧只验证前向不够。

torch.compile 等编译工具尝试优化计算图。动态形状、图断裂和反复编译可能产生额外开销，需分别报告预热、编译和稳定运行时间。具身多视角、可变序列和不同机器人输入尤其需要检查形状分布，不能只测一个理想固定批。PyTorch 编译接口、编译性能分析。

在研究生的学习优先级上，先学会 profiler 与瓶颈定位，再深入算子和编译。优化某个占总时间 2% 的算子，即使提速十倍也只能小幅影响总体；应优先处理主导项，并检查是否改变模型数值和任务质量。

10.7.2 检查点、异常检测与故障恢复

恢复训练需要的不只是权重

磁盘检查点可能包括参数、优化器状态、调度器、梯度缩放器、步数、随机状态、采样器与数据游标、配置和数据版本。具体需要哪些，取决于恢复目标。仅保证继续运行与力求接近原训练轨迹，要求不同。

分布式状态本来可能分散在各卡。保存时需要组织分片与元数据，加载时可能映射到新的布局或 world size。DCP（Distributed Checkpoint，分布式检查点）提供相应保存加载支持，但并非任何框架、状态和版本组合都能无条件重新分片。PyTorch 分布式检查点教程。

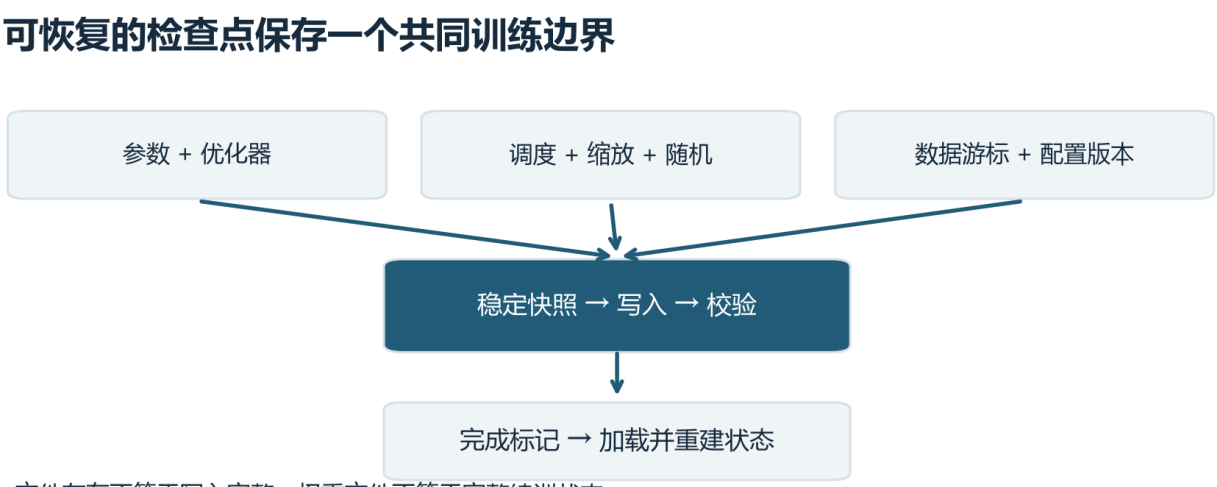


图10.10：恢复要重建模型、优化器、随机和数据位置的共同状态。保存完成的标记应在必要写入成功之后发布。

一致性要求各状态对应同一个训练边界。某些 rank 已更新、另一些未更新时保存，可能混入不同步数。保存中断产生的部分文件也不能当成完整检查点。可采用临时写入、完成清单与校验，再发布可恢复标记；异步保存还要确保写入用的是稳定快照，而不是正在变化的状态。

异常应尽量追到最早来源

非数（Not a Number，NaN）、无穷大（Infinity，Inf）、异常梯度范数和 loss 激增可以来自无效分母、坏数据、精度溢出或优化不稳定。检查梯度与数据范围有助于定位。仅跳过所有异常批可能隐藏系统性数据问题，并改变采样分布。

重启可以恢复到最近可信状态，但若坏样本必然再次出现，重启本身不解决问题。应记录样本标识、步数、配置和异常类型，按明确规则修复或过滤，并更新数据版本。

设置随机种子并不保证跨硬件、版本和分布式规模的位级复现。一些算子、并发顺序和浮点求和会有差异。应区分可重现配置、数值近似复现与统计结论复现，并做中断—恢复的小实验。PyTorch 可复现性说明。

10.7.3 环境、调度与集群运行的必要知识

训练依赖驱动、GPU 运行库、PyTorch、通信库、扩展算子和数据工具。容器把用户空间环境打包，不能凭空替换宿主 GPU 驱动。环境清单应记录版本、镜像或锁定依赖、启动方式和硬件拓扑。

作业调度分配 GPU、CPU、内存、时间和节点，可能通过 Slurm、Kubernetes 或平台界面完成。本书不要求掌握完整集群运维，但你需要理解资源请求与实际使用的关系。请求八卡却不给足解码 CPU，可能浪费昂贵设备；GPU 充足但网络拓扑不合适，模型并行仍可能慢。

抢占（preemption）是任务可能在完成前被系统中断。训练应定期保存或在可靠通知下保存，恢复后避免错误重复更新。检查点频率在保存开销与丢失工作之间折中：保存越频繁不总越好，间隔过长又增加故障代价。

多作业共享存储、网络和 CPU 时，单任务基准速度不等于真实集群表现。评估应记录资源隔离、并发情况和排队时间。研究中报告 GPU 小时有帮助，但时间、能耗或金钱成本需要各自定义，不能简单互相替代。

工程语言按职责深入。Python 组织训练和数据；C++ 常用于高效底层或框架扩展；CUDA/Triton 用于 GPU 算子；Go 等语言可能用于服务与平台。**JD** 列出多种语言不意味着所有训练研究都必须手写算子。先确定研究或岗位所需层次，再练习与实际瓶颈相对应的工具。

10.7.4 检查点保存的是继续学习所需的条件

参数、优化器统计、学习率阶段、随机状态、数据游标、归一化统计、代码配置与环境版本共同影响下一步。只恢复参数可以作为新的微调起点，却不一定是原训练的精确继续。先在小任务中做中断恢复，比较下一批样本与更新，再投入昂贵训练。

编译与性能优化应测量预热、编译时间、稳态时间和重新编译。使用真实输入形状及任务阶段，避免一组固定长度张量的加速结论直接用于动态机器人轨迹。编译教程是机制入口，真实部署仍需本地剖析。

10.7.5 练习题

题10.7-1 GPU 操作是异步的，为什么在调用前后读取 CPU 时钟会低估推理耗时？恢复训练只保存权重又会丢失什么？

10.7.6 参考答案

题10.7-1答案 CPU 提交 GPU 工作后经常立即返回，此时 GPU 还未完成；调用前后读取 CPU 时钟，可能只测到提交开销。若全部工作在同一执行流中，按“开始事件→模型运算→结束事件”排队，等结束事件完成后读两事件的时间差。计时事件需启用时间记录。

若模型使用多个执行流，让相关分支的新工作依赖共同的开始事件；各分支再记录完成事件，汇合流等待它们后才记录总结束事件。只等某个流的结束，不代表整次推理完成。测端到端延迟时可用 CPU 时钟，但停止计时前必须等本次结果真正可用，并把图像预处理、传输和后处理计入所定义的范围。先预热，排除先前遗留工作，再测多次；计时同步用于测量，不应随意塞进正常运行流程而改变吞吐。

只保存权重通常足够重建推理，继续训练还需优化器历史，例如 Adam 的梯度一阶和二阶统计、学习率进度、混合精度缩放状态、训练步数、随机数状态和数据读取位置。在线强化学习还可能需要回放池、价值与目标网络、归一化统计及采样器版本。相同权重配不同 Adam 状态，下一步更新就可能不同。

验证恢复时，保存后分成“继续运行”和“恢复后运行”两条分支，用相同批次比较损失和参数变化；先确认这些状态恢复一致，再解释非确定性运算造成的小差异。

进一步阅读：NVIDIA CUDA C++ Best Practices Guide：Timing and Coalesced Access、PyTorch：Saving and Loading Models、PyTorch 2.9：torch.cuda.Event、PyTorch 2.9：torch.cuda.Stream。

10.8 采样学习系统与多卡 VLA 训练项目

10.8.1 强化学习后训练的采样端与学习端

行动数据是由当前系统产生的

强化学习 Infra 通常分为环境执行端、学习端与评估端。actor 运行策略产生行动和轨迹，learner 用轨迹计算目标并更新参数，评估端在固定条件下检验能力。有时 actor 指整个采样进程，有时指策略网络本身，阅读代码需核对语境。

仿真可以并行多个环境，但并行效率受物理计算、视觉渲染、传感器生成、重置和策略推理共同影响。不同环境必须正确维护各自回合状态，某个环境结束不应重置所有环境。真实机器人则受操作时间、安全与人工恢复约束，不是简单增加 **GPU** 就能扩样本。

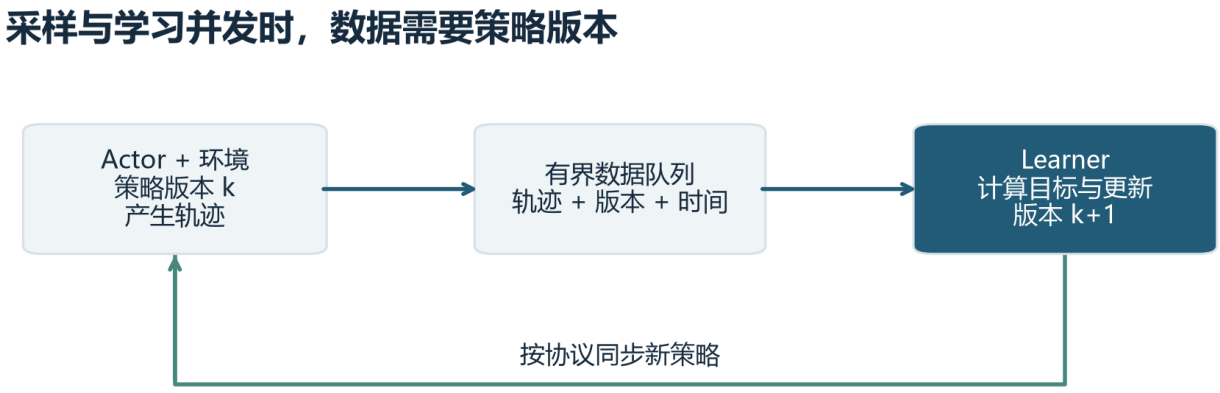


图10.11：采样与更新可并行，但数据携带产生它的策略版本。队列吞吐和学习新鲜度需要共同控制。

异步会带来策略滞后

同步采样时，各 actor 用约定策略生成一批数据，再共同进入更新。异步系统中 learner 更新时 actor 仍可能用旧策略，队列中的数据更旧。某些离策略算法允许重用旧经验；依赖接近当前策略数据的方法则需要控制滞后。

例如 **PPO** 使用新旧策略动作概率之比约束更新，轨迹必须保留产生动作的行为策略信息。若行为 log-prob 错误、版本混乱或动作变换未计入概率，算法含义会改变。对**扩散**式动作策略，真实动作密度可能并不能直接像高斯策略那样方便求得；应使用所选后训练方法实际定义的目标，不能机械套概率比。

增加队列和 actor 可能提升轨迹数/秒，也可能让旧数据占比增加，降低学习效率。应监测策略版本差、数据年龄、有效回合分布与达到目标回报的时间。Ray RLlib 扩展指南。

回合真正终止与时间上限截断也要区分。是否对末状态做价值**自举**由算法和任务定义决定，误把截断当真正终止会改变目标。归一化器、奖励版本、仿真参数与环境随机状态也可能是恢复训练所需状态。

后训练的任务成功不仅依赖 Infra。奖励设计、探索、数据覆盖与策略表示仍需研究验证。Infra 的贡献应通过正确性、吞吐、滞后、恢复与时间到质量来展示，不能仅用 GPU 利用率替代学习效果。

10.8.2 四卡VLA训练的容量与吞吐预算

先给出条件，再算容量

教学设定：十亿参数、四张 24 GB GPU、采用第10.1节的 16 GB 训练状态配置；每卡微批 1 的激活为 4 GB，微批 2 为 8 GB，微批 4 为 16 GB；额外工作区按 2 GB 估计；固定全局批 32。实际激活不保证线性，这里为方便比较而假设线性。

方案	微批与累积	每卡教学容量
DDP	微批 1、累积 8	16+4+2=22 GB
DDP 扩微批	微批 2、累积 4	16+8+2=26 GB，超过设定容量
ZeRO-2	微批 2、累积 4	5.5+8+2=15.5 GB
ZeRO-2 加激活重计算	微批 4、累积 2；激活假设降为 8 GB	5.5+8+2=15.5 GB

这不是 GPU 训练实测，也没有包括所有临时峰值。22 GB 静态估算接近 24 GB 时，实际实现仍可能失败，需要测峰值和保留余量。若使用 **FSDP**，还需计入聚集粒度与预取峰值。

然后比较时间，而非只比较能否放下

设每次更新的教学串行时间如下。为保持可手算，暂不描述真实流水线重叠：

方案	数据、计算、通信、更新耗时	总时间与有效吞吐
基线	0.6、1.2、0.4、0.2 秒	2.4 秒；32/2.4≈13.3 样本/秒
流水线与状态布局调整	0.2、1.0、0.3、0.2 秒	1.7 秒；约 18.8 样本/秒
再启用某重计算配置	0.15、1.15、0.20、0.18 秒	1.68 秒；约 19.0 样本/秒

最后两种相差很小，不能凭这张构造表决定真实最优方案。重计算增加计算时间，但允许其他变化；每一项都应实际测量。稳定输入、预热、重复测量和资源记录是比较所需条件。

从预算到实施的检查顺序

先在小数据上检查输入、动作掩码、损失与一次更新；再测单卡内存和时间线；建立固定全局批 DDP 基线；按主导内存项尝试分片；优化读取和预取；按需要加入重计算或高效算子；检查短程更新是否仍对应同一目标；完成中断恢复试验；最后比较完整学习曲线与任务效果。

保留一个配置表，至少包含数据版本、模型版本、精度、全局批、并行组、采样规则、损失归一化、优化器、预热、检查点与评价协议。只保存一条启动命令不足以解释训练方案。

10.8.3 怎样验证 Infra 改进确实有价值

时间到质量比单步速度更接近目标

time-to-quality（达到目标质量的时间）衡量多久达到事先定义的验证损失或任务成功标准。更大的批让每步更快或吞吐更高，但可能需要更多样本或更新达到同一质量。系统优化可以保持目标基本不变，也可能通过数据、批和精度改变训练动态，需要分别验证。

固定模型、数据、有效样本与全局批的实验，适合检验实现加速；允许改变训练方案的实验，可以比较最佳成本收益，但应明确额外变化。若基线使用慢读取而新方法使用新数据格式，需要消融确认提升来源，不把全部收益归于新的并行机制。

吞吐应报告样本、有效动作或有效 token 的口径。用更多 padding 形成更整齐批次，设备可能更忙，但有效学习信息没有增加。完整报告还应包含峰值显存、通信量、失败与恢复损失、硬件数量、预热及总训练时间。

用有针对性的验证代替无目的跑大实验

数值实现变更可在小模型上比较前向、梯度和一步更新；数据变更可检查采样分布与边界；分布式变更可核对梯度加权和组布局；恢复变更可比较中断前后状态及后续短程行为。通过这些检查后，再看长程质量，能更快发现可定位的问题。

组合现成分片、加载和调度方法可以形成系统贡献，但应有明确瓶颈、机制和证据。若创新点是策略滞后控制，就测队列年龄与学习质量；若是数据解码调度，就测输入分布、等待时间与整体成本。仅展示多个技术缩写，不足以说明新方法的必要性。

后续学习可分三个方向：训练系统与分布式框架、GPU 算子与编译、数据与集群可靠性。共同基础是正确训练、显存需求、时间线和公平比较。完成一个可重复、可恢复、能解释瓶颈的小训练，比一开始部署庞大但无法诊断的平台更能建立理解。

10.8.4 演员学习器系统还需要数据年龄定义

采样进程使用哪个策略版本，轨迹距离当前权重多远，奖励和归一化是否改变，都会影响学习。异步采样提高资源利用率，也增加策略滞后。记录版本、采样与接收时间、旧概率和实际执行动作；设置可解释的滞后处理，并在同资源下评价。

系统研究可提出减少等待、提高有效吞吐或改善恢复的目标，但应保持算法、数据和评价口径。训练快一倍却改变了采样分布，可能是另一项研究，应单独说明。

10.8.5 练习题

题10.8-1 多个机器人或仿真器异步产生轨迹，学习器持续更新策略。为什么每条轨迹需要策略版本？同策略和异策略算法怎样不同地使用旧轨迹？

10.8.6 参考答案

题10.8-1答案 每条轨迹的动作由某个旧版本策略产生，但学习器可能已更新到新版本。版本号让我们知道数据的行为策略是谁，估计策略落后程度，并决定能否继续使用。还应保存行为策略对实际采样动作的对数概率；仅有版本号而没有旧模型或旧概率，通常不足以计算概率比。

同策略方法希望轨迹来自当前采样策略。PPO 会先固定一个采样版本，保存旧概率和价值估计，在这批轨迹上做有限次更新，然后重新采样。若任意拼接很老的轨迹，状态分布、动作分布与优势估计都会更远离当前策略；PPO 的裁剪并不自动修复全部这种偏差。因此通常控制版本差、数据年龄和更新轮次。

异策略方法允许行为策略与学习策略不同。SAC 等把旧转移 (o, a, r, o') 放入回放池，用当前评价器与目标计算更新；它不要求每条轨迹都来自最新策略。某些异步策略梯度系统则用 $\pi_{\text{new}}(a \mid o)/\pi_{\text{behavior}}(a \mid o)$ 修正采样差异，例如 IMPALA 的 V-trace。旧数据仍需有相关状态动作覆盖，权重也会带来方差；“异策略”并不意味着越陈旧越好。

第11章：工程专题：端侧部署与推理加速

端侧部署需要把计算、时效和动作质量一起考虑。本章从图转换与量化讲到 GPU 算子、缓存和异步执行，再把局部加速放回完整机器人链路。

11.1 部署目标、性能需求与硬件瓶颈

11.1.1 部署优化必须以实际机器人循环为目标

相机、预处理、条件编码、动作生成、传输和执行都贡献延迟。单个算子更快，可能只缩短很小一段；平均推理快，尾部延迟仍可能使接触控制失去时效。本章将数值近似、程序优化和时钟约束连接起来，而不是按工具名称罗列。

11.1.2 从机器人任务认识推理

一次推理究竟做什么

推理是使用已经训练好的模型，根据输入计算输出。假设机器人要把桌上的红色杯子放进托盘：输入包括两路相机图像、语言任务和关节状态，输出包括未来一小段末端动作及夹爪命令。负责选择动作的模型是**策略模型**；Vision-Language-Action，简称 **VLA**，是一种常用于实现这种策略的模型类别。VLA 的名称不能代替具体输入、输出和动作定义。

本书采用一个教学系统：两路 224×224 图像，一个任务文本，一组自身状态；视觉编码器把图像变成特征，主干网络融合信息，动作头输出长度 H=16、每步 D=7 的**动作块**。动作块可看成 16×7 的表格，行是未来控制步，列是六个末端位姿增量和一个夹爪量。旋转、单位与归一化方式必须另行约定；不是所有 VLA 都输出这种格式。

训练时，模型通过数据和损失改变参数；部署时，通常保持参数固定，反复输入最新观测并计算动作。**量化**感知训练与**蒸馏**会重新训练模型；更换推理引擎通常不以更新模型知识为目标；**CUDA** 算子优化修改计算实现。三者都可能改变最终数值，却是在不同位置做工作。

模型的快与机器人的快

设教学机器人每 100 ms 请求一次新动作块，即策略更新频率为 10 Hz；底层控制器每 10 ms 跟踪一个目标，即 100 Hz。模型不需要在每个底层控制步完整运行，但生成的新动作必须及时进入执行队列。这里的两个频率只是案例设定，真实要求由任务、动作接口和机器人本体决定。

一条串行链路可能包括图像采集与预处理、设备间传输、视觉编码、主干计算、动作生成、后处理和发送。教学耗时依次为 14、8、22、46、24、6 ms，总计 120 ms。只测主干的 46 ms，会误以为已经满足 100 ms 的预算。机器人真正等待的是可以执行的新结果。

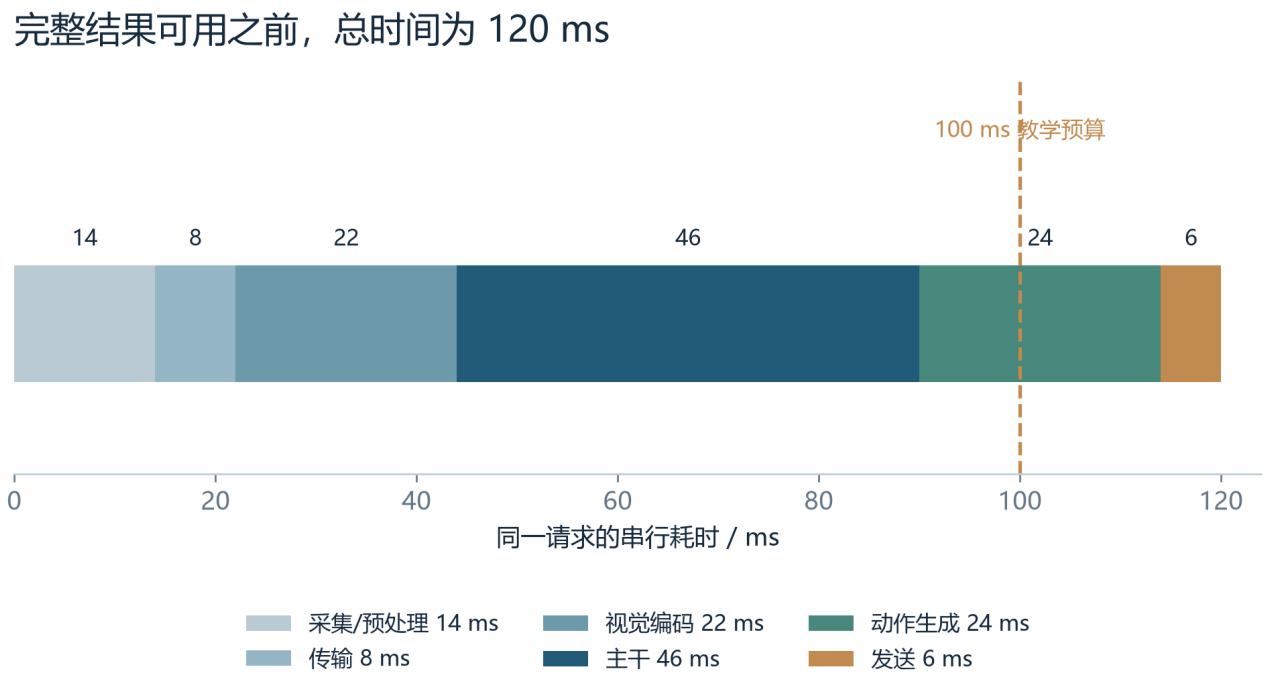


图11.1：同一次请求的串行教学预算，总和为 120 ms。后续流水线可能存在重叠，此时应测关键路径而不是机械相加。

闭环意味着动作会改变环境，下一次观测又会影响动作。输入是 120 ms 前采集的图像，即使动作在该图像上正确，也可能不再适合已经移动的杯子。部署优化最终需要同时回答：模型输出是否正确、输出是否及时、机器人是否仍能成功。

先区分三个优化目标

模型轻量化减少参数、存储或计算需求；执行加速提高同一计算在设备上的效率；系统优化减少等待、重复工作和数据搬运。它们可以组合。例如先用四位权重降低读取量，再用支持该格式的矩阵乘法实现，最后避免每次请求重新分配内存。只完成第一步，可能得到一个更小但并不更快的模型。

11.1.3 张量、算子、计算图与运行时分别是什么

从一个线性层开始

张量是按维度组织的数值数组。图像可表示为批次×通道×高度×宽度；一串特征可表示为批次×Token 数×特征宽度。Token 在这里是模型处理的一项序列元素，可能来自文本，也可能来自图像块，不一定是一个字。

线性层可以写成

$$Y = XW + b.$$

若 X 有 M 行、K 列，W 有 K 行、N 列，Y 就有 M 行、N 列。X 是当前输入特征，W 和 b 是训练得到的权重与偏置。部署时主要改变 X；压缩方法可能改变 W 的表示或尺寸。这个公式不是某个独立网络，只是网络中的一项计算。

算子是有明确输入输出的计算，例如矩阵乘法、加法、归一化或图像缩放。**计算图**记录算子之间的数据依赖：先算哪一个，哪个结果供后续使用。训练框架可能先执行矩阵乘法、再加偏置、再做激活；部署系统可能把三项工作融合为一个实现，避免中间结果多次写入内存。

参数量、计算量和存储不是同一件事

一个权重矩阵只存储一次，却可能被很多 Token 使用。因此同一个模型输入的 Token 越多，计算量可能越大，而参数量保持不变。反过来，给零权重保留原来的密集矩阵形状，参数“有效数量”减少，但程序仍可能执行原来的全部乘法。

矩阵乘法约需要 2MKN 次浮点运算，这里把一次乘法和一次加法各计一次；也有资料把乘加作为一项，因此比较计算量时应说明口径。**FLOPs** 指运算次数，FLOP/s 指每秒运算能力，两者不可混用。

运行时怎样把图变成结果

运行时是实际组织执行的程序：分配或复用内存、调用算子实现、管理设备和队列、返回结果。推理引擎通常还会预先优化计算图并选择执行方案。**GPU** 上实际执行的函数称为 kernel，中文常译为核函数。一个算子可以对应多个 kernel；一个融合 kernel 也可以执行多个算子。

理解层次后，报错就更容易定位：导出失败可能是计算图无法表达某种控制流；引擎构建失败可能是没有兼容算子；输出错误可能是动作归一化或精度问题；运行缓慢可能是某个 kernel 的访存方式，也可能是 **CPU** 迟迟没有提交任务。不能把它们统称为“CUDA 有问题”。开放神经网络交换格式（Open Neural Network Exchange，ONNX）概念文档

11.1.4 为什么 GPU 算力很高，模型仍然很慢

计算需要先拿到数据

CPU 擅长复杂控制和多种工作；GPU 用大量并行执行资源处理适合并行的数据计算；**神经网络处理器**（Neural Processing Unit，NPU）针对特定神经网络计算提供专门执行能力。实际性能还取决于可支持的算子、数值格式和软件实现，不能只比较宣传的峰值算力。

GPU 数据会经过多个位置：容量较大的设备主存，芯片上的缓存，**共享内存**和寄存器。寄存器主要服务单个线程，共享内存供同一线程块协作，二者容量都有限。计算资源可能在等待数据，也可能因为线程或寄存器限制而没有充分使用。

一个粗略下界帮助判断瓶颈

$$T \gtrsim \max\left(\frac{F}{P}, \frac{B}{\beta}\right).$$

F 是运算次数，P 是与该数值格式和算子相匹配的计算速率；B 是相关存储层之间实际需要传输的字节数，β 是该层的传输速率。两个比值分别回答“即使算得很顺，需要多久”和“即使搬得很顺，需要多久”。取最大值，是假设搬运和计算能充分重叠时仍绕不过较慢的一项。启动、同步、资源争用和效率损失会使实际时间更长，它不是精确预测式。

计算强度为什么有用

定义计算强度 I=F/B，单位是每字节执行多少次运算。它回答同一份被搬来的数据被利用得有多充分。Roofline，常译为屋顶线模型，用下面的上界描述可达到的计算速率

$$P_{\text{achievable}} \lesssim \min(P, I\beta).$$

这条线有两段：计算强度小时，数据搬运限制性能；计算强度大时，计算资源成为限制。模型并未说明全部硬件细节，但能帮助形成可验证的优化假设：若是带宽瓶颈，减少字节或增加数据复用更有希望；若是计算瓶颈，应考虑更高效的矩阵执行和减少运算。

以 **FP16** 矩阵乘法为教学例子。M=K=N=1024 时，约有 21.47 亿次运算；若两个输入各读一次、结果写一次，最低数据量约 6.29 MB，计算强度约 341 次/字节。换成 M=1、K=N=4096，计算量约 3355 万次，而权重仍约 33.55 MB，强度接近 1 次/字节。后者只处理一行输入，每个权重难以在多行之间复用，可能更受读取权重限制。这里采用理想最低搬运量；真实重复读取可能更多。

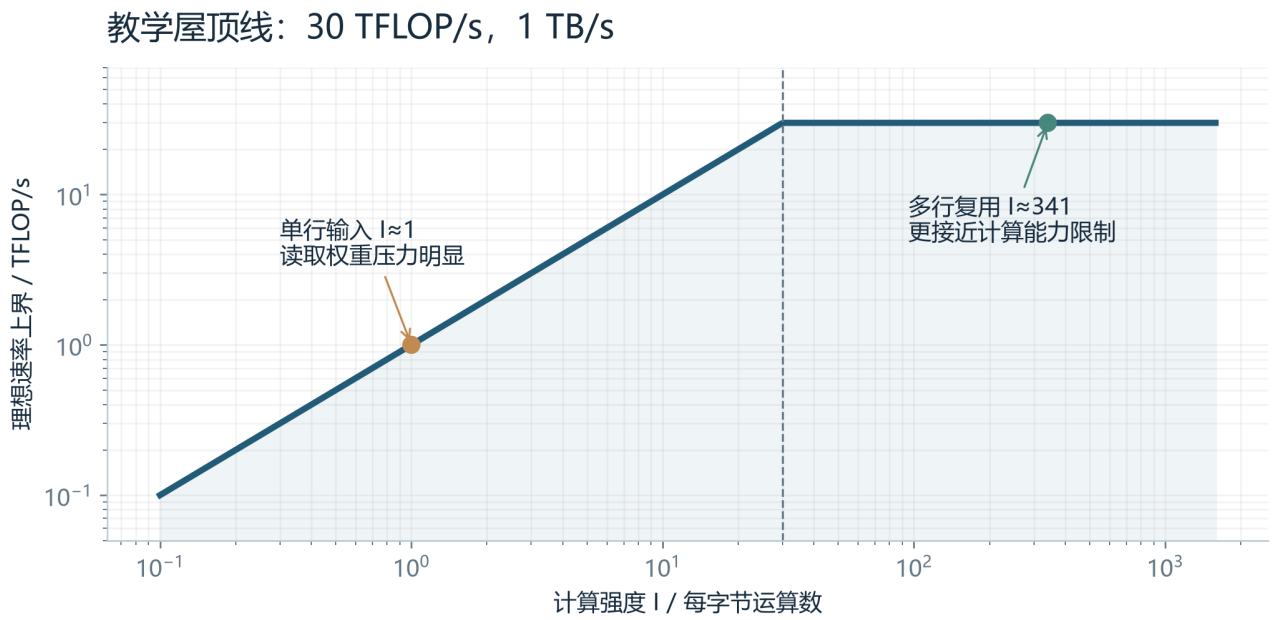


图11.2：教学硬件设定为对应精度下 30 TFLOP/s、1 TB/s。图示用于理解瓶颈，不对应任何具体设备型号。

对机器人部署的启发

一个机器人常以 batch=1 推理，batch 是一次共同执行的请求数。云端可以等待更多请求组成批次，以增加权重复用；机器人等待组成批次会增加反应时间。吞吐优化和低延迟优化需要分别考虑。

Tensor Core 是 NVIDIA GPU 内适合特定矩阵运算和数值格式的专用执行资源。使用较低精度并不保证进入这条高效路径，矩阵形状、对齐和实现都影响结果。应把“设备支持什么”与“程序实际使用什么”分别检查。CUDA 最佳实践

11.1.5 先测什么，怎样才算测对

建立与任务一致的基线

基线是后续优化的比较起点。至少记录模型版本、输入尺寸、动作块长度、生成步数、批次、精度、设备、软件版本及功耗配置。否则一次变快可能来自输入更短，另一次来自更高功率，无法说明优化机制。

延迟是一项请求等待结果的时间；吞吐量是单位时间完成多少请求；峰值内存是运行过程曾达到的最高内存用量。P95 延迟表示约 95% 的请求在该时间内完成，它不等于最大延迟，也不等于 95% 的机器人任务成功。平均快而偶尔长时间停顿，仍可能破坏连续操作。

GPU 异步执行造成的计时陷阱

CPU 调用 GPU 函数后，通常只是把工作放入执行队列，不等待工作完成。若只测函数返回时间，测到的可能主要是提交时间。用 CPU 的墙钟计时测单次端到端 GPU 工作，需要在合适位置等待前后任务完成；用 CUDA event 计时，可测执行队列中两事件之间的设备时间。后者不自动包含相机、CPU 预处理和网络通信。PyTorch CUDA 执行说明

预热，是先运行若干次，让初始化、内存分配、编译和缓存等一次性工作完成。稳态延迟应在预热后报告，首次运行延迟应另外报告。不要用“预热”隐藏产品启动成本，也不要每轮清空缓存后把冷启动当正常推理。

Profiling，性能剖析，是把总时间拆成可定位的执行事件。先看端到端时间轴，确认 GPU 是持续计算、在等数据还是在等 CPU；再看耗时较大的算子和 kernel；最后针对它检查带宽、计算资源和同步。PyTorch Profiler 适合连接模型算子与执行事件，Nsight Systems 侧重系统时间轴，Nsight Compute 侧重具体 kernel 的执行指标。PyTorch Profiler 文档

为什么要关心整个占比

若某部分占总时间的比例为 f，把该部分加速 k 倍，理想串行总加速比为

$$S = \frac{1}{(1-f) + f/k}.$$

分母表示新时间占原时间的多少：不改的部分仍是 1-f，改过的部分只剩 f/k。假设动作头占 20%，加速四倍，总加速只有 1/(0.8+0.05)≈1.176 倍，即总时间减少约 15%。这说明局部指标需要回到端到端占比，并不说明局部优化没有价值。

一套可重复的测量流程

固定代表性回放输入，分别记录模块时间和端到端时间；预热后多次运行，保存原始样本而非只保存均值；固定功耗与散热条件，并测试持续负载；改变一个因素后重复测量；最后在闭环任务中验证。模块均值相加可用于串行预算，模块 P95 不能直接相加当作端到端 P95。异步链路也不能把可重叠事件的持续时间全部相加。

真实设备上至少区分 GPU 内存分配量、运行时保留量和整机内存。Jetson 等集成平台存在共享物理内存，简单相加不同计数器可能重复统计。TensorRT 性能测量

11.1.6 算术强度判断需要哪一种资源

矩阵运算多且数据复用高，可能受计算能力限制；小批线性层不断读大权重，可能受带宽限制；许多短算子可能受启动和调度限制。Roofline 用运算量与搬运量的比值连接计算上限和带宽上限。理论上限不是实测性能，缓存、访问方式与并发还会改变结果。

以实际输入长度、精度和批次计账，再通过剖析验证假设。网络 FLOPs 降低不保证延迟降低；删除少量 token 若不改变内核形状，收益也可能有限。

11.1.7 练习题

题11.1-1 假设端到端流程中的各部分依次执行，模型计算占总延迟的 40%，其余耗时不变。模型计算加速两倍后，整体加速比是多少？若模型计算可无限加速，整体上限又是多少？为什么实际使用这两个估计前，需要检查性能剖析的时间线?

11.1.8 参考答案

题11.1-1答案 假设这些时间在端到端关键路径上依次发生，其他部分耗时不变。把原总时间归一化为 1，模型计算占 0.4，其余占 0.6。模型计算加速两倍后

$$T_{\text{new}} = 0.6 + 0.4/2 = 0.8, \qquad S = 1/0.8 = 1.25.$$

所以整体加速比是 **1.25 倍**，延迟降低 20%。若继续把模型计算无限加速，整体上限才是 $1/0.6 \approx 1.67$ 倍。这就是阿姆达尔定律的含义：局部优化的收益受未优化部分限制。

要用性能剖析确定“40%”到底测了什么。若它只是各个 GPU 算子时间相加的比例，而计算与图像传输存在重叠，这个比例不一定等于端到端关键路径占比；加速被其他工作遮住的运算，整体延迟可能几乎不变。加速后也可能从计算瓶颈转为数据或调度瓶颈。因此公式用于估计优化空间，时间线和端到端实测负责验证其前提。

11.2 计算图、算子语义与部署运行时

11.2.1 训练好的模型怎样进入部署环境

导出的对象不是模型文件名

模型部署需要带走四样东西：网络的计算结构、训练参数、输入输出协议以及预处理后处理规则。协议包括图像排列、颜色顺序、归一化、文本处理、状态字段、动作单位和坐标系。权重相同而颜色顺序错了，输出也会错；动作没有恢复原单位，即使引擎数值完全一致，机器人仍会执行错误命令。

常见路线是 PyTorch 模型导出为 **开放神经网络交换格式**（Open Neural Network Exchange，ONNX），再由 ONNX Runtime 执行，或由 TensorRT 构建引擎。ONNX 是计算图交换格式，ONNX Runtime 是运行它的系统，TensorRT 是 NVIDIA 平台的优化与执行工具。还有保留 PyTorch 并使用编译优化的路线，也有针对特定设备的其他工具链；ONNX 不是所有部署的强制中间步骤。PyTorch ONNX 导出文档

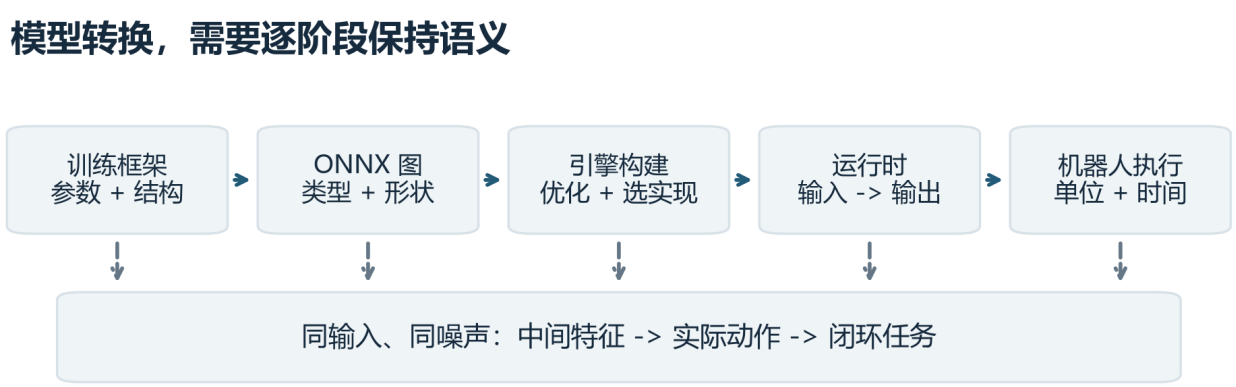


图11.3：每次改变表示或执行方式后，都先使用相同输入比较输出，再检查完整动作链路。

为什么动态形状会增加困难

形状是张量各维度的大小。语言长度变化、视觉 Token 裁剪数量变化、生成动作长度变化，都会改变形状。引擎可以针对一个固定形状选择较高效的实现；支持一大范围形状则需要更灵活的执行方案与内存安排。

TensorRT 的 optimization profile 是构建时声明的一组允许形状及优化目标，通常包括最小、常用和最大形状。它不是“性能分析报告”。例如文本长度允许 8-128，常用 32：构建器知道应该优先为常用形状选择执行方案；运行时仍要提供当前实际形状。范围声明并不保证所有形状同样快。动态形状概念

不兼容算子如何处理

opset 是 ONNX 一组算子定义的版本号。导出器和运行时都认识某个算子名，不等于都支持该版本的全部语义。先确认失败位置和版本，再考虑等价改写、选择支持的导出路径、拆分子模块或补充自定义实现。把某一层送回 **CPU** 是可选方案，但会增加设备切换和传输，应该单独测量。TensorRT ONNX 导入说明

对于迭代动作生成，网络每一步和外层生成循环可以分开部署：引擎负责一次网络前向，外层程序组织多次调用并更新生成变量。这更容易确定时间与形状边界，但调用开销可能累积。是否合并循环要由测量和工具支持决定。

怎样验证转换后的模型

先用固定输入与固定随机噪声比较原模型和部署模型的中间输出；找出误差从哪一层开始增大；再比较恢复物理单位后的动作、夹爪切换及完整**动作块**。随机生成模型若使用不同噪声，输出差异不能直接归因于导出。最后做闭环验证，因为输入回放一致仅证明在这组已有输入上表现接近，不能证明两者执行后会访问相同状态。

版本记录应同时包括模型参数、图文件、引擎、预处理、动作协议及依赖版本。只记录一个权重文件，无法重现完整部署结果。

11.2.2 推理引擎怎样选实现、管理内存并执行

构建阶段和运行阶段

把计算图交给引擎，不意味着立刻开始机器人推理。**构建阶段**先检查图、传播形状、优化依赖、选择算子实现并安排内存；**运行阶段**使用已经构建好的方案处理当前输入。构建可能花费较长时间，通常应在部署准备阶段完成；频繁在控制循环里构建，会造成长停顿。

TensorRT 中，engine 是构建后的执行方案与参数；execution context 是一次执行所需的上下文，包括动态形状和相关执行状态。tactic 是某个算子的一种候选执行方案，例如不同分块大小或不同库实现。构建器可以试运行候选并按条件选择。它优化的是具体设备和形状下的执行方式，不是在重新学习机器人动作。TensorRT 工作机制

图优化为什么能减少时间

设 Y=XW+b，随后 Z=max(Y,0)。分开执行可能先把矩阵乘法结果写回主存，再读取它加偏置，再写回和读取以做激活。融合实现可以在数据仍位于片上时完成偏置与激活，最后只写 Z。减少的是中间搬运和 kernel 启动，而不是把数学上的三个步骤假装不存在。

融合不是越多越好。融合后的程序若需要太多寄存器，可能限制并行；分支和形状也可能让它难以选择高效实现。图上的“可融合”与某设备上的“融合后更快”要分别验证。TensorRT 融合与调优

内存复用为什么需要生命周期

某个中间张量的最后一次使用结束后，其内存才能供其他张量复用。生命周期是从产生到最后一次被使用的区间。运行时利用不重叠的生命周期减少同时占用。**GPU** 工作异步提交时，CPU 函数返回不表示张量已经用完，因此复用还要满足执行顺序。

内存池预先准备或保留可重复使用的内存，避免每次请求向系统申请和释放。它可能降低波动，但保留量会大于当前活跃张量量。workspace 是某种执行方案临时需要的工作空间；限制它可能减少内存，也可能排除较快方案。权重占用、活跃张量、缓存和工作空间应分开记录。

多后端运行时与回退

ONNX Runtime 通过 execution provider，执行提供器，接入不同硬件后端。一张图可能分成多个子图分别执行；不支持的算子可能回退到其他后端。回退保持可运行，却可能增加同步和搬运。因此“使用了 GPU 后端”不等于全部计算都在 GPU 上。ONNX Runtime 执行提供器

一种实际部署策略是先保留标准模型作为正确性参照，逐步替换耗时模块。自定义插件补充缺失算子时，必须同时定义形状、数值类型、序列化信息和执行函数。插件不是仅把一个 **CUDA** 文件复制进模型；第11.7节会把这条路径接完整。

11.2.3 数值精度怎样影响资源和动作

浮点数与整数的职责

FP32、**FP16**、**BF16** 是浮点格式，数字说明总位数；**8 位整数**（8-bit Integer，INT8）和 **4 位整数**（4-bit Integer，INT4）是整数编码格式。在量化计算中，缩放尺度和累加器类型还需另外规定。浮点数用符号、指数和有效数字描述数值，较大范围与细致刻度之间存在取舍。BF16 和 FP16 都占 16 位，但指数与有效数字分配不同，因此适合处理的数值范围和精细程度不同。低位浮点也存在，不能把所有低精度统称为整数**量化**。

权重用某种格式存储，不表示乘法、求和及输出都用同一格式。一个实现可能读取 FP16 输入、用较高精度累计再写 FP16 输出。另一个实现可能读取打包的四位权重，在 kernel 内还原成浮点参与计算。学习精度时需要问：存储格式是什么、计算格式是什么、累加格式是什么？TensorRT 精度与误差

用参数量估算存储

若参数数量为 N_p，每个参数占 b 位，理想权重存储为

$$M_{\text{weights}} = \frac{N_p b}{8} \quad \text{bytes}.$$

一个教学 70 亿参数模型用 16 位保存，约需 14 GB；用四位保存，理想值约 3.5 GB。这使用十进制 GB，不是二进制 GiB。若每 128 个参数另存一个两字节缩放值，还要约 109.4 MB；**零点**、对齐和其他未量化参数可能继续增加占用。

真正运行还需要中间结果、动作生成状态、**注意力**缓存、工作空间和程序本身。训练内存则另外有梯度、优化器状态等。不能拿 3.5 GB 的理想权重值直接断言模型能在任意 4 GB 设备上运行。

哪些误差值得关注

溢出是数值超过格式可表示范围；舍入误差是数值落在两个可表示值之间而被近似；长求和会累计误差；非常接近的数相减可能丢失有效信息。归一化、概率计算和接近动作阈值的输出，可能对这些变化敏感。

如果动作输出是归一化值，0.01 的误差在恢复单位后可能对应 1 mm，也可能对应 1 cm，取决于缩放约定。夹爪开合若使用阈值，0.49 变成 0.51 就可能改变离散指令。评价时要看物理单位和行为后果，不能只看网络输出的平均小数误差。

混合精度是在不同模块或步骤使用不同数值格式。它不是随意保留几层，而是在硬件支持、误差敏感性和时间占比之间做选择。第11.4节给出选择与验证流程。

11.2.4 量化公式背后的建模选择

为什么整数还需要缩放值

整数编码可以占用更少字节，但模型原来使用的是实数。需要规定“一个整数刻度对应多少真实数值”。常见仿射量化使用正的 scale，缩放值 s，和 zero point，零点 z

$$q = \text{clip}\left(\text{round}(x/s) + z, q_{\min}, q_{\max}\right), \qquad \hat{x} = s(q - z).$$

x 是原数，q 是存储的整数，帽子表示还原后的近似数。先除以 s，把真实数换成刻度单位；舍入选最近的整数；加 z 指定真实零的位置；裁剪保证落入有限编码范围。还原时执行相反的缩放，但已经丢失的细节无法恢复。舍入平局的规则依格式和实现而定。

对称量化通常令零点为 0，并围绕零选择范围；**非对称量化**允许非零零点，以适应不对称的数据范围。例如只出现正值的中间结果，如果仍给负数分配很多刻度，就可能浪费精度。是否支持某种零点，还受后端算子限制。ONNX Runtime 量化说明

一个能手算的例子

为便于教学，使用整数范围 [-7,7]、s=0.5、z=0。这是使用 15 个对称等级的教学设定，实际有符号四位整数常用范围是 [-8,7]。

原数 x	整数 q	还原值	还原值减原数
-1.2	-2	-1.0	0.2
-0.2	0	0	0.2
0.7	1	0.5	-0.2
1.8	4	2.0	0.2
4.0	7，发生裁剪	3.5	-0.5

把 s 改为 0.1，刻度更细，但最大可还原值只有 0.7，1.8 会被裁剪到 0.7。这个例子揭示了核心取舍：刻度越细，同样数量的整数能覆盖的范围越小。量化不是只选位数，还在选择有限刻度如何分配给数据。

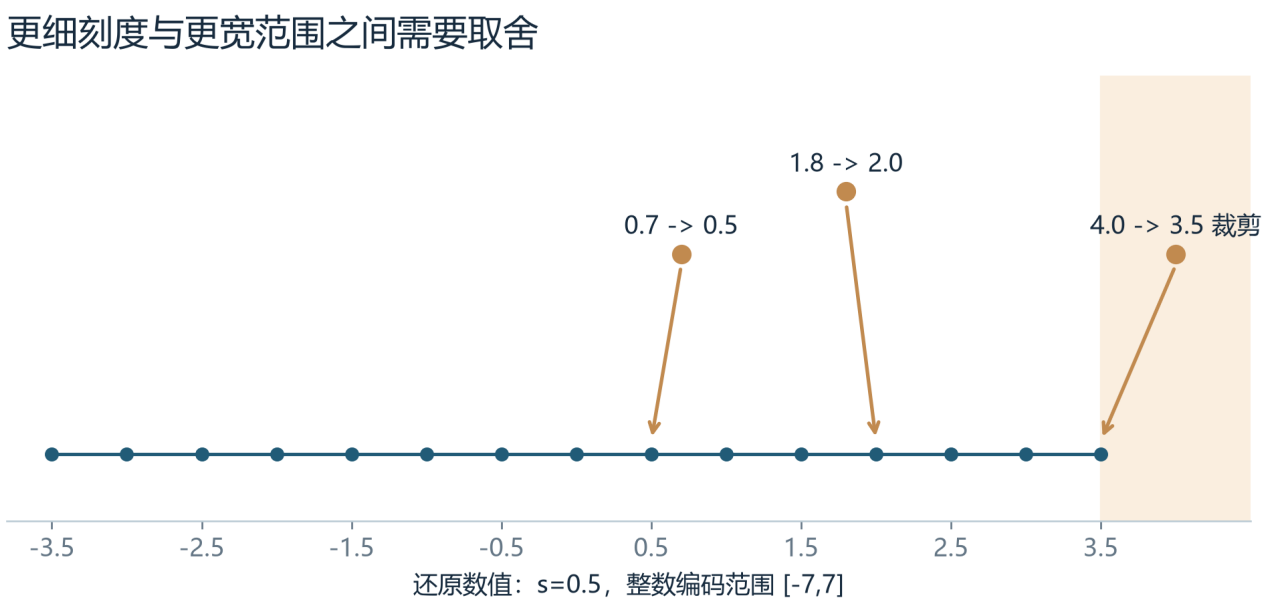


图11.4：教学刻度下，0.7 近似为 0.5，1.8 近似为 2.0；超过范围的 4.0 裁剪为 3.5。

整数矩阵乘法怎样恢复结果

若输入和权重都使用零点 0，近似为 X≈s_x Q_x、W≈s_w Q_w，那么

$$XW \approx s_x s_w (Q_x Q_w).$$

右侧括号可以由适当的整数矩阵实现执行，结果再按两个缩放值的乘积解释。整数乘积通常需要更宽的累计格式，因为很多项相加可能超出单个输入的范围。非零零点则引入补偿项；逐通道缩放还要为不同输出使用不同尺度。具体整数累加和重缩放规则必须与后端一致。

注意，量化与反量化节点可能只是图中表达的语义。优化器可能融合它们，也可能真的执行转换。把整数重新展开成完整高精度权重再调用原计算，能减少文件大小，却不一定减少运行时的权重读取。因此存储压缩和低精度执行应分别检查。TensorRT 量化类型与方案

11.2.5 精确程序变换和近似模型变换需要不同验证

算子融合、布局调整和满足条件的缓存复用，主要改变同一运算怎样执行，需要检查各算子和整体输出，包括浮点顺序带来的误差。量化、剪枝和蒸馏则有意改变模型近似，还要评价任务与鲁棒性。导出验证应覆盖实际形状、分支和自定义算子；一组固定输入能运行，只说明这条路径已经得到支持。

11.2.6 练习题

题11.2-1 导出模型后张量形状正确，但动作结果不同。应检查哪些算子语义？为什么需要中间层数值比较而不只比较最终输出？

11.2.7 参考答案

题11.2-1答案 形状正确只说明张量尺寸兼容，不说明计算含义一致。先固定同一输入、权重、随机噪声和推理模式，并确认图像归一化、通道顺序、动作反归一化、坐标与单位一致，再比较两条计算路径。

重点检查：注意力掩码中 True 表示允许还是屏蔽，是否包含因果约束；softmax 沿哪个维度归一化；归一化层用哪些轴、方差和 epsilon；矩阵转置及广播是否一致；图像缩放的插值方式和像素坐标约定；量化的尺度、零点、舍入和饱和边界。对动作生成器还要检查生成步数、时间输入、求解器更新顺序和初始噪声。不同算子融合或浮点精度可以产生小数值差，需区分预期舍入误差与语义错误。

逐层比较中间结果，可以找到第一处明显偏离。例如视觉编码器输入一致、缩放后立即偏离，就应先查缩放坐标；若直到注意力权重才偏离，就查缩放系数和掩码。只比较最终动作时，多层误差已经混在一起，有的还会暂时抵消。使用最大绝对误差与相对误差，并给接近零的值设置合理绝对容差；最终还需在闭环任务中验证误差影响。

11.3 量化表示、误差、校准与精度分配

11.3.1 为什么异常值、通道和校准如此重要

不是每个数都处于相同范围

激活是模型在当前输入上产生的中间结果，不是另一套固定参数。权重固定时，可以事先看它的数值；激活会随图像、语言、状态和生成进度变化。**异常值**在这里指远大于多数同类值的数，不一定是错误数据，它可能携带重要信息。

假设多数激活在 [-1,1]，少量达到 20。若用最大绝对值设对称四位 scale，s=20/7≈2.857，多数小数会落到零。若把范围裁剪到 [-1,1]，多数值保留得更好，但原来的 20 被改成 1。两种误差谁更危险，需要看这些数怎样影响模型输出，不能只数哪个范围内的样本更多。

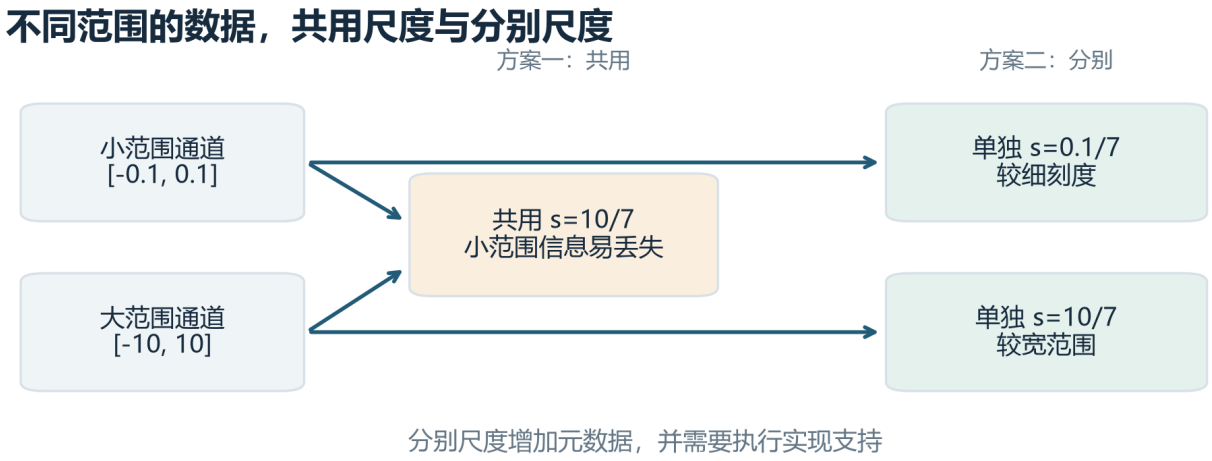


图11.5：共用一个宽刻度会损害小范围数据；分组尺度可以分别适配，但会增加元数据和执行复杂度。

粒度改变了假设

per-tensor 是一个完整张量共用尺度；per-channel 是按通道分别用尺度；per-group 或 per-block 是按固定大小的组分别用尺度。假设一个矩阵的两列范围分别为 [-0.1,0.1] 和 [-10,10]，共用尺度意味着要求两列适应同一套刻度；分别设尺度则承认它们的统计特征不同。

细粒度通常有机会减小误差，但需要更多尺度存储，并增加读取、广播或转换成本。还要说明通道对应哪一维：线性层按输入维还是输出维分组，取决于矩阵布局和算法目的。不是凡是名叫“逐通道”就彼此兼容。

校准究竟在学习什么

校准是在代表性输入上运行原模型，收集中间数值，选择尺度、范围或其他**量化**参数。常见统计包括最大最小值、百分位范围和分布直方图。有些方法还比较量化前后模块输出，选择误差较小的配置。它通常不是重新训练动作策略，也不必使用动作真值标签。

校准集应覆盖预期的图像、任务、状态及困难阶段。例如只用接近杯子之前的图像，无法代表接触、夹爪闭合和遮挡发生后的状态。对迭代生成的动作头，还要运行其生成流程以覆盖不同噪声状态和生成进度。

校准集、选择超参数的验证集和最终测试集承担不同职责。用最终任务结果反复选择裁剪比例，再把同一组任务作为测试，容易高估泛化。没有一个固定校准样本数可以保证所有 **VLA** 都充分；可以通过增加覆盖度并检查误差和成功率是否稳定来判断。

11.3.2 训练后量化怎样实际使用

PTQ 的输入、输出与流程

训练后量化，Post-Training Quantization，简称 **PTQ**，输入是训练完成的模型、量化配置和必要的代表性数据，输出是量化参数与可以部署的模型表示。基本流程是：固定原模型；决定哪些模块和张量量化；收集需要的统计；生成量化表示；检查中间误差；构建目标执行方案；在设备和任务上验证。

PTQ 的“训练后”说明起点，不表示所有 PTQ 都完全不使用数据。有些仅量化权重的方法直接根据权重范围做舍入；**GPTQ** 是一种利用近似二阶信息量化权重的方法，**激活感知权重量化**（Activation-aware Weight Quantization，AWQ）则利用激活信息识别并保护重要权重。两者都要使用校准输入产生的激活，并非只读取权重就完成量化。还有只更新尺度或通过局部重建选配置的方法。应明确究竟改变哪些参数，而非只看名称。

静态与动态说的是尺度何时确定

静态量化事先用校准数据确定有关尺度，部署时复用它们。优点是减少在线尺度计算，代价是需要尺度能覆盖新输入。**动态量化**在推理中根据当前输入或当前数据块计算尺度，适应变化，代价是引入在线计算与可能的同步。动态不等于在线重新训练模型，静态也不等于固定输入形状。

仅权重量化主要压缩权重；权重与激活都量化会进一步改变中间数据与计算方式。W4A16 是常用记号，表示四位权重、16 位激活；它仍未指定是否为整数或浮点、分组方式、累加类型和具体 kernel。W8A8 同样需要补齐这些信息。torchao 量化推理

什么时候优先试哪条路线

如果主要受权重容量与读取限制，可以先尝试目标后端支持的仅权重方案；如果激活搬运和矩阵计算也是明显成本，可评估权重与激活的联合量化。若某模块数值敏感，则保留较高精度并检查整体收益。这个选择来自第11.1节的瓶颈分析，不是认为位数越低越好。

尤其在端侧，先核对具体设备、引擎版本和算子支持。某种格式能被加载，不代表所有层都有高效实现；另一种格式可能需要额外解包、转换或图拆分。用模型文件大小和参数位数预测加速比通常不可靠。

一次失败怎样定位

若输出已经偏离，使用相同输入比较逐层特征，定位偏差首次明显扩大之处，然后分别试更合适的尺度、细粒度、保留敏感模块或 **量化感知训练**（Quantization-Aware Training，QAT）。若输出接近但速度变慢，检查量化转换、后端回退、形状和实际 kernel。若离线输出接近但任务下降，回到动作物理误差、阈值切换、缓存或输入分布变化。三个现象需要不同证据。

11.3.3 GPTQ、AWQ、SmoothQuant 各自在保护什么

权重误差小，输出误差就小吗

设线性层 Y=XW，只量化权重后使用 W+ΔW，输出变化为 XΔW。ΔW 是权重误差，但 X 决定这些误差怎样进入输出。教学输入 x=[10,0.1]，权重 w=[0.1,1]，原输出为 1.1。若两个权重各增加 0.05，输出增加 10×0.05+0.1×0.05=0.505。第一维误差产生的影响是第二维的一百倍。

这就是“考虑激活”的动机：模型对不同通道的使用程度不同，不应把所有权重误差视为同样重要。以下三种方法都是解决数值表示问题的具体路线，不是适用于任意 VLA 的保证。

GPTQ：让量化后的层输出尽量接近原层

GPTQ 使用校准输入和近似二阶信息做权重量化。一个理解其目的的局部目标是

$$\min_{\hat{W}\in\mathcal{Q}} \|XW - X\hat{W}\|_F^2.$$

X 是收集到的该层输入，W 是原权重，带帽的 W 被限制在允许的量化集合 Q 中；F 下标表示把矩阵元素误差平方相加。目标不是让每个权重独自最接近原数，而是让这些权重共同作用后的层输出接近。输入相关性决定哪些权重变化可以互相补偿。对应误差曲面的曲率与 X 的相关矩阵有关；实现使用稳定化和分块等处理，不是直接求整个大模型的完整 Hessian。GPTQ 原始论文

用一个两权重例子说明输出补偿。输入 $x=[1,1]$，权重 $w=[0.6,0.6]$，原输出为 1.2，只允许整数权重。独立舍入为 $[1,1]$，输出 2，误差为 0.8。若先把第一项定为 1，它已经比原来多贡献 0.4；把第二项连续候选从 0.6 减到 0.2，再舍入为 0，输出就成为 1，误差降到 0.2。这个过程展示“一个权重的量化误差可由另一个权重调整来补偿”。真实 GPTQ 用多条校准输入和相应二阶近似组织补偿，下一段检验这种补偿为何依赖输入分布。

如果把上述输入改为 [1,-1]，原输出为 0，独立舍入仍输出 0，而补偿结果 [1,0] 输出 1。这说明补偿依据的数据很重要：在一种相关性下改善，并不保证换一种输入后仍改善。适用条件是校准输入能代表部署、后端能高效执行产生的权重格式，因此还要独立任务验证。

AWQ：用激活识别重要通道，再改变它们的量化条件

AWQ 是激活感知的权重量化。它根据输入激活识别对输出重要的权重通道，用等价缩放改善这些通道的量化误差，而不是简单把所有重要权重留成高精度。AWQ 原始论文

对输入维度的正对角尺度矩阵 S，有

$$XW = (XS^{-1})(SW).$$

左边仍是原线性运算；右边先缩小对应输入通道、放大对应权重通道，两者在未量化时抵消。使用前面的 x 与 w，令 S 的对角项为 [10,1]，新输入为 [1,0.1]，新权重为 [1,1]，输出仍为 1.1。量化后，刻度与通道之间的关系变了，误差可能更有利。具体是否改善取决于组内尺度、裁剪和搜索结果，等价缩放本身不保证减少任何量化误差。

继续把这个例子接到量化：假设两个权重共用最大绝对值除以 7 的尺度，激活保留高精度。原权重 [0.1,1] 近似为 [1/7,1]，输出变成约 1.529；缩放后的 [1,1] 在这套刻度下恰能表示，与新输入 [1,0.1] 相乘得到 1.1。这个特定算例展示缩放为什么可能保护重要通道，不代表真实整层都能无误差。

实际流程是收集统计、选择通道尺度、把可折叠的变换并入邻接参数、量化并交给适配的 kernel。这里的 S 通常是离线选定的参数，不是在每个控制步训练的新网络。

SmoothQuant：把激活的难题转移到更易处理的权重

SmoothQuant 也使用等价缩放，但主要面向权重与激活都量化：当某些激活通道特别大，缩小它们、对应放大权重，把部分量化困难从激活转到权重。尺度在两侧统计之间权衡，并不是让两侧都无限缩小。变换可在合适边界离线折叠；遇到共享输入、残差或非线性，必须保持所有分支语义一致。SmoothQuant 原始论文

一种典型尺度构造为

$$s_j = \frac{a_j^{\alpha}}{w_j^{1-\alpha}}, \qquad 0 \le \alpha \le 1.$$

a_j 是校准得到的输入通道最大绝对值，w_j 是对应权重通道最大绝对值；s_j 是等价变换尺度，不是最后整数编码的刻度。α=1 会把激活各通道最大值缩成 1，让权重承受变化；α=0 会把权重通道最大值变成 1，让激活承受变化；α=0.5 让两侧最大值都成为 sqrt(a_j w_j)。设计思想是分担难度，α 由验证选择，零值与极小值需要稳定化处理。论文第 4 节

方法	核心问题	应重点核对
GPTQ	多个权重的量化误差怎样共同影响该层输出	校准输入、二阶近似、分组及执行格式
AWQ	重要权重通道怎样在低位表示中得到较好保护	激活统计、等价尺度、权重 kernel
SmoothQuant	难量化的激活范围怎样与权重共同分担	两侧误差、变换折叠、联合低精度执行

模型主干中的线性层适合先评估这些思路，但动作头、视觉模块和循环结构不能按语言模型的经验直接推定。是否保留特定模块高精度，应由第11.4节的证据决定。

11.3.4 从实数内积推到整数累加与输出重缩放

量化不仅规定怎么存数，还要规定整数计算结果怎样解释。设 $x_i \approx s_x(q_{x,i} - z_x)$、$w_i \approx s_w(q_{w,i} - z_w)$，则

$$y = \sum_i x_i w_i \approx s_x s_w A, \quad A = \sum_i (q_{x,i} - z_x)(q_{w,i} - z_w).$$

A 是整数累加值，乘 $s_x s_w$ 后才回到实数单位。它通常要用比输入更宽的整数保存，因为很多乘积相加会超过八位范围。如果输出还要存成尺度 s_y、零点 z_y 的整数，就继续计算

$$q_y = \mathrm{clip}\left(\mathrm{round}\left(\frac{s_x s_w}{s_y} A\right) + z_y, q_{\min}, q_{\max}\right).$$

这一步叫**重缩放**：把累加结果的单位换成输出编码的单位，并承担新一轮舍入与饱和误差。实现可以用固定点乘法等方式完成这种换算，不必在所有位置都转回浮点。gemmlowp 的量化算术教学

手算 $x = [0.5, -0.5]$、$w = [0.2, 0.3]$。选择 $s_x = s_w = 0.1, z_x = 128, z_w = 0$，编码为 $q_x = [133, 123], q_w = [2, 3]$。整数累加为 $A = 5 \times 2 - 5 \times 3 = -5$，实值为 $0.01 \times (-5) = -0.05$。若输出尺度为 0.05、零点为 128，输出编码就是 127，再解码仍为 -0.05。

```
import numpy as np

def quantized_dot(qx, qw, sx, sw, zx, zw, sy, zy,
                  qmin=0, qmax=255):
    # 1. 先提升累加位宽，再扣零点；避免八位数组直接相乘溢出。
    x_code = np.asarray(qx, dtype=np.int64) - zx
    w_code = np.asarray(qw, dtype=np.int64) - zw
    accumulator = int(x_code @ w_code)
    # 2. 将整数累加单位转换为输出编码单位。
    output_code = np.rint((sx * sw / sy) * accumulator) + zy
    # 3. 输出编码受有限范围限制，恢复时使用输出自己的尺度。
    output_code = int(np.clip(output_code, qmin, qmax))
    return output_code, sy * (output_code - zy)

print(quantized_dot([133, 123], [2, 3],
                    0.1, 0.1, 128, 0, 0.05, 128))  # (127, -0.05)
```

真实零对应的是编码 z。本例给输入补实数零，应填 128；填编码 0 会被解码成 -12.8，从而产生错误贡献。这解释了零点为何影响卷积填充与矩阵计算，而不只是图中多了一个参数。

展开累加还可得到 $A = \sum q_x q_w - z_w \sum q_x - z_x \sum q_w + n z_x z_w$。对称权重取 $z_w = 0$ 时，有些补偿项便消失。后端对零点、逐通道尺度和打包布局的要求，因此直接关系到能采用哪种高效算术。以上 NumPy 程序演示数值关系；实际低位加速需要对应整数内核与数据布局。TensorRT 量化类型说明

11.3.5 在 CPU 上先看懂量化误差

下面只实现正文的教学对称刻度，没有打包和 **GPU** kernel。它展示数值近似，不能用运行速度评价真实低位推理。

算法步骤：

1. 确认尺度有效。
2. 按尺度取整，并限制到整数表示范围。
3. 将整数编码恢复到近似实值。

```
def fake_quantize(values, scale, qmin=-7, qmax=7):
    # 步骤1：确认尺度有效
    assert scale > 0
    # 步骤2：按尺度取整，并限制到整数表示范围
    codes = [max(qmin, min(qmax, round(v / scale)))
             for v in values]
    # 步骤3：将整数编码恢复到近似实值
    restored = [scale * q for q in codes]
    return codes, restored

x = [-1.2, -0.2, 0.7, 1.8, 4.0]
q, x_hat = fake_quantize(x, 0.5)
print(q)      # [-2, 0, 1, 4, 7]
print(x_hat)  # [-1.0, 0.0, 0.5, 2.0, 3.5]
```

Python 的 round 对平局有其规则，真实引擎需要匹配目标定义。继续试较小 scale，并计算误差如何进入一个线性层，而不是只看整数编码的样子。

11.3.6 理解 Triton 的分块与掩码

这是独立的教学核心片段，需要可用 GPU、匹配 PyTorch/Triton 环境、已在设备上的连续一维输入及正确调用方式。

算法步骤：

1. 为程序块中的每个位置分配元素索引。
2. 标出仍在数组内的位置。
3. 只读取有效元素。
4. 相加并只写回有效位置。

```
import triton
import triton.language as tl

@triton.jit
def add_kernel(x, y, out, n, BLOCK: tl.constexpr):
    # 步骤1：为程序块中的每个位置分配元素索引
    i = tl.program_id(0) * BLOCK + tl.arange(0, BLOCK)
    # 步骤2：标出仍在数组内的位置
    valid = i < n
    # 步骤3：只读取有效元素
    a = tl.load(x + i, mask=valid, other=0)
    b = tl.load(y + i, mask=valid, other=0)
    # 步骤4：相加并只写回有效位置
    tl.store(out + i, a + b, mask=valid)

# out 与输入形状、设备和类型匹配，连续存储
# add_kernel[(triton.cdiv(n, 256),)](x, y, out, n, BLOCK=256)
```

program_id 选择当前块，arange 选择块内元素，valid 防止越界。它表达的是数据分工，不是完整模型插件。实际练习先与框架相加对照，再增加非整块长度，最后按第11.1节规范计时。

11.3.7 训练和部署不要混在一个隐含步骤里

```
PTQ: 原模型 -> 代表性输入 -> 统计与配置 -> 量化图/权重
     -> 目标引擎 -> 数值验证 -> 设备测量 -> 闭环任务

QAT: 原模型 -> 插入假量化 -> 输入与目标前向 -> 损失
     -> 更新指定参数 -> 转换真实格式 -> 引擎与任务复测

蒸馏: 输入 -> 冻结教师提供目标 + 学生预测
      -> 任务/分布/特征损失 -> 更新学生 -> 部署学生
```

每个箭头要保存可追溯产物。若方法添加训练、额外目标或选择器，把它加入记录与成本，不用“进行优化”四个字隐藏过程。

11.3.8 一张实验记录应该有什么

项目	需要填写的内容
任务与模型	任务划分、模型版本、参数量、动作协议
输入与生成	相机与尺寸、Token 长度、状态字段、H、E、生成步
压缩配置	权重/激活格式、粒度、尺度来源、保留模块
数据来源	校准、训练、验证、测试各自来源与数量
训练成本	更新参数、教师与标签、设备时间、额外交互
执行条件	硬件、引擎、依赖、功耗模式、散热、其他负载
测量协议	预热、重复次数、同步、是否含传输与排队
性能记录	原始延迟样本、分位数、峰值内存、观测年龄
任务记录	成功数与分母、任务分层、置信区间、失败类型
比较对象	优化基线、单方法、组合、相同预算对照
可复现产物	图/权重/引擎、协议清单、随机状态与回放

11.3.9 按能力逐步做四个项目

第一，选一个可访问的小模型，建立输入输出协议并测完整链路，能够解释总时间由哪些部分组成。没有目标设备时，可以先做数值和计算图学习，但最终设备结论要等真实测量。

第二，在支持的执行环境对比原精度、PTQ 和一种误差恢复路线，保存中间误差与真实性能，解释为什么某配置变快或变慢。

第三，实现一个基础 GPU 算子，再研究一个真实热点的融合或布局变化。比较成熟实现，覆盖边界，观察局部和整体收益。

第四，把配置接回具身策略，在回放、仿真与可用真机上分层验证，固定控制协议，记录观测年龄与任务效果。此时再讨论任务自适应计算或多模块组合，研究问题会更明确。

11.3.10 练习题

题11.3-1 非对称量化 x≈s(q−z)。两个向量的整数点积怎样还原到实值？为什么直接累加 qx·qw 可能错误?

11.3.11 参考答案

题11.3-1答案 先假设每个向量内部使用同一尺度和零点：$x_i \approx s_x(q_{x,i} - z_x)$，$w_i \approx s_w(q_{w,i} - z_w)$。长度为 n 的点积应为

$$x^\top w \approx s_x s_w \sum_{i=1}^{n} (q_{x,i} - z_x)(q_{w,i} - z_w).$$

展开可得到适合整数累加的形式：

$$s_x s_w \left[\sum_i q_{x,i} q_{w,i} - z_w \sum_i q_{x,i} - z_x \sum_i q_{w,i} + n z_x z_w \right].$$

整数零点表示实数零，不一定等于整数 0。直接累加 $q_x q_w$ 会把零点偏移也乘进去，只有零点均为 0，或实现已经补偿偏移时才可直接采用它。

例如 $s_x = 0.1, z_x = 10, q_x = [12, 9]$，对应 $x = [0.2, -0.1]$；$s_w = 0.2, z_w = 5, q_w = [8, 3]$，对应 $w = [0.6, -0.4]$。正确点积为 $0.12 + 0.04 = 0.16$。整数侧先减零点得到 $[2, -1] \cdot [3, -2] = 8$，再乘 0.02，仍为 0.16；直接用整数原值则得 $0.02 \times 123 = 2.46$。若输出仍需量化，还要除以输出尺度、加输出零点，再舍入和截断。分组或逐通道尺度则需按对应组、通道还原。

11.4 量化训练、低比特适配、剪枝与蒸馏

11.4.1 QAT 为什么要模拟量化，怎样训练

一次性改数值不够时，给模型适应机会

PTQ 直接改变已经训练好的数值表示。如果模型原来依赖的细微差别在**量化**后消失，可以让参数在这些限制下重新适应。**量化感知训练**，Quantization-Aware Training，简称 **QAT**，通常在训练前向中插入 fake quantization，**假量化**，模拟舍入与裁剪；训练参数仍可保存为可更新的浮点数。torchao QAT 流程

“假”不表示误差是假的，而是训练仍使用便于求梯度的表示，同时在前向中经历预期数值限制。假量化可写成 Q_s(x)=反量化(量化(x))。它输出浮点形式的近似值，以便接在其他计算后面。训练结束后转换到实际低位表示并构建执行方案；训练中的假量化速度不能当部署速度。

梯度从哪里来

舍入是阶梯函数，几乎处处导数为零，直接反传无法有效调整权重。**直通估计器**（Straight-Through Estimator，STE）保留前向中的量化值，却在反向中采用替代导数，例如有效范围内令 $\partial Q_s(x)/\partial x \approx 1$。于是输出误差仍能传到保留的浮点参数，推动它跨过下一档量化边界。前向负责让模型经历有限刻度，替代梯度负责让参数能够适应；这是一种优化近似。饱和范围之外的梯度与尺度更新规则，还需与具体量化实现一致。

一个教学任务损失为

$$L(\theta) = \mathbb{E}_{(o,a^*)\sim\mathcal{D}} \left\| \pi_{\theta,\text{fake}}(o) - a^* \right\|_2^2.$$

o 是可用观测和任务条件；a* 是遥操作或示范数据中的目标动作；D 是训练数据分布；π 的下标说明前向包括假量化；更新对象 θ 是策略参数。损失告诉模型“在量化限制下仍应输出示范动作”，而不是“把整数改得越大越好”。离散动作可用分类损失，流或**扩散**动作头应保留其相应生成训练目标，而非一概替换成这个教学**均方误差**。

一个标量算例能看出更新作用：输入 x=1，浮点权重 w=0.7，固定刻度 s=0.5，假量化后权重为 0.5，目标输出为 1，平方损失为 0.25。用有效范围内的直通梯度，权重梯度近似为 -1；学习率 0.2 的更新使 w 变成 0.9，下一次假量化得到 1，损失变成 0。参数跨过了表示等级的边界。这展示近似梯度如何使模型适应有限刻度，不是所有训练都会单步成功的保证。

QAT：前向受数值限制，反向更新主参数

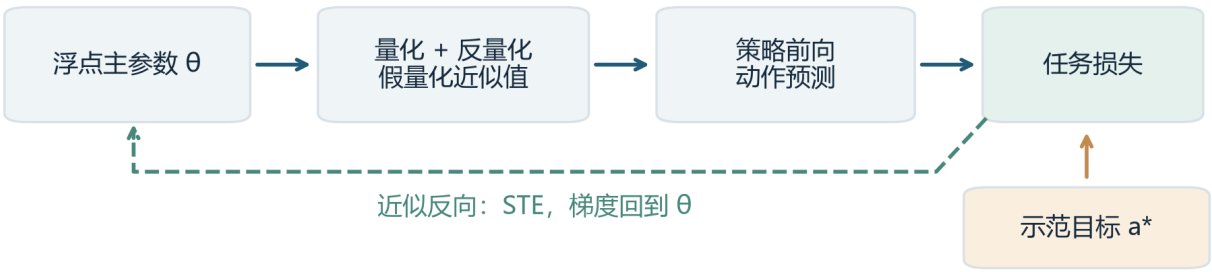


图11.6：前向经历近似数值限制；真实动作标签约束输出；反向通过近似梯度更新浮点主参数。

可执行的训练与转换顺序

先从可用模型初始化，并决定哪些模块参与 QAT；用代表性数据确定初始尺度或观察统计；前向插入假量化；用原任务标签或教师提供的目标计算损失；反向更新选定参数；在训练中逐步稳定或冻结相关统计；转换成与训练假设一致的真实量化格式；构建引擎并重新验证。

这里的 observer，观察器，是收集范围等统计的组件，不一定是一个需要梯度训练的神经网络。某些方案学习尺度，某些固定尺度；冻结统计不等于冻结模型全部参数。论文或实现需要分别说清。

QAT 的条件与成本

QAT 需要训练数据、额外计算和可靠的验证集。若模拟的格式、裁剪、粒度或计算规则与部署不一致，训练改善可能无法传递到设备。数据只覆盖容易阶段，也可能出现总体损失下降而接触阶段退化。建议同时监控模块误差、物理动作偏差和闭环结果；它是精度恢复路线，不是替代性能分析的路线。

11.4.2 怎样选择混合精度并评价敏感性

什么叫某层“敏感”

敏感不是参数很大或名字很重要，而是该层数值限制会明显改变需要的输出或行为。可以先仅量化一个模块，其余保持原模型，在相同输入和生成噪声下比较动作；再组合模块检查交互。单层结果不能简单相加预测全模型。

考虑输入和权重都有变化

$$\Delta Y = X\Delta W + \Delta X W + \Delta X \Delta W.$$

第一项来自权重误差，第二项来自输入激活误差，第三项来自二者交互。后续非线性、归一化和残差还会继续改变影响，因此用一层的权重误差排名代替全模型行为敏感性存在局限。

衡量什么输出

中间特征的相对误差适合定位层，但分母接近零时指标会不稳定，应给出绝对误差或稳定分母。最终动作要看平移误差、旋转表示对应的角度误差、夹爪开合一致率、**动作块**平滑性，以及接触前后是否发生明显跳变。不同维度有不同单位，直接把米、弧度和夹爪量混在一个均方误差里，可能让尺度较大的维度主导结论。

教学例子：配置甲节省 8 ms，但使关键接触位置误差增加 4 mm；配置乙节省 6 ms，位置误差增加 1 mm。哪个可接受由任务容差和成功率决定，而不是只选加速更多的甲。阈值、动作归一化和数值检查必须覆盖部署后的实际输出。

把选择看成有约束的资源分配

设 c 为模块精度配置，一个教学目标是

$$\min_{c} T(c) \quad \text{subject to} \quad M(c) \le M_{\max}, \quad R(c) \ge R_{\min}.$$

T 是目标设备上的时间，M 是峰值内存，R 是任务质量，例如独立任务成功率；边界由部署需要决定。它表达“在满足容量和质量要求的配置中选择较快者”。这些函数通常没有简单解析式，需要实际构建和测量。也可加入功耗、尾部延迟等约束。

一个合理顺序是先使用硬件支持且误差可接受的基础精度，再对主要耗时模块降低位数，对少量敏感计算保留高精度，最后重新检查融合和转换开销。精度切换太碎，可能让混合方案失去原本的执行优势。

11.4.3 剪枝、稀疏化与蒸馏为什么不同

剪枝删除计算结构

剪枝删除权重或网络结构；**稀疏化**使大量元素为零；二者常有联系，但并不自动让执行更快。非结构化**剪枝**可把任意权重置零，若仍调用相同密集矩阵乘法，程序通常没有减少原有乘加工作。结构化剪枝删除整列、通道、头或层，可以改变实际形状，更容易减少工作量，但更可能影响表示能力。

例如某层 X 的形状为 M×1024，W 为 1024×1024。把半数权重设零，密集计算仍按原形状执行。若删除 256 个输出通道，W 变为 1024×768，计算量减少四分之一；但后续输入和**残差连接**也要做一致调整，而且矩阵形状变化后的实际速度仍需测量。

稀疏算子能在支持的布局下跳过部分零值，但要存储位置并执行索引。零值太少、分布太不规则或输入太小，索引开销可能抵消收益。2:4 等结构化稀疏模式对分组和硬件有明确限制，不能把任意 50% 零权重称为相同加速条件。cuSPARSELt 稀疏执行文档

怎样判断哪些结构可以删除

权重幅值是容易计算的候选指标，但小权重可能乘以很大的激活。使用梯度或删除后的输出变化，可以纳入任务信息；这些仍然依赖数据覆盖。建议先明确删的单位，收集候选重要性，尝试少量删除，修复或微调，再评估部署与闭环。原始剪枝工作展示了剪枝后再训练的路线，不能据此推出每个 **VLA** 都存在同样比例的冗余。剪枝与再训练原始论文

蒸馏改变的是学生如何学

知识蒸馏用一个教师模型或系统产生训练信号，训练另一个学生模型。学生可以从一开始就设计成较小结构，不要求它是教师删除几层后的版本。它可能减少参数与计算，也可能只是提升一个同样大小模型的能力。教师负责提供目标，部署时通常运行学生；是否仍需教师在线参与，要看系统设计。

压缩方法的选择由瓶颈决定：想减少权重读取可尝试量化；想改变实际矩阵尺寸可尝试结构剪枝；想换成一套更小架构可尝试**蒸馏**。组合时要逐项确认节省的资源和引入的误差，第11.10节给出组合实验方式。

11.4.4 机器人策略蒸馏的标签、损失和信息边界

教师与学生各自接收什么

教师是提供目标的已训练模型或专家系统；学生是待训练、最终运行的模型。它们不是两种固定网络类别。部署学生可以输入图像、语言和自身状态，输出连续动作、离散动作 Token 或动作分布。训练教师可能使用更丰富信息，但学生部署时没有的信息必须在训练说明中明确。

如果教师使用未来轨迹或完整三维环境，生成的是依赖额外信息的目标。学生可以学习其中在自己输入下可推断的规律，但不能保证恢复那些输入根本无法区分的决定。例如相机看不到杯子是否被固定，教师根据触觉选择不同动作，学生只有单张图像就可能无法确定该选哪一种。应补充学生可用观测、使用历史，或把目标改成适当的不确定分布。

连续动作蒸馏怎样更新学生

一个教学目标为

$$L = \lambda_d \|a_S(o) - a_T(o)\|^2 + \lambda_s \|a_S(o) - a^*\|^2.$$

a_S 是学生预测，a_T 是固定教师输出，a* 是示范动作真值；两项分别让学生学习教师行为和真实示范。只有学生参数更新，教师输出作为目标。权重 λ 控制两类监督的相对影响，不能通过任意增大教师权重保证更好。动作块可以逐步加权，关键接触阶段也可另行评价。

若教师存在多种成功模式，例如从物体左侧或右侧绕过障碍，直接平均两个连续动作可能变成撞向障碍的中间路线。学生应保留与任务相符的分布、模式条件或生成结构，不能把所有策略蒸馏都简化成均方误差。

离散分布与特征蒸馏

离散动作教师输出概率 p_T，学生输出 p_S，可以用 $D_{\mathrm{KL}}(p_T \| p_S)$ 训练。相较于只给一个正确类别，教师还能表达“第二种动作也合理，第三种几乎不可取”，学生因此学习相对偏好。若概率由 logits 经 softmax 得到，温度 T 增大通常使分布更平缓，让原先很小的候选概率也参与监督；训练常配合损失尺度调整。温度只是监督表达的选择，不能保证越大越好。部署仍使用规定的动作解码与采样规则。知识蒸馏。

特征蒸馏让学生的中间表示接近教师，维度不同可使用投影模块。该模块输入学生特征、输出匹配教师的维度，标签来自冻结教师的对应特征。它通常仅训练时使用，但若设计为部署所需模块，必须算进推理成本。特征接近不自动表示动作正确，还需要动作监督和任务证据。

完整数据流程与分布变化

在训练观测上离线运行教师，保存或在线产生目标；学生前向得到动作与必要特征；计算对应损失；更新学生；在独立观测和闭环中评估。闭环学生偏离教师轨迹后会看到新状态，原训练集不一定覆盖。可以收集学生访问状态并请求教师或人工提供目标，再重新训练，但这些额外数据和交互成本必须在实验中说明。

一个公平比较同时报告教师能力、原学生、只用示范训练的学生和蒸馏学生；若教师提供额外数据，增加相应数据基线。教师更大不是贡献本身，学生在明确资源预算下保留了什么能力才是可检验主张。

11.4.5 用代码分开假量化的前向数值与替代梯度

前面的 $w = 0.7 \to 0.9$ 可以用几行代码验证。令 $Q(w)$ 是舍入裁剪后的浮点值。在表示范围内，构造 $Q(w)$ 的前向值、对 w 取 1 的反向导数；范围外，本例取零替代导数。这是一个明确选择的 STE，而不是舍入函数的真实导数。

```
import torch

def fake_quant_ste(weight, scale, qmin=-7, qmax=7):
    # 1. 前向模拟有限刻度与裁剪。
    quantized = torch.clamp(torch.round(weight / scale), qmin, qmax) * scale
    # 2. 范围内传递替代梯度；范围外不继续推动饱和值。
    inside = ((weight >= qmin * scale) & (weight <= qmax * scale))
    # 第一项提供前向值；第二项前向为零，反向提供规定的导数。
    return quantized.detach() + (weight - weight.detach()) * inside

weight = torch.tensor(0.7, requires_grad=True)
prediction = fake_quant_ste(weight, scale=0.5)
loss = (prediction - 1.0).square()
loss.backward()  # 梯度为 -1，来自替代求导路径。
with torch.no_grad():
    weight -= 0.2 * weight.grad
print(weight.item(), fake_quant_ste(weight, 0.5).item())  # 约 0.9，1.0
```

训练保存浮点权重，前向却让它经历有限刻度。这样参数可以逐渐跨越量化分档，而不是因真实舍入导数为零就停住。部署转换还要使用与训练相容的范围、粒度及格式。torchao QAT 教程

这也解释了 QAT 与 QLoRA 的差别：QAT 为未来量化执行调整模型；QLoRA 通常冻结量化的基础权重，只训练适配器以节省微调存储。二者可能组合，但改变的参数和学习目标应分别说明。

11.4.6 量化与蒸馏的流程

```
PTQ：原模型 -> 代表性输入 -> 统计与配置 -> 量化图/权重
     -> 目标引擎 -> 数值验证 -> 设备测量 -> 闭环任务

QAT：原模型 -> 插入假量化 -> 输入与目标前向 -> 损失
     -> 更新指定参数 -> 转换真实格式 -> 引擎与任务复测

蒸馏：输入 -> 冻结教师提供目标 + 学生预测
      -> 任务/分布/特征损失 -> 更新学生 -> 部署学生
```

11.4.7 练习题

题11.4-1 PTQ 在**校准**数据上误差很小，却在机器人闭环中失败；QAT 是否必然解决？如何选择校准与蒸馏目标？

11.4.8 参考答案

题11.4-1答案 PTQ（Post-Training Quantization，训练后量化）在固定校准集上表现好，不保证闭环好。一次小动作偏差会改变下一次看到的画面，随后模型进入校准集没覆盖的状态，误差可能继续放大；接触、急停、遮挡和恢复阶段往往比正常运动更敏感。

QAT（Quantization-Aware Training，量化感知训练）在训练前向中模拟舍入、截断等低精度效果，让参数适应这些误差。它仍依赖训练数据覆盖，并且模拟计算要符合部署后端；若失败来自新状态、错误动作尺度或不支持的算子，仅做 QAT 不会必然解决。

校准集应覆盖真实相机、光照、任务阶段、接触状态和策略会遇到的困难情形，最好加入压缩策略闭环产生的轨迹。除全局平均误差外，分层检查激活的极端值、截断率和关键动作误差，必要时让敏感视觉层、归一化或动作输出保持较高精度。

蒸馏目标应与学生的生成机制对应：自回归学生可拟合教师的 token 概率；流模型可在相同条件、噪声状态和生成时刻拟合教师向量场，再检查生成动作块。若数据包含多种合法路线，不宜把教师的冲突动作直接求平均。最后以任务成功率、接触力和恢复能力检验，配合相同闭环场景中的误差累积分析。

11.5 Token、缓存与 VLA 生成工作量

11.5.1 Token 裁剪、缓存与动态推理怎样减少工作

少处理输入，不等于少存权重

Token 裁剪选择一部分序列元素参与后续计算。教学视觉编码若每路图像拆成 14×14 个块，两路共 392 个视觉 Token，再加 32 个文本 Token 和 1 个状态 Token，总数为 425。这是本书的示意编码方式，实际模型可能使用不同图块大小和多尺度特征。

注意力中若序列长度为 N，成对关系数量通常约为 N^2。N 减半可使这部分关系数量约为四分之一，但线性层一般只减少到一半，视觉编码器和其他固定成本可能保持不变，还要加选择与整理 Token 的开销。因此计算量下降比例不能直接作为端到端加速比。

选择分数可以来自任务相关性、特征变化或一个学习模块。学习模块的输入是可用视觉特征和任务，输出保留分数或比例；训练信号可以来自模仿完整模型、任务标签或计算预算。评分模块本身有成本，且注意力分数高不自动表示那个图块对行动有因果必要性。

缓存为什么需要判断依赖

缓存保存已经算过的结果，后续条件满足时复用。原始静态文本编码可精确复用的前提，是编码器只依赖相同文本及相同参数。某个融合后的文本特征若依赖当前图像，就不能仅因为指令没变而声称精确复用。

相邻相机帧相似时，复用视觉特征通常是一种近似。需要规定哪些位置重算、多久全部刷新、怎样处理遮挡与接触变化。动作生成内部也可能复用相近生成步的中间特征，风险来自当前噪声状态和进度仍在变化。**VLA**-Cache 和 EfficientVLA 分别提供了时间冗余利用与组合加速的研究实例，理解机制后仍需核对其模型与评测条件。VLA-Cache、EfficientVLA

图11.7：裁剪改变本次处理的元素；缓存用过去结果替代部分重算。二者需要不同的有效性条件。

动态推理如何选择工作量

动态推理根据当前输入或过程，改变保留 Token 数、网络深度或生成步数。一个控制模块可以在容易状态减少计算，在遮挡、接触或模型不确定时增加计算。该模块输入当前可得信息，输出计算配置，不应在部署时依赖真实未来成功标签。

如果以任务质量减计算成本训练这个选择器，需要说明任务反馈怎样得到、代价如何定义和探索需要多少机器人交互。若仅用中间差异作启发式规则，就应把它称为规则，并验证该差异能否预测危险误差。变化多的形状还可能降低引擎效率，可将配置限制在少量已构建形状档位。

低计算配置表现差时，改进路径包括补充困难状态、增加强制刷新、保留任务关键区域，或调整选择器训练目标。单纯提高平均缓存比例，只证明少算了，尚未证明机器人仍及时可靠。

11.5.2 语言模型的缓存和服务优化，哪些能迁移到 VLA

Prefill 与 Decode 的工作不同

自回归模型根据已有序列逐项生成新 Token。**Prefill，前缀处理**，是先处理已有输入；**Decode，逐步生成**，是继续计算下一项。前缀通常有多项可在一次前向中共同计算，逐步生成常一次只增加一项。因此相同权重在两个阶段的复用程度、计算形状和瓶颈不同。

TTFT（Time to First Token，首 token 延迟）是从请求到首个输出 token 的时间，TPOT（Time per Output Token，每个输出 token 耗时）是后续每个 token 的生成时间。它们适合语言服务，但机器人可能必须收齐一个动作或**动作块**才能执行，首个 Token 早出现不等于新动作已可用。VLA 部署应另测完整动作可用时间。

KV Cache 保存什么

自回归因果注意力中，过去位置不能读取尚未产生的未来位置。若前缀输入、位置规则和参数不变，过去位置的键和值可以复用，称为 **KV Cache，键值缓存**。新位置仍要生成自己的查询、键和值，并与允许的历史比较。它避免重复计算旧位置的键和值，未消除读取历史或新位置的网络计算。

简单容量估算为

$$M_{\mathrm{KV}} = 2LNh_{\mathrm{KV}}d_h b_e.$$

2 代表键和值，L 是层数，N 是已保存位置数，h_KV 是键值头数，d_h 是每头宽度，b_e 是每元素字节。教学 L=32、N=2048、h_KV=8、d_h=128、b_e=2 时，单序列约占 256 MiB；批次与多序列还会增加。查询头与键值头数量可以不同，公式应使用真实键值头数。

PagedAttention 将缓存按块管理，减少复杂长度与多请求下的分配浪费并支持共享等机制。vLLM 是采用相关机制的推理系统。这个方向对多请求语言服务重要，但一个机器人的单请求 VLA 需要先确认模型结构与执行路径是否适配。PagedAttention 与 vLLM 原始论文

批处理与推测解码

动态或连续批处理把不同请求共同组织，提升资源利用，但可能增加排队或共享资源的等待。机器人系统里多个模型竞争同一 **GPU**，也需要调度，不过决策应考虑动作截止时间，不能只最大化每秒完成的请求数。

推测解码让小模型先提出多个离散 token，再由目标模型并行计算这些位置的概率，并按规则接受或修正。为何它能够保持目标分布？看一个位置：草稿分布为 q，目标分布为 p，从 q 抽到 y 后，以 $\min(1, p(y)/q(y))$ 接受；拒绝时从归一化后的 $[p-q]_+$ 采样，其中正部表示仅保留大于零的差值。接受部分提供 $\min(p,q)$ 的概率质量，修正部分补上目标缺少的质量，合起来便得到 p。多位置算法按已接受前缀逐步处理，在首个拒绝处校正并结束这一轮。加速来自草稿与并行验证的配合，取决于接受率和额外开销。推测解码。

它依赖离散自回归生成的特定结构。流动作头和**扩散**动作头有不同的生成变量与更新规则，不能只把几个未来动作叫“草稿”就认为同一分布保证成立。跨控制步的新图像若改变旧特征，也不能直接把本次 KV 缓存沿用到下一次决策。需要从依赖关系判断精确复用还是近似复用。

11.5.3 VLA 的动作生成结构怎样决定优化方法

先看模型输出是怎样产生的

自回归动作 Token 模型逐项生成离散编码，再恢复连续动作；直接回归动作头一次前向输出动作；扩散或流动作头通过多次网络计算把初始变量变成动作块。它们都能实现策略，但单次请求包含的工作不同。OpenVLA 与 π0 分别提供了不同动作建模路线的原始阅读入口。OpenVLA、π0

不要从“用了 **Transformer**”就推定整套系统按语言生成方式执行。应画出视觉编码器、条件主干、动作头及外层生成循环，记录每一项一轮决策执行几次。某个主干很大但只运行一次，另一个动作头较小却重复运行十次，后者也可能占主要时间。

流动作头在更新哪一个变量

一个简化条件流生成过程，以随机动作变量 A_0 开始，网络给出生成方向 v_θ(A_τ,τ,c)，其中 c 是图像、文本、状态条件。使用教学欧拉更新

$$A_{\tau+\Delta\tau} = A_\tau + \Delta\tau\, v_\theta(A_\tau, \tau, c).$$

τ 是从噪声到目标动作的生成进度，不是机器人真实时间；A 是正在生成的一整块动作变量，v 描述这个变量怎样变化，不一定是末端物理速度；θ 是训练好的参数，在一次推理中通常不更新。每次调用网络得到方向，再由外层程序改变 A。

一种理解训练标签的简化路径是 A_τ=(1-τ)A_0+τA*，A* 来自示范动作块，A_0 来自采样噪声，对应方向目标为 A*-A_0。网络训练学习在给定条件与中间状态下预测方向。具体论文可能使用不同路径、预处理和目标，本式仅解释为什么部署需要反复调用网络。

减少生成步数的代价

若允许条件模块只算一次并复用，时间可粗略写成

$$T \approx T_{\text{condition}} + S(T_{\text{step}} + T_{\text{launch}}) + T_{\text{other}}.$$

S 是内部生成步数，T_step 是每步动作网络时间。少运行几步会减少这部分工作，但积分更粗或去噪过程改变，最终动作可能偏离。由十步改为五步不能直接宣称输出等价，也不能直接把全部延迟折半。可先固定噪声比较动作，再评估平滑性与闭环；必要时采用专门训练或**蒸馏**的少步模型。

条件模块能否复用取决于注意力依赖。若某个特征读取了不断变化的动作变量，它通常需要重算；若其输出只依赖固定条件，就有精确复用机会。不同模块的**量化**敏感性也不同：每步误差会进入下一生成步，因此要评价完整生成链，而非只评价一轮动作头输出。

一个模型对应多张部署清单

视觉部分检查图像尺寸和预处理；主干检查 Token 数、缓存与线性层；动作头检查形状、生成步、数值精度和重复开销；外层程序检查随机状态、时间戳和动作协议。高效 VLA 研究可以跨模块组合，但要明确每一项到底节省了什么。openpi 官方实现

11.5.4 动作生成中的条件缓存，何时才是精确复用

“图像和语言不变”还不足以保证融合后的条件特征不变。将 token 分成条件 C 和当前生成动作 A_τ，假设每层的信息依赖为

$$C^{l+1} = F_l(C^l), \qquad A_\tau^{l+1} = G_l(A_\tau^l, C^l).$$

条件分支只读条件，动作分支读取条件与自身变量。即使 A_τ 在生成过程中持续改变，C^l 仍可复用；每层条件的键和值也可缓存，供动作查询读取。这类单向依赖可以由注意力掩码规定。自回归前缀缓存依赖的也是“旧位置不读取新位置”的信息边界。Hugging Face 的缓存教学

如果条件 token 也读动作，则变成 $C^{l+1} = F_l(C^l, A_\tau^l)$。更新动作以后，融合后的条件特征也会改变；沿用旧值是在做近似，而非完全相同的计算。例如只有一个条件值 1、一个动作值从 0 改成 2，若条件查询对二者各取一半，条件输出就从 0.5 变成 1.5。输入条件虽没变，其隐藏特征已经不同。

因此先画清每层谁能读谁，再决定缓存哪些张量。一轮动作生成内部的固定条件缓存，与跨机器人控制步复用旧图像特征，是两种不同情形。新的图像、任务、位置规则或模型参数到来后，要相应刷新缓存；跨帧视觉近似复用还需专门检查运动、遮挡和接触变化。openpi 的模型与推理实现

11.5.5 练习题

题11.5-1 KV 缓存为何能减少自回归重复计算，却不能把全部注意力计算变成常数？为什么动作流模型不能直接缓存所有去噪 token?

11.5.6 参考答案

题11.5-1答案 在因果自回归模型中，已有 token 的表示不会因追加未来 token 而改变，所以各层可保存它们的键 K 和值 V。生成第 t 个新 token 时，只计算新 token 的投影，再让新查询 q_t 与缓存的 t 个键比较并汇总值，避免重算旧 token 的整套网络。

但新 token 仍需计算 $q_t K_{1:t}^\top$，并对全部可见历史归一化。因此标准稠密注意力的单步工作量随历史长度增长，KV 显存也增长。缓存省掉重复投影和旧位置计算，并未把新查询与历史的比较变成常数；窗口、稀疏或其他结构才可能进一步限制可见历史。

流或扩散动作模型每个生成步都会改变整个待生成动作块，生成时间输入也变化。第 k 步的动作表示产生的 K, V 通常不等于第 $k+1$ 步所需值，直接复用会让网络读取旧动作状态。固定图像和语言的编码可缓存；某层的条件侧 K/V 只有在它们不受动作块或生成时间影响时才可精确缓存。例如条件前缀若会反过来注意动作 token，就不满足这一条件。缓存动态动作分支只能作为另行验证的近似方法。

11.6 CUDA 并行、访存与矩阵分块

11.6.1 CUDA 的线程、线程块和 Warp 在做什么

给很多执行者分配数据

CUDA是 NVIDIA 的 **GPU** 并行计算平台和编程模型。它需要 **CPU** 程序组织输入和提交工作，GPU kernel 处理指定数据。线程是一个执行实例，线程块是一组可以在规定范围内协作的线程，grid 是一次 kernel 启动的全部线程块。CUDA 编程模型

给长度 1000 的两个向量逐项相加，可以让每个线程负责一个位置。每块 256 个线程，启动四块，共 1024 个线程；最后 24 个没有有效数据，所以需要边界判断。数组指针表示数据所在内存的位置，x[i] 表示读取第 i 项。

算法步骤：

1. 计算线程对应的全局元素编号。
2. 只对合法编号读取、相加和写回。

```
__global__ void add(const float* x, const float* y,
                    float* out, int n) {
    // 步骤1：计算线程对应的全局元素编号
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    // 步骤2：只对合法编号读取、相加和写回
    if (i < n) out[i] = x[i] + y[i];
}
// 输入和输出已在设备上分配，n=1000
add<<<(n + 255) / 256, 256>>>(x, y, out, n);
```

blockIdx 是块编号，blockDim 是每块线程数，threadIdx 是块内编号。这个表达式把块内局部位置换成整个向量中的位置。尖括号提供启动规模；函数返回到 CPU 不代表 GPU 已经完成。代码是解释执行分工的片段，省略了分配、复制、错误检查和等待。

Warp 与分支为什么影响效率

在 NVIDIA **CUDA** 模型中，线程块内每 32 个线程形成一个 warp，**线程束**。硬件以相关方式执行这些线程。若同一 warp 的线程走不同分支，部分线程在某条路径上不活跃，执行资源利用可能下降，这称为分支分歧。分支不一定错误，也不必全部消除，应看其发生比例和路径成本。

线程块通常被分配给一个 SM（Streaming Multiprocessor，流式多处理器）。SM 有有限的寄存器、**共享内存**和可容纳线程。更大的线程块不是必然更快，过大的资源需求可能使同时驻留的块数减少。

Occupancy，占用率通常描述实际驻留 warp 数相对硬件允许上限的比例。更多驻留 warp 可以在一组等待数据时执行另一组，但提高该比例也可能降低每个线程可用资源。它是解释性能的指标，不是直接优化目标。

协作需要可见性与顺序

共享内存让块内线程共同使用一份数据，但“大家都能访问”不等于“数据已被写好”。同步使依赖满足后再读取。块级屏障要求尚未退出的线程按一致的控制流参与，不能让部分仍在执行的线程绕过屏障，而让其他线程在同一屏障等待。已经退出的线程不再属于等待对象；但如果它们本来负责加载其他线程要用的共享数据，提前退出仍会破坏计算。下面的分块矩阵乘法因此保留边界线程，使无效输入填零，并继续完成协作加载与同步。线程块之间一般也不能依赖任意执行顺序。正确性优先于减少等待，否则可能只在部分输入上偶然正确。

11.6.2 访存合并、共享内存与矩阵分块

连续数据为什么比较容易读取

访存合并是把一组线程的相关内存访问组织成较高效的传输。若相邻线程读取连续的四字节浮点数，通常能较充分利用传输块；若每个线程跳过很多元素，实际传来的字节中可能只有少量被使用。具体交易数量还取决于对齐、缓存和架构，不应把连续访问固定理解成任何情况下都只进行一次传输。

行优先矩阵把同一行的元素连续保存。若线程按列号读取一行，访问较连续；若按行号读取同一列，步长为整行宽度。相同矩阵公式，布局和线程映射不同，性能可以差很多。转置视图并不一定真的搬动数据，但后续算子可能因非连续布局额外整理数据。

分块是在提高数据复用

矩阵乘法的一项输出 C[i,j] 要读取 A 的第 i 行和 B 的第 j 列。若每个线程各自反复从主存读这些数，相邻输出会重复读共同输入。**Tiling，分块**，让一个线程块协作读一小块 A 和 B 到片上，再生成多项输出。

一个教学 4×4 矩阵乘法，只考虑结果左上角 2×2 子块。直接计算四项输出，每项读四个 A 和四个 B，共 32 次输入元素读取。按 K 维分成两段：每段协作读 A 的 2×2 块和 B 的 2×2 块，共八项，两段合计 16 项，四项结果共享这些读取。这里只比较一种理想访问安排，不是完整 GPU 性能模型。

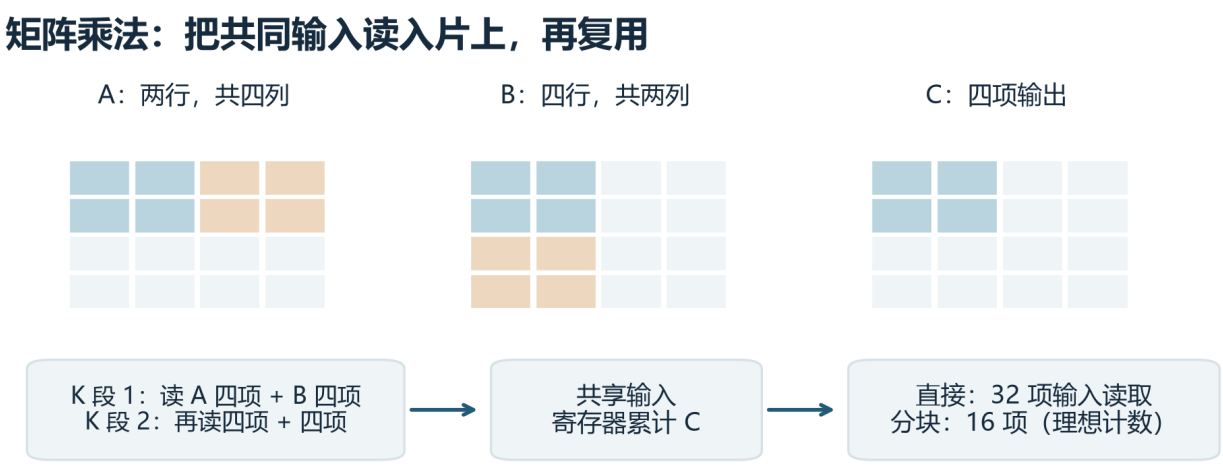


图11.8：计算 C 的 2×2 子块，两段 K 方向输入依次进入共享区域，累计结果保存在寄存器。

为什么块不能无限增大

块增大可以增加复用，却会占用更多共享内存与寄存器，减少并发，也可能增加边界浪费。寄存器不足时，部分内容可能溢出到更慢的存储，称为 spill。共享内存内部的 bank，存储分组，如果同时访问不同地址却集中到相同分组，可能发生冲突；同一地址的广播是另一种情况，不能简单把地址相同都看作冲突。

高性能矩阵乘法还要安排加载、矩阵计算、同步与下一块预取。Tensor Core 实现使用硬件支持的矩阵指令；**CUTLASS**（CUDA Templates for Linear Algebra Subroutines）提供 CUDA 线性代数运算的实现组件，包括 C++ 模板和 CuTe DSL 接口。这里 DSL 指**领域专用语言**（Domain-Specific Language），即面向特定计算领域设计的语言。工程实践通常先使用成熟库，再为真实瓶颈改进布局或特定形状，避免用教学 kernel 的表现推断库性能。CUDA 最佳实践、Triton 矩阵乘法教程

量化算子的额外工作

四位权重通常打包保存，一个字节容纳两个四位编码。kernel 需要取出编码、读尺度并按所需方式计算。能否在片上解包并立即消费，决定是否保留读取优势。若每次推理把全部权重展开成一个新的高精度大矩阵，压缩存储与运行时节省就可能分离。打包布局也是引擎与算子之间的协议，不能随意替换。

11.6.3 手写分块矩阵乘法：共享数据与两次同步

下面实现行优先矩阵 $C = AB$，A 为 $M \times K$，B 为 $K \times N$。每个线程块负责 $T \times T$ 个输出，每个线程维护一项累加值。沿 K 维逐段加载共享块，块内复用后再加载下一段。

```
template<int T>
__global__ void matmul_tiled(const float* A, const float* B,
                             float* C, int M, int N, int K) {
    __shared__ float a_tile[T][T];
    __shared__ float b_tile[T][T];
    int tx = threadIdx.x, ty = threadIdx.y;
    int row = blockIdx.y * T + ty;
    int col = blockIdx.x * T + tx;
    float total = 0.0f;

    for (int start = 0; start < K; start += T) {
        // 1. 协作读取当前两块；超出矩阵的部分按乘法零贡献填充。
        int ak = start + tx, bk = start + ty;
        a_tile[ty][tx] = (row < M && ak < K) ? A[row*K + ak] : 0.0f;
        b_tile[ty][tx] = (bk < K && col < N) ? B[bk*N + col] : 0.0f;
        __syncthreads();  // 2. 等所有线程写好，才能读取其他线程的元素。

        // 3. 一份加载服务多项输出，结果累积在本线程中。
        for (int k = 0; k < T; ++k)
            total += a_tile[ty][k] * b_tile[k][tx];
        __syncthreads();  // 4. 等大家读完，才允许下一轮覆盖共享块。
    }
    if (row < M && col < N) C[row*N + col] = total;
```

```
}
// A、B、C 是已分配的设备内存；启动每块 16×16 个线程。
// matmul_tiled<16><<<dim3((N+15)/16, (M+15)/16), dim3(16,16)>>>(A,B,C,M,N,K);
```

第一次屏障保证“数据已写完”，第二次保证“旧数据已读完”。边界线程也必须参与两次屏障，所以它们没有在函数开头因越界而直接返回。无效输入填零，合法输出才写回，使非整块矩阵也遵守相同运算。

相邻横向线程在加载时读取同一行的连续元素，有利于合并访存。计算时，A 的同一行被多个输出列使用，B 的同一列被多个输出行使用。分块把这些复用从重复主存访问移到共享内存。更大的块增加复用，也增加共享内存、寄存器和边界浪费；实际实现还会采用预取、矩阵指令及更细的线程分工。CUDA 的共享内存矩阵乘法教程、Triton 矩阵乘法教程

这是理解映射、复用与同步的教学内核。它没有使用 Tensor Core，也没有做成熟矩阵库中的多级优化；比较部署方案时，应以相同运算的成熟库为性能参照。

11.6.4 Triton 中的分块索引

每个程序块处理一段连续元素。有效掩码同时用于读取和写回，处理最后一块不足整块的情况。

算法步骤：

1. 为程序块中的每个位置分配元素索引。
2. 标出仍在数组内的位置。
3. 只读取有效元素。
4. 相加并只写回有效位置。

```
import triton
import triton.language as tl

@triton.jit
def add_kernel(x, y, out, n, BLOCK: tl.constexpr):
    # 步骤1：为程序块中的每个位置分配元素索引
    i = tl.program_id(0) * BLOCK + tl.arange(0, BLOCK)
    # 步骤2：标出仍在数组内的位置
    valid = i < n
    # 步骤3：只读取有效元素
    a = tl.load(x + i, mask=valid, other=0)
    b = tl.load(y + i, mask=valid, other=0)
    # 步骤4：相加并只写回有效位置
    tl.store(out + i, a + b, mask=valid)

# out 与输入形状、设备和类型匹配，连续存储
# add_kernel[(triton.cdiv(n, 256),)](x, y, out, n, BLOCK=256)
```

11.6.5 练习题

题11.6-1 CUDA 相邻线程访问连续 float，与以很大固定步长访问有什么差别？为什么一个元素一个线程仍要检查 i<n?

11.6.6 参考答案

题11.6-1答案 GPU 通常把同一个线程束（warp）中线程的访存请求合并成较少的内存事务。以 32 个线程各读一个 float、地址按顺序且适当对齐为例，合计需要 128 字节；在常见采用 32 字节事务粒度的架构上，可由四个事务服务。若各线程地址隔得很远，它们可能落在 32 个不同事务中；每次取一段，却只使用其中一个 float，带宽大量浪费。具体代价还受对齐和缓存影响，但“每个线程读一样多数据”不代表总访存成本一样。

“一个元素一个线程”是索引分配方式，并不保证发射出的线程数恰等于元素数。常用块大小为 256，处理 $n = 1000$ 个元素时需 $\lceil 1000/256 \rceil = 4$ 个块，共 1024 个线程。最后 24 个线程的索引已越界，所以先计算 $i = \mathrm{blockIdx.x} \times \mathrm{blockDim.x} + \mathrm{threadIdx.x}$，只有 $i < n$ 才读写数组。掩码版本也是同样目的：允许整齐分块发射，同时保护尾部无效索引。

11.7 归约、融合、注意力与自定义算子

11.7.1 归约、Softmax 与归一化为什么适合融合

归约把多个元素变成少量统计

Reduction，归约，把一组数合成为和、最大值或其他统计。把 1 到 8 求和，可以先得到 [3,7,11,15]，再得到 [10,26]，最后得到 36。树形协作减少串行依赖长度，但需要交换结果、满足同步；浮点加法的顺序变化还可能改变舍入结果。

Softmax把分数变成和为一的正权重

$$p_i = \frac{\exp(z_i - m)}{\sum_j \exp(z_j - m)}, \qquad m = \max_j z_j.$$

所有分数减去共同最大值，不改变最后比例，却使指数输入不大于零，降低溢出风险。重点不是再学习指数，而是：这个操作同时需要行内最大值、求和和逐项变换，因此不同元素之间存在依赖。

融合减少什么

若最大值、减法、指数、求和和除法各作为独立程序执行，中间张量可能反复写回主存。一个融合 kernel 可以读入一行，完成统计和变换，再写结果。它节省中间读写及多次启动，数学目标仍是同一个归一化。Triton 融合 Softmax 教程

若一行太长，不能方便地保留在片上，就需要更复杂分块或多次扫描。还要正确处理无效位置：带掩码的**注意力**通常使无效分数不参与归一化；若全部位置都无效，结果定义必须明确，否则可能产生非有限数值。

LayerNorm 与 RMSNorm 的作用

LayerNorm 按约定特征维计算均值和方差，得到标准化特征，再用训练得到的 γ、β 缩放和偏移。它不是一个预测标签的新网络，而是主模型内的运算，其参数由任务损失训练。以 [1,3] 为教学输入，均值为 2、方差为 1，忽略稳定小量且 γ=1、β=0 时，输出为 [-1,1]。

RMSNorm 用均方根尺度归一化，通常不减均值。两者处理的是不同计算，不能仅因某个 kernel 更快就把原模型的一种替换成另一种而称等价。求和、平方和与小方差可能敏感，常需要适当的累计精度。融合必须保留输入维度、稳定项和输出类型的实际定义。

对 VLA 的意义

这些算子可能没有最大计算量，却可能频繁启动、搬运中间结果，尤其在较小批次下明显。先看时间占比和设备时间轴，再决定是否优化。正确性测试要覆盖不同长度、非整块边界、大数、接近常数的输入和掩码，而非只测试一组随机数。

11.7.2 注意力与 FlashAttention 为什么能少搬数据

注意力是在按相关性汇总信息

Transformer 把输入投影成查询 Q、键 K、值 V：查询描述当前元素需要什么，键用于计算匹配程度，值提供被汇总的信息。这里的 Q、V 是张量，不是强化学习里的**动作价值**或**状态价值**。

常见缩放点积注意力为

$$O = \mathrm{softmax}(QK^\top/\sqrt{d} + M)V.$$

d 是键特征维度，缩放避免点积分数随维度变化而过大；M 是掩码等加性限制；每个查询先与允许的键比较，形成权重，再对值加权。模型参数学习产生有用的查询、键和值投影，注意力不是人工固定的目标检测规则。

普通实现可能保存一个 N×N 的分数或概率矩阵。序列长时，这个中间矩阵大，需要多次读写。FlashAttention 通过分块和在线统计减少主存与片上之间的传输，在数学上仍计算原注意力；浮点运算顺序变化意味着不保证逐位相同。它的主要思想是改变执行和数据流，不是删掉低分 Token。
FlashAttention 原始论文

不保存完整概率矩阵，怎么保持归一化

对一行查询，分批处理键和值，维护三个统计：已见最大分数 m，归一化分母 l，未除分母的加权和 u。新块分数为 z_j、值为 v_j 时

$$\begin{aligned} m' &= \max(m, \max_j z_j), \\ l' &= e^{m-m'}l + \sum_j e^{z_j - m'}, \\ u' &= e^{m-m'}u + \sum_j e^{z_j - m'}v_j. \end{aligned}$$

初始化可用 m 为负无穷、l=0、u=0，并按数值安全规则实现第一块。结束后 O=u/l。最大值变大时，旧统计需乘 e 的修正因子，使新旧数据使用同一归一化基准。这正是“分块后仍保持全局 **softmax**”的关键。

教学分数为 [0,ln2,ln3]，值为 [2,4,6]，未归一化权重比例是 [1,2,3]，最终结果为 28/6≈4.667。先处理前两项：m=ln2、l=1.5、u=5；第三项让 m'=ln3，旧统计乘 2/3，再加入新项，得到 l'=2、u'=28/3，结果仍为 14/3。这个算例是单行在线统计，不是完整 **GPU** kernel。

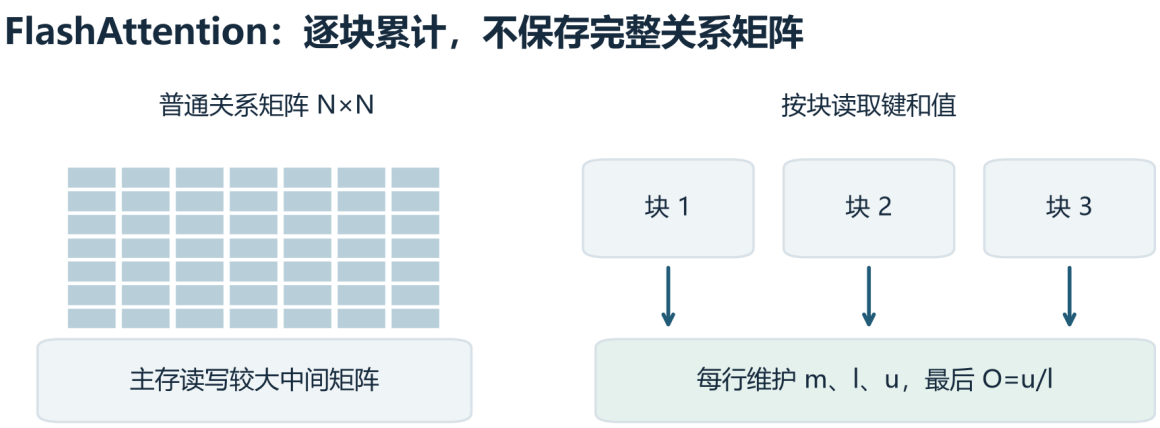


图11.9：右侧保存较小输入块和统计，避免把全部关系矩阵反复写回主存。

什么条件影响收益

序列长度、头维度、批次、掩码与设备支持影响性能。序列很短时，启动和其他模块可能更显著；不支持的掩码或数值类型可能使框架采用其他路径。检查实际调用，而不是仅检查安装了相关包。

视觉 Token 裁剪与 FlashAttention 可以组合：前者改变参与计算的信息，可能引入模型行为误差；后者主要改变执行方式。公平比较要区分两类贡献，并使用相同输入长度下的成熟注意力实现作为基线。

11.7.3 Triton、CUTLASS 与自定义算子怎样进入模型

三种工具处于什么位置

CUDA C++ 让程序员直接组织线程和内存；Triton 编程语言提供接近 Python 的块级表达，由编译器组织 GPU 执行；**CUTLASS** 是构建高性能矩阵等运算的实现组件库。它们都需要理解数据布局和资源条件，表达层次不同。NVIDIA Triton Inference Server 是推理服务软件，与 Triton 编程语言不是同一项目。Triton 文档、CUTLASS 官方项目

Triton 的一个 program 通常处理一块数据。program_id 给出当前块编号，arange 生成块内位置，mask 防止越界，load 与 store 读写数据。这些概念可与第11.6节的分工联系起来，但不能机械把每个 program 等同于某个固定 CUDA 线程布局，具体映射由实现和编译器决定。第11.6节的分块索引示例给出最小片段。

从 kernel 到框架算子

先定义数值语义、输入输出形状和类型；实现 kernel；在框架中注册算子接口；处理形状推断与内存布局；若需要训练，再提供正确的反向规则；若需要导出与 TensorRT 执行，还要补齐导出表示和插件适配。只在 Python 里能调用自定义函数，不代表 **ONNX** 或目标引擎认识它。

TensorRT 插件需要提供形状与类型信息、支持的格式、构建或序列化信息和运行时执行方法。算子依赖 stream，不能随意在内部全设备同步，从而破坏调用方的异步安排。当前版本插件接口与历史示例可能不同，应按所用版本实现。TensorRT 自定义层

怎样证明值得维护

比较成熟框架或库的相同运算，而不是刻意拆成多个低效 Python 操作作为唯一基线。固定输入形状、布局、精度和同步口径，测试多种代表性尺寸及边界。先检查数值，再测 kernel，最后替换进模型测端到端。若局部快两倍却只节省总时间的 1%，要与维护成本和适配需求一起评价。

一段基础向量相加适合学习，不意味着实际项目需要重新实现所有通用算子。岗位中“算子开发”的合理目标，是解决已有路径的缺失或真实瓶颈，并给出可重复的正确性和性能证据。

11.7.4 FlashAttention 的反向为什么可以重新算出注意力权重

前向没有保存完整$N \times N$概率矩阵，反向又需要这些权重，怎么办？先看不含注意力Dropout、每行至少有一个合法键的情况。令$S = QK^\top/\sqrt{d} + M$，M与前向使用同一掩码。前向保存行最大值m_i和指数和ℓ_i，于是$L_i = m_i + \log \ell_i$就是该行的log-sum-exp。反向按块重算分数并重新应用M，就能恢复：

$$P_{ij} = \exp(S_{ij} - L_i).$$

被屏蔽的位置取$P_{ij} = 0$；若允许整行遮蔽，需沿用前向约定单独处理。训练含注意力Dropout时，还要恢复同一随机掩码并应用对应缩放，下面推导不含Dropout的基本形式。Q、K与行统计足够按块重建P，不必永久保存全部P。反向用更多计算，换取较少大矩阵的保存与读取。它和第10章激活重计算具有相同取舍，计算组织则针对注意力的特殊结构。Triton 融合注意力的前向与反向教程

记损失传来的输出梯度为 $G = \partial \mathcal{L}/\partial O$。从 $O = PV$ 可得 $\partial \mathcal{L}/\partial V = P^\top G$、$\partial \mathcal{L}/\partial P = GV^\top$；再经过 softmax，有

$$\frac{\partial \mathcal{L}}{\partial S_{ij}} = P_{ij}\left((GV^\top)_{ij} - \sum_k P_{ik}(GV^\top)_{ik}\right).$$

括号内减去的是该行按 P 加权的平均贡献，反映 softmax 分数彼此竞争：提高一个位置的权重，会相应减少别处权重。掩码固定后，再沿缩放点积计算Q、K的梯度。分块实现按块重建所需概率并累计这些梯度，而不把全矩阵一次展开。FlashAttention 的反向算法

这使 FlashAttention 的学习价值更明确：数学上保留所需注意力关系，工程上重新安排中间量的生命周期。裁剪 token 改变哪些信息参与计算，FlashAttention 则改变这些信息怎样被计算与保存。

11.7.5 在线 softmax 的分块更新

当前块可能提高本行最大分数。旧分母和旧加权和都要换到新尺度，再与当前块相加，才能保持与完整 softmax 同一结果。

算法步骤：

1. 合并旧最大分数与当前块最大分数。
2. 计算把旧累计量换到新尺度的系数。
3. 计算当前块在新尺度下的指数权重。
4. 同步更新加权和、分母及最大值。

```
# 步骤1：合并旧最大分数与当前块最大分数
new_max = np.maximum(maximum, scores.max(axis=1))
safe_max = np.where(np.isfinite(new_max), new_max, 0)
# 步骤2：计算把旧累计量换到新尺度的系数
correction = np.exp(maximum - safe_max)
# 步骤3：计算当前块在新尺度下的指数权重
p = np.exp(scores - safe_max[:, None])
# 步骤4：同步更新加权和、分母及最大值
numerator = correction[:, None]*numerator + p @ value_block
normalizer = correction*normalizer + p.sum(axis=1)
maximum = new_max
```

11.7.6 从 FlashAttention 原图到在线归一化的手算

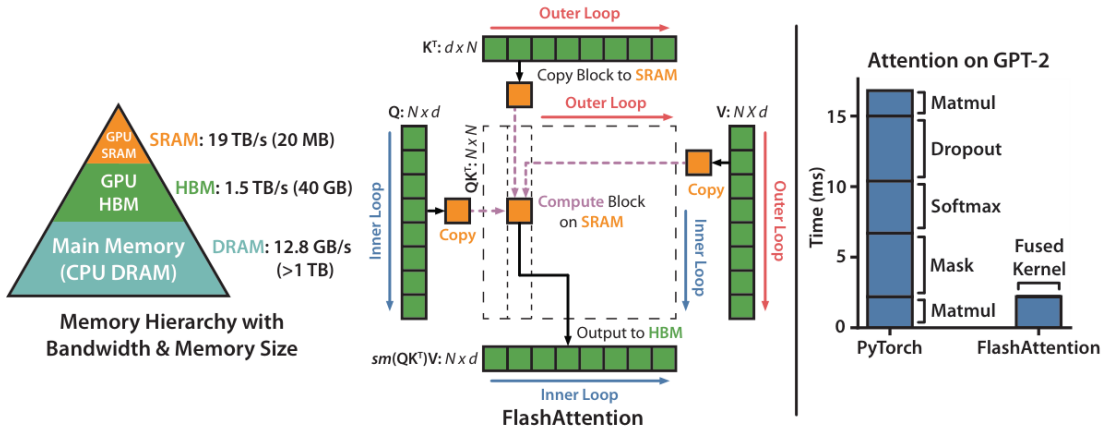


图11.10：Dao 等，FlashAttention 论文 Figure 1。左侧是存储层次，中间为分块注意力，右侧是作者一次实验中的时间比较。原论文。

左图的 SRAM（Static Random-Access Memory，静态随机存储器）在这里指 GPU 上更快但较小的片上存储资源，HBM（High-Bandwidth Memory，高带宽内存）是容量更大的 GPU 显存，DRAM（Dynamic Random-Access Memory，动态随机存储器）则用于图中主机内存。图中的带宽与容量对应作者当时的实验平台；核心关系是快而小的存储与较慢而大的存储之间，搬运中间结果可能比计算本身更昂贵。

中图每次搬入一块 Q、K、V，在片上计算匹配和内容累积。Inner Loop、Outer Loop 分别是内层、外层循环，sm 表示 softmax，虚线的大方框是避免完整写入显存的 $N \times N$ 分数或权重矩阵。右图的 Fused Kernel（融合核函数）把多项工作放进较少的 GPU 执行函数中，减少反复搬运与启动。算法仍汇总所有需要读取的位置，没有把原本可读的信息删掉。

难点是所有键没有一次放进来，如何仍用同一个 softmax 分母。先看两个块：第一块只有分数0和值2，保存最大分数 $m = 0$、指数和 $\ell = 1$、未归一化内容和 $u = 2$。第二块只有分数 $\log 3$ 和值6，新最大分数为 $m' = \log 3$。旧量换到新尺度需乘 $e^{m-m'} = 1/3$，所以

$$\ell' = \frac{1}{3} \times 1 + 1 = \frac{4}{3}, \qquad u' = \frac{1}{3} \times 2 + 1 \times 6 = \frac{20}{3}, \qquad O = \frac{u'}{\ell'} = 5.$$

结果与一次性 softmax 的权重 $(1/4, 3/4)$ 完全对应。这里保存的是未归一化加权和，不能先把每块分别归一化再直接平均；最大分数改变时，分子分母必须一起重缩放。第1章注意力的数学目标与本节分块的数值实现由此接通。Triton 融合注意力教程

11.7.7 练习题

题11.7-1 在线 softmax 处理一个新分块时，为何既要更新最大值，又要重新缩放旧分母和旧加权和？它怎样支持 FlashAttention 的精确注意力？

11.7.8 参考答案

题11.7-1答案 对一个查询，记已处理分数的最大值为 m，分母为 $l = \sum_j e^{s_j - m}$，未归一化加权和为 $r = \sum_j e^{s_j - m} v_j$。输出是 r/l。减去最大值不改变 softmax 比例，却避免指数过大。

加入新分块 B 后，最大值变为 $m' = \max(m, \max_{j \in B} s_j)$。旧项原先按 m 计算，如今必须统一按 m' 表示，所以用 $\alpha = e^{m - m'}$ 同时缩放旧分母和旧加权和：

$$l' = \alpha l + \sum_{j \in B} e^{s_j - m'}, \qquad r' = \alpha r + \sum_{j \in B} e^{s_j - m'} v_j.$$

这来自恒等式 $e^{s_j - m'} = e^{s_j - m} e^{m - m'}$。只更新最大值而不缩放旧项，会混用两个不同基准，改变新旧 token 的相对权重。

例如先读 $s_1 = 0, v_1 = 2$，有 $m = 0, l = 1, r = 2$；再读 $s_2 = \log 2, v_2 = 4$，则 $m' = \log 2, \alpha = 1/2, l' = 1.5, r' = 5$，输出 $10/3$，与直接计算权重 $[1/3, 2/3]$ 相同。

FlashAttention 用这一递推让查询分块依次处理键值分块，只保留最大值、分母和加权和，不把整张注意力分数及权重矩阵写到大显存。它仍考虑全部允许的键，因此在相同掩码下计算标准注意力；“精确”指没有改成稀疏或低秩近似，浮点舍入差仍可能存在。

11.8 异步调度、观测时效与稳定运行

11.8.1 Stream、Event 与 CUDA Graph 怎样减少等待

顺序依赖与可以重叠的工作

Stream，执行流，是 **GPU** 工作的有序队列。同一流的工作按其规则保持顺序，不同流的工作在依赖满足和硬件资源允许时可以重叠。**Event，事件**，可以记录进度、表达等待关系和用于设备计时。不同流不是互相无关，读某张量之前仍需确保写入已完成。CUDA 异步执行

教学有三个独立请求，每个传输 20 ms、计算 60 ms。完全串行耗时 240 ms；若传输与计算可充分重叠，总完工时间可为 20+3×60=200 ms。第一项结果仍在 80 ms 出现，流水线主要提高连续处理效率，并未自动降低首项延迟。机器人相机帧若持续排队，还要检查是否处理了过时输入。

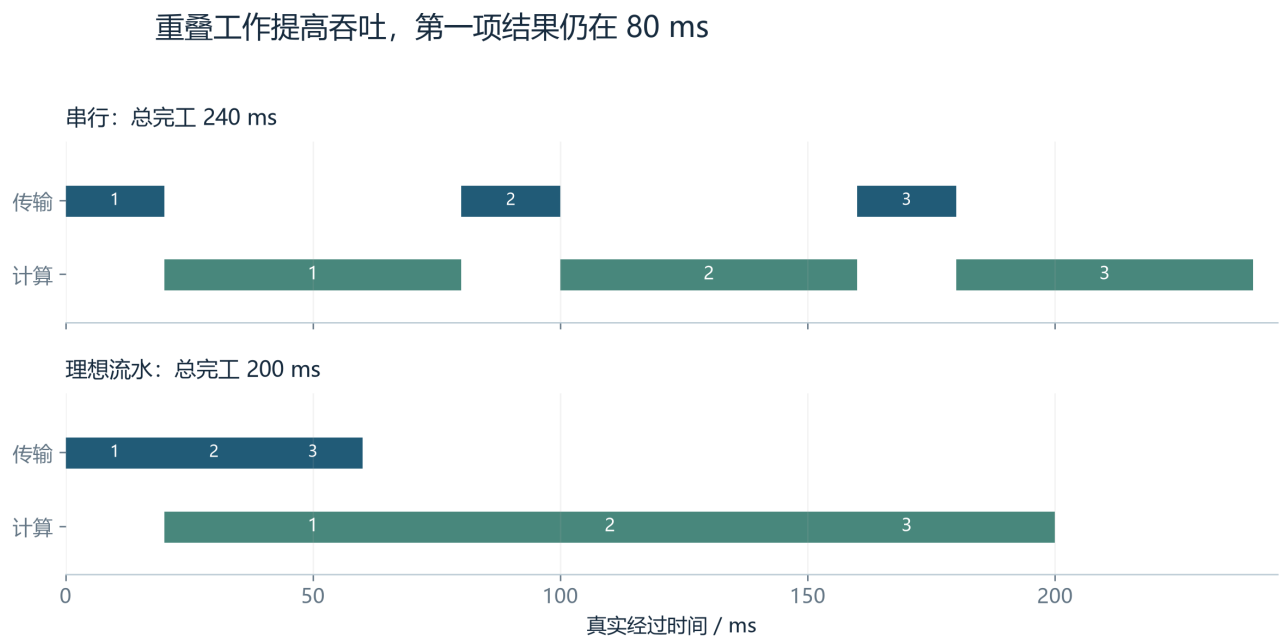


图11.11：理想重叠条件下的教学时间轴，传输依赖用事件表达；真实设备还可能存在资源争用。

内存传输的条件

pinned memory，锁页内存，帮助设备异步传输满足有关主机内存条件；non-blocking 请求表达希望异步，不保证所有数据和设备路径都能并行。源缓冲区在读取完成前不能被相机程序改写，目标缓冲区也不能提前供模型使用。

零拷贝指特定边界不需要再复制数据，并不意味着没有存储访问或同步。GPU 直接访问较慢的主机内存，可能省下一次复制，却让多次读取更慢。集成设备的共享物理内存也不自动消除格式转换与缓存一致性成本。应说明省掉了哪次复制，保留了哪些访问。

CUDA Graph 复用提交过程

CUDA Graph 预先记录一组 GPU 操作及依赖，之后整体重复提交，减少逐次启动和 **CPU** 调度开销。它与模型计算图不同：模型图描述网络运算，CUDA Graph 描述具体执行工作及依赖。适合重复、形状和内存地址等条件可管理的执行过程。CUDA Graph 官方说明

如果实际瓶颈是大型矩阵计算，减少提交开销的收益可能有限；若很多小 kernel 之间存在明显 CPU 空隙，收益可能更大。动态形状和随机生成需要管理输入地址、状态和可更新节点，必要时针对少量形状分别准备执行图。重复提交不应意外重复旧输入或旧随机状态。

机器人系统中的取舍

把摄像头、预处理和模型放到独立工作线程可以减少互相阻塞，但要规定输入时间戳、队列上限和数据所有权。一个多线程系统若无限积压旧帧，吞吐提高而观测变旧。第11.8节会用信息年龄解释这个问题。

11.8.2 动作块、观测年龄与机器人实时性

预测多步与执行多步不是同一决定

动作块长度 H 表示一次预测多少步；执行前缀长度 E 表示在重新使用新结果前执行多少步。H=16 不代表必须连续执行全部 16 步。执行较长前缀减少模型请求频率，却降低对新情况的响应；执行较短前缀更及时，可能增加计算和块之间衔接压力。

策略更新频率、底层控制频率、模型内部生成步数分别属于不同时间或计数。它们可以联系，但不能混为“模型每秒生成多少步”。动作块中的步长还必须与控制器采用的真实时间单位一致。

观测年龄比纯推理时间更贴近响应

定义一个实际执行动作所依赖观测的年龄

$$\mathrm{Age} = t_{\mathrm{execute}} - t_{\mathrm{capture}}.$$

其中采集时间是相应图像或状态的真实采集时间，执行时间是动作实际开始执行的时间。它包括预处理、排队、模型计算、传输和执行等待。一个模型只算 40 ms，但输入已在队列里等待 200 ms，最终行为仍基于过时世界。

不同传感器还有时间对齐问题：相机图像来自 t，关节状态来自 t+50 ms，如果直接拼接，可能构造出真实世界从未出现过的组合。应记录时间戳，根据协议对齐、插值或丢弃过期数据。队列限制和新帧替换策略需要兼顾模型是否必须保留历史；并非所有模型都能任意只取最后一帧。

异步生成怎样与已执行动作衔接

教学底层每 10 ms 执行一步，模型生成用时 80 ms。新块算好时，旧队列已经执行约八步。直接把新块从原来观测的第一步开始执行，会忽略这八步造成的状态变化。新块需要按真实执行时间对齐，并处理在计算期间已承诺执行的动作。

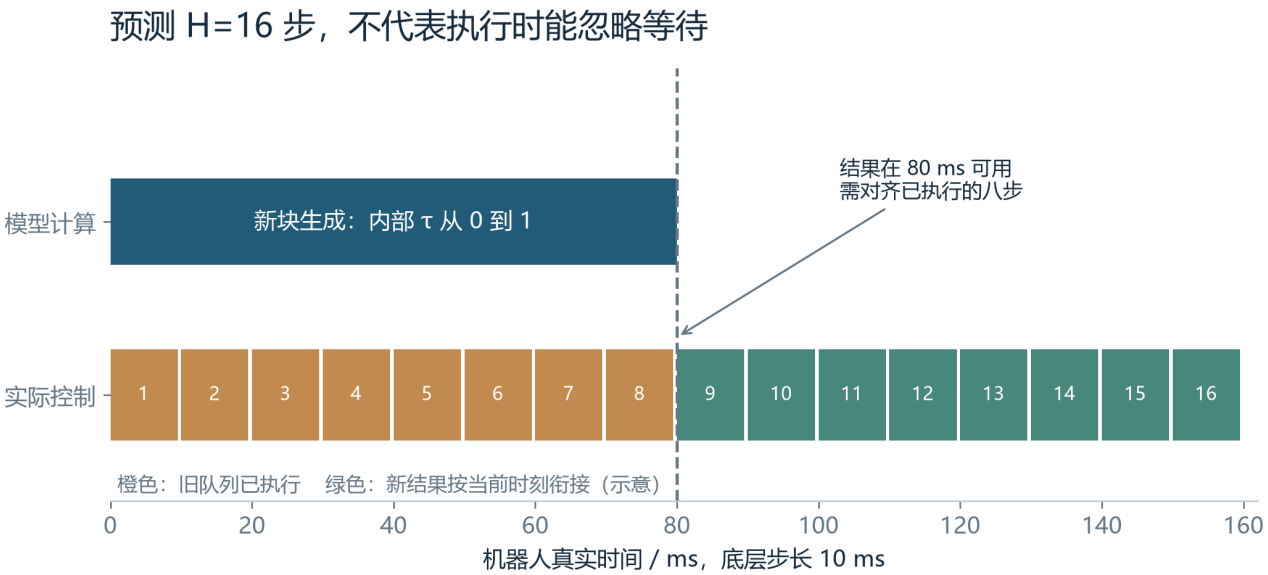


图11.12：内部生成发生于 80 ms 等待期间，机器人已执行旧队列的八个控制步；H=16 仅是本案例的预测长度。

Real-Time Chunking 研究通过在生成中处理已承诺前缀等条件，改善动作块的异步连续执行。这不是简单多开一个线程；关键在新生成过程如何与已经在执行的动作一致。部署能知道已发送或预计将执行的命令，却不知道未来真实环境响应，这条信息边界必须保持。实时动作块执行原始论文

软实时与硬实时

软实时系统允许少量超时但应控制后果；硬实时要求在明确条件下保证截止时间。测得 P99 小于预算不能证明硬实时保证，更没有覆盖驱动、系统调度和异常负载的全部情况。

机器人应规定动作队列不足、观测超时或输出无效时的行为，例如使用经过验证的保持、减速或停止策略。具体行为由本体控制接口和风险约束决定。冻结旧动作并无限等待，不一定安全。高层 **VLA** 的性能优化与底层控制的确定性需要分别验证。

11.8.3 从实验脚本到稳定运行的系统

部署包需要包含协议

交付应包含模型或引擎、必要插件、预处理和后处理版本、输入输出字段、支持形状、随机与缓存规则、设备和依赖要求、质量与性能回归结果。manifest，清单文件，记录这些关联关系，避免更新权重时保留了不匹配的动作缩放。

容器可以固定软件环境，但不能消除硬件、驱动、功耗与外设差异。引擎是否可移植还要查实际兼容条件。构建或加载模型不应未经测量混入控制线程，启动时可做检查与必要预热。

内存、队列与状态的所有权

谁写入相机缓冲区、谁读取模型输入、什么时候可复用、哪个线程消费动作队列，都要明确。双缓冲是轮流使用两组缓冲区，使一组读取时另一组写入；它仍需同步防止写入尚未完成或读取尚未结束。

多模型共用 GPU 时，要看优先级、排队时间与峰值总内存。单独运行的 VLA 达标，不表示同时运行语音、感知和规划后仍达标。ROS2 是机器人通信与系统组织工具，消息质量策略和执行器配置影响数据与回调行为，它不是模型加速引擎，也不自动给出硬实时保证。ROS2 官方消息质量文档源

为什么岗位常要求 C++、Linux 与 Git

Python 便于模型实验，C++ 常用于连接推理库、设备接口、内存和并发执行。需要掌握指针与数据布局、资源生命周期、线程和构建依赖；将程序改写成 C++ 不会自动缩短本来由 GPU 矩阵运算主导的时间，应先确认 CPU 开销。

Linux 是许多机器人计算平台的操作系统环境，部署工作涉及驱动与库版本、进程、日志、设备权限和资源调度。Git 保存代码版本与变更，模型参数和数据还要有相应版本记录。它们服务于兼容、调试和可复现，不是额外的模型算法。学习时可以随着第11.10节的实际流程逐项掌握，避免把所有系统知识当作开始**量化**学习的前置条件。

日志怎样支持失败归因

记录输入时间戳、任务阶段、模型版本、计算配置、缓存命中、生成步、关键执行事件、动作时间及异常。不要只记录“模型推理失败”。输出非有限数值、缓存使用了旧条件、状态没有对齐、GPU 被其他模型占用，可能产生相似外部症状，却需要不同修复。

回放应允许相同输入、参数和可控随机状态下重现，必要时比较中间结果。线上监控关注尾部延迟、队列深度、内存、温度与任务失败分布；性能回归测试用于发现更新后变慢或资源增大，而不是仅检查程序是否能启动。

模型更新与回退

OTA（Over-the-Air Update，无线远程更新）是通过网络更新设备软件或模型；灰度发布是先在有限设备或任务上运行新版本，检查结果后扩大范围。更新需保持模型、引擎和协议匹配，可先完整准备再切换，并保留经过验证的回退版本。缓存与执行状态也要按新模型规则处理。

工程稳定性有可检验目标：持续负载下是否泄漏内存，异常输入是否被正确处理，队列是否有界，超时行为是否符合协议。它与模型研究相互连接，不能用“实验室成功过一次”代替。

11.8.4 延迟与动作年龄分别测量

推理耗时是从开始到完成，动作年龄是执行时距其依据的观测采样已经过去多久。队列堆积、相机缓存和并发调度可能使模型推理很快而动作很旧。应为观测、生成、传输和执行分别记录时间，并测量中位数、尾部和最坏相关事件。

CUDA Graph 减少重复提交开销，通常要求符合捕获与重放条件；异步流需要正确事件依赖与缓冲生命周期。若下一帧覆写了正在计算的输入，速度提升伴随数据竞争。工程优化应在长期运行、温度变化和接触阶段下评价稳定性。

11.8.5 过期动作前缀与轨迹衔接，为什么要一起处理

假设动作块以观测时刻 t_0 为起点，覆盖未来若干控制间隔，生成完毕时已经到了 $t_0 + 3\Delta t$。前面三个动作所对应的执行时刻已经过去，不能将整块当作刚从 t_0 开始生成那样执行；队列需要按计划执行时间选择后缀，而不是按数组第一个元素开始。

但删去三项还不够。机器人在等待期间已经执行了旧策略块，它现在的状态可能与新块原先假设的状态不同。剩余第一个动作若依赖那三个未实际执行的动作，也可能跳变。更完整的衔接要固定已经承诺的前缀，在当前执行状态附近生成或修正后缀，并检查位置、速度和控制约束。延迟补偿、观测更新和动作块内部协调，解决的是同一次异步执行中的三个相连问题。

11.8.6 练习题

题11.8-1 异步动作块推理时，新块前几步在结果到达前已经过去。直接从新块第一个动作开始执行会有什么问题？怎样组织衔接？

11.8.7 参考答案

题11.8-1答案 新块第一步对应的计划时刻可能已过去。直接从第一步执行，会把过期指令当作当前指令，造成重复运动、回拉或速度跳变。每个块应保存观测采集时刻 t_{obs}、计划起点 t_{plan}、控制周期 Δt 及结果可用时刻 t_{avail}；观测年龄和动作计划年龄不是同一个量。

假设第 j 项计划在 $t_{plan} + j\Delta t$ 的固定控制时隙下发，结果到达后可用于下一个时隙，则第一个可用索引为

$$j_0 = \max\left(0, \left\lceil \frac{t_{avail} - t_{plan}}{\Delta t} \right\rceil\right).$$

例如计划从 100 ms 开始，每 20 ms 一项，结果在 155 ms 可用，则 $j_0 = 3$，最早对应 160 ms；索引 0、1、2 已过期。实际控制器还要预留提交时间，按真实截止时间选后缀；整个块过期时不能继续取索引。与此同时，机器人在推理期间继续执行旧块已经承诺的动作。

删过期前缀只解决时间对齐。新块剩余动作可能假设了另一条前缀，因此还要检查它与当前位置、速度和接触状态是否一致。对支持条件补全的流或扩散生成器，可固定旧块将实际执行的前缀，再生成兼顾新观测的后缀；其他模型需要相应训练接口或重新规划，不能只覆盖几个数组元素就认为轨迹连续。结果过时、状态突变或旧块即将耗尽时，按预定安全流程保持或停下，并重新获取观测。用时延抖动实验检查块间连续性及闭环成功率。

进一步阅读：Physical Intelligence：Real-Time Action Chunking with Large Models。

11.9 Jetson、CPU、NPU 与模型编译器

11.9.1 Jetson、ARM CPU 与 NPU 的部署差异

Jetson 的核心约束

Jetson 将 NVIDIA **GPU** 与其他处理部件集成在设备上。部署需要同时考虑型号资源、功耗及 JetPack 软件栈，模型引擎与驱动、运行库、插件相互兼容。引擎的构建与加载范围由相应后端规定，跨设备部署应选择受支持的方式。

共享物理内存减少某些边界成本，也让 **CPU**、GPU 与其他部件共同争用带宽和容量。要测摄像头、感知、模型与控制程序同时运行的状态。最大内存、持续散热与长期功率条件，比一次空闲设备上的短测更接近真实部署。Jetson 功耗与性能文档，r36.4.4 版本示例

ARM CPU 与 NPU

ARM 是常见 CPU 架构家族，并非一种固定性能等级。CPU 推理需要考虑线程数、向量指令、内存带宽和算子实现。增加线程可能改善模型时间，也可能与传感器、控制或其他任务争用资源，因此应在整机负载下选线程配置。

NPU（Neural Processing Unit，神经网络处理器）面向神经网络执行，但产品之间的算子、动态形状与数值格式不同。GPU 访存和调度分析仍能帮助理解性能，具体算子则需对应工具链；例如昇腾使用 CANN 等组件组织编译与运行。昇腾文档。

LiteRT、MNN、**ONNX** Runtime 等提供不同端侧路线。选工具时先看目标硬件、模型结构、支持算子和工程条件，再看是否已有可维护的执行路径。TFLite 是一些岗位与旧资料常用名称，阅读移动端当前资料时也会遇到 LiteRT。LiteRT 官方入口、MNN 官方项目

功率与能耗怎样比较

功率是瞬时使用能量的速率，能耗是某段时间内累计用能。粗略稳定条件下 E≈平均功率×时间。教学配置甲为 15 W、100 ms，一次约 1.5 J；乙为 25 W、50 ms，一次约 1.25 J。功率更高仍可能单次用能更少，但持续运行能否散热、平均占空比怎样，还需要测量。

应说明测的是 GPU、计算板还是整台机器人，是否扣除空闲功率。任务级能耗还包括等待和运动执行，不能用模型单次能耗代替完整任务能耗。功耗模式是设备允许的配置，不等于每时刻实测功率。

跨平台的一条共同流程

先做兼容与输入输出验证，再选后端支持的精度与形状，定位真实瓶颈，调整内存和算子，测试持续负载与任务。跨硬件比较既可采用共同模型配置看平台差异，也可比较各平台充分调优后的部署结果；两种问题应分别报告，避免把工具适配程度误认为模型机制优劣。

11.9.2 模型编译器与分布式推理，学到哪里合适

编译器连接数学运算与机器实现

模型编译器把较高层网络运算转换成可执行实现。IR，Intermediate Representation，中间表示，是转换过程中表达运算、数据与依赖的形式。lowering，降低表示层次，是逐渐把“矩阵乘法”展开到循环、分块、内存与机器指令等更具体层次。

优化包括常量折叠、图融合、布局变换、内存规划和执行调度。schedule 在此指计算与数据访问怎样安排，不是机器人任务日程。两份程序可以执行相同公式，却采用不同分块与内存安排，因此性能不同。

TVM、MLIR、XLA 等经常出现在岗位要求中，但处于不同生态和抽象层。MLIR 是构建多层表示与转换的编译基础设施，不宜简单称为与 TensorRT 完全相同的推理引擎。torch.compile 是 PyTorch 的编译入口，也不是一键保证所有模型完全优化的承诺。TVM 文档、MLIR 项目、PyTorch 编译入口

动态程序为什么会重新编译

为特定形状、类型或控制条件选择实现，称为 specialization，特化。guards 是确认运行时仍满足这些条件的检查。条件改变可能选择另一缓存结果或触发新的编译；无法纳入图的片段可能产生图断裂并回到其他执行路径。

端侧输入如果经常改变长度和分支，平均理论算量低也可能出现编译或调度波动。可采用有限形状档位、固定关键路径或预先编译常见情况，但需要评估补齐 Token 和内存成本。

多机推理的扩展方向

模型很大时，可把层或矩阵切到多设备，分别对应流水或张量等并行方案。分割降低单设备容量压力，也引入通信、同步和设备间调度。若一个请求本来很小，通信可能超过节省的计算。云端系统还有多请求、网络和容错问题，与单机器人本地链路有不同重点。

当模型容量或请求规模需要多机时，通信、拓扑和调度会成为主要问题。掌握前面的张量程序、资源布局与时间线后，可以进一步学习多机网络与编译优化，并在对应硬件上测量。

学完以后应该能解释什么

分析部署方法时，依次说明改变了什么计算、减少了什么工作、引入了什么误差，再用数值、时间和闭环指标检验。训练数据适用于学习与恢复，图转换和调度则改变执行过程；两者需要不同验证。

11.9.3 硬件选择由工作负载和接口约束决定

端侧 GPU、CPU 和神经处理单元支持的算子、精度、内存与编译机制不同。理论算力常以**每秒万亿次运算**（Tera Operations Per Second，TOPS）表示，但需注明数值类型和计数口径；这一数值不代表完整 **VLA** 吞吐，部分算子回退到 CPU 可能增加数据搬运。先用代表性图像长度、动作生成过程和闭环接口做可行性原型，再选择精度与压缩方案。

嵌入式设备还受功耗、温度、频率、传感器和操作系统调度影响。短时冷机测量可能不能代表连续任务。配置、功耗模式和散热条件应进入实验记录。

11.9.4 练习题

题11.9-1 一个低比特模型权重更小，为什么在 NPU 或 Jetson 上仍可能更慢？部署方案必须共同确定哪些因素？

11.9.5 参考答案

题11.9-1答案 权重文件更小，只能直接说明储存量降低。硬件执行时间还取决于它是否有该低比特格式的高效算子、输入是否满足这些算子的形状和对齐要求，以及转换开销。若权重每层都要先解包或反量化，省下的访存可能被转换抵消；若算子不被 NPU 支持，部分计算会回退到 CPU，增加设备间传输和同步。Jetson 的小批次矩阵也可能无法充分利用低精度计算单元。

还可能出现量化边界过多、融合被打断、布局转换频繁，或动态形状使用了不适合实际输入的编译配置。长时间运行时，共享内存带宽、功耗模式和温度也会影响速度。

部署要共同确定模型结构与生成方式、精度及量化粒度、目标硬件、可用算子与编译运行时、实际输入形状和批次、图像传输及动作调度、功耗和热条件。例如权重量化只能降低模型读取成本；若端到端主要在等待相机解码，它不会解决主要瓶颈。最终以目标设备上的层级剖析、稳定端到端延迟和闭环任务表现选择方案，不能仅根据模型文件大小排序。

11.10 端侧策略优化项目与可信性能研究

11.10.1 端侧优化的教学推演：从原模型到执行链路

明确项目条件

任务仍是将杯子放入托盘；输入为两路图像、任务和状态，输出 H=16、D=7 的**动作块**，底层周期 10 ms。目标是让新动作的可用时间尽量满足 100 ms 的软实时预算，同时把峰值内存压到 8 GiB 以下并保留任务能力。

下面使用一组构造记录，展示如何比较同一设备上的部署方案。所有方案使用相同输入与功耗设置；表中数值用于解释测量和取舍，实际项目应替换为目标设备记录。

第一步：做能解释的原始记录

固定模型、输入、生成步和随机状态，在回放上比较输出，在闭环上建立任务基线。保存每次请求的采集、入队、计算开始、结果可用和执行时间。串行教学模块均值如下：

模块，单位 ms	原始	引擎与基础工程优化	量化及进一步执行优化	少步生成与前后处理优化
采集与预处理	14	10	10	8
输入传输	8	6	4	3
视觉编码	22	18	16	15
主干网络	46	34	26	24
动作生成	24	24	20	14
后处理与发送	6	5	5	4
总计	120	97	81	68

这个教学例子组合了多个工程改动，不能据此判断某一种方法的独立贡献；后者需要消融实验。这里只假设串行模块时间不重叠，实际系统应按时间轴核对。后表的P95和P99是为本例单独设定的端到端教学统计，用来说明尾部延迟与均值的区别；它们不是实测结果，也不是将各模块分位数相加得到的。

第二步：先完成转换正确性

建立预处理和动作恢复协议，保留高精度原模型；分模块导出，配置实际常用形状；构建引擎；固定输入与噪声，比较视觉特征、条件特征、每个生成步输出和最终物理动作。若结果偏离，先修正协议或算子，不在错误基线上研究**量化**。

减少重复初始化、内存分配和不必要复制后，再做性能剖析。查看哪一项仍主导时间。若是权重读取，就评估低位权重；若是重复动作步，就评估生成和提交开销；若是等待，则调整管线与同步。

第三步：校准、量化、恢复与复测

收集接近、抓取、遮挡和放置阶段的观测，运行完整生成过程收集中间统计；先尝试后端实际支持的主干权重量化，保留数值敏感模块；对输出异常位置检查尺度和分组；对速度不佳位置检查量化 kernel、解包、回退和转换。必要时尝试 **QAT**，但单独记录训练数据与计算成本。

若降低位数后的误差集中在夹爪阈值或最终动作恢复，不应继续盲目降低整个模型位数。可以保留对应模块、改变适配尺度或补充训练覆盖，并验证问题是否消失。

第四步：降低重复工作并接回控制系统

在模型依赖允许的位置复用固定条件；对少步生成固定噪声检查动作，再闭环验证；为执行线程和推理线程规定队列所有权、时间对齐与失败行为。动态 Token 数若造成频繁构建或性能波动，可选择少量已构建档位。把随机、缓存和异步执行状态纳入回放记录。

第五步：结果该怎样读

配置	平均 / P95 / P99，ms	峰值 GiB	成功数 / 尝试数
原始	120 / 156 / 205	15.8	184 / 200
引擎与基础优化	97 / 124 / 171	15.2	184 / 200
量化及进一步优化	81 / 108 / 147	6.2	182 / 200
少步生成等组合	68 / 88 / 116	5.8	180 / 200

第三个方案的平均延迟低于 100 ms，但 P95 超过时限；最后方案的 P95 达标，P99 仍超时。平均、尾部与最坏情况分别描述不同要求。成功率由 92% 到 90% 的变化还需结合样本量、任务分组和失败后果判断，才能决定这一加速是否值得采用。

还应检查最后配置在遮挡或接触任务上是否集中退化，以及队列是否造成观测变旧。若困难任务退化，可以为这些状态提高计算预算、减少缓存或保留较高精度，再测试门控机制。工程目标是在明确约束下选择配置，而不是把压缩率作为最终目的。

11.10.2 怎样把高效推理工作做成可信研究

问题要连接资源与能力

“把量化、缓存和**剪枝**组合起来”描述了模块，尚未说明研究问题。可进一步问：在端侧观测变化与接触阶段，哪些计算冗余可以去除，哪些必须保留？是否能利用任务信息分配计算，并在固定资源条件下改善成功率与时延的权衡？这些问题需要机制和证据，不要求每个组成模块全新。

先检索与整个目的、数据流和执行机制接近的工作。EfficientVLA 已经研究跨模块组合，因此只用不同名称拼接同类组件，不能直接宣称整体新颖。可以进一步研究新的任务约束、信息选择依据、学习目标或能被验证的失败机制。EfficientVLA 原始工作

分层验证主张

运算统计说明计算量变化，设备测量说明耗时与内存，观测年龄和超时率说明响应，闭环任务说明动作质量。逐层记录可以定位收益和退化的来源：更少 **FLOPs** 仍可能遇到访存瓶颈，较小离线误差也可能被执行延迟放大。

一套研究应同时保留未压缩参考、经过合理引擎优化的参考、单方法基线及组合。若新方法只比原始未优化脚本快，尚不能确定比成熟部署路线更有效。比较时固定模型、任务、输入、动作步长、硬件和功耗；方法确需改变这些条件时明确报告并增加对照。

消融要回答机制问题

可以分别去掉量化、Token 选择、缓存或任务门控；比较相同保留比例的随机选择、简单变化规则和学习选择；比较相同计算预算下的静态与动态配置。若门控依赖额外模型、特征或训练数据，将这部分开销计入完整链路，并使用匹配数据与预算的基线。

为验证“关键接触时保留更多计算”这一机制，可以构造遮挡、物体运动、接触切换等分层任务，记录门控决策与动作误差，查看是否比固定预算更稳。若只在静态简单任务有效，可改善训练覆盖或任务表示，而不是直接把失效解释成真实部署过于困难。

成功率与统计不确定性

成功次数除尝试次数是样本成功率，不是未知总体成功概率。教学 180/200 得到 90%，常用 95% Wilson 区间约为 85.1%-93.4%。相邻任务、不同种子和同一场景重复之间可能有关联，不能盲目按独立样本处理。可按任务或场景分层，并采用匹配的成对统计或分层重采样。

“质量下降不超过某范围”应预先定义容差并使用相应非劣效或等效检验思路，不能把差异不显著当作完全等效证明。同时报告碰撞、超时与接触失败类型，平均成功率相同也可能有不同后果。

Pareto 权衡怎样读

先确定要比较的目标，例如延迟越低越好、成功率越高越好。若方案 A 在两项都不差于 B，且至少一项严格更好，称 A 支配 B；没有被任何其他候选支配的方案组成**帕累托前沿**（Pareto Frontier）。前沿上的快方案可能质量略低，质量高的方案可能更慢，需要按任务约束选择。加入内存和功耗后，应同时检查这些维度。实验得到的是当前候选与测量支持的前沿，不是所有可能算法的理论最优集合。

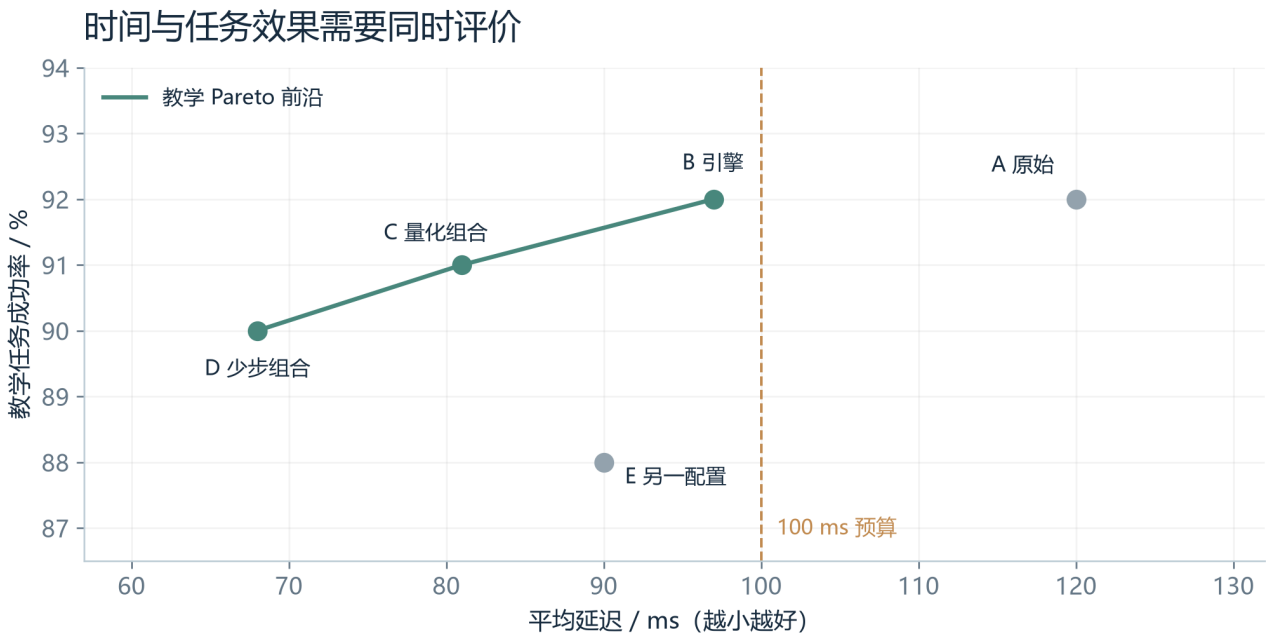


图11.13：使用第11.10节的教学均值与成功率，另加 90 ms、88% 的配置 E；E 被 81 ms、91% 的配置 C 支配。点不是实测结论，也未展示置信区间。

论文叙事应形成闭环：明确约束和瓶颈，解释方法为什么针对该瓶颈，展示数据与执行变化，再用对应层次的实验证明。若一个模块无明显贡献，可分析它何时适用并缩小主张，或重新设计连接方式。

11.10.3 加速研究的完整对照

固定模型、输入、硬件、预热和计时方式，验证数值后报告完整推理与机器人循环；近似方法再报告成功、恢复和分层精度。用同步和设备计时检查真实完成时间，避免只测 **CPU** 提交。若使用多候选筛选、不同精度或额外缓存，应明确作为实验变量。

最终贡献可以是更小误差下的更短延迟，也可以是同延迟下更高能力。把质量与成本放在同一比较表，比只给一个“加速倍数”更适合具身应用。

11.10.4 练习题

题11.10-1 比较原模型与加速模型时，怎样避免“生成更少、任务也更容易”造成不公平速度提升？至少控制哪些预算？

11.10.5 参考答案

题11.10-1答案 首先固定任务集合、初始状态、相机输入、控制频率、动作块与执行前缀、生成步数或候选数，使双方完成同样工作量；硬件、批次、功耗、热状态、预热与统计方法也应一致。

分别报告模型延迟、观测采集到动作可用的链路延迟、任务时长与成功率。吞吐和单机器人响应分开，成功判据、超时、控制接口相同；失败结束得快不能算成功任务更快。

若减少 token、分辨率或生成步数，应公开改动，补同预算对照及速度—成功率曲线，判断收益来自实现效率还是减少工作量。两模型都在固定完整评测集上运行，不能只挑加速模型成功或双方均成功的样本。

第12章：综合项目、研究设计与学习检验

最后将前面各环组成完整项目，并用可检验假设组织研究。系统图、阶段记录与公平对照帮助把学习到的机制转成具体实验。

12.1 把数据、策略、物理与执行组成完整系统

12.1.1 用一份项目规格串起整本书

考虑一个移动操作机器人，在房间中寻找桌上红杯，把它放到托盘，遇到遮挡或抓取失败时允许重新观察和恢复。这个任务同时需要身份语义、空间估计、动作执行、接触与长期流程。我们不假定必须使用所有模块：先建立最小可靠系统，再根据明确失败增加预测、规划、感知或学习机制。

先确定评价边界。物体包含训练实例与留出实例，桌面布局分为已见和未见组合，指令包含目标切换，负载与光照在合理范围变化。成功定义为正确杯子稳定放置、机器人未违反规定成本；接管单独计数。总时限、重试次数和计算预算固定。任务定义在训练之前冻结，避免看过结果后再把困难条件删去。

12.1.2 五份可检查的接口

接口	必须说明的语义	最小验证
观测	图像视角、时间、关节测量、指令与历史	回放同一轨迹得到一致样本
动作	单位、坐标、绝对或增量、周期与夹爪含义	已知目标命令产生预期运动
数据	提议与执行动作、标签来源、有效掩码	事件对齐及标注抽检
学习	可训练参数、目标、采样和有效批次	小集拟合与梯度路径检查
评价	初始分布、成功、成本、结束和接管	人工构造各类结果并核对分母

观测包含固定相机、腕部相机、关节状态与必要历史；动作使用基座坐标中的末端增量和夹爪命令。块长度、执行前缀和控制周期应与任务响应要求匹配。评价按物体与场景留出，记录成功、接管和阶段失败。

观测中没有物体真值位置，但仿真奖励可以使用它判断放置。动作周期是 0.05 秒，预测范围 0.4 秒，执行前缀 0.1 秒。底层控制周期需另行记录。两段历史不一定足够表达所有接触和速度，应以失败诊断决定，而非把配置当作理论保证。

12.1.3 第一阶段：物理与感知基线

使用已知目标位置和简单规划或控制，检查机器人是否能够完成抓取放置。若不能，先修资产、标定、碰撞和动作接口。然后替换为实际感知，测量误差与延迟引起的退化。这个比较定位“物理执行可行但感知不足”，避免用策略学习掩盖基础问题。

移动阶段可以由已有导航系统负责，近距离操作由动作策略负责，也可以研究统一模型。统一是否必要，应由任务交互决定：如果底座位置影响可达性和视角，高层选择与低层操作可能需要协调。比较统一方法时，模块基线也应获得相同观测和任务信息。

12.1.4 第二阶段：示范与策略预训练

采集不同布局、指令和接触阶段的示范，保留失败与纠正来源。按物体和场景拆分，固定归一化和数据版本。先训练简单可解释策略，再选择连续**动作块**、**扩散**或 Flow 路线。不同路线保持任务、数据曝光和评价条件一致，同时报告生成成本。

```
读取轨迹和任务定义
按定义构造观测历史与未来动作标签
应用有效步数和身体维度掩码
计算所选动作生成训练损失
更新允许训练的参数
保存权重、优化状态、归一化和配置
用独立重置条件运行闭环评价
```

实现时，窗口构造、有效掩码和输出解码必须保持同一动作定义。可以先用少量轨迹检查样本与回放，再扩大训练；这样更容易定位时间、坐标和标签错误。

12.1.5 第三阶段：按失败选择后训练信号

如果模型知道怎样抓取，却在小偏差后失效，先补纠错数据或 **DAgger**；如果存在多条示范路线，最终质量不同，可考虑奖励加权或强化学习；如果奖励难定义但人能可靠比较，可研究偏好；如果交互成本高，可先利用离线轨迹和模型，但要处理支持范围与预测偏差。

强化学习采样保留旧策略概率、动作变换、奖励、任务终止和截断。生成模型若没有可计算动作密度，应选择具有明确目标的适配路线，而非强行套用 **PPO**。固定预训练底座、在线交互预算和独立评价，比较后训练前后，并检验目标外的语义与任务保持。

12.1.6 第四阶段：用世界模型或规划处理明确困难

若长期任务需要比较多个路线，规划器可搜索技能和运动；若模型能准确预测不同动作后果，可以用于候选筛选或想象学习。新增模块应该改善指定能力：减少无效抓取、降低碰撞、提升遮挡恢复或降低真实交互。若只是增加候选数量，应与同样计算预算的策略采样基线比较。

评价预测、选择与执行三层，记录模型是否被搜索利用漏洞。感知若需要新视角，主动观察应计入动作与时间成本；规划若使用物体真值，应明确是上限实验，不与仅图像策略直接称为公平比较。

12.1.7 第五阶段：训练与部署的系统检验

多卡训练保持损失归一化和采样口径，检查检查点恢复。部署时先比较导出模型与原模型的输出，再分别评价量化、少步生成和缓存，最后测组合方案。记录观测年龄、端到端延迟、热状态、成本事件和成功率，才能判断哪项优化节省了资源，哪项近似损害了能力。

12.1.8 从小实验连接到完整机器人任务

行为克隆实验使用一维点状态、固定目标 0.8 与一个 32 隐单元网络，动作直接改变位置。初始数据只在 −0.9 至 −0.5，专家动作在此区域接近常量。五个训练种子中，继续窄分布行为克隆的训练**均方误差**约为 10^{-8}，留出起点闭环成功率仍为 0；DAgger 与覆盖充分的普通行为克隆在相同 1120 条专家标签下均达到该教学测试的 100%。这个刻意构造说明覆盖与闭环差异，不证明真实机器人上的普遍排名。

三动作单步实验可以观察 PPO 怎样提高高回报动作的概率：奖励均值为 8、6、1，初始概率为 0.5、0.3、0.2，期望奖励为 6。它适合检查旧概率、优势和裁剪方向。完整机器人任务还需处理状态变化、长期信用与接触，因而要在保持这些计算关系的基础上扩展数据与环境。

线性模型适合分析**可辨识性**（Identifiability），即数据能否区分不同参数或机制，也便于观察重规划；配对统计适合研究相关试验；整数点积适合检查量化尺度。小实验将一个机制单独显露出来，完整任务再检验它与感知、动作和环境结合后的效果。

每个小实践都对应全书中的一个可检查问题。完成后，可将其中一个假设替换为真实数据、图像或机器人接口，再设计下一层实验，而不是将所有小例子的成功相加，推断已经解决完整具身任务。

12.1.9 同一条轨迹里，策略提议、下发动作与真实运动怎样对应

策略提出末端增量 u，执行适配器把它变成关节目标 a，测量得到下一关节位置 q'，三者是不同对象。若学习策略的接口是末端增量，训练动作标签就应保存相应 u 及参照系；若学习关节目标，标签应对应下发的 a；实际状态差 $q' - q$ 则由动作、控制器和物理共同造成，不应无条件当成原始控制命令。

强化学习回放还要保存动作产生时的概率定义。若用已采样的 u 计算 PPO 概率比，就必须让记录的行为概率也对应这个 u；执行器限幅后得到的 a 可能是多种提议映射到同一个动作，不能把 a 直接代入原提议分布而声称得到实际执行密度。可以在算法中明确把 u 当策略变量、把适配与控制器并入环境；也可以为实际接口设计可计算的分布，但数据与概率对象必须一致。

这个接口问题将整本书连接起来：模仿学习需要语义正确的标签，动力学需要与真实转移对应的控制输入，世界模型需要预测这些输入的后果，策略梯度需要动作的生成概率，端侧执行则需要正确的时序与单位。保存清楚这三层记录，才能让后续诊断和学习有共同依据。

12.1.10 练习题

题12.1-1 策略输出动作块后，经限幅、坐标转换和底层控制执行。训练回放应保存策略提议还是实际执行动作？什么时候两者都必要？

12.1.11 参考答案

题12.1-1答案 有后处理时，两者都值得保存，并明确各自的坐标、单位和时间：策略提议说明模型选择了什么；实际执行指令说明限幅、坐标转换或安全模块最终让机器人接收了什么。再保存关节或末端测得的真实运动，因为下发指令也可能未被完全跟踪。

若要学习机器人本体对控制指令的响应，应以实际执行指令配对状态转移；否则会把“系统限幅没有执行”误学成“机器人动力学对这个大动作反应很小”。若建模对象包含限幅器在内的整个系统，也可输入策略提议，但必须把这个接口定义保持一致。行为克隆的标签同样取决于要学哪一个接口：若学

生输出限幅前提议，就需同接口示范；若希望学生直接生成安全可执行动作，就用对应的修正标签，并让部署接口一致。

策略梯度还有额外要求。例如策略采样 $u = 2$，安全限幅后执行 $a = 1$。若策略的随机变量是 u，PPO 应保存并使用 $\log \pi(u = 2 \mid o)$，把限幅看作环境接口的一部分；不能把 $a = 1$ 代入原分布就当成所采样动作的概率。若改为对限幅后动作建模，其分布需要重新推导，边界甚至可能出现点概率。保存两者和干预标记，才能诊断哪些成功来自模型、哪些来自安全模块，也便于重放同一执行流程。

12.2 从研究问题到公平可验证的实验

12.2.1 从能力缺口提出研究问题

“结合 A 与 B”描述了实现来源，还没有说明问题。先观察一个可重复缺口：例如跨负载抓取时，视觉动作策略在接触后失效；或者生成动作候选很多，却无法可靠判断哪条能插入；或者**量化**在静态图像上误差小，在夹爪闭合阶段失败。研究问题应说明条件、现有方法为何不足、希望改变哪种能力，以及怎样验证。

借鉴多个已有机制可以形成创新，前提是组合解决了明确问题，整体逻辑与已有系统不同，并通过合理证据说明必要性。组合的价值不只来自没人使用同一组名词，而来自问题定义、信息连接、训练目标或验证结论的新增内容。

12.2.2 把机制假设写成可反驳预测

假设用接触历史适应负载。机制预测应是：在改变负载但保持视觉和任务相同时，历史提供可辨别响应；历史编码改变策略的接触调整；这种变化改善未见负载。反例包括历史只编码任务阶段，或改善全部来自额外参数。于是实验需要独立改变负载、移除或打乱历史、保持模型容量，以及检查适应延迟。

可以把叙事写成“问题→机制必要性→方法→可检验预测→证据”。理论解释应清楚哪些是假设、哪些是推导、哪些只有经验支持。不能把损失形式看起来合理写成收敛保证，也不能把失败风险当成方法必然没有价值。

12.2.3 基线公平性是一组信息和资源约束

比较维度	必须对齐或明确报告	常见误判
数据	机器人轨迹、外部预训练、反馈和合成量	将更多数据归功于结构
信息	真实状态、相机、历史、教师与奖励	将特权输入归功于算法
计算	训练步数、候选数、模型与搜索	将更多搜索归功于规划方法
交互	真实、仿真、接管与重复试验	只报告自主成功片段
接口	动作、控制周期、控制器与结束规则	比较不同物理任务
评价	相同重置、拆分、统计单位与分母	用近重复条件证明泛化

合理基线不等于强迫所有方法相同结构。应建立一项主要预算，并报告其他资源；当设计本来需要额外信息时，说明这种代价和可用条件，并加入信息匹配对照。调参预算也应尽量可比较，基线未充分调试会削弱整个结论。

12.2.4 消融要区分必要性、贡献与交互

完整方法比基线好，并不说明每个模块都有贡献。移除模块可能同时改变容量、输入或优化难度，因而需要参数匹配、信息匹配或替代模块的对照。若研究A与B的互补，可用相同预算比较基线、仅A、仅B、A+B。以越大越好的评价指标J为例，交互量为$I = J_{AB} - J_A - J_B + J_0$；$I > 0$表示组合增益超过两项单独增益之和。若原指标是越小越好的代价，应先取负值作为收益，或相应反转交互量的符号。交互仍受指标尺度、上限和估计不确定性影响，还需检查组合改变了什么中间行为，才能支持机制解释。

若移除**世界模型**后候选数也下降，消融同时改变搜索预算；若移除历史后网络变小，容量也改变。设计前明确哪些变量应固定，哪些变化本来就是方法的一部分，避免事后解释所有结果。

12.2.5 Benchmark 是研究问题的取样器

短抓取任务检验局部操作，长程基准检验阶段衔接，语言泛化检验条件使用，接触和柔性任务检验物理适应。一个高分不能自动代表全部具身能力。选择主要任务族，再用额外任务检验边界；不要为显著结果不断更换基准而不报告筛选。

评价拆分应对应主张：新物体、新场景、新身体、新负载或新任务组合。不同拆分回答不同泛化问题。记录成功、恢复、成本、时间和计算，按预先定义的主指标判断，再分析子指标和失败。

12.2.6 不确定结果也可以产生科学结论

若模型预测误差下降而任务成功不变，可能说明改善落在无关纹理，或者规划受另一瓶颈限制。若少步生成显著加快而接触任务退化，表明时间成本与动作细节存在权衡。先核对实验，再用针对性测量解释，不用挑最好视频代替结论。

在主要配置上使用多个训练种子，按场景或物体重复试验；报告区间和完整失败。负结果若清楚限定假设与机制，也能帮助提出下一步可验证改进。论文叙事应围绕得到的事实，避免预先决定方法必然有效。

12.2.7 一份研究记录的最小内容

保存研究问题、预期机制、主评价协议、数据与信息预算、基线配置、消融计划、代码与权重版本、完整结果和失败分类。记录训练前决定与观察结果后的探索分析。复现实验要能够重建样本、动作、奖励和评价，而不仅能加载模型。

12.2.8 练习题

题12.2-1 提出“历史编码能改善遮挡下抓取”的假设后，怎样设计能够排除参数量增加、更多帧训练和额外推理预算的对照？

12.2.9 参考答案

题12.2-1答案 把假设限定为：“当前图像因遮挡缺少抓取所需信息，而先前图像包含该信息时，模型能利用历史改善抓取。”先固定示范轨迹、训练样本数、优化步数、动作标签与评测任务，避免历史方法偷偷见到更多数据。

至少设置三类对照：单帧基础模型；增加有效容量并尽量匹配推理计算的单帧模型，用来检验收益是否仅来自模型变大；使用相同历史编码结构、参数量和帧数，但输入重复当前帧的模型，用来控制历史模块本身和计算量。再加入读取真实历史但忽略顺序的多帧聚合模型，区分“多看一些画面”与“确实建模了时间关系”。训练数据的当前帧、历史窗口和标签应来自同一组轨迹，划分按轨迹或物体进行，避免相邻帧泄漏到测试集。

测试分为无遮挡、当前遮挡但历史曾看清、从未看清三种条件，并控制历史年龄、遮挡程度和物体运动。若方法主要依赖历史中曾出现的直接证据，收益应集中在第二种条件。第三种条件若仍有运动、几何或对象先验可用，模型可能间接推断；若现有观测在多个相关真实状态下完全相同，就无法仅靠这些输入可靠区分它们。将历史打乱、替换成其他物体历史或清空记忆，检验预测是否按预期改变。报告局部位姿误差、抓取成功率、延迟及多个训练种子的区间；预算难以精确匹配时给预算曲线，而不是把不同成本的单点结果当成纯机制证据。

12.3 独立学习与实践路线

12.3.1 用四轮实践连接知识

第一轮建立模型图：给定图像、语言和状态，画出编码、融合、动作生成与控制连接，解释每个变量来自哪里。第二轮训练一个小策略，独立构造样本、损失与更新，观察开环预测和闭环执行的差异。第三轮引入回报或预测模型，追踪新标签怎样改变学习。第四轮固定任务和预算，研究某个改动的收益、代价与失败。

每轮都保留一个能运行的最小例子。例如手写**注意力**适合检查信息读取，**行为克隆**适合检查标签与访问状态，**PPO** 适合检查优势与概率，线性**世界模型**适合检查预测和规划，**量化**点积适合检查数值表示。熟悉小例子之后，再把同一机制扩展到视觉和机器人任务。

12.3.2 理论、机器人学习与工程的阅读路线

模型基础可沿 Dive into Deep Learning 与 **Transformer** 原论文补充表示、注意力和生成。强化学习可沿 Sutton 与 Barto 教材、Berkeley CS285 和 Spinning Up补充估计与优化。机器人学习可沿 CMU Robot Learning组织模仿、视频、模型与多任务问题。

物理与控制接口可结合 Modern Robotics 和 MIT Underactuated Robotics。实践从 Hugging Face Robotics Course、MuJoCo、Gymnasium与对应机器人项目开始。训练与部署工程则结合 PyTorch 官方教程、**CUDA** 文档及 Machine Learning Compilation。

这些入口可以按当前问题选读。遇到价值目标不清楚，回到回报与**自举**；遇到接触失败，检查观测和控制接口；遇到推理慢，查看时间线与算子。阅读的目的，是把缺少的机制补回当前系统。

12.3.3 原理理解与现场实现

解释优势时，要连接回报、Q、V 与更新方向；解释**流匹配**时，要连接端点、路径标签和部署积分；解释量化时，要连接数值误差、硬件支持和闭环结果。现场实现也沿这条链组织：先写输入输出和步骤，再完成核心计算，最后用能暴露错误的例子检查。

基础题应能够独立推导和实现，开放题则需要提出可检查的方案。注意力掩码、归一化、缓存、**扩散**与流匹配、PPO 与 **SAC**、运动学、鲁棒估计、分布式平均和量化构成一组适合反复练习的主题。它们连接建模、代码与使用条件，也方便理解不同模型的选择。

12.3.4 从知识选择研究问题

研究问题可以从稳定观察到的失败出发：历史是否帮助辨认遮挡后状态，动作生成是否保留必要模式，价值估计是否改善候选选择，低精度是否破坏特定接触阶段。明确条件与评价，再设计能够排除替代解释的对照。完整实验未必支持最初假设，但能帮助判断哪一环值得继续研究。

12.3.5 练习题

题12.3-1 不查资料，画出“视觉语言表示→动作生成→控制执行→环境反馈→监督或强化更新”的完整图。分别以行为克隆、PPO 与质量加权流匹配填写动作标签、价值用途和更新目标。

12.3.6 参考答案

题12.3-1答案 一条完整的数据流可以画为：

```
图像、语言、机器人状态及历史
            ↓ 编码为条件 c
策略产生动作块 A
            ↓ 限幅、坐标转换、底层控制
机器人执行 → 环境产生下一观测、奖励和接触反馈
            ↓ 保存轨迹及执行信息
动作示范 / 累计回报 / 轨迹质量 → 构造训练目标 → 更新网络
            ↓
用更新后的策略再次推理或采集轨迹
```

行为克隆从示范中的条件 c 与专家动作块 A^* 学习。若策略有可计算的动作概率，最小化 $-\log \pi_\theta(A^* \mid c)$；若是单值回归，最小化预测与标签的平方误差。标签告诉模型“专家在这个条件下怎样做”，不需要价值网络；多种冲突示范下，单值平方回归可能预测不合适的均值，因此还需选对动作生成形式。

PPO 先以可计算动作概率的单步随机策略为例：用旧策略交互，保存采样动作、旧动作概率、奖励和结束标记。价值网络输入条件，预测从这里按采样策略继续的预期回报；用轨迹回报或自举目标训练它，再估计每个已采样动作的优势 $\hat{A}$。策略最大化 $\mathbb{E}[\min(\rho\hat{A}, \mathrm{clip}(\rho, 1-\epsilon, 1+\epsilon)\hat{A})]$，其中 $\rho = \pi_\theta(a \mid c)/\pi_{\mathrm{old}}(a \mid c)$。它用好坏反馈改变动作概率，没有“唯一正确动作标签”；价值预测用于评价与降低估计噪声，通常不直接输出执行动作。若按动作块决策，还需一致地定义整块概率、执行期间累计奖励和跨块折扣。

质量加权流匹配保留动作数据 A，并用成功标签、轨迹评分或评价器估计产生非负权重 w。抽噪声 ξ 与生成时刻 τ，构造 $x_\tau = (1-\tau)\xi + \tau A$，目标向量为 $A - \xi$；最小化

$$L = \mathbb{E}[w \| v_\theta(x_\tau, \tau, c) - (A - \xi) \|^2].$$

在本次策略更新中把质量权重 w 当作固定监督信息：它可以由独立标签给出，也可以由评价器先算出再停止这条权重的梯度。否则网络还可能通过降低自己的评分来降低损失，而没有改善动作。权重应非负且数值有限；整批为零时没有动作监督贡献，应跳过这次监督更新。它们决定哪些已有动作被学得更重。部署时从新噪声出发积分 v_θ，生成动作块。若权重来自价值或优势，评价器只负责产生质量信息；也可直接使用独立成功标签而不训练价值网络。此方法偏向已有数据中的高质量行为，不能自动保证获得数据外的新好动作，也不能把上述流匹配损失直接当成标准 PPO 的概率比目标。

进一步阅读：Proximal Policy Optimization Algorithms、Flow Matching：AffineProbPath 官方教程与接口说明、PyTorch 2.9：Autograd Mechanics。

参考文献与进一步阅读

课程、教材与教程

表中的 IL 指**模仿学习**（Imitation Learning），即利用示范数据学习行为。 表中的**可辨识性**（Identifiability）指数据能否区分不同的模型参数或机制。

教学来源	主要学习主题	入口
MIT Robotic Manipulation	从空间关系、操作接口、接触到规划与学习的连续教学链	https://manipulation.csail.mit.edu/
同课程 Pick	变换、运动学与几何接口	https://manipulation.csail.mit.edu/pick.html
同课程 Clutter	抓取、碰撞与技能组织	https://manipulation.csail.mit.edu/clutter.html
同课程 Force	力控制与阻抗的实际意义	https://manipulation.csail.mit.edu/force.html
同课程 Tactile	触觉、接触、软体操作	https://manipulation.csail.mit.edu/tactile.html
同课程 RL	学习与模型控制的关系	https://manipulation.csail.mit.edu/rl.html
MIT Underactuated Robotics	动力学、辨识、优化、模型假设	https://underactuated.mit.edu/
Modern Robotics，Lynch 与 Park	旋转、位姿、雅可比、机器人动力学	https://modernrobotics.northwestern.edu/nu-gm-book-resource/
Introduction to Robotics and Perception，Dellaert 与 Hutchinson	概率估计如何连接机器人任务	https://www.roboticsbook.org/S44_logistics_perception.html
同书 System Identification	数据激励与参数可辨识性	https://www.roboticsbook.org/S46_logistics_learning.html
Deep Learning，Goodfellow 等	表示、目标、优化、生成建模	https://www.deeplearningbook.org/
Dive into Deep Learning	注意力、位置编码与架构之间的联系	https://d2l.ai/chapter_attention-mechanisms-and-transformers/index.html
Stanford CS231n 教学笔记	神经网络与实际训练	https://cs231n.github.io/neural-networks-1/
Stanford CS234	价值估计、策略优化与实验	https://web.stanford.edu/class/cs234/
Berkeley CS285	模仿、离线、模型强化学习与研究设计	https://rail.eecs.berkeley.edu/deeprlcourse/

教学来源	主要学习主题	入口
CMU 16-831，2026 秋季	视频学习、逆强化学习、安全、多任务与机器人学习的完整覆盖	https://cmu16831.github.io/26fall/
Hugging Face Robotics Course	数据、训练、机器人接入和实践路径	https://huggingface.co/learn/robotics-course/en/unit0/1
Hugging Face Diffusion Course	去噪训练与生成流程的教学衔接	https://huggingface.co/learn/diffusion-course/en/unit1/1
Spinning Up RL Introduction	对象、策略、价值与算法分类	https://spinningup.openai.com/en/latest/spinningup/rl_intro.html
Spinning Up PPO	截断目标的作用与边界	https://spinningup.openai.com/en/latest/algorithms/ppo.html
Flow Matching Guide and Code	条件路径、边缘场与训练思想	https://arxiv.org/abs/2412.06264
PyTorch Autograd 教程	梯度到底更新哪一组参数	https://docs.pytorch.org/tutorials/beginner/basics/autogradqs_tutorial.html
PyTorch DDP 教程	进程、同步、数据划分与梯度归约	https://docs.pytorch.org/tutorials/intermediate/ddp_tutorial.html
PyTorch Compile 教程	图捕获、编译与实际性能验证	https://docs.pytorch.org/tutorials/intermediate/torch_compile_tutorial.html
MuJoCo Overview	模型、状态、仿真与渲染的分层	https://mujoco.readthedocs.io/en/stable/overview.html
Gymnasium 创建环境教程	reset、step、终止与超时的定义	https://gymnasium.farama.org/tutorials/gymnasium_basics/environment_creation/
LeRobot IL 教程	采集、训练与部署的贯通	https://huggingface.co/docs/lerobot/en/il_robots
Isaac Sim 当前教程索引	软件流程、资产、传感、学习与 ROS 接入	https://docs.isaacsim.omniverse.nvidia.com/latest/introduction/tutorial_list.html
Isaac Sim ROS 2 QoS	传输及时性和消息兼容	https://docs.isaacsim.omniverse.nvidia.com/latest/ros2_tutorials/bridge_configuration/tutorial_ros2_qos.html
Isaac Sim TF 与里程计	时间相关坐标变换与导航接口	https://docs.isaacsim.omniverse.nvidia.com/latest/ros2_tutorials/tutorial_series/tutorial_ros2_tf.html
Machine Learning Compilation	张量程序、图优化、共享内存与矩阵分块的教学链	https://book.mlc.ai/chapter_gpu_acceleration/part1.html
CUDA Programming Guide	执行和存储模型	https://docs.nvidia.com/cuda/cuda-programming-guide/index.html

教学来源	主要学习主题	入口
TensorRT 量化文档	量化语义、校准、运行时约束	https://docs.nvidia.com/deeplearning/tensorrt/latest/inference-library/work-quantized-types.html
NIST 工程统计手册	置信区间、实验单位与实验设计	https://www.itl.nist.gov/div898/handbook/
SciPy Bootstrap 文档	重采样方法与参数定义	https://scipy.github.io/devdocs/reference/generated/scipy.stats.bootstrap.html

按机制追溯原始论文与官方实现

1. AdamW：权重衰减与自适应优化。
2. Transformer：注意力与序列架构。
3. LayerNorm：层归一化。
4. RMSNorm：均方根归一化。
5. GLU Variants：门控前馈结构。
6. RoFormer / RoPE：旋转位置编码。
7. GQA：分组查询注意力。
8. FlashAttention：注意力计算与访存。
9. ViT：图像 patch Transformer。
10. CLIP：图文对比预训练。
11. SigLIP：图文 sigmoid 目标。
12. DINOv2：视觉自监督特征。
13. MAE：掩码重建。
14. FiLM：条件特征调制。
15. VAE：潜变量与变分学习。
16. ACT：动作分块与 CVAE。
17. DDPM：去噪扩散。
18. Diffusion Policy：机器人扩散动作生成。
19. Score/SDE：随机过程与 probability flow。
20. DDIM：隐式扩散采样。
21. Flow Matching：连续方向场训练。
22. Rectified Flow：直线与重配对。
23. DiT：Transformer 生成结构。
24. CFG：无分类器条件引导。
25. Consistency Models：一致性与少步生成。

26. Normalizing Flows：可逆生成与密度计算。
27. VQ-VAE：离散表示学习。
28. FAST：动作时间频率编码。
29. RT-1、RT-2：机器人 Transformer 与 VLA 路线。
30. OpenVLA、OFT：开放底座与动作适配。
31. Octo：灵活条件与扩散策略。
32. π0、π0.5：Flow 专家与异质训练。
33. π*0.6：RECAP 与经验学习。
34. π0.7：历史与多模态上下文。
35. GR00T N1 论文、当前仓库、N1.7 模型卡：架构与发布版本。
36. X-VLA：跨形态软提示。
37. LingBot-VLA 2.0、官方仓库：MoE、空间时间监督与适配。
38. DreamZero、官方仓库：世界动作联合生成。
39. SmolVLA：紧凑策略与执行组织。
40. LoRA：低秩适配。
41. Knowledge Insulation：底座与连续专家的学习信号。
42. Mamba：选择性状态空间结构。
43. DAgger：闭环分布与专家标签。
44. 离线人类示范研究：算法、数据与评价条件。
45. Diffusion Policy 作者页：实现与实验入口。
46. RT-2：视觉语言到动作 token。
47. OpenVLA 官方实现：训练与使用条件。
48. Octo：通用策略与接口适配。
49. π_0：视觉语言基础与 Flow 动作生成。
50. Open X-Embodiment：跨机器人数据与模型。
51. LIBERO：受控迁移基准。
52. Sutton 与 Barto，2018，Reinforcement Learning: An Introduction，第二版。先读任务建模、动态规划、MC、TD，再读函数近似和策略梯度。
53. Schulman 等，2015，High-Dimensional Continuous Control Using Generalized Advantage Estimation。对应优势估计和偏差—方差。
54. Schulman 等，2017，Proximal Policy Optimization Algorithms。对应概率比与裁剪代理。
55. Haarnoja 等，2018，Soft Actor-Critic。对应最大熵与 Off-policy 学习。
56. Haarnoja 等，2018，Soft Actor-Critic Algorithms and Applications。了解双 Q、温度与实践版本。

57. Fujimoto 等，2018，Addressing Function Approximation Error in Actor-Critic Methods。TD3 的估计误差处理。

58. Kostrikov 等，2021，Offline Reinforcement Learning with Implicit Q-Learning。IQL 三个模块。

59. Nair 等，2020，AWAC: Accelerating Online Reinforcement Learning with Offline Datasets。离线到在线优势加权。

60. Kumar 等，2020，Conservative Q-Learning for Offline Reinforcement Learning。离线价值保守性。

61. Shao 等，2024，DeepSeekMath。GRPO 最初的算法定义，机器人应用另需适配。

62. Ng、Harada 与 Russell，1999，Policy Invariance under Reward Transformations。势函数塑形条件。

63. Ross、Gordon 与 Bagnell，2011，A Reduction of Imitation Learning and Structured Prediction to No-Regret Online Learning。DAgger 的理论与算法。

64. Kim 等，2024，OpenVLA: An Open-Source Vision-Language-Action Model。先理解 Token 化动作和输入接口。

65. Kim、Finn 与 Liang，2025，Fine-Tuning Vision-Language-Action Models: Optimizing Speed and Success。OFT 的连续动作、并行解码与动作块。

66. Tan 等，2025，Interactive Post-Training for Vision-Language-Action Models（RIPT-VLA）。稀疏奖励与 LOO。

67. Lu 等，2025，VLA-RL: Towards Masterful and General Robotic Manipulation with Scalable Reinforcement Learning。过程反馈与系统化在线训练。

68. Li 等，2025，SimpleVLA-RL: Scaling VLA Training via Reinforcement Learning。并行交互与探索。

69. Chen 等，2025，TGRPO: Fine-tuning VLA via Trajectory-wise Group Relative Policy Optimization。轨迹和步级信号。

70. Chen 等，2025，ConRFT: A Reinforced Fine-tuning Method for VLA Models via Consistency Policy。离线到在线与人工干预；RSS 正式论文。

71. Luo 等，2024，Precise and Dexterous Robotic Manipulation via Human-in-the-Loop Reinforcement Learning（HIL-SERL）。真机学习系统；作者项目。

72. Zhang 等，2025，Balancing Signal and Variance: Adaptive Offline RL Post-Training for VLA Flow Models（ARFM）；AAAI 2026 正式页面。

73. Amin 等，2025，$\pi^*_{0.6}$: a VLA That Learns From Experience（RECAP）。优势条件化与经验纠正。

74. Ren 等，2024，Diffusion Policy Policy Optimization（DPPO）。生成链与策略梯度。

75. Chen 等，2025，πRL: Online RL Fine-tuning for Flow-based VLA Models。Flow-Noise 与 Flow-SDE。

76. Wagenmaker 等，2025，Steering Your Diffusion Policy with Latent Space Reinforcement Learning（DSRL）。冻结策略后的潜在空间适配。

77. McAllister 等，2025，Flow Matching Policy Gradients（FPO）。生成损失代理；ICLR 2026 正式页面。

78. Lyu 等，2025，Reinforcement Fine-Tuning of Flow-Matching Policies for Vision-Language-Action Models。另一篇使用 FPO 缩写的工作。

79. Rafailov 等，2023，Direct Preference Optimization。先理解静态条件输出的偏好目标。

80. Xie 等，2023，Text2Reward: Reward Shaping with Language Models for Reinforcement Learning。奖励代码生成。

81. Ma 等，2023，Eureka: Human-Level Reward Design via Coding Large Language Models。奖励代码的迭代优化。

82. Lee 等，2026，RoboReward: General-Purpose Vision-Language Reward Models for Robotics。机器人奖励模型与负例构造。

83. Schroeder 等，2026，SOLE-R1: Video-Language Reasoning as the Sole Reward for On-Robot Reinforcement Learning。时序进度反馈。

84. 2025，Human-assisted Robotic Policy Refinement via Action Preference Optimization（APO）。动作偏好。

85. Fei 等，2026，SRPO: Self-Referential Policy Optimization for VLA Models。自参考反馈；注意不同领域存在同缩写工作。

86. Xiao 等，2026，RehearseVLA: Simulated Post-Training for VLAs with Physically-Consistent World Model。虚拟交互与终止评价。

87. 2025，VLA-RFT: VLA Reinforcement Fine-tuning with Verified Rewards in World Simulators。世界模型交互与目标参考。

88. Zhong 等，2026，VLA-OPD: Bridging Offline SFT and Online RL via On-Policy Distillation。学生轨迹上的教师分布监督。

89. Chen 等，2025，RoboTwin 2.0。双臂任务与随机化。

90. Mees 等，2021，CALVIN。语言条件长程任务链。

91. 2024，Evaluating Real-World Robot Manipulation Policies in Simulation（SimplerEnv）。仿真评价真实策略。

92. Lynch 与 Park，Modern Robotics 作者资料。按需读刚体运动、雅可比、动态与接触章节。

93. 齐次变换。对应坐标组合与位姿。

94. 开链静力学。对应功率与转置映射。

95. 理解惯性矩阵。对应配置相关惯性与耦合。

96. 正动力学。对应由作用推进运动。

97. 约束动力学。对应允许运动与约束作用。

98. 驱动、传动与摩擦。对应执行器并非理想力矩源。

99. 摩擦。对应接触力可行范围。

100. 非完整轮式运动。对应底盘瞬时运动约束。

101. Tedrake，Robotic Manipulation。按任务问题选择章节。

102. Bin Picking。抓取、接触与摩擦的操作背景。

103. Manipulator Control。选择性阅读力、顺应与接触任务机制。

104. System Identification。辨识模型与信息条件。

105. ROS，REP 145: Conventions for IMU Sensor Drivers，官方草案原文。理解比力与传感坐标约定，实际驱动仍需核查。

106. Yuan、Dong、Adelson，2017，GelSight: High-Resolution Robot Tactile Sensors for Estimating Geometry and Force。触觉形变与作用估计的原始研究入口。

107. Open X-Embodiment Collaboration，2023，Open X-Embodiment: Robotic Learning Datasets and RT-X Models。跨机器人数据与接口语义。

108. MuJoCo Computation。驱动、接触、约束与计算对象。

109. MuJoCo Modeling。模型格式、坐标与接触配置。

110. Chua 等，2018，Deep Reinforcement Learning in a Handful of Trials using Probabilistic Dynamics Models。PETS 的概率动态与集成入口。

111. Kumar 等，2021，RMA: Rapid Motor Adaptation for Legged Robots。历史适应与参数条件化的具体系统。

112. Rudin 等，2021，Learning to Walk in Minutes Using Massively Parallel Deep Reinforcement Learning。并行腿足学习与部署条件。

113. DROID：真实多场景采集。

114. UMI：便携示范与接口设计。

115. LeRobotDataset v3：数据组织与版本接口。

116. MuJoCo 概览。

117. MuJoCo Python：模型、状态与推进。

118. Gymnasium Env：重置、推进和结束。

119. Isaac Sim：平台入口。

120. Isaac Sim URDF：资产导入和检查。

121. Isaac Lab：学习框架。

122. Isaac Lab Quickstart。

123. Isaac Lab 空场景：启动与时钟。

124. ManiSkill Quickstart。

125. ManiSkill 自定义任务。

126. robosuite：操作任务生态。

127. Habitat-Lab：具身场景与任务。

128. Gazebo：系统与传感仿真。

129. 视觉域随机化：视觉训练分布与迁移。

130. 动力学随机化：物理变化下的策略训练。

131. Ha 与 Schmidhuber，2018。World Models。阅读重点：视觉压缩、时间预测与控制器的分工。

132. Hafner 等，2018/2019。Learning Latent Dynamics for Planning from Pixels。PlaNet；重点：潜在状态、规划和多步动态训练。

133. Hafner 等，2019/2020。Dream to Control: Learning Behaviors by Latent Imagination。重点：在潜在想象中学习行为。

134. Hafner 等，2020/2021。Mastering Atari with Discrete World Models。DreamerV2；重点：离散潜在模型与行为学习。

135. Hafner 等，2023。Mastering Diverse Domains through World Models。DreamerV3；正式论文为 2025 年 Mastering diverse control tasks through world models。重点：跨任务训练稳定化，不把所有版本细节混同。

136. Janner 等，2019。When to Trust Your Model: Model-Based Policy Optimization。MBPO；重点：从真实状态分支的短模型展开。

137. Yu 等，2020。MOPO: Model-based Offline Policy Optimization。重点：不确定性相关奖励惩罚及其条件。

138. Kidambi 等，2020。MOReL: Model-Based Offline Reinforcement Learning。重点：未知区域与悲观环境构造。

139. Schrittwieser 等，2019/2020。Mastering Atari, Go, Chess and Shogi by Planning with a Learned Model。MuZero；重点：服务决策的潜在模型与树搜索。

140. Hansen 等，2023/2024。TD-MPC2: Scalable, Robust World Models for Continuous Control。重点：连续控制中的潜在模型预测控制。

141. Micheli 等，2022/2023。Transformers are Sample-Efficient World Models。IRIS；重点：视觉 token 的序列动态。

142. Alonso 等，2024。Diffusion for World Modeling: Visual Details Matter in Atari。DIAMOND；重点：扩散世界模型与视觉细节。

143. Chen 等，2024。Diffusion Forcing: Next-token Prediction Meets Full-Sequence Diffusion。重点：序列预测与不同 token 噪声等级。

144. Yang 等，2023。Learning Interactive Real-World Simulators。UniSim；重点：生成式交互与控制条件。

145. Bruce 等，2024。Genie: Generative Interactive Environments。重点：视频学习与潜在动作接口。

146. Du 等，2023。Learning Universal Policies via Text-Guided Video Generation。UniPi；重点：视觉计划与动作恢复。

147. Wu 等，2023。Unleashing Large-Scale Video Generative Pre-training for Visual Robot Manipulation。GR-1；重点：视频预训练、动作与未来图像联合预测。

148. NVIDIA 作者团队，2025。Cosmos World Foundation Model Platform for Physical AI。重点：平台能力、训练条件与具体接口。

149. Bardes 等，2024。Revisiting Feature Prediction for Learning Visual Representations from Video。V-JEPA；重点：特征目标与视频表示学习。

150. Assran 等，2025。V-JEPA 2: Self-Supervised Video Models Enable Understanding, Prediction and Planning。结合作者代码阅读；重点：视觉预训练与行动条件后训练的区别。

151. Sanchez-Gonzalez 等，2020。Learning to Simulate Complex Physics with Graph Networks。重点：图关系、积分与展开误差。

152. Kerbl 等，2023。3D Gaussian Splatting for Real-Time Radiance Field Rendering。重点：理解三维渲染表示与行动动态的区别。

153. Sutton，1991。Dyna, an Integrated Architecture for Learning, Planning, and Reacting。结合作者 1990 年 Integrated Architectures for Learning, Planning, and Reacting Based on Approximating Dynamic Programming 阅读；重点：真实与模拟经验的结合。

154. Williams 等，2017。Information Theoretic Model Predictive Control: Theory and Applications to Autonomous Driving。重点：采样加权的完整控制推导。

155. Walke 等，2023。BridgeData V2: A Dataset for Robot Learning at Scale。

156. Dasari 等，2019。RoboNet: Large-Scale Multi-Robot Learning。

157. Tassa 等，2018。DeepMind Control Suite。

158. Unterthiner 等，2018。Towards Accurate Generative Models of Video: A New Metric & Challenges。FVD。

159. Zhang 等，2018。The Unreasonable Effectiveness of Deep Features as a Perceptual Metric。LPIPS。

160. Wang 等，2004。Image Quality Assessment: From Error Visibility to Structural Similarity。SSIM。

161. TAMP 综述：离散与连续耦合。

162. MIT Robotic Manipulation：运动规划：IK、约束与轨迹。

163. OMPL 规划器说明：几何、运动约束与方法条件。

164. MoveIt 教程：机器人运动规划工具链。

165. MoveIt Planning Scene：环境与附着几何。

166. BehaviorTree.CPP：执行组织与反应性。

167. SayCan：语言相关性与技能可执行性。

168. OpenCV calib3d：投影、几何和标定。

169. OpenCV 相机标定：数据采集与参数求解。

170. OpenCV PnP：输入、输出和转换方向。
171. Kalibr：多传感器标定。
172. GTSAM 教程：变量、因子、优化和不确定性。
173. PCL ICP：点云对齐实现入口。
174. ORB-SLAM3：视觉与视觉惯性 SLAM。
175. VINS-Mono：视觉惯性估计。
176. FAST-LIO：激光惯性估计。
177. LIO-SAM：基于平滑与建图的激光惯性系统。
178. OctoMap：三维概率占据地图。
179. Voxblox：距离场与机器人地图。
180. NeRF：神经辐射场原始工作。
181. Segment Anything：提示式分割机制。
182. Grounding DINO：语言条件开放集合检测。
183. FoundationPose 论文：新对象位姿与跟踪。
184. FoundationPose 代码：对象信息、输入与复现。
185. Contact-GraspNet：深度场景到抓取候选。
186. BOP 评价定义：可见表面、对称性与位姿指标。
187. TUM RGB-D 基准论文：ATE、RPE 与轨迹评价。
188. KITTI 里程计评价：轨迹段误差与评价条件。
189. PyTorch Autograd：计算图与保存中间张量。
190. PyTorch Adam：优化器更新与状态。
191. PyTorch AMP：精度、缩放、累积与裁剪。
192. PyTorch 激活检查点：重计算与随机状态。
193. QLoRA：量化基础模型的适配训练。
194. PyTorch Distributed：进程组与集合通信。
195. NCCL 集合通信：归约与布局。
196. ZeRO：训练状态冗余与分片。
197. DeepSpeed ZeRO 教程：阶段、配置与卸载。
198. PyTorch FSDP2：完全分片数据并行。
199. Megatron 原论文：模型内部并行。
200. Megatron 并行指南：并行布局与取舍。
201. Megatron CP：上下文分片与注意力通信。
202. PyTorch DataLoader：数据采样、工作进程与内存。

203. PyTorch 编译接口：图捕获、动态形状与配置。
204. PyTorch 编译性能分析：时间线、预热与图断裂。
205. PyTorch DCP：分布式状态保存加载。
206. PyTorch 可复现性：种子与复现边界。
207. RLlib 扩展指南：采样端、环境与学习端扩展。
208. ONNX 基本概念。图、节点、类型与交换格式。
209. PyTorch ONNX 导出。导出路径及接口。
210. ONNX Runtime 执行提供器。运行时和后端关系。
211. ONNX Runtime 量化。静态、动态、误差定位与支持限制。
212. TensorRT 工作机制。构建、执行与内存。
213. TensorRT 动态形状。允许范围、优化形状与执行上下文。
214. TensorRT 量化方案。数值格式、粒度与支持条件。
215. TensorRT 量化工作流。PTQ、QAT 与图表示。
216. TensorRT 精度考虑。溢出、舍入与混合精度。
217. TensorRT 自定义层。插件与自定义算子。
218. TensorRT ONNX 导入说明。格式与算子版本。
219. TensorRT 引擎兼容性。不要默认跨设备与版本可移植。
220. TensorRT 性能测量。计时与环境控制。
221. TensorRT 性能实践。先测量再优化的闭环。
222. CUDA 编程模型。线程、块、warp 与硬件关系。
223. CUDA 最佳实践。访问、并发与性能条件。
224. CUDA 异步执行。流、事件与依赖。
225. CUDA Graph。重复提交与内存语义。
226. PyTorch CUDA 说明。异步、同步与存储管理。
227. PyTorch Profiler。模型事件剖析。
228. Nsight Systems 用户指南。系统时间轴分析。
229. Nsight Compute 剖析指南。kernel 指标与性能分析。
230. Triton 语言文档。GPU kernel 编程，不是同名推理服务器。
231. Triton 向量相加教程。分块、掩码、验证与计时。
232. Triton 融合 Softmax。归约与融合。
233. Triton 矩阵乘法。分块、布局和调优。
234. CUTLASS 官方项目。高性能线性代数组件。
235. torchao 量化推理。格式、实现与硬件条件。

236. torchao QAT。假量化、训练与转换。

237. GPTQ，Frantar 等，2022。近似二阶权重量化。

238. AWQ，Lin 等，2023。激活感知权重量化与执行。

239. SmoothQuant，Xiao 等，2022。等价变换与权重激活联合量化。

240. Distilling the Knowledge in a Neural Network，Hinton 等，2015。教师分布作为监督。

241. Learning both Weights and Connections for Efficient Neural Networks，Han 等，2015。剪枝与再训练。

242. cuSPARSELt 官方文档。受约束稀疏矩阵执行与设备支持。

243. PagedAttention / vLLM，Kwon 等，2023。键值缓存管理与服务。

244. Fast Inference from Transformers via Speculative Decoding，Leviathan 等，2022。草稿与目标验证。

245. openpi 官方实现。模型、生成与部署结构。

246. EfficientVLA，2025。层裁剪、视觉选择与动作头缓存的组合实例。

247. VLA-Cache，2025。视觉时间冗余与缓存。

248. Real-Time Execution of Action Chunking Flow Policies，Black 等，2025。动作块与异步执行。

249. Jetson r36.4.4 功耗和性能文档。选定版本的设备实践入口。

250. 昇腾官方文档。CANN 等资料按产品和版本选择。

251. LiteRT 官方文档。移动与端侧运行路线。

252. MNN 官方项目。端侧模型执行与工具链。

253. ROS2 官方消息质量文档源。通信及时性与队列等概念；网页文档不能直接访问时可读官方源文件。

254. TVM 官方文档。模型编译与运行。

255. MLIR 官方项目。多层中间表示与转换。

256. PyTorch 编译入口。编译与执行集成。

机制推导与实践教程

教程	对应学习内容
Spinning Up：Part 1 — Key Concepts in RL	第1.1、3.1、3.2、3.3、3.9节
Berkeley CS 285：Model-Based RL Algorithms（课程讲义）	第1.1、1.13、2.6、4.5、4.8节

教程	对应学习内容
Berkeley CS 285：Supervised Learning of Behaviors（课程讲义）	第1.2、1.7、2.1、2.2、2.5、2.7节
PyTorch：Autograd mechanics（冻结参数与梯度模式）	第1.2、3.6节
Dive into Deep Learning：Attention Scoring Functions	第1.4、1.5节
Dive into Deep Learning：Multi-Head Attention	第1.4节
RoFormer：Enhanced Transformer with Rotary Position Embedding	第1.5节
Physical Intelligence：VLAs that Train Fast, Run Fast, and Generalize Better	第1.6、1.13、1.14节
ACT/ALOHA：Learning Fine-Grained Bimanual Manipulation with Low-Cost Hardware	第1.7、1.8、2.1、2.3节
Hugging Face Diffusion Course：Unit 1	第1.9节
Step-by-Step Diffusion: An Elementary Tutorial（扩散与流匹配教程）	第1.9、1.10、1.11、3.8节
Physical Intelligence：Our First Generalist Policy	第1.11、1.12、1.13节
Physical Intelligence：Real-Time Action Chunking with Large Models	第1.13、2.3、11.4、11.8、11.10节
Modern Robotics：Singularities	第2.4、4.2节
Berkeley CS 285：Actor-Critic（含 GAE 与重参数化梯度）	第3.3、3.11节
Berkeley CS 285：Policy Gradients（课程讲义）	第3.4、3.5节

教程	对应学习内容
Berkeley CS 285：Off-Policy Policy Gradient（含裁剪代理目标）	第3.5、3.11节
Spinning Up：Soft Actor-Critic	第3.6节
IQL：Offline Reinforcement Learning with Implicit Q-Learning	第3.7节
Diffusion Policy Policy Optimization（作者方法说明）	第3.8节
Modern Robotics：Mobile Manipulation	第4.9节
Berkeley CS285：Model-Based Reinforcement Learning 课程讲义	第5.1节
Meta 作者博客：V-JEPA 2 从动作无关预测到动作条件规划	第5.1、5.9、5.10节
Kendall 与 Gal：视觉模型中的数据不确定性和模型不确定性	第5.2节
DreamerV3：RSSM 与继续概率的作者论文说明	第5.3、5.6节
Berkeley CS285：Model-Based Policy Learning 课程讲义	第5.5、5.8、5.10节
Berkeley CS285：Optimal Control and Planning 课程讲义	第5.7节
Isaac Lab 官方教程：Creating a Direct Workflow RL Environment	第6.1、6.2、6.5节
Isaac Lab 2.1.1 官方源码：ManagerBasedRLEnv 的按实例重置	第6.4、6.5节
Isaac Lab 官方事件随机化参考	第6.7、6.8节
LeRobot 官方教程：Action Representations	第7.1、7.6节

教程	对应学习内容
Berkeley CS285：Supervised Learning of Behaviors 课程讲义	第7.3节
LeRobot 官方教程：Language Columns and Recipes	第7.5节
scikit-learn 官方教程：分组数据与时间序列的交叉验证	第7.6、7.7节
SciPy 官方教程：成对 Bootstrap 重采样	第7.7节
Burr Settles：Active Learning Literature Survey，密度加权与异常样本	第7.8节
BehaviorTree.CPP 官方教程：Blackboard and Ports	第8.2节
BehaviorTree.CPP 官方教程：Reactive and Asynchronous Behaviors	第8.2、8.6节
MIT Underactuated Robotics：Sampling-based Motion Planning	第8.3节
MIT Robotic Manipulation：路径时间参数化	第8.4、8.7节
OpenCV：Camera Calibration and 3D Reconstruction	第9.1、9.9节
MIT Robotic Manipulation：Geometric Pose Estimation	第9.1、9.3、9.4、9.6、9.7、9.8、9.9等节
FilterPy：KalmanFilter 作者文档与示例	第9.2节
Open3D：ICP Registration	第9.4节
MIT Robotic Manipulation：Bin Picking	第9.6、9.7、9.8、12.1节
Simple Online and Realtime Tracking	第9.7节
PyTorch 2.9：Automatic Mixed Precision Examples	第10.2节

教程	对应学习内容
NVIDIA Megatron Core：Mixture of Experts	第10.5节
IMPALA: Scalable Distributed Deep-RL with Importance Weighted Actor-Learner Architectures	第10.6、10.8节
NVIDIA CUDA C++ Best Practices Guide：Timing and Coalesced Access	第10.7、11.6节
PyTorch：Saving and Loading Models	第10.7节
Proximal Policy Optimization Algorithms	第10.8、12.1、12.3节
NVIDIA TensorRT：Best Practices	第11.1、11.10节
ONNX：Resize 算子定义	第11.2节
NVIDIA TensorRT：Optimizing TensorRT Performance	第11.2、11.9节
gemmlowp：Building a Quantization Paradigm from First Principles	第11.3节
NVIDIA TensorRT 10.x：Working with Quantized Types	第11.4、11.9节
Hugging Face Transformers：How Caching Works	第11.5节
Flow Matching：AffineProbPath 官方教程与接口说明	第11.5、12.3节
Deep Reinforcement Learning that Matters	第11.10、12.2节